

ChArUco Board を使ったカメラキャリブレーション

0.0.2

構築: Doxygen 1.15.0

1 ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版.	1
2 calib	3
2.1 概要	3
2.2 主な機能	3
2.3 プログラムの処理の流れ	4
2.4 主要クラスと責務	4
2.4.1 Cameraと入力実装	4
2.4.2 Preference、Intrinsics、Config	5
2.4.3 Menu	5
2.4.4 Calibration	5
2.5 Windowsでの低遅延キャプチャ	6
2.6 macOSでの低遅延キャプチャ	6
2.7 OpenXR 対応	6
2.8 基本操作	7
2.9 構成ファイル	7
2.10 開発環境とビルド	7
2.10.1 Windowsでの基本的なビルド例	8
2.10.2 macOS でのビルド例	8
2.10.3 Raspberry Pi (Linux ARM) でのビルド例	8
2.10.4 Android スマートフォンでのビルド例	8
2.11 開発時の確認事項	9
2.12 ドキュメント・関連資料	9
2.12.1 開発・管理ドキュメント	9
2.12.2 プラットフォーム・機能別ガイド (docs)	9
3 名前空間索引	11
3.1 名前空間一覧	11
4 階層索引	13
4.1 クラス階層	13
5 クラス索引	15
5.1 クラス一覧	15
6 ファイル索引	17
6.1 ファイル一覧	17
7 名前空間詳解	19
7.1 gg 名前空間	19
7.1.1 詳解	22
7.1.2 列挙型詳解	22
7.1.2.1 BindingPoints	22
7.1.3 関数詳解	22

7.1.3.1 _ggError()	22
7.1.3.2 _ggFBOError()	23
7.1.3.3 ggArraysObj()	23
7.1.3.4 ggConjugate()	24
7.1.3.5 ggCreateComputeShader()	24
7.1.3.6 ggCreateNormalMap()	25
7.1.3.7 ggCreateShader()	26
7.1.3.8 ggCross() [1/2]	27
7.1.3.9 ggCross() [2/2]	28
7.1.3.10 ggDistance3() [1/2]	29
7.1.3.11 ggDistance3() [2/2]	29
7.1.3.12 ggDistance4() [1/2]	30
7.1.3.13 ggDistance4() [2/2]	31
7.1.3.14 ggDot3() [1/2]	32
7.1.3.15 ggDot3() [2/2]	32
7.1.3.16 ggDot4() [1/2]	33
7.1.3.17 ggDot4() [2/2]	34
7.1.3.18 ggElementsMesh()	34
7.1.3.19 ggElementsObj()	35
7.1.3.20 ggElementsSphere()	36
7.1.3.21 ggEllipse()	37
7.1.3.22 ggEulerQuaternion() [1/3]	37
7.1.3.23 ggEulerQuaternion() [2/3]	38
7.1.3.24 ggEulerQuaternion() [3/3]	39
7.1.3.25 ggFrustum()	40
7.1.3.26 ggIdentity()	41
7.1.3.27 ggIdentityQuaternion()	41
7.1.3.28 ggInit()	42
7.1.3.29 ggInvert() [1/2]	42
7.1.3.30 ggInvert() [2/2]	43
7.1.3.31 ggLength3() [1/2]	43
7.1.3.32 ggLength3() [2/2]	44
7.1.3.33 ggLength4() [1/2]	45
7.1.3.34 ggLength4() [2/2]	45
7.1.3.35 ggLoadComputeShader()	46
7.1.3.36 ggLoadHeight()	47
7.1.3.37 ggLoadImage()	48
7.1.3.38 ggLoadShader() [1/2]	49
7.1.3.39 ggLoadShader() [2/2]	49
7.1.3.40 ggLoadSimpleObj() [1/2]	50
7.1.3.41 ggLoadSimpleObj() [2/2]	51
7.1.3.42 ggLoadTexture()	52

7.1.3.43 ggLookat() [1/3]	53
7.1.3.44 ggLookat() [2/3]	54
7.1.3.45 ggLookat() [3/3]	54
7.1.3.46 ggMatrixQuaternion() [1/2]	55
7.1.3.47 ggMatrixQuaternion() [2/2]	56
7.1.3.48 ggNorm()	57
7.1.3.49 ggNormal()	57
7.1.3.50 ggNormalize()	58
7.1.3.51 ggNormalize3() [1/4]	58
7.1.3.52 ggNormalize3() [2/4]	59
7.1.3.53 ggNormalize3() [3/4]	59
7.1.3.54 ggNormalize3() [4/4]	60
7.1.3.55 ggNormalize4() [1/4]	60
7.1.3.56 ggNormalize4() [2/4]	61
7.1.3.57 ggNormalize4() [3/4]	62
7.1.3.58 ggNormalize4() [4/4]	62
7.1.3.59 ggOrthogonal()	63
7.1.3.60 ggPerspective()	64
7.1.3.61 ggPointsCube()	64
7.1.3.62 ggPointsSphere()	65
7.1.3.63 ggQuaternionMatrix()	65
7.1.3.64 ggQuaternionTransposeMatrix()	66
7.1.3.65 ggReadImage()	67
7.1.3.66 ggRectangle()	68
7.1.3.67 ggRotate() [1/5]	69
7.1.3.68 ggRotate() [2/5]	69
7.1.3.69 ggRotate() [3/5]	70
7.1.3.70 ggRotate() [4/5]	70
7.1.3.71 ggRotate() [5/5]	71
7.1.3.72 ggRotateQuaternion() [1/3]	72
7.1.3.73 ggRotateQuaternion() [2/3]	72
7.1.3.74 ggRotateQuaternion() [3/3]	73
7.1.3.75 ggRotateX()	74
7.1.3.76 ggRotateY()	74
7.1.3.77 ggRotateZ()	75
7.1.3.78 ggSaveColor()	76
7.1.3.79 ggSaveDepth()	77
7.1.3.80 ggSaveTga()	78
7.1.3.81 ggScale() [1/3]	79
7.1.3.82 ggScale() [2/3]	79
7.1.3.83 ggScale() [3/3]	80
7.1.3.84 ggSlerp() [1/4]	81

7.1.3.85 ggSlerp() [2/4]	81
7.1.3.86 ggSlerp() [3/4]	82
7.1.3.87 ggSlerp() [4/4]	82
7.1.3.88 ggTranslate() [1/3]	83
7.1.3.89 ggTranslate() [2/3]	84
7.1.3.90 ggTranslate() [3/3]	85
7.1.3.91 ggTranspose()	86
7.1.3.92 operator*()	87
7.1.3.93 operator+() [1/2]	87
7.1.3.94 operator+() [2/2]	88
7.1.3.95 operator-() [1/2]	88
7.1.3.96 operator-() [2/2]	88
7.1.3.97 operator/()	89
7.1.4 変数詳解	89
7.1.4.1 ggBufferAlignment	89
8 クラス詳解	91
8.1 Buffer クラス	91
8.1.1 詳解	92
8.1.2 構築子と解体子	92
8.1.2.1 Buffer() [1/4]	92
8.1.2.2 Buffer() [2/4]	93
8.1.2.3 Buffer() [3/4]	94
8.1.2.4 Buffer() [4/4]	94
8.1.2.5 ~Buffer()	95
8.1.3 関数詳解	95
8.1.3.1 bindBuffer()	95
8.1.3.2 channelsToFormat()	96
8.1.3.3 copy()	96
8.1.3.4 copyBuffer()	97
8.1.3.5 create()	98
8.1.3.6 discard()	99
8.1.3.7 formatToChannels()	100
8.1.3.8 getAspect()	101
8.1.3.9 getBufferName()	102
8.1.3.10 getChannels()	102
8.1.3.11 getFormat()	103
8.1.3.12 getHeight()	103
8.1.3.13 getSize()	104
8.1.3.14 getWidth()	105
8.1.3.15 map()	106
8.1.3.16 operator=() [1/2]	106

8.1.3.17 operator=() [2/2]	107
8.1.3.18 resize() [1/2]	107
8.1.3.19 resize() [2/2]	108
8.1.3.20 unbindBuffer()	109
8.1.3.21 unmap()	109
8.2 Calibration クラス	110
8.2.1 詳解	111
8.2.2 構築子と解体子	111
8.2.2.1 Calibration() [1/2]	111
8.2.2.2 Calibration() [2/2]	112
8.2.2.3 ~Calibration()	112
8.2.3 関数詳解	112
8.2.3.1 calibrate()	112
8.2.3.2 createBoard()	113
8.2.3.3 detectBoard()	113
8.2.3.4 detectMarkers()	114
8.2.3.5 discardCorners()	114
8.2.3.6 drawBoard()	115
8.2.3.7 finished()	115
8.2.3.8 getAllMarkerPoses()	115
8.2.3.9 getCameraMatrix()	116
8.2.3.10 getCornersCount()	116
8.2.3.11 getCurrentMotion()	117
8.2.3.12 getDistortionCoefficients()	117
8.2.3.13 getReprojectionError()	117
8.2.3.14 getSampleCount()	117
8.2.3.15 getStableProgress()	117
8.2.3.16 getTotalCount()	118
8.2.3.17 isDiverseEnough()	118
8.2.3.18 isStable()	119
8.2.3.19 loadParameters()	119
8.2.3.20 operator=()	119
8.2.3.21 recordCorners()	120
8.2.3.22 RvecTvecToPose()	120
8.2.3.23 saveParameters()	121
8.2.3.24 setDictionary()	121
8.2.3.25 updateMotion()	122
8.2.4 メンバ詳解	123
8.2.4.1 dictionaryList	123
8.3 CamCv クラス	123
8.3.1 詳解	125
8.3.2 構築子と解体子	125

8.3.2.1 CamCv()	125
8.3.2.2 ~CamCv()	126
8.3.3 関数詳解	126
8.3.3.1 decreaseExposure()	126
8.3.3.2 decreaseGain()	127
8.3.3.3 getCodec() [1/2]	127
8.3.3.4 getCodec() [2/2]	127
8.3.3.5 getFps()	128
8.3.3.6 getPosition()	128
8.3.3.7 increaseExposure()	129
8.3.3.8 increaseGain()	129
8.3.3.9 onClose()	129
8.3.3.10 onStart()	130
8.3.3.11 onStop()	130
8.3.3.12 open() [1/2]	130
8.3.3.13 open() [2/2]	131
8.3.3.14 setExposure()	131
8.3.3.15 setGain()	132
8.3.3.16 setPosition()	132
8.4 Camera クラス	133
8.4.1 詳解	134
8.4.2 構築子と解体子	134
8.4.2.1 Camera()	134
8.4.2.2 ~Camera()	134
8.4.3 関数詳解	135
8.4.3.1 close()	135
8.4.3.2 decreaseExposure()	135
8.4.3.3 decreaseGain()	136
8.4.3.4 getChannels()	136
8.4.3.5 getFormatList()	136
8.4.3.6 getFps()	136
8.4.3.7 getHeight()	137
8.4.3.8 getPrioritizeLatency()	137
8.4.3.9 getSize()	137
8.4.3.10 getWidth()	137
8.4.3.11 increaseExposure()	138
8.4.3.12 increaseGain()	138
8.4.3.13 isCaptured()	138
8.4.3.14 isRunning()	138
8.4.3.15 isStillImage()	139
8.4.3.16 lockFrame()	139
8.4.3.17 onClose()	140

8.4.3.18 onStart()	140
8.4.3.19 onStop()	141
8.4.3.20 selectFormat()	141
8.4.3.21 setPrioritizeLatency()	142
8.4.3.22 start()	142
8.4.3.23 stop()	143
8.4.4 メンバ詳解	143
8.4.4.1 captured	143
8.4.4.2 channels	144
8.4.4.3 height	144
8.4.4.4 image	144
8.4.4.5 interval	144
8.4.4.6 mtx	144
8.4.4.7 prioritizeLatency	144
8.4.4.8 running	145
8.4.4.9 thr	145
8.4.4.10 width	145
8.5 CamImage クラス	145
8.5.1 詳解	147
8.5.2 構築子と解体子	147
8.5.2.1 CamImage() [1/2]	147
8.5.2.2 CamImage() [2/2]	147
8.5.2.3 ~CamImage()	148
8.5.3 関数詳解	148
8.5.3.1 isOpened()	148
8.5.3.2 isStillImage()	149
8.5.3.3 load()	149
8.5.3.4 onClose()	150
8.5.3.5 onStart()	150
8.5.3.6 onStop()	150
8.5.3.7 open()	150
8.5.3.8 save()	151
8.6 CamMf クラス	152
8.6.1 詳解	154
8.6.2 構築子と解体子	154
8.6.2.1 CamMf()	154
8.6.2.2 ~CamMf()	155
8.6.3 関数詳解	155
8.6.3.1 capture()	155
8.6.3.2 getDeviceList()	156
8.6.3.3 getFormatList()	156
8.6.3.4 onClose()	156

8.6.3.5 onStart()	157
8.6.3.6 onStop()	157
8.6.3.7 open()	157
8.6.3.8 select()	158
8.6.3.9 selectFormat()	159
8.7 Capture クラス	159
8.7.1 詳解	160
8.7.2 構築子と解体子	160
8.7.2.1 Capture() [1/2]	160
8.7.2.2 Capture() [2/2]	160
8.7.2.3 ~Capture()	160
8.7.3 関数詳解	161
8.7.3.1 close()	161
8.7.3.2 getFps()	161
8.7.3.3 getPrioritizeLatency()	161
8.7.3.4 getSize()	162
8.7.3.5 isImage()	162
8.7.3.6 isOpened()	162
8.7.3.7 openDevice()	162
8.7.3.8 openImage()	163
8.7.3.9 openMovie()	164
8.7.3.10 operator bool()	164
8.7.3.11 retrieve() [1/2]	164
8.7.3.12 retrieve() [2/2]	165
8.7.3.13 setPrioritizeLatency()	166
8.7.3.14 start()	166
8.7.3.15 stop()	166
8.8 CaptureFormat 構造体	166
8.8.1 詳解	167
8.8.2 構築子と解体子	167
8.8.2.1 CaptureFormat()	167
8.8.3 メンバ詳解	167
8.8.3.1 codec	167
8.8.3.2 fps	168
8.8.3.3 index	168
8.8.3.4 resolution	168
8.9 Compute クラス	168
8.9.1 詳解	168
8.9.2 構築子と解体子	168
8.9.2.1 Compute()	168
8.9.2.2 ~Compute()	169
8.9.3 関数詳解	169

8.9.3.1 execute()	169
8.9.3.2 get()	170
8.9.3.3 use()	170
8.10 Config クラス	170
8.10.1 詳解	171
8.10.2 構築子と解体子	171
8.10.2.1 Config()	171
8.10.2.2 ~Config()	171
8.10.3 関数詳解	171
8.10.3.1 getBackground()	171
8.10.3.2 getCheckerLength()	172
8.10.3.3 getCheckerSize()	172
8.10.3.4 getDictionaryName()	173
8.10.3.5 getHeight()	173
8.10.3.6 getInitialImage()	174
8.10.3.7 getMarkerLength()	174
8.10.3.8 getMenuFont()	174
8.10.3.9 getMenuFontSize()	175
8.10.3.10 getPreferences()	175
8.10.3.11 getSettings()	175
8.10.3.12 getTitle()	175
8.10.3.13 getWidth()	176
8.10.3.14 initialize()	176
8.10.3.15 load()	176
8.10.3.16 save()	177
8.10.3.17 setSettings()	178
8.11 Expand クラス	178
8.11.1 詳解	179
8.11.2 構築子と解体子	179
8.11.2.1 Expand() [1/2]	179
8.11.2.2 Expand() [2/2]	179
8.11.2.3 ~Expand()	180
8.11.3 関数詳解	180
8.11.3.1 getProgram()	180
8.11.3.2 operator=()	180
8.11.3.3 setup()	181
8.12 Framebuffer クラス	182
8.12.1 詳解	184
8.12.2 構築子と解体子	185
8.12.2.1 Framebuffer() [1/4]	185
8.12.2.2 Framebuffer() [2/4]	185
8.12.2.3 Framebuffer() [3/4]	186

8.12.2.4	Framebuffer()	[4 / 4]	186
8.12.2.5	~Framebuffer()		187
8.12.3	関数詳解		187
8.12.3.1	bindFramebuffer()		187
8.12.3.2	copy()		188
8.12.3.3	create()		189
8.12.3.4	discard()		190
8.12.3.5	getChannels()		190
8.12.3.6	getFramebufferName()		191
8.12.3.7	getSize()		191
8.12.3.8	operator=()	[1 / 2]	191
8.12.3.9	operator=()	[2 / 2]	192
8.12.3.10	show()		193
8.12.3.11	unbindFramebuffer()		194
8.12.3.12	update()	[1 / 2]	194
8.12.3.13	update()	[2 / 2]	195
8.13	GgApp クラス		196
8.13.1	詳解		196
8.13.2	構築子と解体子		196
8.13.2.1	GgApp()	[1 / 3]	196
8.13.2.2	GgApp()	[2 / 3]	197
8.13.2.3	GgApp()	[3 / 3]	198
8.13.2.4	~GgApp()		198
8.13.3	関数詳解		198
8.13.3.1	getUsername()		198
8.13.3.2	main()		198
8.13.3.3	operator=()	[1 / 2]	200
8.13.3.4	operator=()	[2 / 2]	200
8.14	gg::GgBuffer< T > クラステンプレート		201
8.14.1	詳解		202
8.14.2	構築子と解体子		202
8.14.2.1	GgBuffer()	[1 / 3]	202
8.14.2.2	GgBuffer()	[2 / 3]	203
8.14.2.3	GgBuffer()	[3 / 3]	203
8.14.2.4	~GgBuffer()		204
8.14.3	関数詳解		204
8.14.3.1	bind()		204
8.14.3.2	copy()		204
8.14.3.3	getBuffer()		205
8.14.3.4	getCount()		205
8.14.3.5	getStride()		206
8.14.3.6	getTarget()		206

8.14.3.7 map() [1/2]	207
8.14.3.8 map() [2/2]	207
8.14.3.9 operator=()	208
8.14.3.10 read()	208
8.14.3.11 send()	209
8.14.3.12 unbind()	210
8.14.3.13 unmap()	210
8.15 gg::GgColorTexture クラス	210
8.15.1 詳解	211
8.15.2 構築子と解体子	211
8.15.2.1 GgColorTexture() [1/3]	211
8.15.2.2 GgColorTexture() [2/3]	211
8.15.2.3 GgColorTexture() [3/3]	212
8.15.2.4 ~GgColorTexture()	212
8.15.3 関数詳解	213
8.15.3.1 bind()	213
8.15.3.2 get()	213
8.15.3.3 getHeight()	213
8.15.3.4 getSize() [1/2]	213
8.15.3.5 getSize() [2/2]	213
8.15.3.6 getTexture()	214
8.15.3.7 getWidth()	214
8.15.3.8 load() [1/2]	214
8.15.3.9 load() [2/2]	215
8.15.3.10 operator bool()	216
8.15.3.11 operator"!()"	216
8.15.3.12 unbind()	216
8.16 gg::GgElements クラス	217
8.16.1 詳解	218
8.16.2 構築子と解体子	218
8.16.2.1 GgElements() [1/2]	218
8.16.2.2 GgElements() [2/2]	219
8.16.2.3 ~GgElements()	220
8.16.3 関数詳解	220
8.16.3.1 draw()	220
8.16.3.2 getIndexBuffer()	221
8.16.3.3 getIndexCount()	221
8.16.3.4 load()	221
8.16.3.5 send()	222
8.17 gg::GgMatrix クラス	223
8.17.1 詳解	226
8.17.2 構築子と解体子	226

8.17.2.1 GgMatrix() [1/6]	226
8.17.2.2 GgMatrix() [2/6]	226
8.17.2.3 GgMatrix() [3/6]	227
8.17.2.4 GgMatrix() [4/6]	227
8.17.2.5 GgMatrix() [5/6]	228
8.17.2.6 GgMatrix() [6/6]	228
8.17.2.7 ~GgMatrix()	229
8.17.3 関数詳解	229
8.17.3.1 frustum()	229
8.17.3.2 get() [1/2]	230
8.17.3.3 get() [2/2]	232
8.17.3.4 invert()	232
8.17.3.5 loadFrustum()	233
8.17.3.6 loadIdentity()	233
8.17.3.7 loadInvert() [1/2]	234
8.17.3.8 loadInvert() [2/2]	234
8.17.3.9 loadLookat() [1/3]	235
8.17.3.10 loadLookat() [2/3]	236
8.17.3.11 loadLookat() [3/3]	236
8.17.3.12 loadNormal() [1/2]	237
8.17.3.13 loadNormal() [2/2]	238
8.17.3.14 loadOrthogonal()	239
8.17.3.15 loadPerspective()	239
8.17.3.16 loadRotate() [1/5]	240
8.17.3.17 loadRotate() [2/5]	241
8.17.3.18 loadRotate() [3/5]	242
8.17.3.19 loadRotate() [4/5]	243
8.17.3.20 loadRotate() [5/5]	244
8.17.3.21 loadRotateX()	245
8.17.3.22 loadRotateY()	246
8.17.3.23 loadRotateZ()	246
8.17.3.24 loadScale() [1/3]	247
8.17.3.25 loadScale() [2/3]	248
8.17.3.26 loadScale() [3/3]	248
8.17.3.27 loadTranslate() [1/3]	249
8.17.3.28 loadTranslate() [2/3]	250
8.17.3.29 loadTranslate() [3/3]	250
8.17.3.30 loadTranspose() [1/2]	251
8.17.3.31 loadTranspose() [2/2]	252
8.17.3.32 lookat() [1/3]	252
8.17.3.33 lookat() [2/3]	253
8.17.3.34 lookat() [3/3]	254

8.17.3.35 normal()	255
8.17.3.36 operator*() [1/3]	256
8.17.3.37 operator*() [2/3]	257
8.17.3.38 operator*() [3/3]	258
8.17.3.39 operator*=() [1/2]	259
8.17.3.40 operator*=() [2/2]	260
8.17.3.41 operator+() [1/2]	260
8.17.3.42 operator+() [2/2]	261
8.17.3.43 operator+=() [1/2]	262
8.17.3.44 operator+=() [2/2]	262
8.17.3.45 operator-() [1/2]	263
8.17.3.46 operator-() [2/2]	264
8.17.3.47 operator-=() [1/2]	265
8.17.3.48 operator-=() [2/2]	265
8.17.3.49 operator/() [1/2]	266
8.17.3.50 operator/() [2/2]	267
8.17.3.51 operator/=() [1/2]	267
8.17.3.52 operator/=() [2/2]	268
8.17.3.53 operator=() [1/3]	269
8.17.3.54 operator=() [2/3]	269
8.17.3.55 operator=() [3/3]	270
8.17.3.56 orthogonal()	270
8.17.3.57 perspective()	271
8.17.3.58 projection() [1/4]	272
8.17.3.59 projection() [2/4]	272
8.17.3.60 projection() [3/4]	272
8.17.3.61 projection() [4/4]	272
8.17.3.62 rotate() [1/5]	273
8.17.3.63 rotate() [2/5]	273
8.17.3.64 rotate() [3/5]	274
8.17.3.65 rotate() [4/5]	274
8.17.3.66 rotate() [5/5]	275
8.17.3.67 rotateX()	276
8.17.3.68 rotateY()	277
8.17.3.69 rotateZ()	277
8.17.3.70 scale() [1/3]	278
8.17.3.71 scale() [2/3]	279
8.17.3.72 scale() [3/3]	280
8.17.3.73 translate() [1/3]	281
8.17.3.74 translate() [2/3]	282
8.17.3.75 translate() [3/3]	282
8.17.3.76 transpose()	283

8.18 gg::GgNormalTexture クラス	284
8.18.1 詳解	284
8.18.2 構築子と解体子	285
8.18.2.1 GgNormalTexture() [1/3]	285
8.18.2.2 GgNormalTexture() [2/3]	285
8.18.2.3 GgNormalTexture() [3/3]	285
8.18.2.4 ~GgNormalTexture()	286
8.18.3 関数詳解	286
8.18.3.1 bind()	286
8.18.3.2 get()	286
8.18.3.3 getHeight()	287
8.18.3.4 getSize() [1/2]	287
8.18.3.5 getSize() [2/2]	287
8.18.3.6 getTexture()	287
8.18.3.7 getWidth()	288
8.18.3.8 load() [1/2]	288
8.18.3.9 load() [2/2]	289
8.18.3.10 operator bool()	289
8.18.3.11 operator"!()	290
8.18.3.12 unbind()	290
8.19 gg::GgPoints クラス	290
8.19.1 詳解	291
8.19.2 構築子と解体子	292
8.19.2.1 GgPoints() [1/2]	292
8.19.2.2 GgPoints() [2/2]	292
8.19.2.3 ~GgPoints()	293
8.19.3 関数詳解	293
8.19.3.1 draw()	293
8.19.3.2 getBuffer()	294
8.19.3.3 getCount()	294
8.19.3.4 load()	295
8.19.3.5 operator bool()	295
8.19.3.6 operator"!()	295
8.19.3.7 send()	296
8.20 gg::GgPointShader クラス	296
8.20.1 詳解	297
8.20.2 構築子と解体子	297
8.20.2.1 GgPointShader() [1/3]	297
8.20.2.2 GgPointShader() [2/3]	298
8.20.2.3 GgPointShader() [3/3]	299
8.20.2.4 ~GgPointShader()	299
8.20.3 関数詳解	300

8.20.3.1	get()	300
8.20.3.2	load() [1/2]	300
8.20.3.3	load() [2/2]	301
8.20.3.4	loadMatrix() [1/2]	302
8.20.3.5	loadMatrix() [2/2]	303
8.20.3.6	loadModelviewMatrix() [1/2]	304
8.20.3.7	loadModelviewMatrix() [2/2]	305
8.20.3.8	loadProjectionMatrix() [1/2]	306
8.20.3.9	loadProjectionMatrix() [2/2]	306
8.20.3.10	unuse()	307
8.20.3.11	use() [1/5]	307
8.20.3.12	use() [2/5]	308
8.20.3.13	use() [3/5]	309
8.20.3.14	use() [4/5]	309
8.20.3.15	use() [5/5]	310
8.21	gg::GgQuaternion クラス	311
8.21.1	詳解	314
8.21.2	構築子と解体子	314
8.21.2.1	GgQuaternion() [1/7]	314
8.21.2.2	GgQuaternion() [2/7]	315
8.21.2.3	GgQuaternion() [3/7]	316
8.21.2.4	GgQuaternion() [4/7]	316
8.21.2.5	GgQuaternion() [5/7]	317
8.21.2.6	GgQuaternion() [6/7]	317
8.21.2.7	GgQuaternion() [7/7]	318
8.21.2.8	~GgQuaternion()	318
8.21.3	関数詳解	318
8.21.3.1	add() [1/4]	318
8.21.3.2	add() [2/4]	319
8.21.3.3	add() [3/4]	319
8.21.3.4	add() [4/4]	320
8.21.3.5	conjugate()	321
8.21.3.6	divide() [1/4]	322
8.21.3.7	divide() [2/4]	322
8.21.3.8	divide() [3/4]	323
8.21.3.9	divide() [4/4]	324
8.21.3.10	euler() [1/3]	325
8.21.3.11	euler() [2/3]	325
8.21.3.12	euler() [3/3]	326
8.21.3.13	get()	327
8.21.3.14	getConjugateMatrix() [1/3]	327
8.21.3.15	getConjugateMatrix() [2/3]	328

8.21.3.16 getConjugateMatrix() [3/3]	328
8.21.3.17 getMatrix() [1/3]	329
8.21.3.18 getMatrix() [2/3]	330
8.21.3.19 getMatrix() [3/3]	331
8.21.3.20 invert()	331
8.21.3.21 loadAdd() [1/4]	332
8.21.3.22 loadAdd() [2/4]	333
8.21.3.23 loadAdd() [3/4]	333
8.21.3.24 loadAdd() [4/4]	334
8.21.3.25 loadConjugate() [1/2]	335
8.21.3.26 loadConjugate() [2/2]	335
8.21.3.27 loadDivide() [1/4]	336
8.21.3.28 loadDivide() [2/4]	336
8.21.3.29 loadDivide() [3/4]	337
8.21.3.30 loadDivide() [4/4]	338
8.21.3.31 loadEuler() [1/2]	339
8.21.3.32 loadEuler() [2/2]	340
8.21.3.33 loadIdentity()	341
8.21.3.34 loadInvert() [1/2]	341
8.21.3.35 loadInvert() [2/2]	342
8.21.3.36 loadMatrix() [1/2]	343
8.21.3.37 loadMatrix() [2/2]	343
8.21.3.38 loadMultiply() [1/4]	344
8.21.3.39 loadMultiply() [2/4]	345
8.21.3.40 loadMultiply() [3/4]	345
8.21.3.41 loadMultiply() [4/4]	346
8.21.3.42 loadNormalize() [1/2]	347
8.21.3.43 loadNormalize() [2/2]	348
8.21.3.44 loadRotate() [1/3]	349
8.21.3.45 loadRotate() [2/3]	349
8.21.3.46 loadRotate() [3/3]	350
8.21.3.47 loadRotateX()	351
8.21.3.48 loadRotateY()	351
8.21.3.49 loadRotateZ()	352
8.21.3.50 loadSlerp() [1/4]	353
8.21.3.51 loadSlerp() [2/4]	354
8.21.3.52 loadSlerp() [3/4]	354
8.21.3.53 loadSlerp() [4/4]	355
8.21.3.54 loadSubtract() [1/4]	356
8.21.3.55 loadSubtract() [2/4]	356
8.21.3.56 loadSubtract() [3/4]	357
8.21.3.57 loadSubtract() [4/4]	358

8.21.3.58 multiply() [1/4]	359
8.21.3.59 multiply() [2/4]	360
8.21.3.60 multiply() [3/4]	361
8.21.3.61 multiply() [4/4]	362
8.21.3.62 norm()	363
8.21.3.63 normalize()	363
8.21.3.64 operator*() [1/3]	364
8.21.3.65 operator*() [2/3]	365
8.21.3.66 operator*() [3/3]	365
8.21.3.67 operator*=() [1/3]	366
8.21.3.68 operator*=() [2/3]	366
8.21.3.69 operator*=() [3/3]	366
8.21.3.70 operator+() [1/3]	367
8.21.3.71 operator+() [2/3]	367
8.21.3.72 operator+() [3/3]	368
8.21.3.73 operator+=() [1/3]	368
8.21.3.74 operator+=() [2/3]	369
8.21.3.75 operator+=() [3/3]	369
8.21.3.76 operator-() [1/3]	369
8.21.3.77 operator-() [2/3]	370
8.21.3.78 operator-() [3/3]	370
8.21.3.79 operator-=() [1/3]	371
8.21.3.80 operator-=() [2/3]	371
8.21.3.81 operator-=() [3/3]	371
8.21.3.82 operator/() [1/3]	372
8.21.3.83 operator/() [2/3]	372
8.21.3.84 operator/() [3/3]	373
8.21.3.85 operator/=() [1/3]	373
8.21.3.86 operator/=() [2/3]	374
8.21.3.87 operator/=() [3/3]	374
8.21.3.88 operator=() [1/4]	374
8.21.3.89 operator=() [2/4]	375
8.21.3.90 operator=() [3/4]	376
8.21.3.91 operator=() [4/4]	376
8.21.3.92 rotate() [1/3]	376
8.21.3.93 rotate() [2/3]	377
8.21.3.94 rotate() [3/3]	378
8.21.3.95 rotateX()	379
8.21.3.96 rotateY()	379
8.21.3.97 rotateZ()	380
8.21.3.98 slerp() [1/2]	380
8.21.3.99 slerp() [2/2]	381

8.21.3.100 subtract() [1/4]	381
8.21.3.101 subtract() [2/4]	382
8.21.3.102 subtract() [3/4]	382
8.21.3.103 subtract() [4/4]	383
8.22 gg::GgShader クラス	384
8.22.1 詳解	384
8.22.2 構築子と解体子	384
8.22.2.1 GgShader() [1/4]	384
8.22.2.2 GgShader() [2/4]	385
8.22.2.3 GgShader() [3/4]	386
8.22.2.4 GgShader() [4/4]	386
8.22.2.5 ~GgShader()	387
8.22.3 関数詳解	387
8.22.3.1 get()	387
8.22.3.2 operator=() [1/2]	387
8.22.3.3 operator=() [2/2]	388
8.22.3.4 unuse()	388
8.22.3.5 use()	389
8.23 gg::GgShape クラス	389
8.23.1 詳解	390
8.23.2 構築子と解体子	390
8.23.2.1 GgShape()	390
8.23.2.2 ~GgShape()	390
8.23.3 関数詳解	390
8.23.3.1 draw()	390
8.23.3.2 get()	391
8.23.3.3 getMode()	392
8.23.3.4 operator bool()	392
8.23.3.5 operator"!()	392
8.23.3.6 setMode()	393
8.24 gg::GgSimpleObj クラス	393
8.24.1 詳解	393
8.24.2 構築子と解体子	393
8.24.2.1 GgSimpleObj()	393
8.24.2.2 ~GgSimpleObj()	394
8.24.3 関数詳解	394
8.24.3.1 draw()	394
8.24.3.2 get()	395
8.24.3.3 operator bool()	395
8.24.3.4 operator"!()	395
8.25 gg::GgSimpleShader クラス	396
8.25.1 詳解	397

8.25.2 構築子と解体子	398
8.25.2.1 GgSimpleShader() [1/4]	398
8.25.2.2 GgSimpleShader() [2/4]	398
8.25.2.3 GgSimpleShader() [3/4]	399
8.25.2.4 GgSimpleShader() [4/4]	400
8.25.2.5 ~GgSimpleShader()	400
8.25.3 関数詳解	400
8.25.3.1 load() [1/2]	400
8.25.3.2 load() [2/2]	401
8.25.3.3 loadMatrix() [1/4]	402
8.25.3.4 loadMatrix() [2/4]	403
8.25.3.5 loadMatrix() [3/4]	403
8.25.3.6 loadMatrix() [4/4]	404
8.25.3.7 loadModelviewMatrix() [1/4]	405
8.25.3.8 loadModelviewMatrix() [2/4]	406
8.25.3.9 loadModelviewMatrix() [3/4]	406
8.25.3.10 loadModelviewMatrix() [4/4]	407
8.25.3.11 operator=()	408
8.25.3.12 use() [1/13]	409
8.25.3.13 use() [2/13]	410
8.25.3.14 use() [3/13]	411
8.25.3.15 use() [4/13]	412
8.25.3.16 use() [5/13]	412
8.25.3.17 use() [6/13]	413
8.25.3.18 use() [7/13]	414
8.25.3.19 use() [8/13]	415
8.25.3.20 use() [9/13]	416
8.25.3.21 use() [10/13]	416
8.25.3.22 use() [11/13]	417
8.25.3.23 use() [12/13]	417
8.25.3.24 use() [13/13]	418
8.26 gg::GgTexture クラス	419
8.26.1 詳解	419
8.26.2 構築子と解体子	419
8.26.2.1 GgTexture() [1/3]	419
8.26.2.2 GgTexture() [2/3]	420
8.26.2.3 GgTexture() [3/3]	421
8.26.2.4 ~GgTexture()	421
8.26.3 関数詳解	422
8.26.3.1 bind()	422
8.26.3.2 getHeight()	422
8.26.3.3 getSize() [1/2]	422

8.26.3.4	getSize() [2/2]	422
8.26.3.5	getTexture()	423
8.26.3.6	getWidth()	423
8.26.3.7	operator=() [1/2]	424
8.26.3.8	operator=() [2/2]	424
8.26.3.9	swapRandB()	425
8.26.3.10	unbind()	425
8.27	gg::GgTrackball クラス	425
8.27.1	詳解	429
8.27.2	構築子と解体子	429
8.27.2.1	GgTrackball()	429
8.27.2.2	~GgTrackball()	430
8.27.3	関数詳解	430
8.27.3.1	begin()	430
8.27.3.2	end()	431
8.27.3.3	get()	431
8.27.3.4	getMatrix()	431
8.27.3.5	getQuaternion()	432
8.27.3.6	getScale() [1/3]	432
8.27.3.7	getScale() [2/3]	432
8.27.3.8	getScale() [3/3]	432
8.27.3.9	getStart() [1/3]	433
8.27.3.10	getStart() [2/3]	433
8.27.3.11	getStart() [3/3]	433
8.27.3.12	motion()	433
8.27.3.13	operator=()	434
8.27.3.14	region() [1/2]	435
8.27.3.15	region() [2/2]	435
8.27.3.16	reset()	436
8.27.3.17	rotate()	437
8.28	gg::GgTriangles クラス	437
8.28.1	詳解	438
8.28.2	構築子と解体子	438
8.28.2.1	GgTriangles() [1/2]	438
8.28.2.2	GgTriangles() [2/2]	439
8.28.2.3	~GgTriangles()	440
8.28.3	関数詳解	440
8.28.3.1	draw()	440
8.28.3.2	getBuffer()	441
8.28.3.3	getCount()	441
8.28.3.4	load()	441
8.28.3.5	send()	442

8.29 gg::GgUniformBuffer< T > クラステンプレート	443
8.29.1 詳解	443
8.29.2 構築子と解体子	443
8.29.2.1 GgUniformBuffer() [1/3]	443
8.29.2.2 GgUniformBuffer() [2/3]	444
8.29.2.3 GgUniformBuffer() [3/3]	445
8.29.2.4 ~GgUniformBuffer()	446
8.29.3 関数詳解	446
8.29.3.1 bind()	446
8.29.3.2 copy()	447
8.29.3.3 fill()	447
8.29.3.4 getBuffer()	448
8.29.3.5 getCount()	449
8.29.3.6 getStride()	450
8.29.3.7 getTarget()	450
8.29.3.8 load() [1/2]	451
8.29.3.9 load() [2/2]	451
8.29.3.10 map() [1/2]	452
8.29.3.11 map() [2/2]	453
8.29.3.12 read()	453
8.29.3.13 send()	454
8.29.3.14 unbind()	455
8.29.3.15 unmap()	456
8.30 gg::GgVector クラス	456
8.30.1 詳解	457
8.30.2 構築子と解体子	457
8.30.2.1 GgVector() [1/6]	457
8.30.2.2 GgVector() [2/6]	458
8.30.2.3 GgVector() [3/6]	459
8.30.2.4 GgVector() [4/6]	459
8.30.2.5 GgVector() [5/6]	460
8.30.2.6 GgVector() [6/6]	460
8.30.2.7 ~GgVector()	461
8.30.3 関数詳解	461
8.30.3.1 distance3()	461
8.30.3.2 distance4()	462
8.30.3.3 dot3()	462
8.30.3.4 dot4()	463
8.30.3.5 length3()	464
8.30.3.6 length4()	464
8.30.3.7 normalize3()	465
8.30.3.8 normalize4()	465

8.30.3.9 operator*() [1/2]	465
8.30.3.10 operator*() [2/2]	466
8.30.3.11 operator*=() [1/2]	467
8.30.3.12 operator*=() [2/2]	468
8.30.3.13 operator+() [1/2]	469
8.30.3.14 operator+() [2/2]	470
8.30.3.15 operator+=() [1/2]	471
8.30.3.16 operator+=() [2/2]	472
8.30.3.17 operator-() [1/2]	473
8.30.3.18 operator-() [2/2]	474
8.30.3.19 operator-=() [1/2]	475
8.30.3.20 operator-=() [2/2]	476
8.30.3.21 operator/() [1/2]	477
8.30.3.22 operator/() [2/2]	478
8.30.3.23 operator/=() [1/2]	479
8.30.3.24 operator/=() [2/2]	480
8.30.3.25 operator=() [1/2]	481
8.30.3.26 operator=() [2/2]	481
8.31 gg::GgVertex 構造体	481
8.31.1 詳解	482
8.31.2 構築子と解体子	483
8.31.2.1 GgVertex() [1/4]	483
8.31.2.2 GgVertex() [2/4]	483
8.31.2.3 GgVertex() [3/4]	483
8.31.2.4 GgVertex() [4/4]	484
8.31.3 メンバ詳解	484
8.31.3.1 normal	484
8.31.3.2 position	485
8.32 gg::GgVertexArray クラス	485
8.32.1 詳解	485
8.32.2 構築子と解体子	485
8.32.2.1 GgVertexArray() [1/3]	485
8.32.2.2 GgVertexArray() [2/3]	486
8.32.2.3 GgVertexArray() [3/3]	487
8.32.2.4 ~GgVertexArray()	488
8.32.3 関数詳解	488
8.32.3.1 bind()	488
8.32.3.2 get()	488
8.32.3.3 operator=() [1/2]	489
8.32.3.4 operator=() [2/2]	490
8.33 Intrinsics 構造体	491
8.33.1 詳解	491

8.33.2 構築子と解体子	491
8.33.2.1 Intrinsic() [1/3]	491
8.33.2.2 Intrinsic() [2/3]	492
8.33.2.3 Intrinsic() [3/3]	492
8.33.3 関数詳解	492
8.33.3.1 setCenter()	492
8.33.3.2 setFov() [1/2]	493
8.33.3.3 setFov() [2/2]	493
8.33.3.4 setFps()	494
8.33.3.5 setSize()	494
8.33.4 メンバ詳解	494
8.33.4.1 center	494
8.33.4.2 fov	495
8.33.4.3 fps	495
8.33.4.4 sensorSize	495
8.33.4.5 size	495
8.34 gg::GgSimpleShader::Light 構造体	496
8.34.1 詳解	496
8.34.2 メンバ詳解	497
8.34.2.1 ambient	497
8.34.2.2 diffuse	497
8.34.2.3 position	497
8.34.2.4 specular	497
8.35 gg::GgSimpleShader::LightBuffer クラス	498
8.35.1 詳解	499
8.35.2 構築子と解体子	499
8.35.2.1 LightBuffer() [1/3]	499
8.35.2.2 LightBuffer() [2/3]	500
8.35.2.3 LightBuffer() [3/3]	501
8.35.2.4 ~LightBuffer()	501
8.35.3 関数詳解	501
8.35.3.1 load() [1/2]	501
8.35.3.2 load() [2/2]	502
8.35.3.3 loadAmbient() [1/3]	503
8.35.3.4 loadAmbient() [2/3]	504
8.35.3.5 loadAmbient() [3/3]	504
8.35.3.6 loadColor()	505
8.35.3.7 loadDiffuse() [1/3]	506
8.35.3.8 loadDiffuse() [2/3]	507
8.35.3.9 loadDiffuse() [3/3]	508
8.35.3.10 loadPosition() [1/4]	508
8.35.3.11 loadPosition() [2/4]	509

8.35.3.12 loadPosition() [3/4]	510
8.35.3.13 loadPosition() [4/4]	511
8.35.3.14 loadSpecular() [1/3]	512
8.35.3.15 loadSpecular() [2/3]	512
8.35.3.16 loadSpecular() [3/3]	513
8.35.3.17 select()	514
8.36 gg::GgSimpleShader::Material 構造体	515
8.36.1 詳解	516
8.36.2 メンバ詳解	516
8.36.2.1 ambient	516
8.36.2.2 diffuse	516
8.36.2.3 shininess	516
8.36.2.4 specular	517
8.37 gg::GgSimpleShader::MaterialBuffer クラス	517
8.37.1 詳解	518
8.37.2 構築子と解体子	519
8.37.2.1 MaterialBuffer() [1/3]	519
8.37.2.2 MaterialBuffer() [2/3]	519
8.37.2.3 MaterialBuffer() [3/3]	520
8.37.2.4 ~MaterialBuffer()	521
8.37.3 関数詳解	521
8.37.3.1 load() [1/2]	521
8.37.3.2 load() [2/2]	521
8.37.3.3 loadAmbient() [1/3]	522
8.37.3.4 loadAmbient() [2/3]	523
8.37.3.5 loadAmbient() [3/3]	524
8.37.3.6 loadAmbientAndDiffuse() [1/3]	524
8.37.3.7 loadAmbientAndDiffuse() [2/3]	525
8.37.3.8 loadAmbientAndDiffuse() [3/3]	526
8.37.3.9 loadDiffuse() [1/3]	527
8.37.3.10 loadDiffuse() [2/3]	528
8.37.3.11 loadDiffuse() [3/3]	529
8.37.3.12 loadShininess() [1/2]	529
8.37.3.13 loadShininess() [2/2]	530
8.37.3.14 loadSpecular() [1/3]	531
8.37.3.15 loadSpecular() [2/3]	532
8.37.3.16 loadSpecular() [3/3]	533
8.37.3.17 select()	533
8.38 Menu クラス	534
8.38.1 詳解	535
8.38.2 構築子と解体子	535
8.38.2.1 Menu() [1/2]	535

8.38.2.2 Menu() [2/2]	536
8.38.2.3 ~Menu()	537
8.38.3 関数詳解	537
8.38.3.1 draw()	537
8.38.3.2 getCheckerLength()	537
8.38.3.3 getCheckerSize()	537
8.38.3.4 getMarkerLength()	538
8.38.3.5 getMenuBarHeight()	538
8.38.3.6 getPose()	539
8.38.3.7 initializeInputIntrinsics()	539
8.38.3.8 operator bool()	539
8.38.3.9 operator=()	539
8.38.3.10 saveImage()	540
8.38.3.11 setup() [1/2]	540
8.38.3.12 setup() [2/2]	541
8.38.3.13 updateAutoCapture()	541
8.38.4 メンバ詳解	542
8.38.4.1 autoCaptureBeep	542
8.38.4.2 autoCaptureCooldown	542
8.38.4.3 autoCaptureCooldownTimer	542
8.38.4.4 autoCaptureEnabled	543
8.38.4.5 autoCaptureMinStableTime	543
8.38.4.6 autoCaptureStatusMessage	543
8.38.4.7 detectBoard	543
8.38.4.8 detectMarker	543
8.38.4.9 prioritizeLatency	543
8.39 Mesh クラス	544
8.39.1 詳解	544
8.39.2 構築子と解体子	544
8.39.2.1 Mesh() [1/2]	544
8.39.2.2 Mesh() [2/2]	545
8.39.2.3 ~Mesh()	545
8.39.3 関数詳解	545
8.39.3.1 draw()	545
8.39.3.2 drawMesh()	545
8.39.3.3 operator=()	546
8.40 Preference クラス	546
8.40.1 詳解	547
8.40.2 構築子と解体子	547
8.40.2.1 Preference() [1/3]	547
8.40.2.2 Preference() [2/3]	548
8.40.2.3 Preference() [3/3]	548

8.40.2.4 ~Preference()	549
8.40.3 関数詳解	549
8.40.3.1 buildShader()	549
8.40.3.2 getDescription()	549
8.40.3.3 getIntrinsics()	549
8.40.3.4 getShader()	549
8.40.3.5 setPreference()	549
8.41 Settings 構造体	550
8.41.1 詳解	551
8.41.2 関数詳解	551
8.41.2.1 getFocal()	551
8.41.3 メンバ詳解	551
8.41.3.1 checkerLength	551
8.41.3.2 checkerSize	551
8.41.3.3 defaultCheckerLength	552
8.41.3.4 defaultCheckerSize	552
8.41.3.5 defaultEuler	552
8.41.3.6 defaultFocal	552
8.41.3.7 defaultFocalRange	552
8.41.3.8 defaultMarkerLength	552
8.41.3.9 dictionaryName	553
8.41.3.10 euler	553
8.41.3.11 focal	553
8.41.3.12 focalRange	553
8.41.3.13 markerLength	553
8.41.3.14 samples	553
8.42 Texture クラス	554
8.42.1 詳解	556
8.42.2 構築子と解体子	556
8.42.2.1 Texture() [1/4]	556
8.42.2.2 Texture() [2/4]	557
8.42.2.3 Texture() [3/4]	557
8.42.2.4 Texture() [4/4]	558
8.42.2.5 ~Texture()	558
8.42.3 関数詳解	559
8.42.3.1 bindTexture()	559
8.42.3.2 copy()	560
8.42.3.3 create()	561
8.42.3.4 discard()	562
8.42.3.5 draw()	563
8.42.3.6 drawPixels() [1/5]	564
8.42.3.7 drawPixels() [2/5]	564

8.42.3.8 drawPixels() [3/5]	565
8.42.3.9 drawPixels() [4/5]	566
8.42.3.10 drawPixels() [5/5]	566
8.42.3.11 getChannels()	567
8.42.3.12 getSize()	567
8.42.3.13 getTextureName()	568
8.42.3.14 operator=() [1/2]	568
8.42.3.15 operator=() [2/2]	569
8.42.3.16 readPixels() [1/3]	570
8.42.3.17 readPixels() [2/3]	571
8.42.3.18 readPixels() [3/3]	571
8.42.3.19 unbindTexture()	572
8.42.4 メンバ詳解	573
8.42.4.1 mesh	573
8.43 GgApp::Window クラス	573
8.43.1 詳解	574
8.43.2 構築子と解体子	575
8.43.2.1 Window() [1/3]	575
8.43.2.2 Window() [2/3]	577
8.43.2.3 Window() [3/3]	577
8.43.2.4 ~Window()	577
8.43.3 関数詳解	578
8.43.3.1 get()	578
8.43.3.2 getAltArrow()	578
8.43.3.3 getAltArrowX()	578
8.43.3.4 getAltArrowY()	579
8.43.3.5 getArrow() [1/2]	580
8.43.3.6 getArrow() [2/2]	580
8.43.3.7 getArrowX()	581
8.43.3.8 getArrowY()	582
8.43.3.9 getAspect()	583
8.43.3.10 getControlArrow()	583
8.43.3.11 getControlArrowX()	584
8.43.3.12 getControlArrowY()	584
8.43.3.13 getFboHeight()	585
8.43.3.14 getFboSize() [1/2]	586
8.43.3.15 getFboSize() [2/2]	586
8.43.3.16 getFboWidth()	586
8.43.3.17 getHeight()	587
8.43.3.18 getKey()	587
8.43.3.19 getLastKey()	588
8.43.3.20 getMouse() [1/3]	588

8.43.3.21	getMouse() [2/3]	588
8.43.3.22	getMouse() [3/3]	588
8.43.3.23	getMouseX()	589
8.43.3.24	getMouseY()	589
8.43.3.25	getRotation()	589
8.43.3.26	getRotationMatrix()	589
8.43.3.27	getScrollMatrix()	590
8.43.3.28	getShiftArrow()	590
8.43.3.29	getShiftArrowX()	591
8.43.3.30	getShiftArrowY()	591
8.43.3.31	getSize() [1/2]	592
8.43.3.32	getSize() [2/2]	592
8.43.3.33	getTranslation()	593
8.43.3.34	getTranslationMatrix()	594
8.43.3.35	getUserPointer()	594
8.43.3.36	getWheel() [1/3]	594
8.43.3.37	getWheel() [2/3]	594
8.43.3.38	getWheel() [3/3]	595
8.43.3.39	getWheelX()	595
8.43.3.40	getWheelY()	595
8.43.3.41	getWidth()	596
8.43.3.42	operator bool()	596
8.43.3.43	operator=() [1/2]	596
8.43.3.44	operator=() [2/2]	597
8.43.3.45	reset()	598
8.43.3.46	resetRotation()	598
8.43.3.47	resetTranslation()	598
8.43.3.48	restoreViewport()	599
8.43.3.49	selectInterface()	599
8.43.3.50	setClose()	599
8.43.3.51	setKeyboardFunc()	599
8.43.3.52	setMouseFunc()	600
8.43.3.53	setResizeFunc()	600
8.43.3.54	setUserPointer()	601
8.43.3.55	setVelocity()	601
8.43.3.56	setWheelFunc()	601
8.43.3.57	shouldClose()	602
8.43.3.58	swapBuffers()	602
8.43.3.59	updateViewport()	603
9	ファイル詳解	605
9.1	Buffer.cpp ファイル	605

9.1.1 詳解	605
9.2 Buffer.cpp	606
9.3 Buffer.h ファイル	608
9.3.1 詳解	609
9.3.2 マクロ定義詳解	609
9.3.2.1 USE_PIXEL_BUFFER_OBJECT	609
9.4 Buffer.h	610
9.5 calib.cpp ファイル	611
9.5.1 詳解	612
9.5.2 マクロ定義詳解	612
9.5.2.1 CONFIG_FILE	612
9.6 calib.cpp	613
9.7 Calibration.cpp ファイル	615
9.7.1 詳解	615
9.7.2 マクロ定義詳解	616
9.7.2.1 USE_RODRIGUES	616
9.8 Calibration.cpp	616
9.9 Calibration.h ファイル	624
9.9.1 詳解	625
9.10 Calibration.h	625
9.11 CamAndroid.cpp ファイル	628
9.11.1 詳解	628
9.12 CamAndroid.cpp	628
9.13 CamAndroid.h ファイル	634
9.13.1 詳解	634
9.14 CamAndroid.h	634
9.15 CamAvf.h ファイル	636
9.15.1 詳解	636
9.16 CamAvf.h	636
9.17 CamAvf.mm ファイル	637
9.17.1 詳解	637
9.18 CamAvf.mm	638
9.19 CamCv.h ファイル	647
9.19.1 詳解	648
9.20 CamCv.h	649
9.21 Camera.h ファイル	652
9.21.1 詳解	653
9.22 Camera.h	654
9.23 CamImage.h ファイル	656
9.23.1 詳解	657
9.24 CamImage.h	658
9.25 CamLibcam.cpp ファイル	660

9.25.1 詳解	660
9.26 CamLibcam.cpp	660
9.27 CamLibcam.h ファイル	668
9.27.1 詳解	668
9.28 CamLibcam.h	669
9.29 CamMf.cpp ファイル	670
9.29.1 詳解	670
9.29.2 関数詳解	671
9.29.2.1 SafeRelease()	671
9.29.2.2 SubTypeToName()	671
9.30 CamMf.cpp	672
9.31 CamMf.h ファイル	687
9.31.1 詳解	688
9.32 CamMf.h	688
9.33 Capture.cpp ファイル	690
9.33.1 詳解	690
9.34 Capture.cpp	691
9.35 Capture.h ファイル	694
9.35.1 詳解	695
9.36 Capture.h	696
9.37 Compute.h ファイル	697
9.37.1 詳解	698
9.38 Compute.h	698
9.39 Config.cpp ファイル	698
9.39.1 詳解	699
9.40 Config.cpp	699
9.41 Config.h ファイル	703
9.41.1 詳解	704
9.42 Config.h	704
9.43 Expand.cpp ファイル	706
9.44 Expand.cpp	706
9.45 Expand.h ファイル	708
9.45.1 詳解	709
9.46 Expand.h	709
9.47 Framebuffer.cpp ファイル	710
9.47.1 詳解	710
9.48 Framebuffer.cpp	711
9.49 Framebuffer.h ファイル	714
9.49.1 詳解	715
9.50 Framebuffer.h	715
9.51 gg.cpp ファイル	716
9.51.1 詳解	716

9.52 gg.cpp	717
9.53 gg.h ファイル	794
9.53.1 詳解	798
9.53.2 マクロ定義詳解	799
9.53.2.1 ggError	799
9.53.2.2 ggFBOError	799
9.53.3 型定義詳解	799
9.53.3.1 pathChar	799
9.53.3.2 pathString	799
9.53.4 関数詳解	799
9.53.4.1 TCharToUtf8()	799
9.53.4.2 Utf8ToTChar()	800
9.54 gg.h	800
9.55 GgApp.cpp ファイル	857
9.55.1 詳解	858
9.56 GgApp.cpp	858
9.57 GgApp.h ファイル	884
9.57.1 詳解	885
9.57.2 マクロ定義詳解	885
9.57.2.1 GG.BUTTON_COUNT	885
9.57.2.2 GG.INTERFACE_COUNT	885
9.58 GgApp.h	886
9.59 Intrinsics.cpp ファイル	897
9.59.1 詳解	897
9.60 Intrinsics.cpp	897
9.61 Intrinsics.h ファイル	898
9.61.1 詳解	899
9.62 Intrinsics.h	900
9.63 main.cpp ファイル	900
9.63.1 詳解	901
9.63.2 マクロ定義詳解	901
9.63.2.1 HEADER_STR	901
9.63.3 関数詳解	902
9.63.3.1 main()	902
9.64 main.cpp	903
9.65 Menu.cpp ファイル	905
9.65.1 詳解	905
9.65.2 関数詳解	906
9.65.2.1 getAnyList()	906
9.66 Menu.cpp	906
9.67 Menu.h ファイル	925
9.67.1 詳解	926

9.68 Menu.h	927
9.69 Mesh.h ファイル	929
9.69.1 詳解	930
9.70 Mesh.h	931
9.71 opencv_link.h ファイル	932
9.71.1 詳解	932
9.72 opencv_link.h	933
9.73 parseconfig.h ファイル	933
9.73.1 詳解	935
9.73.2 関数詳解	935
9.73.2.1 getString() [1/3]	935
9.73.2.2 getString() [2/3]	935
9.73.2.3 getString() [3/3]	936
9.73.2.4 getValue() [1/2]	936
9.73.2.5 getValue() [2/2]	937
9.73.2.6 setString() [1/3]	938
9.73.2.7 setString() [2/3]	938
9.73.2.8 setString() [3/3]	939
9.73.2.9 setValue() [1/2]	939
9.73.2.10 setValue() [2/2]	939
9.74 parseconfig.h	940
9.75 Preference.cpp ファイル	943
9.75.1 詳解	943
9.76 Preference.cpp	943
9.77 Preference.h ファイル	944
9.77.1 詳解	945
9.78 Preference.h	946
9.79 README.md ファイル	947
9.80 Texture.cpp ファイル	947
9.80.1 詳解	947
9.81 Texture.cpp	948
9.82 Texture.h ファイル	951
9.82.1 詳解	952
9.83 Texture.h	952
Index	955

Chapter 1

ゲームグラフィックス特論の宿題用補助 プログラム **GLFW3** 版.

著作権所有

Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies or substantial portions of the Software.

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 2

calib

2.1 概要

本プログラムは、Webカメラ、動画ファイル、静止画像から映像を取得し、選択した投影方式で画像を展開しながら、ArUco Markerの検出とChArUco Boardを用いたカメラ較正を行うC++アプリケーションです。

Windowsのカメラ入力には Microsoft Media Foundation (MSMF) を直接使用します。macOSのカメラ入力には AV Foundation ネイティブバックエンド (CamAvf) を使用し、カメラデバイス一覧や特性（解像度・フレームレート・コーデック）を直接取得して制御します。Raspberry Pi ではネイティブの libcamera バックエンド (CamLibcam) および OpenGL ES 3.1 をサポートします。Android では Camera2 NDK (CamAndroid) をサポートします。その他の動画・静止画像の入力にはOpenCVを使用し、描画とUIにはOpenGL / OpenGL ES、GLFW、Dear ImGuiを使用します。

2.2 主な機能

- Webカメラ、動画ファイル、静止画像からの入力
- Orthographic、Equirectangular、Equidistance、Stereographicなどの投影方式による展開
- 投影方式ごとの画角、主点、既定解像度の管理
- Windows Media FoundationによるH.264/MJPEGの取得、デコード、RGB変換 (CamMf)
- macOS AV Foundationによるカメラデバイス一覧取得、特性選択、BGRA変換 (CamAvf)
- Raspberry Pi ネイティブの libcamera によるカメラ入力 (CamLibcam)
- 解像度、フレームレート、コーデックの組み合わせ選択
- 全フレーム処理とレイテンシ優先処理の切り替え
- ArUco Marker検出、ChArUco Board検出、較正パラメータの保存・読込
- 静止検知と幾何多様性判定による完全自動キャリブレーション（一人での較正作業を支援）
- JSON構成ファイルによる投影方式、表示、較正設定の管理
- OpenXRによるHMD表示（Windows、オプション）

2.3 プログラムの処理の流れ

`calib.cpp`のメインループでは、概ね次の順序で一フレームを処理します。

1. `Menu`がUIを描画し、投影方式、入力、較正設定を更新する。
2. `Capture`が現在の入力源からフレームを取得する。
3. 取得フレームをOpenGLテクスチャへ転送する。
4. `Menu::setup()`が選択中の`Preference`に対応する展開シェーダーを設定する。
5. `Framebuffer`が投影変換後の画像を生成する。
6. 必要に応じて変換後の画像をCPU側へ戻し、`Calibration`がマーカーまたはボードを検出する。
7. 最終画像とImGuiのUIをデスクトップ画面へ描画する。
8. OpenXRが有効なら、各viewのorientationを姿勢へ合成して画像を再展開し、HMDのswapchainへ描画する。

投影方式を変更すると、その方式に保存された画角と中心位置が`Intrinsics`へ反映されます。一方、入力源やビデオフォーマットを変更した直後は、実際のフレーム解像度と現在の焦点距離から、その入力を見やすく表示する初期画角を計算します。このとき中心位置は維持され、投影方式を選び直すと画角もその方式の設定値へ戻ります。

最終画像は、実`Framebuffer`の縦横比に応じて表示矩形の頂点を縮小し、幅または高さの一方を表示領域いっぱいに合わせてます。テクスチャ座標は常に画像全体を参照し、`draw.frag`でCPU/OpenCV側のBGRA配列を画面表示用のRGBAへ変換します。

2.4 主要クラスと責務

2.4.1 Cameraと入力実装

`Camera`はキャプチャスレッド、フレームバッファ、排他制御、およびレイテンシ優先フラグを管理する基底クラスです。外部グラフィックスAPI (OpenGL) や画像処理ライブラリ (OpenCV) への依存を完全に排除し、標準C++ライブラリのみで構成された純粋なフレーム取得レイヤとして再設計されています。

- **NVI (Non-Virtual Interface)** パターン: 公開インターフェース `start()`, `stop()`, `close()` がスレッド状態フラグ (running) の更新、排他制御、およびスレッド合流 (`thr.join()`) などの共通ライフサイクルを一元管理し、派生クラスは保護仮想フック関数 `onStart()`, `onStop()`, `onClose()` にハードウェア固有の処理のみをオーバーライドします。
- 単一バッファ化とコピー削減: 従来の二重バッファ (`frame` と `image`) を廃止し、CPUメモリ上の単一バッファ (`std::vector<std::uint8_t> image`) へ集約してメモリ使用量とコピーのオーバーヘッドを削減しています。
- コールバック式ゼロコピーフレームロック (**lockFrame**): `lockFrame(F&& func)` テンプレートメソッドにより、ミューテックスの非ブロッキングロック (`try-to-lock`) 成功時のみフレームデータポインタをコールバックへ渡し、PBOへの直接転送 (`glBufferSubData`) や `cv::Mat` へのコピーを上位層で安全かつ直接的に行います。

- **dynamic_cast** の排除: 基底クラスに `isStillImage()`, `getFormatList()`, `selectFormat()` の仮想関数を導入し、上位層が特定派生クラスの型チェック (ダウンキャスト) を行わずにポリモーフィックに操作できるように疎結合化しました。
- **CamMf**: Windows Media Foundationによるカメラ入力
- **CamAvf**: macOS AV Foundationによるカメラ入力
- **CamAndroid**: Android Camera2 NDKによるカメラ入力
- **CamLibcam**: Raspberry Pi ネイティブの `libcamera` によるカメラ入力
- **CamCv**: OpenCVによるカメラ、動画、ネットワーク入力
- **CamImage**: 静止画像入力
- **Capture**: 上記入力実装の所有、切り替え、開始・停止を行う窓口

WindowsおよびmacOSのフォーマット列挙結果は**CaptureFormat**として構造化され、解像度、fps、コーデック、選択番号を**Menu**へ渡します。UIはプラットフォーム固有型や表示文字列の解析に依存しません。

2.4.2 Preference、Intrinsics、Config

- **Preference**: 一つの投影方式について、説明、シェーダーファイル、固有の**Intrinsics**を保持する。
- **Intrinsics**: 画角、中心位置、解像度、フレームレートを保持する。
- **Config**: JSON構成ファイルの読込・保存、投影方式一覧、表示設定、校正設定を管理する。

構成ファイルの再読込では、一時領域へ全内容を読み込んで検証し、成功した場合だけ現在の構成を置き換えます。実行中に追加された投影方式についても、置換前に展開シェーダーを構築します。

2.4.3 Menu

MenuはImGuiによる操作画面と、UI操作を各機能へ伝える処理を担当します。描画処理は次の単位に分離されています。

- `draw()`: メインメニューバー、UI各パネル、エラーダイアログの描画統括
- `drawInputPanel()`: 投影方式、姿勢、焦点距離、入力源、フォーマット、開始・停止
- `drawCalibrationPanel()`: 辞書、検出モード、標本取得、校正
- `drawErrorDialog()`: エラーメッセージ表示

投影方式の同期は**selectPreference()**、キャプチャ開始は**startCapture()**に集約し、UI内に同じ状態遷移を重複して実装しない方針です。

2.4.4 Calibration

ArUco MarkerとChArUco Boardの検出、校正用コーナーの記録、カメラ行列と歪み係数の計算、校正パラメータの読込・保存を担当します。

2.5 Windowsでの低遅延キャプチャ

CamMfはMedia Foundation Source Readerから圧縮フレームを取得し、MFTデコーダとカラーコンバータを用いてCPUメモリ上のRGB画像へ変換します。

- フォーマット列挙時にはデコーダを作成せず、「開始」時に選択フォーマットを適用する遅延初期化を行う。
- CODECAPI_AVLowLatencyModeを設定し、デコーダ内部のバッファリングを抑制する。
- レイテンシ優先時は古いフレームを待たず、常に新しいフレームを共有バッファへ反映する。
- 全フレーム処理時は、メインスレッドが取得するまで次のフレームで上書きしない。
- COMオブジェクトとMFTバッファは、早期終了時にも解放漏れが起きないように管理する。

専用プレイヤーが使用するDXVAとGPU上のZero-copy描画に比べると、CPUメモリへの展開とOpenGLへの転送分の遅延は残ります。

2.6 macOSでの低遅延キャプチャ

CamAvfはAV Foundationを用い、カメラ入力の制御とフレーム取得を行います。

- AVCaptureDeviceDiscoverySessionにより、接続されたすべてのカメラデバイス（内蔵カメラ、USB Webカメラ、連係カメラ等）をシステムからネイティブに列挙します。
- AVCaptureDeviceFormat から解像度、最大フレームレート、コーデック（4CC）を抽出し、CaptureFormat 構造体へ格納してUIで選択可能にします。
- デバイス認識時やフォーマット列挙時にはセッション開始を行わず、キャプチャ開始時にのみフォーマット適用・セッション初期化を行う遅延初期化（Lazy Initialization）を行います。
- AVCaptureVideoDataOutput の出力フォーマットに kCVPixelFormatType_32BGRA を指定し、ハードウェアまたはOS内部で高速にBGRA変換を行います。
- alwaysDiscardsLateVideoFrames と連携し、レイテンシ優先時は遅延したフレームを破棄して常に最新フレームを取得します。
- スライド（行パディング）を考慮しながら単一画像バッファへ格納し、lockFrame() コールバックを通じてPBOまたはOpenCVへゼロコピーで安全にフレームを渡します。

2.7 OpenXR 対応

- OpenXR バックエンドは既定では無効です。Windowsで使用する場合はCMakeの構成時に-DGG_ENABLE_OPENXR=ONを指定してください。
- OpenXR ランタイムとHMDを使用する場合は、コマンドラインに--openxrを指定してください。初期化できない場合はデスクトップ表示だけで動作を継続します。
- HMD表示では各viewの向きを画像展開へ反映します。入力を単眼画像として扱うため、眼の位置による視差は付けません。
- API、フレーム処理、リソース管理についてはOpenXRマニュアルを参照してください。

2.8 基本操作

1. 「入力」パネルで投影方式を選択する。
2. 必要に応じて画角、中心、姿勢、焦点距離を調整する。
3. カメラ装置を選択する。
4. WindowsおよびmacOSでは解像度、コマ数、符号化方式を選択する。
5. 必要に応じて「レイテンシ優先」を有効にする。
6. 「開始」を押してキャプチャを開始する。

校正する場合は、「校正」パネルで辞書、ChArUco Boardのマス目数（縦横）および寸法を設定し、← ChArUco Board検出を有効にして標本を取得します。6標本以上取得すると校正を実行できます。

一人で校正作業を行う場合は「自動キャプチャ」を有効にすると、カメラの前でボードを静止させるだけで、ブレのない安定状態が自動判定され、十分な幾何変化（重心位置・距離・傾き）が確認された瞬間に標本が自動記録されます。記録直後は姿勢変更のためのクールダウンに入り、合図音（Windows: ビープ音, Linux: ターミナルベル）が鳴ります。手動の「取得」ボタンやスペースキーでの記録も併用できます。

2.9 構成ファイル

既定の構成ファイルはcalib_config.jsonです。主な項目は次のとおりです。

- ウィンドウサイズと背景色
- 展開メッシュのサンプル数
- カメラ姿勢、焦点距離、焦点距離範囲
- ArUco辞書とChArUco Boardマス目数・寸法
- 初期表示画像
- 投影方式ごとの説明、画角、中心、解像度、fps、シェーダー

構成ファイルを保存すると、UIで変更したSettingsが保存対象へ反映されます。投影方式ごとの固有パラメータはPreferenceが管理します。

2.10 開発環境とビルド

- C++17
- CMake 3.13以降
- Visual Studio 2022以降（Windows、x64）
- Clang / Xcode Command Line Tools（macOS、arm64 / x64）
- OpenCV 4.13.0
- GLFW 3.4
- Dear ImGui 1.92.8
- Native File Dialog Extended 1.3.0

2.10.1 Windowsでの基本的なビルド例

```
cmake -S . -B build
cmake --build build --config Debug
cmake --build build --config Release
```

初回のCMake構成時には、CMakeLists.txtが必要な依存ライブラリをlibs以下へ取得します。ビルド後は、シェーダー、構成ファイル、画像、フォント、OpenCV DLLが実行ファイルのディレクトリへコピーされます。

2.10.2 macOSでのビルド例

Xcode Command Line Tools (xcode-select --install) と CMake がインストールされていれば、Homebrew などの外部パッケージマネージャに依存せず完全自己完結でビルドできます。必要な依存ライブラリ (OpenCV 4.13.0, GLFW 3.4 等) は CMake によりプロジェクト直下の libs ディレクトリへ自動取得・ビルドされます。

```
# ビルド
cmake -B build
cmake --build build -j$(sysctl -n hw.ncpu)

# 実行
./build/calib
```

ビルド完了後、POST.BUILD コマンドによりシェーダーおよび JSON 構成ファイル、画像アセットが build/ ディレクトリへ自動コピーされます。また、AVFoundation および CoreMedia フレームワークが自動的にリンクされます。

2.10.3 Raspberry Pi (Linux ARM)でのビルド例

Raspberry Pi OS (Bookworm / Bullseye, 64-bit / 32-bit) では、標準のパッケージマネージャから必要な開発パッケージを導入してビルドします。

```
# 依存パッケージのインストール
sudo apt update
sudo apt install -y build-essential cmake libopencv-dev libglfw3-dev libgtk-3-dev libgles2-mesa-dev libegl1-mesa-dev

# ビルド (OpenGL ES 3.1 を使用)
cmake -B build -DUSE_GLES=ON
cmake --build build -j$(nproc)

# 実行
./build/calib

# Raspberry Pi Camera Module を使用する場合 (libcamerify 経由)
libcamerify ./build/calib
```

ビルド完了後、POST.BUILD コマンドによりシェーダーおよび JSON 構成ファイル、画像アセットが build/ ディレクトリへ自動コピーされます。

2.10.4 Android スマートフォンでのビルド例

Android Studio で android フォルダを開いてビルド・実行します。実機の事前設定、カメラ権限の手動許可、Logcat によるデバッグ手順などの詳細は Android 実機テストガイド を参照してください。

コマンドラインから Gradle Wrapper を用いてビルドする場合:

```
cd android
.\gradlew.bat assembleDebug
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

2.11 開発時の確認事項

- 入力を開いた直後は実解像度と焦点距離に基づく初期画角となり、中心位置は維持されること。
- 投影方式を選び直すと、その方式に保存された画角と中心位置へ戻ること。
- WindowsおよびmacOSのフォーマット選択が、解像度・fps・コーデックの実在する組み合わせを指していること。
- プラットフォーム固有処理は [CamMf](#)、[CamAvf](#)、[CamAndroid](#)、[CamLibcam](#)、[CamCv](#)、[Capture](#) に閉じ込め、[Menu](#) に固有型を露出させないこと。
- 構成再読込に失敗した場合、現在の構成が部分的に変更されないこと。
- キャプチャ開始処理を追加・変更するときは、`startCapture()` と `openDevice()` へ集約すること。
- `const_cast` や `friend` による不変条件迂回を排除し、`getSettings()` / `setSettings()` 等の公開 API で状態連携すること。
- クラスメンバ変数の初期化はコンストラクタの初期化子リストではなくクラス定義（ヘッダ内）のデフォルトメンバ初期化構文（インクラス初期化）へ集約すること。
- `mfcapture` との共通処理で変数名・関数名は `mfcapture`、コメント・Doxygen 表現は `calib` に統一すること。
- UIの追加は、対応する描画関数の責務を越えないようにすること。
- コメントとDoxygenを実装変更と同時に更新すること。

2.12 ドキュメント・関連資料

2.12.1 開発・管理ドキュメント

- GEMINI.md: プロジェクト開発方針と環境定義
- REQUESTS.md: 変更要求と対応履歴

2.12.2 プラットフォーム・機能別ガイド (docs)

- docs/OpenXR.md: OpenXR バックエンド実装マニュアル
- docs/Android.md: Android 実機テストとビルドガイド
- docs/CamMf.md: Windows Media Foundation ビデオキャプチャクラス [CamMf](#) 完全解説
- docs/CamAvf.md: macOS AV Foundation ビデオキャプチャクラス [CamAvf](#) 完全解説
- docs/CamAndroid.md: Android Camera2 NDK ビデオキャプチャクラス [CamAndroid](#) 完全解説
- docs/CamLibcam.md: Raspberry Pi ネイティブカメラキャプチャクラス [CamLibcam](#) 完全解説

Chapter 3

名前空間索引

3.1 名前空間一覧

全名前空間の一覧です。

99 19

Chapter 4

階層索引

4.1 クラス階層

クラス階層一覧です。大雑把に文字符号順で並べられています。

std::array	
gg::GgMatrix	223
gg::GgVector	456
gg::GgQuaternion	311
gg::GgTrackball	425
Buffer	91
Texture	554
Framebuffer	182
Calibration	110
Camera	133
CamCv	123
CamImage	145
CamMf	152
Capture	159
CaptureFormat	166
Compute	168
Config	170
Expand	178
GgApp	196
gg::GgBuffer< T >	201
gg::GgColorTexture	210
gg::GgNormalTexture	284
gg::GgPointShader	296
gg::GgSimpleShader	396
gg::GgShader	384
gg::GgShape	389
gg::GgPoints	290
gg::GgTriangles	437
gg::GgElements	217
gg::GgSimpleObj	393
gg::GgTexture	419
gg::GgUniformBuffer< T >	443
gg::GgUniformBuffer< Light >	443
gg::GgSimpleShader::LightBuffer	498

gg::GgUniformBuffer< Material >	443
gg::GgSimpleShader::MaterialBuffer	517
gg::GgVertex	481
gg::GgVertexArray	485
Intrinsics	491
gg::GgSimpleShader::Light	496
gg::GgSimpleShader::Material	515
Menu	534
Mesh	544
Preference	546
Settings	550
GgApp::Window	573

Chapter 5

クラス索引

5.1 クラス一覧

クラス・構造体・共用体・インターフェースの一覧です。

Buffer	91
Calibration	110
CamCv	123
Camera	133
CamImage	145
CamMf	152
Capture	159
CaptureFormat	166
Compute	168
Config	170
Expand	178
Framebuffer	182
GgApp	196
gg::GgBuffer< T >	201
gg::GgColorTexture	210
gg::GgElements	217
gg::GgMatrix	223
gg::GgNormalTexture	284
gg::GgPoints	290
gg::GgPointShader	296
gg::GgQuaternion	311
gg::GgShader	384
gg::GgShape	389
gg::GgSimpleObj	393
gg::GgSimpleShader	396
gg::GgTexture	419
gg::GgTrackball	425
gg::GgTriangles	437
gg::GgUniformBuffer< T >	443
gg::GgVector	456
gg::GgVertex	481
gg::GgVertexArray	485
Intrinsics	491
gg::GgSimpleShader::Light	496
gg::GgSimpleShader::LightBuffer	498

gg::GgSimpleShader::Material	515
gg::GgSimpleShader::MaterialBuffer	517
Menu	534
Mesh	544
Preference	546
Settings	550
Texture	554
GgApp::Window	573

Chapter 6

ファイル索引

6.1 ファイル一覧

ファイル一覧です。

Buffer.cpp	605
Buffer.h	608
calib.cpp	611
Calibration.cpp	615
Calibration.h	624
CamAndroid.cpp	628
CamAndroid.h	634
CamAvf.h	636
CamAvf.mm	637
CamCv.h	647
Camera.h	652
CamImage.h	656
CamLibcam.cpp	660
CamLibcam.h	668
CamMf.cpp	670
CamMf.h	687
Capture.cpp	690
Capture.h	694
Compute.h	697
Config.cpp	698
Config.h	703
Expand.cpp	706
Expand.h	708
Framebuffer.cpp	710
Framebuffer.h	714
gg.cpp	716
gg.h	794
GgApp.cpp	857
GgApp.h	884
Intrinsics.cpp	897
Intrinsics.h	898
main.cpp	900
Menu.cpp	905
Menu.h	925
Mesh.h	929

opencv_link.h	932
parseconfig.h	933
Preference.cpp	943
Preference.h	944
Texture.cpp	947
Texture.h	951

Chapter 7

名前空間詳解

7.1 gg 名前空間

クラス

- class [GgVector](#)
- class [GgMatrix](#)
- class [GgQuaternion](#)
- class [GgTrackball](#)
- class [GgTexture](#)
- class [GgColorTexture](#)
- class [GgNormalTexture](#)
- class [GgBuffer](#)
- class [GgUniformBuffer](#)
- class [GgVertexArray](#)
- class [GgShape](#)
- class [GgPoints](#)
- struct [GgVertex](#)
- class [GgTriangles](#)
- class [GgElements](#)
- class [GgShader](#)
- class [GgPointShader](#)
- class [GgSimpleShader](#)
- class [GgSimpleObj](#)

列挙型

- enum [BindingPoints](#) { [LightBindingPoint](#) = 0 , [MaterialBindingPoint](#) }

関数

- void `ggInit` ()
- void `_ggError` (const std::string &name="", unsigned int line=0)
- void `_ggFBOError` (const std::string &name="", unsigned int line=0)
- void `ggCross` (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat `ggDot3` (const GLfloat *a, const GLfloat *b)
- GLfloat `ggLength3` (const GLfloat *a)
- GLfloat `ggDistance3` (const GLfloat *a, const GLfloat *b)
- void `ggNormalize3` (const GLfloat *a, GLfloat *b)
- void `ggNormalize3` (GLfloat *a)
- GLfloat `ggDot4` (const GLfloat *a, const GLfloat *b)
- GLfloat `ggLength4` (const GLfloat *a)
- GLfloat `ggDistance4` (const GLfloat *a, const GLfloat *b)
- void `ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `ggNormalize4` (GLfloat *a)
- const `GgVector` & `operator+` (const `GgVector` &v)
- `GgVector` `operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `operator-` (const `GgVector` &v)
- `GgVector` `operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength3` (const `GgVector` &a)
- GLfloat `ggDistance3` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize3` (const `GgVector` &a)
- void `ggNormalize3` (`GgVector` *a)
- GLfloat `ggDot4` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength4` (const `GgVector` &a)
- GLfloat `ggDistance4` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize4` (const `GgVector` &a)
- void `ggNormalize4` (`GgVector` *a)
- `GgMatrix` `ggIdentity` ()
- `GgMatrix` `ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggTranslate` (const GLfloat *t)
- `GgMatrix` `ggTranslate` (const `GgVector` &t)
- `GgMatrix` `ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggScale` (const GLfloat *s)
- `GgMatrix` `ggScale` (const `GgVector` &s)
- `GgMatrix` `ggRotateX` (GLfloat a)
- `GgMatrix` `ggRotateY` (GLfloat a)
- `GgMatrix` `ggRotateZ` (GLfloat a)
- `GgMatrix` `ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix` `ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r)
- `GgMatrix` `ggRotate` (const `GgVector` &r)
- `GgMatrix` `ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix` `ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix` `ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix` `ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)

- `GgMatrix ggTranspose` (const `GgMatrix` &m)
- `GgMatrix ggInvert` (const `GgMatrix` &m)
- `GgMatrix ggNormal` (const `GgMatrix` &m)
- `GgQuaternion ggIdentityQuaternion` ()
- `GgQuaternion ggMatrixQuaternion` (const `GLfloat` *a)
- `GgQuaternion ggMatrixQuaternion` (const `GgMatrix` &m)
- `GgMatrix ggQuaternionMatrix` (const `GgQuaternion` &q)
- `GgMatrix ggQuaternionTransposeMatrix` (const `GgQuaternion` &q)
- `GgQuaternion ggRotateQuaternion` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` a)
- `GgQuaternion ggRotateQuaternion` (const `GLfloat` *v, `GLfloat` a)
- `GgQuaternion ggRotateQuaternion` (const `GLfloat` *v)
- `GgQuaternion ggEulerQuaternion` (`GLfloat` heading, `GLfloat` pitch, `GLfloat` roll)
- `GgQuaternion ggEulerQuaternion` (const `GLfloat` *e)
- `GgQuaternion ggEulerQuaternion` (const `GgVector` &e)
- `GgQuaternion ggSlerp` (const `GLfloat` *a, const `GLfloat` *b, `GLfloat` t)
- `GgQuaternion ggSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, `GLfloat` t)
- `GgQuaternion ggSlerp` (const `GgQuaternion` &q, const `GLfloat` *a, `GLfloat` t)
- `GgQuaternion ggSlerp` (const `GLfloat` *a, const `GgQuaternion` &q, `GLfloat` t)
- `GLfloat ggNorm` (const `GgQuaternion` &q)
- `GgQuaternion ggNormalize` (const `GgQuaternion` &q)
- `GgQuaternion ggConjugate` (const `GgQuaternion` &q)
- `GgQuaternion ggInvert` (const `GgQuaternion` &q)
- bool `ggSaveTga` (const `std::string` &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool `ggSaveColor` (const `std::string` &name)
- bool `ggSaveDepth` (const `std::string` &name)
- bool `ggReadImage` (const `std::string` &name, `std::vector`< `GLubyte` > &image, `GLsizei` *pWidth, `GLsizei` *pHeight, `GLenum` *pFormat)
- `GLuint ggLoadTexture` (const `GLvoid` *image, `GLsizei` width, `GLsizei` height, `GLenum` format=`GL_RGB`, `GLenum` type=`GL_UNSIGNED_BYTE`, `GLenum` internal=`GL_RGB`, `GLenum` wrap=`GL_CLAMP_TO_EDGE`, bool swizzle=false)
- `GLuint ggLoadImage` (const `std::string` &name, `GLsizei` *pWidth=nullptr, `GLsizei` *pHeight=nullptr, `GLenum` internal=`GL_RGBA`, `GLenum` wrap=`GL_CLAMP_TO_EDGE`)
- void `ggCreateNormalMap` (const `GLubyte` *hmap, `GLsizei` width, `GLsizei` height, `GLenum` format, `GLfloat` nz, `GLenum` internal, `std::vector`< `GgVector` > &nmap)
- `GLuint ggLoadHeight` (const `std::string` &name, `GLfloat` nz, `GLsizei` *pWidth=nullptr, `GLsizei` *pHeight=nullptr, `GLenum` internal=`GL_RGBA`)
- `GLuint ggCreateShader` (const `std::string` &vsrsrc, const `std::string` &fsrc="", const `std::string` &gsrsrc="", `GLint` nvarying=0, const char *const *varyings=nullptr, const `std::string` &vtext="vertex shader", const `std::string` &ftext="fragment shader", const `std::string` >ext="geometry shader")
- `GLuint ggLoadShader` (const `std::string` &vert, const `std::string` &frag="", const `std::string` &geom="", `GLint` nvarying=0, const char *const *varyings=nullptr)
- `GLuint ggLoadShader` (const `std::array`< `std::string`, 3 > &files, `GLint` nvarying=0, const char *const *varyings=nullptr)
- `GLuint ggCreateComputeShader` (const `std::string` &csrsrc, const `std::string` &ctext="compute shader")
- `GLuint ggLoadComputeShader` (const `std::string` &comp)
- `std::shared_ptr`< `GgPoints` > `ggPointsCube` (`GLsizei` countv, `GLfloat` length=1.0f, `GLfloat` cx=0.0f, `GLfloat` cy=0.0f, `GLfloat` cz=0.0f)
- `std::shared_ptr`< `GgPoints` > `ggPointsSphere` (`GLsizei` countv, `GLfloat` radius=0.5f, `GLfloat` cx=0.0f, `GLfloat` cy=0.0f, `GLfloat` cz=0.0f)
- `std::shared_ptr`< `GgTriangles` > `ggRectangle` (`GLfloat` width=1.0f, `GLfloat` height=1.0f)
- `std::shared_ptr`< `GgTriangles` > `ggEllipse` (`GLfloat` width=1.0f, `GLfloat` height=1.0f, `GLuint` slices=16)
- `std::shared_ptr`< `GgTriangles` > `ggArraysObj` (const `std::string` &name, bool normalize=false)
- `std::shared_ptr`< `GgElements` > `ggElementsObj` (const `std::string` &name, bool normalize=false)
- `std::shared_ptr`< `GgElements` > `ggElementsMesh` (`GLuint` slices, `GLuint` stacks, const `GLfloat`(*pos)[3], const `GLfloat`(*norm)[3]=nullptr)

- `std::shared_ptr< GgElements > ggElementsSphere` (GLfloat radius=1.0f, int slices=16, int stacks=8)
- `bool ggLoadSimpleObj` (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< GgSimpleShader::Material > &material, std::vector< GgVertex > &vert, bool normalize=false)
- `bool ggLoadSimpleObj` (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< GgSimpleShader::Material > &material, std::vector< GgVertex > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint `ggBufferAlignment`
使用している GPU のバッファアライメント。

7.1.1 詳解

ゲームグラフィックス特論の宿題用補助プログラムの名前空間

7.1.2 列挙型詳解

7.1.2.1 BindingPoints

```
enum gg::BindingPoints
```

光源と材質の uniform buffer object の結合ポイント。

列挙値

LightBindingPoint	光源の uniform buffer object の結合ポイント。
MaterialBindingPoint	材質の uniform buffer object の結合ポイント。

`gg.h` の 1367 行目に定義があります。

7.1.3 関数詳解

7.1.3.1 _ggError()

```
void gg::_ggError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

OpenGL のエラーをチェックする。

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, nullptr なら何も出力しない。
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない。

覚え書き

OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する。

`gg.cpp` の 2617 行目に定義があります。

7.1.3.2 _ggFBOError()

```
void gg::_ggFBOError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

FBO のエラーをチェックする.

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, nullptr なら何も出力しない.
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない.

覚え書き

FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する.

[gg.cpp](#) の 2661 行目に定義があります。

7.1.3.3 ggArraysObj()

```
std::shared_ptr< gg::GgTriangles > gg::ggArraysObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイルを読み込む (Arrays 形式)

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

[GgTriangles](#) 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイルを読み込んで GgArrays 形式の三角形データを生成する.

[gg.cpp](#) の 5416 行目に定義があります。

呼び出し関係図:



7.1.3.4 ggConjugate()

```
GgQuaternion gg::ggConjugate (
    const GgQuaternion & q) [inline]
```

共役四元数を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* の共役四元数.

gg.h の 4799 行目に定義があります。

呼び出し関係図:



7.1.3.5 ggCreateComputeShader()

```
GLuint gg::ggCreateComputeShader (
    const std::string & csrc,
    const std::string & ctext = "compute shader") [extern]
```

コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する.

引数

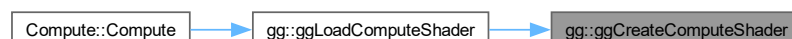
<i>csrc</i>	コンピュートシェーダのソースプログラムの文字列.
<i>ctext</i>	コンピュートシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 5187 行目に定義があります。

被呼び出し関係図:



7.1.3.6 ggCreateNormalMap()

```
void gg::ggCreateNormalMap (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format,
    GLfloat nz,
    GLenum internal,
    std::vector< GgVector > & nmap) [extern]
```

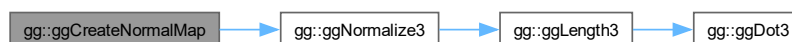
グレースケール画像 (8bit) から法線マップのデータを作成する。

引数

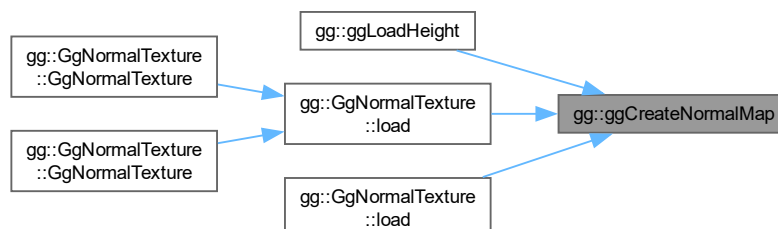
<i>hmap</i>	グレースケール画像のデータ.
<i>width</i>	高さマップのグレースケール画像 hmap の横の画素数.
<i>height</i>	高さマップのグレースケール画像 hmap の縦の画素数.
<i>format</i>	データの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線の z 成分の割合.
<i>internal</i>	法線マップを格納するテクスチャの内部フォーマット.
<i>nmap</i>	法線マップを格納する vector.

gg.cpp の 3927 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.7 ggCreateShader()

```
GLuint gg::ggCreateShader (
    const std::string & vsrc,
    const std::string & fsrc = "",
    const std::string & gsrc = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr,
    const std::string & vtext = "vertex shader",
    const std::string & ftext = "fragment shader",
    const std::string & gtext = "geometry shader") [extern]
```

シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する。

引数

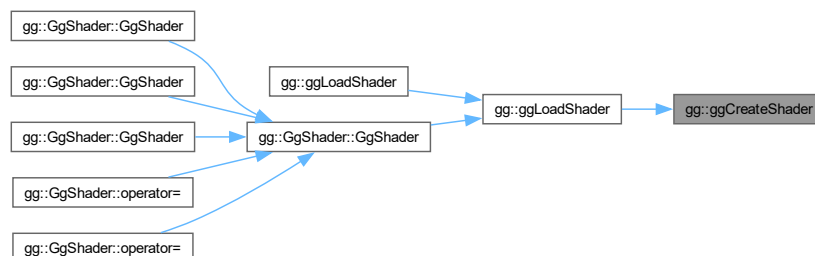
<i>vsrc</i>	バーテックスシェーダのソースプログラムの文字列.
<i>fsrc</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>gsrc</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).
<i>vtext</i>	バーテックスシェーダのコンパイル時のメッセージに追加する文字列.
<i>ftext</i>	フラグメントシェーダのコンパイル時のメッセージに追加する文字列.
<i>gtext</i>	ジオメトリシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の [4954](#) 行目に定義があります。

被呼び出し関係図:



7.1.3.8 ggCross() [1/2]

```
GgVector gg::ggCross (  
    const GgVector & a,  
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の外積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* の外積.

覚え書き

戻り値の *w* (第4) 要素は 0.

gg.h の 2037 行目に定義があります。

呼び出し関係図:



7.1.3.9 ggCross() [2/2]

```
void gg::ggCross (
    GLfloat * c,
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の外積.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.
<i>c</i>	結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1447 行目に定義があります。

被呼び出し関係図:



7.1.3.10 ggDistance3() [1/2]

```
GLfloat gg::ggDistance3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の距離.

引数

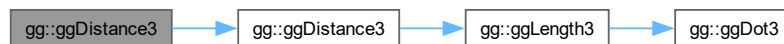
<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* の距離.

gg.h の 2074 行目に定義があります。

呼び出し関係図:



7.1.3.11 ggDistance3() [2/2]

```
GLfloat gg::ggDistance3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の距離.

引数

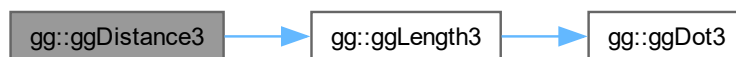
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.

戻り値

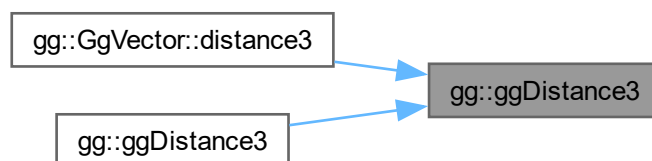
`a` と `b` の距離.

`gg.h` の 1484 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.12 ggDistance4() [1/2]

```

GLfloat gg::ggDistance4 (
    const GgVector & a,
    const GgVector & b) [inline]
  
```

`GgVector` 型の 4 要素の距離.

引数

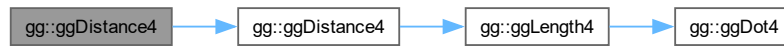
<code>a</code>	<code>GgVector</code> 型の変数.
<code>b</code>	<code>GgVector</code> 型の変数.

戻り値

2 つの `GgVector` `a`, `b` の距離.

`gg.h` の 2138 行目に定義があります。

呼び出し関係図:



7.1.3.13 ggDistance4() [2/2]

```

GLfloat gg::ggDistance4 (
    const GLfloat * a,
    const GLfloat * b) [inline]
  
```

`GLfloat` 型の 4 要素の距離.

引数

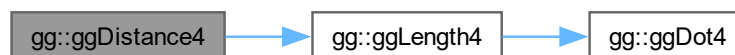
<code>a</code>	<code>GLfloat</code> 型の 4 要素の配列変数.
<code>b</code>	<code>GLfloat</code> 型の 4 要素の配列変数.

戻り値

`a` と `b` のそれぞれの 4 要素の距離.

`gg.h` の 1549 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.14 ggDot3() [1/2]

```
GLfloat gg::ggDot3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の内積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* のそれぞれの 3 要素の内積.

[gg.h](#) の 2051 行目に定義があります。

呼び出し関係図:



7.1.3.15 ggDot3() [2/2]

```
GLfloat gg::ggDot3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の内積.

引数

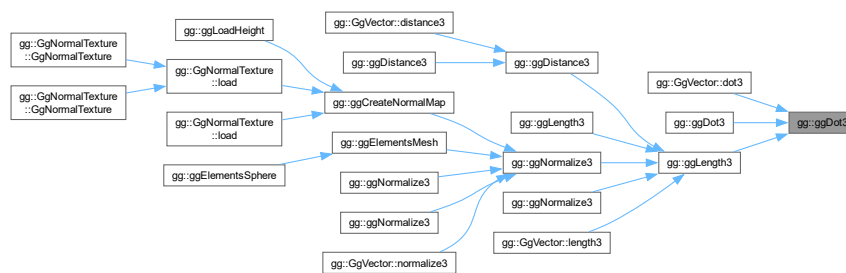
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.

戻り値

a と *b* のそれぞれの 3 要素の内積.

gg.h の 1461 行目に定義があります。

被呼び出し関係図:



7.1.3.16 ggDot4() [1/2]

```

GLfloat gg::ggDot4 (
    const GgVector & a,
    const GgVector & b) [inline]

```

GgVector 型の 4 要素の内積.

引数

<i>a</i>	GgVector 型の変数.
<i>b</i>	GgVector 型の変数.

戻り値

a と *b* のそれぞれの 4 要素の内積.

gg.h の 2115 行目に定義があります。

呼び出し関係図:



<i>pos</i>	メッシュの頂点の位置.
<i>norm</i>	メッシュの頂点の法線, nullptr なら頂点の位置から算出する.

戻り値

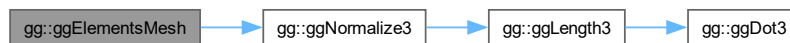
[GgElements](#) 型のポインタ.

覚え書き

メッシュ状に [GgElements](#) 形式の三角形データを生成する.

[gg.cpp](#) の 5450 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.19 ggElementsObj()

```
std::shared_ptr< gg::GgElements > gg::ggElementsObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイル を読み込む (Elements 形式).

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

`GgElements` 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイル を読み込んで `GgElements` 形式の三角形データを生成する.

`gg.cpp` の 5432 行目に定義があります.

呼び出し関係図:



7.1.3.20 ggElementsSphere()

```

std::shared_ptr< GgElements > gg::ggElementsSphere (
    GLfloat radius = 1.0f,
    int slices = 16,
    int stacks = 8) [extern]
  
```

球状に三角形データを生成する (Elements 形式).

引数

<i>radius</i>	球の半径.
<i>slices</i>	球の経度方向の分割数.
<i>stacks</i>	球の緯度方向の分割数.

戻り値

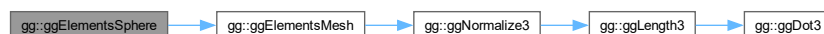
`GgElements` 型のポインタ.

覚え書き

球状に `GgElements` 形式の三角形データを生成する.

`gg.cpp` の 5545 行目に定義があります.

呼び出し関係図:



7.1.3.21 ggEllipse()

```
std::shared_ptr< gg::GgTriangles > gg::ggEllipse (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f,
    GLuint slices = 16) [extern]
```

楕円状に三角形を生成する。

引数

<i>width</i>	楕円の横幅.
<i>height</i>	楕円の高さ.
<i>slices</i>	楕円の分割数.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5391 行目に定義があります。

7.1.3.22 ggEulerQuaternion() [1/3]

```
GgQuaternion gg::ggEulerQuaternion (
    const GgVector & e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す。

引数

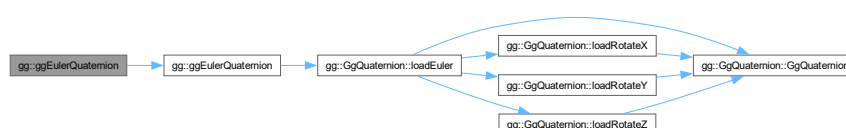
<i>e</i>	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
----------	--

戻り値

回転を表す四元数.

[gg.h](#) の 4713 行目に定義があります。

呼び出し関係図:



7.1.3.23 ggEulerQuaternion() [2/3]

```
GgQuaternion gg::ggEulerQuaternion (  
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す.

引数

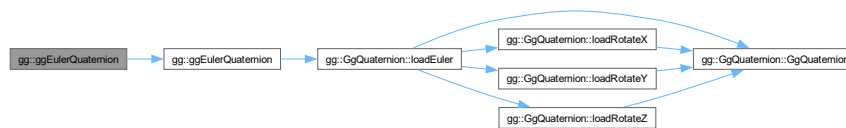
<i>e</i>	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

回転を表す四元数.

gg.h の 4702 行目に定義があります。

呼び出し関係図:



7.1.3.24 ggEulerQuaternion() [3/3]

```

GgQuaternion gg::ggEulerQuaternion (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) [inline]
  
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を返す.

引数

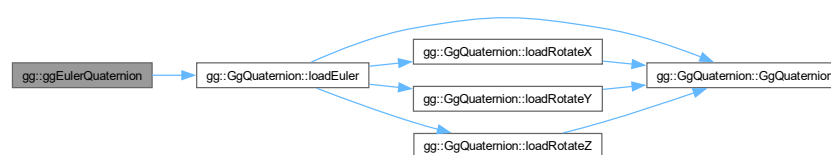
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

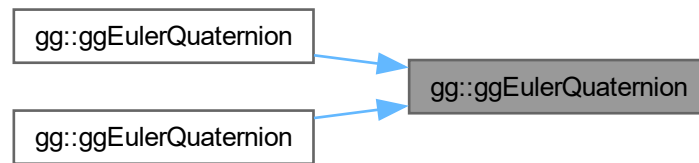
回転を表す四元数.

gg.h の 4690 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.25 ggFrustum()

```
GgMatrix gg::ggFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

透視投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3431 行目に定義があります。

呼び出し関係図:



7.1.3.26 ggIdentity()

```
GgMatrix gg::ggIdentity () [inline]
```

単位行列を返す.

戻り値

単位行列.

gg.h の 3156 行目に定義があります。

呼び出し関係図:



7.1.3.27 ggIdentityQuaternion()

```
GgQuaternion gg::ggIdentityQuaternion () [inline]
```

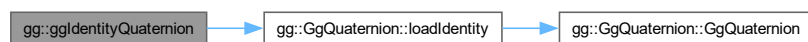
単位四元数を返す.

戻り値

単位四元数.

gg.h の 4588 行目に定義があります。

呼び出し関係図:



7.1.3.28 ggInit()

```
void gg::ggInit () [extern]
```

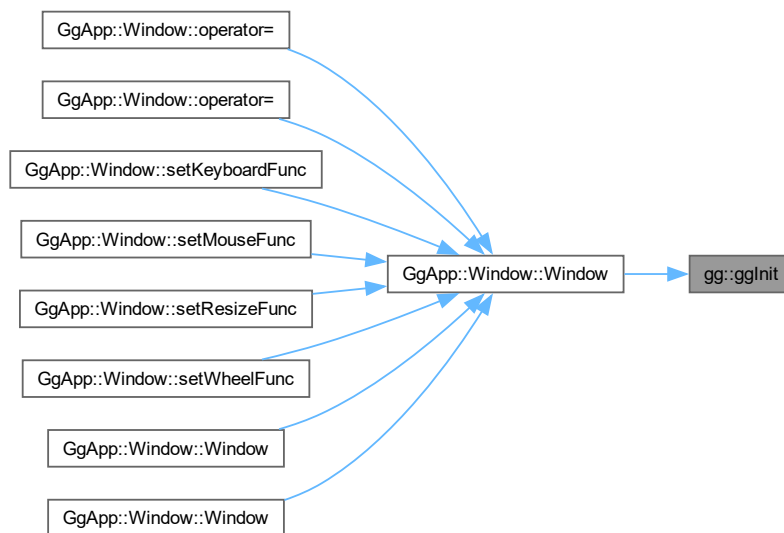
ゲームグラフィックス特論の都合にもとづく初期化を行う。

覚え書き

Windows において OpenGL 1.2 以降の API を有効化する。

gg.cpp の 1362 行目に定義があります。

被呼び出し関係図:



7.1.3.29 ggInvert() [1/2]

```
GgMatrix gg::ggInvert (
    const GgMatrix & m) [inline]
```

逆行列を返す。

引数

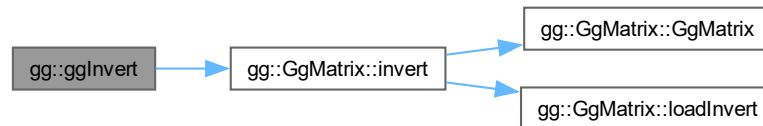
<i>m</i>	元の変換行列.
----------	---------

戻り値

m の逆行列.

gg.h の 3476 行目に定義があります。

呼び出し関係図:



7.1.3.30 ggInvert() [2/2]

```
GgQuaternion gg::ggInvert (
    const GgQuaternion & q) [inline]
```

四元数の逆元を求める.

引数

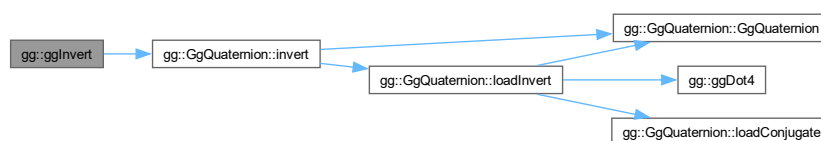
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 q の逆元.

gg.h の 4810 行目に定義があります。

呼び出し関係図:



7.1.3.31 ggLength3() [1/2]

```
GLfloat gg::ggLength3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の長さ.

引数

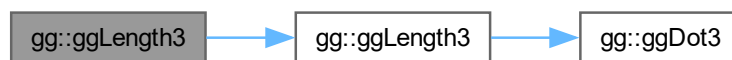
<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

a の 3 要素の長さ.

gg.h の 2062 行目に定義があります。

呼び出し関係図:



7.1.3.32 ggLength3() [2/2]

```
GLfloat gg::ggLength3 (  
    const GLfloat * a) [inline]
```

3 要素の長さ.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
----------	-----------------------

戻り値

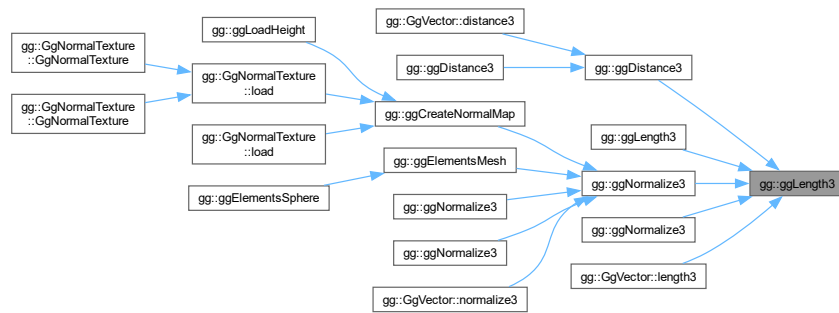
a の 3 要素の長さ.

gg.h の 1472 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.33 ggLength4() [1/2]

```
GLfloat gg::ggLength4 (
    const GgVector & a) [inline]
```

GgVector 型の 4 要素の長さ.

引数

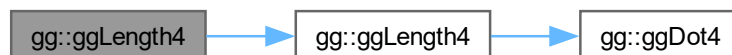
a	GgVector 型の変数.
-----	----------------

戻り値

a の 4 要素の長さ.

gg.h の 2126 行目に定義があります。

呼び出し関係図:



7.1.3.34 ggLength4() [2/2]

```
GLfloat gg::ggLength4 (
    const GLfloat * a) [inline]
```

GLfloat 型の 4 要素の長さ.

引数

<code>a</code>	GLfloat 型の 4 要素の配列変数.
----------------	-----------------------

戻り値

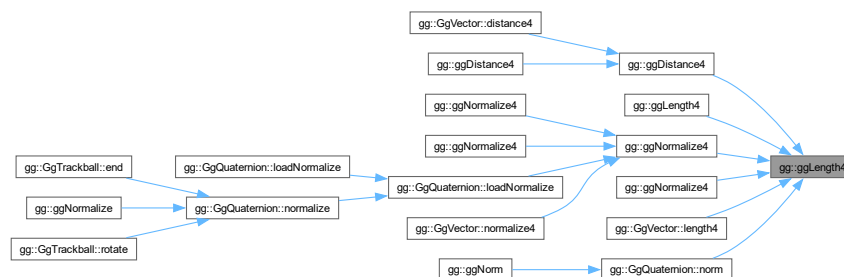
`a` の 4 要素の長さ.

`gg.h` の 1537 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.35 ggLoadComputeShader()

```

GLuint gg::ggLoadComputeShader (
    const std::string & comp) [extern]
  
```

コンピュータシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<code>comp</code>	コンピュータシェーダのソースファイル名.
-------------------	----------------------

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 5228 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.36 ggLoadHeight()

```

GLuint gg::ggLoadHeight (
    const std::string & name,
    GLfloat nz,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA) [extern]
  
```

TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する。

引数

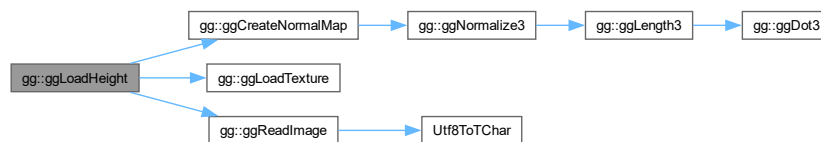
<i>name</i>	読み込むファイル名.
<i>nz</i>	法線の z 成分の割合.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 4012 行目に定義があります。

呼び出し関係図:



7.1.3.37 ggLoadImage()

```

GLuint gg::ggLoadImage (
    const std::string & name,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE) [extern]
  
```

テクスチャを作成して TGA フォーマットの画像ファイルを読み込む。

引数

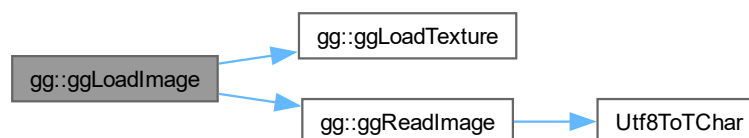
<i>name</i>	読み込むファイル名.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	テクスチャの内部フォーマット, デフォルトは GL_RGBA. 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3878 行目に定義があります。

呼び出し関係図:



7.1.3.38 ggLoadShader() [1/2]

```
GLuint gg::ggLoadShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.h](#) の 5199 行目に定義があります。

呼び出し関係図:



7.1.3.39 ggLoadShader() [2/2]

```
GLuint gg::ggLoadShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [extern]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).

<code>varyings</code>	フィードバックする <code>varying</code> 変数のリスト (<code>nullptr</code> なら不使用).
-----------------------	--

戻り値

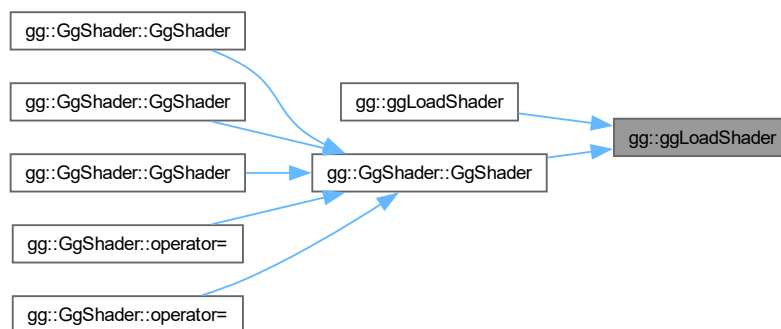
プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 5154 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.40 ggLoadSimpleObj() [1/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルと MTL ファイルを読み込む (Arrays 形式)

引数

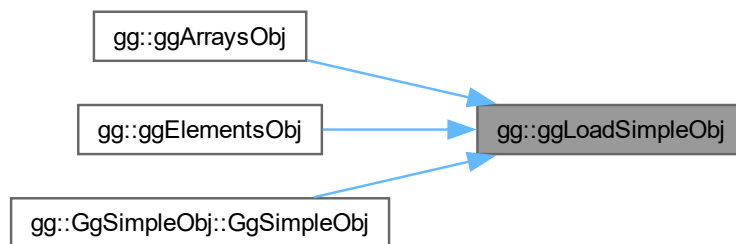
<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの <code>GgSimpleShader::Material</code> 型の材質.
<i>vert</i>	読み込んだデータの頂点属性.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら true.

`gg.cpp` の 4679 行目に定義があります。

被呼び出し関係図:



7.1.3.41 ggLoadSimpleObj() [2/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    std::vector< GLuint > & face,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルを読み込む (Elements 形式).

引数

<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの材質.

<i>vert</i>	読み込んだデータの頂点属性.
<i>face</i>	読み込んだデータの三角形の頂点インデックス.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら true.

[gg.cpp](#) の 4768 行目に定義があります。

7.1.3.42 ggLoadTexture()

```
GLuint gg::ggLoadTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = false) [extern]
```

テクスチャを作成して確保して画像データをテクスチャとして読み込む。

引数

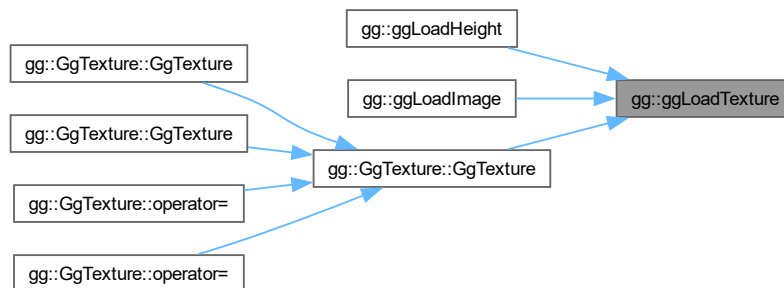
<i>image</i>	テクスチャとして読み込むデータ, nullptr ならテクスチャの作成のみを行う.
<i>width</i>	テクスチャとして読み込むデータ <i>image</i> の横の画素数.
<i>height</i>	テクスチャとして読み込むデータ <i>image</i> の縦の画素数.
<i>format</i>	<i>image</i> のフォーマット.
<i>type</i>	<i>image</i> のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

[gg.cpp](#) の 3823 行目に定義があります。

被呼び出し関係図:



7.1.3.43 ggLookat() [1/3]

```
GgMatrix gg::ggLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]
```

ビュー変換行列を返す.

引数

<i>e</i>	視点の位置を格納した <code>GgVector</code> 型の変数.
<i>t</i>	目標点の位置を格納した <code>GgVector</code> 型の変数.
<i>u</i>	上方向のベクトルを格納した <code>GgVector</code> 型の変数.

戻り値

求めたビュー変換行列.

`gg.h` の 3391 行目に定義があります。

呼び出し関係図:



7.1.3.44 ggLookat() [2/3]

```
GgMatrix gg::ggLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
```

ビュー変換行列を返す.

引数

<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

求めたビュー変換行列.

[gg.h](#) の 3373 行目に定義があります。

呼び出し関係図:



7.1.3.45 ggLookat() [3/3]

```
GgMatrix gg::ggLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz) [inline]
```

ビュー変換行列を返す.

引数

<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

求めたビュー変換行列.

[gg.h](#) の 3355 行目に定義があります。

呼び出し関係図:

gg::ggLookat



gg::GgMatrix::loadLookat

7.1.3.46 ggMatrixQuaternion() [1/2]

```
GgQuaternion gg::ggMatrixQuaternion (  
    const GgMatrix & m) [inline]
```

回転の変換行列 *m* を表す四元数を返す.

引数

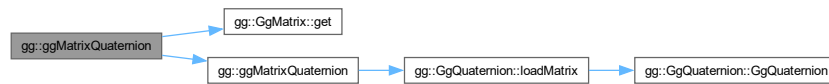
<i>m</i>	GgMatrix 型の変換行列.
----------	----------------------------------

戻り値

`m` による回転の変換に相当する四元数.

`gg.h` の 4612 行目に定義があります。

呼び出し関係図:



7.1.3.47 ggMatrixQuaternion() [2/2]

```
GgQuaternion gg::ggMatrixQuaternion (
    const GLfloat * a) [inline]
```

回転の変換行列 `m` を表す四元数を返す.

引数

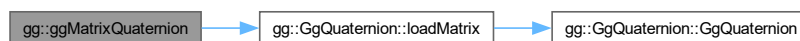
<code>a</code>	GLfloat 型の 16 要素の配列変数.
----------------	------------------------

戻り値

`a` による回転の変換に相当する四元数.

`gg.h` の 4600 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.48 ggNorm()

```
GLfloat gg::ggNorm (
    const GgQuaternion & q) [inline]
```

四元数のノルムを返す。

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* のノルム.

gg.h の 4777 行目に定義があります。

呼び出し関係図:



7.1.3.49 ggNormal()

```
GgMatrix gg::ggNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を返す。

引数

<i>m</i>	元の変換行列.
----------	---------

戻り値

m の法線変換行列.

gg.h の 3487 行目に定義があります。

呼び出し関係図:



7.1.3.50 ggNormalize()

```
GgQuaternion gg::ggNormalize (
    const GgQuaternion & q) [inline]
```

正規化した四元数を返す.

引数

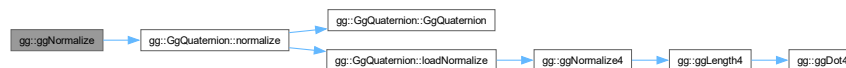
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* を正規化した四元数.

gg.h の 4788 行目に定義があります。

呼び出し関係図:



7.1.3.51 ggNormalize3() [1/4]

```
GgVector gg::ggNormalize3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

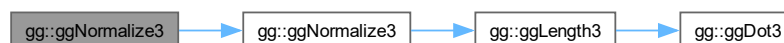
a の 3 要素を正規化したもの.

覚え書き

戻り値の *w* (第4) 要素は操作しない.

gg.h の 2088 行目に定義があります。

呼び出し関係図:



7.1.3.52 ggNormalize3() [2/4]

```
void gg::ggNormalize3 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

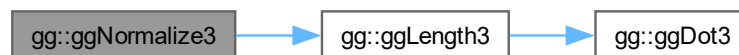
3 要素の正規化.

引数

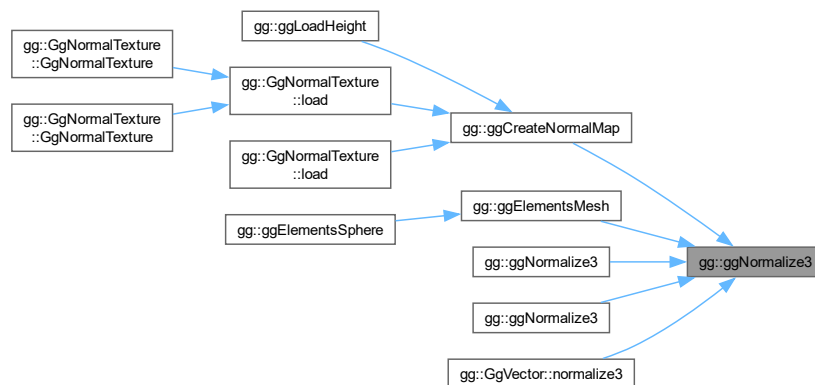
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	<i>a</i> の 3 要素を正規化した結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1496 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



7.1.3.53 ggNormalize3() [3/4]

```
void gg::ggNormalize3 (
    GgVector * a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

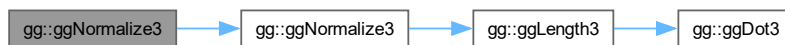
<i>a</i>	<code>GgVector</code> 型の変数のポインタ.
----------	----------------------------------

覚え書き

a の *w* (第4) 要素は操作しない.

`gg.h` の 2103 行目に定義があります。

呼び出し関係図:



7.1.3.54 `ggNormalize3()` [4/4]

```
void gg::ggNormalize3 (
    GLfloat * a) [inline]
```

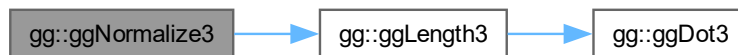
3 要素の正規化.

引数

<i>a</i>	<code>GLfloat</code> 型の 3 要素の配列変数.
----------	------------------------------------

`gg.h` の 1510 行目に定義があります。

呼び出し関係図:



7.1.3.55 `ggNormalize4()` [1/4]

```
GgVector gg::ggNormalize4 (
    const GgVector & a) [inline]
```

`GgVector` 型の 4 要素の正規化.

引数

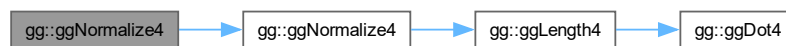
<i>a</i>	GgVector 型の変数.
----------	----------------

戻り値

a の 4 要素を正規化した結果.

gg.h の 2149 行目に定義があります。

呼び出し関係図:



7.1.3.56 ggNormalize4() [2/4]

```
void gg::ggNormalize4 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

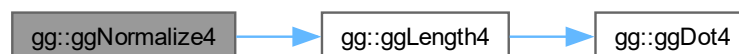
GLfloat 型の 4 要素の正規化.

引数

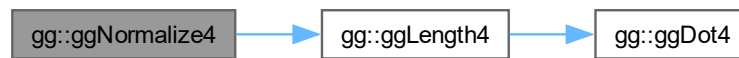
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	<i>a</i> を正規化した結果を格納する GLfloat 型の 4 要素の配列変数.

gg.h の 1561 行目に定義があります。

呼び出し関係図:



呼び出し関係図:



7.1.3.59 ggOrthogonal()

```
GgMatrix gg::ggOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

直交投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた直交投影変換行列.

gg.h の 3412 行目に定義があります。

呼び出し関係図:



7.1.3.60 ggPerspective()

```
GgMatrix gg::ggPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

画角を指定して透視投影変換行列を返す.

引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3450 行目に定義があります。

呼び出し関係図:



7.1.3.61 ggPointsCube()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsCube (
    GLsizei countv,
    GLfloat length = 1.0f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を立方体状に生成する.

引数

<i>countv</i>	生成する点の数.
---------------	----------

<i>length</i>	点群を生成する立方体の一辺の長さ.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5315 行目に定義があります。

7.1.3.62 ggPointsSphere()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsSphere (
    GLsizei countv,
    GLfloat radius = 0.5f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を球状に生成する.

引数

<i>countv</i>	生成する点の数.
<i>radius</i>	点群を生成する半径.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5342 行目に定義があります。

7.1.3.63 ggQuaternionMatrix()

```
GgMatrix gg::ggQuaternionMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の変換行列を返す.

引数

q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する `GgMatrix` 型の変換行列.

`gg.h` の 4623 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.64 ggQuaternionTransposeMatrix()

```
GgMatrix gg::ggQuaternionTransposeMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の転置した変換行列を返す.

引数

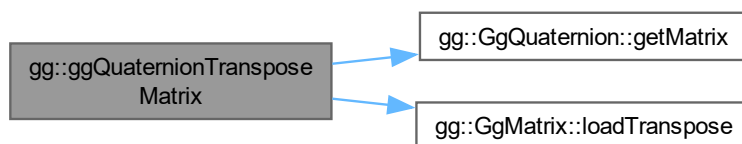
q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する転置した `GgMatrix` 型の変換行列.

`gg.h` の 4636 行目に定義があります。

呼び出し関係図:



7.1.3.65 ggReadImage()

```
bool gg::ggReadImage (
    const std::string & name,
    std::vector< GLubyte > & image,
    GLsizei * pWidth,
    GLsizei * pHeight,
    GLenum * pFormat) [extern]
```

TGA ファイル (8/16/24/32bit) をメモリに読み込む.

引数

<i>name</i>	読み込むファイル名.
<i>image</i>	読み込んだデータを格納する <code>vector</code> .
<i>pWidth</i>	読み込んだ画像の横の画素数の格納先のポインタ, <code>nullptr</code> なら格納しない.
<i>pHeight</i>	読み込んだ画像の縦の画素数の格納先のポインタ, <code>nullptr</code> なら格納しない.
<i>pFormat</i>	読み込んだファイルの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA) の格納先のポインタ, <code>nullptr</code> なら格納しない.

戻り値

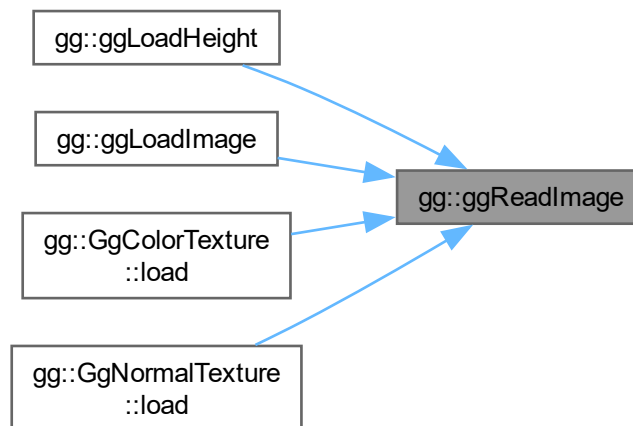
読み込みに成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3690 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.66 ggRectangle()

```

std::shared_ptr< gg::GgTriangles > gg::ggRectangle (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f) [extern]
  
```

矩形状に 2 枚の三角形を生成する.

引数

<i>width</i>	矩形の横幅.
<i>height</i>	矩形の高さ.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5370 行目に定義があります。

7.1.3.67 ggRotate() [1/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルと回転角を表す GgVector 型の変数.
-----	--

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸に $r[3]$ だけ回転する変換行列.

[gg.h](#) の 3335 行目に定義があります。

呼び出し関係図:



7.1.3.68 ggRotate() [2/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルを表す GgVector 型の変数.
a	回転角.

戻り値

r を軸に a だけ回転する変換行列.

[gg.h](#) の 3311 行目に定義があります。

呼び出し関係図:



7.1.3.69 ggRotate() [3/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルと回転角を表す GLfloat 型の 4 要素の配列変数.
-----	---------------------------------------

戻り値

$[r[0], r[1], r[2]]$ を軸に $r[3]$ だけ回転する変換行列.

[gg.h](#) の 3323 行目に定義があります。

呼び出し関係図:



7.1.3.70 ggRotate() [4/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

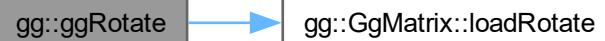
<i>r</i>	回転軸のベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

r を軸に *a* だけ回転する変換行列.

gg.h の 3298 行目に定義があります。

呼び出し関係図:



7.1.3.71 ggRotate() [5/5]

```
GgMatrix gg::ggRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

(x, y, z) を軸にさらに *a* 回転する変換行列.

gg.h の 3285 行目に定義があります。

呼び出し関係図:



7.1.3.72 ggRotateQuaternion() [1/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を返す.

引数

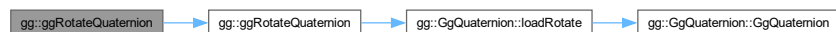
v	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

回転を表す四元数.

gg.h の 4677 行目に定義があります。

呼び出し関係図:



7.1.3.73 ggRotateQuaternion() [2/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を返す.

引数

<i>v</i>	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
<i>a</i>	回転角.

戻り値

回転を表す四元数.

gg.h の 4666 行目に定義があります。

呼び出し関係図:



7.1.3.74 ggRotateQuaternion() [3/3]

```

GgQuaternion gg::ggRotateQuaternion (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
  
```

(x, y, z) を軸として角度 a 回転する四元数を返す.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

戻り値

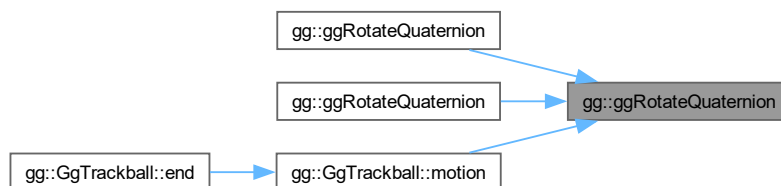
回転を表す四元数.

gg.h の 4653 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.75 ggRotateX()

```
GgMatrix gg::ggRotateX (
    GLfloat a) [inline]
```

x 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

x 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3246 行目に定義があります。

呼び出し関係図:



7.1.3.76 ggRotateY()

```
GgMatrix gg::ggRotateY (
    GLfloat a) [inline]
```

y 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

y 軸中心に *a* だけ回転する変換行列.

gg.h の 3258 行目に定義があります。

呼び出し関係図:



7.1.3.77 ggRotateZ()

```
GgMatrix gg::ggRotateZ (  
    GLfloat a) [inline]
```

z 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

z 軸中心に *a* だけ回転する変換行列.

gg.h の 3270 行目に定義があります。

呼び出し関係図:



7.1.3.78 ggSaveColor()

```
bool gg::ggSaveColor (
    const std::string & name) [extern]
```

カラーバッファの内容を TGA ファイルに保存する.

引数

<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば true, 失敗すれば false.

gg.cpp の 3634 行目に定義があります。

呼び出し関係図:



7.1.3.79 ggSaveDepth()

```
bool gg::ggSaveDepth (  
    const std::string & name) [extern]
```

デブスバッファの内容を TGA ファイルに保存する.

引数

<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば true, 失敗すれば false.

gg.cpp の 3660 行目に定義があります。

呼び出し関係図:



7.1.3.80 ggSaveTga()

```
bool gg::ggSaveTga (
    const std::string & name,
    const void * buffer,
    unsigned int width,
    unsigned int height,
    unsigned int depth) [extern]
```

配列の内容を TGA ファイルに保存する。

引数

<i>name</i>	保存するファイル名.
<i>buffer</i>	画像データを格納した配列.
<i>width</i>	画像の横の画素数.
<i>height</i>	画像の縦の画素数.
<i>depth</i>	1画素のバイト数.

戻り値

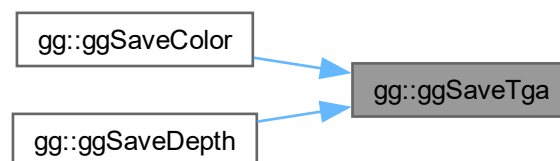
保存に成功すれば true, 失敗すれば false.

gg.cpp の 3558 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.81 ggScale() [1/3]

```
GgMatrix gg::ggScale (  
    const GgVector & s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の GgVector 型の変数.
---	---------------------

戻り値

拡大縮小の変換行列.

gg.h の 3234 行目に定義があります。

呼び出し関係図:

**7.1.3.82 ggScale()** [2/3]

```
GgMatrix gg::ggScale (  
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小の変換行列.

[gg.h](#) の [3222](#) 行目に定義があります。

呼び出し関係図:



7.1.3.83 ggScale() [3/3]

```
GgMatrix gg::ggScale (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w = 1.0f) [inline]
```

拡大縮小の変換行列を返す.

引数

<i>x</i>	<i>x</i> 方向の拡大率.
<i>y</i>	<i>y</i> 方向の拡大率.
<i>z</i>	<i>z</i> 方向の拡大率.
<i>w</i>	拡大率のスケールファクタ (= 1.0f).

戻り値

拡大縮小の変換行列.

[gg.h](#) の [3210](#) 行目に定義があります。

呼び出し関係図:



7.1.3.84 ggSlerp() [1/4]

```
GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

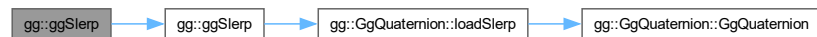
<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

q, *r* を *t* で内分した四元数.

gg.h の 4740 行目に定義があります。

呼び出し関係図:



7.1.3.85 ggSlerp() [2/4]

```
GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
    const GLfloat * a,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

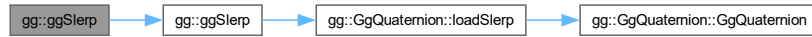
<i>q</i>	GgQuaternion 型の四元数.
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

q, a を t で内分した四元数.

`gg.h` の 4753 行目に定義があります。

呼び出し関係図:



7.1.3.86 ggSlerp() [3/4]

```
GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

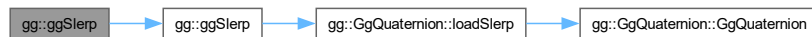
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
q	<code>GgQuaternion</code> 型の四元数.
t	補間パラメータ.

戻り値

a, q を t で内分した四元数.

`gg.h` の 4766 行目に定義があります。

呼び出し関係図:



7.1.3.87 ggSlerp() [4/4]

```
GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

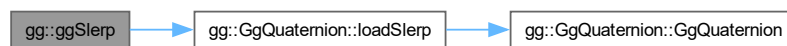
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

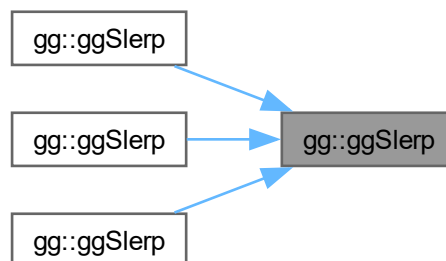
a, *b* を *t* で内分した四元数.

gg.h の 4726 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.88 ggTranslate() [1/3]

```
GgMatrix gg::ggTranslate (
    const GgVector & t) [inline]
```

平行移動の変換行列を返す.

引数

<i>t</i>	移動量の GgVector 型の変数.
----------	---------------------

戻り値

平行移動の変換行列.

[gg.h](#) の [3195](#) 行目に定義があります。

呼び出し関係図:



7.1.3.89 ggTranslate() [2/3]

```
GgMatrix gg::ggTranslate (  
    const GLfloat * t) [inline]
```

平行移動の変換行列を返す.

引数

<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

平行移動の変換行列.

[gg.h](#) の [3183](#) 行目に定義があります。

呼び出し関係図:



7.1.3.90 ggTranslate() [3/3]

```
GgMatrix gg::ggTranslate (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w = 1.0f) [inline]
```

平行移動の変換行列を返す.

引数

<i>x</i>	x 方向の移動量.
<i>y</i>	y 方向の移動量.
<i>z</i>	z 方向の移動量.
<i>w</i>	移動量のスケールファクタ (= 1.0f).

戻り値

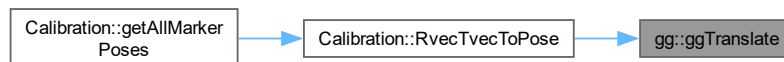
平行移動の変換行列.

[gg.h](#) の 3171 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.1.3.91 ggTranspose()

```
GgMatrix gg::ggTranspose (
    const GgMatrix & m) [inline]
```

転置行列を返す.

引数

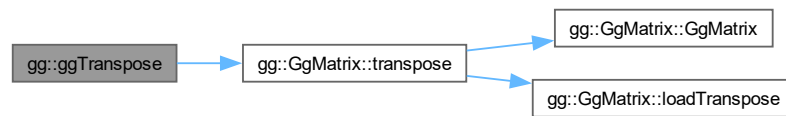
<i>m</i>	元の変換行列.
----------	---------

戻り値

m の転置行列.

gg.h の 3465 行目に定義があります。

呼び出し関係図:



7.1.3.92 operator*()

```
GgVector gg::operator* (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに GgVector 型の各要素を乗じた積を返す.

引数

a	GLfloat 型の値.
b	GgVector 型のベクトル.

戻り値

a に b の各要素を乗じた積.

gg.h の 2010 行目に定義があります。

7.1.3.93 operator+() [1/2]

```
const GgVector & gg::operator+ (
    const GgVector & v) [inline]
```

単項の + 演算子なので何もしない.

引数

v	GgVector 型のベクトル.
---	------------------

戻り値

v の値.

gg.h の 1963 行目に定義があります。

7.1.3.94 operator+() [2/2]

```
GgVector gg::operator+ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに **GgVector** 型の各要素を足した和を返す.

引数

a	GLfloat 型の値.
b	GgVector 型のベクトル.

戻り値

a に **b** の各要素を足した和.

gg.h の 1975 行目に定義があります。

7.1.3.95 operator-() [1/2]

```
const GgVector gg::operator- (
    const GgVector & v) [inline]
```

符号の反転.

引数

v	GgVector 型のベクトル.
----------	-------------------------

戻り値

v の値の符号を反転した結果.

gg.h の 1986 行目に定義があります。

7.1.3.96 operator-() [2/2]

```
GgVector gg::operator- (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーから **GgVector** 型の各要素を引いた差を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a から *b* の各要素を引いた差.

gg.h の 1998 行目に定義があります。

7.1.3.97 operator/()

```
GgVector gg::operator/ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーを GgVector 型の各要素で割った商を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a を *b* の各要素で割った商.

gg.h の 2022 行目に定義があります。

7.1.4 変数詳解

7.1.4.1 ggBufferAlignment

```
GLint gg::ggBufferAlignment [extern]
```

使用している GPU のバッファアライメント.

使用している GPU のバッファオブジェクトのアライメント, 初期化に取得される.

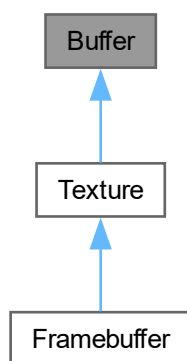
Chapter 8

クラス詳解

8.1 Buffer クラス

```
#include <Buffer.h>
```

Buffer の継承関係図



公開メンバ関数

- `Buffer ()=default`
- `Buffer (GLsizei width, GLsizei height, int channels)`
- `Buffer (const Buffer &buffer)`
- `Buffer (Buffer &&buffer) noexcept`
- `virtual ~Buffer ()`
- `Buffer & operator= (const Buffer &buffer)`
- `Buffer & operator= (Buffer &&buffer) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `void resize (GLsizei width, GLsizei height, int channels)`
- `void resize (const Buffer &buffer)`

- virtual void `copy` (const `Buffer` &buffer) noexcept
- virtual void `discard` ()
- auto `getBufferName` () const
- virtual const std::array< GLsizei, 2 > & `getSize` () const
- GLsizei `getWidth` () const
- GLsizei `getHeight` () const
- virtual int `getChannels` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

限定公開 メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const `Buffer` &buffer) noexcept

8.1.1 詳解

バッファクラス

フレームを格納するピクセルバッファオブジェクト

`Buffer.h` の 24 行目に定義があります。

8.1.2 構築子と解体子

8.1.2.1 `Buffer()` [1/4]

```
Buffer::Buffer () [default]
```


[Buffer.h](#) の 94 行目に定義があります。

呼び出し関係図:



8.1.2.3 Buffer() [3/4]

```
Buffer::Buffer (
    const Buffer & buffer) [inline]
```

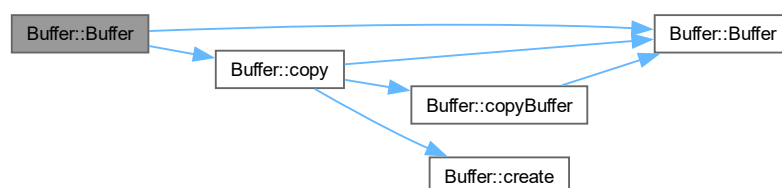
コピーコンストラクタ

引数

<i>buffer</i>	コピー元のバッファ
---------------	-----------

[Buffer.h](#) の 104 行目に定義があります。

呼び出し関係図:



8.1.2.4 Buffer() [4/4]

```
Buffer::Buffer (
    Buffer && buffer) [inline], [noexcept]
```

ムーブコンストラクタ

引数

<i>buffer</i>	ムーブ元のバッファ
---------------	-----------

[Buffer.h](#) の 114 行目に定義があります。

呼び出し関係図:



8.1.2.5 ~Buffer()

```
virtual Buffer::~~Buffer () [inline], [virtual]
```

デストラクタ

[Buffer.h](#) の 135 行目に定義があります。

呼び出し関係図:



8.1.3 関数詳解

8.1.3.1 bindBuffer()

```
void Buffer::bindBuffer (  
    GLenum target = GL_PIXEL_PACK_BUFFER) const [inline]
```

バッファのピクセルバッファオブジェクトを結合する

引数

<i>target</i>	結合するターゲット
---------------	-----------

[Buffer.h](#) の 294 行目に定義があります。

8.1.3.2 channelsToFormat()

```
auto Buffer::channelsToFormat (
    int channels) const [inline], [protected]
```

チャンネル数からフォーマットを求める

引数

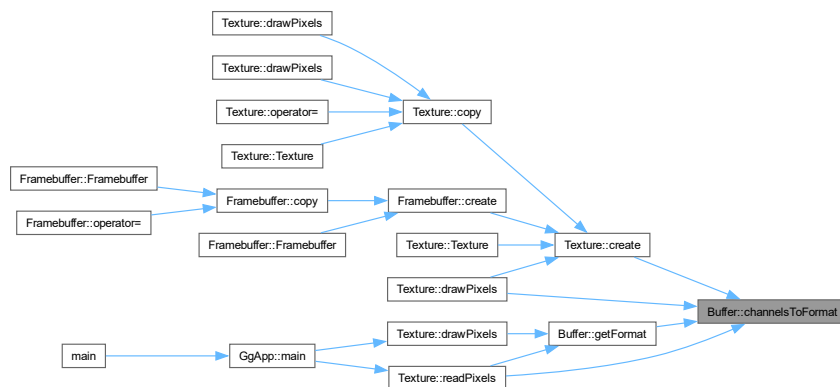
<i>channels</i>	フレームのチャンネル数
-----------------	-------------

戻り値

テクスチャのフォーマット

[Buffer.h](#) の 52 行目に定義があります。

被呼び出し関係図:



8.1.3.3 copy()

```
void Buffer::copy (
    const Buffer & buffer) [virtual], [noexcept]
```

バッファをコピーする

引数

<code>buffer</code>	コピー元のバッファ
---------------------	-----------

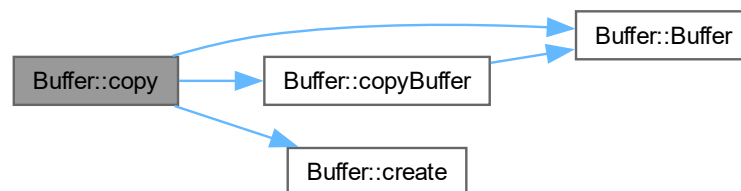
覚え書き

このバッファのサイズがコピー元のバッファのサイズと異なれば、このバッファを削除して新しいバッファを作り直してコピーする。

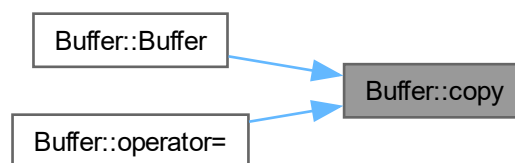
`Framebuffer`, `Texture`で再実装されています。

`Buffer.cpp` の 93 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.4 copyBuffer()

```
void Buffer::copyBuffer (
    const Buffer & buffer) [protected], [noexcept]
```

既存のバッファにコピーする

引数

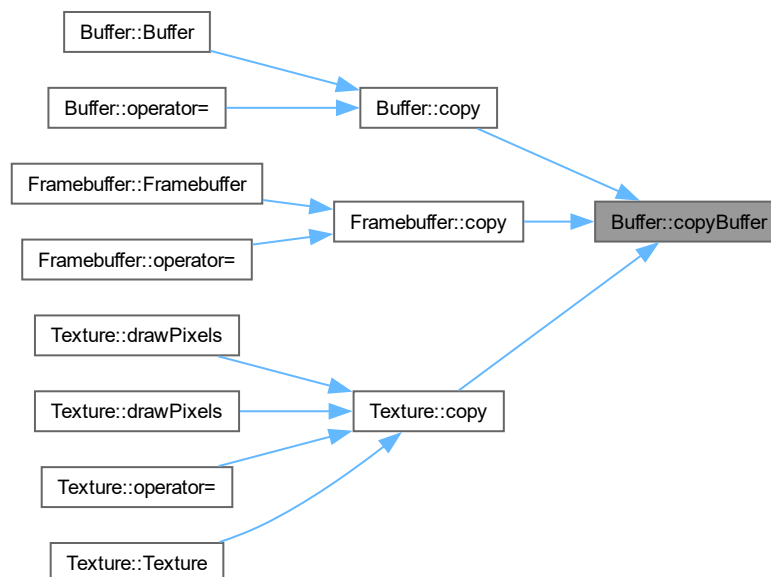
<i>buffer</i>	コピー元のバッファ
---------------	-----------

[Buffer.cpp](#) の 39 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.5 create()

```

void Buffer::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
  
```

バッファを作成する

引数

<i>width</i>	バッファに格納するフレームの横の画素数
<i>height</i>	バッファに格納するフレームの縦の画素数
<i>channels</i>	バッファに格納するフレームのチャンネル数

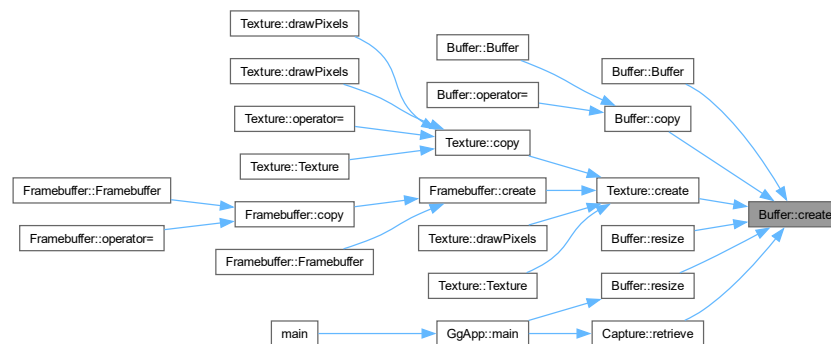
覚え書き

このバッファのサイズが引数で指定したサイズと異なれば、このバッファを削除して新しいバッファを作り直す。

Framebuffer, Textureで再実装されています。

Buffer.cpp の 60 行目に定義があります。

被呼び出し関係図:



8.1.3.6 discard()

```
void Buffer::discard () [virtual]
```

バッファを破棄する

Framebuffer, Textureで再実装されています。

Buffer.cpp の 187 行目に定義があります。

被呼び出し関係図:



8.1.3.7 formatToChannels()

```
int Buffer::formatToChannels (
    GLenum format) const [protected]
```

フォーマットからチャンネル数を求める

引数

<i>format</i>	テクスチャのフォーマット
---------------	--------------

戻り値

フレームのチャンネル数

[Buffer.cpp](#) の 16 行目に定義があります。

8.1.3.8 getAspect()

```
auto Buffer::getAspect () const [inline]
```

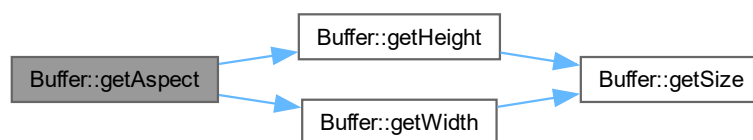
格納されているフレームの縦横比を得る

戻り値

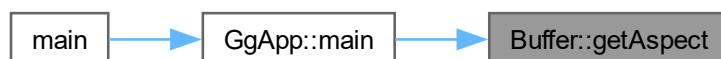
格納されているフレームの縦横比

[Buffer.h](#) の 284 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.9 getBufferName()

```
auto Buffer::getBufferName () const [inline]
```

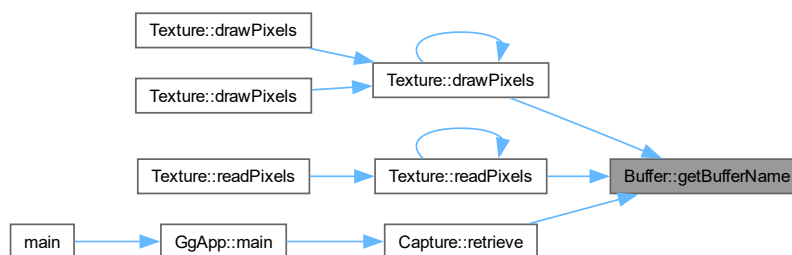
バッファのピクセルバッファオブジェクト名を得る

戻り値

ピクセルバッファオブジェクト名

[Buffer.h](#) の 221 行目に定義があります。

被呼び出し関係図:



8.1.3.10 getChannels()

```
virtual int Buffer::getChannels () const [inline], [virtual]
```

格納されているフレームのチャンネル数を得る

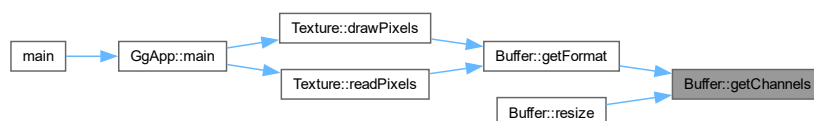
戻り値

格納されているフレームのチャンネル数

[Framebuffer](#), [Texture](#) で再実装されています。

[Buffer.h](#) の 264 行目に定義があります。

被呼び出し関係図:



8.1.3.11 getFormat()

```
auto Buffer::getFormat () const [inline]
```

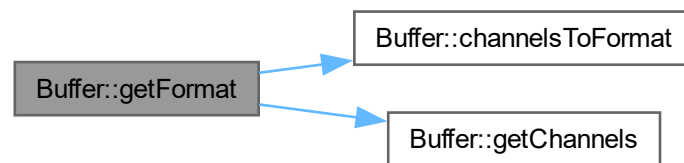
格納されているフレームのフォーマットを得る

戻り値

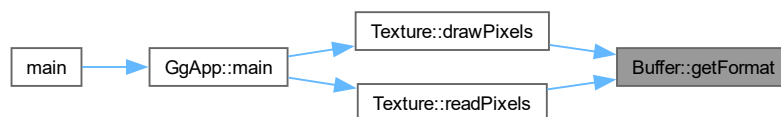
格納されているフレームのフォーマット

[Buffer.h](#) の 274 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.12 getHeight()

```
GLsizei Buffer::getHeight () const [inline]
```

格納されているフレームの縦の画素数を得る

戻り値

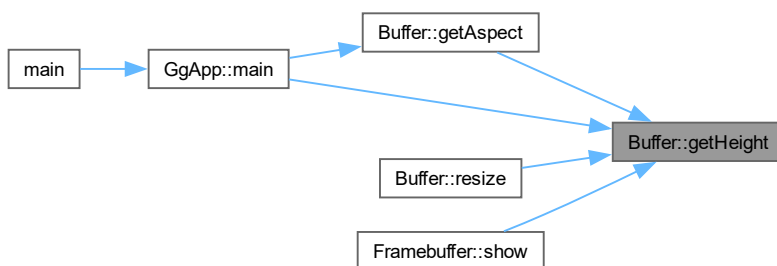
格納されているフレームの縦の画素数

[Buffer.h](#) の 254 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.13 getSize()

```
virtual const std::array< GLsizei, 2 > & Buffer::getSize () const [inline], [virtual]
```

格納されているフレームのサイズを得る

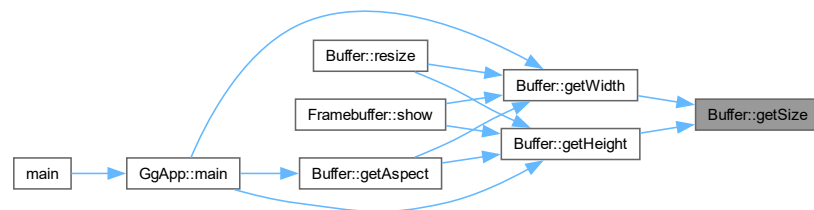
戻り値

格納されているフレームのサイズ

[Framebuffer](#), [Texture](#)で再実装されています。

[Buffer.h](#) の 234 行目に定義があります。

被呼び出し関係図:



8.1.3.14 getWidth()

```
GLsizei Buffer::getWidth () const [inline]
```

格納されているフレームの横の画素数を得る

戻り値

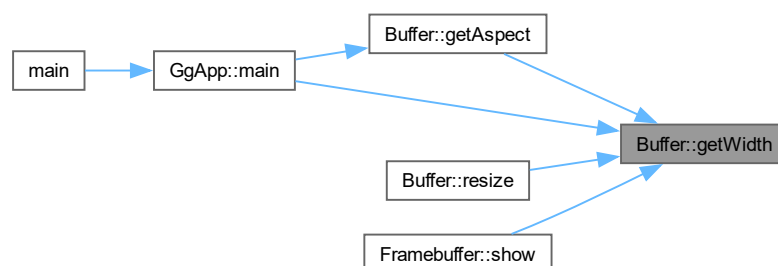
格納されているフレームの横の画素数

[Buffer.h](#) の 244 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.15 map()

```
GLvoid * Buffer::map () const
```

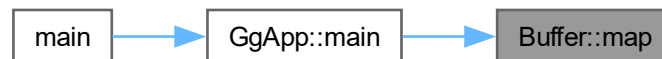
バッファのピクセルバッファオブジェクトをマップする

戻り値

ピクセルバッファオブジェクトをマップしたメモリ

[Buffer.cpp](#) の [152](#) 行目に定義があります。

被呼び出し関係図:



8.1.3.16 operator=() [1/2]

```
Buffer & Buffer::operator= (
    Buffer && buffer) [noexcept]
```

ムーブ代入演算子

引数

<i>buffer</i>	ムーブ代入元のバッファ
---------------	-------------

戻り値

ムーブ代入結果のバッファの参照

[Buffer.cpp](#) の [121](#) 行目に定義があります。

呼び出し関係図:



8.1.3.17 operator=() [2/2]

```
Buffer & Buffer::operator= (  
    const Buffer & buffer)
```

代入演算子

引数

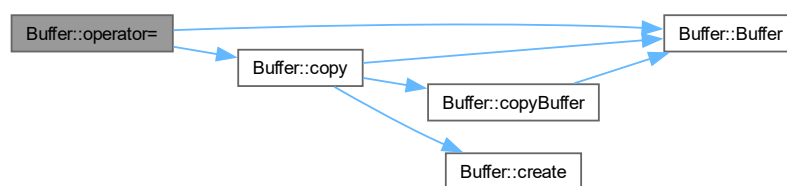
<i>buffer</i>	代入元のバッファ
---------------	----------

戻り値

代入結果のバッファの参照

[Buffer.cpp](#) の 106 行目に定義があります。

呼び出し関係図:



8.1.3.18 resize() [1/2]

```
void Buffer::resize (  
    const Buffer & buffer) [inline]
```

オブジェクトが保持するフレームのサイズを引数に指定したオブジェクトと同じにする

引数

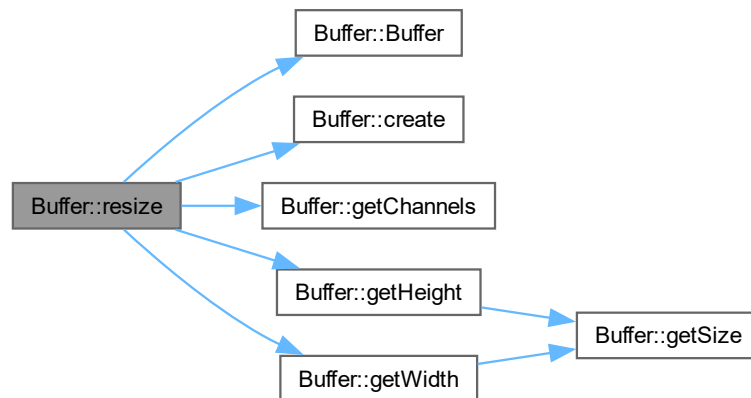
<i>buffer</i>	サイズの基準に用いるオブジェクト
---------------	------------------

覚え書き

このオブジェクトのサイズが引数で指定したオブジェクトのサイズと異なれば、このオブジェクトを削除して新しいオブジェクトを作り直す。

Buffer.h の 194 行目に定義があります。

呼び出し関係図:



8.1.3.19 `resize()` [2/2]

```
void Buffer::resize (
    GLsizei width,
    GLsizei height,
    int channels) [inline]
```

オブジェクトが保持するフレームのサイズを変更する

引数

<i>width</i>	フレームの横の画素数
<i>height</i>	フレームの縦の画素数
<i>channels</i>	フレームのチャンネル数

覚え書き

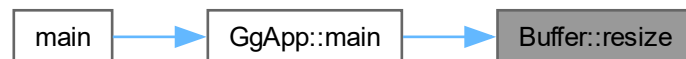
このオブジェクトのサイズが引数で指定したオブジェクトのサイズと異なれば、このオブジェクトを削除して新しいオブジェクトを作り直す。

[Buffer.h](#) の 180 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.1.3.20 unbindBuffer()

```
void Buffer::unbindBuffer (
    GLenum target = GL_PIXEL_PACK_BUFFER) const [inline]
```

バッファのピクセルバッファオブジェクトの結合を解除する

引数

<i>target</i>	結合を解除するターゲット
---------------	--------------

[Buffer.h](#) の 306 行目に定義があります。

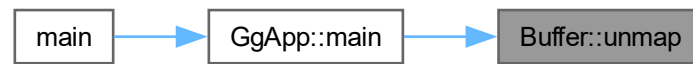
8.1.3.21 unmap()

```
void Buffer::unmap () const
```

バッファのピクセルバッファオブジェクトをアンマップする

[Buffer.cpp](#) の 173 行目に定義があります。

被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Buffer.h](#)
- [Buffer.cpp](#)

8.2 Calibration クラス

```
#include <Calibration.h>
```

公開メンバ関数

- [Calibration](#) (const std::string &dictionaryName, const std::array< int, 2 > &checkerSize, const std::array< float, 2 > &checkerLength)
- [Calibration](#) (const [Calibration](#) &calibration)=delete
- virtual ~[Calibration](#) ()
- [Calibration](#) & operator= (const [Calibration](#) &calibration)=delete
- void [createBoard](#) (const std::array< int, 2 > &checkerSize, const std::array< float, 2 > &checkerLength)
- void [setDictionary](#) (const std::string &dictionaryName, const std::array< int, 2 > &checkerSize, const std::array< float, 2 > &checkerLength)
- void [drawBoard](#) (cv::Mat &boardImage, int width, int height)
- void [detectBoard](#) (cv::Mat &image)
- void [detectMarkers](#) (cv::Mat &image, float markerLength)
- void [recordCorners](#) ()
- void [discardCorners](#) ()
- void [updateMotion](#) (float deltaTime, float motionThresholdPx=2.0f, float minStableTime=0.6f, int minCorners=6)
- bool [isStable](#) () const
- float [getStableProgress](#) (float minStableTime=0.6f) const
- float [getCurrentMotion](#) () const
- bool [isDiverseEnough](#) (float minDistanceRatio=0.05f, float minScaleRatio=0.12f, float minAngleDeg=8.0f) const
- bool [calibrate](#) ()
- auto [getCornersCount](#) () const
- auto [getSampleCount](#) () const
- auto [getTotalCount](#) () const
- const auto & [getCameraMatrix](#) () const
- const auto & [getDistortionCoefficients](#) () const
- auto [getReprojectionError](#) () const
- auto [finished](#) ()
- [GgMatrix RvecTvecToPose](#) (const cv::Vec3d &rvec, const cv::Vec3d &tvec)
- void [getAllMarkerPoses](#) (float markerLength, std::map< int, [GgMatrix](#) > &poses)
- bool [loadParameters](#) (const std::string &filename)
- bool [saveParameters](#) (const std::string &filename) const

静的公開変数類

- static const std::map< const std::string, const cv::aruco::PredefinedDictionaryType > [dictionaryList](#)
ArUco Marker 辞書のリスト

8.2.1 詳解

較正クラス

[Calibration.h](#) の 24 行目に定義があります。

8.2.2 構築子と解体子

8.2.2.1 Calibration() [1/2]

```
Calibration::Calibration (
    const std::string & dictionaryName,
    const std::array< int, 2 > & checkerSize,
    const std::array< float, 2 > & checkerLength)
```

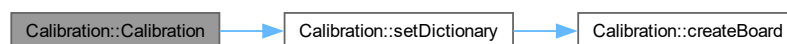
較正オブジェクトのコンストラクタ

引数

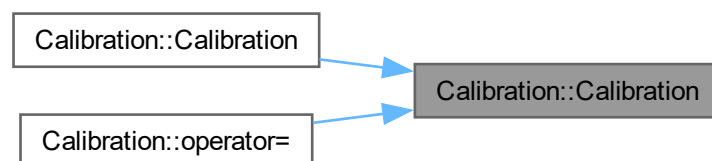
<i>dictionaryName</i>	ArUco Marker の辞書名
<i>checkerSize</i>	ChArUco Board のマス目の横と縦の数
<i>checkerLength</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 29 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.2.2 Calibration() [2/2]

```
Calibration::Calibration (
    const Calibration & calibration) [delete]
```

コピーコンストラクタは使用しない

引数

<i>calibration</i>	コピー元
--------------------	------

呼び出し関係図:



8.2.2.3 ~Calibration()

```
Calibration::~~Calibration () [virtual]
```

デストラクタ

[Calibration.cpp](#) の 39 行目に定義があります。

8.2.3 関数詳解

8.2.3.1 calibrate()

```
bool Calibration::calibrate ()
```

校正する

戻り値

校正に成功したら true

[Calibration.cpp](#) の 256 行目に定義があります。

呼び出し関係図:



8.2.3.2 createBoard()

```
void Calibration::createBoard (
    const std::array< int, 2 > & checkerSize,
    const std::array< float, 2 > & checkerLength)
```

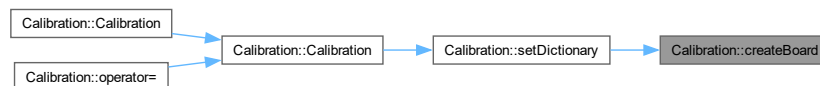
ChArUco Board を作成する

引数

<i>checkerSize</i>	ChArUco Board のマス目の横と縦の数
<i>checkerLength</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 46 行目に定義があります。

被呼び出し関係図:



8.2.3.3 detectBoard()

```
void Calibration::detectBoard (
    cv::Mat & image)
```

ChArUco Board を検出する

引数

<i>image</i>	ChArUco Board を検出する画像
--------------	-----------------------

[Calibration.cpp](#) の 97 行目に定義があります。

被呼び出し関係図:



8.2.3.4 detectMarkers()

```
void Calibration::detectMarkers (
    cv::Mat & image,
    float markerLength)
```

ArUco Marker を検出する

引数

<i>image</i>	ArUco Marker を検出する画像
<i>markerLength</i>	ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 133 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.3.5 discardCorners()

```
void Calibration::discardCorners ()
```

取得した標本と校正結果を破棄する

[Calibration.cpp](#) の 228 行目に定義があります。

被呼び出し関係図:



8.2.3.6 drawBoard()

```
void Calibration::drawBoard (
    cv::Mat & boardImage,
    int width,
    int height)
```

ChArUco Board を描く

引数

<i>boardImage</i>	ChArUco Board を描き込む画像
<i>width</i>	ChArUco Board の横の画素数
<i>height</i>	ChArUco Board の縦の画素数

[Calibration.cpp](#) の 89 行目に定義があります。

8.2.3.7 finished()

```
auto Calibration::finished () [inline]
```

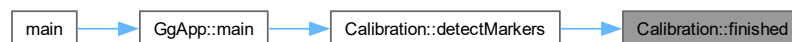
較正が完了したかどうかを調べる

戻り値

較正が完了していたら true

[Calibration.h](#) の 318 行目に定義があります。

被呼び出し関係図:



8.2.3.8 getAllMarkerPoses()

```
void Calibration::getAllMarkerPoses (
    float markerLength,
    std::map< int, GgMatrix > & poses)
```

ArUco Marker の 3 次元姿勢の変換行列を求める

引数

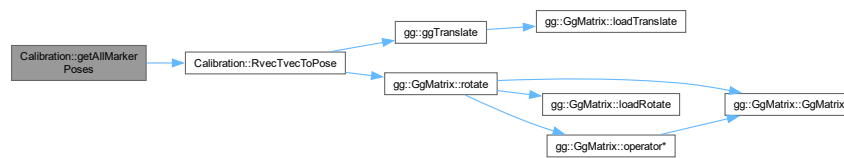
<i>markerLength</i>	ArUco Marker の一辺の長さ (単位 cm)
<i>poses</i>	推定した ArUco Marker の 3 次元姿勢

覚え書き

これはキャリブレーション終了後に単独マーカの位置推定に用いる

[Calibration.cpp](#) の 349 行目に定義があります。

呼び出し関係図:



8.2.3.9 getCameraMatrix()

```
const auto & Calibration::getCameraMatrix () const [inline]
```

カメラ行列を取り出す

戻り値

カメラ行列

[Calibration.h](#) の 288 行目に定義があります。

8.2.3.10 getCornersCount()

```
auto Calibration::getCornersCount () const [inline]
```

検出数を取得する

戻り値

検出されたコーナーの数

[Calibration.h](#) の 258 行目に定義があります。

8.2.3.11 getCurrentMotion()

```
float Calibration::getCurrentMotion () const [inline]
```

現在フレームの平均変位量 (ピクセル) を取得する

戻り値

平均変位量

[Calibration.h](#) の 234 行目に定義があります。

8.2.3.12 getDistortionCoefficients()

```
const auto & Calibration::getDistortionCoefficients () const [inline]
```

歪パラメータを取り出す

戻り値

歪パラメータ

[Calibration.h](#) の 298 行目に定義があります。

8.2.3.13 getReprojectionError()

```
auto Calibration::getReprojectionError () const [inline]
```

再投影誤差を得る

戻り値

再投影誤差

[Calibration.h](#) の 308 行目に定義があります。

8.2.3.14 getSampleCount()

```
auto Calibration::getSampleCount () const [inline]
```

標本数を取得する

戻り値

保存された標本の数

[Calibration.h](#) の 268 行目に定義があります。

8.2.3.15 getStableProgress()

```
float Calibration::getStableProgress (  
    float minStableTime = 0.6f) const [inline]
```

静止継続時間の達成度 (0.0 1.0) を取得する (UI プログレスバー用)

引数

<i>minStableTime</i>	静止とみなすために必要な継続時間 (秒)
----------------------	----------------------

戻り値

達成度 (0.0f 1.0f)

[Calibration.h](#) の 224 行目に定義があります。

8.2.3.16 getTotalCount()

```
auto Calibration::getTotalCount () const [inline]
```

検出数の合計を取得する

戻り値

検出されたコーナーの数

[Calibration.h](#) の 278 行目に定義があります。

8.2.3.17 isDiverseEnough()

```
bool Calibration::isDiverseEnough (
    float minDistanceRatio = 0.05f,
    float minScaleRatio = 0.12f,
    float minAngleDeg = 8.0f) const
```

直前に記録された標本に対して十分な姿勢・位置の多様性があるかを調べる

引数

<i>minDistanceRatio</i>	画像対角線長に対する重心移動量の最小比率
<i>minScaleRatio</i>	スケール (慣性半径) の最小変化率
<i>minAngleDeg</i>	主軸角度の最小変化量 (度)

戻り値

多様性があれば true

[Calibration.cpp](#) の 646 行目に定義があります。

8.2.3.18 isStable()

```
bool Calibration::isStable () const [inline]
```

現在静止しているか否かを判定する

戻り値

静止継続時間を満たしていれば true

[Calibration.h](#) の 216 行目に定義があります。

8.2.3.19 loadParameters()

```
bool Calibration::loadParameters (
    const std::string & filename)
```

ファイルからキャリブレーションパラメータを読み込む

引数

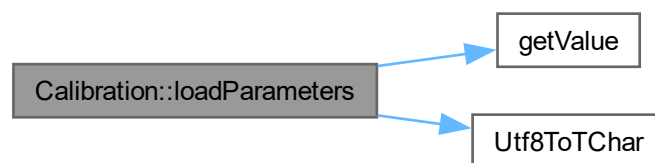
<i>filename</i>	保存するファイル名
-----------------	-----------

戻り値

パラメータの読み込みに成功したら true

[Calibration.cpp](#) の 433 行目に定義があります。

呼び出し関係図:



8.2.3.20 operator=()

```
Calibration & Calibration::operator= (
    const Calibration & calibration) [delete]
```

代入演算子は使用しない

引数

<i>calibration</i>	代入元
--------------------	-----

呼び出し関係図:



8.2.3.21 recordCorners()

```
void Calibration::recordCorners ()
```

標本を取得する

[Calibration.cpp](#) の 191 行目に定義があります。

8.2.3.22 RvecTvecToPose()

```
GgMatrix Calibration::RvecTvecToPose (
    const cv::Vec3d & rvec,
    const cv::Vec3d & tvec)
```

回転ベクトルと並進ベクトルから姿勢の変換行列を求める

引数

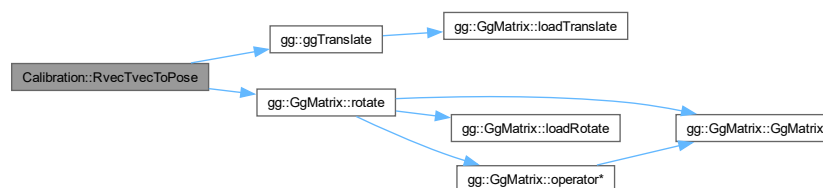
<i>rvec</i>	回転ベクトル
<i>tvec</i>	並進ベクトル

戻り値

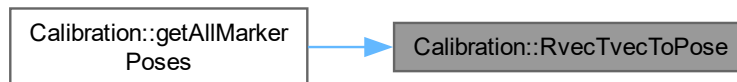
姿勢の変換行列

[Calibration.cpp](#) の 300 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.3.23 saveParameters()

```
bool Calibration::saveParameters (
    const std::string & filename) const
```

キャリブレーションパラメータをファイルに保存する

引数

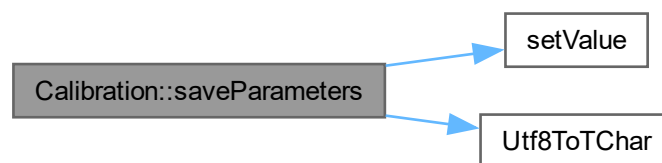
<i>filename</i>	保存するファイル名
-----------------	-----------

戻り値

パラメータの書き込みに成功したら true

[Calibration.cpp](#) の [469](#) 行目に定義があります。

呼び出し関係図:



8.2.3.24 setDictionary()

```
void Calibration::setDictionary (
    const std::string & dictionaryName,
    const std::array< int, 2 > & checkerSize,
    const std::array< float, 2 > & checkerLength)
```

ArUco Marker の辞書と検出器を設定する

引数

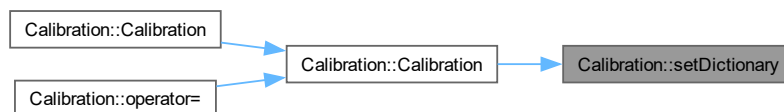
<i>dictionaryName</i>	ArUco Marker の辞書名
<i>checkerSize</i>	ChArUco Board のマス目の横と縦の数
<i>checkerLength</i>	ChArUco Board のマス目の一辺の長さ と ArUco Marker の一辺の長さ (単位 cm)

[Calibration.cpp](#) の 66 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.2.3.25 updateMotion()

```

void Calibration::updateMotion (
    float deltaTime,
    float motionThresholdPx = 2.0f,
    float minStableTime = 0.6f,
    int minCorners = 6)
  
```

モーション状態（静止判定）を更新する

引数

<i>deltaTime</i>	前フレームからの経過時間 (秒)
<i>motionThresholdPx</i>	静止とみなす最大変位閾値 (ピクセル)
<i>minStableTime</i>	静止とみなすために必要な継続時間 (秒)
<i>minCorners</i>	静止判定に必要な最低コーナー数

[Calibration.cpp](#) の 573 行目に定義があります。

8.2.4 メンバ詳解

8.2.4.1 dictionaryList

```
const std::map< const std::string, const cv::aruco::PredefinedDictionaryType > Calibration←
::dictionaryList [static]
```

初期値:

```
{
  { "DICT_4X4_50", cv::aruco::DICT_4X4_50 },
  { "DICT_4X4_100", cv::aruco::DICT_4X4_100 },
  { "DICT_4X4_250", cv::aruco::DICT_4X4_250 },
  { "DICT_4X4_1000", cv::aruco::DICT_4X4_1000 },
  { "DICT_5X5_50", cv::aruco::DICT_5X5_50 },
  { "DICT_5X5_100", cv::aruco::DICT_5X5_100 },
  { "DICT_5X5_250", cv::aruco::DICT_5X5_250 },
  { "DICT_5X5_1000", cv::aruco::DICT_5X5_1000 },
  { "DICT_6X6_50", cv::aruco::DICT_6X6_50 },
  { "DICT_6X6_100", cv::aruco::DICT_6X6_100 },
  { "DICT_6X6_250", cv::aruco::DICT_6X6_250 },
  { "DICT_6X6_1000", cv::aruco::DICT_6X6_1000 },
  { "DICT_7X7_50", cv::aruco::DICT_7X7_50 },
  { "DICT_7X7_100", cv::aruco::DICT_7X7_100 },
  { "DICT_7X7_250", cv::aruco::DICT_7X7_250 },
  { "DICT_7X7_1000", cv::aruco::DICT_7X7_1000 },
  { "DICT_ARUCO_ORIGINAL", cv::aruco::DICT_ARUCO_ORIGINAL },
  { "DICT_APRILTAG_16h5", cv::aruco::DICT_APRILTAG_16h5 },
  { "DICT_APRILTAG_25h9", cv::aruco::DICT_APRILTAG_25h9 },
  { "DICT_APRILTAG_36h10", cv::aruco::DICT_APRILTAG_36h10 },
  { "DICT_APRILTAG_36h11", cv::aruco::DICT_APRILTAG_36h11 }
}
```

ArUco Marker 辞書のリスト

[Calibration.h](#) の 497 行目に定義があります。

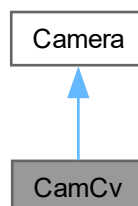
このクラス詳解は次のファイルから抽出されました:

- [Calibration.h](#)
- [Calibration.cpp](#)

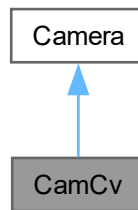
8.3 CamCv クラス

```
#include <CamCv.h>
```

CamCv の継承関係図



CamCv 連携図



公開メンバ関数

- `CamCv` ()=default
- virtual `~CamCv` ()
- bool `open` (int device, int `width`=0, int `height`=0, double `fps`=0.0, const char *`fourcc`="", int `pref`=cv::CAP_ANY)
- bool `open` (const std::string &file, int `width`=0, int `height`=0, double `fps`=0.0, const char *`fourcc`="", int `pref`=cv::CAP_ANY)
- virtual double `getFps` () const override
- unsigned int `getCodec` () const
- void `getCodec` (char *`fourcc`) const
- double `getPosition` () const
- void `setPosition` (double frame)
- void `setExposure` (double exposure)
- void `increaseExposure` () override
- void `decreaseExposure` () override
- void `setGain` (double gain)
- void `increaseGain` () override
- void `decreaseGain` () override

基底クラス `Camera` に属する継承公開メンバ関数

- `Camera` ()=default
- virtual `~Camera` ()=default
- void `start` ()
- void `stop` ()
- void `close` ()
- bool `isRunning` () const
- bool `isCaptured` () const
- template<typename F>
bool `lockFrame` (F &&func)
- virtual bool `isStillImage` () const
- virtual const std::vector< `CaptureFormat` > & `getFormatList` () const
- virtual bool `selectFormat` (int index)
- std::array< int, 2 > `getSize` () const
- int `getWidth` () const
- int `getHeight` () const
- int `getChannels` () const
- virtual void `setPrioritizeLatency` (bool mode)
- bool `getPrioritizeLatency` () const

限定公開メンバ関数

- bool `onStart` () override
- void `onStop` () override
- void `onClose` () override

その他の継承メンバ

基底クラス `Camera` に属する継承限定公開変数類

- int `width` { 0 }
解像度 (幅)
- int `height` { 0 }
解像度 (高さ)
- int `channels` { 0 }
チャンネル数
- double `interval` { 10.0 }
キャプチャした画像のフレーム間隔 (ミリ秒)
- std::vector< std::uint8_t > `image`
キャプチャしたフレームを保持する単一バッファ (CPU メモリ上の生データ)
- std::mutex `mtx`
排他制御用ミューテックス
- std::atomic< bool > `running` { false }
キャプチャスレッドが実行中なら `true`
- std::atomic< bool > `captured` { false }
新しいフレームが取得されたら `true`
- std::atomic< bool > `prioritizeLatency` { false }
レイテンシを優先するなら `true`
- std::thread `thr`
キャプチャを非同期に行うためのワーカーズレッド

8.3.1 詳解

OpenCV を使ってビデオをキャプチャするクラス

`CamCv.h` の 24 行目に定義があります。

8.3.2 構築子と解体子

8.3.2.1 `CamCv()`

```
CamCv::CamCv () [default]
```

コンストラクタ

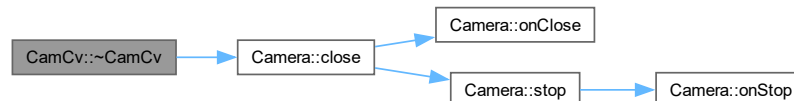
8.3.2.2 ~CamCv()

```
virtual CamCv::~~CamCv () [inline], [virtual]
```

デストラクタ

[CamCv.h](#) の 240 行目に定義があります。

呼び出し関係図:



8.3.3 関数詳解

8.3.3.1 decreaseExposure()

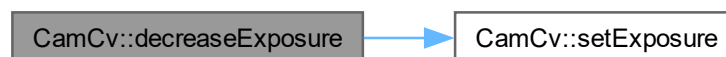
```
void CamCv::decreaseExposure () [inline], [override], [virtual]
```

露出を一段階下げる

[Camera](#)を再実装しています。

[CamCv.h](#) の 356 行目に定義があります。

呼び出し関係図:



8.3.3.2 decreaseGain()

```
void CamCv::decreaseGain () [inline], [override], [virtual]
```

利得を一段階下げる

[Camera](#)を再実装しています。

[CamCv.h](#) の [382](#) 行目に定義があります。

呼び出し関係図:



8.3.3.3 getCodec() [1/2]

```
unsigned int CamCv::getCodec () const [inline]
```

コーデックを調べる

戻り値

使用しているコーデックを表す4バイト

[CamCv.h](#) の [294](#) 行目に定義があります。

被呼び出し関係図:



8.3.3.4 getCodec() [2/2]

```
void CamCv::getCodec (  
    char * fourcc) const [inline]
```

コーデックを調べる

引数

<i>fourcc</i>	使用しているコーデックを表す 4 文字の格納先
---------------	-------------------------

[CamCv.h](#) の 304 行目に定義があります。

呼び出し関係図:



8.3.3.5 getFps()

```
virtual double CamCv::getFps () const [inline], [override], [virtual]
```

キャプチャしたフレームのフレームレートを得る

戻り値

キャプチャしたフレームのフレームレート

[Camera](#)を再実装しています。

[CamCv.h](#) の 284 行目に定義があります。

8.3.3.6 getPosition()

```
double CamCv::getPosition () const [inline]
```

ファイルから入力しているとき現在のフレーム番号を得る

戻り値

現在入力しているフレーム番号

[CamCv.h](#) の 320 行目に定義があります。

8.3.3.7 increaseExposure()

```
void CamCv::increaseExposure () [inline], [override], [virtual]
```

露出を一段階上げる

[Camera](#)を再実装しています。

[CamCv.h](#) の 348 行目に定義があります。

呼び出し関係図:



8.3.3.8 increaseGain()

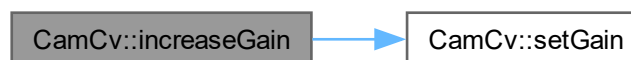
```
void CamCv::increaseGain () [inline], [override], [virtual]
```

利得を一段階上げる

[Camera](#)を再実装しています。

[CamCv.h](#) の 374 行目に定義があります。

呼び出し関係図:



8.3.3.9 onClose()

```
void CamCv::onClose () [inline], [override], [protected], [virtual]
```

キャプチャデバイスを閉じる処理を行う

[Camera](#)を実装しています。

[CamCv.h](#) の 221 行目に定義があります。

8.3.3.10 onStart()

```
bool CamCv::onStart () [inline], [override], [protected], [virtual]
```

キャプチャ開始処理を行う

戻り値

正常に開始できたら `true`

[Camera](#)を実装しています。

[CamCv.h](#) の 203 行目に定義があります。

8.3.3.11 onStop()

```
void CamCv::onStop () [inline], [override], [protected], [virtual]
```

キャプチャ停止処理を行う

[Camera](#)を実装しています。

[CamCv.h](#) の 213 行目に定義があります。

8.3.3.12 open() [1/2]

```
bool CamCv::open (  
    const std::string & file,  
    int width = 0,  
    int height = 0,  
    double fps = 0.0,  
    const char * fourcc = "",  
    int pref = cv::CAP_ANY) [inline]
```

ファイルまたはネットワークから入力する

引数

<i>file</i>	入力するファイルまたはネットワーク URL
<i>width</i>	入力するファイルを開く際に期待するフレームの横の画素数, 0 ならお任せ
<i>height</i>	入力するファイルを開く際に期待するフレームの縦の画素数, 0 ならお任せ
<i>fps</i>	入力するファイルを開く際に期待するフレームレート, 0 ならお任せ
<i>fourcc</i>	入力するファイルを開く際に期待するコーデックの 4 文字, "" ならお任せ
<i>pref</i>	OpenCV が使用する VideoCapture バックエンド

戻り値

入力するファイルが使用可能なら `true`

[CamCv.h](#) の 273 行目に定義があります。

8.3.3.13 open() [2/2]

```
bool CamCv::open (
    int device,
    int width = 0,
    int height = 0,
    double fps = 0.0,
    const char * fourcc = "",
    int pref = cv::CAP_ANY) [inline]
```

キャプチャデバイスを開く

引数

<i>device</i>	キャプチャデバイスの番号
<i>width</i>	キャプチャデバイスを開く際に期待するフレームの横の画素数, 0 ならお任せ
<i>height</i>	キャプチャデバイスを開く際に期待するフレームの縦の画素数, 0 ならお任せ
<i>fps</i>	キャプチャデバイスを開く際に期待するフレームレート, 0 ならお任せ
<i>fourcc</i>	キャプチャデバイスを開く際に期待するコーデックの 4 文字, "" ならお任せ
<i>pref</i>	OpenCV が使用する VideoCapture バックエンド

戻り値

キャプチャデバイスが使用可能なら true

[CamCv.h](#) の 256 行目に定義があります。

8.3.3.14 setExposure()

```
void CamCv::setExposure (
    double exposure) [inline]
```

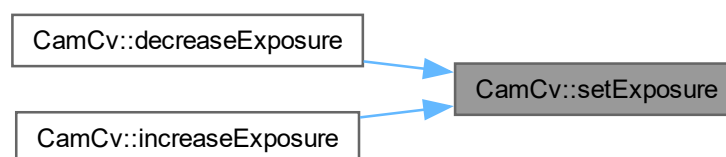
露出を設定する

引数

<i>exposure</i>	設定する露出
-----------------	--------

[CamCv.h](#) の 340 行目に定義があります。

被呼び出し関係図:



8.3.3.15 setGain()

```
void CamCv::setGain (
    double gain) [inline]
```

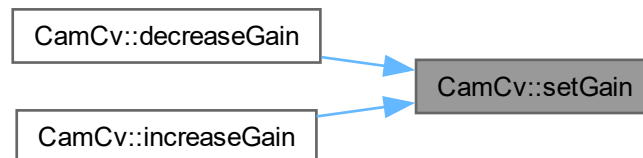
利得を設定する

引数

<i>gain</i>	設定する利得
-------------	--------

[CamCv.h](#) の 366 行目に定義があります。

被呼び出し関係図:



8.3.3.16 setPosition()

```
void CamCv::setPosition (
    double frame) [inline]
```

ファイルから入力しているとき再生位置を指定する

引数

<i>frame</i>	再生位置
--------------	------

[CamCv.h](#) の 330 行目に定義があります。

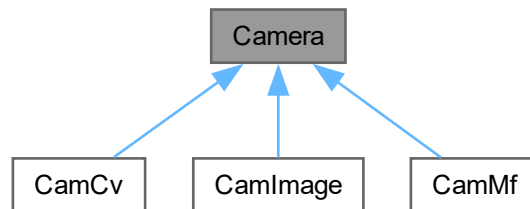
このクラス詳解は次のファイルから抽出されました:

- [CamCv.h](#)

8.4 Camera クラス

```
#include <Camera.h>
```

Camera の継承関係図



公開メンバ関数

- `Camera ()=default`
- `virtual ~Camera ()=default`
- `void start ()`
- `void stop ()`
- `void close ()`
- `bool isRunning () const`
- `bool isCaptured () const`
- `template<typename F>`
`bool lockFrame (F &&func)`
- `virtual bool isStillImage () const`
- `virtual const std::vector< CaptureFormat > & getFormatList () const`
- `virtual bool selectFormat (int index)`
- `std::array< int, 2 > getSize () const`
- `int getWidth () const`
- `int getHeight () const`
- `int getChannels () const`
- `virtual double getFps () const`
- `virtual void setPrioritizeLatency (bool mode)`
- `bool getPrioritizeLatency () const`
- `virtual void increaseExposure ()`
- `virtual void decreaseExposure ()`
- `virtual void increaseGain ()`
- `virtual void decreaseGain ()`

限定公開メンバ関数

- `virtual bool onStart ()=0`
- `virtual void onStop ()=0`
- `virtual void onClose ()=0`

限定公開変数類

- `int width { 0 }`
解像度 (幅)
- `int height { 0 }`
解像度 (高さ)
- `int channels { 0 }`
チャンネル数
- `double interval { 10.0 }`
キャプチャした画像のフレーム間隔 (ミリ秒)
- `std::vector< std::uint8_t > image`
キャプチャしたフレームを保持する単一バッファ (CPU メモリ上の生データ)
- `std::mutex mtx`
排他制御用ミューテックス
- `std::atomic< bool > running { false }`
キャプチャスレッドが実行中なら `true`
- `std::atomic< bool > captured { false }`
新しいフレームが取得されたら `true`
- `std::atomic< bool > prioritizeLatency { false }`
レイテンシを優先するなら `true`
- `std::thread thr`
キャプチャを非同期に行うためのワーカースレッド

8.4.1 詳解

キャプチャデバイス関連の基底クラス

[Camera.h](#) の 58 行目に定義があります。

8.4.2 構築子と解体子

8.4.2.1 Camera()

```
Camera::Camera () [default]
```

コンストラクタ

8.4.2.2 ~Camera()

```
virtual Camera::~Camera () [virtual], [default]
```

デストラクタ

8.4.3 関数詳解

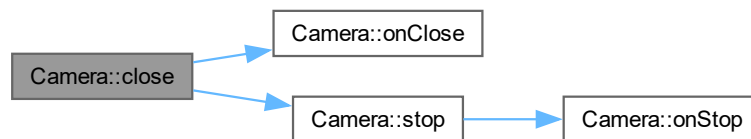
8.4.3.1 close()

```
void Camera::close () [inline]
```

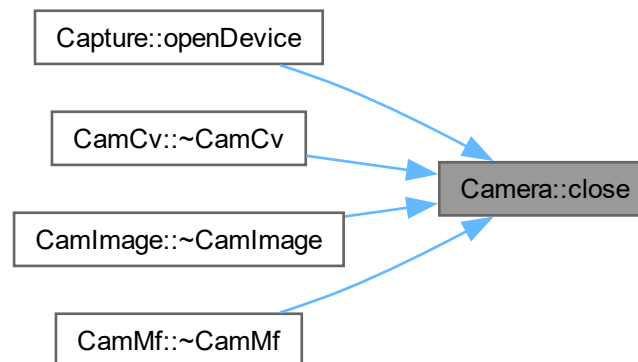
キャプチャデバイスを閉じる

[Camera.h](#) の 157 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.4.3.2 decreaseExposure()

```
virtual void Camera::decreaseExposure () [inline], [virtual]
```

露出を下げる

[CamCv](#) で再実装されています。

[Camera.h](#) の 322 行目に定義があります。

8.4.3.3 decreaseGain()

```
virtual void Camera::decreaseGain () [inline], [virtual]
```

利得を下げる

[CamCv](#)で再実装されています。

[Camera.h](#) の 336 行目に定義があります。

8.4.3.4 getChannels()

```
int Camera::getChannels () const [inline]
```

キャプチャしたフレームのチャンネル数を調べる

戻り値

キャプチャしたフレームのチャンネル数

[Camera.h](#) の 277 行目に定義があります。

8.4.3.5 getFormatList()

```
virtual const std::vector< CaptureFormat > & Camera::getFormatList () const [inline], [virtual]
```

キャプチャデバイスが対応するビデオフォーマットのリストを得る

戻り値

ビデオフォーマット情報のリスト

[CamMf](#)で再実装されています。

[Camera.h](#) の 225 行目に定義があります。

8.4.3.6 getFps()

```
virtual double Camera::getFps () const [inline], [virtual]
```

キャプチャデバイスのフレームレートを得る

戻り値

キャプチャデバイスのフレームレート

[CamCv](#)で再実装されています。

[Camera.h](#) の 287 行目に定義があります。

8.4.3.7 getHeight()

```
int Camera::getHeight () const [inline]
```

キャプチャしたフレームの縦の画素数を得る

戻り値

キャプチャ中のフレームの縦の画素数

[Camera.h](#) の 267 行目に定義があります。

8.4.3.8 getPrioritizeLatency()

```
bool Camera::getPrioritizeLatency () const [inline]
```

レイテンシ優先モードかどうか調べる

戻り値

レイテンシを優先する場合は true

[Camera.h](#) の 307 行目に定義があります。

8.4.3.9 getSize()

```
std::array< int, 2 > Camera::getSize () const [inline]
```

キャプチャしたフレームのサイズを得る

戻り値

キャプチャしたフレームのサイズ

[Camera.h](#) の 247 行目に定義があります。

8.4.3.10 getWidth()

```
int Camera::getWidth () const [inline]
```

キャプチャしたフレームの横の画素数を得る

戻り値

キャプチャ中のフレームの横の画素数

[Camera.h](#) の 257 行目に定義があります。

8.4.3.11 increaseExposure()

```
virtual void Camera::increaseExposure () [inline], [virtual]
```

露出を上げる

CamCvで再実装されています。

Camera.h の 315 行目に定義があります。

8.4.3.12 increaseGain()

```
virtual void Camera::increaseGain () [inline], [virtual]
```

利得を上げる

CamCvで再実装されています。

Camera.h の 329 行目に定義があります。

8.4.3.13 isCaptured()

```
bool Camera::isCaptured () const [inline]
```

新しいフレームが取得されているか調べる

戻り値

新しいフレームが取得されていれば true

Camera.h の 184 行目に定義があります。

8.4.3.14 isRunning()

```
bool Camera::isRunning () const [inline]
```

キャプチャスレッドが実行中かどうか調べる

戻り値

キャプチャ中なら true

Camera.h の 174 行目に定義があります。

8.4.3.15 isStillImage()

```
virtual bool Camera::isStillImage () const [inline], [virtual]
```

静止画像入力であるかどうかを調べる

戻り値

静止画像なら true

[CamImage](#)で再実装されています。

[Camera.h](#) の 215 行目に定義があります。

被呼び出し関係図:



8.4.3.16 lockFrame()

```
template<typename F>
bool Camera::lockFrame (
    F && func) [inline]
```

フレームデータをロックして処理関数を実行する

テンプレート引数

<i>F</i>	処理関数の型
----------	--------

引数

<i>func</i>	フレームデータを処理する関数 (引数: const std::uint8_t* data, size_t length, int width, int height, int channels)
-------------	---

戻り値

フレームが取得できて処理関数が実行されたら true

[Camera.h](#) の 197 行目に定義があります。

呼び出し関係図:



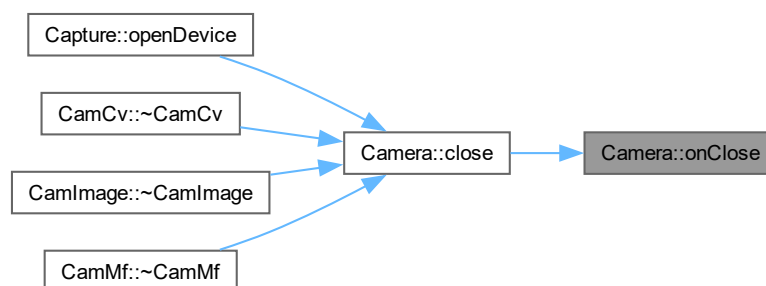
8.4.3.17 onClose()

```
virtual void Camera::onClose () [protected], [pure virtual]
```

キャプチャデバイスを閉じる処理を行う（派生クラス固有の実装）

[CamCv](#), [CamImage](#), [CamMf](#)で実装されています。

被呼び出し関係図:



8.4.3.18 onStart()

```
virtual bool Camera::onStart () [protected], [pure virtual]
```

キャプチャ開始処理を行う（派生クラス固有の実装）

戻り値

正常に開始できたら true

CamCv, CamImage, CamMfで実装されています。

被呼び出し関係図:



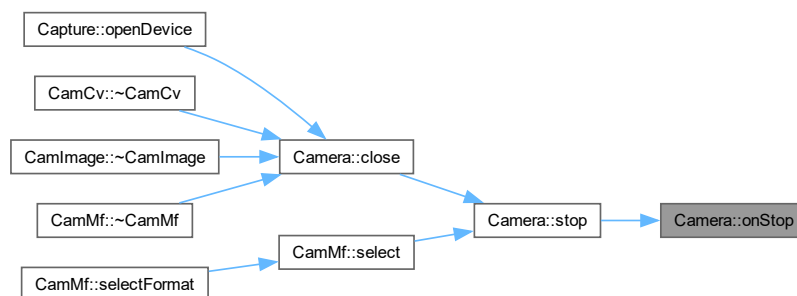
8.4.3.19 onStop()

```
virtual void Camera::onStop () [protected], [pure virtual]
```

キャプチャ停止処理を行う（派生クラス固有の実装）

CamCv, CamImage, CamMfで実装されています。

被呼び出し関係図:



8.4.3.20 selectFormat()

```
virtual bool Camera::selectFormat (
    int index) [inline], [virtual]
```

ビデオフォーマットを選択して設定する

引数

<i>index</i>	選択するフォーマットのインデックス
--------------	-------------------

戻り値

正常に設定できたら `true`

[CamMf](#)で再実装されています。

[Camera.h](#) の 237 行目に定義があります。

8.4.3.21 `setPrioritizeLatency()`

```
virtual void Camera::setPrioritizeLatency (  
    bool mode) [inline], [virtual]
```

レイテンシ優先モードを設定する

引数

<i>mode</i>	レイテンシを優先する場合は <code>true</code>
-------------	---------------------------------

[Camera.h](#) の 297 行目に定義があります。

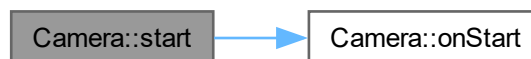
8.4.3.22 `start()`

```
void Camera::start () [inline]
```

キャプチャを開始する

[Camera.h](#) の 130 行目に定義があります。

呼び出し関係図:



8.4.3.23 stop()

```
void Camera::stop () [inline]
```

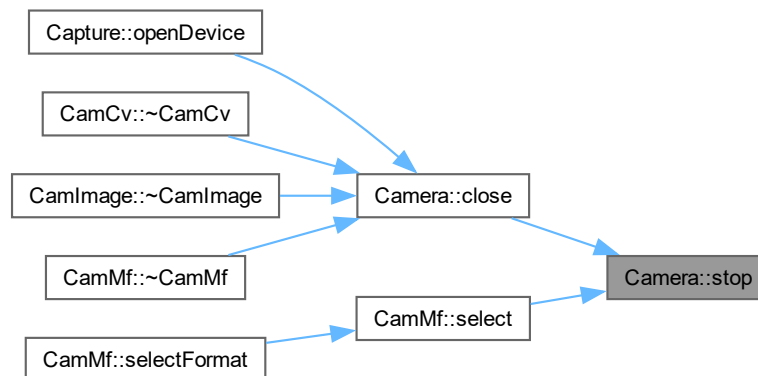
キャプチャを停止する

[Camera.h](#) の 143 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.4.4 メンバ詳解

8.4.4.1 captured

```
std::atomic<bool> Camera::captured { false } [protected]
```

新しいフレームが取得されたら `true`

[Camera.h](#) の 90 行目に定義があります。

8.4.4.2 channels

```
int Camera::channels { 0 } [protected]
```

チャンネル数

[Camera.h](#) の 75 行目に定義があります。

8.4.4.3 height

```
int Camera::height { 0 } [protected]
```

解像度 (高さ)

[Camera.h](#) の 72 行目に定義があります。

8.4.4.4 image

```
std::vector<std::uint8_t> Camera::image [protected]
```

キャプチャしたフレームを保持する単一バッファ (CPU メモリ上の生データ)

[Camera.h](#) の 81 行目に定義があります。

8.4.4.5 interval

```
double Camera::interval { 10.0 } [protected]
```

キャプチャした画像のフレーム間隔 (ミリ秒)

[Camera.h](#) の 78 行目に定義があります。

8.4.4.6 mtx

```
std::mutex Camera::mtx [mutable], [protected]
```

排他制御用ミューテックス

[Camera.h](#) の 84 行目に定義があります。

8.4.4.7 prioritizeLatency

```
std::atomic<bool> Camera::prioritizeLatency { false } [protected]
```

レイテンシを優先するなら true

[Camera.h](#) の 93 行目に定義があります。

8.4.4.8 running

```
std::atomic<bool> Camera::running { false } [protected]
```

キャプチャスレッドが実行中なら true

[Camera.h](#) の 87 行目に定義があります。

8.4.4.9 thr

```
std::thread Camera::thr [protected]
```

キャプチャを非同期に行うためのワーカースレッド

[Camera.h](#) の 96 行目に定義があります。

8.4.4.10 width

```
int Camera::width { 0 } [protected]
```

解像度（幅）

[Camera.h](#) の 69 行目に定義があります。

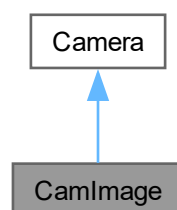
このクラス詳解は次のファイルから抽出されました:

- [Camera.h](#)

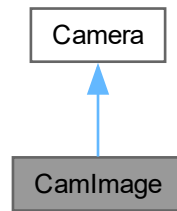
8.5 CamImage クラス

```
#include <CamImage.h>
```

CamImage の継承関係図



CamImage 連携図



公開メンバ関数

- `CamImage ()=default`
- `CamImage (const std::string &filename, bool flip=false)`
- `virtual ~CamImage ()`
- `bool isStillImage () const override`
- `bool open (const std::string &filename, bool flip=false)`
- `bool isOpened () const`

基底クラス **Camera** に属する継承公開メンバ関数

- `Camera ()=default`
- `virtual ~Camera ()=default`
- `void start ()`
- `void stop ()`
- `void close ()`
- `bool isRunning () const`
- `bool isCaptured () const`
- `template<typename F>`
`bool lockFrame (F &&func)`
- `virtual const std::vector< CaptureFormat > & getFormatList () const`
- `virtual bool selectFormat (int index)`
- `std::array< int, 2 > getSize () const`
- `int getWidth () const`
- `int getHeight () const`
- `int getChannels () const`
- `virtual double getFps () const`
- `virtual void setPrioritizeLatency (bool mode)`
- `bool getPrioritizeLatency () const`
- `virtual void increaseExposure ()`
- `virtual void decreaseExposure ()`
- `virtual void increaseGain ()`
- `virtual void decreaseGain ()`

静的公開メンバ関数

- `static bool load (const std::string &filename, cv::Mat &frame)`
- `static bool save (const std::string &filename, const cv::Mat &image)`

限定公開メンバ関数

- bool `onStart()` override
- void `onStop()` override
- void `onClose()` override

その他の継承メンバ

基底クラス **Camera** に属する継承限定公開変数類

- int `width` { 0 }
解像度 (幅)
- int `height` { 0 }
解像度 (高さ)
- int `channels` { 0 }
チャンネル数
- double `interval` { 10.0 }
キャプチャした画像のフレーム間隔 (ミリ秒)
- `std::vector< std::uint8_t >` `image`
キャプチャしたフレームを保持する単一バッファ (CPU メモリ上の生データ)
- `std::mutex` `mtx`
排他制御用ミューテックス
- `std::atomic< bool >` `running` { false }
キャプチャスレッドが実行中なら `true`
- `std::atomic< bool >` `captured` { false }
新しいフレームが取得されたら `true`
- `std::atomic< bool >` `prioritizeLatency` { false }
レイテンシを優先するなら `true`
- `std::thread` `thr`
キャプチャを非同期に行うためのワーカースレッド

8.5.1 詳解

OpenCV を使って画像ファイルを読み込むクラス

`CamImage.h` の 23 行目に定義があります。

8.5.2 構築子と解体子

8.5.2.1 CamImage() [1/2]

```
CamImage::CamImage () [default]
```

コンストラクタ

8.5.2.2 CamImage() [2/2]

```
CamImage::CamImage (
    const std::string & filename,
    bool flip = false) [inline]
```

画像ファイルを開いて読み込むコンストラクタ

引数

<i>filename</i>	画像ファイル名
<i>flip</i>	上下を反転するときは true

[CamImage.h](#) の 64 行目に定義があります。

呼び出し関係図:



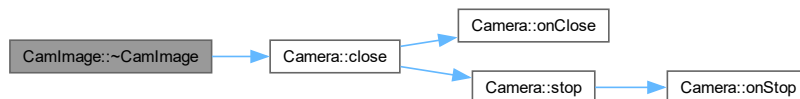
8.5.2.3 ~CamImage()

```
virtual CamImage::~CamImage () [inline], [virtual]
```

デストラクタ

[CamImage.h](#) の 72 行目に定義があります。

呼び出し関係図:



8.5.3 関数詳解

8.5.3.1 isOpened()

```
bool CamImage::isOpened () const [inline]
```

画像ファイルが開けたかどうか調べる

戻り値

ファイルが開けていたら true

[CamImage.h](#) の 123 行目に定義があります。

8.5.3.2 isStillImage()

```
bool CamImage::isStillImage () const [inline], [override], [virtual]
```

静止画像入力であるかどうかを調べる

戻り値

静止画像なので true

Cameraを再実装しています。

CamImage.h の 82 行目に定義があります。

8.5.3.3 load()

```
bool CamImage::load (
    const std::string & filename,
    cv::Mat & frame) [inline], [static]
```

画像ファイルを読み込む

引数

<i>filename</i>	読み込む画像ファイルのパス
<i>frame</i>	読み込んだ画像

戻り値

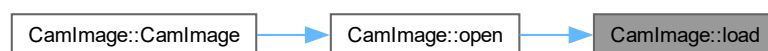
読み込みに成功したら true

CamImage.h の 135 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.5.3.4 onClose()

```
void CamImage::onClose () [inline], [override], [protected], [virtual]
```

キャプチャデバイスを閉じる処理

[Camera](#)を実装しています。

[CamImage.h](#) の 47 行目に定義があります。

8.5.3.5 onStart()

```
bool CamImage::onStart () [inline], [override], [protected], [virtual]
```

キャプチャ開始処理（静止画ではスレッドを起動しない）

戻り値

常に true

[Camera](#)を実装しています。

[CamImage.h](#) の 32 行目に定義があります。

8.5.3.6 onStop()

```
void CamImage::onStop () [inline], [override], [protected], [virtual]
```

キャプチャ停止処理

[Camera](#)を実装しています。

[CamImage.h](#) の 40 行目に定義があります。

8.5.3.7 open()

```
bool CamImage::open (  
    const std::string & filename,  
    bool flip = false) [inline]
```

画像ファイルを開く

引数

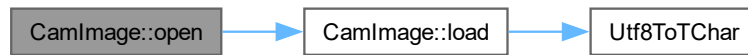
<i>filename</i>	画像ファイル名
<i>flip</i>	上下を反転するときは true

戻り値

正常に開けたら true

CamImage.h の 94 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.5.3.8 save()

```
bool CamImage::save (  
    const std::string & filename,  
    const cv::Mat & image) [inline], [static]
```

配列に格納されている画像をファイルに保存する

引数

<i>filename</i>	保存する画像ファイルのパス
<i>image</i>	保存する画像を保持している配列

戻り値

保存に成功したら true

[CamImage.h](#) の 176 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



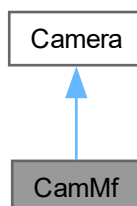
このクラス詳解は次のファイルから抽出されました:

- [CamImage.h](#)

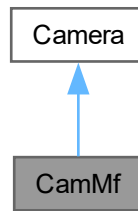
8.6 CamMf クラス

```
#include <CamMf.h>
```

CamMf の継承関係図



CamMf 連携図



公開メンバ関数

- `CamMf ()`=default
- virtual `~CamMf ()`
- bool `open` (int device, bool setupFormat=true)
- virtual const std::vector< `CaptureFormat` > & `getFormatList` () const override
- bool `select` (int index)
- virtual bool `selectFormat` (int index) override
- void `capture` ()

基底クラス `Camera` に属する継承公開メンバ関数

- `Camera ()`=default
- virtual `~Camera ()`=default
- void `start` ()
- void `stop` ()
- void `close` ()
- bool `isRunning` () const
- bool `isCaptured` () const
- template<typename F>
bool `lockFrame` (F &&func)
- virtual bool `isStillImage` () const
- std::array< int, 2 > `getSize` () const
- int `getWidth` () const
- int `getHeight` () const
- int `getChannels` () const
- virtual double `getFps` () const
- virtual void `setPrioritizeLatency` (bool mode)
- bool `getPrioritizeLatency` () const
- virtual void `increaseExposure` ()
- virtual void `decreaseExposure` ()
- virtual void `increaseGain` ()
- virtual void `decreaseGain` ()

静的公開メンバ関数

- static const auto & `getDeviceList` ()

限定公開メンバ関数

- virtual bool `onStart()` override
- virtual void `onStop()` override
- virtual void `onClose()` override

その他の継承メンバ

基底クラス `Camera` に属する継承限定公開変数類

- int `width` { 0 }
解像度 (幅)
- int `height` { 0 }
解像度 (高さ)
- int `channels` { 0 }
チャンネル数
- double `interval` { 10.0 }
キャプチャした画像のフレーム間隔 (ミリ秒)
- `std::vector< std::uint8_t >` `image`
キャプチャしたフレームを保持する単一バッファ (CPU メモリ上の生データ)
- `std::mutex` `mtx`
排他制御用ミューテックス
- `std::atomic< bool >` `running` { false }
キャプチャスレッドが実行中なら `true`
- `std::atomic< bool >` `captured` { false }
新しいフレームが取得されたら `true`
- `std::atomic< bool >` `prioritizeLatency` { false }
レイテンシを優先するなら `true`
- `std::thread` `thr`
キャプチャを非同期に行うためのワーカーズレッド

8.6.1 詳解

Microsoft Media Foundation を使ってビデオをキャプチャするクラス

`CamMf.h` の 25 行目に定義があります。

8.6.2 構築子と解体子

8.6.2.1 `CamMf()`

```
CamMf::CamMf () [default]
```

コンストラクタ

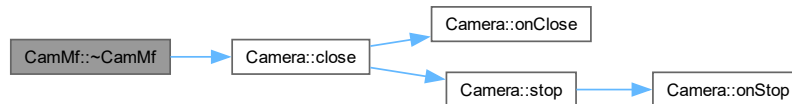
8.6.2.2 ~CamMf()

```
virtual CamMf::~~CamMf () [inline], [virtual]
```

デストラクタ

[CamMf.h](#) の 248 行目に定義があります。

呼び出し関係図:



8.6.3 関数詳解

8.6.3.1 capture()

```
void CamMf::capture ()
```

フレームをキャプチャする（別スレッドでループ実行される）

[CamMf.cpp](#) の 824 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.6.3.2 getDeviceList()

```
const auto & CamMf::getDeviceList () [inline], [static]
```

Media Foundation のビデオデバイスの一覧を返す

戻り値

デバイス名のリスト

[CamMf.h](#) の 259 行目に定義があります。

被呼び出し関係図:



8.6.3.3 getFormatList()

```
virtual const std::vector< CaptureFormat > & CamMf::getFormatList () const [inline], [override], [virtual]
```

使用可能なビデオフォーマットの表示・選択情報を返す

戻り値

enumerateFormats() で作成した構造化フォーマットのリスト

[Camera](#) を再実装しています。

[CamMf.h](#) の 278 行目に定義があります。

8.6.3.4 onClose()

```
void CamMf::onClose () [override], [protected], [virtual]
```

キャプチャデバイスを閉じる処理を行う

[Camera](#) を実装しています。

[CamMf.cpp](#) の 1310 行目に定義があります。

呼び出し関係図:



8.6.3.5 onStart()

```
bool CamMf::onStart () [override], [protected], [virtual]
```

キャプチャ開始処理を行う

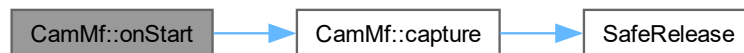
戻り値

正常に開始できたら true

[Camera](#)を実装しています。

[CamMf.cpp](#) の 1285 行目に定義があります。

呼び出し関係図:



8.6.3.6 onStop()

```
void CamMf::onStop () [override], [protected], [virtual]
```

キャプチャ停止処理を行う

[Camera](#)を実装しています。

[CamMf.cpp](#) の 1298 行目に定義があります。

8.6.3.7 open()

```
bool CamMf::open (
    int device,
    bool setupFormat = true)
```

カメラを開く

引数

<i>device</i>	デバイスの番号
<i>setupFormat</i>	最初のフォーマットを設定するかどうか (遅延初期化時は false を指定)

戻り値

開くことができたなら true

CamMf.cpp の 757 行目に定義があります。

呼び出し関係図:



8.6.3.8 select()

```
bool CamMf::select (
    int index)
```

フォーマットを選択して設定する

引数

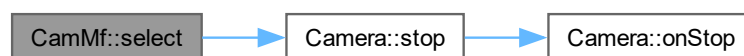
<i>index</i>	選択するフォーマットのリストインデックス
--------------	----------------------

戻り値

成功したら true

CamMf.cpp の 809 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.6.3.9 selectFormat()

```
virtual bool CamMf::selectFormat (
    int index) [inline], [override], [virtual]
```

ビデオフォーマットを選択して設定する

引数

<i>index</i>	選択するフォーマットのインデックス
--------------	-------------------

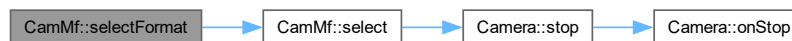
戻り値

正常に設定できたら true

[Camera](#)を再実装しています。

[CamMf.h](#) の 297 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [CamMf.h](#)
- [CamMf.cpp](#)

8.7 Capture クラス

```
#include <Capture.h>
```

公開メンバ関数

- [Capture](#) ()=default
- [Capture](#) (const std::string &filename)
- virtual [~Capture](#) ()
- bool [openImage](#) (const std::string &filename)
- bool [openMovie](#) (const std::string &filename, cv::VideoCaptureAPIs backend=cv::CAP_FFMPEG)
- bool [openDevice](#) (int deviceNumber, std::array< int, 2 > &size, double &fps, cv::VideoCaptureAPIs backend, char *fourcc)
- void [start](#) ()
- void [stop](#) ()
- void [close](#) ()
- [operator bool](#) () const
- bool [isOpen](#) () const
- bool [isImage](#) () const
- std::array< int, 2 > [getSize](#) () const
- double [getFps](#) () const
- bool [retrieve](#) (Buffer &buffer)
- bool [retrieve](#) (cv::Mat &frame)
- void [setPrioritizeLatency](#) (bool mode)
- bool [getPrioritizeLatency](#) () const

8.7.1 詳解

キャプチャクラス

[Capture.h](#) の 44 行目に定義があります。

8.7.2 構築子と解体子

8.7.2.1 Capture() [1/2]

```
Capture::Capture () [default]
```

キャプチャーオブジェクトのデフォルトコンストラクタ

8.7.2.2 Capture() [2/2]

```
Capture::Capture (  
    const std::string & filename) [inline]
```

キャプチャするファイルを指定するコンストラクタ

引数

<i>filename</i>	キャプチャするファイルのパス名
-----------------	-----------------

[Capture.h](#) の 69 行目に定義があります。

呼び出し関係図:



8.7.2.3 ~Capture()

```
virtual Capture::~~Capture () [inline], [virtual]
```

デストラクタ

[Capture.h](#) の 77 行目に定義があります。

呼び出し関係図:



8.7.3 関数詳解

8.7.3.1 close()

```
void Capture::close ()
```

キャプチャデバイスを閉じる

[Capture.cpp](#) の 268 行目に定義があります。

被呼び出し関係図:



8.7.3.2 getFps()

```
double Capture::getFps () const
```

キャプチャデバイスのフレームレートを得る

戻り値

キャプチャデバイスのフレームレート

[Capture.cpp](#) の 290 行目に定義があります。

8.7.3.3 getPrioritizeLatency()

```
bool Capture::getPrioritizeLatency () const [inline]
```

レイテンシ優先モードかどうか調べる

戻り値

レイテンシを優先する場合は true

[Capture.h](#) の 241 行目に定義があります。

8.7.3.4 getSize()

```
std::array< int, 2 > Capture::getSize () const
```

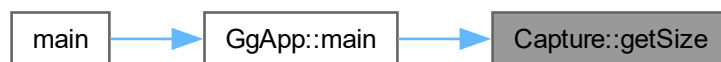
キャプチャデバイスのフレームの解像度を得る

戻り値

キャプチャデバイスのフレームの解像度

[Capture.cpp](#) の 281 行目に定義があります。

被呼び出し関係図:



8.7.3.5 isImage()

```
bool Capture::isImage () const [inline]
```

現在開いているのが静止画像かどうか

[Capture.h](#) の 190 行目に定義があります。

8.7.3.6 isOpened()

```
bool Capture::isOpened () const [inline]
```

キャプチャデバイスが有効かどうか

[Capture.h](#) の 182 行目に定義があります。

8.7.3.7 openDevice()

```
bool Capture::openDevice (  
    int deviceNumber,  
    std::array< int, 2 > & size,  
    double & fps,  
    cv::VideoCaptureAPIs backend,  
    char * fourcc)
```

キャプチャデバイスを開く (その他用: OpenCV)

引数

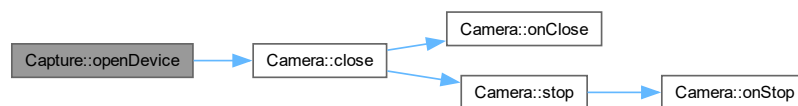
<i>deviceNumber</i>	開くデバイス番号
<i>size</i>	キャプチャデバイスのフレームの解像度
<i>fps</i>	キャプチャデバイスのフレームレート
<i>backend</i>	バックエンドの種類
<i>fourcc</i>	コーデックの 4 文字

戻り値

開くことができたなら `true`

`Capture.cpp` の 196 行目に定義があります。

呼び出し関係図:



8.7.3.8 openImage()

```
bool Capture::openImage (
    const std::string & filename)
```

画像ファイルを開く

引数

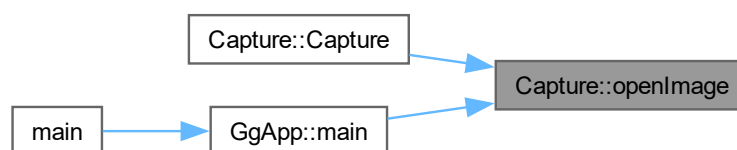
<i>filename</i>	開く画像ファイル名
-----------------	-----------

戻り値

開くことができたなら `true`

`Capture.cpp` の 18 行目に定義があります。

被呼び出し関係図:



8.7.3.9 openMovie()

```
bool Capture::openMovie (  
    const std::string & filename,  
    cv::VideoCaptureAPIs backend = cv::CAP_FFMPEG)
```

動画ファイルを開く

引数

<i>filename</i>	開く動画ファイル名
<i>backend</i>	バックエンドの種類

戻り値

開くことができたなら true

[Capture.cpp](#) の 38 行目に定義があります。

8.7.3.10 operator bool()

```
Capture::operator bool () const [inline], [explicit]
```

キャプチャ中か否か

戻り値

キャプチャ中なら true

[Capture.h](#) の 174 行目に定義があります。

8.7.3.11 retrieve() [1/2]

```
bool Capture::retrieve (  
    Buffer & buffer)
```

新しいフレームを GPU の PBO に取得する

引数

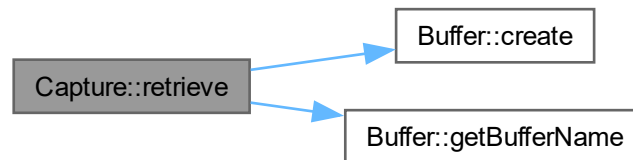
<i>buffer</i>	取得したフレームを格納するバッファ
---------------	-------------------

戻り値

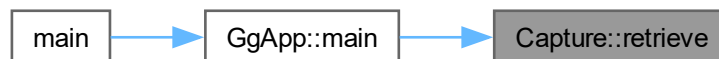
新しいフレームを取得できたら true

[Capture.cpp](#) の 299 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.7.3.12 retrieve() [2/2]

```
bool Capture::retrieve (  
    cv::Mat & frame)
```

新しいフレームを CPU のメモリに取得する

引数

<i>frame</i>	取得したフレーム
--------------	----------

戻り値

新しいフレームを取得できたら true

覚え書き

OpenCV で処理する場合。

[Capture.cpp](#) の 316 行目に定義があります。

8.7.3.13 setPrioritizeLatency()

```
void Capture::setPrioritizeLatency (  
    bool mode) [inline]
```

レイテンシ優先モードを設定する

引数

<i>mode</i>	レイテンシを優先する場合は true
-------------	--------------------

[Capture.h](#) の 231 行目に定義があります。

8.7.3.14 start()

```
void Capture::start ()
```

キャプチャ開始

[Capture.cpp](#) の 250 行目に定義があります。

8.7.3.15 stop()

```
void Capture::stop ()
```

キャプチャ終了

[Capture.cpp](#) の 259 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Capture.h](#)
- [Capture.cpp](#)

8.8 CaptureFormat 構造体

```
#include <Camera.h>
```

公開メンバ関数

- [CaptureFormat](#) (const std::string &resolution, const std::string &fps, const std::string &codec, int index)

公開変数類

- `std::string resolution`
解像度の表示文字列（例: "1920 x 1080"）
- `std::string fps`
フレームレートの表示文字列（例: "30.00"）
- `std::string codec`
コーデックの表示文字列（例: "NV12"）
- `int index { 0 }`
バックエンドのフォーマットリストにおける選択番号

8.8.1 詳解

キャプチャデバイスが対応するビデオフォーマットの表示・選択情報

バックエンド固有のメディア型を UI が解釈し直さなくて済むよう、解像度、フレームレート、コーデックを表示用文字列として保持する。index はバックエンドが保持する実フォーマットの選択に使用する。

[Camera.h](#) の 30 行目に定義があります。

8.8.2 構築子と解体子

8.8.2.1 CaptureFormat()

```
CaptureFormat::CaptureFormat (
    const std::string & resolution,
    const std::string & fps,
    const std::string & codec,
    int index) [inline]
```

コンストラクタ

引数

<i>resolution</i>	解像度の表示文字列（例: "1920 x 1080"）
<i>fps</i>	フレームレートの表示文字列（例: "30.00"）
<i>codec</i>	コーデックの表示文字列（例: "NV12"）
<i>index</i>	バックエンドのフォーマットリストにおける選択番号

[Camera.h](#) の 45 行目に定義があります。

8.8.3 メンバ詳解

8.8.3.1 codec

```
std::string CaptureFormat::codec
```

コーデックの表示文字列（例: "NV12"）

[Camera.h](#) の 34 行目に定義があります。

8.8.3.2 fps

```
std::string CaptureFormat::fps
```

フレームレートの表示文字列（例: "30.00"）

[Camera.h](#) の 33 行目に定義があります。

8.8.3.3 index

```
int CaptureFormat::index { 0 }
```

バックエンドのフォーマットリストにおける選択番号

[Camera.h](#) の 35 行目に定義があります。

8.8.3.4 resolution

```
std::string CaptureFormat::resolution
```

解像度の表示文字列（例: "1920 x 1080"）

[Camera.h](#) の 32 行目に定義があります。

この構造体詳解は次のファイルから抽出されました:

- [Camera.h](#)

8.9 Compute クラス

```
#include <Compute.h>
```

公開メンバ関数

- [Compute](#) (const char *comp)
- virtual [~Compute](#) ()
- auto [get](#) () const
- void [use](#) () const
- void [execute](#) (GLuint width, GLuint height, GLuint local_size_x=1, GLuint local_size_y=1) const

8.9.1 詳解

画像処理クラスの定義（コンピュータシェーダ版）

[Compute.h](#) の 18 行目に定義があります。

8.9.2 構築子と解体子

8.9.2.1 Compute()

```
Compute::Compute (  
    const char * comp) [inline]
```

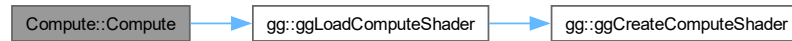
コンストラクタ

引数

<i>comp</i>	コンピュートシェーダのソースファイル名
-------------	---------------------

[Compute.h](#) の 30 行目に定義があります。

呼び出し関係図:



8.9.2.2 ~Compute()

```
virtual Compute::~Compute () [inline], [virtual]
```

デストラクタ

[Compute.h](#) の 38 行目に定義があります。

8.9.3 関数詳解

8.9.3.1 execute()

```
void Compute::execute (
    GLuint width,
    GLuint height,
    GLuint local_size_x = 1,
    GLuint local_size_y = 1) const [inline]
```

計算を実行する

引数

<i>width</i>	画像の横の画素数
<i>height</i>	画像の縦の画素数
<i>local_size_x</i>	ワークグループの横方向のスレッド数
<i>local_size_y</i>	ワークグループの縦方向のスレッド数

[Compute.h](#) の 70 行目に定義があります。

8.9.3.2 get()

```
auto Compute::get () const [inline]
```

シェーダプログラムを得る

戻り値

シェーダのプログラムオブジェクト

[Compute.h](#) の 49 行目に定義があります。

8.9.3.3 use()

```
void Compute::use () const [inline]
```

計算用のシェーダプログラムの使用を開始する

[Compute.h](#) の 57 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Compute.h](#)

8.10 Config クラス

```
#include <Config.h>
```

公開メンバ関数

- [Config](#) (const std::string &filename)
- virtual [~Config](#) ()
- void [initialize](#) ()
- bool [load](#) (const [pathString](#) &filename)
- bool [save](#) (const [pathString](#) &filename) const
- const [Settings](#) & [getSettings](#) () const
- void [setSettings](#) (const [Settings](#) &value)
- const auto & [getPreferences](#) () const
- const auto & [getBackground](#) () const
- const auto & [getMenuFont](#) () const
- auto [getMenuFontSize](#) () const
- const auto & [getTitle](#) () const
- auto [getWidth](#) () const
- auto [getHeight](#) () const
- const auto & [getInitialImage](#) () const
- const auto & [getDictionaryName](#) () const
- const auto & [getCheckerSize](#) () const
- const auto & [getCheckerLength](#) () const
- auto [getMarkerLength](#) () const

8.10.1 詳解

構成データクラス

[Config.h](#) の 74 行目に定義があります。

8.10.2 構築子と解体子

8.10.2.1 Config()

```
Config::Config (  
    const std::string & filename)
```

コンストラクタ

引数

<i>filename</i>	構成データの初期化に用いる JSON 形式のファイル名
-----------------	-----------------------------

[Config.cpp](#) の 39 行目に定義があります。

呼び出し関係図:



8.10.2.2 ~Config()

```
Config::~Config () [virtual]
```

デストラクタ

[Config.cpp](#) の 89 行目に定義があります。

8.10.3 関数詳解

8.10.3.1 getBackground()

```
const auto & Config::getBackground () const [inline]
```

背景色を得る

戻り値

RGBA 形式の背景色への読み取り専用参照

[Config.h](#) の 183 行目に定義があります。

8.10.3.2 getConfigLength()

```
const auto & Config::getConfigLength () const [inline]
```

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さを得る

戻り値

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さ

[Config.h](#) の 273 行目に定義があります。

被呼び出し関係図:



8.10.3.3 getConfigSize()

```
const auto & Config::getConfigSize () const [inline]
```

検出する ChArUco Board のマス目の横と縦の数を取得

戻り値

検出する ChArUco Board のマス目の横と縦の数

[Config.h](#) の 263 行目に定義があります。

被呼び出し関係図:



8.10.3.4 getDictionaryName()

```
const auto & Config::getDictionaryName () const [inline]
```

使用中の ArUco Marker の辞書名を得る

戻り値

使用中の ArUco Marker の辞書名

[Config.h](#) の 253 行目に定義があります。

被呼び出し関係図:



8.10.3.5 getHeight()

```
auto Config::getHeight () const [inline]
```

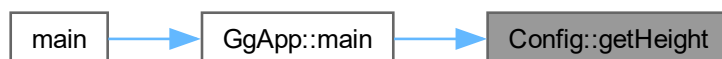
ウィンドウの高さを得る

戻り値

ウィンドウの高さ

[Config.h](#) の 233 行目に定義があります。

被呼び出し関係図:



8.10.3.6 getInitialImage()

```
const auto & Config::getInitialImage () const [inline]
```

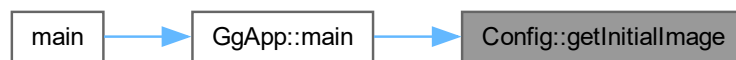
初期画像ファイル名を得る

戻り値

初期画像ファイル名

[Config.h](#) の 243 行目に定義があります。

被呼び出し関係図:



8.10.3.7 getMarkerLength()

```
auto Config::getMarkerLength () const [inline]
```

検出する ArUco Marker の一辺の長さを得る

戻り値

検出する ArUco Marker の一辺の長さ

[Config.h](#) の 283 行目に定義があります。

8.10.3.8 getMenuFont()

```
const auto & Config::getMenuFont () const [inline]
```

メニュー用フォントのファイル名を得る

戻り値

フォントファイル名への読み取り専用参照

[Config.h](#) の 193 行目に定義があります。

8.10.3.9 getMenuFontSize()

```
auto Config::getMenuFontSize () const [inline]
```

メニュー用フォントサイズを得る

戻り値

フォントサイズ

[Config.h](#) の 203 行目に定義があります。

8.10.3.10 getPreferences()

```
const auto & Config::getPreferences () const [inline]
```

利用可能な投影方式の一覧を得る

戻り値

投影方式一覧への読み取り専用参照

[Config.h](#) の 173 行目に定義があります。

8.10.3.11 getSettings()

```
const Settings & Config::getSettings () const [inline]
```

現在の表示・較正設定を得る

戻り値

設定データへの読み取り専用参照

[Config.h](#) の 153 行目に定義があります。

8.10.3.12 getTitle()

```
const auto & Config::getTitle () const [inline]
```

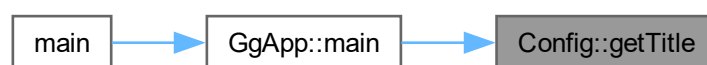
ウィンドウタイトルの文字列を得る

戻り値

ウィンドウのタイトル文字列

[Config.h](#) の 213 行目に定義があります。

被呼び出し関係図:



8.10.3.13 getWidth()

```
auto Config::getWidth () const [inline]
```

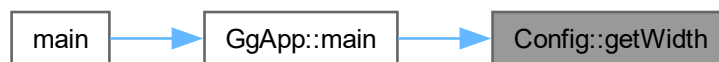
ウィンドウの横幅を得る

戻り値

ウィンドウの横幅

[Config.h](#) の 223 行目に定義があります。

被呼び出し関係図:



8.10.3.14 initialize()

```
void Config::initialize ()
```

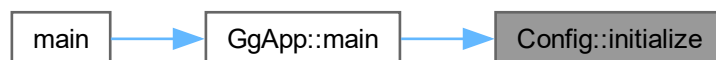
構成データを初期化する

覚え書き

OpenGL のレンダリングコンテキスト作成後に実行する

[Config.cpp](#) の 96 行目に定義があります。

被呼び出し関係図:



8.10.3.15 load()

```
bool Config::load (  
    const pathString & filename)
```

構成ファイルを読み込む

引数

<i>filename</i>	構成データの JSON 形式のファイル名
-----------------	----------------------

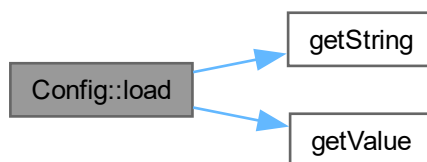
戻り値

構成ファイルの読み込みに成功したら `true`

ファイル内容を一時領域へ読み込んで検証し、成功した場合だけ現在の構成を置き換える。`initialize()` 実行後の再読み込みでは、新しい投影方式のシェーダーも構築する。

`Config.cpp` の 111 行目に定義があります。

呼び出し関係図:



8.10.3.16 save()

```
bool Config::save (  
    const pathString & filename) const
```

構成ファイルを保存する

引数

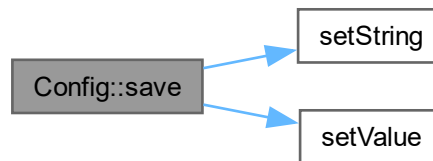
<i>filename</i>	構成データの JSON 形式のファイル名
-----------------	----------------------

戻り値

構成ファイルの保存に成功したら `true`

`Config.cpp` の 222 行目に定義があります。

呼び出し関係図:



8.10.3.17 setSettings()

```
void Config::setSettings (
    const Settings & value) [inline]
```

保存対象の表示・較正設定を更新する

引数

<i>value</i>	新しい設定データ
--------------	----------

[Config.h](#) の 163 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Config.h](#)
- [Config.cpp](#)
- [Menu.cpp](#)

8.11 Expand クラス

```
#include <Expand.h>
```

公開メンバ関数

- [Expand](#) (const std::string &vert, const std::string &frag)
- [Expand](#) (const [Expand](#) &shader)=delete
- virtual ~[Expand](#) ()
- [Expand](#) & operator= (const [Expand](#) &shader)=delete
- auto [getProgram](#) () const
- std::array< GLsizei, 2 > [setup](#) (int samples, GLfloat aspect, const [gg::GgMatrix](#) &pose, const std::array< GLfloat, 2 > &fov, const std::array< GLfloat, 2 > ¢er, GLfloat focal, const std::array< GLfloat, 4 > &border, int unit=0) const

8.11.1 詳解

展開用シェーダクラス

[Expand.h](#) の 17 行目に定義があります。

8.11.2 構築子と解体子

8.11.2.1 Expand() [1/2]

```
Expand::Expand (  
    const std::string & vert,  
    const std::string & frag)
```

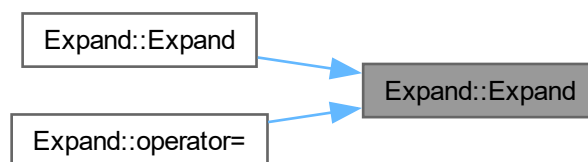
コンストラクタ

引数

<i>vert</i>	バーテックスシェーダのソースファイル名
<i>frag</i>	フラグメントシェーダのソースファイル名

[Expand.cpp](#) の 12 行目に定義があります。

被呼び出し関係図:



8.11.2.2 Expand() [2/2]

```
Expand::Expand (  
    const Expand & shader) [delete]
```

コピーコンストラクタは使用しない

引数

<i>shader</i>	コピー元のシェーダ
---------------	-----------

呼び出し関係図:



8.11.2.3 ~Expand()

```
Expand::~~Expand () [virtual]
```

デストラクタ

[Expand.cpp](#) の 29 行目に定義があります。

8.11.3 関数詳解

8.11.3.1 getProgram()

```
auto Expand::getProgram () const [inline]
```

展開用シェーダのプログラムオブジェクトを得る

戻り値

展開用シェーダのプログラムオブジェクト

[Expand.h](#) の 78 行目に定義があります。

8.11.3.2 operator=()

```
Expand & Expand::operator= (
    const Expand & shader) [delete]
```

代入演算子を使用しない

引数

<i>shader</i>	代入元のシェーダ
---------------	----------

戻り値

代入後のこのシェーダの参照

呼び出し関係図:



8.11.3.3 setup()

```

std::array< int, 2 > Expand::setup (
    int samples,
    GLfloat aspect,
    const gg::GgMatrix & pose,
    const std::array< GLfloat, 2 > & fov,
    const std::array< GLfloat, 2 > & center,
    GLfloat focal,
    const std::array< GLfloat, 4 > & border,
    int unit = 0) const
  
```

展開

引数

<i>samples</i>	展開するテクスチャをサンプリングする数
<i>aspect</i>	展開するテクスチャの縦横比
<i>pose</i>	サンプリングに用いるカメラの姿勢
<i>fov</i>	サンプリングに用いるカメラの相対画角 (単位は度)
<i>center</i>	サンプリングに用いるカメラの撮像面上の中心 (主点) 位置
<i>focal</i>	サンプリングに用いるカメラの主点とスクリーンの距離
<i>border</i>	展開後のフレームの境界色
<i>unit</i>	テクスチャユニット番号

戻り値

描画すべきメッシュの横と縦の格子点数

覚え書き

展開するテクスチャをマッピングするメッシュの解像度を `samples` と `aspect` から個々のメッシュが正方形に近くなるよう決定し、それをもとに格子の間隔を求める

展開

`Expand.cpp` の 37 行目に定義があります。

呼び出し関係図:



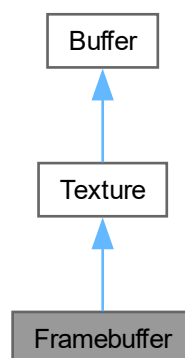
このクラス詳解は次のファイルから抽出されました:

- [Expand.h](#)
- [Expand.cpp](#)

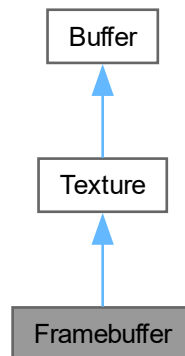
8.12 Framebuffer クラス

```
#include <Framebuffer.h>
```

Framebuffer の継承関係図



Framebuffer 連携図



公開メンバ関数

- `Framebuffer ()=default`
- `Framebuffer (GLsizei width, GLsizei height, int channels=3, GLenum attachment=GL_COLOR_ATTACHMENT0)`
- `Framebuffer (const Framebuffer &framebuffer)`
- `Framebuffer (Framebuffer &&framebuffer) noexcept`
- `virtual ~Framebuffer ()`
- `Framebuffer & operator= (const Framebuffer &framebuffer)`
- `Framebuffer & operator= (Framebuffer &&framebuffer) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `virtual void copy (const Buffer &framebuffer) noexcept`
- `virtual void discard ()`
- `auto getFramebufferName () const`
- `virtual const std::array< GLsizei, 2 > & getSize () const`
- `virtual int getChannels () const`
- `void bindFramebuffer ()`
- `void unbindFramebuffer () const`
- `void update (const std::array< int, 2 > &size)`
- `void update (const std::array< int, 2 > &size, const Texture &frame, int unit=0)`
- `void show (GLsizei width, GLsizei height) const`

基底クラス **Texture** に属する継承公開メンバ関数

- `Texture ()=default`
- `Texture (GLsizei width, GLsizei height, int channels)`
- `Texture (const Texture &texture)`
- `Texture (Texture &&texture) noexcept`
- `virtual ~Texture ()`
- `Texture & operator= (const Texture &texture)`
- `Texture & operator= (Texture &&texture) noexcept`
- `auto getTextureName () const`

- void `bindTexture` (int unit=0) const
- void `unbindTexture` () const
- void `draw` (GLsizei width, GLsizei height, int unit=0) const
- void `readPixels` (GLuint buffer) const
- void `readPixels` (Buffer &buffer) const
- void `readPixels` () const
- void `drawPixels` (GLuint buffer) const
- void `drawPixels` (const Texture &texture)
- void `drawPixels` (const Buffer &buffer)
- void `drawPixels` () const
- void `drawPixels` (GLsizei width, GLsizei height, int channels, const void *pixels)

基底クラス **Buffer** に属する継承公開メンバ関数

- Buffer ()=default
- Buffer (GLsizei width, GLsizei height, int channels)
- Buffer (const Buffer &buffer)
- Buffer (Buffer &&buffer) noexcept
- virtual ~Buffer ()
- Buffer & operator= (const Buffer &buffer)
- Buffer & operator= (Buffer &&buffer) noexcept
- void `resize` (GLsizei width, GLsizei height, int channels)
- void `resize` (const Buffer &buffer)
- auto `getBufferName` () const
- GLsizei `getWidth` () const
- GLsizei `getHeight` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

その他の継承メンバ

基底クラス **Buffer** に属する継承限定公開メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const Buffer &buffer) noexcept

基底クラス **Texture** に属する継承静的限定公開変数類

- static std::shared_ptr< Mesh > `mesh`
テクスチャ展開に用いるメッシュの頂点配列オブジェクト

8.12.1 詳解

フレームバッファオブジェクトクラス

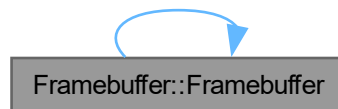
`Framebuffer.h` の 17 行目に定義があります。

8.12.2 構築子と解体子

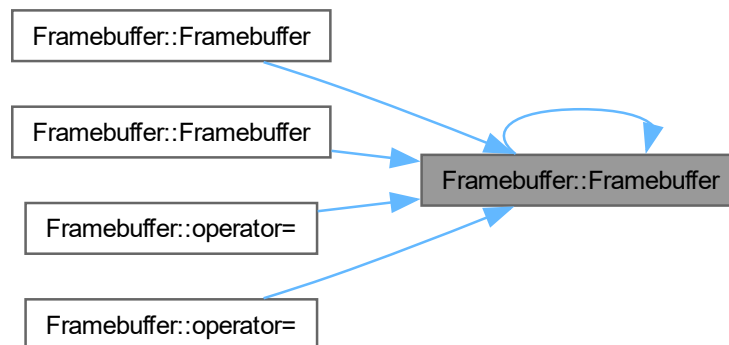
8.12.2.1 Framebuffer() [1/4]

```
Framebuffer::Framebuffer () [default]
```

デフォルトコンストラクタ 呼び出し関係図:



被呼び出し関係図:



8.12.2.2 Framebuffer() [2/4]

```
Framebuffer::Framebuffer (
    GLsizei width,
    GLsizei height,
    int channels = 3,
    GLenum attachment = GL_COLOR_ATTACHMENT0)
```

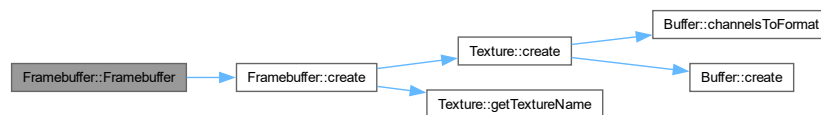
フレームバッファオブジェクトを作成するコンストラクタ

引数

<i>width</i>	作成するフレームバッファオブジェクトの横の画素数
<i>height</i>	作成するフレームバッファオブジェクトの縦の画素数
<i>channels</i>	作成するフレームバッファオブジェクトのチャンネル数
<i>attachment</i>	フレームバッファオブジェクトのカラーバッファのアタッチメント

[Framebuffer.cpp](#) の 13 行目に定義があります。

呼び出し関係図:



8.12.2.3 Framebuffer() [3/4]

```
Framebuffer::Framebuffer (
    const Framebuffer & framebuffer)
```

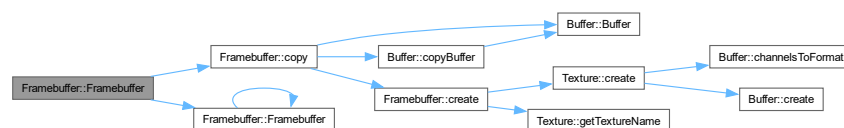
コピーコンストラクタ

引数

<i>framebuffer</i>	コピー元のフレームバッファオブジェクト
--------------------	---------------------

[Framebuffer.cpp](#) の 26 行目に定義があります。

呼び出し関係図:



8.12.2.4 Framebuffer() [4/4]

```
Framebuffer::Framebuffer (
    Framebuffer && framebuffer) [inline], [noexcept]
```

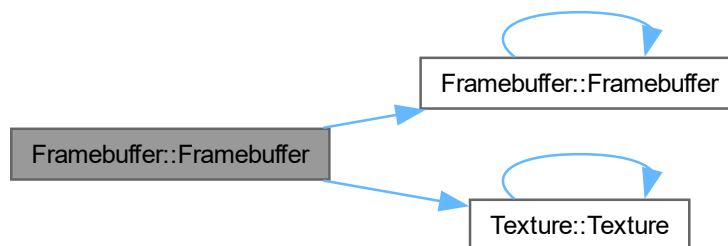
ムーブコンストラクタ

引数

<code>framebuffer</code>	ムーブ元のフレームバッファオブジェクト
--------------------------	---------------------

[Framebuffer.h](#) の 61 行目に定義があります。

呼び出し関係図:



8.12.2.5 ~Framebuffer()

```
Framebuffer::~~Framebuffer () [virtual]
```

デストラクタ

[Framebuffer.cpp](#) の 36 行目に定義があります。

呼び出し関係図:



8.12.3 関数詳解

8.12.3.1 bindFramebuffer()

```
void Framebuffer::bindFramebuffer ()
```

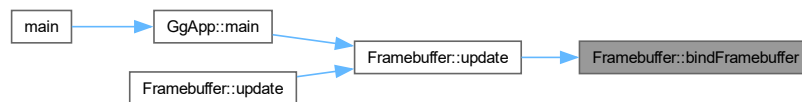
レンダリング先をフレームバッファオブジェクトに切り替える

覚え書き

この後から `unbindFramebuffer()` の前まで、レンダリング先がフレームバッファオブジェクトになる。この時点のビューポートを保存する。この間でレンダリング処理を実行する。

`Framebuffer.cpp` の 154 行目に定義があります。

被呼び出し関係図:



8.12.3.2 copy()

```
void Framebuffer::copy (
    const Buffer & framebuffer) [virtual], [noexcept]
```

フレームバッファオブジェクトをコピーする

引数

<code>framebuffer</code>	コピー元のフレームバッファオブジェクト
--------------------------	---------------------

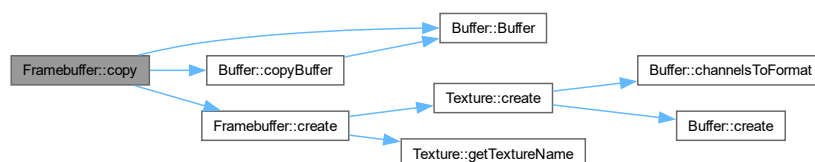
覚え書き

このフレームバッファオブジェクトのサイズがコピー元と異なれば、このフレームバッファオブジェクトを削除して、新しいフレームバッファオブジェクトを作り直してコピーする。引数の型が `Buffer` なのは、このオブジェクトの型が `Buffer` のとき `Buffer::copy()`、`Texture` のとき `Texture::copy()`、`Framebuffer` のとき `Framebuffer::copy()` を呼ぶようにするため。

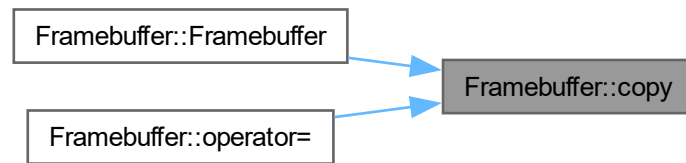
`Texture` を再実装しています。

`Framebuffer.cpp` の 141 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.12.3.3 create()

```

void Framebuffer::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
  
```

フレームバッファオブジェクトを作成する

引数

<i>width</i>	作成するフレームバッファオブジェクトの横の画素数
<i>height</i>	作成するフレームバッファオブジェクトの縦の画素数
<i>channels</i>	作成するフレームバッファオブジェクトのチャンネル数

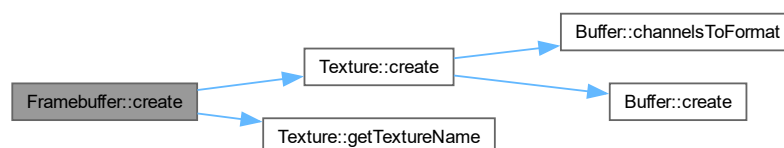
覚え書き

このフレームバッファオブジェクトのサイズが引数で指定したサイズと異なれば、このフレームバッファオブジェクトを削除して、新しいフレームバッファオブジェクトを作り直す。

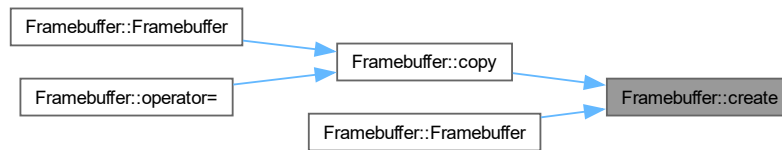
[Texture](#)を再実装しています。

[Framebuffer.cpp](#) の 110 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.12.3.4 discard()

```
void Framebuffer::discard () [virtual]
```

フレームバッファオブジェクトを破棄する

[Texture](#)を再実装しています。

[Framebuffer.cpp](#) の 92 行目に定義があります。

被呼び出し関係図:



8.12.3.5 getChannels()

```
virtual int Framebuffer::getChannels () const [inline], [virtual]
```

フレームバッファオブジェクトのチャンネル数を得る

戻り値

フレームバッファオブジェクトのチャンネル数

[Texture](#)を再実装しています。

[Framebuffer.h](#) の 156 行目に定義があります。

被呼び出し関係図:



8.12.3.6 getFramebufferName()

```
auto Framebuffer::getFramebufferName () const [inline]
```

フレームバッファオブジェクト名を得る

戻り値

フレームバッファオブジェクト名

[Framebuffer.h](#) の 136 行目に定義があります。

8.12.3.7 getSize()

```
virtual const std::array< GLsizei, 2 > & Framebuffer::getSize () const [inline], [virtual]
```

フレームバッファオブジェクトのサイズを得る

戻り値

フレームバッファオブジェクトのサイズ

[Texture](#)を再実装しています。

[Framebuffer.h](#) の 146 行目に定義があります。

8.12.3.8 operator=() [1/2]

```
Framebuffer & Framebuffer::operator= (
    const Framebuffer & framebuffer)
```

代入演算子

引数

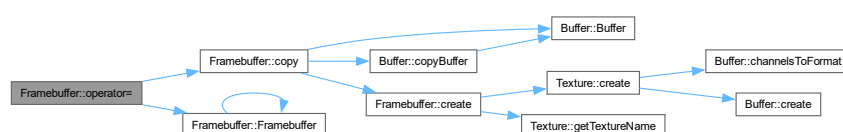
<i>framebuffer</i>	代入元のフレームバッファオブジェクト
--------------------	--------------------

戻り値

代入結果のテクスチャ

[Framebuffer.cpp](#) の 45 行目に定義があります。

呼び出し関係図:



8.12.3.9 operator=() [2/2]

```
Framebuffer & Framebuffer::operator= (  
    Framebuffer && framebuffer) [noexcept]
```

ムーブ代入演算子

引数

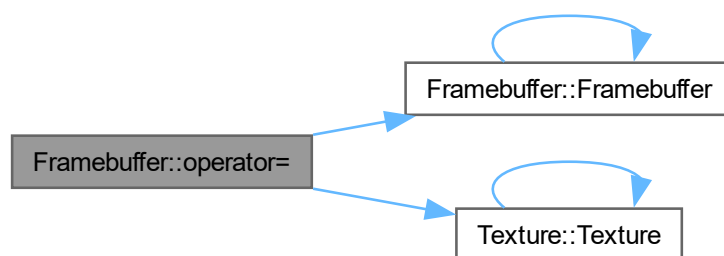
<code>framebuffer</code>	ムーブ代入元のフレームバッファオブジェクト
--------------------------	-----------------------

戻り値

ムーブ代入結果のフレームバッファオブジェクト

[Framebuffer.cpp](#) の 61 行目に定義があります。

呼び出し関係図:



8.12.3.10 show()

```
void Framebuffer::show (
    GLsizei width,
    GLsizei height) const
```

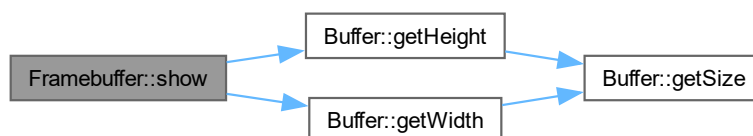
フレームバッファオブジェクトの内容を表示する

引数

<i>width</i>	フレームバッファオブジェクトの内容を表示する横の画素数
<i>height</i>	フレームバッファオブジェクトの内容を表示する縦の画素数

[Framebuffer.cpp](#) の 216 行目に定義があります。

呼び出し関係図:



8.12.3.11 unbindFramebuffer()

```
void Framebuffer::unbindFramebuffer () const
```

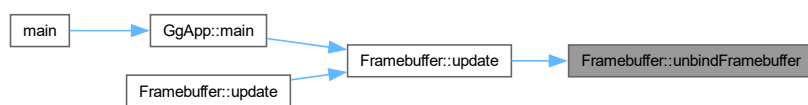
レンダリング先を通常のフレームバッファに戻す

覚え書き

`bindFramebuffer()` の後からこの前まで、レンダリング先がフレームバッファオブジェクトになる。保存したビューポートはここで復帰する。この間でレンダリング処理を実行する。

`Framebuffer.cpp` の 166 行目に定義があります。

被呼び出し関係図:



8.12.3.12 update() [1/2]

```
void Framebuffer::update (
    const std::array< int, 2 > & size)
```

テクスチャを展開してフレームバッファオブジェクトを更新する

引数

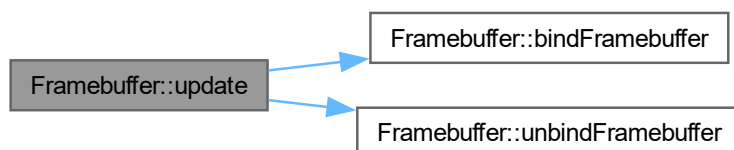
<code>size</code>	展開に用いるメッシュの分割数
-------------------	----------------

覚え書き

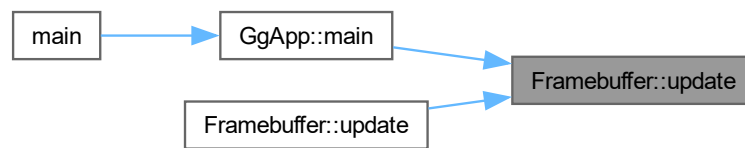
レンダリング先をフレームバッファオブジェクトに切り替えて、フレームバッファオブジェクト全体を覆うメッシュを描画する。これより前でシェードの選択やテクスチャの結合を行う。

`Framebuffer.cpp` の 186 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.12.3.13 update() [2/2]

```

void Framebuffer::update (
    const std::array< int, 2 > & size,
    const Texture & frame,
    int unit = 0)
  
```

テクスチャを展開してフレームバッファオブジェクトを更新する

引数

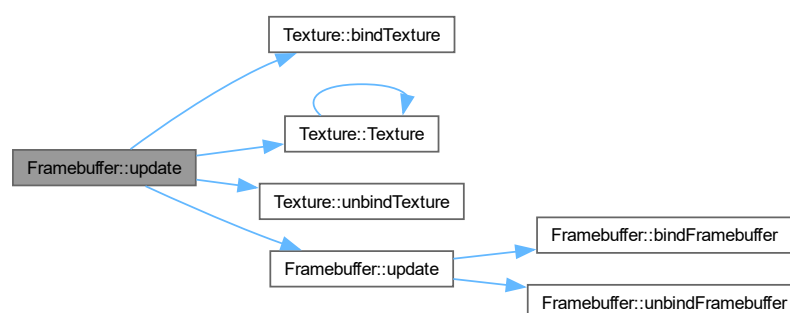
<i>size</i>	展開に用いるメッシュの分割数
<i>frame</i>	展開するフレームを格納したテクスチャ
<i>unit</i>	テクスチャのマッピングに使うテクスチャユニットの番号

覚え書き

レンダリング先をフレームバッファオブジェクトに切り替えて、テクスチャユニット *unit* を指定して *frame* に格納されたテクスチャを結合してから、フレームバッファオブジェクト全体を覆うメッシュを描画する。これより前でシェードの選択を行う。*unit* にはシェードにおいてテクスチャのサンブラの *uniform* 変数に設定する `GL.TEXTUREi` の *i* と一致させること。

[Framebuffer.cpp](#) の 201 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Framebuffer.h](#)
- [Framebuffer.cpp](#)

8.13 GgApp クラス

```
#include <GgApp.h>
```

クラス

- class [Window](#)

公開メンバ関数

- [GgApp](#) (int major=0, int minor=1)
- [GgApp](#) (const [GgApp](#) &w)=delete
- [GgApp](#) ([GgApp](#) &&w)=default
- virtual [~GgApp](#) ()
- [GgApp](#) & [operator=](#) (const [GgApp](#) &w)=delete
- [GgApp](#) & [operator=](#) ([GgApp](#) &&w)=default
- int [main](#) (int argc, const char *const *argv)

静的公開メンバ関数

- static std::string [getUsername](#) ()

8.13.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラス.

[GgApp.h](#) の 133 行目に定義があります。

8.13.2 構築子と解体子

8.13.2.1 GgApp() [1/3]

```
GgApp::GgApp (
    int major = 0,
    int minor = 1)
```

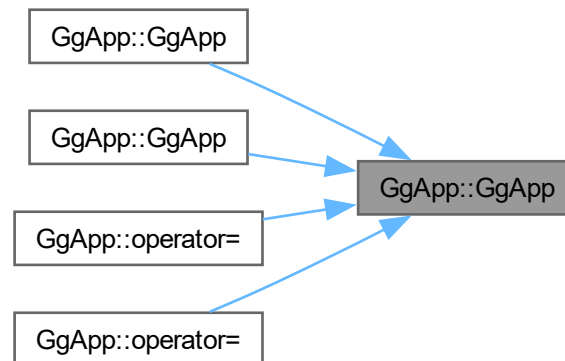
コンストラクタ.

引数

<i>major</i>	使用する OpenGL の major 番号, 0 なら無指定.
<i>minor</i>	使用する OpenGL の minor 番号, major 番号が 0 なら無視.

[GgApp.cpp](#) の 53 行目に定義があります。

被呼び出し関係図:



8.13.2.2 GgApp() [2/3]

```
GgApp::GgApp (
    const GgApp & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のオブジェクト.
----------	--------------

呼び出し関係図:



8.13.2.3 GgApp() [3/3]

```
GgApp::GgApp (
    GgApp && w) [default]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ元のオブジェクト.
----------	--------------

呼び出し関係図:



8.13.2.4 ~GgApp()

```
GgApp::~GgApp () [virtual]
```

デストラクタ.

[GgApp.cpp](#) の 100 行目に定義があります。

8.13.3 関数詳解

8.13.3.1 getUsername()

```
std::string GgApp::getUsername () [static]
```

ユーザ名を得る.

戻り値

ユーザ名の文字列.

[GgApp.cpp](#) の 2216 行目に定義があります。

8.13.3.2 main()

```
int GgApp::main (
    int argc,
    const char *const * argv)
```

アプリケーション本体.

引数

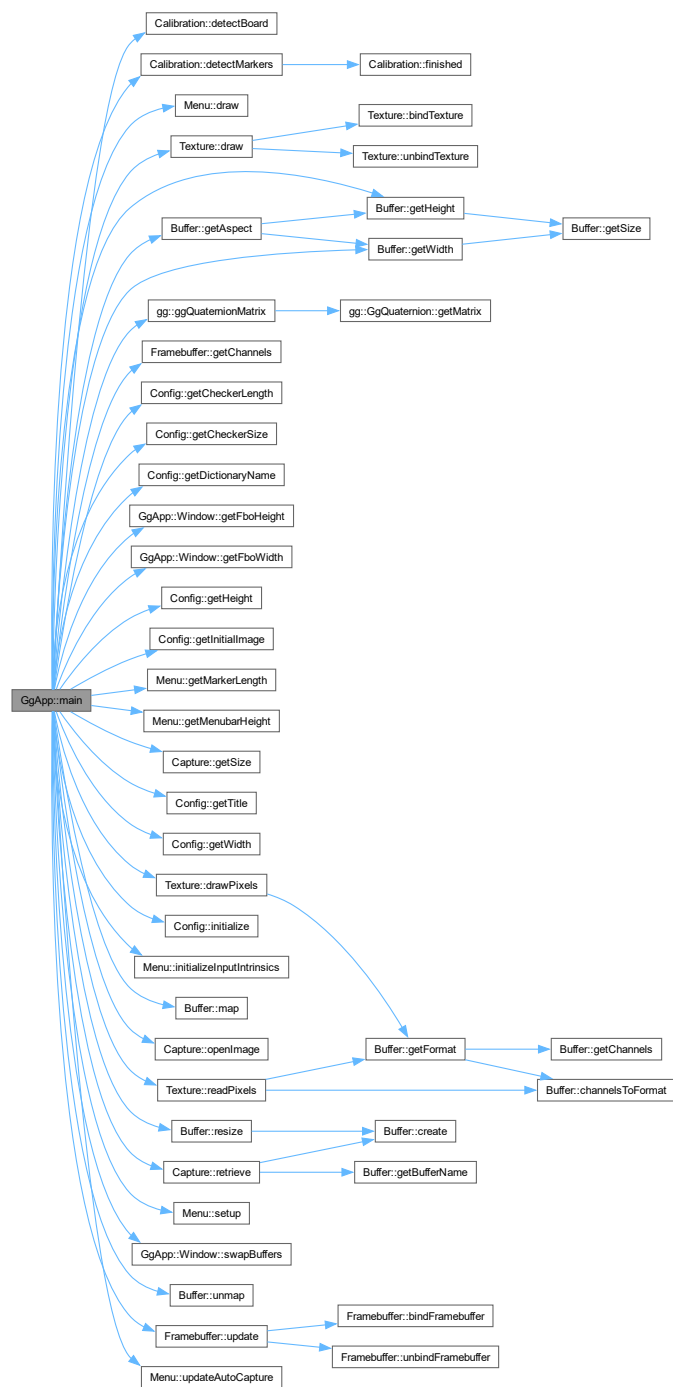
<code>argc</code>	コマンドライン引数の数.
<code>argv</code>	コマンドライン引数の配列.

戻り値

アプリケーションの終了ステータス.

`calib.cpp` の 38 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.13.3.3 operator=() [1/2]

```
GgApp & GgApp::operator= (  
    const GgApp & w) [delete]
```

代入演算子は使用しない.

引数

<i>w</i>	代入元のオブジェクト.
----------	-------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



8.13.3.4 operator=() [2/2]

```
GgApp & GgApp::operator= (  
    GgApp && w) [default]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のオブジェクト.
----------	----------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [calib.cpp](#)
- [GgApp.cpp](#)

8.14 gg::GgBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgBuffer](#) (GLenum target, const T *data, GLsizei stride, GLsizei count, GLenum usage)
- [GgBuffer](#) (const GgBuffer< T > &buffer)=delete
- [GgBuffer](#) (GgBuffer< T > &&o) noexcept
- virtual [~GgBuffer](#) ()
- [GgBuffer](#)< T > & [operator=](#) (const [GgBuffer](#)< T > &buffer)=delete
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLint first, GLsizei count) const
- void [unmap](#) () const
- void [send](#) (const T *data, GLint first, GLsizei count) const
- void [read](#) (T *data, GLint first, GLsizei count) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.14.1 詳解

```
template<typename T>
class gg::GgBuffer< T >
```

バッファオブジェクト.

覚え書き

頂点属性／頂点インデックス／ユニフォーム変数を格納するバッファオブジェクトの基底クラス.

[gg.h](#) の 5834 行目に定義があります。

8.14.2 構築子と解体子

8.14.2.1 GgBuffer() [1/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GLenum target,
    const T * data,
    GLsizei stride,
    GLsizei count,
    GLenum usage) [inline]
```

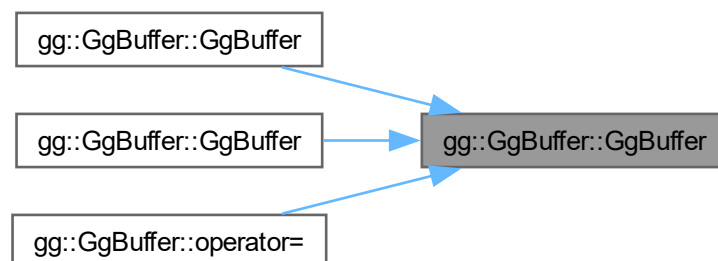
コンストラクタ.

引数

<i>target</i>	バッファオブジェクトのターゲット.
<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>stride</i>	データの間隔.
<i>usage</i>	バッファオブジェクトの使い方.

[gg.h](#) の 5859 行目に定義があります。

被呼び出し関係図:



8.14.2.2 GgBuffer() [2/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    const GgBuffer< T > & buffer) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>buffer</i>	コピー元のバッファ.
---------------	------------

呼び出し関係図:



8.14.2.3 GgBuffer() [3/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GgBuffer< T > && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

<i>o</i>	ムーブ元のバッファ.
----------	------------

覚え書き

バッファオブジェクト名の所有権を移動し, ムーブ元のバッファオブジェクト名を 0 にする. ムーブ元のデストラクタが同じバッファオブジェクトを削除しないようにするため.

gg.h の 5893 行目に定義があります。

呼び出し関係図:



8.14.2.4 ~GgBuffer()

```
template<typename T>
virtual gg::GgBuffer< T >::~~GgBuffer () [inline], [virtual]
```

デストラクタ.

gg.h の 5905 行目に定義があります。

8.14.3 関数詳解

8.14.3.1 bind()

```
template<typename T>
void gg::GgBuffer< T >::bind () const [inline]
```

バッファオブジェクトを結合する.

gg.h の 5963 行目に定義があります。

8.14.3.2 copy()

```
template<typename T>
void gg::GgBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
```

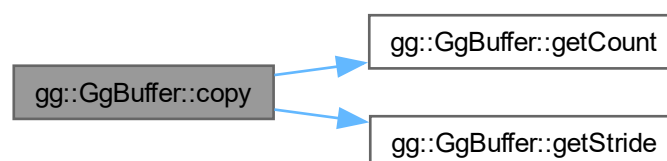
別のバッファオブジェクトからデータを複写する.

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (<i>src_buffer</i>) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6068 行目に定義があります。

呼び出し関係図:



8.14.3.3 getBuffer()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getBuffer () const [inline]
```

バッファオブジェクト名を取り出す.

戻り値

このバッファオブジェクト名.

gg.h の 5955 行目に定義があります。

8.14.3.4 getCount()

```
template<typename T>
const GLsizei & gg::GgBuffer< T >::getCount () const [inline]
```

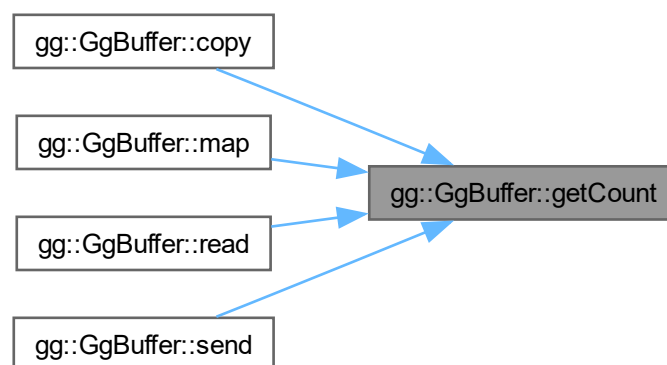
バッファオブジェクトが保持するデータの数を取り出す.

戻り値

このバッファオブジェクトが保持するデータの数.

gg.h の 5945 行目に定義があります。

被呼び出し関係図:



8.14.3.5 getStride()

```
template<typename T>
GLsizeiPtr gg::GgBuffer< T >::getStride () const [inline]
```

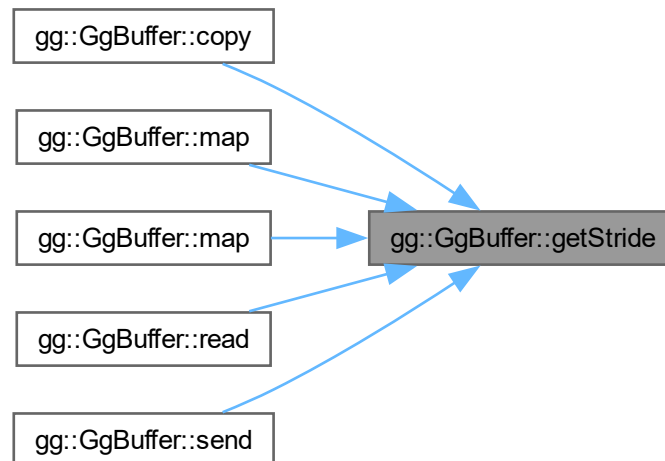
バッファオブジェクトのアライメントを考慮したデータの間隔を取り出す。

戻り値

このバッファオブジェクトのデータの間隔。

gg.h の 5935 行目に定義があります。

被呼び出し関係図:



8.14.3.6 getTarget()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getTarget () const [inline]
```

バッファオブジェクトのターゲットを取り出す。

戻り値

このバッファオブジェクトのターゲット。

gg.h の 5925 行目に定義があります。

8.14.3.7 map() [1/2]

```
template<typename T>
void * gg::GgBuffer< T >::map () const [inline]
```

バッファオブジェクトをマップする.

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5981 行目に定義があります。

呼び出し関係図:



8.14.3.8 map() [2/2]

```
template<typename T>
void * gg::GgBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

バッファオブジェクトの指定した範囲をマップする.

引数

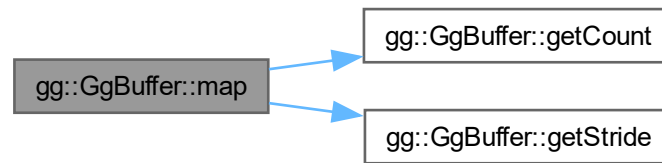
<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5998 行目に定義があります。

呼び出し関係図:



8.14.3.9 operator=()

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    const GgBuffer< T > & buffer) [delete]
```

代入演算子は使用しない.

引数

<i>buffer</i>	代入元のバッファ.
---------------	-----------

戻り値

代入後のこのバッファの参照.

呼び出し関係図:



8.14.3.10 read()

```
template<typename T>
void gg::GgBuffer< T >::read (
    T * data,
    GLint first,
    GLsizei count) const [inline]
```

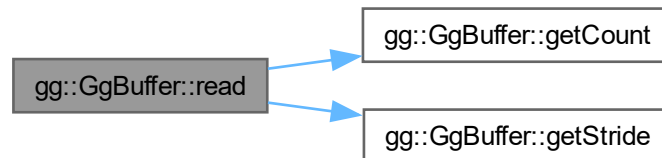
バッファオブジェクトのデータから抽出する.

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>first</i>	抽出元のバッファオブジェクトの取り出すデータの領域の先頭の要素番号.
<i>count</i>	抽出するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6041 行目に定義があります。

呼び出し関係図:



8.14.3.11 send()

```

template<typename T>
void gg::GgBuffer< T >::send (
    const T * data,
    GLint first,
    GLsizei count) const [inline]
  
```

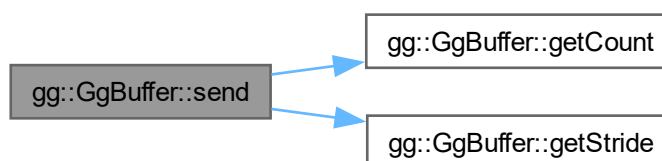
すでに確保したバッファオブジェクトにデータを転送する。

引数

<i>data</i>	転送元のデータが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6023 行目に定義があります。

呼び出し関係図:



8.14.3.12 unbind()

```
template<typename T>
void gg::GgBuffer< T >::unbind () const [inline]
```

バッファオブジェクトを解放する。

gg.h の 5971 行目に定義があります。

8.14.3.13 unmap()

```
template<typename T>
void gg::GgBuffer< T >::unmap () const [inline]
```

バッファオブジェクトをアンマップする。

gg.h の 6011 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.15 gg::GgColorTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgColorTexture \(\)](#)
- [GgColorTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- [GgColorTexture](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- virtual [~GgColorTexture \(\)](#)
- void [load](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- void [load](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- [operator bool \(\)](#) const noexcept
- bool [operator!](#) () const noexcept
- const [GgTexture](#) * [get](#) () const
- GLuint [getTexture](#) () const
- GLsizei [getWidth](#) () const
- GLsizei [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- void [bind](#) () const
- void [unbind](#) () const

8.15.1 詳解

カラーマップ.

覚え書き

カラー画像を読み込んでテクスチャを作成する.

gg.h の 5421 行目に定義があります。

8.15.2 構築子と解体子

8.15.2.1 GgColorTexture() [1/3]

```
gg::GgColorTexture::GgColorTexture () [inline]
```

コンストラクタ.

gg.h の 5431 行目に定義があります。

8.15.2.2 GgColorTexture() [2/3]

```
gg::GgColorTexture::GgColorTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = true) [inline]
```

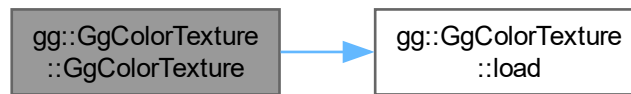
メモリ上のデータからカラーのテクスチャを作成するコンストラクタ.

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	読み込む画像の横の画素数.
<i>height</i>	読み込む画像の縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

gg.h の 5447 行目に定義があります。

呼び出し関係図:



8.15.2.3 GgColorTexture() [3/3]

```

gg::GgColorTexture::GgColorTexture (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE) [inline]
  
```

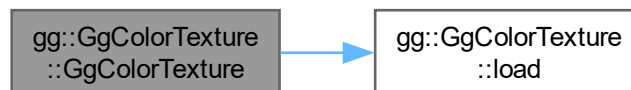
TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成するコンストラクタ.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, GL_TEXTURE_WRAP_S および GL_TEXTURE_WRAP_T に設定する値.

[gg.h](#) の 5468 行目に定義があります。

呼び出し関係図:



8.15.2.4 ~GgColorTexture()

```

virtual gg::GgColorTexture::~~GgColorTexture () [inline], [virtual]
  
```

デストラクタ.

[gg.h](#) の 5480 行目に定義があります。

8.15.3 関数詳解

8.15.3.1 bind()

```
void gg::GgColorTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

[gg.h](#) の 5607 行目に定義があります。

8.15.3.2 get()

```
const GgTexture * gg::GgColorTexture::get () const [inline]
```

テクスチャオブジェクトを取り出す.

戻り値

[GgTexture](#) オブジェクトのポインタ.

[gg.h](#) の 5549 行目に定義があります。

8.15.3.3 getHeight()

```
GLsizei gg::GgColorTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す.

戻り値

テクスチャの縦の画素数.

[gg.h](#) の 5579 行目に定義があります。

8.15.3.4 getSize() [1/2]

```
const GLsizei * gg::GgColorTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す.

戻り値

テクスチャのサイズを格納した配列へのポインタ.

[gg.h](#) の 5599 行目に定義があります。

8.15.3.5 getSize() [2/2]

```
void gg::GgColorTexture::getSize (  
    GLsizei * size) const [inline]
```

使用しているテクスチャのサイズを取り出す.

引数

<i>size</i>	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数.
-------------	--------------------------------------

[gg.h](#) の 5589 行目に定義があります。

8.15.3.6 getTexture()

```
GLuint gg::GgColorTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る.

戻り値

テクスチャ名.

[gg.h](#) の 5559 行目に定義があります。

8.15.3.7 getWidth()

```
GLsizei gg::GgColorTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す.

戻り値

テクスチャの横の画素数.

[gg.h](#) の 5569 行目に定義があります。

8.15.3.8 load() [1/2]

```
void gg::GgColorTexture::load (  
    const GLvoid * image,  
    GLsizei width,  
    GLsizei height,  
    GLenum format = GL_RGB,  
    GLenum type = GL_UNSIGNED_BYTE,  
    GLenum internal = GL_RGB,  
    GLenum wrap = GL_CLAMP_TO_EDGE,  
    bool swizzle = true) [inline]
```

メモリ上のデータを読み込んでテクスチャを作成する.

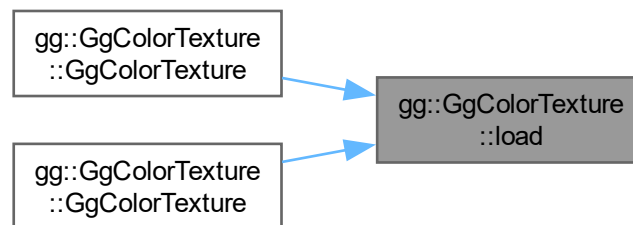
引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.

<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

gg.h の 5496 行目に定義があります。

被呼び出し関係図:



8.15.3.9 load() [2/2]

```

void gg::GgColorTexture::load (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE)
  
```

TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 ならファイルの画像フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).

gg.cpp の 4074 行目に定義があります。

呼び出し関係図:



8.15.3.10 operator bool()

```
gg::GgColorTexture::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる.

戻り値

オブジェクトが有効なら **true**.

[gg.h](#) の 5529 行目に定義があります。

8.15.3.11 operator"!()"

```
bool gg::GgColorTexture::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する.

戻り値

オブジェクトが有効なら **false**, 無効なら **true**.

[gg.h](#) の 5539 行目に定義があります。

8.15.3.12 unbind()

```
void gg::GgColorTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

[gg.h](#) の 5615 行目に定義があります。

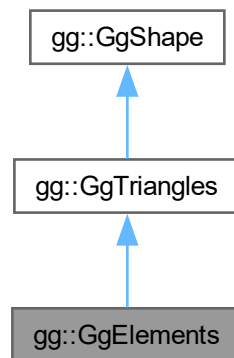
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

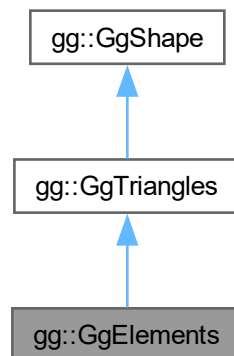
8.16 gg::GgElements クラス

```
#include <gg.h>
```

gg::GgElements の継承関係図



gg::GgElements 連携図



公開メンバ関数

- [GgElements](#) (GLenum mode=GL_TRIANGLES)
- [GgElements](#) (const [GgVertex](#) *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgElements](#) ()
- const GLsizei & [getIndexCount](#) () const

- const GLuint & [getIndexBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLuint firstv, GLsizei countv, const GLuint *face=NULLPTR, GLuint firstf=0, GLsizei countf=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLuint first=0, GLsizei count=0) const

基底クラス [gg::GgTriangles](#) に属する継承公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLuint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

8.16.1 詳解

三角形で表した形状データ (Elements 形式).

[gg.h](#) の 6901 行目に定義があります。

8.16.2 構築子と解体子

8.16.2.1 GgElements() [1/2]

```
gg::GgElements::GgElements (
    GLenum mode = GL_TRIANGLES) [inline]
```

コンストラクタ.

引数

<i>mode</i>	描画する基本図形の種類.
-------------	--------------

[gg.h](#) の 6914 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.2.2 GgElements() [2/2]

```

gg::GgElements::GgElements (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

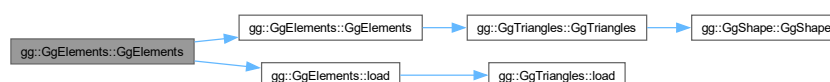
コンストラクタ.

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>face</i>	三角形の頂点インデックス.
<i>countf</i>	三角形の頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6929 行目に定義があります。

呼び出し関係図:



8.16.2.3 ~GgElements()

```
virtual gg::GgElements::~~GgElements () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6945 行目に定義があります。

8.16.3 関数詳解

8.16.3.1 draw()

```
void gg::GgElements::draw (  
    GLint first = 0,  
    GLsizei count = 0) const [virtual]
```

インデックスを使った三角形の描画.

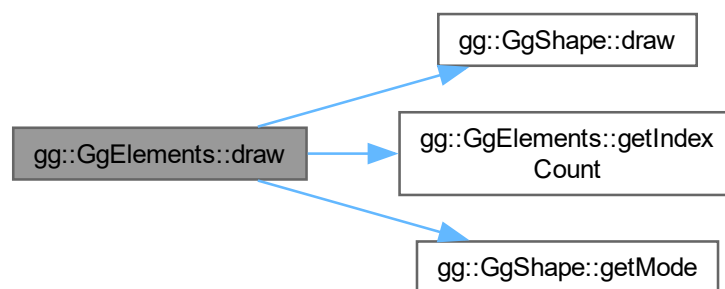
引数

<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

[gg::GgTriangles](#) を再実装しています。

[gg.cpp](#) の 5302 行目に定義があります。

呼び出し関係図:



8.16.3.2 getIndexBuffer()

```
const GLuint & gg::GgElements::getIndexBuffer () const [inline]
```

三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名を取り出す。

戻り値

この図形の三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名。

gg.h の 6963 行目に定義があります。

8.16.3.3 getIndexCount()

```
const GLsizei & gg::GgElements::getIndexCount () const [inline]
```

データの数を取り出す。

戻り値

この図形の三角形数。

gg.h の 6953 行目に定義があります。

被呼び出し関係図:



8.16.3.4 load()

```
void gg::GgElements::load (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

バッファオブジェクトを確保して頂点属性と三角形の頂点インデックスデータを格納する。

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>countv</i>	頂点のデータの数 (頂点数).
<i>face</i>	三角形の頂点インデックスデータ.
<i>countf</i>	三角形の頂点数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 7000 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.16.3.5 send()

```

void gg::GgElements::send (
    const GgVertex * vert,
    GLuint firstv,
    GLsizei countv,
    const GLuint * face = nullptr,
    GLuint firstf = 0,
    GLsizei countf = 0) const [inline]
  
```

既存のバッファオブジェクトに頂点属性と三角形の頂点インデックスデータを転送する。

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>firstv</i>	頂点属性の転送先のバッファオブジェクトの先頭の要素番号.
<i>countv</i>	頂点のデータの数 (頂点数).

<i>face</i>	三角形の頂点インデックスデータ.
<i>firstf</i>	インデックスの転送先のバッファオブジェクトの先頭の要素番号.
<i>countf</i>	三角形の頂点数.

gg.h の 6978 行目に定義があります。

呼び出し関係図:



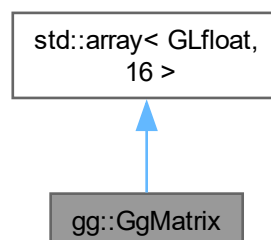
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

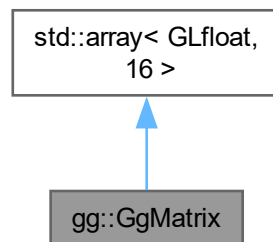
8.17 gg::GgMatrix クラス

```
#include <gg.h>
```

gg::GgMatrix の継承関係図



gg::GgMatrix 連携図



公開メンバ関数

- **GgMatrix** ()=default
- constexpr **GgMatrix** (GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03, GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13, GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23, GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33)
- constexpr **GgMatrix** (const GLfloat *a)
- constexpr **GgMatrix** (GLfloat c)
- **GgMatrix** (const GgMatrix &m)=default
- **GgMatrix** (GgMatrix &&m)=default
- ~**GgMatrix** ()=default
- **GgMatrix** & operator= (const **GgMatrix** &m)=default
- **GgMatrix** & operator= (**GgMatrix** &&m)=default
- **GgMatrix** & operator= (const GLfloat *a)
- **GgMatrix** operator+ (const GLfloat *a) const
- **GgMatrix** operator+ (const **GgMatrix** &m) const
- **GgMatrix** & operator+= (const GLfloat *a)
- **GgMatrix** & operator+= (const **GgMatrix** &m)
- **GgMatrix** operator- (const GLfloat *a) const
- **GgMatrix** operator- (const **GgMatrix** &m) const
- **GgMatrix** & operator-= (const GLfloat *a)
- **GgMatrix** & operator-= (const **GgMatrix** &m)
- **GgMatrix** operator* (const GLfloat *a) const
- **GgMatrix** operator* (const **GgMatrix** &m) const
- **GgMatrix** & operator*= (const GLfloat *a)
- **GgMatrix** & operator*= (const **GgMatrix** &m)
- **GgMatrix** operator/ (const GLfloat *a) const
- **GgMatrix** operator/ (const **GgMatrix** &m) const
- **GgMatrix** & operator/= (const GLfloat *a)
- **GgMatrix** & operator/= (const **GgMatrix** &m)
- **GgMatrix** & loadIdentity ()
- **GgMatrix** & loadTranslate (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- **GgMatrix** & loadTranslate (const GLfloat *t)
- **GgMatrix** & loadTranslate (const **GgVector** &t)
- **GgMatrix** & loadScale (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- **GgMatrix** & loadScale (const GLfloat *s)
- **GgMatrix** & loadScale (const **GgVector** &s)

- `GgMatrix & loadRotateX` (GLfloat a)
- `GgMatrix & loadRotateY` (GLfloat a)
- `GgMatrix & loadRotateZ` (GLfloat a)
- `GgMatrix & loadRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix & loadRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix & loadRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix & loadRotate` (const GLfloat *r)
- `GgMatrix & loadRotate` (const `GgVector` &r)
- `GgMatrix & loadLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix & loadLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix & loadLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix & loadOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadTranspose` (const GLfloat *a)
- `GgMatrix & loadTranspose` (const `GgMatrix` &m)
- `GgMatrix & loadInvert` (const GLfloat *a)
- `GgMatrix & loadInvert` (const `GgMatrix` &m)
- `GgMatrix & loadNormal` (const GLfloat *a)
- `GgMatrix & loadNormal` (const `GgMatrix` &m)
- `GgMatrix translate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix translate` (const GLfloat *t) const
- `GgMatrix translate` (const `GgVector` &t) const
- `GgMatrix scale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix scale` (const GLfloat *s) const
- `GgMatrix scale` (const `GgVector` &s) const
- `GgMatrix rotateX` (GLfloat a) const
- `GgMatrix rotateY` (GLfloat a) const
- `GgMatrix rotateZ` (GLfloat a) const
- `GgMatrix rotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r, GLfloat a) const
- `GgMatrix rotate` (const `GgVector` &r, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r) const
- `GgMatrix rotate` (const `GgVector` &r) const
- `GgMatrix lookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz) const
- `GgMatrix lookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u) const
- `GgMatrix lookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u) const
- `GgMatrix orthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix frustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix perspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar) const
- `GgMatrix transpose` () const
- `GgMatrix invert` () const
- `GgMatrix normal` () const
- void `projection` (GLfloat *c, const GLfloat *v) const
- void `projection` (GLfloat *c, const `GgVector` &v) const
- void `projection` (`GgVector` &c, const GLfloat *v) const
- void `projection` (`GgVector` &c, const `GgVector` &v) const
- `GgVector operator*` (const `GgVector` &v) const
- const GLfloat * `get` () const
- void `get` (GLfloat *a) const

8.17.1 詳解

変換行列.

[gg.h](#) の 2169 行目に定義があります。

8.17.2 構築子と解体子

8.17.2.1 GgMatrix() [1/6]

```
gg::GgMatrix::GgMatrix () [default]
```

コンストラクタ.

8.17.2.2 GgMatrix() [2/6]

```
gg::GgMatrix::GgMatrix (  
    GLfloat m00,  
    GLfloat m01,  
    GLfloat m02,  
    GLfloat m03,  
    GLfloat m10,  
    GLfloat m11,  
    GLfloat m12,  
    GLfloat m13,  
    GLfloat m20,  
    GLfloat m21,  
    GLfloat m22,  
    GLfloat m23,  
    GLfloat m30,  
    GLfloat m31,  
    GLfloat m32,  
    GLfloat m33) [inline], [constexpr]
```

コンストラクタ.

引数

<i>m00</i>	GLfloat 型の値.
<i>m01</i>	GLfloat 型の値.
<i>m02</i>	GLfloat 型の値.
<i>m03</i>	GLfloat 型の値.
<i>m10</i>	GLfloat 型の値.
<i>m11</i>	GLfloat 型の値.
<i>m12</i>	GLfloat 型の値.
<i>m13</i>	GLfloat 型の値.
<i>m20</i>	GLfloat 型の値.
<i>m21</i>	GLfloat 型の値.

<i>m22</i>	GLfloat 型の値.
<i>m23</i>	GLfloat 型の値.
<i>m30</i>	GLfloat 型の値.
<i>m31</i>	GLfloat 型の値.
<i>m32</i>	GLfloat 型の値.
<i>m33</i>	GLfloat 型の値.

gg.h の 2204 行目に定義があります。

8.17.2.3 GgMatrix() [3/6]

```
gg::GgMatrix::GgMatrix (
    const GLfloat * a) [inline], [constexpr]
```

コンストラクタ.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

gg.h の 2219 行目に定義があります。

呼び出し関係図:



8.17.2.4 GgMatrix() [4/6]

```
gg::GgMatrix::GgMatrix (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

gg.h の 2229 行目に定義があります。

呼び出し関係図:



8.17.2.5 GgMatrix() [5/6]

```
gg::GgMatrix::GgMatrix (  
    const GgMatrix & m) [default]
```

コピーコンストラクタ.

引数

<i>m</i>	コピー元の変換行列.
----------	------------

呼び出し関係図:



8.17.2.6 GgMatrix() [6/6]

```
gg::GgMatrix::GgMatrix (  
    GgMatrix && m) [default]
```

ムーブコンストラクタ.

引数

<i>m</i>	ムーブ元の変換行列.
----------	------------

呼び出し関係図:



8.17.2.7 ~GgMatrix()

```
gg::GgMatrix::~~GgMatrix () [default]
```

デストラクタ.

8.17.3 関数詳解

8.17.3.1 frustum()

```
GgMatrix gg::GgMatrix::frustum (  
    GLfloat left,  
    GLfloat right,  
    GLfloat bottom,  
    GLfloat top,  
    GLfloat zNear,  
    GLfloat zFar) const [inline]
```

透視投影変換を乗じた結果を返す.

引数

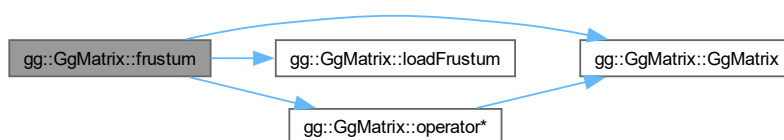
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

[gg.h](#) の [3012](#) 行目に定義があります。

呼び出し関係図:



8.17.3.2 `get()` [1/2]

```
const GLfloat * gg::GgMatrix::get () const [inline]
```

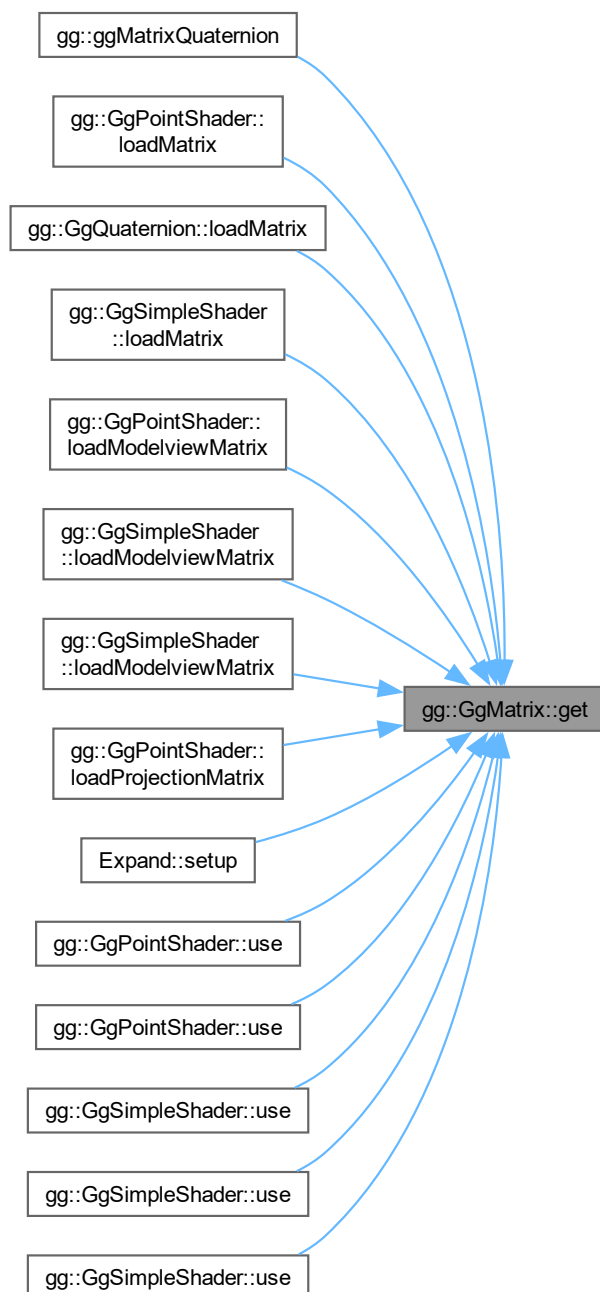
変換行列を取り出す.

戻り値

変換行列を格納した GLfloat 型の 16 要素の配列変数.

gg.h の 3135 行目に定義があります。

被呼び出し関係図:



8.17.3.3 get() [2/2]

```
void gg::GgMatrix::get (  
    GLfloat * a) const [inline]
```

変換行列を取り出す.

引数

<i>a</i>	変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

[gg.h](#) の 3145 行目に定義があります。

8.17.3.4 invert()

```
GgMatrix gg::GgMatrix::invert () const [inline]
```

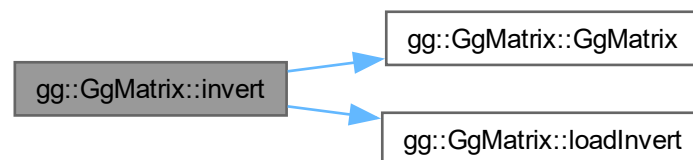
逆行列を返す.

戻り値

逆行列.

[gg.h](#) の 3056 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.5 loadFrustum()

```

gg::GgMatrix & gg::GgMatrix::loadFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)

```

透視投影変換行列を格納する.

引数

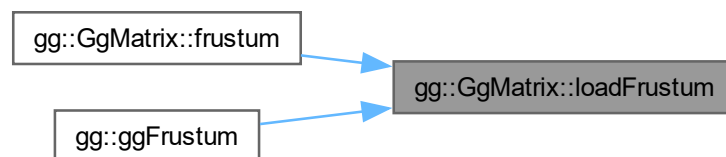
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

gg.cpp の 3131 行目に定義があります。

被呼び出し関係図:



8.17.3.6 loadIdentity()

```

gg::GgMatrix & gg::GgMatrix::loadIdentity ()

```

単位行列を格納する.

gg.cpp の 2742 行目に定義があります。

被呼び出し関係図:



8.17.3.7 loadInvert() [1/2]

```
GgMatrix & gg::GgMatrix::loadInvert (
    const GgMatrix & m) [inline]
```

逆行列を格納する.

引数

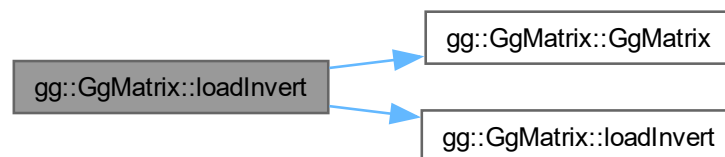
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

設定した *m* の逆行列.

gg.h の 2735 行目に定義があります。

呼び出し関係図:



8.17.3.8 loadInvert() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadInvert (
    const GLfloat * a)
```

逆行列を格納する.

引数

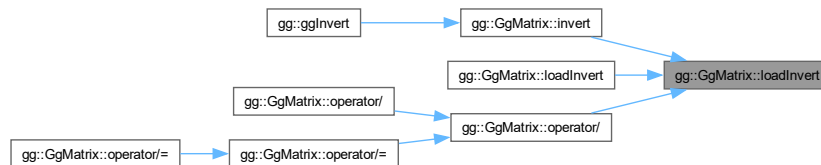
a	GLfloat 型の 16 要素の変換行列.
-----	------------------------

戻り値

設定した a の逆行列.

gg.cpp の 2917 行目に定義があります。

被呼び出し関係図:



8.17.3.9 loadLookat() [1/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]
```

ビュー変換行列を格納する.

引数

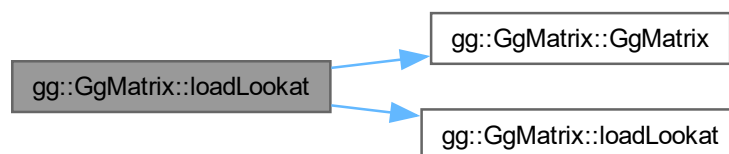
e	視点の位置の GgVector 型の変数.
t	目標点の位置の GgVector 型の変数.
u	上方向のベクトルの GgVector 型の変数.

戻り値

設定したビュー変換行列.

gg.h の 2655 行目に定義があります。

呼び出し関係図:



8.17.3.10 loadLookat() [2/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
```

ビュー変換行列を格納する.

引数

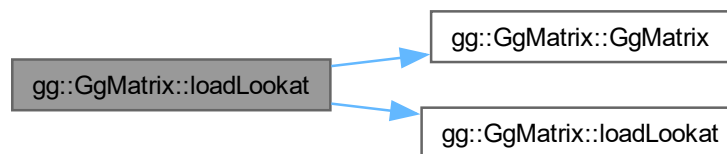
<i>e</i>	視点の位置の配列変数.
<i>t</i>	目標点の位置の配列変数.
<i>u</i>	上方向のベクトルの配列変数.

戻り値

設定したビュー変換行列.

[gg.h](#) の 2642 行目に定義があります。

呼び出し関係図:



8.17.3.11 loadLookat() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz)
```

ビュー変換行列を格納する.

引数

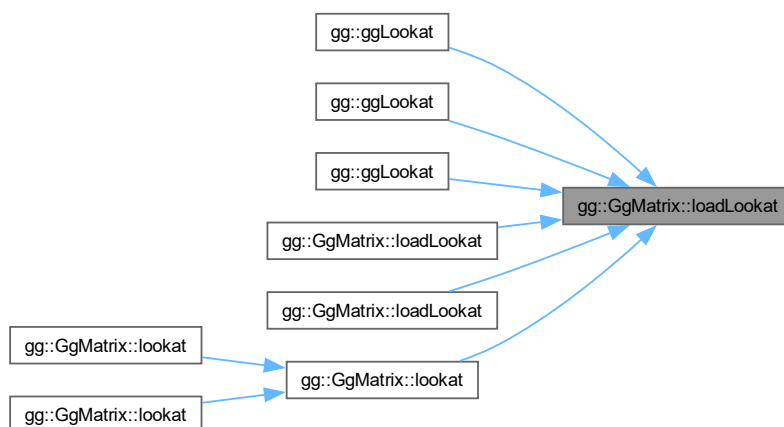
ex	視点の位置の x 座標値.
ey	視点の位置の y 座標値.
ez	視点の位置の z 座標値.
tx	目標点の位置の x 座標値.
ty	目標点の位置の y 座標値.
tz	目標点の位置の z 座標値.
ux	上方向のベクトルの x 成分.
uy	上方向のベクトルの y 成分.
uz	上方向のベクトルの z 成分.

戻り値

設定したビュー変換行列.

gg.cpp の 3031 行目に定義があります。

被呼び出し関係図:



8.17.3.12 loadNormal() [1/2]

```
GgMatrix & gg::GgMatrix::loadNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を格納する.

引数

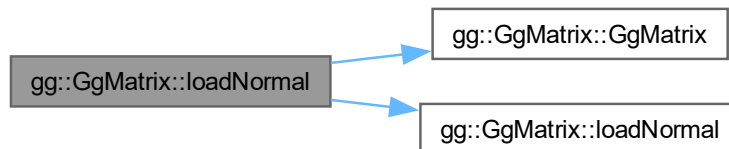
m	GgMatrix 型の変換行列.
-----	------------------

戻り値

設定した `m` の法線変換行列.

`gg.h` の 2754 行目に定義があります。

呼び出し関係図:



8.17.3.13 loadNormal() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadNormal (
    const GLfloat * a)
```

法線変換行列を格納する.

引数

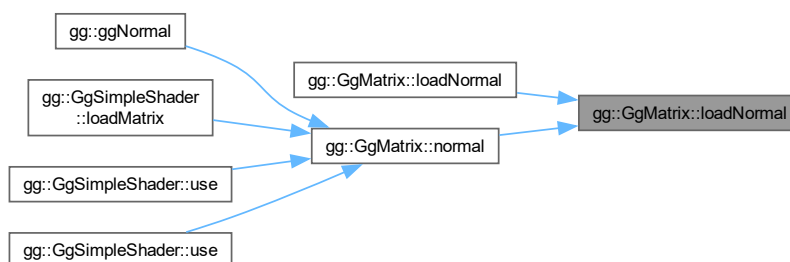
<code>a</code>	GLfloat 型の 16 要素の変換行列.
----------------	------------------------

戻り値

設定した `a` の法線変換行列.

`gg.cpp` の 3000 行目に定義があります。

被呼び出し関係図:



8.17.3.14 loadOrthogonal()

```
gg::GgMatrix & gg::GgMatrix::loadOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)
```

直交投影変換行列を格納する。

引数

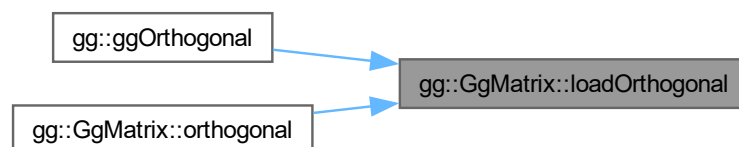
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した直交投影変換行列.

gg.cpp の 3090 行目に定義があります。

被呼び出し関係図:



8.17.3.15 loadPerspective()

```
gg::GgMatrix & gg::GgMatrix::loadPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar)
```

画角を指定して透視投影変換行列を格納する。

引数

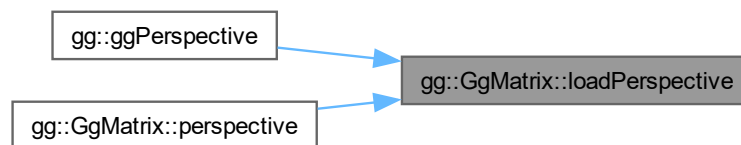
<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

[gg.cpp](#) の 3172 行目に定義があります。

被呼び出し関係図:



8.17.3.16 loadRotate() [1/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

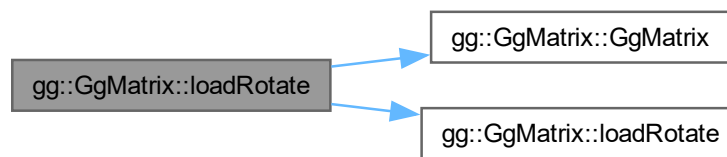
<i>r</i>	回転軸の方向ベクトルと回転角を格納した GgVector 型のベクトル, 第 4 要素を回転角に用いる.
----------	--

戻り値

設定した変換行列.

[gg.h](#) の 2611 行目に定義があります。

呼び出し関係図:



8.17.3.17 loadRotate() [2/5]

```
GgMatrix & gg::GgMatrix::loadRotate (  
    const GgVector & r,  
    GLfloat a) [inline]
```

`r` 方向のベクトルを軸とする回転の変換行列を格納する.

引数

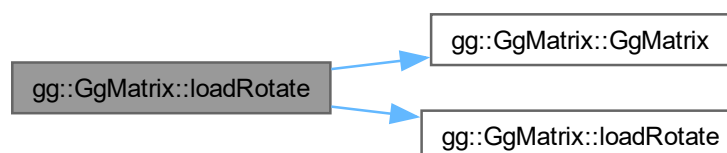
<i>r</i>	回転軸の方向ベクトルを格納した <code>GgVector</code> 型のベクトル, 第 4 要素は無視する.
<i>a</i>	回転角.

戻り値

設定した変換行列.

`gg.h` の 2589 行目に定義があります。

呼び出し関係図:



8.17.3.18 loadRotate() [3/5]

```
GgMatrix & gg::GgMatrix::loadRotate (  
    const GLfloat * r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

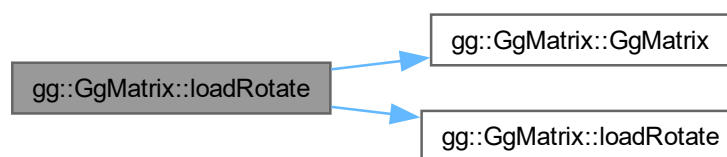
<i>r</i>	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数, 第 4 要素を回転角に用いる.
----------	---

戻り値

設定した変換行列.

gg.h の 2600 行目に定義があります。

呼び出し関係図:



8.17.3.19 loadRotate() [4/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

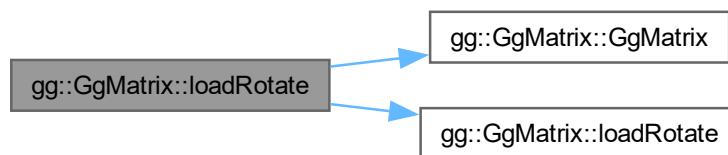
<i>r</i>	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
<i>a</i>	回転角.

戻り値

設定した変換行列.

gg.h の 2577 行目に定義があります。

呼び出し関係図:



8.17.3.20 loadRotate() [5/5]

```
gg::GgMatrix & gg::GgMatrix::loadRotate (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat a)
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を格納する.

引数

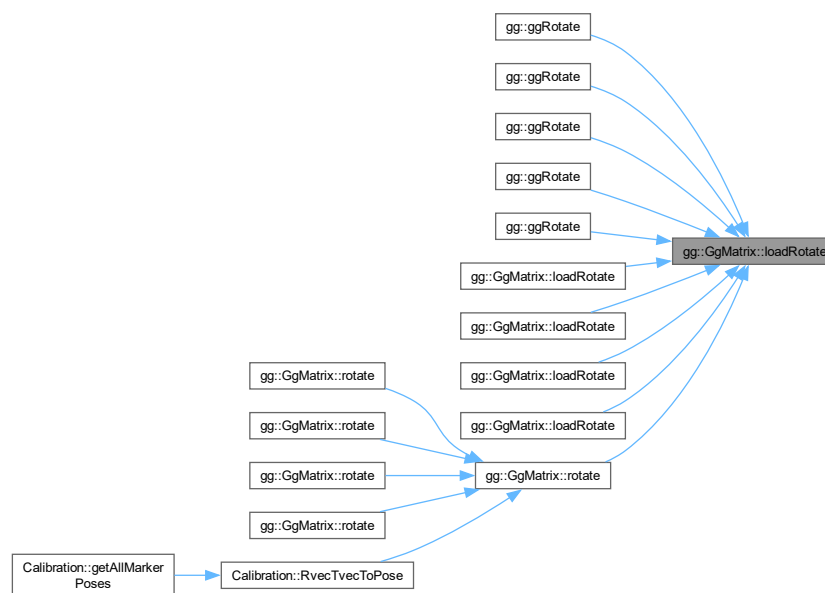
<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

設定した変換行列.

[gg.cpp](#) の [2847](#) 行目に定義があります。

被呼び出し関係図:



8.17.3.21 loadRotateX()

```
gg::GgMatrix & gg::GgMatrix::loadRotateX (
    GLfloat a)
```

x 軸中心の回転の変換行列を格納する.

引数

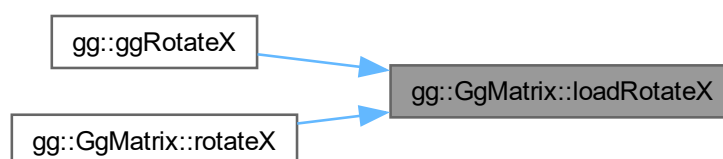
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

[gg.cpp](#) の 2790 行目に定義があります.

被呼び出し関係図:



8.17.3.22 loadRotateY()

```
gg::GgMatrix & gg::GgMatrix::loadRotateY (  
    GLfloat a)
```

y 軸中心の回転の変換行列を格納する.

引数

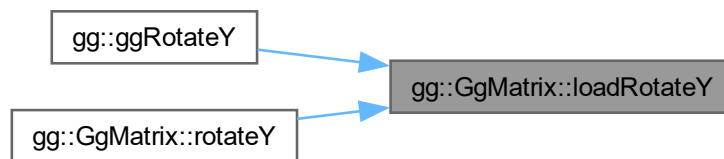
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

`gg.cpp` の 2809 行目に定義があります。

被呼び出し関係図:



8.17.3.23 loadRotateZ()

```
gg::GgMatrix & gg::GgMatrix::loadRotateZ (  
    GLfloat a)
```

z 軸中心の回転の変換行列を格納する.

引数

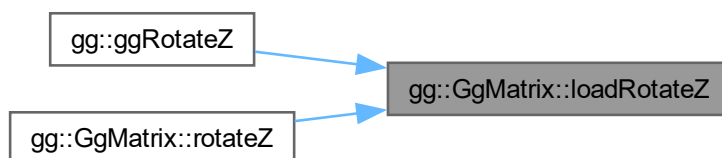
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

[gg.cpp](#) の 2828 行目に定義があります。

被呼び出し関係図:



8.17.3.24 loadScale() [1/3]

```
GgMatrix & gg::GgMatrix::loadScale (  
    const GgVector & s) [inline]
```

拡大縮小の変換行列を格納する.

引数

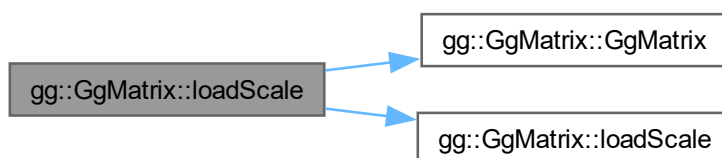
s	拡大率の GgVector 型の変数.
---	-------------------------------------

戻り値

設定した変換行列.

[gg.h](#) の 2530 行目に定義があります。

呼び出し関係図:



8.17.3.25 loadScale() [2/3]

```
GgMatrix & gg::GgMatrix::loadScale (
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を格納する.

引数

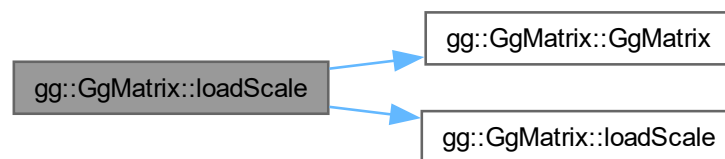
s	拡大率の GLfloat 型の配列 (x, y, z).
----------	------------------------------

戻り値

設定した変換行列.

[gg.h](#) の 2519 行目に定義があります。

呼び出し関係図:



8.17.3.26 loadScale() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadScale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

拡大縮小の変換行列を格納する.

引数

x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	w 拡大率のスケールファクタ (= 1.0f).

8.17.3.28 loadTranslate() [2/3]

```
GgMatrix & gg::GgMatrix::loadTranslate (
    const GLfloat * t) [inline]
```

平行移動の変換行列を格納する.

引数

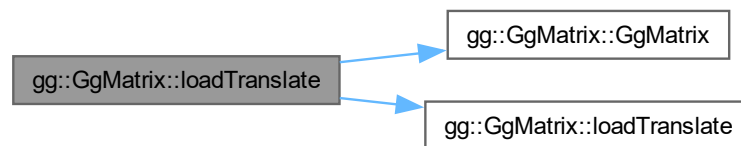
<i>t</i>	移動量の GLfloat 型の配列 (x, y, z).
----------	------------------------------

戻り値

設定した変換行列.

gg.h の 2486 行目に定義があります。

呼び出し関係図:



8.17.3.29 loadTranslate() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

平行移動の変換行列を格納する.

引数

<i>x</i>	x 方向の移動量.
<i>y</i>	y 方向の移動量.
<i>z</i>	z 方向の移動量.
<i>w</i>	w 移動量のスケールファクタ (= 1.0f).

8.17.3.31 loadTranspose() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadTranspose (
    const GLfloat * a)
```

転置行列を格納する.

引数

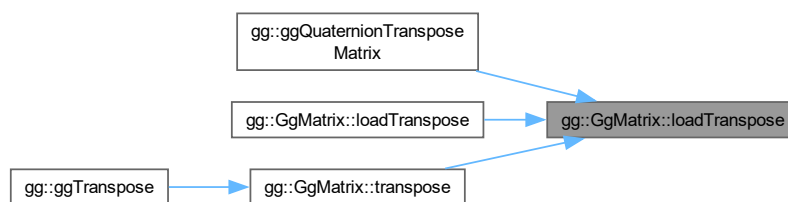
a	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した **a** の転置行列.

[gg.cpp](#) の 2889 行目に定義があります。

被呼び出し関係図:



8.17.3.32 lookat() [1/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

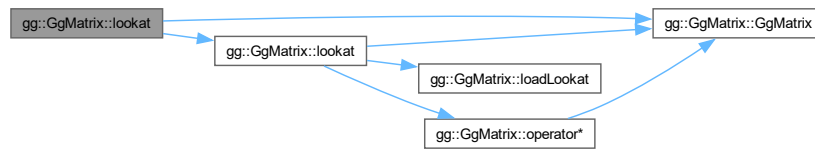
e	視点の位置を格納した GgVector 型の変数.
t	目標点の位置を格納した GgVector 型の変数.
u	上方向のベクトルを格納した GgVector 型の変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2975 行目に定義があります。

呼び出し関係図:



8.17.3.33 lookat() [2/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

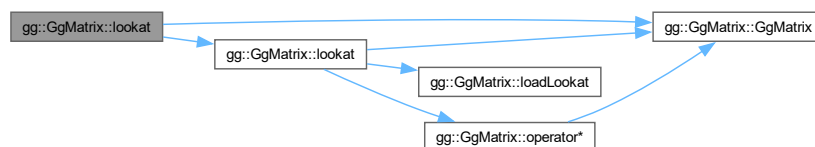
<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2962 行目に定義があります。

呼び出し関係図:



8.17.3.34 `lookat()` [3/3]

```
GgMatrix gg::GgMatrix::lookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz) const [inline]
```

ビュー変換を乗じた結果を返す。

引数

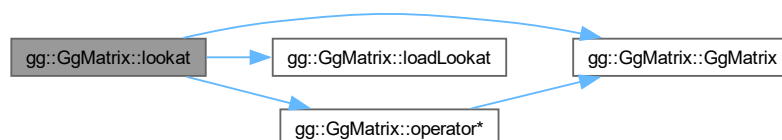
<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

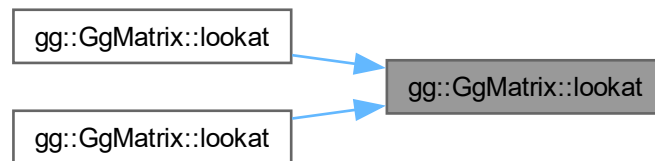
ビュー変換行列を乗じた変換行列。

[gg.h](#) の 2944 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.35 normal()

```
GgMatrix gg::GgMatrix::normal () const [inline]
```

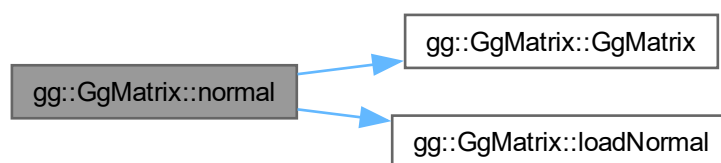
法線変換行列を返す.

戻り値

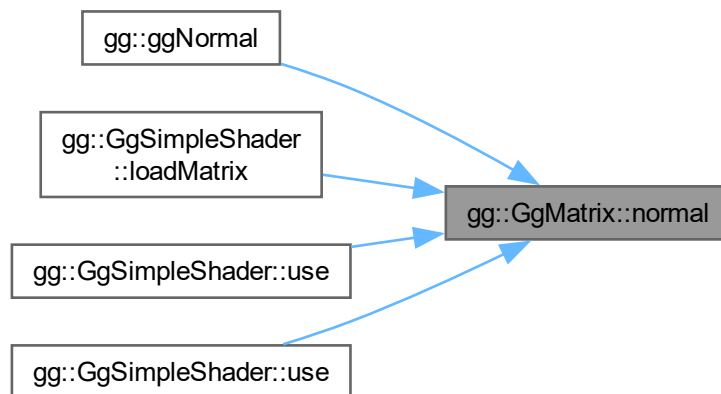
法線変換行列.

[gg.h](#) の [3067](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.36 operator*() [1/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を乗算した値を返す.

引数

<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

変換行列に *m* を乗じた `GgMatrix` 型の変換行列.

`gg.h` の 2388 行目に定義があります。

呼び出し関係図:



8.17.3.37 operator*() [2/3]

```
GgVector gg::GgMatrix::operator* (  
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

v	元のベクトルの GgVector 型の変数.
----------	-------------------------------

戻り値

v 変換結果の **GgVector** 型のベクトル.

gg.h の 3123 行目に定義があります。

8.17.3.38 operator*() [3/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を乗算した値を返す.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	---

戻り値

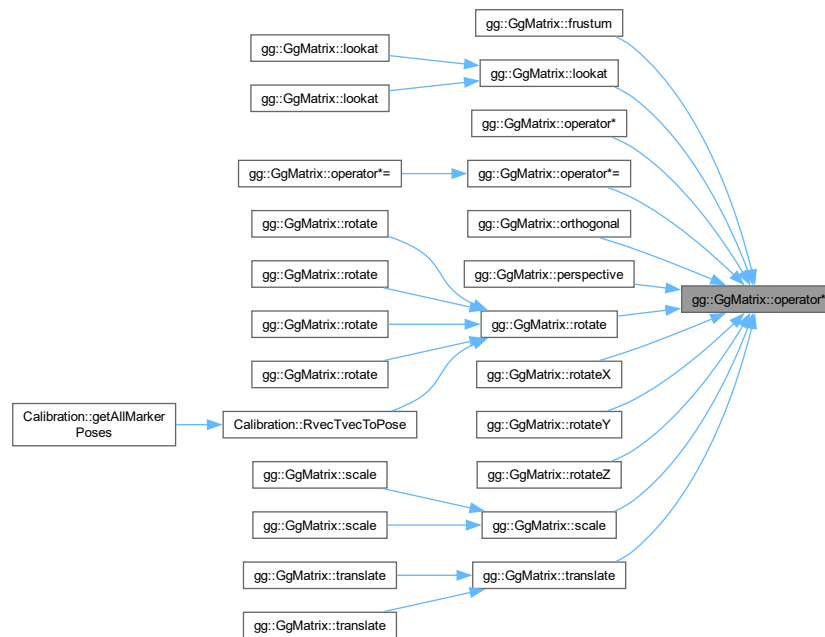
変換行列に **a** を乗じた **GgMatrix** 型の変換行列.

gg.h の 2375 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.39 operator*=() [1/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を乗算した結果を格納する.

引数

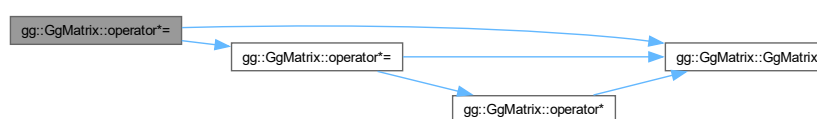
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を掛けた変換行列の参照.

gg.h の 2411 行目に定義があります。

呼び出し関係図:



8.17.3.40 operator*=() [2/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を乗算した結果を格納する.

引数

a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を掛けた変換行列の参照.

gg.h の 2399 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.41 operator+() [1/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を加算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列に *m* を加えた GgMatrix 型の変換行列.

gg.h の 2294 行目に定義があります。

呼び出し関係図:



8.17.3.42 operator+() [2/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を加算した値を返す.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列に *a* を加えた GgMatrix 型の変換行列.

gg.h の 2281 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.43 operator+=() [1/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を加算した結果を格納する.

引数

<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

m を加えた変換行列の参照.

`gg.h` の 2317 行目に定義があります。

呼び出し関係図:



8.17.3.44 operator+=() [2/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を加算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を加えた変換行列の参照.

gg.h の 2305 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.45 operator-() [1/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GgMatrix & m) const [inline]
```

変換行列から別の変換行列を減算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列から `m` を引いた `GgMatrix` 型の変換行列.

`gg.h` の 2341 行目に定義があります。

呼び出し関係図:



8.17.3.46 operator-() [2/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GLfloat * a) const [inline]
```

変換行列から配列に格納した変換行列を減算した結果を格納する.

引数

<code>a</code>	変換行列を格納した <code>GLfloat</code> 型の 16 要素の配列変数.
----------------	---

戻り値

変換行列から `a` を引いた `GgMatrix` 型の変換行列.

`gg.h` の 2328 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.47 operator-=() [1/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を減算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を引いた変換行列の参照.

gg.h の 2364 行目に定義があります。

呼び出し関係図:



8.17.3.48 operator-=() [2/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

`a` を引いた変換行列の参照.

[gg.h](#) の 2352 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.49 operator/() [1/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GgMatrix & m) const [inline]
```

変換行列を変換行列で除算した値を返す.

引数

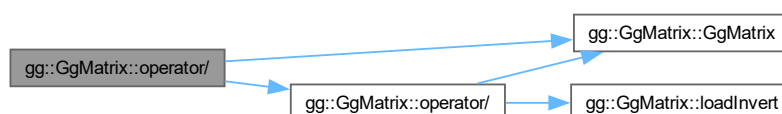
<i>m</i>	GgMatrix 型の変換行列.
----------	----------------------------------

戻り値

変換行列を `m` で割った [GgMatrix](#) 型の変換行列.

[gg.h](#) の 2436 行目に定義があります。

呼び出し関係図:



8.17.3.50 operator/() [2/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GLfloat * a) const [inline]
```

変換行列を配列に格納した変換行列で除算した値を返す.

引数

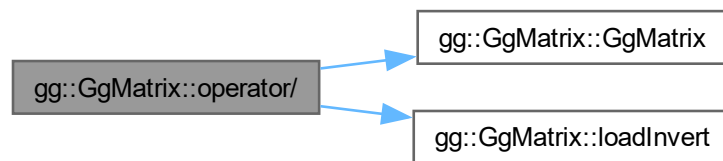
a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
---	----------------------------------

戻り値

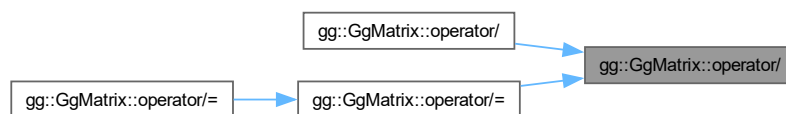
変換行列を a で割った GgMatrix 型の変換行列.

gg.h の 2422 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.51 operator/=() [1/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GgMatrix & m) [inline]
```

変換行列を別の変換行列で除算した結果を格納する.

引数

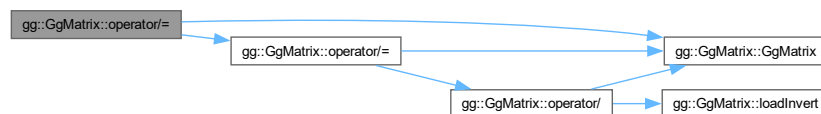
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m で割った変換行列の参照.

gg.h の 2459 行目に定義があります。

呼び出し関係図:



8.17.3.52 operator/=() [2/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GLfloat * a) [inline]
```

変換行列を配列に格納した変換行列で除算した結果を格納する.

引数

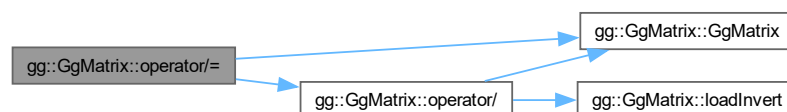
<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a で割った変換行列の参照.

gg.h の 2447 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.53 operator=() [1/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GgMatrix & m) [default]
```

代入演算子. 呼び出し関係図:



8.17.3.54 operator=() [2/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GLfloat * a) [inline]
```

配列変数の値を格納する.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

戻り値

a を代入したこのオブジェクトの参照.

[gg.h](#) の 2269 行目に定義があります。

呼び出し関係図:



8.17.3.55 operator=() [3/3]

```
GgMatrix & gg::GgMatrix::operator= (
    GgMatrix && m) [default]
```

ムーブ代入演算子. 呼び出し関係図:



8.17.3.56 orthogonal()

```
GgMatrix gg::GgMatrix::orthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

直交投影変換を乗じた結果を返す.

引数

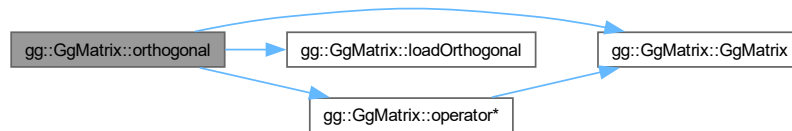
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

直交投影変換行列を乗じた変換行列.

gg.h の 2991 行目に定義があります。

呼び出し関係図:



8.17.3.57 perspective()

```

GgMatrix gg::GgMatrix::perspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) const [inline]
  
```

画角を指定して透視投影変換を乗じた結果を返す.

引数

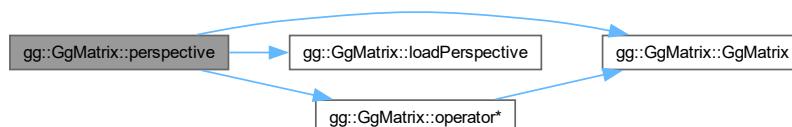
<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

gg.h の 3031 行目に定義があります。

呼び出し関係図:



8.17.3.58 projection() [1/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GgVector 型の変数.
v	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3112 行目に定義があります。

8.17.3.59 projection() [2/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GgVector 型の変数.
v	元のベクトルの GLfloat 型の 4 要素の配列変数.

[gg.h](#) の 3101 行目に定義があります。

8.17.3.60 projection() [3/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GLfloat 型の 4 要素の配列変数.
v	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3090 行目に定義があります。

8.17.3.61 projection() [4/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<i>c</i>	変換結果を格納する GLfloat 型の 4 要素の配列変数.
<i>v</i>	元のベクトルの GLfloat 型の 4 要素の配列変数.

gg.h の 3079 行目に定義があります。

8.17.3.62 rotate() [1/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<i>r</i>	回転軸の方向ベクトルと回転角を格納した GgVector 型の変数).
----------	-------------------------------------

戻り値

(r[0], r[1], r[2]) を軸にさらに r[3] 回転した変換行列.

gg.h の 2925 行目に定義があります。

呼び出し関係図:



8.17.3.63 rotate() [2/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

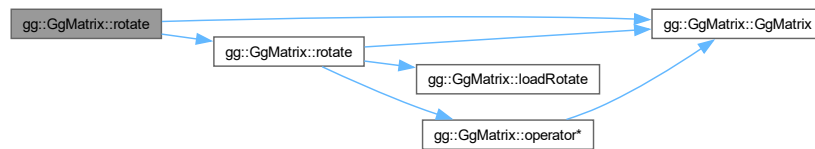
<i>r</i>	回転軸の方向ベクトルを格納した GgVector 型の変数.
<i>a</i>	回転角.

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに a 回転した変換行列.

gg.h の 2903 行目に定義があります。

呼び出し関係図:



8.17.3.64 rotate() [3/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

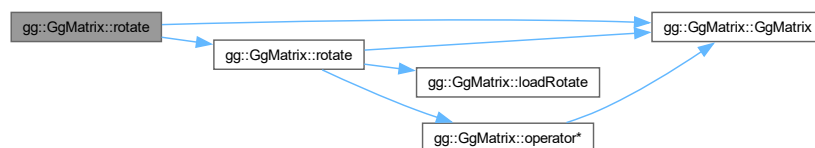
r	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数 (x, y, z, a).
-----	--

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸にさらに $r[3]$ 回転した変換行列.

gg.h の 2914 行目に定義があります。

呼び出し関係図:



8.17.3.65 rotate() [4/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

<i>r</i>	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
<i>a</i>	回転角.

戻り値

(*r*[0], *r*[1], *r*[2]) を軸にさらに *a* 回転した変換行列.

gg.h の 2891 行目に定義があります。

呼び出し関係図:



8.17.3.66 rotate() [5/5]

```

GgMatrix gg::GgMatrix::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]

```

(x, y, z) 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

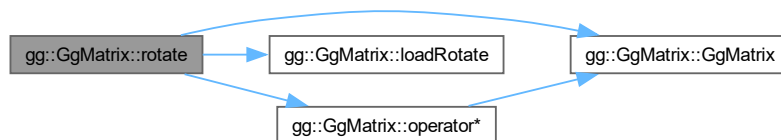
<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

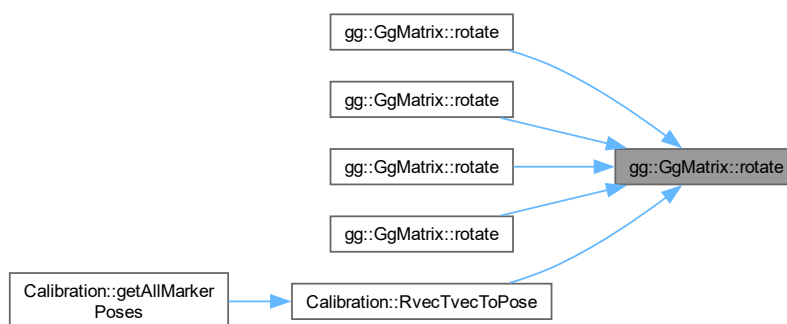
(x, y, z) を軸にさらに *a* 回転した変換行列.

gg.h の 2878 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.67 rotateX()

```
GgMatrix gg::GgMatrix::rotateX (
    GLfloat a) const [inline]
```

x 軸中心の回転変換を乗じた結果を返す.

引数

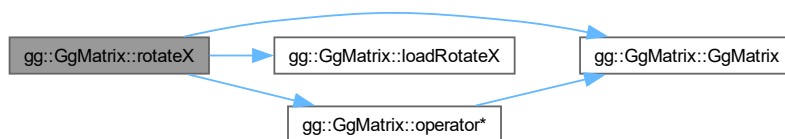
<i>a</i>	回転角.
----------	------

戻り値

x 軸中心にさらに *a* 回転した変換行列.

[gg.h](#) の 2839 行目に定義があります。

呼び出し関係図:



8.17.3.68 rotateY()

```
GgMatrix gg::GgMatrix::rotateY (
    GLfloat a) const [inline]
```

y 軸中心の回転変換を乗じた結果を返す.

引数

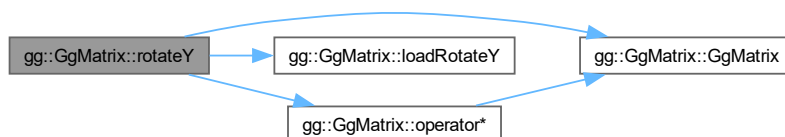
<i>a</i>	回転角.
----------	------

戻り値

y 軸中心にさらに *a* 回転した変換行列.

[gg.h](#) の 2851 行目に定義があります。

呼び出し関係図:



8.17.3.69 rotateZ()

```
GgMatrix gg::GgMatrix::rotateZ (
    GLfloat a) const [inline]
```

z 軸中心の回転変換を乗じた結果を返す.

引数

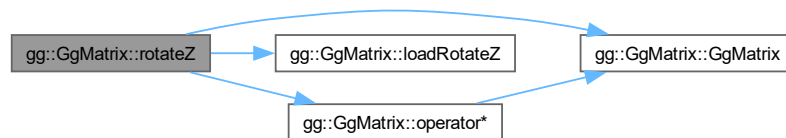
<code>a</code>	回転角.
----------------	------

戻り値

z 軸中心にさらに a 回転した変換行列.

`gg.h` の 2863 行目に定義があります。

呼び出し関係図:



8.17.3.70 scale() [1/3]

```
GgMatrix gg::GgMatrix::scale (
    const GgVector & s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

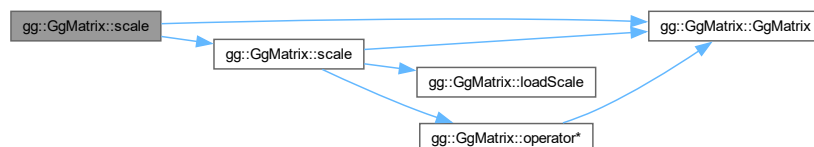
<code>s</code>	拡大率の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

拡大縮小した結果の変換行列.

`gg.h` の 2828 行目に定義があります。

呼び出し関係図:



8.17.3.71 scale() [2/3]

```
GgMatrix gg::GgMatrix::scale (  
    const GLfloat * s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

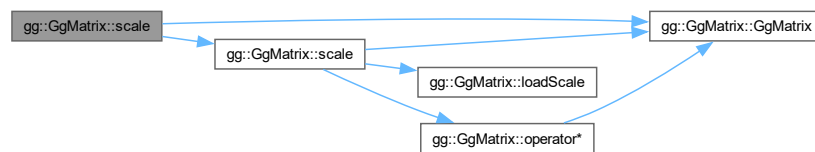
s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小した結果の変換行列.

gg.h の 2817 行目に定義があります。

呼び出し関係図:



8.17.3.72 scale() [3/3]

```

GgMatrix gg::GgMatrix::scale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]

```

拡大縮小変換を乗じた結果を返す.

引数

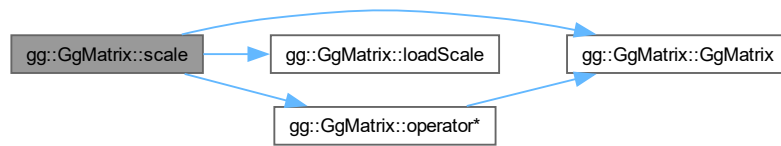
x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	w 移動量のスケールファクタ (= 1.0f).

戻り値

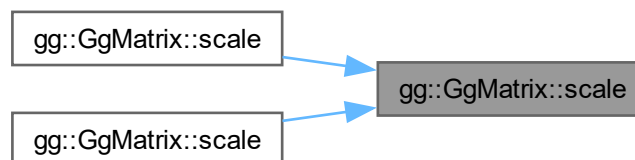
拡大縮小した結果の変換行列.

gg.h の 2805 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.73 translate() [1/3]

```
GgMatrix gg::GgMatrix::translate (
    const GgVector & t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

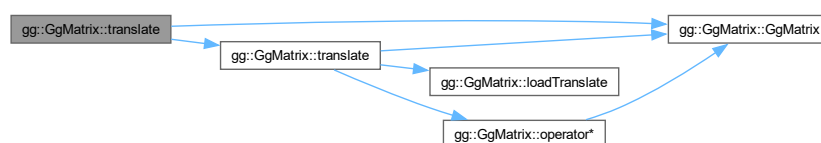
<i>t</i>	移動量の GgVector 型の変数.
----------	-------------------------------------

戻り値

平行移動した結果の変換行列.

[gg.h](#) の 2791 行目に定義があります。

呼び出し関係図:



8.17.3.74 translate() [2/3]

```
GgMatrix gg::GgMatrix::translate (
    const GLfloat * t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

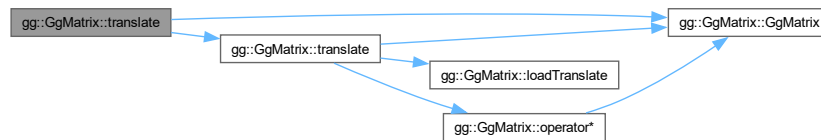
<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

平行移動した結果の変換行列.

gg.h の 2780 行目に定義があります。

呼び出し関係図:



8.17.3.75 translate() [3/3]

```
GgMatrix gg::GgMatrix::translate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

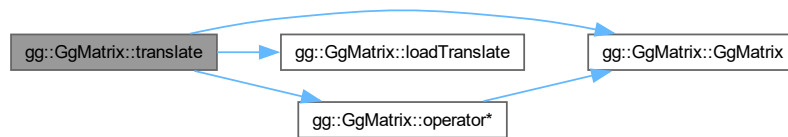
<i>x</i>	<i>x</i> 方向の移動量.
<i>y</i>	<i>y</i> 方向の移動量.
<i>z</i>	<i>z</i> 方向の移動量.
<i>w</i>	<i>w</i> 移動量のスケールファクタ (= 1.0f).

戻り値

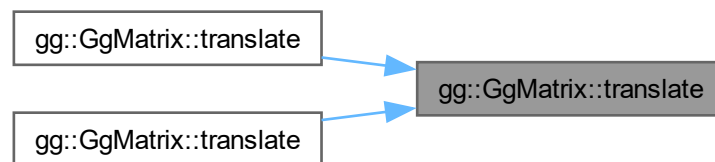
平行移動した結果の変換行列.

gg.h の 2768 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.17.3.76 transpose()

```
GgMatrix gg::GgMatrix::transpose () const [inline]
```

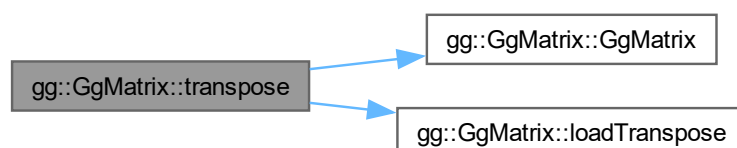
転置行列を返す.

戻り値

転置行列.

gg.h の 3045 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.18 gg::GgNormalTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgNormalTexture](#) ()
- [GgNormalTexture](#) (const GLubyte *image, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- [GgNormalTexture](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- virtual [~GgNormalTexture](#) ()
- void [load](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- void [load](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const [GgTexture](#) * [get](#) () const
- GLuint [getTexture](#) () const
- GLsizei [getWidth](#) () const
- GLsizei [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- void [bind](#) () const
- void [unbind](#) () const

8.18.1 詳解

法線マップ.

覚え書き

高さマップ（グレイスケール画像）を読み込んで法線マップのテクスチャを作成する.

[gg.h](#) の 5627 行目に定義があります。

8.18.2 構築子と解体子

8.18.2.1 GgNormalTexture() [1/3]

```
gg::GgNormalTexture::GgNormalTexture () [inline]
```

コンストラクタ.

[gg.h](#) の 5637 行目に定義があります。

8.18.2.2 GgNormalTexture() [2/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const GLubyte * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

メモリ上のデータから法線マップのテクスチャを作成するコンストラクタ.

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

[gg.h](#) の 5651 行目に定義があります。

呼び出し関係図:



8.18.2.3 GgNormalTexture() [3/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

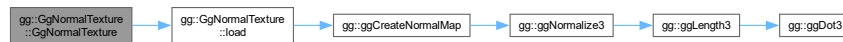
ファイルからデータを読み込んで法線マップのテクスチャを作成するコンストラクタ.

引数

<i>name</i>	画像ファイル名.
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

[gg.h](#) の 5671 行目に定義があります。

呼び出し関係図:



8.18.2.4 ~GgNormalTexture()

```
virtual gg::GgNormalTexture::~~GgNormalTexture () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 5684 行目に定義があります。

8.18.3 関数詳解

8.18.3.1 bind()

```
void gg::GgNormalTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

[gg.h](#) の 5813 行目に定義があります。

8.18.3.2 get()

```
const GgTexture * gg::GgNormalTexture::get () const [inline]
```

テクスチャオブジェクトを取り出す.

戻り値

[GgTexture](#) オブジェクトのポインタ.

[gg.h](#) の 5755 行目に定義があります。

8.18.3.3 getHeight()

```
GLsizei gg::GgNormalTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す。

戻り値

テクスチャの縦の画素数。

gg.h の 5785 行目に定義があります。

8.18.3.4 getSize() [1/2]

```
const GLsizei * gg::GgNormalTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す。

戻り値

テクスチャのサイズを格納した配列へのポインタ。

gg.h の 5805 行目に定義があります。

8.18.3.5 getSize() [2/2]

```
void gg::GgNormalTexture::getSize (
    GLsizei * size) const [inline]
```

使用しているテクスチャのサイズを取り出す。

引数

size	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数。
------	--------------------------------------

gg.h の 5795 行目に定義があります。

8.18.3.6 getTexture()

```
GLuint gg::GgNormalTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る。

戻り値

テクスチャ名。

gg.h の 5765 行目に定義があります。

8.18.3.7 getWidth()

```
GLsizei gg::GgNormalTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す。

戻り値

テクスチャの横の画素数。

[gg.h](#) の 5775 行目に定義があります。

8.18.3.8 load() [1/2]

```
void gg::GgNormalTexture::load (  
    const GLubyte * hmap,  
    GLsizei width,  
    GLsizei height,  
    GLenum format = GL_RED,  
    GLfloat nz = 1.0f,  
    GLenum internal = GL_RGBA) [inline]
```

メモリ上のデータから法線マップのテクスチャを作成する。

引数

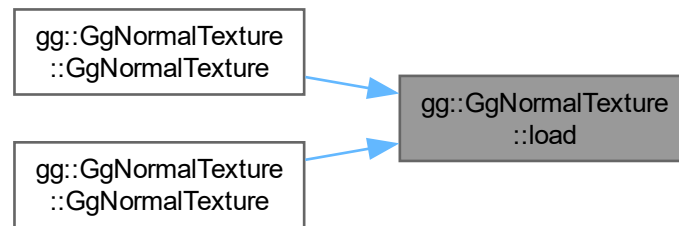
<i>hmap</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

[gg.h](#) の 5698 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.18.3.9 load() [2/2]

```
void gg::GgNormalTexture::load (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA)
```

TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する.

引数

<i>name</i>	画像ファイル名 (1 チャンネルの TGA 画像).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.cpp の 4110 行目に定義があります。

呼び出し関係図:



8.18.3.10 operator bool()

```
gg::GgNormalTexture::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる.

戻り値

オブジェクトが有効なら true.

gg.h の 5735 行目に定義があります。

8.18.3.11 operator"!()

```
bool gg::GgNormalTexture::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する.

戻り値

オブジェクトが有効なら **false**, 無効なら **true**.

[gg.h](#) の 5745 行目に定義があります。

8.18.3.12 unbind()

```
void gg::GgNormalTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

[gg.h](#) の 5821 行目に定義があります。

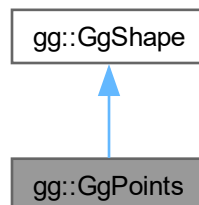
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

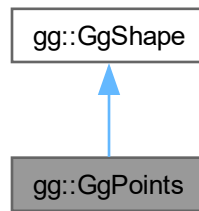
8.19 gg::GgPoints クラス

```
#include <gg.h>
```

gg::GgPoints の継承関係図



gg::GgPoints 連携図



公開メンバ関数

- `GgPoints` (GLenum mode=GL_POINTS)
- `GgPoints` (const `GgVector` *pos, GLsizei countv, GLenum mode=GL_POINTS, GLenum usage=GL_STATIC_DRAW)
- virtual `~GgPoints` ()
- `operator bool` () const noexcept
- `bool operator!` () const noexcept
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `send` (const `GgVector` *pos, GLint first=0, GLsizei count=0) const
- void `load` (const `GgVector` *pos, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void `draw` (GLint first=0, GLsizei count=0) const

基底クラス `gg::GgShape` に属する継承公開メンバ関数

- `GgShape` (GLenum mode=0)
- virtual `~GgShape` ()
- const GLuint & `get` () const
- void `setMode` (GLenum mode)
- const GLenum & `getMode` () const

8.19.1 詳解

点.

`gg.h` の 6628 行目に定義があります。

8.19.2 構築子と解体子

8.19.2.1 GgPoints() [1/2]

```
gg::GgPoints::GgPoints (
    GLenum mode = GL_POINTS) [inline]
```

コンストラクタ.

[gg.h](#) の 6639 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.19.2.2 GgPoints() [2/2]

```
gg::GgPoints::GgPoints (
    const GgVector * pos,
    GLsizei countv,
    GLenum mode = GL_POINTS,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

コンストラクタ.

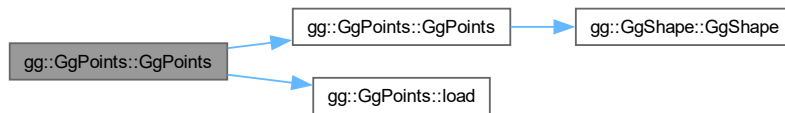
引数

<i>pos</i>	この図形の頂点の位置のデータの配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.

<i>usage</i>	バッファオブジェクトの使い方.
--------------	-----------------

gg.h の 6652 行目に定義があります。

呼び出し関係図:



8.19.2.3 ~GgPoints()

```
virtual gg::GgPoints::~~GgPoints () [inline], [virtual]
```

デストラクタ.

gg.h の 6666 行目に定義があります。

8.19.3 関数詳解

8.19.3.1 draw()

```
void gg::GgPoints::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

点の描画.

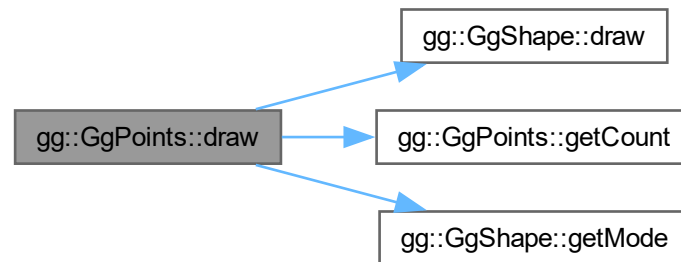
引数

<i>first</i>	描画を開始する最初の点の番号.
<i>count</i>	描画する点の数, 0 なら全部の点を描く.

gg::GgShape を再実装しています。

gg.cpp の 5259 行目に定義があります。

呼び出し関係図:



8.19.3.2 getBuffer()

```
const GLuint & gg::GgPoints::getBuffer () const [inline]
```

頂点の位置データを格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の頂点の位置データを格納した頂点バッファオブジェクト名.

[gg.h](#) の 6705 行目に定義があります。

8.19.3.3 getCount()

```
const GLsizei & gg::GgPoints::getCount () const [inline]
```

データの数を取り出す.

戻り値

この図形の頂点の位置データの数 (頂点数).

[gg.h](#) の 6695 行目に定義があります。

被呼び出し関係図:



8.19.3.4 load()

```
void gg::GgPoints::load (
    const GgVector * pos,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点の位置データを格納する。

引数

<i>pos</i>	頂点の位置データが格納されている領域の先頭のポインタ。
<i>count</i>	頂点のデータの数 (頂点数)。
<i>usage</i>	バッファオブジェクトの使い方。

gg.cpp の 5246 行目に定義があります。

被呼び出し関係図:



8.19.3.5 operator bool()

```
gg::GgPoints::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

バッファが有効かどうか調べる。

戻り値

バッファが有効なら true

gg::GgShape を再実装しています。

gg.h の 6675 行目に定義があります。

8.19.3.6 operator"!()

```
bool gg::GgPoints::operator! () const [inline], [virtual], [noexcept]
```

バッファが有効かどうかの結果を反転する。

戻り値

バッファが有効なら false, 無効なら true.

gg::GgShape を再実装しています。

gg.h の 6685 行目に定義があります。

8.19.3.7 send()

```
void gg::GgPoints::send (
    const GgVector * pos,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点の位置データを転送する.

引数

<i>pos</i>	転送元の頂点の位置データが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6717 行目に定義があります。

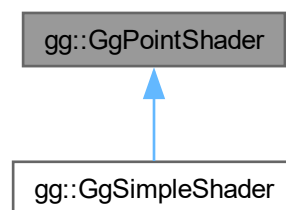
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.20 gg::GgPointShader クラス

```
#include <gg.h>
```

gg::GgPointShader の継承関係図



公開メンバ関数

- [GgPointShader](#) ()
- [GgPointShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgPointShader](#) (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual [~GgPointShader](#) ()
- bool [load](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool [load](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void [loadProjectionMatrix](#) (const GLfloat *mp) const
- virtual void [loadProjectionMatrix](#) (const [GgMatrix](#) &mp) const
- virtual void [loadModelviewMatrix](#) (const GLfloat *mv) const
- virtual void [loadModelviewMatrix](#) (const [GgMatrix](#) &mv) const
- virtual void [loadMatrix](#) (const GLfloat *mp, const GLfloat *mv) const
- virtual void [loadMatrix](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- virtual void [use](#) () const
- void [use](#) (const GLfloat *mp) const
- void [use](#) (const [GgMatrix](#) &mp) const
- void [use](#) (const GLfloat *mp, const GLfloat *mv) const
- void [use](#) (const [GgMatrix](#) &mp, const [GgMatrix](#) &mv) const
- void [unuse](#) () const
- GLuint [get](#) () const

8.20.1 詳解

点のシェーダ.

[gg.h](#) の 7298 行目に定義があります。

8.20.2 構築子と解体子

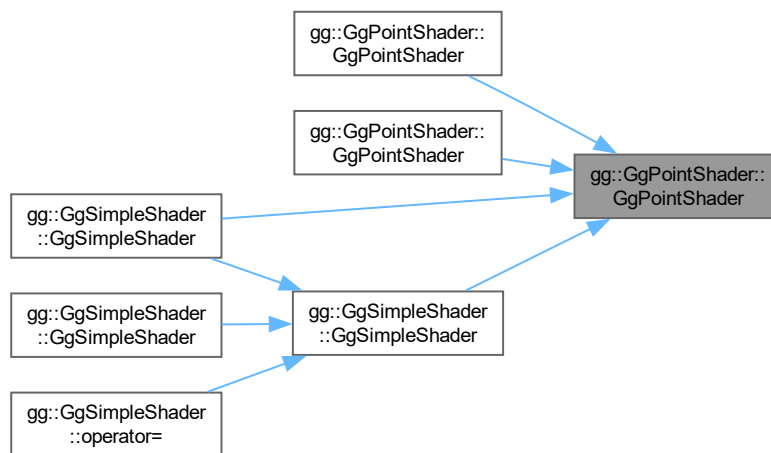
8.20.2.1 GgPointShader() [1/3]

```
gg::GgPointShader::GgPointShader () [inline]
```

コンストラクタ.

[gg.h](#) の 7314 行目に定義があります。

被呼び出し関係図:



8.20.2.2 GgPointShader() [2/3]

```

gg::GgPointShader::GgPointShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]

```

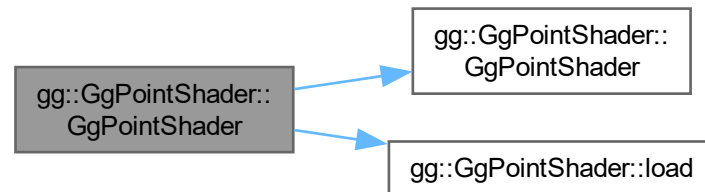
コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7329 行目に定義があります。

呼び出し関係図:



8.20.2.3 GgPointShader() [3/3]

```

gg::GgPointShader::GgPointShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

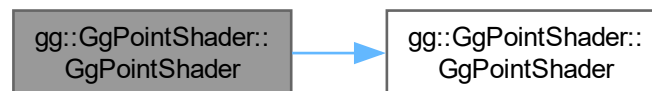
コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

[gg.h](#) の 7348 行目に定義があります。

呼び出し関係図:



8.20.2.4 ~GgPointShader()

```

virtual gg::GgPointShader::~GgPointShader () [inline], [virtual]
  
```

デストラクタ.

[gg.h](#) の 7360 行目に定義があります。

8.20.3 関数詳解

8.20.3.1 get()

```
GLuint gg::GgPointShader::get () const [inline]
```

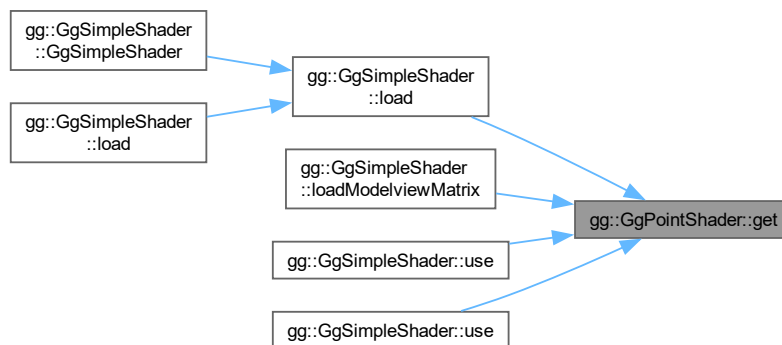
シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

[gg.h](#) の 7544 行目に定義があります。

被呼び出し関係図:



8.20.3.2 load() [1/2]

```
bool gg::GgPointShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら true.

gg.h の 7407 行目に定義があります。

呼び出し関係図:



8.20.3.3 load() [2/2]

```

bool gg::GgPointShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

シェーダのソースファイルを読み込む。

引数

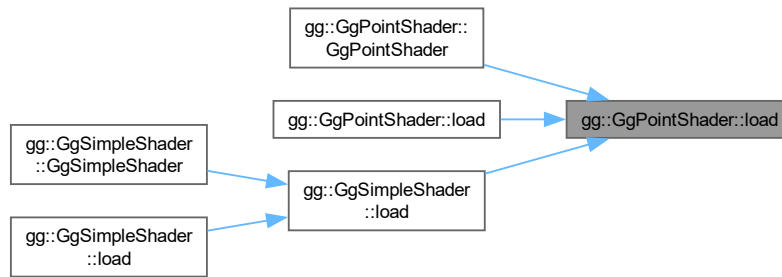
<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

戻り値

プログラムオブジェクトが作成できれば true.

gg.h の 7374 行目に定義があります。

被呼び出し関係図:



8.20.3.4 loadMatrix() [1/2]

```
virtual void gg::GgPointShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する.

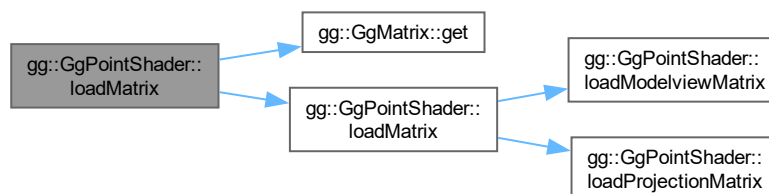
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

[gg::GgSimpleShader](#)で再実装されています。

[gg.h](#) の 7474 行目に定義があります。

呼び出し関係図:



8.20.3.5 loadMatrix() [2/2]

```
virtual void gg::GgPointShader::loadMatrix (  
    const GLfloat * mp,  
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する.

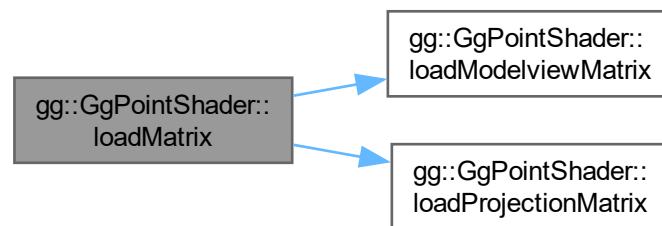
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

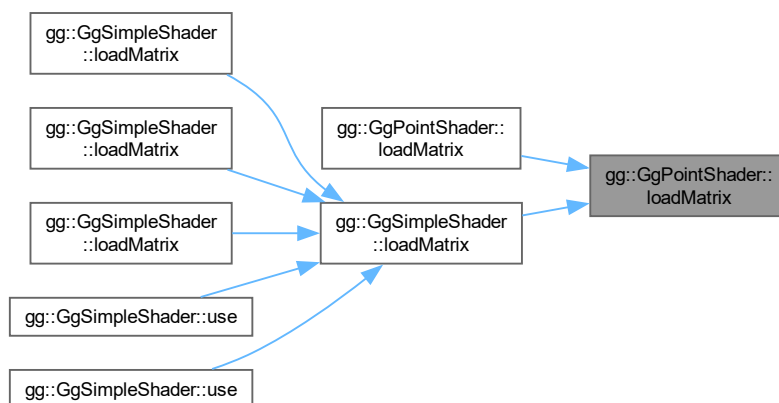
`gg::GgSimpleShader`で再実装されています。

`gg.h` の 7462 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.20.3.6 loadModelviewMatrix() [1/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する.

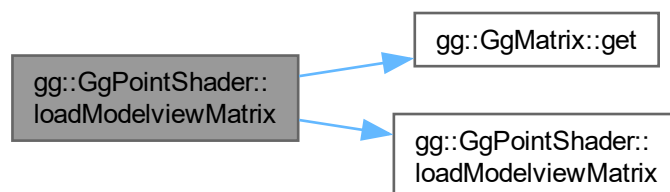
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	------------------------

gg::GgSimpleShaderで再実装されています。

gg.h の 7451 行目に定義があります。

呼び出し関係図:



8.20.3.7 loadModelviewMatrix() [2/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

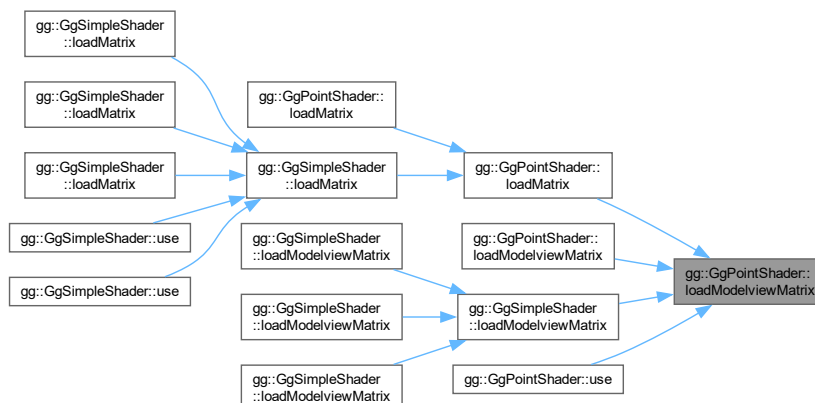
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgSimpleShaderで再実装されています。

gg.h の 7441 行目に定義があります。

被呼び出し関係図:



8.20.3.8 loadProjectionMatrix() [1/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (  
    const GgMatrix & mp) const [inline], [virtual]
```

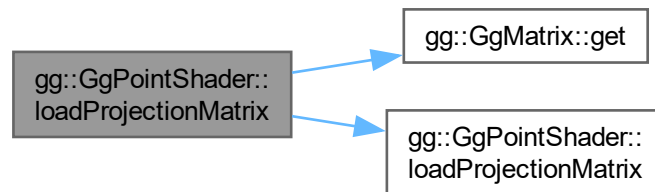
投影変換行列を設定する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
-----------	--------------------

gg.h の 7431 行目に定義があります。

呼び出し関係図:



8.20.3.9 loadProjectionMatrix() [2/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (  
    const GLfloat * mp) const [inline], [virtual]
```

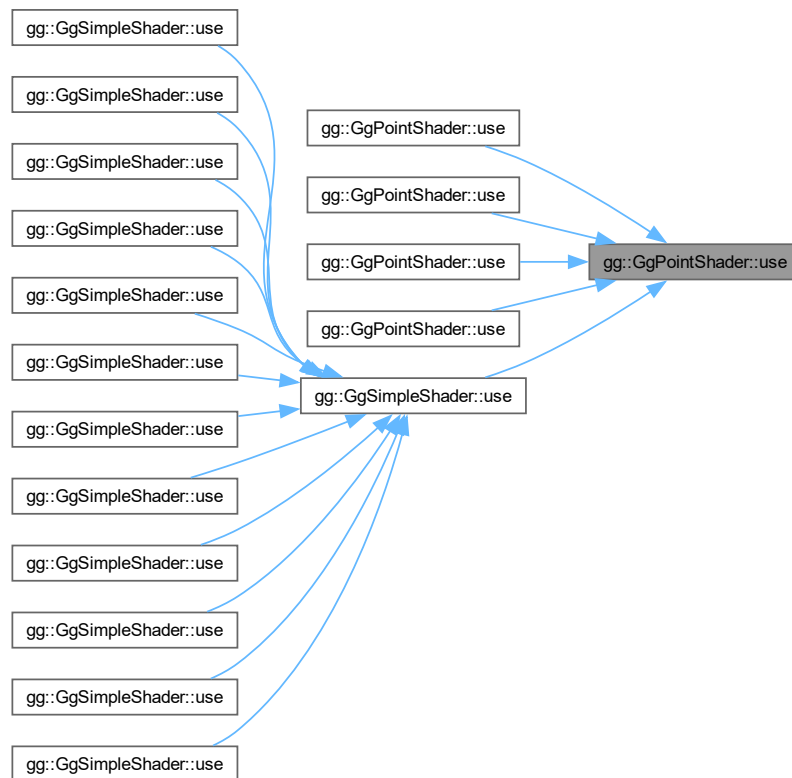
投影変換行列を設定する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

gg.h の 7421 行目に定義があります。

被呼び出し関係図:



8.20.3.12 use() [2/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp) const [inline]
```

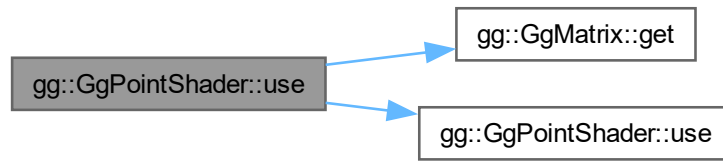
投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
-----------	--------------------

gg.h の 7503 行目に定義があります。

呼び出し関係図:



8.20.3.13 use() [3/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

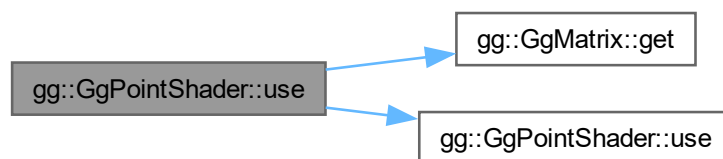
投影変換行列とモデルビューを設定してシェードプログラムの使用を開始する.

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

[gg.h](#) の 7526 行目に定義があります。

呼び出し関係図:



8.20.3.14 use() [4/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp) const [inline]
```

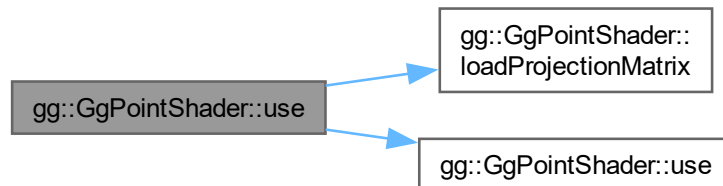
投影変換行列を設定してシェードプログラムの使用を開始する.

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

[gg.h](#) の 7492 行目に定義があります。

呼び出し関係図:



8.20.3.15 use() [5/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

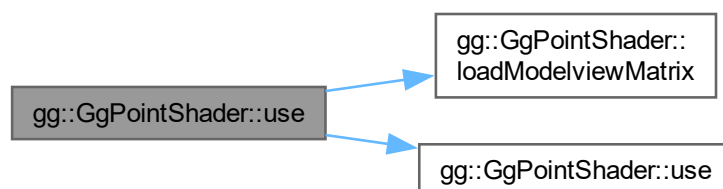
投影変換行列とモデルビューを設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

[gg.h](#) の 7514 行目に定義があります。

呼び出し関係図:



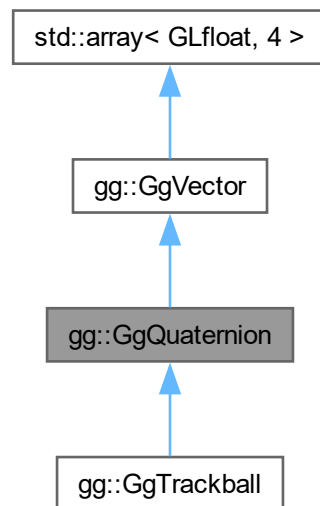
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

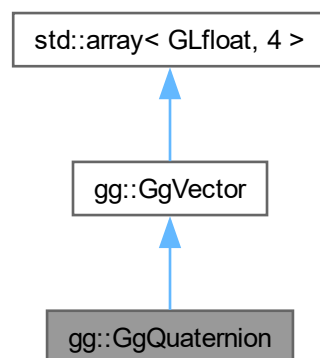
8.21 gg::GgQuaternion クラス

```
#include <gg.h>
```

gg::GgQuaternion の継承関係図



gg::GgQuaternion 連携図



公開メンバ関数

- [GgQuaternion](#) ()=default

- constexpr [GgQuaternion](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- constexpr [GgQuaternion](#) (GLfloat c)
- [GgQuaternion](#) (const GLfloat *a)
- [GgQuaternion](#) (const [GgVector](#) &v)
- [GgQuaternion](#) (const [GgQuaternion](#) &q)=default
- [GgQuaternion](#) ([GgQuaternion](#) &&q)=default
- [~GgQuaternion](#) ()=default
- [GgQuaternion](#) & operator= (const [GgQuaternion](#) &q)=default
- [GgQuaternion](#) & operator= ([GgQuaternion](#) &&q)=default
- GLfloat [norm](#) () const
- [GgQuaternion](#) & [loadAdd](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadAdd](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadAdd](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadAdd](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadSubtract](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadSubtract](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadMultiply](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadMultiply](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadDivide](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadDivide](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadDivide](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadDivide](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) add (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) add (const GLfloat *a) const
- [GgQuaternion](#) add (const [GgVector](#) &v) const
- [GgQuaternion](#) add (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) subtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) subtract (const GLfloat *a) const
- [GgQuaternion](#) subtract (const [GgVector](#) &v) const
- [GgQuaternion](#) subtract (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) multiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) multiply (const GLfloat *a) const
- [GgQuaternion](#) multiply (const [GgVector](#) &v) const
- [GgQuaternion](#) multiply (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) divide (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) divide (const GLfloat *a) const
- [GgQuaternion](#) divide (const [GgVector](#) &v) const
- [GgQuaternion](#) divide (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) & operator= (const GLfloat *a)
- [GgQuaternion](#) & operator= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator+= (const GLfloat *a)
- [GgQuaternion](#) & operator+= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator+= (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & operator-= (const GLfloat *a)
- [GgQuaternion](#) & operator-= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator-= (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & operator*= (const GLfloat *a)
- [GgQuaternion](#) & operator*= (const [GgVector](#) &v)
- [GgQuaternion](#) & operator*= (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & operator/= (const GLfloat *a)
- [GgQuaternion](#) & operator/= (const [GgVector](#) &v)

- GgQuaternion & operator/= (const GgQuaternion &q)
- GgQuaternion operator+ (const GLfloat *a) const
- GgQuaternion operator+ (const GgVector &v) const
- GgQuaternion operator+ (const GgQuaternion &q) const
- GgQuaternion operator- (const GLfloat *a) const
- GgQuaternion operator- (const GgVector &v) const
- GgQuaternion operator- (const GgQuaternion &q) const
- GgQuaternion operator* (const GLfloat *a) const
- GgQuaternion operator* (const GgVector &v) const
- GgQuaternion operator* (const GgQuaternion &q) const
- GgQuaternion operator/ (const GLfloat *a) const
- GgQuaternion operator/ (const GgVector &v) const
- GgQuaternion operator/ (const GgQuaternion &q) const
- GgQuaternion & loadMatrix (const GLfloat *a)
- GgQuaternion & loadMatrix (const GgMatrix &m)
- GgQuaternion & loadIdentity ()
- GgQuaternion & loadRotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- GgQuaternion & loadRotate (const GLfloat *v, GLfloat a)
- GgQuaternion & loadRotate (const GLfloat *v)
- GgQuaternion & loadRotateX (GLfloat a)
- GgQuaternion & loadRotateY (GLfloat a)
- GgQuaternion & loadRotateZ (GLfloat a)
- GgQuaternion rotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- GgQuaternion rotate (const GLfloat *v, GLfloat a) const
- GgQuaternion rotate (const GLfloat *v) const
- GgQuaternion rotateX (GLfloat a) const
- GgQuaternion rotateY (GLfloat a) const
- GgQuaternion rotateZ (GLfloat a) const
- GgQuaternion & loadEuler (GLfloat heading, GLfloat pitch, GLfloat roll)
- GgQuaternion & loadEuler (const GLfloat *e)
- GgQuaternion euler (GLfloat heading, GLfloat pitch, GLfloat roll) const
- GgQuaternion euler (const GLfloat *e) const
- GgQuaternion euler (const GgVector &e) const
- GgQuaternion & loadSlerp (const GLfloat *a, const GLfloat *b, GLfloat t)
- GgQuaternion & loadSlerp (const GgQuaternion &q, const GgQuaternion &r, GLfloat t)
- GgQuaternion & loadSlerp (const GgQuaternion &q, const GLfloat *a, GLfloat t)
- GgQuaternion & loadSlerp (const GLfloat *a, const GgQuaternion &q, GLfloat t)
- GgQuaternion & loadNormalize (const GLfloat *a)
- GgQuaternion & loadNormalize (const GgQuaternion &q)
- GgQuaternion & loadConjugate (const GLfloat *a)
- GgQuaternion & loadConjugate (const GgQuaternion &q)
- GgQuaternion & loadInvert (const GLfloat *a)
- GgQuaternion & loadInvert (const GgQuaternion &q)
- GgQuaternion slerp (GLfloat *a, GLfloat t) const
- GgQuaternion slerp (const GgQuaternion &q, GLfloat t) const
- GgQuaternion normalize () const
- GgQuaternion conjugate () const
- GgQuaternion invert () const
- void get (GLfloat *a) const
- void getMatrix (GLfloat *a) const
- void getMatrix (GgMatrix &m) const
- GgMatrix getMatrix () const
- void getConjugateMatrix (GLfloat *a) const
- void getConjugateMatrix (GgMatrix &m) const
- GgMatrix getConjugateMatrix () const

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & operator= (const **GgVector** &v)=default
- **GgVector** & operator= (**GgVector** &&v)=default
- constexpr **GgVector** operator+ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator+ (GLfloat c) const noexcept
- constexpr **GgVector** & operator+= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator+= (GLfloat c) noexcept
- constexpr **GgVector** operator- (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator- (GLfloat c) const noexcept
- constexpr **GgVector** & operator-= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator-= (GLfloat c) noexcept
- constexpr **GgVector** operator* (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator* (GLfloat c) const noexcept
- constexpr **GgVector** & operator*= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator*= (GLfloat c) noexcept
- constexpr **GgVector** operator/ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator/ (GLfloat c) const noexcept
- constexpr **GgVector** & operator/= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator/= (GLfloat c) noexcept
- GLfloat dot3 (const **GgVector** &v) const
- GLfloat length3 () const
- GLfloat distance3 (const **GgVector** &v) const
- **GgVector** normalize3 () const
- GLfloat dot4 (const **GgVector** &v) const
- GLfloat length4 () const
- GLfloat distance4 (const **GgVector** &v) const
- **GgVector** normalize4 () const

8.21.1 詳解

四元数.

[gg.h](#) の 3495 行目に定義があります。

8.21.2 構築子と解体子

8.21.2.1 GgQuaternion() [1/7]

```
gg::GgQuaternion::GgQuaternion () [default]
```

コンストラクタ.

8.21.2.2 GgQuaternion() [2/7]

```
gg::GgQuaternion::GgQuaternion (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w) [inline], [constexpr]
```

コンストラクタ.

引数

x	四元数の x 要素.
y	四元数の y 要素.
z	四元数の z 要素.
w	四元数の w 要素.

[gg.h](#) の 3524 行目に定義があります。

呼び出し関係図:



8.21.2.3 GgQuaternion() [3/7]

```
gg::GgQuaternion::GgQuaternion (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

c	GLfloat 型の値.
-----	--------------

[gg.h](#) の 3534 行目に定義があります。

呼び出し関係図:



8.21.2.4 GgQuaternion() [4/7]

```
gg::GgQuaternion::GgQuaternion (
    const GLfloat * a) [inline]
```

コンストラクタ.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

gg.h の 3544 行目に定義があります。

呼び出し関係図:



8.21.2.5 GgQuaternion() [5/7]

```
gg::GgQuaternion::GgQuaternion (  
    const GgVector & v) [inline]
```

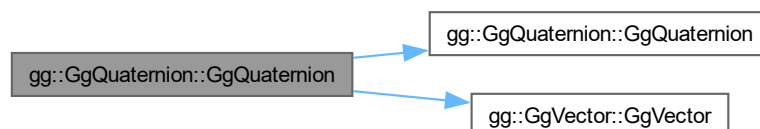
コンストラクタ.

引数

<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

gg.h の 3554 行目に定義があります。

呼び出し関係図:



8.21.2.6 GgQuaternion() [6/7]

```
gg::GgQuaternion::GgQuaternion (  
    const GgQuaternion & q) [default]
```

コピーコンストラクタ.

引数

q	コピー元の四元数.
-----	-----------

呼び出し関係図:



8.21.2.7 GgQuaternion() [7/7]

```
gg::GgQuaternion::GgQuaternion (
    GgQuaternion && q) [default]
```

ムーブコンストラクタ.

引数

q	ムーブ元の四元数.
-----	-----------

呼び出し関係図:



8.21.2.8 ~GgQuaternion()

```
gg::GgQuaternion::~~GgQuaternion () [default]
```

デストラクタ.

8.21.3 関数詳解

8.21.3.1 add() [1/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgQuaternion & q) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

q を加えた四元数.

gg.h の 3847 行目に定義があります。

呼び出し関係図:



8.21.3.2 add() [2/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgVector & v) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

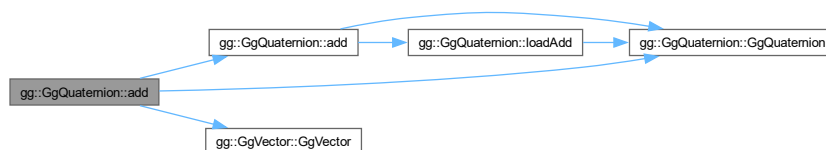
v	四元数を格納した GgVector 型の変数.
-----	-------------------------

戻り値

v を加えた四元数.

gg.h の 3836 行目に定義があります。

呼び出し関係図:



8.21.3.3 add() [3/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GLfloat * a) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を加えた四元数.

gg.h の 3825 行目に定義があります。

呼び出し関係図:



8.21.3.4 add() [4/4]

```

GgQuaternion gg::GgQuaternion::add (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
  
```

四元数に別の四元数を加算した結果を返す.

引数

<i>x</i>	加える四元数の x 要素.
<i>y</i>	加える四元数の y 要素.
<i>z</i>	加える四元数の z 要素.
<i>w</i>	加える四元数の w 要素.

戻り値

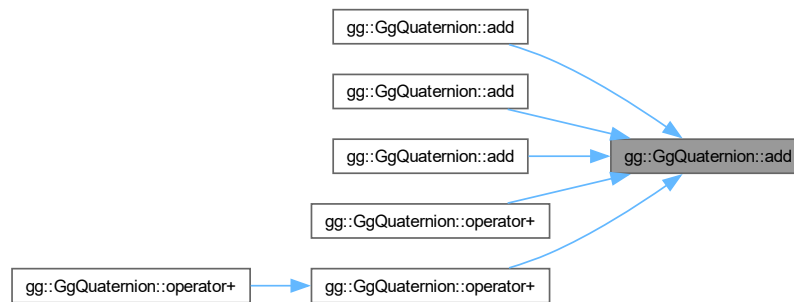
(x, y, z, w) を加えた四元数.

gg.h の 3813 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.5 conjugate()

```
GgQuaternion gg::GgQuaternion::conjugate () const [inline]
```

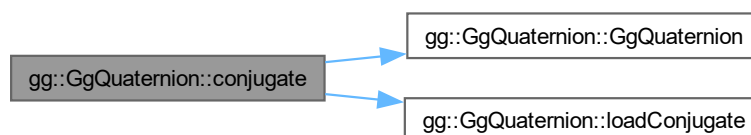
共役四元数に変換する.

戻り値

共役四元数.

gg.h の 4484 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.6 divide() [1/4]

```
GgQuaternion gg::GgQuaternion::divide (  
    const GgQuaternion & q) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q で割った四元数.

gg.h の 3996 行目に定義があります。

呼び出し関係図:



8.21.3.7 divide() [2/4]

```
GgQuaternion gg::GgQuaternion::divide (  
    const GgVector & v) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

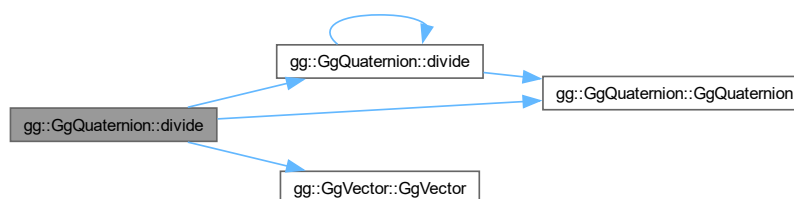
<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v で割った四元数.

gg.h の 3985 行目に定義があります。

呼び出し関係図:



8.21.3.8 divide() [3/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GLfloat * a) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

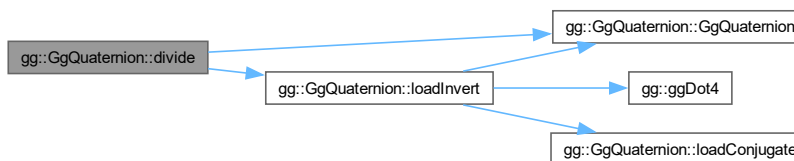
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a で割った四元数.

gg.h の 3971 行目に定義があります。

呼び出し関係図:



8.21.3.9 divide() [4/4]

```
GgQuaternion gg::GgQuaternion::divide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

戻り値

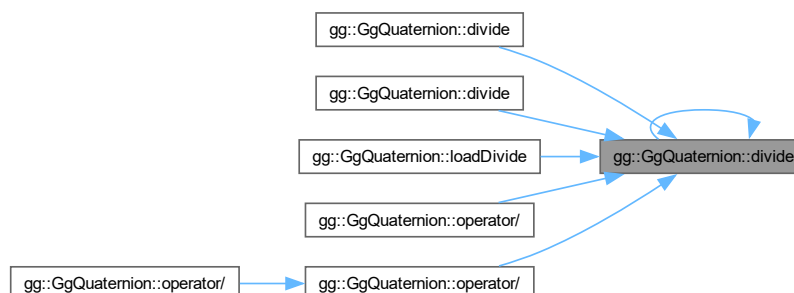
(x, y, z, w) を割った四元数.

[gg.h](#) の 3959 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.10 euler() [1/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GgVector & e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

e	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
---	---

戻り値

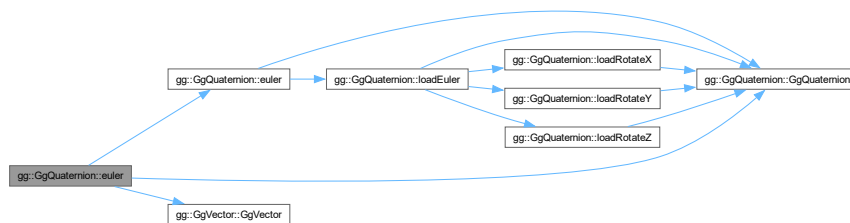
回転した四元数.

覚え書き

第 4 要素は無視する.

[gg.h](#) の 4324 行目に定義があります。

呼び出し関係図:



8.21.3.11 euler() [2/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GLfloat * e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

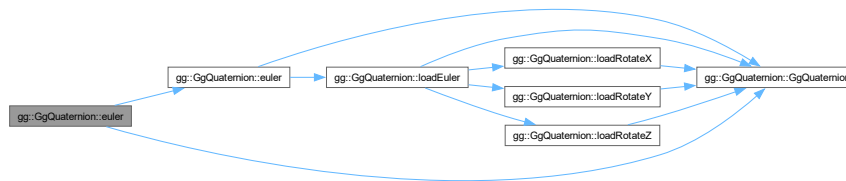
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
---	---

戻り値

回転した四元数.

[gg.h](#) の [4310](#) 行目に定義があります。

呼び出し関係図:



8.21.3.12 euler() [3/3]

```

GgQuaternion gg::GgQuaternion::euler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) const [inline]
  
```

四元数をオイラー角 (heading, pitch, roll) で回転した四元数を返す.

引数

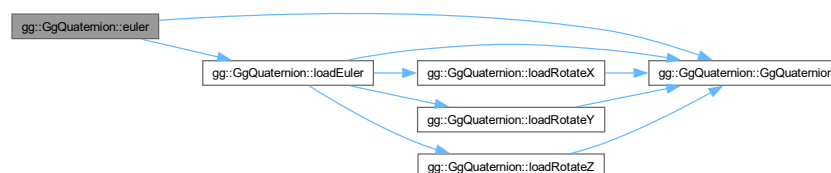
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

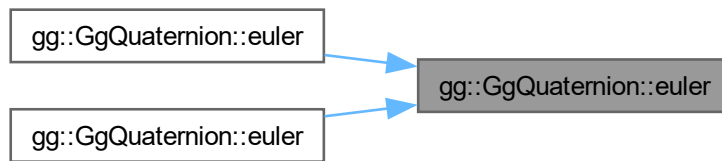
回転した四元数.

[gg.h](#) の [4298](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.13 get()

```
void gg::GgQuaternion::get (
    GLfloat * a) const [inline]
```

四元数を取り出す.

引数

<i>a</i>	四元数を格納する GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

[gg.h](#) の 4508 行目に定義があります。

8.21.3.14 getConjugateMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getConjugateMatrix () const [inline]
```

四元数の共役が表す回転の変換行列を取り出す.

戻り値

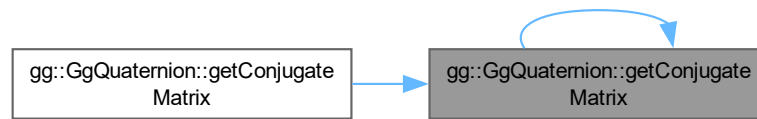
回転の変換を表す [GgMatrix](#) 型の変換行列.

[gg.h](#) の 4575 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.15 getConjugateMatrix() [2/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GgMatrix & m) const [inline]
```

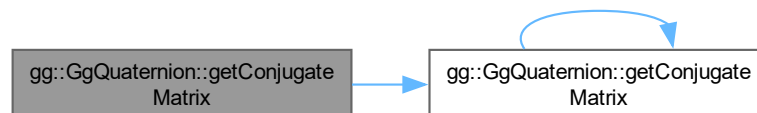
四元数の共役が表す回転の変換行列を m に求める。

引数

m	回転の変換行列を格納する <code>GgMatrix</code> 型の変数.
-----	--

`gg.h` の 4565 行目に定義があります。

呼び出し関係図:



8.21.3.16 getConjugateMatrix() [3/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GLfloat * a) const [inline]
```

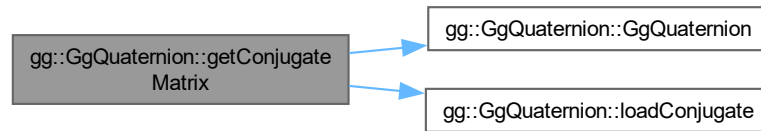
四元数の共役が表す回転の変換行列を a に求める。

引数

a	回転の変換行列を格納する <code>GLfloat</code> 型の 16 要素の配列変数.
-----	--

gg.h の 4553 行目に定義があります。

呼び出し関係図:



8.21.3.17 getMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getMatrix () const [inline]
```

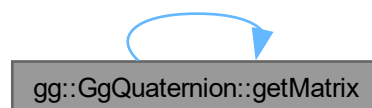
四元数が表す回転の変換行列を取り出す。

戻り値

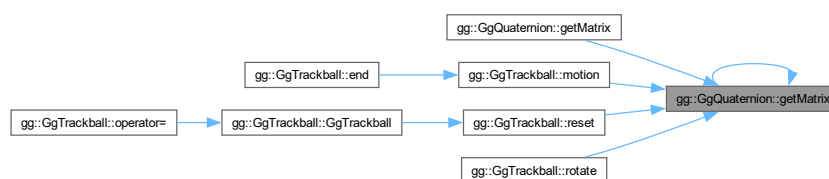
回転の変換を表す `GgMatrix` 型の変換行列。

gg.h の 4541 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.18 getMatrix() [2/3]

```
void gg::GgQuaternion::getMatrix (  
    GgMatrix & m) const [inline]
```

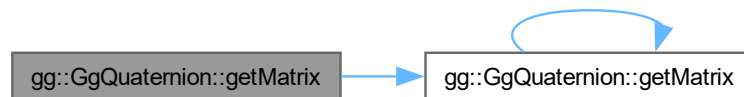
四元数が表す回転の変換行列を m に求める.

引数

<i>m</i>	回転の変換行列を格納する GgMatrix 型の変数.
----------	------------------------------------

[gg.h](#) の 4531 行目に定義があります。

呼び出し関係図:



8.21.3.19 getMatrix() [3/3]

```
void gg::GgQuaternion::getMatrix (
    GLfloat * a) const [inline]
```

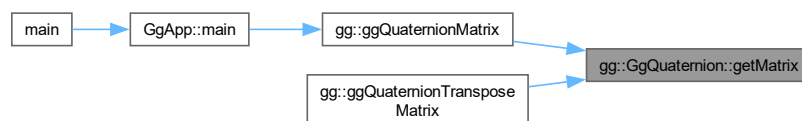
四元数が表す回転の変換行列を **a** に求める。

引数

<i>a</i>	回転の変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	-------------------------------------

[gg.h](#) の 4521 行目に定義があります。

被呼び出し関係図:



8.21.3.20 invert()

```
GgQuaternion gg::GgQuaternion::invert () const [inline]
```

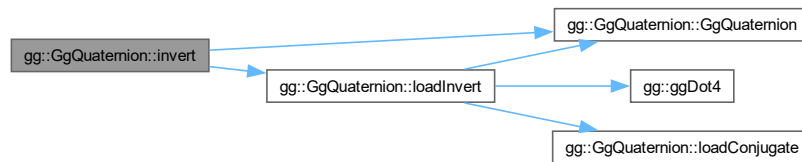
逆元に変換する。

戻り値

四元数の逆元.

[gg.h](#) の 4496 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.21 loadAdd() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q を加えた四元数.

[gg.h](#) の 3651 行目に定義があります。

呼び出し関係図:



8.21.3.22 loadAdd() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgVector & v) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

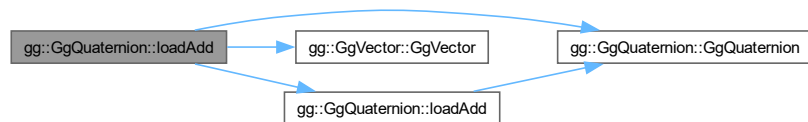
v	四元数を格納した GgVector 型の変数.
----------	--------------------------------

戻り値

v を加えた四元数.

gg.h の 3640 行目に定義があります。

呼び出し関係図:



8.21.3.23 loadAdd() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GLfloat * a) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	---------------------------------------

戻り値

a を加えた四元数.

gg.h の 3629 行目に定義があります。

呼び出し関係図:



8.21.3.24 loadAdd() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>x</i>	加える四元数の <i>x</i> 要素.
<i>y</i>	加える四元数の <i>y</i> 要素.
<i>z</i>	加える四元数の <i>z</i> 要素.
<i>w</i>	加える四元数の <i>w</i> 要素.

戻り値

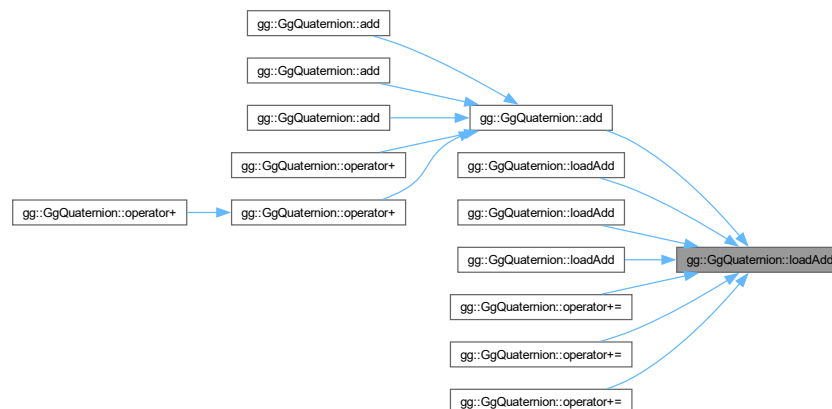
(*x*, *y*, *z*, *w*) を加えた四元数.

[gg.h](#) の 3613 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.25 loadConjugate() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadConjugate (  
    const GgQuaternion & q) [inline]
```

引数に指定した四元数の共役四元数を格納する.

引数

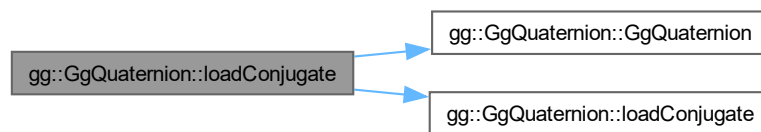
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

共役四元数.

gg.h の 4415 行目に定義があります。

呼び出し関係図:



8.21.3.26 loadConjugate() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadConjugate (  
    const GLfloat * a)
```

引数に指定した四元数の共役四元数を格納する.

引数

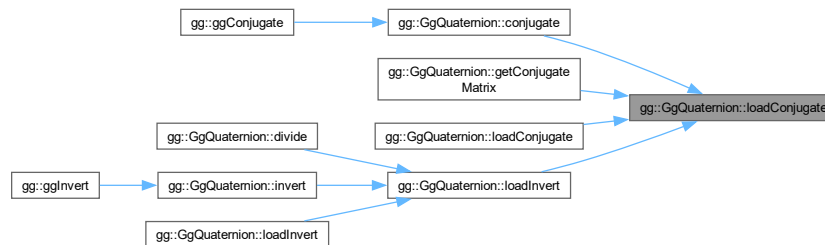
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

共役四元数.

`gg.cpp` の 3401 行目に定義があります。

被呼び出し関係図:



8.21.3.27 loadDivide() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgQuaternion & q) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

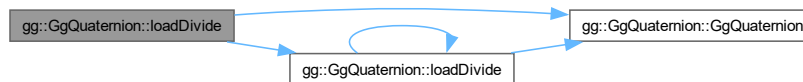
<code>q</code>	<code>GgQuaternion</code> 型の四元数.
----------------	----------------------------------

戻り値

`q` で割った四元数.

`gg.h` の 3799 行目に定義があります。

呼び出し関係図:



8.21.3.28 loadDivide() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgVector & v) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

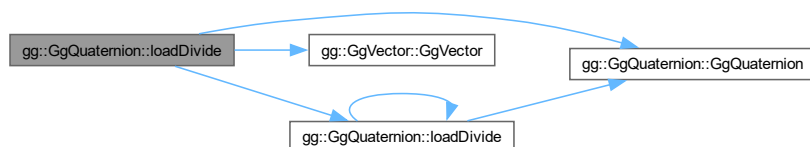
v	四元数を格納した GgVector 型の変数.
-----	--------------------------------

戻り値

v で割った四元数.

gg.h の 3788 行目に定義があります。

呼び出し関係図:



8.21.3.29 loadDivide() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GLfloat * a) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

a で割った四元数.

gg.h の 3777 行目に定義があります。

呼び出し関係図:



8.21.3.30 loadDivide() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

戻り値

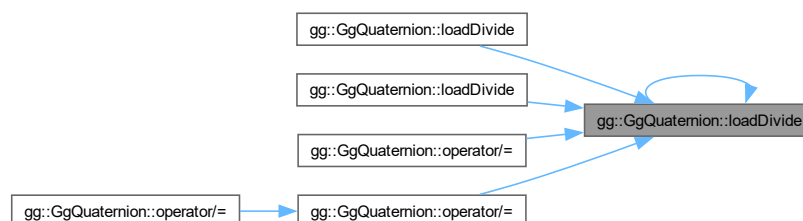
(x , y , z , w) を割った四元数.

[gg.h](#) の 3765 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.31 loadEuler() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadEuler (  
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を格納する.

引数

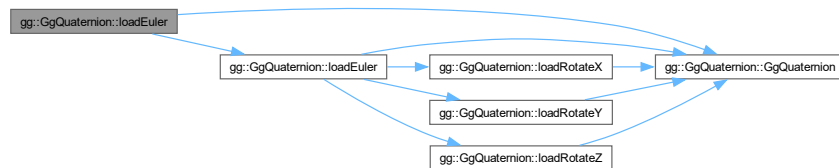
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

格納した回転を表す四元数.

[gg.h](#) の 4285 行目に定義があります。

呼び出し関係図:



8.21.3.32 loadEuler() [2/2]

```

gg::GgQuaternion & gg::GgQuaternion::loadEuler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll)

```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を格納する.

引数

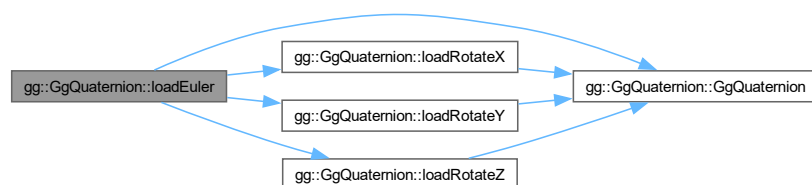
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

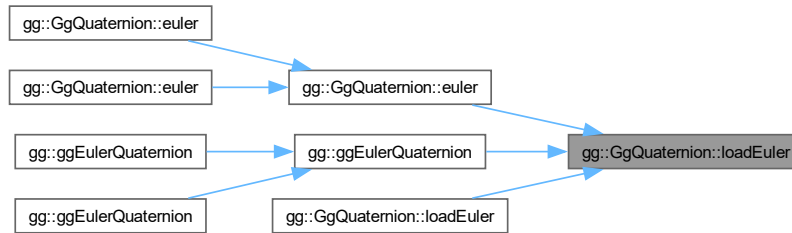
格納した回転を表す四元数.

[gg.cpp](#) の 3373 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.33 loadIdentity()

`GgQuaternion` & `gg::GgQuaternion::loadIdentity ()` [inline]

単位元を格納する.

戻り値

格納された単位元.

`gg.h` の 4135 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.34 loadInvert() [1/2]

`GgQuaternion` & `gg::GgQuaternion::loadInvert (`
`const GgQuaternion & q)` [inline]

引数に指定した四元数の逆元を格納する.

引数

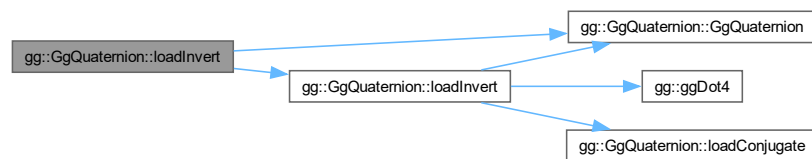
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

四元数の逆元.

gg.h の 4434 行目に定義があります。

呼び出し関係図:



8.21.3.35 loadInvert() [2/2]

```

gg::GgQuaternion & gg::GgQuaternion::loadInvert (
    const GLfloat * a)
  
```

引数に指定した四元数の逆元を格納する.

引数

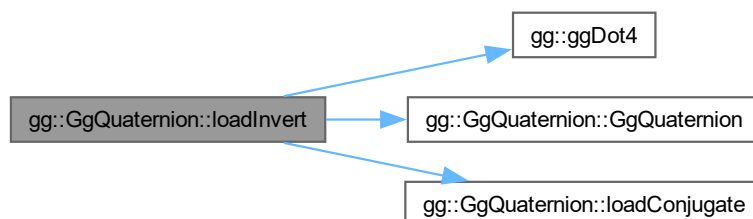
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

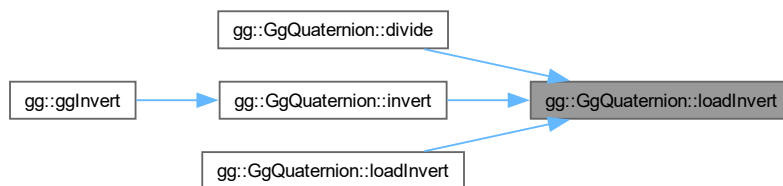
四元数の逆元.

gg.cpp の 3415 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.36 loadMatrix() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GgMatrix & m) [inline]
```

回転の変換行列 m を表す四元数を格納する.

引数

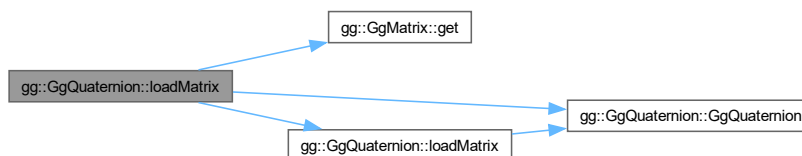
m	<code>GgMatrix</code> 型の変換行列.
-----	-------------------------------

戻り値

m による回転の変換に相当する四元数.

`gg.h` の 4125 行目に定義があります。

呼び出し関係図:



8.21.3.37 loadMatrix() [2/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GLfloat * a) [inline]
```

回転の変換行列を表す四元数を格納する.

引数

<i>a</i>	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

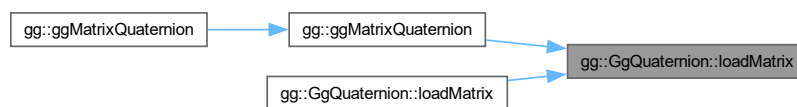
a による回転の変換に相当する四元数.

[gg.h](#) の 4113 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



8.21.3.38 loadMultiply() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

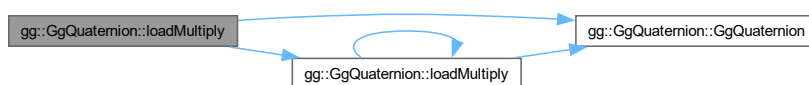
<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q を乗じた四元数.

[gg.h](#) の 3751 行目に定義があります.

呼び出し関係図:



8.21.3.39 loadMultiply() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgVector & v) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

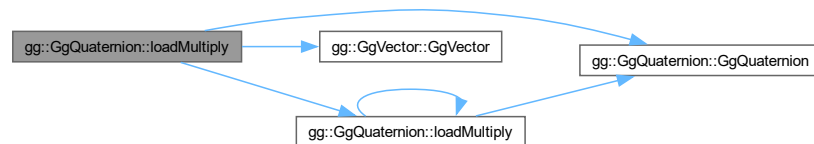
<code>v</code>	四元数を格納した <code>GgVector</code> 型の変数.
----------------	--------------------------------------

戻り値

`v` を乗じた四元数.

`gg.h` の 3740 行目に定義があります。

呼び出し関係図:



8.21.3.40 loadMultiply() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GLfloat * a) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

<code>a</code>	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
----------------	---

戻り値

`a` を乗じた四元数.

`gg.h` の 3729 行目に定義があります。

呼び出し関係図:



8.21.3.41 loadMultiply() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数に別の四元数を乗算した結果を格納する。

引数

<i>x</i>	掛ける四元数の <i>x</i> 要素.
<i>y</i>	掛ける四元数の <i>y</i> 要素.
<i>z</i>	掛ける四元数の <i>z</i> 要素.
<i>w</i>	掛ける四元数の <i>w</i> 要素.

戻り値

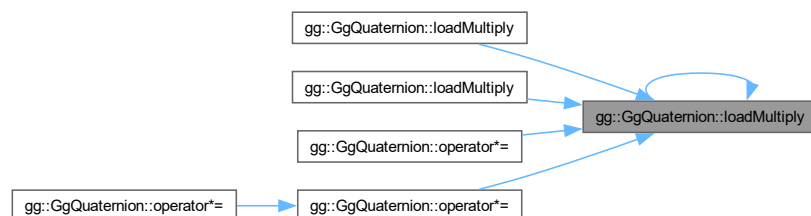
(*x*, *y*, *z*, *w*) を掛けた四元数.

[gg.h](#) の 3717 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.42 loadNormalize() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadNormalize (  
    const GgQuaternion & q) [inline]
```

引数に指定した四元数を正規化して格納する.

引数

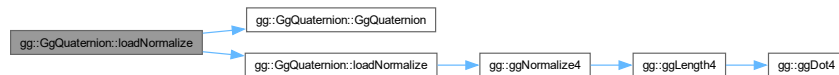
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

正規化された四元数.

gg.h の 4396 行目に定義があります。

呼び出し関係図:



8.21.3.43 loadNormalize() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadNormalize (
    const GLfloat * a)
```

引数に指定した四元数を正規化して格納する.

引数

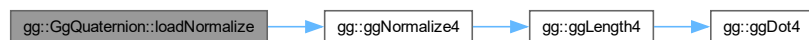
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

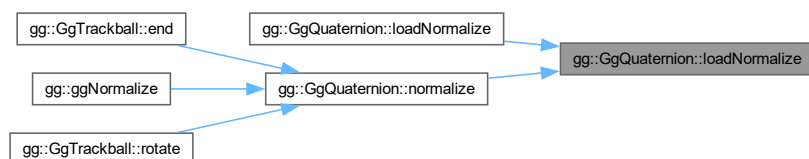
正規化された四元数.

gg.cpp の 3389 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.44 loadRotate() [1/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を格納する.

引数

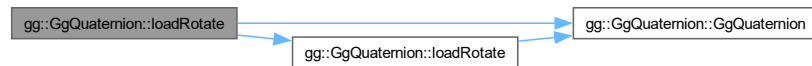
v	軸ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------------

戻り値

格納された回転を表す四元数.

gg.h の 4169 行目に定義があります。

呼び出し関係図:



8.21.3.45 loadRotate() [2/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を格納する.

引数

v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

格納された回転を表す四元数.

gg.h の 4158 行目に定義があります。

呼び出し関係図:



8.21.3.47 loadRotateX()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateX (
    GLfloat a)
```

x 軸中心に角度 *a* 回転する四元数を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

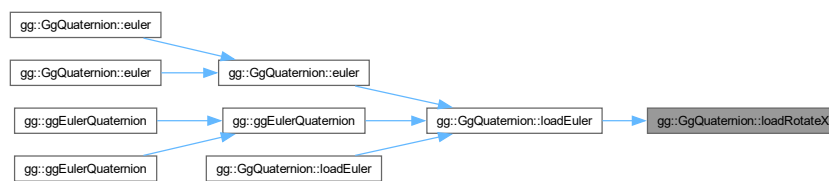
格納された回転を表す四元数.

gg.cpp の 3337 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.48 loadRotateY()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateY (
    GLfloat a)
```

y 軸中心に角度 *a* 回転する四元数を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

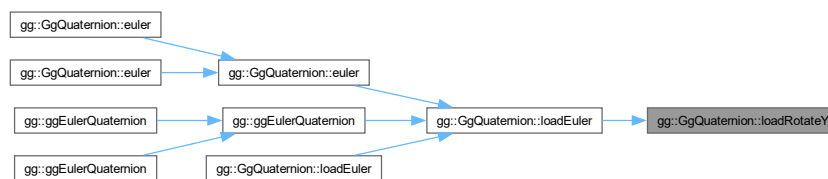
格納された回転を表す四元数.

`gg.cpp` の 3349 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.49 loadRotateZ()

```

gg::GgQuaternion & gg::GgQuaternion::loadRotateZ (
    GLfloat a)
  
```

z 軸中心に角度 *a* 回転する四元数を格納する.

引数

<i>a</i>	回転角.
----------	------

戻り値

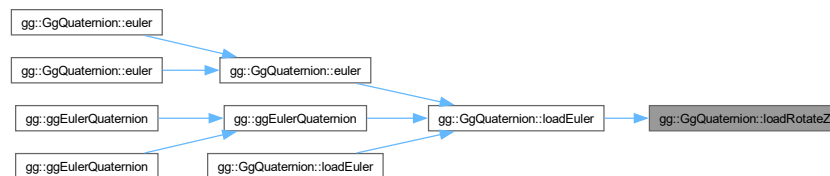
格納された回転を表す四元数.

gg.cpp の 3361 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.50 loadSlerp() [1/4]

```

GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
  
```

球面線形補間の結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した q, r を t で内分した四元数.

[gg.h](#) の 4351 行目に定義があります。

呼び出し関係図:



8.21.3.51 loadSlerp() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GLfloat * a,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

q	GgQuaternion 型の四元数.
a	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
t	補間パラメータ.

戻り値

格納した q, a を t で内分した四元数.

[gg.h](#) の 4364 行目に定義があります。

呼び出し関係図:



8.21.3.52 loadSlerp() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した *a*, *q* を *t* で内分した四元数.

gg.h の 4377 行目に定義があります。

呼び出し関係図:



8.21.3.53 loadSlerp() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する。

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

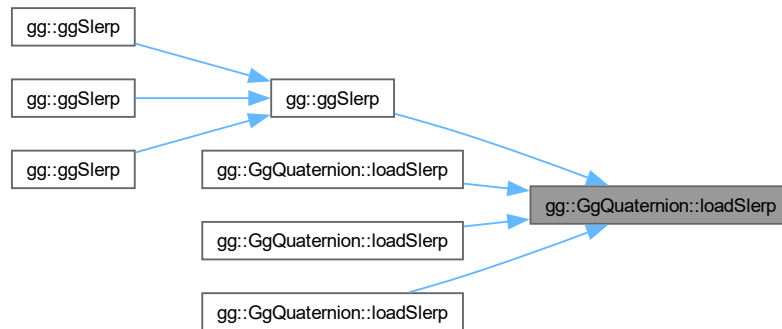
格納した *a*, *b* を *t* で内分した四元数.

gg.h の 4337 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.54 loadSubtract() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgQuaternion & q) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

<i>q</i>	<code>GgQuaternion</code> 型の四元数.
----------	----------------------------------

戻り値

q を引いた四元数.

`gg.h` の 3703 行目に定義があります。

呼び出し関係図:



8.21.3.55 loadSubtract() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgVector & v) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

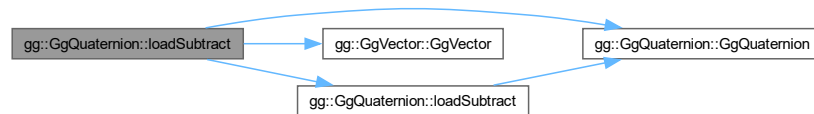
<code>v</code>	四元数を格納した <code>GgVector</code> 型の変数.
----------------	--------------------------------------

戻り値

`v` を引いた四元数.

`gg.h` の 3692 行目に定義があります.

呼び出し関係図:



8.21.3.56 loadSubtract() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GLfloat * a) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

<code>a</code>	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
----------------	---

戻り値

`a` を引いた四元数.

`gg.h` の 3681 行目に定義があります.

呼び出し関係図:



8.21.3.57 loadSubtract() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w) [inline]
```

四元数から別の四元数を減算した結果を格納する。

引数

x	引く四元数の x 要素.
y	引く四元数の y 要素.
z	引く四元数の z 要素.
w	引く四元数の w 要素.

戻り値

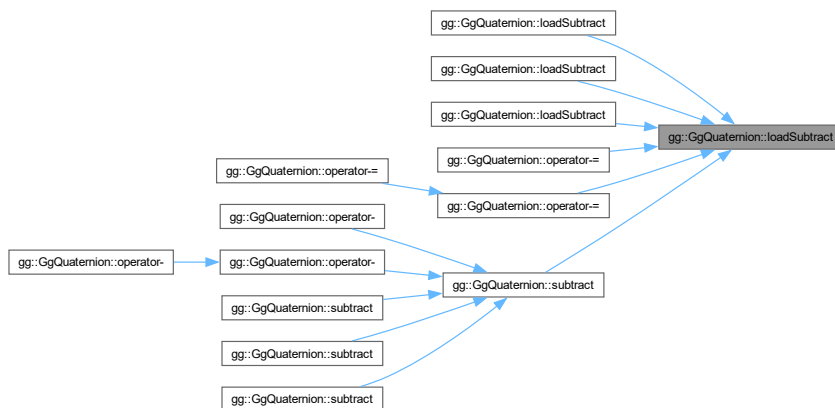
(x, y, z, w) を引いた四元数.

gg.h の 3665 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.58 multiply() [1/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GgQuaternion & q) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

q	<code>GgQuaternion</code> 型の四元数.
-----	----------------------------------

戻り値

q を掛けた四元数.

`gg.h` の 3945 行目に定義があります。

呼び出し関係図:



8.21.3.59 multiply() [2/4]

```
GgQuaternion gg::GgQuaternion::multiply (  
    const GgVector & v) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

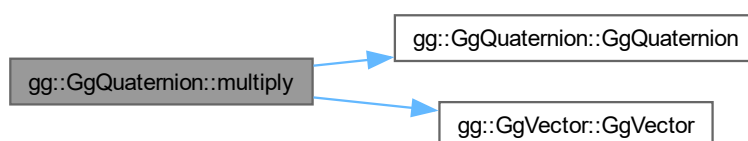
v	四元数を格納した <code>GgVector</code> 型の変数.
-----	--------------------------------------

戻り値

v を掛けた四元数.

`gg.h` の 3934 行目に定義があります。

呼び出し関係図:



8.21.3.60 multiply() [3/4]

```
GgQuaternion gg::GgQuaternion::multiply (  
    const GLfloat * a) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を掛けた四元数.

[gg.h](#) の 3921 行目に定義があります。

呼び出し関係図:



8.21.3.61 multiply() [4/4]

```

GgQuaternion gg::GgQuaternion::multiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
  
```

四元数に別の四元数を乗算した結果を返す.

引数

<i>x</i>	掛ける四元数の <i>x</i> 要素.
<i>y</i>	掛ける四元数の <i>y</i> 要素.
<i>z</i>	掛ける四元数の <i>z</i> 要素.
<i>w</i>	掛ける四元数の <i>w</i> 要素.

戻り値

(*x*, *y*, *z*, *w*) を掛けた四元数.

[gg.h](#) の 3909 行目に定義があります。

呼び出し関係図:



8.21.3.62 norm()

```
GLfloat gg::GgQuaternion::norm () const [inline]
```

四元数のノルムを求める。

戻り値

四元数のノルム。

[gg.h](#) の [3599](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.63 normalize()

```
GgQuaternion gg::GgQuaternion::normalize () const [inline]
```

正規化する。

戻り値

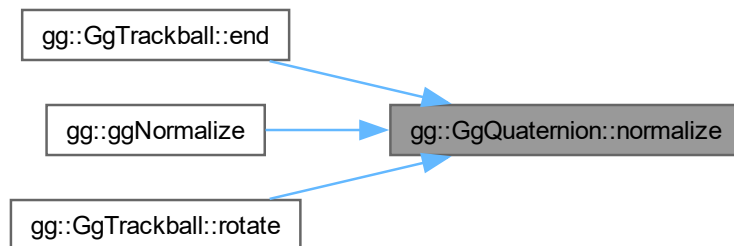
正規化された四元数.

[gg.h](#) の 4472 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.64 operator*() [1/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GgQuaternion & q) const [inline]
```

[gg.h](#) の 4090 行目に定義があります。

呼び出し関係図:

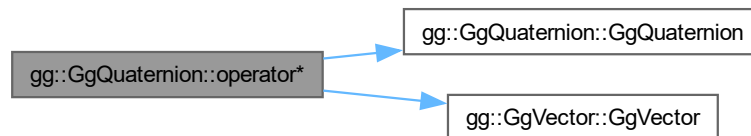


8.21.3.65 operator*() [2/3]

```
GgQuaternion gg::GgQuaternion::operator* (  
    const GgVector & v) const [inline]
```

gg.h の 4086 行目に定義があります。

呼び出し関係図:

**8.21.3.66 operator*() [3/3]**

```
GgQuaternion gg::GgQuaternion::operator* (  
    const GLfloat * a) const [inline]
```

gg.h の 4082 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.67 operator*=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgQuaternion & q) [inline]
```

gg.h の 4042 行目に定義があります。

呼び出し関係図:

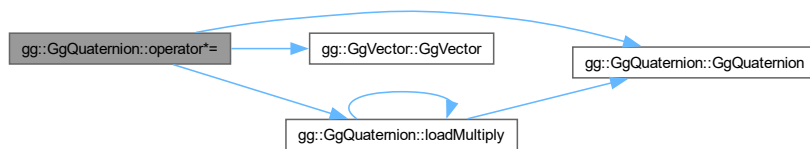


8.21.3.68 operator*=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgVector & v) [inline]
```

gg.h の 4038 行目に定義があります。

呼び出し関係図:



8.21.3.69 operator*=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GLfloat * a) [inline]
```

gg.h の 4034 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.70 operator+() [1/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgQuaternion & q) const [inline]
```

gg.h の 4066 行目に定義があります。

呼び出し関係図:

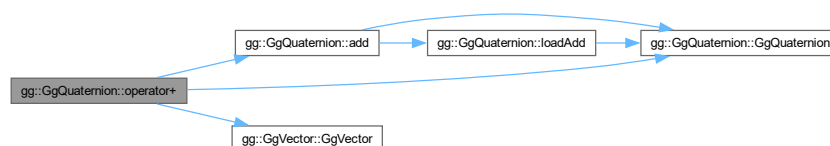


8.21.3.71 operator+() [2/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgVector & v) const [inline]
```

gg.h の 4062 行目に定義があります。

呼び出し関係図:



8.21.3.72 operator+() [3/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GLfloat * a) const [inline]
```

gg.h の 4058 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

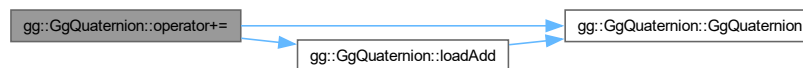


8.21.3.73 operator+=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgQuaternion & q) [inline]
```

gg.h の 4018 行目に定義があります。

呼び出し関係図:

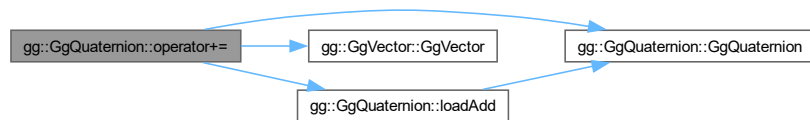


8.21.3.74 operator+=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgVector & v) [inline]
```

gg.h の 4014 行目に定義があります。

呼び出し関係図:

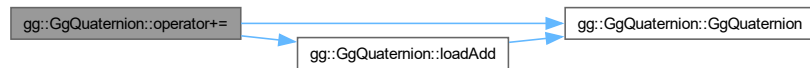


8.21.3.75 operator+=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GLfloat * a) [inline]
```

gg.h の 4010 行目に定義があります。

呼び出し関係図:

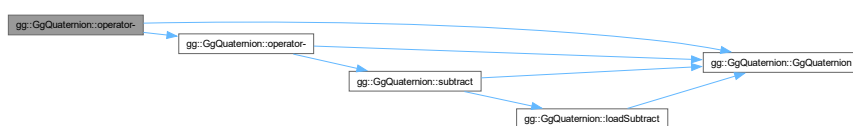


8.21.3.76 operator-() [1/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgQuaternion & q) const [inline]
```

gg.h の 4078 行目に定義があります。

呼び出し関係図:



8.21.3.77 operator-() [2/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgVector & v) const [inline]
```

gg.h の 4074 行目に定義があります。

呼び出し関係図:



8.21.3.78 operator-() [3/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GLfloat * a) const [inline]
```

gg.h の 4070 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.79 operator-=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgQuaternion & q) [inline]
```

gg.h の 4030 行目に定義があります。

呼び出し関係図:

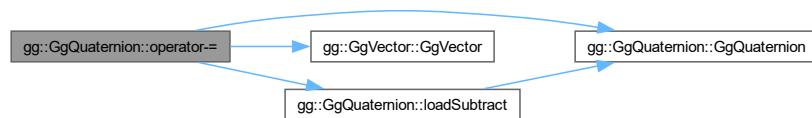


8.21.3.80 operator-=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgVector & v) [inline]
```

gg.h の 4026 行目に定義があります。

呼び出し関係図:



8.21.3.81 operator-=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GLfloat * a) [inline]
```

gg.h の 4022 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

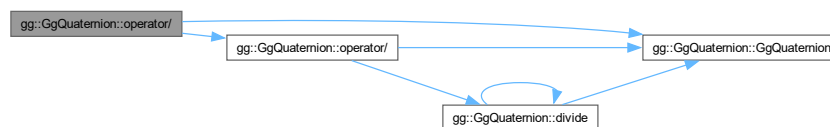


8.21.3.82 operator/() [1/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgQuaternion & q) const [inline]
```

`gg.h` の 4102 行目に定義があります。

呼び出し関係図:

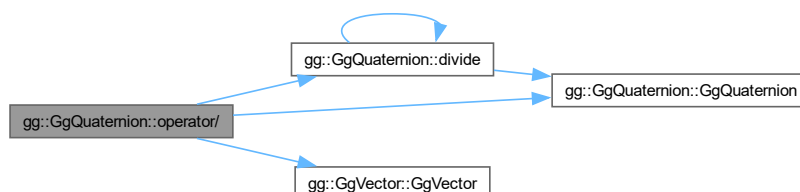


8.21.3.83 operator/() [2/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgVector & v) const [inline]
```

`gg.h` の 4098 行目に定義があります。

呼び出し関係図:

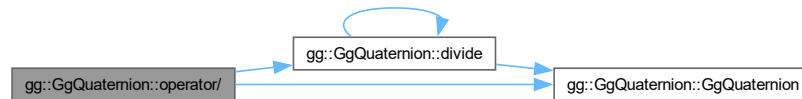


8.21.3.84 operator/() [3/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GLfloat * a) const [inline]
```

gg.h の 4094 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.85 operator/=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgQuaternion & q) [inline]
```

gg.h の 4054 行目に定義があります。

呼び出し関係図:

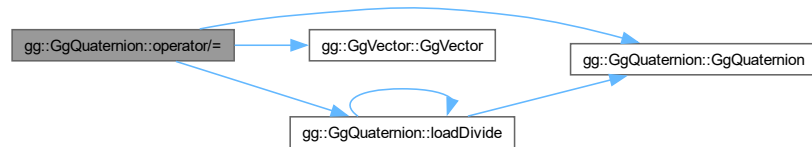


8.21.3.86 operator/=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgVector & v) [inline]
```

gg.h の 4050 行目に定義があります。

呼び出し関係図:

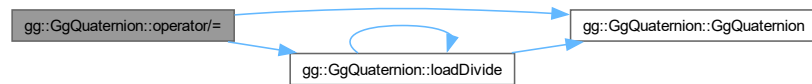


8.21.3.87 operator/=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GLfloat * a) [inline]
```

gg.h の 4046 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.88 operator=() [1/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GgQuaternion & q) [default]
```

代入演算子.

引数

q	代入元の四元数.
-----	----------

戻り値

代入後のこの四元数の参照.

呼び出し関係図:



被呼び出し関係図:

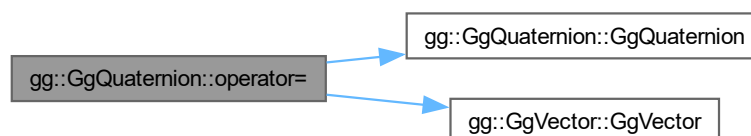


8.21.3.89 operator=() [2/4]

```
GgQuaternion & gg::GgQuaternion::operator= (  
    const GgVector & v) [inline]
```

gg.h の 4006 行目に定義があります。

呼び出し関係図:



8.21.3.90 operator=() [3/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GLfloat * a) [inline]
```

gg.h の 4002 行目に定義があります。

呼び出し関係図:

**8.21.3.91 operator=()** [4/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    GgQuaternion && q) [default]
```

ムーブ代入演算子.

引数

<i>q</i>	ムーブ代入元の四元数.
----------	-------------

戻り値

ムーブ代入後のこの四元数の参照.

呼び出し関係図:

**8.21.3.92 rotate()** [1/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 v[3] 回転した四元数を返す.

引数

v	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

回転した四元数.

gg.h の 4231 行目に定義があります.

呼び出し関係図:



8.21.3.93 rotate() [2/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v,
    GLfloat a) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 a 回転した四元数を返す.

引数

v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

回転した四元数.

gg.h の 4220 行目に定義があります.

呼び出し関係図:



8.21.3.94 rotate() [3/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

四元数を (x, y, z) を軸として角度 a 回転した四元数を返す.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

戻り値

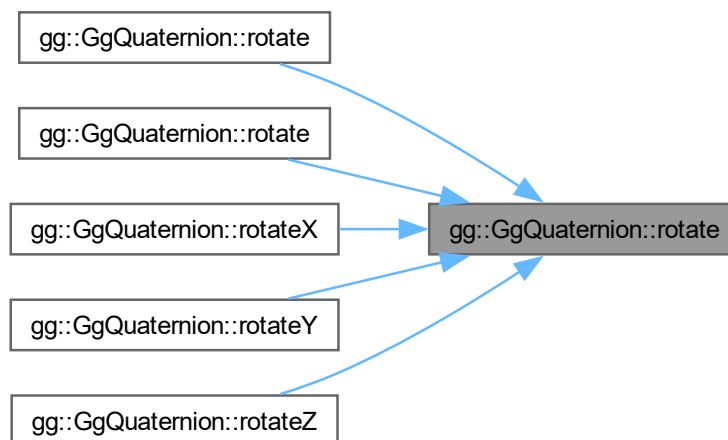
回転した四元数.

[gg.h](#) の 4207 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.21.3.95 rotateX()

```
GgQuaternion gg::GgQuaternion::rotateX (
    GLfloat a) const [inline]
```

四元数を x 軸中心に角度 **a** 回転した四元数を返す.

引数

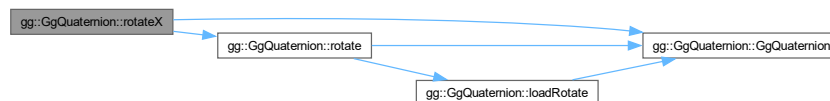
a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4242 行目に定義があります。

呼び出し関係図:



8.21.3.96 rotateY()

```
GgQuaternion gg::GgQuaternion::rotateY (
    GLfloat a) const [inline]
```

四元数を y 軸中心に角度 **a** 回転した四元数を返す.

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4253 行目に定義があります。

呼び出し関係図:



8.21.3.97 rotateZ()

```
GgQuaternion gg::GgQuaternion::rotateZ (
    GLfloat a) const [inline]
```

四元数を z 軸中心に角度 **a** 回転した四元数を返す。

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4264 行目に定義があります。

呼び出し関係図:



8.21.3.98 slerp() [1/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    const GgQuaternion & q,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す。

引数

q	GgQuaternion 型の四元数.
t	補間パラメータ.

戻り値

四元数を **q** に対して **t** で内分した結果.

[gg.h](#) の 4460 行目に定義があります。

呼び出し関係図:



8.21.3.99 slerp() [2/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    GLfloat * a,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

四元数を *a* に対して *t* で内分した結果.

gg.h の 4446 行目に定義があります。

呼び出し関係図:



8.21.3.100 subtract() [1/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgQuaternion & q) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を引いた四元数.

gg.h の 3895 行目に定義があります。

呼び出し関係図:



8.21.3.101 subtract() [2/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgVector & v) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<code>v</code>	四元数を格納した <code>GgVector</code> 型の変数.
----------------	--------------------------------------

戻り値

`v` を引いた四元数.

`gg.h` の 3884 行目に定義があります.

呼び出し関係図:

**8.21.3.102 subtract()** [3/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GLfloat * a) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<code>a</code>	四元数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
----------------	---

戻り値

`a` を引いた四元数.

`gg.h` の 3873 行目に定義があります.

呼び出し関係図:



8.21.3.103 subtract() [4/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>x</i>	引く四元数の <i>x</i> 要素.
<i>y</i>	引く四元数の <i>y</i> 要素.
<i>z</i>	引く四元数の <i>z</i> 要素.
<i>w</i>	引く四元数の <i>w</i> 要素.

戻り値

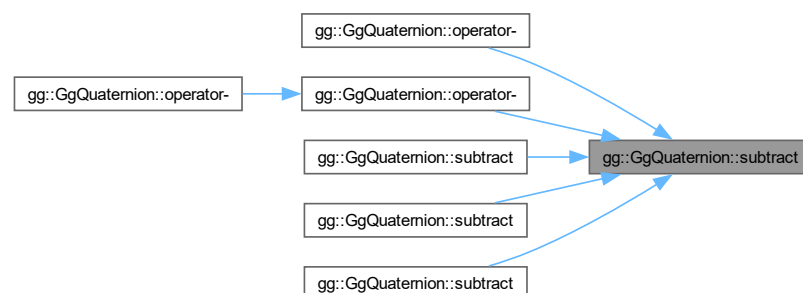
(*x*, *y*, *z*, *w*) を引いた四元数.

[gg.h](#) の 3861 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.22 gg::GgShader クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgShader` (const std::string &vert, const std::string &frag="", const std::string &geom="", int nvarying=0, const char *const *varyings=nullptr)
- `GgShader` (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- `GgShader` (const GgShader &shader)=delete
- `GgShader` (GgShader &&o) noexcept
- virtual `~GgShader` ()
- `GgShader` & `operator=` (const `GgShader` &shader)=delete
- `GgShader` & `operator=` (`GgShader` &&o) noexcept
- void `use` () const
- void `unuse` () const
- GLuint `get` () const

8.22.1 詳解

シェーダの基底クラス.

覚え書き

シェーダのクラスはこのクラスを派生して作る.

`gg.h` の 7159 行目に定義があります.

8.22.2 構築子と解体子

8.22.2.1 GgShader() [1/4]

```
gg::GgShader::GgShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

コンストラクタ.

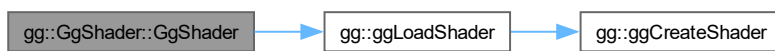
引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).

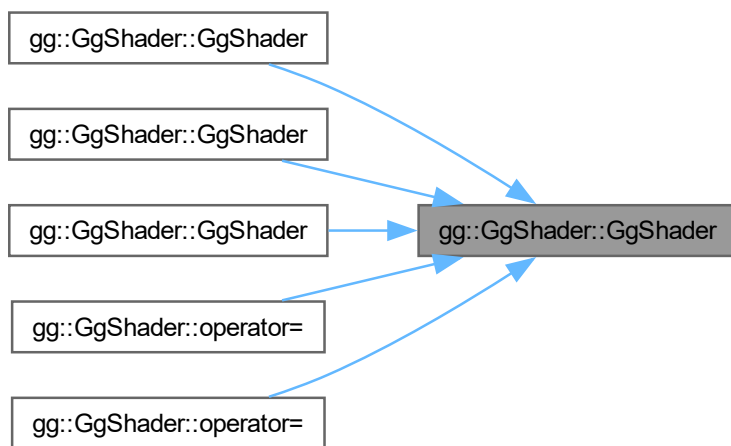
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.
-----------------	----------------------------------

gg.h の 7175 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.22.2.2 GgShader() [2/4]

```

gg::GgShader::GgShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

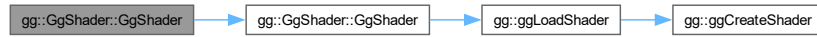
引数

<i>files</i>	パーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).

<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (<i>nullptr</i> なら不使用).
-----------------	--

[gg.h](#) の 7193 行目に定義があります。

呼び出し関係図:



8.22.2.3 GgShader() [3/4]

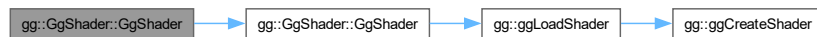
```
gg::GgShader::GgShader (
    const GgShader & shader) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>shader</i>	コピー元のシェーダ.
---------------	------------

呼び出し関係図:



8.22.2.4 GgShader() [4/4]

```
gg::GgShader::GgShader (
    GgShader && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

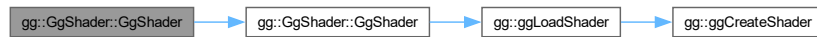
<i>o</i>	ムーブ元のシェーダ.
----------	------------

覚え書き

プログラム名の所有権を移動し, ムーブ元のプログラム名を 0 にする. ムーブ元のデストラクタが同じシェーダプログラムを削除しないようにするため.

gg.h の 7218 行目に定義があります。

呼び出し関係図:



8.22.2.5 ~GgShader()

```
virtual gg::GgShader::~GgShader () [inline], [virtual]
```

デストラクタ.

gg.h の 7227 行目に定義があります。

8.22.3 関数詳解

8.22.3.1 get()

```
GLuint gg::GgShader::get () const [inline]
```

シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

gg.h の 7289 行目に定義があります。

8.22.3.2 operator=() [1/2]

```
GgShader & gg::GgShader::operator= (
    const GgShader & shader) [delete]
```

代入演算子は使用しない.

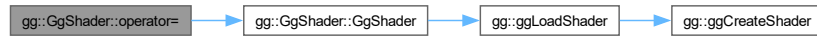
引数

<i>shader</i>	代入元のシェーダ.
---------------	-----------

戻り値

代入後のこのシェーダの参照.

呼び出し関係図:



8.22.3.3 operator=() [2/2]

```
GgShader & gg::GgShader::operator= (
    GgShader && o) [inline], [noexcept]
```

ムーブ代入演算子.

引数

<i>o</i>	ムーブ代入元のシェーダ.
----------	--------------

戻り値

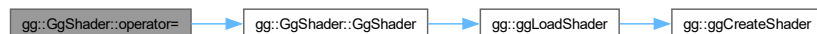
ムーブ代入後のこのシェーダの参照.

覚え書き

元々保持していたシェーダプログラムを削除してから所有権を移動し, ムーブ代入元のプログラム名を0にする.

[gg.h](#) の 7252 行目に定義があります。

呼び出し関係図:



8.22.3.4 unuse()

```
void gg::GgShader::unuse () const [inline]
```

シェーダプログラムの使用を終了する.

[gg.h](#) の 7279 行目に定義があります。

8.22.3.5 use()

```
void gg::GgShader::use () const [inline]
```

シェーダプログラムの使用を開始する。

[gg.h](#) の 7271 行目に定義があります。

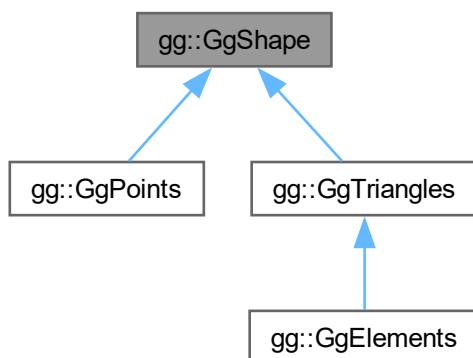
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.23 gg::GgShape クラス

```
#include <gg.h>
```

gg::GgShape の継承関係図



公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

8.23.1 詳解

形状データの基底クラス.

覚え書き

形状データのクラスはこのクラスを派生して作る. 基本図形の種類と頂点配列オブジェクトを保持する.

[gg.h](#) の [6535](#) 行目に定義があります。

8.23.2 構築子と解体子

8.23.2.1 GgShape()

```
gg::GgShape::GgShape (
    GLenum mode = 0) [inline]
```

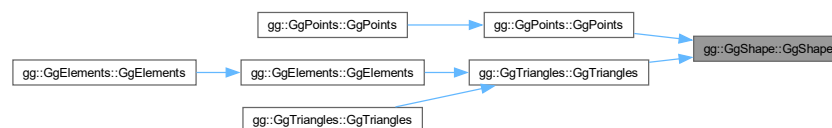
コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

[gg.h](#) の [6550](#) 行目に定義があります。

被呼び出し関係図:



8.23.2.2 ~GgShape()

```
virtual gg::GgShape::~~GgShape () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の [6559](#) 行目に定義があります。

8.23.3 関数詳解

8.23.3.1 draw()

```
virtual void gg::GgShape::draw (
    GLint first = 0,
    GLsizei count = 0) const [inline], [virtual]
```

図形の描画, 派生クラスでこの手続きをオーバーライドする.

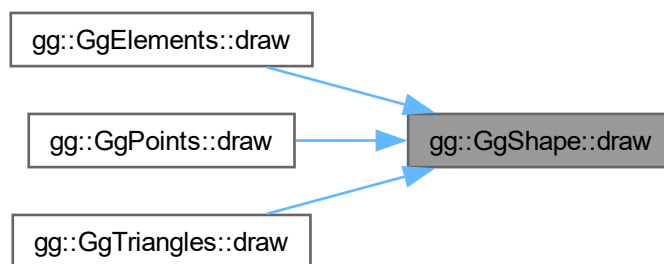
引数

<i>first</i>	描画する最初のアイテム.
<i>count</i>	描画するアイテムの数, 0 なら全部のアイテムを描画する.

[gg::GgElements](#), [gg::GgPoints](#), [gg::GgTriangles](#) で再実装されています。

[gg.h](#) の 6619 行目に定義があります。

被呼び出し関係図:



8.23.3.2 get()

```
const GLuint & gg::GgShape::get () const [inline]
```

頂点配列オブジェクト名を取り出す。

戻り値

頂点配列オブジェクト名。

[gg.h](#) の 6588 行目に定義があります。

被呼び出し関係図:



8.23.3.3 `getMode()`

```
const GEnum & gg::GgShape::getMode () const [inline]
```

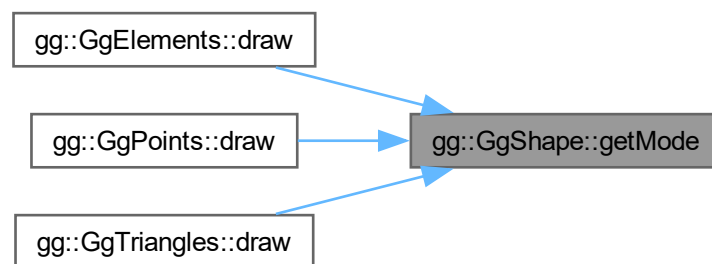
基本図形の検査.

戻り値

この頂点配列オブジェクトの基本図形の種類.

[gg.h](#) の 6608 行目に定義があります。

被呼び出し関係図:



8.23.3.4 `operator bool()`

```
virtual gg::GgShape::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうか調べる.

戻り値

頂点配列オブジェクトが有効なら `true`

[gg::GgPoints](#) で再実装されています。

[gg.h](#) の 6568 行目に定義があります。

8.23.3.5 `operator"!()`

```
virtual bool gg::GgShape::operator! () const [inline], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうかの結果を反転する.

戻り値

頂点配列オブジェクトが有効なら `false`, 無効なら `true`.

[gg::GgPoints](#) で再実装されています。

[gg.h](#) の 6578 行目に定義があります。

8.23.3.6 setMode()

```
void gg::GgShape::setMode (
    GLenum mode) [inline]
```

基本図形の設定.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6598 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.24 gg::GgSimpleObj クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgSimpleObj](#) (const std::string &name, bool normalize=false)
- virtual [~GgSimpleObj](#) ()
デストラクタ.
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const [GgTriangles](#) * [get](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

8.24.1 詳解

Wavefront OBJ 形式のファイル (Elements 形式).

gg.h の 8565 行目に定義があります。

8.24.2 構築子と解体子

8.24.2.1 GgSimpleObj()

```
gg::GgSimpleObj::GgSimpleObj (
    const std::string & name,
    bool normalize = false)
```

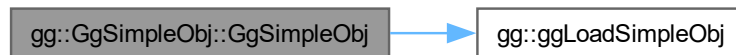
コンストラクタ.

引数

<i>name</i>	三角形分割された Alias OBJ 形式のファイルのファイル名.
<i>normalize</i>	true なら図形のサイズを [-1, 1] に正規化する.

gg.cpp の 6125 行目に定義があります。

呼び出し関係図:



8.24.2.2 ~GgSimpleObj()

```
virtual gg::GgSimpleObj::~~GgSimpleObj () [inline], [virtual]
```

デストラクタ.

gg.h の 8588 行目に定義があります。

8.24.3 関数詳解

8.24.3.1 draw()

```
void gg::GgSimpleObj::draw (  
    GLint first = 0,  
    GLsizei count = 0) const [virtual]
```

Wavefront OBJ 形式のデータを描画する手続き.

引数

<i>first</i>	描画する最初のパーツ番号.
<i>count</i>	描画するパーツの数, 0 なら全部のパーツを描く.

gg.cpp の 6153 行目に定義があります。

8.24.3.2 get()

```
const GgTriangles * gg::GgSimpleObj::get () const [inline]
```

形状データの取り出し。

戻り値

[GgTriangles](#) 型の形状データのポインタ。

[gg.h](#) の 8617 行目に定義があります。

呼び出し関係図:



8.24.3.3 operator bool()

```
gg::GgSimpleObj::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる。

戻り値

オブジェクトが有効なら true

[gg.h](#) の 8597 行目に定義があります。

8.24.3.4 operator"!()

```
bool gg::GgSimpleObj::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する。

戻り値

オブジェクトが有効なら false, 無効なら true.

[gg.h](#) の 8607 行目に定義があります。

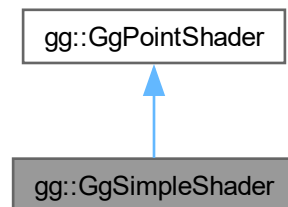
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

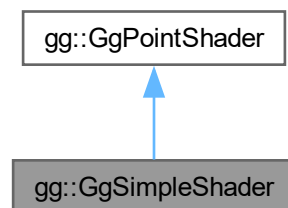
8.25 gg::GgSimpleShader クラス

```
#include <gg.h>
```

gg::GgSimpleShader の継承関係図



gg::GgSimpleShader 連携図



クラス

- struct [Light](#)
- class [LightBuffer](#)
- struct [Material](#)
- class [MaterialBuffer](#)

公開メンバ関数

- [GgSimpleShader](#) ()
- [GgSimpleShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const GgSimpleShader &o)

- virtual ~GgSimpleShader ()
- GgSimpleShader & operator= (const GgSimpleShader &o)
- bool load (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool load (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void loadModelviewMatrix (const GLfloat *mv, const GLfloat *mn) const
- virtual void loadModelviewMatrix (const GgMatrix &mv, const GgMatrix &mn) const
- virtual void loadModelviewMatrix (const GLfloat *mv) const
- virtual void loadModelviewMatrix (const GgMatrix &mv) const
- virtual void loadMatrix (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- virtual void loadMatrix (const GgMatrix &mp, const GgMatrix &mv, const GgMatrix &mn) const
- virtual void loadMatrix (const GLfloat *mp, const GLfloat *mv) const
- virtual void loadMatrix (const GgMatrix &mp, const GgMatrix &mv) const
- void use () const
- void use (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- void use (const GgMatrix &mp, const GgMatrix &mv, const GgMatrix &mn) const
- void use (const GLfloat *mp, const GLfloat *mv) const
- void use (const GgMatrix &mp, const GgMatrix &mv) const
- void use (const LightBuffer *light, GLint i=0) const
- void use (const LightBuffer &light, GLint i=0) const
- void use (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn, const LightBuffer *light, GLint i=0) const
- void use (const GgMatrix &mp, const GgMatrix &mv, const GgMatrix &mn, const LightBuffer &light, GLint i=0) const
- void use (const GLfloat *mp, const GLfloat *mv, const LightBuffer *light, GLint i=0) const
- void use (const GgMatrix &mp, const GgMatrix &mv, const LightBuffer &light, GLint i=0) const
- void use (const GLfloat *mp, const LightBuffer *light, GLint i=0) const
- void use (const GgMatrix &mp, const LightBuffer &light, GLint i=0) const

基底クラス **gg::GgPointShader** に属する継承公開メンバ関数

- GgPointShader ()
- GgPointShader (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GgPointShader (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual ~GgPointShader ()
- bool load (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool load (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void loadProjectionMatrix (const GLfloat *mp) const
- virtual void loadProjectionMatrix (const GgMatrix &mp) const
- void use (const GLfloat *mp) const
- void use (const GgMatrix &mp) const
- void use (const GLfloat *mp, const GLfloat *mv) const
- void use (const GgMatrix &mp, const GgMatrix &mv) const
- void unuse () const
- GLuint get () const

8.25.1 詳解

三角形に単純な陰影付けを行うシェーダ。

[gg.h](#) の 7553 行目に定義があります。

8.25.2 構築子と解体子

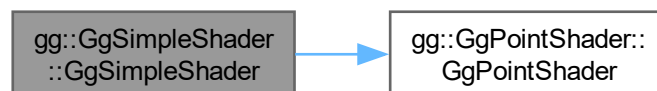
8.25.2.1 GgSimpleShader() [1/4]

```
gg::GgSimpleShader::GgSimpleShader () [inline]
```

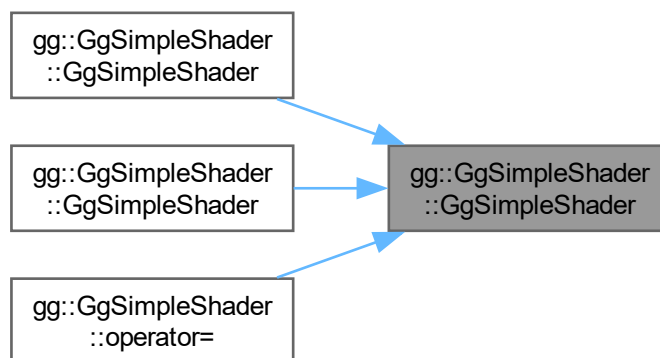
コンストラクタ.

[gg.h](#) の 7564 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.2.2 GgSimpleShader() [2/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

gg.h の 7579 行目に定義があります。

呼び出し関係図:



8.25.2.3 GgSimpleShader() [3/4]

```

gg::GgSimpleShader::GgSimpleShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

gg.h の 7597 行目に定義があります。

呼び出し関係図:



8.25.2.4 GgSimpleShader() [4/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const GgSimpleShader & o) [inline]
```

コピーコンストラクタ.

gg.h の 7609 行目に定義があります。

呼び出し関係図:



8.25.2.5 ~GgSimpleShader()

```
virtual gg::GgSimpleShader::~~GgSimpleShader () [inline], [virtual]
```

デストラクタ.

gg.h の 7618 行目に定義があります。

8.25.3 関数詳解

8.25.3.1 load() [1/2]

```
bool gg::GgSimpleShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	パーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら `true`.

[gg.h](#) の 7665 行目に定義があります。

呼び出し関係図:



8.25.3.2 load() [2/2]

```

bool gg::GgSimpleShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr)
  
```

シェーダのソースプログラムの文字列からプログラムオブジェクトを作成する。

引数

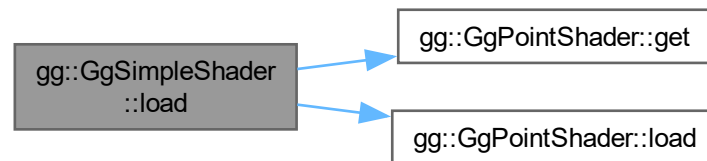
<i>vert</i>	バーテックスシェーダのソースプログラムの文字列.
<i>frag</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>geom</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>nvarying</i>	フィードバックする <code>varying</code> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <code>varying</code> 変数のリスト (nullptr なら不使用).

戻り値

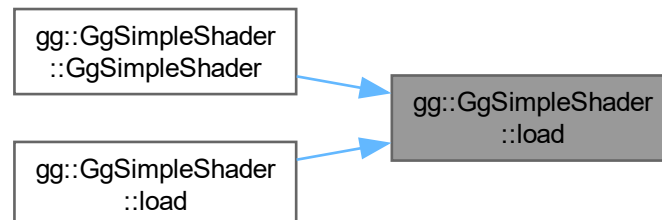
プログラムオブジェクトの作成に成功したら `true`.

[gg.cpp](#) の 6108 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.3.3 loadMatrix() [1/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
  
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する.

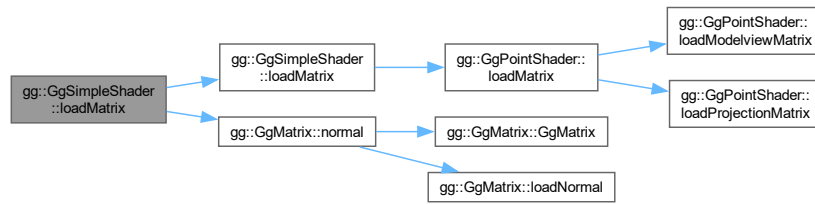
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg::GgPointShaderを再実装しています。

gg.h の 7759 行目に定義があります。

呼び出し関係図:



8.25.3.4 loadMatrix() [2/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]

```

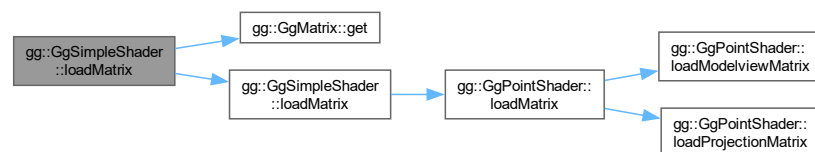
投影変換行列とモデルビュー変換行列と法線変換行列を設定する.

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7737 行目に定義があります。

呼び出し関係図:



8.25.3.5 loadMatrix() [3/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv) const [inline], [virtual]

```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する.

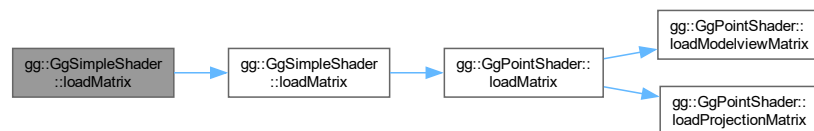
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

`gg::GgPointShader` を再実装しています。

`gg.h` の 7748 行目に定義があります。

呼び出し関係図:



8.25.3.6 loadMatrix() [4/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]

```

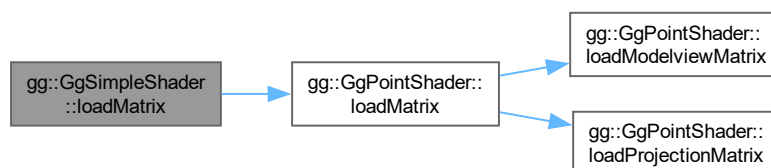
投影変換行列とモデルビュー変換行列と法線変換行列を設定する。

引数

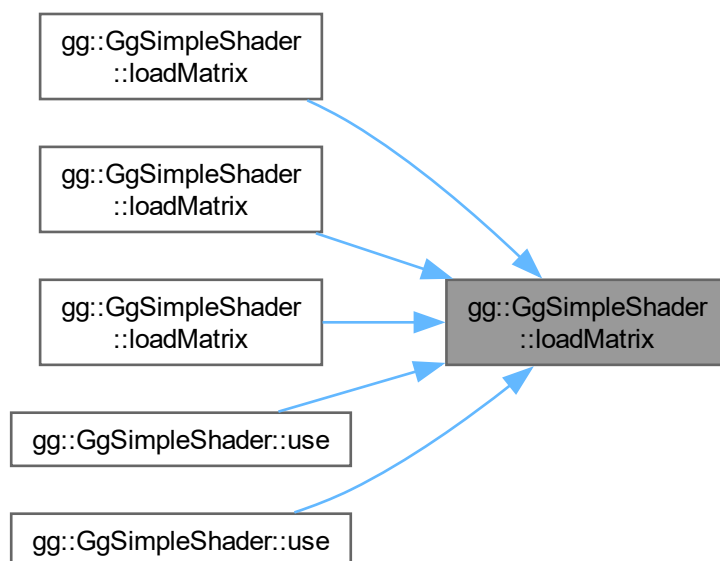
<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

`gg.h` の 7724 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.3.7 loadModelviewMatrix() [1/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

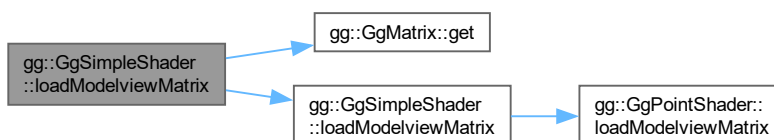
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	------------------------

gg::GgPointShaderを再実装しています。

gg.h の 7712 行目に定義があります。

呼び出し関係図:



8.25.3.8 loadModelviewMatrix() [2/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]
```

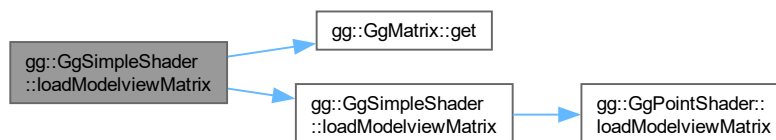
モデルビュー変換行列と法線変換行列を設定する。

引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7692 行目に定義があります。

呼び出し関係図:



8.25.3.9 loadModelviewMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

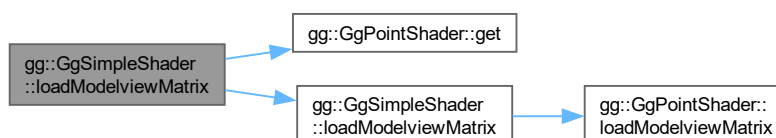
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgPointShader を再実装しています。

gg.h の 7702 行目に定義があります。

呼び出し関係図:



8.25.3.10 loadModelviewMatrix() [4/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]
```

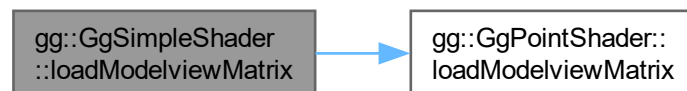
モデルビュー変換行列と法線変換行列を設定する。

引数

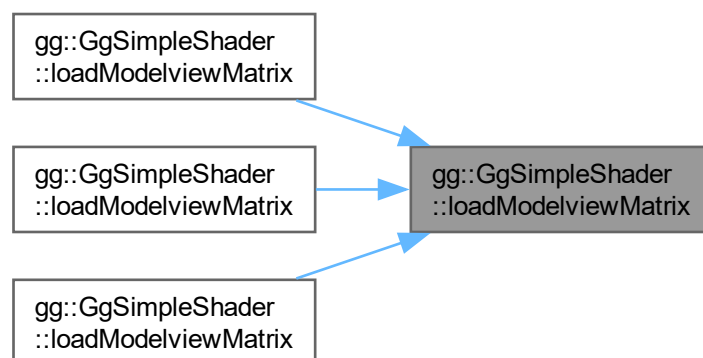
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 7680 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.25.3.11 operator=()

```
GgSimpleShader & gg::GgSimpleShader::operator= (  
    const GgSimpleShader & o) [inline]
```

代入演算子.

引数

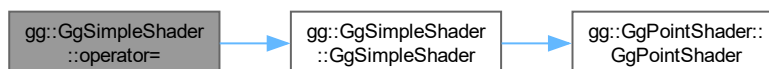
o 代入する [GgSimpleShader](#) 型のオブジェクト.

戻り値

代入後の [GgSimpleShader](#) 型のオブジェクトの参照.

[gg.h](#) の 7628 行目に定義があります。

呼び出し関係図:



8.25.3.12 use() [1/13]

```
void gg::GgSimpleShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する.

[gg::GgPointShader](#)を再実装しています。

[gg.h](#) の 8332 行目に定義があります。

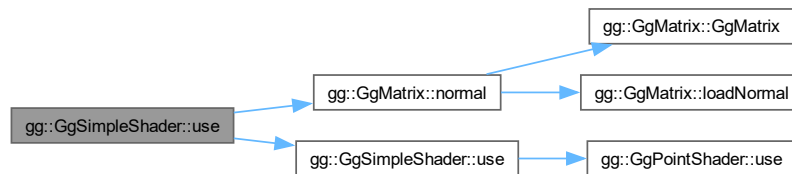
呼び出し関係図:



<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

[gg.h](#) の 8383 行目に定義があります。

呼び出し関係図:



8.25.3.14 use() [3/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline]
  
```

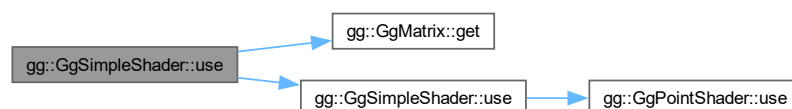
投影変換行列とモデルビュー変換行列と法線変換行列を指定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

[gg.h](#) の 8361 行目に定義があります。

呼び出し関係図:



8.25.3.15 use() [4/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

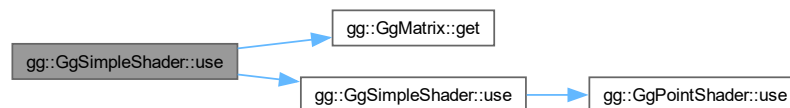
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8447 行目に定義があります。

呼び出し関係図:



8.25.3.16 use() [5/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

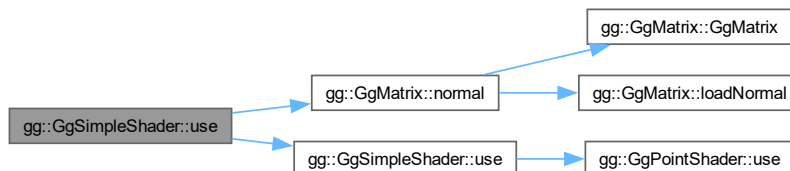
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.

<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

gg.h の 8484 行目に定義があります。

呼び出し関係図:



8.25.3.17 use() [6/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const LightBuffer & light,
    GLint i = 0) const [inline]
  
```

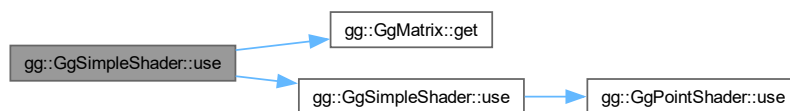
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8517 行目に定義があります。

呼び出し関係図:



8.25.3.18 use() [7/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

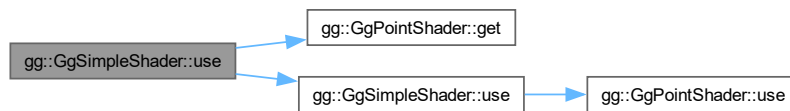
投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg.h の 8372 行目に定義があります。

呼び出し関係図:



8.25.3.19 use() [8/13]

```

void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline]
  
```

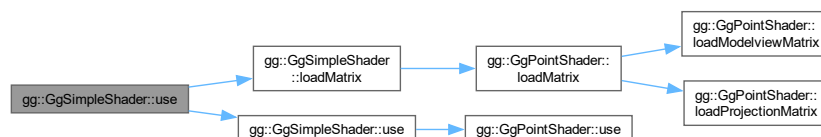
投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 8345 行目に定義があります。

呼び出し関係図:



8.25.3.20 use() [9/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

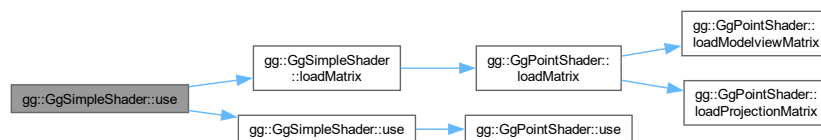
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8423 行目に定義があります。

呼び出し関係図:



8.25.3.21 use() [10/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

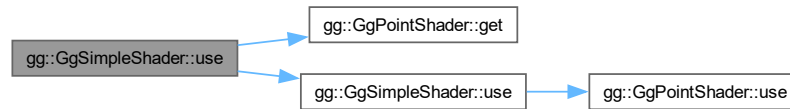
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.

<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

gg.h の 8466 行目に定義があります。

呼び出し関係図:



8.25.3.22 use() [11/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

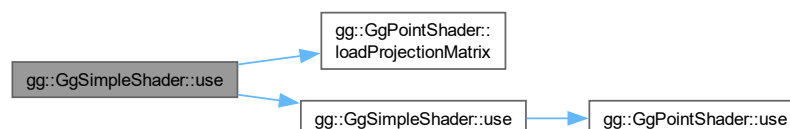
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8501 行目に定義があります。

呼び出し関係図:



8.25.3.23 use() [12/13]

```
void gg::GgSimpleShader::use (
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

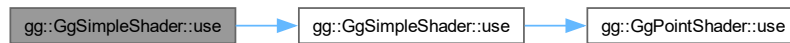
光源を指定してシェードプログラムの使用を開始する。

引数

<i>light</i>	光源の特性の <code>gg::LightBuffer</code> 構造体.
<i>i</i>	光源データの <code>uniform block</code> のインデックス.

`gg.h` の 8409 行目に定義があります。

呼び出し関係図:



8.25.3.24 use() [13/13]

```
void gg::GgSimpleShader::use (
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

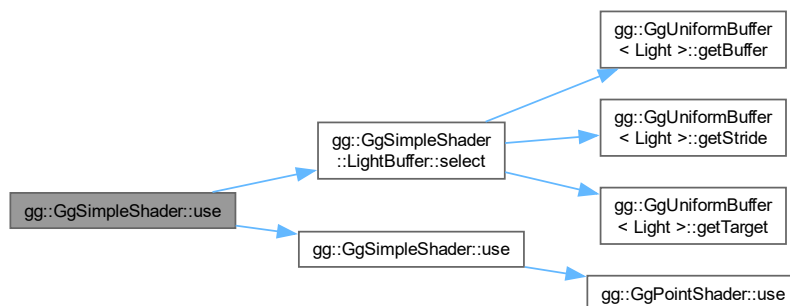
光源を指定してシェードプログラムの使用を開始する。

引数

<i>light</i>	光源の特性の <code>gg::LightBuffer</code> 構造体のポインタ.
<i>i</i>	光源データの <code>uniform block</code> のインデックス.

`gg.h` の 8394 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- `gg.h`
- `gg.cpp`

8.26 gg::GgTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- [GgTexture](#) (const GgTexture &texture)=delete
- [GgTexture](#) (GgTexture &&o) noexcept
- virtual [~GgTexture](#) ()
- [GgTexture](#) & [operator=](#) (const [GgTexture](#) &texture)=delete
- [GgTexture](#) & [operator=](#) ([GgTexture](#) &&o) noexcept
- void [bind](#) () const
- void [unbind](#) () const
- void [swapRandB](#) (bool swizzle) const
- const GLsizei & [getWidth](#) () const
- const GLsizei & [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- const GLuint & [getTexture](#) () const

8.26.1 詳解

テクスチャ.

覚え書き

画像データを読み込んでテクスチャマップを作成する.

[gg.h](#) の 5236 行目に定義があります。

8.26.2 構築子と解体子

8.26.2.1 GgTexture() [1/3]

```
gg::GgTexture::GgTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = false) [inline]
```

メモリ上のデータからテクスチャを作成するコンストラクタ.

引数

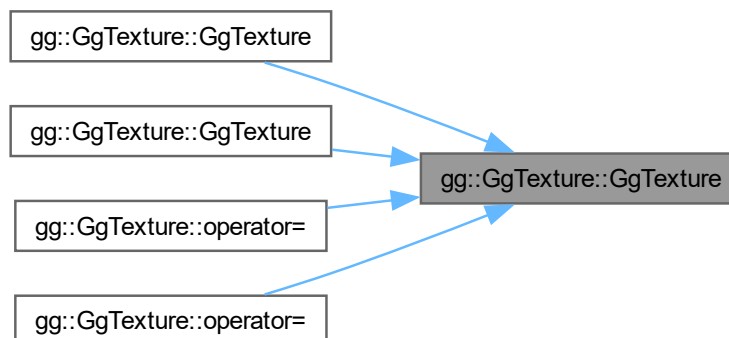
<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL.CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

gg.h の 5258 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.26.2.2 GgTexture() [2/3]

```
gg::GgTexture::GgTexture (
    const GgTexture & texture) [delete]
```

コピーコンストラクタは使用しない.

引数

<code>texture</code>	コピー元のテクスチャ.
----------------------	-------------

呼び出し関係図:



8.26.2.3 GgTexture() [3/3]

```
gg::GgTexture::GgTexture (
    GgTexture && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

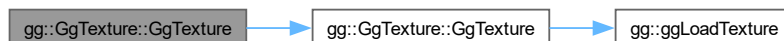
<code>o</code>	ムーブ元のテクスチャ.
----------------	-------------

覚え書き

テクスチャ名の所有権を移動し, ムーブ元のテクスチャ名を 0 にする. ムーブ元のデストラクタが同じテクスチャを削除しないようにするため.

[gg.h](#) の 5289 行目に定義があります。

呼び出し関係図:



8.26.2.4 ~GgTexture()

```
virtual gg::GgTexture::~~GgTexture () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 5299 行目に定義があります。

8.26.3 関数詳解

8.26.3.1 bind()

```
void gg::GgTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

gg.h の 5343 行目に定義があります。

8.26.3.2 getHeight()

```
const GLsizei & gg::GgTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す.

戻り値

テクスチャの縦の画素数.

gg.h の 5378 行目に定義があります。

被呼び出し関係図:



8.26.3.3 getSize() [1/2]

```
const GLsizei * gg::GgTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す.

戻り値

テクスチャのサイズを格納した配列へのポインタ.

gg.h の 5399 行目に定義があります。

8.26.3.4 getSize() [2/2]

```
void gg::GgTexture::getSize (  
    GLsizei * size) const [inline]
```

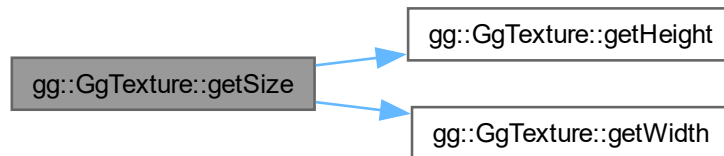
使用しているテクスチャのサイズを取り出す.

引数

<i>size</i>	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数.
-------------	--------------------------------------

gg.h の 5388 行目に定義があります。

呼び出し関係図:



8.26.3.5 getTexture()

```
const GLuint & gg::GgTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る.

戻り値

テクスチャ名.

gg.h の 5409 行目に定義があります。

8.26.3.6 getWidth()

```
const GLsizei & gg::GgTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す.

戻り値

テクスチャの横の画素数.

gg.h の 5368 行目に定義があります。

被呼び出し関係図:



8.26.3.7 operator=() [1/2]

```
GgTexture & gg::GgTexture::operator= (
    const GgTexture & texture) [delete]
```

代入演算子は使用しない.

引数

<i>texture</i>	代入元のテクスチャ.
----------------	------------

戻り値

代入後のこのテクスチャの参照.

呼び出し関係図:



8.26.3.8 operator=() [2/2]

```
GgTexture & gg::GgTexture::operator= (
    GgTexture && o) [inline], [noexcept]
```

ムーブ代入演算子.

引数

<i>o</i>	ムーブ代入元のテクスチャ.
----------	---------------

戻り値

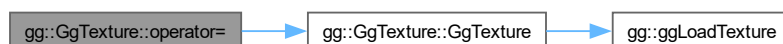
ムーブ代入後のこのテクスチャの参照.

覚え書き

元々保持していたテクスチャを削除してから所有権を移動する.

[gg.h](#) の 5322 行目に定義があります。

呼び出し関係図:



8.26.3.9 swapRandB()

```
void gg::GgTexture::swapRandB (
    bool swizzle) const
```

テクスチャの赤と青を交換する

引数

<i>swizzle</i>	赤と青を交換するなら true
----------------	-----------------

[gg.cpp](#) の 4052 行目に定義があります。

8.26.3.10 unbind()

```
void gg::GgTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

[gg.h](#) の 5351 行目に定義があります。

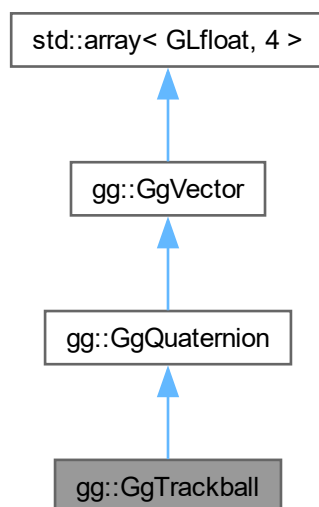
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

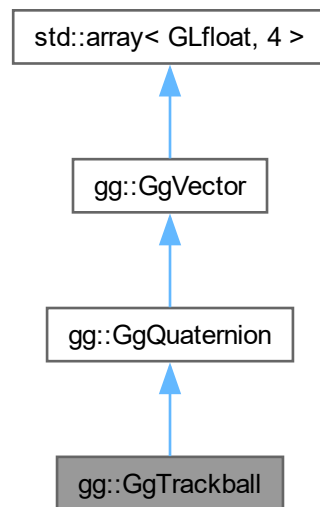
8.27 gg::GgTrackball クラス

```
#include <gg.h>
```

gg::GgTrackball の継承関係図



gg::GgTrackball 連携図



公開メンバ関数

- `GgTrackball` (`const GgQuaternion &q=ggIdentityQuaternion()`)
- `~GgTrackball` `()=default`
- `GgTrackball & operator=` (`const GgQuaternion &q`)
- `void region` (`GLfloat w, GLfloat h`)
- `void region` (`int w, int h`)
- `void begin` (`GLfloat x, GLfloat y`)
- `void motion` (`GLfloat x, GLfloat y`)
- `void rotate` (`const GgQuaternion &q`)
- `void end` (`GLfloat x, GLfloat y`)
- `void reset` (`const GgQuaternion &q=ggIdentityQuaternion()`)
- `const GLfloat * getStart` `() const`
- `const GLfloat & getStart` (`int direction`) `const`
- `void getStart` (`GLfloat *position`) `const`
- `const GLfloat * getScale` `() const`
- `const GLfloat getScale` (`int direction`) `const`
- `void getScale` (`GLfloat *factor`) `const`
- `const GgQuaternion & getQuaternion` `() const`
- `const GgMatrix & getMatrix` `() const`
- `const GLfloat * get` `() const`

基底クラス `gg::GgQuaternion` に属する継承公開メンバ関数

- `GgQuaternion` `()=default`
- `constexpr GgQuaternion` (`GLfloat x, GLfloat y, GLfloat z, GLfloat w`)
- `constexpr GgQuaternion` (`GLfloat c`)
- `GgQuaternion` (`const GLfloat *a`)

- GgQuaternion (const GgVector &v)
- GgQuaternion (const GgQuaternion &q)=default
- GgQuaternion (GgQuaternion &&q)=default
- ~GgQuaternion ()=default
- GgQuaternion & operator= (const GgQuaternion &q)=default
- GgQuaternion & operator= (GgQuaternion &&q)=default
- GLfloat norm () const
- GgQuaternion & loadAdd (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadAdd (const GLfloat *a)
- GgQuaternion & loadAdd (const GgVector &v)
- GgQuaternion & loadAdd (const GgQuaternion &q)
- GgQuaternion & loadSubtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadSubtract (const GLfloat *a)
- GgQuaternion & loadSubtract (const GgVector &v)
- GgQuaternion & loadSubtract (const GgQuaternion &q)
- GgQuaternion & loadMultiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadMultiply (const GLfloat *a)
- GgQuaternion & loadMultiply (const GgVector &v)
- GgQuaternion & loadMultiply (const GgQuaternion &q)
- GgQuaternion & loadDivide (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- GgQuaternion & loadDivide (const GLfloat *a)
- GgQuaternion & loadDivide (const GgVector &v)
- GgQuaternion & loadDivide (const GgQuaternion &q)
- GgQuaternion add (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion add (const GLfloat *a) const
- GgQuaternion add (const GgVector &v) const
- GgQuaternion add (const GgQuaternion &q) const
- GgQuaternion subtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion subtract (const GLfloat *a) const
- GgQuaternion subtract (const GgVector &v) const
- GgQuaternion subtract (const GgQuaternion &q) const
- GgQuaternion multiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion multiply (const GLfloat *a) const
- GgQuaternion multiply (const GgVector &v) const
- GgQuaternion multiply (const GgQuaternion &q) const
- GgQuaternion divide (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- GgQuaternion divide (const GLfloat *a) const
- GgQuaternion divide (const GgVector &v) const
- GgQuaternion divide (const GgQuaternion &q) const
- GgQuaternion & operator= (const GLfloat *a)
- GgQuaternion & operator= (const GgVector &v)
- GgQuaternion & operator+= (const GLfloat *a)
- GgQuaternion & operator+= (const GgVector &v)
- GgQuaternion & operator+= (const GgQuaternion &q)
- GgQuaternion & operator-= (const GLfloat *a)
- GgQuaternion & operator-= (const GgVector &v)
- GgQuaternion & operator-= (const GgQuaternion &q)
- GgQuaternion & operator*= (const GLfloat *a)
- GgQuaternion & operator*= (const GgVector &v)
- GgQuaternion & operator*= (const GgQuaternion &q)
- GgQuaternion & operator/= (const GLfloat *a)
- GgQuaternion & operator/= (const GgVector &v)
- GgQuaternion & operator/= (const GgQuaternion &q)
- GgQuaternion operator+ (const GLfloat *a) const
- GgQuaternion operator+ (const GgVector &v) const

- `GgQuaternion operator+ (const GgQuaternion &q) const`
- `GgQuaternion operator- (const GLfloat *a) const`
- `GgQuaternion operator- (const GgVector &v) const`
- `GgQuaternion operator- (const GgQuaternion &q) const`
- `GgQuaternion operator* (const GLfloat *a) const`
- `GgQuaternion operator* (const GgVector &v) const`
- `GgQuaternion operator* (const GgQuaternion &q) const`
- `GgQuaternion operator/ (const GLfloat *a) const`
- `GgQuaternion operator/ (const GgVector &v) const`
- `GgQuaternion operator/ (const GgQuaternion &q) const`
- `GgQuaternion & loadMatrix (const GLfloat *a)`
- `GgQuaternion & loadMatrix (const GgMatrix &m)`
- `GgQuaternion & loadIdentity ()`
- `GgQuaternion & loadRotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a)`
- `GgQuaternion & loadRotate (const GLfloat *v, GLfloat a)`
- `GgQuaternion & loadRotate (const GLfloat *v)`
- `GgQuaternion & loadRotateX (GLfloat a)`
- `GgQuaternion & loadRotateY (GLfloat a)`
- `GgQuaternion & loadRotateZ (GLfloat a)`
- `GgQuaternion rotate (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const`
- `GgQuaternion rotate (const GLfloat *v, GLfloat a) const`
- `GgQuaternion rotate (const GLfloat *v) const`
- `GgQuaternion rotateX (GLfloat a) const`
- `GgQuaternion rotateY (GLfloat a) const`
- `GgQuaternion rotateZ (GLfloat a) const`
- `GgQuaternion & loadEuler (GLfloat heading, GLfloat pitch, GLfloat roll)`
- `GgQuaternion & loadEuler (const GLfloat *e)`
- `GgQuaternion euler (GLfloat heading, GLfloat pitch, GLfloat roll) const`
- `GgQuaternion euler (const GLfloat *e) const`
- `GgQuaternion euler (const GgVector &e) const`
- `GgQuaternion & loadSlerp (const GLfloat *a, const GLfloat *b, GLfloat t)`
- `GgQuaternion & loadSlerp (const GgQuaternion &q, const GgQuaternion &r, GLfloat t)`
- `GgQuaternion & loadSlerp (const GgQuaternion &q, const GLfloat *a, GLfloat t)`
- `GgQuaternion & loadSlerp (const GLfloat *a, const GgQuaternion &q, GLfloat t)`
- `GgQuaternion & loadNormalize (const GLfloat *a)`
- `GgQuaternion & loadNormalize (const GgQuaternion &q)`
- `GgQuaternion & loadConjugate (const GLfloat *a)`
- `GgQuaternion & loadConjugate (const GgQuaternion &q)`
- `GgQuaternion & loadInvert (const GLfloat *a)`
- `GgQuaternion & loadInvert (const GgQuaternion &q)`
- `GgQuaternion slerp (GLfloat *a, GLfloat t) const`
- `GgQuaternion slerp (const GgQuaternion &q, GLfloat t) const`
- `GgQuaternion normalize () const`
- `GgQuaternion conjugate () const`
- `GgQuaternion invert () const`
- `void get (GLfloat *a) const`
- `void getMatrix (GLfloat *a) const`
- `void getMatrix (GgMatrix &m) const`
- `GgMatrix getMatrix () const`
- `void getConjugateMatrix (GLfloat *a) const`
- `void getConjugateMatrix (GgMatrix &m) const`
- `GgMatrix getConjugateMatrix () const`

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & operator= (const **GgVector** &v)=default
- **GgVector** & operator= (**GgVector** &&v)=default
- constexpr **GgVector** operator+ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator+ (GLfloat c) const noexcept
- constexpr **GgVector** & operator+= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator+= (GLfloat c) noexcept
- constexpr **GgVector** operator- (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator- (GLfloat c) const noexcept
- constexpr **GgVector** & operator-= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator-= (GLfloat c) noexcept
- constexpr **GgVector** operator* (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator* (GLfloat c) const noexcept
- constexpr **GgVector** & operator*= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator*= (GLfloat c) noexcept
- constexpr **GgVector** operator/ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator/ (GLfloat c) const noexcept
- constexpr **GgVector** & operator/= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator/= (GLfloat c) noexcept
- GLfloat dot3 (const **GgVector** &v) const
- GLfloat length3 () const
- GLfloat distance3 (const **GgVector** &v) const
- **GgVector** normalize3 () const
- GLfloat dot4 (const **GgVector** &v) const
- GLfloat length4 () const
- GLfloat distance4 (const **GgVector** &v) const
- **GgVector** normalize4 () const

8.27.1 詳解

簡易トラックボール処理.

gg.h の 4818 行目に定義があります。

8.27.2 構築子と解体子

8.27.2.1 GgTrackball()

```
gg::GgTrackball::GgTrackball (
    const GgQuaternion & q = ggIdentityQuaternion()) [inline]
```

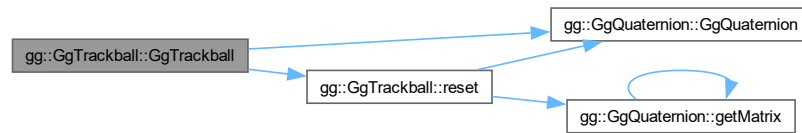
コンストラクタ.

引数

q	トラックボールの回転の初期値の四元数.
-----	---------------------

gg.h の 4833 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.27.2.2 ~GgTrackball()

```
gg::GgTrackball::~~GgTrackball () [default]
```

デストラクタ.

8.27.3 関数詳解

8.27.3.1 begin()

```
void gg::GgTrackball::begin (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を開始する.

引数

x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ開始時 (マウスボタンを押したとき) に呼び出す.

gg.cpp の 3470 行目に定義があります。

8.27.3.2 end()

```
void gg::GgTrackball::end (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を停止する。

引数

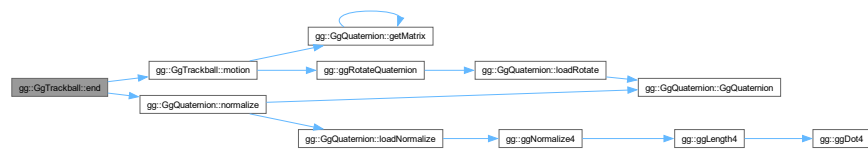
<i>x</i>	現在のマウスの <i>x</i> 座標.
<i>y</i>	現在のマウスの <i>y</i> 座標.

覚え書き

マウスのドラッグ終了時 (マウスボタンを離したとき) に呼び出す。

gg.cpp の 3536 行目に定義があります。

呼び出し関係図:



8.27.3.3 get()

```
const GLfloat * gg::GgTrackball::get () const [inline]
```

現在の回転の変換行列を取り出す。

戻り値

回転の変換を表す GLfloat 型の 16 要素の配列。

gg.h の 5011 行目に定義があります。

8.27.3.4 getMatrix()

```
const GgMatrix & gg::GgTrackball::getMatrix () const [inline]
```

現在の回転の変換行列を取り出す。

戻り値

回転の変換を表す GgMatrix 型の変換行列。

gg.h の 5001 行目に定義があります。

8.27.3.5 getQuaternion()

```
const GgQuaternion & gg::GgTrackball::getQuaternion () const [inline]
```

現在の回転の四元数を取り出す.

戻り値

回転の変換を表す Quaternion 型の四元数.

gg.h の 4991 行目に定義があります。

呼び出し関係図:



8.27.3.6 getScale() [1/3]

```
const GLfloat * gg::GgTrackball::getScale () const [inline]
```

トラックボール処理の換算係数を取り出す.

戻り値

トラックボールの換算係数のポインタ.

gg.h の 4960 行目に定義があります。

8.27.3.7 getScale() [2/3]

```
void gg::GgTrackball::getScale (
    GLfloat * factor) const [inline]
```

トラックボール処理の換算係数を取り出す.

引数

<i>factor</i>	トラックボールの換算係数を格納する 2 要素の配列.
---------------	----------------------------

gg.h の 4980 行目に定義があります。

8.27.3.8 getScale() [3/3]

```
const GLfloat gg::GgTrackball::getScale (
    int direction) const [inline]
```

トラックボール処理の換算係数を取り出す.

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

gg.h の 4970 行目に定義があります。

8.27.3.9 getStart() [1/3]

```
const GLfloat * gg::GgTrackball::getStart () const [inline]
```

トラックボール処理の開始位置を取り出す。

戻り値

トラックボールの開始位置のポインタ。

gg.h の 4931 行目に定義があります。

8.27.3.10 getStart() [2/3]

```
void gg::GgTrackball::getStart (
    GLfloat * position) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>position</i>	トラックボールの開始位置を格納する 2 要素の配列.
-----------------	----------------------------

gg.h の 4949 行目に定義があります。

8.27.3.11 getStart() [3/3]

```
const GLfloat & gg::GgTrackball::getStart (
    int direction) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

gg.h の 4939 行目に定義があります。

8.27.3.12 motion()

```
void gg::GgTrackball::motion (
    GLfloat x,
    GLfloat y)
```

回転の変換行列を計算する。

引数

x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ中に呼び出す.

`gg.cpp` の 3486 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.27.3.13 operator=()

```
GgTrackball & gg::GgTrackball::operator= (
    const GgQuaternion & q) [inline]
```

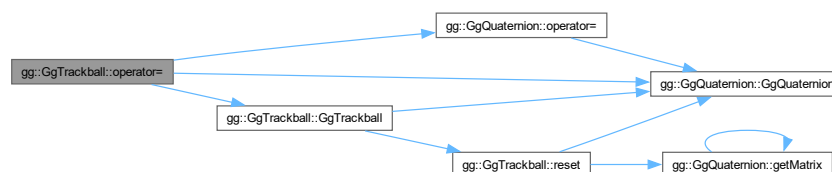
代入.

引数

q	トラックボールの回転の初期値の四元数.
-----	---------------------

`gg.h` の 4848 行目に定義があります。

呼び出し関係図:



8.27.3.14 region() [1/2]

```
void gg::GgTrackball::region (  
    GLfloat w,  
    GLfloat h)
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

[gg.cpp](#) の 3457 行目に定義があります。

被呼び出し関係図:



8.27.3.15 region() [2/2]

```
void gg::GgTrackball::region (  
    int w,  
    int h) [inline]
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す。

[gg.h](#) の [4874](#) 行目に定義があります。

呼び出し関係図:



8.27.3.16 reset()

```
void gg::GgTrackball::reset (
    const GgQuaternion & q = ggIdentityQuaternion())
```

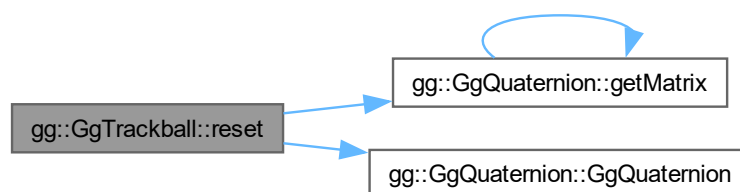
トラックボールをリセットする。

引数

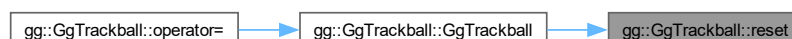
<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

[gg.cpp](#) の [3439](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.27.3.17 rotate()

```
void gg::GgTrackball::rotate (
    const GgQuaternion & q)
```

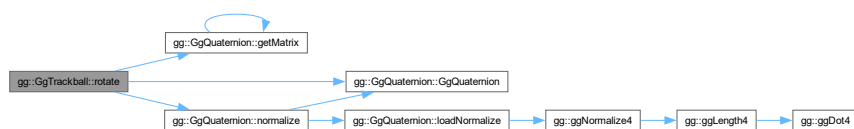
トラックボールの回転角を修正する。

引数

<i>q</i>	修正分の回転角の四元数.
----------	--------------

gg.cpp の 3515 行目に定義があります。

呼び出し関係図:



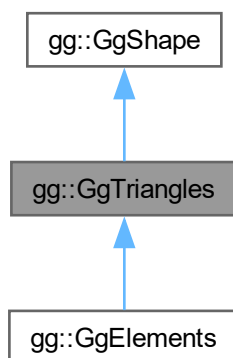
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

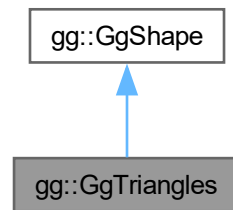
8.28 gg::GgTriangles クラス

```
#include <gg.h>
```

gg::GgTriangles の継承関係図



gg::GgTriangles 連携図



公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

8.28.1 詳解

三角形で表した形状データ (Arrays 形式).

[gg.h](#) の 6804 行目に定義があります。

8.28.2 構築子と解体子

8.28.2.1 GgTriangles() [1/2]

```
gg::GgTriangles::GgTriangles (
    GLenum mode = GL_TRIANGLES) [inline]
```

コンストラクタ.

引数

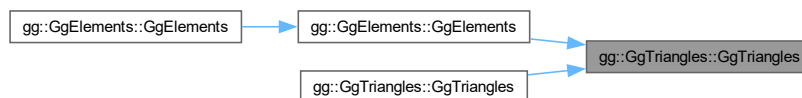
<i>mode</i>	描画する基本図形の種類.
-------------	--------------

gg.h の 6817 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.28.2.2 GgTriangles() [2/2]

```

gg::GgTriangles::GgTriangles (
    const GgVertex * vert,
    GLsizei count,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

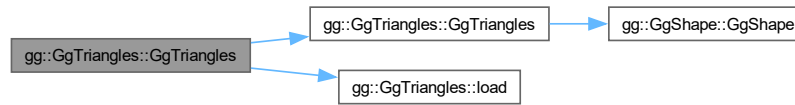
コンストラクタ.

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>count</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6830 行目に定義があります。

呼び出し関係図:



8.28.2.3 ~GgTriangles()

```
virtual gg::GgTriangles::~~GgTriangles () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の [6844](#) 行目に定義があります。

8.28.3 関数詳解

8.28.3.1 draw()

```
void gg::GgTriangles::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

三角形の描画.

引数

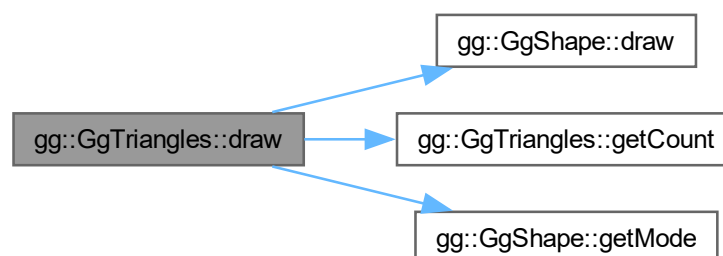
<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

[gg::GgShape](#) を再実装しています。

[gg::GgElements](#) で再実装されています。

[gg.cpp](#) の [5290](#) 行目に定義があります。

呼び出し関係図:



8.28.3.2 getBuffer()

```
const GLuint & gg::GgTriangles::getBuffer () const [inline]
```

頂点属性を格納した頂点バッファオブジェクト名を取り出す。

戻り値

この図形の頂点属性を格納した頂点バッファオブジェクト名。

gg.h の 6863 行目に定義があります。

8.28.3.3 getCount()

```
const GLsizei & gg::GgTriangles::getCount () const [inline]
```

データの数を取り出す。

戻り値

この図形の頂点属性の数 (頂点数)。

gg.h の 6853 行目に定義があります。

被呼び出し関係図:



8.28.3.4 load()

```
void gg::GgTriangles::load (
    const GgVertex * vert,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点属性を格納する。

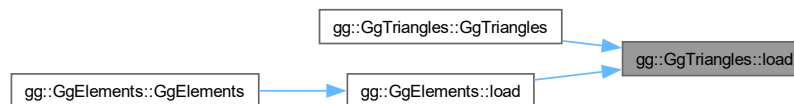
引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ。
-------------	-------------------------

<i>count</i>	頂点のデータの数 (頂点数).
<i>usage</i>	バッファオブジェクトの使い方.

[gg.cpp](#) の 5271 行目に定義があります。

被呼び出し関係図:



8.28.3.5 send()

```
void gg::GgTriangles::send (
    const GgVertex * vert,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点属性を転送する。

引数

<i>vert</i>	転送元の頂点属性が格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6875 行目に定義があります。

被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.29 gg::GgUniformBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgUniformBuffer](#) ()
- [GgUniformBuffer](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- [GgUniformBuffer](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgUniformBuffer](#) ()
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLuint first, GLsizei count) const
- void [unmap](#) () const
- void [load](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [load](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [send](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [fill](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [read](#) (GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.29.1 詳解

```
template<typename T>
class gg::GgUniformBuffer< T >
```

ユニフォームバッファオブジェクト.

覚え書き

ユニフォーム変数を格納するバッファオブジェクトの基底クラス.

[gg.h](#) の [6094](#) 行目に定義があります。

8.29.2 構築子と解体子

8.29.2.1 GgUniformBuffer() [1/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer () [inline]
```

コンストラクタ.

[gg.h](#) の [6104](#) 行目に定義があります。

8.29.2.2 GgUniformBuffer() [2/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

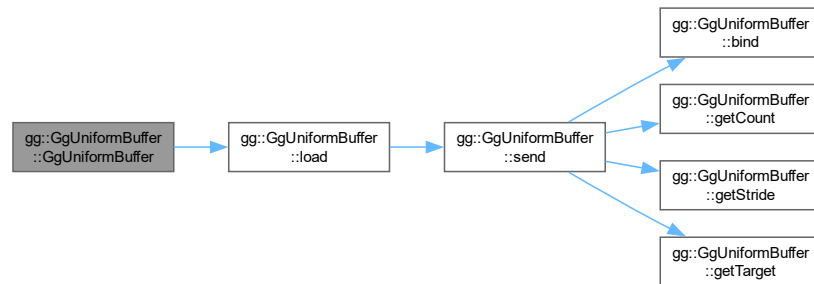
ユニフォームバッファオブジェクトのブロックごとにデータを転送するコンストラクタ.

引数

<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6115 行目に定義があります。

呼び出し関係図:



8.29.2.3 GgUniformBuffer() [3/3]

```

template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

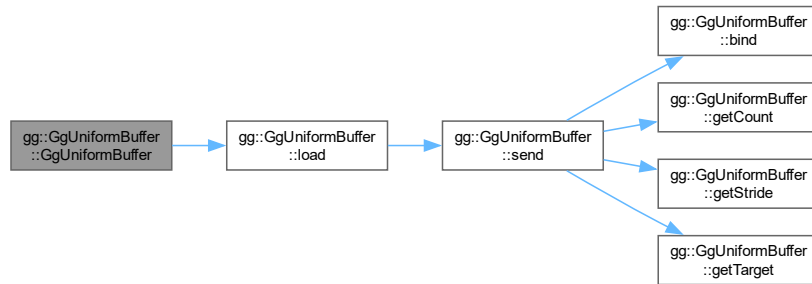
ユニフォームバッファオブジェクトの全ブロックに同じデータを格納するコンストラクタ。

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6127 行目に定義があります。

呼び出し関係図:



8.29.2.4 ~GgUniformBuffer()

```
template<typename T>
virtual gg::GgUniformBuffer< T >::~~GgUniformBuffer () [inline], [virtual]
```

デストラクタ.

gg.h の 6135 行目に定義があります。

8.29.3 関数詳解

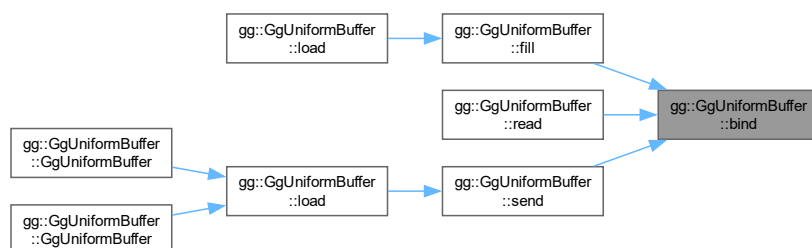
8.29.3.1 bind()

```
template<typename T>
void gg::GgUniformBuffer< T >::bind () const [inline]
```

ユニフォームバッファオブジェクトを結合する.

gg.h の 6182 行目に定義があります。

被呼び出し関係図:



8.29.3.2 copy()

```
template<typename T>
void gg::GgUniformBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
```

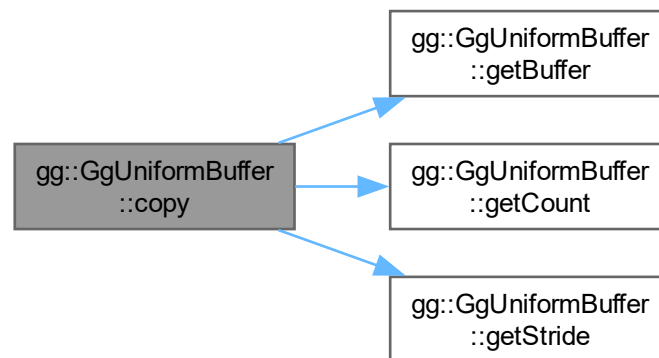
別のバッファオブジェクトからデータを複写する.

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (buffer) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6393 行目に定義があります.

呼び出し関係図:



8.29.3.3 fill()

```
template<typename T>
void gg::GgUniformBuffer< T >::fill (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

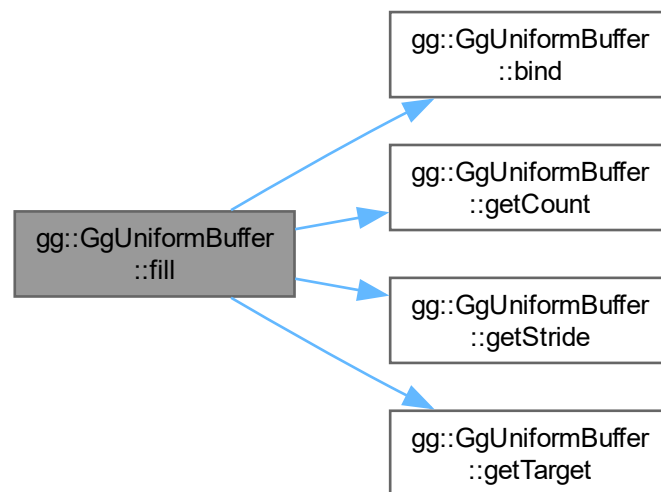
ユニフォームバッファオブジェクトの全ブロックのメンバーを同じデータを格納する.

引数

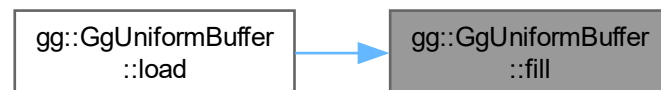
<i>data</i>	格納するデータ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

gg.h の 6310 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.29.3.4 getBuffer()

```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getBuffer () const [inline]
```

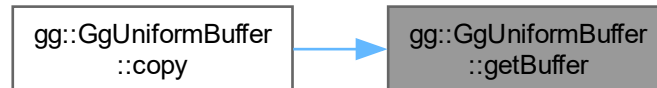
ユニフォームバッファオブジェクト名を取り出す。

戻り値

このユニフォームバッファオブジェクト名.

gg.h の 6174 行目に定義があります。

被呼び出し関係図:



8.29.3.5 getCount()

```
template<typename T>
const GLsizei & gg::GgUniformBuffer< T >::getCount () const [inline]
```

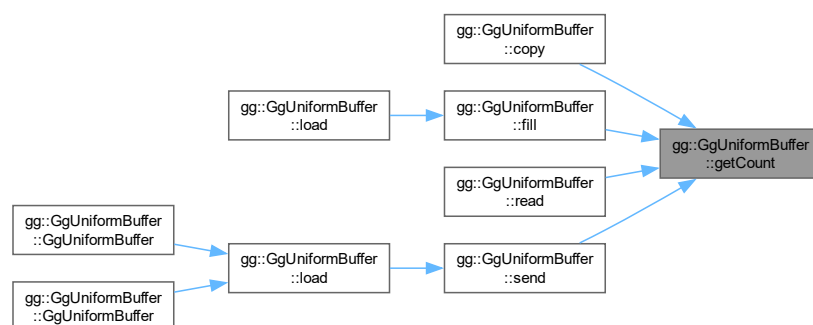
データの数を取り出す.

戻り値

このユニフォームバッファオブジェクトのデータの数.

gg.h の 6164 行目に定義があります。

被呼び出し関係図:



8.29.3.6 getStride()

```
template<typename T>
GLsizeiptr gg::GgUniformBuffer< T >::getStride () const [inline]
```

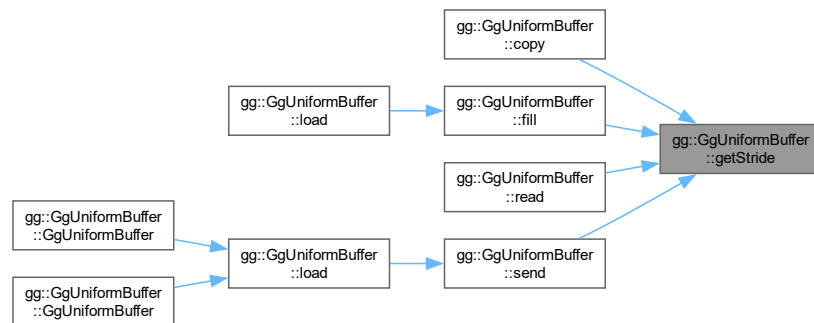
ユニフォームバッファオブジェクトのアライメントを考慮したデータの間隔を取り出す。

戻り値

このユニフォームバッファオブジェクトのデータの間隔。

gg.h の 6154 行目に定義があります。

被呼び出し関係図:



8.29.3.7 getTarget()

```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getTarget () const [inline]
```

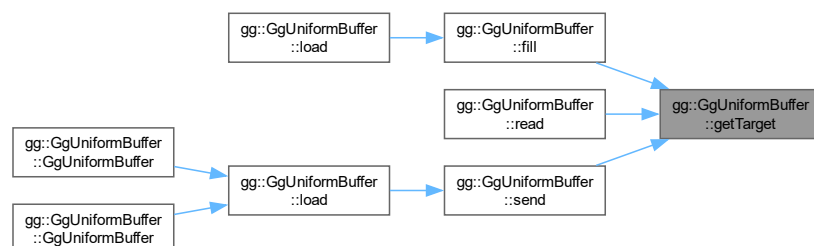
ユニフォームバッファオブジェクトのターゲットを取り出す。

戻り値

このユニフォームバッファオブジェクトのターゲット。

gg.h の 6144 行目に定義があります。

被呼び出し関係図:



8.29.3.8 load() [1/2]

```
template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

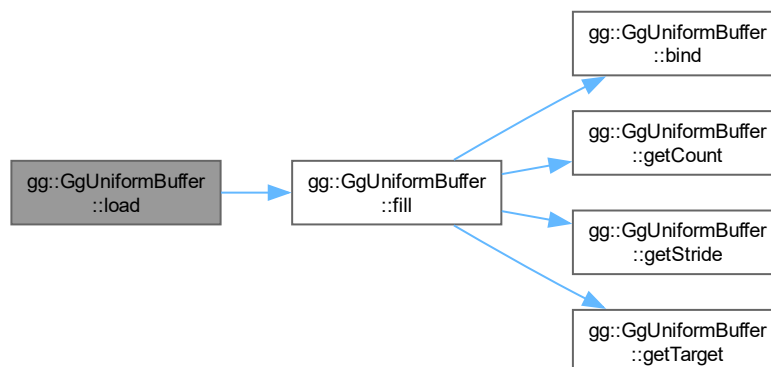
ユニフォームバッファオブジェクトを確保して全てのブロックに同じデータを格納する。

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6251 行目に定義があります。

呼び出し関係図:



8.29.3.9 load() [2/2]

```
template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

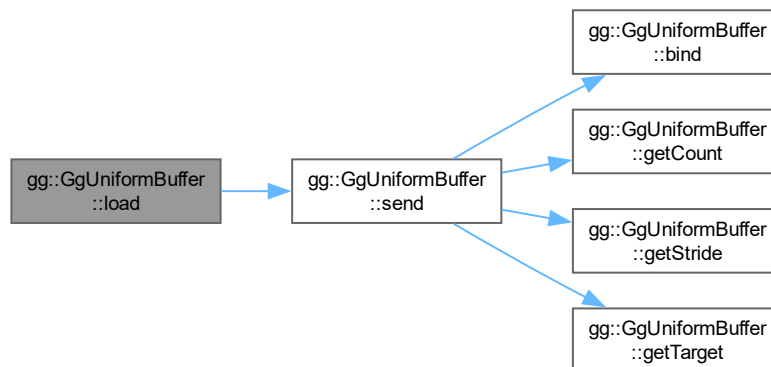
ユニフォームバッファオブジェクトを確保してブロックごとにデータを転送する。

引数

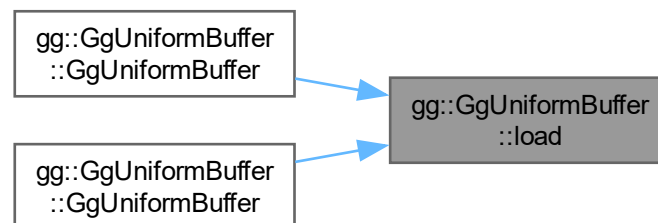
<i>data</i>	データが格納されている領域の先頭のポインタ (<code>nullptr</code> ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

[gg.h](#) の 6232 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.29.3.10 map() [1/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map () const [inline]
```

ユニフォームバッファオブジェクトをマップする.

戻り値

マップしたメモリの先頭のポインタ.

[gg.h](#) の 6200 行目に定義があります。

8.29.3.11 map() [2/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

ユニフォームバッファオブジェクトの指定した範囲をマップする。

引数

<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 6212 行目に定義があります。

8.29.3.12 read()

```
template<typename T>
void gg::GgUniformBuffer< T >::read (
    GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

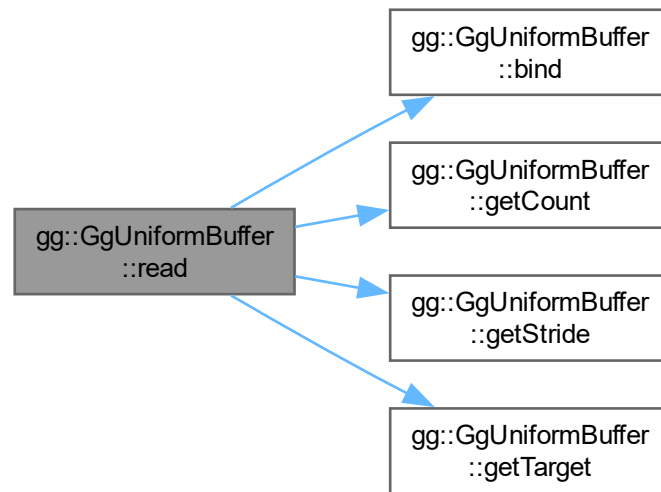
ユニフォームバッファオブジェクトからデータを抽出する。

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>offset</i>	抽出元のユニフォームバッファオブジェクトのメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	抽出するデータの一個あたりのバイト数.
<i>first</i>	抽出元のユニフォームバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	抽出するデータの数 (0 ならユニフォームバッファオブジェクト全体).

gg.h の 6345 行目に定義があります。

呼び出し関係図:



8.29.3.13 send()

```

template<typename T>
void gg::GgUniformBuffer< T >::send (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
  
```

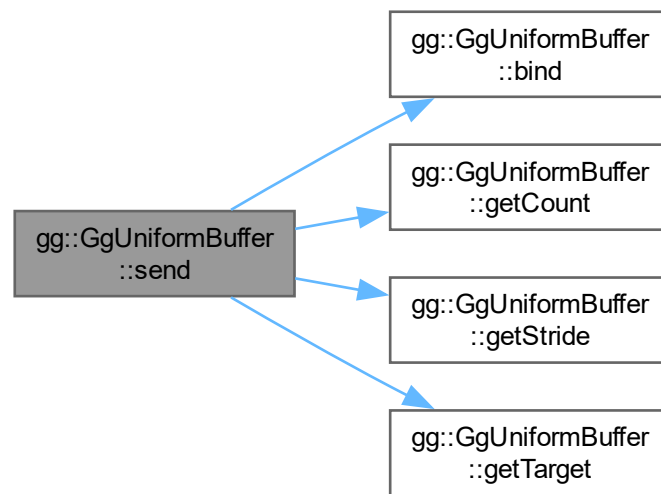
ユニフォームバッファオブジェクトを確保してユニフォームバッファオブジェクトのブロックごとのメンバを同じデータで埋める。

引数

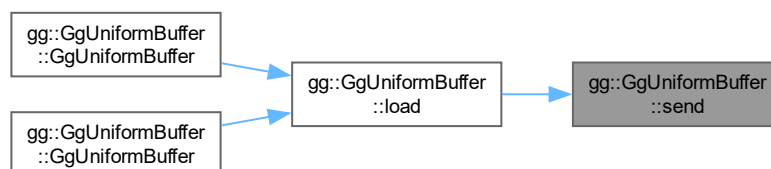
<i>data</i>	データが格納されている領域の先頭のポインタ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

[gg.h](#) の 6272 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.29.3.14 unbind()

```
template<typename T>
void gg::GgUniformBuffer< T >::unbind () const [inline]
```

ユニフォームバッファオブジェクトを解放する。

gg.h の 6190 行目に定義があります。

8.29.3.15 unmap()

```
template<typename T>
void gg::GgUniformBuffer< T >::unmap () const [inline]
```

バッファオブジェクトをアンマップする。

[gg.h](#) の 6220 行目に定義があります。

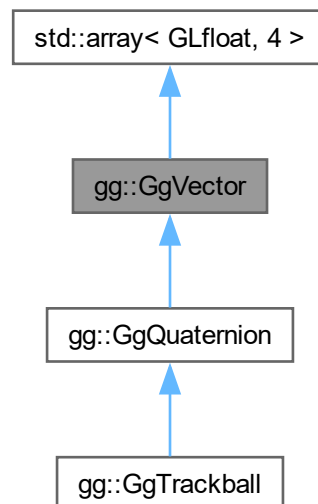
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

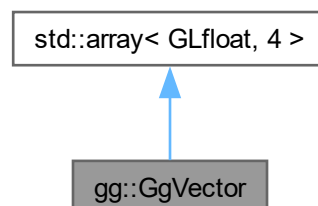
8.30 gg::GgVector クラス

```
#include <gg.h>
```

gg::GgVector の継承関係図



gg::GgVector 連携図



公開メンバ関数

- `GgVector` ()=default
- constexpr `GgVector` (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr `GgVector` (const GLfloat *a)
- constexpr `GgVector` (GLfloat c)
- `GgVector` (const GgVector &v)=default
- `GgVector` (GgVector &&v)=default
- `~GgVector` ()=default
- `GgVector & operator=` (const `GgVector` &v)=default
- `GgVector & operator=` (GgVector &&v)=default
- constexpr `GgVector operator+` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator+` (GLfloat c) const noexcept
- constexpr `GgVector & operator+=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator+=` (GLfloat c) noexcept
- constexpr `GgVector operator-` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator-` (GLfloat c) const noexcept
- constexpr `GgVector & operator-=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator-=` (GLfloat c) noexcept
- constexpr `GgVector operator*` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator*` (GLfloat c) const noexcept
- constexpr `GgVector & operator*=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator*=` (GLfloat c) noexcept
- constexpr `GgVector operator/` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator/` (GLfloat c) const noexcept
- constexpr `GgVector & operator/=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator/=` (GLfloat c) noexcept
- GLfloat `dot3` (const `GgVector` &v) const
- GLfloat `length3` () const
- GLfloat `distance3` (const `GgVector` &v) const
- `GgVector normalize3` () const
- GLfloat `dot4` (const `GgVector` &v) const
- GLfloat `length4` () const
- GLfloat `distance4` (const `GgVector` &v) const
- `GgVector normalize4` () const

8.30.1 詳解

単精度実数の4要素のベクトル。

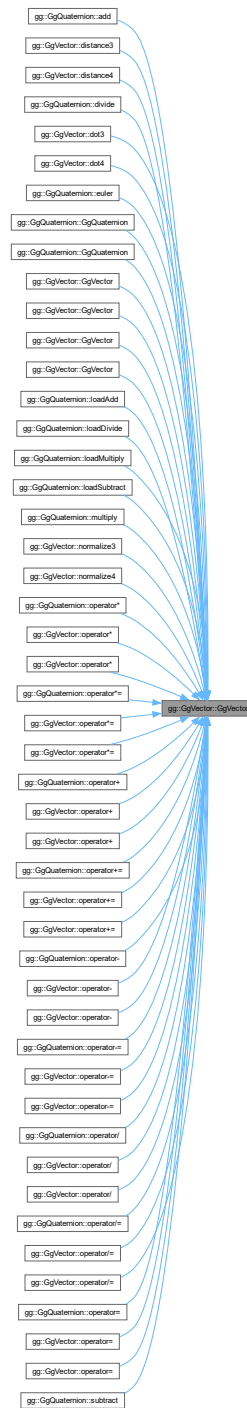
`gg.h` の 1589 行目に定義があります。

8.30.2 構築子と解体子

8.30.2.1 `GgVector()` [1/6]

```
gg::GgVector::GgVector () [default]
```

コンストラクタ. 被呼び出し関係図:



8.30.2.2 GgVector() [2/6]

```

gg::GgVector::GgVector (
    GLfloat v0,
    GLfloat v1,

```



```
GLfloat v2,
GLfloat v3) [inline], [constexpr]
```

コンストラクタ.

引数

<i>v0</i>	GLfloat 型の値.
<i>v1</i>	GLfloat 型の値.
<i>v2</i>	GLfloat 型の値.
<i>v3</i>	GLfloat 型の値.

gg.h の 1606 行目に定義があります。

8.30.2.3 GgVector() [3/6]

```
gg::GgVector::GgVector (
    const GLfloat * a) [inline], [constexpr]
```

コンストラクタ.

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

gg.h の 1616 行目に定義があります。

呼び出し関係図:



8.30.2.4 GgVector() [4/6]

```
gg::GgVector::GgVector (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

c	GLfloat 型の値.
---	--------------

[gg.h](#) の 1626 行目に定義があります。

呼び出し関係図:



8.30.2.5 GgVector() [5/6]

```
gg::GgVector::GgVector (  
    const GgVector & v) [default]
```

コピーコンストラクタ.

引数

v	コピー元のベクトル.
---	------------

呼び出し関係図:



8.30.2.6 GgVector() [6/6]

```
gg::GgVector::GgVector (  
    GgVector && v) [default]
```

ムーブコンストラクタ.

引数

<code>v</code>	ムーブ元のベクトル.
----------------	------------

呼び出し関係図:



8.30.2.7 ~GgVector()

```
gg::GgVector::~~GgVector () [default]
```

デストラクタ.

8.30.3 関数詳解

8.30.3.1 distance3()

```
GLfloat gg::GgVector::distance3 (
    const GgVector & v) const [inline]
```

`GgVector` 型の 3 要素の距離.

引数

<code>v</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

オブジェクトと `v` の 3 要素の距離.

`gg.h` の 1895 行目に定義があります。

呼び出し関係図:



8.30.3.2 distance4()

```
GLfloat gg::GgVector::distance4 (  
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の距離.

引数

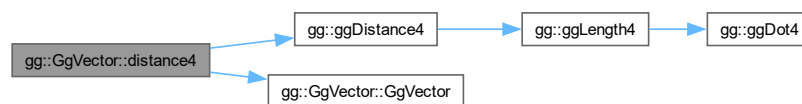
<i>v</i>	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと *v* の 4 要素の距離.

gg.h の 1939 行目に定義があります。

呼び出し関係図:



8.30.3.3 dot3()

```
GLfloat gg::GgVector::dot3 (  
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の内積.

引数

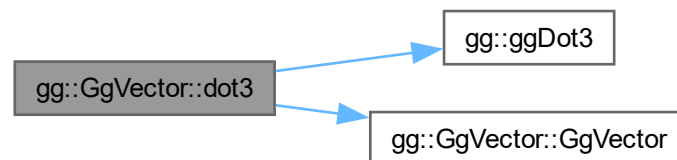
<i>v</i>	GgVector 型のベクトル.
----------	-------------------------

戻り値

オブジェクトと v のそれぞれの 3 要素の内積.

gg.h の 1874 行目に定義があります。

呼び出し関係図:



8.30.3.4 dot4()

```
GLfloat gg::GgVector::dot4 (  
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の内積.

引数

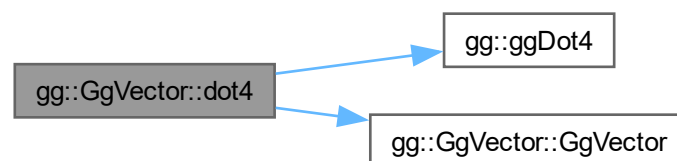
v	GgVector 型のベクトル.
-----	------------------

戻り値

オブジェクトと v のそれぞれの 4 要素の内積.

gg.h の 1918 行目に定義があります。

呼び出し関係図:



8.30.3.5 length3()

```
GLfloat gg::GgVector::length3 () const [inline]
```

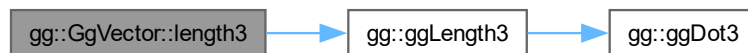
GgVector 型の 3 要素の長さ.

戻り値

オブジェクトの 3 要素の長さ.

gg.h の 1884 行目に定義があります。

呼び出し関係図:



8.30.3.6 length4()

```
GLfloat gg::GgVector::length4 () const [inline]
```

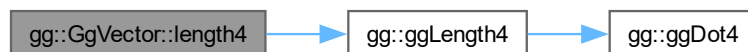
GgVector 型の 4 要素の長さ.

戻り値

オブジェクトの 4 要素の長さ.

gg.h の 1928 行目に定義があります。

呼び出し関係図:



8.30.3.7 normalize3()

`GgVector` `gg::GgVector::normalize3 () const [inline]`

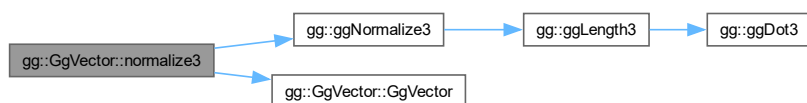
`GgVector` 型の 3 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

`gg.h` の 1905 行目に定義があります。

呼び出し関係図:



8.30.3.8 normalize4()

`GgVector` `gg::GgVector::normalize4 () const [inline]`

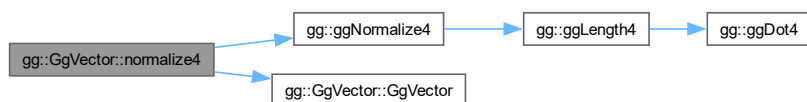
`GgVector` 型の 4 要素の正規化.

戻り値

GLfloat 型の 4 要素の配列変数.

`gg.h` の 1949 行目に定義があります。

呼び出し関係図:



8.30.3.9 operator*() [1/2]

`GgVector` `gg::GgVector::operator* (const GgVector & v) const [inline], [constexpr], [noexcept]`

`GgVector` 型の積を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの積のオブジェクト.

`gg.h` の 1770 行目に定義があります.

呼び出し関係図:



8.30.3.10 operator*() [2/2]

```
GgVector gg::GgVector::operator* (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを乗じた積を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素に `c` を乗じたオブジェクト.

`gg.h` の 1781 行目に定義があります.

呼び出し関係図:



8.30.3.11 operator*=() [1/2]

```
GgVector & gg::GgVector::operator*= (
    const GgVector & v) [inline], [constexpr], [noexcept]
```

GgVector 型を乗算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ乗算したオブジェクトの参照.

`gg.h` の 1792 行目に定義があります。

呼び出し関係図:



8.30.3.12 `operator*=()` [2/2]

```
GgVector & gg::GgVector::operator*= (  
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを乗じる.

引数

<code>c</code>	<code>GLfloat</code> 型の値.
----------------	---------------------------

戻り値

オブジェクトの各要素に `c` を乗じたオブジェクトの参照.

`gg.h` の 1807 行目に定義があります。

呼び出し関係図:



8.30.3.13 operator+() [1/2]

```
GgVector gg::GgVector::operator+ (  
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の和を返す.

引数

<code>v</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの和のオブジェクト.

`gg.h` の 1666 行目に定義があります.

呼び出し関係図:



8.30.3.14 operator+() [2/2]

```
GgVector gg::GgVector::operator+ (  
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを足した和を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の値.
----------------	---------------------------

戻り値

`a` の各要素に `b` を足した和のオブジェクト.

`gg.h` の 1677 行目に定義があります.

呼び出し関係図:



8.30.3.15 operator+=() [1/2]

```
GgVector & gg::GgVector::operator+= (  
    const GgVector & v) [inline], [constexpr], [noexcept]
```

GgVector 型を加算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ加算したオブジェクトの参照.

`gg.h` の 1688 行目に定義があります。

呼び出し関係図:



8.30.3.16 operator+=() [2/2]

```
GgVector & gg::GgVector::operator+= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを加算する.

引数

<code>c</code>	<code>GLfloat</code> 型の値.
----------------	---------------------------

戻り値

オブジェクトの各要素に `c` を足したオブジェクトの参照.

`gg.h` の 1703 行目に定義があります。

呼び出し関係図:



8.30.3.17 operator-() [1/2]

```
GgVector gg::GgVector::operator- (
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の差を返す.

引数

v	GgVector 型の変数.
----------	-----------------------

戻り値

オブジェクトの各要素と **v** の各要素の要素ごとの差のオブジェクト.

gg.h の 1718 行目に定義があります。

呼び出し関係図:



8.30.3.18 operator-() [2/2]

```
GgVector gg::GgVector::operator- (  
    GLfloat c) const [inline], [constexpr], [noexcept]
```

GgVector 型の各要素からスカラーを引いた差を返す.

引数

c	GLfloat 型の変数.
----------	---------------

戻り値

オブジェクトの各要素から **c** を引いたオブジェクト.

gg.h の 1729 行目に定義があります。

呼び出し関係図:



8.30.3.19 operator-=() [1/2]

```
GgVector & gg::GgVector::operator-= (  
    const GgVector & v) [inline], [constexpr], [noexcept]
```

GgVector 型を減算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ減算したオブジェクトの参照.

`gg.h` の 1740 行目に定義があります。

呼び出し関係図:



8.30.3.20 operator-=() [2/2]

```
GgVector & gg::GgVector::operator-= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素からスカラーを引く.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素から `c` を引いたオブジェクトの参照.

`gg.h` の 1755 行目に定義があります。

呼び出し関係図:



8.30.3.21 operator/() [1/2]

```
GgVector gg::GgVector::operator/ (
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の商を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素を `v` の各要素で要素ごとに割った結果のオブジェクト.

`gg.h` の 1822 行目に定義があります.

呼び出し関係図:



8.30.3.22 operator/() [2/2]

```
GgVector gg::GgVector::operator/ (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素をスカラーで割った商を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素を `c` で割ったオブジェクト.

`gg.h` の 1833 行目に定義があります.

呼び出し関係図:



8.30.3.23 operator/=() [1/2]

```
GgVector & gg::GgVector::operator/= (
    const GgVector & v) [inline], [constexpr], [noexcept]
```

GgVector 型を除算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素を `v` の各要素でそれぞれ除算したオブジェクトの参照.

`gg.h` の 1844 行目に定義があります。

呼び出し関係図:



8.30.3.24 operator/=() [2/2]

```
GgVector & gg::GgVector::operator/= (  
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素をスカラーで割る.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素から `c` で割ったオブジェクトの参照.

`gg.h` の 1859 行目に定義があります。

呼び出し関係図:



8.30.3.25 operator=() [1/2]

```
GgVector & gg::GgVector::operator= (  
    const GgVector & v) [default]
```

代入演算子. 呼び出し関係図:



8.30.3.26 operator=() [2/2]

```
GgVector & gg::GgVector::operator= (  
    GgVector && v) [default]
```

ムーブ代入演算子. 呼び出し関係図:



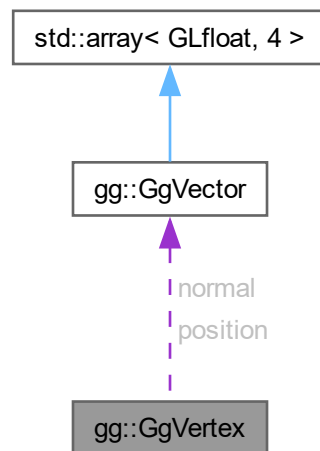
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.31 gg::GgVertex 構造体

```
#include <gg.h>
```

gg::GgVertex 連携図



公開メンバ関数

- `GgVertex ()`
- `GgVertex (const GgVector &pos, const GgVector &norm)`
- `GgVertex (GLfloat px, GLfloat py, GLfloat pz, GLfloat nx, GLfloat ny, GLfloat nz)`
- `GgVertex (const GLfloat *pos, const GLfloat *norm)`

公開変数類

- `GgVector position`
位置.
- `GgVector normal`
法線.

8.31.1 詳解

三角形の頂点データ.

`gg.h` の 6743 行目に定義があります。

8.31.2 構築子と解体子

8.31.2.1 GgVertex() [1/4]

```
gg::GgVertex::GgVertex () [inline]
```

コンストラクタ.

gg.h の 6754 行目に定義があります。

被呼び出し関係図:



8.31.2.2 GgVertex() [2/4]

```
gg::GgVertex::GgVertex (
    const GgVector & pos,
    const GgVector & norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	GgVector 型の位置データ.
<i>norm</i>	GgVector 型の法線データ.

gg.h の 6764 行目に定義があります。

8.31.2.3 GgVertex() [3/4]

```
gg::GgVertex::GgVertex (
    GLfloat px,
    GLfloat py,
    GLfloat pz,
    GLfloat nx,
    GLfloat ny,
    GLfloat nz) [inline]
```

コンストラクタ.

引数

<i>px</i>	GgVector 型の位置データの x 成分.
<i>py</i>	GgVector 型の位置データの y 成分.
<i>pz</i>	GgVector 型の位置データの z 成分.
<i>nx</i>	GgVector 型の法線データの x 成分.
<i>ny</i>	GgVector 型の法線データの y 成分.
<i>nz</i>	GgVector 型の法線データの z 成分.

gg.h の 6780 行目に定義があります。

8.31.2.4 GgVertex() [4/4]

```
gg::GgVertex::GgVertex (
    const GLfloat * pos,
    const GLfloat * norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	3 要素の GLfloat 型の位置データのポインタ.
<i>norm</i>	3 要素の GLfloat 型の法線データのポインタ.

gg.h の 6795 行目に定義があります。

呼び出し関係図:



8.31.3 メンバ詳解

8.31.3.1 normal

GgVector gg::GgVertex::normal

法線.

gg.h の 6749 行目に定義があります。

8.31.3.2 position

`GgVector` gg::GgVertex::position

位置.

gg.h の 6746 行目に定義があります。

この構造体詳解は次のファイルから抽出されました:

- [gg.h](#)

8.32 gg::GgVertexArray クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgVertexArray` (GLenum mode=0)
- `GgVertexArray` (const GgVertexArray &array)=delete
- `GgVertexArray` (GgVertexArray &&o) noexcept
- virtual `~GgVertexArray` ()
- `GgVertexArray & operator=` (const `GgVertexArray` &array)=delete
- `GgVertexArray & operator=` (`GgVertexArray` &&o) noexcept
- const GLuint & `get` () const
- void `bind` () const

8.32.1 詳解

頂点配列 クラス.

gg.h の 6424 行目に定義があります。

8.32.2 構築子と解体子

8.32.2.1 GgVertexArray() [1/3]

```
gg::GgVertexArray::GgVertexArray (
    GLenum mode = 0) [inline]
```

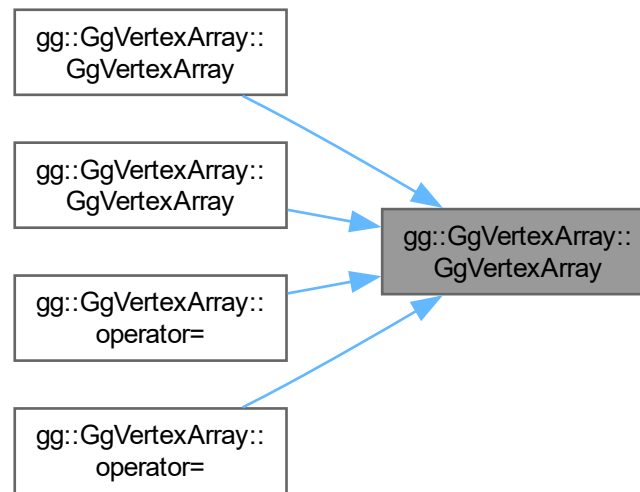
コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6436 行目に定義があります。

被呼び出し関係図:



8.32.2.2 GgVertexArray() [2/3]

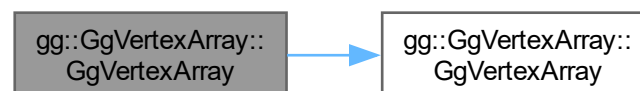
```
gg::GgVertexArray::GgVertexArray (
    const GgVertexArray & array) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>array</i>	コピー元の頂点配列オブジェクト.
--------------	------------------

呼び出し関係図:



8.32.2.3 GgVertexArray() [3/3]

```
gg::GgVertexArray::GgVertexArray (  
    GgVertexArray && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

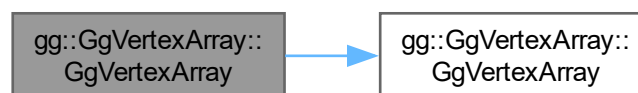
o	ムーブ元の頂点配列オブジェクト.
---	------------------

覚え書き

頂点配列オブジェクト名の所有権を移動し, ムーブ元の頂点配列オブジェクト名を 0 にする. ムーブ元のデストラクタが同じ頂点配列オブジェクトを削除しないようにするため.

gg.h の 6459 行目に定義があります。

呼び出し関係図:



8.32.2.4 ~GgVertexArray()

```
virtual gg::GgVertexArray::~~GgVertexArray () [inline], [virtual]
```

デストラクタ.

gg.h の 6468 行目に定義があります。

8.32.3 関数詳解

8.32.3.1 bind()

```
void gg::GgVertexArray::bind () const [inline]
```

頂点配列オブジェクトを結合する.

gg.h の 6522 行目に定義があります。

8.32.3.2 get()

```
const GLuint & gg::GgVertexArray::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

gg.h の 6514 行目に定義があります。

8.32.3.3 operator=() [1/2]

```
GgVertexArray & gg::GgVertexArray::operator= (  
    const GgVertexArray & array) [delete]
```

代入演算子は使用しない.

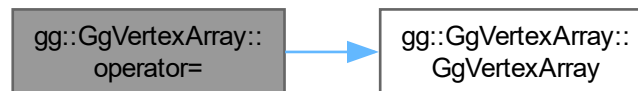
引数

<code>array</code>	代入元の頂点配列オブジェクト.
--------------------	-----------------

戻り値

代入後のこの頂点配列オブジェクトの参照.

呼び出し関係図:



8.32.3.4 operator=() [2/2]

```
GgVertexArray & gg::GgVertexArray::operator= (
    GgVertexArray && o) [inline], [noexcept]
```

ムーブ代入演算子.

引数

<code>o</code>	ムーブ代入元の頂点配列オブジェクト.
----------------	--------------------

戻り値

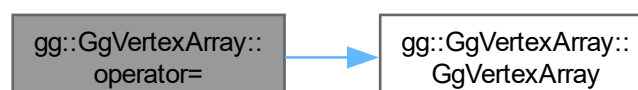
ムーブ代入後のこの頂点配列オブジェクトの参照.

覚え書き

元々保持していた頂点配列オブジェクトを削除してから所有権を移動し, ムーブ代入元の頂点配列オブジェクト名を 0 にする.

[gg.h](#) の 6493 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

8.33 Intrinsics 構造体

```
#include <Intrinsics.h>
```

公開メンバ関数

- [Intrinsics](#) ()=default
- [Intrinsics](#) (const std::array< float, 2 > &[fov](#), const std::array< float, 2 > &[center](#), const std::array< int, 2 > [size](#), double [fps](#))
- [Intrinsics](#) (const picojson::object &object)
- void [setFov](#) (float focal)
- void [setFov](#) (float fovx, float fovy)
- void [setCenter](#) (float x, float y)
- void [setSize](#) (int width, int height)
- void [setFps](#) (double frequency)

公開変数類

- std::array< float, 2 > [fov](#) { 50.03f, 38.58f }
キャプチャデバイスのレンズの縦横の画角
- std::array< float, 2 > [center](#) { 0.0f, 0.0f }
キャプチャデバイスのレンズの中心 (主点) の位置
- std::array< int, 2 > [size](#) { 640, 480 }
キャプチャデバイスの解像度
- double [fps](#) { 30.0 }
キャプチャデバイスのフレームレート

静的公開変数類

- static constexpr auto [sensorSize](#) { 35.0f }
展開時のセンサーサイズ

8.33.1 詳解

キャプチャデバイス固有のパラメータ

[Intrinsics.h](#) の 20 行目に定義があります。

8.33.2 構築子と解体子

8.33.2.1 Intrinsics() [1/3]

```
Intrinsics::Intrinsics () [default]
```

キャプチャデバイス固有のパラメータの構造体のデフォルトコンストラクタ

覚え書き

構成ファイルが読めなかったときにしか使わない

8.33.2.2 Intrinsic() [2/3]

```
Intrinsics::Intrinsics (
    const std::array< float, 2 > & fov,
    const std::array< float, 2 > & center,
    const std::array< int, 2 > size,
    double fps) [inline]
```

パラメータを指定するときに使う キャプチャデバイス固有のパラメータの構造体のコンストラクタ

引数

<i>fov</i>	キャプチャデバイスのレンズの縦横の画角
<i>center</i>	キャプチャデバイスのレンズの中心 (主点) の位置
<i>size</i>	キャプチャデバイスの解像度
<i>fps</i>	キャプチャデバイスのフレームレート

[Intrinsics.h](#) の 50 行目に定義があります。

8.33.2.3 Intrinsic() [3/3]

```
Intrinsics::Intrinsics (
    const picojson::object & object)
```

構成ファイルを読み込むときに使う キャプチャデバイス固有のパラメータの構造体のコンストラクタ

[Intrinsics.cpp](#) の 14 行目に定義があります。

呼び出し関係図:



8.33.3 関数詳解

8.33.3.1 setCenter()

```
void Intrinsics::setCenter (
    float x,
    float y) [inline]
```

キャプチャデバイスのレンズの中心の位置を変更する

引数

<i>x</i>	キャプチャデバイスのレンズの中心の位置の <i>x</i> 座標
<i>y</i>	キャプチャデバイスのレンズの中心の位置の <i>y</i> 座標

[Intrinsics.h](#) の 94 行目に定義があります。

8.33.3.2 setFov() [1/2]

```
void Intrinsics::setFov (
    float focal)
```

スクリーンの高さをもとにしてキャプチャデバイスのレンズの縦横の画角を変更する

引数

<i>focal</i>	撮像面の対角線長が <i>sensorSize</i> のときの焦点距離
--------------	--------------------------------------

[Intrinsics.cpp](#) の 32 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.33.3.3 setFov() [2/2]

```
void Intrinsics::setFov (
    float fovx,
    float fovy) [inline]
```

キャプチャデバイスのレンズの縦横の画角を変更する

引数

<i>fovx</i>	キャプチャデバイスのレンズの fovx 方向の画角
<i>fovy</i>	キャプチャデバイスのレンズの fovy 方向の画角

[Intrinsics.h](#) の 81 行目に定義があります。

8.33.3.4 setFps()

```
void Intrinsics::setFps (  
    double frequency) [inline]
```

キャプチャデバイスのフレームレートを変更する

引数

<i>frequency</i>	フレームレート
------------------	---------

[Intrinsics.h](#) の 121 行目に定義があります。

8.33.3.5 setSize()

```
void Intrinsics::setSize (  
    int width,  
    int height) [inline]
```

キャプチャデバイスの解像度を変更する

引数

<i>width</i>	キャプチャデバイスのフレームの横の画素数
<i>height</i>	キャプチャデバイスのフレームの縦の画素数

[Intrinsics.h](#) の 108 行目に定義があります。

8.33.4 メンバ詳解

8.33.4.1 center

```
std::array<float, 2> Intrinsics::center { 0.0f, 0.0f }
```

キャプチャデバイスのレンズの中心 (主点) の位置

[Intrinsics.h](#) の 26 行目に定義があります。

8.33.4.2 fov

```
std::array<float, 2> Intrinsic::fov { 50.03f, 38.58f }
```

キャプチャデバイスのレンズの縦横の画角

[Intrinsic.h](#) の 23 行目に定義があります。

8.33.4.3 fps

```
double Intrinsic::fps { 30.0 }
```

キャプチャデバイスのフレームレート

[Intrinsic.h](#) の 32 行目に定義があります。

8.33.4.4 sensorSize

```
auto Intrinsic::sensorSize { 35.0f } [static], [constexpr]
```

展開時のセンサーサイズ

[Intrinsic.h](#) の 66 行目に定義があります。

8.33.4.5 size

```
std::array<int, 2> Intrinsic::size { 640, 480 }
```

キャプチャデバイスの解像度

[Intrinsic.h](#) の 29 行目に定義があります。

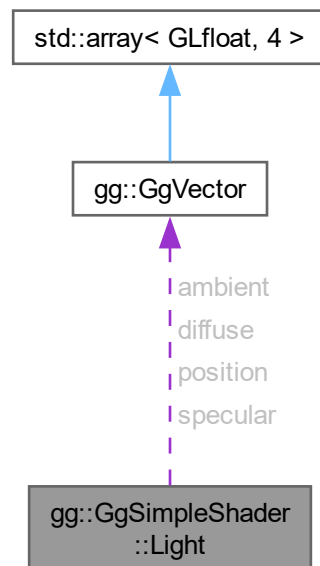
この構造体詳解は次のファイルから抽出されました:

- [Intrinsic.h](#)
- [Intrinsic.cpp](#)

8.34 gg::GgSimpleShader::Light 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Light 連携図



公開変数類

- **GgVector ambient**
光源強度の環境光成分.
- **GgVector diffuse**
光源強度の拡散反射光成分.
- **GgVector specular**
光源強度の鏡面反射光成分.
- **GgVector position**
光源の位置.

8.34.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データ.

gg.h の 7767 行目に定義があります。

8.34.2 メンバ詳解

8.34.2.1 ambient

`GgVector gg::GgSimpleShader::Light::ambient`

光源強度の環境光成分.

`gg.h` の 7769 行目に定義があります。

8.34.2.2 diffuse

`GgVector gg::GgSimpleShader::Light::diffuse`

光源強度の拡散反射光成分.

`gg.h` の 7770 行目に定義があります。

8.34.2.3 position

`GgVector gg::GgSimpleShader::Light::position`

光源の位置.

`gg.h` の 7772 行目に定義があります。

8.34.2.4 specular

`GgVector gg::GgSimpleShader::Light::specular`

光源強度の鏡面反射光成分.

`gg.h` の 7771 行目に定義があります。

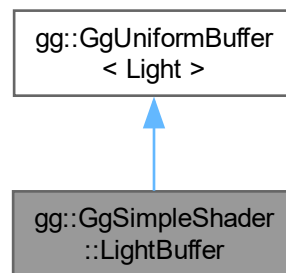
この構造体詳解は次のファイルから抽出されました:

- `gg.h`

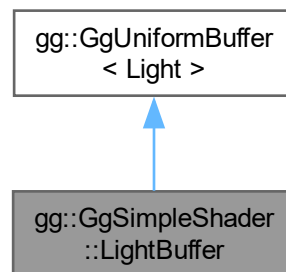
8.35 gg::GgSimpleShader::LightBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::LightBuffer の継承関係図



gg::GgSimpleShader::LightBuffer 連携図



公開メンバ関数

- [LightBuffer](#) (const [Light](#) *light=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- [LightBuffer](#) (const [Light](#) &light, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- [LightBuffer](#) ([GgVector](#) ambient, [GgVector](#) diffuse, [GgVector](#) specular, [GgVector](#) position, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual [~LightBuffer](#) ()
- void [loadAmbient](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadAmbient](#) (const [GgVector](#) &ambient, GLint first=0, GLsizei count=1) const
- void [loadAmbient](#) (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void [loadDiffuse](#) (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void [loadDiffuse](#) (const [GgVector](#) &diffuse, GLint first=0, GLsizei count=1) const

- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GgVector &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadColor` (const Light &color, GLint first=0, GLsizei count=1) const
- void `loadPosition` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GgVector &position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GLfloat *position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GgVector *position, GLint first=0, GLsizei count=1) const
- void `load` (const Light *light, GLint first=0, GLsizei count=1) const
- void `load` (const Light &light, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Light >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const Light *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.35.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データのユニフォームバッファオブジェクト。

`gg.h` の 7778 行目に定義があります。

8.35.2 構築子と解体子

8.35.2.1 LightBuffer() [1/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light * light = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

デフォルトコンストラクタ。

引数

<i>light</i>	GgSimpleShader::Light 型の光源データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7790 行目に定義があります。

呼び出し関係図:



8.35.2.2 LightBuffer() [2/3]

```

gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light & light,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

同じデータで埋めるコンストラクタ.

引数

<i>light</i>	GgSimpleShader::Light 型の光源データ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7806 行目に定義があります。

呼び出し関係図:



8.35.2.3 LightBuffer() [3/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GgVector position,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

同じ引数で埋めるコンストラクタ。

引数

<i>ambient</i>	光源強度の環境光成分.
<i>diffuse</i>	光源強度の拡散反射光成分.
<i>specular</i>	光源強度の鏡面反射光成分.
<i>position</i>	光源の位置.
<i>count</i>	バッファ中の <code>GgSimpleShader::Light</code> 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

`gg.h` の 7825 行目に定義があります。

呼び出し関係図:



8.35.2.4 ~LightBuffer()

```
virtual gg::GgSimpleShader::LightBuffer::~~LightBuffer () [inline], [virtual]
```

デストラクタ。

`gg.h` の 7840 行目に定義があります。

8.35.3 関数詳解

8.35.3.1 load() [1/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light & light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

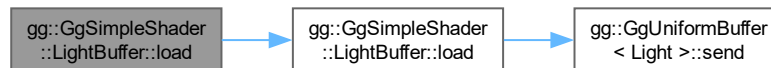
光源の色と位置を設定する。

引数

<i>light</i>	光源の特性の GgSimpleShader::Light 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 8032 行目に定義があります。

呼び出し関係図:



8.35.3.2 load() [2/2]

```

void gg::GgSimpleShader::LightBuffer::load (
    const Light * light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

光源の色と位置を設定する。

引数

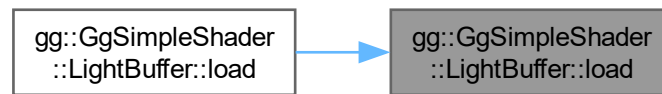
<i>light</i>	光源の特性の GgSimpleShader::Light 構造体のポインタ.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 8020 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.35.3.3 loadAmbient() [1/3]

```

void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
  
```

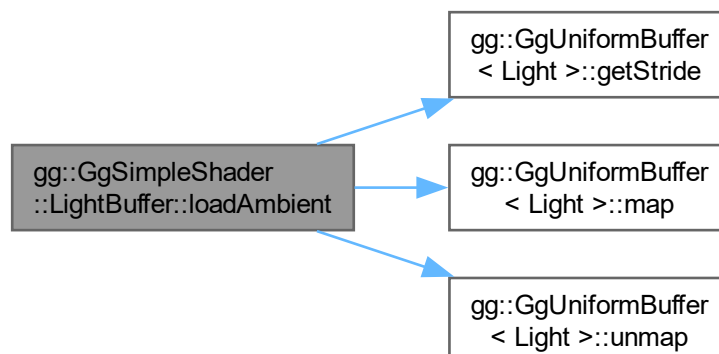
光源の強度の環境光成分を設定する.

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5618 行目に定義があります。

呼び出し関係図:



8.35.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7875 行目に定義があります。

呼び出し関係図:



8.35.3.5 loadAmbient() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

光源の強度の環境光成分を設定する。

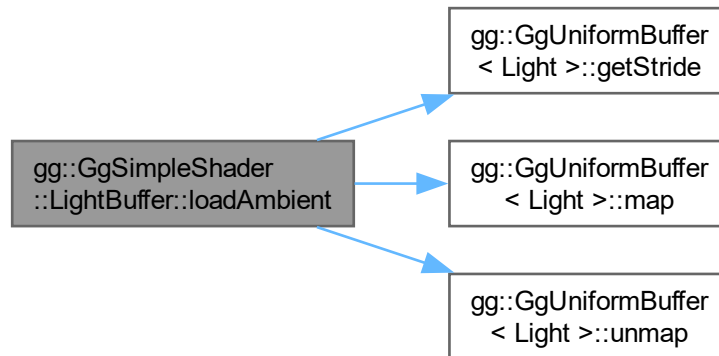
引数

<i>r</i>	光源の強度の環境光成分の赤成分.
<i>g</i>	光源の強度の環境光成分の緑成分.
<i>b</i>	光源の強度の環境光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.

<i>count</i>	値を設定する光源データの数, デフォルトは 1.
--------------	--------------------------

gg.cpp の 5592 行目に定義があります。

呼び出し関係図:



8.35.3.6 loadColor()

```

void gg::GgSimpleShader::LightBuffer::loadColor (
    const Light & color,
    GLint first = 0,
    GLsizei count = 1) const
  
```

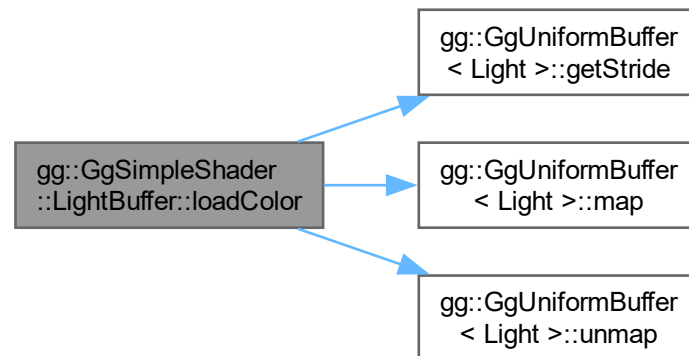
光源の色を設定するが位置は変更しない。

引数

<i>color</i>	光源の特性の <code>GgSimpleShader::Light</code> 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5745 行目に定義があります。

呼び出し関係図:



8.35.3.7 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
```

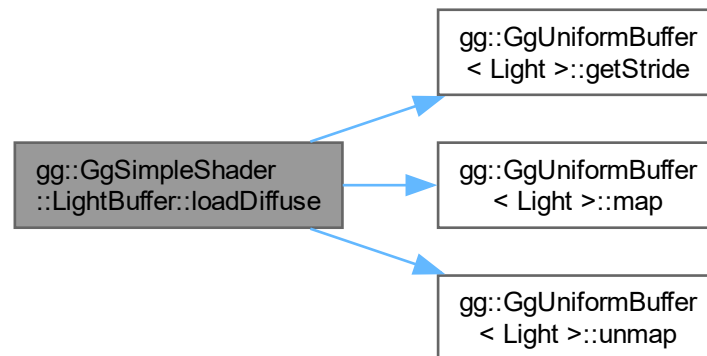
光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5670 行目に定義があります。

呼び出し関係図:



8.35.3.8 loadDiffuse() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7912 行目に定義があります。

呼び出し関係図:



8.35.3.9 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

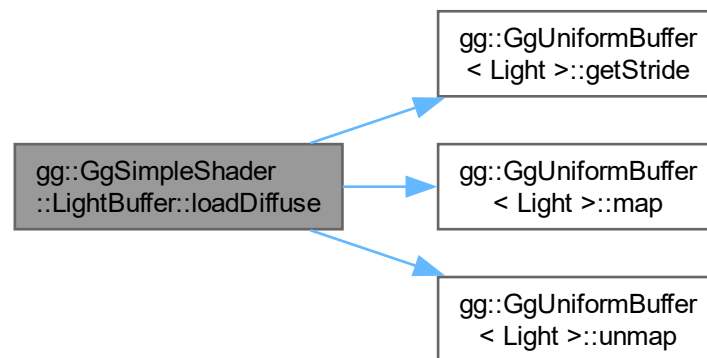
光源の強度の拡散反射光成分を設定する。

引数

<i>r</i>	光源の強度の拡散反射光成分の赤成分.
<i>g</i>	光源の強度の拡散反射光成分の緑成分.
<i>b</i>	光源の強度の拡散反射光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5644 行目に定義があります。

呼び出し関係図:



8.35.3.10 loadPosition() [1/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector & position,
    GLint first = 0,
    GLsizei count = 1) const
```

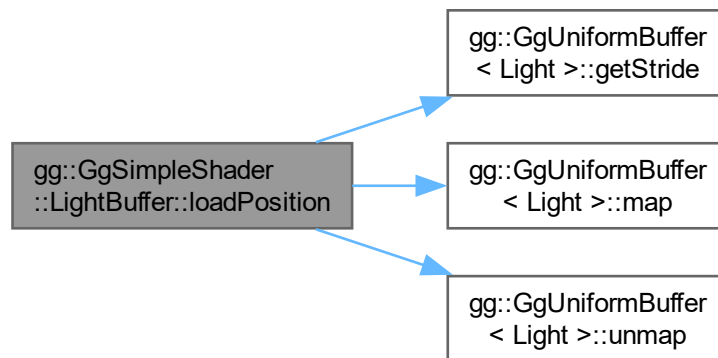
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5799 行目に定義があります。

呼び出し関係図:



8.35.3.11 loadPosition() [2/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

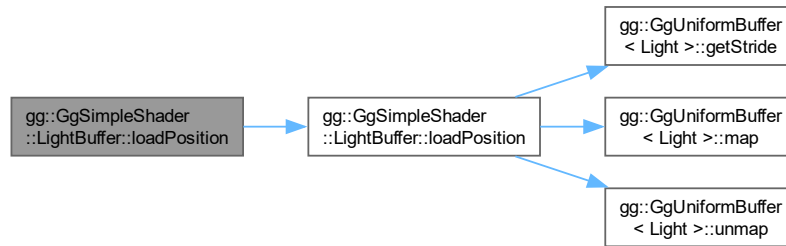
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の配列.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 8008 行目に定義があります。

呼び出し関係図:



8.35.3.12 loadPosition() [3/4]

```

void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GLfloat * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

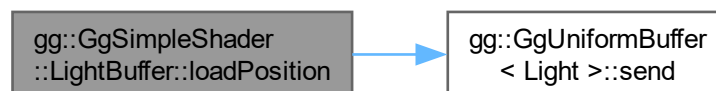
光源の位置を設定する.

引数

<i>position</i>	光源の位置の同次座標を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7995 行目に定義があります。

呼び出し関係図:



8.35.3.13 loadPosition() [4/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

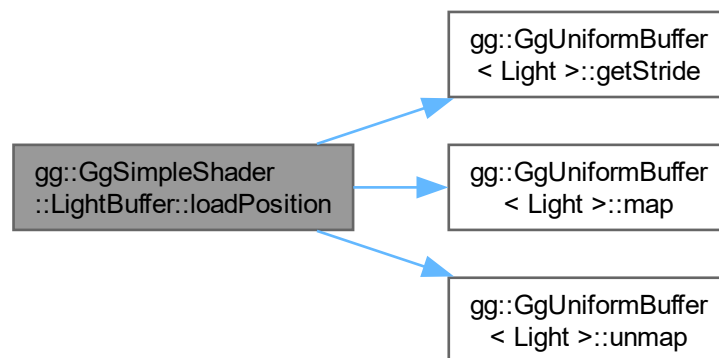
光源の位置を設定する。

引数

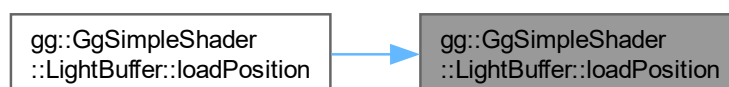
<i>x</i>	光源の位置の <i>x</i> 座標.
<i>y</i>	光源の位置の <i>y</i> 座標.
<i>z</i>	光源の位置の <i>z</i> 座標.
<i>w</i>	光源の位置の <i>w</i> 座標, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5773 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.35.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

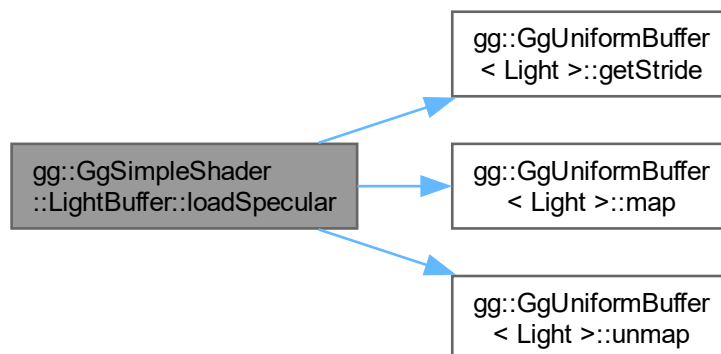
光源の強度の鏡面反射光成分を設定する。

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5722 行目に定義があります。

呼び出し関係図:



8.35.3.15 loadSpecular() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の鏡面反射光成分を設定する。

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GLfloat 型の 4 要素の配列変数.
-----------------	---

<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7949 行目に定義があります。

呼び出し関係図:



8.35.3.16 loadSpecular() [3/3]

```

void gg::GgSimpleShader::LightBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

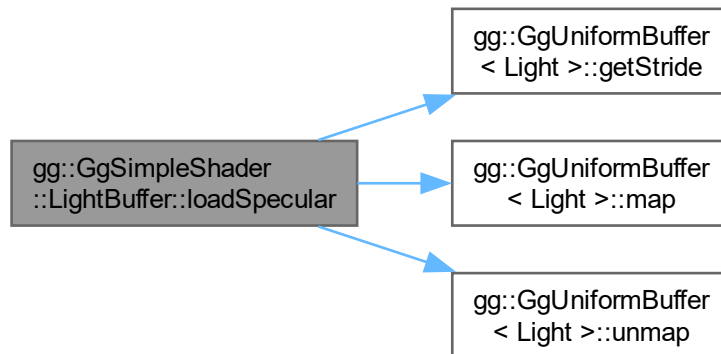
光源の強度の鏡面反射光成分を設定する。

引数

<i>r</i>	光源の強度の鏡面反射光成分の赤成分.
<i>g</i>	光源の強度の鏡面反射光成分の緑成分.
<i>b</i>	光源の強度の鏡面反射光成分の青成分.
<i>a</i>	光源の強度の鏡面反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5696 行目に定義があります。

呼び出し関係図:



8.35.3.17 select()

```
void gg::GgSimpleShader::LightBuffer::select (
    GLint i = 0) const [inline]
```

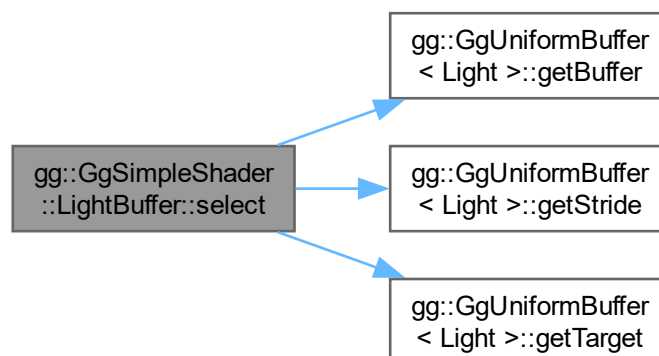
光源を選択する.

引数

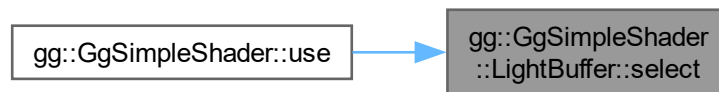
<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

[gg.h](#) の 8042 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



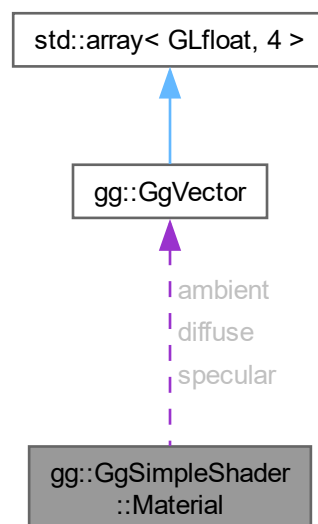
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

8.36 gg::GgSimpleShader::Material 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Material 連携図



公開変数類

- [GgVector ambient](#)
環境光に対する反射係数.
- [GgVector diffuse](#)
拡散反射係数.
- [GgVector specular](#)
鏡面反射係数.
- [GLfloat shininess](#)
輝き係数.

8.36.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データ.

[gg.h](#) の 8053 行目に定義があります。

8.36.2 メンバ詳解

8.36.2.1 ambient

```
GgVector gg::GgSimpleShader::Material::ambient
```

環境光に対する反射係数.

[gg.h](#) の 8055 行目に定義があります。

8.36.2.2 diffuse

```
GgVector gg::GgSimpleShader::Material::diffuse
```

拡散反射係数.

[gg.h](#) の 8056 行目に定義があります。

8.36.2.3 shininess

```
GLfloat gg::GgSimpleShader::Material::shininess
```

輝き係数.

[gg.h](#) の 8058 行目に定義があります。

8.36.2.4 specular

GgVector gg::GgSimpleShader::Material::specular

鏡面反射係数.

gg.h の 8057 行目に定義があります。

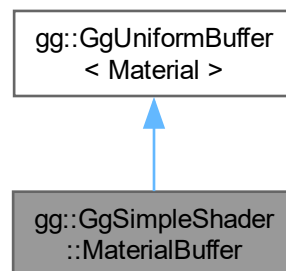
この構造体詳解は次のファイルから抽出されました:

- gg.h

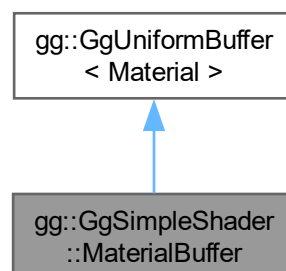
8.37 gg::GgSimpleShader::MaterialBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::MaterialBuffer の継承関係図



gg::GgSimpleShader::MaterialBuffer 連携図



公開メンバ関数

- `MaterialBuffer` (const `Material` *material=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `MaterialBuffer` (const `Material` &material, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `MaterialBuffer` (`GgVector` ambient, `GgVector` diffuse, `GgVector` specular, GLfloat shininess, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual `~MaterialBuffer` ()
- void `loadAmbient` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const `GgVector` &ambient, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const `GgVector` &diffuse, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const `GgVector` &color, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const GLfloat *color, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const `GgVector` &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadShininess` (GLfloat shininess, GLint first=0, GLsizei count=1) const
- void `loadShininess` (const GLfloat *shininess, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` *material, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` &material, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Material >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const `Material` *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(`Material`), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(`Material`), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(`Material`), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

8.37.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データのユニフォームバッファオブジェクト。

`gg.h` の 8064 行目に定義があります。

8.37.2 構築子と解体子

8.37.2.1 MaterialBuffer() [1/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material * material = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

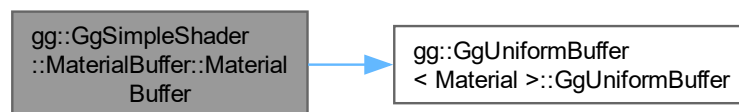
デフォルトコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 8076 行目に定義があります。

呼び出し関係図:



8.37.2.2 MaterialBuffer() [2/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material & material,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

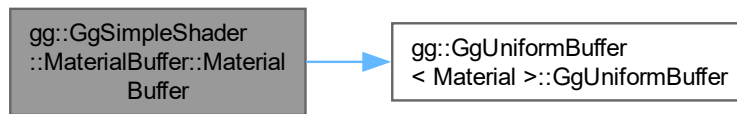
同じデータで埋めるコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 8092 行目に定義があります。

呼び出し関係図:



8.37.2.3 MaterialBuffer() [3/3]

```

gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GLfloat shininess,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

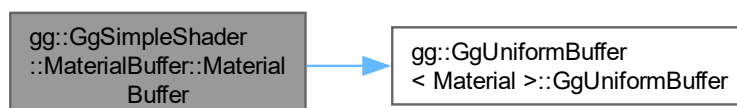
同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	環境光に対する反射係数.
<i>diffuse</i>	拡散反射係数.
<i>specular</i>	鏡面反射係数.
<i>shininess</i>	輝き係数.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 8111 行目に定義があります。

呼び出し関係図:



8.37.2.4 ~MaterialBuffer()

```
virtual gg::GgSimpleShader::MaterialBuffer::~MaterialBuffer () [inline], [virtual]
```

デストラクタ.

gg.h の 8126 行目に定義があります。

8.37.3 関数詳解

8.37.3.1 load() [1/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material & material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

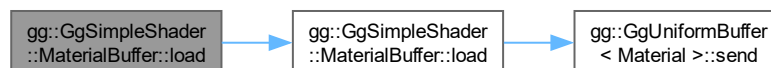
材質を設定する.

引数

<i>material</i>	材質の特性の GgSimpleShader::Material 構造体.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 8311 行目に定義があります。

呼び出し関係図:



8.37.3.2 load() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material * material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

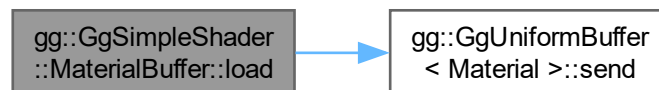
材質を設定する.

引数

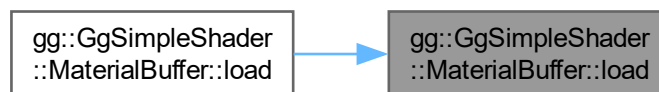
<i>material</i>	材質の特性の GgSimpleShader::Material 構造体のポインタ.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8299 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.37.3.3 loadAmbient() [1/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
  
```

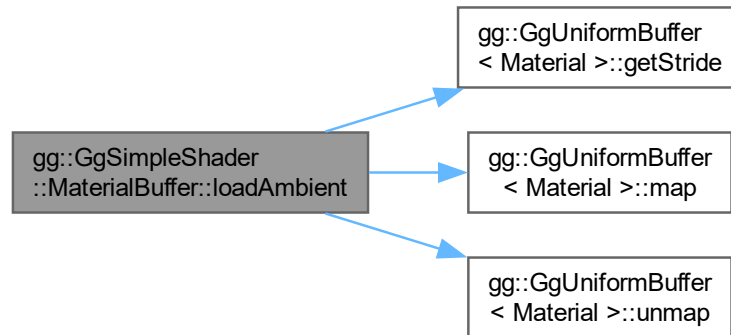
三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する。

引数

<i>ambient</i>	環境光に対する反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5851 行目に定義があります。

呼び出し関係図:



8.37.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

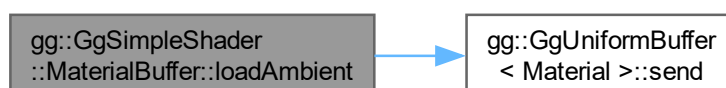
環境光に対する反射係数を設定する。

引数

<i>ambient</i>	環境光に対する反射係数を格納した GLfloat 型の 4 要素の配列変数。
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8161 行目に定義があります。

呼び出し関係図:



8.37.3.5 loadAmbient() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

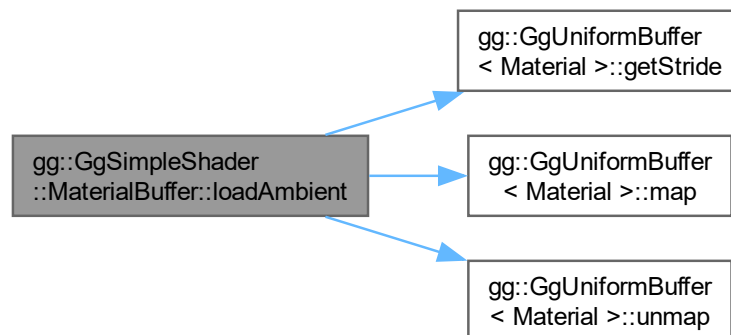
環境光に対する反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数の赤成分.
<i>g</i>	環境光に対する反射係数の緑成分.
<i>b</i>	環境光に対する反射係数の青成分.
<i>a</i>	環境光に対する反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5825 行目に定義があります。

呼び出し関係図:



8.37.3.6 loadAmbientAndDiffuse() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GgVector & color,
    GLint first = 0,
    GLsizei count = 1) const
```

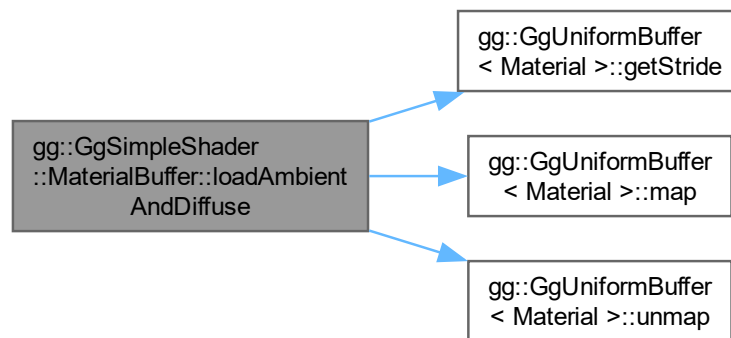
三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5955 行目に定義があります。

呼び出し関係図:



8.37.3.7 loadAmbientAndDiffuse() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GLfloat * color,
    GLint first = 0,
    GLsizei count = 1) const
```

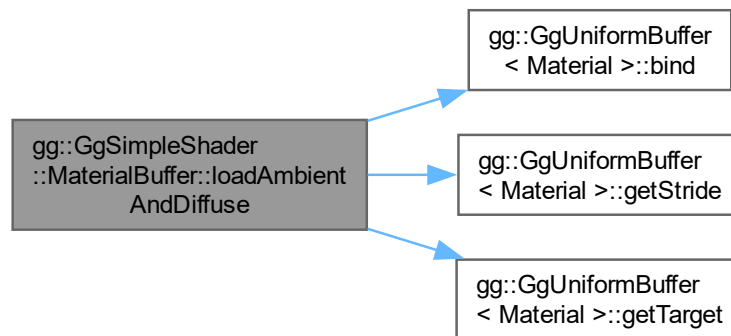
環境光に対する反射係数と拡散反射係数を設定する.

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5978 行目に定義があります。

呼び出し関係図:



8.37.3.8 loadAmbientAndDiffuse() [3/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

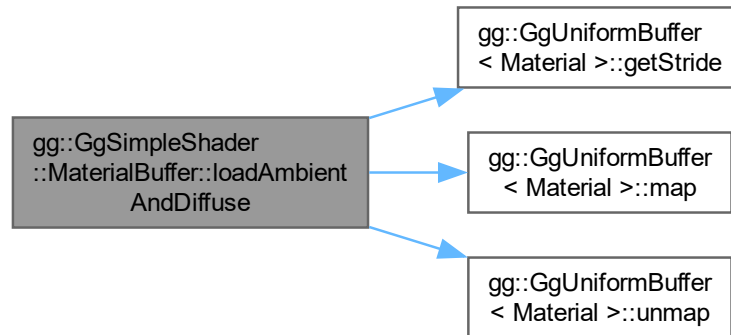
環境光に対する反射係数と拡散反射係数を設定する.

引数

<i>r</i>	環境光に対する反射係数と拡散反射係数の赤成分.
<i>g</i>	環境光に対する反射係数と拡散反射係数の緑成分.
<i>b</i>	環境光に対する反射係数と拡散反射係数の青成分.
<i>a</i>	環境光に対する反射係数と拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5929 行目に定義があります。

呼び出し関係図:



8.37.3.9 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
```

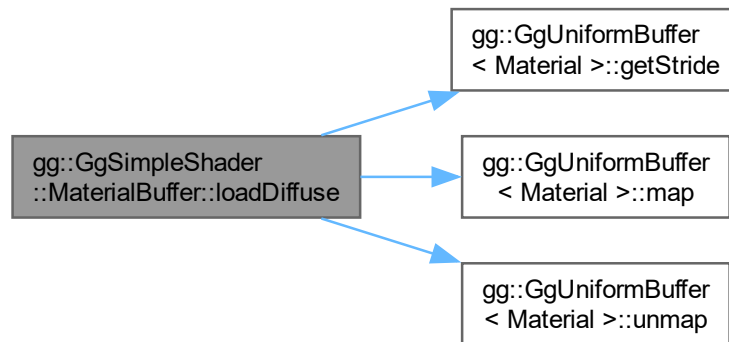
三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する。

引数

<i>diffuse</i>	拡散反射係数を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

`gg.cpp` の 5903 行目に定義があります。

呼び出し関係図:



8.37.3.10 loadDiffuse() [2/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

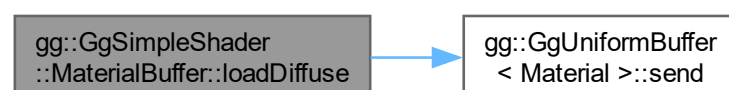
拡散反射係数を設定する.

引数

<i>diffuse</i>	拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8198 行目に定義があります。

呼び出し関係図:



8.37.3.11 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

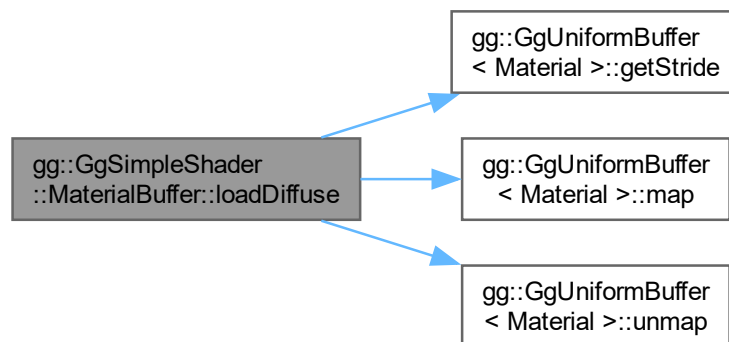
拡散反射係数を設定する。

引数

<i>r</i>	拡散反射係数の赤成分.
<i>g</i>	拡散反射係数の緑成分.
<i>b</i>	拡散反射係数の青成分.
<i>a</i>	拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5877 行目に定義があります。

呼び出し関係図:



8.37.3.12 loadShininess() [1/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    const GLfloat * shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

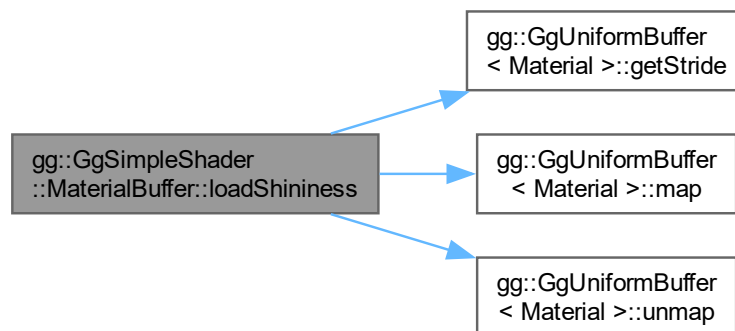
輝き係数を設定する。

引数

<i>shininess</i>	輝き 係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 6091 行目に定義があります。

呼び出し関係図:



8.37.3.13 loadShininess() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    GLfloat shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

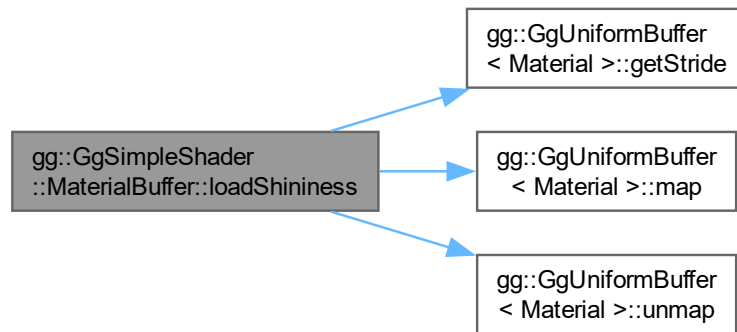
輝き係数を設定する.

引数

<i>shininess</i>	輝き 係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 6070 行目に定義があります。

呼び出し関係図:



8.37.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

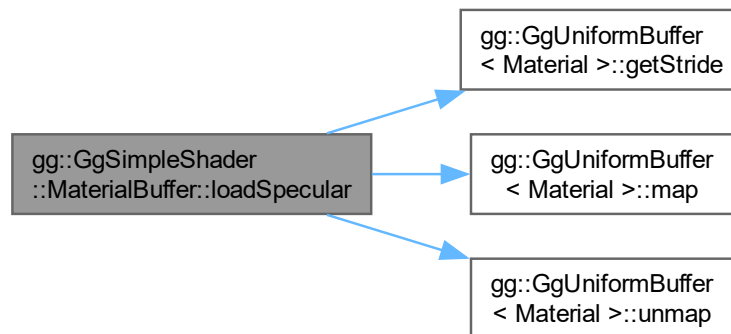
三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する。

引数

<i>specular</i>	鏡面反射係数を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

`gg.cpp` の 6047 行目に定義があります。

呼び出し関係図:



8.37.3.15 loadSpecular() [2/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

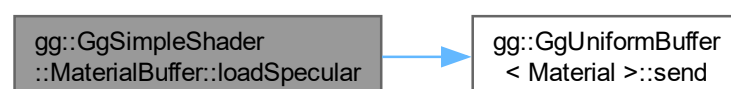
鏡面反射係数を設定する.

引数

<i>specular</i>	鏡面反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8268 行目に定義があります。

呼び出し関係図:



8.37.3.16 loadSpecular() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

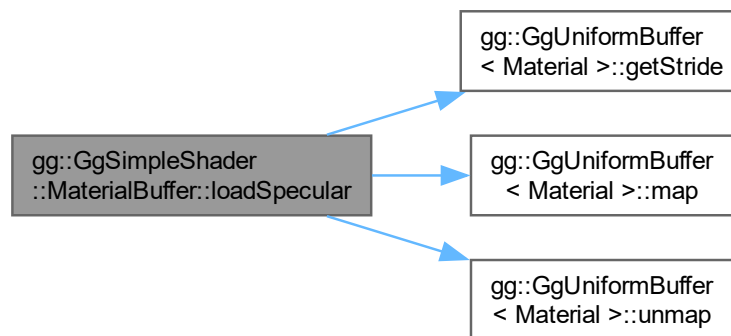
鏡面反射係数を設定する。

引数

<i>r</i>	鏡面反射係数の赤成分.
<i>g</i>	鏡面反射係数の緑成分.
<i>b</i>	鏡面反射係数の青成分.
<i>a</i>	鏡面反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 6021 行目に定義があります。

呼び出し関係図:



8.37.3.17 select()

```
void gg::GgSimpleShader::MaterialBuffer::select (
    GLint i = 0) const [inline]
```

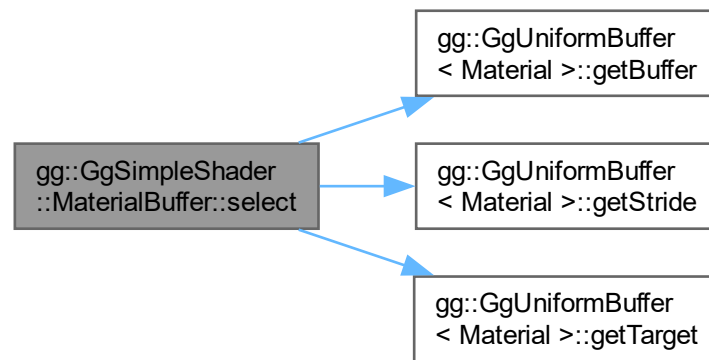
材質を選択する。

引数

`i` | 材質データの uniform block のインデックス.

`gg.h` の 8321 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- `gg.h`
- `gg.cpp`

8.38 Menu クラス

```
#include <Menu.h>
```

公開メンバ関数

- `bool updateAutoCapture (float deltaTime)`
- `Menu (Config &config, Capture &capture, Calibration &calibration)`
- `Menu (const Menu &menu)=delete`
- `virtual ~Menu ()`
- `Menu & operator= (const Menu &menu)=delete`
- `operator bool () const`
- `const auto & getPose () const`
- `auto getMenuubarHeight () const`
- `void initializeInputIntrinsics (const std::array< int, 2 > &size)`
- `const auto & getCheckerSize () const`
- `const auto & getCheckerLength () const`
- `auto getMarkerLength () const`
- `std::array< GLsizei, 2 > setup (GLfloat aspect) const`
- `std::array< GLsizei, 2 > setup (GLfloat aspect, const gg::GgMatrix &viewPose) const`
- `void draw ()`
- `void saveImage (const cv::Mat &image, const std::string &filename="*.jpg") const`

公開変数類

- bool `prioritizeLatency` { false }
レイテンシを優先するなら *true*
- bool `detectMarker` { false }
ArUco Marker を検出するなら *true*
- bool `detectBoard` { false }
ChArUco Board を検出するなら *true*
- bool `autoCaptureEnabled` { false }
自動キャプチャを行うなら *true*
- float `autoCaptureMinStableTime` { 0.6f }
静止判定に必要な継続時間 (秒)
- float `autoCaptureCooldown` { 1.5f }
姿勢変更クールダウン時間 (秒)
- float `autoCaptureCooldownTimer` { 0.0f }
姿勢変更クールダウンタイマー (秒)
- bool `autoCaptureBeep` { true }
自動キャプチャ時の音響フィードバックを行うなら *true*
- std::string `autoCaptureStatusMessage`
自動キャプチャのステータスメッセージ

8.38.1 詳解

メニューの描画

`Menu.h` の 23 行目に定義があります。

8.38.2 構築子と解体子

8.38.2.1 Menu() [1/2]

```
Menu::Menu (
    Config & config,
    Capture & capture,
    Calibration & calibration)
```

コンストラクタ

引数

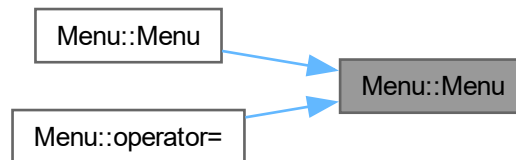
<code>config</code>	構成データ
<code>capture</code>	入力フレームを取得するキャプチャデバイス
<code>calibration</code>	較正オブジェクト

`Menu.cpp` の 612 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.38.2.2 Menu() [2/2]

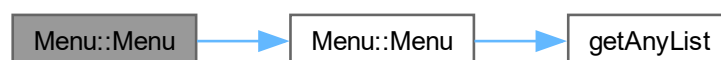
```
Menu::Menu (
    const Menu & menu) [delete]
```

コピーコンストラクタは使用しない

引数

<i>menu</i>	コピー元
-------------	------

呼び出し関係図:



8.38.2.3 ~Menu()

```
Menu::~Menu () [virtual]
```

デストラクタ

[Menu.cpp](#) の 696 行目に定義があります。

8.38.3 関数詳解

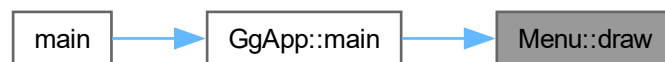
8.38.3.1 draw()

```
void Menu::draw ()
```

メニューを描画する

[Menu.cpp](#) の 1492 行目に定義があります。

被呼び出し関係図:



8.38.3.2 getCheckerLength()

```
const auto & Menu::getCheckerLength () const [inline]
```

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さを得る

戻り値

検出する ChArUco Board のマス目の一辺の長さと ArUco Marker の一辺の長さ

[Menu.h](#) の 393 行目に定義があります。

8.38.3.3 getCheckerSize()

```
const auto & Menu::getCheckerSize () const [inline]
```

検出する ChArUco Board のマス目の横と縦の数を得る

戻り値

検出する ChArUco Board のマス目の横と縦の数

[Menu.h](#) の 383 行目に定義があります。

8.38.3.4 getMarkerLength()

```
auto Menu::getMarkerLength () const [inline]
```

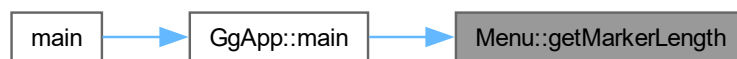
検出する ArUco Marker の一辺の長さを得る

戻り値

検出する ArUco Marker の一辺の長さ

[Menu.h](#) の 403 行目に定義があります。

被呼び出し関係図:



8.38.3.5 getMenubarHeight()

```
auto Menu::getMenubarHeight () const [inline]
```

メニューバーの高さを得る

戻り値

メニューバーの高さ

[Menu.h](#) の 363 行目に定義があります。

被呼び出し関係図:



8.38.3.6 getPose()

```
const auto & Menu::getPose () const [inline]
```

キャプチャデバイスの姿勢を得る

戻り値

図形の姿勢

[Menu.h](#) の 353 行目に定義があります。

8.38.3.7 initializeInputIntrinsics()

```
void Menu::initializeInputIntrinsics (
    const std::array< int, 2 > & size)
```

入力画像に合わせて内部パラメータを初期化する

引数

<i>size</i>	開かれた入力フレームの解像度
-------------	----------------

実解像度を反映し、現在の焦点距離から画像全体が見やすい初期画角を計算する。中心位置は投影方式の設定値を維持する。

[Menu.cpp](#) の 713 行目に定義があります。

被呼び出し関係図:



8.38.3.8 operator bool()

```
Menu::operator bool () const [inline], [explicit]
```

処理を継続するかどうか調べる

戻り値

処理を継続するなら true

[Menu.h](#) の 343 行目に定義があります。

8.38.3.9 operator=()

```
Menu & Menu::operator= (
    const Menu & menu) [delete]
```

代入演算子は使用しない

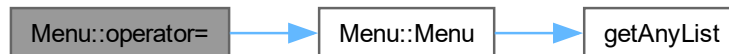
引数

<i>menu</i>	代入元のメニュー
-------------	----------

戻り値

代入後のこのメニューの参照

呼び出し関係図:



8.38.3.10 saveImage()

```

void Menu::saveImage (
    const cv::Mat & image,
    const std::string & filename = "*.jpg") const
  
```

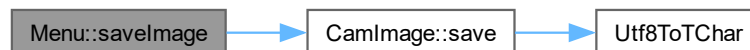
画像の保存

引数

<i>image</i>	保存する画像データ
<i>filename</i>	保存する画像ファイル名のテンプレート

[Menu.cpp](#) の 1515 行目に定義があります。

呼び出し関係図:



8.38.3.11 setup() [1/2]

```

std::array< GLsizei, 2 > Menu::setup (
    GLfloat aspect) const
  
```

シェーダを設定する

引数

<i>aspect</i>	表示領域の縦横比
---------------	----------

戻り値

描画すべきメッシュの横と縦の格子点数

覚え書き

格子点数は画角 *aspect* と展開用メッシュのサンプル点数 *samples* から求める。

[Menu.cpp](#) の 804 行目に定義があります。

被呼び出し関係図:



8.38.3.12 setup() [2/2]

```
std::array< GLsizei, 2 > Menu::setup (
    GLfloat aspect,
    const gg::GgMatrix & viewPose) const
```

指定した姿勢でシェーダを設定する

引数

<i>aspect</i>	表示領域の縦横比
<i>viewPose</i>	メニューの補正姿勢へ追加する視点姿勢

戻り値

描画すべきメッシュの横と縦の格子点数

[Menu.cpp](#) の 814 行目に定義があります。

8.38.3.13 updateAutoCapture()

```
bool Menu::updateAutoCapture (
    float deltaTime)
```

自動キャプチャ処理を更新する

引数

<code>deltaTime</code>	前フレームからの経過時間 (秒)
------------------------	------------------

戻り値

自動記録が行われたら true

[Menu.cpp](#) の 1545 行目に定義があります。

被呼び出し関係図:



8.38.4 メンバ詳解

8.38.4.1 autoCaptureBeep

```
bool Menu::autoCaptureBeep { true }
```

自動キャプチャ時の音響フィードバックを行うなら true

[Menu.h](#) の 296 行目に定義があります。

8.38.4.2 autoCaptureCooldown

```
float Menu::autoCaptureCooldown { 1.5f }
```

姿勢変更クールダウン時間 (秒)

[Menu.h](#) の 290 行目に定義があります。

8.38.4.3 autoCaptureCooldownTimer

```
float Menu::autoCaptureCooldownTimer { 0.0f }
```

姿勢変更クールダウンタイマー (秒)

[Menu.h](#) の 293 行目に定義があります。

8.38.4.4 autoCaptureEnabled

```
bool Menu::autoCaptureEnabled { false }
```

自動キャプチャを行うなら true

[Menu.h](#) の 284 行目に定義があります。

8.38.4.5 autoCaptureMinStableTime

```
float Menu::autoCaptureMinStableTime { 0.6f }
```

静止判定に必要な継続時間 (秒)

[Menu.h](#) の 287 行目に定義があります。

8.38.4.6 autoCaptureStatusMessage

```
std::string Menu::autoCaptureStatusMessage
```

自動キャプチャのステータスメッセージ

[Menu.h](#) の 299 行目に定義があります。

8.38.4.7 detectBoard

```
bool Menu::detectBoard { false }
```

ChArUco Board を検出するなら true

[Menu.h](#) の 281 行目に定義があります。

8.38.4.8 detectMarker

```
bool Menu::detectMarker { false }
```

ArUco Marker を検出するなら true

[Menu.h](#) の 278 行目に定義があります。

8.38.4.9 prioritizeLatency

```
bool Menu::prioritizeLatency { false }
```

レイテンシを優先するなら true

[Menu.h](#) の 275 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Menu.h](#)
- [Menu.cpp](#)

8.39 Mesh クラス

```
#include <Mesh.h>
```

公開メンバ関数

- `Mesh ()`
- `Mesh (const Mesh &mesh)=delete`
- `virtual ~Mesh ()`
- `Mesh & operator= (const Mesh &mesh)=delete`
- `void draw (const std::array< GLfloat, 2 > &scale, GLint unit=0) const`
- `void drawMesh (const std::array< GLsizei, 2 > &size) const`

8.39.1 詳解

メッシュの描画クラス

`Mesh.h` の 20 行目に定義があります。

8.39.2 構築子と解体子

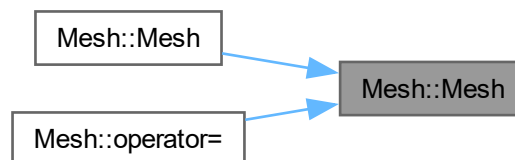
8.39.2.1 Mesh() [1/2]

```
Mesh::Mesh () [inline]
```

コンストラクタ

`Mesh.h` の 39 行目に定義があります。

被呼び出し関係図:



8.39.2.2 Mesh() [2/2]

```
Mesh::Mesh (
    const Mesh & mesh) [delete]
```

コピーコンストラクタは使用しない呼び出し関係図:



8.39.2.3 ~Mesh()

```
virtual Mesh::~Mesh () [inline], [virtual]
```

デストラクタ

[Mesh.h](#) の 60 行目に定義があります。

8.39.3 関数詳解

8.39.3.1 draw()

```
void Mesh::draw (
    const std::array< GLfloat, 2 > & scale,
    GLint unit = 0) const [inline]
```

シェーダを指定して矩形を描画する

引数

<i>scale</i>	マッピングするテクスチャのスケール
<i>unit</i>	使用するテクスチャユニット

覚え書き

この前にテクスチャユニット `GL_TEXTURE0 + unit` を指定し、テクスチャを結合しておく必要がある。

[Mesh.h](#) の 84 行目に定義があります。

8.39.3.2 drawMesh()

```
void Mesh::drawMesh (
    const std::array< GLsizei, 2 > & size) const [inline]
```

メッシュを描画する

引数

<i>size</i>	描画するメッシュの横と縦の格子点数
-------------	-------------------

覚え書き

この前に別途シェードを指定し、メッシュの格子点数を得ておく必要がある。

[Mesh.h](#) の 116 行目に定義があります。

8.39.3.3 operator=()

```
Mesh & Mesh::operator= (  
    const Mesh & mesh) [delete]
```

代入演算子は使用しない

引数

<i>mesh</i>	代入元のオブジェクト
-------------	------------

戻り値

代入後のこのオブジェクトの参照

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Mesh.h](#)

8.40 Preference クラス

```
#include <Preference.h>
```


公開メンバ関数

- [Preference](#) ()
- [Preference](#) (const std::string &description, const std::string &vert, const std::string &frag, const [Intrinsics](#) &intrinsics=[Intrinsics](#){})
- [Preference](#) (const picojson::object &object)
- virtual [~Preference](#) ()
- void [buildShader](#) ()
- const auto & [getDescription](#) () const
- const auto & [getIntrinsics](#) () const
- const auto & [getShader](#) () const
- void [setPreference](#) (picojson::object &object) const

8.40.1 詳解

キャプチャデバイスの構成データ

キャプチャデバイスごとの特性を記述する. キャプチャデバイス固有のパラメータはカメラの内部パラメータなどである. キャプチャデバイスの投影方式は平面展開用のシェードとして記述する.

[Preference.h](#) の 24 行目に定義があります。

8.40.2 構築子と解体子

8.40.2.1 Preference() [1/3]

```
Preference::Preference ()
```

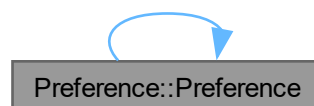
キャプチャデバイスの構成データのデフォルトコンストラクタ

覚え書き

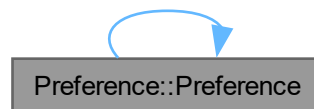
構成ファイルが読めなかったときにしか使わない.

[Preference.cpp](#) の 13 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.40.2.2 Preference() [2/3]

```

Preference::Preference (
    const std::string & description,
    const std::string & vert,
    const std::string & frag,
    const Intrinsic & intrinsics = Intrinsic{})
  
```

シェーダのソースファイルを指定するときに使う キャプチャデバイスの構成データのコンストラクタ

引数

<i>description</i>	このキャプチャデバイスの説明の文字列
<i>vert</i>	展開用のバーテックスシェーダのソースファイル名
<i>frag</i>	展開用のフラグメントシェーダのソースファイル名
<i>intrinsics</i>	キャプチャデバイス固有のパラメータ

[Preference.cpp](#) の 22 行目に定義があります。

8.40.2.3 Preference() [3/3]

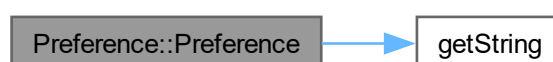
```

Preference::Preference (
    const picojson::object & object)
  
```

構成ファイルを読み込むときに使う キャプチャデバイスの構成データのコンストラクタ

[Preference.cpp](#) の 35 行目に定義があります。

呼び出し関係図:



8.40.2.4 ~Preference()

```
Preference::~~Preference () [virtual]
```

デストラクタ

[Preference.cpp](#) の 48 行目に定義があります。

8.40.3 関数詳解

8.40.3.1 buildShader()

```
void Preference::buildShader ()
```

展開用のシェーダをビルドする

[Preference.cpp](#) の 57 行目に定義があります。

8.40.3.2 getDescription()

```
const auto & Preference::getDescription () const [inline]
```

説明の文字列を取り出す

戻り値

構成の説明文

[Preference.h](#) の 84 行目に定義があります。

8.40.3.3 getIntrinsics()

```
const auto & Preference::getIntrinsics () const [inline]
```

キャプチャデバイス固有のパラメータを取り出す

戻り値

キャプチャデバイス固有のパラメータ

[Preference.h](#) の 94 行目に定義があります。

8.40.3.4 getShader()

```
const auto & Preference::getShader () const [inline]
```

シェーダを取り出す

戻り値

この構成のシェーダの参照

[Preference.h](#) の 104 行目に定義があります。

8.40.3.5 setPreference()

```
void Preference::setPreference (  
    picojson::object & object) const
```

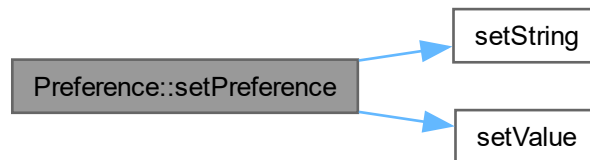
JSON オブジェクトに構成を格納する

引数

<i>object</i>	格納先の JSON オブジェクト
---------------	------------------

[Preference.cpp](#) の 69 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [Preference.h](#)
- [Preference.cpp](#)

8.41 Settings 構造体

```
#include <Config.h>
```

公開メンバ関数

- auto [getFocal](#) () const

公開変数類

- int [samples](#) { 57600 }
展開に用いるメッシュのサンプル数
- std::array< float, 3 > [euler](#) { [defaultEuler](#) }
キャプチャデバイスの姿勢のオイラー角
- float [focal](#) { [defaultFocal](#) }
描画時の焦点距離
- std::array< float, 2 > [focalRange](#) { [defaultFocalRange](#) }
描画時の焦点距離の範囲
- std::string [dictionaryName](#) { "DICT_4X4_50" }
使用中の *ArUco Marker* 辞書名
- std::array< int, 2 > [checkerSize](#) { [defaultCheckerSize](#) }
検出する *ChArUco Board* のマス目の横と縦の数
- std::array< float, 2 > [checkerLength](#) { [defaultCheckerLength](#) }
検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さ (単位 *cm*)
- float [markerLength](#) { [defaultMarkerLength](#) }
検出する *ArUco Marker* の一辺の長さ (単位 *cm*)

静的公開変数類

- static constexpr decltype([euler](#)) [defaultEuler](#) { 0.0f, 0.0f, 0.0f }
キャプチャデバイスの姿勢のデフォルト値
- static constexpr decltype([focal](#)) [defaultFocal](#) { 50.0f }
展開時の焦点距離のデフォルト値
- static constexpr decltype([focalRange](#)) [defaultFocalRange](#) { 10.0f, 200.0f }
描画時の焦点距離の範囲のデフォルト値
- static constexpr decltype([checkerSize](#)) [defaultCheckerSize](#) { 10, 7 }
検出する *ChArUco Board* のマス目の横と縦の数のデフォルト値
- static constexpr decltype([checkerLength](#)) [defaultCheckerLength](#) { 4.0f, 2.0f }
検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さのデフォルト値
- static constexpr decltype([markerLength](#)) [defaultMarkerLength](#) { 5.0f }
検出する *ArUco Marker* の一辺の長さのデフォルト値

8.41.1 詳解

表示関連の設定データ

[Config.h](#) の 17 行目に定義があります。

8.41.2 関数詳解

8.41.2.1 getFocal()

```
auto Settings::getFocal () const [inline]
```

正規化デバイス座標系における焦点距離を求める

[Config.h](#) の 64 行目に定義があります。

8.41.3 メンバ詳解

8.41.3.1 checkerLength

```
std::array<float, 2> Settings::checkerLength { defaultCheckerLength }
```

検出する *ChArUco Board* のマス目一辺の長さ と *ArUco Marker* の一辺の長さ (単位 cm)

[Config.h](#) の 50 行目に定義があります。

8.41.3.2 checkerSize

```
std::array<int, 2> Settings::checkerSize { defaultCheckerSize }
```

検出する *ChArUco Board* のマス目の横と縦の数

[Config.h](#) の 44 行目に定義があります。

8.41.3.3 defaultCheckerLength

```
decltype(checkerLength) Settings::defaultCheckerLength { 4.0f, 2.0f } [static], [constexpr]
```

検出する ChArUco Board のマス目一辺の長さ と ArUco Marker の一辺の長さのデフォルト値

[Config.h](#) の 53 行目に定義があります。

8.41.3.4 defaultCheckerSize

```
decltype(checkerSize) Settings::defaultCheckerSize { 10, 7 } [static], [constexpr]
```

検出する ChArUco Board のマス目の横と縦の数のデフォルト値

[Config.h](#) の 47 行目に定義があります。

8.41.3.5 defaultEuler

```
decltype(euler) Settings::defaultEuler { 0.0f, 0.0f, 0.0f } [static], [constexpr]
```

キャプチャデバイスの姿勢のデフォルト値

[Config.h](#) の 26 行目に定義があります。

8.41.3.6 defaultFocal

```
decltype(focal) Settings::defaultFocal { 50.0f } [static], [constexpr]
```

展開時の焦点距離のデフォルト値

[Config.h](#) の 32 行目に定義があります。

8.41.3.7 defaultFocalRange

```
decltype(focalRange) Settings::defaultFocalRange { 10.0f, 200.0f } [static], [constexpr]
```

描画時の焦点距離の範囲のデフォルト値

[Config.h](#) の 38 行目に定義があります。

8.41.3.8 defaultMarkerLength

```
decltype(markerLength) Settings::defaultMarkerLength { 5.0f } [static], [constexpr]
```

検出する ArUco Marker の一辺の長さのデフォルト値

[Config.h](#) の 59 行目に定義があります。

8.41.3.9 dictionaryName

```
std::string Settings::dictionaryName { "DICT_4X4_50" }
```

使用中の ArUco Marker 辞書名

[Config.h](#) の 41 行目に定義があります。

8.41.3.10 euler

```
std::array<float, 3> Settings::euler { defaultEuler }
```

キャプチャデバイスの姿勢のオイラー角

[Config.h](#) の 23 行目に定義があります。

8.41.3.11 focal

```
float Settings::focal { defaultFocal }
```

描画時の焦点距離

[Config.h](#) の 29 行目に定義があります。

8.41.3.12 focalRange

```
std::array<float, 2> Settings::focalRange { defaultFocalRange }
```

描画時の焦点距離の範囲

[Config.h](#) の 35 行目に定義があります。

8.41.3.13 markerLength

```
float Settings::markerLength { defaultMarkerLength }
```

検出する ArUco Marker の一辺の長さ (単位 cm)

[Config.h](#) の 56 行目に定義があります。

8.41.3.14 samples

```
int Settings::samples { 57600 }
```

展開に用いるメッシュのサンプル数

[Config.h](#) の 20 行目に定義があります。

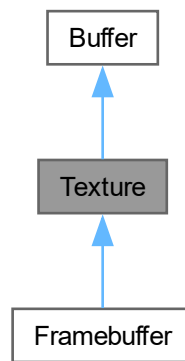
この構造体詳解は次のファイルから抽出されました:

- [Config.h](#)

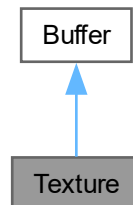
8.42 Texture クラス

```
#include <Texture.h>
```

Texture の継承関係図



Texture 連携図



公開メンバ関数

- `Texture ()=default`
- `Texture (GLsizei width, GLsizei height, int channels)`
- `Texture (const Texture &texture)`
- `Texture (Texture &&texture) noexcept`
- `virtual ~Texture ()`
- `Texture & operator= (const Texture &texture)`
- `Texture & operator= (Texture &&texture) noexcept`
- `virtual void create (GLsizei width, GLsizei height, int channels)`
- `virtual void copy (const Buffer &texture) noexcept`
- `virtual void discard ()`

- auto `getTextureName` () const
- virtual const std::array< GLsizei, 2 > & `getSize` () const
- virtual int `getChannels` () const
- void `bindTexture` (int unit=0) const
- void `unbindTexture` () const
- void `draw` (GLsizei width, GLsizei height, int unit=0) const
- void `readPixels` (GLuint buffer) const
- void `readPixels` (Buffer &buffer) const
- void `readPixels` () const
- void `drawPixels` (GLuint buffer) const
- void `drawPixels` (const Texture &texture)
- void `drawPixels` (const Buffer &buffer)
- void `drawPixels` () const
- void `drawPixels` (GLsizei width, GLsizei height, int channels, const void *pixels)

基底クラス **Buffer** に属する継承公開メンバ関数

- **Buffer** ()=default
- **Buffer** (GLsizei width, GLsizei height, int channels)
- **Buffer** (const Buffer &buffer)
- **Buffer** (Buffer &&buffer) noexcept
- virtual ~**Buffer** ()
- **Buffer** & operator= (const Buffer &buffer)
- **Buffer** & operator= (Buffer &&buffer) noexcept
- void `resize` (GLsizei width, GLsizei height, int channels)
- void `resize` (const Buffer &buffer)
- auto `getBufferName` () const
- GLsizei `getWidth` () const
- GLsizei `getHeight` () const
- auto `getFormat` () const
- auto `getAspect` () const
- void `bindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- void `unbindBuffer` (GLenum target=GL_PIXEL_PACK_BUFFER) const
- GLvoid * `map` () const
- void `unmap` () const

静的限定公開変数類

- static std::shared_ptr< Mesh > `mesh`
テクスチャ展開に用いるメッシュの頂点配列オブジェクト

その他の継承メンバ

基底クラス **Buffer** に属する継承限定公開メンバ関数

- auto `channelsToFormat` (int channels) const
- int `formatToChannels` (GLenum format) const
- void `copyBuffer` (const Buffer &buffer) noexcept

8.42.1 詳解

テクスチャクラス

[Texture.h](#) の 20 行目に定義があります。

8.42.2 構築子と解体子

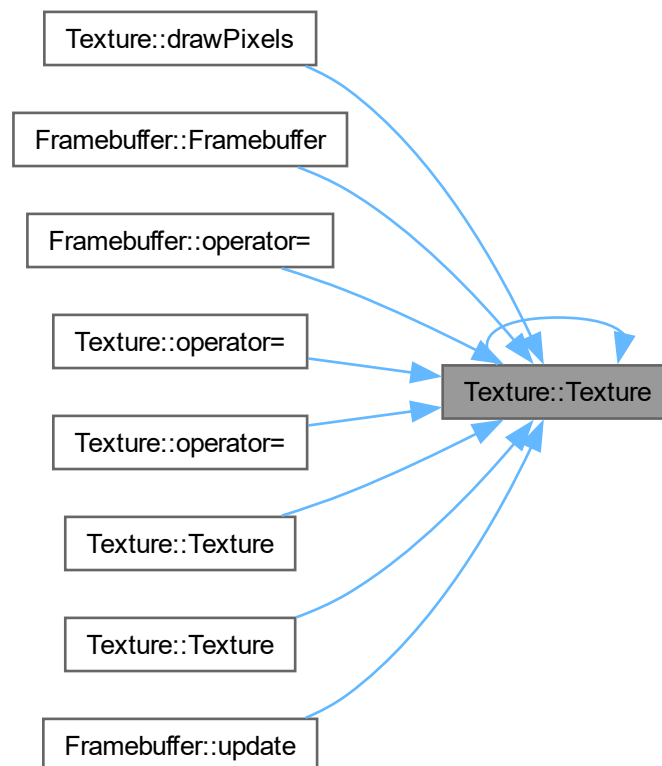
8.42.2.1 Texture() [1/4]

```
Texture::Texture () [default]
```

テクスチャのデフォルトコンストラクタ 呼び出し関係図:



被呼び出し関係図:



8.42.2.2 Texture() [2/4]

```
Texture::Texture (
    GLsizei width,
    GLsizei height,
    int channels)
```

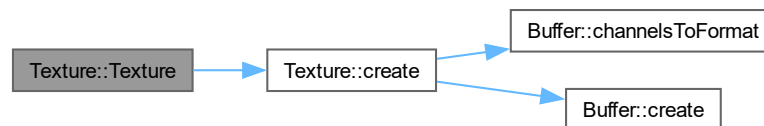
テクスチャを作成するコンストラクタ

引数

<i>width</i>	作成するテクスチャの横の画素数
<i>height</i>	作成するテクスチャの縦の画素数
<i>channels</i>	作成するテクスチャのチャンネル数

[Texture.cpp](#) の 13 行目に定義があります。

呼び出し関係図:



8.42.2.3 Texture() [3/4]

```
Texture::Texture (
    const Texture & texture)
```

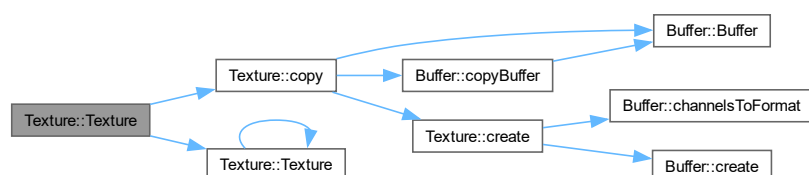
コピーコンストラクタ

引数

<i>texture</i>	コピー元のテクスチャ
----------------	------------

[Texture.cpp](#) の 22 行目に定義があります。

呼び出し関係図:



8.42.2.4 Texture() [4/4]

```
Texture::Texture (  
    Texture && texture) [inline], [noexcept]
```

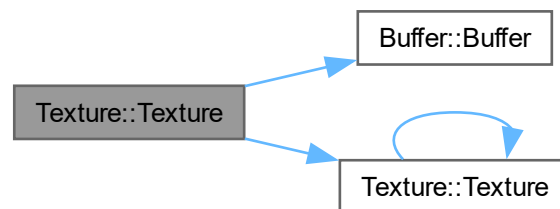
ムーブコンストラクタ

引数

<i>texture</i>	ムーブ元のテクスチャ
----------------	------------

[Texture.h](#) の 64 行目に定義があります。

呼び出し関係図:



8.42.2.5 ~Texture()

```
Texture::~Texture () [virtual]
```

デストラクタ

[Texture.cpp](#) の 31 行目に定義があります。

呼び出し関係図:



8.42.3 関数詳解

8.42.3.1 bindTexture()

```
void Texture::bindTexture (  
    int unit = 0) const [inline]
```

テクスチャユニットを指定してテクスチャを結合する

引数

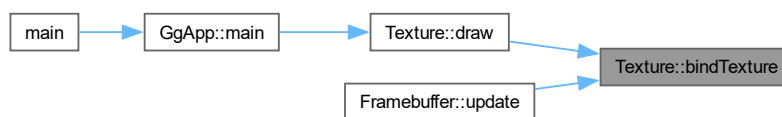
<i>unit</i>	テクスチャユニット番号
-------------	-------------

覚え書き

`unit` にはシェーダにおいてテクスチャのサンプラの `uniform` 変数に設定する `GL_TEXTUREi` の `i` と一致させること。

`Texture.h` の 165 行目に定義があります。

被呼び出し関係図:



8.42.3.2 copy()

```
void Texture::copy (
    const Buffer & texture) [virtual], [noexcept]
```

テクスチャをコピーする

引数

<i>texture</i>	コピー元のテクスチャ
----------------	------------

覚え書き

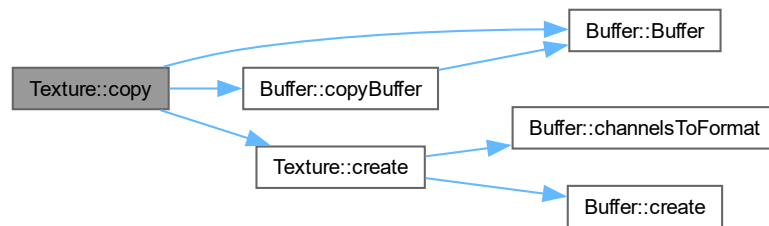
このテクスチャのサイズがコピー元と異なれば、このテクスチャを削除して新しいテクスチャを作り直してコピーする。引数の型が `Buffer` なのは、このオブジェクトの型が `Buffer` のとき `Buffer::copy()`、`Texture` のとき `Texture::copy()`、`Framebuffer` のとき `Framebuffer::copy()` を呼ぶようにするため。

`Buffer` を再実装しています。

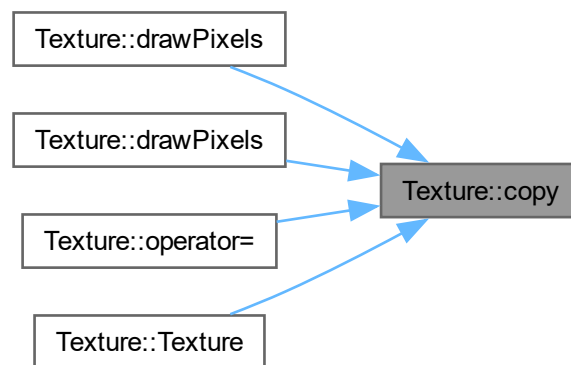
`Framebuffer` で再実装されています。

`Texture.cpp` の 121 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.3 create()

```

void Texture::create (
    GLsizei width,
    GLsizei height,
    int channels) [virtual]
  
```

テクスチャを作成する

引数

<i>width</i>	作成するテクスチャの横の画素数
<i>height</i>	作成するテクスチャの縦の画素数
<i>channels</i>	作成するテクスチャのチャンネル数

覚え書き

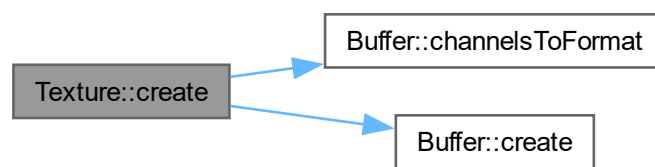
このテクスチャのサイズが引数で指定したサイズと異なれば、このテクスチャを削除して新しいテクスチャを作り直す。

`Buffer`を再実装しています。

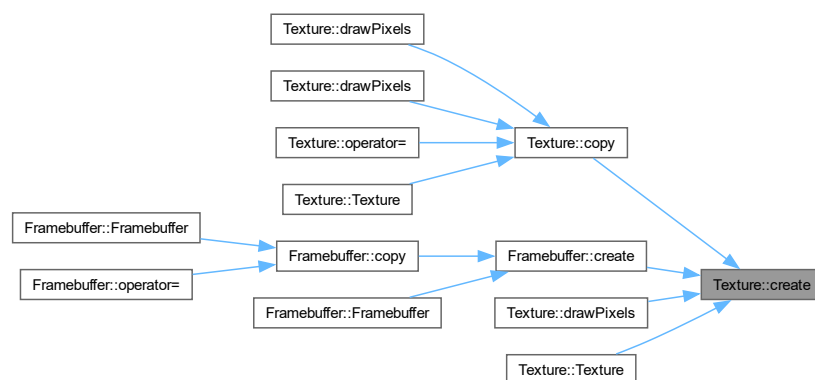
`Framebuffer`で再実装されています。

`Texture.cpp` の 85 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.4 discard()

```
void Texture::discard () [virtual]
```

テクスチャを破棄する

`Buffer`を再実装しています。

`Framebuffer`で再実装されています。

[Texture.cpp](#) の 134 行目に定義があります。

被呼び出し関係図:



8.42.3.5 draw()

```
void Texture::draw (  
    GLsizei width,  
    GLsizei height,  
    int unit = 0) const
```

このテクスチャをマッピングして矩形を描画する

引数

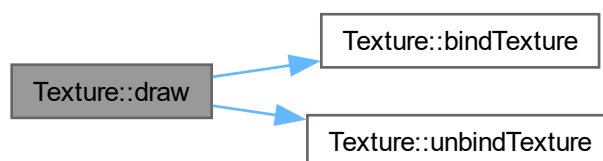
<i>width</i>	表示領域の横の画素数
<i>height</i>	表示領域の縦の画素数
<i>unit</i>	使用するテクスチャユニット番号

覚え書き

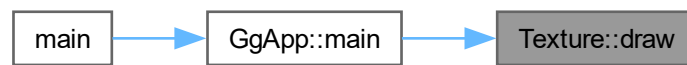
フレームバッファ全体を覆うメッシュを描画する。unit にはシェードにおいてテクスチャのサンブラの uniform 変数に設定する GL_TEXTUREi の i と一致させること。

[Texture.cpp](#) の 151 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



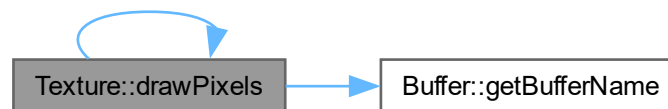
8.42.3.6 drawPixels() [1/5]

```
void Texture::drawPixels () const [inline]
```

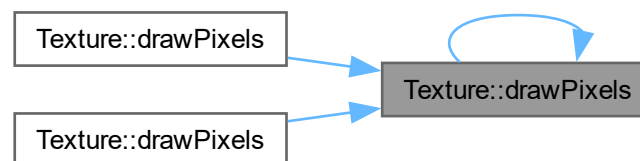
ピクセルバッファオブジェクトからテクスチャにデータをコピーする

[Texture.h](#) の 301 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.7 drawPixels() [2/5]

```
void Texture::drawPixels (
    const Buffer & buffer) [inline]
```

指定したオブジェクトからテクスチャにデータをコピーする

引数

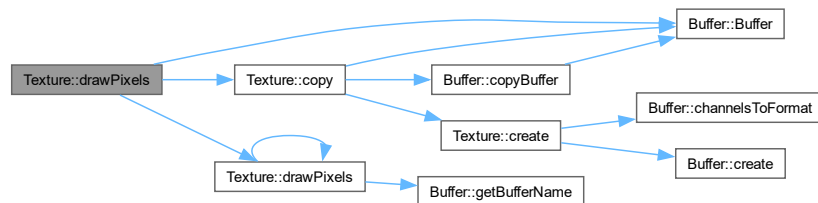
<i>buffer</i>	テクスチャをコピーする先のオブジェクト
---------------	---------------------

覚え書き

テクスチャのサイズをコピー元のオブジェクトのバッファに合わせる。

[Texture.h](#) の 289 行目に定義があります。

呼び出し関係図:



8.42.3.8 drawPixels() [3/5]

```
void Texture::drawPixels (
    const Texture & texture) [inline]
```

指定したテクスチャからからデータをコピーする

引数

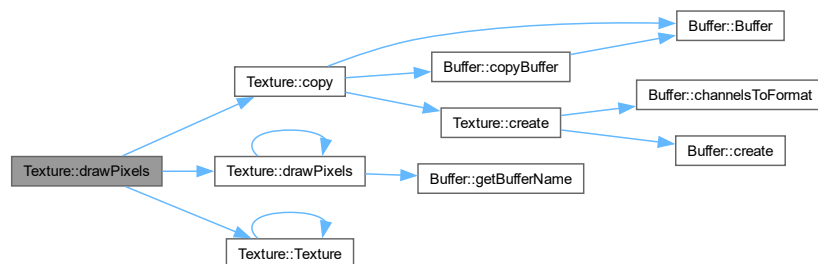
<i>texture</i>	コピー元のテクスチャ
----------------	------------

覚え書き

テクスチャのサイズをコピー元のテクスチャに合わせる。

[Texture.h](#) の 272 行目に定義があります。

呼び出し関係図:



8.42.3.9 drawPixels() [4/5]

```
void Texture::drawPixels (
    GLsizei width,
    GLsizei height,
    int channels,
    const void * pixels)
```

CPU メモリ上の画像をテクスチャへ転送する

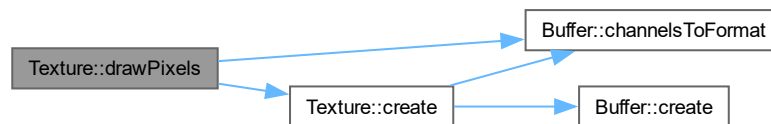
引数

<i>width</i>	画像の幅
<i>height</i>	画像の高さ
<i>channels</i>	画像のチャンネル数
<i>pixels</i>	画像の先頭画素へのポインタ

OpenCV で補正した `cv::Mat` を PBO を介さずGPUへ転送するために使う。

[Texture.cpp](#) の 275 行目に定義があります。

呼び出し関係図:



8.42.3.10 drawPixels() [5/5]

```
void Texture::drawPixels (
    GLuint buffer) const
```

指定したピクセルバッファオブジェクトからテクスチャにデータをコピーする

引数

<i>buffer</i>	コピーするテクスチャを格納したピクセルバッファオブジェクト名
---------------	--------------------------------

覚え書き

コピーするデータのサイズが一致している必要がある。

Texture.cpp の 237 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.11 getChannels()

```
virtual int Texture::getChannels () const [inline], [virtual]
```

テクスチャのチャンネル数を得る

Bufferを再実装しています。

Framebufferで再実装されています。

Texture.h の 151 行目に定義があります。

8.42.3.12 getSize()

```
virtual const std::array< GLsizei, 2 > & Texture::getSize () const [inline], [virtual]
```

テクスチャのサイズを得る

Bufferを再実装しています。

Framebufferで再実装されています。

Texture.h の 143 行目に定義があります。

8.42.3.13 `getTextureName()`

```
auto Texture::getTextureName () const [inline]
```

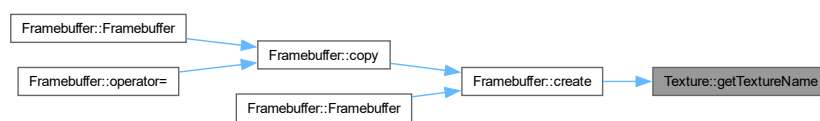
テクスチャ名を得る

戻り値

テクスチャ名

[Texture.h](#) の 135 行目に定義があります。

被呼び出し関係図:



8.42.3.14 `operator=()` [1/2]

```
Texture & Texture::operator= (
    const Texture & texture)
```

代入演算子

引数

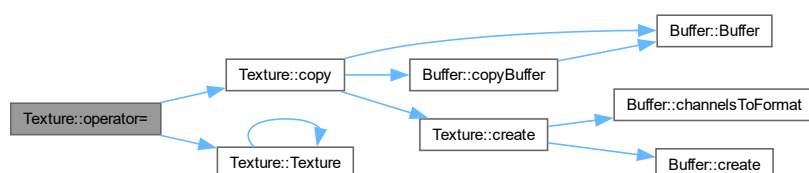
<i>texture</i>	代入元のテクスチャ
----------------	-----------

戻り値

代入結果のテクスチャ

[Texture.cpp](#) の 40 行目に定義があります。

呼び出し関係図:



8.42.3.15 operator=() [2/2]

```
Texture & Texture::operator= (  
    Texture && texture) [noexcept]
```

ムーブ代入演算子

引数

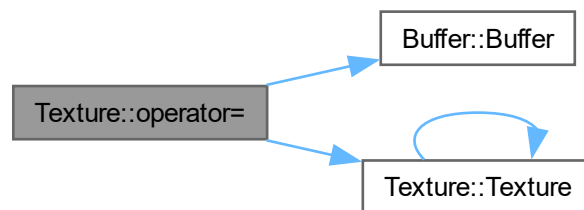
<code>texture</code>	ムーブ代入元のテクスチャ
----------------------	--------------

戻り値

ムーブ代入結果のテクスチャ

[Texture.cpp](#) の 55 行目に定義があります。

呼び出し関係図:



8.42.3.16 readPixels() [1/3]

```
void Texture::readPixels () const [inline]
```

テクスチャからピクセルバッファオブジェクトにデータをコピーする

[Texture.h](#) の 239 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.42.3.17 readPixels() [2/3]

```
void Texture::readPixels (
    Buffer & buffer) const [inline]
```

テクスチャから指定したオブジェクトのバッファにデータをコピーする

引数

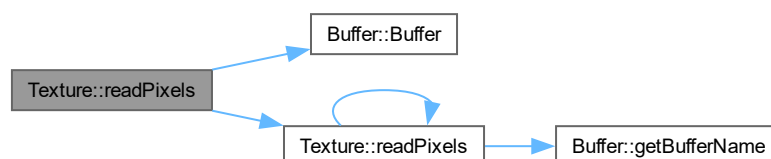
<i>buffer</i>	テクスチャをコピーする先のオブジェクト
---------------	---------------------

覚え書き

コピーする先のオブジェクトのサイズをテクスチャに合わせる。

[Texture.h](#) の 224 行目に定義があります。

呼び出し関係図:



8.42.3.18 readPixels() [3/3]

```
void Texture::readPixels (
    GLuint buffer) const
```

テクスチャから指定したピクセルバッファオブジェクトにデータをコピーする

引数

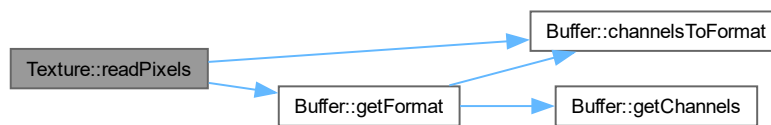
<code>buffer</code>	テクスチャをコピーする先のピクセルバッファオブジェクト名
---------------------	------------------------------

覚え書き

コピーするデータのサイズが一致している必要がある。

`Texture.cpp` の 178 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



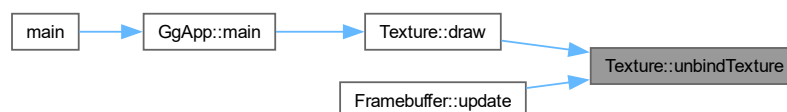
8.42.3.19 unbindTexture()

```
void Texture::unbindTexture () const [inline]
```

テクスチャの結合を解除する

`Texture.h` の 177 行目に定義があります。

被呼び出し関係図:



8.42.4 メンバ詳解

8.42.4.1 mesh

```
std::shared_ptr< Mesh > Texture::mesh [static], [protected]
```

テクスチャ展開に用いるメッシュの頂点配列オブジェクト

[Texture.h](#) の 34 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [Texture.h](#)
- [Texture.cpp](#)

8.43 GgApp::Window クラス

```
#include <GgApp.h>
```

公開メンバ関数

- [Window](#) (const std::string &title="GLFW Window", int width=640, int height=480, int fullscreen=0, GLFWwindow *share=nullptr)
- [Window](#) (const Window &w)=delete
- [Window](#) (Window &&w) noexcept
- virtual [~Window](#) ()
- [Window](#) & [operator=](#) (const [Window](#) &w)=delete
- [Window](#) & [operator=](#) ([Window](#) &&w) noexcept
- auto * [get](#) () const
- void [setClose](#) (int flag=1) const
- bool [shouldClose](#) () const
- [operator](#) bool ()
- void [swapBuffers](#) () const
- void [restoreViewport](#) () const
- void [updateViewport](#) ()
- auto [getWidth](#) () const
- auto [getHeight](#) () const
- auto [getFboWidth](#) () const
- auto [getFboHeight](#) () const
- const auto & [getSize](#) () const
- void [getSize](#) (GLsizei *size) const
- const auto & [getFboSize](#) () const
- void [getFboSize](#) (GLsizei *fboSize) const
- auto [getAspect](#) () const
- bool [getKey](#) (int key) const
- void [selectInterface](#) (int no)
- void [setVelocity](#) (GLfloat vx, GLfloat vy, GLfloat vz=0.1f)
- int [getLastKey](#) ()
- auto [getArrow](#) (int direction=0, int mods=0) const
- auto [getArrowX](#) (int mods=0) const
- auto [getArrowY](#) (int mods=0) const

- void [getArrow](#) (GLfloat *arrow, int mods=0) const
- auto [getShiftArrowX](#) () const
- auto [getShiftArrowY](#) () const
- void [getShiftArrow](#) (GLfloat *shift_arrow) const
- auto [getControlArrowX](#) () const
- auto [getControlArrowY](#) () const
- void [getControlArrow](#) (GLfloat *control_arrow) const
- auto [getAltArrowX](#) () const
- auto [getAltArrowY](#) () const
- void [getAltArrow](#) (GLfloat *alt_arrow) const
- const auto * [getMouse](#) () const
- void [getMouse](#) (GLfloat *position) const
- auto [getMouse](#) (int direction) const
- auto [getMouseX](#) () const
- auto [getMouseY](#) () const
- const auto * [getWheel](#) () const
- void [getWheel](#) (GLfloat *rotation) const
- auto [getWheel](#) (int direction) const
- auto [getWheelX](#) () const
- auto [getWheelY](#) () const
- const auto & [getTranslation](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getTranslationMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getScrollMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getRotation](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getRotationMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- void [resetRotation](#) ()
- void [resetTranslation](#) ()
- void [reset](#) ()
- void * [getUserPointer](#) () const
- void [setUserPointer](#) (void *pointer)
- void [setResizeFunc](#) (void(*func)(const [Window](#) *window, int width, int height))
- void [setKeyboardFunc](#) (void(*func)(const [Window](#) *window, int key, int scancode, int action, int mods))
- void [setMouseFunc](#) (void(*func)(const [Window](#) *window, int button, int action, int mods))
- void [setWheelFunc](#) (void(*func)(const [Window](#) *window, double x, double y))

8.43.1 詳解

ウィンドウ関連の処理.

覚え書き

GLFW を使って OpenGL のウィンドウを操作するラッパークラス.

[GgApp.h](#) の 200 行目に定義があります。

8.43.2 構築子と解体子

8.43.2.1 Window() [1/3]

```
GgApp::Window::Window (  
    const std::string & title = "GLFW Window",  
    int width = 640,  
    int height = 480,  
    int fullscreen = 0,  
    GLFWwindow * share = nullptr)
```

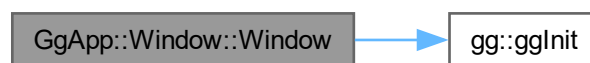
コンストラクタ.

引数

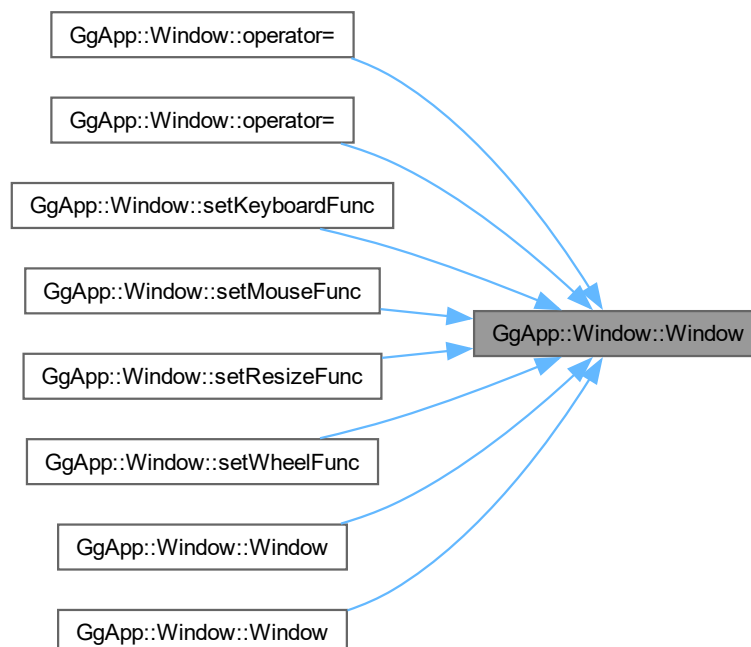
<i>title</i>	ウィンドウタイトルの文字列.
<i>width</i>	開くウィンドウの幅, フルスクリーン時は無視され実際のディスプレイの幅が使われる.
<i>height</i>	開くウィンドウの高さ, フルスクリーン時は無視され実際のディスプレイの高さが使われる.
<i>fullscreen</i>	フルスクリーン表示を行うディスプレイ番号, 0 ならフルスクリーン表示を行わない.
<i>share</i>	共有するコンテキスト, nullptr ならコンテキストを共有しない.

GgApp.cpp の 506 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.2.2 Window() [2/3]

```
GgApp::Window::Window (  
    const Window & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のウィンドウ.
----------	-------------

呼び出し関係図:



8.43.2.3 Window() [3/3]

```
GgApp::Window::Window (  
    Window && w) [noexcept]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

[GgApp.cpp](#) の 587 行目に定義があります。

呼び出し関係図:



8.43.2.4 ~Window()

```
virtual GgApp::Window::~~Window () [inline], [virtual]
```

デストラクタ.

[GgApp.h](#) の 349 行目に定義があります。

8.43.3 関数詳解

8.43.3.1 get()

```
auto * GgApp::Window::get () const [inline]
```

ウィンドウの識別子のポインタを取得する.

戻り値

GLFWwindow 型のウィンドウ識別子のポインタ.

[GgApp.h](#) の 383 行目に定義があります。

8.43.3.2 getAltArrow()

```
void GgApp::Window::getAltArrow (
    GLfloat * alt_arrow) const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>alt_arrow</i>	ALT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
------------------	---

[GgApp.h](#) の 742 行目に定義があります。

呼び出し関係図:



8.43.3.3 getAltArrowX()

```
auto GgApp::Window::getAltArrowX () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の X 値.

GgApp.h の 722 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.4 getAltArrowY()

```
auto GgApp::Window::getAltArrowY () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の Y 値.

GgApp.h の 732 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.5 getArrow() [1/2]

```
void GgApp::Window::getArrow (
    GLfloat * arrow,
    int mods = 0) const [inline]
```

矢印キーの現在の値を得る.

引数

<i>arrow</i>	矢印キーの値を格納する GLfloat[2] の配列.
<i>mods</i>	修飾キーの状態 (0:なし, 1, SHIFT, 2: CTRL, 3: ALT).

[GgApp.h](#) の [649](#) 行目に定義があります。

呼び出し関係図:



8.43.3.6 getArrow() [2/2]

```
auto GgApp::Window::getArrow (
    int direction = 0,
    int mods = 0) const [inline]
```

矢印キーの現在の値を得る.

引数

<i>direction</i>	方向 (0: X, 1:Y).
------------------	-----------------

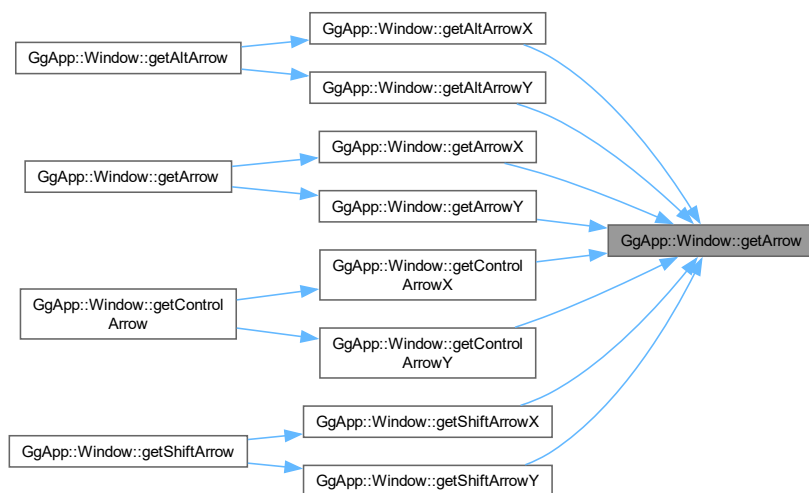
<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの値.

GgApp.h の 615 行目に定義があります。

被呼び出し関係図:



8.43.3.7 getArrowX()

```
auto GgApp::Window::getArrowX (
    int mods = 0) const [inline]
```

矢印キーの現在の X 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの X 値.

[GgApp.h](#) の [627](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.8 getArrowY()

```
auto GgApp::Window::getArrowY (  
    int mods = 0) const [inline]
```

矢印キーの現在の Y 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0: なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	---

戻り値

矢印キーの Y 値.

[GgApp.h](#) の [638](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.9 getAspect()

```
auto GgApp::Window::getAspect () const [inline]
```

ウィンドウの縦横比を得る.

戻り値

ウィンドウの縦横比.

[GgApp.h](#) の 547 行目に定義があります。

8.43.3.10 getControlArrow()

```
void GgApp::Window::getControlArrow (
    GLfloat * control_arrow) const [inline]
```

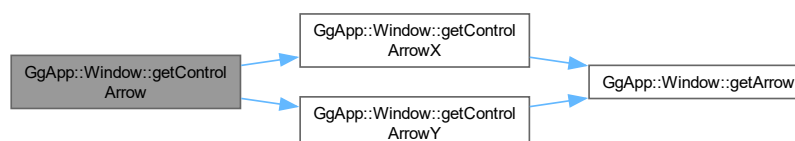
CTRL キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>control_arrow</i>	CTRL キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
----------------------	--

[GgApp.h](#) の 711 行目に定義があります。

呼び出し関係図:



8.43.3.11 getControlArrowX()

```
auto GgApp::Window::getControlArrowX () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

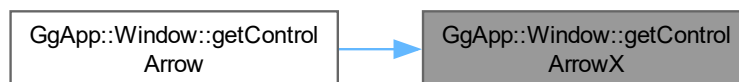
CTRL キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 691 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.12 getControlArrowY()

```
auto GgApp::Window::getControlArrowY () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

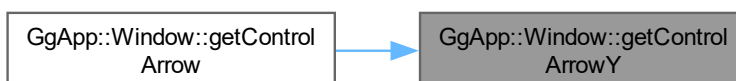
CTRL キーを押しながら矢印キーを押したときの現在の Y 値.

GgApp.h の 701 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.13 getFboHeight()

```
auto GgApp::Window::getFboHeight () const [inline]
```

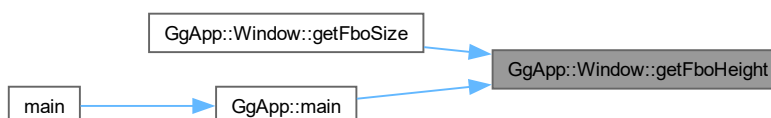
FBO の高さを得る.

戻り値

FBO の高さ.

GgApp.h の 495 行目に定義があります。

被呼び出し関係図:



8.43.3.14 getFboSize() [1/2]

```
const auto & GgApp::Window::getFboSize () const [inline]
```

FBO のサイズを得る.

戻り値

FBO の幅と高さを格納した GLsizei 型の 2 要素の配列.

[GgApp.h](#) の 526 行目に定義があります。

8.43.3.15 getFboSize() [2/2]

```
void GgApp::Window::getFboSize (  
    GLsizei * fboSize) const [inline]
```

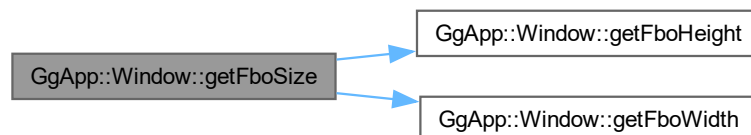
FBO のサイズを得る.

引数

<i>fboSize</i>	FBO の幅と高さを格納する GLsizei 型の 2 要素の配列.
----------------	------------------------------------

[GgApp.h](#) の 536 行目に定義があります。

呼び出し関係図:



8.43.3.16 getFboWidth()

```
auto GgApp::Window::getFboWidth () const [inline]
```

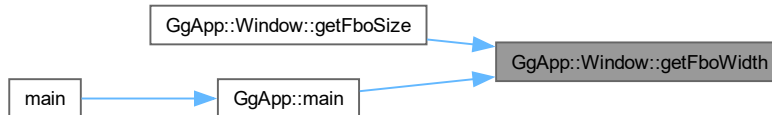
FBO の横幅を得る.

戻り値

FBO の横幅.

[GgApp.h](#) の 485 行目に定義があります。

被呼び出し関係図:



8.43.3.17 getHeight()

```
auto GgApp::Window::getHeight () const [inline]
```

ウィンドウの高さを得る.

戻り値

ウィンドウの高さ.

[GgApp.h](#) の 475 行目に定義があります。

被呼び出し関係図:



8.43.3.18 getKey()

```
bool GgApp::Window::getKey (  
    int key) const [inline]
```

キーが押されているかどうかを判定する.

戻り値

キーが押されていれば true.

[GgApp.h](#) の 557 行目に定義があります。

8.43.3.19 getLastKey()

```
int GgApp::Window::getLastKey () [inline]
```

最後にタイプしたキーを得る.

戻り値

最後にタイプしたキーの文字.

[GgApp.h](#) の 600 行目に定義があります。

8.43.3.20 getMouse() [1/3]

```
const auto * GgApp::Window::getMouse () const [inline]
```

マウスカーソルの現在位置を得る.

戻り値

マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.

[GgApp.h](#) の 753 行目に定義があります。

8.43.3.21 getMouse() [2/3]

```
void GgApp::Window::getMouse (
    GLfloat * position) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>position</i>	マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.
-----------------	---------------------------------------

[GgApp.h](#) の 764 行目に定義があります。

8.43.3.22 getMouse() [3/3]

```
auto GgApp::Window::getMouse (
    int direction) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスカーソルの現在位置.

GgApp.h の 777 行目に定義があります。

8.43.3.23 getMouseX()

```
auto GgApp::Window::getMouseX () const [inline]
```

マウスカーソルの現在位置の X 座標を得る.

戻り値

direction 方向のマウスカーソルの X 方向の現在位置.

GgApp.h の 788 行目に定義があります。

8.43.3.24 getMouseY()

```
auto GgApp::Window::getMouseY () const [inline]
```

マウスカーソルの現在位置の Y 座標を得る.

戻り値

direction 方向のマウスカーソルの Y 方向の現在位置.

GgApp.h の 799 行目に定義があります。

8.43.3.25 getRotation()

```
auto GgApp::Window::getRotation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW.MOUSE.BUTTON_[1,2]).
---------------	--

戻り値

回転を行う GgQuaternion 型の四元数.

GgApp.h の 927 行目に定義があります。

8.43.3.26 getRotationMatrix()

```
auto GgApp::Window::getRotationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う **GgMatrix** 型の変換行列.

GgApp.h の 940 行目に定義があります。

8.43.3.27 getScrollMatrix()

```
auto GgApp::Window::getScrollMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによってオブジェクトの平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

クリッピング座標系で平行移動を行う **GgMatrix** 型の変換行列.

GgApp.h の 904 行目に定義があります。

8.43.3.28 getShiftArrow()

```
void GgApp::Window::getShiftArrow (
    GLfloat * shift_arrow) const [inline]
```

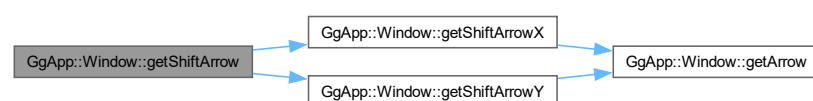
SHIFT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>shift_arrow</i>	SHIFT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
--------------------	---

GgApp.h の 680 行目に定義があります。

呼び出し関係図:



8.43.3.29 getShiftArrowX()

```
auto GgApp::Window::getShiftArrowX () const [inline]
```

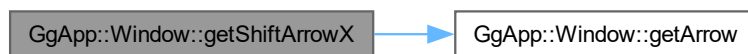
SHIFT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 660 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.30 getShiftArrowY()

```
auto GgApp::Window::getShiftArrowY () const [inline]
```

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 670 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



8.43.3.31 getSize() [1/2]

```
const auto & GgApp::Window::getSize () const [inline]
```

ウィンドウのサイズを得る.

戻り値

ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列の参照.

[GgApp.h](#) の 505 行目に定義があります.

8.43.3.32 getSize() [2/2]

```
void GgApp::Window::getSize (
    GLsizei * size) const [inline]
```

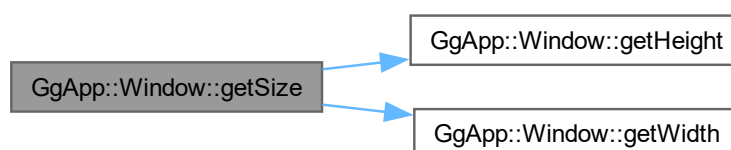
ウィンドウのサイズを得る.

引数

<i>size</i>	ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列.
-------------	-------------------------------------

[GgApp.h](#) の 515 行目に定義があります.

呼び出し関係図:



8.43.3.33 getTranslation()

```
const auto & GgApp::Window::getTranslation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボール処理を考慮したマウスによるスクロールの変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

平行移動量を格納した GLfloat[3] の配列のポインタ.

GgApp.h の 868 行目に定義があります。

8.43.3.34 getTranslationMatrix()

```
auto GgApp::Window::getTranslationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによって視点の平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

視点座標系で平行移動を行う GgMatrix 型の変換行列.

GgApp.h の 881 行目に定義があります。

8.43.3.35 getUserPointer()

```
void * GgApp::Window::getUserPointer () const [inline]
```

ユーザーポインタを取り出す.

戻り値

保存されているユーザポインタ.

GgApp.h の 982 行目に定義があります。

8.43.3.36 getWheel() [1/3]

```
const auto * GgApp::Window::getWheel () const [inline]
```

マウスホイールの回転量を得る.

戻り値

マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.

GgApp.h の 810 行目に定義があります。

8.43.3.37 getWheel() [2/3]

```
void GgApp::Window::getWheel (
    GLfloat * rotation) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>rotation</i>	マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.
-----------------	--------------------------------------

[GgApp.h](#) の 821 行目に定義があります。

8.43.3.38 getWheel() [3/3]

```
auto GgApp::Window::getWheel (
    int direction) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスホイールの回転量.

[GgApp.h](#) の 834 行目に定義があります。

8.43.3.39 getWheelX()

```
auto GgApp::Window::getWheelX () const [inline]
```

マウスホイールの X 方向の回転量を得る.

戻り値

マウスホイールの X 方向の回転量.

[GgApp.h](#) の 845 行目に定義があります。

8.43.3.40 getWheelY()

```
auto GgApp::Window::getWheelY () const [inline]
```

マウスホイールの Y 方向の回転量を得る.

戻り値

マウスホイールの Y 方向の回転量.

[GgApp.h](#) の 856 行目に定義があります。

8.43.3.41 getWidth()

```
auto GgApp::Window::getWidth () const [inline]
```

ウィンドウの横幅を得る.

戻り値

ウィンドウの横幅.

GgApp.h の 465 行目に定義があります。

被呼び出し関係図:



8.43.3.42 operator bool()

```
GgApp::Window::operator bool () [explicit]
```

イベントを取得してループを継続すべきかどうか調べる.

戻り値

ループを継続すべきなら true.

GgApp.cpp の 648 行目に定義があります。

呼び出し関係図:



8.43.3.43 operator=() [1/2]

```
Window & GgApp::Window::operator= (  
    const Window & w) [delete]
```

代入演算子を使用しない.

引数

<i>w</i>	代入元のウィンドウ.
----------	------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



8.43.3.44 operator=() [2/2]

```
GgApp::Window & GgApp::Window::operator= (  
    Window && w) [noexcept]
```

ムーブ代入演算子.

引数

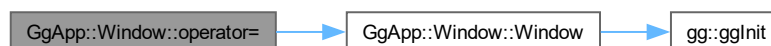
<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

[GgApp.cpp](#) の 611 行目に定義があります。

呼び出し関係図:



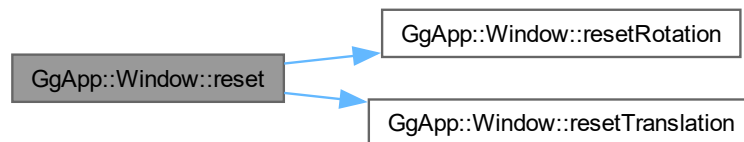
8.43.3.45 reset()

```
void GgApp::Window::reset () [inline]
```

トラックボール・マウスホイール・矢印キーの値を初期化する。

[GgApp.h](#) の 968 行目に定義があります。

呼び出し関係図:



8.43.3.46 resetRotation()

```
void GgApp::Window::resetRotation () [inline]
```

トラックボール処理を初期化する。

[GgApp.h](#) の 950 行目に定義があります。

被呼び出し関係図:



8.43.3.47 resetTranslation()

```
void GgApp::Window::resetTranslation () [inline]
```

平行移動量を初期化する。

[GgApp.h](#) の 959 行目に定義があります。

被呼び出し関係図:



8.43.3.48 restoreViewport()

```
void GgApp::Window::restoreViewport () const [inline]
```

ビューポートを元のサイズに復帰する.

[GgApp.h](#) の 430 行目に定義があります.

被呼び出し関係図:

**8.43.3.49 selectInterface()**

```
void GgApp::Window::selectInterface (
    int no) [inline]
```

インタフェースを選択する.

引数

<i>no</i>	インターフェース番号.
-----------	-------------

[GgApp.h](#) の 577 行目に定義があります.

8.43.3.50 setClose()

```
void GgApp::Window::setClose (
    int flag = 1) const [inline]
```

ウィンドウのクローズフラグを設定する.

引数

<i>flag</i>	クローズフラグ, 0 (GLFW.FALSE) 以外ならウィンドウを閉じる.
-------------	--

[GgApp.h](#) の 393 行目に定義があります.

8.43.3.51 setKeyboardFunc()

```
void GgApp::Window::setKeyboardFunc (
    void(* func ) (const Window *window, int key, int scancode, int action, int mods))
[inline]
```

ユーザ定義の keyboard 関数を設定する.

引数

<i>func</i>	ユーザ定義の keyboard 関数, キーボードの操作時に呼び出される.
-------------	--

GgApp.h の 1012 行目に定義があります。

呼び出し関係図:



8.43.3.52 setMouseFunc()

```
void GgApp::Window::setMouseFunc (  
    void(* func )(const Window *window, int button, int action, int mods)) [inline]
```

ユーザ定義の **mouse** 関数を設定する.

引数

<i>func</i>	ユーザ定義の mouse 関数, マウスボタンの操作時に呼び出される.
-------------	--

GgApp.h の 1022 行目に定義があります。

呼び出し関係図:



8.43.3.53 setResizeFunc()

```
void GgApp::Window::setResizeFunc (  
    void(* func )(const Window *window, int width, int height)) [inline]
```

ユーザ定義の **resize** 関数を設定する.

引数

<i>func</i>	ユーザ定義の resize 関数, ウィンドウのサイズ変更時に呼び出される.
-------------	---

[GgApp.h](#) の 1002 行目に定義があります。

呼び出し関係図:



8.43.3.54 setUserPointer()

```
void GgApp::Window::setUserPointer (
    void * pointer) [inline]
```

任意のユーザポインタを保存する。

引数

<i>pointer</i>	保存するユーザポインタ.
----------------	--------------

[GgApp.h](#) の 992 行目に定義があります。

8.43.3.55 setVelocity()

```
void GgApp::Window::setVelocity (
    GLfloat vx,
    GLfloat vy,
    GLfloat vz = 0.1f) [inline]
```

マウスの移動速度を設定する。

引数

<i>vx</i>	x 方向の移動速度.
<i>vy</i>	y 方向の移動速度.
<i>vz</i>	z 方向の移動速度.

[GgApp.h](#) の 590 行目に定義があります。

8.43.3.56 setWheelFunc()

```
void GgApp::Window::setWheelFunc (
    void(* func) (const Window *window, double x, double y)) [inline]
```

ユーザ定義の wheel 関数を設定する。

引数

<code>func</code>	ユーザ定義の <code>wheel</code> 関数, マウスホイールの操作時に呼び出される.
-------------------	---

[GgApp.h](#) の 1032 行目に定義があります。

呼び出し関係図:



8.43.3.57 shouldClose()

```
bool GgApp::Window::shouldClose () const [inline]
```

ウィンドウを閉じるべきかどうか調べる.

戻り値

ウィンドウを閉じるべきなら `true`.

[GgApp.h](#) の 405 行目に定義があります。

被呼び出し関係図:



8.43.3.58 swapBuffers()

```
void GgApp::Window::swapBuffers () const
```

カラーバッファを入れ替える.

[GgApp.cpp](#) の 732 行目に定義があります。

被呼び出し関係図:



8.43.3.59 updateViewport()

```
void GgApp::Window::updateViewport ()
```

ビューポートのサイズを更新する.

[GgApp.cpp](#) の 758 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

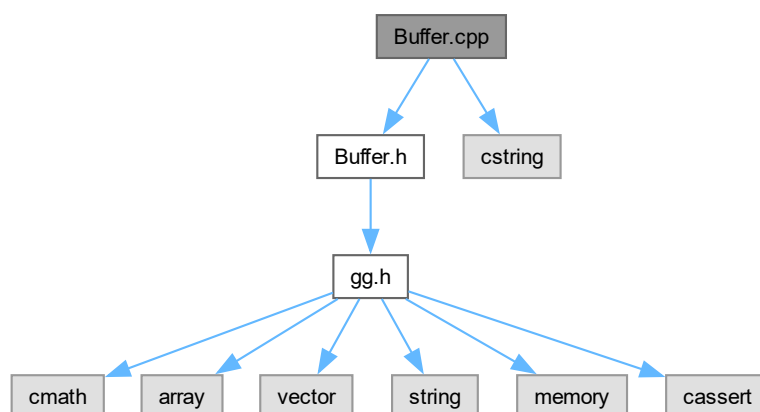
- [GgApp.h](#)
- [GgApp.cpp](#)

Chapter 9

ファイル詳解

9.1 Buffer.cpp ファイル

```
#include "Buffer.h"  
#include <cstring>  
Buffer.cpp の依存先関係図:
```



9.1.1 詳解

バッファクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Buffer.cpp](#) に定義があります。

9.2 Buffer.cpp

[詳解]

```

00001
00008 #include "Buffer.h"
00009
00010 // 標準ライブラリ
00011 #include <cstring>
00012
00013 //
00014 // フォーマットからチャンネル数を求める
00015 //
00016 int Buffer::formatToChannels(GLenum format) const
00017 {
00018     switch (format)
00019     {
00020     case GL_RED:
00021         return 1;
00022     case GL_RG:
00023         return 2;
00024     case GL_RGB:
00025         return 3;
00026     case GL_RGBA:
00027         return 4;
00028     default:
00029         assert(false);
00030         break;
00031     }
00032
00033     return 0;
00034 };
00035
00036 //
00037 // 既存のバッファにコピーする
00038 //
00039 void Buffer::copyBuffer(const Buffer& buffer) noexcept
00040 {
00041     #if defined(USE_PIXEL_BUFFER_OBJECT)
00042         // コピー元のピクセルバッファオブジェクト
00043         glBindBuffer(GL_COPY_READ_BUFFER, buffer.bufferName);
00044
00045         // コピー先のピクセルバッファオブジェクト
00046         glBindBuffer(GL_COPY_WRITE_BUFFER, bufferName);
00047
00048         // バッファオブジェクトをコピーする
00049         glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
00050                             0, 0, std::min(bufferLength, buffer.bufferLength));
00051     #else
00052         // フレームのデータをコピーする
00053         memcpy(bufferName.data(), buffer.bufferName.data(), bufferName.size());
00054     #endif
00055 }
00056
00057 //
00058 // バッファを作成する
00059 //
00060 void Buffer::create(GLsizei width, GLsizei height, int channels)
00061 {
00062     // 指定したサイズが保持しているフレームのサイズと同じなら何もしない
00063     if (width == bufferSize[0] && height == bufferSize[1]
00064         && channels == bufferChannels) return;
00065
00066     // フレームのサイズとチャンネル数を記録する
00067     bufferSize = std::array<int, 2>{ width, height };
00068     bufferChannels = channels;
00069
00070     #if defined(USE_PIXEL_BUFFER_OBJECT)
00071         // フレームの保存に必要なメモリ量を求める
00072         bufferLength = width * height * channels;
00073
00074         // 以前のピクセルバッファオブジェクトを破棄する
00075         glDeleteBuffers(1, &bufferName);
00076
00077         // 新しいピクセルバッファオブジェクトを作成する
00078         glGenBuffers(1, &bufferName);
00079
00080         // ピクセルバッファオブジェクトのメモリを確保してデータを転送する
00081         glBindBuffer(GL_PIXEL_PACK_BUFFER, bufferName);
00082         glBufferData(GL_PIXEL_PACK_BUFFER, bufferLength, nullptr, GL_DYNAMIC_COPY);
00083         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00084     #else
00085         // メモリを確保する
00086         bufferName.resize(width * height * channels);
00087     #endif
00088 }

```

```

00089
00090 //
00091 // バッファをコピーする
00092 //
00093 void Buffer::copy(const Buffer& buffer) noexcept
00094 {
00095     // コピー元と同じサイズの空のバッファを作る
00096     Buffer::create(buffer.bufferSize[0], buffer.bufferSize[1],
00097         buffer.bufferChannels);
00098
00099     // 作成されたバッファにコピーする
00100     copyBuffer(buffer);
00101 }
00102
00103 //
00104 // 代入演算子
00105 //
00106 Buffer& Buffer::operator=(const Buffer& buffer)
00107 {
00108     // 代入元と代入先が同じなら何もしない
00109     if (&buffer == this) return *this;
00110
00111     // 引数のバッファをバッファをこのバッファにコピーする
00112     Buffer::copy(buffer);
00113
00114     // このバッファを返す
00115     return *this;
00116 }
00117
00118 //
00119 // ムーブ代入演算子
00120 //
00121 Buffer& Buffer::operator=(Buffer&& buffer) noexcept
00122 {
00123     // ムーブ代入元とムーブ代入先が同じなら何もしない
00124     if (&buffer == this) return *this;
00125
00126     // ムーブ代入元のバッファのメンバをムーブする
00127     bufferSize = buffer.bufferSize;
00128     buffer.bufferSize = { 0, 0 };
00129     buffer.channels = buffer.bufferChannels;
00130     buffer.bufferChannels = 0;
00131 #if defined(USE_PIXEL_BUFFER_OBJECT)
00132     buffer.length = buffer.bufferLength;
00133     buffer.bufferLength = 0;
00134
00135     // ピクセルバッファオブジェクトを削除する
00136     glDeleteBuffers(1, &bufferName);
00137
00138     // ムーブ代入元のピクセルバッファオブジェクト名をムーブ代入先に移す
00139     bufferName = buffer.bufferName;
00140
00141     // ムーブ代入元のデストラクタでバッファが削除されないよう 0 にする
00142     buffer.bufferName = 0;
00143 #endif
00144
00145     // このバッファを返す
00146     return *this;
00147 }
00148
00149 //
00150 // ピクセルバッファオブジェクトをマップする
00151 //
00152 GLvoid* Buffer::map()
00153 #if defined(USE_PIXEL_BUFFER_OBJECT)
00154 const
00155 #endif
00156 {
00157     #if defined(USE_PIXEL_BUFFER_OBJECT)
00158         // ピクセルバッファオブジェクトを参照する
00159         glBindBuffer(GL_PIXEL_PACK_BUFFER, bufferName);
00160
00161         // ピクセルバッファオブジェクトをマップする
00162         return glMapBufferRange(GL_PIXEL_PACK_BUFFER, 0, bufferLength,
00163             GL_MAP_READ_BIT | GL_MAP_WRITE_BIT);
00164     #else
00165         // データの格納場所を返す
00166         return bufferName.data();
00167     #endif
00168 }
00169
00170 //
00171 // ピクセルバッファオブジェクトをアンマップする
00172 //
00173 void Buffer::unmap() const
00174 {
00175     #if defined(USE_PIXEL_BUFFER_OBJECT)

```

```

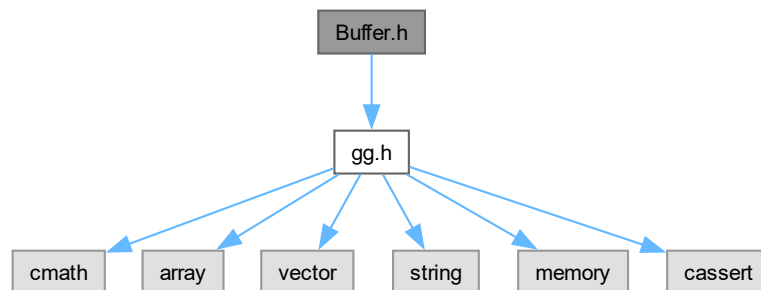
00176 // ピクセルバッファオブジェクトのマップを解除する
00177 glUnmapBuffer(GL_PIXEL_PACK_BUFFER);
00178
00179 // アップロード先のピクセルバッファオブジェクトの結合を解除する
00180 glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00181 #endif
00182 }
00183
00184 //
00185 // バッファを破棄する
00186 //
00187 void Buffer::discard()
00188 {
00189     #if defined(USE_PIXEL_BUFFER_OBJECT)
00190         // ピクセルバッファオブジェクトの結合を解除する
00191         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00192         glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
00193
00194         // ピクセルバッファオブジェクトを削除する
00195         glDeleteBuffers(1, &bufferName);
00196         bufferName = 0;
00197     #else
00198         // メモリを消去する
00199         bufferName.clear();
00200     #endif
00201
00202     // バッファのサイズを 0 にする
00203     bufferSize = std::array<int, 2>{ 0, 0 };
00204     bufferChannels = 0;
00205 }

```

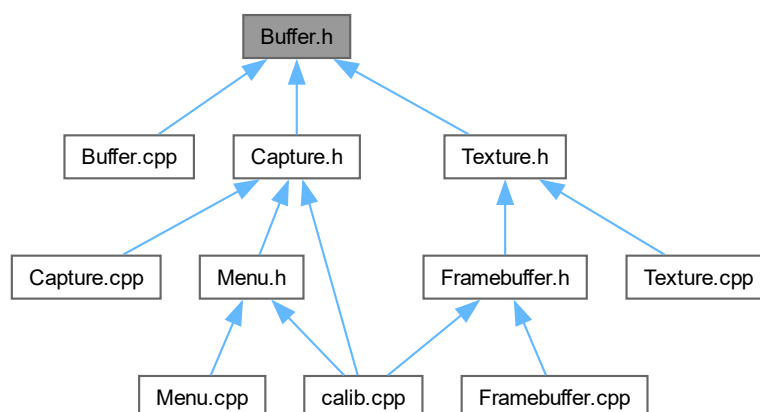
9.3 Buffer.h ファイル

```
#include "gg.h"
```

Buffer.h の依存先関係図:



被依存関係図:



クラス

- class [Buffer](#)

マクロ定義

- `#define` [USE_PIXEL_BUFFER_OBJECT](#)

9.3.1 詳解

バッファクラスの定義

著者

Kohe Tokoi

日付

Aplil 3, 2023

[Buffer.h](#) に定義があります。

9.3.2 マクロ定義詳解

9.3.2.1 USE_PIXEL_BUFFER_OBJECT

```
#define USE_PIXEL_BUFFER_OBJECT
```

[Buffer.h](#) の 16 行目に定義があります。

9.4 Buffer.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00015 // ピクセルバッファオブジェクトを使うとき
00016 #define USE_PIXEL_BUFFER_OBJECT
00017
00024 class Buffer
00025 {
00026     std::array<int, 2> bufferSize{ 0, 0 };
00027
00028     int bufferChannels{ 0 };
00029
00032 #if defined(USE_PIXEL_BUFFER_OBJECT)
00033     GLsizei bufferLength{ 0 };
00034
00037     GLuint
00038 #else
00039     std::vector<GLubyte>
00040 #endif
00041     bufferName{ 0 };
00042
00043 protected:
00044
00052     auto channelsToFormat(int channels) const
00053     {
00054         // OpenGL のテクスチャフォーマットのリスト
00055         static constexpr GLenum toFormat[] { GL_RED, GL_RG, GL_RGB, GL_RGBA };
00056
00057         // フォーマットを返す
00058         return toFormat[(channels - 1) & 3];
00059     }
00060
00067     int formatToChannels(GLenum format) const;
00068
00074     void copyBuffer(const Buffer& buffer) noexcept;
00075
00076 public:
00077
00081     Buffer() = default;
00082
00094     Buffer(GLsizei width, GLsizei height, int channels)
00095     {
00096         Buffer::create(width, height, channels);
00097     }
00098
00104     Buffer(const Buffer& buffer)
00105     {
00106         Buffer::copy(buffer);
00107     }
00108
00114     Buffer(Buffer&& buffer) noexcept
00115         : bufferSize{ buffer.bufferSize }
00116         , bufferChannels{ buffer.bufferChannels }
00117     #if defined(USE_PIXEL_BUFFER_OBJECT)
00118         , bufferLength{ buffer.bufferLength }
00119         , bufferName{ buffer.bufferName }
00120     #endif
00121     {
00122         buffer.bufferSize = { 0, 0 };
00123         buffer.bufferChannels = 0;
00124     #if defined(USE_PIXEL_BUFFER_OBJECT)
00125         buffer.bufferLength = 0;
00126
00127         // ムーブ元のデストラクタでバッファが削除されないよう 0 にする
00128         buffer.bufferName = 0;
00129     #endif
00130     }
00131
00135     virtual ~Buffer()
00136     {
00137         Buffer::discard();
00138     }
00139
00146     Buffer& operator=(const Buffer& buffer);
00147
00154     Buffer& operator=(Buffer&& buffer) noexcept;
00155
00167     virtual void create(GLsizei width, GLsizei height, int channels);

```



```

00168
00180 void resize(GLsizei width, GLsizei height, int channels)
00181 {
00182     create(width, height, channels);
00183 }
00184
00194 void resize(const Buffer& buffer)
00195 {
00196     create(buffer.getWidth(), buffer.getHeight(), buffer.getChannels());
00197 }
00198
00208 virtual void copy(const Buffer& buffer) noexcept;
00209
00213 virtual void discard();
00214
00220 #if defined(USE_PIXEL_BUFFER_OBJECT)
00221 auto getBufferName() const
00222 #else
00223 auto& getBufferName()
00224 #endif
00225 {
00226     return bufferName;
00227 }
00228
00234 virtual const std::array<GLsizei, 2>& getSize() const
00235 {
00236     return bufferSize;
00237 }
00238
00244 GLsizei getWidth() const
00245 {
00246     return getSize()[0];
00247 }
00248
00254 GLsizei getHeight() const
00255 {
00256     return getSize()[1];
00257 }
00258
00264 virtual int getChannels() const
00265 {
00266     return bufferChannels;
00267 }
00268
00274 auto getFormat() const
00275 {
00276     return channelsToFormat(getChannels());
00277 }
00278
00284 auto getAspect() const
00285 {
00286     return static_cast<GLfloat>(getWidth()) / static_cast<GLfloat>(getHeight());
00287 }
00288
00294 void bindBuffer(GLenum target = GL_PIXEL_PACK_BUFFER) const
00295 {
00296     #if defined(USE_PIXEL_BUFFER_OBJECT)
00297         glBindBuffer(target, bufferName);
00298     #endif
00299 }
00300
00306 void unbindBuffer(GLenum target = GL_PIXEL_PACK_BUFFER) const
00307 {
00308     #if defined(USE_PIXEL_BUFFER_OBJECT)
00309         glBindBuffer(target, 0);
00310     #endif
00311 }
00312
00318 GLvoid* map()
00319 #if defined(USE_PIXEL_BUFFER_OBJECT)
00320     const
00321 #endif
00322     ;
00323
00327 void unmap() const;
00328 };

```

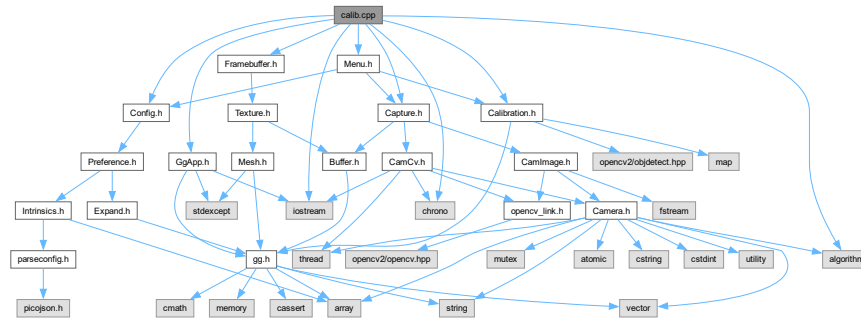
9.5 calib.cpp ファイル

```

#include "GgApp.h"
#include "Config.h"

```

```
#include "Capture.h"
#include "Calibration.h"
#include "Menu.h"
#include "Framebuffer.h"
#include <algorithm>
#include <chrono>
#include <iostream>
calib.cpp の依存先関係図:
```



マクロ定義

- #define **CONFIG_FILE** PROJECT_NAME "_config.json"

9.5.1 詳解

ChArUco Board によるカメラキャリブレーション

著者

Kohe Tokoi

日付

March 6, 2024

calib.cpp に定義があります。

9.5.2 マクロ定義詳解

9.5.2.1 CONFIG_FILE

```
#define CONFIG_FILE PROJECT_NAME "_config.json"
```

calib.cpp の 33 行目に定義があります。

9.6 calib.cpp

[詳解]

```

00001
00008
00009 // ウィンドウ関連の処理
00010 #include "GgApp.h"
00011
00012 // 構成データ
00013 #include "Config.h"
00014
00015 // キャプチャデバイス
00016 #include "Capture.h"
00017
00018 // 校正
00019 #include "Calibration.h"
00020
00021 // メニュー
00022 #include "Menu.h"
00023
00024 // フレームバッファオブジェクト
00025 #include "Framebuffer.h"
00026
00027 // 標準ライブラリ
00028 #include <algorithm>
00029 #include <chrono>
00030 #include <iostream>
00031
00032 // 構成ファイル名
00033 #define CONFIG_FILE PROJECT_NAME "_config.json"
00034
00035 //
00036 // アプリケーション本体
00037 //
00038 int GgApp::main(int argc, const char* const* argv)
00039 {
00040     #if defined(GG_USE_OPENXR)
00041         // --openxr が指定されたときだけ HMD を起動する
00042         const bool useOpenXr{ std::find(argv + 1, argv + argc, std::string{ "--openxr" }) != argv + argc };
00043     #endif
00044
00045     // 構成ファイルを読み込む
00046     Config config{ CONFIG_FILE };
00047
00048     // 構成にもとづいてウィンドウを作成する
00049     GgApp::Window window{ config.getTitle(), config.getWidth(), config.getHeight() };
00050
00051     #if defined(GG_USE_OPENXR)
00052         // OpenXR の初期化
00053         GgApp::OpenXR* openxr{ nullptr };
00054         if (useOpenXr)
00055         {
00056             try
00057             {
00058                 openxr = &GgApp::OpenXR::initialize(window, XR_REFERENCE_SPACE_TYPE_STAGE,
00059                 config.getTitle().c_str());
00059             }
00060             catch (const std::exception& e)
00061             {
00062                 std::cerr << "OpenXR: " << e.what() << '\n';
00063                 std::cerr << "OpenXR is not available; continuing with the desktop display.\n";
00064             }
00065         }
00066     #endif
00067
00068     // 開いたウィンドウに対して初期化処理を実行する
00069     config.initialize();
00070
00071     // キャプチャデバイスを作る
00072     Capture capture;
00073
00074     // 校正オブジェクトを作成する
00075     Calibration calibration{ config.getDictionaryName(), config.getCheckerSize(),
00076     config.getCheckerLength() };
00077
00078     // メニューを作る
00079     Menu menu{ config, capture, calibration };
00080
00081     // キャプチャデバイスで初期画像を開く
00082     if (!capture.openImage(config.getInitialImage())) throw std::runtime_error("Cannot open initial
00083     image.");
00084
00085     // 初期画像の実解像度と焦点距離から、見やすい初期画角を設定する
00086     menu.initializeInputIntrinsics(capture.getSize());
00087

```

```

00086 // キャプチャしたフレームを保持するテクスチャ
00087 Texture frame;
00088
00089 // 画像の展開に用いるフレームバッファオブジェクトのサイズを初期ウィンドウに合わせる
00090 Framebuffer framebuffer{ config.getWidth(), config.getHeight() };
00091
00092 // ウィンドウが開いている間繰り返す
00093 while (window && menu)
00094 {
00095     // フレーム間の実経過時間 (deltaTime) を計測する
00096     static auto lastFrameTime{ std::chrono::steady_clock::now() };
00097     const auto currentFrameTime{ std::chrono::steady_clock::now() };
00098     float deltaTime{ std::chrono::duration<float>(currentFrameTime - lastFrameTime).count() };
00099     lastFrameTime = currentFrameTime;
00100     if (deltaTime <= 0.0f || deltaTime > 0.5f) deltaTime = 0.033f;
00101
00102     // 描画フレームレートの実測と診断出力 (1秒ごと)
00103     static auto lastRenderFpsReport{ std::chrono::steady_clock::now() };
00104     static int renderFrameCount{ 0 };
00105     ++renderFrameCount;
00106     const auto renderElapsed{ std::chrono::duration<double>(currentFrameTime -
lastRenderFpsReport).count() };
00107     if (renderElapsed >= 2.0)
00108     {
00109         const double rFps{ renderFrameCount / renderElapsed };
00110         std::cout << "calib: Render FPS = " << rFps << std::endl;
00111         renderFrameCount = 0;
00112         lastRenderFpsReport = currentFrameTime;
00113     }
00114
00115     // メニューを表示して設定を更新する
00116     menu.draw();
00117
00118     // 選択しているキャプチャデバイスから1フレーム取得する
00119     capture.retrieve(frame);
00120
00121     // ピクセルバッファオブジェクトの内容をテクスチャに転送する
00122     frame.drawPixels();
00123
00124     // フレームバッファオブジェクトのサイズをキャプチャしたフレームに合わせる
00125     framebuffer.resize(frame);
00126
00127     // シェーダの設定を行う
00128     const auto&& size{ menu.setup(framebuffer.getAspect()) };
00129
00130     // フレームバッファオブジェクトにフレームを展開する
00131     framebuffer.update(size, frame);
00132
00133     // ArUco Marker を検出するなら
00134     if (menu.detectMarker || menu.detectBoard)
00135     {
00136         // フレームバッファオブジェクトの内容をピクセルバッファオブジェクトに転送する
00137         framebuffer.readPixels();
00138
00139         // 入力画像のサイズを調べる
00140         const auto size{ cv::Size{ framebuffer.getWidth(), framebuffer.getHeight() } };
00141
00142         // ピクセルバッファオブジェクトを CPU のメモリ空間にマップする
00143         cv::Mat image{ size, CV_8UC(framebuffer.getChannels()), framebuffer.map() };
00144
00145         // ChArUco Board を認識するなら
00146         if (menu.detectBoard)
00147         {
00148             // ChArUco Board を検出する
00149             calibration.detectBoard(image);
00150
00151             // 自動キャプチャの静止・多様性判定および記録処理
00152             menu.updateAutoCapture(deltaTime);
00153         }
00154         else
00155         {
00156             // ArUco Marker を検出する
00157             calibration.detectMarkers(image, menu.getMarkerLength());
00158         }
00159
00160         // ピクセルバッファオブジェクトのマップを解除する
00161         framebuffer.unmap();
00162
00163         // ピクセルバッファオブジェクトの内容をフレームバッファオブジェクトに書き戻す
00164         framebuffer.drawPixels();
00165     }
00166
00167     // 表示するウィンドウのビューポートを再設定する
00168     window.setMenuBarHeight(menu.getMenuBarHeight());
00169
00170     // シェーダーでBGRAをRGBAへ変換し、縦横比を維持して実Framebuffer領域へ中央表示する
00171     framebuffer.draw(window.getFboWidth(), window.getFboHeight());

```

```

00172
00173 #if defined(GG_USE_OPENXR)
00174 // OpenXR が実行中なら、各 view の swapchain にレンダリングする
00175 if (openxr && openxr->isRunning() && openxr->begin())
00176 {
00177     const auto viewCount{ openxr->getViewCount() };
00178     for (uint32_t view = 0; view < viewCount; ++view)
00179     {
00180         // 各眼の向きで入力画像を再展開する。単眼画像なので位置による視差は付けない。
00181         const auto& pose{ openxr->getPose(view) };
00182         const auto viewPose{ gg::ggQuaternionMatrix(gg::GgQuaternion{
00183             pose.orientation.x, pose.orientation.y,
00184             pose.orientation.z, pose.orientation.w }) };
00185         const auto& xrSize{ menu.setup(openxr->getAspect(view), viewPose) };
00186         framebuffer.update(xrSize, frame);
00187
00188         openxr->select(view);
00189         framebuffer.draw(openxr->getWidth(view), openxr->getHeight(view));
00190         openxr->commit(view);
00191     }
00192     openxr->submit(false);
00193 }
00194 #endif
00195
00196 // カラーバッファを入れ替えてイベントを取り出す
00197 window.swapBuffers();
00198 }
00199
00200 return 0;
00201 }

```

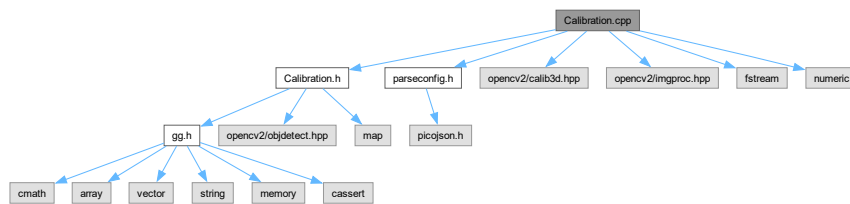
9.7 Calibration.cpp ファイル

```

#include "Calibration.h"
#include "parseconfig.h"
#include <opencv2/calib3d.hpp>
#include <opencv2/imgproc.hpp>
#include <fstream>
#include <numeric>

```

Calibration.cpp の依存先関係図:



マクロ定義

- `#define USE_RODRIGUES`

9.7.1 詳解

較正用フレームバッファオブジェクトクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Calibration.cpp](#) に定義があります。

9.7.2 マクロ定義詳解

9.7.2.1 USE_RODRIGUES

```
#define USE_RODRIGUES
```

[Calibration.cpp](#) の 23 行目に定義があります。

9.8 Calibration.cpp

[詳解]

```
00001
00008 #include "Calibration.h"
00009
00010 // 構成ファイルの読み取り補助
00011 #include "parseconfig.h"
00012
00013 // OpenCV
00014 #pragma warning(disable:4819)
00015 #include <opencv2/calib3d.hpp>
00016 #include <opencv2/imgproc.hpp>
00017
00018 // 標準ライブラリ
00019 #include <fstream>
00020 #include <numeric>
00021
00022 // cv::Rodrigues() を使う
00023 #define USE_RODRIGUES
00024
00025
00026 //
00027 // コンストラクタ
00028 //
00029 Calibration::Calibration(const std::string& dictionaryName,
00030     const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength)
00031 {
00032     // ArUco Marker の辞書を選択する
00033     setDictionary(dictionaryName, checkerSize, checkerLength);
00034 }
00035
00036 //
00037 // デストラクタ
00038 //
00039 Calibration::~Calibration()
00040 {
00041 }
00042
00043 //
00044 // ChArUco Board を作成する
00045 //
00046 void Calibration::createBoard(const std::array<int, 2>& checkerSize,
00047     const std::array<float, 2>& checkerLength)
00048 {
00049     // キャリブレーション用の ChArUco Board を作成する
00050     board = new cv::aruco::CharucoBoard(cv::Size{ checkerSize[0], checkerSize[1] },
00051         checkerLength[0] * 0.01f, checkerLength[1] * 0.01f, dictionary);
00052 }
```

```

00053 // キャリブレーション用の ChArUco Board の検出器を作成する
00054 boardDetector = new cv::aruco::CharucoDetector(*board);
00055
00056 // board は boardDetector->getBoard() で取り出すことができるか
00057 // 実行中に board を作り直すことがあるので cv::Ptr に持たせる
00058
00059 // 較正結果を再利用しない
00060 calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00061 }
00062
00063 //
00064 // ArUco Marker の辞書と検出器を設定する
00065 //
00066 void Calibration::setDictionary(const std::string& dictionaryName,
00067     const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength)
00068 {
00069     // ArUco Marker の辞書を検索する
00070     auto dictionaryItem{ dictionaryList.find(dictionaryName) };
00071
00072     // ArUco Marker の辞書が見つからなかったら辞書リストの最初の辞書を使う
00073     if (dictionaryItem == dictionaryList.end()) dictionaryItem = dictionaryList.begin();
00074
00075     // ArUco Marker の辞書を設定する
00076     dictionary = cv::aruco::getPredefinedDictionary(dictionaryItem->second);
00077
00078     // ArUco Marker の検出器を作成する
00079     cv::aruco::DetectorParameters detectorParams = cv::aruco::DetectorParameters();
00080     detector = new cv::aruco::ArucoDetector(dictionary, detectorParams);
00081
00082     // キャリブレーション用の ChArUco Board を作成する
00083     createBoard(checkerSize, checkerLength);
00084 }
00085
00086 //
00087 // ChArUco Board を描く
00088 //
00089 void Calibration::drawBoard(cv::Mat& boardImage, int width, int height)
00090 {
00091     board->generateImage(cv::Size{ width, height }, boardImage, 10, 1);
00092 }
00093
00094 //
00095 // ChArUco Board を検出する
00096 //
00097 void Calibration::detectBoard(cv::Mat& image)
00098 {
00099     // 画像のサイズを保存しておく
00100     size = image.size();
00101
00102     // 4 チャンネル画像の場合は一時的に3チャンネル画像を作成する
00103     cv::Mat tempImage;
00104     const auto isFourChannels{ image.channels() == 4 };
00105     if (isFourChannels)
00106     {
00107         cv::cvtColor(image, tempImage, cv::COLOR_BGRA2BGR);
00108     }
00109     else
00110     {
00111         tempImage = image;
00112     }
00113
00114     // ChArUco Board のコーナーを検出する
00115     boardDetector->detectBoard(tempImage, charucoCorners, charucoIds);
00116
00117     // コーナーが見つからなかったら何もしない
00118     if (charucoCorners.empty()) return;
00119
00120     // ChArUco Board のコーナーの位置を表示に描き込む
00121     cv::aruco::drawDetectedCornersCharuco(tempImage, charucoCorners, charucoIds, cv::Scalar(0, 0, 255));
00122
00123     // 4 チャンネル画像の場合は結果を書き戻す
00124     if (isFourChannels)
00125     {
00126         cv::cvtColor(tempImage, image, cv::COLOR_BGR2BGRA);
00127     }
00128 }
00129
00130 //
00131 // ArUco Marker を検出する
00132 //
00133 void Calibration::detectMarkers(cv::Mat& image, float markerLength)
00134 {
00135     // 4 チャンネル画像の場合は一時的に3チャンネル画像を作成する
00136     cv::Mat tempImage;
00137     const auto isFourChannels{ image.channels() == 4 };
00138     if (isFourChannels)
00139     {

```

```

00140     cv::cvtColor(image, tempImage, cv::COLOR_BGRA2BGR);
00141 }
00142 else
00143 {
00144     tempImage = image;
00145 }
00146
00147 // ArUco Marker のコーナーを検出する
00148 detector->detectMarkers(tempImage, corners, ids, rejected);
00149
00150 // コーナーが見つからなければ戻る
00151 if (corners.empty()) return;
00152
00153 // キャリブレーションが完了していれば
00154 if (finished())
00155 {
00156     // 各マーカに対応する3次元空間の点
00157     const float markerCenter{ markerLength * 0.5f };
00158     std::vector<cv::Point3f> markerObjPoints;
00159     markerObjPoints.push_back(cv::Point3f(-markerCenter, markerCenter, 0.0f));
00160     markerObjPoints.push_back(cv::Point3f(markerCenter, markerCenter, 0.0f));
00161     markerObjPoints.push_back(cv::Point3f(markerCenter, -markerCenter, 0.0f));
00162     markerObjPoints.push_back(cv::Point3f(-markerCenter, -markerCenter, 0.0f));
00163
00164     // 個々のマーカーについて
00165     for (size_t i = 0; i < corners.size(); ++i)
00166     {
00167         // マーカーのコーナー検出位置から3次元姿勢（回転・平行移動）を推定する
00168         cv::Vec3d rvec, tvec;
00169         cv::solvePnP(markerObjPoints, corners[i], cameraMatrix, distCoeffs, rvec, tvec, false,
00170             cv::SOLVEPNP_IPPE_SQUARE);
00171
00172         // 座標軸を描く
00173         cv::drawFrameAxes(tempImage, cameraMatrix, distCoeffs, rvec, tvec, markerLength);
00174     }
00175 }
00176 else
00177 {
00178     // ArUco Marker の場所に矩形と番号を描き込む
00179     cv::aruco::drawDetectedMarkers(tempImage, corners, ids);
00180 }
00181
00182 // 4 チャンネル画像の場合は結果を書き戻す
00183 if (isFourChannels)
00184 {
00185     cv::cvtColor(tempImage, image, cv::COLOR_BGR2BGRA);
00186 }
00187
00188 //
00189 // 標本を取得する
00190 //
00191 void Calibration::recordCorners()
00192 {
00193     // ChArUco Board のコーナーが4 つ以上見つければ
00194     if (charucoCorners.size() >= 4)
00195     {
00196         // ChArUco Board のレイアウトと検出されたコーナーから
00197         // ChArUco Board 上の点と対応する画像上の点を求める
00198         board->matchImagePoints(charucoCorners, charucoIds, objectPoints, imagePoints);
00199
00200         // ChArUco Board 上の点と対応する画像上の点が見つければ
00201         if (!imagePoints.empty() && !objectPoints.empty())
00202         {
00203             // ChArUco Board のコーナーを記録する
00204             allCorners.push_back(charucoCorners);
00205             allIds.push_back(charucoIds);
00206             allImagePoints.push_back(imagePoints);
00207             allObjectPoints.push_back(objectPoints);
00208
00209             // 記録したコーナーの数の合計を求める
00210             totalCorners += static_cast<int>(charucoCorners.size());
00211
00212             // 記録したショットの幾何特徴を保存する（多様性チェック用）
00213             lastRecordedFeatures = computeFeatures(charucoCorners);
00214             hasRecordedShot = true;
00215             stableDuration = 0.0f;
00216             isCurrentlyStable = false;
00217         }
00218     }
00219
00220 #if defined(_DEBUG)
00221     std::cerr << "charucoCorners = " << charucoCorners.size()
00222     << ", allCorners = " << allCorners.size() << "\n";
00223 #endif
00224 }
00225 //

```



```

00226 // 標本と校正結果を破棄する
00227 //
00228 void Calibration::discardCorners()
00229 {
00230     // 記録した標本を消去する
00231     allCorners.clear();
00232     allIds.clear();
00233
00234     // 校正結果を消去する
00235     cameraMatrix.release();
00236     distCoeffs.release();
00237
00238     // 検出したコーナー数の合計を 0 にする
00239     totalCorners = 0;
00240
00241     // 校正の計算結果を再利用しない
00242     calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00243
00244     // 自動キャプチャ関連の状態をリセットする
00245     hasRecordedShot = false;
00246     stableDuration = 0.0f;
00247     isCurrentlyStable = false;
00248     currentMotion = 999.0f;
00249     prevCharucoCorners.clear();
00250     prevCharucoIds.clear();
00251 }
00252
00253 //
00254 // 校正する
00255 //
00256 bool Calibration::calibrate()
00257 {
00258     // 再投影誤差はとりあえず 0 にしておく
00259     repError = 0.0f;
00260
00261     // コーナーを合計 6 つ以上検出できていれば
00262     if (allCorners.size() >= 6) try
00263     {
00264         // CALIB_USE_INTRINSIC_GUESS が設定されていない場合に、
00265         // fx と fy をしてしたアスペクト比に強制する
00266         if (calibrationFlags & cv::CALIB_FIX_ASPECT_RATIO)
00267         {
00268             const auto aspect{ static_cast<double>(size.width) / size.height };
00269             cameraMatrix = cv::Mat_<eye(3, 3, CV_64F);
00270             cameraMatrix.at<double>(0, 0) = aspect;
00271         }
00272
00273         // ChArUco Board の姿勢 (使わないので捨ててしまう)
00274         std::vector<cv::Mat> boardRvecs, boardTvecs;
00275
00276         // 取得した全てのコーナーからカメラパラメータを推定する
00277         repError = cv::calibrateCamera(allObjectPoints, allImagePoints, size,
00278             cameraMatrix, distCoeffs, boardRvecs, boardTvecs, cv::noArray(),
00279             cv::noArray(), cv::noArray(), calibrationFlags);
00280     }
00281     catch (const cv::Exception&)
00282     {
00283         // 校正に失敗した場合は計算結果を捨てる
00284         discardCorners();
00285
00286         // 校正に失敗したことを報告する
00287         return false;
00288     }
00289
00290     // 校正の計算結果を再利用する
00291     calibrationFlags |= cv::CALIB_USE_INTRINSIC_GUESS;
00292
00293     // 校正に成功したことを報告する
00294     return true;
00295 }
00296
00297 //
00298 // 回転ベクトルから姿勢の変換行列を求める
00299 //
00300 GgMatrix Calibration::RvecTvecToPose(const cv::Vec3d& rvec, const cv::Vec3d& tvec)
00301 {
00302     #if defined(USE_RODRIGUES)
00303         // 回転軸と回転角から回転の変換行列を求める
00304         cv::Mat_<double> r(3, 3);
00305         cv::Rodrigues(rvec, r);
00306
00307         // 姿勢の変換行列
00308         return GgMatrix
00309         {
00310             static_cast<GLfloat>(r[0][0]),
00311             static_cast<GLfloat>(r[1][0]),
00312             static_cast<GLfloat>(r[2][0]),

```

```

00313     0.0f,
00314     static_cast<GLfloat>(r[0][1]),
00315     static_cast<GLfloat>(r[1][1]),
00316     static_cast<GLfloat>(r[2][1]),
00317     0.0f,
00318     static_cast<GLfloat>(r[0][2]),
00319     static_cast<GLfloat>(r[1][2]),
00320     static_cast<GLfloat>(r[2][2]),
00321     0.0f,
00322     static_cast<GLfloat>(tvec[0]),
00323     static_cast<GLfloat>(tvec[1]),
00324     static_cast<GLfloat>(tvec[2]),
00325     1.0f
00326 };
00327 #else
00328 // 回転角
00329 const auto d{ cv::norm(rvec) };
00330
00331 // 回転軸ベクトル
00332 const auto rx{ static_cast<GLfloat>(rvec[0] / d) };
00333 const auto ry{ static_cast<GLfloat>(rvec[1] / d) };
00334 const auto rz{ static_cast<GLfloat>(rvec[2] / d) };
00335
00336 // 平行移動量
00337 const auto tx{ static_cast<GLfloat>(tvec[0]) };
00338 const auto ty{ static_cast<GLfloat>(tvec[1]) };
00339 const auto tz{ static_cast<GLfloat>(tvec[2]) };
00340
00341 // 姿勢の変換行列
00342 return ggTranslate(tx, ty, tz).rotate(rx, ry, rz, static_cast<GLfloat>(d));
00343 #endif
00344 }
00345
00346 //
00347 // ArUco Marker の3次元姿勢の変換行列を求める
00348 //
00349 void Calibration::getAllMarkerPoses(float markerLength, std::map<int, GgMatrix>& poses)
00350 {
00351     // 各マーカに対応する3次元空間の点
00352     const float markerCenter = markerLength * 0.5f;
00353     std::vector<cv::Point3f> markerObjPoints;
00354     markerObjPoints.push_back(cv::Point3f(-markerCenter, markerCenter, 0.0f));
00355     markerObjPoints.push_back(cv::Point3f(markerCenter, markerCenter, 0.0f));
00356     markerObjPoints.push_back(cv::Point3f(markerCenter, -markerCenter, 0.0f));
00357     markerObjPoints.push_back(cv::Point3f(-markerCenter, -markerCenter, 0.0f));
00358
00359     // 個々のマーカについて
00360     for (size_t i = 0; i < corners.size(); ++i)
00361     {
00362         // マーカーのコーナー検出位置から3次元姿勢（回転・平行移動）を推定する
00363         cv::Vec3d rvec, tvec;
00364         cv::solvePnP(markerObjPoints, corners[i], cameraMatrix, distCoeffs, rvec, tvec, false,
00365             cv::SOLVEPNP_IPPE_SQUARE);
00366
00367         // 各マーカの姿勢の変換行列を求める
00368         poses[ids[i]] = RvecTvecToPose(rvec, tvec);
00369     }
00370 }
00371 //
00372 // カメラパラメータの JSON オブジェクトから数値の配列を取得する
00373 //
00374 static bool getMatrix(const picojson::object& object,
00375     const std::string& key, cv::Mat& mat, int cols, int rows)
00376 {
00377     // key に一致するオブジェクトを探す
00378     const auto&& value{ object.find(key) };
00379
00380     // オブジェクトが無い配列でなかったら戻る
00381     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00382
00383     // 配列を取り出す
00384     const auto& array{ value->second.get<picojson::array>() };
00385
00386     // 配列の要素数とデータの格納先の行列の要素数が一致していなければ戻る
00387     if (static_cast<size_t>(cols) * rows != array.size()) return false;
00388
00389     // カメラ行列の要素を確保する
00390     mat = cv::Mat::zeros(rows, cols, CV_64F);
00391
00392     // 配列の要素について
00393     for (size_t i = 0; i < array.size(); ++i)
00394     {
00395         // 行列の要素の位置
00396         const auto x{ static_cast<int>(i % cols) };
00397         const auto y{ static_cast<int>(i / cols) };
00398     }

```

```

00399 // 要素が数値なら格納する
00400 if (array[i].is<double>()) mat.at<double>(y, x) = array[i].get<double>();
00401 }
00402
00403 return true;
00404 }
00405
00406 //
00407 // カメラパラメータの JSON オブジェクトから数値の配列を取得する
00408 //
00409 static void setMatrix(picojson::object& object,
00410 const std::string& key, const cv::Mat& mat)
00411 {
00412 // picojson の配列
00413 picojson::array array;
00414
00415 // 配列の要素について
00416 for (size_t i = 0; i < mat.total(); ++i)
00417 {
00418 // 行列の要素の位置
00419 const auto x{ static_cast<int>(i % mat.cols) };
00420 const auto y{ static_cast<int>(i / mat.cols) };
00421
00422 // 要素を picojson::array に追加する
00423 array.emplace_back(picojson::value(mat.at<double>(y, x)));
00424 }
00425
00426 // オブジェクトに追加する
00427 object.emplace(key, array);
00428 }
00429
00430 //
00431 // ファイルからキャリブレーションパラメータを読み込む
00432 //
00433 bool Calibration::loadParameters(const std::string& filename)
00434 {
00435 // パラメータファイルの読み込み
00436 std::ifstream json{ Utf8ToTChar(filename) };
00437 if (!json) return false;
00438
00439 // JSON の読み込み
00440 picojson::value value;
00441 json >> value;
00442 json.close();
00443
00444 // 構成内容の取り出し
00445 const auto& object{ value.get<picojson::object>() };
00446
00447 // オブジェクトが空だったらエラー
00448 if (object.empty()) return false;
00449
00450 // カメラ行列
00451 if (!getMatrix(object, "camera matrix", cameraMatrix, 3, 3)) return false;
00452
00453 // 歪み定数
00454 if (!getMatrix(object, "distortion", distCoeffs, 5, 1)) return false;
00455
00456 // 再投影誤差
00457 getValue(object, "error", repError);
00458
00459 // 校正の計算結果を再利用しない
00460 calibrationFlags &= ~cv::CALIB_USE_INTRINSIC_GUESS;
00461
00462 // キャリブレーションパラメータの読み込み
00463 return true;
00464 }
00465
00466 //
00467 // キャリブレーションパラメータをファイルに保存する
00468 //
00469 bool Calibration::saveParameters(const std::string& filename) const
00470 {
00471 // 設定値を保存する
00472 std::ofstream config{ Utf8ToTChar(filename) };
00473 if (!config) return false;
00474
00475 // オブジェクト
00476 picojson::object object;
00477
00478 // カメラ行列
00479 setMatrix(object, "camera matrix", cameraMatrix);
00480
00481 // 歪み定数
00482 setMatrix(object, "distortion", distCoeffs);
00483
00484 // 再投影誤差
00485 setValue(object, "error", repError);

```

```

00486
00487 // 構成をシリアルライズして保存
00488 picojson::value v{ object };
00489 config << v.serialize(true);
00490 config.close();
00491
00492 // キャリブレーションパラメータの書き込み
00493 return true;
00494 }
00495
00496 // ArUco Marker 辞書のリスト
00497 const std::map<const std::string, const cv::aruco::PredefinedDictionaryType>
Calibration::dictionaryList
00498 {
00499 { "DICT_4X4_50", cv::aruco::DICT_4X4_50 },
00500 { "DICT_4X4_100", cv::aruco::DICT_4X4_100 },
00501 { "DICT_4X4_250", cv::aruco::DICT_4X4_250 },
00502 { "DICT_4X4_1000", cv::aruco::DICT_4X4_1000 },
00503 { "DICT_5X5_50", cv::aruco::DICT_5X5_50 },
00504 { "DICT_5X5_100", cv::aruco::DICT_5X5_100 },
00505 { "DICT_5X5_250", cv::aruco::DICT_5X5_250 },
00506 { "DICT_5X5_1000", cv::aruco::DICT_5X5_1000 },
00507 { "DICT_6X6_50", cv::aruco::DICT_6X6_50 },
00508 { "DICT_6X6_100", cv::aruco::DICT_6X6_100 },
00509 { "DICT_6X6_250", cv::aruco::DICT_6X6_250 },
00510 { "DICT_6X6_1000", cv::aruco::DICT_6X6_1000 },
00511 { "DICT_7X7_50", cv::aruco::DICT_7X7_50 },
00512 { "DICT_7X7_100", cv::aruco::DICT_7X7_100 },
00513 { "DICT_7X7_250", cv::aruco::DICT_7X7_250 },
00514 { "DICT_7X7_1000", cv::aruco::DICT_7X7_1000 },
00515 { "DICT_ARUCO_ORIGINAL", cv::aruco::DICT_ARUCO_ORIGINAL },
00516 { "DICT_APRILTAG_16h5", cv::aruco::DICT_APRILTAG_16h5 },
00517 { "DICT_APRILTAG_25h9", cv::aruco::DICT_APRILTAG_25h9 },
00518 { "DICT_APRILTAG_36h10", cv::aruco::DICT_APRILTAG_36h10 },
00519 { "DICT_APRILTAG_36h11", cv::aruco::DICT_APRILTAG_36h11 }
00520 };
00521
00522 //
00523 // コーナー群から幾何特徴を計算する
00524 //
00525 // 【目的】
00526 //   ChArUco ボードの画像内での配置状態（位置、見かけの大きさ、傾き）を
00527 //   統計的モーメントを用いてロバストに要約し、特徴量として抽出する。
00528 //   直前に記録した標本の特徴量と比較することで、サンプルの多様性（位置移動、
00529 //   カメラとの距離変化、回転・傾き）を判定するために利用する。
00530 //
00531 Calibration::BoardPoseFeatures Calibration::computeFeatures(const std::vector<cv::Point2f>& corners)
00532 {
00533     BoardPoseFeatures features;
00534
00535     // 4点未満では面としての幾何特徴（慣性楕円など）を安定して算出できないため除外
00536     if (corners.size() >= 4)
00537     {
00538         // コーナー点群の統計モーメントを算出
00539         //   m00: 点の総数（質量）
00540         //   m10, m01: 1次モーメント（座標総和）
00541         //   mu20, mu02, mu11: 重心まわりの2次中心モーメント（分散・共分散）
00542         cv::Moments m = cv::moments(corners);
00543         if (m.m00 > 0.0)
00544         {
00545             // 1. 重心位置 (Centroid): 画像内でのボードの中心座標
00546             features.centroid = cv::Point2f(static_cast<float>(m.m10 / m.m00), static_cast<float>(m.m01 /
m.m00));
00547
00548             const double mu20 = m.mu20 / m.m00;
00549             const double mu02 = m.mu02 / m.m00;
00550             const double mu11 = m.mu11 / m.m00;
00551
00552             // 2. 慣性半径 (Spread / Scale): コーナー点群の重心からの平均的な広がり
00553             //   カメラとボード間の距離（見かけの大きさ）を表す指標となる。
00554             //   単純な外接矩形や一部コーナーのオクルージョンに比べ外れ値に強い。
00555             features.spread = static_cast<float>(std::sqrt(std::max(0.0, mu20 + mu02)));
00556
00557             // 3. 主軸角度 (Orientation Angle): 慣性主軸の傾き角度（度数法, -90° +90°）
00558             //   画像平面内におけるボードの回転・傾き状態を表す指標となる。
00559             features.angleDeg = static_cast<float>(0.5 * std::atan2(2.0 * mu11, mu20 - mu02) * 180.0 /
CV_PI);
00560         }
00561     }
00562     return features;
00563 }
00564
00565 //
00566 // モーション状態（静止判定）を更新する
00567 //
00568 // 【目的】
00569 //   直前フレームと同一の ChArUco コーナー ID を照合してフレーム間の変位量を追跡し、

```

```

00570 //  作業者の手振れやボード移動に伴う「モーションブラー（ブレ）」のない
00571 //  安定した静止状態が一定時間継続しているかを判定する。
00572 //
00573 void Calibration::updateMotion(float deltaTime, float motionThresholdPx, float minStableTime, int
    minCorners)
00574 {
00575     //  検出コーナー数が最低必要数未満、または直前フレームのデータが存在しない場合
00576     if (charucoCorners.size() < static_cast<size_t>(minCorners) || prevCharucoCorners.empty())
00577     {
00578         //  移動中または未検出とみなし、変位量を大きく設定して静止時間をリセット
00579         currentMotion = 999.0f;
00580         stableDuration = 0.0f;
00581         isCurrentlyStable = false;
00582         prevCharucoCorners = charucoCorners;
00583         prevCharucoIds = charucoIds;
00584         return;
00585     }
00586
00587     //  直前フレームと現在フレームで共通するコーナー ID を探索し、
00588     //  ピクセル座標の移動量（ユークリッド距離）を合算する
00589     float totalDist = 0.0f;
00590     int commonCount = 0;
00591
00592     for (size_t i = 0; i < charucoIds.size(); ++i)
00593     {
00594         const int id = charucoIds[i];
00595         for (size_t j = 0; j < prevCharucoIds.size(); ++j)
00596         {
00597             if (prevCharucoIds[j] == id)
00598             {
00599                 totalDist += static_cast<float>(cv::norm(charucoCorners[i] - prevCharucoCorners[j]));
00600                 ++commonCount;
00601                 break;
00602             }
00603         }
00604     }
00605
00606     //  共通コーナーが一定数以上見つかった場合は平均変位量（px）を算出
00607     if (commonCount >= std::min(4, minCorners))
00608     {
00609         currentMotion = totalDist / commonCount;
00610     }
00611     else
00612     {
00613         //  共通点が少なすぎる場合はトラッキング不可（大きな移動があった）とみなす
00614         currentMotion = 999.0f;
00615     }
00616
00617     //  平均変位量が許容閾値（motionThresholdPx、例：2.0px）以下なら静止中と判定
00618     if (currentMotion <= motionThresholdPx)
00619     {
00620         //  静止継続時間（秒）を積算
00621         stableDuration += deltaTime;
00622     }
00623     else
00624     {
00625         //  動いている場合は静止タイマーを即座にリセット
00626         stableDuration = 0.0f;
00627     }
00628
00629     //  静止継続時間が必要時間（minStableTime、例：0.6秒）に達していれば安定と判定
00630     isCurrentlyStable = (stableDuration >= minStableTime);
00631
00632     //  次フレームの変位計算用に現在のコーナー情報を保持
00633     prevCharucoCorners = charucoCorners;
00634     prevCharucoIds = charucoIds;
00635 }
00636
00637 //
00638 //  直前に記録された標本に対して十分な姿勢・位置の多様性があるかを調べる
00639 //
00640 //  【目的】
00641 //  静止が検知された場合でも、直前に記録したサンプルとはほぼ同じ場所・距離・角度の
00642 //  重複標本が連続して保存されるのを防止する。
00643 //  画面内の平行移動、カメラとの距離変化、ボードの傾き変化のいずれかが
00644 //  閾値以上ある場合のみ「多様性あり」と判定して記録を許可する。
00645 //
00646 bool Calibration::isDiverseEnough(float minDistanceRatio, float minScaleRatio, float minAngleDeg)
    const
00647 {
00648     //  初回記録時、または過去の標本がない場合は比較対象がないため常に合格
00649     if (!hasRecordedShot || allCorners.empty()) return true;
00650
00651     //  コーナーが少なすぎる場合は判定不可
00652     if (charucoCorners.size() < 4) return false;
00653
00654     //  現在フレームの幾何特徴を算出

```

```

00655     const auto cur = computeFeatures(charucoCorners);
00656
00657     // 1. 重心移動量の比率 (画像対角線長に対する割合)
00658     //   ボードが画面の中央、四隅、端など異なる位置に移動したかを判定
00659     const float diag = std::hypot(static_cast<float>(size.width), static_cast<float>(size.height));
00660     const float dist = static_cast<float>(cv::norm(cur.centroid - lastRecordedFeatures.centroid));
00661     if (diag > 0.0f && (dist / diag) >= minDistanceRatio) return true;
00662
00663     // 2. サイズ (慣性半径・スケール) の変化率
00664     //   カメラに近づいたか、遠ざかったか (奥行き方向のバリエーション) を判定
00665     if (lastRecordedFeatures.spread > 0.0f)
00666     {
00667         const float scaleDiff = std::abs(cur.spread - lastRecordedFeatures.spread) /
lastRecordedFeatures.spread;
00668         if (scaleDiff >= minScaleRatio) return true;
00669     }
00670
00671     // 3. 主軸角度の変化 (度)
00672     //   ボードが回転または斜めに傾けられたかを判定 (周期性を考慮して 0°90° の最小差を算出)
00673     float angleDiff = std::abs(cur.angleDeg - lastRecordedFeatures.angleDeg);
00674     while (angleDiff > 90.0f) angleDiff = std::abs(180.0f - angleDiff);
00675     if (angleDiff >= minAngleDeg) return true;
00676
00677     // いずれの幾何変化も閾値に満たない場合は「同一姿勢の重複」とみなして不合格
00678     return false;
00679 }

```

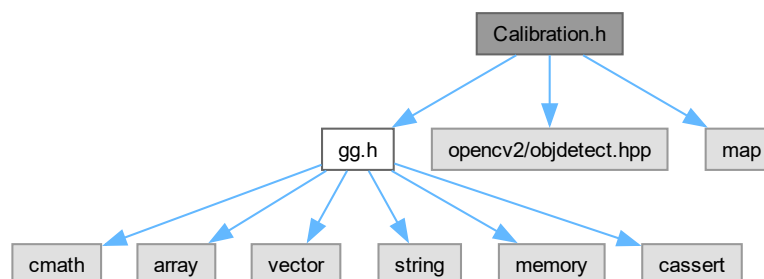
9.9 Calibration.h ファイル

```

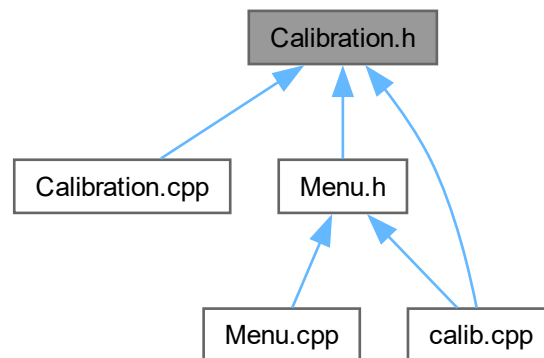
#include "gg.h"
#include <opencv2/objdetect.hpp>
#include <map>

```

Calibration.h の依存先関係図:



被依存関係図:



クラス

- class [Calibration](#)

9.9.1 詳解

校正クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Calibration.h](#) に定義があります。

9.10 Calibration.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00015 // OpenCV ArUco & ChArUco (modern OpenCV 4.7+)
00016 #include <opencv2/objdetect.hpp>
00017
00018 // 標準ライブラリ
00019 #include <map>
```

```

00020
00024 class Calibration
00025 {
00027     cv::aruco::Dictionary dictionary;
00028
00030     cv::Ptr<cv::aruco::ArucoDetector> detector;
00031
00033     cv::Ptr<cv::aruco::CharucoBoard> board;
00034
00036     cv::Ptr<cv::aruco::CharucoDetector> boardDetector;
00037
00039     std::vector<std::vector<cv::Point2f>> corners, rejected;
00040     std::vector<int> ids;
00041
00043     std::vector<cv::Point2f> charucoCorners;
00044     std::vector<int> charucoIds;
00045     std::vector<cv::Point3f> objectPoints;
00046     std::vector<cv::Point2f> imagePoints;
00047
00049     std::vector<std::vector<cv::Point2f>> allCorners;
00050     std::vector<std::vector<int>> allIds;
00051     std::vector<std::vector<cv::Point2f>> allImagePoints;
00052     std::vector<std::vector<cv::Point3f>> allObjectPoints;
00053
00055     cv::Mat cameraMatrix;
00056
00058     cv::Mat distCoeffs;
00059
00061     cv::Size size{ 0, 0 };
00062
00064     double repError{ 0.0 };
00065
00067     int totalCorners{ 0 };
00068
00070     int calibrationFlags
00071     {
00072         0
00073         ///| cv::CALIB_USE_INTRINSIC_GUESS // cameraMatrix contains valid initial values of fx, fy, cx, cy
that are optimized further. Otherwise, (cx, cy) is initially set to the image center (imageSize is
used), and focal distances are computed in a least - squares fashion. Note, that if intrinsic
parameters are known, there is no need to use this function just to estimate extrinsic parameters. Use
solvePnP instead.
00074         ///| cv::CALIB_FIX_PRINCIPAL_POINT // The principal point is not changed during the global
optimization. It stays at the center or at a different location specified when CALIB_USE_INTRINSIC_GUESS
is set too.
00075         ///| cv::CALIB_FIX_ASPECT_RATIO // The functions consider only fy as a free parameter. The ratio
fx / fy stays the same as in the input cameraMatrix. When CALIB_USE_INTRINSIC_GUESS is not set, the
actual input values of fx and fy are ignored, only their ratio is computed and used further.
00076         ///| cv::CALIB_ZERO_TANGENT_DIST // Tangential distortion coefficients (p1, p2) are set to zeros
and stay zero.
00077         ///| cv::CALIB_FIX_FOCAL_LENGTH // The focal length is not changed during the global optimization
if CALIB_USE_INTRINSIC_GUESS is set.
00078         ///| cv::CALIB_FIX_K1 // , ..., CALIB_FIX_K6 The corresponding radial distortion
coefficient is not changed during the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient
from the supplied distCoeffs matrix is used. Otherwise, it is set to 0.
00079         ///| cv::CALIB_RATIONAL_MODEL // Coefficients k4, k5, and k6 are enabled. To provide the
backward compatibility, this extra flag should be explicitly specified to make the calibration
function use the rational model and return 8 coefficients or more.
00080         ///| cv::CALIB_THIN_PRISM_MODEL // Coefficients s1, s2, s3 and s4 are enabled. To provide the
backward compatibility, this extra flag should be explicitly specified to make the calibration
function use the thin prism model and return 12 coefficients or more.
00081         ///| cv::CALIB_FIX_S1_S2_S3_S4 // The thin prism distortion coefficients are not changed during
the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient from the supplied distCoeffs
matrix is used. Otherwise, it is set to 0.
00082         ///| cv::CALIB_TILTED_MODEL // Coefficients tauX and tauY are enabled. To provide the backward
compatibility, this extra flag should be explicitly specified to make the calibration function use the
tilted sensor model and return 14 coefficients.
00083         ///| cv::CALIB_FIX_TAUX_TAUY // The coefficients of the tilted sensor model are not changed
during the optimization. If CALIB_USE_INTRINSIC_GUESS is set, the coefficient from the supplied
distCoeffs matrix is used. Otherwise, it is set to 0.
00084     };
00085
00089     struct BoardPoseFeatures
00090     {
00091         cv::Point2f centroid{ 0.0f, 0.0f };
00092         float spread{ 0.0f };
00093         float angleDeg{ 0.0f };
00094     };
00095
00097     std::vector<cv::Point2f> prevCharucoCorners;
00098     std::vector<int> prevCharucoIds;
00099
00101     float currentMotion{ 999.0f };
00102
00104     float stableDuration{ 0.0f };
00105
00107     bool isCurrentlyStable{ false };

```



```

00108
00110 BoardPoseFeatures lastRecordedFeatures;
00111
00113 bool hasRecordedShot{ false };
00114
00116 static BoardPoseFeatures computeFeatures(const std::vector<cv::Point2f>& corners);
00117
00118 public:
00119
00127 Calibration(const std::string& dictionaryName,
00128             const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength);
00129
00135 Calibration(const Calibration& calibration) = delete;
00136
00140 virtual ~Calibration();
00141
00147 Calibration& operator=(const Calibration& calibration) = delete;
00148
00155 void createBoard(const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength);
00156
00164 void setDictionary(const std::string& dictionaryName,
00165                  const std::array<int, 2>& checkerSize, const std::array<float, 2>& checkerLength);
00166
00174 void drawBoard(cv::Mat& boardImage, int width, int height);
00175
00181 void detectBoard(cv::Mat& image);
00182
00189 void detectMarkers(cv::Mat& image, float markerLength);
00190
00194 void recordCorners();
00195
00199 void discardCorners();
00200
00209 void updateMotion(float deltaTime, float motionThresholdPx = 2.0f, float minStableTime = 0.6f, int
minCorners = 6);
00210
00216 bool isStable() const { return isCurrentlyStable; }
00217
00224 float getStableProgress(float minStableTime = 0.6f) const
00225 {
00226     return minStableTime > 0.0f ? std::min(1.0f, stableDuration / minStableTime) : 0.0f;
00227 }
00228
00234 float getCurrentMotion() const { return currentMotion; }
00235
00244 bool isDiverseEnough(float minDistanceRatio = 0.05f, float minScaleRatio = 0.12f, float minAngleDeg
= 8.0f) const;
00245
00251 bool calibrate();
00252
00258 auto getCornersCount() const
00259 {
00260     return static_cast<int>(corners.size());
00261 }
00262
00268 auto getSampleCount() const
00269 {
00270     return static_cast<int>(allCorners.size());
00271 }
00272
00278 auto getTotalCount() const
00279 {
00280     return totalCorners;
00281 }
00282
00288 const auto& getCameraMatrix() const
00289 {
00290     return cameraMatrix;
00291 }
00292
00298 const auto& getDistortionCoefficients() const
00299 {
00300     return distCoeffs;
00301 }
00302
00308 auto getReprojectionError() const
00309 {
00310     return repError;
00311 }
00312
00318 auto finished()
00319 {
00320     return cameraMatrix.total() == 9 && distCoeffs.total() == 5;
00321 }
00322
00330 GgMatrix RvecTvecToPose(const cv::Vec3d& rvec, const cv::Vec3d& tvec);
00331

```

```

00341 void getAllMarkerPoses(float markerLength, std::map<int, GgMatrix>& poses);
00342
00349 bool loadParameters(const std::string& filename);
00350
00357 bool saveParameters(const std::string& filename) const;
00358
00360 static const std::map<const std::string, const cv::aruco::PredefinedDictionaryType> dictionaryList;
00361 };

```

9.11 CamAndroid.cpp ファイル

9.11.1 詳解

Android Camera2 NDK を使ったビデオキャプチャクラスの実装

著者

Kohe Tokoi

日付

September 2026

[CamAndroid.cpp](#) に定義があります。

9.12 CamAndroid.cpp

[詳解]

```

00001
00008
00009 #if defined(_ANDROID_)
00010
00011 #include "CamAndroid.h"
00012 #include <android/log.h>
00013 #include <algorithm>
00014 #include <sstream>
00015
00016 #define LOG_TAG "CamAndroid"
00017 #define LOGI(...) _android_log_print(ANDROID_LOG_INFO, LOG_TAG, __VA_ARGS__)
00018 #define LOGE(...) _android_log_print(ANDROID_LOG_ERROR, LOG_TAG, __VA_ARGS__)
00019
00020 std::vector<std::string> CamAndroid::deviceList;
00021
00022 //
00023 // コンストラクタ
00024 //
00025 CamAndroid::CamAndroid(int deviceNumber, int initial_width, int initial_height, double initial_fps)
00026 {
00027     open(deviceNumber, initial_width, initial_height, initial_fps);
00028 }
00029
00030 //
00031 // デストラクタ
00032 //
00033 CamAndroid::~CamAndroid()
00034 {
00035     close();
00036 }
00037
00038 //
00039 // 使用可能なカメラデバイスのリストを返す
00040 //
00041 const std::vector<std::string>& CamAndroid::getDeviceList()
00042 {
00043     deviceList.clear();

```

```

00044
00045   ACameraManager* mgr{ ACameraManager.create() };
00046   if (!mgr) return deviceList;
00047
00048   ACameraIdList* idList{ nullptr };
00049   camera_status_t status{ ACameraManager.getCameraIdList(mgr, &idList) };
00050   if (status == ACAMERA_OK && idList)
00051   {
00052       for (int i = 0; i < idList->numCameras; ++i)
00053       {
00054           const char* id{ idList->cameraIds[i] };
00055           ACameraMetadata* chars{ nullptr };
00056           status = ACameraManager.getCameraCharacteristics(mgr, id, &chars);
00057
00058           std::string facingStr{ "External" };
00059           if (status == ACAMERA_OK && chars)
00060           {
00061               ACameraMetadata::const_entry entry;
00062               if (ACameraMetadata.getConstEntry(chars, ACAMERA_LENS_FACING, &entry) == ACAMERA_OK)
00063               {
00064                   auto facing{ entry.data.u8[0] };
00065                   if (facing == ACAMERA_LENS_FACING_BACK)
00066                   {
00067                       facingStr = "Back";
00068                   }
00069                   else if (facing == ACAMERA_LENS_FACING_FRONT)
00070                   {
00071                       facingStr = "Front";
00072                   }
00073               }
00074               ACameraMetadata.free(chars);
00075           }
00076
00077           std::ostringstream oss;
00078           oss << i << ": Camera " << id << " (" << facingStr << ")";
00079           deviceList.push_back(oss.str());
00080       }
00081       ACameraManager.deleteCameraIdList(idList);
00082   }
00083
00084   ACameraManager.delete(mgr);
00085   return deviceList;
00086 }
00087
00088 //
00089 // キャプチャデバイスを開く
00090 //
00091 bool CamAndroid::open(int deviceNumber, int initial_width, int initial_height, double initial_fps)
00092 {
00093     close();
00094
00095     cameraManager = ACameraManager.create();
00096     if (!cameraManager)
00097     {
00098         LOGE("Failed to create ACameraManager");
00099         return false;
00100     }
00101
00102     ACameraIdList* idList{ nullptr };
00103     camera_status_t status{ ACameraManager.getCameraIdList(cameraManager, &idList) };
00104     if (status != ACAMERA_OK || !idList || deviceNumber < 0 || deviceNumber >= idList->numCameras)
00105     {
00106         if (idList) ACameraManager.deleteCameraIdList(idList);
00107         LOGE("Invalid camera device number: %d", deviceNumber);
00108         return false;
00109     }
00110
00111     selectedCameraId = idList->cameraIds[deviceNumber];
00112     deviceIndex = deviceNumber;
00113     ACameraManager.deleteCameraIdList(idList);
00114
00115     // カメラ特性から解像度一覧を取得
00116     ACameraMetadata* chars{ nullptr };
00117     status = ACameraManager.getCameraCharacteristics(cameraManager, selectedCameraId.c_str(), &chars);
00118     if (status == ACAMERA_OK && chars)
00119     {
00120         enumerateFormats(chars);
00121         ACameraMetadata.free(chars);
00122     }
00123
00124     // デフォルト解像度の決定
00125     if (initial_width > 0 && initial_height > 0)
00126     {
00127         width = initial_width;
00128         height = initial_height;
00129     }
00130     else if (!formatList.empty())

```

```

00131 {
00132     selectFormat(0);
00133 }
00134 else
00135 {
00136     width = 1280;
00137     height = 720;
00138 }
00139
00140 channels = 4; // BGRA
00141 interval = (initial.fps > 0.0) ? (1000.0 / initial.fps) : 33.3;
00142
00143 LOGI("Opened camera %s (%d x %d)", selectedCameraId.c_str(), width, height);
00144 return true;
00145 }
00146
00147 //
00148 // 利用可能なカメラフォーマットを列挙する
00149 //
00150 void CamAndroid::enumerateFormats(ACameraMetadata* metadata)
00151 {
00152     formatList.clear();
00153
00154     ACameraMetadata_const_entry entry;
00155     camera.status_t status{ ACameraMetadata_getConstEntry(metadata,
00156 ACAMERA_SCALER_AVAILABLE_STREAM_CONFIGURATIONS, &entry) };
00157     if (status != ACAMERA_OK) return;
00158
00159     // 各エントリは (format, width, height, isInput) の 4 要素
00160     int formatIndex{ 0 };
00161     for (uint32_t i = 0; i < entry.count; i += 4)
00162     {
00163         int32_t fmt{ entry.data.i32[i + 0] };
00164         int32_t w{ entry.data.i32[i + 1] };
00165         int32_t h{ entry.data.i32[i + 2] };
00166         int32_t isInput{ entry.data.i32[i + 3] };
00167
00168         // YUV_420_888 の出力ストリームのみ対象
00169         if (fmt == AIMAGE_FORMAT_YUV_420_888 && isInput == 0)
00170         {
00171             std::ostringstream resOss;
00172             resOss << w << " x " << h;
00173             std::string resStr{ resOss.str() };
00174
00175             // 重複チェック
00176             bool exists{ false };
00177             for (const auto& item : formatList)
00178             {
00179                 if (item.resolution == resStr)
00180                 {
00181                     exists = true;
00182                     break;
00183                 }
00184             }
00185             if (!exists)
00186             {
00187                 formatList.emplace_back(resStr, "30.00", "YUV420", formatIndex++);
00188             }
00189         }
00190     }
00191
00192     // 解像度の降順にソート
00193     std::sort(formatList.begin(), formatList.end(), [](const CaptureFormat& a, const CaptureFormat& b) {
00194         int wA = 0, hA = 0, wB = 0, hB = 0;
00195         sscanf(a.resolution.c_str(), "%d x %d", &wA, &hA);
00196         sscanf(b.resolution.c_str(), "%d x %d", &wB, &hB);
00197         return (wA * hA) > (wB * hB);
00198     });
00199
00200     // index の再割り当て
00201     for (size_t i = 0; i < formatList.size(); ++i)
00202     {
00203         formatList[i].index = static_cast<int>(i);
00204     }
00205 }
00206
00207 //
00208 // フォーマットを選択する
00209 //
00210 bool CamAndroid::selectFormat(int index)
00211 {
00212     if (index < 0 || index >= static_cast<int>(formatList.size())) return false;
00213
00214     int w{ 0 }, h{ 0 };
00215     if (sscanf(formatList[index].resolution.c_str(), "%d x %d", &w, &h) == 2)
00216     {

```

```

00217     bool restart{ running };
00218     if (restart) stop();
00219
00220     width = w;
00221     height = h;
00222
00223     if (restart) start();
00224     return true;
00225 }
00226
00227 return false;
00228 }
00229
00230 //
00231 // キャプチャ開始処理
00232 //
00233 bool CamAndroid::onStart()
00234 {
00235     if (!cameraManager || selectedCameraId.empty()) return false;
00236
00237     // AImageReader の作成
00238     media_status_t mStatus{ AImageReader_new(width, height, AIMAGE_FORMAT_YUV420_888, 4, &imageReader) };
00239     if (mStatus != AMEDIA_OK || !imageReader)
00240     {
00241         LOGE("Failed to create AImageReader (%d x %d)", width, height);
00242         return false;
00243     }
00244
00245     AImageReader_ImageListener listener;
00246     listener.context = this;
00247     listener.onImageAvailable = onImageAvailableCallback;
00248     AImageReader_setImageListener(imageReader, &listener);
00249
00250     mStatus = AImageReader_getWindow(imageReader, &imageWindow);
00251     if (mStatus != AMEDIA_OK || !imageWindow)
00252     {
00253         LOGE("Failed to get ANativeWindow from AImageReader");
00254         AImageReader_delete(imageReader);
00255         imageReader = nullptr;
00256         return false;
00257     }
00258
00259     // カメラデバイスのオープン
00260     static ACameraDevice_stateCallbacks devCallbacks;
00261     devCallbacks.context = this;
00262     devCallbacks.onDisconnected = [](void*, ACameraDevice*) {};
00263     devCallbacks.onError = [](void*, ACameraDevice*, int error) {
00264         LOGE("CameraDevice error: %d", error);
00265     };
00266
00267     camera_status_t cStatus{ ACameraManager_openCamera(cameraManager, selectedCameraId.c_str(),
00268 &devCallbacks, &cameraDevice) };
00269     if (cStatus != ACAMERA_OK || !cameraDevice)
00270     {
00271         LOGE("Failed to open camera %s", selectedCameraId.c_str());
00272         AImageReader_delete(imageReader);
00273         imageReader = nullptr;
00274         imageWindow = nullptr;
00275         return false;
00276     }
00277
00278     // キャプチャリクエスト作成
00279     cStatus = ACameraDevice_createCaptureRequest(cameraDevice, TEMPLATE_PREVIEW, &captureRequest);
00280     if (cStatus != ACAMERA_OK || !captureRequest)
00281     {
00282         LOGE("Failed to create capture request");
00283         onStop();
00284         return false;
00285     }
00286
00287     // ターゲットの関連付け
00288     ACameraOutputTarget_create(imageWindow, &outputTarget);
00289     ACaptureRequest_addTarget(captureRequest, outputTarget);
00290
00291     // セッション出力コンテナ
00292     ACaptureSessionOutputContainer_create(&outputContainer);
00293     ACaptureSessionOutput_create(imageWindow, &sessionOutput);
00294     ACaptureSessionOutputContainer_add(outputContainer, sessionOutput);
00295
00296     // キャプチャセッション作成
00297     static ACameraCaptureSession_stateCallbacks sessionCallbacks;
00298     sessionCallbacks.context = this;
00299     sessionCallbacks.onActive = [](void*, ACameraCaptureSession*) {};
00300     sessionCallbacks.onClosed = [](void*, ACameraCaptureSession*) {};
00301     sessionCallbacks.onReady = [](void*, ACameraCaptureSession*) {};
00302
00303     cStatus = ACameraDevice_createCaptureSession(cameraDevice, outputContainer, &sessionCallbacks,

```

```

        &captureSession);
00303     if (cStatus != ACAMERA_OK || !captureSession)
00304     {
00305         LOGE("Failed to create capture session");
00306         onStop();
00307         return false;
00308     }
00309
00310     // プレビューの連続キャプチャ要求開始
00311     cStatus = ACameraCaptureSession_setRepeatingRequest(captureSession, nullptr, 1, &captureRequest,
    nullptr);
00312     if (cStatus != ACAMERA_OK)
00313     {
00314         LOGE("Failed to set repeating request");
00315         onStop();
00316         return false;
00317     }
00318
00319     LOGI("Camera capture started successfully: %s (%d x %d)", selectedCameraId.c_str(), width, height);
00320     return true;
00321 }
00322
00323 //
00324 // キャプチャ停止処理
00325 //
00326 void CamAndroid::onStop()
00327 {
00328     if (captureSession)
00329     {
00330         ACameraCaptureSession_stopRepeating(captureSession);
00331         ACameraCaptureSession_close(captureSession);
00332         captureSession = nullptr;
00333     }
00334
00335     if (captureRequest)
00336     {
00337         if (outputTarget)
00338         {
00339             ACaptureRequest_removeTarget(captureRequest, outputTarget);
00340         }
00341         ACaptureRequest_free(captureRequest);
00342         captureRequest = nullptr;
00343     }
00344
00345     if (outputTarget)
00346     {
00347         ACameraOutputTarget_free(outputTarget);
00348         outputTarget = nullptr;
00349     }
00350
00351     if (sessionOutput)
00352     {
00353         ACaptureSessionOutputContainer_remove(outputContainer, sessionOutput);
00354         ACaptureSessionOutput_free(sessionOutput);
00355         sessionOutput = nullptr;
00356     }
00357
00358     if (outputContainer)
00359     {
00360         ACaptureSessionOutputContainer_free(outputContainer);
00361         outputContainer = nullptr;
00362     }
00363
00364     if (cameraDevice)
00365     {
00366         ACameraDevice_close(cameraDevice);
00367         cameraDevice = nullptr;
00368     }
00369
00370     if (imageReader)
00371     {
00372         AImageReader_setImageListener(imageReader, nullptr);
00373         AImageReader_delete(imageReader);
00374         imageReader = nullptr;
00375         imageWindow = nullptr;
00376     }
00377
00378     LOGI("Camera capture stopped");
00379 }
00380
00381 //
00382 // キャプチャデバイスを閉じる処理
00383 //
00384 void CamAndroid::onClose()
00385 {
00386     onStop();
00387 }

```

```

00388     if (cameraManager)
00389     {
00390         ACameraManager_delete(cameraManager);
00391         cameraManager = nullptr;
00392     }
00393
00394     selectedCameraId.clear();
00395 }
00396
00397 //
00398 // 画像取得時のコールバック
00399 //
00400 void CamAndroid::onImageAvailableCallback(void* context, AImageReader* reader)
00401 {
00402     auto* self{ static_cast<CamAndroid*>(context) };
00403     if (!self) return;
00404
00405     AImage* image{ nullptr };
00406     media_status_t status;
00407
00408     if (self->prioritizeLatency)
00409     {
00410         // レイテンシ優先時は最新フレームを取得し、古いフレームを破棄
00411         status = AImageReader_acquireLatestImage(reader, &image);
00412     }
00413     else
00414     {
00415         status = AImageReader_acquireNextImage(reader, &image);
00416     }
00417
00418     if (status == AMEDIA_OK && image)
00419     {
00420         self->convertYuvToBgra(image);
00421         AImage_delete(image);
00422     }
00423 }
00424
00425 //
00426 // YUV420-888 から BGRA8888 へ的高速カラー変換
00427 //
00428 void CamAndroid::convertYuvToBgra (AImage* img)
00429 {
00430     int32_t w{ 0 }, h{ 0 };
00431     AImage_getWidth(img, &w);
00432     AImage_getHeight(img, &h);
00433     if (w <= 0 || h <= 0) return;
00434
00435     uint8_t* yPlane{ nullptr };
00436     uint8_t* uPlane{ nullptr };
00437     uint8_t* vPlane{ nullptr };
00438     int yLen{ 0 }, uLen{ 0 }, vLen{ 0 };
00439     int yRowStride{ 0 }, uRowStride{ 0 }, vRowStride{ 0 };
00440     int uPixelStride{ 0 }, vPixelStride{ 0 };
00441
00442     AImage_getPlaneData(img, 0, &yPlane, &yLen);
00443     AImage_getPlaneRowStride(img, 0, &yRowStride);
00444
00445     AImage_getPlaneData(img, 1, &uPlane, &uLen);
00446     AImage_getPlaneRowStride(img, 1, &uRowStride);
00447     AImage_getPlanePixelStride(img, 1, &uPixelStride);
00448
00449     AImage_getPlaneData(img, 2, &vPlane, &vLen);
00450     AImage_getPlaneRowStride(img, 2, &vRowStride);
00451     AImage_getPlanePixelStride(img, 2, &vPixelStride);
00452
00453     if (!yPlane || !uPlane || !vPlane) return;
00454
00455     const size_t outSize{ static_cast<size_t>(w * h * 4) };
00456
00457     // フレームバッファの排他更新
00458     std::unique_lock<std::mutex> lock(mtx, std::try_to_lock);
00459     if (!lock.owns_lock())
00460     {
00461         // メインスレッドが読み出し中の場合はスキップ
00462         return;
00463     }
00464
00465     if (image.size() != outSize)
00466     {
00467         image.resize(outSize);
00468         width = w;
00469         height = h;
00470         channels = 4;
00471     }
00472
00473     uint8_t* dst{ image.data() };
00474

```

```

00475 // YUV420 -> BGRA 変換ループ
00476 for (int y = 0; y < h; ++y)
00477 {
00478     const uint8_t* yRow{ yPlane + y * yRowStride };
00479     const uint8_t* uRow{ uPlane + (y / 2) * uRowStride };
00480     const uint8_t* vRow{ vPlane + (y / 2) * vRowStride };
00481     uint8_t* dstRow{ dst + y * w * 4 };
00482
00483     for (int x = 0; x < w; ++x)
00484     {
00485         const int Y{ yRow[x] };
00486         const int U{ uRow[(x / 2) * uPixelStride] };
00487         const int V{ vRow[(x / 2) * vPixelStride] };
00488
00489         const int C{ Y - 16 };
00490         const int D{ U - 128 };
00491         const int E{ V - 128 };
00492
00493         int R{ (298 * C + 409 * E + 128) >> 8 };
00494         int G{ (298 * C - 100 * D - 208 * E + 128) >> 8 };
00495         int B{ (298 * C + 516 * D + 128) >> 8 };
00496
00497         dstRow[x * 4 + 0] = static_cast<uint8_t>(std::clamp(B, 0, 255));
00498         dstRow[x * 4 + 1] = static_cast<uint8_t>(std::clamp(G, 0, 255));
00499         dstRow[x * 4 + 2] = static_cast<uint8_t>(std::clamp(R, 0, 255));
00500         dstRow[x * 4 + 3] = 255;
00501     }
00502 }
00503
00504 captured = true;
00505 }
00506
00507 #endif // defined(__ANDROID__)

```

9.13 CamAndroid.h ファイル

9.13.1 詳解

Android Camera2 NDK を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

September 2026

[CamAndroid.h](#) に定義があります。

9.14 CamAndroid.h

[詳解]

```

00001 #pragma once
00002
00010
00011 #if defined(__ANDROID__)
00012
00013 // カメラ関連の基底クラス
00014 #include "Camera.h"
00015
00016 // Android NDK Camera2 & Media API
00017 #include <camera/NdkCameraManager.h>
00018 #include <camera/NdkCameraDevice.h>
00019 #include <camera/NdkCameraCaptureSession.h>

```



```

00020 #include <camera/NdkCameraMetadata.h>
00021 #include <media/NdkImageReader.h>
00022 #include <media/NdkImage.h>
00023
00024 // 標準ライブラリ
00025 #include <memory>
00026 #include <vector>
00027 #include <string>
00028 #include <mutex>
00029 #include <condition_variable>
00030
00034 class CamAndroid : public Camera
00035 {
00037     ACameraManager* cameraManager{ nullptr };
00038
00040     ACameraDevice* cameraDevice{ nullptr };
00041
00043     ACameraCaptureSession* captureSession{ nullptr };
00044
00046     ACaptureRequest* captureRequest{ nullptr };
00047
00049     ACaptureSessionOutputContainer* outputContainer{ nullptr };
00050
00052     ACaptureSessionOutput* sessionOutput{ nullptr };
00053
00055     ACameraOutputTarget* outputTarget{ nullptr };
00056
00058     AImageReader* imageReader{ nullptr };
00059
00061     ANativeWindow* imageWindow{ nullptr };
00062
00064     int deviceIndex{ 0 };
00065
00067     std::string selectedCameraId;
00068
00070     std::vector<CaptureFormat> formatList;
00071
00073     static std::vector<std::string> deviceList;
00074
00080     void enumerateFormats(ACameraMetadata* metadata);
00081
00087     static void onImageAvailableCallback(void* context, AImageReader* reader);
00088
00094     void convertYuvToBgra(AImage* img);
00095
00096 protected:
00097
00103     virtual bool onStart() override;
00104
00108     virtual void onStop() override;
00109
00113     virtual void onClose() override;
00114
00115 public:
00116
00125     CamAndroid(int deviceNumber = 0, int initialWidth = 0, int initialHeight = 0, double initialFps =
0.0);
00126
00130     virtual ~CamAndroid();
00131
00141     bool open(int deviceNumber, int initialWidth = 0, int initialHeight = 0, double initialFps = 0.0);
00142
00148     bool isOpened() const
00149     {
00150         return cameraManager != nullptr && !selectedCameraId.empty();
00151     }
00152
00158     static const std::vector<std::string>& getDeviceList();
00159
00165     const std::vector<CaptureFormat>& getFormatList() const override
00166     {
00167         return formatList;
00168     }
00169
00176     bool selectFormat(int index) override;
00177 };
00178
00179 #endif // defined(__ANDROID__)

```

9.15 CamAvf.h ファイル

9.15.1 詳解

AV Foundation を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

September 2026

[CamAvf.h](#) に定義があります。

9.16 CamAvf.h

[詳解]

```
00001 #pragma once
00002
00010
00011 #if defined(__APPLE__)
00012
00013 // カメラ関連の処理
00014 #include "Camera.h"
00015
00016 // 標準ライブラリ
00017 #include <vector>
00018 #include <string>
00019 #include <memory>
00020
00024 class CamAvf : public Camera
00025 {
00029     struct VideoFormat
00030     {
00031         int width{ 0 };
00032         int height{ 0 };
00033         double fps{ 0.0 };
00034         std::string codec;
00035         int formatIndex{ 0 };
00036         int rangeIndex{ 0 };
00037
00048         VideoFormat(int width, int height, double fps, const std::string& codec,
00049                     int formatIndex, int rangeIndex)
00050             : width{ width }
00051             , height{ height }
00052             , fps{ fps }
00053             , codec{ codec }
00054             , formatIndex{ formatIndex }
00055             , rangeIndex{ rangeIndex }
00056     {
00057     };
00058 };
00059
00061 struct Impl;
00062 std::unique_ptr<Impl> impl;
00063
00065 std::vector<VideoFormat> availableFormats;
00066
00068 std::vector<CaptureFormat> formatList;
00069
00071 int selectedFormatIndex{ 0 };
00072
00074 static std::vector<std::string> deviceList;
00075
00081 bool enumerateFormats();
00082
```

```
00083 protected:
00084
00090     virtual bool onStart() override;
00091
00095     virtual void onStop() override;
00096
00100     virtual void onClose() override;
00101
00102 public:
00103
00107     CamAvf();
00108
00112     virtual ~CamAvf();
00113
00119     static const std::vector<std::string>& getDeviceList();
00120
00128     bool open(int deviceNumber, bool setupFormat = true);
00129
00135     virtual const std::vector<CaptureFormat>& getFormatList() const override
00136     {
00137         return formatList;
00138     }
00139
00146     bool select(int index);
00147
00154     virtual bool selectFormat(int index) override
00155     {
00156         return select(index);
00157     }
00158
00164     virtual void setPrioritizeLatency(bool mode) override;
00165
00171     void handleSampleBuffer(const void* sampleBuffer);
00172 };
00173
00174 #endif // defined(__APPLE__)
```

9.17 CamAvf.mm ファイル

9.17.1 詳解

AV Foundation を使ったビデオキャプチャクラスの実装

著者

Kohe Tokoi

日付

September 2026

macOS の AV Foundation ネイティブ API を直接使用し、カメラデバイスの列挙、フォーマット（解像度・フレームレート・コーデック）の取得、および高速なビデオフレームのキャプチャを行います。OpenCV の VideoCapture におけるデバイス名取得不可やフォーマット変更不可といった制約を回避し、Windows (CamMf) と同等の低遅延・高機能なキャプチャ機能を提供します。

[CamAvf.mm](#) に定義があります。

9.18 CamAvf.mm

[詳解]

```

00001
00014
00015 #if defined(__APPLE__)
00016
00017 #include "CamAvf.h"
00018
00019 // AV Foundation
00020 #import <AVFoundation/AVFoundation.h>
00021 #import <CoreVideo/CoreVideo.h>
00022 #import <CoreMedia/CoreMedia.h>
00023
00024 // 標準ライブラリ
00025 #include <sstream>
00026 #include <iomanip>
00027 #include <cmath>
00028 #include <cctype>
00029 #include <algorithm>
00030 #include <chrono>
00031 #include <thread>
00032
00033 // キャプチャデバイス名のリスト
00034 std::vector<std::string> CamAvf::deviceList;
00035
00045 @interface CamAvfDelegate : NSObject <AVCaptureVideoDataOutputSampleBufferDelegate>
00046 {
00047     CamAvf* parent;
00048 }
00049
00055 - (id)initWithParent:(CamAvf*)p;
00056
00057 @end
00058
00059 @implementation CamAvfDelegate
00060
00061 //
00062 // 初期化
00063 //
00064 - (id)initWithParent:(CamAvf*)p
00065 {
00066     self = [super init];
00067     if (self)
00068     {
00069         // 親となる C++ オブジェクトのポインタを保持する
00070         parent = p;
00071     }
00072     return self;
00073 }
00074
00075 //
00076 // 新しいビデオフレームが届いたときに呼ばれるコールバック
00077 //
00078 - (void)captureOutput:(AVCaptureOutput*)output
00079 didOutputSampleBuffer:(CMSampleBufferRef)sampleBuffer
00080 fromConnection:(AVCaptureConnection*)connection
00081 {
00082     // 親オブジェクトが存在していればフレーム処理関数を呼び出す
00083     if (parent)
00084     {
00085         parent->handleSampleBuffer(sampleBuffer);
00086     }
00087 }
00088
00089 @end
00090
00097 struct CamAvf::Impl
00098 {
00099     AVCaptureSession* session{ nil };
00100     AVCaptureDevice* device{ nil };
00101     AVCaptureDeviceInput* input{ nil };
00102     AVCaptureVideoDataOutput* output{ nil };
00103     CamAvfDelegate* delegate{ nil };
00104     dispatch_queue_t queue{ nil };
00105     int deviceIndex{ -1 };
00106
00112 ~Impl()
00113 {
00114     if (session)
00115     {
00116         // セッションが動作中であれば停止する
00117         if (session.isRunning)
00118         {
00119             [session stopRunning];

```

```

00120     }
00121
00122     // セッションの構成変更を開始する
00123     [session beginConfiguration];
00124
00125     // セッションから入力を削除する
00126     if (input)
00127     {
00128         [session removeInput:input];
00129         input = nil;
00130     }
00131
00132     // デリゲートを解除してからセッションから出力を削除する
00133     if (output)
00134     {
00135         [output setSampleBufferDelegate:nil queue:NULL];
00136         [session removeOutput:output];
00137         output = nil;
00138     }
00139
00140     // セッションの構成変更を確定する
00141     [session commitConfiguration];
00142     session = nil;
00143 }
00144
00145 // デリゲートとキューの参照をクリアする
00146 delegate = nil;
00147 queue = nil;
00148 device = nil;
00149 }
00150 };
00151
00152 namespace
00153 {
00161     NSArray<AVCaptureDevice*>* getVideoDevices()
00162     {
00163         // 検索対象とするデバイスタイプのリストを作成する
00164         NSMutableArray<AVCaptureDeviceType*> types = [NSMutableArray array];
00165
00166         // 内蔵カメラ (FaceTime HD カメラ等) を追加する
00167         [types addObject:AVCaptureDeviceTypeBuiltInWideAngleCamera];
00168
00169         #if defined(__MAC_14_0)
00170             if (@available(macOS 14.0, *))
00171             {
00172                 // macOS 14 (Sonoma) 以降では外部 UVC カメラと連係カメラが個別に分類される
00173                 [types addObject:AVCaptureDeviceTypeExternal];
00174                 [types addObject:AVCaptureDeviceTypeContinuityCamera];
00175             }
00176             else
00177         #endif
00178         {
00179             // macOS 13 以前では外部カメラ全般を ExternalUnknown で検索する
00180             [types addObject:AVCaptureDeviceTypeExternalUnknown];
00181         }
00182
00183         // 指定されたタイプのビデオ入力デバイスを検索・列挙するセッションを作成する
00184         AVCaptureDeviceDiscoverySession* discoverySession = [AVCaptureDeviceDiscoverySession
00185             discoverySessionWithDeviceTypes:types
00186             mediaType:AVMediaTypeVideo
00187             position:AVCaptureDevicePositionUnspecified];
00188
00189         // 発見されたデバイスの配列を返す
00190         return discoverySession.devices;
00191     }
00192
00193     AVCaptureDevice* getDeviceByIndex(int index)
00194     {
00195         // 不正なインデックスの場合は nil を返す
00196         if (index < 0) return nil;
00197
00198         // デバイス一覧を取得する
00199         NSArray<AVCaptureDevice*>* devices = getVideoDevices();
00200
00201         // 範囲内であれば該当するデバイスを返す
00202         if (static_cast<NSUInteger>(index) < devices.count)
00203         {
00204             return devices[index];
00205         }
00206
00207         return nil;
00208     }
00209 }
00210
00211 //
00212 // コンストラクタ
00213 //

```

```

00220 CamAvf::CamAvf()
00221 {
00222 }
00223
00224 //
00225 // デストラクタ
00226 //
00227 CamAvf::~CamAvf()
00228 {
00229     close();
00230 }
00231
00242 static std::string sanitizeDeviceName(const std::string& name)
00243 {
00244     std::string result;
00245     result.reserve(name.size());
00246
00247     const auto len{ name.size() };
00248     for (size_t i = 0; i < len; )
00249     {
00250         const auto c{ static_cast<unsigned char>(name[i]) };
00251
00252         // 1 バイト (ASCII: 0x00 - 0x7F)
00253         if (c <= 0x7F)
00254         {
00255             if (c < 0x20 || c == 0x7F)
00256             {
00257                 result += ' '; // 制御文字は空白に置換
00258             }
00259             else
00260             {
00261                 result += static_cast<char>(c);
00262             }
00263             ++i;
00264         }
00265         // 2 バイト文字 (0xC2 - 0xDF)
00266         else if ((c & 0xE0) == 0xC0)
00267         {
00268             if (i + 1 < len)
00269             {
00270                 result += name.substr(i, 2);
00271                 i += 2;
00272             }
00273             else
00274             {
00275                 break; // 不正なバイト列
00276             }
00277         }
00278         // 3 バイト文字 (0xE0 - 0xEF)
00279         else if ((c & 0xF0) == 0xE0)
00280         {
00281             if (i + 2 < len)
00282             {
00283                 const auto b1{ static_cast<unsigned char>(name[i + 1]) };
00284                 const auto b2{ static_cast<unsigned char>(name[i + 2]) };
00285
00286                 // 特殊なクォーテーションの正規化
00287                 // U+2018 (E2 80 98) -> '
00288                 // U+2019 (E2 80 99) -> '
00289                 if (c == 0xE2 && b1 == 0x80 && (b2 == 0x98 || b2 == 0x99))
00290                 {
00291                     result += '\'';
00292                 }
00293                 // U+201C (E2 80 9C) -> "
00294                 // U+201D (E2 80 9D) -> "
00295                 else if (c == 0xE2 && b1 == 0x80 && (b2 == 0x9C || b2 == 0x9D))
00296                 {
00297                     result += '"';
00298                 }
00299                 else
00300                 {
00301                     result += name.substr(i, 3);
00302                 }
00303                 i += 3;
00304             }
00305             else
00306             {
00307                 break; // 不正なバイト列
00308             }
00309         }
00310         // 4 バイト文字 (0xF0 - 0xF7: U+10000 以上の絵文字等)
00311         else if ((c & 0xF8) == 0xF0)
00312         {
00313             // フォントに収録されておらず ImGui で表示できないためスキップする
00314             i += (i + 4 <= len) ? 4 : (len - i);
00315         }
00316         else

```

```

00317     {
00318         // 不正なバイトはスキップ
00319         ++i;
00320     }
00321 }
00322
00323 // 連続する空白文字を 1 つにまとめ、前後の空白を除去する
00324 std::string trimmed;
00325 trimmed.reserve(result.size());
00326 bool inSpace{ false };
00327 for (char ch : result)
00328 {
00329     if (ch == ' ')
00330     {
00331         if (!inSpace && !trimmed.empty())
00332         {
00333             trimmed += ' ';
00334             inSpace = true;
00335         }
00336     }
00337     else
00338     {
00339         trimmed += ch;
00340         inSpace = false;
00341     }
00342 }
00343 if (!trimmed.empty() && trimmed.back() == ' ')
00344 {
00345     trimmed.pop_back();
00346 }
00347
00348 // 万一すべて除去されて空になった場合は既定の名前を使用する
00349 return trimmed.empty() ? "Camera" : trimmed;
00350 }
00351
00352 //
00353 // ビデオデバイスの一覧を取得する
00354 //
00355 const std::vector<std::string>& CamAvf::getDeviceList()
00356 {
00357     // 以前のリストをクリアする
00358     deviceList.clear();
00359
00360     // Objective-C の一時オブジェクトを即座に解放するための自動解放プール
00361     @autoreleasepool {
00362         // カメラのアクセス許可状態を確認する (macOS TCC: Transparency, Consent, and Control)
00363         AVAuthorizationStatus status = [AVCaptureDevice authorizationStatusForMediaType:AVMediaTypeVideo];
00364
00365         // アクセス権限が未決定 (初回起動時など) の場合は許可をリクエストし、ユーザーの選択を待機する
00366         if (status == AVAuthorizationStatusNotDetermined)
00367         {
00368             dispatch_semaphore_t sem = dispatch_semaphore_create(0);
00369             [AVCaptureDevice requestAccessForMediaType:AVMediaTypeVideo completionHandler:^(BOOL granted) {
00370                 dispatch_semaphore_signal(sem);
00371             }];
00372             dispatch_semaphore_wait(sem, DISPATCH_TIME_FOREVER);
00373         }
00374
00375         // 利用可能なデバイスの一覧を取得する
00376         NSArray<AVCaptureDevice*> devices = getVideoDevices();
00377
00378         // 各デバイスの表示名を取得してリストに追加する
00379         for (NSUInteger i = 0; i < devices.count; ++i)
00380         {
00381             AVCaptureDevice* dev = devices[i];
00382
00383             // デバイスのローカライズ表示名を取り出す
00384             const char* rawName = [dev.localizedName UTF8String];
00385
00386             // ImGui 表示用にサニタイズする
00387             std::string name = sanitizeDeviceName(rawName ? rawName : "");
00388
00389             // 同一名称のカメラが存在する場合に識別できるよう "名前##番号" とする
00390             name += "##" + std::to_string(i);
00391
00392             // リストに追加する
00393             deviceList.push_back(name);
00394         }
00395     }
00396
00397     // デバイス名のリストを返す
00398     return deviceList;
00399 }
00400
00401 //
00402 // カメラを開く
00403 //

```

```

00404 bool CamAvf::open(int deviceNumber, bool setupFormat)
00405 {
00406     // 既に開かれているデバイスがあれば閉じる
00407     close();
00408
00409     @autoreleasepool {
00410         // 指定された番号のデバイスを取得する
00411         AVCaptureDevice* dev = getDeviceByIndex(deviceNumber);
00412         if (!dev) return false;
00413
00414         // 内部実装オブジェクトを作成し、デバイス情報を設定する
00415         impl = std::make_unique<Impl>();
00416         impl->device = dev;
00417         impl->deviceIndex = deviceNumber;
00418
00419         // デバイスがサポートするフォーマット一覧を列挙する
00420         if (!enumerateFormats())
00421         {
00422             close();
00423             return false;
00424         }
00425
00426         if (setupFormat)
00427         {
00428             // 即時初期化モード：既定（先頭）のフォーマットを適用する
00429             if (!select(0))
00430             {
00431                 close();
00432                 return false;
00433             }
00434         }
00435         else
00436         {
00437             // 遅延初期化モード：カメラ認識時にはハードウェアストリームやセッションを起動せず、
00438             // 既定フォーマットの解像度・フレームレート等のメタ情報のみを保持しておく
00439             if (!availableFormats.empty())
00440             {
00441                 selectedFormatIndex = 0;
00442                 const auto& vFmt = availableFormats[0];
00443                 width = vFmt.width;
00444                 height = vFmt.height;
00445                 channels = 4;
00446                 interval = vFmt.fps > 0.0 ? 1000.0 / vFmt.fps : 10.0;
00447                 image.resize(static_cast<size_t>(width) * height * channels);
00448             }
00449         }
00450     }
00451
00452     return true;
00453 }
00454
00455 //
00456 // 使用可能な解像度、フレームレート、コーデックのリストを作成する
00457 //
00458 bool CamAvf::enumerateFormats()
00459 {
00460     // 以前のリストをクリアする
00461     formatList.clear();
00462     availableFormats.clear();
00463
00464     // デバイスが有効でなければ戻る
00465     if (!impl || !impl->device) return false;
00466
00467     @autoreleasepool {
00468         // デバイスがサポートするすべてのフォーマットを走査する
00469         NSArray<AVCaptureDeviceFormat*>* formats = impl->device.formats;
00470         for (NSUInteger fIdx = 0; fIdx < formats.count; ++fIdx)
00471         {
00472             AVCaptureDeviceFormat* format = formats[fIdx];
00473
00474             // フォーマット記述子から解像度を取得する
00475             CMVideoFormatDescriptionRef formatDesc = format.formatDescription;
00476             CMVideoDimensions dims = CMVideoFormatDescriptionGetDimensions(formatDesc);
00477             int w = dims.width;
00478             int h = dims.height;
00479
00480             // フォーマットのメディアサブタイプ（FourCC コーデックコード）を取得する
00481             FourCharCode subType = CMFormatDescriptionGetMediaSubType(formatDesc);
00482
00483             // サブタイプに対応するコーデック名文字列を判定する
00484             std::string codecName;
00485             switch (subType)
00486             {
00487             case kCVPixelFormatType_420YpCbCr8BiPlanarVideoRange:
00488             case kCVPixelFormatType_420YpCbCr8BiPlanarFullRange:
00489                 codecName = "NV12";
00490                 break;

```



```

00491     case kCVPixelFormatType_422YpCbCr8:
00492         codecName = "2vuy";
00493         break;
00494     case kCVPixelFormatType_422YpCbCr8_yuvs:
00495         codecName = "YUY2";
00496         break;
00497     case kCVPixelFormatType_32BGRA:
00498         codecName = "BGRA";
00499         break;
00500     case kCVPixelFormatType_32ARGB:
00501         codecName = "ARGB";
00502         break;
00503     case kCMVideoCodecType_JPEG:
00504         codecName = "MJPG";
00505         break;
00506     case kCMVideoCodecType_H264:
00507         codecName = "H264";
00508         break;
00509     case kCMVideoCodecType_HEVC:
00510         codecName = "HEVC";
00511         break;
00512     default:
00513     {
00514         // 既知の定数以外は FourCC 文字列または 16 進数値へ変換する
00515         char fcc[5] = {
00516             static_cast<char>((subType >> 24) & 0xFF),
00517             static_cast<char>((subType >> 16) & 0xFF),
00518             static_cast<char>((subType >> 8) & 0xFF),
00519             static_cast<char>(subType & 0xFF),
00520             '\0'
00521         };
00522         if (std::isprint(static_cast<unsigned char>(fcc[0])) &&
00523             std::isprint(static_cast<unsigned char>(fcc[1])) &&
00524             std::isprint(static_cast<unsigned char>(fcc[2])) &&
00525             std::isprint(static_cast<unsigned char>(fcc[3])))
00526         {
00527             codecName = fcc;
00528         }
00529         else
00530         {
00531             std::ostringstream oss;
00532             oss << "0x" << std::hex << subType;
00533             codecName = oss.str();
00534         }
00535         break;
00536     }
00537 }
00538
00539 // 各フォーマットがサポートするフレームレート範囲を走査する
00540 for (NSUInteger rIdx = 0; rIdx < format.videoSupportedFrameRateRanges.count; ++rIdx)
00541 {
00542     AVFrameRateRange* range = format.videoSupportedFrameRateRanges[rIdx];
00543     double maxFps = range.maxFrameRate;
00544
00545     // 実用的なフレームレート (5 fps 以上) のみを対象とする
00546     if (maxFps < 5.0) continue;
00547
00548     // UI 表示用の文字列を作成する
00549     std::ostringstream resOss;
00550     resOss << w << " x " << h;
00551     std::ostringstream fpsOss;
00552     fpsOss << std::fixed << std::setprecision(2) << maxFps;
00553
00554     // 重複チェック (同一解像度、同一 fps、同一コーデックのエントリは重複して追加しない)
00555     bool duplicate = false;
00556     for (const auto& fmt : formatList)
00557     {
00558         if (fmt.resolution == resOss.str() && fmt.fps == fpsOss.str() && fmt.codec == codecName)
00559         {
00560             duplicate = true;
00561             break;
00562         }
00563     }
00564     if (!duplicate)
00565     {
00566         // UI 表示用の構造体を追加する
00567         formatList.push_back({ resOss.str(), fpsOss.str(), codecName,
00568             static_cast<int>(availableFormats.size()) });
00569
00570         // ハードウェア設定用の詳細フォーマット情報を追加する
00571         availableFormats.emplace_back(w, h, maxFps, codecName, static_cast<int>(fIdx),
00572             static_cast<int>(rIdx));
00573     }
00574 }
00575

```

```

00576 // 1 つ以上のフォーマットが取得できていれば成功
00577 return !formatList.empty();
00578 }
00579
00580 //
00581 // フォーマットを選択して設定する
00582 //
00583 bool CamAvf::select(int index)
00584 {
00585     // インデックスの妥当性を検査する
00586     if (!impl || !impl->device || index < 0 || index >= static_cast<int>(availableFormats.size()))
00587     {
00588         return false;
00589     }
00590
00591     // キャプチャが実行中であれば一時停止する
00592     const bool restart{ running };
00593     if (restart) stop();
00594
00595     // 選択されたフォーマットのインデックスを保持する
00596     selectedFormatIndex = index;
00597     const auto& vFmt = availableFormats[index];
00598
00599     @autoreleasepool {
00600         NSError* error = nil;
00601
00602         // デバイスの設定を変更するために排他ロックを取得する
00603         if ([impl->device lockForConfiguration:&error])
00604         {
00605             if (static_cast<NSUInteger>(vFmt.formatIndex) < impl->device.formats.count)
00606             {
00607                 // 選択されたフォーマットをアクティブフォーマットとして適用する
00608                 AVCaptureDeviceFormat* fmt = impl->device.formats[vFmt.formatIndex];
00609                 impl->device.activeFormat = fmt;
00610
00611                 // フレームレートを CMTime 形式で設定する (最小・最大フレーム期間を指定)
00612                 int32_t fpsInt = static_cast<int32_t>(std::round(vFmt.fps));
00613                 if (fpsInt > 0)
00614                 {
00615                     impl->device.activeVideoMinFrameDuration = CMTimeMake(1, fpsInt);
00616                     impl->device.activeVideoMaxFrameDuration = CMTimeMake(1, fpsInt);
00617                 }
00618             }
00619
00620             // デバイス設定の排他ロックを解除する
00621             [impl->device unlockForConfiguration];
00622         }
00623     }
00624
00625     // カメラ基底クラスのプロパティを更新する
00626     width = vFmt.width;
00627     height = vFmt.height;
00628     channels = 4;
00629     interval = vFmt.fps > 0.0 ? 1000.0 / vFmt.fps : 10.0;
00630
00631     // 画像バッファのメモリ領域を確保する
00632     image.resize(static_cast<size_t>(width) * height * channels);
00633
00634     // 以前に実行中だった場合はキャプチャを再開する
00635     if (restart) start();
00636
00637     return true;
00638 }
00639
00640 //
00641 // レイテンシ優先モードを設定する
00642 //
00643 void CamAvf::setPrioritizeLatency(bool mode)
00644 {
00645     // 基底クラスの設定を更新する
00646     Camera::setPrioritizeLatency(mode);
00647
00648     // AV Foundation 出力側のフレーム破棄ポリシーに反映する
00649     if (impl && impl->output)
00650     {
00651         // レイテンシ優先時は遅延した古いフレームを自動破棄 (YES) し、
00652         // 全フレーム処理時はフレームを破棄せず順番通りに処理 (NO) する
00653         impl->output.alwaysDiscardsLateVideoFrames = mode ? YES : NO;
00654     }
00655 }
00656
00657 //
00658 // キャプチャ開始処理を行う
00659 //
00660 bool CamAvf::onStart()
00661 {
00662     if (!impl || !impl->device) return false;

```

```

00663
00664 @autoreleasepool {
00665     // セッションがまだ作成されていなければ初期化する
00666     if (!impl->session)
00667     {
00668         // キャプチャセッションを生成する
00669         impl->session = [[AVCaptureSession alloc] init];
00670
00671         // セッションの構成変更を開始する
00672         [impl->session beginConfiguration];
00673
00674         // デバイス入力を作成してセッションに追加する
00675         NSError* error = nil;
00676         impl->input = [AVCaptureDeviceInput deviceInputWithDevice:impl->device error:&error];
00677         if (!impl->input || ![impl->session canAddInput:impl->input])
00678         {
00679             [impl->session commitConfiguration];
00680             impl->session = nil;
00681             return false;
00682         }
00683         [impl->session addInput:impl->input];
00684
00685         // ビデオデータ出力を生成する
00686         impl->output = [[AVCaptureVideoDataOutput alloc] init];
00687
00688         // 出力フォーマットを 32-bit BGRA に指定する
00689         // OS / ハードウェア側で BGRA への変換が自動的に行われ、CPU 側での無駄な色変換を排除する
00690         impl->output.videoSettings = @{
00691             (id)kCVPixelBufferPixelFormatTypeKey : @(kCVPixelFormatType_32BGRA)
00692         };
00693
00694         // レイテンシ優先設定に応じてフレーム破棄を制御する
00695         impl->output.alwaysDiscardsLateVideoFrames = prioritizeLatency ? YES : NO;
00696
00697         // フレーム受信用のデリゲートとシリアルディスパッチキューを作成する
00698         impl->delegate = [[CamAvfDelegate alloc] initWithParent:this];
00699         impl->queue = dispatch_queue_create("CamAvfQueue", DISPATCH_QUEUE_SERIAL);
00700         [impl->output setSampleBufferDelegate:impl->delegate queue:impl->queue];
00701
00702         // セッションに出力を追加する
00703         if (![impl->session canAddOutput:impl->output])
00704         {
00705             [impl->session commitConfiguration];
00706             impl->session = nil;
00707             return false;
00708         }
00709         [impl->session addOutput:impl->output];
00710
00711         // セッションの構成変更を確定する
00712         [impl->session commitConfiguration];
00713     }
00714
00715     // セッション構成後に、選択されたフォーマットをデバイスに適用する
00716     if (selectedFormatIndex >= 0 && selectedFormatIndex < static_cast<int>(availableFormats.size()))
00717     {
00718         const auto& vFmt = availableFormats[selectedFormatIndex];
00719         NSError* error = nil;
00720         if ([impl->device lockForConfiguration:&error])
00721         {
00722             if (static_cast<NSUInteger>(vFmt.formatIndex) < impl->device.formats.count)
00723             {
00724                 impl->device.activeFormat = impl->device.formats[vFmt.formatIndex];
00725                 int32_t fpsInt = static_cast<int32_t>(std::round(vFmt.fps));
00726                 if (fpsInt > 0)
00727                 {
00728                     impl->device.activeVideoMinFrameDuration = CMTimeMake(1, fpsInt);
00729                     impl->device.activeVideoMaxFrameDuration = CMTimeMake(1, fpsInt);
00730                 }
00731             }
00732             [impl->device unlockForConfiguration];
00733         }
00734     }
00735
00736     // キャプチャセッションを開始する
00737     [impl->session startRunning];
00738
00739     // セッションが動作中であれば成功を返す
00740     return impl->session.isRunning;
00741 }
00742 }
00743
00744 //
00745 // キャプチャ停止処理を行う
00746 //
00747 void CamAvf::onStop()
00748 {
00749     if (impl)

```

```

00750 {
00751     // キャプチャセッションが動作中であれば停止する
00752     if (impl->session && impl->session.isRunning)
00753     {
00754         @autoreleasepool {
00755             [impl->session stopRunning];
00756         }
00757     }
00758
00759     // ディスバッatchキューに残っているフレーム処理の完了を待機する
00760     // これにより、停止後にコールバックが実行されて不正なメモリアクセスが発生するのを防ぐ
00761     if (impl->queue)
00762     {
00763         dispatch_sync(impl->queue, ^{});
00764     }
00765 }
00766 }
00767
00768 //
00769 // キャプチャデバイスを閉じる処理を行う
00770 //
00771 void CamAvf::onClose()
00772 {
00773     // 内部実装オブジェクトを解放する（セッション停止とリソース解放が行われる）
00774     if (impl)
00775     {
00776         impl.reset();
00777     }
00778
00779     // フォーマット情報をクリアする
00780     availableFormats.clear();
00781     formatList.clear();
00782     selectedFormatIndex = 0;
00783 }
00784
00785 //
00786 // サンプルバッファからフレームデータを取得してバッファを更新する
00787 //
00788 void CamAvf::handleSampleBuffer(const void* sampleBufferRef)
00789 {
00790     // キャプチャが停止していれば何もしない
00791     if (!running) return;
00792
00793     // 全フレーム処理モードの場合、メインスレッドが前フレームを処理（取得）するまで待機する
00794     if (!prioritizeLatency)
00795     {
00796         while (running && !prioritizeLatency && captured)
00797         {
00798             std::this_thread::sleep_for(std::chrono::milliseconds(1));
00799         }
00800         if (!running) return;
00801     }
00802
00803     // CMSampleBuffer から画像バッファ (CVImageBufferRef) を取得する
00804     CMSampleBufferRef sampleBuffer = (CMSampleBufferRef) sampleBufferRef;
00805     CVImageBufferRef imageBuffer = CMSampleBufferGetImageBuffer(sampleBuffer);
00806     if (!imageBuffer) return;
00807
00808     // 画像バッファのアドレスを読み取り専用でロックする
00809     CVPixelBufferLockBaseAddress(imageBuffer, kCVPixelBufferLock_ReadOnly);
00810
00811     // ピクセルバッファのサイズとストライド（1行あたりのバイト数）、先頭ポインタを取得する
00812     const size_t w = CVPixelBufferGetWidth(imageBuffer);
00813     const size_t h = CVPixelBufferGetHeight(imageBuffer);
00814     const size_t bytesPerRow = CVPixelBufferGetBytesPerRow(imageBuffer);
00815     const uint8_t* src = static_cast<const uint8_t*>(CVPixelBufferGetBaseAddress(imageBuffer));
00816
00817     if (src && w > 0 && h > 0)
00818     {
00819         // メインスレッドとのバッファ競合を防ぐためにミューテックスをロックする
00820         std::lock_guard<std::mutex> lock{ mtx };
00821
00822         // 解像度やチャンネル数が変更されていれば内部バッファをリサイズする
00823         if (width != static_cast<int>(w) || height != static_cast<int>(h) || channels != 4)
00824         {
00825             width = static_cast<int>(w);
00826             height = static_cast<int>(h);
00827             channels = 4;
00828             image.resize(static_cast<size_t>(width) * height * channels);
00829         }
00830
00831         // 1行あたりの有効バイト数を計算する (width * 4)
00832         const size_t rowBytes = static_cast<size_t>(width * channels);
00833
00834         // アライメントパディングの有無を判定してコピーする
00835         if (bytesPerRow == rowBytes)
00836         {

```

```

00837     // バディングがない場合は連続領域として一括コピーする
00838     std::memcpy(image.data(), src, std::min(image.size(), rowBytes * h));
00839 }
00840 else
00841 {
00842     // 各行末尾にバディングが含まれる場合は行ごとにコピーする
00843     const size_t copyWidth = std::min(rowBytes, bytesPerRow);
00844     for (size_t y = 0; y < h; ++y)
00845     {
00846         std::memcpy(image.data() + y * rowBytes, src + y * bytesPerRow, copyWidth);
00847     }
00848 }
00849
00850 // 新しいフレームが取得されたことを通知するフラグを立てる
00851 captured = true;
00852 }
00853
00854 // 画像バッファのアドレスロックを解除する
00855 CVPixelBufferUnlockBaseAddress(imageBuffer, kCVPixelBufferLock_ReadOnly);
00856 }
00857
00858 #endif // defined(__APPLE__)

```

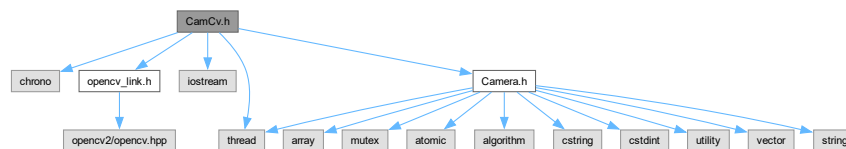
9.19 CamCv.h ファイル

```

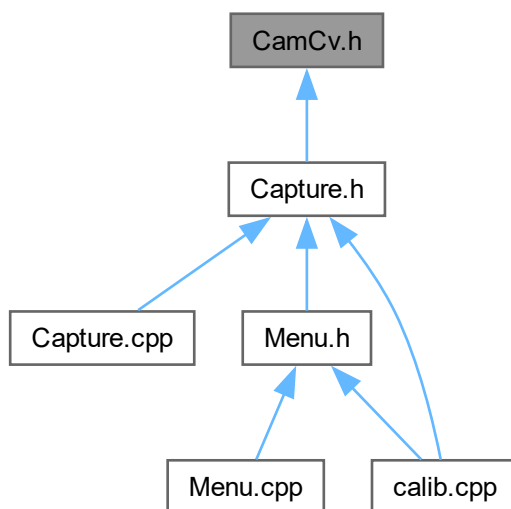
#include <chrono>
#include <thread>
#include <iostream>
#include "opencv_link.h"
#include "Camera.h"

```

CamCv.h の依存先関係図:



被依存関係図:



クラス

- class [CamCv](#)

9.19.1 詳解

OpenCV を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

December 27, 2022

[CamCv.h](#) に定義があります。

9.20 CamCv.h

[詳解]

```

00001 #pragma once
00002
00010
00011 #include <chrono>
00012 #include <thread>
00013 #include <iostream>
00014
00015 // OpenCV へのリンクとインクルード
00016 #include "opencv_link.h"
00017
00018 // カメラ関連の処理
00019 #include "Camera.h"
00020
00024 class CamCv : public Camera
00025 {
00026     cv::VideoCapture camera;
00027
00028     cv::Mat cvFrame;
00029
00030     double elapsedTime{ 0.0 };
00031
00032     double in{ 0.0 };
00033
00034     double out{ 0.0 };
00035
00036     double total{ 0.0 };
00037
00038     int exposure{ 0 }, gain{ 0 };
00039
00040     static double getTime()
00041     {
00042         using namespace std::chrono;
00043         return duration<double>(steady_clock::now().time_since_epoch()).count();
00044     }
00045
00046     bool init(int initialWidth, int initialHeight, double initialFps, const char* fourcc = "")
00047     {
00048         // カメラのコーデック・解像度・フレームレートを設定する
00049         if (fourcc[0] != '\0') camera.set(cv::CAP_PROP_FOURCC,
00050             cv::VideoWriter::fourcc(fourcc[0], fourcc[1], fourcc[2], fourcc[3]));
00051         if (initialWidth > 0) camera.set(cv::CAP_PROP_FRAME_WIDTH, initialWidth);
00052         if (initialHeight > 0) camera.set(cv::CAP_PROP_FRAME_HEIGHT, initialHeight);
00053         if (initialFps > 0.0) camera.set(cv::CAP_PROP_FPS, initialFps);
00054
00055         // fps が 0 より大きければ逆数をカメラの遅延に使う
00056         const auto fps{ camera.get(cv::CAP_PROP_FPS) };
00057         if (fps > 0.0) interval = 1000.0 / fps;
00058
00059         // ムービーファイルのインポイント・アウトポイントの初期値とフレーム数
00060         in = camera.get(cv::CAP_PROP_POS_FRAMES);
00061         out = total = camera.get(cv::CAP_PROP_FRAME_COUNT);
00062
00063         // 経過時間
00064         elapsedTime = 0.0;
00065
00066         // カメラから最初のフレームをキャプチャできなかったらカメラは使えない
00067         if (!camera.grab()) return false;
00068
00069         // カメラの利得と露出を取得する
00070         gain = static_cast<int>(camera.get(cv::CAP_PROP_GAIN));
00071         exposure = static_cast<int>(camera.get(cv::CAP_PROP_EXPOSURE) * 10.0);
00072
00073         // フレームを取り出してキャプチャ用のメモリを確保する
00074         camera.retrieve(cvFrame);
00075
00076 #if defined(DEBUG)
00077         char codec[5]{ 0, 0, 0, 0, 0 };
00078         getCodec(codec);
00079         std::cerr << "in:" << in << ", out:" << out
00080             << ", width:" << cvFrame.cols << ", height:" << cvFrame.rows
00081             << ", fourcc: " << codec << "\n";
00082 #endif
00083
00084         // 基底クラスのバッファとメンバを更新
00085         width = cvFrame.cols;
00086         height = cvFrame.rows;
00087         channels = cvFrame.channels();
00088         {
00089             const auto size{ static_cast<size_t>(cvFrame.total() * cvFrame.elemSize()) };
00090             image.resize(size);
00091             std::memcpy(image.data(), cvFrame.data, size);
00092         }
00093     }

```

```

00114
00115 // フレームがキャプチャされたことを記録する
00116 captured = true;
00117
00118 // カメラが使える
00119 return true;
00120 }
00121
00122 void capture()
00123 {
00124 // 再生開始時刻
00125 auto startTime{ getTime() };
00126
00127 // スレッドが実行可の間
00128 while (running)
00129 {
00130 // フレームを取り出せたら true
00131 auto status{ (total <= 0.0 || camera.get(cv::CAP_PROP_POS_FRAMES) < out) && camera.grab() };
00132
00133 // ムービーファイルでないかムービーファイルの終端でなければ次のフレームを取り出して
00134 if (status && camera.retrieve(cvFrame))
00135 {
00136 // 単一バッファをロックしてから
00137 std::lock_guard<std::mutex> lock{ mtx };
00138
00139 // 基底クラスのバッファとメンバを直接更新
00140 width = cvFrame.cols;
00141 height = cvFrame.rows;
00142 channels = cvFrame.channels();
00143 const auto size{ static_cast<size_t>(cvFrame.total() * cvFrame.elemSize()) };
00144 image.resize(size);
00145 std::memcpy(image.data(), cvFrame.data, size);
00146
00147 // 新しいフレームがキャプチャされたことを通知する
00148 captured = true;
00149 }
00150
00151 // 遅延時間
00152 auto deferred{ 0.0 };
00153
00154 // ムービーファイルから入力しているとき
00155 if (total > 0.0)
00156 {
00157 // ムービーファイルの終端に到達していたら
00158 if (!status)
00159 {
00160 // インポイントまで巻き戻す
00161 camera.set(cv::CAP_PROP_POS_FRAMES, in);
00162
00163 // 経過時間を戻す
00164 elapsedTime = 0.0;
00165
00166 // 開始時間を更新する
00167 startTime = getTime();
00168 }
00169 else
00170 {
00171 // 現在のフレームのインポイントからの経過時間
00172 const auto pos{ camera.get(cv::CAP_PROP_POS_MSEC) - in };
00173
00174 // 再生位置の次のフレームの時刻に対する経過時間
00175 const auto now{ (elapsedTime + getTime() - startTime) * 1000.0 + interval };
00176
00177 // 遅延時間はフレームの経過時間と現在の経過時間の差
00178 deferred = pos - now;
00179 }
00180 }
00181
00182 // 遅延時間あれば待つ
00183 if (deferred > 0.0)
00184 {
00185 std::this_thread::sleep_for(std::chrono::milliseconds(static_cast<int>(deferred)));
00186 }
00187
00188 // 再生停止までの経過時間を積算する
00189 elapsedTime += getTime() - startTime;
00190 }
00191
00192 protected:
00193
00203 bool onStart() override
00204 {
00205 if (!camera.isOpened()) return false;
00206 thr = std::thread(&CamCv::capture, this);
00207 return true;
00208 }

```



```

00209
00213 void onStop() override
00214 {
00215     // capture() の while (running) か抜ける
00216 }
00217
00221 void onClose() override
00222 {
00223     camera.release();
00224     in = 0.0;
00225     out = 0.0;
00226     total = 0.0;
00227     elapsedTime = 0.0;
00228 }
00229
00230 public:
00231
00235 CamCv() = default;
00236
00240 virtual ~CamCv()
00241 {
00242     close();
00243 }
00244
00256 bool open(int device, int width = 0, int height = 0, double fps = 0.0, const char* fourcc = "", int
pref = cv::CAP_ANY)
00257 {
00258     // カメラを開いて初期化する
00259     return camera.open(device, pref) && init(width, height, fps, fourcc);
00260 }
00261
00273 bool open(const std::string& file, int width = 0, int height = 0, double fps = 0.0, const char*
fourcc = "", int pref = cv::CAP_ANY)
00274 {
00275     // ファイル／ネットワークを開いて初期化する
00276     return camera.open(file, pref) && init(width, height, fps, fourcc);
00277 }
00278
00284 virtual double getFps() const override
00285 {
00286     return camera.get(cv::CAP_PROP_FPS);
00287 }
00288
00294 unsigned int getCodec() const
00295 {
00296     return static_cast<unsigned int>(camera.get(cv::CAP_PROP_FOURCC));
00297 }
00298
00304 void getCodec(char* fourcc) const
00305 {
00306     auto cc{ getCodec() };
00307     for (int i = 0; i < 4; ++i)
00308     {
00309         fourcc[i] = static_cast<char>(cc & 0x7f);
00310         if (!isalnum(fourcc[i])) fourcc[i] = '?';
00311         cc >>= 8;
00312     }
00313 }
00314
00320 double getPosition() const
00321 {
00322     return camera.get(cv::CAP_PROP_POS_FRAMES);
00323 }
00324
00330 void setPosition(double frame)
00331 {
00332     camera.set(cv::CAP_PROP_POS_FRAMES, frame);
00333 }
00334
00340 void setExposure(double exposure)
00341 {
00342     if (camera.isOpened()) camera.set(cv::CAP_PROP_EXPOSURE, exposure);
00343 }
00344
00348 void increaseExposure() override
00349 {
00350     setExposure(++exposure * 0.1);
00351 }
00352
00356 void decreaseExposure() override
00357 {
00358     setExposure(--exposure * 0.1);
00359 }
00360
00366 void setGain(double gain)
00367 {
00368     if (camera.isOpened()) camera.set(cv::CAP_PROP_GAIN, gain);

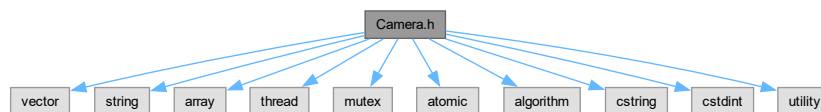
```

```
00369     }  
00370  
00374     void increaseGain() override  
00375     {  
00376         setGain(++gain);  
00377     }  
00378  
00382     void decreaseGain() override  
00383     {  
00384         setGain(--gain);  
00385     }  
00386 };
```

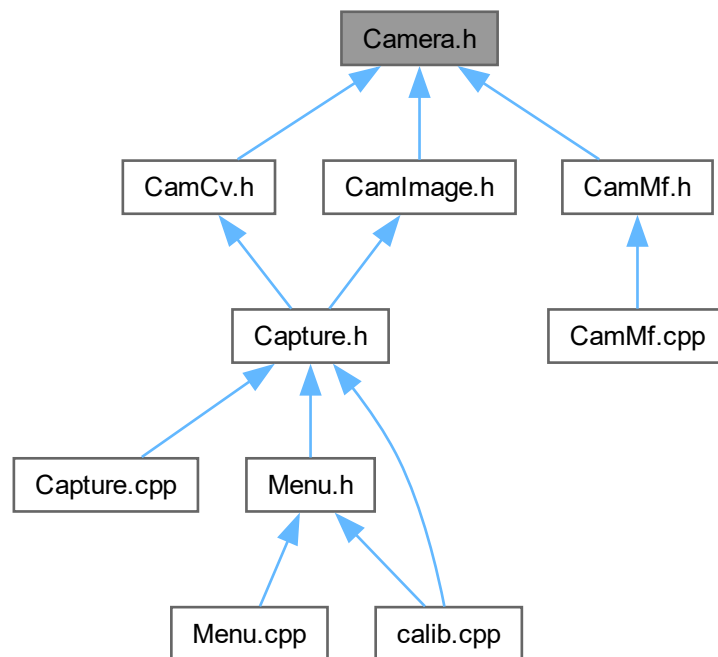
9.21 Camera.h ファイル

```
#include <vector>  
#include <string>  
#include <array>  
#include <thread>  
#include <mutex>  
#include <atomic>  
#include <algorithm>  
#include <cstring>  
#include <cstdint>  
#include <utility>
```

Camera.h の依存先関係図:



被依存関係図:



クラス

- struct [CaptureFormat](#)
- class [Camera](#)

9.21.1 詳解

キャプチャデバイス関連の基底クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Camera.h](#) に定義があります。

9.22 Camera.h

[詳解]

```

00001 #pragma once
00002
00010
00011 #include <vector>
00012 #include <string>
00013 #include <array>
00014 #include <thread>
00015 #include <mutex>
00016 #include <atomic>
00017 #include <algorithm>
00018 #include <cstring>
00019 #include <cstdint>
00020 #include <utility>
00021
00030 struct CaptureFormat
00031 {
00032     std::string resolution;
00033     std::string fps;
00034     std::string codec;
00035     int index{ 0 };
00036
00045     CaptureFormat(const std::string& resolution, const std::string& fps,
00046                 const std::string& codec, int index)
00047         : resolution{ resolution }
00048         , fps{ fps }
00049         , codec{ codec }
00050         , index{ index }
00051     {
00052     }
00053 };
00054
00058 class Camera
00059 {
00060 private:
00061
00062     // コピー・代入は禁止
00063     Camera(const Camera&) = delete;
00064     Camera& operator=(const Camera&) = delete;
00065
00066 protected:
00067
00069     int width{ 0 };
00070
00072     int height{ 0 };
00073
00075     int channels{ 0 };
00076
00078     double interval{ 10.0 };
00079
00081     std::vector<std::uint8_t> image;
00082
00084     mutable std::mutex mtx;
00085
00087     std::atomic<bool> running{ false };
00088
00090     std::atomic<bool> captured{ false };
00091
00093     std::atomic<bool> prioritizeLatency{ false };
00094
00096     std::thread thr;
00097
00103     virtual bool onStart() = 0;
00104
00108     virtual void onStop() = 0;
00109
00113     virtual void onClose() = 0;
00114
00115 public:
00116
00120     Camera() = default;
00121
00125     virtual ~Camera() = default;
00126
00130     void start()
00131     {
00132         std::lock_guard<std::mutex> lock{ mtx };
00133         if (running) return;
00134         if (onStart())
00135         {
00136             running = true;
00137         }
00138     }

```

```

00139
00143 void stop()
00144 {
00145     if (!running) return;
00146     running = false;
00147     onStop();
00148     if (thr.joinable())
00149     {
00150         thr.join();
00151     }
00152 }
00153
00157 void close()
00158 {
00159     stop();
00160     std::lock_guard<std::mutex> lock{ mtx };
00161     onClose();
00162     captured = false;
00163     width = 0;
00164     height = 0;
00165     channels = 0;
00166     image.clear();
00167 }
00168
00174 bool isRunning() const
00175 {
00176     return running;
00177 }
00178
00184 bool isCaptured() const
00185 {
00186     return captured;
00187 }
00188
00196 template <typename F>
00197 bool lockFrame(F&& func)
00198 {
00199     std::unique_lock<std::mutex> lock{ mtx, std::try_to_lock };
00200     if (lock.owns_lock() && captured && !image.empty())
00201     {
00202         const auto length{ static_cast<size_t>(width) * height * channels };
00203         func(image.data(), std::min(image.size(), length), width, height, channels);
00204         if (!isStillImage()) captured = false;
00205         return true;
00206     }
00207     return false;
00208 }
00209
00215 virtual bool isStillImage() const
00216 {
00217     return false;
00218 }
00219
00225 virtual const std::vector<CaptureFormat>& getFormatList() const
00226 {
00227     static const std::vector<CaptureFormat> empty;
00228     return empty;
00229 }
00230
00237 virtual bool selectFormat(int index)
00238 {
00239     return false;
00240 }
00241
00247 std::array<int, 2> getSize() const
00248 {
00249     return std::array<int, 2>{ width, height };
00250 }
00251
00257 int getWidth() const
00258 {
00259     return width;
00260 }
00261
00267 int getHeight() const
00268 {
00269     return height;
00270 }
00271
00277 int getChannels() const
00278 {
00279     return channels;
00280 }
00281
00287 virtual double getFps() const
00288 {
00289     return interval > 0.0 ? 1000.0 / interval : 0.0;

```

```

00290     }
00291
00297     virtual void setPrioritizeLatency(bool mode)
00298     {
00299         prioritizeLatency = mode;
00300     }
00301
00307     bool getPrioritizeLatency() const
00308     {
00309         return prioritizeLatency;
00310     }
00311
00315     virtual void increaseExposure()
00316     {
00317     }
00318
00322     virtual void decreaseExposure()
00323     {
00324     }
00325
00329     virtual void increaseGain()
00330     {
00331     }
00332
00336     virtual void decreaseGain()
00337     {
00338     }
00339 };

```

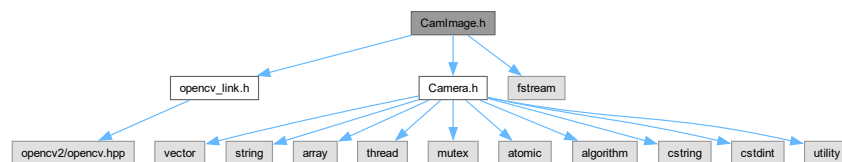
9.23 CamImage.h ファイル

```

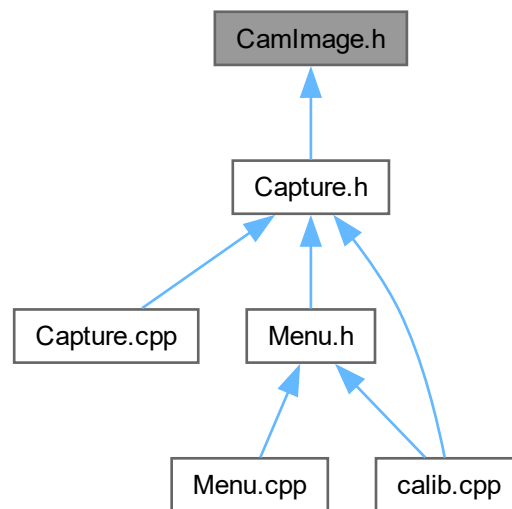
#include "opencv_link.h"
#include "Camera.h"
#include <fstream>

```

CamImage.h の依存先関係図:



被依存関係図:



クラス

- class [CamImage](#)

9.23.1 詳解

OpenCV を使って画像ファイルを読み込むクラス

著者

Kohe Tokoi

日付

November 15, 2022

[CamImage.h](#) に定義があります。

9.24 CamImage.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // OpenCV へのリンクとインクルード
00012 #include "opencv_link.h"
00013
00014 // カメラ関連の処理
00015 #include "Camera.h"
00016
00017 // ファイル入出力
00018 #include <fstream>
00019
00023 class CamImage : public Camera
00024 {
00025 protected:
00026
00032     bool onStart() override
00033     {
00034         return true;
00035     }
00036
00040     void onStop() override
00041     {
00042     }
00043
00047     void onClose() override
00048     {
00049     }
00050
00051 public:
00052
00056     CamImage() = default;
00057
00064     CamImage(const std::string& filename, bool flip = false)
00065     {
00066         open(filename, flip);
00067     }
00068
00072     virtual ~CamImage()
00073     {
00074         close();
00075     }
00076
00082     bool isStillImage() const override
00083     {
00084         return true;
00085     }
00086
00094     bool open(const std::string& filename, bool flip = false)
00095     {
00096         // 画像ファイルを読み込む
00097         cv::Mat cvFrame;
00098         if (!load(filename, cvFrame)) return false;
00099
00100         // 必要なら上下を反転する
00101         if (flip) cv::flip(cvFrame, cvFrame, 1);
00102
00103         // 基底クラスの単一バッファに安全にコピー
00104         {
00105             std::lock_guard<std::mutex> lock{ mtx };
00106             width = cvFrame.cols;
00107             height = cvFrame.rows;
00108             channels = cvFrame.channels();
00109             const size_t size = cvFrame.total() * cvFrame.elemSize();
00110             image.resize(size);
00111             std::memcpy(image.data(), cvFrame.data, size);
00112             captured = true;
00113         }
00114
00115         return true;
00116     }
00117
00123     bool isOpened() const
00124     {
00125         return !image.empty();
00126     }
00127
00135     static bool load(const std::string& filename, cv::Mat& frame)
00136     {
00137         // 画像ファイルをバイナリモードで開いて読み込み位置を最後に移動する
00138         std::ifstream file(UTF8ToTChar(filename),
00139             std::ifstream::in |

```



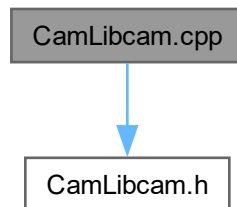
```

00140         std::ifstream::binary |
00141         std::ifstream::ate);
00142
00143     // 画像ファイルが開けたら
00144     if (file.is_open())
00145     {
00146         // 画像ファイルを読み込むメモリを確保する
00147         std::vector<char> buffer(static_cast<std::vector<char>::size_type>(file.tellg()));
00148
00149         // 画像ファイルを先頭から全部読み込む
00150         file.seekg(0, std::ifstream::beg);
00151         file.read(buffer.data(), buffer.size());
00152
00153         // 画像ファイルを閉じる
00154         file.close();
00155
00156         // 画像ファイルが読み込めたら
00157         if (file.good())
00158         {
00159             // 読み込んだ画像データを復号して返す
00160             frame = cv::imdecode(buffer, cv::IMREAD_COLOR);
00161             return true;
00162         }
00163     }
00164
00165     // 読み込めなかった
00166     return false;
00167 }
00168
00176 static bool save(const std::string& filename, const cv::Mat& image)
00177 {
00178     // ファイル名の拡張子の場所を取り出す
00179     const auto pos{ filename.find_last_of('.') };
00180
00181     // 拡張子があればそれを取り出し、無ければ ".png" にする
00182     const std::string ext{ pos != std::string::npos ? filename.substr(pos) : ".png" };
00183
00184     // 画像の符号化に失敗したら戻る
00185     std::vector<uchar> buffer;
00186     if (!cv::imencode(ext, image, buffer)) return false;
00187
00188     // 画像ファイルをバイナリモードで開く
00189     std::ofstream file(UTF8ToTChar(filename),
00190         std::ofstream::out |
00191         std::ofstream::binary);
00192
00193     // 画像ファイルが開けなかったら戻る
00194     if (file.fail()) return false;
00195
00196     // 画像データをファイルに書き込む
00197     const auto ptr{ reinterpret_cast<char*>(buffer.data()) };
00198     file.write(ptr, buffer.size());
00199
00200     // 画像ファイルを閉じる
00201     file.close();
00202
00203     // 読み込めたかどうかを返す
00204     return file.good();
00205 }
00206 };

```

9.25 CamLibcam.cpp ファイル

`#include "CamLibcam.h"`
CamLibcam.cpp の依存先関係図:



9.25.1 詳解

libcamera を使ったビデオキャプチャクラスの実装

著者

Kohe Tokoi

日付

March 2026

[CamLibcam.cpp](#) に定義があります。

9.26 CamLibcam.cpp

[[詳解](#)]

```
00001
00008
00009 #include "CamLibcam.h"
00010
00011 #if defined(USE_LIBCAMERA)
00012
00013 // libcamera formats
00014 #include <libcamera/formats.h>
00015 #include <libcamera/control_ids.h>
00016
00017 // POSIX
00018 #include <sys/mman.h>
00019 #include <unistd.h>
00020 #include <cstring>
00021 #include <iostream>
00022 #include <algorithm>
00023
00024 //
00025 // libcamera の CameraManager を管理するクラスのコンストラクタ
00026 //
```

```

00027 CamLibcam::Manager::Manager()
00028 : cm{ std::make_unique<libcamera::CameraManager>() }
00029 {
00030     // カメラマネージャを起動する
00031     const int ret{ cm->start() };
00032
00033     // 起動に成功したかどうかを記録する
00034     started = (ret == 0);
00035
00036     // 起動に失敗した場合はエラーメッセージを出力する
00037     if (!started)
00038     {
00039         std::cerr << "Failed to start libcamera CameraManager: " << ret << std::endl;
00040     }
00041 }
00042
00043 //
00044 // libcamera の CameraManager を管理するクラスのデストラクタ
00045 //
00046 CamLibcam::Manager::~Manager()
00047 {
00048     // カメラマネージャが起動していれば停止する
00049     if (started)
00050     {
00051         cm->stop();
00052     }
00053 }
00054
00055 //
00056 // CameraManager シングルトンインスタンスへの参照を返す
00057 //
00058 CamLibcam::Manager& CamLibcam::Manager::getInstance()
00059 {
00060     // シングルトンインスタンス
00061     static Manager instance;
00062
00063     // インスタンスへの参照を返す
00064     return instance;
00065 }
00066
00067 //
00068 // 検出されたカメラデバイスの表示名リストを返す
00069 //
00070 const std::vector<std::string>& CamLibcam::Manager::getDeviceList()
00071 {
00072     // デバイスリストを初期化する
00073     deviceList.clear();
00074
00075     // カメラマネージャが正常に起動していれば
00076     if (started)
00077     {
00078         // カメラデバイスのインデックス
00079         int index{ 0 };
00080
00081         // システムに存在するカメラデバイスを順次走査する
00082         for (const auto& cam : cm->cameras())
00083         {
00084             // デバイスインデックスとカメラ ID を組み合わせた表示名を作成する
00085             std::string name{ "Camera " + std::to_string(index) + ": " + cam->id() };
00086
00087             // 表示名リストに追加する
00088             deviceList.emplace_back(name);
00089             ++index;
00090         }
00091     }
00092
00093     // デバイスリストを返す
00094     return deviceList;
00095 }
00096
00097 //
00098 // コンストラクタ
00099 //
00100 CamLibcam::CamLibcam(int deviceNumber, int initial.width, int initial.height, double initial.fps)
00101 {
00102     // 指定されたパラメータでカメラを開く
00103     open(deviceNumber, initial.width, initial.height, initial.fps);
00104 }
00105
00106 //
00107 // デストラクタ
00108 //
00109 CamLibcam::~CamLibcam()
00110 {
00111     // カメラを閉じてリソースを解放する
00112     close();
00113 }

```

```

00114
00115 //
00116 // カメラを開く
00117 //
00118 bool CamLibcam::open(int deviceNumber, int initial.width, int initial.height, double initial.fps)
00119 {
00120     // 既に開いているカメラがあれば閉じる
00121     close();
00122
00123     // カメラマネージャを取得する
00124     auto* cm{ Manager::getInstance().get() };
00125     if (!cm) return false;
00126
00127     // 利用可能なカメラ一覧を取得する
00128     const auto cameras{ cm->cameras() };
00129
00130     // デバイス番号が有効な範囲内にあるか検証する
00131     if (deviceNumber < 0 || deviceNumber >= static_cast<int>(cameras.size()))
00132     {
00133         std::cerr << "libcamera: Invalid camera index " << deviceNumber << std::endl;
00134         return false;
00135     }
00136
00137     // 指定されたインデックスのカメラを取得する
00138     camera = cameras[deviceNumber];
00139     if (!camera) return false;
00140
00141     // カメラデバイスの排他制御権を獲得する
00142     if (camera->acquire() != 0)
00143     {
00144         std::cerr << "libcamera: Failed to acquire camera " << deviceNumber << " (" << camera->id() << ")"
00145         << std::endl;
00146         camera.reset();
00147         return false;
00148     }
00149
00150     std::cout << "libcamera: Opened camera " << deviceNumber << ": " << camera->id() << std::endl;
00151
00152     // ストリームロールを順次試行する (Viewfinder -> VideoRecording -> Raw)
00153     const std::vector<libcamera::StreamRole> roles = {
00154         libcamera::StreamRole::Viewfinder,
00155         libcamera::StreamRole::VideoRecording,
00156         libcamera::StreamRole::Raw
00157     };
00158
00159     // 設定成功フラグ
00160     bool configured{ false };
00161
00162     // 各ロールについて設定生成を試行する
00163     for (const auto role : roles)
00164     {
00165         // ロールに応じた既定設定を生成する
00166         config = camera->generateConfiguration({ role });
00167         if (!config || config->empty()) continue;
00168
00169         // ストリーム設定の参照を取得する
00170         auto& streamConfig{ config->at(0) };
00171
00172         // 希望解像度が指定されている場合は設定に反映する
00173         if (initial.width > 0 && initial.height > 0)
00174         {
00175             streamConfig.size.width = initial.width;
00176             streamConfig.size.height = initial.height;
00177         }
00178
00179         // サポートされているピクセルフォーマット一覧を取得する
00180         const auto supportedFormats{ streamConfig.formats().pixelformats() };
00181         std::cout << "libcamera: Supported pixel formats: ";
00182         for (const auto& fmt : supportedFormats)
00183         {
00184             std::cout << fmt.toString() << " ";
00185         }
00186         std::cout << std::endl;
00187
00188         // 優先順位リストに基づいてピクセルフォーマットを選択する (XBGR8888 は memcpy 可能で最速)
00189         const std::vector<libcamera::PixelFormat> preferenceList = {
00190             libcamera::formats::R8, // モノクロ 8-bit (OV9281 等)
00191             libcamera::formats::XBGR8888, // 4-byte BGRA 互換 (memcpy 可能・超高速)
00192             libcamera::formats::BGRX8888,
00193             libcamera::formats::XRGB8888,
00194             libcamera::formats::RGBX8888,
00195             libcamera::formats::BGR888,
00196             libcamera::formats::RGB888,
00197             libcamera::formats::YUYV,
00198             libcamera::formats::NV12,
00199             libcamera::formats::YUV420,
00200         };

```

```

00200
00201 // 優先順位順にサポートフォーマットを探索する
00202 bool formatSelected{ false };
00203 for (const auto& pref : preferenceList)
00204 {
00205     if (std::find(supportedFormats.begin(), supportedFormats.end(), pref) != supportedFormats.end())
00206     {
00207         streamConfig.pixelFormat = pref;
00208         formatSelected = true;
00209         break;
00210     }
00211 }
00212
00213 // 優先フォーマットが見つからなければサポート一覧の先頭を採用する
00214 if (!formatSelected && !supportedFormats.empty())
00215 {
00216     streamConfig.pixelFormat = supportedFormats.front();
00217 }
00218
00219 // パイプラインによる設定の検証と調整を行う
00220 const auto status = config->validate();
00221 if (status == libcamera::CameraConfiguration::Invalid)
00222 {
00223     std::cerr << "libcamera: Configuration invalid for role, trying next..." << std::endl;
00224     continue;
00225 }
00226 else if (status == libcamera::CameraConfiguration::Adjusted)
00227 {
00228     std::cout << "libcamera: Configuration was adjusted by pipeline" << std::endl;
00229 }
00230
00231 // カメラに設定を適用する
00232 if (camera->configure(config.get()) == 0)
00233 {
00234     configured = true;
00235     break;
00236 }
00237 else
00238 {
00239     std::cerr << "libcamera: configure() failed for role, trying next..." << std::endl;
00240 }
00241 }
00242
00243 // いずれのロールでも設定に失敗した場合はカメラを解放して戻る
00244 if (!configured)
00245 {
00246     std::cerr << "libcamera: Failed to configure camera with any stream role" << std::endl;
00247     camera->release();
00248     camera.reset();
00249     return false;
00250 }
00251
00252 // 確定したストリーム設定からパラメータを取得する
00253 auto& streamConfig{ config->at(0) };
00254 stream = streamConfig.stream();
00255 pixelFormat = streamConfig.pixelFormat;
00256 stride = streamConfig.stride;
00257 width = streamConfig.size.width;
00258 height = streamConfig.size.height;
00259 channels = 4; // 内部バッファは常に BGRA 4チャンネル
00260
00261 std::cout << "libcamera: Final configuration - format: " << pixelFormat.toString()
00262           << ", resolution: " << width << "x" << height
00263           << ", stride: " << stride << std::endl;
00264
00265 // 要求フレームレートからフレーム時間（マイクロ秒）とインターバルを計算する
00266 if (initial.fps > 0.0)
00267 {
00268     frameDurationUs = static_cast<int64_t>(1000000.0 / initial.fps);
00269     interval = 1000.0 / initial.fps;
00270 }
00271 else
00272 {
00273     frameDurationUs = 33333; // 既定 30fps
00274     interval = 33.3;
00275 }
00276
00277 // フレームデータ格納用のバッファメモリを確保する
00278 const size_t bufferSize{ static_cast<size_t>(width * height * channels) };
00279 frame.resize(bufferSize, 255);
00280 image.resize(bufferSize, 255);
00281
00282 // フレームバッファアロケータを作成しストリームにバッファを割り当てる
00283 allocator = std::make_unique<libcamera::FrameBufferAllocator>(camera);
00284 if (allocator->allocate(stream) < 0)
00285 {
00286     std::cerr << "libcamera: Failed to allocate buffers" << std::endl;

```

```

00287     close();
00288     return false;
00289 }
00290
00291 // 割り当てられたバッファの各プレーンをプロセスのアドレス空間にマッピングする
00292 for (const auto& buffer : allocator->buffers(stream))
00293 {
00294     std::vector<MappedPlane> planes;
00295     for (const auto& plane : buffer->planes())
00296     {
00297         // ファイルディスクリプタからメモリマップを作成する
00298         void* memory{ ::mmap(NULL, plane.length, PROT_READ, MAP_SHARED, plane.fd.get(), plane.offset) };
00299         if (memory == MAP_FAILED)
00300         {
00301             std::cerr << "libcamera: Failed to mmap plane" << std::endl;
00302             memory = nullptr;
00303         }
00304         planes.push_back({ memory, plane.length });
00305     }
00306     mappedBuffers[buffer.get()] = std::move(planes);
00307
00308     // キャプチャ用リクエストを作成する
00309     auto request{ camera->createRequest() };
00310     if (!request)
00311     {
00312         std::cerr << "libcamera: Failed to create request" << std::endl;
00313         close();
00314         return false;
00315     }
00316
00317     // リクエストにバッファを割り当てる
00318     if (request->addBuffer(stream, buffer.get()) < 0)
00319     {
00320         std::cerr << "libcamera: Failed to add buffer to request" << std::endl;
00321         close();
00322         return false;
00323     }
00324
00325     // リクエストリストに追加する
00326     requests.push_back(std::move(request));
00327 }
00328
00329 // フレーム完了通知コールバックを接続する
00330 camera->requestCompleted.connect(this, &CamLibcam::requestComplete);
00331
00332 // キャプチャフラグを初期化する
00333 captured = false;
00334 return true;
00335 }
00336
00337 //
00338 // メモリマッピングされたバッファを解放する
00339 //
00340 void CamLibcam::unmapBuffers()
00341 {
00342     // すべてのマッピング済みプレーンについて munmap を実行する
00343     for (auto& [buf, planes] : mappedBuffers)
00344     {
00345         for (auto& plane : planes)
00346         {
00347             if (plane.address && plane.address != MAP_FAILED)
00348             {
00349                 ::munmap(plane.address, plane.length);
00350             }
00351         }
00352     }
00353
00354     // マッピングテーブルをクリアする
00355     mappedBuffers.clear();
00356 }
00357
00358 //
00359 // カメラを閉じる処理を行う
00360 //
00361 void CamLibcam::onClose()
00362 {
00363     // コールバック接続を解除する
00364     if (camera)
00365     {
00366         camera->requestCompleted.disconnect(this, &CamLibcam::requestComplete);
00367     }
00368
00369     // リクエスト一覧をクリアする
00370     requests.clear();
00371
00372     // メモリマッピングを解除する
00373     unmapBuffers();

```

```

00374
00375 // アロケータと設定オブジェクトを解放する
00376 allocator.reset();
00377 config.reset();
00378
00379 // カメラデバイスの排他制御権を解放する
00380 if (camera)
00381 {
00382     camera->release();
00383     camera.reset();
00384 }
00385
00386 // ストリームをリセットする
00387 stream = nullptr;
00388 }
00389
00390 //
00391 // キャプチャを開始する
00392 //
00393 bool CamLibcam::onStart()
00394 {
00395     // カメラが存在しなければ開始できない
00396     if (!camera) return false;
00397
00398     // キャプチャ開始時のコントロールパラメータを設定する
00399     libcamera::ControlList startControls;
00400
00401     // 自動露出 (AE) を有効化し、フレームレート (フレーム時間) を設定する
00402     startControls.set(libcamera::controls::AeEnable, true);
00403     startControls.set(libcamera::controls::FrameDurationLimits,
00404         libcamera::Span<const int64_t, 2>({ frameDurationUs, frameDurationUs }));
00405
00406     // カメラのストリーミングを開始する
00407     if (camera->start(&startControls) < 0)
00408     {
00409         std::cerr << "libcamera: Failed to start camera" << std::endl;
00410         return false;
00411     }
00412
00413     // 準備したすべてのリクエストをカメラのキューに投入する
00414     for (auto& request : requests)
00415     {
00416         camera->queueRequest(request.get());
00417     }
00418
00419     return true;
00420 }
00421
00422 //
00423 // キャプチャを停止する
00424 //
00425 void CamLibcam::onStop()
00426 {
00427     // カメラが存在すればストリーミングを停止する
00428     if (camera)
00429     {
00430         camera->stop();
00431     }
00432 }
00433
00434 //
00435 // フレーム完了コールバック
00436 //
00437 void CamLibcam::requestComplete(libcamera::Request* request)
00438 {
00439     // リクエストがキャンセルされた場合は何もしない
00440     if (request->status() == libcamera::Request::RequestCancelled)
00441     {
00442         return;
00443     }
00444
00445     // 完了したリクエストのバッファマップを取得する
00446     auto bufferMap{ request->buffers() };
00447     auto it{ bufferMap.find(stream) };
00448     if (it != bufferMap.end())
00449     {
00450         // ストリームのフレームバッファとマッピングプレーンを取得する
00451         libcamera::FrameBuffer* buffer{ it->second };
00452         const auto& planes{ mappedBuffers[buffer] };
00453
00454         // バッファアドレスが有効であれば画像データを変換・格納する
00455         if (!planes.empty() && planes[0].address)
00456         {
00457             // ソースメモリの先頭アドレス
00458             const uint8_t* src{ static_cast<const uint8_t*>(planes[0].address) };
00459
00460             // バッファアクセスをスレッドセーフに行うための排他ロック

```

```

00461     std::lock_guard<std::mutex> lock(mtx);
00462
00463     // 格納先バッファの先頭アドレス
00464     uint8_t* dst{ image.data() };
00465
00466     // ピクセルフォーマットに応じて BGRA 4チャンネルへ変換する
00467     if (pixelFormat == libcamera::formats::BGR888)
00468     {
00469         // 3バイト BGR から 4バイト BGRA へ展開する
00470         for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00471         {
00472             const uint8_t* rowSrc{ src + y * stride };
00473             uint8_t* rowDst{ dst + y * width * 4 };
00474             for (size_t x = 0; x < static_cast<size_t>(width); ++x)
00475             {
00476                 rowDst[0] = rowSrc[0]; // B
00477                 rowDst[1] = rowSrc[1]; // G
00478                 rowDst[2] = rowSrc[2]; // R
00479                 rowDst[3] = 255;      // A
00480                 rowSrc += 3;
00481                 rowDst += 4;
00482             }
00483         }
00484     }
00485     else if (pixelFormat == libcamera::formats::RGB888)
00486     {
00487         // 3バイト RGB から 4バイト BGRA へ色順序を入れ替えて展開する
00488         for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00489         {
00490             const uint8_t* rowSrc{ src + y * stride };
00491             uint8_t* rowDst{ dst + y * width * 4 };
00492             for (size_t x = 0; x < static_cast<size_t>(width); ++x)
00493             {
00494                 rowDst[0] = rowSrc[2]; // B
00495                 rowDst[1] = rowSrc[1]; // G
00496                 rowDst[2] = rowSrc[0]; // R
00497                 rowDst[3] = 255;      // A
00498                 rowSrc += 3;
00499                 rowDst += 4;
00500             }
00501         }
00502     }
00503     else if (pixelFormat == libcamera::formats::XBGR8888 || pixelFormat ==
libcamera::formats::BGRX8888)
00504     {
00505         // BGRA 互換フォーマットの高速コピー
00506         if (stride == static_cast<unsigned int>(width * 4))
00507         {
00508             // スライド幅*4と一致する場合は全体を一括 memcpy する
00509             std::memcpy(dst, src, static_cast<size_t>(width * height * 4));
00510         }
00511         else
00512         {
00513             // スライドパディングが存在する場合は行単位で memcpy する
00514             for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00515             {
00516                 std::memcpy(dst + y * width * 4, src + y * stride, width * 4);
00517             }
00518         }
00519     }
00520     else if (pixelFormat == libcamera::formats::XRGB8888 || pixelFormat ==
libcamera::formats::RGBX8888)
00521     {
00522         // 4バイト RGBX から 4バイト BGRA へ色順序を入れ替えて展開する
00523         for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00524         {
00525             const uint8_t* rowSrc{ src + y * stride };
00526             uint8_t* rowDst{ dst + y * width * 4 };
00527             for (size_t x = 0; x < static_cast<size_t>(width); ++x)
00528             {
00529                 rowDst[0] = rowSrc[2]; // B
00530                 rowDst[1] = rowSrc[1]; // G
00531                 rowDst[2] = rowSrc[0]; // R
00532                 rowDst[3] = 255;      // A
00533                 rowSrc += 4;
00534                 rowDst += 4;
00535             }
00536         }
00537     }
00538     else if (pixelFormat == libcamera::formats::R8)
00539     {
00540         // 8-bit モノクロ (OV9281 等) の輝度値を BGR 各チャンネルに複製して展開する
00541         for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00542         {
00543             const uint8_t* rowSrc{ src + y * stride };
00544             uint8_t* rowDst{ dst + y * width * 4 };
00545             for (size_t x = 0; x < static_cast<size_t>(width); ++x)

```



```

00546         {
00547             const uint8_t val = *rowSrc++;
00548             rowDst[0] = val; // B
00549             rowDst[1] = val; // G
00550             rowDst[2] = val; // R
00551             rowDst[3] = 255; // A
00552             rowDst += 4;
00553         }
00554     }
00555 }
00556 else if (pixelFormat == libcamera::formats::YUYV)
00557 {
00558     // YUYV 4:2:2 バックアップフォーマットから BGRA へ色空間変換する
00559     for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00560     {
00561         const uint8_t* rowSrc{ src + y * stride };
00562         uint8_t* rowDst{ dst + y * width * 4 };
00563         for (size_t x = 0; x < static_cast<size_t>(width); x += 2)
00564         {
00565             const int y0 = rowSrc[0] - 16;
00566             const int u = rowSrc[1] - 128;
00567             const int y1 = rowSrc[2] - 16;
00568             const int v = rowSrc[3] - 128;
00569             rowSrc += 4;
00570
00571             // 1 画素目の RGB 変換
00572             rowDst[0] = static_cast<uint8_t>(std::clamp((298 * y0 + 516 * u + 128) >> 8, 0, 255));
00573             rowDst[1] = static_cast<uint8_t>(std::clamp((298 * y0 - 100 * u - 208 * v + 128) >> 8, 0,
255));
00574             rowDst[2] = static_cast<uint8_t>(std::clamp((298 * y0 + 409 * v + 128) >> 8, 0, 255));
00575             rowDst[3] = 255;
00576             rowDst += 4;
00577
00578             // 2 画素目の RGB 変換
00579             rowDst[0] = static_cast<uint8_t>(std::clamp((298 * y1 + 516 * u + 128) >> 8, 0, 255));
00580             rowDst[1] = static_cast<uint8_t>(std::clamp((298 * y1 - 100 * u - 208 * v + 128) >> 8, 0,
255));
00581             rowDst[2] = static_cast<uint8_t>(std::clamp((298 * y1 + 409 * v + 128) >> 8, 0, 255));
00582             rowDst[3] = 255;
00583             rowDst += 4;
00584         }
00585     }
00586 }
00587 else if (pixelFormat == libcamera::formats::NV12 || pixelFormat == libcamera::formats::YUV420)
00588 {
00589     // 輝度プレーン (Y) を BGRA の各チャンネルへ複製して展開する
00590     for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00591     {
00592         const uint8_t* rowSrc{ src + y * stride };
00593         uint8_t* rowDst{ dst + y * width * 4 };
00594         for (size_t x = 0; x < static_cast<size_t>(width); ++x)
00595         {
00596             const uint8_t val = *rowSrc++;
00597             rowDst[0] = val; // B
00598             rowDst[1] = val; // G
00599             rowDst[2] = val; // R
00600             rowDst[3] = 255; // A
00601             rowDst += 4;
00602         }
00603     }
00604 }
00605 else
00606 {
00607     // 未知のフォーマットに対するフォールバック (輝度として展開)
00608     for (size_t y = 0; y < static_cast<size_t>(height); ++y)
00609     {
00610         const uint8_t* rowSrc{ src + y * stride };
00611         uint8_t* rowDst{ dst + y * width * 4 };
00612         for (size_t x = 0; x < static_cast<size_t>(width); ++x)
00613         {
00614             const uint8_t val = *rowSrc++;
00615             rowDst[0] = val; // B
00616             rowDst[1] = val; // G
00617             rowDst[2] = val; // R
00618             rowDst[3] = 255; // A
00619             rowDst += 4;
00620         }
00621     }
00622 }
00623
00624 // フレーム取得完了フラグを立てる
00625 captured = true;
00626
00627 // キャプチャフレームレートの実測と診断出力 (1秒ごと)
00628 static auto lastFpsReport{ std::chrono::steady_clock::now() };
00629 static int capturedFrameCount{ 0 };
00630

```

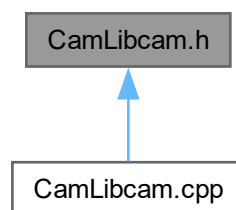
```

00631         ++capturedFrameCount;
00632         const auto now{ std::chrono::steady_clock::now() };
00633         const auto elapsed{ std::chrono::duration<double>(now - lastFpsReport).count() };
00634         if (elapsed >= 1.0)
00635         {
00636             const double currentFps{ capturedFrameCount / elapsed };
00637             std::cout << "libcamera: Capture FPS = " << currentFps << std::endl;
00638             capturedFrameCount = 0;
00639             lastFpsReport = now;
00640         }
00641     }
00642 }
00643
00644 // キャプチャ実行中であればリクエストを再利用してカメラキューに再投入する
00645 if (running)
00646 {
00647     request->reuse(libcamera::Request::ReuseBuffers);
00648     camera->queueRequest(request);
00649 }
00650 }
00651
00652 //
00653 // 現在のピクセルフォーマット名を返す
00654 //
00655 std::string CamLibcam::getPixelFormatName() const
00656 {
00657     // ピクセルフォーマットの文字列表現を返す
00658     return pixelFormat.toString();
00659 }
00660
00661 //
00662 // ビデオキャプチャデバイスの表示名リストを返す
00663 //
00664 const std::vector<std::string>& CamLibcam::getDeviceList()
00665 {
00666     // カメラマネージャからデバイスリストを取得して返す
00667     return Manager::getInstance().getDeviceList();
00668 }
00669
00670 #endif // USE_LIBCAMERA

```

9.27 CamLibcam.h ファイル

被依存関係図:



9.27.1 詳解

libcamera を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

March 2026

[CamLibcam.h](#) に定義があります。

9.28 CamLibcam.h

[詳解]

```

00001 #pragma once
00002
00010
00011 #if defined(USE_LIBCAMERA)
00012
00013 // カメラ関連の基底クラス
00014 #include "Camera.h"
00015
00016 // libcamera C++ API
00017 #include <libcamera/libcamera.h>
00018
00019 // 標準ライブラリ
00020 #include <memory>
00021 #include <vector>
00022 #include <string>
00023 #include <map>
00024 #include <mutex>
00025 #include <condition_variable>
00026
00030 class CamLibcam : public Camera
00031 {
00032     class Manager
00033     {
00034     {
00035         std::unique_ptr<libcamera::CameraManager> cm;
00036         std::vector<std::string> deviceList;
00037         bool started{ false };
00038
00039         Manager();
00040         ~Manager();
00041
00042     public:
00043         Manager(const Manager&) = delete;
00044         Manager& operator=(const Manager&) = delete;
00045
00046         static Manager& getInstance();
00047         libcamera::CameraManager* get() { return cm.get(); }
00048         const std::vector<std::string>& getDeviceList();
00049     };
00050
00052     std::shared_ptr<libcamera::Camera> camera;
00053
00055     std::unique_ptr<libcamera::CameraConfiguration> config;
00056
00058     std::unique_ptr<libcamera::FrameBufferAllocator> allocator;
00059
00061     std::vector<std::unique_ptr<libcamera::Request>> requests;
00062
00064     struct MappedPlane
00065     {
00066         void* address{ nullptr };
00067         size_t length{ 0 };
00068     };
00069
00071     std::map<const libcamera::FrameBuffer*, std::vector<MappedPlane>> mappedBuffers;
00072
00074     libcamera::Stream* stream{ nullptr };
00075
00077     libcamera::PixelFormat pixelFormat;
00078
00080     unsigned int stride{ 0 };
00081
00083     int64_t frameDurationUs{ 33333 };
00084
00086     std::condition_variable cv;
00087     bool frameReady{ false };
00088
00094     void requestComplete(libcamera::Request* request);
00095
00099     void unmapBuffers();

```

```

00100
00101 public:
00102
00111     CamLibcam(int deviceNumber, int initial.width = 0, int initial.height = 0, double initial_fps = 0.0);
00112
00116     virtual ~CamLibcam();
00117
00118 protected:
00119
00125     virtual bool onStart() override;
00126
00130     virtual void onStop() override;
00131
00135     virtual void onClose() override;
00136
00137 public:
00138
00148     bool open(int deviceNumber, int initial.width = 0, int initial.height = 0, double initial_fps = 0.0);
00149
00155     bool isOpened() const
00156     {
00157         return bool(camera);
00158     }
00159
00165     static const std::vector<std::string>& getDeviceList();
00166
00172     std::string getPixelFormatName() const;
00173 };
00174
00175 #endif // USE_LIBCAMERA

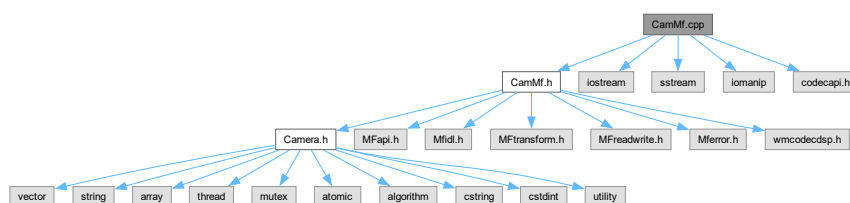
```

9.29 CamMf.cpp ファイル

```

#include "CamMf.h"
#include <iostream>
#include <sstream>
#include <iomanip>
#include <codecapi.h>
CamMf.cpp の依存先関係図:

```



関数

- template<class T>
void [SafeRelease](#) (T **ppT)
- std::string [SubTypeToName](#) (const GUID &subType)

9.29.1 詳解

Microsoft Media Foundation を使ったビデオキャプチャクラスの実装

著者

Kohe Tokoi

日付

December 24, 2024

[CamMf.cpp](#) に定義があります。

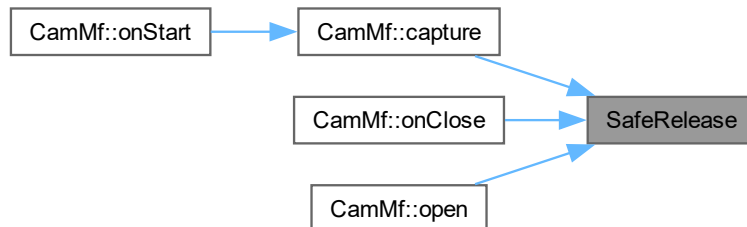
9.29.2 関数詳解

9.29.2.1 SafeRelease()

```
template<class T>
void SafeRelease (
    T ** ppT)
```

[CamMf.cpp](#) の 27 行目に定義があります。

被呼び出し関係図:



9.29.2.2 SubTypeToName()

```
std::string SubTypeToName (
    const GUID & subType)
```

[CamMf.cpp](#) の 169 行目に定義があります。

9.30 CamMf.cpp

[詳解]

```

00001
00008 #include "CamMf.h"
00009 #include <iostream>
00010 #include <sstream>
00011 #include <iomanip>
00012 #include <codecapi.h>
00013
00014 // Microsoft Media Foundation
00015 #pragma comment(lib, "MF.lib")
00016 #pragma comment(lib, "MFplat.lib")
00017 #pragma comment(lib, "MFuuid.lib")
00018 #pragma comment(lib, "MFreadwrite.lib")
00019 #pragma comment(lib, "wmcodecdspuuid.lib")
00020
00021 // COM ライブラリの初期化と終了を行うオブジェクト
00022 CamMf::ComInitializer CamMf::ComInitializer::instance;
00023
00024 //
00025 // メモリの解放
00026 //
00027 template <class T> void SafeRelease(T** ppT)
00028 {
00029     if (*ppT)
00030     {
00031         (*ppT)->Release();
00032         *ppT = nullptr;
00033     }
00034 }
00035
00036 //
00037 // WCHAR (UTF-16) 文字列を UTF-8 の std::string に変換する
00038 //
00039 static std::string WCharToUtf8(const WCHAR* wstr)
00040 {
00041     if (!wstr) return {};
00042     const int sizeNeeded{ WideCharToMultiByte(CP_UTF8, 0, wstr, -1, nullptr, 0, nullptr, nullptr) };
00043     if (sizeNeeded <= 1) return {};
00044     std::string str(sizeNeeded - 1, '\0');
00045     WideCharToMultiByte(CP_UTF8, 0, wstr, -1, str.data(), sizeNeeded, nullptr, nullptr);
00046     return str;
00047 }
00048
00056 static std::string sanitizeDeviceName(const std::string& name)
00057 {
00058     std::string result;
00059     result.reserve(name.size());
00060
00061     const auto len{ name.size() };
00062     for (size_t i = 0; i < len; )
00063     {
00064         const auto c{ static_cast<unsigned char>(name[i]) };
00065
00066         // 1 バイト (ASCII: 0x00 - 0x7F)
00067         if (c <= 0x7F)
00068         {
00069             if (c < 0x20 || c == 0x7F)
00070             {
00071                 result += ' '; // 制御文字は空白に置換
00072             }
00073             else
00074             {
00075                 result += static_cast<char>(c);
00076             }
00077             ++i;
00078         }
00079         // 2 バイト文字 (0xC2 - 0xDF)
00080         else if ((c & 0xE0) == 0xC0)
00081         {
00082             if (i + 1 < len)
00083             {
00084                 result += name.substr(i, 2);
00085                 i += 2;
00086             }
00087             else
00088             {
00089                 break; // 不正なバイト列
00090             }
00091         }
00092         // 3 バイト文字 (0xE0 - 0xEF)
00093         else if ((c & 0xF0) == 0xE0)
00094         {
00095             if (i + 2 < len)

```

```

00096     {
00097         const auto b1{ static_cast<unsigned char>(name[i + 1]) };
00098         const auto b2{ static_cast<unsigned char>(name[i + 2]) };
00099
00100         // 特殊なクォーテーションの正規化
00101         // U+2018 (E2 80 98) -> '
00102         // U+2019 (E2 80 99) -> '
00103         if (c == 0xE2 && b1 == 0x80 && (b2 == 0x98 || b2 == 0x99))
00104         {
00105             result += '\'';
00106         }
00107         // U+201C (E2 80 9C) -> "
00108         // U+201D (E2 80 9D) -> "
00109         else if (c == 0xE2 && b1 == 0x80 && (b2 == 0x9C || b2 == 0x9D))
00110         {
00111             result += '"';
00112         }
00113         else
00114         {
00115             result += name.substr(i, 3);
00116         }
00117         i += 3;
00118     }
00119     else
00120     {
00121         break; // 不正なバイト列
00122     }
00123 }
00124 // 4 バイト文字 (0xF0 - 0xF7: U+10000 以上の絵文字等)
00125 else if ((c & 0xF8) == 0xF0)
00126 {
00127     // フォントに収録されておらず ImGui で表示できないためスキップする
00128     i += (i + 4 <= len) ? 4 : (len - i);
00129 }
00130 else
00131 {
00132     // 不正なバイトはスキップ
00133     ++i;
00134 }
00135 }
00136
00137 // 連続する空白文字を 1 つにまとめ、前後の空白を除去する
00138 std::string trimmed;
00139 trimmed.reserve(result.size());
00140 bool inSpace{ false };
00141 for (char ch : result)
00142 {
00143     if (ch == ' ')
00144     {
00145         if (!inSpace && !trimmed.empty())
00146         {
00147             trimmed += ' ';
00148             inSpace = true;
00149         }
00150     }
00151     else
00152     {
00153         trimmed += ch;
00154         inSpace = false;
00155     }
00156 }
00157 if (!trimmed.empty() && trimmed.back() == ' ')
00158 {
00159     trimmed.pop_back();
00160 }
00161
00162 // 万一すべて除去されて空になった場合は既定の名前を使用する
00163 return trimmed.empty() ? "Camera" : trimmed;
00164 }
00165
00166 //
00167 // GUID から人間が読める形式の名前を返すヘルパー関数
00168 //
00169 std::string SubTypeToName(const GUID& subType)
00170 {
00171     if (subType == MFVideoFormat_YUY2) return "YUY2";
00172     if (subType == MFVideoFormat_NV12) return "NV12";
00173     if (subType == MFVideoFormat_MJPEG) return "MJPG";
00174     if (subType == MFVideoFormat_H264) return "H264";
00175
00176     // TODO: 他に使用するフォーマットがあればここに追加
00177     //if (subType == MFVideoFormat_RGB24) return "RGB24";
00178     if (subType == MFVideoFormat_RGB32) return "RGB32";
00179
00180     return "";
00181 }
00182

```

```

00183 //
00184 // COM ライブラリの初期化と終了を行うクラスのコンストラクタ
00185 //
00186 CamMf::ComInitializer::ComInitializer() = default;
00187
00188 //
00189 // COM ライブラリの初期化と終了を行うクラスのデストラクタ
00190 //
00191 CamMf::ComInitializer::~ComInitializer()
00192 {
00193     // メディアソースのリストを取得していれば
00194     if (ppSourceActivate)
00195     {
00196         // すべてのメディアソースを解放して
00197         for (DWORD i = 0; i < cSourceActivate; ++i) SafeRelease(&ppSourceActivate[i]);
00198
00199         // メディアソースのリストに使ったメモリを解放して
00200         CoTaskMemFree(ppSourceActivate);
00201         ppSourceActivate = nullptr;
00202
00203         // ビデオキャプチャデバイスの表示名のリストを空にする
00204         deviceList.clear();
00205     }
00206
00207     // Media Foundation が起動していればシャットダウンする
00208     if (mfStarted) MFShutdown();
00209
00210     // COM ライブラリが初期化されていれば終了する
00211     if (coInitialized) CoUninitialize();
00212 }
00213
00214 //
00215 // 初期化
00216 //
00217 const char* CamMf::ComInitializer::initialize()
00218 {
00219     // エラーメッセージ
00220     const char* message{ nullptr };
00221
00222     // 検索条件を保持する属性ストア
00223     IMFAttributes* pAttributes{ nullptr };
00224
00225     // COM ライブラリを初期化する
00226     if (FAILED(CoInitializeEx(nullptr, COINIT_MULTITHREADED)))
00227     {
00228         // COM ライブラリの初期化に失敗した
00229         message = "Failed to initialize COM library.";
00230         goto done;
00231     }
00232
00233     // COM ライブラリの初期化に成功した
00234     coInitialized = true;
00235
00236     // Media Foundation を起動する
00237     if (FAILED(MFStartup(MF_VERSION, MFSTARTUP_FULL)))
00238     {
00239         // Media Foundation の起動に失敗した
00240         message = "Failed to start Media Foundation.";
00241         goto done;
00242     }
00243
00244     // Media Foundation の起動に成功した
00245     mfStarted = true;
00246
00247     // 検索条件を保持する属性ストアを作成する
00248     if (FAILED(MFCreateAttributes(&pAttributes, 1)))
00249     {
00250         // 属性ストアの作成失敗した
00251         message = "Failed to create attribute store.";
00252         goto done;
00253     }
00254
00255     // 属性ストアにビデオキャプチャデバイスの属性を設定する
00256     if (FAILED(pAttributes->SetGUID(
00257         MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE,
00258         MF_DEVSOURCE_ATTRIBUTE_SOURCE_TYPE_VIDCAP_GUID)))
00259     {
00260         // ビデオキャプチャデバイスの属性の設定に失敗した
00261         message = "Failed to set attribute for video capture device.";
00262         goto done;
00263     }
00264
00265     // メディアソースを列挙する
00266     if (FAILED(MFEnumDeviceSources(pAttributes,
00267         &ppSourceActivate, &cSourceActivate)))
00268     {
00269         // メディアソースの列挙に失敗した

```



```

00270     message = "Failed to enumerate media sources.";
00271     goto done;
00272 }
00273
00274 // すべてのメディアソースについて
00275 for (DWORD i = 0; i < cSourceActivate; ++i)
00276 {
00277     // メディアソースの表示名の文字列ポインタ
00278     WCHAR* szFriendlyName{ nullptr };
00279
00280     // メディアソースの表示名の文字数
00281     UINT32 cFriendlyName{ 0 };
00282
00283     // メディアソースの表示名を取得する
00284     if (SUCCEEDED(ppSourceActivate[i]->GetAllocatedString(
00285         MF_DEVSOURCE_ATTRIBUTE_FRIENDLY_NAME, &szFriendlyName, &cFriendlyName)))
00286     {
00287         // ビデオキャプチャデバイス名を作る
00288         std::stringstream ss;
00289         ss << sanitizeDeviceName(WCharToUtf8(szFriendlyName)) << "##" << i;
00290
00291         // ビデオキャプチャデバイス名をリストに追加する
00292         deviceList.emplace_back(ss.str());
00293     }
00294
00295     // 表示名の文字列に使ったメモリを解放する
00296     CoTaskMemFree(szFriendlyName);
00297 }
00298
00299 done:
00300
00301 // 属性ストアはもう使わないので解放する
00302 SafeRelease(&pAttributes);
00303
00304 // エラーメッセージを返す
00305 return message;
00306 }
00307
00308 //
00309 // COM ライブラリのシングルトンインスタンスを返す
00310 //
00311 const CamMf::ComInitializer& CamMf::ComInitializer::getInstance()
00312 {
00313     // COM ライブラリが初期化され Media Foundation が起動されていなければ
00314     if (!instance.ppSourceActivate)
00315     {
00316         // COM ライブラリを初期化して Media Foundation を起動する
00317         auto message{ instance.initialize() };
00318
00319         // COM ライブラリの初期化と Media Foundation の起動に失敗したら例外を投げる
00320         if (message) throw std::runtime_error(message);
00321     }
00322
00323     // COM ライブラリのシングルトンインスタンスへの参照を返す
00324     return instance;
00325 }
00326
00327 //
00328 // 有効化
00329 //
00330 bool CamMf::ComInitializer::activate(int device, IMFMediaSource** pMediaSource)
00331 {
00332     // メディアソースを作成して結果を返す
00333     return device >= 0 && static_cast<UINT32>(device) < instance.cSourceActivate
00334         && SUCCEEDED(instance.ppSourceActivate[device]->ActivateObject(IID_PPV_ARGS(pMediaSource)))
00335         && pMediaSource;
00336 }
00337
00338 //
00339 // ビデオキャプチャデバイスの表示名のリストを返す
00340 //
00341 const std::vector<std::string>& CamMf::ComInitializer::getDeviceList()
00342 {
00343     // ビデオキャプチャデバイスの表示名のリストを返す
00344     return getInstance().deviceList;
00345 }
00346
00347 //
00348 // 使用可能な解像度、フレームレート、コーデックのリストを作成する
00349 //
00350 bool CamMf::enumerateFormats()
00351 {
00352     // Source Reader が作成されていなければ戻る
00353     if (!pSourceReader) return false;
00354
00355     // 以前に作成したリストをクリアする
00356     availableFormats.clear();

```

```

00357     formatList.clear();
00358
00359     // ネイティブメディアタイプを一つずつ列挙する
00360     IMFMediaType* pMediaType{ nullptr };
00361     for (DWORD dwMediaTypeIndex = 0;
00362          SUCCEEDED(pSourceReader->GetNativeMediaType(MF_SOURCE_READER_FIRST_VIDEO_STREAM,
00363              dwMediaTypeIndex, &pMediaType));
00364          ++dwMediaTypeIndex
00365      )
00366     {
00367         // スコープを抜けるときにメディアタイプを解放する
00368         struct MediaTypeReleaser { IMFMediaType*& p; ~MediaTypeReleaser() { SafeRelease(&p); } } releaser{
00369             pMediaType };
00370
00371         // メディアタイプを取得する
00372         GUID majorType{};
00373         if (FAILED(pMediaType->GetGUID(MF_MT_MAJOR_TYPE, &majorType))) continue;
00374
00375         // メディアタイプがビデオで無ければ次へ
00376         if (majorType != MFMediaType_Video) continue;
00377
00378         // ビデオフォーマットを取得する
00379         GUID subType{};
00380         if (FAILED(pMediaType->GetGUID(MF_MT_SUBTYPE, &subType))) continue;
00381
00382         // 解像度を取得する
00383         UINT32 width{ 0 }, height{ 0 };
00384         if (FAILED(MFGetAttributeSize(pMediaType, MF_MT_FRAME_SIZE, &width, &height))) continue;
00385
00386         // フレームレートを取得する
00387         UINT32 numerator{ 0 }, denominator{ 0 };
00388         if (FAILED(MFGetAttributeRatio(pMediaType, MF_MT_FRAME_RATE, &numerator, &denominator))) continue;
00389
00390         // コーデックの文字列を取り出す
00391         const auto& codecName{ SubTypeToName(subType) };
00392
00393         // 対応できないコーデックかフレームレートに問題があれば次へ
00394         if (codecName.empty() || denominator == 0 || numerator == 0) continue;
00395
00396         // フレームレートを求める
00397         const double fps{ static_cast<double>(numerator) / static_cast<double>(denominator) };
00398
00399         // フレームレートが 5 未満なら次へ
00400         if (fps < 5.0) continue;
00401
00402         // UI が Media Foundation 固有の型を扱わずに済むよう、解像度と fps を表示文字列に変換する
00403         std::ostringstream resolution;
00404         resolution << width << " x " << height;
00405         std::ostringstream frameRate;
00406         frameRate << std::fixed << std::setprecision(2) << fps;
00407
00408         // availableFormats と同じ並び順の選択番号を関連付けて UI 用リストへ追加する
00409         formatList.push_back({ resolution.str(), frameRate.str(), codecName,
00410             static_cast<int>(availableFormats.size()) });
00411
00412         // 使用可能なビデオフォーマットのリストに追加する
00413         availableFormats.emplace_back(width, height, numerator, denominator, subType);
00414     }
00415
00416     // リストが空でなければ成功
00417     return !formatList.empty();
00418 }
00419
00420 // 指定されたサブタイプに対応するビデオデコーダを探す
00421 //
00422 HRESULT CamMf::findVideoDecoder(
00423     const GUID& subtype,
00424     IMFTransform** ppDecoder,
00425     BOOL bAllowAsync,
00426     BOOL bAllowHardware,
00427     BOOL bAllowTranscode
00428 ) const
00429 {
00430     // 結果
00431     HRESULT hr{ S_OK };
00432
00433     // 見つかったデコーダの数
00434     UINT32 count{ 0 };
00435
00436     // デコーダのアクティベーションオブジェクトのリスト
00437     IMFActivate** ppActivate{ nullptr };
00438
00439     // 検索条件
00440     MFT_REGISTER_TYPE_INFO inputInfo{ MFMediaType_Video, subtype };
00441     MFT_REGISTER_TYPE_INFO outputInfo{ MFMediaType_Video, MFVideoFormat_NV12 };
00442

```

```

00443 // 列挙のフラグ
00444 UINT32 unFlags
00445 {
00446     MFT_ENUM_FLAG_SYNCMFT
00447     | MFT_ENUM_FLAG_LOCALMFT
00448     | MFT_ENUM_FLAG_SORTANDFILTER
00449 };
00450
00451 // 非同期デコードも含めるなら
00452 if (bAllowAsync) unFlags |= MFT_ENUM_FLAG_ASYNCMFT;
00453
00454 // ハードウェアデコードも含めるなら
00455 if (bAllowHardware) unFlags |= MFT_ENUM_FLAG_HARDWARE;
00456
00457 // トランスコード専用のデコードも含めるなら
00458 if (bAllowTranscode) unFlags |= MFT_ENUM_FLAG_TRANSCODE_ONLY;
00459
00460 // 条件に合う MFT のアクティベーションオブジェクトを列挙する
00461 hr = MFTEnumEx(MFT_CATEGORY_VIDEO_DECODER,
00462     unFlags, &inputInfo, &outputInfo, &ppActivate, &count);
00463
00464 // デコーダが見つからなかったらエラー
00465 if (SUCCEEDED(hr) && count == 0) hr = MF_E_TOPO_CODEC_NOT_FOUND;
00466
00467 // リストの最初のデコーダをインスタンス化する
00468 if (SUCCEEDED(hr)) hr = ppActivate[0]->ActivateObject(IID_PPV_ARGS(ppDecoder));
00469
00470 // デコーダのアクティベーションオブジェクトのリストを解放する
00471 for (UINT32 i = 0; i < count; i++) ppActivate[i]->Release();
00472
00473 // デコーダのアクティベーションオブジェクトのリストに使ったメモリを解放する
00474 CoTaskMemFree(ppActivate);
00475
00476 // 結果を返す
00477 return hr;
00478 }
00479
00480 //
00481 // MFT のセットアップと接続を行う
00482 //
00483 HRESULT CamMf::setUpPipeline(IMFTransform* pTransform, const VideoFormat& format, const GUID& subType)
00484 {
00485     #if defined(_DEBUG)
00486         std::cerr << SubTypeToName(format.subType) << " -> " << SubTypeToName(subType) << std::endl;
00487     #endif
00488
00489     // 入力と出力のメディアタイプ
00490     IMFMediaType* pInputType{ nullptr };
00491     IMFMediaType* pOutputType{ nullptr };
00492
00493     // 結果
00494     HRESULT hr{ S_OK };
00495
00496     // MFT の入力タイプの設定
00497     hr = MFCreateMediaType(&pInputType);
00498     if (SUCCEEDED(hr)) hr = pInputType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00499     if (SUCCEEDED(hr)) hr = pInputType->SetGUID(MF_MT_SUBTYPE, format.subType);
00500     if (SUCCEEDED(hr)) hr = MFSetAttributeSize(pInputType, MF_MT_FRAME_SIZE, format.width,
00501         format.height);
00502     if (SUCCEEDED(hr)) hr = MFSetAttributeRatio(pInputType, MF_MT_FRAME_RATE, format.fpsNum,
00503         format.fpsDenom);
00504
00505     // MFT に入力タイプを設定する
00506     if (SUCCEEDED(hr)) hr = pTransform->SetInputType(0, pInputType, 0);
00507     if (FAILED(hr)) goto done;
00508
00509     // MFT の出力タイプの設定
00510     hr = MFCreateMediaType(&pOutputType);
00511     if (SUCCEEDED(hr)) hr = pOutputType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00512     if (SUCCEEDED(hr)) hr = pOutputType->SetGUID(MF_MT_SUBTYPE, subType);
00513     if (SUCCEEDED(hr)) hr = MFSetAttributeSize(pOutputType, MF_MT_FRAME_SIZE, format.width,
00514         format.height);
00515     if (SUCCEEDED(hr)) hr = MFSetAttributeRatio(pOutputType, MF_MT_FRAME_RATE, format.fpsNum,
00516         format.fpsDenom);
00517
00518     // MFT に出力タイプを設定する
00519     if (SUCCEEDED(hr)) hr = pTransform->SetOutputType(0, pOutputType, 0);
00520     if (FAILED(hr)) goto done;
00521
00522     // MFT をアクティブにする
00523     hr = pTransform->ProcessMessage(MFT_MESSAGE_COMMAND_FLUSH, NULL);
00524     if (SUCCEEDED(hr)) hr = pTransform->ProcessMessage(MFT_MESSAGE_NOTIFY_START_OF_STREAM, NULL);
00525     if (SUCCEEDED(hr)) hr = pTransform->ProcessMessage(MFT_MESSAGE_NOTIFY_BEGIN_STREAMING, NULL);
00526
00527 done:
00528

```

```

00525 // メディアタイプの取得に使ったメモリを解放する
00526 SafeRelease(&pInputType);
00527 SafeRelease(&pOutputType);
00528
00529 // 結果を返す
00530 return hr;
00531 }
00532
00533 //
00534 // MFT を解放する
00535 //
00536 void CamMf::cleanUpTransform(IMFTransform** pTransform) const
00537 {
00538     // MFT が作成されていれば
00539     if (*pTransform)
00540     {
00541         // MFT のストリーミングの終了を通知する
00542         (*pTransform)->ProcessMessage(MFT_MESSAGE_NOTIFY_END_STREAMING, NULL);
00543         (*pTransform)->ProcessMessage(MFT_MESSAGE_NOTIFY_END_OF_STREAM, NULL);
00544
00545         // MFT を解放する
00546         (*pTransform)->Release();
00547         *pTransform = nullptr;
00548     }
00549 }
00550
00551 //
00552 // デコーダの出力バッファを作成する
00553 //
00554 HRESULT CamMf::createDecoderBuffer()
00555 {
00556     // デコーダの出力ストリーム情報を取得する
00557     MFT_OUTPUT_STREAM_INFO streamInfo{ 0 };
00558     if (SUCCEEDED(pDecoder->GetOutputStreamInfo(0, &streamInfo))) {}
00559
00560     // NV12 は通常 16 ピクセル単位に切り上げられたサイズなので幅を 16 の倍数に丸める
00561     const auto alignedWidth{ static_cast<UINT32>((width + 15) & ~15) };
00562     const auto alignedHeight{ static_cast<UINT32>((height + 15) & ~15) };
00563
00564     // NV12 フォーマットの理論上の必要バッファサイズを計算する
00565     const auto cbDecoderCalc{ static_cast<UINT32>((alignedWidth * alignedHeight * 3) / 2) };
00566
00567     // MFT が要求するサイズと理論計算値の大きい方を採用する
00568     const auto cbDecoder{ static_cast<UINT32>((streamInfo.cbSize > cbDecoderCalc) ? streamInfo.cbSize :
cbDecoderCalc) };
00569
00570     // MFT が要求するメモリアラインメントを取得する
00571     const auto alignmentDecoder{ static_cast<DWORD>((streamInfo.cbAlignment > 0) ?
(streamInfo.cbAlignment - 1) : 63) };
00572
00573     // 現在の出力フォーマット用に確保されている出力バッファがあれば解放する
00574     SafeRelease(&pDecoderBuffer);
00575
00576     // デコーダの出力フレームを書き込むためのアラインメント付きメモリバッファを作成する
00577     return MFCCreateAlignedMemoryBuffer(cbDecoder, alignmentDecoder, &pDecoderBuffer);
00578 }
00579
00580 //
00581 // カラーコンバータの出力バッファを作成する
00582 //
00583 HRESULT CamMf::createConverterBuffer()
00584 {
00585     // カラーコンバータの出力ストリーム情報を取得する
00586     MFT_OUTPUT_STREAM_INFO convStreamInfo{ 0 };
00587     if (SUCCEEDED(pConverter->GetOutputStreamInfo(0, &convStreamInfo))) {}
00588
00589     // RGB32 フォーマットの理論上の必要バッファサイズを計算する
00590     UINT32 cbConverterCalc{ 0 };
00591     MFCalculateImageSize(MFVideoFormat_RGB32, width, height, &cbConverterCalc);
00592
00593     // MFT が要求するサイズと理論計算値の大きい方を採用する
00594     UINT32 cbConverter = (convStreamInfo.cbSize > cbConverterCalc) ? convStreamInfo.cbSize :
cbConverterCalc;
00595
00596     // MFT が要求するメモリアラインメントを取得する
00597     DWORD alignmentConverter = (convStreamInfo.cbAlignment > 0) ? (convStreamInfo.cbAlignment - 1) : 63;
00598
00599     // 既存の出力バッファがあれば解放する
00600     SafeRelease(&pConverterBuffer);
00601
00602     // カラーコンバータが要求するサイズとアラインメントを満たす RGB32 用の出力バッファを作成する
00603     return MFCCreateAlignedMemoryBuffer(cbConverter, alignmentConverter, &pConverterBuffer);
00604 }
00605
00606 //
00607 // Source Reader の出力フォーマットを設定し、基底クラスの frame を初期化する
00608 //

```

```

00609 bool CamMf::setFormat(int index)
00610 {
00611     // 使用可能なフォーマットのリストから選択されたフォーマットを取得する
00612     VideoFormat selectedFormat{ availableFormats[index] };
00613
00614     // デコード方法
00615     enum class DecodeMethod { None = 0, Convert, Decode } decodeMethod
00616     {
00617         selectedFormat.subType == MFVideoFormat_MJPG ? DecodeMethod::Decode :
00618         selectedFormat.subType == MFVideoFormat_H264 ? DecodeMethod::Decode :
00619         selectedFormat.subType == MFVideoFormat_NV12 ? DecodeMethod::Convert :
00620         selectedFormat.subType == MFVideoFormat_YUY2 ? DecodeMethod::Convert :
00621         DecodeMethod::None
00622     };
00623
00624     // デコーダとカラーコンバータを未設定にする
00625     cleanUpTransform(&pDecoder);
00626     cleanUpTransform(&pConverter);
00627
00628     // 結果
00629     HRESULT hr{ S_OK };
00630
00631     // 新しいメディアタイプを作成する
00632     IMFMediaType* pMediaType{ nullptr };
00633     hr = MFCreateMediaType(&pMediaType);
00634     if (FAILED(hr)) goto done;
00635
00636     // メジャータイプにビデオを指定する
00637     hr = pMediaType->SetGUID(MF_MT_MAJOR_TYPE, MFMediaType_Video);
00638     if (FAILED(hr)) goto done;
00639
00640     // ビデオフォーマットを指定する
00641     hr = pMediaType->SetGUID(MF_MT_SUBTYPE, selectedFormat.subType);
00642     if (FAILED(hr)) goto done;
00643
00644     // 解像度を設定する
00645     hr = MFSetAttributeSize(pMediaType, MF_MT_FRAME_SIZE, selectedFormat.width, selectedFormat.height);
00646     if (FAILED(hr)) goto done;
00647
00648     // フレームレートを設定する
00649     hr = MFSetAttributeRatio(pMediaType, MF_MT_FRAME_RATE, selectedFormat.fpsNum,
selectedFormat.fpsDenom);
00650     if (FAILED(hr)) goto done;
00651
00652     // Source Reader の出力メディアタイプを設定する
00653     hr = pSourceReader->SetCurrentMediaType(MF_SOURCE_READER_FIRST_VIDEO_STREAM, nullptr, pMediaType);
00654     if (FAILED(hr)) goto done;
00655
00656     // 基底クラスの解像度とチャンネル数を設定し、バッファサイズを設定する
00657     width = selectedFormat.width;
00658     height = selectedFormat.height;
00659     channels = 4;
00660     image.resize(width * height * channels);
00661
00662     // インターバルを計算する
00663     interval = (selectedFormat.fpsDenom != 0)
00664         ? (1000.0 * selectedFormat.fpsDenom / selectedFormat.fpsNum)
00665         : 10.0;
00666
00667     // デコードまたはカラー変換が必要なとき
00668     if (decodeMethod != DecodeMethod::None)
00669     {
00670         // デコードが必要ななら
00671         if (decodeMethod == DecodeMethod::Decode)
00672         {
00673             // 選択されたビデオフォーマットのデコーダを探す
00674             hr = findVideoDecoder(selectedFormat.subType, &pDecoder);
00675             if (FAILED(hr)) goto done;
00676
00677             // デコーダから ICodecAPI インターフェイスの取得に成功したら
00678             ICodecAPI* pCodecAPI{ nullptr };
00679             if (SUCCEEDED(pDecoder->QueryInterface(IID_PPV_ARGS(&pCodecAPI))))
00680             {
00681                 // コーデック設定値を渡すための VARIANT を初期化する
00682                 VARIANT var;
00683                 VariantInit(&var);
00684
00685                 // Low Latency Mode を VT_UI4 型で有効にする
00686                 var.vt = VT_UI4;
00687
00688                 // 1 = Low Latency Mode
00689                 var.ulVal = 1;
00690
00691                 // デコーダに Low Latency Mode を要求する
00692                 hr = pCodecAPI->SetValue(&CODECAPI_AVLowLatencyMode, &var);
00693
00694                 // VT_UI4 型で Low Latency Mode を有効にできなかったら

```

```

00695         if (FAILED(hr))
00696         {
00697             // Low Latency Mode を VT_BOOL 型で有効にする
00698             var.vt = VT_BOOL;
00699
00700             // VARIANT_TRUE = Low Latency Mode
00701             var.boolVal = VARIANT_TRUE;
00702
00703             // デコーダに Low Latency Mode を要求する
00704             pCodecAPI->SetValue(&CODECAPI_AVLowLatencyMode, &var);
00705         }
00706
00707         // VARIANT が保持しているリソースを解放する38
00708         VariantClear(&var);
00709
00710         // ICodecAPI インターフェイスを解放する
00711         SafeRelease(&pCodecAPI);
00712     }
00713
00714     // MFT デコーダのセットアップと接続を行う
00715     hr = setUpPipeline(pDecoder, selectedFormat, MFVideoFormat_NV12);
00716     if (FAILED(hr)) goto done;
00717
00718     // デコード後のビデオフォーマットは NV12 にしている
00719     selectedFormat.subType = MFVideoFormat_NV12;
00720
00721     // デコーダの出力バッファを作成する
00722     hr = createDecoderBuffer();
00723     if (FAILED(hr)) goto done;
00724 }
00725
00726 // MFT をインスタンス化する
00727 hr = CoCreateInstance(CLSID_CColorConvertDMO, nullptr, CLSCTX_INPROC_SERVER,
IID_PPV_ARGS(&pConverter));
00728 if (FAILED(hr)) goto done;
00729
00730 // MFT カラーコンバータのセットアップと接続を行う
00731 hr = setUpPipeline(pConverter, selectedFormat, MFVideoFormat_RGB32);
00732 if (FAILED(hr)) goto done;
00733
00734 // カラー変換用の出力バッファを作成する
00735 hr = createConverterBuffer();
00736 if (FAILED(hr)) goto done;
00737 }
00738
00739 done:
00740
00741 // メディアタイプはもう使わないので解放する
00742 SafeRelease(&pMediaType);
00743
00744 // 成功したら true を返す
00745 if (SUCCEEDED(hr)) return true;
00746
00747 // 失敗したときはカメラを閉じて
00748 close();
00749
00750 // false を返す
00751 return false;
00752 }
00753
00754 //
00755 // カメラを開く
00756 //
00757 bool CamMf::open(int device, bool setupFormat)
00758 {
00759     // メディアソースを作成する
00760     if (!ComInitializer::activate(device, &pMediaSource)) return false;
00761
00762     // 結果
00763     HRESULT hr{ S_OK };
00764
00765     // Source Reader の属性ストアを作成する
00766     IMFAttributes* pAttributes{ nullptr };
00767     hr = MFCreateAttributes(&pAttributes, 1);
00768     if (FAILED(hr)) goto done;
00769
00770     // Source Reader の属性ストアにデコード能力を設定する
00771     pAttributes->SetUINT32(MF_READWRITE_ENABLE_HARDWARE_TRANSFORMS, TRUE);
00772     pAttributes->SetUINT32(MF_READWRITE_DISABLE_CONVERTERS, FALSE);
00773     pAttributes->SetUINT32(MF_SOURCE_READER_ENABLE_VIDEO_PROCESSING, FALSE);
00774
00775     // Source Reader に低遅延モードを要求する
00776     pAttributes->SetUINT32(MF_LOW_LATENCY, TRUE);
00777
00778     // Source Reader の解放時に Media Source をシャットダウンするようにする
00779     pAttributes->SetUINT32(MF_SOURCE_READER_DISCONNECT_MEDIASOURCE_ON_SHUTDOWN, TRUE);
00780

```

```

00781 // Source Reader を作成する
00782 hr = MFCreateSourceReaderFromMediaSource(pMediaSource, pAttributes, &pSourceReader);
00783 if (FAILED(hr)) goto done;
00784
00785 // 属性ストアはもう必要ないので解放する
00786 SafeRelease(&pAttributes);
00787
00788 // 使用可能なフォーマットを列挙して最初のフォーマットを選択する
00789 if (enumerateFormats())
00790 {
00791 // 要求された場合のみ最初のフォーマットを設定する
00792 if (!setupFormat || setFormat(0)) return true;
00793 }
00794
00795 done:
00796
00797 // フォーマットの設定に失敗したら全てを解放して戻る
00798 SafeRelease(&pSourceReader);
00799 SafeRelease(&pAttributes);
00800 availableFormats.clear();
00801 formatList.clear();
00802
00803 return false;
00804 }
00805
00806 //
00807 // フォーマットを選択して設定する
00808 //
00809 bool CamMf::select(int index)
00810 {
00811 // カメラが開かれていないかインデックスが範囲外なら戻る
00812 if (!pSourceReader || index < 0 || index >= availableFormats.size()) return false;
00813
00814 // キャプチャスレッドが実行中であれば安全のために停止する
00815 stop();
00816
00817 // 選択されたフォーマットを設定する
00818 return setFormat(index);
00819 }
00820
00821 //
00822 // フレームをキャプチャする
00823 //
00824 void CamMf::capture()
00825 {
00826 // スレッドが実行可の間
00827 while (running)
00828 {
00829 // サンプルを読み出して
00830 DWORD dwStreamIndex{ 0 };
00831 DWORD dwStreamFlags{ 0 };
00832 LONGLONG llTimestamp{ 0 };
00833 IMFSample* pSample{ nullptr };
00834 if (FAILED(pSourceReader->ReadSample(MF_SOURCE_READER_FIRST_VIDEO_STREAM,
00835 0, &dwStreamIndex, &dwStreamFlags, &llTimestamp, &pSample))) continue;
00836
00837 // サンプルが取得できていなければ戻る
00838 if (!pSample) continue;
00839
00840 // サンプルから取り出したメディアバッファ
00841 IMFMediaBuffer* pBuffer{ nullptr };
00842
00843 // デコーダの出力フレームを保持するサンプル
00844 IMFSample* pDecodedSample{ nullptr };
00845
00846 // カラーコンバータの出力フレームを保持するサンプル
00847 IMFSample* pConvertedSample{ nullptr };
00848
00849 // ProcessOutput の呼び出しの状態
00850 DWORD dwStatus{ 0 };
00851
00852 // デコーダが設定されていれば (MJPG か H264 の場合)
00853 if (pDecoder)
00854 {
00855 // サンプルをデコーダに渡す
00856 HRESULT hr{ pDecoder->ProcessInput(0, pSample, 0) };
00857
00858 // デコーダへの入力に失敗したら
00859 if (FAILED(hr))
00860 {
00861 #if defined(_DEBUG)
00862 if (hr != MF_E_NOTACCEPTING)
00863 {
00864 std::cerr << "Decoder process input failed: ";
00865 switch (hr)
00866 {
00867 case E_INVALIDARG:

```

```

00868         std::cerr << "Invalid argument." << std::endl; break;
00869     case E_UNEXPECTED:
00870         std::cerr << "Unexpected error." << std::endl; break;
00871     case E_FAIL:
00872         std::cerr << "Unspecified error." << std::endl; break;
00873     case MF_E_INVALIDSTREAMNUMBER:
00874         std::cerr << "Invalid stream number." << std::endl; break;
00875     case MF_E_NO_SAMPLE_DURATION:
00876         std::cerr << "No sample duration." << std::endl; break;
00877     case MF_E_NO_SAMPLE_TIMESTAMP:
00878         std::cerr << "No sample timestamp." << std::endl; break;
00879     case MF_E_NOTACCEPTING:
00880         std::cerr << "Not accepting input." << std::endl; break;
00881     case MF_E_TRANSFORM_TYPE_NOT_SET:
00882         std::cerr << "Transform type not set." << std::endl; break;
00883     case MF_E_UNSUPPORTED_D3D_TYPE:
00884         std::cerr << "Unsupported D3D type." << std::endl; break;
00885     default:
00886         std::cerr << "Code: " << std::hex << hr << std::endl; break;
00887     }
00888 }
00889 #endif
00890 goto done;
00891 }
00892
00893 // デコーダの出力ストリーム情報を取得し、誰かサンプルを提供するべきかを判定する
00894 MFT_OUTPUT_STREAM_INFO streamInfo{};
00895 bool mftProvidesSamples{ false };
00896 if (SUCCEEDED(pDecoder->GetOutputStreamInfo(0, &streamInfo)))
00897 {
00898     mftProvidesSamples = (streamInfo.dwFlags & MFT_OUTPUT_STREAM_PROVIDES_SAMPLES) != 0;
00899 }
00900
00901 // デコーダから出力サンプルを取り出す
00902 IMFSample* pLatestDecodedSample{ nullptr };
00903 bool hasOutput{ false };
00904
00905 while (true)
00906 {
00907     DWORD prevLength = 0;
00908
00909     if (mftProvidesSamples)
00910     {
00911         // MFT が自前でサンプルを割り当てるので nullptr を渡す
00912         pDecodedSample = nullptr;
00913     }
00914     else
00915     {
00916         // 呼び出し側がサンプルを提供する
00917         hr = MFCreateSample(&pDecodedSample);
00918         if (FAILED(hr)) goto done;
00919
00920         // クラスメンバの pDecoderBuffer が有効であることを確認
00921         if (!pDecoderBuffer)
00922         {
00923             hr = createDecoderBuffer();
00924             if (FAILED(hr))
00925             {
00926                 SafeRelease(&pDecodedSample);
00927                 goto done;
00928             }
00929         }
00930
00931         // バッファの有効データ長を 0 に設定して、MFT が先頭から書き込めるようにする
00932         pDecoderBuffer->GetCurrentLength(&prevLength);
00933         pDecoderBuffer->SetCurrentLength(0);
00934
00935         hr = pDecodedSample->AddBuffer(pDecoderBuffer);
00936         if (FAILED(hr))
00937         {
00938             SafeRelease(&pDecodedSample);
00939             goto done;
00940         }
00941     }
00942
00943     MFT_OUTPUT_DATA_BUFFER decodedBuffer{ 0, pDecodedSample, 0, nullptr };
00944     hr = pDecoder->ProcessOutput(0, 1, &decodedBuffer, &dwStatus);
00945
00946     // ストリームのフォーマットが変化した場合の処理
00947     if (hr == MF_E_TRANSFORM_STREAM_CHANGE)
00948     {
00949         #if defined(_DEBUG)
00950             std::cerr << "Transform stream change." << std::endl;
00951         #endif
00952
00953         // 1. 利用可能な出力タイプから NV12 を探索・選択する
00954         IMFMediaType* pNewOutputType{ nullptr };
00955         GUID subtype{ 0 };

```



```

00955         DWORD dwTypeIndex = 0;
00956         bool foundNV12 = false;
00957         hr = S_OK; // hr を S_OK にリセット
00958         while (SUCCEEDED(pDecoder->GetOutputAvailableType(0, dwTypeIndex++, &pNewOutputType))
00959             {
00960             GUID subType{};
00961             if (SUCCEEDED(pNewOutputType->GetGUID(MF_MT_SUBTYPE, &subType)) && subType ==
MFVideoFormat_NV12)
00962             {
00963                 foundNV12 = true;
00964                 subtype = subType;
00965                 #if defined(_DEBUG)
00966                 std::cerr << "Found NV12 output type at index " << (dwTypeIndex - 1) << std::endl;
00967                 #endif
00968                 break;
00969             }
00970             SafeRelease(&pNewOutputType);
00971         }
00972
00973         // 見つからなかった場合はインデックス 0 を取得してサブタイプを NV12 に書きする
00974         if (!foundNV12)
00975         {
00976             #if defined(_DEBUG)
00977             std::cerr << "NV12 output type not found. Forcing fallback to index 0." << std::endl;
00978             #endif
00979             hr = pDecoder->GetOutputAvailableType(0, 0, &pNewOutputType);
00980             if (SUCCEEDED(hr))
00981             {
00982                 hr = pNewOutputType->SetGUID(MF_MT_SUBTYPE, MFVideoFormat_NV12);
00983                 subtype = MFVideoFormat_NV12;
00984             }
00985         }
00986
00987         // 2. 現在の入力メディアタイプから解像度などの属性をコピーする
00988         if (SUCCEEDED(hr) && pNewOutputType)
00989         {
00990             IMFMediaType* pInputType{ nullptr };
00991             if (SUCCEEDED(pDecoder->GetInputCurrentType(0, &pInputType)))
00992             {
00993                 UINT32 w{ 0 }, h{ 0 };
00994                 if (SUCCEEDED(MFGetAttributeSize(pInputType, MF_MT_FRAME_SIZE, &w, &h)))
00995                 {
00996                     MFSetAttributeSize(pNewOutputType, MF_MT_FRAME_SIZE, w, h);
00997                 }
00998
00999                 UINT32 fpsNum{ 0 }, fpsDenom{ 0 };
01000                 if (SUCCEEDED(MFGetAttributeRatio(pInputType, MF_MT_FRAME_RATE, &fpsNum, &fpsDenom)))
01001                 {
01002                     MFSetAttributeRatio(pNewOutputType, MF_MT_FRAME_RATE, fpsNum, fpsDenom);
01003                 }
01004
01005                 UINT32 aspectNum{ 0 }, aspectDenom{ 0 };
01006                 if (SUCCEEDED(MFGetAttributeRatio(pInputType, MF_MT_PIXEL_ASPECT_RATIO, &aspectNum,
&aspectDenom)))
01007                 {
01008                     MFSetAttributeRatio(pNewOutputType, MF_MT_PIXEL_ASPECT_RATIO, aspectNum, aspectDenom);
01009                 }
01010
01011                 PROPVARIANT var;
01012                 PropVariantInit(&var);
01013                 if (SUCCEEDED(pInputType->GetItem(MF_MT_INTERLACE_MODE, &var)))
01014                 {
01015                     if (var.vt == VT_UI4) pNewOutputType->SetUINT32(MF_MT_INTERLACE_MODE, var.ulVal);
01016                     PropVariantClear(&var);
01017                 }
01018                 SafeRelease(&pInputType);
01019             }
01020
01021             // 新しい出力メディアタイプを設定する
01022             hr = pDecoder->SetOutputType(0, pNewOutputType, 0);
01023             #if defined(_DEBUG)
01024             if (FAILED(hr)) std::cerr << "pDecoder->SetOutputType failed: " << std::hex << hr <<
std::endl;
01025             else std::cerr << "pDecoder->SetOutputType succeeded." << std::endl;
01026             #endif
01027         }
01028         else if (SUCCEEDED(hr))
01029         {
01030             hr = E_FAIL;
01031             #if defined(_DEBUG)
01032             std::cerr << "No output type available to set!" << std::endl;
01033             #endif
01034         }
01035     }
01036
01037     // 3. 設定された出力タイプから属性を取得してバッファとカラーコンバータを更新する
01038     if (SUCCEEDED(hr) && pNewOutputType)

```

```

01039         {
01040             UINT32 newWidth{ 0 }, newHeight{ 0 };
01041             if (SUCCEEDED(MFGetAttributeSize(pNewOutputType, MF_MT_FRAME_SIZE, &newWidth, &newHeight))
&& newWidth > 0 && newHeight > 0)
01042             {
01043                 width = newWidth;
01044                 height = newHeight;
01045             }
01046
01047             UINT32 fpsNum{ 0 }, fpsDenom{ 0 };
01048             MFGetAttributeRatio(pNewOutputType, MF_MT_FRAME_RATE, &fpsNum, &fpsDenom);
01049
01050             // デコーダの出力バッファ要件の取得と再作成
01051             hr = createDecoderBuffer();
01052
01053             // カラーコンバータの再設定
01054             if (SUCCEEDED(hr) && pConverter)
01055             {
01056                 VideoFormat newFormat{ static_cast<UINT32>(width), static_cast<UINT32>(height), fpsNum,
fpsDenom, subtype };
01057
01058                 // カラーコンバータのセットアップ
01059                 hr = setUpPipeline(pConverter, newFormat, MFVideoFormat_RGB32);
01060
01061                 if (SUCCEEDED(hr))
01062                 {
01063                     hr = createConverterBuffer();
01064                 }
01065             }
01066
01067             // 基底クラスのバッファリサイズ
01068             if (SUCCEEDED(hr))
01069             {
01070                 image.resize(width * height * channels);
01071             }
01072         }
01073
01074         // 出力サンプルとイベントを破棄する
01075         if (!mftProvidesSamples) SafeRelease(&pDecodedSample);
01076         else SafeRelease(&decodedBuffer.pSample);
01077         SafeRelease(&decodedBuffer.pEvents);
01078
01079         // リソースの解放
01080         SafeRelease(&pNewOutputType);
01081
01082         if (FAILED(hr))
01083         {
01084             #if defined(_DEBUG)
01085                 std::cerr << "Failed to handle stream change: " << std::hex << hr << std::endl;
01086             #endif
01087             goto done;
01088         }
01089
01090         // ストリーム情報を再取得する (ストリームチェンジによってフラグが変わる可能性があるため)
01091         if (SUCCEEDED(pDecoder->GetOutputStreamInfo(0, &streamInfo)))
01092         {
01093             mftProvidesSamples = (streamInfo.dwFlags & MFT_OUTPUT_STREAM_PROVIDES_SAMPLES) != 0;
01094             #if defined(_DEBUG)
01095                 std::cerr << "After stream change, mftProvidesSamples = " << (mftProvidesSamples ? "true"
: "false") << ", cbSize = " << streamInfo.cbSize << std::endl;
01096             #endif
01097         }
01098
01099         #if defined(_DEBUG)
01100             std::cerr << "Stream change handled successfully, retrying ProcessOutput." << std::endl;
01101         #endif
01102
01103         // ストリーム変更を行ったので、今回の ProcessOutput はやり直す
01104         if (!mftProvidesSamples && pDecoderBuffer)
01105         {
01106             pDecoderBuffer->SetCurrentLength(prevLength);
01107             continue;
01108         }
01109
01110         // デコーダがさらなる入力を要求している場合 (正常動作)
01111         if (hr == MF_E_TRANSFORM_NEED_MORE_INPUT)
01112         {
01113             if (!mftProvidesSamples)
01114             {
01115                 if (pDecoderBuffer) pDecoderBuffer->SetCurrentLength(prevLength);
01116                 SafeRelease(&pDecodedSample);
01117             }
01118             else
01119             {
01120                 SafeRelease(&decodedBuffer.pSample);
01121             }
01122             break;

```

```

01123     }
01124
01125     // その他のエラー
01126     if (FAILED(hr))
01127     {
01128     #if defined(_DEBUG)
01129         std::cerr << "Decoder process output failed (loop bottom): " << std::hex << hr << std::endl;
01130     #endif
01131         if (!mftProvidesSamples)
01132         {
01133             if (pDecoderBuffer) pDecoderBuffer->SetCurrentLength(prevLength);
01134             SafeRelease(&pDecodedSample);
01135         }
01136         else
01137         {
01138             SafeRelease(&decodedBuffer.pSample);
01139         }
01140         SafeRelease(&decodedBuffer.pEvents);
01141         goto done;
01142     }
01143
01144     // デコード成功!
01145     SafeRelease(&pLatestDecodedSample);
01146     if (mftProvidesSamples)
01147     {
01148         pLatestDecodedSample = decodedBuffer.pSample;
01149     }
01150     else
01151     {
01152         pLatestDecodedSample = pDecodedSample;
01153     }
01154     pDecodedSample = nullptr;
01155     SafeRelease(&decodedBuffer.pEvents);
01156     hasOutput = true;
01157
01158     // レイテンシ優先なら
01159     if (prioritizeLatency)
01160     {
01161         // 最新を取得し続けるためループを継続する
01162         continue;
01163     }
01164     else
01165     {
01166         // 全フレーム処理モードならループを抜ける
01167         break;
01168     }
01169 }
01170
01171 if (hasOutput && pLatestDecodedSample)
01172 {
01173     // 元のサンプルはもう使わないので解放する
01174     pSample->Release();
01175
01176     // デコードに成功したらデコードされたサンプルを使う
01177     pSample = pLatestDecodedSample;
01178     pLatestDecodedSample = nullptr;
01179 }
01180 else
01181 {
01182     // 今回の入力ではデコード完了フレームが得られなかったため、
01183     // 入力サンプルを解放して次のフレームの読み込みに進む
01184     pSample->Release();
01185     pSample = nullptr;
01186     continue;
01187 }
01188 }
01189
01190 // カラーコンバータが設定されていれば (NV12 か YUY2 か MJPG か H264 の場合)
01191 if (pConverter)
01192 {
01193     // サンプルをカラーコンバータに渡す
01194     HRESULT hr{ pConverter->ProcessInput(0, pSample, 0) };
01195     if (FAILED(hr)) goto done;
01196
01197     // カラーコンバータの出力サンプルを作成する
01198     hr = MFCreateSample(&pConvertedSample);
01199     if (FAILED(hr)) goto done;
01200
01201     // バッファの長さを 0 に設定して書き込み可能にする
01202     pConverterBuffer->SetCurrentLength(0);
01203
01204     // カラーコンバータの出力サンプルにバッファを追加する
01205     hr = pConvertedSample->AddBuffer(pConverterBuffer);
01206     if (FAILED(hr)) goto done;
01207
01208     // カラーコンバータの出力バッファ
01209     MFT_OUTPUT_DATA_BUFFER convertedBuffer{ 0, pConvertedSample, 0, nullptr };

```

```

01210
01211 // カラー変換処理を実行
01212 hr = pConverter->ProcessOutput(0, 1, &convertedBuffer, &dwStatus);
01213 if (FAILED(hr)) goto done;
01214
01215 // 元のサンプルはもう使わないので解放する
01216 pSample->Release();
01217
01218 // コンバートに成功したらコンバートされたサンプルを使う
01219 pSample = pConvertedSample;
01220 pConvertedSample = nullptr;
01221 }
01222
01223 // サンプルからメディアバッファを取得して
01224 if (FAILED(pSample->GetBufferByIndex(0, &pBuffer))) goto done;
01225
01226 // メディアバッファが取得できていれば
01227 if (pBuffer)
01228 {
01229     // メディアバッファから取り出したデータ
01230     BYTE* pData{ nullptr };
01231
01232     // メディアバッファから取り出したデータの長さ
01233     DWORD cbDataLength{ 0 };
01234
01235     // メディアバッファからフレームの情報を取得できたら
01236     if (SUCCEEDED(pBuffer->Lock(&pData, nullptr, &cbDataLength)) && pData)
01237     {
01238         // 全フレーム処理モードなら
01239         if (!prioritizeLatency)
01240         {
01241             // キャプチャしている間は
01242             while (running && captured)
01243             {
01244                 // スレッドを一時停止して CPU リソースを解放する
01245                 std::this_thread::yield();
01246             }
01247         }
01248
01249         // 一時メモリをロックして
01250         std::lock_guard<std::mutex> lock{ mtx };
01251
01252         // キャプチャしたデータを一時メモリにコピーしたら
01253         if (image.size() < cbDataLength) image.resize(cbDataLength);
01254         memcpy(image.data(), pData, cbDataLength);
01255
01256         // 新しいフレームがキャプチャされたことを通知して
01257         captured = true;
01258
01259         // メディアバッファのロックを解除する
01260         pBuffer->Unlock();
01261     }
01262
01263     // メディアバッファを解放する
01264     pBuffer->Release();
01265     pBuffer = nullptr;
01266 }
01267
01268 done:
01269
01270 // デコーダの出力サンプルとバッファを解放する
01271 if (pDecodedSample) pDecodedSample->Release();
01272
01273 // カラーコンバータの出力サンプルバッファを解放する
01274 if (pConvertedSample) pConvertedSample->Release();
01275
01276 // サンプルを解放する
01277 pSample->Release();
01278 pSample = nullptr;
01279 }
01280 }
01281
01282 //
01283 // キャプチャを開始する
01284 //
01285 bool CamMf::onStart()
01286 {
01287     // Source Reader が準備できていなければ開始できない
01288     if (!pSourceReader) return false;
01289
01290     // キャプチャスレッドを起動する
01291     thr = std::thread(&CamMf::capture, this);
01292     return true;
01293 }
01294
01295 //
01296 // キャプチャスレッドを停止する

```

```

01297 //
01298 void CamMf::onStop()
01299 {
01300     // ReadSample のプロッキングを解除する
01301     if (pSourceReader)
01302     {
01303         pSourceReader->Flush(MF_SOURCE_READER_FIRST_VIDEO_STREAM);
01304     }
01305 }
01306
01307 //
01308 // カメラを閉じる
01309 //
01310 void CamMf::onClose()
01311 {
01312     // MFT デコーダを解放する
01313     cleanUpTransform(&pDecoder);
01314
01315     // MFT カラーコンバータを解放する
01316     cleanUpTransform(&pConverter);
01317
01318     // MFT デコーダのバッファを解放する
01319     SafeRelease(&pDecoderBuffer);
01320
01321     // MFT カラーコンバータのバッファを解放する
01322     SafeRelease(&pConverterBuffer);
01323
01324     // Source Reader を解放する
01325     SafeRelease(&pSourceReader);
01326
01327     // Media Source 解放する
01328     SafeRelease(&pMediaSource);
01329
01330     // フォーマットリストをクリアする
01331     availableFormats.clear();
01332     formatList.clear();
01333 }

```

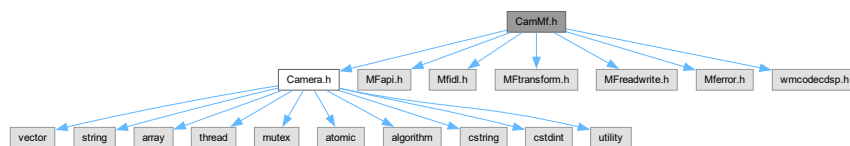
9.31 CamMf.h ファイル

```

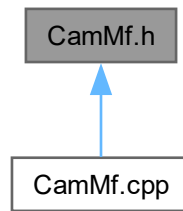
#include "Camera.h"
#include <MFapi.h>
#include <Mfidl.h>
#include <MFtransform.h>
#include <MFreadwrite.h>
#include <Mferror.h>
#include <wmcodecdsp.h>

```

CamMf.h の依存先関係図:



被依存関係図:



クラス

- class [CamMf](#)

9.31.1 詳解

Microsoft Media Foundation を使ったビデオキャプチャクラスの定義

著者

Kohe Tokoi

日付

December 24, 2024

[CamMf.h](#) に定義があります。

9.32 CamMf.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // カメラ関連の処理
00012 #include "Camera.h"
00013
00014 // Microsoft Media Foundation
00015 #include <MFapi.h>
00016 #include <Mfidl.h>
00017 #include <MFtransform.h>
00018 #include <MFreadwrite.h>
00019 #include <Mferror.h>
00020 #include <wmcodecdsp.h>
00021
00025 class CamMf : public Camera
00026 {
00030     struct VideoFormat
00031     {
00032         UINT32 width;
```

```

00033     UINT32 height;
00034     UINT32 fpsNum;
00035     UINT32 fpsDenom;
00036     GUID subType;
00037
00047     VideoFormat(UINT32 width, UINT32 height,
00048                 UINT32 fpsNum, UINT32 fpsDenom, GUID subType)
00049     : width{ width }
00050     , height{ height }
00051     , fpsNum{ fpsNum }
00052     , fpsDenom{ fpsDenom }
00053     , subType{ subType }
00054     {
00055     };
00056 };
00057
00061 class ComInitializer
00062 {
00064     static ComInitializer instance;
00065
00067     IMFActivate** ppSourceActivate{ nullptr };
00068
00070     UINT32 cSourceActivate{ 0 };
00071
00073     std::vector<std::string> deviceList;
00074
00076     bool coInitialized{ false };
00077
00079     bool mfStarted{ false };
00080
00084     ComInitializer();
00085
00089     ~ComInitializer();
00090
00096     const char* initialize();
00097
00098 public:
00099
00100     // シングルトンなのでコピーは作らせない
00101     ComInitializer(const ComInitializer& com) = delete;
00102     ComInitializer(ComInitializer&& com) = delete;
00103     ComInitializer& operator=(const ComInitializer& com) = delete;
00104     ComInitializer& operator=(ComInitializer&&) = delete;
00105
00111     static const ComInitializer& getInstance();
00112
00120     static bool activate(int device, IMFMediaSource** pMediaSource);
00121
00127     static const std::vector<std::string>& getDeviceList();
00128 };
00129
00131 IMFMediaSource* pMediaSource{ nullptr };
00132
00134 IMFSourceReader* pSourceReader{ nullptr };
00135
00137 IMFTransform* pDecoder{ nullptr };
00138
00140 IMFMediaBuffer* pDecoderBuffer{ nullptr };
00141
00143 IMFTransform* pConverter{ nullptr };
00144
00146 IMFMediaBuffer* pConverterBuffer{ nullptr };
00147
00149 std::vector<VideoFormat> availableFormats;
00150
00152 std::vector<CaptureFormat> formatList;
00153
00159 bool enumerateFormats();
00160
00171 HRESULT findVideoDecoder(
00172     const GUID& subtype,
00173     IMFTransform** ppDecoder,
00174     BOOL bAllowAsync = FALSE,
00175     BOOL bAllowHardware = FALSE,
00176     BOOL bAllowTranscode = FALSE
00177 ) const;
00178
00187 HRESULT setUpPipeline(IMFTransform* pTransform,
00188     const VideoFormat& format, const GUID& subType) const;
00189
00195 void cleanUpTransform(IMFTransform** pTransform) const;
00196
00202 HRESULT createDecoderBuffer();
00203
00209 HRESULT createConverterBuffer();
00210
00217 bool setFormat(int index);

```

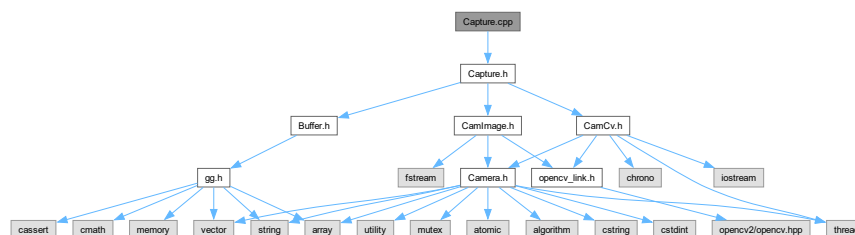
```

00218
00219 protected:
00220
00226     virtual bool onStart() override;
00227
00231     virtual void onStop() override;
00232
00236     virtual void onClose() override;
00237
00238 public:
00239
00243     CamMf() = default;
00244
00248     virtual ~CamMf()
00249     {
00250         // カメラを閉じる
00251         close();
00252     }
00253
00259     static const auto& getDeviceList()
00260     {
00261         return ComInitializer::getDeviceList();
00262     }
00263
00271     bool open(int device, bool setupFormat = true);
00272
00278     virtual const std::vector<CaptureFormat>& getFormatList() const override
00279     {
00280         return formatList;
00281     }
00282
00289     bool select(int index);
00290
00297     virtual bool selectFormat(int index) override
00298     {
00299         return select(index);
00300     }
00301
00305     void capture();
00306 };

```

9.33 Capture.cpp ファイル

#include "Capture.h"
 Capture.cpp の依存先関係図:



9.33.1 詳解

キャプチャクラスの実装

著者

Kohe Tokoi

日付

April 3, 2023

[Capture.cpp](#) に定義があります。

9.34 Capture.cpp

[詳解]

```

00001
00002 #include "Capture.h"
00003
00004 #if defined(_WIN32) || defined(__ANDROID__) || defined(__APPLE__)
00005 const std::vector<CaptureFormat> Capture::emptyFormatList;
00006 #endif
00007
00008 //
00009 // 画像ファイルを開く
00010 //
00011 bool Capture::openImage(const std::string& filename)
00012 {
00013     // 新しいキャプチャデバイスを作成したら
00014     auto camImage{ std::make_unique<CamImage>() };
00015
00016     // キャプチャデバイスを開く
00017     if (camImage->open(filename))
00018     {
00019         // このキャプチャデバイスを使うことにする
00020         camera = std::move(camImage);
00021         return true;
00022     }
00023
00024     // 開けなかった
00025     return false;
00026 }
00027
00028 //
00029 // 動画ファイルを開く
00030 //
00031 bool Capture::openMovie(const std::string& filename,
00032                          cv::VideoCaptureAPIs backend)
00033 {
00034     // 新しいキャプチャデバイスを作成したら
00035     auto camCv{ std::make_unique<CamCv>() };
00036
00037     // キャプチャデバイスを開く
00038     if (camCv->open(filename, 0, 0, 0.0, "", backend))
00039     {
00040         // このキャプチャデバイスを使うことにする
00041         camera = std::move(camCv);
00042         return true;
00043     }
00044
00045     // 開けなかった
00046     return false;
00047 }
00048
00049 #if defined(_WIN32) || defined(__ANDROID__) || defined(__APPLE__)
00050 //
00051 // デバイスを開く (Windows用: MSMF, Android用: Camera2, macOS用: AVFoundation)
00052 //
00053 bool Capture::openDevice(int deviceNumber)
00054 {
00055     // 既にカメラが有効なら一旦閉じる
00056     if (camera) camera->close();
00057
00058     #if defined(_WIN32)
00059         // 新しいキャプチャデバイスを作成したら
00060         auto camMf{ std::make_unique<CamMf>() };
00061
00062         // このデバイスをデバイス番号で開いて
00063         if (camMf->open(deviceNumber, false))
00064         {
00065             // このキャプチャデバイスを使うことにする
00066             camera = std::move(camMf);
00067
00068             // 開けた
00069             return true;
00070         }
00071     #elif defined(__ANDROID__)
00072         // 新しいキャプチャデバイスを作成したら
00073         auto camAndroid{ std::make_unique<CamAndroid>() };
00074
00075         // このデバイスをデバイス番号で開いて
00076         if (camAndroid->open(deviceNumber))
00077         {
00078             // このキャプチャデバイスを使うことにする
00079             camera = std::move(camAndroid);
00080
00081             // 開けた
00082             return true;
00083         }
00084     #endif
00085 }
00086
00087 #endif

```

```

00090     }
00091 #elif defined(__APPLE__)
00092     // 新しいキャプチャデバイスを作成したら
00093     auto camAvf{ std::make_unique<CamAvf>() };
00094
00095     // このデバイスをデバイス番号で開いて
00096     if (camAvf->open(deviceNumber, false))
00097     {
00098         // このキャプチャデバイスを使うことにする
00099         camera = std::move(camAvf);
00100
00101         // 開けた
00102         return true;
00103     }
00104 #endif
00105
00106     // カメラを無効にしておく
00107     camera.reset();
00108
00109     // 開けなかった
00110     return false;
00111 }
00112
00113 //
00114 // ビデオフォーマット選択
00115 //
00116 bool Capture::select(int index)
00117 {
00118     // カメラが有効でなければ戻る
00119     if (!camera) return false;
00120
00121     // ビデオフォーマットを選択する
00122     return camera->selectFormat(index);
00123 }
00124
00125 void Capture::updateFormatList(int deviceNumber)
00126 {
00127     #if defined(_WIN32)
00128     // 実際の入力状態を変更せずに選択肢だけ取得する一時カメラ
00129     CamMf temp;
00130
00131     // デバイスを遅延初期化で開く
00132     if (temp.open(deviceNumber, false))
00133     {
00134         // 開けたら列挙されたフォーマットリストを保存する
00135         deviceFormatList = temp.getFormatList();
00136         temp.close();
00137     }
00138     else
00139     {
00140         // 開けなかったらフォーマットリストを空にする
00141         deviceFormatList.clear();
00142     }
00143 #elif defined(__ANDROID__)
00144     // 実際の入力状態を変更せずに選択肢だけ取得する一時カメラ
00145     CamAndroid temp;
00146
00147     // デバイスを遅延初期化で開く
00148     if (temp.open(deviceNumber))
00149     {
00150         // 開けたら列挙されたフォーマットリストを保存する
00151         deviceFormatList = temp.getFormatList();
00152         temp.close();
00153     }
00154     else
00155     {
00156         // 開けなかったらフォーマットリストを空にする
00157         deviceFormatList.clear();
00158     }
00159 #elif defined(__APPLE__)
00160     // 実際の入力状態を変更せずに選択肢だけ取得する一時カメラ
00161     CamAvf temp;
00162
00163     // デバイスを遅延初期化で開く
00164     if (temp.open(deviceNumber, false))
00165     {
00166         // 開けたら列挙されたフォーマットリストを保存する
00167         deviceFormatList = temp.getFormatList();
00168         temp.close();
00169     }
00170     else
00171     {
00172         // 開けなかったらフォーマットリストを空にする
00173         deviceFormatList.clear();
00174     }
00175 #endif
00176 }

```

```

00177
00178 //
00179 // フォーマットリストを取り出す
00180 //
00181 const std::vector<CaptureFormat>& Capture::getFormatList() const
00182 {
00183     // 現在開いているカメラが有効かつフォーマットを保持している場合はその一覧を返し、
00184     // 静止画像 (CamImage) 表示中やカメラ未開始時は事前取得済みのデバイスフォーマット一覧を返す
00185     if (camera && !camera->getFormatList().empty())
00186     {
00187         return camera->getFormatList();
00188     }
00189     return deviceFormatList;
00190 }
00191
00192 #else
00193 //
00194 // デバイスを開く (Windows以外用: OpenCV)
00195 //
00196 bool Capture::openDevice(int deviceNumber, std::array<int, 2>& size, double& fps,
00197     cv::VideoCaptureAPIs backend, char* fourcc)
00198 {
00199     // 既にカメラが有効なら一旦閉じる
00200     if (camera) camera->close();
00201
00202     #if defined(USE_LIBCAMERA)
00203     // libcamera バックエンドの場合
00204     if (backend == CAP_LIBCAMERA)
00205     {
00206         auto camLibcam{ std::make_unique<CamLibcam>(deviceNumber, size[0], size[1], fps) };
00207         if (camLibcam->isOpened())
00208         {
00209             size[0] = camLibcam->getWidth();
00210             size[1] = camLibcam->getHeight();
00211             fps = camLibcam->getFps();
00212             if (fourcc)
00213             {
00214                 const auto name{ camLibcam->getPixelFormatName() };
00215                 std::strncpy(fourcc, name.c_str(), 4);
00216                 fourcc[4] = '\0';
00217             }
00218             camera = std::move(camLibcam);
00219             return true;
00220         }
00221         return false;
00222     }
00223     #endif
00224
00225     // 新しいキャプチャデバイスを作成したら
00226     auto camCv{ std::make_unique<CamCv>() };
00227
00228     // このデバイスをデバイス番号で開いて
00229     if (camCv->open(deviceNumber, size[0], size[1], fps, fourcc, backend))
00230     {
00231         // 実際に開いた設定を書き戻す
00232         size[0] = camCv->getWidth();
00233         size[1] = camCv->getHeight();
00234         fps = camCv->getFps();
00235         camCv->getCodec(fourcc);
00236
00237         // このキャプチャデバイスを使うことにする
00238         camera = std::move(camCv);
00239         return true;
00240     }
00241
00242     // 開けなかった
00243     return false;
00244 }
00245 #endif
00246
00247 //
00248 // キャプチャ開始
00249 //
00250 void Capture::start()
00251 {
00252     // キャプチャデバイスが有効ならキャプチャスレッドを起動する
00253     if (camera) camera->start();
00254 }
00255
00256 //
00257 // キャプチャ終了
00258 //
00259 void Capture::stop()
00260 {
00261     // キャプチャデバイスが有効ならキャプチャスレッドを停止する
00262     if (camera) camera->stop();
00263 }

```

```

00264
00265 //
00266 // キャプチャデバイスを閉じる
00267 //
00268 void Capture::close()
00269 {
00270     // キャプチャデバイスが有効ならキャプチャスレッドを停止する
00271     if (camera)
00272     {
00273         camera->close();
00274         camera.reset();
00275     }
00276 }
00277
00278 //
00279 // キャプチャデバイスの解像度とフレームレートを取得する
00280 //
00281 std::array<int, 2> Capture::getSize() const
00282 {
00283     // キャプチャデバイスが有効ならその解像度を返す
00284     return camera ? camera->getSize() : std::array<int, 2>{ 0, 0 };
00285 }
00286
00287 //
00288 // キャプチャデバイスのフレームレートを得る
00289 //
00290 double Capture::getFps() const
00291 {
00292     // キャプチャデバイスが有効ならそのフレームレートを返す
00293     return camera ? camera->getFps() : 0.0;
00294 }
00295
00296 //
00297 // 新しいフレームを GPU の PBO に取得する
00298 //
00299 bool Capture::retrieve(Buffer& buffer)
00300 {
00301     // キャプチャデバイスが無効なら失敗
00302     if (!camera) return false;
00303
00304     // フレームデータをロックして PBO に転送する
00305     return camera->lockFrame([&buffer](const std::uint8_t* data, size_t length, int width, int height,
int channels) {
00306         buffer.create(width, height, channels);
00307         glBindBuffer(GL_PIXEL_PACK_BUFFER, buffer.getBufferName());
00308         glBufferSubData(GL_PIXEL_PACK_BUFFER, 0, length, data);
00309         glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00310     });
00311 }
00312
00313 //
00314 // 新しいフレームを CPU のメモリに取得する
00315 //
00316 bool Capture::retrieve(cv::Mat& frame)
00317 {
00318     // OpenCV 補正を選んだときだけ使用し、PBO へ送る前の画像を cv::Mat として得る。
00319     if (!camera) return false;
00320
00321     // フレームデータをロックして cv::Mat にコピーする
00322     return camera->lockFrame([&frame](const std::uint8_t* data, size_t length, int width, int height, int
channels) {
00323         frame.create(height, width, ((channels - 1) << 3));
00324         const auto copySize{ std::min(static_cast<size_t>(frame.total() * frame.elemSize()), length) };
00325         std::memcpy(frame.data, data, copySize);
00326     });
00327 }

```

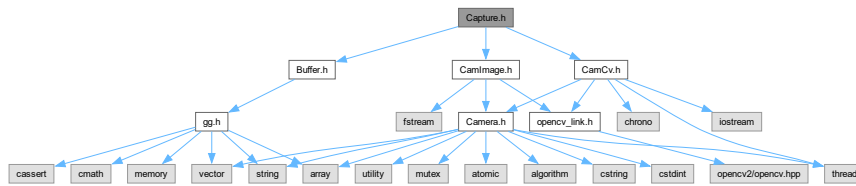
9.35 Capture.h ファイル

```

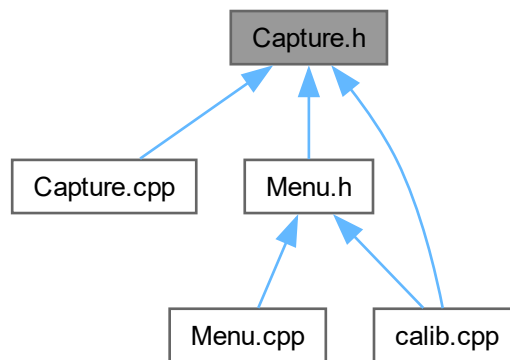
#include "Buffer.h"
#include "CamImage.h"
#include "CamCv.h"

```

Capture.h の依存先関係図:



被依存関係図:



クラス

- class [Capture](#)

9.35.1 詳解

キャプチャクラスの定義

著者

Kohe Tokoi

日付

April 3, 2023

[Capture.h](#) に定義があります。

9.36 Capture.h

[詳解]

```

00001 #pragma once
00002
00010 // バッファクラス
00011 // OpenCV による画像ファイルの入力
00012 #include "Buffer.h"
00013
00014 // OpenCV による動画の入力
00015 #include "CamImage.h"
00016
00017 // OpenCV による動画の入力
00018 #include "CamCv.h"
00019
00020 // Windows のみ Microsoft Media Foundation による動画の入力
00021 #if defined(_WIN32)
00022 #include "CamMf.h"
00023 #endif
00024
00025 // Android のみ Camera2 NDK によるカメラの入力
00026 #if defined(_ANDROID_)
00027 #include "CamAndroid.h"
00028 #endif
00029
00030 // macOS のみ AV Foundation による動画の入力
00031 #if defined(__APPLE__)
00032 #include "CamAvf.h"
00033 #endif
00034
00035 // libcamera による動画の入力
00036 #if defined(USE_LIBCAMERA)
00037 #include "CamLibcam.h"
00038 constexpr cv::VideoCaptureAPIs CAP_LIBCAMERA{ static_cast<cv::VideoCaptureAPIs>(9000) };
00039 #endif
00040
00041 class Capture
00042 {
00043     std::unique_ptr<Camera> camera{};
00044
00045 #if defined(_WIN32) || defined(_ANDROID_) || defined(__APPLE__)
00046     std::vector<CaptureFormat> deviceFormatList;
00047     static const std::vector<CaptureFormat> emptyFormatList;
00048 #endif
00049 public:
00050     Capture() = default;
00051     Capture(const std::string& filename)
00052     {
00053         openImage(filename);
00054     }
00055     virtual ~Capture()
00056     {
00057         close();
00058     }
00059     bool openImage(const std::string& filename);
00060     bool openMovie(const std::string& filename,
00061 #if defined(_WIN32)
00062         cv::VideoCaptureAPIs backend = cv::CAP_ANY;
00063 #else
00064         cv::VideoCaptureAPIs backend = cv::CAP_FFMPEG;
00065 #endif
00066     );
00067     bool openDevice(int deviceNumber);
00068     const std::vector<CaptureFormat>& getFormatList() const;
00069     bool select(int index);
00070     void updateFormatList(int deviceNumber);
00071 #else
00072     bool openDevice(int deviceNumber,
00073         std::array<int, 2>& size, double& fps,
00074         cv::VideoCaptureAPIs backend,
00075         char* fourcc);
00076 #endif
00077     void start();

```

```

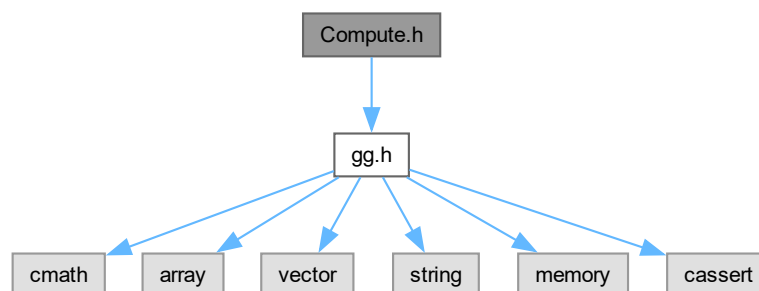
00158
00162 void stop();
00163
00167 void close();
00168
00174 explicit operator bool() const
00175 {
00176     return camera && camera->isRunning();
00177 }
00178
00182 bool isOpened() const
00183 {
00184     return bool(camera);
00185 }
00186
00190 bool isImage() const
00191 {
00192     return camera && camera->isStillImage();
00193 }
00194
00200 std::array<int, 2> getSize() const;
00201
00207 double getFps() const;
00208
00215 bool retrieve(Buffer& buffer);
00216
00224 bool retrieve(cv::Mat& frame);
00225
00231 void setPrioritizeLatency(bool mode)
00232 {
00233     if (camera) camera->setPrioritizeLatency(mode);
00234 }
00235
00241 bool getPrioritizeLatency() const
00242 {
00243     return camera ? camera->getPrioritizeLatency() : false;
00244 }
00245 };

```

9.37 Compute.h ファイル

```
#include "gg.h"
```

Compute.h の依存先関係図:



クラス

- class [Compute](#)

9.37.1 詳解

画像処理クラスの定義（コンピュートシェーダ版）

著者

Kohe Tokoi

日付

November 15, 2022

[Compute.h](#) に定義があります。

9.38 Compute.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013 using namespace gg;
00014
00018 class Compute
00019 {
00021     const GLuint program;
00022
00023 public:
00024
00030     Compute(const char* comp)
00031         : program(ggLoadComputeShader(comp))
00032     {
00033     }
00034
00038     virtual ~Compute()
00039     {
00040         // シェーダプログラムを削除する
00041         glDeleteShader(program);
00042     }
00043
00049     auto get() const
00050     {
00051         return program;
00052     }
00053
00057     void use() const
00058     {
00059         glUseProgram(program);
00060     }
00061
00070     void execute(GLuint width, GLuint height, GLuint local_size_x = 1, GLuint local_size_y = 1) const
00071     {
00072         glDispatchCompute((width + local_size_x - 1) / local_size_x, (height + local_size_y - 1) /
00073             local_size_y, 1);
00074     };
00075 }
```

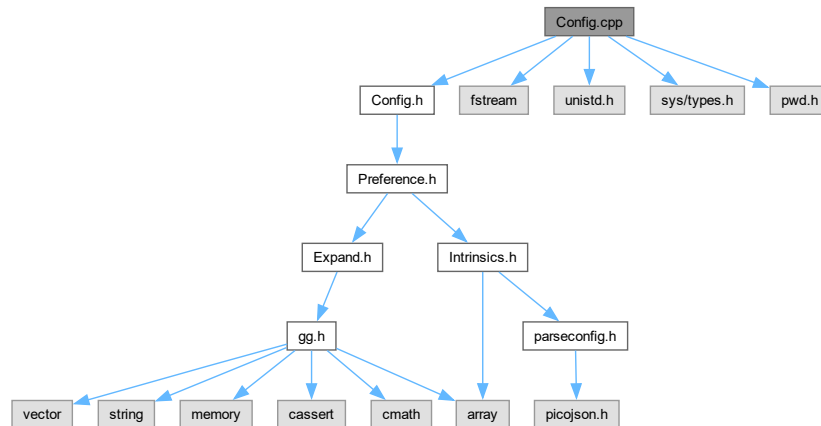
9.39 Config.cpp ファイル

```
#include "Config.h"
#include <fstream>
#include <unistd.h>
```



```
#include <sys/types.h>
#include <pwd.h>
```

Config.cpp の依存先関係図:



9.39.1 詳解

構成データクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Config.cpp](#) に定義があります。

9.40 Config.cpp

[詳解]

```

00001
00008 #include "Config.h"
00009
00010 // 標準ライブラリ
00011 #include <fstream>
00012
00013 #if defined(_WIN32)
00014 // Microsoft Media Foundation によるキャプチャ
00015 #include "CamMf.h"
00016 #elif defined(__ANDROID__)
00017 // Android Camera2 NDK によるキャプチャ
00018 #include "CamAndroid.h"
00019 #elif defined(__APPLE__)
00020 // AV Foundation によるキャプチャ
00021 #include "CamAvf.h"
00022 #endif
00023
00024 #if !defined(_DEBUG)

```

```

00025 # if defined(_WIN32)
00026 // appData のパスを得るときに使う
00027 # include <shlobj_core.h>
00028 # else
00029 // ホームディレクトリのパスを得るときに使う
00030 # include <unistd.h>
00031 # include <sys/types.h>
00032 # include <pwd.h>
00033 # endif
00034 #endif
00035
00036 //
00037 // コンストラクタ
00038 //
00039 Config::Config(const std::string& filename)
00040 #if defined(_WIN32)
00041 : deviceList{ CamMf::getDeviceList() }
00042 #elif defined(_ANDROID_)
00043 : deviceList{ CamAndroid::getDeviceList() }
00044 #elif defined(_APPLE_)
00045 : deviceList{ CamAvf::getDeviceList() }
00046 #endif
00047 {
00048     // 構成ファイルの保存場所を決定する
00049 #if defined(_DEBUG)
00050     const auto path{ Utf8ToTChar(filename) };
00051 #else
00052 # if defined(_WIN32)
00053     // 構成ファイルの保存先のパス
00054     wchar_t appDataPath[MAX_PATH];
00055
00056     // AppData のパスを取得する
00057     SHGetSpecialFolderPathW(NULL, appDataPath, CSIDL_APPDATA, 0);
00058
00059     // AppData のパスに構成ファイル名を連結する
00060     const auto path{ CString(appDataPath) + TEXT("\\") + Utf8ToTChar(filename) };
00061 # else
00062     // ホームディレクトリのパスを取得する
00063     const char* homeDir{ std::getenv("HOME") };
00064     if (!homeDir)
00065     {
00066         const struct passwd* const pw{ getpwuid(getuid()) };
00067         if (pw && pw->pw_dir) homeDir = pw->pw_dir;
00068     }
00069
00070     // ホームディレクトリのパスに構成ファイル名を連結する
00071     const auto path{ (homeDir ? std::string(homeDir) : ".") + "/" + filename };
00072 # endif
00073 #endif
00074
00075     // 構成ファイルを読み込む
00076     if (!load(path))
00077     {
00078         // デフォルトの設定を追加する
00079         if (!load(Utf8ToTChar(filename))) preferenceList.emplace_back();
00080
00081         // デフォルトの設定を入れた構成ファイルを作成する
00082         save(path);
00083     }
00084 }
00085
00086 //
00087 // デストラクタ
00088 //
00089 Config::~Config()
00090 {
00091 }
00092
00093 //
00094 // 構成データを初期化する
00095 //
00096 void Config::initialize()
00097 {
00098     // 構成リストのすべて構成についてシェーダをビルドする
00099     for (auto& preference : preferenceList) preference.buildShader();
00100
00101     // 初期化済みであることを記録しておく
00102     initialized = true;
00103
00104     // 背景色を設定する
00105     glClearColor(background[0], background[1], background[2], background[3]);
00106 }
00107
00108 //
00109 // 構成ファイルを読み込む
00110 //
00111 bool Config::load(const pathString& filename)

```

```

00112 {
00113     // 構成ファイルの読み込み
00114     std::ifstream json{ filename };
00115     if (!json) return false;
00116
00117     // JSON の読み込み
00118     picojson::value value;
00119     json >> value;
00120     json.close();
00121
00122     // ルートが JSON オブジェクトでなければ、既存構成を変更せず失敗とする
00123     if (!value.is<picojson::object>()) return false;
00124
00125     // 構成内容の取り出し
00126     const auto& object{ value.get<picojson::object>() };
00127
00128     // オブジェクトが空だったらエラー
00129     if (object.empty()) return false;
00130
00131     // 読み込みに失敗しても現在の構成を壊さないよう、一時領域へ読み込む
00132     auto loadedWindowSize{ windowSize };
00133     auto loadedBackground{ background };
00134     auto loadedSettings{ settings };
00135     auto loadedInitialImage{ initialImage };
00136     std::vector<Preference> loadedPreferences;
00137
00138     // ウィンドウのサイズ
00139     getValue(object, "size", loadedWindowSize);
00140
00141     // ウィンドウの背景色
00142     getValue(object, "background", loadedBackground);
00143
00144     // メッシュのサンプル数
00145     getValue(object, "samples", loadedSettings.samples);
00146
00147     // キャプチャデバイスの姿勢
00148     getValue(object, "pose", loadedSettings.euler);
00149
00150     // 描画時の焦点距離
00151     getValue(object, "focal", loadedSettings.focal);
00152
00153     // 描画時の焦点距離の範囲
00154     getValue(object, "range", loadedSettings.focalRange);
00155
00156     // ArUco Marker の辞書名
00157     getString(object, "dictionary", loadedSettings.dictionaryName);
00158
00159     // ChArUco Board のマス目の横と縦の数
00160     if (!getValue(object, "squares", loadedSettings.checkerSize))
00161         getValue(object, "checkerSize", loadedSettings.checkerSize);
00162
00163     // ChArUco Board のマス目とマーカーの一辺の長さ
00164     if (!getValue(object, "length", loadedSettings.checkerLength))
00165         getValue(object, "checkerLength", loadedSettings.checkerLength);
00166
00167     // ArUco Marker の一辺の長さ
00168     if (!getValue(object, "marker", loadedSettings.markerLength))
00169         getValue(object, "markerLength", loadedSettings.markerLength);
00170
00171     // 初期表示画像
00172     getString(object, "initial", loadedInitialImage);
00173
00174     // キャプチャデバイスの構成を探す
00175     const auto& camera{ object.find("camera") };
00176
00177     // キャプチャデバイスの構成の配列が見つからなければエラー
00178     if (camera == object.end() || !camera->second.is<picojson::array>()) return false;
00179
00180     // 配列の個々の要素について
00181     for (const auto& value : camera->second.get<picojson::array>())
00182     {
00183         // 配列中に投影方式以外の値があれば、不完全な構成を適用せず失敗とする
00184         if (!value.is<picojson::object>()) return false;
00185
00186         // キャプチャデバイスの構成のオブジェクトを取り出す
00187         const auto& preference{ value.get<picojson::object>() };
00188
00189         // キャプチャデバイスの構成をリストに追加
00190         loadedPreferences.emplace_back(preference);
00191     }
00192
00193     // キャプチャデバイスの構成が一つも読み取れなければエラー
00194     if (loadedPreferences.empty()) return false;
00195
00196     // 実行時の再読み込みでは、新しい構成のシェードも準備する
00197     if (initialized)
00198     {

```

```

00199 // OpenGL 初期化後の再読み込みなので、置換前に全投影方式を描画可能にする
00200 for (auto& preference : loadedPreferences) preference.buildShader();
00201 }
00202
00203 // 読み込みと検証がすべて成功してから現在の構成を置き換える
00204 windowSize = loadedWindowSize;
00205 background = loadedBackground;
00206 settings = loadedSettings;
00207 initialImage = std::move(loadedInitialImage);
00208 preferenceList.swap(loadedPreferences);
00209
00210 if (initialized)
00211 {
00212     // 実行中の再読み込みでは、新しい背景色を現在の OpenGL 状態にも反映する
00213     glClearColor(background[0], background[1], background[2], background[3]);
00214 }
00215
00216 return true;
00217 }
00218
00219 //
00220 // 構成ファイルを保存する
00221 //
00222 bool Config::save(const pathString& filename) const
00223 {
00224     // 設定値を保存する
00225     std::ofstream config{ filename };
00226     if (!config) return false;
00227
00228     // オブジェクト
00229     picojson::object object;
00230
00231     // ウィンドウのサイズ
00232     setValue(object, "size", windowSize);
00233
00234     // ウィンドウの背景色
00235     setValue(object, "background", background);
00236
00237     // メッシュのサンプル数
00238     setValue(object, "samples", settings.samples);
00239
00240     // キャプチャデバイスの姿勢
00241     setValue(object, "pose", settings.euler);
00242
00243     // 描画時の焦点距離
00244     setValue(object, "focal", settings.focal);
00245
00246     // 描画時の焦点距離の範囲
00247     setValue(object, "range", settings.focalRange);
00248
00249     // ArUco Marker 辞書名
00250     setString(object, "dictionary", settings.dictionaryName);
00251
00252     // ChArUco Board のマス目の横と縦の数
00253     setValue(object, "squares", settings.checkerSize);
00254
00255     // ChArUco Board のマス目とマーカーの一辺の長さ
00256     setValue(object, "length", settings.checkerLength);
00257
00258     // ArUco Marker の一辺の長さ
00259     setValue(object, "marker", settings.markerLength);
00260
00261     // 初期表示画像
00262     setString(object, "initial", initialImage);
00263
00264     // 配列
00265     picojson::array array;
00266
00267     // 構成リストのすべて構成について
00268     for (const auto& preference : preferenceList)
00269     {
00270         // 構成の要素
00271         picojson::object camera;
00272
00273         // 構成を JSON オブジェクトに格納する
00274         preference.setPreference(camera);
00275
00276         // 要素を picojson::array に追加する
00277         array.emplace_back(picojson::value(camera));
00278     }
00279
00280     // オブジェクトに追加する
00281     object.emplace("camera", array);
00282
00283     // 構成をシリアライズして保存
00284     picojson::value v{ object };
00285     config << v.serialize(true);

```

```

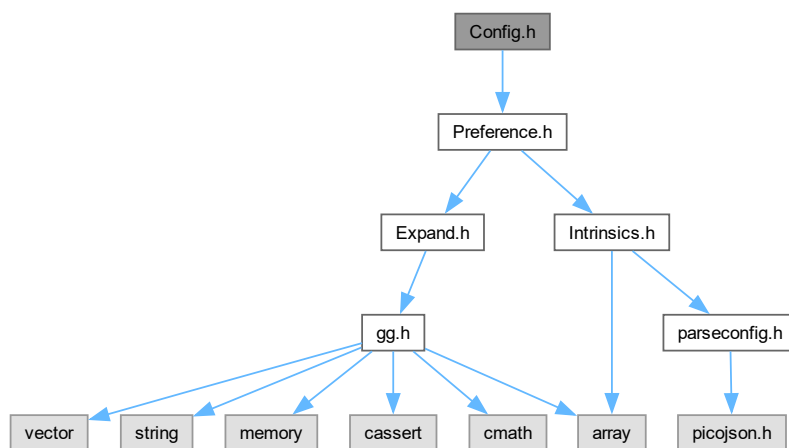
00286     config.close();
00287
00288     // 構成データの書き込み成功
00289     return true;
00290 }

```

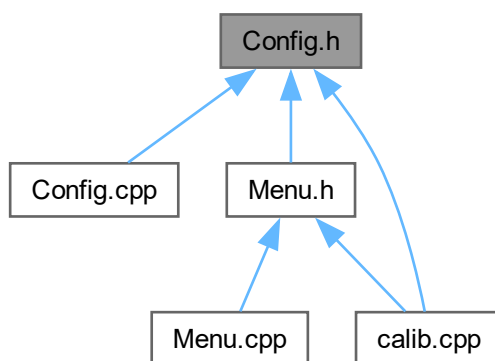
9.41 Config.h ファイル

```
#include "Preference.h"
```

Config.h の依存先関係図:



被依存関係図:



クラス

- struct [Settings](#)
- class [Config](#)

9.41.1 詳解

構成データクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Config.h](#) に定義があります。

9.42 Config.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // キャプチャデバイスの構成
00012 #include "Preference.h"
00013
00017 struct Settings
00018 {
00020     int samples{ 57600 };
00021
00023     std::array<float, 3> euler{ defaultEuler };
00024
00026     static constexpr decltype(euler) defaultEuler{ 0.0f, 0.0f, 0.0f };
00027
00029     float focal{ defaultFocal };
00030
00032     static constexpr decltype(focal) defaultFocal{ 50.0f };
00033
00035     std::array<float, 2> focalRange{ defaultFocalRange };
00036
00038     static constexpr decltype(focalRange) defaultFocalRange{ 10.0f, 200.0f };
00039
00041     std::string dictionaryName{ "DICT_4X4_50" };
00042
00044     std::array<int, 2> checkerSize{ defaultCheckerSize };
00045
00047     static constexpr decltype(checkerSize) defaultCheckerSize{ 10, 7 };
00048
00050     std::array<float, 2> checkerLength{ defaultCheckerLength };
00051
00053     static constexpr decltype(checkerLength) defaultCheckerLength{ 4.0f, 2.0f };
00054
00056     float markerLength{ defaultMarkerLength };
00057
00059     static constexpr decltype(markerLength) defaultMarkerLength{ 5.0f };
00060
00064     auto getFocal() const
00065     {
00066         // 投影面の対角線長は 35mm (17.5mm × 2) とする
00067         return focal / 17.5f;
00068     }
00069 };
00070
00074 class Config
00075 {
00077     std::string title{ PROJECT_NAME };
00078
00080     std::array<GLsizei, 2> windowSize{ 1280, 720 };
00081
00083     std::array<GLfloat, 4> background{ 0.2f, 0.3f, 0.4f, 1.0f };
00084
00086     Settings settings{};
00087
00089     std::string menuFont{ "Mplus1-Regular.ttf" };
```

```

00090
00092     float menuFontSize{ 20.0f };
00093
00095     static std::string initialImage;
00096
00098     std::vector<Preference> preferenceList;
00099
00101     bool initialized{ false };
00102
00103     #if defined(_WIN32) || defined(_ANDROID_) || defined(_APPLE_)
00105     const std::vector<std::string>& deviceList;
00106     #endif
00107
00108     public:
00109
00115     Config(const std::string& filename);
00116
00120     virtual ~Config();
00121
00127     void initialize();
00128
00138     bool load(const pathString& filename);
00139
00146     bool save(const pathString& filename) const;
00147
00153     const Settings& getSettings() const
00154     {
00155         return settings;
00156     }
00157
00163     void setSettings(const Settings& value)
00164     {
00165         settings = value;
00166     }
00167
00173     const auto& getPreferences() const
00174     {
00175         return preferenceList;
00176     }
00177
00183     const auto& getBackground() const
00184     {
00185         return background;
00186     }
00187
00193     const auto& getMenuFont() const
00194     {
00195         return menuFont;
00196     }
00197
00203     auto getMenuFontSize() const
00204     {
00205         return menuFontSize;
00206     }
00207
00213     const auto& getTitle() const
00214     {
00215         return title;
00216     }
00217
00223     auto getWidth() const
00224     {
00225         return windowSize[0];
00226     }
00227
00233     auto getHeight() const
00234     {
00235         return windowSize[1];
00236     }
00237
00243     const auto& getInitialImage() const
00244     {
00245         return initialImage;
00246     }
00247
00253     const auto& getDictionaryName() const
00254     {
00255         return settings.dictionaryName;
00256     }
00257
00263     const auto& getCheckerSize() const
00264     {
00265         return settings.checkerSize;
00266     }
00267
00273     const auto& getCheckerLength() const
00274     {

```

```

00275     return settings.checkerLength;
00276 }
00277
00283 auto getMarkerLength() const
00284 {
00285     return settings.markerLength;
00286 }
00287
00288 #if defined(_WIN32) || defined(_ANDROID_) || defined(_APPLE_)
00294 const auto& getDeviceList() const
00295 {
00296     return deviceList;
00297 }
00298
00305 const auto& getDeviceName(int number) const
00306 {
00307     static const std::string empty{};
00308     return deviceList.empty() ? empty : deviceList[number];
00309 }
00310 #endif
00311 };

```

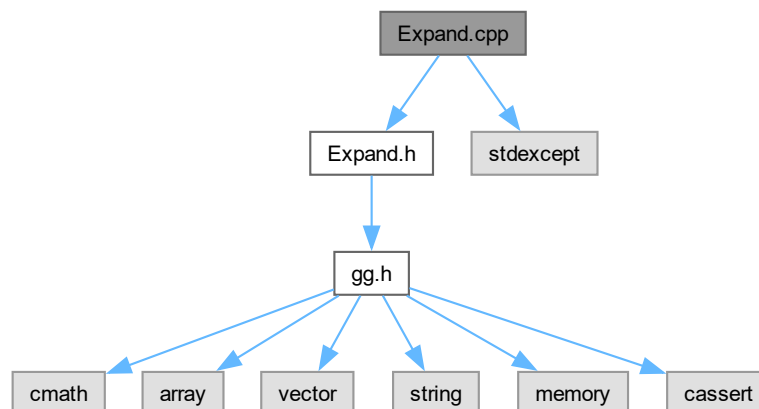
9.43 Expand.cpp ファイル

```

#include "Expand.h"
#include <stdexcept>

```

Expand.cpp の依存先関係図:



9.44 Expand.cpp

[詳解]

```

00001 //
00002 // 展開用シェーダ
00003 //
00004 #include "Expand.h"
00005
00006 // 標準ライブラリ
00007 #include <stdexcept>
00008
00009 //
00010 // コンストラクタ
00011 //
00012 Expand::Expand(const std::string& vert, const std::string& frag)
00013 : program{ gg::ggLoadShader(vert, frag) }

```



```

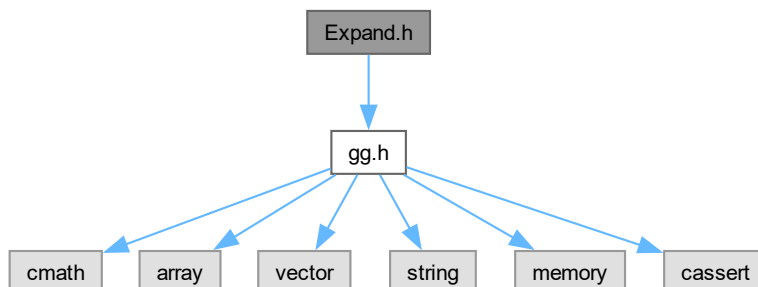
00014 , imageLoc{ glGetUniformLocation(program, "image") }
00015 , screenLoc{ glGetUniformLocation(program, "screen") }
00016 , focalLoc{ glGetUniformLocation(program, "focal") }
00017 , rotationLoc{ glGetUniformLocation(program, "rotation") }
00018 , circleLoc{ glGetUniformLocation(program, "circle") }
00019 , borderLoc{ glGetUniformLocation(program, "border") }
00020 , gapLoc{ glGetUniformLocation(program, "gap") }
00021 {
00022     // プログラムオブジェクトが作れなかったら落とす
00023     if (program == 0) throw std::runtime_error("Cannot create expand shader: " + vert + " / " + frag);
00024 }
00025
00026 //
00027 // デストラクタ
00028 //
00029 Expand::~Expand()
00030 {
00031     glDeleteProgram(program);
00032 }
00033
00037 std::array<int, 2> Expand::setup(int samples, GLfloat aspect, const gg::GgMatrix& pose,
00038     const std::array<GLfloat, 2>& fov, const std::array<GLfloat, 2>& center, GLfloat focal,
00039     const std::array<GLfloat, 4>& border, int unit) const
00040 {
00041     // プログラムオブジェクトの指定
00042     glUseProgram(program);
00043
00044     // テクスチャユニットの指定
00045     glUniform1i(imageLoc, unit);
00046
00047     // 境界色
00048     glUniform4fv(borderLoc, 1, border.data());
00049
00050     // 投影像の画角（度）と中心位置
00051     glUniform4f(circleLoc, fov[0], fov[1], center[0], center[1]);
00052
00053     // スクリーンのサイズと中心位置
00054     // screen[0] = (right - left) / 2
00055     // screen[1] = (top - bottom) / 2
00056     // screen[2] = (right + left) / 2
00057     // screen[3] = (top + bottom) / 2
00058     const GLfloat screen[] = { aspect, 1.0f, 0.0f, 0.0f };
00059     glUniform4fv(screenLoc, 1, screen);
00060
00061     // レンズの主点とスクリーンの距離
00062     glUniform1f(focalLoc, focal);
00063
00064     // 背景に対する視線の回転行列
00065     glUniformMatrix4fv(rotationLoc, 1, GL_FALSE, pose.get());
00066
00067     // メッシュの横の格子点数
00068     // 標本点の数（頂点数） samples = w * h とするとき、これに縦横比
00069     // aspect = w / h をかければ aspect * samples = w * h となるから、
00070     // w = sqrt(aspect * samples), h = samples / w で求められる
00071     const auto w{ static_cast<int>(sqrt(aspect * samples)) };
00072     const auto h{ samples / w };
00073
00074     // 格子間隔
00075     // クリッピング空間全体を埋める四角形は [-1, 1] の範囲すなわち
00076     // 縦横 2 の大きさだから、それを縦横の（格子数 - 1）で割る
00077     std::array<GLfloat, 2> gap{ 2.0f / (w - 1), 2.0f / (h - 1) };
00078     glUniform2fv(gapLoc, 1, gap.data());
00079
00080     // 描画するメッシュの横と縦の格子点数を返す
00081     return std::array<int, 2>{ w, h };
00082 }

```

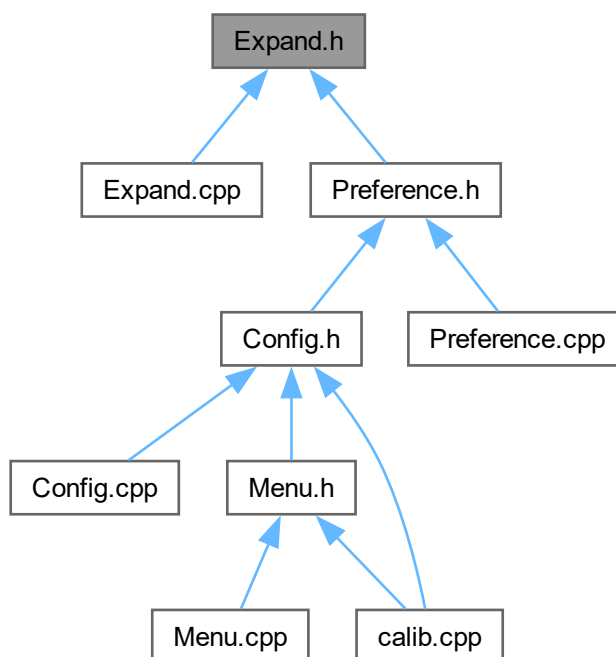
9.45 Expand.h ファイル

```
#include "gg.h"
```

Expand.h の依存先関係図:



被依存関係図:



クラス

- class [Expand](#)

9.45.1 詳解

展開用シェークラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Expand.h](#) に定義があります。

9.46 Expand.h

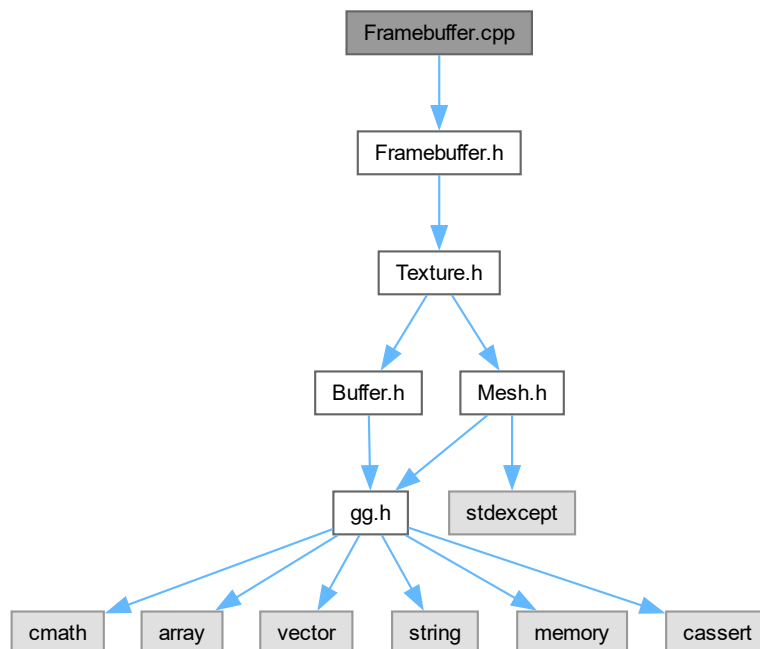
[詳解]

```
00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
00017 class Expand
00018 {
00020     const GLuint program;
00021
00023     const GLint imageLoc;
00024
00026     const GLint screenLoc;
00027
00029     const GLint focalLoc;
00030
00032     const GLint rotationLoc;
00033
00035     const GLint circleLoc;
00036
00038     const GLint borderLoc;
00039
00041     const GLint gapLoc;
00042
00043 public:
00044
00051     Expand(const std::string& vert, const std::string& frag);
00052
00058     Expand(const Expand& shader) = delete;
00059
00063     virtual ~Expand();
00064
00071     Expand& operator=(const Expand& shader) = delete;
00072
00078     auto getProgram() const
00079     {
00080         return program;
00081     }
00082
00100     std::array<GLsizei, 2> setup(int samples, GLfloat aspect, const gg::GgMatrix& pose,
00101         const std::array<GLfloat, 2>& fov, const std::array<GLfloat, 2>& center, GLfloat focal,
00102         const std::array<GLfloat, 4>& border, int unit = 0) const;
00103 };
```

9.47 Framebuffer.cpp ファイル

```
#include "Framebuffer.h"
```

Framebuffer.cpp の依存先関係図:



9.47.1 詳解

フレームバッファオブジェクトクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Framebuffer.cpp](#) に定義があります。

9.48 Framebuffer.cpp

[詳解]

```

00001
00008 #include "Framebuffer.h"
00009
00010 //
00011 // フレームバッファオブジェクトを作成するコンストラクタ
00012 //
00013 Framebuffer::Framebuffer(GLsizei width, GLsizei height, int channels,
00014                           GLenum attachment)
00015 : attachment{ attachment }
00016 {
00017     // フレームバッファオブジェクトを作る
00018     Framebuffer::create(width, height, channels);
00019 }
00020
00026 Framebuffer::Framebuffer(const Framebuffer& framebuffer)
00027 : attachment{ framebuffer.attachment }
00028 {
00029     // フレームバッファオブジェクトをコピーして作成する
00030     Framebuffer::copy(framebuffer);
00031 }
00032
00033 //
00034 // デストラクタ
00035 //
00036 Framebuffer::~Framebuffer()
00037 {
00038     // フレームバッファオブジェクトを破棄する
00039     Framebuffer::discard();
00040 }
00041
00042 //
00043 // 代入演算子
00044 //
00045 Framebuffer& Framebuffer::operator=(const Framebuffer& framebuffer)
00046 {
00047     // 代入元と代入先が同じでなければ
00048     if (&framebuffer != this)
00049     {
00050         // 引数のフレームバッファオブジェクトをコピーする
00051         Framebuffer::copy(framebuffer);
00052     }
00053
00054     // このフレームバッファオブジェクトを返す
00055     return *this;
00056 }
00057
00058 //
00059 // ムーブ代入演算子
00060 //
00061 Framebuffer& Framebuffer::operator=(Framebuffer&& framebuffer) noexcept
00062 {
00063     // ムーブ代入元とムーブ代入先が同じなら何もしない
00064     if (&framebuffer == this) return *this;
00065
00066     // ムーブ代入元の基底クラスをムーブする
00067     static_cast<Texture&>(*this) = std::move(framebuffer);
00068
00069     // ムーブ代入元のテクスチャのメンバをムーブする
00070     framebufferSize = framebuffer.framebufferSize;
00071     framebuffer.framebufferSize = { 0, 0 };
00072     framebufferChannels = framebuffer.framebufferChannels;
00073     framebuffer.framebufferChannels = 0;
00074     framebuffer.attachment = GL_COLOR_ATTACHMENT0;
00075
00076     // ムーブ代入先のフレームバッファを削除する
00077     glDeleteFramebuffers(1, &framebufferName);
00078
00079     // ムーブ代入元のフレームバッファ名をムーブ代入先に移す
00080     framebufferName = framebuffer.framebufferName;
00081
00082     // ムーブ代入元のデストラクタでフレームバッファが削除されないよう 0 にする
00083     framebuffer.framebufferName = 0;
00084
00085     // このフレームバッファオブジェクトを返す
00086     return *this;
00087 }
00088
00089 //
00090 // フレームバッファオブジェクトを破棄する
00091 //
00092 void Framebuffer::discard()
00093 {

```

```

00094 // デフォルトのフレームバッファオブジェクトに戻す
00095 glBindFramebuffer(GL_FRAMEBUFFER, 0);
00096
00097 // フレームバッファオブジェクトを削除する
00098 glDeleteFramebuffers(1, &framebufferName);
00099 framebufferName = 0;
00100
00101 // バッファオブジェクトのメンバを初期化する
00102 framebufferSize = std::array<int, 2>{ 0, 0 };
00103 framebufferChannels = 0;
00104 attachment = GL_COLOR_ATTACHMENT0;
00105 }
00106
00107 //
00108 // フレームバッファオブジェクトを作成する
00109 //
00110 void Framebuffer::create(GLsizei width, GLsizei height, int channels)
00111 {
00112     // 既存のテクスチャを破棄して新しいテクスチャを作成する
00113     Texture::create(width, height, channels);
00114
00115     // まだメッシュの頂点配列オブジェクトが作られていなければ作る
00116     if (!mesh) mesh = std::make_shared<Mesh>();
00117
00118     // 指定したサイズがフレームバッファオブジェクトのサイズと同じなら何もしない
00119     if (width == framebufferSize[0] && height == framebufferSize[1]
00120         && channels == framebufferChannels) return;
00121
00122     // フレームバッファオブジェクトのサイズとチャンネル数を記録する
00123     framebufferSize = std::array<int, 2>{ width, height };
00124     framebufferChannels = channels;
00125
00126     // 以前のフレームバッファオブジェクトを削除する
00127     glDeleteFramebuffers(1, &framebufferName);
00128
00129     // フレームバッファオブジェクトを作成する
00130     glGenFramebuffers(1, &framebufferName);
00131     glBindFramebuffer(GL_FRAMEBUFFER, framebufferName);
00132     glFramebufferTexture2D(GL_FRAMEBUFFER, attachment, GL_TEXTURE_2D, getTextureName(), 0);
00133     glDrawBuffers(1, &attachment);
00134     glReadBuffer(attachment);
00135     glBindFramebuffer(GL_FRAMEBUFFER, 0);
00136 }
00137
00138 //
00139 // フレームバッファオブジェクトをコピーする
00140 //
00141 void Framebuffer::copy(const Buffer& framebuffer) noexcept
00142 {
00143     // コピー元と同じサイズの空のフレームバッファオブジェクトを作る
00144     Framebuffer::create(framebuffer.getWidth(), framebuffer.getHeight(),
00145         framebuffer.getChannels());
00146
00147     // 作成したフレームバッファオブジェクトのバッファにコピーする
00148     copyBuffer(framebuffer);
00149 }
00150
00151 //
00152 // レンダリング先をフレームバッファオブジェクトに切り替える
00153 //
00154 void Framebuffer::bindFramebuffer()
00155 {
00156     // 描画先をフレームバッファオブジェクトに切り替える
00157     glBindFramebuffer(GL_FRAMEBUFFER, framebufferName);
00158
00159     // ビューポートをフレームバッファオブジェクトに合わせる
00160     glViewport(0, 0, framebufferSize[0], framebufferSize[1]);
00161 }
00162
00163 //
00164 // レンダリング先を通常のフレームバッファに戻す
00165 //
00166 void Framebuffer::unbindFramebuffer() const
00167 {
00168     // 描画先を通常のフレームバッファに戻す
00169     glBindFramebuffer(GL_FRAMEBUFFER, 0);
00170
00171     // 読み書きを通常のフレームバッファのバックバッファに対して行う
00172     static constexpr GLenum bufs{
00173 #if defined(GL_GLES_PROTOTYPES)
00174         GL_BACK
00175 #else
00176         GL_BACK_LEFT
00177 #endif
00178     };
00179     glDrawBuffers(1, &bufs);
00180     glReadBuffer(GL_BACK);

```

```

00181 }
00182
00183 //
00184 // テクスチャを展開してフレームバッファオブジェクトを更新する
00185 //
00186 void Framebuffer::update(const std::array<int, 2>& size)
00187 {
00188     // 描画先をフレームバッファオブジェクトに切り替える
00189     glBindFramebuffer();
00190
00191     // テクスチャをフレームバッファオブジェクトに展開する
00192     mesh->drawMesh(size);
00193
00194     // 描画先を通常のフレームバッファに戻す
00195     glBindFramebuffer();
00196 }
00197
00198 //
00199 // テクスチャを展開してフレームバッファオブジェクトを更新する
00200 //
00201 void Framebuffer::update(const std::array<int, 2>& size, const Texture& frame, int unit)
00202 {
00203     // 展開するするテクスチャを指定する
00204     frame.bindTexture(unit);
00205
00206     // テクスチャをフレームバッファオブジェクトに展開する
00207     update(size);
00208
00209     // 展開するテクスチャの指定を解除する
00210     frame.unbindTexture();
00211 }
00212
00213 //
00214 // フレームバッファオブジェクトの表示
00215 //
00216 void Framebuffer::show(GLsizei width, GLsizei height) const
00217 {
00218     // フレームバッファオブジェクトの縦横比
00219     const auto f{ static_cast<float>(framebufferSize[0] * height) };
00220
00221     // ウィンドウの表示領域の縦横比
00222     const auto d{ static_cast<float>(framebufferSize[1] * width) };
00223
00224     // 実際に描画する領域
00225     GLint dx0, dy0, dx1, dy1;
00226
00227     // 表示領域の右上端の位置を求める
00228     --width;
00229     --height;
00230
00231     // フレームバッファオブジェクトの縦横比が大きかったら
00232     if (f > d)
00233     {
00234         // ウィンドウの表示領域の高さを求める
00235         const auto h{ static_cast<GLint>(d / getWidth() + 0.5f) };
00236
00237         // 表示が横長なので表示領域の横幅いっぱいに表示する
00238         dx0 = 0;
00239         dx1 = width;
00240
00241         // 高さは縦横比を維持して描画する領域の中央に描く
00242         dy0 = (height - h) / 2;
00243         dy1 = dy0 + h;
00244     }
00245     else
00246     {
00247         // ウィンドウの表示領域の幅を求める
00248         const auto w{ static_cast<GLint>(f / getHeight() + 0.5f) };
00249
00250         // 表示が縦長なので表示領域の高さいっぱいに表示する
00251         dy0 = 0;
00252         dy1 = height;
00253
00254         // 横幅は縦横比を維持して描画する領域の中央に描く
00255         dx0 = (width - w) / 2;
00256         dx1 = dx0 + w;
00257     }
00258
00259     // フレームバッファオブジェクトを読み込み元にする
00260     glBindFramebuffer(GL_READ_FRAMEBUFFER, framebufferName);
00261     glReadBuffer(GL_READ_FRAMEBUFFER);
00262
00263     // 現在のフレームバッファを背景色で塗りつぶす
00264     glClear(GL_COLOR_BUFFER_BIT);
00265
00266     // フレームバッファオブジェクトの内容を通常のフレームバッファに書き込む
00267     glBlitFramebuffer(0, 0, getWidth() - 1, getHeight() - 1,

```

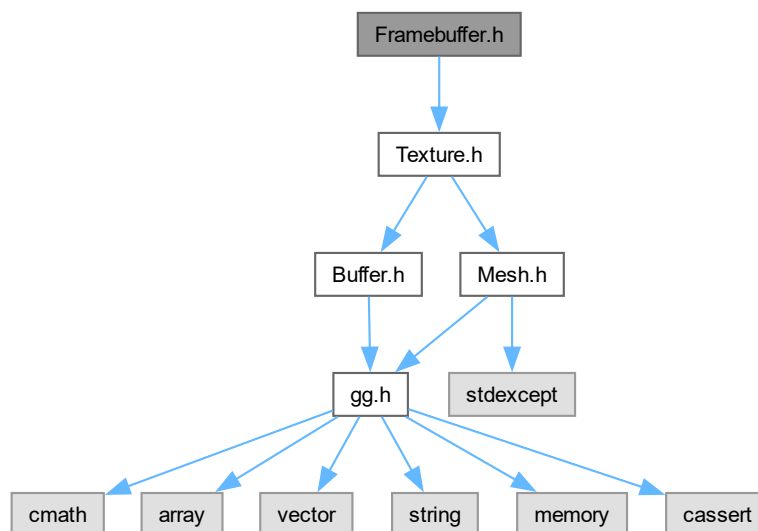
```

00268     dx0, dy1, dx1, dy0, GL_COLOR_BUFFER_BIT, GL_NEAREST);
00269
00270     // 読み込み元を通常のフレームバッファに戻す
00271     glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
00272     glReadBuffer(GL_BACK);
00273 }

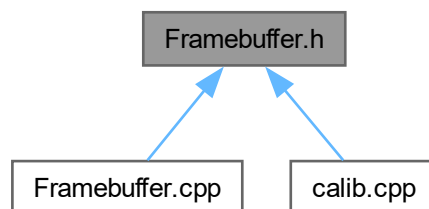
```

9.49 Framebuffer.h ファイル

#include "Texture.h"
 Framebuffer.h の依存先関係図:



被依存関係図:



クラス

- class [Framebuffer](#)

9.49.1 詳解

フレームバッファオブジェクトクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Framebuffer.h](#) に定義があります。

9.50 Framebuffer.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // テクスチャ
00012 #include "Texture.h"
00013
00017 class Framebuffer : public Texture
00018 {
00020     std::array<int, 2> framebufferSize{ 0, 0 };
00021
00023     int framebufferChannels{ 0 };
00024
00026     GLuint framebufferName{ 0 };
00027
00029     GLenum attachment{ GL_COLOR_ATTACHMENT0 };
00030
00031 public:
00032
00036     Framebuffer() = default;
00037
00046     Framebuffer(GLsizei width, GLsizei height, int channels = 3,
00047                 GLenum attachment = GL_COLOR_ATTACHMENT0);
00048
00054     Framebuffer(const Framebuffer& framebuffer);
00055
00061     Framebuffer(Framebuffer&& framebuffer) noexcept
00062         : Texture{ std::move(framebuffer) }
00063         , framebufferSize{ framebuffer.framebufferSize }
00064         , framebufferChannels{ framebuffer.framebufferChannels }
00065         , framebufferName{ framebuffer.framebufferName }
00066         , attachment{ framebuffer.attachment }
00067     {
00068         framebuffer.framebufferSize = { 0, 0 };
00069         framebuffer.framebufferChannels = 0;
00070
00071         // ムーブ元のデストラクタでフレームバッファが削除されないよう 0 にする
00072         framebuffer.framebufferName = 0;
00073         framebuffer.attachment = GL_COLOR_ATTACHMENT0;
00074     }
00075
00079     virtual ~Framebuffer();
00080
00087     Framebuffer& operator=(const Framebuffer& framebuffer);
00088
00095     Framebuffer& operator=(Framebuffer&& framebuffer) noexcept;
00096
00109     virtual void create(GLsizei width, GLsizei height, int channels);
00110
00124     virtual void copy(const Buffer& framebuffer) noexcept;
00125
00129     virtual void discard();
00130
00136     auto getFramebufferName() const
00137     {
```

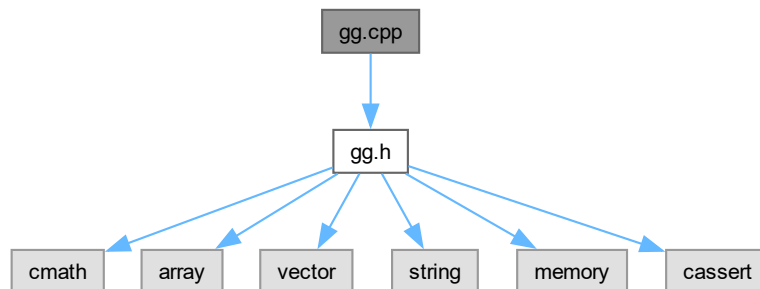
```

00138     return framebufferName;
00139 }
00140
00146 virtual const std::array<GLsizei, 2>& getSize() const
00147 {
00148     return framebufferSize;
00149 }
00150
00156 virtual int getChannels() const
00157 {
00158     return framebufferChannels;
00159 }
00160
00170 void bindFramebuffer();
00171
00181 void unbindFramebuffer() const;
00182
00193 void update(const std::array<int, 2>& size);
00194
00210 void update(const std::array<int, 2>& size, const Texture& frame, int unit = 0);
00211
00218 void show(GLsizei width, GLsizei height) const;
00219 };

```

9.51 gg.cpp ファイル

#include "gg.h"
gg.cpp の依存先関係図:



9.51.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の定義.

著者

Kohe Tokoi

日付

July 17, 2025

[gg.cpp](#) に定義があります。

9.52 gg.cpp

[詳解]

```

00001  /*
00002
00003  ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005  Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007  Permission is hereby granted, free of charge, to any person obtaining a copy
00008  of this software and associated documentation files (the "Software"), to deal
00009  in the Software without restriction, including without limitation the rights
00010  to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011  copies or substantial portions of the Software.
00012
00013  The above copyright notice and this permission notice shall be included in
00014  all copies or substantial portions of the Software.
00015
00016  THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017  IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018  FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019  KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020  AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021  CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023  */
00024
00025  #include "gg.h"
00026
00027  // 標準ライブラリ
00028  #include <cmath>
00029  #include <cstdlib>
00030  #include <iostream>
00031  #include <fstream>
00032  #include <sstream>
00033  #include <limits>
00034  #include <map>
00035  #if !defined(_WIN32)
00036  #   include <unistd.h>
00037  #   include <limits.h>
00038  #endif
00039  #if defined(_ANDROID_)
00040  #   include <android/asset_manager.h>
00041  void* ggAndroidAssetManager{ nullptr };
00042  #endif
00043
00044  #define READ_TEXTURE_COORDINATE_FROM_OBJ 0
00045
00046  // Windows (Visual Studio) のとき
00047  #if defined(_MSC_VER)
00048  // リリースビルドではコンソールを出さない
00049  #   if !defined(_DEBUG)
00050  #       pragma comment(linker, "/subsystem:windows /entry:mainCRTStartup")
00051  #   endif
00052  // リンクするライブラリ
00053  #   pragma comment(lib, "lib\\GLFW3-PLATFORM\\GLFW3-CONFIGURATION\\glfw3.lib")
00054
00055  // For VC++ MFC Convert UTF-8 to TCHAR, or Convert TCHAR to UTF-8. VC++ MFC用 UTF-8⇔TCHARの変換処理
00056  // Original author: mt-u
00057  // Copyright (c) 2013 mt-u
00058  // https://gist.github.com/mt-u/6878251
00059  // Modified by: Kohe Tokoi
00060
00061  // UTF-8 文字列を CString に変換する
00062  //
00063  pathString Utf8ToTChar(const std::string& string)
00064  {
00065  // UTF-8 文字列を UTF-16 に変換した後の文字列の長さを求める
00066  const INT length{ MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, NULL, 0) };
00067
00068  // 変換結果の格納に必要な長さのメモリを確保する
00069  std::vector<WCHAR> utf16(length);
00070
00071  // UTF-8 文字列を UTF-16 に変換する
00072  MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, utf16.data(), length);
00073
00074  // 変換した文字列を CString にして返す
00075  return CString{ utf16.data(), static_cast<int>(wcslen(utf16.data())) };
00076  }

```

```

00092
00093 //
00094 // CString を UTF-8 文字列に変換する
00095 //
00096 std::string TCharToUtf8(const pathString& cstring)
00097 {
00098     // 与えられた文字列を一旦 UTF-16 に変換する
00099     CStringW cstringw{ cstring };
00100
00101     // UTF-16 を UTF-8 に変換した後の文字列の長さを求める
00102     const INT length{ WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, NULL, 0, NULL, NULL) };
00103
00104     // 変換結果の格納に必要な長さのメモリを確保する
00105     std::vector<CHAR> utf8(length);
00106
00107     // UTF-16 を UTF-8 に変換する
00108     WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, utf8.data(), length, NULL, NULL);
00109
00110     // 変換した文字列を std::string にして返す
00111     return std::string{ utf8.data(), strlen(utf8.data()) };
00112 }
00113 #endif
00114
00115 // OpenGL 3.2 の API のエントリポイント
00116 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00117 PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00118 PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00119 PFNGLACTIVETEXTUREPROC glActiveTexture;
00120 PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00121 PFNGLATTACHSHADERPROC glAttachShader;
00122 PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00123 PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00124 PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00125 PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00126 PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00127 PFNGLBEGINQUERYPROC glBeginQuery;
00128 PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00129 PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00130 PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00131 PFNGLBINDBUFFERPROC glBindBuffer;
00132 PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00133 PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00134 PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00135 PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00136 PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00137 PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00138 PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00139 PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00140 PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00141 PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00142 PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00143 PFNGLBINDSAMPLERPROC glBindSampler;
00144 PFNGLBINDSAMPLERSPROC glBindSamplers;
00145 PFNGLBINDTEXTUREPROC glBindTexture;
00146 PFNGLBINDTEXTURESPROC glBindTextures;
00147 PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00148 PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00149 PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00150 PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00151 PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00152 PFNGLBLENDARRIERKHRPROC glBlendBarrierKHR;
00153 PFNGLBLENDARRIERNVPROC glBlendBarrierNV;
00154 PFNGLBLENDCOLORPROC glBlendColor;
00155 PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00156 PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00157 PFNGLBLEND EQUATIONNPROC glBlendEquation;
00158 PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00159 PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00160 PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00161 PFNGLBLENDFUNCTIONIARBPROC glBlendFunciARB;
00162 PFNGLBLENDFUNCTIONIPROC glBlendFunci;
00163 PFNGLBLENDFUNCTIONPROC glBlendFunc;
00164 PFNGLBLENDFUNCTIONSEPARATEIARBPROC glBlendFuncSeparateiARB;
00165 PFNGLBLENDFUNCTIONSEPARATEIPROC glBlendFuncSeparatei;
00166 PFNGLBLENDFUNCTIONSEPARATEPROC glBlendFuncSeparate;
00167 PFNGLBLENDPARAMETERINVPROC glBlendParameteriNV;
00168 PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00169 PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00170 PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00171 PFNGLBUFFERDATAPROC glBufferData;
00172 PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00173 PFNGLBUFFERSTORAGEPROC glBufferStorage;
00174 PFNGLBUFFERSUBDATAPROC glBufferSubData;
00175 PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00176 PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00177 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00178 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;

```

```
00179 PFNGLCLAMPColorPROC glClampColor;
00180 PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00181 PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00182 PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00183 PFNGLCLEARBUFFERIVPROC glClearBufferiv;
00184 PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00185 PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00186 PFNGLCLEARColorPROC glClearColor;
00187 PFNGLCLEARDEPTHFPROC glClearDepthf;
00188 PFNGLCLEARDEPTHPROC glClearDepth;
00189 PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00190 PFNGLCLEARNAMEDBUFFERDATAAPROC glClearNamedBufferData;
00191 PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00192 PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00193 PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00194 PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00195 PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glClearNamedFramebufferiv;
00196 PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glClearNamedFramebufferuiv;
00197 PFNGLCLEARPROC glClear;
00198 PFNGLCLEARSTENCILPROC glClearStencil;
00199 PFNGLCLEARTEXIMAGEPROC glClearTexImage;
00200 PFNGLCLEARTEXSUBIMAGEPROC glClearTexSubImage;
00201 PFNGLCLIENTATTRIBDEFAULTTEXTPROC glClientAttribDefaultEXT;
00202 PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
00203 PFNGLCLIPCONTROLPROC glClipControl;
00204 PFNGLCOLORFORMATNVPROC glColorFormatNV;
00205 PFNGLCOLORMASKIPROC glColorMaski;
00206 PFNGLCOLORMASKPROC glColorMask;
00207 PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00208 PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00209 PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00210 PFNGLCOMPILESHADERPROC glCompileShader;
00211 PFNGLCOMPRESSEDMULTITEXIMAGE1DXTPROC glCompressedMultiTexImage1DXT;
00212 PFNGLCOMPRESSEDMULTITEXIMAGE2DXTPROC glCompressedMultiTexImage2DXT;
00213 PFNGLCOMPRESSEDMULTITEXIMAGE3DXTPROC glCompressedMultiTexImage3DXT;
00214 PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DXTPROC glCompressedMultiTexSubImage1DXT;
00215 PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DXTPROC glCompressedMultiTexSubImage2DXT;
00216 PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DXTPROC glCompressedMultiTexSubImage3DXT;
00217 PFNGLCOMPRESSEDTEXIMAGE1DPROC glCompressedTexImage1D;
00218 PFNGLCOMPRESSEDTEXIMAGE2DPROC glCompressedTexImage2D;
00219 PFNGLCOMPRESSEDTEXIMAGE3DPROC glCompressedTexImage3D;
00220 PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glCompressedTexSubImage1D;
00221 PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glCompressedTexSubImage2D;
00222 PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glCompressedTexSubImage3D;
00223 PFNGLCOMPRESSEDTEXTUREIMAGE1DXTPROC glCompressedTextureImage1DXT;
00224 PFNGLCOMPRESSEDTEXTUREIMAGE2DXTPROC glCompressedTextureImage2DXT;
00225 PFNGLCOMPRESSEDTEXTUREIMAGE3DXTPROC glCompressedTextureImage3DXT;
00226 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DXTPROC glCompressedTextureSubImage1DXT;
00227 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00228 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DXTPROC glCompressedTextureSubImage2DXT;
00229 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00230 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DXTPROC glCompressedTextureSubImage3DXT;
00231 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00232 PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00233 PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00234 PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00235 PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00236 PFNGLCOPYMULTITEXIMAGE1DXTPROC glCopyMultiTexImage1DXT;
00237 PFNGLCOPYMULTITEXIMAGE2DXTPROC glCopyMultiTexImage2DXT;
00238 PFNGLCOPYMULTITEXSUBIMAGE1DXTPROC glCopyMultiTexSubImage1DXT;
00239 PFNGLCOPYMULTITEXSUBIMAGE2DXTPROC glCopyMultiTexSubImage2DXT;
00240 PFNGLCOPYMULTITEXSUBIMAGE3DXTPROC glCopyMultiTexSubImage3DXT;
00241 PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00242 PFNGLCOPYPATHNVPROC glCopyPathNV;
00243 PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00244 PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00245 PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00246 PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00247 PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00248 PFNGLCOPYTEXTUREIMAGE1DXTPROC glCopyTextureImage1DXT;
00249 PFNGLCOPYTEXTUREIMAGE2DXTPROC glCopyTextureImage2DXT;
00250 PFNGLCOPYTEXTURESUBIMAGE1DXTPROC glCopyTextureSubImage1DXT;
00251 PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00252 PFNGLCOPYTEXTURESUBIMAGE2DXTPROC glCopyTextureSubImage2DXT;
00253 PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00254 PFNGLCOPYTEXTURESUBIMAGE3DXTPROC glCopyTextureSubImage3DXT;
00255 PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00256 PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00257 PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00258 PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00259 PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00260 PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00261 PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00262 PFNGLCREATEBUFFERSPROC glCreateBuffers;
00263 PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00264 PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00265 PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
```

```
00266 PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00267 PFNGLCREATEPROGRAMPROC glCreateProgram;
00268 PFNGLCREATEQUERIESPROC glCreateQueries;
00269 PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00270 PFNGLCREATESAMPLERSPROC glCreateSamplers;
00271 PFNGLCREATESHADERPROC glCreateShader;
00272 PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00273 PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00274 PFNGLCREATESTATESNVPROC glCreateStatesNV;
00275 PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLEventARB;
00276 PFNGLCREATETEXTURESPROC glCreateTextures;
00277 PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00278 PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00279 PFNGLCULLFACEPROC glCullFace;
00280 PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00281 PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00282 PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00283 PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00284 PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00285 PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00286 PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00287 PFNGLDELETECOMMANDLISTSNVPROC glDeleteCommandListsNV;
00288 PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00289 PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00290 PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00291 PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00292 PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00293 PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00294 PFNGLDELETEPROGRAMPROC glDeleteProgram;
00295 PFNGLDELETEQUERIESPROC glDeleteQueries;
00296 PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00297 PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00298 PFNGLDELETESHADERPROC glDeleteShader;
00299 PFNGLDELETESSTATESNVPROC glDeleteStatesNV;
00300 PFNGLDELETESYNCPROC glDeleteSync;
00301 PFNGLDELETETEXTURESPROC glDeleteTextures;
00302 PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00303 PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00304 PFNGLDEPTHFUNCPROC glDepthFunc;
00305 PFNGLDEPTHMASKPROC glDepthMask;
00306 PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00307 PFNGLDEPTHRANGEFPROC glDepthRangef;
00308 PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00309 PFNGLDEPTHRANGEPROC glDepthRange;
00310 PFNGLDETACHSHADERPROC glDetachShader;
00311 PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00312 PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00313 PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00314 PFNGLDISABLEIPROC glDisablei;
00315 PFNGLDISABLEPROC glDisable;
00316 PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00317 PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00318 PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00319 PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00320 PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00321 PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00322 PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00323 PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00324 PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00325 PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00326 PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00327 PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00328 PFNGLDRAWARRAYSPROC glDrawArrays;
00329 PFNGLDRAWBUFFERPROC glDrawBuffer;
00330 PFNGLDRAWBUFFERSPROC glDrawBuffers;
00331 PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00332 PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00333 PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00334 PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00335 PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00336 PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00337 PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00338 PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00339 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC glDrawElementsInstancedBaseVertexBaseInstance;
00340 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00341 PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00342 PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00343 PFNGLDRAWELEMENTSPROC glDrawElements;
00344 PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00345 PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00346 PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00347 PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00348 PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00349 PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00350 PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00351 PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00352 PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
```

```
00353 PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00354 PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00355 PFNGLENABLEIPROC glEnablei;
00356 PFNGLENABLEPROC glEnable;
00357 PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00358 PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00359 PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00360 PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00361 PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00362 PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00363 PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00364 PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00365 PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00366 PFNGLENDQUERYPROC glEndQuery;
00367 PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00368 PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00369 PFNGLFENCESYNCPROC glFenceSync;
00370 PFNGLFINISHPROC glFinish;
00371 PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00372 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00373 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00374 PFNGLFLUSHPROC glFlush;
00375 PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00376 PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00377 PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00378 PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00379 PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00380 PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00381 PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00382 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00383 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00384 PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00385 PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00386 PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00387 PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00388 PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00389 PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00390 PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00391 PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00392 PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00393 PFNGLFRONTFACEPROC glFrontFace;
00394 PFNGLGENBUFFERSPROC glGenBuffers;
00395 PFNGLGENERATEMIPMAPPROC glGenerateMipmap;
00396 PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenerateMultiTexMipmapEXT;
00397 PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenerateTextureMipmapEXT;
00398 PFNGLGENERATETEXTUREMIPMAPPROC glGenerateTextureMipmap;
00399 PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00400 PFNGLGENPATHSNVPROC glGenPathsNV;
00401 PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00402 PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00403 PFNGLGENQUERIESPROC glGenQueries;
00404 PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00405 PFNGLGENSAMPLERSPROC glGenSamplers;
00406 PFNGLGENTEXTURESPROC glGenTextures;
00407 PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00408 PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00409 PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00410 PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00411 PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00412 PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00413 PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00414 PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00415 PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00416 PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00417 PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00418 PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00419 PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00420 PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00421 PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00422 PFNGLGETBOOLEANIVPROC glGetBooleani.v;
00423 PFNGLGETBOOLEANVPROC glGetBooleany;
00424 PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00425 PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00426 PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00427 PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00428 PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00429 PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00430 PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00431 PFNGLGETCOMPRESSEDTEXTIMAGEPROC glGetCompressedTexImage;
00432 PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00433 PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00434 PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00435 PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00436 PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00437 PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00438 PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00439 PFNGLGETDOUBLEIVEXTPROC glGetDoublei.vEXT;
```

```
00440 PFNGLGETDOUBLEI.VPROC glGetDoublei.v;
00441 PFNGLGETDOUBLEVPROC glGetDoublev;
00442 PFNGLGETERRORPROC glGetError;
00443 PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00444 PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00445 PFNGLGETFLOATI.VEXTPROC glGetFloati.vEXT;
00446 PFNGLGETFLOATI.VPROC glGetFloati.v;
00447 PFNGLGETFLOATVPROC glGetFloatv;
00448 PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00449 PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00450 PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00451 PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00452 PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00453 PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00454 PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00455 PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00456 PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00457 PFNGLGETINTEGER64I.VPROC glGetInteger64i.v;
00458 PFNGLGETINTEGER64VPROC glGetInteger64v;
00459 PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00460 PFNGLGETINTEGERI.VPROC glGetIntegeri.v;
00461 PFNGLGETINTEGERUI64I.VNVPROC glGetIntegerui64i.vNV;
00462 PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00463 PFNGLGETINTEGERVPROC glGetIntegerv;
00464 PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00465 PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00466 PFNGLGETINTERNALFORMATSAMPLEIIVNVPROC glGetInternalformatSampleivNV;
00467 PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00468 PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00469 PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00470 PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00471 PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00472 PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00473 PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00474 PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00475 PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00476 PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00477 PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameteriivEXT;
00478 PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameteriuiEXT;
00479 PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00480 PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00481 PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00482 PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00483 PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00484 PFNGLGETNAMEDBUFFERPOINTERVEXTPROC glGetNamedBufferPointervEXT;
00485 PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00486 PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00487 PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00488 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC glGetNamedFramebufferAttachmentParameterivEXT;
00489 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00490 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00491 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00492 PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00493 PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00494 PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00495 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameteriivEXT;
00496 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameteriuiEXT;
00497 PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00498 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00499 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00500 PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00501 PFNGLGETNAMEDSTRINGIARBPROC glGetNamedStringiARB;
00502 PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00503 PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00504 PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00505 PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00506 PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00507 PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvarB;
00508 PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00509 PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvarB;
00510 PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00511 PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64varB;
00512 PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00513 PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00514 PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64varB;
00515 PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivarB;
00516 PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00517 PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00518 PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00519 PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00520 PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00521 PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00522 PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00523 PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00524 PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00525 PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00526 PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
```



```
00527 PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00528 PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00529 PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00530 PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00531 PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00532 PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00533 PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00534 PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00535 PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00536 PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00537 PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00538 PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00539 PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00540 PFNGLGETPOINTERIVEXTPROC glGetPointerivEXT;
00541 PFNGLGETPOINTERVPROC glGetPointerv;
00542 PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00543 PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00544 PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00545 PFNGLGETPROGRAMIVPROC glGetProgramiv;
00546 PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00547 PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00548 PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00549 PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00550 PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00551 PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00552 PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00553 PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00554 PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00555 PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00556 PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00557 PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00558 PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00559 PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00560 PFNGLGETQUERYIVPROC glGetQueryiv;
00561 PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00562 PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00563 PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00564 PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00565 PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00566 PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00567 PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00568 PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00569 PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00570 PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00571 PFNGLGETSHADERIVPROC glGetShaderiv;
00572 PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00573 PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00574 PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00575 PFNGLGETSTRINGIPROC glGetStringi;
00576 PFNGLGETSTRINGPROC glGetString;
00577 PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00578 PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00579 PFNGLGETSYNCPROC glGetSynciv;
00580 PFNGLGETTEXIMAGEPROC glGetTexImage;
00581 PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00582 PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00583 PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00584 PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00585 PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00586 PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00587 PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00588 PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00589 PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00590 PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00591 PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00592 PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00593 PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00594 PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00595 PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00596 PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00597 PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00598 PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00599 PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00600 PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00601 PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00602 PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00603 PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00604 PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00605 PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00606 PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64v;
00607 PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00608 PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00609 PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00610 PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00611 PFNGLGETUNIFORMDVPROC glGetUniformdv;
00612 PFNGLGETUNIFORMFVPROC glGetUniformfv;
00613 PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
```

```
00614 PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00615 PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00616 PFNGLGETUNIFORMIVPROC glGetUniformiv;
00617 PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00618 PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00619 PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00620 PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00621 PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00622 PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00623 PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00624 PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00625 PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayInteger_vEXT;
00626 PFNGLGETVERTEXARRAYIVPROC glGetVertexArrayiv;
00627 PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00628 PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointer_vEXT;
00629 PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00630 PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00631 PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00632 PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00633 PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00634 PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribLdv;
00635 PFNGLGETVERTEXATTRIBLI64VNVPROC glGetVertexAttribLi64vNV;
00636 PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00637 PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00638 PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointer_v;
00639 PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00640 PFNGLHINTPROC glHint;
00641 PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00642 PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00643 PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00644 PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00645 PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00646 PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00647 PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00648 PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00649 PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00650 PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00651 PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00652 PFNGLISBUFFERPROC glIsBuffer;
00653 PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00654 PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00655 PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00656 PFNGLISENABLEDIPROC glIsEnabledi;
00657 PFNGLISENABLEDIPROC glIsEnabled;
00658 PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00659 PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00660 PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00661 PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00662 PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00663 PFNGLISPATHTNVPROC glIsPathNV;
00664 PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00665 PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00666 PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00667 PFNGLISPROGRAMPROC glIsProgram;
00668 PFNGLISQUERYPROC glIsQuery;
00669 PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00670 PFNGLISSAMPLERPROC glIsSampler;
00671 PFNGLISSHADERPROC glIsShader;
00672 PFNGLISSTATENVPROC glIsStateNV;
00673 PFNGLISSYNCPROC glIsSync;
00674 PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00675 PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00676 PFNGLISTEXTUREPROC glIsTexture;
00677 PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00678 PFNGLISVERTEXARRAYPROC glIsVertexArray;
00679 PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00680 PFNGLLINEWIDTHPROC glLineWidth;
00681 PFNGLLINKPROGRAMPROC glLinkProgram;
00682 PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC glListDrawCommandsStatesClientNV;
00683 PFNGLLOGICOPPROC glLogicOp;
00684 PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00685 PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00686 PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00687 PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00688 PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00689 PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00690 PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00691 PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00692 PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00693 PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00694 PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00695 PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00696 PFNGLMAPBUFFERPROC glMapBuffer;
00697 PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00698 PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00699 PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00700 PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
```

```
00701 PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00702 PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00703 PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00704 PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00705 PFNGLMATRIXLOADDEXTPROC glMatrixLoadEXT;
00706 PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00707 PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00708 PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00709 PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00710 PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00711 PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00712 PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00713 PFNGLMATRIXMULTDEXTPROC glMatrixMultdEXT;
00714 PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00715 PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00716 PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00717 PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00718 PFNGLMATRIXORTHOEXTPROC glMatrixOrthoEXT;
00719 PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00720 PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00721 PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00722 PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00723 PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00724 PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00725 PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00726 PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00727 PFNGLMAXSHADERCOMPILETHREADSARBPROC glMaxShaderCompilerThreadsARB;
00728 PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00729 PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00730 PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00731 PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00732 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00733 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00734 PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00735 PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00736 PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00737 PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00738 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00739 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00740 PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00741 PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00742 PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00743 PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00744 PFNGLMULTITEXCOORDPOINTEREXTPROC glMultiTexCoordPointerEXT;
00745 PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00746 PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00747 PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00748 PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00749 PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00750 PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00751 PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00752 PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00753 PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00754 PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00755 PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00756 PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00757 PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00758 PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00759 PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00760 PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameterIEXT;
00761 PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameterIivEXT;
00762 PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameterIuivEXT;
00763 PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00764 PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00765 PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00766 PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00767 PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00768 PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00769 PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00770 PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00771 PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00772 PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00773 PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00774 PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00775 PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00776 PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00777 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00778 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00779 PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameterIEXT;
00780 PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameterI;
00781 PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00782 PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00783 PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00784 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00785 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00786 PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00787 PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
```

```
00788 PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00789 PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00790 PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00791 PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00792 PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00793 PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00794 PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00795 PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00796 PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00797 PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00798 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00799 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00800 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00801 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00802 PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00803 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC glNamedProgramLocalParametersI4ivEXT;
00804 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00805 PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00806 PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00807 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEEXTPROC
glNamedRenderbufferStorageMultisampleCoverageEXT;
00808 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00809 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00810 PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00811 PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00812 PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00813 PFNGLOBJCTLABELPROC glObjectLabel;
00814 PFNGLOBJECTPTRLABELPROC glObjectPtrLabel;
00815 PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00816 PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00817 PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00818 PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00819 PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00820 PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00821 PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00822 PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00823 PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00824 PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00825 PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00826 PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00827 PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00828 PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00829 PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00830 PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00831 PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00832 PFNGLPATHSTRINGNVPROC glPathStringNV;
00833 PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00834 PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00835 PFNGLPAUSESETTRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00836 PFNGLPIXELSTOREFPROC glPixelStoref;
00837 PFNGLPIXELSTOREIPROC glPixelStorei;
00838 PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00839 PFNGLPOINTPARAMETERFPROC glPointParameterf;
00840 PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00841 PFNGLPOINTPARAMETERIPROC glPointParameteri;
00842 PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00843 PFNGLPOINTSIZEPROC glPointSize;
00844 PFNGLPOLYGONMODEPROC glPolygonMode;
00845 PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00846 PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00847 PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00848 PFNGLPOPGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00849 PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00850 PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00851 PFNGLPROGRAMBINARYPROC glProgramBinary;
00852 PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00853 PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00854 PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00855 PFNGLPROGRAMUNIFORM1DEXTPROC glProgramUniform1dEXT;
00856 PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00857 PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00858 PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00859 PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00860 PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00861 PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00862 PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00863 PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00864 PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00865 PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00866 PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00867 PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00868 PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00869 PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00870 PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00871 PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00872 PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00873 PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
```

```
00874 PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00875 PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00876 PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00877 PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00878 PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00879 PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00880 PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00881 PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00882 PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00883 PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00884 PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00885 PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00886 PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00887 PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00888 PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00889 PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00890 PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00891 PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00892 PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00893 PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00894 PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00895 PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00896 PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00897 PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00898 PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00899 PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00900 PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00901 PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00902 PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00903 PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00904 PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00905 PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00906 PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00907 PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00908 PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00909 PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00910 PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00911 PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00912 PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00913 PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00914 PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00915 PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00916 PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00917 PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00918 PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00919 PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00920 PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00921 PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00922 PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00923 PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00924 PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00925 PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00926 PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00927 PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00928 PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00929 PFNGLPROGRAMUNIFORM4DVEXTPROC glProgramUniform4dvEXT;
00930 PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00931 PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00932 PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00933 PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00934 PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00935 PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00936 PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00937 PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00938 PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00939 PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00940 PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00941 PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00942 PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00943 PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00944 PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00945 PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00946 PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00947 PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00948 PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00949 PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00950 PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00951 PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00952 PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00953 PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00954 PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00955 PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00956 PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00957 PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00958 PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00959 PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00960 PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
```

```
00961 PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00962 PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00963 PFNGLPROGRAMUNIFORMMATRIX2X4DVECTPROC glProgramUniformMatrix2x4dvEXT;
00964 PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00965 PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00966 PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00967 PFNGLPROGRAMUNIFORMMATRIX3DVECTPROC glProgramUniformMatrix3dvEXT;
00968 PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00969 PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00970 PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00971 PFNGLPROGRAMUNIFORMMATRIX3X2DVECTPROC glProgramUniformMatrix3x2dvEXT;
00972 PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00973 PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00974 PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00975 PFNGLPROGRAMUNIFORMMATRIX3X4DVECTPROC glProgramUniformMatrix3x4dvEXT;
00976 PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00977 PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00978 PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00979 PFNGLPROGRAMUNIFORMMATRIX4DVECTPROC glProgramUniformMatrix4dvEXT;
00980 PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00981 PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00982 PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00983 PFNGLPROGRAMUNIFORMMATRIX4X2DVECTPROC glProgramUniformMatrix4x2dvEXT;
00984 PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00985 PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00986 PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00987 PFNGLPROGRAMUNIFORMMATRIX4X3DVECTPROC glProgramUniformMatrix4x3dvEXT;
00988 PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00989 PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00990 PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00991 PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00992 PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00993 PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00994 PFNGLPUSHCLIENTATTRIBDEFAULTTEXTPROC glPushClientAttribDefaultEXT;
00995 PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00996 PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00997 PFNGLQUERYCOUNTERPROC glQueryCounter;
00998 PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00999 PFNGLREADBUFFERPROC glReadBuffer;
01000 PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
01001 PFNGLREADNPIXELSPROC glReadnPixels;
01002 PFNGLREADPIXELSPROC glReadPixels;
01003 PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
01004 PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
01005 PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
01006 PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
01007 PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
01008 PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01009 PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01010 PFNGLSAMPLEMASKIPROC glSampleMaski;
01011 PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01012 PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
01013 PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01014 PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01015 PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01016 PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01017 PFNGLSCISSORARRAYVPROC glScissorArrayv;
01018 PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01019 PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01020 PFNGLSCISSORPROC glScissor;
01021 PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01022 PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01023 PFNGLSHADERBINARYPROC glShaderBinary;
01024 PFNGLSHADERSOURCEPROC glShaderSource;
01025 PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01026 PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01027 PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01028 PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01029 PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01030 PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01031 PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01032 PFNGLSTENCILFUNCPROC glStencilFunc;
01033 PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01034 PFNGLSTENCILMASKPROC glStencilMask;
01035 PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01036 PFNGLSTENCILOPPROC glStencilOp;
01037 PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01038 PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01039 PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01040 PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01041 PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01042 PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01043 PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01044 PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01045 PFNGLTEXBUFFERARBPROC glTexBufferARB;
01046 PFNGLTEXBUFFERPROC glTexBuffer;
01047 PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
```

```
01048 PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01049 PFNGLTEXIMAGE1DPROC glTexImage1D;
01050 PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01051 PFNGLTEXIMAGE2DPROC glTexImage2D;
01052 PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01053 PFNGLTEXIMAGE3DPROC glTexImage3D;
01054 PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01055 PFNGLTEXPARAMETERFPROC glTexParameterf;
01056 PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01057 PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01058 PFNGLTEXPARAMETERIPROC glTexParameterI;
01059 PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01060 PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01061 PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01062 PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01063 PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01064 PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01065 PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01066 PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01067 PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01068 PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01069 PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01070 PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01071 PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01072 PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01073 PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01074 PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01075 PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01076 PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01077 PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01078 PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01079 PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01080 PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01081 PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01082 PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01083 PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01084 PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01085 PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01086 PFNGLTEXTUREPARAMETERIPROC glTextureParameterI;
01087 PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01088 PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01089 PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01090 PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01091 PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01092 PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01093 PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01094 PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01095 PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01096 PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01097 PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01098 PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01099 PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01100 PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01101 PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01102 PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01103 PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01104 PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01105 PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01106 PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01107 PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01108 PFNGLTEXTUREVIEWPROC glTextureView;
01109 PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01110 PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01111 PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01112 PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01113 PFNGLUNIFORM1DPROC glUniform1d;
01114 PFNGLUNIFORM1DVPROC glUniform1dv;
01115 PFNGLUNIFORM1FPROC glUniform1f;
01116 PFNGLUNIFORM1FVPROC glUniform1fv;
01117 PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01118 PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01119 PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01120 PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01121 PFNGLUNIFORM1IPROC glUniform1i;
01122 PFNGLUNIFORM1IVPROC glUniform1iv;
01123 PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01124 PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01125 PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01126 PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01127 PFNGLUNIFORM1UIPROC glUniform1ui;
01128 PFNGLUNIFORM1UIVPROC glUniform1uiv;
01129 PFNGLUNIFORM2DPROC glUniform2d;
01130 PFNGLUNIFORM2DVPROC glUniform2dv;
01131 PFNGLUNIFORM2FPROC glUniform2f;
01132 PFNGLUNIFORM2FVPROC glUniform2fv;
01133 PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01134 PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
```

```
01135 PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01136 PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01137 PFNGLUNIFORM2IPROC glUniform2i;
01138 PFNGLUNIFORM2IVPROC glUniform2iv;
01139 PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01140 PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01141 PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01142 PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01143 PFNGLUNIFORM2UIPROC glUniform2ui;
01144 PFNGLUNIFORM2UIVPROC glUniform2uiv;
01145 PFNGLUNIFORM3DPROC glUniform3d;
01146 PFNGLUNIFORM3DVPROC glUniform3dv;
01147 PFNGLUNIFORM3FPROC glUniform3f;
01148 PFNGLUNIFORM3FVPROC glUniform3fv;
01149 PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01150 PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01151 PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01152 PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01153 PFNGLUNIFORM3IPROC glUniform3i;
01154 PFNGLUNIFORM3IVPROC glUniform3iv;
01155 PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01156 PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01157 PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01158 PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01159 PFNGLUNIFORM3UIPROC glUniform3ui;
01160 PFNGLUNIFORM3UIVPROC glUniform3uiv;
01161 PFNGLUNIFORM4DPROC glUniform4d;
01162 PFNGLUNIFORM4DVPROC glUniform4dv;
01163 PFNGLUNIFORM4FPROC glUniform4f;
01164 PFNGLUNIFORM4FVPROC glUniform4fv;
01165 PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01166 PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01167 PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01168 PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01169 PFNGLUNIFORM4IPROC glUniform4i;
01170 PFNGLUNIFORM4IVPROC glUniform4iv;
01171 PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01172 PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01173 PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01174 PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01175 PFNGLUNIFORM4UIPROC glUniform4ui;
01176 PFNGLUNIFORM4UIVPROC glUniform4uiv;
01177 PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01178 PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01179 PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01180 PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01181 PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01182 PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01183 PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01184 PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01185 PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01186 PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01187 PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01188 PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01189 PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01190 PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01191 PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01192 PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01193 PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01194 PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01195 PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01196 PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01197 PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01198 PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01199 PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01200 PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01201 PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01202 PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01203 PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01204 PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01205 PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01206 PFNGLUSEPROGRAMPROC glUseProgram;
01207 PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01208 PFNGLUSERESHADEPROGRAMEXTPROC glUseProgramEXT;
01209 PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01210 PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01211 PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01212 PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01213 PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01214 PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01215 PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01216 PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01217 PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01218 PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01219 PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01220 PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01221 PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
```



```
01222 PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01223 PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01224 PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01225 PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01226 PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01227 PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01228 PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01229 PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01230 PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01231 PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01232 PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01233 PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01234 PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01235 PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01236 PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01237 PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01238 PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01239 PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01240 PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01241 PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01242 PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01243 PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01244 PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01245 PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01246 PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01247 PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01248 PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01249 PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01250 PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01251 PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01252 PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01253 PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01254 PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01255 PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01256 PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01257 PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01258 PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01259 PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01260 PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01261 PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01262 PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01263 PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01264 PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01265 PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01266 PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubv;
01267 PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01268 PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01269 PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01270 PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01271 PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01272 PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01273 PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01274 PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01275 PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01276 PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01277 PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01278 PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01279 PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01280 PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01281 PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01282 PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01283 PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01284 PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01285 PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01286 PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01287 PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01288 PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01289 PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01290 PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01291 PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01292 PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01293 PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01294 PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01295 PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01296 PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01297 PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01298 PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01299 PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01300 PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01301 PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01302 PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01303 PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01304 PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01305 PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01306 PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01307 PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01308 PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
```

```

01309 PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribLui64vNV;
01310 PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01311 PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01312 PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01313 PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01314 PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01315 PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01316 PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01317 PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01318 PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01319 PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01320 PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01321 PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01322 PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01323 PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01324 PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01325 PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01326 PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01327 PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01328 PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribLFormatNV;
01329 PFNGLVERTEXATTRIBFORMATPROC glVertexAttribLFormat;
01330 PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01331 PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01332 PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01333 PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01334 PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01335 PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01336 PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01337 PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01338 PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01339 PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01340 PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01341 PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01342 PFNGLVIEWPORTARRAYVPROC glViewportArrayv;
01343 PFNGLVIEWPORTINDEXEDFPROC glViewportIndexdf;
01344 PFNGLVIEWPORTINDEXEDFVPROC glViewportIndexdfv;
01345 PFNGLVIEWPORTPOSITIONWSCALENVPROC glViewportPositionWScaleNV;
01346 PFNGLVIEWPORTPROC glViewport;
01347 PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01348 PFNGLWAITSYNCPROC glWaitSync;
01349 PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01350 PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01351 PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01352 #endif
01353
01355
01357 GLint gg::ggBufferAlignment(0);
01358
01359 //
01360 // ゲームグラフィックス特論の都合にもとづく初期化
01361 //
01362 void gg::ggInit()
01363 {
01364     //すでにこの関数が実行されていたら以降の処理を行わない
01365     if (ggBufferAlignment) return;
01366
01367     // macOS 以外で OpenGL 3.2 以降の API を取得する
01368     #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
01369         glActiveProgramEXT = PFNGLACTIVEPROGRAMEXTPROC(glwGetProcAddress("glActiveProgramEXT"));
01370         glActiveShaderProgram = PFNGLACTIVESHADERPROGRAMPROC(glwGetProcAddress("glActiveShaderProgram"));
01371         glActiveTexture = PFNGLACTIVETEXTUREPROC(glwGetProcAddress("glActiveTexture"));
01372         glApplyFramebufferAttachmentCMAAINTEL =
01373             PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC(glwGetProcAddress("glApplyFramebufferAttachmentCMAAINTEL"));
01374         glAttachShader = PFNGLATTACHSHADERPROC(glwGetProcAddress("glAttachShader"));
01375         glBeginConditionalRender =
01376             PFNGLBEGINCONDITIONALRENDERPROC(glwGetProcAddress("glBeginConditionalRender"));
01377         glBeginConditionalRenderNV =
01378             PFNGLBEGINCONDITIONALRENDERNVPROC(glwGetProcAddress("glBeginConditionalRenderNV"));
01379         glBeginPerfMonitorAMD = PFNGLBEGINPERFMONITORAMDPROC(glwGetProcAddress("glBeginPerfMonitorAMD"));
01380         glBeginPerfQueryINTEL = PFNGLBEGINPERFQUERYINTELPROC(glwGetProcAddress("glBeginPerfQueryINTEL"));
01381         glBeginQuery = PFNGLBEGINQUERYPROC(glwGetProcAddress("glBeginQuery"));
01382         glBeginQueryIndexed = PFNGLBEGINQUERYINDEXEDPROC(glwGetProcAddress("glBeginQueryIndexed"));
01383         glBeginTransformFeedback =
01384             PFNGLBEGINTRANSFORMFEEDBACKPROC(glwGetProcAddress("glBeginTransformFeedback"));
01385         glBindAttribLocation = PFNGLBINDATTRIBLOCATIONPROC(glwGetProcAddress("glBindAttribLocation"));
01386         glBindBuffer = PFNGLBINDBUFFERPROC(glwGetProcAddress("glBindBuffer"));
01387         glBindBufferBase = PFNGLBINDBUFFERBASEPROC(glwGetProcAddress("glBindBufferBase"));
01388         glBindBufferRange = PFNGLBINDBUFFERRANGEPROC(glwGetProcAddress("glBindBufferRange"));
01389         glBindBuffersBase = PFNGLBINDBUFFERSBASEPROC(glwGetProcAddress("glBindBuffersBase"));
01390         glBindBuffersRange = PFNGLBINDBUFFERSRANGEPROC(glwGetProcAddress("glBindBuffersRange"));
01391         glBindFragDataLocation =
01392             PFNGLBINDFRAGDATALOCATIONPROC(glwGetProcAddress("glBindFragDataLocation"));
01393         glBindFragDataLocationIndexed =
01394             PFNGLBINDFRAGDATALOCATIONINDEXEDPROC(glwGetProcAddress("glBindFragDataLocationIndexed"));
01395         glBindFramebuffer = PFNGLBINDFRAMEBUFFERPROC(glwGetProcAddress("glBindFramebuffer"));
01396         glBindImageTexture = PFNGLBINDIMAGETEXTUREPROC(glwGetProcAddress("glBindImageTexture"));
01397         glBindImageTextures = PFNGLBINDIMAGETEXTURESPROC(glwGetProcAddress("glBindImageTextures"));

```

```

01392     glBindMultiTextureEXT = PFNGLBINDMULTITEXTUREEXTPROC (glfwGetProcAddress("glBindMultiTextureEXT"));
01393     glBindProgramPipeline = PFNGLBINDPROGRAMPIPELINEPROC (glfwGetProcAddress("glBindProgramPipeline"));
01394     glBindRenderbuffer = PFNGLBINDRENDERBUFFERPROC (glfwGetProcAddress("glBindRenderbuffer"));
01395     glBindSampler = PFNGLBINDSAMPLERPROC (glfwGetProcAddress("glBindSampler"));
01396     glBindSamplers = PFNGLBINDSAMPLERSPROC (glfwGetProcAddress("glBindSamplers"));
01397     glBindTexture = PFNGLBINDTEXTUREPROC (glfwGetProcAddress("glBindTexture"));
01398     glBindTextureUnit = PFNGLBINDTEXTUREUNITPROC (glfwGetProcAddress("glBindTextureUnit"));
01399     glBindTextures = PFNGLBINDTEXTURESPROC (glfwGetProcAddress("glBindTextures"));
01400     glBindTransformFeedback =
PFNGLBINDTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glBindTransformFeedback"));
01401     glBindVertexArray = PFNGLBINDVERTEXARRAYPROC (glfwGetProcAddress("glBindVertexArray"));
01402     glBindVertexBuffer = PFNGLBINDVERTEXBUFFERPROC (glfwGetProcAddress("glBindVertexBuffer"));
01403     glBindVertexBuffers = PFNGLBINDVERTEXBUFFERSPROC (glfwGetProcAddress("glBindVertexBuffers"));
01404     glBlendBarrierKHR = PFNGLBLENDBARRIERKHRPROC (glfwGetProcAddress("glBlendBarrierKHR"));
01405     glBlendBarrierNV = PFNGLBLENDBARRIERNVPROC (glfwGetProcAddress("glBlendBarrierNV"));
01406     glBlendColor = PFNGLBLENDCOLORPROC (glfwGetProcAddress("glBlendColor"));
01407     glBlendEquation = PFNGLBLEND EQUATIONPROC (glfwGetProcAddress("glBlendEquation"));
01408     glBlendEquationSeparate =
PFNGLBLEND EQUATIONSEPARATEPROC (glfwGetProcAddress("glBlendEquationSeparate"));
01409     glBlendEquationSeparatei =
PFNGLBLEND EQUATIONSEPARATEIPROC (glfwGetProcAddress("glBlendEquationSeparatei"));
01410     glBlendEquationSeparateiARB =
PFNGLBLEND EQUATIONSEPARATEIARBPROC (glfwGetProcAddress("glBlendEquationSeparateiARB"));
01411     glBlendEquationi = PFNGLBLEND EQUATIONIPROC (glfwGetProcAddress("glBlendEquationi"));
01412     glBlendEquationiARB = PFNGLBLEND EQUATIONIARBPROC (glfwGetProcAddress("glBlendEquationiARB"));
01413     glBlendFunc = PFNGLBLEND FUNCPROC (glfwGetProcAddress("glBlendFunc"));
01414     glBlendFuncSeparate = PFNGLBLEND FUNCSEPARATEPROC (glfwGetProcAddress("glBlendFuncSeparate"));
01415     glBlendFuncSeparatei = PFNGLBLEND FUNCSEPARATEIPROC (glfwGetProcAddress("glBlendFuncSeparatei"));
01416     glBlendFuncSeparateiARB =
PFNGLBLEND FUNCSEPARATEIARBPROC (glfwGetProcAddress("glBlendFuncSeparateiARB"));
01417     glBlendFunci = PFNGLBLEND FUNCIPROC (glfwGetProcAddress("glBlendFunci"));
01418     glBlendFunciARB = PFNGLBLEND FUNCIARBPROC (glfwGetProcAddress("glBlendFunciARB"));
01419     glBlendParameteriNV = PFNGLBLEND PARAMETERINVPROC (glfwGetProcAddress("glBlendParameteriNV"));
01420     glBlitFramebuffer = PFNGLBLITFRAMEBUFFERPROC (glfwGetProcAddress("glBlitFramebuffer"));
01421     glBlitNamedFramebuffer =
PFNGLBLITNAMEDFRAMEBUFFERPROC (glfwGetProcAddress("glBlitNamedFramebuffer"));
01422     glBufferAddressRangeNV =
PFNGLBUFFERADDRESSRANGENVPROC (glfwGetProcAddress("glBufferAddressRangeNV"));
01423     glBufferData = PFNGLBUFFERDATAPROC (glfwGetProcAddress("glBufferData"));
01424     glBufferPageCommitmentARB =
PFNGLBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress("glBufferPageCommitmentARB"));
01425     glBufferStorage = PFNGLBUFFERSTORAGEPROC (glfwGetProcAddress("glBufferStorage"));
01426     glBufferSubData = PFNGLBUFFERSUBDATAPROC (glfwGetProcAddress("glBufferSubData"));
01427     glCallCommandListNV = PFNGLCALLCOMMANDLISTNVPROC (glfwGetProcAddress("glCallCommandListNV"));
01428     glCheckFramebufferStatus =
PFNGLCHECKFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckFramebufferStatus"));
01429     glCheckNamedFramebufferStatus =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckNamedFramebufferStatus"));
01430     glCheckNamedFramebufferStatusEXT =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC (glfwGetProcAddress("glCheckNamedFramebufferStatusEXT"));
01431     glClampColor = PFNGLCLAMP COLORPROC (glfwGetProcAddress("glClampColor"));
01432     glClear = PFNGLCLEARPROC (glfwGetProcAddress("glClear"));
01433     glClearBufferData = PFNGLCLEARBUFFERDATAPROC (glfwGetProcAddress("glClearBufferData"));
01434     glClearBufferSubData = PFNGLCLEARBUFFERSUBDATAPROC (glfwGetProcAddress("glClearBufferSubData"));
01435     glClearBufferfi = PFNGLCLEARBUFFERFIPROC (glfwGetProcAddress("glClearBufferfi"));
01436     glClearBufferfv = PFNGLCLEARBUFFERFVPROC (glfwGetProcAddress("glClearBufferfv"));
01437     glClearBufferiv = PFNGLCLEARBUFFERIVPROC (glfwGetProcAddress("glClearBufferiv"));
01438     glClearBufferuiv = PFNGLCLEARBUFFERUIVPROC (glfwGetProcAddress("glClearBufferuiv"));
01439     glClearColor = PFNGLCLEAR COLORPROC (glfwGetProcAddress("glClearColor"));
01440     glClearDepth = PFNGLCLEAR DEPTHPROC (glfwGetProcAddress("glClearDepth"));
01441     glClearDepthf = PFNGLCLEAR DEPTHFPROC (glfwGetProcAddress("glClearDepthf"));
01442     glClearNamedBufferData =
PFNGLCLEARNAMEDBUFFERDATAPROC (glfwGetProcAddress("glClearNamedBufferData"));
01443     glClearNamedBufferDataEXT =
PFNGLCLEARNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferDataEXT"));
01444     glClearNamedBufferSubData =
PFNGLCLEARNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress("glClearNamedBufferSubData"));
01445     glClearNamedBufferSubDataEXT =
PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferSubDataEXT"));
01446     glClearNamedFramebufferfi =
PFNGLCLEARNAMEDFRAMEBUFFERFIPROC (glfwGetProcAddress("glClearNamedFramebufferfi"));
01447     glClearNamedFramebufferfv =
PFNGLCLEARNAMEDFRAMEBUFFERFVPROC (glfwGetProcAddress("glClearNamedFramebufferfv"));
01448     glClearNamedFramebufferiv =
PFNGLCLEARNAMEDFRAMEBUFFERIVPROC (glfwGetProcAddress("glClearNamedFramebufferiv"));
01449     glClearNamedBufferuiv =
PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC (glfwGetProcAddress("glClearNamedBufferuiv"));
01450     glClearStencil = PFNGLCLEARSTENCILPROC (glfwGetProcAddress("glClearStencil"));
01451     glClearTexImage = PFNGLCLEARTEXTIMAGEPROC (glfwGetProcAddress("glClearTexImage"));
01452     glClearTexSubImage = PFNGLCLEARTEXTSUBIMAGEPROC (glfwGetProcAddress("glClearTexSubImage"));
01453     glClientAttribDefaultEXT =
PFNGLCLIENTATTRIBDEFAULTTEXTPROC (glfwGetProcAddress("glClientAttribDefaultEXT"));
01454     glClientWaitSync = PFNGLCLIENTWAITSYNCPROC (glfwGetProcAddress("glClientWaitSync"));
01455     glClipControl = PFNGLCLIPCONTROLPROC (glfwGetProcAddress("glClipControl"));
01456     glColorFormatNV = PFNGLCOLORFORMATNVPROC (glfwGetProcAddress("glColorFormatNV"));
01457     glColorMask = PFNGLCOLORMASKPROC (glfwGetProcAddress("glColorMask"));
01458     glColorMaski = PFNGLCOLORMASKIPROC (glfwGetProcAddress("glColorMaski"));

```

```

01459     glCommandListSegmentsNV =
PFNGLCOMMANDLISTSEGMENTSNVPROC (glfwGetProcAddress ("glCommandListSegmentsNV"));
01460     glCompileCommandListNV =
PFNGLCOMPILECOMMANDLISTNVPROC (glfwGetProcAddress ("glCompileCommandListNV"));
01461     glCompileShader = PFNGLCOMPILESHADERPROC (glfwGetProcAddress ("glCompileShader"));
01462     glCompileShaderIncludeARB =
PFNGLCOMPILESHADERINCLUDEARBPROC (glfwGetProcAddress ("glCompileShaderIncludeARB"));
01463     glCompressedMultiTexImage1DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage1DEXT"));
01464     glCompressedMultiTexImage2DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage2DEXT"));
01465     glCompressedMultiTexImage3DEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage3DEXT"));
01466     glCompressedMultiTexSubImage1DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage1DEXT"));
01467     glCompressedMultiTexSubImage2DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage2DEXT"));
01468     glCompressedMultiTexSubImage3DEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage3DEXT"));
01469     glCompressedTexImage1D =
PFNGLCOMPRESSEDTEXIMAGE1DPROC (glfwGetProcAddress ("glCompressedTexImage1D"));
01470     glCompressedTexImage2D =
PFNGLCOMPRESSEDTEXIMAGE2DPROC (glfwGetProcAddress ("glCompressedTexImage2D"));
01471     glCompressedTexImage3D =
PFNGLCOMPRESSEDTEXIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexImage3D"));
01472     glCompressedTexSubImage1D =
PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTexSubImage1D"));
01473     glCompressedTexSubImage2D =
PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTexSubImage2D"));
01474     glCompressedTexSubImage3D =
PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexSubImage3D"));
01475     glCompressedTextureImage1DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage1DEXT"));
01476     glCompressedTextureImage2DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage2DEXT"));
01477     glCompressedTextureImage3DEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedTextureImage3DEXT"));
01478     glCompressedTextureSubImage1D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTextureSubImage1D"));
01479     glCompressedTextureSubImage1DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage1DEXT"));
01480     glCompressedTextureSubImage2D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTextureSubImage2D"));
01481     glCompressedTextureSubImage2DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage2DEXT"));
01482     glCompressedTextureSubImage3D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTextureSubImage3D"));
01483     glCompressedTextureSubImage3DEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage3DEXT"));
01484     glConservativeRasterParameterfNV =
PFNGLCONSERVATIVERASTERPARAMETERFNVPROC (glfwGetProcAddress ("glConservativeRasterParameterfNV"));
01485     glConservativeRasterParameteriNV =
PFNGLCONSERVATIVERASTERPARAMETERINVPROC (glfwGetProcAddress ("glConservativeRasterParameteriNV"));
01486     glCopyBufferSubData = PFNGLCOPYBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyBufferSubData"));
01487     glCopyImageSubData = PFNGLCOPYIMAGESUBDATAPROC (glfwGetProcAddress ("glCopyImageSubData"));
01488     glCopyMultiTexImage1DEXT =
PFNGLCOPYMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage1DEXT"));
01489     glCopyMultiTexImage2DEXT =
PFNGLCOPYMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage2DEXT"));
01490     glCopyMultiTexSubImage1DEXT =
PFNGLCOPYMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage1DEXT"));
01491     glCopyMultiTexSubImage2DEXT =
PFNGLCOPYMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage2DEXT"));
01492     glCopyMultiTexSubImage3DEXT =
PFNGLCOPYMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage3DEXT"));
01493     glCopyNamedBufferSubData =
PFNGLCOPYNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyNamedBufferSubData"));
01494     glCopyPathNV = PFNGLCOPYPATHNVPROC (glfwGetProcAddress ("glCopyPathNV"));
01495     glCopyTexImage1D = PFNGLCOPYTEXIMAGE1DPROC (glfwGetProcAddress ("glCopyTexImage1D"));
01496     glCopyTexImage2D = PFNGLCOPYTEXIMAGE2DPROC (glfwGetProcAddress ("glCopyTexImage2D"));
01497     glCopyTexSubImage1D = PFNGLCOPYTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTexSubImage1D"));
01498     glCopyTexSubImage2D = PFNGLCOPYTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTexSubImage2D"));
01499     glCopyTexSubImage3D = PFNGLCOPYTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTexSubImage3D"));
01500     glCopyTextureImage1DEXT =
PFNGLCOPYTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureImage1DEXT"));
01501     glCopyTextureImage2DEXT =
PFNGLCOPYTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureImage2DEXT"));
01502     glCopyTextureSubImage1D =
PFNGLCOPYTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTextureSubImage1D"));
01503     glCopyTextureSubImage1DEXT =
PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage1DEXT"));
01504     glCopyTextureSubImage2D =
PFNGLCOPYTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTextureSubImage2D"));
01505     glCopyTextureSubImage2DEXT =
PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage2DEXT"));
01506     glCopyTextureSubImage3D =
PFNGLCOPYTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTextureSubImage3D"));

```



```

01507     glCopyTextureSubImage3DEXT =
PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress("glCopyTextureSubImage3DEXT"));
01508     glCoverFillPathInstancedNV =
PFNGLCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress("glCoverFillPathInstancedNV"));
01509     glCoverFillPathNV = PFNGLCOVERFILLPATHNVPROC (glfwGetProcAddress("glCoverFillPathNV"));
01510     glCoverStrokePathInstancedNV =
PFNGLCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress("glCoverStrokePathInstancedNV"));
01511     glCoverStrokePathNV = PFNGLCOVERSTROKEPATHNVPROC (glfwGetProcAddress("glCoverStrokePathNV"));
01512     glCoverageModulationNV =
PFNGLCOVERAGEMODULATIONNVPROC (glfwGetProcAddress("glCoverageModulationNV"));
01513     glCoverageModulationTableNV =
PFNGLCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress("glCoverageModulationTableNV"));
01514     glCreateBuffers = PFNGLCREATEBUFFERSPROC (glfwGetProcAddress("glCreateBuffers"));
01515     glCreateCommandListsNV =
PFNGLCREATECOMMANDLISTSNVPROC (glfwGetProcAddress("glCreateCommandListsNV"));
01516     glCreateFramebuffers = PFNGLCREATEFRAMEBUFFERSPROC (glfwGetProcAddress("glCreateFramebuffers"));
01517     glCreatePerfQueryINTEL =
PFNGLCREATEPERFQUERYINTELPROC (glfwGetProcAddress("glCreatePerfQueryINTEL"));
01518     glCreateProgram = PFNGLCREATEPROGRAMPROC (glfwGetProcAddress("glCreateProgram"));
01519     glCreateProgramPipelines =
PFNGLCREATEPROGRAMPIPELINESPROC (glfwGetProcAddress("glCreateProgramPipelines"));
01520     glCreateQueries = PFNGLCREATEQUERIESPROC (glfwGetProcAddress("glCreateQueries"));
01521     glCreateRenderbuffers = PFNGLCREATERENDERBUFFERSPROC (glfwGetProcAddress("glCreateRenderbuffers"));
01522     glCreateSamplers = PFNGLCREATESAMPLERSPROC (glfwGetProcAddress("glCreateSamplers"));
01523     glCreateShader = PFNGLCREATESHADERPROC (glfwGetProcAddress("glCreateShader"));
01524     glCreateShaderProgramEXT =
PFNGLCREATESHADERPROGRAMEXTPROC (glfwGetProcAddress("glCreateShaderProgramEXT"));
01525     glCreateShaderProgramv =
PFNGLCREATESHADERPROGRAMVPROC (glfwGetProcAddress("glCreateShaderProgramv"));
01526     glCreateStatesNV = PFNGLCREATESTATESNVPROC (glfwGetProcAddress("glCreateStatesNV"));
01527     glCreateSyncFromCLeventARB =
PFNGLCREATESYNCFROMCLEVENTARBPROC (glfwGetProcAddress("glCreateSyncFromCLeventARB"));
01528     glCreateTextures = PFNGLCREATETEXTURESPROC (glfwGetProcAddress("glCreateTextures"));
01529     glCreateTransformFeedbacks =
PFNGLCREATETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glCreateTransformFeedbacks"));
01530     glCreateVertexArrays = PFNGLCREATEVERTEXARRAYSPROC (glfwGetProcAddress("glCreateVertexArrays"));
01531     glCullFace = PFNGLCULLFACEPROC (glfwGetProcAddress("glCullFace"));
01532     glDebugMessageCallback =
PFNGLDEBUGMESSAGECALLBACKPROC (glfwGetProcAddress("glDebugMessageCallback"));
01533     glDebugMessageCallbackARB =
PFNGLDEBUGMESSAGECALLBACKARBPROC (glfwGetProcAddress("glDebugMessageCallbackARB"));
01534     glDebugMessageControl = PFNGLDEBUGMESSAGECONTROLPROC (glfwGetProcAddress("glDebugMessageControl"));
01535     glDebugMessageControlARB =
PFNGLDEBUGMESSAGECONTROLARBPROC (glfwGetProcAddress("glDebugMessageControlARB"));
01536     glDebugMessageInsert = PFNGLDEBUGMESSAGEINSERTPROC (glfwGetProcAddress("glDebugMessageInsert"));
01537     glDebugMessageInsertARB =
PFNGLDEBUGMESSAGEINSERTARBPROC (glfwGetProcAddress("glDebugMessageInsertARB"));
01538     glDeleteBuffers = PFNGLDELETEBUFFERSPROC (glfwGetProcAddress("glDeleteBuffers"));
01539     glDeleteCommandListsNV =
PFNGLDELETECOMMANDLISTSNVPROC (glfwGetProcAddress("glDeleteCommandListsNV"));
01540     glDeleteFramebuffers = PFNGLDELETEFRAMEBUFFERSPROC (glfwGetProcAddress("glDeleteFramebuffers"));
01541     glDeleteNamedStringARB =
PFNGLDELETENAMEDSTRINGARBPROC (glfwGetProcAddress("glDeleteNamedStringARB"));
01542     glDeletePathsNV = PFNGLDELETEPATHSNVPROC (glfwGetProcAddress("glDeletePathsNV"));
01543     glDeletePerfMonitorsAMD =
PFNGLDELETEPERFMONITORSAMDPROC (glfwGetProcAddress("glDeletePerfMonitorsAMD"));
01544     glDeletePerfQueryINTEL =
PFNGLDELETEPERFQUERYINTELPROC (glfwGetProcAddress("glDeletePerfQueryINTEL"));
01545     glDeleteProgram = PFNGLDELETEPROGRAMPROC (glfwGetProcAddress("glDeleteProgram"));
01546     glDeleteProgramPipelines =
PFNGLDELETEPROGRAMPIPELINESPROC (glfwGetProcAddress("glDeleteProgramPipelines"));
01547     glDeleteQueries = PFNGLDELETEQUERIESPROC (glfwGetProcAddress("glDeleteQueries"));
01548     glDeleteRenderbuffers = PFNGLDELETERENDERBUFFERSPROC (glfwGetProcAddress("glDeleteRenderbuffers"));
01549     glDeleteSamplers = PFNGLDELETESAMPLERSPROC (glfwGetProcAddress("glDeleteSamplers"));
01550     glDeleteShader = PFNGLDELETESHADERPROC (glfwGetProcAddress("glDeleteShader"));
01551     glDeleteStatesNV = PFNGLDELETESATESNVPROC (glfwGetProcAddress("glDeleteStatesNV"));
01552     glDeleteSync = PFNGLDELETESYNCPROC (glfwGetProcAddress("glDeleteSync"));
01553     glDeleteTextures = PFNGLDELETETEXTURESPROC (glfwGetProcAddress("glDeleteTextures"));
01554     glDeleteTransformFeedbacks =
PFNGLDELETETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glDeleteTransformFeedbacks"));
01555     glDeleteVertexArrays = PFNGLDELETEVERTEXARRAYSPROC (glfwGetProcAddress("glDeleteVertexArrays"));
01556     glDepthFunc = PFNGLDEPTHFUNCPROC (glfwGetProcAddress("glDepthFunc"));
01557     glDepthMask = PFNGLDEPTHMASKPROC (glfwGetProcAddress("glDepthMask"));
01558     glDepthRange = PFNGLDEPTHRANGEPROC (glfwGetProcAddress("glDepthRange"));
01559     glDepthRangeArrayv = PFNGLDEPTHRANGEARRAYVPROC (glfwGetProcAddress("glDepthRangeArrayv"));
01560     glDepthRangeIndexed = PFNGLDEPTHRANGEINDEXEDPROC (glfwGetProcAddress("glDepthRangeIndexed"));
01561     glDepthRangef = PFNGLDEPTHRANGEFPROC (glfwGetProcAddress("glDepthRangef"));
01562     glDetachShader = PFNGLDETACHSHADERPROC (glfwGetProcAddress("glDetachShader"));
01563     glDisable = PFNGLDISABLEPROC (glfwGetProcAddress("glDisable"));
01564     glDisableClientStateIndexedEXT =
PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glDisableClientStateIndexedEXT"));
01565     glDisableClientStateiEXT =
PFNGLDISABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glDisableClientStateiEXT"));
01566     glDisableIndexedEXT = PFNGLDISABLEINDEXEDEXTPROC (glfwGetProcAddress("glDisableIndexedEXT"));
01567     glDisableVertexArrayAttrib =
PFNGLDISABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glDisableVertexArrayAttrib"));
01568     glDisableVertexArrayAttribEXT =

```

```

    PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glDisableVertexArrayAttribEXT"));
01569     glDisableVertexArrayEXT =
    PFNGLDISABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glDisableVertexArrayEXT"));
01570     glDisableVertexArrayAttribArray =
    PFNGLDISABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glDisableVertexArrayAttribArray"));
01571     glDisablei = PFNGLDISABLEIPROC (glfwGetProcAddress("glDisablei"));
01572     glDispatchCompute = PFNGLDISPATCHCOMPUTEPROC (glfwGetProcAddress("glDispatchCompute"));
01573     glDispatchComputeGroupSizeARB =
    PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC (glfwGetProcAddress("glDispatchComputeGroupSizeARB"));
01574     glDispatchComputeIndirect =
    PFNGLDISPATCHCOMPUTEINDIRECTPROC (glfwGetProcAddress("glDispatchComputeIndirect"));
01575     glDrawArrays = PFNGLDRAWARRAYSPROC (glfwGetProcAddress("glDrawArrays"));
01576     glDrawArraysIndirect = PFNGLDRAWARRAYSINDIRECTPROC (glfwGetProcAddress("glDrawArraysIndirect"));
01577     glDrawArraysInstanced = PFNGLDRAWARRAYSINSTANCEDPROC (glfwGetProcAddress("glDrawArraysInstanced"));
01578     glDrawArraysInstancedARB =
    PFNGLDRAWARRAYSINSTANCEDARBPROC (glfwGetProcAddress("glDrawArraysInstancedARB"));
01579     glDrawArraysInstancedBaseInstance =
    PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress("glDrawArraysInstancedBaseInstance"));
01580     glDrawArraysInstancedEXT =
    PFNGLDRAWARRAYSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawArraysInstancedEXT"));
01581     glDrawBuffer = PFNGLDRAWBUFFERPROC (glfwGetProcAddress("glDrawBuffer"));
01582     glDrawBuffers = PFNGLDRAWBUFFERSPROC (glfwGetProcAddress("glDrawBuffers"));
01583     glDrawCommandsAddressNV =
    PFNGLDRAWCOMMANDSADDRESSNVPROC (glfwGetProcAddress("glDrawCommandsAddressNV"));
01584     glDrawCommandsNV = PFNGLDRAWCOMMANDSNVPROC (glfwGetProcAddress("glDrawCommandsNV"));
01585     glDrawCommandsStatesAddressNV =
    PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC (glfwGetProcAddress("glDrawCommandsStatesAddressNV"));
01586     glDrawCommandsStatesNV =
    PFNGLDRAWCOMMANDSSTATESNVPROC (glfwGetProcAddress("glDrawCommandsStatesNV"));
01587     glDrawElements = PFNGLDRAWELEMENTSPROC (glfwGetProcAddress("glDrawElements"));
01588     glDrawElementsBaseVertex =
    PFNGLDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glDrawElementsBaseVertex"));
01589     glDrawElementsIndirect =
    PFNGLDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress("glDrawElementsIndirect"));
01590     glDrawElementsInstanced =
    PFNGLDRAWELEMENTSINSTANCEDPROC (glfwGetProcAddress("glDrawElementsInstanced"));
01591     glDrawElementsInstancedARB =
    PFNGLDRAWELEMENTSINSTANCEDARBPROC (glfwGetProcAddress("glDrawElementsInstancedARB"));
01592     glDrawElementsInstancedBaseInstance =
    PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress("glDrawElementsInstancedBaseInstance"));
01593     glDrawElementsInstancedBaseVertex =
    PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC (glfwGetProcAddress("glDrawElementsInstancedBaseVertex"));
01594     glDrawElementsInstancedBaseVertexBaseInstance =
    PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC (glfwGetProcAddress("glDrawElementsInstancedBaseVertexBaseInstance"));
01595     glDrawElementsInstancedEXT =
    PFNGLDRAWELEMENTSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawElementsInstancedEXT"));
01596     glDrawRangeElements = PFNGLDRAWRANGEELEMENTSPROC (glfwGetProcAddress("glDrawRangeElements"));
01597     glDrawRangeElementsBaseVertex =
    PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glDrawRangeElementsBaseVertex"));
01598     glDrawTransformFeedback =
    PFNGLDRAWTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glDrawTransformFeedback"));
01599     glDrawTransformFeedbackInstanced =
    PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackInstanced"));
01600     glDrawTransformFeedbackStream =
    PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC (glfwGetProcAddress("glDrawTransformFeedbackStream"));
01601     glDrawTransformFeedbackStreamInstanced =
    PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackStreamInstanced"));
01602     glDrawVkImageNV = PFNGLDRAWVKIMAGENVPROC (glfwGetProcAddress("glDrawVkImageNV"));
01603     glEdgeFlagFormatNV = PFNGLEDGEFLAGFORMATNVPROC (glfwGetProcAddress("glEdgeFlagFormatNV"));
01604     glEnable = PFNGLENABLEPROC (glfwGetProcAddress("glEnable"));
01605     glEnableClientStateIndexedEXT =
    PFNGLENABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glEnableClientStateIndexedEXT"));
01606     glEnableClientStateiEXT =
    PFNGLENABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glEnableClientStateiEXT"));
01607     glEnableIndexedEXT = PFNGLENABLEINDEXEDEXTPROC (glfwGetProcAddress("glEnableIndexedEXT"));
01608     glEnableVertexArrayAttrib =
    PFNGLENABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glEnableVertexArrayAttrib"));
01609     glEnableVertexArrayAttribEXT =
    PFNGLENABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glEnableVertexArrayAttribEXT"));
01610     glEnableVertexArrayEXT =
    PFNGLENABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glEnableVertexArrayEXT"));
01611     glEnableVertexArrayAttribArray =
    PFNGLENABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glEnableVertexArrayAttribArray"));
01612     glEnablei = PFNGLENABLEIPROC (glfwGetProcAddress("glEnablei"));
01613     glEndConditionalRender =
    PFNGLENDCONDITIONALRENDERPROC (glfwGetProcAddress("glEndConditionalRender"));
01614     glEndConditionalRenderNV =
    PFNGLENDCONDITIONALRENDERNVPROC (glfwGetProcAddress("glEndConditionalRenderNV"));
01615     glEndPerfMonitorAMD = PFNGLENDPERFMONITORAMDPROC (glfwGetProcAddress("glEndPerfMonitorAMD"));
01616     glEndPerfQueryINTEL = PFNGLENDPERFQUERYINTELPROC (glfwGetProcAddress("glEndPerfQueryINTEL"));
01617     glEndQuery = PFNGLENDQUERYPROC (glfwGetProcAddress("glEndQuery"));
01618     glEndQueryIndexed = PFNGLENDQUERYINDEXEDPROC (glfwGetProcAddress("glEndQueryIndexed"));
01619     glEndTransformFeedback =
    PFNGLENDTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glEndTransformFeedback"));
01620     glEvaluateDepthValuesARB =
    PFNGLEVALUATEDEPTHVALUESARBPROC (glfwGetProcAddress("glEvaluateDepthValuesARB"));
01621     glFenceSync = PFNGLFENCESYNCPROC (glfwGetProcAddress("glFenceSync"));

```

```

01622     glFinish = PFNGLFINISHPROC (glfwGetProcAddress ("glFinish"));
01623     glFlush = PFNGLFLUSHPROC (glfwGetProcAddress ("glFlush"));
01624     glFlushMappedBufferRange =
PFNGLFLUSHMAPPEDBUFFERRANGEPROC (glfwGetProcAddress ("glFlushMappedBufferRange"));
01625     glFlushMappedNamedBufferRange =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC (glfwGetProcAddress ("glFlushMappedNamedBufferRange"));
01626     glFlushMappedNamedBufferRangeEXT =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress ("glFlushMappedNamedBufferRangeEXT"));
01627     glFogCoordFormatNV = PFNGLFOGCOORDFORMATNVPROC (glfwGetProcAddress ("glFogCoordFormatNV"));
01628     glFragmentCoverageColorNV =
PFNGLFRAGMENTCOVERAGECOLORNVPROC (glfwGetProcAddress ("glFragmentCoverageColorNV"));
01629     glFramebufferDrawBufferEXT =
PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC (glfwGetProcAddress ("glFramebufferDrawBufferEXT"));
01630     glFramebufferDrawBuffersEXT =
PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC (glfwGetProcAddress ("glFramebufferDrawBuffersEXT"));
01631     glFramebufferParameteri =
PFNGLFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress ("glFramebufferParameteri"));
01632     glFramebufferReadBufferEXT =
PFNGLFRAMEBUFFERREADBUFFEREXTPROC (glfwGetProcAddress ("glFramebufferReadBufferEXT"));
01633     glFramebufferRenderbuffer =
PFNGLFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress ("glFramebufferRenderbuffer"));
01634     glFramebufferSampleLocationsfvARB =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress ("glFramebufferSampleLocationsfvARB"));
01635     glFramebufferSampleLocationsfvNV =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress ("glFramebufferSampleLocationsfvNV"));
01636     glFramebufferTexture = PFNGLFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress ("glFramebufferTexture"));
01637     glFramebufferTexture1D =
PFNGLFRAMEBUFFERTEXTURE1DPROC (glfwGetProcAddress ("glFramebufferTexture1D"));
01638     glFramebufferTexture2D =
PFNGLFRAMEBUFFERTEXTURE2DPROC (glfwGetProcAddress ("glFramebufferTexture2D"));
01639     glFramebufferTexture3D =
PFNGLFRAMEBUFFERTEXTURE3DPROC (glfwGetProcAddress ("glFramebufferTexture3D"));
01640     glFramebufferTextureARB =
PFNGLFRAMEBUFFERTEXTUREARBPROC (glfwGetProcAddress ("glFramebufferTextureARB"));
01641     glFramebufferTextureFaceARB =
PFNGLFRAMEBUFFERTEXTUREFACEARBPROC (glfwGetProcAddress ("glFramebufferTextureFaceARB"));
01642     glFramebufferTextureLayer =
PFNGLFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress ("glFramebufferTextureLayer"));
01643     glFramebufferTextureLayerARB =
PFNGLFRAMEBUFFERTEXTURELAYERARBPROC (glfwGetProcAddress ("glFramebufferTextureLayerARB"));
01644     glFramebufferTextureMultiviewOVR =
PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC (glfwGetProcAddress ("glFramebufferTextureMultiviewOVR"));
01645     glFrontFace = PFNGLFRONTFACEPROC (glfwGetProcAddress ("glFrontFace"));
01646     glGenBuffers = PFNGLGENBUFFERSPROC (glfwGetProcAddress ("glGenBuffers"));
01647     glGenFramebuffers = PFNGLGENFRAMEBUFFERSPROC (glfwGetProcAddress ("glGenFramebuffers"));
01648     glGenPathsNV = PFNGLGENPATHSNVPROC (glfwGetProcAddress ("glGenPathsNV"));
01649     glGenPerfMonitorsAMD = PFNGLGENPERFMONITORSAMDPROC (glfwGetProcAddress ("glGenPerfMonitorsAMD"));
01650     glGenProgramPipelines = PFNGLGENPROGRAMPIPELINESPROC (glfwGetProcAddress ("glGenProgramPipelines"));
01651     glGenQueries = PFNGLGENQUERIESPROC (glfwGetProcAddress ("glGenQueries"));
01652     glGenRenderbuffers = PFNGLGENRENDERBUFFERSPROC (glfwGetProcAddress ("glGenRenderbuffers"));
01653     glGenSamplers = PFNGLGENSAMPLERSPROC (glfwGetProcAddress ("glGenSamplers"));
01654     glGenTextures = PFNGLGENTEXTURESPROC (glfwGetProcAddress ("glGenTextures"));
01655     glGenTransformFeedbacks =
PFNGLGENTRANSFORMFEEDBACKSPROC (glfwGetProcAddress ("glGenTransformFeedbacks"));
01656     glGenVertexArrays = PFNGLGENVERTEXARRAYSPROC (glfwGetProcAddress ("glGenVertexArrays"));
01657     glGenerateMipmap = PFNGLGENERATEMIPMAPPROC (glfwGetProcAddress ("glGenerateMipmap"));
01658     glGenerateMultiTexMipmapEXT =
PFNGLGENERATEMULTITEXMIPMAPEXTPROC (glfwGetProcAddress ("glGenerateMultiTexMipmapEXT"));
01659     glGenerateTextureMipmap =
PFNGLGENERATETEXTUREMIPMAPPROC (glfwGetProcAddress ("glGenerateTextureMipmap"));
01660     glGenerateTextureMipmapEXT =
PFNGLGENERATETEXTUREMIPMAPEXTPROC (glfwGetProcAddress ("glGenerateTextureMipmapEXT"));
01661     glGetActiveAtomicCounterBufferiv =
PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC (glfwGetProcAddress ("glGetActiveAtomicCounterBufferiv"));
01662     glGetActiveAttrib = PFNGLGETACTIVEATTRIBPROC (glfwGetProcAddress ("glGetActiveAttrib"));
01663     glGetActiveSubroutineName =
PFNGLGETACTIVESUBROUTINENAMEPROC (glfwGetProcAddress ("glGetActiveSubroutineName"));
01664     glGetActiveSubroutineUniformName =
PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC (glfwGetProcAddress ("glGetActiveSubroutineUniformName"));
01665     glGetActiveSubroutineUniformiv =
PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC (glfwGetProcAddress ("glGetActiveSubroutineUniformiv"));
01666     glGetActiveUniform = PFNGLGETACTIVEUNIFORMPROC (glfwGetProcAddress ("glGetActiveUniform"));
01667     glGetActiveUniformBlockName =
PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC (glfwGetProcAddress ("glGetActiveUniformBlockName"));
01668     glGetActiveUniformBlockiv =
PFNGLGETACTIVEUNIFORMBLOCKIVPROC (glfwGetProcAddress ("glGetActiveUniformBlockiv"));
01669     glGetActiveUniformName =
PFNGLGETACTIVEUNIFORMNAMEPROC (glfwGetProcAddress ("glGetActiveUniformName"));
01670     glGetActiveUniformsiv = PFNGLGETACTIVEUNIFORMSIVPROC (glfwGetProcAddress ("glGetActiveUniformsiv"));
01671     glGetAttachedShaders = PFNGLGETATTACHEDSHADERSPROC (glfwGetProcAddress ("glGetAttachedShaders"));
01672     glGetAttribLocation = PFNGLGETATTRIBLOCATIONPROC (glfwGetProcAddress ("glGetAttribLocation"));
01673     glGetBooleanIndexedvEXT =
PFNGLGETBOOLEANINDEXEDVEXTPROC (glfwGetProcAddress ("glGetBooleanIndexedvEXT"));
01674     glGetBooleani_v = PFNGLGETBOOLEANI_VPROC (glfwGetProcAddress ("glGetBooleani_v"));
01675     glGetBooleanv = PFNGLGETBOOLEANVPROC (glfwGetProcAddress ("glGetBooleanv"));
01676     glGetBufferParameteri64v =
PFNGLGETBUFFERPARAMETERI64VPROC (glfwGetProcAddress ("glGetBufferParameteri64v"));

```

```

01677     glGetBufferParameteriv =
PFNGLGETBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetBufferParameteriv"));
01678     glGetBufferParameterui64vNV =
PFNGLGETBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress("glGetBufferParameterui64vNV"));
01679     glGetBufferPointerv = PFNGLGETBUFFERPOINTERVPROC (glfwGetProcAddress("glGetBufferPointerv"));
01680     glGetBufferSubData = PFNGLGETBUFFERSUBDATAPROC (glfwGetProcAddress("glGetBufferSubData"));
01681     glGetCommandHeaderNV = PFNGLGETCOMMANDHEADERNVPROC (glfwGetProcAddress("glGetCommandHeaderNV"));
01682     glGetCompressedMultiTexImageEXT =
PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC (glfwGetProcAddress("glGetCompressedMultiTexImageEXT"));
01683     glGetCompressedTexImage =
PFNGLGETCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress("glGetCompressedTexImage"));
01684     glGetCompressedTextureImage =
PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC (glfwGetProcAddress("glGetCompressedTextureImage"));
01685     glGetCompressedTextureImageEXT =
PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC (glfwGetProcAddress("glGetCompressedTextureImageEXT"));
01686     glGetCompressedTextureSubImage =
PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC (glfwGetProcAddress("glGetCompressedTextureSubImage"));
01687     glGetCoverageModulationTableNV =
PFNGLGETCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress("glGetCoverageModulationTableNV"));
01688     glGetDebugMessageLog = PFNGLGETDEBUGMESSAGELOGPROC (glfwGetProcAddress("glGetDebugMessageLog"));
01689     glGetDebugMessageLogARB =
PFNGLGETDEBUGMESSAGELOGARBPROC (glfwGetProcAddress("glGetDebugMessageLogARB"));
01690     glGetDoubleIndexeddvEXT =
PFNGLGETDOUBLEINDEXEDVEXTPROC (glfwGetProcAddress("glGetDoubleIndexeddvEXT"));
01691     glGetDoublei_v = PFNGLGETDOUBLEI_VPROC (glfwGetProcAddress("glGetDoublei_v"));
01692     glGetDoublei_vEXT = PFNGLGETDOUBLEI_VEXTPROC (glfwGetProcAddress("glGetDoublei_vEXT"));
01693     glGetDoublev = PFNGLGETDOUBLEVPROC (glfwGetProcAddress("glGetDoublev"));
01694     glGetError = PFNGLGETERRORPROC (glfwGetProcAddress("glGetError"));
01695     glGetFirstPerfQueryIdINTEL =
PFNGLGETFIRSTPERFQUERYIDINTELPROC (glfwGetProcAddress("glGetFirstPerfQueryIdINTEL"));
01696     glGetFloatIndexeddvEXT = PFNGLGETFLOATINDEXEDVEXTPROC (glfwGetProcAddress("glGetFloatIndexeddvEXT"));
01697     glGetFloati_v = PFNGLGETFLOATI_VPROC (glfwGetProcAddress("glGetFloati_v"));
01698     glGetFloati_vEXT = PFNGLGETFLOATI_VEXTPROC (glfwGetProcAddress("glGetFloati_vEXT"));
01699     glGetFloatv = PFNGLGETFLOATVPROC (glfwGetProcAddress("glGetFloatv"));
01700     glGetFragDataIndex = PFNGLGETFRAGDATAINDEXPROC (glfwGetProcAddress("glGetFragDataIndex"));
01701     glGetFragDataLocation = PFNGLGETFRAGDATALOCATIONPROC (glfwGetProcAddress("glGetFragDataLocation"));
01702     glGetFramebufferAttachmentParameteriv =
PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress("glGetFramebufferAttachmentParameteriv"));
01703     glGetFramebufferParameteriv =
PFNGLGETFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetFramebufferParameteriv"));
01704     glGetFramebufferParameterivEXT =
PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress("glGetFramebufferParameterivEXT"));
01705     glGetGraphicsResetStatus =
PFNGLGETGRAPHICSRESETSTATUSPROC (glfwGetProcAddress("glGetGraphicsResetStatus"));
01706     glGetGraphicsResetStatusARB =
PFNGLGETGRAPHICSRESETSTATUSARBPROC (glfwGetProcAddress("glGetGraphicsResetStatusARB"));
01707     glGetImageHandleARB = PFNGLGETIMAGEHANDLEARBPROC (glfwGetProcAddress("glGetImageHandleARB"));
01708     glGetImageHandleNV = PFNGLGETIMAGEHANDLENVPROC (glfwGetProcAddress("glGetImageHandleNV"));
01709     glGetInteger64i_v = PFNGLGETINTEGER64I_VPROC (glfwGetProcAddress("glGetInteger64i_v"));
01710     glGetInteger64v = PFNGLGETINTEGER64VPROC (glfwGetProcAddress("glGetInteger64v"));
01711     glGetIntegerIndexeddvEXT =
PFNGLGETINTEGERINDEXEDVEXTPROC (glfwGetProcAddress("glGetIntegerIndexeddvEXT"));
01712     glGetIntegeri_v = PFNGLGETINTEGERI_VPROC (glfwGetProcAddress("glGetIntegeri_v"));
01713     glGetIntegerui64i_vNV = PFNGLGETINTEGERUI64I_VNVPROC (glfwGetProcAddress("glGetIntegerui64i_vNV"));
01714     glGetIntegerui64vNV = PFNGLGETINTEGERUI64VNVPROC (glfwGetProcAddress("glGetIntegerui64vNV"));
01715     glGetIntegerv = PFNGLGETINTEGERVPROC (glfwGetProcAddress("glGetIntegerv"));
01716     glGetInternalformatSampleivNV =
PFNGLGETINTERNALFORMATSAMPLEIVNVPROC (glfwGetProcAddress("glGetInternalformatSampleivNV"));
01717     glGetInternalformati64v =
PFNGLGETINTERNALFORMATI64VPROC (glfwGetProcAddress("glGetInternalformati64v"));
01718     glGetInternalformativ = PFNGLGETINTERNALFORMATIVPROC (glfwGetProcAddress("glGetInternalformativ"));
01719     glGetMultiTexEnvfvEXT = PFNGLGETMULTITEXENVFVEXTPROC (glfwGetProcAddress("glGetMultiTexEnvfvEXT"));
01720     glGetMultiTexEnvivEXT = PFNGLGETMULTITEXENVIVEXTPROC (glfwGetProcAddress("glGetMultiTexEnvivEXT"));
01721     glGetMultiTexGendvEXT = PFNGLGETMULTITEXGENDVEXTPROC (glfwGetProcAddress("glGetMultiTexGendvEXT"));
01722     glGetMultiTexGenfvEXT = PFNGLGETMULTITEXGENFVEXTPROC (glfwGetProcAddress("glGetMultiTexGenfvEXT"));
01723     glGetMultiTexGenivEXT = PFNGLGETMULTITEXGENIVEXTPROC (glfwGetProcAddress("glGetMultiTexGenivEXT"));
01724     glGetMultiTexImageEXT = PFNGLGETMULTITEXIMAGEEXTPROC (glfwGetProcAddress("glGetMultiTexImageEXT"));
01725     glGetMultiTexLevelParameterfvEXT =
PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC (glfwGetProcAddress("glGetMultiTexLevelParameterfvEXT"));
01726     glGetMultiTexLevelParameterivEXT =
PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC (glfwGetProcAddress("glGetMultiTexLevelParameterivEXT"));
01727     glGetMultiTexParameterIivEXT =
PFNGLGETMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress("glGetMultiTexParameterIivEXT"));
01728     glGetMultiTexParameterIuivEXT =
PFNGLGETMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress("glGetMultiTexParameterIuivEXT"));
01729     glGetMultiTexParameterfvEXT =
PFNGLGETMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress("glGetMultiTexParameterfvEXT"));
01730     glGetMultiTexParameterivEXT =
PFNGLGETMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress("glGetMultiTexParameterivEXT"));
01731     glGetMultisamplefv = PFNGLGETMULTISAMPLEFVPROC (glfwGetProcAddress("glGetMultisamplefv"));
01732     glGetNamedBufferParameteri64v =
PFNGLGETNAMEDBUFFERPARAMETERI64VPROC (glfwGetProcAddress("glGetNamedBufferParameteri64v"));
01733     glGetNamedBufferParameteriv =
PFNGLGETNAMEDBUFFERPARAMETERIVPROC (glfwGetProcAddress("glGetNamedBufferParameteriv"));
01734     glGetNamedBufferParameterivEXT =
PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress("glGetNamedBufferParameterivEXT"));
01735     glGetNamedBufferParameterui64vNV =

```



```

    PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetNamedBufferParameterui64vNV"));
01736     glGetNamedBufferPointerv =
    PFNGLGETNAMEDBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetNamedBufferPointerv"));
01737     glGetNamedBufferPointervEXT =
    PFNGLGETNAMEDBUFFERPOINTERVEXTPROC (glfwGetProcAddress ("glGetNamedBufferPointervEXT"));
01738     glGetNamedBufferSubData =
    PFNGLGETNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetNamedBufferSubData"));
01739     glGetNamedBufferSubDataEXT =
    PFNGLGETNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glGetNamedBufferSubDataEXT"));
01740     glGetNamedFramebufferAttachmentParameteriv =
    PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameteriv"));
01741     glGetNamedFramebufferAttachmentParameterivEXT =
    PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferAttachmentParameterivEXT"));
01742     glGetNamedFramebufferParameteriv =
    PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedFramebufferParameteriv"));
01743     glGetNamedFramebufferParameterivEXT =
    PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedFramebufferParameterivEXT"));
01744     glGetNamedProgramLocalParameterIivEXT =
    PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIivEXT"));
01745     glGetNamedProgramLocalParameterIuivEXT =
    PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterIuivEXT"));
01746     glGetNamedProgramLocalParameterdvEXT =
    PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVECTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterdvEXT"));
01747     glGetNamedProgramLocalParameterfvEXT =
    PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetNamedProgramLocalParameterfvEXT"));
01748     glGetNamedProgramStringEXT =
    PFNGLGETNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glGetNamedProgramStringEXT"));
01749     glGetNamedProgramivEXT =
    PFNGLGETNAMEDPROGRAMIIVEXTPROC (glfwGetProcAddress ("glGetNamedProgramivEXT"));
01750     glGetNamedRenderbufferParameteriv =
    PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameteriv"));
01751     glGetNamedRenderbufferParameterivEXT =
    PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedRenderbufferParameterivEXT"));
01752     glGetNamedStringARB = PFNGLGETNAMEDSTRINGARBPROC (glfwGetProcAddress ("glGetNamedStringARB"));
01753     glGetNamedStringivARB = PFNGLGETNAMEDSTRINGIVARBPROC (glfwGetProcAddress ("glGetNamedStringivARB"));
01754     glGetNextPerfQueryIdINTEL =
    PFNGLGETNEXTPERQUERYIDINTELPROC (glfwGetProcAddress ("glGetNextPerfQueryIdINTEL"));
01755     glGetObjectLabel = PFNGLGETOBJECTLABELPROC (glfwGetProcAddress ("glGetObjectLabel"));
01756     glGetObjectLabelEXT = PFNGLGETOBJECTLABELEXTPROC (glfwGetProcAddress ("glGetObjectLabelEXT"));
01757     glGetObjectPtrLabel = PFNGLGETOBJECTPTRLABELPROC (glfwGetProcAddress ("glGetObjectPtrLabel"));
01758     glGetPathCommandsNV = PFNGLGETPATHCOMMANDSNVPROC (glfwGetProcAddress ("glGetPathCommandsNV"));
01759     glGetPathCoordsNV = PFNGLGETPATHCOORDSNVPROC (glfwGetProcAddress ("glGetPathCoordsNV"));
01760     glGetPathDashArrayNV = PFNGLGETPATHDASHARRAYNVPROC (glfwGetProcAddress ("glGetPathDashArrayNV"));
01761     glGetPathLengthNV = PFNGLGETPATHLENGTHNVPROC (glfwGetProcAddress ("glGetPathLengthNV"));
01762     glGetPathMetricRangeNV =
    PFNGLGETPATHMETRICRANGENVPROC (glfwGetProcAddress ("glGetPathMetricRangeNV"));
01763     glGetPathMetricsNV = PFNGLGETPATHMETRICSNVPROC (glfwGetProcAddress ("glGetPathMetricsNV"));
01764     glGetPathParameterfvNV =
    PFNGLGETPATHPARAMETERFVNVPROC (glfwGetProcAddress ("glGetPathParameterfvNV"));
01765     glGetPathParameterivNV =
    PFNGLGETPATHPARAMETERIVNVPROC (glfwGetProcAddress ("glGetPathParameterivNV"));
01766     glGetPathSpacingNV = PFNGLGETPATHSPACINGNVPROC (glfwGetProcAddress ("glGetPathSpacingNV"));
01767     glGetPerfCounterInfoINTEL =
    PFNGLGETPERFCOUNTERINFOINTELPROC (glfwGetProcAddress ("glGetPerfCounterInfoINTEL"));
01768     glGetPerfMonitorCounterDataAMD =
    PFNGLGETPERFMONITORCOUNTERDATAAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterDataAMD"));
01769     glGetPerfMonitorCounterInfoAMD =
    PFNGLGETPERFMONITORCOUNTERINFOAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterInfoAMD"));
01770     glGetPerfMonitorCounterStringAMD =
    PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCounterStringAMD"));
01771     glGetPerfMonitorCountersAMD =
    PFNGLGETPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress ("glGetPerfMonitorCountersAMD"));
01772     glGetPerfMonitorGroupStringAMD =
    PFNGLGETPERFMONITORGROUPSTRINGAMDPROC (glfwGetProcAddress ("glGetPerfMonitorGroupStringAMD"));
01773     glGetPerfMonitorGroupsAMD =
    PFNGLGETPERFMONITORGROUPSAMDPROC (glfwGetProcAddress ("glGetPerfMonitorGroupsAMD"));
01774     glGetPerfQueryDataINTEL =
    PFNGLGETPERFQUERYDATAINTELPROC (glfwGetProcAddress ("glGetPerfQueryDataINTEL"));
01775     glGetPerfQueryIdByNameINTEL =
    PFNGLGETPERFQUERYIDBYNAMEINTELPROC (glfwGetProcAddress ("glGetPerfQueryIdByNameINTEL"));
01776     glGetPerfQueryInfoINTEL =
    PFNGLGETPERFQUERYINFOINTELPROC (glfwGetProcAddress ("glGetPerfQueryInfoINTEL"));
01777     glGetPointerIndexedvEXT =
    PFNGLGETPOINTERINDEXEDVEXTPROC (glfwGetProcAddress ("glGetPointerIndexedvEXT"));
01778     glGetPointerivEXT = PFNGLGETPOINTERIVEXTPROC (glfwGetProcAddress ("glGetPointerivEXT"));
01779     glGetPointerv = PFNGLGETPOINTERVPROC (glfwGetProcAddress ("glGetPointerv"));
01780     glGetProgramBinary = PFNGLGETPROGRAMBINARYPROC (glfwGetProcAddress ("glGetProgramBinary"));
01781     glGetProgramInfoLog = PFNGLGETPROGRAMINFOLOGPROC (glfwGetProcAddress ("glGetProgramInfoLog"));
01782     glGetProgramInterfaceiv =
    PFNGLGETPROGRAMINTERFACEIVPROC (glfwGetProcAddress ("glGetProgramInterfaceiv"));
01783     glGetProgramPipelineInfoLog =
    PFNGLGETPROGRAMPIPELINEINFOLOGPROC (glfwGetProcAddress ("glGetProgramPipelineInfoLog"));
01784     glGetProgramPipelineiv =
    PFNGLGETPROGRAMPIPELINEIVPROC (glfwGetProcAddress ("glGetProgramPipelineiv"));
01785     glGetProgramResourceIndex =
    PFNGLGETPROGRAMRESOURCEINDEXPROC (glfwGetProcAddress ("glGetProgramResourceIndex"));
01786     glGetProgramResourceLocation =

```

```

    PFNGLGETPROGRAMRESOURCELOCATIONPROC (glfwGetProcAddress ("glGetProgramResourceLocation"));
01787   glGetProgramResourceLocationIndex =
    PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC (glfwGetProcAddress ("glGetProgramResourceLocationIndex"));
01788   glGetProgramResourceName =
    PFNGLGETPROGRAMRESOURCENAMEPROC (glfwGetProcAddress ("glGetProgramResourceName"));
01789   glGetProgramResourcefvNV =
    PFNGLGETPROGRAMRESOURCEFVNVPROC (glfwGetProcAddress ("glGetProgramResourcefvNV"));
01790   glGetProgramResourceiv =
    PFNGLGETPROGRAMRESOURCEIVPROC (glfwGetProcAddress ("glGetProgramResourceiv"));
01791   glGetProgramStageiv = PFNGLGETPROGRAMSTAGEIVPROC (glfwGetProcAddress ("glGetProgramStageiv"));
01792   glGetProgramiv = PFNGLGETPROGRAMIVPROC (glfwGetProcAddress ("glGetProgramiv"));
01793   glGetQueryBufferObjecti64v =
    PFNGLGETQUERYBUFFEROBJECTI64VPROC (glfwGetProcAddress ("glGetQueryBufferObjecti64v"));
01794   glGetQueryBufferObjectiv =
    PFNGLGETQUERYBUFFEROBJECTIVPROC (glfwGetProcAddress ("glGetQueryBufferObjectiv"));
01795   glGetQueryBufferObjectui64v =
    PFNGLGETQUERYBUFFEROBJECTUI64VPROC (glfwGetProcAddress ("glGetQueryBufferObjectui64v"));
01796   glGetQueryBufferObjectuiv =
    PFNGLGETQUERYBUFFEROBJECTUIVPROC (glfwGetProcAddress ("glGetQueryBufferObjectuiv"));
01797   glGetQueryIndexediv = PFNGLGETQUERYINDEXEDIVPROC (glfwGetProcAddress ("glGetQueryIndexediv"));
01798   glGetQueryObjecti64v = PFNGLGETQUERYOBJECTI64VPROC (glfwGetProcAddress ("glGetQueryObjecti64v"));
01799   glGetQueryObjectiv = PFNGLGETQUERYOBJECTIVPROC (glfwGetProcAddress ("glGetQueryObjectiv"));
01800   glGetQueryObjectui64v = PFNGLGETQUERYOBJECTUI64VPROC (glfwGetProcAddress ("glGetQueryObjectui64v"));
01801   glGetQueryObjectuiv = PFNGLGETQUERYOBJECTUIVPROC (glfwGetProcAddress ("glGetQueryObjectuiv"));
01802   glGetQueryiv = PFNGLGETQUERYIVPROC (glfwGetProcAddress ("glGetQueryiv"));
01803   glGetRenderbufferParameteriv =
    PFNGLGETRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetRenderbufferParameteriv"));
01804   glGetSamplerParameterIiv =
    PFNGLGETSAMPLERPARAMETERIIVPROC (glfwGetProcAddress ("glGetSamplerParameterIiv"));
01805   glGetSamplerParameterIuiv =
    PFNGLGETSAMPLERPARAMETERUIVPROC (glfwGetProcAddress ("glGetSamplerParameterIuiv"));
01806   glGetSamplerParameterfiv =
    PFNGLGETSAMPLERPARAMETERFVPROC (glfwGetProcAddress ("glGetSamplerParameterfiv"));
01807   glGetSamplerParameteriv =
    PFNGLGETSAMPLERPARAMETERIVPROC (glfwGetProcAddress ("glGetSamplerParameteriv"));
01808   glGetShaderInfoLog = PFNGLGETSHADERINFOLOGPROC (glfwGetProcAddress ("glGetShaderInfoLog"));
01809   glGetShaderPrecisionFormat =
    PFNGLGETSHADERPRECISIONFORMATPROC (glfwGetProcAddress ("glGetShaderPrecisionFormat"));
01810   glGetShaderSource = PFNGLGETSHADERSOURCEPROC (glfwGetProcAddress ("glGetShaderSource"));
01811   glGetShaderiv = PFNGLGETSHADERIVPROC (glfwGetProcAddress ("glGetShaderiv"));
01812   glGetStageIndexNV = PFNGLGETSTAGEINDEXNVPROC (glfwGetProcAddress ("glGetStageIndexNV"));
01813   glGetString = PFNGLGETSTRINGPROC (glfwGetProcAddress ("glGetString"));
01814   glGetStringi = PFNGLGETSTRINGIPROC (glfwGetProcAddress ("glGetStringi"));
01815   glGetSubroutineIndex = PFNGLGETSUBROUTINEINDEXPROC (glfwGetProcAddress ("glGetSubroutineIndex"));
01816   glGetSubroutineUniformLocation =
    PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC (glfwGetProcAddress ("glGetSubroutineUniformLocation"));
01817   glGetSynciv = PFNGLGETSYNCIVPROC (glfwGetProcAddress ("glGetSynciv"));
01818   glGetTexImage = PFNGLGETTEXIMAGEPROC (glfwGetProcAddress ("glGetTexImage"));
01819   glGetTexLevelParameterfiv =
    PFNGLGETTEXLEVELPARAMETERFVPROC (glfwGetProcAddress ("glGetTexLevelParameterfiv"));
01820   glGetTexLevelParameteriv =
    PFNGLGETTEXLEVELPARAMETERIVPROC (glfwGetProcAddress ("glGetTexLevelParameteriv"));
01821   glGetTexParameterIiv = PFNGLGETTEXPARAMETERIIVPROC (glfwGetProcAddress ("glGetTexParameterIiv"));
01822   glGetTexParameterIuiv = PFNGLGETTEXPARAMETERUIVPROC (glfwGetProcAddress ("glGetTexParameterIuiv"));
01823   glGetTexParameterfiv = PFNGLGETTEXPARAMETERFVPROC (glfwGetProcAddress ("glGetTexParameterfiv"));
01824   glGetTexParameteriv = PFNGLGETTEXPARAMETERIVPROC (glfwGetProcAddress ("glGetTexParameteriv"));
01825   glGetTextureHandleARB = PFNGLGETTEXTUREHANDLEARBPROC (glfwGetProcAddress ("glGetTextureHandleARB"));
01826   glGetTextureHandleNV = PFNGLGETTEXTUREHANDLENVPROC (glfwGetProcAddress ("glGetTextureHandleNV"));
01827   glGetTextureImage = PFNGLGETTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetTextureImage"));
01828   glGetTextureImageEXT = PFNGLGETTEXTUREIMAGEEXTPROC (glfwGetProcAddress ("glGetTextureImageEXT"));
01829   glGetTextureLevelParameterfiv =
    PFNGLGETTEXTURELEVELPARAMETERFVPROC (glfwGetProcAddress ("glGetTextureLevelParameterfiv"));
01830   glGetTextureLevelParameterfivEXT =
    PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetTextureLevelParameterfivEXT"));
01831   glGetTextureLevelParameteriv =
    PFNGLGETTEXTURELEVELPARAMETERIVPROC (glfwGetProcAddress ("glGetTextureLevelParameteriv"));
01832   glGetTextureLevelParameterivEXT =
    PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetTextureLevelParameterivEXT"));
01833   glGetTextureParameterIiv =
    PFNGLGETTEXTUREPARAMETERIIVPROC (glfwGetProcAddress ("glGetTextureParameterIiv"));
01834   glGetTextureParameterIivEXT =
    PFNGLGETTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterIivEXT"));
01835   glGetTextureParameterIuiv =
    PFNGLGETTEXTUREPARAMETERUIVPROC (glfwGetProcAddress ("glGetTextureParameterIuiv"));
01836   glGetTextureParameterIuivEXT =
    PFNGLGETTEXTUREPARAMETERUIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterIuivEXT"));
01837   glGetTextureParameterfiv =
    PFNGLGETTEXTUREPARAMETERFVPROC (glfwGetProcAddress ("glGetTextureParameterfiv"));
01838   glGetTextureParameterfivEXT =
    PFNGLGETTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetTextureParameterfivEXT"));
01839   glGetTextureParameteriv =
    PFNGLGETTEXTUREPARAMETERIVPROC (glfwGetProcAddress ("glGetTextureParameteriv"));
01840   glGetTextureParameterivEXT =
    PFNGLGETTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterivEXT"));
01841   glGetTextureSamplerHandleARB =
    PFNGLGETTEXTURESAMPLERHANDLEARBPROC (glfwGetProcAddress ("glGetTextureSamplerHandleARB"));
01842   glGetTextureSamplerHandleNV =

```

```

    PFNGLGETTEXTURESAMPLERHANDLENVPROC (glfwGetProcAddress ("glGetTextureSamplerHandleNV"));
01843   glGetTextureSubImage = PFNGLGETTEXTURESUBIMAGEPROC (glfwGetProcAddress ("glGetTextureSubImage"));
01844   glGetTransformFeedbackVarying =
    PFNGLGETTRANSFORMFEEDBACKVARYINGPROC (glfwGetProcAddress ("glGetTransformFeedbackVarying"));
01845   glGetTransformFeedbacki64_v =
    PFNGLGETTRANSFORMFEEDBACKI64_VPROC (glfwGetProcAddress ("glGetTransformFeedbacki64_v"));
01846   glGetTransformFeedbacki_v =
    PFNGLGETTRANSFORMFEEDBACKI_VPROC (glfwGetProcAddress ("glGetTransformFeedbacki_v"));
01847   glGetTransformFeedbackiv =
    PFNGLGETTRANSFORMFEEDBACKIVPROC (glfwGetProcAddress ("glGetTransformFeedbackiv"));
01848   glGetUniformBlockIndex =
    PFNGLGETUNIFORMBLOCKINDEXPROC (glfwGetProcAddress ("glGetUniformBlockIndex"));
01849   glGetUniformIndices = PFNGLGETUNIFORMINDICESPROC (glfwGetProcAddress ("glGetUniformIndices"));
01850   glGetUniformLocation = PFNGLGETUNIFORMLOCATIONPROC (glfwGetProcAddress ("glGetUniformLocation"));
01851   glGetUniformSubroutineuiv =
    PFNGLGETUNIFORMSUBROUTINEUIVPROC (glfwGetProcAddress ("glGetUniformSubroutineuiv"));
01852   glGetUniformdv = PFNGLGETUNIFORMDVPROC (glfwGetProcAddress ("glGetUniformdv"));
01853   glGetUniformfv = PFNGLGETUNIFORMFVPROC (glfwGetProcAddress ("glGetUniformfv"));
01854   glGetUniformi64vARB = PFNGLGETUNIFORMI64VARBPROC (glfwGetProcAddress ("glGetUniformi64vARB"));
01855   glGetUniformi64vNV = PFNGLGETUNIFORMI64VNVPROC (glfwGetProcAddress ("glGetUniformi64vNV"));
01856   glGetUniformiv = PFNGLGETUNIFORMIVPROC (glfwGetProcAddress ("glGetUniformiv"));
01857   glGetUniformui64vARB = PFNGLGETUNIFORMUI64VARBPROC (glfwGetProcAddress ("glGetUniformui64vARB"));
01858   glGetUniformui64vNV = PFNGLGETUNIFORMUI64VNVPROC (glfwGetProcAddress ("glGetUniformui64vNV"));
01859   glGetUniformuiv = PFNGLGETUNIFORMUIVPROC (glfwGetProcAddress ("glGetUniformuiv"));
01860   glGetVertexArrayIndexed64iv =
    PFNGLGETVERTEXARRAYINDEXED64IVPROC (glfwGetProcAddress ("glGetVertexArrayIndexed64iv"));
01861   glGetVertexArrayIndexedi_v =
    PFNGLGETVERTEXARRAYINDEXEDI_VPROC (glfwGetProcAddress ("glGetVertexArrayIndexedi_v"));
01862   glGetVertexArrayIntegeri_vEXT =
    PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC (glfwGetProcAddress ("glGetVertexArrayIntegeri_vEXT"));
01863   glGetVertexArrayIntegervEXT =
    PFNGLGETVERTEXARRAYINTEGERVEXTPROC (glfwGetProcAddress ("glGetVertexArrayIntegervEXT"));
01864   glGetVertexArrayPointeri_vEXT =
    PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC (glfwGetProcAddress ("glGetVertexArrayPointeri_vEXT"));
01865   glGetVertexArrayPointervEXT =
    PFNGLGETVERTEXARRAYPOINTERVEXTPROC (glfwGetProcAddress ("glGetVertexArrayPointervEXT"));
01866   glGetVertexArrayiv = PFNGLGETVERTEXARRAYIVPROC (glfwGetProcAddress ("glGetVertexArrayiv"));
01867   glGetVertexAttribi_v = PFNGLGETVERTEXATTRIBI_VPROC (glfwGetProcAddress ("glGetVertexAttribi_v"));
01868   glGetVertexAttribiui_v = PFNGLGETVERTEXATTRIBUIVPROC (glfwGetProcAddress ("glGetVertexAttribiui_v"));
01869   glGetVertexAttribLdv = PFNGLGETVERTEXATTRIBLDVPROC (glfwGetProcAddress ("glGetVertexAttribLdv"));
01870   glGetVertexAttribLi64vNV =
    PFNGLGETVERTEXATTRIBLI64VNVPROC (glfwGetProcAddress ("glGetVertexAttribLi64vNV"));
01871   glGetVertexAttribLui64vARB =
    PFNGLGETVERTEXATTRIBLUI64VARBPROC (glfwGetProcAddress ("glGetVertexAttribLui64vARB"));
01872   glGetVertexAttribLui64vNV =
    PFNGLGETVERTEXATTRIBLUI64VNVPROC (glfwGetProcAddress ("glGetVertexAttribLui64vNV"));
01873   glGetVertexAttribPointerv =
    PFNGLGETVERTEXATTRIBPOINTERVPROC (glfwGetProcAddress ("glGetVertexAttribPointerv"));
01874   glGetVertexAttribdv = PFNGLGETVERTEXATTRIBDVPROC (glfwGetProcAddress ("glGetVertexAttribdv"));
01875   glGetVertexAttribfv = PFNGLGETVERTEXATTRIBFVPROC (glfwGetProcAddress ("glGetVertexAttribfv"));
01876   glGetVertexAttribiv = PFNGLGETVERTEXATTRIBIVPROC (glfwGetProcAddress ("glGetVertexAttribiv"));
01877   glGetVkProcAddrNV = PFNGLGETVKPROCADDRNVPROC (glfwGetProcAddress ("glGetVkProcAddrNV"));
01878   glGetnCompressedTexImage =
    PFNGLGETNCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress ("glGetnCompressedTexImage"));
01879   glGetnCompressedTexImageARB =
    PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC (glfwGetProcAddress ("glGetnCompressedTexImageARB"));
01880   glGetnTexImage = PFNGLGETNTEXIMAGEPROC (glfwGetProcAddress ("glGetnTexImage"));
01881   glGetnTexImageARB = PFNGLGETNTEXIMAGEARBPROC (glfwGetProcAddress ("glGetnTexImageARB"));
01882   glGetnUniformdv = PFNGLGETNUNIFORMDVPROC (glfwGetProcAddress ("glGetnUniformdv"));
01883   glGetnUniformdvarb = PFNGLGETNUNIFORMDVARBPROC (glfwGetProcAddress ("glGetnUniformdvarb"));
01884   glGetnUniformfv = PFNGLGETNUNIFORMFVPROC (glfwGetProcAddress ("glGetnUniformfv"));
01885   glGetnUniformfvarb = PFNGLGETNUNIFORMFVARBPROC (glfwGetProcAddress ("glGetnUniformfvarb"));
01886   glGetnUniformi64vARB = PFNGLGETNUNIFORMI64VARBPROC (glfwGetProcAddress ("glGetnUniformi64vARB"));
01887   glGetnUniformiv = PFNGLGETNUNIFORMIVPROC (glfwGetProcAddress ("glGetnUniformiv"));
01888   glGetnUniformivarb = PFNGLGETNUNIFORMIVARBPROC (glfwGetProcAddress ("glGetnUniformivarb"));
01889   glGetnUniformui64vARB = PFNGLGETNUNIFORMUI64VARBPROC (glfwGetProcAddress ("glGetnUniformui64vARB"));
01890   glGetnUniformuiv = PFNGLGETNUNIFORMUIVPROC (glfwGetProcAddress ("glGetnUniformuiv"));
01891   glGetnUniformuivarb = PFNGLGETNUNIFORMUIVARBPROC (glfwGetProcAddress ("glGetnUniformuivarb"));
01892   glHint = PFNGLHINTPROC (glfwGetProcAddress ("glHint"));
01893   glIndexFormatNV = PFNGLINDEXFORMATNVPROC (glfwGetProcAddress ("glIndexFormatNV"));
01894   glInsertEventMarkerEXT =
    PFNGLINSERTEVENTMARKEREXTPROC (glfwGetProcAddress ("glInsertEventMarkerEXT"));
01895   glInterpolatePathsNV = PFNGLINTERPOLATEPATHSNVPROC (glfwGetProcAddress ("glInterpolatePathsNV"));
01896   glInvalidateBufferData =
    PFNGLINVALIDATEBUFFERDATAPROC (glfwGetProcAddress ("glInvalidateBufferData"));
01897   glInvalidateBufferSubData =
    PFNGLINVALIDATEBUFFERSUBDATAPROC (glfwGetProcAddress ("glInvalidateBufferSubData"));
01898   glInvalidateFramebuffer =
    PFNGLINVALIDATEFRAMEBUFFERPROC (glfwGetProcAddress ("glInvalidateFramebuffer"));
01899   glInvalidateNamedFramebufferData =
    PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC (glfwGetProcAddress ("glInvalidateNamedFramebufferData"));
01900   glInvalidateNamedFramebufferSubData =
    PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC (glfwGetProcAddress ("glInvalidateNamedFramebufferSubData"));
01901   glInvalidateSubFramebuffer =
    PFNGLINVALIDATESUBFRAMEBUFFERPROC (glfwGetProcAddress ("glInvalidateSubFramebuffer"));
01902   glInvalidateTexImage = PFNGLINVALIDATETEXIMAGEPROC (glfwGetProcAddress ("glInvalidateTexImage"));
01903   glInvalidateTexSubImage =

```

```

PFNGLINVALIDATETEXSUBIMAGEPROC (glfwGetProcAddress("glInvalidateTexSubImage"));
01904 glIsBuffer = PFNGLISBUFFERPROC (glfwGetProcAddress("glIsBuffer"));
01905 glIsBufferResidentNV = PFNGLISBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsBufferResidentNV"));
01906 glIsCommandListNV = PFNGLISCOMMANDLISTNVPROC (glfwGetProcAddress("glIsCommandListNV"));
01907 glIsEnabled = PFNGLISENABLEDPROC (glfwGetProcAddress("glIsEnabled"));
01908 glIsEnabledIndexedEXT = PFNGLISENABLEDINDEXEDEXTPROC (glfwGetProcAddress("glIsEnabledIndexedEXT"));
01909 glIsEnabledi = PFNGLISENABLEDIPROC (glfwGetProcAddress("glIsEnabledi"));
01910 glIsFramebuffer = PFNGLISFRAMEBUFFERPROC (glfwGetProcAddress("glIsFramebuffer"));
01911 glIsImageHandleResidentARB =
PFNGLISIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsImageHandleResidentARB"));
01912 glIsImageHandleResidentNV =
PFNGLISIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsImageHandleResidentNV"));
01913 glIsNamedBufferResidentNV =
PFNGLISNAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsNamedBufferResidentNV"));
01914 glIsNamedStringARB = PFNGLISNAMEDSTRINGARBPROC (glfwGetProcAddress("glIsNamedStringARB"));
01915 glIsPathNV = PFNGLISPATHNVPROC (glfwGetProcAddress("glIsPathNV"));
01916 glIsPointInFillPathNV = PFNGLISPOINTINFILLPATHNVPROC (glfwGetProcAddress("glIsPointInFillPathNV"));
01917 glIsPointInStrokePathNV =
PFNGLISPOINTINSTROKEPATHNVPROC (glfwGetProcAddress("glIsPointInStrokePathNV"));
01918 glIsProgram = PFNGLISPROGRAMPROC (glfwGetProcAddress("glIsProgram"));
01919 glIsProgramPipeline = PFNGLISPROGRAMPIPELINEPROC (glfwGetProcAddress("glIsProgramPipeline"));
01920 glIsQuery = PFNGLISQUERYPROC (glfwGetProcAddress("glIsQuery"));
01921 glIsRenderbuffer = PFNGLISRENDERBUFFERPROC (glfwGetProcAddress("glIsRenderbuffer"));
01922 glIsSampler = PFNGLISSAMPLERPROC (glfwGetProcAddress("glIsSampler"));
01923 glIsShader = PFNGLISSHADERPROC (glfwGetProcAddress("glIsShader"));
01924 glIsStateNV = PFNGLISSTATENVPROC (glfwGetProcAddress("glIsStateNV"));
01925 glIsSync = PFNGLISSYNCPROC (glfwGetProcAddress("glIsSync"));
01926 glIsTexture = PFNGLISTEXTUREPROC (glfwGetProcAddress("glIsTexture"));
01927 glIsTextureHandleResidentARB =
PFNGLISTEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsTextureHandleResidentARB"));
01928 glIsTextureHandleResidentNV =
PFNGLISTEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsTextureHandleResidentNV"));
01929 glIsTransformFeedback = PFNGLISTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glIsTransformFeedback"));
01930 glIsVertexArray = PFNGLISVERTEXARRAYPROC (glfwGetProcAddress("glIsVertexArray"));
01931 glLabelObjectEXT = PFNGLLABELOBJECTEXTPROC (glfwGetProcAddress("glLabelObjectEXT"));
01932 glLineWidth = PFNGLLINEWIDTHPROC (glfwGetProcAddress("glLineWidth"));
01933 glLinkProgram = PFNGLLINKPROGRAMPROC (glfwGetProcAddress("glLinkProgram"));
01934 glListDrawCommandsStatesClientNV =
PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC (glfwGetProcAddress("glListDrawCommandsStatesClientNV"));
01935 glLogicOp = PFNGLLOGICOPPROC (glfwGetProcAddress("glLogicOp"));
01936 glMakeBufferNonResidentNV =
PFNGLMAKEBUFFERNONRESIDENTNVPROC (glfwGetProcAddress("glMakeBufferNonResidentNV"));
01937 glMakeBufferResidentNV =
PFNGLMAKEBUFFERRESIDENTNVPROC (glfwGetProcAddress("glMakeBufferResidentNV"));
01938 glMakeImageHandleNonResidentARB =
PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC (glfwGetProcAddress("glMakeImageHandleNonResidentARB"));
01939 glMakeImageHandleNonResidentNV =
PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC (glfwGetProcAddress("glMakeImageHandleNonResidentNV"));
01940 glMakeImageHandleResidentARB =
PFNGLMAKEIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glMakeImageHandleResidentARB"));
01941 glMakeImageHandleResidentNV =
PFNGLMAKEIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glMakeImageHandleResidentNV"));
01942 glMakeNamedBufferNonResidentNV =
PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC (glfwGetProcAddress("glMakeNamedBufferNonResidentNV"));
01943 glMakeNamedBufferResidentNV =
PFNGLMAKENAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glMakeNamedBufferResidentNV"));
01944 glMakeTextureHandleNonResidentARB =
PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC (glfwGetProcAddress("glMakeTextureHandleNonResidentARB"));
01945 glMakeTextureHandleNonResidentNV =
PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC (glfwGetProcAddress("glMakeTextureHandleNonResidentNV"));
01946 glMakeTextureHandleResidentARB =
PFNGLMAKETEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress("glMakeTextureHandleResidentARB"));
01947 glMakeTextureHandleResidentNV =
PFNGLMAKETEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress("glMakeTextureHandleResidentNV"));
01948 glMapBuffer = PFNGLMAPBUFFERPROC (glfwGetProcAddress("glMapBuffer"));
01949 glMapBufferRange = PFNGLMAPBUFFERRANGEPROC (glfwGetProcAddress("glMapBufferRange"));
01950 glMapNamedBuffer = PFNGLMAPNAMEDBUFFERPROC (glfwGetProcAddress("glMapNamedBuffer"));
01951 glMapNamedBufferEXT = PFNGLMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress("glMapNamedBufferEXT"));
01952 glMapNamedBufferRange = PFNGLMAPNAMEDBUFFERRANGEPROC (glfwGetProcAddress("glMapNamedBufferRange"));
01953 glMapNamedBufferRangeEXT =
PFNGLMAPNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress("glMapNamedBufferRangeEXT"));
01954 glMatrixFrustumEXT = PFNGLMATRIXFRUSTUMEXTPROC (glfwGetProcAddress("glMatrixFrustumEXT"));
01955 glMatrixLoad3x2fNV = PFNGLMATRIXLOAD3X2FNVPROC (glfwGetProcAddress("glMatrixLoad3x2fNV"));
01956 glMatrixLoad3x3fNV = PFNGLMATRIXLOAD3X3FNVPROC (glfwGetProcAddress("glMatrixLoad3x3fNV"));
01957 glMatrixLoadIdentityEXT =
PFNGLMATRIXLOADIDENTITYEXTPROC (glfwGetProcAddress("glMatrixLoadIdentityEXT"));
01958 glMatrixLoadTranspose3x3fNV =
PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC (glfwGetProcAddress("glMatrixLoadTranspose3x3fNV"));
01959 glMatrixLoadTransposedEXT =
PFNGLMATRIXLOADTRANPOSEDEXTPROC (glfwGetProcAddress("glMatrixLoadTransposedEXT"));
01960 glMatrixLoadTransposefEXT =
PFNGLMATRIXLOADTRANPOSEFEFEXTPROC (glfwGetProcAddress("glMatrixLoadTransposefEXT"));
01961 glMatrixLoadEXT = PFNGLMATRIXLOADDEXTPROC (glfwGetProcAddress("glMatrixLoadEXT"));
01962 glMatrixLoadfEXT = PFNGLMATRIXLOADFEFEXTPROC (glfwGetProcAddress("glMatrixLoadfEXT"));
01963 glMatrixMult3x2fNV = PFNGLMATRIXMULT3X2FNVPROC (glfwGetProcAddress("glMatrixMult3x2fNV"));
01964 glMatrixMult3x3fNV = PFNGLMATRIXMULT3X3FNVPROC (glfwGetProcAddress("glMatrixMult3x3fNV"));
01965 glMatrixMultTranspose3x3fNV =

```



```

    PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC (glfwGetProcAddress ("glMatrixMultTranspose3x3fNV"));
01966 glMatrixMultTransposedEXT =
    PFNGLMATRIXMULTTRANSPOSEDEXTPROC (glfwGetProcAddress ("glMatrixMultTransposedEXT"));
01967 glMatrixMultTransposefEXT =
    PFNGLMATRIXMULTTRANSPOSEFEXTPROC (glfwGetProcAddress ("glMatrixMultTransposefEXT"));
01968 glMatrixMultdEXT = PFNGLMATRIXMULTDEXTPROC (glfwGetProcAddress ("glMatrixMultdEXT"));
01969 glMatrixMultfEXT = PFNGLMATRIXMULTFEXTPROC (glfwGetProcAddress ("glMatrixMultfEXT"));
01970 glMatrixOrthoEXT = PFNGLMATRIXORTHOEXTPROC (glfwGetProcAddress ("glMatrixOrthoEXT"));
01971 glMatrixPopEXT = PFNGLMATRIXPOEXTPROC (glfwGetProcAddress ("glMatrixPopEXT"));
01972 glMatrixPushEXT = PFNGLMATRIXPUSHEXTPROC (glfwGetProcAddress ("glMatrixPushEXT"));
01973 glMatrixRotatedEXT = PFNGLMATRIXROTATEDEXTPROC (glfwGetProcAddress ("glMatrixRotatedEXT"));
01974 glMatrixRotatefEXT = PFNGLMATRIXROTATEFEXTPROC (glfwGetProcAddress ("glMatrixRotatefEXT"));
01975 glMatrixScaledEXT = PFNGLMATRIXSCALEDEXTPROC (glfwGetProcAddress ("glMatrixScaledEXT"));
01976 glMatrixScalefEXT = PFNGLMATRIXSCALEFEXTPROC (glfwGetProcAddress ("glMatrixScalefEXT"));
01977 glMatrixTranslatedEXT = PFNGLMATRIXTRANSLATEDEXTPROC (glfwGetProcAddress ("glMatrixTranslatedEXT"));
01978 glMatrixTranslatefEXT = PFNGLMATRIXTRANSLATEFEXTPROC (glfwGetProcAddress ("glMatrixTranslatefEXT"));
01979 glMaxShaderCompilerThreadsARB =
    PFNGLMAXSHADERCOMPILETHREADSARBPROC (glfwGetProcAddress ("glMaxShaderCompilerThreadsARB"));
01980 glMemoryBarrier = PFNGLMEMORYBARRIERPROC (glfwGetProcAddress ("glMemoryBarrier"));
01981 glMemoryBarrierByRegion =
    PFNGLMEMORYBARRIERBYREGIONPROC (glfwGetProcAddress ("glMemoryBarrierByRegion"));
01982 glMinSampleShading = PFNGLMINSAMPLESHADINGPROC (glfwGetProcAddress ("glMinSampleShading"));
01983 glMinSampleShadingARB = PFNGLMINSAMPLESHADINGARBPROC (glfwGetProcAddress ("glMinSampleShadingARB"));
01984 glMultiDrawArrays = PFNGLMULTIDRAWARRAYSPROC (glfwGetProcAddress ("glMultiDrawArrays"));
01985 glMultiDrawArraysIndirect =
    PFNGLMULTIDRAWARRAYSINDIRECTPROC (glfwGetProcAddress ("glMultiDrawArraysIndirect"));
01986 glMultiDrawArraysIndirectBindlessCountNV =
    PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectBindlessCountNV"));
01987 glMultiDrawArraysIndirectBindlessNV =
    PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectBindlessNV"));
01988 glMultiDrawArraysIndirectCountARB =
    PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectCountARB"));
01989 glMultiDrawElements = PFNGLMULTIDRAWELEMENTSPROC (glfwGetProcAddress ("glMultiDrawElements"));
01990 glMultiDrawElementsBaseVertex =
    PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress ("glMultiDrawElementsBaseVertex"));
01991 glMultiDrawElementsIndirect =
    PFNGLMULTIDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress ("glMultiDrawElementsIndirect"));
01992 glMultiDrawElementsIndirectBindlessCountNV =
    PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectBindlessCountNV"));
01993 glMultiDrawElementsIndirectBindlessNV =
    PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectBindlessNV"));
01994 glMultiDrawElementsIndirectCountARB =
    PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectCountARB"));
01995 glMultiTexBufferEXT = PFNGLMULTITEXBUFFEREXTPROC (glfwGetProcAddress ("glMultiTexBufferEXT"));
01996 glMultiTexCoordPointerEXT =
    PFNGLMULTITEXCOORDPOINTEREXTPROC (glfwGetProcAddress ("glMultiTexCoordPointerEXT"));
01997 glMultiTexEnvfEXT = PFNGLMULTITEXENVFEXTPROC (glfwGetProcAddress ("glMultiTexEnvfEXT"));
01998 glMultiTexEnvfvEXT = PFNGLMULTITEXENVFVEXTPROC (glfwGetProcAddress ("glMultiTexEnvfvEXT"));
01999 glMultiTexEnvfEXT = PFNGLMULTITEXENVFEXTPROC (glfwGetProcAddress ("glMultiTexEnvfEXT"));
02000 glMultiTexEnvivEXT = PFNGLMULTITEXENVIVEXTPROC (glfwGetProcAddress ("glMultiTexEnvivEXT"));
02001 glMultiTexGendEXT = PFNGLMULTITEXGENDEXTPROC (glfwGetProcAddress ("glMultiTexGendEXT"));
02002 glMultiTexGendvEXT = PFNGLMULTITEXGENDVEXTPROC (glfwGetProcAddress ("glMultiTexGendvEXT"));
02003 glMultiTexGenfEXT = PFNGLMULTITEXGENFEXTPROC (glfwGetProcAddress ("glMultiTexGenfEXT"));
02004 glMultiTexGenfvEXT = PFNGLMULTITEXGENFVEXTPROC (glfwGetProcAddress ("glMultiTexGenfvEXT"));
02005 glMultiTexGeniEXT = PFNGLMULTITEXGENIEXTPROC (glfwGetProcAddress ("glMultiTexGeniEXT"));
02006 glMultiTexGenivEXT = PFNGLMULTITEXGENIVEXTPROC (glfwGetProcAddress ("glMultiTexGenivEXT"));
02007 glMultiTexImage1DEXT = PFNGLMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glMultiTexImage1DEXT"));
02008 glMultiTexImage2DEXT = PFNGLMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glMultiTexImage2DEXT"));
02009 glMultiTexImage3DEXT = PFNGLMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress ("glMultiTexImage3DEXT"));
02010 glMultiTexParameterIivEXT =
    PFNGLMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress ("glMultiTexParameterIivEXT"));
02011 glMultiTexParameterIuivEXT =
    PFNGLMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glMultiTexParameterIuivEXT"));
02012 glMultiTexParameterfEXT =
    PFNGLMULTITEXPARAMETERFEXTPROC (glfwGetProcAddress ("glMultiTexParameterfEXT"));
02013 glMultiTexParameterfvEXT =
    PFNGLMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress ("glMultiTexParameterfvEXT"));
02014 glMultiTexParameteriEXT =
    PFNGLMULTITEXPARAMETERIEXTPROC (glfwGetProcAddress ("glMultiTexParameteriEXT"));
02015 glMultiTexParameterivEXT =
    PFNGLMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress ("glMultiTexParameterivEXT"));
02016 glMultiTexRenderbufferEXT =
    PFNGLMULTITEXRENDERBUFFEREXTPROC (glfwGetProcAddress ("glMultiTexRenderbufferEXT"));
02017 glMultiTexSubImage1DEXT =
    PFNGLMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glMultiTexSubImage1DEXT"));
02018 glMultiTexSubImage2DEXT =
    PFNGLMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glMultiTexSubImage2DEXT"));
02019 glMultiTexSubImage3DEXT =
    PFNGLMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glMultiTexSubImage3DEXT"));
02020 glNamedBufferData = PFNGLNAMEDBUFFERDATAPROC (glfwGetProcAddress ("glNamedBufferData"));
02021 glNamedBufferDataEXT = PFNGLNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress ("glNamedBufferDataEXT"));
02022 glNamedBufferPageCommitmentARB =
    PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress ("glNamedBufferPageCommitmentARB"));
02023 glNamedBufferPageCommitmentEXT =
    PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC (glfwGetProcAddress ("glNamedBufferPageCommitmentEXT"));
02024 glNamedBufferStorage = PFNGLNAMEDBUFFERSTORAGEPROC (glfwGetProcAddress ("glNamedBufferStorage"));
02025 glNamedBufferStorageEXT =

```

```

PFNGLNAMEDBUFFERSTORAGEEXTPROC (glfwGetProcAddress ("glNamedBufferStorageEXT"));
02026   glNamedBufferSubData = PFNGLNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glNamedBufferSubData"));
02027   glNamedBufferSubDataEXT =
PFNGLNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glNamedBufferSubDataEXT"));
02028   glNamedCopyBufferSubDataEXT =
PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glNamedCopyBufferSubDataEXT"));
02029   glNamedFramebufferDrawBuffer =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferDrawBuffer"));
02030   glNamedFramebufferDrawBuffers =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC (glfwGetProcAddress ("glNamedFramebufferDrawBuffers"));
02031   glNamedFramebufferParameteri =
PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress ("glNamedFramebufferParameteri"));
02032   glNamedFramebufferParameteriEXT =
PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC (glfwGetProcAddress ("glNamedFramebufferParameteriEXT"));
02033   glNamedFramebufferReadBuffer =
PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferReadBuffer"));
02034   glNamedFramebufferRenderbuffer =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferRenderbuffer"));
02035   glNamedFramebufferRenderbufferEXT =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC (glfwGetProcAddress ("glNamedFramebufferRenderbufferEXT"));
02036   glNamedFramebufferSampleLocationsfvARB =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress ("glNamedFramebufferSampleLocationsfvARB"));
02037   glNamedFramebufferSampleLocationsfvNV =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNPROC (glfwGetProcAddress ("glNamedFramebufferSampleLocationsfvNV"));
02038   glNamedFramebufferTexture =
PFNGLNAMEDFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress ("glNamedFramebufferTexture"));
02039   glNamedFramebufferTexture1DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture1DEXT"));
02040   glNamedFramebufferTexture2DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture2DEXT"));
02041   glNamedFramebufferTexture3DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture3DEXT"));
02042   glNamedFramebufferTextureEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureEXT"));
02043   glNamedFramebufferTextureFaceEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureFaceEXT"));
02044   glNamedFramebufferTextureLayer =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayer"));
02045   glNamedFramebufferTextureLayerEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayerEXT"));
02046   glNamedProgramLocalParameter4dEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dEXT"));
02047   glNamedProgramLocalParameter4dvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dvEXT"));
02048   glNamedProgramLocalParameter4fEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fEXT"));
02049   glNamedProgramLocalParameter4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fvEXT"));
02050   glNamedProgramLocalParameterI4iEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4iEXT"));
02051   glNamedProgramLocalParameterI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4ivEXT"));
02052   glNamedProgramLocalParameterI4uiEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uiEXT"));
02053   glNamedProgramLocalParameterI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uivEXT"));
02054   glNamedProgramLocalParameters4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameters4fvEXT"));
02055   glNamedProgramLocalParametersI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4ivEXT"));
02056   glNamedProgramLocalParametersI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4uivEXT"));
02057   glNamedProgramStringEXT =
PFNGLNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glNamedProgramStringEXT"));
02058   glNamedRenderbufferStorage =
PFNGLNAMEDRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glNamedRenderbufferStorage"));
02059   glNamedRenderbufferStorageEXT =
PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageEXT"));
02060   glNamedRenderbufferStorageMultisample =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisample"));
02061   glNamedRenderbufferStorageMultisampleCoverageEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleCoverageEXT"));
02062   glNamedRenderbufferStorageMultisampleEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleEXT"));
02063   glNamedStringARB = PFNGLNAMEDSTRINGARBPROC (glfwGetProcAddress ("glNamedStringARB"));
02064   glNormalFormatNV = PFNGLNORMALFORMATNVPROC (glfwGetProcAddress ("glNormalFormatNV"));
02065   glObjectLabel = PFNGLOBJECTLABELPROC (glfwGetProcAddress ("glObjectLabel"));
02066   glObjectPtrLabel = PFNGLOBJECTPTRLABELPROC (glfwGetProcAddress ("glObjectPtrLabel"));
02067   glPatchParameterfv = PFNGLPATCHPARAMETERFVPROC (glfwGetProcAddress ("glPatchParameterfv"));
02068   glPatchParameteri = PFNGLPATCHPARAMETERIPROC (glfwGetProcAddress ("glPatchParameteri"));
02069   glPathCommandsNV = PFNGLPATHCOMMANDSNVPROC (glfwGetProcAddress ("glPathCommandsNV"));
02070   glPathCoordsNV = PFNGLPATHCOORDSNVPROC (glfwGetProcAddress ("glPathCoordsNV"));
02071   glPathCoverDepthFuncNV =
PFNGLPATHCOVERDEPTHFUNCNVPROC (glfwGetProcAddress ("glPathCoverDepthFuncNV"));
02072   glPathDashArrayNV = PFNGLPATHDASHARRAYNVPROC (glfwGetProcAddress ("glPathDashArrayNV"));
02073   glPathGlyphIndexArrayNV =
PFNGLPATHGLYPHINDEXARRAYNVPROC (glfwGetProcAddress ("glPathGlyphIndexArrayNV"));

```

```

02074     glPathGlyphIndexRangeNV =
        PFNGLPATHGLYPHINDEXRANGENVPROC (glfwGetProcAddress("glPathGlyphIndexRangeNV"));
02075     glPathGlyphRangeNV = PFNGLPATHGLYPHRANGENVPROC (glfwGetProcAddress("glPathGlyphRangeNV"));
02076     glPathGlyphsNV = PFNGLPATHGLYPHSNVPROC (glfwGetProcAddress("glPathGlyphsNV"));
02077     glPathMemoryGlyphIndexArrayNV =
        PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC (glfwGetProcAddress("glPathMemoryGlyphIndexArrayNV"));
02078     glPathParameterfNV = PFNGLPATHPARAMETERFNVPROC (glfwGetProcAddress("glPathParameterfNV"));
02079     glPathParameterfvNV = PFNGLPATHPARAMETERFVNVPROC (glfwGetProcAddress("glPathParameterfvNV"));
02080     glPathSubCommandsNV = PFNGLPATHSUBCOMMANDSNVPROC (glfwGetProcAddress("glPathSubCommandsNV"));
02081     glPathParameterivNV = PFNGLPATHPARAMETERIVNVPROC (glfwGetProcAddress("glPathParameterivNV"));
02082     glPathStencilDepthOffsetNV =
        PFNGLPATHSTENCILDEPTHOFFSETNVPROC (glfwGetProcAddress("glPathStencilDepthOffsetNV"));
02083     glPathStencilFuncNV = PFNGLPATHSTENCILFUNCNVPROC (glfwGetProcAddress("glPathStencilFuncNV"));
02084     glPathStringNV = PFNGLPATHSTRINGNVPROC (glfwGetProcAddress("glPathStringNV"));
02085     glPathSubCommandsNV = PFNGLPATHSUBCOMMANDSNVPROC (glfwGetProcAddress("glPathSubCommandsNV"));
02086     glPathSubCoordsNV = PFNGLPATHSUBCOORDSNVPROC (glfwGetProcAddress("glPathSubCoordsNV"));
02087     glPauseTransformFeedback =
        PFNGLPAUSETRANSFORMFEEDBACKPROC (glfwGetProcAddress("glPauseTransformFeedback"));
02088     glPixelStoref = PFNGLPIXELSTOREFPROC (glfwGetProcAddress("glPixelStoref"));
02089     glPixelStorei = PFNGLPIXELSTOREIPROC (glfwGetProcAddress("glPixelStorei"));
02090     glPointAlongPathNV = PFNGLPOINTALONGPATHNVPROC (glfwGetProcAddress("glPointAlongPathNV"));
02091     glPointParameterf = PFNGLPOINTPARAMETERFPROC (glfwGetProcAddress("glPointParameterf"));
02092     glPointParameterfv = PFNGLPOINTPARAMETERFVPROC (glfwGetProcAddress("glPointParameterfv"));
02093     glPointParameteri = PFNGLPOINTPARAMETERIPROC (glfwGetProcAddress("glPointParameteri"));
02094     glPointParameteriv = PFNGLPOINTPARAMETERIVPROC (glfwGetProcAddress("glPointParameteriv"));
02095     glPointSize = PFNGLPOINTSIZEPROC (glfwGetProcAddress("glPointSize"));
02096     glPolygonMode = PFNGLPOLYGONMODEPROC (glfwGetProcAddress("glPolygonMode"));
02097     glPolygonOffset = PFNGLPOLYGONOFFSETPROC (glfwGetProcAddress("glPolygonOffset"));
02098     glPolygonOffsetClampEXT =
        PFNGLPOLYGONOFFSETCLAMPEXTPROC (glfwGetProcAddress("glPolygonOffsetClampEXT"));
02099     glPopDebugGroup = PFNGLPOPDEBUGGROUPPROC (glfwGetProcAddress("glPopDebugGroup"));
02100     glPopGroupMarkerEXT = PFNGLPOPGROUPMARKEREXTPROC (glfwGetProcAddress("glPopGroupMarkerEXT"));
02101     glPrimitiveBoundingBoxARB =
        PFNGLPRIMITIVEBOUNDINGBOXARBPROC (glfwGetProcAddress("glPrimitiveBoundingBoxARB"));
02102     glPrimitiveRestartIndex =
        PFNGLPRIMITIVERESTARTINDEXPROC (glfwGetProcAddress("glPrimitiveRestartIndex"));
02103     glProgramBinary = PFNGLPROGRAMBINARYPROC (glfwGetProcAddress("glProgramBinary"));
02104     glProgramParameteri = PFNGLPROGRAMPARAMETERIPROC (glfwGetProcAddress("glProgramParameteri"));
02105     glProgramParameteriARB =
        PFNGLPROGRAMPARAMETERIARBPROC (glfwGetProcAddress("glProgramParameteriARB"));
02106     glProgramPathFragmentInputGenNV =
        PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC (glfwGetProcAddress("glProgramPathFragmentInputGenNV"));
02107     glProgramUniformId = PFNGLPROGRAMUNIFORMIDPROC (glfwGetProcAddress("glProgramUniformId"));
02108     glProgramUniformIdEXT = PFNGLPROGRAMUNIFORMIDEXTPROC (glfwGetProcAddress("glProgramUniformIdEXT"));
02109     glProgramUniformIdv = PFNGLPROGRAMUNIFORMIDVPROC (glfwGetProcAddress("glProgramUniformIdv"));
02110     glProgramUniformIdvEXT =
        PFNGLPROGRAMUNIFORMIDVEXTPROC (glfwGetProcAddress("glProgramUniformIdvEXT"));
02111     glProgramUniform1f = PFNGLPROGRAMUNIFORM1FPROC (glfwGetProcAddress("glProgramUniform1f"));
02112     glProgramUniform1fEXT = PFNGLPROGRAMUNIFORM1FEXTPROC (glfwGetProcAddress("glProgramUniform1fEXT"));
02113     glProgramUniform1fv = PFNGLPROGRAMUNIFORM1FVPROC (glfwGetProcAddress("glProgramUniform1fv"));
02114     glProgramUniform1fvEXT =
        PFNGLPROGRAMUNIFORM1FVEXTPROC (glfwGetProcAddress("glProgramUniform1fvEXT"));
02115     glProgramUniform1i = PFNGLPROGRAMUNIFORM1IPROC (glfwGetProcAddress("glProgramUniform1i"));
02116     glProgramUniform1i64ARB =
        PFNGLPROGRAMUNIFORM1I64ARBPROC (glfwGetProcAddress("glProgramUniform1i64ARB"));
02117     glProgramUniform1i64NV =
        PFNGLPROGRAMUNIFORM1I64NVPROC (glfwGetProcAddress("glProgramUniform1i64NV"));
02118     glProgramUniform1i64vARB =
        PFNGLPROGRAMUNIFORM1I64VARBPROC (glfwGetProcAddress("glProgramUniform1i64vARB"));
02119     glProgramUniform1i64vNV =
        PFNGLPROGRAMUNIFORM1I64VNVPROC (glfwGetProcAddress("glProgramUniform1i64vNV"));
02120     glProgramUniform1iEXT = PFNGLPROGRAMUNIFORM1IEXTPROC (glfwGetProcAddress("glProgramUniform1iEXT"));
02121     glProgramUniform1iv = PFNGLPROGRAMUNIFORM1IVPROC (glfwGetProcAddress("glProgramUniform1iv"));
02122     glProgramUniform1ivEXT =
        PFNGLPROGRAMUNIFORM1IVEXTPROC (glfwGetProcAddress("glProgramUniform1ivEXT"));
02123     glProgramUniform1ui = PFNGLPROGRAMUNIFORM1UIPROC (glfwGetProcAddress("glProgramUniform1ui"));
02124     glProgramUniform1ui64ARB =
        PFNGLPROGRAMUNIFORM1UI64ARBPROC (glfwGetProcAddress("glProgramUniform1ui64ARB"));
02125     glProgramUniform1ui64NV =
        PFNGLPROGRAMUNIFORM1UI64NVPROC (glfwGetProcAddress("glProgramUniform1ui64NV"));
02126     glProgramUniform1ui64vARB =
        PFNGLPROGRAMUNIFORM1UI64VARBPROC (glfwGetProcAddress("glProgramUniform1ui64vARB"));
02127     glProgramUniform1ui64vNV =
        PFNGLPROGRAMUNIFORM1UI64VNVPROC (glfwGetProcAddress("glProgramUniform1ui64vNV"));
02128     glProgramUniform1uiEXT =
        PFNGLPROGRAMUNIFORM1UIEXTPROC (glfwGetProcAddress("glProgramUniform1uiEXT"));
02129     glProgramUniform1uiv = PFNGLPROGRAMUNIFORM1UIVPROC (glfwGetProcAddress("glProgramUniform1uiv"));
02130     glProgramUniform1uivEXT =
        PFNGLPROGRAMUNIFORM1UIVEXTPROC (glfwGetProcAddress("glProgramUniform1uivEXT"));
02131     glProgramUniform2d = PFNGLPROGRAMUNIFORM2DPROC (glfwGetProcAddress("glProgramUniform2d"));
02132     glProgramUniform2dEXT = PFNGLPROGRAMUNIFORM2DEXTPROC (glfwGetProcAddress("glProgramUniform2dEXT"));
02133     glProgramUniform2dv = PFNGLPROGRAMUNIFORM2DVPROC (glfwGetProcAddress("glProgramUniform2dv"));
02134     glProgramUniform2dvEXT =
        PFNGLPROGRAMUNIFORM2DVEXTPROC (glfwGetProcAddress("glProgramUniform2dvEXT"));
02135     glProgramUniform2f = PFNGLPROGRAMUNIFORM2FPROC (glfwGetProcAddress("glProgramUniform2f"));
02136     glProgramUniform2fEXT = PFNGLPROGRAMUNIFORM2FEXTPROC (glfwGetProcAddress("glProgramUniform2fEXT"));
02137     glProgramUniform2fv = PFNGLPROGRAMUNIFORM2FVPROC (glfwGetProcAddress("glProgramUniform2fv"));

```

```

02138     glProgramUniform2fvEXT =
PFNGLPROGRAMUNIFORM2FVEXTPROC (glfwGetProcAddress ("glProgramUniform2fvEXT"));
02139     glProgramUniform2i = PFNGLPROGRAMUNIFORM2IPROC (glfwGetProcAddress ("glProgramUniform2i"));
02140     glProgramUniform2i64ARB =
PFNGLPROGRAMUNIFORM2I64ARBPROC (glfwGetProcAddress ("glProgramUniform2i64ARB"));
02141     glProgramUniform2i64NV =
PFNGLPROGRAMUNIFORM2I64NVPROC (glfwGetProcAddress ("glProgramUniform2i64NV"));
02142     glProgramUniform2i64vARB =
PFNGLPROGRAMUNIFORM2I64VARBPROC (glfwGetProcAddress ("glProgramUniform2i64vARB"));
02143     glProgramUniform2i64vNV =
PFNGLPROGRAMUNIFORM2I64VNVPROC (glfwGetProcAddress ("glProgramUniform2i64vNV"));
02144     glProgramUniform2iEXT = PFNGLPROGRAMUNIFORM2IEXTPROC (glfwGetProcAddress ("glProgramUniform2iEXT"));
02145     glProgramUniform2iv = PFNGLPROGRAMUNIFORM2IVPROC (glfwGetProcAddress ("glProgramUniform2iv"));
02146     glProgramUniform2ivEXT =
PFNGLPROGRAMUNIFORM2IVEXTPROC (glfwGetProcAddress ("glProgramUniform2ivEXT"));
02147     glProgramUniform2ui = PFNGLPROGRAMUNIFORM2UIPROC (glfwGetProcAddress ("glProgramUniform2ui"));
02148     glProgramUniform2ui64ARB =
PFNGLPROGRAMUNIFORM2UI64ARBPROC (glfwGetProcAddress ("glProgramUniform2ui64ARB"));
02149     glProgramUniform2ui64NV =
PFNGLPROGRAMUNIFORM2UI64NVPROC (glfwGetProcAddress ("glProgramUniform2ui64NV"));
02150     glProgramUniform2ui64vARB =
PFNGLPROGRAMUNIFORM2UI64VARBPROC (glfwGetProcAddress ("glProgramUniform2ui64vARB"));
02151     glProgramUniform2ui64vNV =
PFNGLPROGRAMUNIFORM2UI64VNVPROC (glfwGetProcAddress ("glProgramUniform2ui64vNV"));
02152     glProgramUniform2uiEXT =
PFNGLPROGRAMUNIFORM2UIEXTPROC (glfwGetProcAddress ("glProgramUniform2uiEXT"));
02153     glProgramUniform2uiv = PFNGLPROGRAMUNIFORM2UIVPROC (glfwGetProcAddress ("glProgramUniform2uiv"));
02154     glProgramUniform2uivEXT =
PFNGLPROGRAMUNIFORM2UIVEXTPROC (glfwGetProcAddress ("glProgramUniform2uivEXT"));
02155     glProgramUniform3d = PFNGLPROGRAMUNIFORM3DPROC (glfwGetProcAddress ("glProgramUniform3d"));
02156     glProgramUniform3dEXT = PFNGLPROGRAMUNIFORM3DEXTPROC (glfwGetProcAddress ("glProgramUniform3dEXT"));
02157     glProgramUniform3dv = PFNGLPROGRAMUNIFORM3DVPROC (glfwGetProcAddress ("glProgramUniform3dv"));
02158     glProgramUniform3dvEXT =
PFNGLPROGRAMUNIFORM3DVEXTPROC (glfwGetProcAddress ("glProgramUniform3dvEXT"));
02159     glProgramUniform3f = PFNGLPROGRAMUNIFORM3FPROC (glfwGetProcAddress ("glProgramUniform3f"));
02160     glProgramUniform3fEXT = PFNGLPROGRAMUNIFORM3FEXTPROC (glfwGetProcAddress ("glProgramUniform3fEXT"));
02161     glProgramUniform3fv = PFNGLPROGRAMUNIFORM3FVPROC (glfwGetProcAddress ("glProgramUniform3fv"));
02162     glProgramUniform3fvEXT =
PFNGLPROGRAMUNIFORM3FVEXTPROC (glfwGetProcAddress ("glProgramUniform3fvEXT"));
02163     glProgramUniform3i = PFNGLPROGRAMUNIFORM3IPROC (glfwGetProcAddress ("glProgramUniform3i"));
02164     glProgramUniform3i64ARB =
PFNGLPROGRAMUNIFORM3I64ARBPROC (glfwGetProcAddress ("glProgramUniform3i64ARB"));
02165     glProgramUniform3i64NV =
PFNGLPROGRAMUNIFORM3I64NVPROC (glfwGetProcAddress ("glProgramUniform3i64NV"));
02166     glProgramUniform3i64vARB =
PFNGLPROGRAMUNIFORM3I64VARBPROC (glfwGetProcAddress ("glProgramUniform3i64vARB"));
02167     glProgramUniform3i64vNV =
PFNGLPROGRAMUNIFORM3I64VNVPROC (glfwGetProcAddress ("glProgramUniform3i64vNV"));
02168     glProgramUniform3iEXT = PFNGLPROGRAMUNIFORM3IEXTPROC (glfwGetProcAddress ("glProgramUniform3iEXT"));
02169     glProgramUniform3iv = PFNGLPROGRAMUNIFORM3IVPROC (glfwGetProcAddress ("glProgramUniform3iv"));
02170     glProgramUniform3ivEXT =
PFNGLPROGRAMUNIFORM3IVEXTPROC (glfwGetProcAddress ("glProgramUniform3ivEXT"));
02171     glProgramUniform3ui = PFNGLPROGRAMUNIFORM3UIPROC (glfwGetProcAddress ("glProgramUniform3ui"));
02172     glProgramUniform3ui64ARB =
PFNGLPROGRAMUNIFORM3UI64ARBPROC (glfwGetProcAddress ("glProgramUniform3ui64ARB"));
02173     glProgramUniform3ui64NV =
PFNGLPROGRAMUNIFORM3UI64NVPROC (glfwGetProcAddress ("glProgramUniform3ui64NV"));
02174     glProgramUniform3ui64vARB =
PFNGLPROGRAMUNIFORM3UI64VARBPROC (glfwGetProcAddress ("glProgramUniform3ui64vARB"));
02175     glProgramUniform3ui64vNV =
PFNGLPROGRAMUNIFORM3UI64VNVPROC (glfwGetProcAddress ("glProgramUniform3ui64vNV"));
02176     glProgramUniform3uiEXT =
PFNGLPROGRAMUNIFORM3UIEXTPROC (glfwGetProcAddress ("glProgramUniform3uiEXT"));
02177     glProgramUniform3uiv = PFNGLPROGRAMUNIFORM3UIVPROC (glfwGetProcAddress ("glProgramUniform3uiv"));
02178     glProgramUniform3uivEXT =
PFNGLPROGRAMUNIFORM3UIVEXTPROC (glfwGetProcAddress ("glProgramUniform3uivEXT"));
02179     glProgramUniform4d = PFNGLPROGRAMUNIFORM4DPROC (glfwGetProcAddress ("glProgramUniform4d"));
02180     glProgramUniform4dEXT = PFNGLPROGRAMUNIFORM4DEXTPROC (glfwGetProcAddress ("glProgramUniform4dEXT"));
02181     glProgramUniform4dv = PFNGLPROGRAMUNIFORM4DVPROC (glfwGetProcAddress ("glProgramUniform4dv"));
02182     glProgramUniform4dvEXT =
PFNGLPROGRAMUNIFORM4DVEXTPROC (glfwGetProcAddress ("glProgramUniform4dvEXT"));
02183     glProgramUniform4f = PFNGLPROGRAMUNIFORM4FPROC (glfwGetProcAddress ("glProgramUniform4f"));
02184     glProgramUniform4fEXT = PFNGLPROGRAMUNIFORM4FEXTPROC (glfwGetProcAddress ("glProgramUniform4fEXT"));
02185     glProgramUniform4fv = PFNGLPROGRAMUNIFORM4FVPROC (glfwGetProcAddress ("glProgramUniform4fv"));
02186     glProgramUniform4fvEXT =
PFNGLPROGRAMUNIFORM4FVEXTPROC (glfwGetProcAddress ("glProgramUniform4fvEXT"));
02187     glProgramUniform4i = PFNGLPROGRAMUNIFORM4IPROC (glfwGetProcAddress ("glProgramUniform4i"));
02188     glProgramUniform4i64ARB =
PFNGLPROGRAMUNIFORM4I64ARBPROC (glfwGetProcAddress ("glProgramUniform4i64ARB"));
02189     glProgramUniform4i64NV =
PFNGLPROGRAMUNIFORM4I64NVPROC (glfwGetProcAddress ("glProgramUniform4i64NV"));
02190     glProgramUniform4i64vARB =
PFNGLPROGRAMUNIFORM4I64VARBPROC (glfwGetProcAddress ("glProgramUniform4i64vARB"));
02191     glProgramUniform4i64vNV =
PFNGLPROGRAMUNIFORM4I64VNVPROC (glfwGetProcAddress ("glProgramUniform4i64vNV"));
02192     glProgramUniform4iEXT = PFNGLPROGRAMUNIFORM4IEXTPROC (glfwGetProcAddress ("glProgramUniform4iEXT"));
02193     glProgramUniform4iv = PFNGLPROGRAMUNIFORM4IVPROC (glfwGetProcAddress ("glProgramUniform4iv"));

```



```

02194     glProgramUniform4ivEXT =
PFNGLPROGRAMUNIFORM4IIVEXTPROC (glfwGetProcAddress ("glProgramUniform4ivEXT"));
02195     glProgramUniform4ui = PFNGLPROGRAMUNIFORM4UIPROC (glfwGetProcAddress ("glProgramUniform4ui"));
02196     glProgramUniform4ui64ARB =
PFNGLPROGRAMUNIFORM4UI64ARBPROC (glfwGetProcAddress ("glProgramUniform4ui64ARB"));
02197     glProgramUniform4ui64NV =
PFNGLPROGRAMUNIFORM4UI64NVPROC (glfwGetProcAddress ("glProgramUniform4ui64NV"));
02198     glProgramUniform4ui64vARB =
PFNGLPROGRAMUNIFORM4UI64VARBPROC (glfwGetProcAddress ("glProgramUniform4ui64vARB"));
02199     glProgramUniform4ui64vNV =
PFNGLPROGRAMUNIFORM4UI64VNVPROC (glfwGetProcAddress ("glProgramUniform4ui64vNV"));
02200     glProgramUniform4uiEXT =
PFNGLPROGRAMUNIFORM4UIEXTPROC (glfwGetProcAddress ("glProgramUniform4uiEXT"));
02201     glProgramUniform4uiv = PFNGLPROGRAMUNIFORM4UIVPROC (glfwGetProcAddress ("glProgramUniform4uiv"));
02202     glProgramUniform4uivEXT =
PFNGLPROGRAMUNIFORM4UIVEXTPROC (glfwGetProcAddress ("glProgramUniform4uivEXT"));
02203     glProgramUniformHandleui64ARB =
PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64ARB"));
02204     glProgramUniformHandleui64NV =
PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress ("glProgramUniformHandleui64NV"));
02205     glProgramUniformHandleui64vARB =
PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64vARB"));
02206     glProgramUniformHandleui64vNV =
PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress ("glProgramUniformHandleui64vNV"));
02207     glProgramUniformMatrix2dv =
PFNGLPROGRAMUNIFORMMATRIX2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2dv"));
02208     glProgramUniformMatrix2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2dvEXT"));
02209     glProgramUniformMatrix2fv =
PFNGLPROGRAMUNIFORMMATRIX2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2fv"));
02210     glProgramUniformMatrix2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2fvEXT"));
02211     glProgramUniformMatrix2x3dv =
PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dv"));
02212     glProgramUniformMatrix2x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dvEXT"));
02213     glProgramUniformMatrix2x3fv =
PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fv"));
02214     glProgramUniformMatrix2x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fvEXT"));
02215     glProgramUniformMatrix2x4dv =
PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dv"));
02216     glProgramUniformMatrix2x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dvEXT"));
02217     glProgramUniformMatrix2x4fv =
PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fv"));
02218     glProgramUniformMatrix2x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fvEXT"));
02219     glProgramUniformMatrix3dv =
PFNGLPROGRAMUNIFORMMATRIX3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3dv"));
02220     glProgramUniformMatrix3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3dvEXT"));
02221     glProgramUniformMatrix3fv =
PFNGLPROGRAMUNIFORMMATRIX3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3fv"));
02222     glProgramUniformMatrix3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3fvEXT"));
02223     glProgramUniformMatrix3x2dv =
PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dv"));
02224     glProgramUniformMatrix3x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dvEXT"));
02225     glProgramUniformMatrix3x2fv =
PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fv"));
02226     glProgramUniformMatrix3x2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fvEXT"));
02227     glProgramUniformMatrix3x4dv =
PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dv"));
02228     glProgramUniformMatrix3x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dvEXT"));
02229     glProgramUniformMatrix3x4fv =
PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fv"));
02230     glProgramUniformMatrix3x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fvEXT"));
02231     glProgramUniformMatrix4dv =
PFNGLPROGRAMUNIFORMMATRIX4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4dv"));
02232     glProgramUniformMatrix4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4dvEXT"));
02233     glProgramUniformMatrix4fv =
PFNGLPROGRAMUNIFORMMATRIX4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4fv"));
02234     glProgramUniformMatrix4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4fvEXT"));
02235     glProgramUniformMatrix4x2dv =
PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dv"));
02236     glProgramUniformMatrix4x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dvEXT"));
02237     glProgramUniformMatrix4x2fv =
PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fv"));
02238     glProgramUniformMatrix4x2fvEXT =

```

```

PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fvEXT"));
02239 glProgramUniformMatrix4x3dv =
PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dv"));
02240 glProgramUniformMatrix4x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dvEXT"));
02241 glProgramUniformMatrix4x3fv =
PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fv"));
02242 glProgramUniformMatrix4x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fvEXT"));
02243 glProgramUniformi64NV =
PFNGLPROGRAMUNIFORMUI64NVPROC (glfwGetProcAddress ("glProgramUniformui64NV"));
02244 glProgramUniformui64vNV =
PFNGLPROGRAMUNIFORMUI64VNVPROC (glfwGetProcAddress ("glProgramUniformui64vNV"));
02245 glProvokingVertex = PFNGLPROVOKINGVERTEXPROC (glfwGetProcAddress ("glProvokingVertex"));
02246 glPushClientAttribDefaultEXT =
PFNGLPUSHCLIENTATTRIBDEFAULTTEXTPROC (glfwGetProcAddress ("glPushClientAttribDefaultEXT"));
02247 glPushDebugGroup = PFNGLPUSHDEBUGGROUPPROC (glfwGetProcAddress ("glPushDebugGroup"));
02248 glPushGroupMarkerEXT = PFNGLPUSHGROUPMARKEREXTPROC (glfwGetProcAddress ("glPushGroupMarkerEXT"));
02249 glQueryCounter = PFNGLQUERYCOUNTERPROC (glfwGetProcAddress ("glQueryCounter"));
02250 glRasterSamplesEXT = PFNGLRASTERSAMPLESEXTPROC (glfwGetProcAddress ("glRasterSamplesEXT"));
02251 glReadBuffer = PFNGLREADBUFFERPROC (glfwGetProcAddress ("glReadBuffer"));
02252 glReadPixels = PFNGLREADPIXELSPROC (glfwGetProcAddress ("glReadPixels"));
02253 glReadnPixels = PFNGLREADNPIXELSPROC (glfwGetProcAddress ("glReadnPixels"));
02254 glReadnPixelsARB = PFNGLREADNPIXELSARBPROC (glfwGetProcAddress ("glReadnPixelsARB"));
02255 glReleaseShaderCompiler =
PFNGLRELEASESHADERCOMPILERPROC (glfwGetProcAddress ("glReleaseShaderCompiler"));
02256 glRenderbufferStorage = PFNGLRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glRenderbufferStorage"));
02257 glRenderbufferStorageMultisample =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC (glfwGetProcAddress ("glRenderbufferStorageMultisample"));
02258 glRenderbufferStorageMultisampleCoverageNV =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEVPROC (glfwGetProcAddress ("glRenderbufferStorageMultisampleCoverageNV"));
02259 glResolveDepthValuesNV =
PFNGLRESOLVEDEPTHVALUESNVPROC (glfwGetProcAddress ("glResolveDepthValuesNV"));
02260 glResumeTransformFeedback =
PFNGLRESUMETRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glResumeTransformFeedback"));
02261 glSampleCoverage = PFNGLSAMPLECOVERAGEPROC (glfwGetProcAddress ("glSampleCoverage"));
02262 glSampleMaski = PFNGLSAMPLEMASKIPROC (glfwGetProcAddress ("glSampleMaski"));
02263 glSamplerParameterIiv = PFNGLSAMPLERPARAMETERIIVPROC (glfwGetProcAddress ("glSamplerParameterIiv"));
02264 glSamplerParameterIuiv =
PFNGLSAMPLERPARAMETERIUIVPROC (glfwGetProcAddress ("glSamplerParameterIuiv"));
02265 glSamplerParameterf = PFNGLSAMPLERPARAMETERFPROC (glfwGetProcAddress ("glSamplerParameterf"));
02266 glSamplerParameterfv = PFNGLSAMPLERPARAMETERFVPROC (glfwGetProcAddress ("glSamplerParameterfv"));
02267 glSamplerParameteri = PFNGLSAMPLERPARAMETERIPROC (glfwGetProcAddress ("glSamplerParameteri"));
02268 glSamplerParameteriv = PFNGLSAMPLERPARAMETERIVPROC (glfwGetProcAddress ("glSamplerParameteriv"));
02269 glScissor = PFNGLSCISSORPROC (glfwGetProcAddress ("glScissor"));
02270 glScissorArrayv = PFNGLSCISSORARRAYVPROC (glfwGetProcAddress ("glScissorArrayv"));
02271 glScissorIndexed = PFNGLSCISSORINDEXEDPROC (glfwGetProcAddress ("glScissorIndexed"));
02272 glScissorIndexedv = PFNGLSCISSORINDEXEDVPROC (glfwGetProcAddress ("glScissorIndexedv"));
02273 glSecondaryColorFormatNV =
PFNGLSECONDARYCOLORFORMATNVPROC (glfwGetProcAddress ("glSecondaryColorFormatNV"));
02274 glSelectPerfMonitorCountersAMD =
PFNGLSELECTPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress ("glSelectPerfMonitorCountersAMD"));
02275 glShaderBinary = PFNGLSHADERBINARYPROC (glfwGetProcAddress ("glShaderBinary"));
02276 glShaderSource = PFNGLSHADERSOURCEPROC (glfwGetProcAddress ("glShaderSource"));
02277 glShaderStorageBlockBinding =
PFNGLSHADERSTORAGEBLOCKBINDINGPROC (glfwGetProcAddress ("glShaderStorageBlockBinding"));
02278 glSignalVkFenceNV = PFNGLSIGNALVKFENCEVPROC (glfwGetProcAddress ("glSignalVkFenceNV"));
02279 glSignalVkSemaphoreNV = PFNGLSIGNALVKSEMAPHORENVPROC (glfwGetProcAddress ("glSignalVkSemaphoreNV"));
02280 glSpecializeShaderARB = PFNGLSPECIALIZESHADERARBPROC (glfwGetProcAddress ("glSpecializeShaderARB"));
02281 glStateCaptureNV = PFNGLSTATECAPTURENVPROC (glfwGetProcAddress ("glStateCaptureNV"));
02282 glStencilFillPathInstancedNV =
PFNGLSTENCILFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilFillPathInstancedNV"));
02283 glStencilFillPathNV = PFNGLSTENCILFILLPATHNVPROC (glfwGetProcAddress ("glStencilFillPathNV"));
02284 glStencilFunc = PFNGLSTENCILFUNCPROC (glfwGetProcAddress ("glStencilFunc"));
02285 glStencilFuncSeparate = PFNGLSTENCILFUNCSEPARATEPROC (glfwGetProcAddress ("glStencilFuncSeparate"));
02286 glStencilMask = PFNGLSTENCILMASKPROC (glfwGetProcAddress ("glStencilMask"));
02287 glStencilMaskSeparate = PFNGLSTENCILMASKSEPARATEPROC (glfwGetProcAddress ("glStencilMaskSeparate"));
02288 glStencilOp = PFNGLSTENCILOPPROC (glfwGetProcAddress ("glStencilOp"));
02289 glStencilOpSeparate = PFNGLSTENCILOPSEPARATEPROC (glfwGetProcAddress ("glStencilOpSeparate"));
02290 glStencilStrokePathInstancedNV =
PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilStrokePathInstancedNV"));
02291 glStencilStrokePathNV = PFNGLSTENCILSTROKEPATHNVPROC (glfwGetProcAddress ("glStencilStrokePathNV"));
02292 glStencilThenCoverFillPathInstancedNV =
PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilThenCoverFillPathInstancedNV"));
02293 glStencilThenCoverFillPathNV =
PFNGLSTENCILTHENCOVERFILLPATHNVPROC (glfwGetProcAddress ("glStencilThenCoverFillPathNV"));
02294 glStencilThenCoverStrokePathInstancedNV =
PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilThenCoverStrokePathInstancedNV"));
02295 glStencilThenCoverStrokePathNV =
PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC (glfwGetProcAddress ("glStencilThenCoverStrokePathNV"));
02296 glSubpixelPrecisionBiasNV =
PFNGLSUBPIXELPRECISIONBIASNVPROC (glfwGetProcAddress ("glSubpixelPrecisionBiasNV"));
02297 glTexBuffer = PFNGLTEXBUFFERPROC (glfwGetProcAddress ("glTexBuffer"));
02298 glTexBufferARB = PFNGLTEXBUFFERARBPROC (glfwGetProcAddress ("glTexBufferARB"));
02299 glTexBufferRange = PFNGLTEXBUFFERRANGEPROC (glfwGetProcAddress ("glTexBufferRange"));
02300 glTexCoordFormatNV = PFNGLTEXCOORDFORMATNVPROC (glfwGetProcAddress ("glTexCoordFormatNV"));
02301 glTexImage1D = PFNGLTEXIMAGE1DPROC (glfwGetProcAddress ("glTexImage1D"));

```

```
02302     glTexImage2D = PFNGLTEXIMAGE2DPROC (glfwGetProcAddress("glTexImage2D"));
02303     glTexImage2DMultisample =
PFNGLTEXIMAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage2DMultisample"));
02304     glTexImage3D = PFNGLTEXIMAGE3DPROC (glfwGetProcAddress("glTexImage3D"));
02305     glTexImage3DMultisample =
PFNGLTEXIMAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexImage3DMultisample"));
02306     glTexPageCommitmentARB =
PFNGLTEXPAGECOMMITMENTARBPROC (glfwGetProcAddress("glTexPageCommitmentARB"));
02307     glTexParameterIiv = PFNGLTEXPARAMETERIIVPROC (glfwGetProcAddress("glTexParameterIiv"));
02308     glTexParameterIuiv = PFNGLTEXPARAMETERIUIVPROC (glfwGetProcAddress("glTexParameterIuiv"));
02309     glTexParameterf = PFNGLTEXPARAMETERFPROC (glfwGetProcAddress("glTexParameterf"));
02310     glTexParameterfv = PFNGLTEXPARAMETERFVPROC (glfwGetProcAddress("glTexParameterfv"));
02311     glTexParameteri = PFNGLTEXPARAMETERIPROC (glfwGetProcAddress("glTexParameteri"));
02312     glTexParameteriv = PFNGLTEXPARAMETERIVPROC (glfwGetProcAddress("glTexParameteriv"));
02313     glTexStorage1D = PFNGLTEXSTORAGE1DPROC (glfwGetProcAddress("glTexStorage1D"));
02314     glTexStorage2D = PFNGLTEXSTORAGE2DPROC (glfwGetProcAddress("glTexStorage2D"));
02315     glTexStorage2DMultisample =
PFNGLTEXSTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage2DMultisample"));
02316     glTexStorage3D = PFNGLTEXSTORAGE3DPROC (glfwGetProcAddress("glTexStorage3D"));
02317     glTexStorage3DMultisample =
PFNGLTEXSTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage3DMultisample"));
02318     glTexSubImage1D = PFNGLTEXSUBIMAGE1DPROC (glfwGetProcAddress("glTexSubImage1D"));
02319     glTexSubImage2D = PFNGLTEXSUBIMAGE2DPROC (glfwGetProcAddress("glTexSubImage2D"));
02320     glTexSubImage3D = PFNGLTEXSUBIMAGE3DPROC (glfwGetProcAddress("glTexSubImage3D"));
02321     glTextureBarrier = PFNGLTEXTUREBARRIERPROC (glfwGetProcAddress("glTextureBarrier"));
02322     glTextureBarrierNV = PFNGLTEXTUREBARRIERNVPROC (glfwGetProcAddress("glTextureBarrierNV"));
02323     glTextureBuffer = PFNGLTEXTUREBUFFERPROC (glfwGetProcAddress("glTextureBuffer"));
02324     glTextureBufferEXT = PFNGLTEXTUREBUFFEREXTPROC (glfwGetProcAddress("glTextureBufferEXT"));
02325     glTextureBufferRange = PFNGLTEXTUREBUFFERRANGEPROC (glfwGetProcAddress("glTextureBufferRange"));
02326     glTextureBufferRangeEXT =
PFNGLTEXTUREBUFFERRANGEEXTPROC (glfwGetProcAddress("glTextureBufferRangeEXT"));
02327     glTextureImage1DEXT = PFNGLTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress("glTextureImage1DEXT"));
02328     glTextureImage2DEXT = PFNGLTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress("glTextureImage2DEXT"));
02329     glTextureImage3DEXT = PFNGLTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress("glTextureImage3DEXT"));
02330     glTexturePageCommitmentEXT =
PFNGLTEXTUREPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glTexturePageCommitmentEXT"));
02331     glTextureParameterIiv = PFNGLTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glTextureParameterIiv"));
02332     glTextureParameterIivEXT =
PFNGLTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress("glTextureParameterIivEXT"));
02333     glTextureParameterIuiv =
PFNGLTEXTUREPARAMETERIUIVPROC (glfwGetProcAddress("glTextureParameterIuiv"));
02334     glTextureParameterIuivEXT =
PFNGLTEXTUREPARAMETERIUIVEXTPROC (glfwGetProcAddress("glTextureParameterIuivEXT"));
02335     glTextureParameterf = PFNGLTEXTUREPARAMETERFPROC (glfwGetProcAddress("glTextureParameterf"));
02336     glTextureParameterfEXT =
PFNGLTEXTUREPARAMETERFEXTPROC (glfwGetProcAddress("glTextureParameterfEXT"));
02337     glTextureParameterfv = PFNGLTEXTUREPARAMETERFVPROC (glfwGetProcAddress("glTextureParameterfv"));
02338     glTextureParameterfvEXT =
PFNGLTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress("glTextureParameterfvEXT"));
02339     glTextureParameteri = PFNGLTEXTUREPARAMETERIPROC (glfwGetProcAddress("glTextureParameteri"));
02340     glTextureParameteriEXT =
PFNGLTEXTUREPARAMETERIEXTPROC (glfwGetProcAddress("glTextureParameteriEXT"));
02341     glTextureParameteriv = PFNGLTEXTUREPARAMETERIVPROC (glfwGetProcAddress("glTextureParameteriv"));
02342     glTextureParameterivEXT =
PFNGLTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress("glTextureParameterivEXT"));
02343     glTextureRenderbufferEXT =
PFNGLTEXTURERENDERBUFFEREXTPROC (glfwGetProcAddress("glTextureRenderbufferEXT"));
02344     glTextureStorage1D = PFNGLTEXTURESTORAGE1DPROC (glfwGetProcAddress("glTextureStorage1D"));
02345     glTextureStorage1DEXT = PFNGLTEXTURESTORAGE1DEXTPROC (glfwGetProcAddress("glTextureStorage1DEXT"));
02346     glTextureStorage2D = PFNGLTEXTURESTORAGE2DPROC (glfwGetProcAddress("glTextureStorage2D"));
02347     glTextureStorage2DEXT = PFNGLTEXTURESTORAGE2DEXTPROC (glfwGetProcAddress("glTextureStorage2DEXT"));
02348     glTextureStorage2DMultisample =
PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage2DMultisample"));
02349     glTextureStorage2DMultisampleEXT =
PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage2DMultisampleEXT"));
02350     glTextureStorage3D = PFNGLTEXTURESTORAGE3DPROC (glfwGetProcAddress("glTextureStorage3D"));
02351     glTextureStorage3DEXT = PFNGLTEXTURESTORAGE3DEXTPROC (glfwGetProcAddress("glTextureStorage3DEXT"));
02352     glTextureStorage3DMultisample =
PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage3DMultisample"));
02353     glTextureStorage3DMultisampleEXT =
PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage3DMultisampleEXT"));
02354     glTextureSubImage1D = PFNGLTEXTURESUBIMAGE1DPROC (glfwGetProcAddress("glTextureSubImage1D"));
02355     glTextureSubImage1DEXT =
PFNGLTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress("glTextureSubImage1DEXT"));
02356     glTextureSubImage2D = PFNGLTEXTURESUBIMAGE2DPROC (glfwGetProcAddress("glTextureSubImage2D"));
02357     glTextureSubImage2DEXT =
PFNGLTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress("glTextureSubImage2DEXT"));
02358     glTextureSubImage3D = PFNGLTEXTURESUBIMAGE3DPROC (glfwGetProcAddress("glTextureSubImage3D"));
02359     glTextureSubImage3DEXT =
PFNGLTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress("glTextureSubImage3DEXT"));
02360     glTextureView = PFNGLTEXTUREVIEWPROC (glfwGetProcAddress("glTextureView"));
02361     glTransformFeedbackBufferBase =
PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC (glfwGetProcAddress("glTransformFeedbackBufferBase"));
02362     glTransformFeedbackBufferRange =
PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC (glfwGetProcAddress("glTransformFeedbackBufferRange"));
02363     glTransformFeedbackVaryings =
PFNGLTRANSFORMFEEDBACKVARYINGSPROC (glfwGetProcAddress("glTransformFeedbackVaryings"));
```



```
02364 glTransformPathNV = PFNGLTRANSFORMPATHNVPROC (glfwGetProcAddress("glTransformPathNV"));
02365 glUniformld = PFNGLUNIFORMLDPROC (glfwGetProcAddress("glUniformld"));
02366 glUniformldv = PFNGLUNIFORMLDVPROC (glfwGetProcAddress("glUniformldv"));
02367 glUniformlf = PFNGLUNIFORMLFPROC (glfwGetProcAddress("glUniformlf"));
02368 glUniformlfv = PFNGLUNIFORMLFVPROC (glfwGetProcAddress("glUniformlfv"));
02369 glUniformli = PFNGLUNIFORMLIPROC (glfwGetProcAddress("glUniformli"));
02370 glUniformli64ARB = PFNGLUNIFORMLI64ARBPROC (glfwGetProcAddress("glUniformli64ARB"));
02371 glUniformli64NV = PFNGLUNIFORMLI64NVPROC (glfwGetProcAddress("glUniformli64NV"));
02372 glUniformli64vARB = PFNGLUNIFORMLI64VARBPROC (glfwGetProcAddress("glUniformli64vARB"));
02373 glUniformli64vNV = PFNGLUNIFORMLI64VNVPROC (glfwGetProcAddress("glUniformli64vNV"));
02374 glUniformliv = PFNGLUNIFORMLIVPROC (glfwGetProcAddress("glUniformliv"));
02375 glUniformlui = PFNGLUNIFORMLUIPROC (glfwGetProcAddress("glUniformlui"));
02376 glUniformlui64ARB = PFNGLUNIFORMLUI64ARBPROC (glfwGetProcAddress("glUniformlui64ARB"));
02377 glUniformlui64NV = PFNGLUNIFORMLUI64NVPROC (glfwGetProcAddress("glUniformlui64NV"));
02378 glUniformlui64vARB = PFNGLUNIFORMLUI64VARBPROC (glfwGetProcAddress("glUniformlui64vARB"));
02379 glUniformlui64vNV = PFNGLUNIFORMLUI64VNVPROC (glfwGetProcAddress("glUniformlui64vNV"));
02380 glUniformluiv = PFNGLUNIFORMLUIVPROC (glfwGetProcAddress("glUniformluiv"));
02381 glUniform2d = PFNGLUNIFORM2DPROC (glfwGetProcAddress("glUniform2d"));
02382 glUniform2dv = PFNGLUNIFORM2DVPROC (glfwGetProcAddress("glUniform2dv"));
02383 glUniform2f = PFNGLUNIFORM2FPROC (glfwGetProcAddress("glUniform2f"));
02384 glUniform2fv = PFNGLUNIFORM2FVPROC (glfwGetProcAddress("glUniform2fv"));
02385 glUniform2i = PFNGLUNIFORM2IPROC (glfwGetProcAddress("glUniform2i"));
02386 glUniform2i64ARB = PFNGLUNIFORM2I64ARBPROC (glfwGetProcAddress("glUniform2i64ARB"));
02387 glUniform2i64NV = PFNGLUNIFORM2I64NVPROC (glfwGetProcAddress("glUniform2i64NV"));
02388 glUniform2i64vARB = PFNGLUNIFORM2I64VARBPROC (glfwGetProcAddress("glUniform2i64vARB"));
02389 glUniform2i64vNV = PFNGLUNIFORM2I64VNVPROC (glfwGetProcAddress("glUniform2i64vNV"));
02390 glUniform2iv = PFNGLUNIFORM2IVPROC (glfwGetProcAddress("glUniform2iv"));
02391 glUniform2ui = PFNGLUNIFORM2UIPROC (glfwGetProcAddress("glUniform2ui"));
02392 glUniform2ui64ARB = PFNGLUNIFORM2UI64ARBPROC (glfwGetProcAddress("glUniform2ui64ARB"));
02393 glUniform2ui64NV = PFNGLUNIFORM2UI64NVPROC (glfwGetProcAddress("glUniform2ui64NV"));
02394 glUniform2ui64vARB = PFNGLUNIFORM2UI64VARBPROC (glfwGetProcAddress("glUniform2ui64vARB"));
02395 glUniform2ui64vNV = PFNGLUNIFORM2UI64VNVPROC (glfwGetProcAddress("glUniform2ui64vNV"));
02396 glUniform2uiv = PFNGLUNIFORM2UIVPROC (glfwGetProcAddress("glUniform2uiv"));
02397 glUniform3d = PFNGLUNIFORM3DPROC (glfwGetProcAddress("glUniform3d"));
02398 glUniform3dv = PFNGLUNIFORM3DVPROC (glfwGetProcAddress("glUniform3dv"));
02399 glUniform3f = PFNGLUNIFORM3FPROC (glfwGetProcAddress("glUniform3f"));
02400 glUniform3fv = PFNGLUNIFORM3FVPROC (glfwGetProcAddress("glUniform3fv"));
02401 glUniform3i = PFNGLUNIFORM3IPROC (glfwGetProcAddress("glUniform3i"));
02402 glUniform3i64ARB = PFNGLUNIFORM3I64ARBPROC (glfwGetProcAddress("glUniform3i64ARB"));
02403 glUniform3i64NV = PFNGLUNIFORM3I64NVPROC (glfwGetProcAddress("glUniform3i64NV"));
02404 glUniform3i64vARB = PFNGLUNIFORM3I64VARBPROC (glfwGetProcAddress("glUniform3i64vARB"));
02405 glUniform3i64vNV = PFNGLUNIFORM3I64VNVPROC (glfwGetProcAddress("glUniform3i64vNV"));
02406 glUniform3iv = PFNGLUNIFORM3IVPROC (glfwGetProcAddress("glUniform3iv"));
02407 glUniform3ui = PFNGLUNIFORM3UIPROC (glfwGetProcAddress("glUniform3ui"));
02408 glUniform3ui64ARB = PFNGLUNIFORM3UI64ARBPROC (glfwGetProcAddress("glUniform3ui64ARB"));
02409 glUniform3ui64NV = PFNGLUNIFORM3UI64NVPROC (glfwGetProcAddress("glUniform3ui64NV"));
02410 glUniform3ui64vARB = PFNGLUNIFORM3UI64VARBPROC (glfwGetProcAddress("glUniform3ui64vARB"));
02411 glUniform3ui64vNV = PFNGLUNIFORM3UI64VNVPROC (glfwGetProcAddress("glUniform3ui64vNV"));
02412 glUniform3uiv = PFNGLUNIFORM3UIVPROC (glfwGetProcAddress("glUniform3uiv"));
02413 glUniform4d = PFNGLUNIFORM4DPROC (glfwGetProcAddress("glUniform4d"));
02414 glUniform4dv = PFNGLUNIFORM4DVPROC (glfwGetProcAddress("glUniform4dv"));
02415 glUniform4f = PFNGLUNIFORM4FPROC (glfwGetProcAddress("glUniform4f"));
02416 glUniform4fv = PFNGLUNIFORM4FVPROC (glfwGetProcAddress("glUniform4fv"));
02417 glUniform4i = PFNGLUNIFORM4IPROC (glfwGetProcAddress("glUniform4i"));
02418 glUniform4i64ARB = PFNGLUNIFORM4I64ARBPROC (glfwGetProcAddress("glUniform4i64ARB"));
02419 glUniform4i64NV = PFNGLUNIFORM4I64NVPROC (glfwGetProcAddress("glUniform4i64NV"));
02420 glUniform4i64vARB = PFNGLUNIFORM4I64VARBPROC (glfwGetProcAddress("glUniform4i64vARB"));
02421 glUniform4i64vNV = PFNGLUNIFORM4I64VNVPROC (glfwGetProcAddress("glUniform4i64vNV"));
02422 glUniform4iv = PFNGLUNIFORM4IVPROC (glfwGetProcAddress("glUniform4iv"));
02423 glUniform4ui = PFNGLUNIFORM4UIPROC (glfwGetProcAddress("glUniform4ui"));
02424 glUniform4ui64ARB = PFNGLUNIFORM4UI64ARBPROC (glfwGetProcAddress("glUniform4ui64ARB"));
02425 glUniform4ui64NV = PFNGLUNIFORM4UI64NVPROC (glfwGetProcAddress("glUniform4ui64NV"));
02426 glUniform4ui64vARB = PFNGLUNIFORM4UI64VARBPROC (glfwGetProcAddress("glUniform4ui64vARB"));
02427 glUniform4ui64vNV = PFNGLUNIFORM4UI64VNVPROC (glfwGetProcAddress("glUniform4ui64vNV"));
02428 glUniform4uiv = PFNGLUNIFORM4UIVPROC (glfwGetProcAddress("glUniform4uiv"));
02429 glUniformBlockBinding = PFNGLUNIFORMBLOCKBINDINGPROC (glfwGetProcAddress("glUniformBlockBinding"));
02430 glUniformHandleui64ARB =
PFNGLUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress("glUniformHandleui64ARB"));
02431 glUniformHandleui64NV = PFNGLUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress("glUniformHandleui64NV"));
02432 glUniformHandleui64vARB =
PFNGLUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress("glUniformHandleui64vARB"));
02433 glUniformHandleui64vNV =
PFNGLUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress("glUniformHandleui64vNV"));
02434 glUniformMatrix2dv = PFNGLUNIFORMMATRIX2DVPROC (glfwGetProcAddress("glUniformMatrix2dv"));
02435 glUniformMatrix2fv = PFNGLUNIFORMMATRIX2FVPROC (glfwGetProcAddress("glUniformMatrix2fv"));
02436 glUniformMatrix2x3dv = PFNGLUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress("glUniformMatrix2x3dv"));
02437 glUniformMatrix2x3fv = PFNGLUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress("glUniformMatrix2x3fv"));
02438 glUniformMatrix2x4dv = PFNGLUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress("glUniformMatrix2x4dv"));
02439 glUniformMatrix2x4fv = PFNGLUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress("glUniformMatrix2x4fv"));
02440 glUniformMatrix3dv = PFNGLUNIFORMMATRIX3DVPROC (glfwGetProcAddress("glUniformMatrix3dv"));
02441 glUniformMatrix3fv = PFNGLUNIFORMMATRIX3FVPROC (glfwGetProcAddress("glUniformMatrix3fv"));
02442 glUniformMatrix3x2dv = PFNGLUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress("glUniformMatrix3x2dv"));
02443 glUniformMatrix3x2fv = PFNGLUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress("glUniformMatrix3x2fv"));
02444 glUniformMatrix3x4dv = PFNGLUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress("glUniformMatrix3x4dv"));
02445 glUniformMatrix3x4fv = PFNGLUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress("glUniformMatrix3x4fv"));
02446 glUniformMatrix4dv = PFNGLUNIFORMMATRIX4DVPROC (glfwGetProcAddress("glUniformMatrix4dv"));
02447 glUniformMatrix4fv = PFNGLUNIFORMMATRIX4FVPROC (glfwGetProcAddress("glUniformMatrix4fv"));
```

```

02448     glUniformMatrix4x2dv = PFNGLUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress ("glUniformMatrix4x2dv"));
02449     glUniformMatrix4x2fv = PFNGLUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress ("glUniformMatrix4x2fv"));
02450     glUniformMatrix4x3dv = PFNGLUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress ("glUniformMatrix4x3dv"));
02451     glUniformMatrix4x3fv = PFNGLUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress ("glUniformMatrix4x3fv"));
02452     glUniformSubroutinesuiv =
PFNGLUNIFORMSUBROUTINESUIVPROC (glfwGetProcAddress ("glUniformSubroutinesuiv"));
02453     glUniformui64NV = PFNGLUNIFORMUI64NVPROC (glfwGetProcAddress ("glUniformui64NV"));
02454     glUniformui64vNV = PFNGLUNIFORMUI64VNVPROC (glfwGetProcAddress ("glUniformui64vNV"));
02455     glUnmapBuffer = PFNGLUNMAPBUFFERPROC (glfwGetProcAddress ("glUnmapBuffer"));
02456     glUnmapNamedBuffer = PFNGLUNMAPNAMEDBUFFERPROC (glfwGetProcAddress ("glUnmapNamedBuffer"));
02457     glUnmapNamedBufferEXT = PFNGLUNMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress ("glUnmapNamedBufferEXT"));
02458     glUseProgram = PFNGLUSEPROGRAMPROC (glfwGetProcAddress ("glUseProgram"));
02459     glUseProgramStages = PFNGLUSEPROGRAMSTAGESPROC (glfwGetProcAddress ("glUseProgramStages"));
02460     glUseShaderProgramEXT = PFNGLUSESHADERPROGRAMEXTPROC (glfwGetProcAddress ("glUseShaderProgramEXT"));
02461     glValidateProgram = PFNGLVALIDATEPROGRAMPROC (glfwGetProcAddress ("glValidateProgram"));
02462     glValidateProgramPipeline =
PFNGLVALIDATEPROGRAMPIPELINEPROC (glfwGetProcAddress ("glValidateProgramPipeline"));
02463     glVertexArrayAttribBinding =
PFNGLVERTEXARRAYATTRIBBINDINGPROC (glfwGetProcAddress ("glVertexArrayAttribBinding"));
02464     glVertexArrayAttribFormat =
PFNGLVERTEXARRAYATTRIBFORMATPROC (glfwGetProcAddress ("glVertexArrayAttribFormat"));
02465     glVertexArrayAttribIFormat =
PFNGLVERTEXARRAYATTRIBIFORMATPROC (glfwGetProcAddress ("glVertexArrayAttribIFormat"));
02466     glVertexArrayAttribLFormat =
PFNGLVERTEXARRAYATTRIBLFORMATPROC (glfwGetProcAddress ("glVertexArrayAttribLFormat"));
02467     glVertexArrayBindVertexBufferEXT =
PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC (glfwGetProcAddress ("glVertexArrayBindVertexBufferEXT"));
02468     glVertexArrayBindingDivisor =
PFNGLVERTEXARRAYBINDINGDIVISORPROC (glfwGetProcAddress ("glVertexArrayBindingDivisor"));
02469     glVertexArrayColorOffsetEXT =
PFNGLVERTEXARRAYCOLOROFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayColorOffsetEXT"));
02470     glVertexArrayEdgeFlagOffsetEXT =
PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayEdgeFlagOffsetEXT"));
02471     glVertexArrayElementBuffer =
PFNGLVERTEXARRAYELEMENTBUFFERPROC (glfwGetProcAddress ("glVertexArrayElementBuffer"));
02472     glVertexArrayFogCoordOffsetEXT =
PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayFogCoordOffsetEXT"));
02473     glVertexArrayIndexOffsetEXT =
PFNGLVERTEXARRAYINDEXOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayIndexOffsetEXT"));
02474     glVertexArrayMultiTexCoordOffsetEXT =
PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayMultiTexCoordOffsetEXT"));
02475     glVertexArrayNormalOffsetEXT =
PFNGLVERTEXARRAYNORMALOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayNormalOffsetEXT"));
02476     glVertexArraySecondaryColorOffsetEXT =
PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC (glfwGetProcAddress ("glVertexArraySecondaryColorOffsetEXT"));
02477     glVertexArrayTexCoordOffsetEXT =
PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayTexCoordOffsetEXT"));
02478     glVertexArrayVertexAttribBindingEXT =
PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribBindingEXT"));
02479     glVertexArrayVertexAttribDivisorEXT =
PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribDivisorEXT"));
02480     glVertexArrayVertexAttribFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribFormatEXT"));
02481     glVertexArrayVertexAttribIFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIFormatEXT"));
02482     glVertexArrayVertexAttribIOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIOffsetEXT"));
02483     glVertexArrayVertexAttribLFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLFormatEXT"));
02484     glVertexArrayVertexAttribLOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLOffsetEXT"));
02485     glVertexArrayVertexAttribOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribOffsetEXT"));
02486     glVertexArrayVertexBindingDivisorEXT =
PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexBindingDivisorEXT"));
02487     glVertexArrayVertexBuffer =
PFNGLVERTEXARRAYVERTEXBUFFERPROC (glfwGetProcAddress ("glVertexArrayVertexBuffer"));
02488     glVertexArrayVertexBuffers =
PFNGLVERTEXARRAYVERTEXBUFFERSPROC (glfwGetProcAddress ("glVertexArrayVertexBuffers"));
02489     glVertexArrayVertexOffsetEXT =
PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC (glfwGetProcAddress ("glVertexArrayVertexOffsetEXT"));
02490     glVertexAttribld = PFNGLVERTEXATTRIBLDPROC (glfwGetProcAddress ("glVertexAttribld"));
02491     glVertexAttribldv = PFNGLVERTEXATTRIBLDVPROC (glfwGetProcAddress ("glVertexAttribldv"));
02492     glVertexAttribl1f = PFNGLVERTEXATTRIB1FPROC (glfwGetProcAddress ("glVertexAttribl1f"));
02493     glVertexAttribl1fv = PFNGLVERTEXATTRIB1FVPROC (glfwGetProcAddress ("glVertexAttribl1fv"));
02494     glVertexAttribl1s = PFNGLVERTEXATTRIB1SPROC (glfwGetProcAddress ("glVertexAttribl1s"));
02495     glVertexAttribl1sv = PFNGLVERTEXATTRIB1SVPROC (glfwGetProcAddress ("glVertexAttribl1sv"));
02496     glVertexAttrib2d = PFNGLVERTEXATTRIB2DPROC (glfwGetProcAddress ("glVertexAttrib2d"));
02497     glVertexAttrib2dv = PFNGLVERTEXATTRIB2DVPROC (glfwGetProcAddress ("glVertexAttrib2dv"));
02498     glVertexAttrib2f = PFNGLVERTEXATTRIB2FPROC (glfwGetProcAddress ("glVertexAttrib2f"));
02499     glVertexAttrib2fv = PFNGLVERTEXATTRIB2FVPROC (glfwGetProcAddress ("glVertexAttrib2fv"));
02500     glVertexAttrib2s = PFNGLVERTEXATTRIB2SPROC (glfwGetProcAddress ("glVertexAttrib2s"));
02501     glVertexAttrib2sv = PFNGLVERTEXATTRIB2SVPROC (glfwGetProcAddress ("glVertexAttrib2sv"));
02502     glVertexAttrib3d = PFNGLVERTEXATTRIB3DPROC (glfwGetProcAddress ("glVertexAttrib3d"));
02503     glVertexAttrib3dv = PFNGLVERTEXATTRIB3DVPROC (glfwGetProcAddress ("glVertexAttrib3dv"));
02504     glVertexAttrib3f = PFNGLVERTEXATTRIB3FPROC (glfwGetProcAddress ("glVertexAttrib3f"));
02505     glVertexAttrib3fv = PFNGLVERTEXATTRIB3FVPROC (glfwGetProcAddress ("glVertexAttrib3fv"));

```

```
02506     glVertexAttrib3s = PFNGLVERTEXATTRIB3SPROC (glfwGetProcAddress("glVertexAttrib3s"));
02507     glVertexAttrib3sv = PFNGLVERTEXATTRIB3SVPROC (glfwGetProcAddress("glVertexAttrib3sv"));
02508     glVertexAttrib4Nubv = PFNGLVERTEXATTRIB4NubVPROC (glfwGetProcAddress("glVertexAttrib4Nubv"));
02509     glVertexAttrib4Niv = PFNGLVERTEXATTRIB4NIVPROC (glfwGetProcAddress("glVertexAttrib4Niv"));
02510     glVertexAttrib4Nsv = PFNGLVERTEXATTRIB4NSVPROC (glfwGetProcAddress("glVertexAttrib4Nsv"));
02511     glVertexAttrib4Nubv = PFNGLVERTEXATTRIB4NubVPROC (glfwGetProcAddress("glVertexAttrib4Nubv"));
02512     glVertexAttrib4Nuiv = PFNGLVERTEXATTRIB4NuIVPROC (glfwGetProcAddress("glVertexAttrib4Nuiv"));
02513     glVertexAttrib4Nusv = PFNGLVERTEXATTRIB4NusVPROC (glfwGetProcAddress("glVertexAttrib4Nusv"));
02514     glVertexAttrib4bv = PFNGLVERTEXATTRIB4BVPROC (glfwGetProcAddress("glVertexAttrib4bv"));
02515     glVertexAttrib4d = PFNGLVERTEXATTRIB4DPROC (glfwGetProcAddress("glVertexAttrib4d"));
02516     glVertexAttrib4dv = PFNGLVERTEXATTRIB4DVPROC (glfwGetProcAddress("glVertexAttrib4dv"));
02517     glVertexAttrib4f = PFNGLVERTEXATTRIB4FPROC (glfwGetProcAddress("glVertexAttrib4f"));
02518     glVertexAttrib4fv = PFNGLVERTEXATTRIB4FVPROC (glfwGetProcAddress("glVertexAttrib4fv"));
02519     glVertexAttrib4iv = PFNGLVERTEXATTRIB4IVPROC (glfwGetProcAddress("glVertexAttrib4iv"));
02520     glVertexAttrib4s = PFNGLVERTEXATTRIB4SPROC (glfwGetProcAddress("glVertexAttrib4s"));
02521     glVertexAttrib4sv = PFNGLVERTEXATTRIB4SVPROC (glfwGetProcAddress("glVertexAttrib4sv"));
02522     glVertexAttrib4ubv = PFNGLVERTEXATTRIB4UBVPROC (glfwGetProcAddress("glVertexAttrib4ubv"));
02523     glVertexAttrib4uiv = PFNGLVERTEXATTRIB4UIVPROC (glfwGetProcAddress("glVertexAttrib4uiv"));
02524     glVertexAttrib4usv = PFNGLVERTEXATTRIB4USVPROC (glfwGetProcAddress("glVertexAttrib4usv"));
02525     glVertexAttribBinding = PFNGLVERTEXATTRIBBINDINGPROC (glfwGetProcAddress("glVertexAttribBinding"));
02526     glVertexAttribDivisor = PFNGLVERTEXATTRIBDIVISORPROC (glfwGetProcAddress("glVertexAttribDivisor"));
02527     glVertexAttribDivisorARB = PFNGLVERTEXATTRIBDIVISORARBPROC (glfwGetProcAddress("glVertexAttribDivisorARB"));
02528     PFNGLVERTEXATTRIBDIVISORARBPROC (glfwGetProcAddress("glVertexAttribDivisorARB"));
02529     glVertexAttribFormat = PFNGLVERTEXATTRIBFORMATPROC (glfwGetProcAddress("glVertexAttribFormat"));
02530     glVertexAttribFormatNV = PFNGLVERTEXATTRIBFORMATNVPROC (glfwGetProcAddress("glVertexAttribFormatNV"));
02531     glVertexAttribI1i = PFNGLVERTEXATTRIBI1IPROC (glfwGetProcAddress("glVertexAttribI1i"));
02532     glVertexAttribI1iv = PFNGLVERTEXATTRIBI1IVPROC (glfwGetProcAddress("glVertexAttribI1iv"));
02533     glVertexAttribI1ui = PFNGLVERTEXATTRIBI1UIPROC (glfwGetProcAddress("glVertexAttribI1ui"));
02534     glVertexAttribI1uiv = PFNGLVERTEXATTRIBI1UIVPROC (glfwGetProcAddress("glVertexAttribI1uiv"));
02535     glVertexAttribI2i = PFNGLVERTEXATTRIBI2IPROC (glfwGetProcAddress("glVertexAttribI2i"));
02536     glVertexAttribI2iv = PFNGLVERTEXATTRIBI2IVPROC (glfwGetProcAddress("glVertexAttribI2iv"));
02537     glVertexAttribI2ui = PFNGLVERTEXATTRIBI2UIPROC (glfwGetProcAddress("glVertexAttribI2ui"));
02538     glVertexAttribI2uiv = PFNGLVERTEXATTRIBI2UIVPROC (glfwGetProcAddress("glVertexAttribI2uiv"));
02539     glVertexAttribI3i = PFNGLVERTEXATTRIBI3IPROC (glfwGetProcAddress("glVertexAttribI3i"));
02540     glVertexAttribI3iv = PFNGLVERTEXATTRIBI3IVPROC (glfwGetProcAddress("glVertexAttribI3iv"));
02541     glVertexAttribI3ui = PFNGLVERTEXATTRIBI3UIPROC (glfwGetProcAddress("glVertexAttribI3ui"));
02542     glVertexAttribI3uiv = PFNGLVERTEXATTRIBI3UIVPROC (glfwGetProcAddress("glVertexAttribI3uiv"));
02543     glVertexAttribI4bv = PFNGLVERTEXATTRIBI4BVPROC (glfwGetProcAddress("glVertexAttribI4bv"));
02544     glVertexAttribI4i = PFNGLVERTEXATTRIBI4IPROC (glfwGetProcAddress("glVertexAttribI4i"));
02545     glVertexAttribI4iv = PFNGLVERTEXATTRIBI4IVPROC (glfwGetProcAddress("glVertexAttribI4iv"));
02546     glVertexAttribI4sv = PFNGLVERTEXATTRIBI4SVPROC (glfwGetProcAddress("glVertexAttribI4sv"));
02547     glVertexAttribI4ubv = PFNGLVERTEXATTRIBI4UBVPROC (glfwGetProcAddress("glVertexAttribI4ubv"));
02548     glVertexAttribI4ui = PFNGLVERTEXATTRIBI4UIPROC (glfwGetProcAddress("glVertexAttribI4ui"));
02549     glVertexAttribI4uiv = PFNGLVERTEXATTRIBI4UIVPROC (glfwGetProcAddress("glVertexAttribI4uiv"));
02550     glVertexAttribI4usv = PFNGLVERTEXATTRIBI4USVPROC (glfwGetProcAddress("glVertexAttribI4usv"));
02551     glVertexAttribIFormat = PFNGLVERTEXATTRIBIFORMATPROC (glfwGetProcAddress("glVertexAttribIFormat"));
02552     glVertexAttribIFormatNV = PFNGLVERTEXATTRIBIFORMATNVPROC (glfwGetProcAddress("glVertexAttribIFormatNV"));
02553     glVertexAttribIPointer = PFNGLVERTEXATTRIBIPOINTERPROC (glfwGetProcAddress("glVertexAttribIPointer"));
02554     glVertexAttribL1d = PFNGLVERTEXATTRIBL1DPROC (glfwGetProcAddress("glVertexAttribL1d"));
02555     glVertexAttribL1dv = PFNGLVERTEXATTRIBL1DVPROC (glfwGetProcAddress("glVertexAttribL1dv"));
02556     glVertexAttribL1i64NV = PFNGLVERTEXATTRIBL1i64NVPROC (glfwGetProcAddress("glVertexAttribL1i64NV"));
02557     glVertexAttribL1i64vNV = PFNGLVERTEXATTRIBL1i64vNVPROC (glfwGetProcAddress("glVertexAttribL1i64vNV"));
02558     glVertexAttribL1ui64ARB = PFNGLVERTEXATTRIBL1UI64ARBPROC (glfwGetProcAddress("glVertexAttribL1ui64ARB"));
02559     glVertexAttribL1ui64NV = PFNGLVERTEXATTRIBL1UI64NVPROC (glfwGetProcAddress("glVertexAttribL1ui64NV"));
02560     glVertexAttribL1ui64vARB = PFNGLVERTEXATTRIBL1UI64vARBPROC (glfwGetProcAddress("glVertexAttribL1ui64vARB"));
02561     glVertexAttribL1ui64vNV = PFNGLVERTEXATTRIBL1UI64vNVPROC (glfwGetProcAddress("glVertexAttribL1ui64vNV"));
02562     glVertexAttribL2d = PFNGLVERTEXATTRIBL2DPROC (glfwGetProcAddress("glVertexAttribL2d"));
02563     glVertexAttribL2dv = PFNGLVERTEXATTRIBL2DVPROC (glfwGetProcAddress("glVertexAttribL2dv"));
02564     glVertexAttribL2i64NV = PFNGLVERTEXATTRIBL2i64NVPROC (glfwGetProcAddress("glVertexAttribL2i64NV"));
02565     glVertexAttribL2i64vNV = PFNGLVERTEXATTRIBL2i64vNVPROC (glfwGetProcAddress("glVertexAttribL2i64vNV"));
02566     glVertexAttribL2ui64NV = PFNGLVERTEXATTRIBL2UI64NVPROC (glfwGetProcAddress("glVertexAttribL2ui64NV"));
02567     glVertexAttribL2ui64vNV = PFNGLVERTEXATTRIBL2UI64vNVPROC (glfwGetProcAddress("glVertexAttribL2ui64vNV"));
02568     glVertexAttribL3d = PFNGLVERTEXATTRIBL3DPROC (glfwGetProcAddress("glVertexAttribL3d"));
02569     glVertexAttribL3dv = PFNGLVERTEXATTRIBL3DVPROC (glfwGetProcAddress("glVertexAttribL3dv"));
02570     glVertexAttribL3i64NV = PFNGLVERTEXATTRIBL3i64NVPROC (glfwGetProcAddress("glVertexAttribL3i64NV"));
02571     glVertexAttribL3i64vNV = PFNGLVERTEXATTRIBL3i64vNVPROC (glfwGetProcAddress("glVertexAttribL3i64vNV"));
02572     glVertexAttribL3ui64NV = PFNGLVERTEXATTRIBL3UI64NVPROC (glfwGetProcAddress("glVertexAttribL3ui64NV"));
02573     glVertexAttribL3ui64vNV = PFNGLVERTEXATTRIBL3UI64vNVPROC (glfwGetProcAddress("glVertexAttribL3ui64vNV"));
02574     glVertexAttribL4d = PFNGLVERTEXATTRIBL4DPROC (glfwGetProcAddress("glVertexAttribL4d"));
02575     glVertexAttribL4dv = PFNGLVERTEXATTRIBL4DVPROC (glfwGetProcAddress("glVertexAttribL4dv"));
02576     glVertexAttribL4i64NV = PFNGLVERTEXATTRIBL4i64NVPROC (glfwGetProcAddress("glVertexAttribL4i64NV"));
02577     glVertexAttribL4i64vNV = PFNGLVERTEXATTRIBL4i64vNVPROC (glfwGetProcAddress("glVertexAttribL4i64vNV"));
```

```

    PFNGLVERTEXATTRIBL4I64VNVPROC (glfwGetProcAddress("glVertexAttribL4i64vNV"));
22578     glVertexAttribL4ui64NV =
    PFNGLVERTEXATTRIBL4UI64NVPROC (glfwGetProcAddress("glVertexAttribL4ui64vNV"));
22579     glVertexAttribL4ui64vNV =
    PFNGLVERTEXATTRIBL4UI64VNVPROC (glfwGetProcAddress("glVertexAttribL4ui64vNV"));
22580     glVertexAttribLFormat = PFNGLVERTEXATTRIBLFORMATPROC (glfwGetProcAddress("glVertexAttribLFormat"));
22581     glVertexAttribLFormatNV =
    PFNGLVERTEXATTRIBLFORMATNVPROC (glfwGetProcAddress("glVertexAttribLFormatNV"));
22582     glVertexAttribLPointer =
    PFNGLVERTEXATTRIBLPOINTERPROC (glfwGetProcAddress("glVertexAttribLPointer"));
22583     glVertexAttribP1ui = PFNGLVERTEXATTRIBP1UIPROC (glfwGetProcAddress("glVertexAttribP1ui"));
22584     glVertexAttribP1uiv = PFNGLVERTEXATTRIBP1UIVPROC (glfwGetProcAddress("glVertexAttribP1uiv"));
22585     glVertexAttribP2ui = PFNGLVERTEXATTRIBP2UIPROC (glfwGetProcAddress("glVertexAttribP2ui"));
22586     glVertexAttribP2uiv = PFNGLVERTEXATTRIBP2UIVPROC (glfwGetProcAddress("glVertexAttribP2uiv"));
22587     glVertexAttribP3ui = PFNGLVERTEXATTRIBP3UIPROC (glfwGetProcAddress("glVertexAttribP3ui"));
22588     glVertexAttribP3uiv = PFNGLVERTEXATTRIBP3UIVPROC (glfwGetProcAddress("glVertexAttribP3uiv"));
22589     glVertexAttribP4ui = PFNGLVERTEXATTRIBP4UIPROC (glfwGetProcAddress("glVertexAttribP4ui"));
22590     glVertexAttribP4uiv = PFNGLVERTEXATTRIBP4UIVPROC (glfwGetProcAddress("glVertexAttribP4uiv"));
22591     glVertexAttribPointer = PFNGLVERTEXATTRIBPOINTERPROC (glfwGetProcAddress("glVertexAttribPointer"));
22592     glVertexBindingDivisor =
    PFNGLVERTEXBINDINGDIVISORPROC (glfwGetProcAddress("glVertexBindingDivisor"));
22593     glVertexFormatNV = PFNGLVERTEXFORMATNVPROC (glfwGetProcAddress("glVertexFormatNV"));
22594     glViewport = PFNGLVIEWPORTPROC (glfwGetProcAddress("glViewport"));
22595     glViewportArrayv = PFNGLVIEWPORTARRAYVPROC (glfwGetProcAddress("glViewportArrayv"));
22596     glViewportIndexedf = PFNGLVIEWPORTINDEXEDFPROC (glfwGetProcAddress("glViewportIndexedf"));
22597     glViewportIndexedfv = PFNGLVIEWPORTINDEXEDFVPROC (glfwGetProcAddress("glViewportIndexedfv"));
22598     glViewportPositionWScaleNV =
    PFNGLVIEWPORTPOSITIONWSCALENVPROC (glfwGetProcAddress("glViewportPositionWScaleNV"));
22599     glViewportSwizzleNV = PFNGLVIEWPORTSWIZZLENVPROC (glfwGetProcAddress("glViewportSwizzleNV"));
22600     glWaitSync = PFNGLWAITSYNCPROC (glfwGetProcAddress("glWaitSync"));
22601     glWaitVkSemaphoreNV = PFNGLWAITVKSEMAPHORENVPROC (glfwGetProcAddress("glWaitVkSemaphoreNV"));
22602     glWeightPathsNV = PFNGLWEIGHTPATHSNVPROC (glfwGetProcAddress("glWeightPathsNV"));
22603     glWindowRectanglesEXT = PFNGLWINDOWRECTANGLESEXTPROC (glfwGetProcAddress("glWindowRectanglesEXT"));
22604 #endif
22605
22606     // 使用している GPU のバッファアライメントを調べる
22607     glGetIntegerv(GL_UNIFORM_BUFFER_OFFSET_ALIGNMENT, &ggBufferAlignment);
22608 }
22609
22610 //
22611 // OpenGL のエラーをチェックする
22612 //
22613 //   OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
22614 //
22615 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
22616 //
22617 void gg::ggError(const std::string& name, unsigned int line)
22618 {
22619     const GLenum error{ glGetError() };
22620
22621     if (error != GL_NO_ERROR)
22622     {
22623         if (!name.empty())
22624         {
22625             std::cerr << name;
22626             if (line > 0) std::cerr << " (" << line << ")";
22627             std::cerr << ": ";
22628         }
22629
22630         switch (error)
22631         {
22632             case GL_INVALID_ENUM:
22633                 std::cerr << "An unacceptable value is specified for an enumerated argument" << std::endl;
22634                 break;
22635             case GL_INVALID_VALUE:
22636                 std::cerr << "A numeric argument is out of range" << std::endl;
22637                 break;
22638             case GL_INVALID_OPERATION:
22639                 std::cerr << "The specified operation is not allowed in the current state" << std::endl;
22640                 break;
22641             case GL_OUT_OF_MEMORY:
22642                 std::cerr << "There is not enough memory left to execute the command" << std::endl;
22643                 break;
22644             case GL_INVALID_FRAMEBUFFER_OPERATION:
22645                 std::cerr << "The specified operation is not allowed current frame buffer" << std::endl;
22646                 break;
22647             default:
22648                 std::cerr << "An OpenGL error has occurred: " << std::hex << std::showbase << error << std::endl;
22649                 break;
22650         }
22651     }
22652 }
22653
22654 //
22655 // FBO のエラーをチェックする
22656 //
22657 //   FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する

```



```

02658 //
02659 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02660 //
02661 void gg::ggFBOError(const std::string& name, unsigned int line)
02662 {
02663     const auto status{ glCheckFramebufferStatus(GL_FRAMEBUFFER) };
02664
02665     if (status != GL_FRAMEBUFFER_COMPLETE)
02666     {
02667         if (!name.empty())
02668         {
02669             std::cerr << name;
02670             if (line > 0) std::cerr << " (" << line << ")";
02671             std::cerr << ": ";
02672         }
02673
02674         switch (status)
02675         {
02676             case GL_FRAMEBUFFER_UNDEFINED:
02677                 std::cerr << "the default framebuffer does not exist" << std::endl;
02678                 break;
02679             case GL_FRAMEBUFFER_INCOMPLETE_ATTACHMENT:
02680                 std::cerr << "Framebuffer incomplete, duplicate attachment" << std::endl;
02681                 break;
02682             case GL_FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT:
02683                 std::cerr << "Framebuffer incomplete, missing attachment" << std::endl;
02684                 break;
02685             case GL_FRAMEBUFFER_UNSUPPORTED:
02686                 std::cerr << "Unsupported framebuffer internal" << std::endl;
02687                 break;
02688             case GL_FRAMEBUFFER_INCOMPLETE_MULTISAMPLE:
02689                 std::cerr << "The value of GL_RENDERBUFFER_SAMPLES is not"
02690                     " the same for all attached renderbuffers or,"
02691                     " if the attached images are a mix of renderbuffers and textures,"
02692                     " the value of GL_RENDERBUFFER_SAMPLES is not zero" << std::endl;
02693                 break;
02694             #if !defined(GL_GLES_PROTOTYPES)
02695             case GL_FRAMEBUFFER_INCOMPLETE_LAYER_TARGETS:
02696                 std::cerr << "Any framebuffer attachment is layered,"
02697                     " and any populated attachment is not layered,"
02698                     " or if all populated color attachments are not from textures"
02699                     " of the same target" << std::endl;
02700                 break;
02701             case GL_FRAMEBUFFER_INCOMPLETE_DRAW_BUFFER:
02702                 std::cerr << "Framebuffer incomplete, missing draw buffer" << std::endl;
02703                 break;
02704             case GL_FRAMEBUFFER_INCOMPLETE_READ_BUFFER:
02705                 std::cerr << "Framebuffer incomplete, missing read buffer" << std::endl;
02706                 break;
02707             #endif
02708             default:
02709                 std::cerr << "Programming error; will fail on all hardware: " << std::hex << std::showbase <<
02710                     status << std::endl;
02711                 break;
02712         }
02713     }
02714
02715     //
02716     //   変換行列: 行列とベクトルの積  $c \leftarrow a \times b$ 
02717     //
02718     void gg::GgMatrix::projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02719     {
02720         for (int i = 0; i < 4; ++i)
02721         {
02722             c[i] = a[0 + i] * b[0] + a[4 + i] * b[1] + a[8 + i] * b[2] + a[12 + i] * b[3];
02723         }
02724     }
02725
02726     //
02727     //   変換行列: 行列と行列の積  $c \leftarrow a \times b$ 
02728     //
02729     void gg::GgMatrix::multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02730     {
02731         for (int i = 0; i < 16; ++i)
02732         {
02733             int j = i & 3, k = i ^ 3;
02734
02735             c[i] = a[0 + j] * b[k + 0] + a[4 + j] * b[k + 1] + a[8 + j] * b[k + 2] + a[12 + j] * b[k + 3];
02736         }
02737     }
02738
02739     //
02740     //   変換行列: 単位行列を設定する
02741     //
02742     gg::GgMatrix& gg::GgMatrix::loadIdentity()
02743     {

```



```

02744     *this =
02745     {
02746         1.0f, 0.0f, 0.0f, 0.0f,
02747         0.0f, 1.0f, 0.0f, 0.0f,
02748         0.0f, 0.0f, 1.0f, 0.0f,
02749         0.0f, 0.0f, 0.0f, 1.0f
02750     };
02751
02752     return *this;
02753 }
02754
02755 //
02756 // 変換行列：平行移動変換行列を設定する
02757 //
02758 gg::GgMatrix& gg::GgMatrix::loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02759 {
02760     *this =
02761     {
02762         w,      0.0f, 0.0f, 0.0f,
02763         0.0f, w,      0.0f, 0.0f,
02764         0.0f, 0.0f, w,      0.0f,
02765         x,      y,      z,      w
02766     };
02767
02768     return *this;
02769 }
02770
02771 //
02772 // 変換行列：拡大縮小変換行列を設定する
02773 //
02774 gg::GgMatrix& gg::GgMatrix::loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02775 {
02776     *this =
02777     {
02778         x,      0.0f, 0.0f, 0.0f,
02779         0.0f, y,      0.0f, 0.0f,
02780         0.0f, 0.0f, z,      0.0f,
02781         0.0f, 0.0f, 0.0f, w
02782     };
02783
02784     return *this;
02785 }
02786
02787 //
02788 // 変換行列：x 軸中心の回転変換行列を設定する
02789 //
02790 gg::GgMatrix& gg::GgMatrix::loadRotateX(GLfloat a)
02791 {
02792     const auto c{ cosf(a) };
02793     const auto s{ sinf(a) };
02794
02795     *this =
02796     {
02797         1.0f, 0.0f, 0.0f, 0.0f,
02798         0.0f, c,      s,      0.0f,
02799         0.0f, -s,     c,      0.0f,
02800         0.0f, 0.0f, 0.0f, 1.0f
02801     };
02802
02803     return *this;
02804 }
02805
02806 //
02807 // 変換行列：y 軸中心の回転変換行列を設定する
02808 //
02809 gg::GgMatrix& gg::GgMatrix::loadRotateY(GLfloat a)
02810 {
02811     const auto c{ cosf(a) };
02812     const auto s{ sinf(a) };
02813
02814     *this =
02815     {
02816         c,      0.0f, -s,      0.0f,
02817         0.0f, 1.0f, 0.0f, 0.0f,
02818         s,      0.0f, c,      0.0f,
02819         0.0f, 0.0f, 0.0f, 1.0f
02820     };
02821
02822     return *this;
02823 }
02824
02825 //
02826 // 変換行列：z 軸中心の回転変換行列を設定する
02827 //
02828 gg::GgMatrix& gg::GgMatrix::loadRotateZ(GLfloat a)
02829 {
02830     const auto c{ cosf(a) };

```

```

02831     const auto s{ sinf(a) };
02832
02833     *this =
02834     {
02835         c,      s,      0.0f, 0.0f,
02836         -s,     c,      0.0f, 0.0f,
02837         0.0f, 0.0f, 1.0f, 0.0f,
02838         0.0f, 0.0f, 0.0f, 1.0f
02839     };
02840
02841     return *this;
02842 }
02843
02844 //
02845 // 変換行列：任意軸中心の回転変換行列を設定する
02846 //
02847 gg::GgMatrix& gg::GgMatrix::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
02848 {
02849     const auto d{ sqrtf(x * x + y * y + z * z) };
02850
02851     if (d > 0.0f)
02852     {
02853         const auto l{ x / d }, m{ y / d }, n{ z / d };
02854         const auto l2{ l * l }, m2{ m * m }, n2{ n * n };
02855         const auto lm{ l * m }, mn{ m * n }, nl{ n * l };
02856         const auto c{ cosf(a) }, cl{ 1.0f - c };
02857         const auto s{ sinf(a) };
02858
02859         *this =
02860         {
02861             (1.0f - l2) * c + l2,
02862             lm * c1 + n * s,
02863             nl * c1 - m * s,
02864             0.0f,
02865
02866             lm * c1 - n * s,
02867             (1.0f - m2) * c + m2,
02868             mn * c1 + l * s,
02869             0.0f,
02870
02871             nl * c1 + m * s,
02872             mn * c1 - l * s,
02873             (1.0f - n2) * c + n2,
02874             0.0f,
02875
02876             0.0f,
02877             0.0f,
02878             0.0f,
02879             1.0f,
02880         };
02881     }
02882
02883     return *this;
02884 }
02885
02886 //
02887 // 変換行列：転置行列を設定する
02888 //
02889 gg::GgMatrix& gg::GgMatrix::loadTranspose(const GLfloat* marray)
02890 {
02891     *this =
02892     {
02893         marray[ 0],
02894         marray[ 4],
02895         marray[ 8],
02896         marray[12],
02897         marray[ 1],
02898         marray[ 5],
02899         marray[ 9],
02900         marray[13],
02901         marray[ 2],
02902         marray[ 6],
02903         marray[10],
02904         marray[14],
02905         marray[ 3],
02906         marray[ 7],
02907         marray[11],
02908         marray[15]
02909     };
02910
02911     return *this;
02912 }
02913
02914 //
02915 // 変換行列：逆行列を設定する
02916 //
02917 gg::GgMatrix& gg::GgMatrix::loadInvert(const GLfloat* marray)

```

```

02918 {
02919     GLfloat lu[20];
02920     GLfloat* plu[4];
02921
02922     // j 行の要素の値の絶対値の最大値を plu[j][4] に求める
02923     for (int j = 0; j < 4; ++j)
02924     {
02925         auto max{ fabs(*(plu[j] = lu + 5 * j) = *(marray++)) };
02926
02927         for (int i = 0; ++i < 4;)
02928         {
02929             auto a{ fabs(plu[j][i] = *(marray++)) };
02930             if (a > max) max = a;
02931         }
02932         if (max == 0.0f) return *this;
02933         plu[j][4] = 1.0f / max;
02934     }
02935
02936     // ピボットを考慮した LU 分解
02937     for (int j = 0; j < 4; ++j)
02938     {
02939         auto max{ fabs(plu[j][j] * plu[j][4]) };
02940         int i = j;
02941
02942         for (int k = j; ++k < 4;)
02943         {
02944             auto a{ fabs(plu[k][j] * plu[k][4]) };
02945             if (a > max)
02946             {
02947                 max = a;
02948                 i = k;
02949             }
02950         }
02951         if (i > j)
02952         {
02953             auto* t{ plu[j] };
02954             plu[j] = plu[i];
02955             plu[i] = t;
02956         }
02957         if (plu[j][j] == 0.0f) return *this;
02958         for (int k = j; ++k < 4;)
02959         {
02960             plu[k][j] /= plu[j][j];
02961             for (int i = j; ++i < 4;)
02962             {
02963                 plu[k][i] -= plu[j][i] * plu[k][j];
02964             }
02965         }
02966     }
02967
02968     // LU 分解から逆行列を求める
02969     for (int k = 0; k < 4; ++k)
02970     {
02971         // array に単位行列を設定する
02972         for (int i = 0; i < 4; ++i)
02973         {
02974             (*this)[i * 4 + k] = (plu[i] == lu + k * 5) ? 1.0f : 0.0f;
02975         }
02976         // lu から逆行列を求める
02977         for (int i = 0; i < 4; ++i)
02978         {
02979             for (int j = i; ++j < 4;)
02980             {
02981                 (*this)[j * 4 + k] -= (*this)[i * 4 + k] * plu[j][i];
02982             }
02983         }
02984         for (int i = 4; --i >= 0;)
02985         {
02986             for (int j = i; ++j < 4;)
02987             {
02988                 (*this)[i * 4 + k] -= plu[i][j] * (*this)[j * 4 + k];
02989             }
02990             (*this)[i * 4 + k] /= plu[i][i];
02991         }
02992     }
02993
02994     return *this;
02995 }
02996
02997 //
02998 // 変換行列：法線変換行列を設定する
02999 //
03000 gg::GgMatrix& gg::GgMatrix::loadNormal(const GLfloat* marray)
03001 {
03002     *this =
03003     {
03004         marray[ 5] * marray[10] - marray[ 6] * marray[ 9],

```

```

03005     marray[ 6] * marray[ 8] - marray[ 4] * marray[10],
03006     marray[ 4] * marray[ 9] - marray[ 5] * marray[ 8],
03007     0.0f,
03008
03009     marray[ 9] * marray[ 2] - marray[10] * marray[ 1],
03010     marray[10] * marray[ 0] - marray[ 8] * marray[ 2],
03011     marray[ 8] * marray[ 1] - marray[ 9] * marray[ 0],
03012     0.0f,
03013
03014     marray[ 1] * marray[ 6] - marray[ 2] * marray[ 5],
03015     marray[ 2] * marray[ 4] - marray[ 0] * marray[ 6],
03016     marray[ 0] * marray[ 5] - marray[ 1] * marray[ 4],
03017     0.0f,
03018
03019     0.0f,
03020     0.0f,
03021     0.0f,
03022     1.0f
03023 };
03024
03025 return *this;
03026 }
03027
03028 //
03029 // 変換行列：ビュー変換行列を設定する
03030 //
03031 gg::GgMatrix& gg::GgMatrix::loadLookat(
03032     GLfloat ex, GLfloat ey, GLfloat ez,
03033     GLfloat tx, GLfloat ty, GLfloat tz,
03034     GLfloat ux, GLfloat uy, GLfloat uz
03035 )
03036 {
03037     // z 軸 = e - t
03038     const auto zx{ ex - tx };
03039     const auto zy{ ey - ty };
03040     const auto zz{ ez - tz };
03041
03042     // z 軸の長さ
03043     const auto z{ sqrtf(zx * zx + zy * zy + zz * zz) };
03044
03045     // x 軸 = u × z 軸
03046     const auto xx{ uy * zz - uz * zy };
03047     const auto xy{ uz * zx - ux * zz };
03048     const auto xz{ ux * zy - uy * zx };
03049
03050     // x 軸の長さ
03051     const auto x{ sqrtf(xx * xx + xy * xy + xz * xz) };
03052
03053     // y 軸 = z 軸 × x 軸
03054     const auto yx{ zy * xz - zz * xy };
03055     const auto yy{ zz * xx - zx * xz };
03056     const auto yz{ zx * xy - zy * xx };
03057
03058     // y 軸の長さ
03059     const auto y{ sqrtf(yx * yx + yy * yy + yz * yz) };
03060
03061     // y 軸の長さをチェック
03062     if (fabs(y) < std::numeric_limits<float>::epsilon()) return *this;
03063
03064     (*this)[ 0] = xx / x;
03065     (*this)[ 1] = yx / y;
03066     (*this)[ 2] = zx / z;
03067     (*this)[ 3] = 0.0f;
03068
03069     (*this)[ 4] = xy / x;
03070     (*this)[ 5] = yy / y;
03071     (*this)[ 6] = zy / z;
03072     (*this)[ 7] = 0.0f;
03073
03074     (*this)[ 8] = xz / x;
03075     (*this)[ 9] = yz / y;
03076     (*this)[10] = zz / z;
03077     (*this)[11] = 0.0f;
03078
03079     (*this)[12] = -(ex * (*this)[ 0] + ey * (*this)[ 4] + ez * (*this)[ 8]);
03080     (*this)[13] = -(ex * (*this)[ 1] + ey * (*this)[ 5] + ez * (*this)[ 9]);
03081     (*this)[14] = -(ex * (*this)[ 2] + ey * (*this)[ 6] + ez * (*this)[10]);
03082     (*this)[15] = 1.0f;
03083
03084     return *this;
03085 }
03086
03087 //
03088 // 変換行列：平行投影変換行列を設定する
03089 //
03090 gg::GgMatrix& gg::GgMatrix::loadOrthogonal(
03091     GLfloat left, GLfloat right,

```

```

03092 GLfloat bottom, GLfloat top,
03093 GLfloat zNear, GLfloat zFar
03094 )
03095 {
03096     const auto dx{ right - left };
03097     const auto dy{ top - bottom };
03098     const auto dz{ zFar - zNear };
03099
03100     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03101
03102     *this =
03103     {
03104         2.0f / dx,
03105         0.0f,
03106         0.0f,
03107         0.0f,
03108
03109         0.0f,
03110         2.0f / dy,
03111         0.0f,
03112         0.0f,
03113
03114         0.0f,
03115         0.0f,
03116         -2.0f / dz,
03117         0.0f,
03118
03119         -(right + left) / dx,
03120         -(top + bottom) / dy,
03121         -(zFar + zNear) / dz,
03122         1.0f
03123     };
03124
03125     return *this;
03126 }
03127
03128 //
03129 // 変換行列：透視投影変換行列を設定する
03130 //
03131 gg::GgMatrix& gg::GgMatrix::loadFrustum(
03132     GLfloat left, GLfloat right,
03133     GLfloat bottom, GLfloat top,
03134     GLfloat zNear, GLfloat zFar
03135 )
03136 {
03137     const auto dx{ right - left };
03138     const auto dy{ top - bottom };
03139     const auto dz{ zFar - zNear };
03140
03141     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03142
03143     *this =
03144     {
03145         2.0f * zNear / dx,
03146         0.0f,
03147         0.0f,
03148         0.0f,
03149
03150         0.0f,
03151         2.0f * zNear / dy,
03152         0.0f,
03153         0.0f,
03154
03155         (right + left) / dx,
03156         (top + bottom) / dy,
03157         -(zFar + zNear) / dz,
03158         -1.0f,
03159
03160         0.0f,
03161         0.0f,
03162         -2.0f * zFar * zNear / dz,
03163         0.0f
03164     };
03165
03166     return *this;
03167 }
03168
03169 //
03170 // 変換行列：画角から透視投影変換行列を設定する
03171 //
03172 gg::GgMatrix& gg::GgMatrix::loadPerspective(
03173     GLfloat fovy, GLfloat aspect,
03174     GLfloat zNear, GLfloat zFar
03175 )
03176 {
03177     const auto dz{ zFar - zNear };
03178

```

```

03179     if (dz == 0.0f) return *this;
03180
03181     const auto f{ 1.0f / tanf(fovy * 0.5f) };
03182
03183     *this =
03184     {
03185         f / aspect,
03186         0.0f,
03187         0.0f,
03188         0.0f,
03189
03190         0.0f,
03191         f,
03192         0.0f,
03193         0.0f,
03194
03195         0.0f,
03196         0.0f,
03197         -(zFar + zNear) / dz,
03198         -1.0f,
03199
03200         0.0f,
03201         0.0f,
03202         -2.0f * zFar * zNear / dz,
03203         0.0f
03204     };
03205
03206     return *this;
03207 }
03208
03209 //
03210 // 四元数: GgQuaternion 型の四元数 p, q の積を r に求める
03211 //
03212 void gg::GgQuaternion::multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const
03213 {
03214     r[0] = p[1] * q[2] - p[2] * q[1] + p[0] * q[3] + p[3] * q[0];
03215     r[1] = p[2] * q[0] - p[0] * q[2] + p[1] * q[3] + p[3] * q[1];
03216     r[2] = p[0] * q[1] - p[1] * q[0] + p[2] * q[3] + p[3] * q[2];
03217     r[3] = p[3] * q[3] - p[0] * q[0] - p[1] * q[1] - p[2] * q[2];
03218 }
03219
03220 //
03221 // 四元数: GgQuaternion 型の四元数 q が表す変換行列を m に求める
03222 //
03223 void gg::GgQuaternion::toMatrix(GLfloat* m, const GLfloat* q) const
03224 {
03225     const auto xx{ q[0] * q[0] * 2.0f };
03226     const auto yy{ q[1] * q[1] * 2.0f };
03227     const auto zz{ q[2] * q[2] * 2.0f };
03228     const auto xy{ q[0] * q[1] * 2.0f };
03229     const auto yz{ q[1] * q[2] * 2.0f };
03230     const auto zx{ q[2] * q[0] * 2.0f };
03231     const auto xw{ q[0] * q[3] * 2.0f };
03232     const auto yw{ q[1] * q[3] * 2.0f };
03233     const auto zw{ q[2] * q[3] * 2.0f };
03234
03235     m[ 0] = 1.0f - yy - zz;
03236     m[ 1] = xy + zw;
03237     m[ 2] = zx - yw;
03238     m[ 4] = xy - zw;
03239     m[ 5] = 1.0f - zz - xx;
03240     m[ 6] = yz + xw;
03241     m[ 8] = zx + yw;
03242     m[ 9] = yz - xw;
03243     m[10] = 1.0f - xx - yy;
03244     m[ 3] = m[ 7] = m[11] = m[12] = m[13] = m[14] = 0.0f;
03245     m[15] = 1.0f;
03246 }
03247
03248 //
03249 // 四元数: 回転変換行列 a が表す四元数を q に求める
03250 //
03251 void gg::GgQuaternion::toQuaternion(GLfloat* q, const GLfloat* a) const
03252 {
03253     const auto tr{ a[0] + a[5] + a[10] + a[15] };
03254
03255     if (tr > 0.0f)
03256     {
03257         q[3] = sqrtf(tr) * 0.5f;
03258         q[0] = (a[6] - a[9]) * 0.25f / q[3];
03259         q[1] = (a[8] - a[2]) * 0.25f / q[3];
03260         q[2] = (a[1] - a[4]) * 0.25f / q[3];
03261     }
03262 }
03263
03264 //
03265 // 四元数: 球面線形補間 p に q と r を t で補間した四元数を求める

```

```

03266 //
03267 void gg::GgQuaternion::slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const
03268 {
03269     auto qr{ ggDot4(q, r) };
03270     GLfloat s[4]{ r[0], r[1], r[2], r[3] };
03271
03272     // 内積が負なら最短経路を通るように符号を反転する
03273     if (qr < 0.0f)
03274     {
03275         qr = -qr;
03276         s[0] = -s[0];
03277         s[1] = -s[1];
03278         s[2] = -s[2];
03279         s[3] = -s[3];
03280     }
03281
03282     // 浮動小数点誤差で 1.0f を超える場合をクランプ
03283     if (qr > 1.0f) qr = 1.0f;
03284
03285     const auto ss{ 1.0f - qr * qr };
03286
03287     if (ss <= 0.0f)
03288     {
03289         if (p != q)
03290         {
03291             p[0] = q[0];
03292             p[1] = q[1];
03293             p[2] = q[2];
03294             p[3] = q[3];
03295         }
03296     }
03297     else
03298     {
03299         const auto sp{ sqrtf(ss) };
03300         const auto ph{ acosf(qr) };
03301         const auto pt{ ph * t };
03302         const auto t1{ sinf(pt) / sp };
03303         const auto t0{ sinf(ph - pt) / sp };
03304
03305         p[0] = q[0] * t0 + s[0] * t1;
03306         p[1] = q[1] * t0 + s[1] * t1;
03307         p[2] = q[2] * t0 + s[2] * t1;
03308         p[3] = q[3] * t0 + s[3] * t1;
03309     }
03310 }
03311
03312 //
03313 // 四元数 (x, y, z) を軸とし角度 a 回転する四元数を求める
03314 //
03315 gg::GgQuaternion& gg::GgQuaternion::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03316 {
03317     const auto l{ x * x + y * y + z * z };
03318     const auto w{ cosf(a * 0.5f) };
03319
03320     if (fabs(l) > std::numeric_limits<float>::epsilon())
03321     {
03322         const auto s{ sinf(a) / sqrtf(l) };
03323
03324         *this = GgQuaternion{ x * s, y * s, z * s, w };
03325     }
03326     else
03327     {
03328         *this = GgQuaternion{ 0.0f, 0.0f, 0.0f, w };
03329     }
03330
03331     return *this;
03332 }
03333
03334 //
03335 // x 軸中心に角度 a 回転する四元数を格納する
03336 //
03337 gg::GgQuaternion& gg::GgQuaternion::loadRotateX(GLfloat a)
03338 {
03339     const auto t{ a * 0.5f };
03340
03341     *this = GgQuaternion{ sinf(t), 0.0f, 0.0f, cosf(t) };
03342
03343     return *this;
03344 }
03345
03346 //
03347 // y 軸中心に角度 a 回転する四元数を格納する
03348 //
03349 gg::GgQuaternion& gg::GgQuaternion::loadRotateY(GLfloat a)
03350 {
03351     const auto t{ a * 0.5f };
03352

```

```

03353     *this = GgQuaternion{ 0.0f, sinf(t), 0.0f, cosf(t) };
03354
03355     return *this;
03356 }
03357
03358 //
03359 // z 軸中心に角度 a 回転する四元数を格納する
03360 //
03361 gg::GgQuaternion& gg::GgQuaternion::loadRotateZ(GLfloat a)
03362 {
03363     const auto t{ a * 0.5f };
03364
03365     *this = GgQuaternion{ 0.0f, 0.0f, sinf(t), cosf(t) };
03366
03367     return *this;
03368 }
03369
03370 //
03371 // 四元数：オイラー角 (heading, pitch, roll) にもとづいて四元数を求める
03372 //
03373 gg::GgQuaternion& gg::GgQuaternion::loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll)
03374 {
03375     GgQuaternion h, p, r;
03376
03377     h.loadRotateY(heading);
03378     p.loadRotateX(pitch);
03379     r.loadRotateZ(roll);
03380
03381     *this = h * p * r;
03382
03383     return *this;
03384 }
03385
03386 //
03387 // 四元数：正規化して格納する
03388 //
03389 gg::GgQuaternion& gg::GgQuaternion::loadNormalize(const GLfloat* a)
03390 {
03391     *this = a;
03392
03393     ggNormalize4(data());
03394
03395     return *this;
03396 }
03397
03398 //
03399 // 四元数：共役四元数を格納する
03400 //
03401 gg::GgQuaternion& gg::GgQuaternion::loadConjugate(const GLfloat* a)
03402 {
03403     // w 要素を反転する
03404     data()[0] = a[0];
03405     data()[1] = a[1];
03406     data()[2] = a[2];
03407     data()[3] = -a[3];
03408
03409     return *this;
03410 }
03411
03412 //
03413 // 四元数：逆元を格納する
03414 //
03415 gg::GgQuaternion& gg::GgQuaternion::loadInvert(const GLfloat* a)
03416 {
03417     // ノルムの二乗を求める
03418     const auto l(ggDot4(a, a));
03419
03420     if (l > 0.0f)
03421     {
03422         // 共役四元数を求める
03423         GgQuaternion r;
03424         r.loadConjugate(a);
03425
03426         // ノルムの二乗で割る
03427         data()[0] = r[0] / l;
03428         data()[1] = r[1] / l;
03429         data()[2] = r[2] / l;
03430         data()[3] = r[3] / l;
03431     }
03432
03433     return *this;
03434 }
03435
03436 //
03437 // 簡易トラックボール処理：リセット
03438 //
03439 void gg::GgTrackball::reset(const GgQuaternion& q)

```



```

03440 {
03441     // ドラッグ中ではない
03442     drag = false;
03443
03444     // 単位クォーターニオンに初期値を与える
03445     *this = cq = q;
03446
03447     // 回転行列を初期化する
03448     GgQuaternion::getMatrix(rt);
03449 }
03450
03451 //
03452 // 簡易トラックボール処理：トラックボールする領域の設定
03453 //
03454 //   Reshape コールバック (resize) の中で実行する
03455 //   (w, h) ウィンドウサイズ
03456 //
03457 void gg::GgTrackball::region(GLfloat w, GLfloat h)
03458 {
03459     // マウスポインタ位置のウィンドウ内の相対的位置への換算用
03460     scale[0] = 2.0f / w;
03461     scale[1] = 2.0f / h;
03462 }
03463
03464 //
03465 // 簡易トラックボール処理：ドラッグ開始時の処理
03466 //
03467 //   マウスボタンを押したときに実行する
03468 //   (x, y) 現在のマウス位置
03469 //
03470 void gg::GgTrackball::begin(GLfloat x, GLfloat y)
03471 {
03472     // ドラッグ開始
03473     drag = true;
03474
03475     // ドラッグ開始点を記録する
03476     start[0] = x;
03477     start[1] = y;
03478 }
03479
03480 //
03481 // 簡易トラックボール処理：ドラッグ中の処理
03482 //
03483 //   マウスのドラッグ中に実行する
03484 //   (x, y) 現在のマウス位置
03485 //
03486 void gg::GgTrackball::motion(GLfloat x, GLfloat y)
03487 {
03488     // ドラッグ中でなければ何もしない
03489     if (!drag) return;
03490
03491     // マウスポインタの位置のドラッグ開始位置からの変位
03492     const auto dx{ (x - start[0]) * scale[0] };
03493     const auto dy{ (y - start[1]) * scale[1] };
03494
03495     // マウスポインタの位置のドラッグ開始位置からの距離
03496     const auto a{ hypot(dx, dy) };
03497
03498     // マウスポインタの位置がドラッグ開始位置から移動していれば
03499     if (a > 0.0)
03500     {
03501         // 現在の回転の四元数に作った四元数を掛けて合成する
03502         *this = ggRotateQuaternion(dy, dx, 0.0f, a * 3.1415926536f) * cq;
03503
03504         // 合成した四元数から回転の変換行列を求める
03505         GgQuaternion::getMatrix(rt);
03506     }
03507 }
03508
03509 //
03510 // 簡易トラックボール処理：回転角の修正
03511 //
03512 //   現在の回転角を修正する
03513 //   q 修正分の回転角を表す四元数
03514 //
03515 void gg::GgTrackball::rotate(const GgQuaternion& q)
03516 {
03517     // ドラッグ中なら何もしない
03518     if (drag) return;
03519
03520     // 保存されている四元数に修正分の四元数を掛けて合成する
03521     *this = q * cq;
03522
03523     // 合成した四元数から回転の変換行列を求める
03524     GgQuaternion::getMatrix(rt);
03525
03526     // 誤差を吸収するために正規化して保存する

```

```

03527     cq = normalize();
03528 }
03529
03530 //
03531 // 簡易トラックボール処理：停止時の処理
03532 //
03533 // マウスボタンを離れたときに実行する
03534 // (x, y) 現在のマウス位置
03535 //
03536 void gg::GgTrackball::end(GLfloat x, GLfloat y)
03537 {
03538     // ドラッグ終了点における回転を求める
03539     motion(x, y);
03540
03541     // 誤差を吸収するために正規化して保存する
03542     cq = normalize();
03543
03544     // ドラッグ終了
03545     drag = false;
03546 }
03547
03548 //
03549 // 配列に格納された画像の内容を TGA ファイルに保存する
03550 //
03551 // name ファイル名
03552 // buffer 画像データ
03553 // width 画像の横の画素数
03554 // height 画像の縦の画素数
03555 // depth 画像の 1 画素のバイト数
03556 // 戻り値 保存に成功すれば true, 失敗すれば false
03557 //
03558 bool gg::ggSaveTga(
03559     const std::string& name,
03560     const void* buffer,
03561     unsigned int width,
03562     unsigned int height,
03563     unsigned int depth
03564 )
03565 {
03566     // ファイルを開く
03567     std::ofstream file{ Utf8ToTChar(name), std::ios::binary };
03568
03569     // ファイルが開けなかったら戻る
03570     if (file.fail()) return false;
03571
03572     // 画像のヘッダ
03573     const unsigned char type{ static_cast<unsigned char>(depth == 0 ? 0 : depth < 3 ? 3 : 2) };
03574     const unsigned char alpha{ static_cast<unsigned char>(depth == 2 || depth == 4 ? 8 : 0) };
03575     const unsigned char header[18]
03576     {
03577         0,           // ID length
03578         0,           // Color map type (none)
03579         type,        // Image Type (2:RGB, 3:Grayscale)
03580         0, 0,        // Offset into the color map table
03581         0, 0,        // Number of color map entries
03582         0,           // Number of a color map entry bits per pixel
03583         0, 0,        // Horizontal image position
03584         0, 0,        // Vertical image position
03585         static_cast<unsigned char>(width & 0xff),
03586         static_cast<unsigned char>(width >> 8),
03587         static_cast<unsigned char>(height & 0xff),
03588         static_cast<unsigned char>(height >> 8),
03589         static_cast<unsigned char>(depth * 8),
03590         alpha        // Image descriptor
03591     };
03592
03593     // ヘッダを書き込む
03594     file.write(reinterpret_cast<const char*>(header), sizeof header);
03595
03596     // ヘッダの書き込みに失敗したら戻る
03597     if (file.bad()) return false;
03598
03599     // データを書き込む
03600     const unsigned int size{ width * height * depth };
03601     if (type == 2)
03602     {
03603         // フルカラー
03604         std::vector<char> temp(size);
03605         for (GLuint i = 0; i < size; i += depth)
03606         {
03607             temp[i + 2] = static_cast<const char*>(buffer)[i + 0];
03608             temp[i + 1] = static_cast<const char*>(buffer)[i + 1];
03609             temp[i + 0] = static_cast<const char*>(buffer)[i + 2];
03610             if (depth == 4) temp[i + 3] = static_cast<const char*>(buffer)[i + 3];
03611         }
03612         file.write(temp.data(), size);
03613     }

```

```

03614     else if (type == 3)
03615     {
03616         // グレースケール
03617         file.write(static_cast<const char*>(buffer), size);
03618     }
03619
03620     // フッタを書き込む
03621     constexpr char footer[] = "\\0\\0\\0\\0\\0\\0\\0TRUEVISION-XFILE.";
03622     file.write(footer, sizeof footer);
03623
03624     // データの書き込みに失敗していなければ true を返す
03625     return !file.bad();
03626 }
03627
03628 //
03629 // カラーバッファの内容を TGA ファイルに保存する
03630 //
03631 //     name 保存するファイル名
03632 //     戻り値 保存に成功すれば true, 失敗すれば false
03633 //
03634 bool gg::ggSaveColor(const std::string& name)
03635 {
03636     // 現在のビューポートのサイズを得る
03637     GLint viewport[4];
03638     glGetIntegerv(GL_VIEWPORT, viewport);
03639
03640     // ビューポートのサイズ分のメモリを確保する
03641     std::vector<GLubyte> buffer(viewport[2] * viewport[3] * 3);
03642
03643     // 画面表示の完了を待つ
03644     glFinish();
03645
03646     // カラーバッファを読み込む
03647     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03648                 GL_RGB, GL_UNSIGNED_BYTE, buffer.data());
03649
03650     // 読み込んだデータをファイルに書き込む
03651     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 3);
03652 }
03653
03654 //
03655 // デプスバッファの内容を TGA ファイルに保存する
03656 //
03657 //     name 保存するファイル名
03658 //     戻り値 保存に成功すれば true, 失敗すれば false
03659 //
03660 bool gg::ggSaveDepth(const std::string& name)
03661 {
03662     // 現在のビューポートのサイズを得る
03663     GLint viewport[4];
03664     glGetIntegerv(GL_VIEWPORT, viewport);
03665
03666     // ビューポートのサイズ分のメモリを確保する
03667     std::vector<GLubyte> buffer(viewport[2] * viewport[3]);
03668
03669     // 画面表示の完了を待つ
03670     glFinish();
03671
03672     // デプスバッファを読み込む
03673     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03674                 GL_DEPTH_COMPONENT, GL_UNSIGNED_BYTE, buffer.data());
03675
03676     // 読み込んだデータをファイルに書き込む
03677     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 1);
03678 }
03679
03680 //
03681 // TGA ファイル (8/16/24/32bit) を読み込む
03682 //
03683 //     name 読み込むファイル名
03684 //     pWidth 読み込んだファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03685 //     pHeight 読み込んだファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03686 //     pFormat 読み込んだファイルのフォーマットの格納先のポインタ (nullptr なら格納しない)
03687 //     image 読み込んだ画像を格納する vector
03688 //     戻り値 読み込みに成功すれば true, 失敗すれば false
03689 //
03690 bool gg::ggReadImage(
03691     const std::string& name,
03692     std::vector<GLubyte>& image,
03693     GLsizei* pWidth,
03694     GLsizei* pHeight,
03695     GLenum* pFormat
03696 )
03697 {
03698     // ファイルを開く
03699     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
03700

```

```

03701 // ファイルが開けなかったら戻る
03702 if (file.fail()) return false;
03703
03704 // ヘッダを読み込む
03705 unsigned char header[18];
03706 file.read(reinterpret_cast<char*>(header), sizeof header);
03707
03708 // ヘッダの読み込みに失敗したら戻る
03709 if (file.bad()) return false;
03710
03711 // 深度
03712 const auto depth{ header[16] / 8 };
03713 GLenum format;
03714 switch (depth)
03715 {
03716 case 1:
03717     format = GL_RED;
03718     break;
03719 case 2:
03720     format = GL_RG;
03721     break;
03722 case 3:
03723 #if defined(GL_GLES_PROTOTYPES)
03724     format = GL_RGB;
03725 #else
03726     format = GL_BGR;
03727 #endif
03728     break;
03729 case 4:
03730 #if defined(GL_GLES_PROTOTYPES)
03731     format = GL_RGBA;
03732 #else
03733     format = GL_BGRA;
03734 #endif
03735     break;
03736 default:
03737     // 取り扱えないフォーマットだったら戻る
03738     return false;
03739 }
03740
03741 // フォーマットを格納する
03742 if (pFormat) *pFormat = format;
03743
03744 // 画像の縦横の画素数
03745 const auto width{ static_cast<GLsizei>(header[13] << 8 | header[12]) };
03746 if (pWidth) *pWidth = width;
03747 const auto height{ static_cast<GLsizei>(header[15] << 8 | header[14]) };
03748 if (pHeight) *pHeight = height;
03749
03750 // データサイズ
03751 const auto size{ width * height * depth };
03752
03753 // サイズが小さすぎたら戻る
03754 if (size < 2) return false;
03755
03756 // 読み込みに使うメモリを確保する
03757 image.resize(size);
03758
03759 // データを読み込む
03760 if (header[2] & 8)
03761 {
03762     // RLE
03763     int p{ 0 };
03764     char c;
03765     while (file.get(c))
03766     {
03767         if (c & 0x80)
03768         {
03769             // run-length packet
03770             const auto count{ (c & 0x7f) + 1 };
03771             if (p + depth * count > size) break;
03772             char temp[4];
03773             file.read(temp, depth);
03774             for (int i = 0; i < count; ++i)
03775             {
03776                 for (int j = 0; j < depth; j++) image[p++] = temp[j];
03777             }
03778         }
03779         else
03780         {
03781             // raw packet
03782             const auto count{ (c + 1) * depth };
03783             if (p + count > size) break;
03784             file.read(reinterpret_cast<char*>(image.data() + p), count);
03785             p += count;
03786         }
03787     }
}

```

```

03788     }
03789     else
03790     {
03791         // 非圧縮
03792         file.read(reinterpret_cast<char*>(image.data()), size);
03793     }
03794
03795     #if defined(GL_GLES_PROTOTYPES)
03796     // OpenGL ES では GL_BGR / GL_BGRA が使えないため R と B を入れ替える
03797     if (!file.bad() && depth >= 3)
03798     {
03799         for (auto* p = image.data(); p < image.data() + size; p += depth)
03800         {
03801             std::swap(p[0], p[2]);
03802         }
03803     }
03804     #endif
03805
03806     // 読み込みに失敗していなければ true を返す
03807     return !file.bad();
03808 }
03809
03810 //
03811 // テクスチャを作成して画像を読み込む
03812 //
03813 // image 画像データ, nullptr ならメモリの確保だけを行う
03814 // width 画像の横の画素数
03815 // height 画像の縦の画素数
03816 // format 画像データのフォーマット
03817 // type 画像のデータ型
03818 // internal テクスチャの内部フォーマット
03819 // wrap テクスチャのラッピングモード
03820 // swizzle true ならテクスチャの赤と青を入れ替える
03821 // 戻り値 テクスチャ名
03822 //
03823 GLuint gg::ggLoadTexture(
03824     const GLvoid* image,
03825     GLsizei width,
03826     GLsizei height,
03827     GLenum format,
03828     GLenum type,
03829     GLenum internal,
03830     GLenum wrap,
03831     bool swizzle
03832 )
03833 {
03834     // テクスチャを作成する
03835     GLuint texture;
03836     glGenTextures(1, &texture);
03837
03838     // 作成したテクスチャを 2D テクスチャとしてバインドする
03839     glBindTexture(GL_TEXTURE_2D, texture);
03840
03841     // アルファチャンネルがついていれば 4 バイト境界に設定する
03842     #if defined(GL_GLES_PROTOTYPES)
03843     glPixelStorei(GL_UNPACK_ALIGNMENT, format == GL_RGBA ? 4 : 1);
03844     #else
03845     glPixelStorei(GL_UNPACK_ALIGNMENT, format == GL_RGBA || format == GL_BGRA ? 4 : 1);
03846     #endif
03847
03848     // テクスチャを割り当てる
03849     glTexImage2D(GL_TEXTURE_2D, 0, internal, width, height, 0, format, type, image);
03850
03851     // バイリニア (ミップマップなし)、エッジでクランプ
03852     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
03853     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
03854     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, wrap);
03855     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, wrap);
03856
03857     if (swizzle)
03858     {
03859         // テクスチャのサンプリング時に赤と青を入れ替える
03860         glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, GL_BLUE);
03861         glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, GL_RED);
03862     }
03863
03864     // テクスチャ名を返す
03865     return texture;
03866 }
03867
03868 //
03869 // テクスチャを作成して TGA フォーマットの画像ファイルを読み込む
03870 //
03871 // name TGA ファイル名
03872 // pWidth 読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03873 // pHeight 読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03874 // internal テクスチャの内部フォーマット

```

```

03875 // wrap テクスチャのラッピングモード
03876 // 戻り値 テクスチャ名
03877 //
03878 GLuint gg::ggLoadImage(
03879     const std::string& name,
03880     GLsizei* pWidth,
03881     GLsizei* pHeight,
03882     GLenum internal,
03883     GLenum wrap
03884 )
03885 {
03886     // 画像データ
03887     std::vector<GLubyte> image;
03888
03889     // 画像サイズ
03890     GLsizei width, height;
03891
03892     // 画像フォーマット
03893     GLenum format;
03894
03895     // 画像を読み込む
03896     ggReadImage(name, image, &width, &height, &format);
03897
03898     // 画像が読み込めなかったら戻る
03899     if (image.empty()) return 0;
03900
03901     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
03902     if (internal == 0) internal = format;
03903
03904     // テクスチャに読み込む (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA、OpenGL ES では GL_RGB / GL_RGBA)
03905     const auto tex{ ggLoadTexture(image.data(), width, height,
03906         format, GL_UNSIGNED_BYTE, internal, wrap, false) };
03907
03908     // 画像サイズを返す
03909     if (pWidth) *pWidth = width;
03910     if (pHeight) *pHeight = height;
03911
03912     // テクスチャ名を返す
03913     return tex;
03914 }
03915
03916 //
03917 // グレースケール画像 (8bit) から法線マップのデータを作成する
03918 //
03919 // width 高さマップのグレースケール画像 hmap の横の画素数
03920 // height 高さマップのグレースケール画像のデータ hmap の縦の画素数
03921 // stride データの間隔
03922 // hmap グレースケール画像のデータ
03923 // nz 法線の z 成分の割合
03924 // internal テクスチャの内部フォーマット
03925 // nmap 法線マップを格納する vector
03926 //
03927 void gg::ggCreateNormalMap(
03928     const GLubyte* hmap,
03929     GLsizei width,
03930     GLsizei height,
03931     GLenum format,
03932     GLfloat nz,
03933     GLenum internal,
03934     std::vector<GgVector>& nmap
03935 )
03936 {
03937     // メモリサイズ
03938     const GLsizei size{ width * height };
03939
03940     // 法線マップのメモリを確保する
03941     nmap.resize(size);
03942
03943     // 画素のバイト数
03944     GLint stride;
03945     switch (format)
03946     {
03947     case GL_RED:
03948         stride = 1;
03949         break;
03950     case GL_RG:
03951         stride = 2;
03952         break;
03953     case GL_RGB:
03954         stride = 3;
03955         break;
03956     case GL_RGBA:
03957         stride = 4;
03958         break;
03959     default:
03960         stride = 1;
03961         break;

```

```

03962     }
03963
03964     // 法線マップの作成
03965     for (GLsizei i = 0; i < size; ++i)
03966     {
03967         const auto x{ i % width };
03968         const auto y{ i - x };
03969         const auto u0{ (y + (x - 1 + width) % width) * stride };
03970         const auto u1{ (y + (x + 1) % width) * stride };
03971         const auto v0{ ((y - width + size) % size + x) * stride };
03972         const auto v1{ ((y + width) % size + x) * stride };
03973
03974         // 隣接する画素との値の差を法線の成分に用いる
03975         nmap[i][0] = static_cast<GLfloat>(hmap[u0] - hmap[u1]);
03976         nmap[i][1] = static_cast<GLfloat>(hmap[v0] - hmap[v1]);
03977         nmap[i][2] = nz;
03978         nmap[i][3] = hmap[i * stride];
03979
03980         // 法線ベクトルを正規化する
03981         ggNormalize3(&nmap[i]);
03982     }
03983
03984     // 内部フォーマットが浮動小数点テクスチャでなければ [0,1] に正規化する
03985     if (
03986         internal != GL_RGB16F &&
03987         internal != GL_RGBA16F &&
03988         internal != GL_RGB32F &&
03989         internal != GL_RGBA32F
03990     )
03991     {
03992         for (GLsizei i = 0; i < size; ++i)
03993         {
03994             nmap[i][0] = nmap[i][0] * 0.5f + 0.5f;
03995             nmap[i][1] = nmap[i][1] * 0.5f + 0.5f;
03996             nmap[i][2] = nmap[i][2] * 0.5f + 0.5f;
03997             nmap[i][3] *= 0.0039215686f; // == 1/255
03998         }
03999     }
04000 }
04001
04002 //
04003 // TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する
04004 //
04005 //     name TGA ファイル名
04006 //     nz 作成した法線の z 成分の割合
04007 //     pWidth 読み込んだ画像の横の画素数の格納先のポインタ (nullptr なら格納しない)
04008 //     pHeight 読み込んだ画像の縦の画素数の格納先のポインタ (nullptr なら格納しない)
04009 //     internal テクスチャの内部フォーマット
04010 //     戻り値 テクスチャ名
04011 //
04012 GLuint gg::ggLoadHeight(
04013     const std::string& name,
04014     GLfloat nz,
04015     GLsizei* pWidth,
04016     GLsizei* pHeight,
04017     GLenum internal
04018 )
04019 {
04020     // 画像データ
04021     std::vector<GLubyte> hmap;
04022
04023     // 画像サイズ
04024     GLsizei width, height;
04025
04026     // 画像フォーマット
04027     GLenum format;
04028
04029     // 高さマップの画像を読み込む
04030     ggReadImage(name, hmap, &width, &height, &format);
04031
04032     // 画像が読み込めなかったら戻る
04033     if (hmap.empty()) return 0;
04034
04035     // 法線マップ
04036     std::vector<GgVector> nmap;
04037
04038     // 法線マップを作成する
04039     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04040
04041     // 画像サイズを返す
04042     if (pWidth) *pWidth = width;
04043     if (pHeight) *pHeight = height;
04044
04045     // テクスチャを作成して返す
04046     return ggLoadTexture(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal, GL_REPEAT);
04047 }
04048

```

```

04049 //
04050 // テクスチャの赤と青を交換する
04051 //
04052 void gg::GgTexture::swapRandB(bool swap) const
04053 {
04054     const auto swizzle
04055     {
04056         swap ?
04057         std::pair<GLint, GLint>{ GL_BLUE, GL_RED } :
04058         std::pair<GLint, GLint>{ GL_RED, GL_BLUE }
04059     };
04060
04061     glBindTexture(GL_TEXTURE_2D, texture);
04062     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, swizzle.first);
04063     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, swizzle.second);
04064     glBindTexture(GL_TEXTURE_2D, 0);
04065 }
04066
04067 //
04068 // TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する
04069 //
04070 // name 読み込むファイル名
04071 // internal glTexImage2D() に指定するテクスチャの内部フォーマット. 0 なら外部フォーマットに合わせる
04072 // 戻り値 テクスチャの作成に成功すれば true, 失敗すれば false
04073 //
04074 void gg::GgColorTexture::load(
04075     const std::string& name,
04076     GLenum internal,
04077     GLenum wrap
04078 )
04079 {
04080     // 画像データ
04081     std::vector<GLubyte> image;
04082
04083     // 画像サイズ
04084     GLsizei width, height;
04085
04086     // 画像フォーマット
04087     GLenum format;
04088
04089     // 画像を読み込む
04090     ggReadImage(name, image, &width, &height, &format);
04091
04092     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
04093     if (internal == 0) internal = format;
04094
04095     // テクスチャを作成する (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA、OpenGL ES では GL_RGB / GL_RGBA)
04096     texture = std::make_shared<GgTexture>(image.data(), width, height,
04097         format, GL_UNSIGNED_BYTE, internal, wrap, false);
04098 }
04099
04100 //
04101 // TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する
04102 //
04103 // name 画像ファイル名
04104 // width テクスチャとして用いる画像データの横幅
04105 // height テクスチャとして用いる画像データの高さ
04106 // format テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA)
04107 // nz 法線マップの z 成分の値
04108 // internal テクスチャの内部フォーマット
04109 //
04110 void gg::GgNormalTexture::load(
04111     const std::string& name,
04112     GLfloat nz,
04113     GLenum internal
04114 )
04115 {
04116     // 高さマップ
04117     std::vector<GLubyte> hmap;
04118
04119     // 画像サイズ
04120     GLsizei width, height;
04121
04122     // 画像フォーマット
04123     GLenum format;
04124
04125     // 高さマップの画像を読み込む
04126     ggReadImage(name, hmap, &width, &height, &format);
04127
04128     // 法線マップ
04129     std::vector<GgVector> nmap;
04130
04131     // 法線マップを作成する
04132     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04133
04134     // テクスチャを作成する
04135     texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,

```



```

        GL_REPEAT);
04136 }
04137
04139
04140 //
04141 // OBJ ファイルの読み込みに使うデータ型と関数
04142 //
04143 namespace gg
04144 {
04145     // GLfloat 型の 2 要素のベクトル
04146     using vec2 = std::array<GLfloat, 2>;
04147
04148     // GLfloat 型の 3 要素のベクトル
04149     using vec3 = std::array<GLfloat, 3>;
04150
04151     // 三角形データ
04152     struct fidx
04153     {
04154         GLuint p[3];          // 頂点座標番号
04155         GLuint n[3];          // 頂点法線番号
04156         GLuint t[3];          // テクスチャ座標番号
04157         bool smooth;          // スムーズシェーディングの有無
04158     };
04159
04160     // ポリゴングループ
04161     struct fgrp
04162     {
04163         GLuint nextgroup;     // 次のポリゴングループの最初の三角形番号
04164         GLuint mtlno;         // このポリゴングループの材質番号
04165
04166         // コンストラクタ
04167         fgrp(GLuint nextgroup, GLuint mtlno) :
04168             nextgroup(nextgroup),
04169             mtlno(mtlno)
04170         {
04171         }
04172     };
04173
04174     // デフォルトの材質
04175     constexpr GgSimpleShader::Material defaultMaterial
04176     {
04177         { 0.1f, 0.1f, 0.1f, 1.0f },
04178         { 0.6f, 0.6f, 0.6f, 0.0f },
04179         { 0.3f, 0.3f, 0.3f, 0.0f },
04180         60.0f
04181     };
04182
04183     // デフォルトの材質名
04184     constexpr char defaultMaterialName[] = "_default_";
04185
04186     //
04187     // Alias OBJ 形式の MTL ファイルを読み込む
04188     //
04189     // mtlpath MTL ファイルのパス名
04190     // mtl 読み込んだ材質名をキー、材質番号を値にした map
04191     // material 材質データ
04192     //
04193     static bool ggLoadMtl(const std::string& mtlpath,
04194         std::map<std::string, GLuint>& mtl,
04195         std::vector<GgSimpleShader::Material>& material)
04196     {
04197         // MTL ファイルが無ければ戻る
04198         std::ifstream mtlfile{ Utf8ToTChar(mtlpath), std::ios::binary };
04199         if (!mtlfile)
04200         {
04201             #if defined(DEBUG)
04202                 std::cerr << "Warning: Can't open MTL file: " << mtlpath << std::endl;
04203             #endif
04204             return false;
04205         }
04206
04207         // 一行読み込み用のバッファ
04208         std::string mtlline;
04209
04210         // 材質名（ループの外に置く）
04211         std::string mtlname{ defaultMaterialName };
04212
04213         // 現在の材質番号を登録する
04214         mtl[mtlname] = static_cast<GLuint>(material.size());
04215
04216         // 現在の材質にデフォルトの材質を設定する
04217         material.emplace_back(defaultMaterial);
04218
04219         // 材質データを読み込む
04220         while (std::getline(mtlfile, mtlline))
04221         {
04222             // 空行は読み飛ばす

```

```

04223     if (mtlline == "") continue;
04224
04225     // 最後の文字が '\r' なら
04226     if (*(mtlline.end() - 1) == '\r')
04227     {
04228         // 最後の文字を削除する
04229         mtlline.erase(mtlline.end() - 1, mtlline.end());
04230
04231         // 空行になったら読み飛ばす
04232         if (mtlline == "") continue;
04233     }
04234
04235     // 読み込んだ行を文字列ストリームにする
04236     std::istringstream mtlstr{ mtlline };
04237
04238     // オペレータ
04239     std::string mtlop;
04240
04241     // 文字列ストリームから材質パラメータの種類を取り出す
04242     mtlstr >> mtlop;
04243
04244     // '#' で始まる場合はコメントとして行末まで読み飛ばす
04245     if (mtlop[0] == '#') continue;
04246
04247     if (mtlop == "newmtl")
04248     {
04249         // 新規作成する材質名を取り出す
04250         mtlstr >> mtlname;
04251
04252         // 材質名が既に存在するかどうか調べる
04253         const auto m{ mtl.find(mtlname) };
04254         if (m == mtl.end())
04255         {
04256             // 存在しないので新規作成する材質の番号をその材質名に割り当てる
04257             mtl[mtlname] = static_cast<GLuint>(material.size());
04258
04259             // 新規作成する材質にデフォルトの材質を設定しておく
04260             material.emplace_back(defaultMaterial);
04261         }
04262
04263         #if defined(DEBUG)
04264             std::cerr << "newmtl: " << mtlname << std::endl;
04265         #endif
04266     }
04267     else if (mtlop == "Ka")
04268     {
04269         // 環境光の反射係数を登録する
04270         mtlstr
04271             >> material.back().ambient[0]
04272             >> material.back().ambient[1]
04273             >> material.back().ambient[2];
04274     }
04275     else if (mtlop == "Kd")
04276     {
04277         // 拡散反射係数を登録する
04278         mtlstr
04279             >> material.back().diffuse[0]
04280             >> material.back().diffuse[1]
04281             >> material.back().diffuse[2];
04282     }
04283     else if (mtlop == "Ks")
04284     {
04285         // 鏡面反射係数を登録する
04286         mtlstr
04287             >> material.back().specular[0]
04288             >> material.back().specular[1]
04289             >> material.back().specular[2];
04290     }
04291     else if (mtlop == "Ns")
04292     {
04293         // 輝き係数を登録する
04294         GLfloat shininess;
04295         mtlstr >> shininess;
04296         material.back().shininess = shininess * 0.1f;
04297     }
04298     else if (mtlop == "d")
04299     {
04300         // 不透明度を登録する
04301         mtlstr >> material.back().ambient[3];
04302     }
04303 }
04304
04305 // MTL ファイルの読み込みに失敗したら戻る
04306 if (mtlfile.bad())
04307 {
04308     #if defined(DEBUG)
04309         std::cerr << "Warning: Can't read MTL file: " << mtlpath << std::endl;

```

```

04310 #endif
04311     return false;
04312 }
04313
04314 // MTL ファイルの読み込みに成功した
04315 return true;
04316 }
04317
04318 //
04319 // Alias OBJ 形式のファイルを解析する
04320 //
04321 // name Alias OBJ 形式のファイルのファイル名
04322 // group 同じ材質を割り当てるポリゴングループ
04323 // mtl 読み込んだ材質名をキーにした map
04324 // pos 頂点の位置
04325 // norm 頂点の法線
04326 // tex 頂点のテクスチャ座標
04327 // face 三角形のデータ
04328 //
04329 static bool ggParseObj(
04330     const std::string& name,
04331     std::vector<fgrp>& group,
04332     std::vector<GgSimpleShader::Material>& material,
04333     std::vector<vec3>& pos,
04334     std::vector<vec3>& norm,
04335     std::vector<vec2>& tex,
04336     std::vector<fidx>& face,
04337     bool normalize
04338 )
04339 {
04340     // ファイルパスからディレクトリ名を取り出す
04341     const std::string path{ name };
04342     const size_t base{ path.find_last_of("/\\") };
04343     const std::string dirname{ (base == std::string::npos) ? "" : path.substr(0, base + 1) };
04344
04345     // OBJ ファイルを読み込む
04346     std::ifstream file{ Utf8ToTChar(path) };
04347
04348     // 読み込みに失敗したら戻る
04349     if (file.fail())
04350     {
04351         #if defined(DEBUG)
04352             std::cerr << "Error: Can't open OBJ file: " << path << std::endl;
04353         #endif
04354         return false;
04355     }
04356
04357     // ポリゴングループの最初の三角形番号
04358     GLsizei startgroup{ static_cast<GLsizei>(group.size()) };
04359
04360     // スムーズシェーディングのスイッチ
04361     bool smooth{ false };
04362
04363     // 材質のテーブル
04364     std::map<std::string, GLuint> mtl;
04365
04366     // 現在の材質名 (ループの外で宣言する)
04367     std::string mtlname;
04368
04369     // 座標値の最小値・最大値
04370     vec3 bmin{ FLT_MAX }, bmax{ -FLT_MAX };
04371
04372     // 一行読み込み用のバッファ
04373     std::string line;
04374
04375     // データの読み込み
04376     while (std::getline(file, line))
04377     {
04378         // 空行は読み飛ばす
04379         if (line == "") continue;
04380
04381         // 最後の文字が '\r' なら
04382         if (*(line.end() - 1) == '\r')
04383         {
04384             // 最後の文字を削除する
04385             line.erase(line.end() - 1, line.end());
04386
04387             // 空行になったら読み飛ばす
04388             if (line == "") continue;
04389         }
04390
04391         // 一行を文字列ストリームに入れる
04392         std::istringstream str(line);
04393
04394         // 最初のトークンを命令 (op) とみなす
04395         std::string op;
04396         str >> op;

```

```

04397
04398     if (op[0] == '#') continue;
04399
04400     if (op == "v")
04401     {
04402         // 頂点位置
04403         vec3 v;
04404
04405         // 頂点位置はスペースで区切られている
04406         str >> v[0] >> v[1] >> v[2];
04407
04408         // 頂点位置を記録する
04409         pos.emplace_back(v);
04410
04411         // 頂点位置の最小値と最大値を求める (AABB)
04412         for (int i = 0; i < 3; ++i)
04413         {
04414             bmin[i] = std::min(bmin[i], v[i]);
04415             bmax[i] = std::max(bmax[i], v[i]);
04416         }
04417     }
04418     else if (op == "vt")
04419     {
04420         // テクスチャ座標
04421         vec2 t;
04422
04423         // 頂点位置はスペースで区切られている
04424         str >> t[0] >> t[1];
04425
04426         // テクスチャ座標を記録する
04427         tex.emplace_back(t);
04428     }
04429     else if (op == "vn")
04430     {
04431         // 頂点法線
04432         vec3 n;
04433
04434         // 頂点法線はスペースで区切られている
04435         str >> n[0] >> n[1] >> n[2];
04436
04437         // 頂点法線を記録する
04438         norm.emplace_back(n);
04439     }
04440     else if (op == "f")
04441     {
04442         // 三角形データ
04443         fidx f;
04444
04445         // スムースシェーディング
04446         f.smooth = smooth;
04447
04448         // 三頂点のそれぞれについて
04449         for (int i = 0; i < 3; ++i)
04450         {
04451             // 1項目取り出す
04452             std::string s;
04453             str >> s;
04454
04455             // 文字の位置
04456             auto c{ s.begin() };
04457
04458             // テクスチャ座標と法線の番号は未定義を表す 0 にしておく
04459             f.p[i] = f.t[i] = f.n[i] = 0;
04460
04461             // 項目の最初の要素は頂点座標番号
04462             while (c != s.end() && isdigit(*c)) f.p[i] = f.p[i] * 10 + *c++ - '0';
04463             if (c == s.end() || *c++ != '/') continue;
04464
04465             // 二つ目の項目はテクスチャ座標
04466             while (c != s.end() && isdigit(*c)) f.t[i] = f.t[i] * 10 + *c++ - '0';
04467             if (c == s.end() || *c++ != '/') continue;
04468
04469             // 三つ目の項目は法線番号
04470             while (c != s.end() && isdigit(*c)) f.n[i] = f.n[i] * 10 + *c++ - '0';
04471         }
04472
04473         // 三角形データを登録する
04474         face.emplace_back(f);
04475     }
04476     else if (op == "s")
04477     {
04478         // '1' だったらスムースシェーディング有効
04479         std::string s;
04480         str >> s;
04481         smooth = s == "1";
04482     }
04483     else if (op == "usemtl")

```

```

04484     {
04485         // 次のポリゴングループの最初の三角形番号
04486         const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04487
04488         // ポリゴングループに三角形が存在すれば
04489         if (nextgroup > startgroup)
04490         {
04491             // ポリゴングループの三角形数と材質番号を記録する
04492             group.emplace_back(nextgroup, mtl[mtlname]);
04493
04494             // 次のポリゴングループの開始番号を保存しておく
04495             startgroup = nextgroup;
04496         }
04497
04498         // 次に usemtl が来るまで材質名を保持する
04499         str >> mtlname;
04500
04501         // 材質の存在チェック
04502         if (mtl.find(mtlname) == mtl.end())
04503         {
04504             #if defined(DEBUG)
04505                 std::cerr << "Warning: Undefined material: " << mtlname << std::endl;
04506             #endif
04507
04508             // デフォルトの材質を割り当てておく
04509             mtlname = defaultMaterialName;
04510         }
04511         #if defined(DEBUG)
04512             else std::cerr << "usemtl: " << mtlname << std::endl;
04513         #endif
04514     }
04515     else if (op == "mtllib")
04516     {
04517         // MTL ファイルのパス名を作る
04518         str >> std::ws;
04519         std::string mtlpath;
04520         std::getline(str, mtlpath);
04521
04522         // MTL ファイルを読み込む
04523         ggLoadMtl(dirname + mtlpath, mtl, material);
04524     }
04525 }
04526
04527 // OBJ ファイルの読み込みに失敗したら戻る
04528 if (file.bad())
04529 {
04530     #if defined(DEBUG)
04531         std::cerr << "Error: Can't read OBJ file: " << path << std::endl;
04532     #endif
04533     return false;
04534 }
04535
04536 // 最後のポリゴングループの次の三角形番号
04537 const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04538 if (nextgroup > startgroup)
04539 {
04540     // 最後のポリゴングループの三角形数と材質を記録する
04541     group.emplace_back(nextgroup, static_cast<GLuint>(mtl[mtlname]));
04542 }
04543
04544 // スムーズシェーディングしない三角形の頂点を追加する
04545 for (auto& f : face)
04546 {
04547     if (!f.smooth)
04548     {
04549         // 三頂点のそれぞれについて
04550         for (int i = 0; i < 3; ++i)
04551         {
04552             // 新しい頂点座標を生成する (std::array の要素は emplace_back できない)
04553             pos.emplace_back(pos[f.p[i] - 1]);
04554             f.p[i] = static_cast<GLuint>(pos.size());
04555
04556             if (f.t[i] > 0)
04557             {
04558                 // 新しいテクスチャ座標を生成する
04559                 tex.emplace_back(tex[f.t[i] - 1]);
04560                 f.t[i] = static_cast<GLuint>(tex.size());
04561             }
04562
04563             if (f.n[i] > 0)
04564             {
04565                 // 新しい法線を生成する
04566                 norm.emplace_back(norm[f.n[i] - 1]);
04567                 f.n[i] = static_cast<GLuint>(norm.size());
04568             }
04569         }
04570     }
}

```

```

04571     }
04572
04573     // 法線データがなければ算出しておく
04574     if (norm.empty())
04575     {
04576         // 法線データ数の初期値は頂点数と同じでスムーズシェーディングのために初期値は 0
04577         norm.resize(pos.size(), { 0.0f, 0.0f, 0.0f });
04578
04579         // 面の法線の算出と頂点法線の算出
04580         for (auto& f : face)
04581         {
04582             // 頂点座標番号
04583             const auto v0{ f.p[0] - 1 };
04584             const auto v1{ f.p[1] - 1 };
04585             const auto v2{ f.p[2] - 1 };
04586
04587             // v1 - v0, v2 - v0 を求める
04588             const GLfloat d1[] { pos[v1][0] - pos[v0][0], pos[v1][1] - pos[v0][1], pos[v1][2] - pos[v0][2] };
04589             const GLfloat d2[] { pos[v2][0] - pos[v0][0], pos[v2][1] - pos[v0][1], pos[v2][2] - pos[v0][2] };
04590
04591             // 外積により面法線を求める
04592             vec3 n;
04593             ggCross(n.data(), d1, d2);
04594
04595             if (f.smooth)
04596             {
04597                 // スムーズシェーディングを行うときは
04598                 for (int i = 0; i < 3; ++i)
04599                 {
04600                     // 面法線を頂点法線に積算する
04601                     norm[v0][i] += n[i];
04602                     norm[v1][i] += n[i];
04603                     norm[v2][i] += n[i];
04604
04605                     // 面の各頂点の法線番号は頂点番号と同じにする
04606                     f.n[i] = f.p[i];
04607                 }
04608             }
04609             else
04610             {
04611                 // 面法線を最初の頂点に保存する
04612                 norm[v0] = n;
04613                 f.n[0] = f.p[0];
04614
04615                 // 2 頂点追加
04616                 for (int i = 1; i < 3; ++i)
04617                 {
04618                     norm.emplace_back(n);
04619                     f.n[i] = static_cast<GLuint>(norm.size());
04620                 }
04621             }
04622         }
04623
04624         // 頂点の法線ベクトルを正規化する
04625         for (auto& n : norm) ggNormalize3(n.data());
04626     }
04627
04628     // 図形の正規化
04629     if (normalize)
04630     {
04631         // 図形の大きさ
04632         const auto sx{ bmax[0] - bmin[0] };
04633         const auto sy{ bmax[1] - bmin[1] };
04634         const auto sz{ bmax[2] - bmin[2] };
04635
04636         // 図形のスケール
04637         GLfloat s{ sx };
04638         if (sy > s) s = sy;
04639         if (sz > s) s = sz;
04640         const auto scale{ s != 0.0f ? 2.0f / s : 1.0f };
04641
04642         // 図形の中心位置
04643         const auto cx{ (bmax[0] + bmin[0]) * 0.5f };
04644         const auto cy{ (bmax[1] + bmin[1]) * 0.5f };
04645         const auto cz{ (bmax[2] + bmin[2]) * 0.5f };
04646
04647         // 図形の大きさと位置を正規化する
04648         for (auto& p : pos)
04649         {
04650             p[0] = (p[0] - cx) * scale;
04651             p[1] = (p[1] - cy) * scale;
04652             p[2] = (p[2] - cz) * scale;
04653         }
04654     }
04655

```

```

04656 #if defined(DEBUG)
04657     std::cerr
04658     << "[" << name << "]"\\n(Parsed) Group: " << group.size() << ", Material: " << mtl.size()
04659     << ", Pos: " << pos.size() << ", Norm: " << norm.size() << ", Tex: " << tex.size()
04660     << ", Face: " << face.size() << "\\n";
04661 #endif
04662
04663     // OBJ ファイルの読み込み成功
04664     return true;
04665 }
04666 }
04667
04668 //
04669 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Arrays 形式)
04670 //
04671 //
04672 //     name 読み込むOBJ ファイル名
04673 //     group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04674 //     material 読み込んだデータのポリゴングループごとの材質
04675 //     vert 読み込んだデータの頂点属性
04676 //     normalize true ならサイズを正規化する
04677 //     戻り値 読み込みに成功したら true
04678 //
04679 bool gg::ggLoadSimpleObj(const std::string& name,
04680     std::vector<std::array<GLuint, 3>>& group,
04681     std::vector<GgSimpleShader::Material>& material,
04682     std::vector<GgVertex>& vert,
04683     bool normalize)
04684 {
04685     // 読み込み用の一時記憶領域
04686     std::vector<fgrp> tgroup;
04687     std::vector<vec3> tpos;
04688     std::vector<vec3> tnorm;
04689     std::vector<vec2> ttex;
04690     std::vector<fidx> tface;
04691
04692     // OBJ ファイルを解析する
04693     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04694
04695     // 頂点属性データのメモリを確保する
04696     vert.reserve(vert.size() + tface.size() * 3);
04697
04698     // ポリゴングループデータのメモリを確保する
04699     group.reserve(group.size() + tgroup.size());
04700     material.reserve(material.size() + tgroup.size());
04701
04702     // ポリゴングループの最初の三角形番号
04703     GLuint startgroup{ 0 };
04704
04705     // ポリゴングループデータの作成
04706     for (auto& g : tgroup)
04707     {
04708         // このポリゴングループの最初の頂点番号と頂点数・材質番号
04709         std::array<GLuint, 3> v;
04710
04711         // このポリゴングループの最初の頂点番号を保存する
04712         v[0] = static_cast<GLuint>(vert.size());
04713
04714         // 三角形ごとの頂点データの作成
04715         for (GLuint j = startgroup; j < g.nextgroup; ++j)
04716         {
04717             // 処理対象の三角形
04718             auto& f = tface[j];
04719
04720             // 三頂点のそれぞれについて
04721             for (int i = 0; i < 3; ++i)
04722             {
04723                 // テクスチャ座標
04724                 vec2 tex{ 0.0f, 0.0f };
04725                 if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04726
04727                 // 頂点法線
04728                 vec3 norm{ 0.0f, 0.0f, 0.0f };
04729                 if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04730
04731                 // 頂点属性の追加
04732                 vert.emplace_back(tpos[f.p[i] - 1].data(), norm.data());
04733             }
04734         }
04735
04736         // このポリゴングループの頂点数を保存する
04737         v[1] = static_cast<GLuint>(vert.size()) - v[0];
04738         v[2] = g.mtlno;
04739
04740         // このポリゴングループの最初の頂点番号と頂点数・材質番号を登録する
04741         group.emplace_back(v);
04742
04743         // 次のポリゴングループの最初の三角形番号を求める

```

```

04744     startgroup = g.nextgroup;
04745 }
04746
04747 #if defined(DEBUG)
04748     std::cerr
04749         << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04750         << ", Vertex: " << vert.size() << "\n";
04751 #endif
04752
04753 // OBJ ファイルの読み込み成功
04754 return true;
04755 }
04756
04757 //
04758 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Elements 形式)
04759 //
04760 // name 読み込むOBJ ファイル名
04761 // group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04762 // material 読み込んだデータのポリゴングループごとの材質
04763 // vert 読み込んだデータの頂点属性
04764 // face 読み込んだデータの三角形の頂点インデックス
04765 // normalize true ならサイズを正規化する
04766 // 戻り値 読み込みに成功したら true
04767 //
04768 bool gg::ggLoadSimpleObj(const std::string& name,
04769     std::vector<std::array<GLuint, 3>>& group,
04770     std::vector<GgSimpleShader::Material>& material,
04771     std::vector<GgVertex>& vert,
04772     std::vector<GLuint>& face,
04773     bool normalize)
04774 {
04775     // 読み込み用の一時記憶領域
04776     std::vector<fgrp> tgroup;
04777     std::vector<vec3> tpos;
04778     std::vector<vec3> tnorm;
04779     std::vector<vec2> ttex;
04780     std::vector<fidx> tface;
04781
04782     // OBJ ファイルを解析する
04783     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04784
04785     // 頂点属性データの最初の頂点番号
04786     const auto vertbase{ static_cast<GLuint>(vert.size()) };
04787
04788     // 頂点属性データのメモリを確保する
04789     vert.resize(vertbase + tpos.size());
04790
04791     // 三角形データのメモリを確保する
04792     face.reserve(face.size() + tface.size());
04793
04794     // ポリゴングループデータのメモリを確保する
04795     group.reserve(group.size() + tgroup.size());
04796     material.reserve(material.size() + tgroup.size());
04797
04798     // ポリゴングループの最初の三角形番号
04799     GLuint startgroup{ 0 };
04800
04801     // ポリゴングループデータの作成
04802     for (auto& g : tgroup)
04803     {
04804         // このポリゴングループの最初の頂点番号
04805         const auto first{ static_cast<GLuint>(face.size()) };
04806
04807         // 三角形ごとの頂点データの作成
04808         for (GLuint j = startgroup; j < g.nextgroup; ++j)
04809         {
04810             // 処理対象の三角形
04811             auto& f{ tface[j] };
04812
04813             // 三頂点のそれぞれについて
04814             for (int i = 0; i < 3; ++i)
04815             {
04816                 // 追加する三角形データの頂点番号
04817                 const auto q{ f.p[i] - 1 + vertbase };
04818
04819                 // 三角形データの追加
04820                 face.emplace_back(q);
04821
04822                 // テクスチャ座標番号
04823                 vec2 tex{ 0.0f, 0.0f };
04824                 if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04825
04826                 // 頂点法線番号
04827                 vec3 norm{ 0.0f, 0.0f, 0.0f };
04828                 if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04829
04830                 // 頂点の格納

```



```

04831         vert[q] = GgVertex(tpos[f.p[i] - 1].data(), norm.data());
04832     }
04833 }
04834
04835 // このポリゴングループの最初の三角形番号と三角形数・材質番号を登録する
04836 group.emplace_back(std::array<GLuint, 3>{ first, static_cast<GLuint>(face.size()) - first, g.mtlno
});
04837
04838 // 次のポリゴングループの最初の三角形番号を求める
04839 startgroup = g.nextgroup;
04840 }
04841
04842 #if defined(DEBUG)
04843     std::cerr
04844         << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04845         << ", Vertex: " << vert.size() << ", Face: " << face.size() << "\n";
04846 #endif
04847
04848 // OBJ ファイルの読み込み成功
04849 return true;
04850 }
04851
04852 //
04853 // シェーダオブジェクトのコンパイル結果を表示する
04854 //
04855 static GLboolean printShaderInfoLog(GLuint shader, const std::string& str)
04856 {
04857     // コンパイル結果を取得する
04858     GLint status;
04859     glGetShaderiv(shader, GL_COMPILE_STATUS, &status);
04860     if (status == GL_FALSE)
04861     {
04862         std::cerr << "Compile Error in " << str << std::endl;
04863
04864         // シェーダのコンパイル時のログの長さを取得する
04865         GLsizei bufSize;
04866         glGetShaderiv(shader, GL_INFO_LOG_LENGTH, &bufSize);
04867
04868         if (bufSize > 1)
04869         {
04870             // シェーダのコンパイル時のログの内容を取得する
04871             std::vector<GLchar> infoLog(bufSize);
04872             GLsizei length;
04873             glGetShaderInfoLog(shader, bufSize, &length, infoLog.data());
04874             std::cerr << infoLog.data() << std::endl;
04875         }
04876     }
04877
04878     // コンパイル結果を返す
04879     return static_cast<GLboolean>(status);
04880 }
04881
04882 //
04883 // プログラムオブジェクトのリンク結果を表示する
04884 //
04885 static GLboolean printProgramInfoLog(GLuint program)
04886 {
04887     // リンク結果を取得する
04888     GLint status;
04889     glGetProgramiv(program, GL_LINK_STATUS, &status);
04890     if (status == GL_FALSE)
04891     {
04892         std::cerr << "Link Error." << std::endl;
04893
04894         // シェーダのリンク時のログの長さを取得する
04895         GLsizei bufSize;
04896         glGetProgramiv(program, GL_INFO_LOG_LENGTH, &bufSize);
04897
04898         if (bufSize > 1)
04899         {
04900             // シェーダのリンク時のログの内容を取得する
04901             std::vector<GLchar> infoLog(bufSize);
04902             GLsizei length;
04903             glGetProgramInfoLog(program, bufSize, &length, infoLog.data());
04904             std::cerr << infoLog.data() << std::endl;
04905         }
04906     }
04907
04908     // リンク結果を返す
04909     return static_cast<GLboolean>(status);
04910 }
04911
04912 #if defined(GL_GLES_PROTOTYPES)
04913 //
04914 // OpenGL ES 3.1 向けにシェーダのソースコードを変換する
04915 //
04916 static std::string adaptShaderSourceForGles(const std::string& src, bool isFragment)

```

```

04917 {
04918     if (src.empty()) return src;
04919
04920     std::string adapted{ src };
04921     const auto versionPos{ adapted.find("#version") };
04922     if (versionPos != std::string::npos)
04923     {
04924         const auto eol{ adapted.find_first_of("\r\n", versionPos) };
04925         const auto line{ adapted.substr(versionPos, (eol != std::string::npos ? eol - versionPos :
adapted.size() - versionPos)) };
04926         if (line.find("es") == std::string::npos)
04927         {
04928             std::string header{ "#version 310 es\nprecision highp float;\nprecision highp int;" };
04929             adapted.replace(versionPos, line.size(), header);
04930         }
04931         else if (adapted.find("precision ") == std::string::npos)
04932         {
04933             adapted.insert(eol != std::string::npos ? eol + 1 : adapted.size(), "\nprecision highp
float;\nprecision highp int;\n");
04934         }
04935     }
04936     return adapted;
04937 }
04938 #endif
04939 //
04940 // シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
04941 //
04942 // vsrc パーテックスシェーダのソースプログラムの文字列
04943 // fsrc フラグメントシェーダのソースプログラムの文字列 (nullptr なら不使用)
04944 // gsrc ジオメトリシェーダのソースプログラムの文字列 (nullptr なら不使用)
04945 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
04946 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
04947 // vtext パーテックスシェーダのコンパイル時のメッセージに追加する文字列
04948 // ftext フラグメントシェーダのコンパイル時のメッセージに追加する文字列
04949 // gtext ジオメトリシェーダのコンパイル時のメッセージに追加する文字列
04950 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
04951 //
04952 //
04953 //
04954 GLuint gg::ggCreateShader(
04955     const std::string& vsrc,
04956     const std::string& fsrc,
04957     const std::string& gsrc,
04958     GLint nvarying,
04959     const char* const* varyings,
04960     const std::string& vtext,
04961     const std::string& ftext,
04962     const std::string& gtext)
04963 {
04964     // シェーダプログラムの作成
04965     const auto program{ glCreateProgram() };
04966
04967     if (program > 0)
04968     {
04969         bool status = true;
04970
04971         if (!vsrc.empty())
04972         {
04973             #if defined(GL_GLES_PROTOTYPES)
04974                 const std::string vsrcGles{ adaptShaderSourceForGles(vsrc, false) };
04975                 const auto* vsrcp{ vsrcGles.c_str() };
04976             #else
04977                 const auto* vsrcp{ vsrc.c_str() };
04978             #endif
04979             // パーテックスシェーダのシェーダオブジェクトを作成する
04980             const auto vertShader{ glCreateShader(GL_VERTEX_SHADER) };
04981             glShaderSource(vertShader, 1, &vsrcp, nullptr);
04982             glCompileShader(vertShader);
04983
04984             // パーテックスシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04985             if (printShaderInfoLog(vertShader, vtext))
04986                 glAttachShader(program, vertShader);
04987             else
04988                 status = false;
04989             glDeleteShader(vertShader);
04990         }
04991
04992         if (!fsrc.empty())
04993         {
04994             #if defined(GL_GLES_PROTOTYPES)
04995                 const std::string fsrcGles{ adaptShaderSourceForGles(fsrc, true) };
04996                 const auto* fsrcp{ fsrcGles.c_str() };
04997             #else
04998                 const auto* fsrcp{ fsrc.c_str() };
04999             #endif
05000             // フラグメントシェーダのシェーダオブジェクトを作成する
05001             const auto fragShader{ glCreateShader(GL_FRAGMENT_SHADER) };

```

```

05002     glShaderSource(fragShader, 1, &fsrcp, nullptr);
05003     glCompileShader(fragShader);
05004
05005     // フラグメントシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
05006     if (printShaderInfoLog(fragShader, ftext))
05007         glAttachShader(program, fragShader);
05008     else
05009         status = false;
05010     glDeleteShader(fragShader);
05011 }
05012
05013     if (!gsrc.empty())
05014     {
05015         #if defined(GL_GLES_PROTOTYPES)
05016         #   if defined(DEBUG)
05017             std::cerr << gtext << ": The geometry shader is not supported." << std::endl;
05018             status = false;
05019         #   endif
05020         #else
05021             // ジオメトリシェーダのシェーダオブジェクトを作成する
05022             const auto geomShader{ glCreateShader(GL_GEOMETRY_SHADER) };
05023             const auto* gsrcp{ gsrc.c_str() };
05024             glShaderSource(geomShader, 1, &gsrcp, nullptr);
05025             glCompileShader(geomShader);
05026
05027             // ジオメトリシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
05028             if (printShaderInfoLog(geomShader, gtext))
05029                 glAttachShader(program, geomShader);
05030             else
05031                 status = false;
05032             glDeleteShader(geomShader);
05033         #endif
05034     }
05035
05036     // 全てのシェーダオブジェクトのコンパイルに成功したら
05037     if (status)
05038     {
05039         // feedback に使う varying 変数を指定する
05040         if (nvarying > 0)
05041             glTransformFeedbackVaryings(program, nvarying, varyings, GL_SEPARATE_ATTRIBS);
05042
05043         // シェーダプログラムをリンクする
05044         glLinkProgram(program);
05045
05046         // リンクに成功したらプログラムオブジェクト名を返す
05047         if (printProgramInfoLog(program) != GL_FALSE) return program;
05048     }
05049 }
05050
05051 // プログラムオブジェクトが作成できなかった
05052 glDeleteProgram(program);
05053 return 0;
05054 }
05055
05056 //
05057 // シェーダのソースファイルを読み込んだ vector を返す
05058 //
05059 //     name ソースファイル名
05060 //     src 読み込んだソースファイルの文字列
05061 //     戻り値 読み込みの成功すれば true, 失敗したら false
05062 //
05063 static bool readShaderSource(const std::string& name, std::string& src)
05064 {
05065     // ファイル名が空ならそのまま戻る
05066     if (name.empty()) return true;
05067
05068     // ソースファイルを開く
05069     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
05070
05071     #if defined(_WIN32)
05072     // カレントディレクトリで開けなかった場合、実行ファイルと同じディレクトリから探す
05073     if (file.fail())
05074     {
05075         wchar_t exePath[MAX_PATH]{};
05076         if (GetModuleFileNameW(NULL, exePath, MAX_PATH) > 0)
05077         {
05078             CString exeDir{ exePath };
05079             const auto lastSlash{ exeDir.ReverseFind(L'\\') };
05080             if (lastSlash >= 0)
05081             {
05082                 exeDir = exeDir.Left(lastSlash + 1);
05083                 file.open(exeDir + Utf8ToTChar(name), std::ios::binary);
05084             }
05085         }
05086     }
05087     #elif defined(_ANDROID_)
05088     // カレントディレクトリで開けなかった場合、AAssetManager から探す

```

```

05089     if (file.fail() && ggAndroidAssetManager)
05090     {
05091         AAsset* asset{ AAssetManager.open(static_cast<AAssetManager*>(ggAndroidAssetManager), name.c_str(),
AASSET_MODE_BUFFER) };
05092         if (asset)
05093         {
05094             const size_t size{ static_cast<size_t>(AAsset.getLength(asset)) };
05095             src.resize(size);
05096             AAsset.read(asset, &src[0], size);
05097             AAsset.close(asset);
05098             return true;
05099         }
05100     }
05101 #else
05102     // カレントディレクトリで開けなかった場合、実行ファイルと同じディレクトリから探す
05103     if (file.fail())
05104     {
05105         char exePath[PATH_MAX]{};
05106         const ssize_t len{ readlink("/proc/self/exe", exePath, sizeof(exePath) - 1) };
05107         if (len > 0)
05108         {
05109             exePath[len] = '\0';
05110             std::string exeDir{ exePath };
05111             const auto lastSlash{ exeDir.find_last_of('/') };
05112             if (lastSlash != std::string::npos)
05113             {
05114                 exeDir = exeDir.substr(0, lastSlash + 1);
05115                 file.open(exeDir + name, std::ios::binary);
05116             }
05117         }
05118     }
05119 #endif
05120
05121     if (file.fail())
05122     {
05123         // ファイルが開けなければエラーを出力して戻る
05124         std::cerr << "Error: Can't open source file: " << name << std::endl;
05125         return false;
05126     }
05127
05128     // ファイル全体を文字列として読み込む
05129     std::istreambuf_iterator<char> it{ file };
05130     std::istreambuf_iterator<char> last;
05131     src = std::string(it, last);
05132
05133     // ファイルがうまく読み込めなければ戻る
05134     if (file.bad())
05135     {
05136         std::cerr << "Error: Could not read source file: " << name << std::endl;
05137         return false;
05138     }
05139
05140     // ファイルを閉じて戻る
05141     return true;
05142 }
05143
05144 //
05145 // シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05146 //
05147 //   vert   パーテックスシェーダのソースファイル名
05148 //   frag   フラグメントシェーダのソースファイル名 (nullptr なら不使用)
05149 //   geom   ジオメトリシェーダのソースファイル名 (nullptr なら不使用)
05150 //   nvarying フィードバックする varying 変数の数 (0 なら不使用)
05151 //   varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
05152 //   戻り値 シェーダプログラムのプログラム名 (作成できなければ 0)
05153 //
05154 GLuint gg::ggLoadShader(
05155     const std::string& vert,
05156     const std::string& frag,
05157     const std::string& geom,
05158     GLint nvarying,
05159     const char* const* varyings
05160 )
05161 {
05162     // 読み込んだシェーダのソースプログラム
05163     std::string vsrc, fsrc, gsrc;
05164
05165     // 読み込んだ結果
05166     bool status;
05167
05168     // シェーダのソースファイルを読み込む
05169     status = readShaderSource(vert, vsrc);
05170     status = readShaderSource(frag, fsrc) && status;
05171     status = readShaderSource(geom, gsrc) && status;
05172
05173     // 全てのソースファイルが読み込めていなかったらエラー
05174     if (!status) return 0;

```

```

05175
05176 // プログラムオブジェクトを作成する
05177 return ggCreateShader(vsrc, fsrc, gsrc, nvarying, varyings, vert, frag, geom);
05178 }
05179
05180 #if !defined(__APPLE__) && !defined(GL_GLES_PROTOTYPES)
05181 //
05182 // コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
05183 //
05184 // csrc コンピュートシェーダのソースプログラムの文字列
05185 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05186 //
05187 GLuint gg::ggCreateComputeShader(const std::string& csrc, const std::string& ctext)
05188 {
05189 // シェーダプログラムの作成
05190 const auto program{ glCreateProgram() };
05191
05192 if (program > 0)
05193 {
05194 // ソースプログラムの文字列が空だったら 0 を返す
05195 if (csrc.empty()) return 0;
05196
05197 // コンピュートシェーダのシェーダオブジェクトを作成する
05198 const auto compShader{ glCreateShader(GL_COMPUTE_SHADER) };
05199 const auto* csrccp{ csrc.c_str() };
05200 glShaderSource(compShader, 1, &csrccp, nullptr);
05201 glCompileShader(compShader);
05202
05203 // コンピュートシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
05204 if (printShaderInfoLog(compShader, ctext)) glAttachShader(program, compShader);
05205 glDeleteShader(compShader);
05206
05207 // シェーダプログラムをリンクする
05208 glLinkProgram(program);
05209
05210 // プログラムオブジェクトが作成できなければ 0 を返す
05211 if (printProgramInfoLog(program) == GL_FALSE)
05212 {
05213 glDeleteProgram(program);
05214 return 0;
05215 }
05216 }
05217
05218 // プログラムオブジェクトを返す
05219 return program;
05220 }
05221
05222 //
05223 // コンピュートシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05224 //
05225 // comp コンピュートシェーダのソースファイル名
05226 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05227 //
05228 GLuint gg::ggLoadComputeShader(const std::string& comp)
05229 {
05230 // シェーダのソースファイルを読み込む
05231 std::string csrc;
05232 if (readShaderSource(comp, csrc))
05233 {
05234 // プログラムオブジェクトを作成する
05235 return ggCreateComputeShader(csrc.data(), comp);
05236 }
05237
05238 // プログラムオブジェクト作成失敗
05239 return 0;
05240 }
05241 #endif
05242
05243 //
05244 // 点: データ作成
05245 //
05246 void gg::GgPoints::load(const GgVector* pos, GLsizei count, GLenum usage)
05247 {
05248 // 頂点バッファオブジェクトを作成する
05249 position = std::make_shared<GgBuffer<GgVector>>(GL_ARRAY_BUFFER, pos,
static_cast<GLsizei>(sizeof(GgVector)), count, usage);
05250
05251 // このバッファオブジェクトは index == 0 の in 変数から入力する
05252 glVertexAttribPointer(0, static_cast<GLint>(pos->size()), GL_FLOAT, GL_FALSE, 0, 0);
05253 glEnableVertexAttribArray(0);
05254 }
05255
05256 //
05257 // 点: 描画
05258 //
05259 void gg::GgPoints::draw(GLint first, GLsizei count) const
05260 {

```

```

05261 // 頂点配列オブジェクトを指定する
05262 GgShape::draw(first, count);
05263
05264 // 図形を描画する
05265 glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05266 }
05267
05268 //
05269 // 三角形：データ作成
05270 //
05271 void gg::GgTriangles::load(const GgVertex* vert, GLsizei count, GLenum usage)
05272 {
05273     // 頂点バッファオブジェクトを作成する
05274     vertex = std::make_shared<GgBuffer<GgVertex>>(GL_ARRAY_BUFFER, vert,
05275         static_cast<GLsizei>(sizeof(GgVertex)), count, usage);
05276
05277     // 頂点の位置は index == 0 の in 変数から入力する
05278     glVertexAttribPointer(0, static_cast<GLint>(vert->position.size()), GL_FLOAT, GL_FALSE,
05279         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, position));
05280     glEnableVertexAttribArray(0);
05281
05282     // 頂点の法線は index == 1 の in 変数から入力する
05283     glVertexAttribPointer(1, static_cast<GLint>(vert->normal.size()), GL_FLOAT, GL_FALSE,
05284         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, normal));
05285     glEnableVertexAttribArray(1);
05286 }
05287
05288 //
05289 // 三角形：描画
05290 //
05291 void gg::GgTriangles::draw(GLint first, GLsizei count) const
05292 {
05293     // 頂点配列オブジェクトを指定する
05294     GgShape::draw(first, count);
05295
05296     // 図形を描画する
05297     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05298 }
05299
05300 //
05301 // オブジェクト：描画
05302 //
05303 void gg::GgElements::draw(GLint first, GLsizei count) const
05304 {
05305     // 頂点配列オブジェクトを指定する
05306     GgShape::draw(first, count);
05307
05308     // 図形を描画する
05309     glDrawElements(getMode(), count > 0 ? count : getIndexCount() - first,
05310         GL_UNSIGNED_INT, static_cast<GLuint*>(0) + first);
05311 }
05312
05313 //
05314 // 点群を立方体状に生成する
05315 //
05316 std::shared_ptr<gg::GgPoints> gg::ggPointsCube(GLsizei count, GLfloat length, GLfloat cx, GLfloat cy,
05317     GLfloat cz)
05318 {
05319     // メモリを確保する
05320     std::vector<GgVector> pos;
05321     pos.reserve(count);
05322
05323     // 点を生成する
05324     for (GLsizei v = 0; v < count; ++v)
05325     {
05326         const GgVector p
05327         {
05328             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cx,
05329             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cy,
05330             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cz,
05331             1.0f
05332         };
05333         pos.emplace_back(p);
05334     }
05335
05336     // 点データの GgPoints オブジェクトを作成して返す
05337     return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05338 }
05339
05340 //
05341 // 点群を球状に生成する
05342 //
05343 std::shared_ptr<gg::GgPoints> gg::ggPointsSphere(GLsizei count, GLfloat radius,
05344     GLfloat cx, GLfloat cy, GLfloat cz)
05345 {
05346     // メモリを確保する

```

```

05346     std::vector<GgVector> pos;
05347     pos.reserve(count);
05348
05349     // 点を生成する
05350     for (GLsizei v = 0; v < count; ++v)
05351     {
05352         const auto r{ radius * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) };
05353         const auto t{ 6.28318530718f * static_cast<GLfloat>(rand()) / (static_cast<GLfloat>(RAND_MAX) +
1.0f) };
05354         const auto cp{ 2.0f * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 1.0f };
05355         const auto sp{ sqrtf(1.0f - cp * cp) };
05356         const auto ct{ cosf(t) };
05357         const auto st{ sinf(t) };
05358         const GgVector p{ r * sp * ct + cx, r * sp * st + cy, r * cp + cz, 1.0f };
05359
05360         pos.emplace_back(p);
05361     }
05362
05363     // 点データの GgPoints オブジェクトを作成して返す
05364     return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05365 }
05366
05367 //
05368 // 矩形状に 2 枚の三角形を生成する
05369 //
05370 std::shared_ptr<gg::GgTriangles> gg::ggRectangle(GLfloat width, GLfloat height)
05371 {
05372     // 頂点属性
05373     std::array<gg::GgVertex, 4> vert;
05374
05375     // 頂点位置と法線を求める
05376     for (int v = 0; v < 4; ++v)
05377     {
05378         const auto x{ ((v & 1) * 2 - 1) * width };
05379         const auto y{ ((v & 2) - 1) * height };
05380
05381         vert[v] = gg::GgVertex{ x, y, 0.0f, 0.0f, 0.0f, 1.0f };
05382     }
05383
05384     // 矩形の GgTriangles オブジェクトを作成する
05385     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_STRIP);
05386 }
05387
05388 //
05389 // 楕円状に三角形を生成する
05390 //
05391 std::shared_ptr<gg::GgTriangles> gg::ggEllipse(GLfloat width, GLfloat height, GLuint slices)
05392 {
05393     // 楕円のスケール
05394     constexpr GLfloat scale{ 0.5f };
05395
05396     // 作業用のメモリ
05397     std::vector<gg::GgVertex> vert;
05398
05399     // 頂点位置と法線を求める
05400     for (GLuint v = 0; v < slices; ++v)
05401     {
05402         const auto t{ 6.28318530717f * static_cast<GLfloat>(v) / static_cast<GLfloat>(slices) };
05403         const auto x{ cosf(t) * width * scale };
05404         const auto y{ sinf(t) * height * scale };
05405
05406         vert.emplace_back(x, y, 0.0f, 0.0f, 0.0f, 1.0f);
05407     }
05408
05409     // GgTriangles オブジェクトを作成する
05410     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_FAN);
05411 }
05412
05413 //
05414 // Wavefront OBJ ファイルを読み込む (Arrays 形式)
05415 //
05416 std::shared_ptr<gg::GgTriangles> gg::ggArraysObj(const std::string& name, bool normalize)
05417 {
05418     std::vector<std::array<GLuint, 3>> group;
05419     std::vector<gg::GgSimpleShader::Material> material;
05420     std::vector<gg::GgVertex> vert;
05421
05422     // ファイルを読み込む
05423     if (!ggLoadSimpleObj(name, group, material, vert, normalize)) return nullptr;
05424
05425     // GgTriangles オブジェクトを作成する
05426     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLES);
05427 }
05428

```

```

05429 //
05430 // Wavefront OBJ ファイル を読み込む (Elements 形式)
05431 //
05432 std::shared_ptr<gg::GgElements> gg::ggElementsObj(const std::string& name, bool normalize)
05433 {
05434     std::vector<std::array<GLuint, 3>> group;
05435     std::vector<gg::GgSimpleShader::Material> material;
05436     std::vector<gg::GgVertex> vert;
05437     std::vector<GLuint> face;
05438
05439     // ファイルを読み込む
05440     if (!ggLoadSimpleObj(name, group, material, vert, face, normalize)) return nullptr;
05441
05442     // GgElements オブジェクトを作成する
05443     return std::make_shared<gg::GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
05444         face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05445 }
05446
05447 //
05448 // メッシュ形状を作成する (Elements 形式)
05449 //
05450 std::shared_ptr<gg::GgElements> gg::ggElementsMesh(GLuint slices, GLuint stacks, const
    GLfloat (*pos)[3], const GLfloat (*norm)[3])
05451 {
05452     // 頂点属性
05453     std::vector<gg::GgVertex> vert((slices + 1) * (stacks + 1));
05454
05455     // 頂点の法線を求める
05456     for (GLuint j = 0; j <= stacks; ++j)
05457     {
05458         for (GLuint i = 0; i <= slices; ++i)
05459         {
05460             // 処理対象の頂点番号
05461             const auto k{ j * (slices + 1) + i };
05462
05463             // 頂点の法線
05464             gg::GgVector tnorm;
05465             tnorm[3] = 0.0f;
05466
05467             if (norm)
05468             {
05469                 tnorm[0] = norm[k][0];
05470                 tnorm[1] = norm[k][1];
05471                 tnorm[2] = norm[k][2];
05472             }
05473             else
05474             {
05475                 // 処理対象の頂点の周囲の頂点番号
05476                 const auto kim{ i > 0 ? k - 1 : k };
05477                 const auto kip{ i < slices ? k + 1 : k };
05478                 const auto kjm{ j > 0 ? k - slices - 1 : k };
05479                 const auto kjp{ j < stacks ? k + slices + 1 : k };
05480
05481                 // 接線ベクトル
05482                 const std::array<GLfloat, 3> t
05483                 {
05484                     pos[kip][0] - pos[kim][0],
05485                     pos[kip][1] - pos[kim][1],
05486                     pos[kip][2] - pos[kim][2]
05487                 };
05488
05489                 // 従接線ベクトル
05490                 const std::array<GLfloat, 3> b
05491                 {
05492                     pos[kjp][0] - pos[kjm][0],
05493                     pos[kjp][1] - pos[kjm][1],
05494                     pos[kjp][2] - pos[kjm][2]
05495                 };
05496
05497                 // 法線
05498                 tnorm[0] = t[1] * b[2] - t[2] * b[1];
05499                 tnorm[1] = t[2] * b[0] - t[0] * b[2];
05500                 tnorm[2] = t[0] * b[1] - t[1] * b[0];
05501
05502                 // 法線の正規化
05503                 ggNormalize3(tnorm);
05504             }
05505
05506             // 頂点の位置
05507             const gg::GgVector tpos{ pos[k][0], pos[k][1], pos[k][2], 1.0f };
05508
05509             // 頂点属性の保存
05510             vert[k] = gg::GgVertex{ tpos, tnorm };
05511         }
05512     }
05513
05514     // 頂点のインデックス (三角形データ)

```



```

05515     std::vector<GLuint> face;
05516
05517     // 頂点のインデックスを求める
05518     for (GLuint j = 0; j < stacks; ++j)
05519     {
05520         for (GLuint i = 0; i < slices; ++i)
05521         {
05522             // 処理対象のマス
05523             const auto k{ (slices + 1) * j + i };
05524
05525             // マスの上半分の三角形
05526             face.emplace_back(k);
05527             face.emplace_back(k + slices + 2);
05528             face.emplace_back(k + 1);
05529
05530             // マスの下半分の三角形
05531             face.emplace_back(k);
05532             face.emplace_back(k + slices + 1);
05533             face.emplace_back(k + slices + 2);
05534         }
05535     }
05536
05537     // GgElements オブジェクトを作成する
05538     return std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
05539         face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05540 }
05541
05542 //
05543 // 球状に三角形データを生成する (Elements 形式)
05544 //
05545 std::shared_ptr<gg::GgElements> gg::ggElementsSphere(GLfloat radius, int slices, int stacks)
05546 {
05547     // 頂点の位置と法線
05548     std::vector<GLfloat> p, n;
05549
05550     // 頂点の位置と法線を求める
05551     for (int j = 0; j <= stacks; ++j)
05552     {
05553         const auto t{ static_cast<GLfloat>(j) / static_cast<GLfloat>(stacks) };
05554         const auto ph{ 3.1415926536f * t };
05555         const auto y{ cosf(ph) };
05556         const auto r{ sinf(ph) };
05557
05558         for (int i = 0; i <= slices; ++i)
05559         {
05560             const auto s{ static_cast<GLfloat>(i) / static_cast<GLfloat>(slices) };
05561             const auto th{ -2.0f * 3.1415926536f * s };
05562             const auto x{ r * cosf(th) };
05563             const auto z{ r * sinf(th) };
05564
05565             // 頂点の座標値
05566             p.emplace_back(x * radius);
05567             p.emplace_back(y * radius);
05568             p.emplace_back(z * radius);
05569
05570             // 頂点の法線
05571             n.emplace_back(x);
05572             n.emplace_back(y);
05573             n.emplace_back(z);
05574         }
05575     }
05576
05577     // GgElements オブジェクトを作成する
05578     return ggElementsMesh(slices, stacks, reinterpret_cast<GLfloat*>[3]>(p.data()),
05579         reinterpret_cast<GLfloat*>[3]>(n.data()));
05580 }
05581
05582 //
05583 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05584 //
05585 //   r 光源の強度の環境光成分の赤成分
05586 //   g 光源の強度の環境光成分の緑成分
05587 //   b 光源の強度の環境光成分の青成分
05588 //   a 光源の強度の環境光成分の不透明度
05589 //   first 値を設定する光源データの最初の番号
05590 //   count 値を設定する光源データの数
05591 //
05592 void gg::GgSimpleShader::LightBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05593     GLint first, GLsizei count) const
05594 {
05595     // データを格納するバッファオブジェクトの先頭のポインタ
05596     const auto start{ static_cast<char*>(map(first, count)) };
05597     for (GLsizei i = 0; i < count; ++i)
05598     {
05599         // バッファオブジェクトの i 番目のブロックのポインタ
05600         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05601

```

```

05602 // 光源の環境光成分を設定する
05603 light->ambient[0] = r;
05604 light->ambient[1] = g;
05605 light->ambient[2] = b;
05606 light->ambient[3] = a;
05607 }
05608 unmap();
05609 }
05610
05611 //
05612 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05613 //
05614 // ambient 光源の強度の環境光成分
05615 // first 値を設定する光源データの最初の番号
05616 // count 値を設定する光源データの数
05617 //
05618 void gg::GgSimpleShader::LightBuffer::loadAmbient(const GgVector& ambient,
05619 GLint first, GLsizei count) const
05620 {
05621 // データを格納するバッファオブジェクトの先頭のポインタ
05622 const auto start{ static_cast<char*>(map(first, count)) };
05623 for (GLsizei i = 0; i < count; ++i)
05624 {
05625 // バッファオブジェクトの i 番目のブロックのポインタ
05626 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05627
05628 // 光源の強度の環境光成分を設定する
05629 light->ambient = ambient;
05630 }
05631 unmap();
05632 }
05633
05634 //
05635 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05636 //
05637 // r 光源の強度の拡散反射光成分の赤成分
05638 // g 光源の強度の拡散反射光成分の緑成分
05639 // b 光源の強度の拡散反射光成分の青成分
05640 // a 光源の強度の拡散反射光成分の不透明度
05641 // first 値を設定する光源データの最初の番号
05642 // count 値を設定する光源データの数
05643 //
05644 void gg::GgSimpleShader::LightBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05645 GLint first, GLsizei count) const
05646 {
05647 // データを格納するバッファオブジェクトの先頭のポインタ
05648 const auto start{ static_cast<char*>(map(first, count)) };
05649 for (GLsizei i = 0; i < count; ++i)
05650 {
05651 // バッファオブジェクトの i 番目のブロックのポインタ
05652 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05653
05654 // 光源の拡散反射光成分を設定する
05655 light->diffuse[0] = r;
05656 light->diffuse[1] = g;
05657 light->diffuse[2] = b;
05658 light->diffuse[3] = a;
05659 }
05660 unmap();
05661 }
05662
05663 //
05664 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05665 //
05666 // ambient 光源の強度の拡散反射光成分
05667 // first 値を設定する光源データの最初の番号
05668 // count 値を設定する光源データの数
05669 //
05670 void gg::GgSimpleShader::LightBuffer::loadDiffuse(const GgVector& diffuse,
05671 GLint first, GLsizei count) const
05672 {
05673 // データを格納するバッファオブジェクトの先頭のポインタ
05674 const auto start{ static_cast<char*>(map(first, count)) };
05675 for (GLsizei i = 0; i < count; ++i)
05676 {
05677 // バッファオブジェクトの i 番目のブロックのポインタ
05678 const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05679
05680 // 光源の強度の拡散反射光成分を設定する
05681 light->diffuse = diffuse;
05682 }
05683 unmap();
05684 }
05685
05686 //
05687 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05688 //

```

```

05689 //   r 光源の強度の鏡面反射光成分の赤成分
05690 //   g 光源の強度の鏡面反射光成分の緑成分
05691 //   b 光源の強度の鏡面反射光成分の青成分
05692 //   a 光源の強度の鏡面反射光成分の不透明度
05693 //   first 値を設定する光源データの最初の番号
05694 //   count 値を設定する光源データの数
05695 //
05696 void gg::GgSimpleShader::LightBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05697   GLint first, GLsizei count) const
05698 {
05699     // データを格納するバッファオブジェクトの先頭のポインタ
05700     const auto start{ static_cast<char*>(map(first, count)) };
05701     for (GLsizei i = 0; i < count; ++i)
05702     {
05703         // バッファオブジェクトの i 番目のブロックのポインタ
05704         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05705
05706         // 光源の鏡面反射光成分を設定する
05707         light->specular[0] = r;
05708         light->specular[1] = g;
05709         light->specular[2] = b;
05710         light->specular[3] = a;
05711     }
05712     unmap();
05713 }
05714 //
05715 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05716 //
05717 //   ambient 光源の強度の鏡面反射光成分
05718 //   first 値を設定する光源データの最初の番号
05719 //   count 値を設定する光源データの数
05720 //
05721 //
05722 void gg::GgSimpleShader::LightBuffer::loadSpecular(const GgVector& specular,
05723   GLint first, GLsizei count) const
05724 {
05725     // データを格納するバッファオブジェクトの先頭のポインタ
05726     const auto start{ static_cast<char*>(map(first, count)) };
05727     for (GLsizei i = 0; i < count; ++i)
05728     {
05729         // バッファオブジェクトの i 番目のブロックのポインタ
05730         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05731
05732         // 光源の強度の鏡面反射光成分を設定する
05733         light->specular = specular;
05734     }
05735     unmap();
05736 }
05737 //
05738 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の色を設定するか位置は変更しない
05739 //
05740 //   material 光源の特性の GgSimpleShader::Light 構造体
05741 //   first 値を設定する光源データの最初の番号
05742 //   count 値を設定する光源データの数
05743 //
05744 //
05745 void gg::GgSimpleShader::LightBuffer::loadColor(const Light& color,
05746   GLint first, GLsizei count) const
05747 {
05748     // データを格納するバッファオブジェクトの先頭のポインタ
05749     const auto start{ static_cast<char*>(map(first, count)) };
05750     for (GLsizei i = 0; i < count; ++i)
05751     {
05752         // バッファオブジェクトの i 番目のブロックのポインタ
05753         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05754
05755         // 光源の色を設定する
05756         light->ambient = color.ambient;
05757         light->diffuse = color.diffuse;
05758         light->specular = color.specular;
05759     }
05760     unmap();
05761 }
05762 //
05763 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05764 //
05765 //   x 光源の位置の x 座標
05766 //   y 光源の位置の y 座標
05767 //   z 光源の位置の z 座標
05768 //   w 光源の位置の w 座標
05769 //   first 値を設定する光源データの最初の番号
05770 //   count 値を設定する光源データの数
05771 //
05772 //
05773 void gg::GgSimpleShader::LightBuffer::loadPosition(GLfloat x, GLfloat y, GLfloat z, GLfloat w,
05774   GLint first, GLsizei count) const
05775 {

```

```

05776 // データを格納するバッファオブジェクトの先頭のポインタ
05777 const auto start{ static_cast<char*>(map(first, count)) };
05778 for (GLsizei i = 0; i < count; ++i)
05779 {
05780     // バッファオブジェクトの i 番目のブロックのポインタ
05781     const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05782
05783     // 光源の位置を設定する
05784     light->position[0] = x;
05785     light->position[1] = y;
05786     light->position[2] = z;
05787     light->position[3] = w;
05788 }
05789 unmap();
05790 }
05791
05792 //
05793 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05794 //
05795 // position 光源の位置
05796 // first 値を設定する光源データの最初の番号
05797 // count 値を設定する光源データの数
05798 //
05799 void gg::GgSimpleShader::LightBuffer::loadPosition(const GgVector& position,
05800 GLint first, GLsizei count) const
05801 {
05802     // データを格納するバッファオブジェクトの先頭のポインタ
05803     const auto start{ static_cast<char*>(map(first, count)) };
05804     for (GLsizei i = 0; i < count; ++i)
05805     {
05806         // バッファオブジェクトの i 番目のブロックのポインタ
05807         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05808
05809         // 光源の位置を設定する
05810         light->position = position;
05811     }
05812     unmap();
05813 }
05814
05815 //
05816 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05817 //
05818 // r 環境光に対する反射係数の赤成分
05819 // g 環境光に対する反射係数の緑成分
05820 // b 環境光に対する反射係数の青成分
05821 // a 環境光に対する反射係数の不透明度
05822 // first 値を設定する材質データの最初の番号
05823 // count 値を設定する材質データの数
05824 //
05825 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05826 GLint first, GLsizei count) const
05827 {
05828     // データを格納するバッファオブジェクトの先頭のポインタ
05829     const auto start{ static_cast<char*>(map(first, count)) };
05830     for (GLsizei i = 0; i < count; ++i)
05831     {
05832         // バッファオブジェクトの i 番目のブロックのポインタ
05833         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05834
05835         // 環境光に対する反射係数を設定する
05836         material->ambient[0] = r;
05837         material->ambient[1] = g;
05838         material->ambient[2] = b;
05839         material->ambient[3] = a;
05840     }
05841     unmap();
05842 }
05843
05844 //
05845 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05846 //
05847 // ambient 環境光に対する反射係数
05848 // first 値を設定する光源データの最初の番号
05849 // count 値を設定する光源データの数
05850 //
05851 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(const GgVector& ambient,
05852 GLint first, GLsizei count) const
05853 {
05854     // データを格納するバッファオブジェクトの先頭のポインタ
05855     const auto start{ static_cast<char*>(map(first, count)) };
05856     for (GLsizei i = 0; i < count; ++i)
05857     {
05858         // バッファオブジェクトの i 番目のブロックのポインタ
05859         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05860
05861         // 光源の強度の環境光成分を設定する
05862         material->ambient = ambient;

```

```

05863     }
05864     unmap();
05865 }
05866 //
05867 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05868 //
05869 //   r  拡散反射係数の赤成分
05870 //   g  拡散反射係数の緑成分
05871 //   b  拡散反射係数の青成分
05872 //   a  拡散反射係数の不透明度
05873 //   first 値を設定する材質データの最初の番号
05874 //   count 値を設定する材質データの数
05875 //
05876 //
05877 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05878 GLint first, GLsizei count) const
05879 {
05880     // データを格納するバッファオブジェクトの先頭のポインタ
05881     const auto start{ static_cast<char*>(map(first, count)) };
05882     for (GLsizei i = 0; i < count; ++i)
05883     {
05884         // バッファオブジェクトの i 番目のブロックのポインタ
05885         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05886
05887         // 拡散反射係数を設定する
05888         material->diffuse[0] = r;
05889         material->diffuse[1] = g;
05890         material->diffuse[2] = b;
05891         material->diffuse[3] = a;
05892     }
05893     unmap();
05894 }
05895 //
05896 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05897 //
05898 //   diffuse 拡散反射係数
05899 //   first 値を設定する光源データの最初の番号
05900 //   count 値を設定する光源データの数
05901 //
05902 //
05903 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(const GgVector& diffuse,
05904 GLint first, GLsizei count) const
05905 {
05906     // データを格納するバッファオブジェクトの先頭のポインタ
05907     const auto start{ static_cast<char*>(map(first, count)) };
05908     for (GLsizei i = 0; i < count; ++i)
05909     {
05910         // バッファオブジェクトの i 番目のブロックのポインタ
05911         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05912
05913         // 光源の強度の環境光成分を設定する
05914         material->diffuse = diffuse;
05915     }
05916     unmap();
05917 }
05918 //
05919 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05920 //
05921 //   r  環境光に対する反射係数と拡散反射係数の赤成分
05922 //   g  環境光に対する反射係数と拡散反射係数の緑成分
05923 //   b  環境光に対する反射係数と拡散反射係数の青成分
05924 //   a  環境光に対する反射係数と拡散反射係数の不透明度
05925 //   first 値を設定する材質データの最初の番号
05926 //   count 値を設定する材質データの数
05927 //
05928 //
05929 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(GLfloat r, GLfloat g, GLfloat b,
05930 GLfloat a,
05931 GLint first, GLsizei count) const
05932 {
05933     // データを格納するバッファオブジェクトの先頭のポインタ
05934     const auto start{ static_cast<char*>(map(first, count)) };
05935     for (GLsizei i = 0; i < count; ++i)
05936     {
05937         // バッファオブジェクトの i 番目のブロックのポインタ
05938         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05939
05940         // 環境光に対する反射係数と拡散反射係数を設定する
05941         material->ambient[0] = material->diffuse[0] = r;
05942         material->ambient[1] = material->diffuse[1] = g;
05943         material->ambient[2] = material->diffuse[2] = b;
05944         material->ambient[3] = material->diffuse[3] = a;
05945     }
05946     unmap();
05947 }
05948 //

```

```

05949 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05950 //
05951 //   color 環境光に対する反射係数と拡散反射係数
05952 //   first 値を設定する光源データの最初の番号
05953 //   count 値を設定する光源データの数
05954 //
05955 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GgVector& color,
05956 GLint first, GLsizei count) const
05957 {
05958     // データを格納するバッファオブジェクトの先頭のポインタ
05959     const auto start{ static_cast<char*>(map(first, count)) };
05960     for (GLsizei i = 0; i < count; ++i)
05961     {
05962         // バッファオブジェクトの i 番目のブロックのポインタ
05963         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05964
05965         // 光源の強度の環境光成分を設定する
05966         material->ambient = material->diffuse = color;
05967     }
05968     unmap();
05969 }
05970
05971 //
05972 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05973 //
05974 //   color 環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列
05975 //   first 値を設定する材質データの最初の番号
05976 //   count 値を設定する材質データの数
05977 //
05978 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GLfloat* color,
05979 GLint first, GLsizei count) const
05980 {
05981     // ambient 要素のバイトオフセット
05982     constexpr auto ambientOffset{ offsetof(Material, ambient) };
05983
05984     // ambient 要素のバイト数
05985     constexpr auto ambientSize{ sizeof(Material::diffuse) };
05986
05987     // diffuse 要素のバイトオフセット
05988     constexpr auto diffuseOffset{ offsetof(Material, diffuse) };
05989
05990     // diffuse 要素のバイト数
05991     constexpr auto diffuseSize{ sizeof(Material::diffuse) };
05992
05993     // 元のデータの先頭
05994     const auto* source{ reinterpret_cast<const char*>(color) };
05995
05996     // first 番目のブロックから count 個の ambient 要素と diffuse 要素に値を設定する
05997     bind();
05998     for (GLsizei i = 0; i < count; ++i)
05999     {
06000         // 格納先
06001         const auto destination{ getStride() * (first + i) };
06002
06003         // first + i 番目のブロックの ambient 要素に値を設定する
06004         glBufferSubData(getTarget(), destination + ambientOffset, ambientSize, source + i * ambientSize);
06005
06006         // first + i 番目のブロックの diffuse 要素に値を設定する
06007         glBufferSubData(getTarget(), destination + diffuseOffset, diffuseSize, source + i * diffuseSize);
06008     }
06009 }
06010
06011 //
06012 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
06013 //
06014 //   r 鏡面反射係数の赤成分
06015 //   g 鏡面反射係数の緑成分
06016 //   b 鏡面反射係数の青成分
06017 //   a 鏡面反射係数の不透明度
06018 //   first 値を設定する材質データの最初の番号
06019 //   count 値を設定する材質データの数
06020 //
06021 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
06022 GLint first, GLsizei count) const
06023 {
06024     // データを格納するバッファオブジェクトの先頭のポインタ
06025     const auto start{ static_cast<char*>(map(first, count)) };
06026     for (GLsizei i = 0; i < count; ++i)
06027     {
06028         // バッファオブジェクトの i 番目のブロックのポインタ
06029         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
06030
06031         // 鏡面反射係数を設定する
06032         material->specular[0] = r;
06033         material->specular[1] = g;
06034         material->specular[2] = b;
06035         material->specular[3] = a;

```

```

06036     }
06037     unmap();
06038 }
06039
06040 //
06041 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
06042 //
06043 // specular 鏡面反射係数
06044 // first 値を設定する光源データの最初の番号
06045 // count 値を設定する光源データの数
06046 //
06047 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(const GgVector& specular,
06048 GLint first, GLsizei count) const
06049 {
06050     // データを格納するバッファオブジェクトの先頭のポインタ
06051     const auto start{ static_cast<char*>(map(first, count)) };
06052     for (GLsizei i = 0; i < count; ++i)
06053     {
06054         // バッファオブジェクトの i 番目のブロックのポインタ
06055         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
06056
06057         // 光源の強度の環境光成分を設定する
06058         material->specular = specular;
06059     }
06060     unmap();
06061 }
06062
06063 //
06064 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
06065 //
06066 // shininess 輝き係数
06067 // first 値を設定する材質データの最初の番号
06068 // count 値を設定する材質データの数
06069 //
06070 void gg::GgSimpleShader::MaterialBuffer::loadShininess(GLfloat shininess,
06071 GLint first, GLsizei count) const
06072 {
06073     // データを格納するバッファオブジェクトの先頭のポインタ
06074     const auto start{ static_cast<char*>(map(first, count)) };
06075     for (GLsizei i = 0; i < count; ++i)
06076     {
06077         // バッファオブジェクトの i 番目のブロックのポインタ
06078         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
06079         material->shininess = shininess;
06080     }
06081     unmap();
06082 }
06083
06084 //
06085 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
06086 //
06087 // shininess 輝き係数
06088 // first 値を設定する材質データの最初の番号
06089 // count 値を設定する材質データの数
06090 //
06091 void gg::GgSimpleShader::MaterialBuffer::loadShininess(const GLfloat* shininess,
06092 GLint first, GLsizei count) const
06093 {
06094     // データを格納するバッファオブジェクトの先頭のポインタ
06095     const auto start{ static_cast<char*>(map(first, count)) };
06096     for (GLsizei i = 0; i < count; ++i)
06097     {
06098         // バッファオブジェクトの i 番目のブロックのポインタ
06099         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
06100         material->shininess = shininess[i];
06101     }
06102     unmap();
06103 }
06104
06105 //
06106 // 三角形に単純な陰影付けを行うシェーダ：シェーダのソースファイルの読み込み
06107 //
06108 bool gg::GgSimpleShader::load(const std::string& vert, const std::string& frag, const std::string&
geom,
06109 GLint nvarying, const char* const* varyings)
06110 {
06111     if (!GgPointShader::load(vert, frag, geom, nvarying, varyings)) return false;
06112
06113     mnLoc = glGetUniformLocation(get(), "mn");
06114     const auto lightIndex{ glGetUniformLocation(get(), "Light") };
06115     glUniformBlockBinding(get(), lightIndex, LightBindingPoint);
06116     const auto materialIndex{ glGetUniformLocation(get(), "Material") };
06117     glUniformBlockBinding(get(), materialIndex, MaterialBindingPoint);
06118
06119     return true;
06120 }
06121

```



```

06122 //
06123 // Wavefront OBJ 形式のデータ：コンストラクタ
06124 //
06125 gg::GgSimpleObj::GgSimpleObj(const std::string& name, bool normalize)
06126 {
06127     // 作業用のメモリ
06128     std::vector<GgSimpleShader::Material> mat;
06129     std::vector<GgVertex> vert;
06130     std::vector<GLuint> face;
06131
06132     // グループのデータのメモリを確保する
06133     group = std::make_shared<std::vector<std::array<GLuint, 3>>>();
06134
06135     // ファイルを読み込む
06136     if (ggLoadSimpleObj(name, *group, mat, vert, face, normalize))
06137     {
06138         // 頂点バッファオブジェクトを作成する
06139         data = std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
06140             face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
06141
06142         // 描画するオブジェクトを切り替えるために頂点配列オブジェクトを閉じておく
06143         glBindVertexArray(0);
06144
06145         // 材質データを設定する
06146         material = std::make_shared<GgSimpleShader::MaterialBuffer>(mat.data(),
06147             static_cast<GLsizei>(mat.size()));
06148     }
06149 }
06150 //
06151 // Wavefront OBJ 形式のデータ：図形の描画
06152 //
06153 void gg::GgSimpleObj::draw(GLint first, GLsizei count) const
06154 {
06155     // 保持しているグループの数
06156     const auto ng{ static_cast<GLsizei>(group->size()) };
06157
06158     // 描画する最後のグループの次
06159     auto last{ count <= 0 ? ng : first + count };
06160     if (last > ng) last = ng;
06161
06162     for (GLsizei i = first; i < last; ++i)
06163     {
06164         // グループのデータ
06165         const auto& g{ (*group)[i] };
06166
06167         // 材質を設定する
06168         material->select(g[2]);
06169
06170         // 図形を描画する
06171         data->draw(g[0], g[1]);
06172     }
06173 }

```

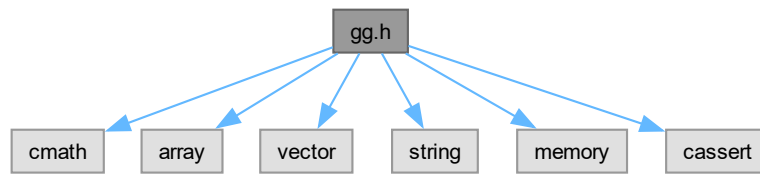
9.53 gg.h ファイル

```

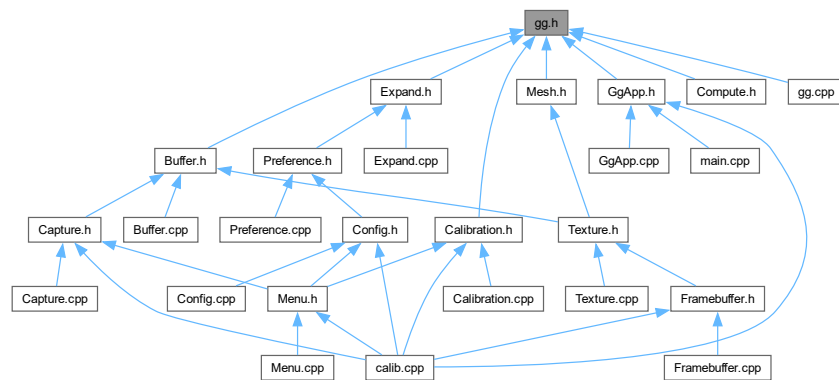
#include <cmath>
#include <array>
#include <vector>
#include <string>
#include <memory>
#include <cassert>

```


gg.h の依存先関係図:



被依存関係図:



クラス

- class gg::GgVector
- class gg::GgMatrix
- class gg::GgQuaternion
- class gg::GgTrackball
- class gg::GgTexture
- class gg::GgColorTexture
- class gg::GgNormalTexture
- class gg::GgBuffer< T >
- class gg::GgUniformBuffer< T >
- class gg::GgVertexArray
- class gg::GgShape
- class gg::GgPoints
- struct gg::GgVertex
- class gg::GgTriangles
- class gg::GgElements
- class gg::GgShader
- class gg::GgPointShader
- class gg::GgSimpleShader
- struct gg::GgSimpleShader::Light
- class gg::GgSimpleShader::LightBuffer
- struct gg::GgSimpleShader::Material
- class gg::GgSimpleShader::MaterialBuffer
- class gg::GgSimpleObj

名前空間

- namespace `gg`

マクロ定義

- `#define ggError()`
- `#define ggFBOError()`

型定義

- using `pathString` = `std::string`
- using `pathChar` = `char`

列挙型

- enum `gg::BindingPoints` { `gg::LightBindingPoint` = 0 , `gg::MaterialBindingPoint` }

関数

- `pathString Utf8ToTChar` (const `std::string` &string)
- `std::string TCharToUtf8` (const `pathString` &cstring)
- void `gg::ggInit` ()
- void `gg::ggError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggFBOError` (const `std::string` &name="", unsigned int line=0)
- void `gg::ggCross` (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggDot3` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength3` (const GLfloat *a)
- GLfloat `gg::ggDistance3` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize3` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize3` (GLfloat *a)
- GLfloat `gg::ggDot4` (const GLfloat *a, const GLfloat *b)
- GLfloat `gg::ggLength4` (const GLfloat *a)
- GLfloat `gg::ggDistance4` (const GLfloat *a, const GLfloat *b)
- void `gg::ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `gg::ggNormalize4` (GLfloat *a)
- const `GgVector` & `gg::operator+` (const `GgVector` &v)
- `GgVector` `gg::operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `gg::operator-` (const `GgVector` &v)
- `GgVector` `gg::operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `gg::ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggLength3` (const `GgVector` &a)
- GLfloat `gg::ggDistance3` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `gg::ggNormalize3` (const `GgVector` &a)
- void `gg::ggNormalize3` (`GgVector` *a)
- GLfloat `gg::ggDot4` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `gg::ggLength4` (const `GgVector` &a)
- GLfloat `gg::ggDistance4` (const `GgVector` &a, const `GgVector` &b)

- `GgVector gg::ggNormalize4` (const `GgVector` &a)
- `void gg::ggNormalize4` (`GgVector` *a)
- `GgMatrix gg::ggIdentity` ()
- `GgMatrix gg::ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix gg::ggTranslate` (const GLfloat *t)
- `GgMatrix gg::ggTranslate` (const `GgVector` &t)
- `GgMatrix gg::ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix gg::ggScale` (const GLfloat *s)
- `GgMatrix gg::ggScale` (const `GgVector` &s)
- `GgMatrix gg::ggRotateX` (GLfloat a)
- `GgMatrix gg::ggRotateY` (GLfloat a)
- `GgMatrix gg::ggRotateZ` (GLfloat a)
- `GgMatrix gg::ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix gg::ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix gg::ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix gg::ggRotate` (const GLfloat *r)
- `GgMatrix gg::ggRotate` (const `GgVector` &r)
- `GgMatrix gg::ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix gg::ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix gg::ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix gg::ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggTranspose` (const `GgMatrix` &m)
- `GgMatrix gg::ggInvert` (const `GgMatrix` &m)
- `GgMatrix gg::ggNormal` (const `GgMatrix` &m)
- `GgQuaternion gg::ggIdentityQuaternion` ()
- `GgQuaternion gg::ggMatrixQuaternion` (const GLfloat *a)
- `GgQuaternion gg::ggMatrixQuaternion` (const `GgMatrix` &m)
- `GgMatrix gg::ggQuaternionMatrix` (const `GgQuaternion` &q)
- `GgMatrix gg::ggQuaternionTransposeMatrix` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggRotateQuaternion` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgQuaternion gg::ggRotateQuaternion` (const GLfloat *v, GLfloat a)
- `GgQuaternion gg::ggRotateQuaternion` (const GLfloat *v)
- `GgQuaternion gg::ggEulerQuaternion` (GLfloat heading, GLfloat pitch, GLfloat roll)
- `GgQuaternion gg::ggEulerQuaternion` (const GLfloat *e)
- `GgQuaternion gg::ggEulerQuaternion` (const `GgVector` &e)
- `GgQuaternion gg::ggSlerp` (const GLfloat *a, const GLfloat *b, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const GLfloat *a, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const GLfloat *a, const `GgQuaternion` &q, GLfloat t)
- `GLfloat gg::ggNorm` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggNormalize` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggConjugate` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggInvert` (const `GgQuaternion` &q)
- `bool gg::ggSaveTga` (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- `bool gg::ggSaveColor` (const std::string &name)
- `bool gg::ggSaveDepth` (const std::string &name)
- `bool gg::ggReadImage` (const std::string &name, std::vector< GLubyte > &image, GLsizei *pWidth, GLsizei *pHeight, GLenum *pFormat)

- GLuint [gg::ggLoadTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_↵RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_↵EDGE, bool swizzle=false)
- GLuint [gg::ggLoadImage](#) (const std::string &name, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE)
- void [gg::ggCreateNormalMap](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format, GLfloat nz, GLenum internal, std::vector< [GgVector](#) > &nmap)
- GLuint [gg::ggLoadHeight](#) (const std::string &name, GLfloat nz, GLsizei *pWidth=nullptr, GLsizei *p↵Height=nullptr, GLenum internal=GL_RGBA)
- GLuint [gg::ggCreateShader](#) (const std::string &vsrsrc, const std::string &fsrsrc="", const std::string &gsrsrc="", GLint nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std↵::string &ftext="fragment shader", const std::string >ext="geometry shader")
- GLuint [gg::ggLoadShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggLoadShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggCreateComputeShader](#) (const std::string &csrsrc, const std::string &ctext="compute shader")
- GLuint [gg::ggLoadComputeShader](#) (const std::string &comp)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsCube](#) (GLsizei countv, GLfloat length=1.0f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsSphere](#) (GLsizei countv, GLfloat radius=0.5f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggRectangle](#) (GLfloat width=1.0f, GLfloat height=1.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggEllipse](#) (GLfloat width=1.0f, GLfloat height=1.0f, GLuint slices=16)
- std::shared_ptr< [GgTriangles](#) > [gg::ggArraysObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsMesh](#) (GLuint slices, GLuint stacks, const GLfloat(*pos)[3], const GLfloat(*norm)[3]=nullptr)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsSphere](#) (GLfloat radius=1.0f, int slices=16, int stacks=8)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std↵::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, bool normalize=false)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std↵::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint [gg::ggBufferAlignment](#)
 使用している GPU のバッファアライメント。

9.53.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の宣言。

著者

Kohe Tokoi

日付

July 17, 2025

[gg.h](#) に定義があります。

9.53.2 マクロ定義詳解

9.53.2.1 ggError

```
#define ggError()
```

OpenGL のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で OpenGL のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

gg.h の 1410 行目に定義があります。

9.53.2.2 ggFBOError

```
#define ggFBOError()
```

FBO のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で FBO のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

gg.h の 1437 行目に定義があります。

9.53.3 型定義詳解

9.53.3.1 pathChar

```
using pathChar = char
```

gg.h の 78 行目に定義があります。

9.53.3.2 pathString

```
using pathString = std::string
```

gg.h の 77 行目に定義があります。

9.53.4 関数詳解

9.53.4.1 TCharToUtf8()

```
std::string TCharToUtf8 (  
    const pathString & cstring) [inline]
```

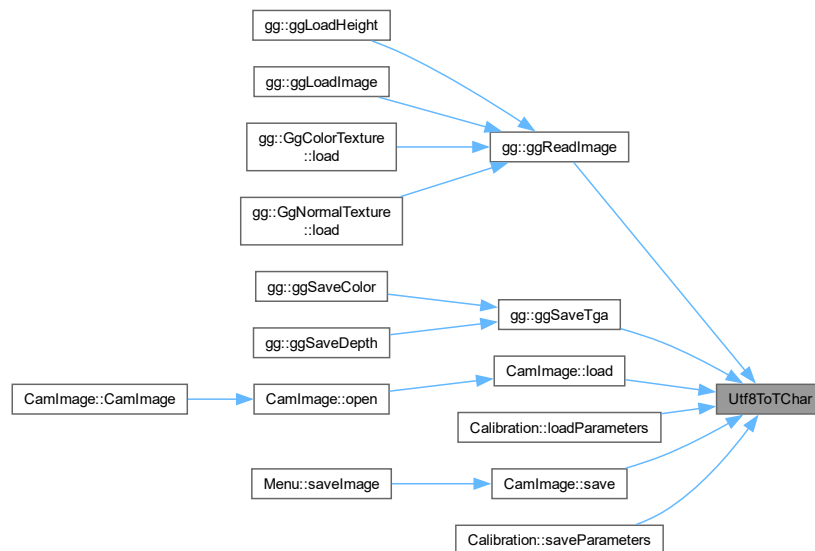
gg.h の 80 行目に定義があります。

9.53.4.2 Utf8ToTChar()

```
pathString Utf8ToTChar (
    const std::string & string) [inline]
```

gg.h の 79 行目に定義があります。

被呼び出し関係図:



9.54 gg.h

[詳解]

```

00001 #pragma once
00002
00026
00034
00035 // 標準ライブラリ
00036 #include <cmath>
00037 #include <array>
00038 #include <vector>
00039 #include <string>
00040 #include <memory>
00041 #include <cassert>
00042
00043 // Windows (Visual Studio) のとき
00044 #if defined(_MSC_VER)
00045 // 非推奨の警告を出さない
00046 # pragma warning(disable:4996)
00047 // 数学ライブラリの定数を使う
00048 # define _USE_MATH_DEFINES
00049 // MIN() / MAX マクロは使わない
00050 # define NOMINMAX
00051 // ファイルパスの文字コード
00052 #include <atlstr.h>
00053 using pathString = CString;
00054 using pathChar = wchar_t;
00055 extern pathString Utf8ToTChar(const std::string& string);
00056 extern std::string TCharToUtf8(const pathString& cstring);
00057 // デバッグビルドかどうか調べる
00058 # if defined(_DEBUG)
00059 #   if !defined(DEBUG)
00060 #     define DEBUG
  
```

```

00061 #      endif
00062 #      define GLFW3_CONFIGURATION "Debug"
00063 #      else
00064 #          if !defined(NDEBUG)
00065 #              define NDEBUG
00066 #          endif
00067 #          define GLFW3_CONFIGURATION "Release"
00068 #      endif
00069 // プラットフォームを調べる
00070 #      if defined(_WIN64)
00071 #          define GLFW3_PLATFORM "x64"
00072 #      else
00073 #          define GLFW3_PLATFORM "Win32"
00074 #      endif
00075 #else
00076 // ファイルパスの文字コード
00077 using pathString = std::string;
00078 using pathChar = char;
00079 inline pathString Utf8ToTChar(const std::string& string) { return string; }
00080 inline std::string TCharToUtf8(const pathString& cstring) { return cstring; }
00081 #endif
00082
00083
00084
00085 // macOS で "OpenGL deprecated" の警告を出さない
00086 #if defined(__APPLE__)
00087 #    define GL_SILENCE_DEPRECATED
00088 #endif
00089
00090 // フレームワークに GLFW 3 を使う (Android では EGL / GLES3 を使用)
00091 #if defined(__ANDROID__)
00092 #    if !defined(GL_GLES_PROTOTYPES)
00093 #        define GL_GLES_PROTOTYPES 1
00094 #    endif
00095 #    if !defined(IMGUI_IMPL_OPENGL_ES3)
00096 #        define IMGUI_IMPL_OPENGL_ES3 1
00097 #    endif
00098 #    include <EGL/egl.h>
00099 #    include <GLES3/gl31.h>
00100 #    include <GLES3/gl3ext.h>
00101 #else
00102 #    if defined(IMGUI_IMPL_OPENGL_ES2)
00103 #        define GLFW_INCLUDE_ES2
00104 #    elif defined(IMGUI_IMPL_OPENGL_ES3)
00105 #        define GLFW_INCLUDE_ES3
00106 #    else
00107 #        define GLFW_INCLUDE_GLCOREARB
00108 #    endif
00109 #    include <GLFW/glfw3.h>
00110 #endif
00111 #if !defined(__gl_h__)
00112 #    define __gl_h__
00113 #endif
00114 #if !defined(__GL_H__)
00115 #    define __GL_H__
00116 #endif
00117
00118 // OpenGL 3.2 の API のエントリポイント
00119 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00120 extern PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00121 extern PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00122 extern PFNGLACTIVETEXTUREPROC glActiveTexture;
00123 extern PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00124 extern PFNGLATTACHSHADERPROC glAttachShader;
00125 extern PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00126 extern PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00127 extern PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00128 extern PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00129 extern PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00130 extern PFNGLBEGINQUERYPROC glBeginQuery;
00131 extern PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00132 extern PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00133 extern PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00134 extern PFNGLBINDBUFFERPROC glBindBuffer;
00135 extern PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00136 extern PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00137 extern PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00138 extern PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00139 extern PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00140 extern PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00141 extern PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00142 extern PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00143 extern PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00144 extern PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00145 extern PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00146 extern PFNGLBINDSAMPLERPROC glBindSampler;
00147 extern PFNGLBINDSAMPLERSPROC glBindSamplers;
00148 extern PFNGLBINDTEXTUREPROC glBindTexture;

```

```

00149 extern PFNGLBINDTEXTURESPROC glBindTextures;
00150 extern PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00151 extern PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00152 extern PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00153 extern PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00154 extern PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00155 extern PFNGLBLENDARRIERKHRPROC glBlendBarrierKHR;
00156 extern PFNGLBLENDARRIERNVPROC glBlendBarrierNV;
00157 extern PFNGLBLENDCOLORPROC glBlendColor;
00158 extern PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00159 extern PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00160 extern PFNGLBLEND EQUATIONPROC glBlendEquation;
00161 extern PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00162 extern PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00163 extern PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00164 extern PFNGLBLEND FUNC IARBPROC glBlendFunciARB;
00165 extern PFNGLBLEND FUNC IPROC glBlendFunci;
00166 extern PFNGLBLEND FUNC PROC glBlendFunc;
00167 extern PFNGLBLEND FUNC SEPARATEIARBPROC glBlendFuncSeparateiARB;
00168 extern PFNGLBLEND FUNC SEPARATEIPROC glBlendFuncSeparatei;
00169 extern PFNGLBLEND FUNC SEPARATEPROC glBlendFuncSeparate;
00170 extern PFNGLBLEND PARAMETER INVPROC glBlendParameteriNV;
00171 extern PFNGLBLIT FRAME BUFFER PROC glBlitFramebuffer;
00172 extern PFNGLBLIT NAMED FRAME BUFFER PROC glBlitNamedFramebuffer;
00173 extern PFNGLBUFFER ADDRESS RANGE NVPROC glBufferAddressRangeNV;
00174 extern PFNGLBUFFER DATA PROC glBufferData;
00175 extern PFNGLBUFFER PAGE COMMITMENT ARBPROC glBufferPageCommitmentARB;
00176 extern PFNGLBUFFER STORAGE PROC glBufferStorage;
00177 extern PFNGLBUFFER SUB DATA PROC glBufferSubData;
00178 extern PFNGLCALL COMMAND LIST NVPROC glCallCommandListNV;
00179 extern PFNGLCHECK FRAME BUFFER STATUS PROC glCheckFramebufferStatus;
00180 extern PFNGLCHECK NAMED FRAME BUFFER STATUS EXT PROC glCheckNamedFramebufferStatusEXT;
00181 extern PFNGLCHECK NAMED FRAME BUFFER STATUS PROC glCheckNamedFramebufferStatus;
00182 extern PFNGLCLAMP COLOR PROC glClampColor;
00183 extern PFNGLCLEAR BUFFER DATA PROC glClearBufferData;
00184 extern PFNGLCLEAR BUFFER FIPROC glClearBufferfi;
00185 extern PFNGLCLEAR BUFFER FVPROC glClearBufferfv;
00186 extern PFNGLCLEAR BUFFER IVPROC glClearBufferiv;
00187 extern PFNGLCLEAR BUFFER SUB DATA PROC glClearBufferSubData;
00188 extern PFNGLCLEAR BUFFER UI VPROC glClearBufferuiv;
00189 extern PFNGLCLEAR COLOR PROC glClearColor;
00190 extern PFNGLCLEAR DEPTH FPROC glClearDepthf;
00191 extern PFNGLCLEAR DEPTH PROC glClearDepth;
00192 extern PFNGLCLEAR NAMED BUFFER DATA EXT PROC glClearNamedBufferDataEXT;
00193 extern PFNGLCLEAR NAMED BUFFER DATA PROC glClearNamedBufferData;
00194 extern PFNGLCLEAR NAMED BUFFER SUB DATA EXT PROC glClearNamedBufferSubDataEXT;
00195 extern PFNGLCLEAR NAMED BUFFER SUB DATA PROC glClearNamedBufferSubData;
00196 extern PFNGLCLEAR NAMED FRAME BUFFER FIPROC glClearNamedFramebufferfi;
00197 extern PFNGLCLEAR NAMED FRAME BUFFER FVPROC glClearNamedFramebufferfv;
00198 extern PFNGLCLEAR NAMED FRAME BUFFER IVPROC glClearNamedFramebufferiv;
00199 extern PFNGLCLEAR NAMED FRAME BUFFER UI VPROC glClearNamedFramebufferuiv;
00200 extern PFNGLCLEAR PROC glClear;
00201 extern PFNGLCLEAR STENCIL PROC glClearStencil;
00202 extern PFNGLCLEAR TEX IMAGE PROC glClearTexImage;
00203 extern PFNGLCLEAR TEX SUB IMAGE PROC glClearTexSubImage;
00204 extern PFNGLCLIENT ATTRIB DEFAULT TEXT PROC glClientAttribDefaultEXT;
00205 extern PFNGLCLIENT WAIT SYNC PROC glClientWaitSync;
00206 extern PFNGLCLIP CONTROL PROC glClipControl;
00207 extern PFNGLCOLOR FORMAT NVPROC glColorFormatNV;
00208 extern PFNGLCOLOR MASK IPROC glColorMaski;
00209 extern PFNGLCOLOR MASK PROC glColorMask;
00210 extern PFNGLCOMMAND LIST SEGMENTS NVPROC glCommandListSegmentsNV;
00211 extern PFNGLCOMPILE COMMAND LIST NVPROC glCompileCommandListNV;
00212 extern PFNGLCOMPILE SHADER INCLUDE ARBPROC glCompileShaderIncludeARB;
00213 extern PFNGLCOMPILE SHADER PROC glCompileShader;
00214 extern PFNGLCOMPRESSED MULTI TEX IMAGE 1 D EXT PROC glCompressedMultiTexImage1DEXT;
00215 extern PFNGLCOMPRESSED MULTI TEX IMAGE 2 D EXT PROC glCompressedMultiTexImage2DEXT;
00216 extern PFNGLCOMPRESSED MULTI TEX IMAGE 3 D EXT PROC glCompressedMultiTexImage3DEXT;
00217 extern PFNGLCOMPRESSED MULTI TEX SUB IMAGE 1 D EXT PROC glCompressedMultiTexSubImage1DEXT;
00218 extern PFNGLCOMPRESSED MULTI TEX SUB IMAGE 2 D EXT PROC glCompressedMultiTexSubImage2DEXT;
00219 extern PFNGLCOMPRESSED MULTI TEX SUB IMAGE 3 D EXT PROC glCompressedMultiTexSubImage3DEXT;
00220 extern PFNGLCOMPRESSED TEX IMAGE 1 D PROC glCompressedTexImage1D;
00221 extern PFNGLCOMPRESSED TEX IMAGE 2 D PROC glCompressedTexImage2D;
00222 extern PFNGLCOMPRESSED TEX IMAGE 3 D PROC glCompressedTexImage3D;
00223 extern PFNGLCOMPRESSED TEX SUB IMAGE 1 D PROC glCompressedTexSubImage1D;
00224 extern PFNGLCOMPRESSED TEX SUB IMAGE 2 D PROC glCompressedTexSubImage2D;
00225 extern PFNGLCOMPRESSED TEX SUB IMAGE 3 D PROC glCompressedTexSubImage3D;
00226 extern PFNGLCOMPRESSED TEXTURE IMAGE 1 D EXT PROC glCompressedTextureImage1DEXT;
00227 extern PFNGLCOMPRESSED TEXTURE IMAGE 2 D EXT PROC glCompressedTextureImage2DEXT;
00228 extern PFNGLCOMPRESSED TEXTURE IMAGE 3 D EXT PROC glCompressedTextureImage3DEXT;
00229 extern PFNGLCOMPRESSED TEXTURE SUB IMAGE 1 D EXT PROC glCompressedTextureSubImage1DEXT;
00230 extern PFNGLCOMPRESSED TEXTURE SUB IMAGE 1 D PROC glCompressedTextureSubImage1D;
00231 extern PFNGLCOMPRESSED TEXTURE SUB IMAGE 2 D EXT PROC glCompressedTextureSubImage2DEXT;
00232 extern PFNGLCOMPRESSED TEXTURE SUB IMAGE 2 D PROC glCompressedTextureSubImage2D;
00233 extern PFNGLCOMPRESSED TEXTURE SUB IMAGE 3 D EXT PROC glCompressedTextureSubImage3DEXT;
00234 extern PFNGLCOMPRESSED TEXTURE SUB IMAGE 3 D PROC glCompressedTextureSubImage3D;
00235 extern PFNGLCONSERVATIVE RASTER PARAMETER FNVPROC glConservativeRasterParameterfNV;

```



```
00236 extern PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00237 extern PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00238 extern PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00239 extern PFNGLCOPYMULTITEXIMAGE1DEXTPROC glCopyMultiTexImage1DEXT;
00240 extern PFNGLCOPYMULTITEXIMAGE2DEXTPROC glCopyMultiTexImage2DEXT;
00241 extern PFNGLCOPYMULTITEXSUBIMAGE1DEXTPROC glCopyMultiTexSubImage1DEXT;
00242 extern PFNGLCOPYMULTITEXSUBIMAGE2DEXTPROC glCopyMultiTexSubImage2DEXT;
00243 extern PFNGLCOPYMULTITEXSUBIMAGE3DEXTPROC glCopyMultiTexSubImage3DEXT;
00244 extern PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00245 extern PFNGLCOPYPATHNVPROC glCopyPathNV;
00246 extern PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00247 extern PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00248 extern PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00249 extern PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00250 extern PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00251 extern PFNGLCOPYTEXTUREIMAGE1DEXTPROC glCopyTextureImage1DEXT;
00252 extern PFNGLCOPYTEXTUREIMAGE2DEXTPROC glCopyTextureImage2DEXT;
00253 extern PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC glCopyTextureSubImage1DEXT;
00254 extern PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00255 extern PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC glCopyTextureSubImage2DEXT;
00256 extern PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00257 extern PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC glCopyTextureSubImage3DEXT;
00258 extern PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00259 extern PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00260 extern PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00261 extern PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00262 extern PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00263 extern PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00264 extern PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00265 extern PFNGLCREATEBUFFERSPROC glCreateBuffers;
00266 extern PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00267 extern PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00268 extern PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00269 extern PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00270 extern PFNGLCREATEPROGRAMPROC glCreateProgram;
00271 extern PFNGLCREATEQUERIESPROC glCreateQueries;
00272 extern PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00273 extern PFNGLCREATESAMPLERSPROC glCreateSamplers;
00274 extern PFNGLCREATESHADERPROC glCreateShader;
00275 extern PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00276 extern PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00277 extern PFNGLCREATESTATESNVPROC glCreateStatesNV;
00278 extern PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLeventARB;
00279 extern PFNGLCREATETEXTURESPROC glCreateTextures;
00280 extern PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00281 extern PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00282 extern PFNGLCULLFACEPROC glCullFace;
00283 extern PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00284 extern PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00285 extern PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00286 extern PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00287 extern PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00288 extern PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00289 extern PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00290 extern PFNGLDELETECOMMANDLISTSNVPROC glDeleteCommandListsNV;
00291 extern PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00292 extern PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00293 extern PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00294 extern PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00295 extern PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00296 extern PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00297 extern PFNGLDELETEPROGRAMPROC glDeleteProgram;
00298 extern PFNGLDELETEQUERIESPROC glDeleteQueries;
00299 extern PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00300 extern PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00301 extern PFNGLDELETESHADERPROC glDeleteShader;
00302 extern PFNGLDELETESSTATESNVPROC glDeleteStatesNV;
00303 extern PFNGLDELETESYNCPROC glDeleteSync;
00304 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00305 extern PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00306 extern PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00307 extern PFNGLDEPTHFUNCPROC glDepthFunc;
00308 extern PFNGLDEPTHMASKPROC glDepthMask;
00309 extern PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00310 extern PFNGLDEPTHRANGEFPROC glDepthRangef;
00311 extern PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00312 extern PFNGLDEPTHRANGEPROC glDepthRange;
00313 extern PFNGLDETACHSHADERPROC glDetachShader;
00314 extern PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00315 extern PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00316 extern PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00317 extern PFNGLDISABLEIPROC glDisablei;
00318 extern PFNGLDISABLEPROC glDisable;
00319 extern PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00320 extern PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00321 extern PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00322 extern PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
```

```
00323 extern PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00324 extern PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00325 extern PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00326 extern PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00327 extern PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00328 extern PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00329 extern PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00330 extern PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00331 extern PFNGLDRAWARRAYSPROC glDrawArrays;
00332 extern PFNGLDRAWBUFFERPROC glDrawBuffer;
00333 extern PFNGLDRAWBUFFERSPROC glDrawBuffers;
00334 extern PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00335 extern PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00336 extern PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00337 extern PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00338 extern PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00339 extern PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00340 extern PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00341 extern PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00342 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC
glDrawElementsInstancedBaseVertexBaseInstance;
00343 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00344 extern PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00345 extern PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00346 extern PFNGLDRAWELEMENTSPROC glDrawElements;
00347 extern PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00348 extern PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00349 extern PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00350 extern PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00351 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00352 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00353 extern PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00354 extern PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00355 extern PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00356 extern PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00357 extern PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00358 extern PFNGLENABLEIPROC glEnablei;
00359 extern PFNGLENABLEPROC glEnable;
00360 extern PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00361 extern PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00362 extern PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00363 extern PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexArray;
00364 extern PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00365 extern PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00366 extern PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00367 extern PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00368 extern PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00369 extern PFNGLENDQUERYPROC glEndQuery;
00370 extern PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00371 extern PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00372 extern PFNGLFENCESYNCPROC glFenceSync;
00373 extern PFNGLFINISHPROC glFinish;
00374 extern PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00375 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00376 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00377 extern PFNGLFLUSHPROC glFlush;
00378 extern PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00379 extern PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00380 extern PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00381 extern PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00382 extern PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00383 extern PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00384 extern PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00385 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00386 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00387 extern PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00388 extern PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00389 extern PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00390 extern PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00391 extern PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00392 extern PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00393 extern PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00394 extern PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00395 extern PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00396 extern PFNGLFRONTFACEPROC glFrontFace;
00397 extern PFNGLGENBUFFERSPROC glGenBuffers;
00398 extern PFNGLGENERATEMIPMAPPROC glGenMipmap;
00399 extern PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenMultiTexMipmapEXT;
00400 extern PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenTextureMipmapEXT;
00401 extern PFNGLGENERATETEXTUREMIPMAPPROC glGenTextureMipmap;
00402 extern PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00403 extern PFNGLGENPATHSNVPROC glGenPathsNV;
00404 extern PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00405 extern PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00406 extern PFNGLGENQUERIESPROC glGenQueries;
00407 extern PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00408 extern PFNGLGENSAMPLERSPROC glGenSamplers;
```

```

00409 extern PFNGLGENTEXTURESPROC glGenTextures;
00410 extern PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00411 extern PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00412 extern PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00413 extern PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00414 extern PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00415 extern PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00416 extern PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00417 extern PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00418 extern PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00419 extern PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00420 extern PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00421 extern PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00422 extern PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00423 extern PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00424 extern PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00425 extern PFNGLGETBOOLEANI_VPROC glGetBooleani_v;
00426 extern PFNGLGETBOOLEANVPROC glGetBooleanv;
00427 extern PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00428 extern PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00429 extern PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00430 extern PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00431 extern PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00432 extern PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00433 extern PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00434 extern PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00435 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00436 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00437 extern PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00438 extern PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00439 extern PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00440 extern PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00441 extern PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00442 extern PFNGLGETDOUBLEI_VEXTPROC glGetDoublei_vEXT;
00443 extern PFNGLGETDOUBLEI_VPROC glGetDoublei_v;
00444 extern PFNGLGETDOUBLEVPROC glGetDoublev;
00445 extern PFNGLGETERRORPROC glGetError;
00446 extern PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00447 extern PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00448 extern PFNGLGETFLOATI_VEXTPROC glGetFloati_vEXT;
00449 extern PFNGLGETFLOATI_VPROC glGetFloati_v;
00450 extern PFNGLGETFLOATVPROC glGetFloatv;
00451 extern PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00452 extern PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00453 extern PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00454 extern PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00455 extern PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00456 extern PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00457 extern PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00458 extern PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00459 extern PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00460 extern PFNGLGETINTEGER64I_VPROC glGetInteger64i_v;
00461 extern PFNGLGETINTEGER64VPROC glGetInteger64v;
00462 extern PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00463 extern PFNGLGETINTEGERI_VPROC glGetIntegeri_v;
00464 extern PFNGLGETINTEGERUI64I_VNVPROC glGetIntegerui64i_vNV;
00465 extern PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00466 extern PFNGLGETINTEGERVPROC glGetIntegerv;
00467 extern PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00468 extern PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00469 extern PFNGLGETINTERNALFORMATSAMPLEIVNVPROC glGetInternalformatSampleivNV;
00470 extern PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00471 extern PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00472 extern PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00473 extern PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00474 extern PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00475 extern PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00476 extern PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00477 extern PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00478 extern PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00479 extern PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00480 extern PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00481 extern PFNGLGETMULTITEXPARAMETERUIVEXTPROC glGetMultiTexParameterIuivEXT;
00482 extern PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00483 extern PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00484 extern PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00485 extern PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00486 extern PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00487 extern PFNGLGETNAMEDBUFFERPOINTERVEXTPROC glGetNamedBufferPointervEXT;
00488 extern PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00489 extern PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00490 extern PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00491 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC
    glGetNamedFramebufferAttachmentParameterivEXT;
00492 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00493 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00494 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;

```

```

00495 extern PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00496 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00497 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00498 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00499 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00500 extern PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00501 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00502 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00503 extern PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00504 extern PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00505 extern PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00506 extern PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00507 extern PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00508 extern PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00509 extern PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00510 extern PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvARB;
00511 extern PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00512 extern PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvARB;
00513 extern PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00514 extern PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64vARB;
00515 extern PFNGLGETNUNIFORMIVARBPROC glGetnUniformivARB;
00516 extern PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00517 extern PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64vARB;
00518 extern PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivARB;
00519 extern PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00520 extern PFNGLGETOBJECTLABELPROC glGetObjectLabelEXT;
00521 extern PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00522 extern PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00523 extern PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00524 extern PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00525 extern PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00526 extern PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00527 extern PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00528 extern PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00529 extern PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00530 extern PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00531 extern PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00532 extern PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00533 extern PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00534 extern PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00535 extern PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00536 extern PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00537 extern PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00538 extern PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00539 extern PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00540 extern PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00541 extern PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00542 extern PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00543 extern PFNGLGETPOINTERI_VEXTPROC glGetPointeri_vEXT;
00544 extern PFNGLGETPOINTERVPROC glGetPointeriv;
00545 extern PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00546 extern PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00547 extern PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00548 extern PFNGLGETPROGRAMIVPROC glGetProgramiv;
00549 extern PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00550 extern PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00551 extern PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00552 extern PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00553 extern PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00554 extern PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00555 extern PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00556 extern PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00557 extern PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00558 extern PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00559 extern PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00560 extern PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00561 extern PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00562 extern PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00563 extern PFNGLGETQUERYIVPROC glGetQueryiv;
00564 extern PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00565 extern PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00566 extern PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00567 extern PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00568 extern PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00569 extern PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00570 extern PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00571 extern PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00572 extern PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00573 extern PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00574 extern PFNGLGETSHADERIVPROC glGetShaderiv;
00575 extern PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00576 extern PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00577 extern PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00578 extern PFNGLGETSTRINGIPROC glGetStringi;
00579 extern PFNGLGETSTRINGPROC glGetString;
00580 extern PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00581 extern PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;

```



```
00582 extern PFNGLGETSYNCCIVPROC glGetSynciv;
00583 extern PFNGLGETTEXIMAGEPROC glGetTexImage;
00584 extern PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00585 extern PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00586 extern PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00587 extern PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00588 extern PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00589 extern PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00590 extern PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00591 extern PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00592 extern PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00593 extern PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00594 extern PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00595 extern PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00596 extern PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00597 extern PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00598 extern PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00599 extern PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00600 extern PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00601 extern PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00602 extern PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00603 extern PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00604 extern PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00605 extern PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00606 extern PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00607 extern PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00608 extern PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00609 extern PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64v;
00610 extern PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00611 extern PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00612 extern PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00613 extern PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00614 extern PFNGLGETUNIFORMDVPROC glGetUniformdv;
00615 extern PFNGLGETUNIFORMFVPROC glGetUniformfv;
00616 extern PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00617 extern PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00618 extern PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00619 extern PFNGLGETUNIFORMIVPROC glGetUniformiv;
00620 extern PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00621 extern PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00622 extern PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00623 extern PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00624 extern PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00625 extern PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00626 extern PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00627 extern PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00628 extern PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayInteger_vEXT;
00629 extern PFNGLGETVERTEXARRAYIIVPROC glGetVertexArrayiiv;
00630 extern PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00631 extern PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointer_vEXT;
00632 extern PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00633 extern PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00634 extern PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribiiv;
00635 extern PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00636 extern PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00637 extern PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribldv;
00638 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00639 extern PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00640 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00641 extern PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointerv;
00642 extern PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00643 extern PFNGLHINTPROC glHint;
00644 extern PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00645 extern PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00646 extern PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00647 extern PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00648 extern PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00649 extern PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00650 extern PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00651 extern PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00652 extern PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00653 extern PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00654 extern PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00655 extern PFNGLISBUFFERPROC glIsBuffer;
00656 extern PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00657 extern PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00658 extern PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00659 extern PFNGLISENABLEDIPROC glIsEnabledi;
00660 extern PFNGLISENABLEDPROC glIsEnabled;
00661 extern PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00662 extern PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00663 extern PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00664 extern PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00665 extern PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00666 extern PFNGLISPATHTVPROC glIsPathNV;
00667 extern PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00668 extern PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
```

```

00669 extern PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00670 extern PFNGLISPROGRAMPROC glIsProgram;
00671 extern PFNGLISQUERYPROC glIsQuery;
00672 extern PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00673 extern PFNGLISSAMPLERPROC glIsSampler;
00674 extern PFNGLISSHADERPROC glIsShader;
00675 extern PFNGLISSTATENVPROC glIsStateNV;
00676 extern PFNGLISSYNCPROC glIsSync;
00677 extern PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00678 extern PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00679 extern PFNGLISTEXTUREPROC glIsTexture;
00680 extern PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00681 extern PFNGLISVERTEXARRAYPROC glIsVertexArray;
00682 extern PFNGLLABELOBJECTTEXTPROC glLabelObjectEXT;
00683 extern PFNGLLINEWIDTHPROC glLineWidth;
00684 extern PFNGLLINKPROGRAMPROC glLinkProgram;
00685 extern PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC glListDrawCommandsStatesClientNV;
00686 extern PFNGLLOGICOPPROC glLogicOp;
00687 extern PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00688 extern PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00689 extern PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00690 extern PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00691 extern PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00692 extern PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00693 extern PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00694 extern PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00695 extern PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00696 extern PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00697 extern PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00698 extern PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00699 extern PFNGLMAPBUFFERPROC glMapBuffer;
00700 extern PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00701 extern PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00702 extern PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00703 extern PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00704 extern PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00705 extern PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00706 extern PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00707 extern PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00708 extern PFNGLMATRIXLOADDEXTPROC glMatrixLoaddEXT;
00709 extern PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00710 extern PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00711 extern PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00712 extern PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00713 extern PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00714 extern PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00715 extern PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00716 extern PFNGLMATRIXMULTDEXTPROC glMatrixMultdEXT;
00717 extern PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00718 extern PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00719 extern PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00720 extern PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00721 extern PFNGLMATRIXORTHODEXTPROC glMatrixOrthoEXT;
00722 extern PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00723 extern PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00724 extern PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00725 extern PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00726 extern PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00727 extern PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00728 extern PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00729 extern PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00730 extern PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00731 extern PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00732 extern PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00733 extern PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00734 extern PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00735 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00736 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00737 extern PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00738 extern PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00739 extern PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00740 extern PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00741 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00742 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00743 extern PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00744 extern PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00745 extern PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00746 extern PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00747 extern PFNGLMULTITEXCOORDPOINTINTEREXTPROC glMultiTexCoordPointerEXT;
00748 extern PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00749 extern PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00750 extern PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00751 extern PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00752 extern PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00753 extern PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00754 extern PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00755 extern PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;

```

```
00756 extern PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00757 extern PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00758 extern PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00759 extern PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00760 extern PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00761 extern PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00762 extern PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00763 extern PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00764 extern PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameteriivEXT;
00765 extern PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameteriuiEXT;
00766 extern PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00767 extern PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00768 extern PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00769 extern PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00770 extern PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00771 extern PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00772 extern PFNGLNAMEDBUFFERDATAAPROC glNamedBufferData;
00773 extern PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00774 extern PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00775 extern PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00776 extern PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00777 extern PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00778 extern PFNGLNAMEDBUFFERSUBDATAAPROC glNamedBufferSubData;
00779 extern PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00780 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00781 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00782 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00783 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00784 extern PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00785 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00786 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00787 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00788 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00789 extern PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00790 extern PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00791 extern PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00792 extern PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00793 extern PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00794 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00795 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00796 extern PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00797 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00798 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00799 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00800 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00801 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00802 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00803 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00804 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00805 extern PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00806 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC glNamedProgramLocalParametersI4ivEXT;
00807 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00808 extern PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00809 extern PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00810 extern PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEEXTPROC
glNamedRenderbufferStorageMultisampleCoverageEXT;
00811 extern PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00812 extern PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00813 extern PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00814 extern PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00815 extern PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00816 extern PFNGLOBJEC LABELPROC glObjectLabel;
00817 extern PFNGLOBJECTPTRLABELPROC glObjectPtrLabel;
00818 extern PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00819 extern PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00820 extern PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00821 extern PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00822 extern PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00823 extern PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00824 extern PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00825 extern PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00826 extern PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00827 extern PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00828 extern PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00829 extern PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00830 extern PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00831 extern PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00832 extern PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00833 extern PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00834 extern PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00835 extern PFNGLPATHSTRINGNVPROC glPathStringNV;
00836 extern PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00837 extern PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00838 extern PFNGLPAUSESTRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00839 extern PFNGLPIXELSTOREFPROC glPixelStoref;
00840 extern PFNGLPIXELSTOREIPROC glPixelStorei;
00841 extern PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
```

```

00842 extern PFNGLPOINTPARAMETERFPROC glPointParameterf;
00843 extern PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00844 extern PFNGLPOINTPARAMETERIPROC glPointParameteri;
00845 extern PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00846 extern PFNGLPOINTSIZEPROC glPointSize;
00847 extern PFNGLPOLYGONMODEPROC glPolygonMode;
00848 extern PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00849 extern PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00850 extern PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00851 extern PFNGLPOPGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00852 extern PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00853 extern PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00854 extern PFNGLPROGRAMBINARYPROC glProgramBinary;
00855 extern PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00856 extern PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00857 extern PFNGLPROGRAMPATHFRAGMENTINPUTGENVPROC glProgramPathFragmentInputGenNV;
00858 extern PFNGLPROGRAMUNIFORM1DEXTPROC glProgramUniform1dEXT;
00859 extern PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00860 extern PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00861 extern PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00862 extern PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00863 extern PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00864 extern PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00865 extern PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00866 extern PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00867 extern PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00868 extern PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00869 extern PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00870 extern PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00871 extern PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00872 extern PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00873 extern PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00874 extern PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00875 extern PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00876 extern PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00877 extern PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00878 extern PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00879 extern PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00880 extern PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00881 extern PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00882 extern PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00883 extern PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00884 extern PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00885 extern PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00886 extern PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00887 extern PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00888 extern PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00889 extern PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00890 extern PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00891 extern PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00892 extern PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00893 extern PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00894 extern PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00895 extern PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00896 extern PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00897 extern PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00898 extern PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00899 extern PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00900 extern PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00901 extern PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00902 extern PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00903 extern PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00904 extern PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00905 extern PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00906 extern PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00907 extern PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00908 extern PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00909 extern PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00910 extern PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00911 extern PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00912 extern PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00913 extern PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00914 extern PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00915 extern PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00916 extern PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00917 extern PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00918 extern PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00919 extern PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00920 extern PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00921 extern PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00922 extern PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00923 extern PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00924 extern PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00925 extern PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00926 extern PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00927 extern PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00928 extern PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;

```



```
00929 extern PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00930 extern PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00931 extern PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00932 extern PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dvEXT;
00933 extern PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00934 extern PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00935 extern PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00936 extern PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00937 extern PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00938 extern PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00939 extern PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00940 extern PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00941 extern PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00942 extern PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00943 extern PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00944 extern PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00945 extern PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00946 extern PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00947 extern PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00948 extern PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00949 extern PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00950 extern PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00951 extern PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00952 extern PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00953 extern PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00954 extern PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00955 extern PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00956 extern PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00957 extern PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00958 extern PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00959 extern PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00960 extern PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00961 extern PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00962 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00963 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00964 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00965 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00966 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00967 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00968 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00969 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00970 extern PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00971 extern PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00972 extern PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00973 extern PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00974 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00975 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00976 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00977 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00978 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00979 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00980 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00981 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00982 extern PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00983 extern PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00984 extern PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00985 extern PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00986 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00987 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00988 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00989 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00990 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00991 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00992 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00993 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00994 extern PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00995 extern PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00996 extern PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00997 extern PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC glPushClientAttribDefaultEXT;
00998 extern PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00999 extern PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
01000 extern PFNGLQUERYCOUNTERPROC glQueryCounter;
01001 extern PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
01002 extern PFNGLREADBUFFERPROC glReadBuffer;
01003 extern PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
01004 extern PFNGLREADNPIXELSPROC glReadnPixels;
01005 extern PFNGLREADPIXELSPROC glReadPixels;
01006 extern PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
01007 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
01008 extern PFNGLRENDERBUFFERSTAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
01009 extern PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
01010 extern PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
01011 extern PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01012 extern PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01013 extern PFNGLSAMPLEMASKIPROC glSampleMaski;
01014 extern PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01015 extern PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
```

```

01016 extern PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01017 extern PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01018 extern PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01019 extern PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01020 extern PFNGLSCISSORARRAYVPROC glScissorArrayv;
01021 extern PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01022 extern PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01023 extern PFNGLSCISSORPROC glScissor;
01024 extern PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01025 extern PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01026 extern PFNGLSHADERBINARYPROC glShaderBinary;
01027 extern PFNGLSHADERSOURCEPROC glShaderSource;
01028 extern PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01029 extern PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01030 extern PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01031 extern PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01032 extern PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01033 extern PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01034 extern PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01035 extern PFNGLSTENCILFUNCPROC glStencilFunc;
01036 extern PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01037 extern PFNGLSTENCILMASKPROC glStencilMask;
01038 extern PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01039 extern PFNGLSTENCILOPPROC glStencilOp;
01040 extern PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01041 extern PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01042 extern PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01043 extern PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01044 extern PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01045 extern PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01046 extern PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01047 extern PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01048 extern PFNGLTEXBUFFERARBPROC glTexBufferARB;
01049 extern PFNGLTEXBUFFERPROC glTexBuffer;
01050 extern PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01051 extern PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01052 extern PFNGLTEXIMAGE1DPROC glTexImage1D;
01053 extern PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01054 extern PFNGLTEXIMAGE2DPROC glTexImage2D;
01055 extern PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01056 extern PFNGLTEXIMAGE3DPROC glTexImage3D;
01057 extern PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01058 extern PFNGLTEXPARAMETERFPROC glTexParameterf;
01059 extern PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01060 extern PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01061 extern PFNGLTEXPARAMETERIPROC glTexParameterIi;
01062 extern PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01063 extern PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01064 extern PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01065 extern PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01066 extern PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01067 extern PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01068 extern PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01069 extern PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01070 extern PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01071 extern PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01072 extern PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01073 extern PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01074 extern PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01075 extern PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01076 extern PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01077 extern PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01078 extern PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01079 extern PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01080 extern PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01081 extern PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01082 extern PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01083 extern PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01084 extern PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01085 extern PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01086 extern PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01087 extern PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01088 extern PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01089 extern PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01090 extern PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01091 extern PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01092 extern PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01093 extern PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01094 extern PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01095 extern PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01096 extern PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01097 extern PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01098 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01099 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01100 extern PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01101 extern PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01102 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;

```

```
01103 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01104 extern PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01105 extern PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01106 extern PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01107 extern PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01108 extern PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01109 extern PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01110 extern PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01111 extern PFNGLTEXTUREVIEWPROC glTextureView;
01112 extern PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01113 extern PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01114 extern PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01115 extern PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01116 extern PFNGLUNIFORM1DPROC glUniform1d;
01117 extern PFNGLUNIFORM1DVPROC glUniform1dv;
01118 extern PFNGLUNIFORM1FPROC glUniform1f;
01119 extern PFNGLUNIFORM1FVPROC glUniform1fv;
01120 extern PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01121 extern PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01122 extern PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01123 extern PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01124 extern PFNGLUNIFORM1IPROC glUniform1i;
01125 extern PFNGLUNIFORM1IVPROC glUniform1iv;
01126 extern PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01127 extern PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01128 extern PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01129 extern PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01130 extern PFNGLUNIFORM1UIPROC glUniform1ui;
01131 extern PFNGLUNIFORM1UIVPROC glUniform1uiv;
01132 extern PFNGLUNIFORM2DPROC glUniform2d;
01133 extern PFNGLUNIFORM2DVPROC glUniform2dv;
01134 extern PFNGLUNIFORM2FPROC glUniform2f;
01135 extern PFNGLUNIFORM2FVPROC glUniform2fv;
01136 extern PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01137 extern PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01138 extern PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01139 extern PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01140 extern PFNGLUNIFORM2IPROC glUniform2i;
01141 extern PFNGLUNIFORM2IVPROC glUniform2iv;
01142 extern PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01143 extern PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01144 extern PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01145 extern PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01146 extern PFNGLUNIFORM2UIPROC glUniform2ui;
01147 extern PFNGLUNIFORM2UIVPROC glUniform2uiv;
01148 extern PFNGLUNIFORM3DPROC glUniform3d;
01149 extern PFNGLUNIFORM3DVPROC glUniform3dv;
01150 extern PFNGLUNIFORM3FPROC glUniform3f;
01151 extern PFNGLUNIFORM3FVPROC glUniform3fv;
01152 extern PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01153 extern PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01154 extern PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01155 extern PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01156 extern PFNGLUNIFORM3IPROC glUniform3i;
01157 extern PFNGLUNIFORM3IVPROC glUniform3iv;
01158 extern PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01159 extern PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01160 extern PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01161 extern PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01162 extern PFNGLUNIFORM3UIPROC glUniform3ui;
01163 extern PFNGLUNIFORM3UIVPROC glUniform3uiv;
01164 extern PFNGLUNIFORM4DPROC glUniform4d;
01165 extern PFNGLUNIFORM4DVPROC glUniform4dv;
01166 extern PFNGLUNIFORM4FPROC glUniform4f;
01167 extern PFNGLUNIFORM4FVPROC glUniform4fv;
01168 extern PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01169 extern PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01170 extern PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01171 extern PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01172 extern PFNGLUNIFORM4IPROC glUniform4i;
01173 extern PFNGLUNIFORM4IVPROC glUniform4iv;
01174 extern PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01175 extern PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01176 extern PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01177 extern PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01178 extern PFNGLUNIFORM4UIPROC glUniform4ui;
01179 extern PFNGLUNIFORM4UIVPROC glUniform4uiv;
01180 extern PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01181 extern PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01182 extern PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01183 extern PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01184 extern PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01185 extern PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01186 extern PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01187 extern PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01188 extern PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01189 extern PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
```

```

01190 extern PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01191 extern PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01192 extern PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01193 extern PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01194 extern PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01195 extern PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01196 extern PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01197 extern PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01198 extern PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01199 extern PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01200 extern PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01201 extern PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01202 extern PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01203 extern PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01204 extern PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01205 extern PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01206 extern PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01207 extern PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01208 extern PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01209 extern PFNGLUSEPROGRAMPROC glUseProgram;
01210 extern PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01211 extern PFNGLUSERESOURCEPROGRAMEXTPROC glUseProgramEXT;
01212 extern PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01213 extern PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01214 extern PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01215 extern PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01216 extern PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01217 extern PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01218 extern PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01219 extern PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01220 extern PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01221 extern PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01222 extern PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01223 extern PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01224 extern PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01225 extern PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01226 extern PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01227 extern PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01228 extern PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01229 extern PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01230 extern PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01231 extern PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01232 extern PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01233 extern PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01234 extern PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01235 extern PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01236 extern PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01237 extern PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01238 extern PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01239 extern PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01240 extern PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01241 extern PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01242 extern PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01243 extern PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01244 extern PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01245 extern PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01246 extern PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01247 extern PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01248 extern PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01249 extern PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01250 extern PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01251 extern PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01252 extern PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01253 extern PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01254 extern PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01255 extern PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01256 extern PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01257 extern PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01258 extern PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01259 extern PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01260 extern PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01261 extern PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01262 extern PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01263 extern PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01264 extern PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01265 extern PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbvp;
01266 extern PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01267 extern PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01268 extern PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01269 extern PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubvp;
01270 extern PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01271 extern PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusvp;
01272 extern PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01273 extern PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01274 extern PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubvp;
01275 extern PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01276 extern PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usvp;

```



```

01277 extern PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01278 extern PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01279 extern PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01280 extern PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01281 extern PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01282 extern PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01283 extern PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01284 extern PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01285 extern PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01286 extern PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01287 extern PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01288 extern PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01289 extern PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01290 extern PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01291 extern PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01292 extern PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01293 extern PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01294 extern PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01295 extern PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01296 extern PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01297 extern PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01298 extern PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01299 extern PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01300 extern PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01301 extern PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01302 extern PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01303 extern PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01304 extern PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01305 extern PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01306 extern PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01307 extern PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01308 extern PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01309 extern PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01310 extern PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01311 extern PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01312 extern PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01313 extern PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01314 extern PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01315 extern PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01316 extern PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01317 extern PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01318 extern PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01319 extern PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01320 extern PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01321 extern PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01322 extern PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01323 extern PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01324 extern PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01325 extern PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01326 extern PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01327 extern PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01328 extern PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01329 extern PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01330 extern PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01331 extern PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01332 extern PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01333 extern PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01334 extern PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01335 extern PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01336 extern PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01337 extern PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01338 extern PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01339 extern PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01340 extern PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01341 extern PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01342 extern PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01343 extern PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01344 extern PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01345 extern PFNGLVIEWPORTARRAYVPROC glViewportArrayv;
01346 extern PFNGLVIEWPORTINDEXEDFPROC glViewportIndexdf;
01347 extern PFNGLVIEWPORTINDEXEDFVPROC glViewportIndexdfv;
01348 extern PFNGLVIEWPORTPOSITIONWSALENVPROC glViewportPositionWScaleNV;
01349 extern PFNGLVIEWPORTPROC glViewport;
01350 extern PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01351 extern PFNGLWAITSYNCPROC glWaitSync;
01352 extern PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01353 extern PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01354 extern PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01355 #endif
01356
01358
01362 namespace gg
01363 {
01367     enum BindingPoints
01368     {
01369         LightBindingPoint = 0,
01370         MaterialBindingPoint,

```

```

01371     };
01372
01376     extern GLint ggBufferAlignment;
01377
01384     extern void ggInit();
01385
01395     extern void _ggError(const std::string& name = "", unsigned int line = 0);
01396
01407 #if defined(DEBUG)
01408 #   define ggError() gg::_ggError(__FILE__, __LINE__)
01409 #else
01410 #   define ggError()
01411 #endif
01412
01422     extern void _ggFBOError(const std::string& name = "", unsigned int line = 0);
01423
01434 #if defined(DEBUG)
01435 #   define ggFBOError() gg::_ggFBOError(__FILE__, __LINE__)
01436 #else
01437 #   define ggFBOError()
01438 #endif
01439
01447     inline void ggCross(GLfloat* c, const GLfloat* a, const GLfloat* b)
01448     {
01449         c[0] = a[1] * b[2] - a[2] * b[1];
01450         c[1] = a[2] * b[0] - a[0] * b[2];
01451         c[2] = a[0] * b[1] - a[1] * b[0];
01452     }
01453
01461     inline GLfloat ggDot3(const GLfloat* a, const GLfloat* b)
01462     {
01463         return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
01464     }
01465
01472     inline GLfloat ggLength3(const GLfloat* a)
01473     {
01474         return sqrtf(ggDot3(a, a));
01475     }
01476
01484     inline GLfloat ggDistance3(const GLfloat* a, const GLfloat* b)
01485     {
01486         const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], 0.0f };
01487         return ggLength3(c);
01488     }
01489
01496     inline void ggNormalize3(const GLfloat* a, GLfloat* b)
01497     {
01498         const GLfloat l { ggLength3(a) };
01499         assert(l > 0.0f);
01500         b[0] = a[0] / l;
01501         b[1] = a[1] / l;
01502         b[2] = a[2] / l;
01503     }
01504
01510     inline void ggNormalize3(GLfloat* a)
01511     {
01512         const GLfloat l { ggLength3(a) };
01513         assert(l > 0.0f);
01514         a[0] /= l;
01515         a[1] /= l;
01516         a[2] /= l;
01517     }
01518
01526     inline GLfloat ggDot4(const GLfloat* a, const GLfloat* b)
01527     {
01528         return a[0] * b[0] + a[1] * b[1] + a[2] * b[2] + a[3] * b[3];
01529     }
01530
01537     inline GLfloat ggLength4(const GLfloat* a)
01538     {
01539         return sqrtf(ggDot4(a, a));
01540     }
01541
01549     inline GLfloat ggDistance4(const GLfloat* a, const GLfloat* b)
01550     {
01551         const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], a[3] - b[3] };
01552         return ggLength4(c);
01553     }
01554
01561     inline void ggNormalize4(const GLfloat* a, GLfloat* b)
01562     {
01563         const GLfloat l { ggLength4(a) };
01564         assert(l > 0.0f);
01565         b[0] = a[0] / l;
01566         b[1] = a[1] / l;
01567         b[2] = a[2] / l;
01568         b[3] = a[3] / l;

```

```

01569     }
01570
01576     inline void ggNormalize4(GLfloat* a)
01577     {
01578         const GLfloat l{ ggLength4(a) };
01579         assert(l > 0.0f);
01580         a[0] /= l;
01581         a[1] /= l;
01582         a[2] /= l;
01583         a[3] /= l;
01584     }
01585
01589     class GgVector : public std::array<GLfloat, 4>
01590     {
01591     public:
01592
01596         GgVector() = default;
01597
01606         constexpr GgVector(GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3) :
01607             std::array<GLfloat, 4>{ v0, v1, v2, v3 }
01608         {
01609         }
01610
01616         constexpr GgVector(const GLfloat* a) :
01617             GgVector{ a[0], a[1], a[2], a[3] }
01618         {
01619         }
01620
01626         constexpr GgVector(GLfloat c) :
01627             GgVector{ c, c, c, c }
01628         {
01629         }
01630
01636         GgVector(const GgVector& v) = default;
01637
01643         GgVector(GgVector&& v) = default;
01644
01648         ~GgVector() = default;
01649
01653         GgVector& operator=(const GgVector& v) = default;
01654
01658         GgVector& operator=(GgVector&& v) = default;
01659
01666         constexpr GgVector operator+(const GgVector& v) const noexcept
01667         {
01668             return GgVector{ (*this)[0] + v[0], (*this)[1] + v[1], (*this)[2] + v[2], (*this)[3] + v[3] };
01669         }
01670
01677         constexpr GgVector operator+(GLfloat c) const noexcept
01678         {
01679             return GgVector{ (*this)[0] + c, (*this)[1] + c, (*this)[2] + c, (*this)[3] + c };
01680         }
01681
01688         constexpr GgVector& operator+=(const GgVector& v) noexcept
01689         {
01690             (*this)[0] += v[0];
01691             (*this)[1] += v[1];
01692             (*this)[2] += v[2];
01693             (*this)[3] += v[3];
01694             return *this;
01695         }
01696
01703         constexpr GgVector& operator+=(GLfloat c) noexcept
01704         {
01705             (*this)[0] += c;
01706             (*this)[1] += c;
01707             (*this)[2] += c;
01708             (*this)[3] += c;
01709             return *this;
01710         }
01711
01718         constexpr GgVector operator-(const GgVector& v) const noexcept
01719         {
01720             return GgVector{ (*this)[0] - v[0], (*this)[1] - v[1], (*this)[2] - v[2], (*this)[3] - v[3] };
01721         }
01722
01729         constexpr GgVector operator-(GLfloat c) const noexcept
01730         {
01731             return GgVector{ (*this)[0] - c, (*this)[1] - c, (*this)[2] - c, (*this)[3] - c };
01732         }
01733
01740         constexpr GgVector& operator-=(const GgVector& v) noexcept
01741         {
01742             (*this)[0] -= v[0];
01743             (*this)[1] -= v[1];
01744             (*this)[2] -= v[2];
01745             (*this)[3] -= v[3];

```

```

01746     return *this;
01747 }
01748
01755 constexpr GgVector& operator--(GLfloat c) noexcept
01756 {
01757     (*this)[0] -= c;
01758     (*this)[1] -= c;
01759     (*this)[2] -= c;
01760     (*this)[3] -= c;
01761     return *this;
01762 }
01763
01770 constexpr GgVector operator*(const GgVector& v) const noexcept
01771 {
01772     return GgVector{ (*this)[0] * v[0], (*this)[1] * v[1], (*this)[2] * v[2], (*this)[3] * v[3] };
01773 }
01774
01781 constexpr GgVector operator*(GLfloat c) const noexcept
01782 {
01783     return GgVector{ (*this)[0] * c, (*this)[1] * c, (*this)[2] * c, (*this)[3] * c };
01784 }
01785
01792 constexpr GgVector& operator*=(const GgVector& v) noexcept
01793 {
01794     (*this)[0] *= v[0];
01795     (*this)[1] *= v[1];
01796     (*this)[2] *= v[2];
01797     (*this)[3] *= v[3];
01798     return *this;
01799 }
01800
01807 constexpr GgVector& operator*=(GLfloat c) noexcept
01808 {
01809     (*this)[0] *= c;
01810     (*this)[1] *= c;
01811     (*this)[2] *= c;
01812     (*this)[3] *= c;
01813     return *this;
01814 }
01815
01822 constexpr GgVector operator/(const GgVector& v) const noexcept
01823 {
01824     return GgVector{ (*this)[0] / v[0], (*this)[1] / v[1], (*this)[2] / v[2], (*this)[3] / v[3] };
01825 }
01826
01833 constexpr GgVector operator/(GLfloat c) const noexcept
01834 {
01835     return GgVector{ (*this)[0] / c, (*this)[1] / c, (*this)[2] / c, (*this)[3] / c };
01836 }
01837
01844 constexpr GgVector& operator/=(const GgVector& v) noexcept
01845 {
01846     (*this)[0] /= v[0];
01847     (*this)[1] /= v[1];
01848     (*this)[2] /= v[2];
01849     (*this)[3] /= v[3];
01850     return *this;
01851 }
01852
01859 constexpr GgVector& operator/=(GLfloat c) noexcept
01860 {
01861     (*this)[0] /= c;
01862     (*this)[1] /= c;
01863     (*this)[2] /= c;
01864     (*this)[3] /= c;
01865     return *this;
01866 }
01867
01874 inline GLfloat dot3(const GgVector& v) const
01875 {
01876     return ggDot3(data(), v.data());
01877 }
01878
01884 inline GLfloat length3() const
01885 {
01886     return ggLength3(data());
01887 }
01888
01895 inline GLfloat distance3(const GgVector& v) const
01896 {
01897     return ggDistance3(data(), v.data());
01898 }
01899
01905 inline GgVector normalize3() const
01906 {
01907     GgVector b;
01908     ggNormalize3(data(), b.data());

```



```

01909     return b;
01910 }
01911
01918 inline GLfloat dot4(const GgVector& v) const
01919 {
01920     return ggDot4(data(), v.data());
01921 }
01922
01928 inline GLfloat length4() const
01929 {
01930     return ggLength4(data());
01931 }
01932
01939 inline GLfloat distance4(const GgVector& v) const
01940 {
01941     return ggDistance4(data(), v.data());
01942 }
01943
01949 inline GgVector normalize4() const
01950 {
01951     GgVector b;
01952     ggNormalize4(data(), b.data());
01953     return b;
01954 }
01955 };
01956
01963 inline const GgVector& operator+(const GgVector& v)
01964 {
01965     return v;
01966 }
01967
01975 inline GgVector operator+(GLfloat a, const GgVector& b)
01976 {
01977     return GgVector{ a + b[0], a + b[1], a + b[2], a + b[3] };
01978 }
01979
01986 inline const GgVector operator-(const GgVector& v)
01987 {
01988     return GgVector{ -v[0], -v[1], -v[2], -v[3] };
01989 }
01990
01998 inline GgVector operator-(GLfloat a, const GgVector& b)
01999 {
02000     return GgVector{ a - b[0], a - b[1], a - b[2], a - b[3] };
02001 }
02002
02010 inline GgVector operator*(GLfloat a, const GgVector& b)
02011 {
02012     return GgVector{ a * b[0], a * b[1], a * b[2], a * b[3] };
02013 }
02014
02022 inline GgVector operator/(GLfloat a, const GgVector& b)
02023 {
02024     return GgVector{ a / b[0], a / b[1], a / b[2], a / b[3] };
02025 }
02026
02037 inline GgVector ggCross(const GgVector& a, const GgVector& b)
02038 {
02039     GgVector c;
02040     ggCross(c.data(), a.data(), b.data());
02041     return c;
02042 }
02043
02051 inline GLfloat ggDot3(const GgVector& a, const GgVector& b)
02052 {
02053     return ggDot3(a.data(), b.data());
02054 }
02055
02062 inline GLfloat ggLength3(const GgVector& a)
02063 {
02064     return ggLength3(a.data());
02065 }
02066
02074 inline GLfloat ggDistance3(const GgVector& a, const GgVector& b)
02075 {
02076     return ggDistance3(a.data(), b.data());
02077 }
02078
02088 inline GgVector ggNormalize3(const GgVector& a)
02089 {
02090     GgVector b;
02091     ggNormalize3(a.data(), b.data());
02092     return b;
02093 }
02094
02103 inline void ggNormalize3(GgVector* a)
02104 {

```

```

02105     ggNormalize3(a->data());
02106 }
02107
02115 inline GLfloat ggDot4(const GgVector& a, const GgVector& b)
02116 {
02117     return ggDot4(a.data(), b.data());
02118 }
02119
02126 inline GLfloat ggLength4(const GgVector& a)
02127 {
02128     return ggLength4(a.data());
02129 }
02130
02138 inline GLfloat ggDistance4(const GgVector& a, const GgVector& b)
02139 {
02140     return ggDistance4(a.data(), b.data());
02141 }
02142
02149 inline GgVector ggNormalize4(const GgVector& a)
02150 {
02151     GgVector b;
02152     ggNormalize4(a.data(), b.data());
02153     return b;
02154 }
02155
02161 inline void ggNormalize4(GgVector* a)
02162 {
02163     ggNormalize4(a->data());
02164 }
02165
02169 class GgMatrix : public std::array<GLfloat, 16>
02170 {
02171     // 行列 a とベクトル b の積をベクトル c に格納する
02172     void projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02173
02174     // 行列 a と行列 b の積を行列 c に格納する
02175     void multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02176
02177 public:
02178
02182     GgMatrix() = default;
02183
02204     constexpr GgMatrix(
02205         GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03,
02206         GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13,
02207         GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23,
02208         GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33
02209     ) :
02210         std::array<GLfloat, 16>{ m00, m01, m02, m03, m10, m11, m12, m13, m20, m21, m22, m23, m30, m31,
02211             m32, m33 }
02212     {
02213     }
02214
02219     constexpr GgMatrix(const GLfloat* a) :
02220         GgMatrix{ a[0], a[1], a[2], a[3], a[4], a[5], a[6], a[7], a[8], a[9], a[10], a[11], a[12],
02221             a[13], a[14], a[15] }
02222     {
02223     }
02224
02229     constexpr GgMatrix(GLfloat c) :
02230         GgMatrix{ c, c, c, c, c, c, c, c, c, c, c, c, c, c, c, c }
02231     {
02232     }
02233
02239     GgMatrix(const GgMatrix& m) = default;
02240
02246     GgMatrix(GgMatrix&& m) = default;
02247
02251     ~GgMatrix() = default;
02252
02256     GgMatrix& operator=(const GgMatrix& m) = default;
02257
02261     GgMatrix& operator=(GgMatrix&& m) = default;
02262
02269     GgMatrix& operator=(const GLfloat* a)
02270     {
02271         *this = GgMatrix(a);
02272         return *this;
02273     }
02274
02281     GgMatrix operator+(const GLfloat* a) const
02282     {
02283         GgMatrix t;
02284         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] + a[i];
02285         return t;
02286     }
02287

```

```

02294 GgMatrix operator+(const GgMatrix& m) const
02295 {
02296     return operator+(m.data());
02297 }
02298
02305 GgMatrix& operator+=(const GLfloat* a)
02306 {
02307     for (std::size_t i = 0; i < size(); ++i) data()[i] += a[i];
02308     return *this;
02309 }
02310
02317 GgMatrix& operator+=(const GgMatrix& m)
02318 {
02319     return operator+=(m.data());
02320 }
02321
02328 GgMatrix operator-(const GLfloat* a) const
02329 {
02330     GgMatrix t;
02331     for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] - a[i];
02332     return t;
02333 }
02334
02341 GgMatrix operator-(const GgMatrix& m) const
02342 {
02343     return operator-(m.data());
02344 }
02345
02352 GgMatrix& operator-=(const GLfloat* a)
02353 {
02354     for (std::size_t i = 0; i < size(); ++i) data()[i] -= a[i];
02355     return *this;
02356 }
02357
02364 GgMatrix& operator-=(const GgMatrix& m)
02365 {
02366     return operator-=(m.data());
02367 }
02368
02375 GgMatrix operator*(const GLfloat* a) const
02376 {
02377     GgMatrix t;
02378     multiply(t.data(), data(), a);
02379     return t;
02380 }
02381
02388 GgMatrix operator*(const GgMatrix& m) const
02389 {
02390     return operator*(m.data());
02391 }
02392
02399 GgMatrix& operator*=(const GLfloat* a)
02400 {
02401     *this = operator*(a);
02402     return *this;
02403 }
02404
02411 GgMatrix& operator*=(const GgMatrix& m)
02412 {
02413     return operator*=(m.data());
02414 }
02415
02422 GgMatrix operator/(const GLfloat* a) const
02423 {
02424     GgMatrix i, t;
02425     i.loadInvert(a);
02426     multiply(t.data(), data(), i.data());
02427     return t;
02428 }
02429
02436 GgMatrix operator/(const GgMatrix& m) const
02437 {
02438     return operator/(m.data());
02439 }
02440
02447 GgMatrix& operator/=(const GLfloat* a)
02448 {
02449     *this = operator/(a);
02450     return *this;
02451 }
02452
02459 GgMatrix& operator/=(const GgMatrix& m)
02460 {
02461     return operator/=(m.data());
02462 }
02463
02467 GgMatrix& loadIdentity();

```

```

02468
02478 GgMatrix& loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02479
02486 GgMatrix& loadTranslate(const GLfloat* t)
02487 {
02488     return loadTranslate(t[0], t[1], t[2]);
02489 }
02490
02497 GgMatrix& loadTranslate(const GgVector& t)
02498 {
02499     return loadTranslate(t[0], t[1], t[2], t[3]);
02500 }
02501
02511 GgMatrix& loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02512
02519 GgMatrix& loadScale(const GLfloat* s)
02520 {
02521     return loadScale(s[0], s[1], s[2]);
02522 }
02523
02530 GgMatrix& loadScale(const GgVector& s)
02531 {
02532     return loadScale(s[0], s[1], s[2], s[3]);
02533 }
02534
02541 GgMatrix& loadRotateX(GLfloat a);
02542
02549 GgMatrix& loadRotateY(GLfloat a);
02550
02557 GgMatrix& loadRotateZ(GLfloat a);
02558
02568 GgMatrix& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
02569
02577 GgMatrix& loadRotate(const GLfloat* r, GLfloat a)
02578 {
02579     return loadRotate(r[0], r[1], r[2], a);
02580 }
02581
02589 GgMatrix& loadRotate(const GgVector& r, GLfloat a)
02590 {
02591     return loadRotate(r[0], r[1], r[2], a);
02592 }
02593
02600 GgMatrix& loadRotate(const GLfloat* r)
02601 {
02602     return loadRotate(r[0], r[1], r[2], r[3]);
02603 }
02604
02611 GgMatrix& loadRotate(const GgVector& r)
02612 {
02613     return loadRotate(r[0], r[1], r[2], r[3]);
02614 }
02615
02630 GgMatrix& loadLookat(GLfloat ex, GLfloat ey, GLfloat ez,
02631     GLfloat tx, GLfloat ty, GLfloat tz,
02632     GLfloat ux, GLfloat uy, GLfloat uz);
02633
02642 GgMatrix& loadLookat(const GLfloat* e, const GLfloat* t, const GLfloat* u)
02643 {
02644     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02645 }
02646
02655 GgMatrix& loadLookat(const GgVector& e, const GgVector& t, const GgVector& u)
02656 {
02657     return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02658 }
02659
02671 GgMatrix& loadOrthogonal(GLfloat left, GLfloat right,
02672     GLfloat bottom, GLfloat top,
02673     GLfloat zNear, GLfloat zFar);
02674
02686 GgMatrix& loadFrustum(GLfloat left, GLfloat right,
02687     GLfloat bottom, GLfloat top,
02688     GLfloat zNear, GLfloat zFar);
02689
02699 GgMatrix& loadPerspective(GLfloat fovy, GLfloat aspect,
02700     GLfloat zNear, GLfloat zFar);
02701
02708 GgMatrix& loadTranspose(const GLfloat* a);
02709
02716 GgMatrix& loadTranspose(const GgMatrix& m)
02717 {
02718     return loadTranspose(m.data());
02719 }
02720
02727 GgMatrix& loadInvert(const GLfloat* a);
02728

```

```

02735     GgMatrix& loadInvert(const GgMatrix& m)
02736     {
02737         return loadInvert(m.data());
02738     }
02739
02746     GgMatrix& loadNormal(const GLfloat* a);
02747
02754     GgMatrix& loadNormal(const GgMatrix& m)
02755     {
02756         return loadNormal(m.data());
02757     }
02758
02768     GgMatrix translate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02769     {
02770         GgMatrix m;
02771         return operator*(m.loadTranslate(x, y, z, w));
02772     }
02773
02780     GgMatrix translate(const GLfloat* t) const
02781     {
02782         return translate(t[0], t[1], t[2]);
02783     }
02784
02791     GgMatrix translate(const GgVector& t) const
02792     {
02793         return translate(t[0], t[1], t[2], t[3]);
02794     }
02795
02805     GgMatrix scale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02806     {
02807         GgMatrix m;
02808         return operator*(m.loadScale(x, y, z, w));
02809     }
02810
02817     GgMatrix scale(const GLfloat* s) const
02818     {
02819         return scale(s[0], s[1], s[2]);
02820     }
02821
02828     GgMatrix scale(const GgVector& s) const
02829     {
02830         return scale(s[0], s[1], s[2], s[3]);
02831     }
02832
02839     GgMatrix rotateX(GLfloat a) const
02840     {
02841         GgMatrix m;
02842         return operator*(m.loadRotateX(a));
02843     }
02844
02851     GgMatrix rotateY(GLfloat a) const
02852     {
02853         GgMatrix m;
02854         return operator*(m.loadRotateY(a));
02855     }
02856
02863     GgMatrix rotateZ(GLfloat a) const
02864     {
02865         GgMatrix m;
02866         return operator*(m.loadRotateZ(a));
02867     }
02868
02878     GgMatrix rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
02879     {
02880         GgMatrix m;
02881         return operator*(m.loadRotate(x, y, z, a));
02882     }
02883
02891     GgMatrix rotate(const GLfloat* r, GLfloat a) const
02892     {
02893         return rotate(r[0], r[1], r[2], a);
02894     }
02895
02903     GgMatrix rotate(const GgVector& r, GLfloat a) const
02904     {
02905         return rotate(r[0], r[1], r[2], a);
02906     }
02907
02914     GgMatrix rotate(const GLfloat* r) const
02915     {
02916         return rotate(r[0], r[1], r[2], r[3]);
02917     }
02918
02925     GgMatrix rotate(const GgVector& r) const
02926     {
02927         return rotate(r[0], r[1], r[2], r[3]);
02928     }

```

```

02929
02944 GgMatrix lookout(
02945     GLfloat ex, GLfloat ey, GLfloat ez,
02946     GLfloat tx, GLfloat ty, GLfloat tz,
02947     GLfloat ux, GLfloat uy, GLfloat uz
02948 ) const
02949 {
02950     GgMatrix m;
02951     return operator*(m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz));
02952 }
02953
02962 GgMatrix lookout(const GLfloat* e, const GLfloat* t, const GLfloat* u) const
02963 {
02964     return lookout(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02965 }
02966
02975 GgMatrix lookout(const GgVector& e, const GgVector& t, const GgVector& u) const
02976 {
02977     return lookout(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02978 }
02979
02991 GgMatrix orthogonal(
02992     GLfloat left, GLfloat right,
02993     GLfloat bottom, GLfloat top,
02994     GLfloat zNear, GLfloat zFar
02995 ) const
02996 {
02997     GgMatrix m;
02998     return operator*(m.loadOrthogonal(left, right, bottom, top, zNear, zFar));
02999 }
03000
03012 GgMatrix frustum(
03013     GLfloat left, GLfloat right,
03014     GLfloat bottom, GLfloat top,
03015     GLfloat zNear, GLfloat zFar
03016 ) const
03017 {
03018     GgMatrix m;
03019     return operator*(m.loadFrustum(left, right, bottom, top, zNear, zFar));
03020 }
03021
03031 GgMatrix perspective(
03032     GLfloat fovy, GLfloat aspect,
03033     GLfloat zNear, GLfloat zFar
03034 ) const
03035 {
03036     GgMatrix m;
03037     return operator*(m.loadPerspective(fovy, aspect, zNear, zFar));
03038 }
03039
03045 GgMatrix transpose() const
03046 {
03047     GgMatrix t;
03048     return t.loadTranspose(*this);
03049 }
03050
03056 GgMatrix invert() const
03057 {
03058     GgMatrix t;
03059     return t.loadInvert(*this);
03060 }
03061
03067 GgMatrix normal() const
03068 {
03069     GgMatrix t;
03070     return t.loadNormal(*this);
03071 }
03072
03079 void projection(GLfloat* c, const GLfloat* v) const
03080 {
03081     projection(c, data(), v);
03082 }
03083
03090 void projection(GLfloat* c, const GgVector& v) const
03091 {
03092     projection(c, v.data());
03093 }
03094
03101 void projection(GgVector& c, const GLfloat* v) const
03102 {
03103     projection(c.data(), v);
03104 }
03105
03112 void projection(GgVector& c, const GgVector& v) const
03113 {
03114     projection(c.data(), v.data());
03115 }

```

```

03116
03123     GgVector operator*(const GgVector& v) const
03124     {
03125         GgVector c;
03126         projection(c, v);
03127         return c;
03128     }
03129
03135     const GLfloat* get() const
03136     {
03137         return data();
03138     }
03139
03145     void get(GLfloat* a) const
03146     {
03147         std::copy(data(), data() + size(), a);
03148     }
03149 };
03150
03156     inline GgMatrix ggIdentity()
03157     {
03158         GgMatrix t;
03159         return t.loadIdentity();
03160     }
03161
03171     inline GgMatrix ggTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03172     {
03173         GgMatrix m;
03174         return m.loadTranslate(x, y, z, w);
03175     }
03176
03183     inline GgMatrix ggTranslate(const GLfloat* t)
03184     {
03185         GgMatrix m;
03186         return m.loadTranslate(t[0], t[1], t[2]);
03187     }
03188
03195     inline GgMatrix ggTranslate(const GgVector& t)
03196     {
03197         GgMatrix m;
03198         return m.loadTranslate(t[0], t[1], t[2], t[3]);
03199     }
03200
03210     inline GgMatrix ggScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03211     {
03212         GgMatrix m;
03213         return m.loadScale(x, y, z, w);
03214     }
03215
03222     inline GgMatrix ggScale(const GLfloat* s)
03223     {
03224         GgMatrix m;
03225         return m.loadScale(s[0], s[1], s[2]);
03226     }
03227
03234     inline GgMatrix ggScale(const GgVector& s)
03235     {
03236         GgMatrix m;
03237         return m.loadScale(s[0], s[1], s[2], s[3]);
03238     }
03239
03246     inline GgMatrix ggRotateX(GLfloat a)
03247     {
03248         GgMatrix m;
03249         return m.loadRotateX(a);
03250     }
03251
03258     inline GgMatrix ggRotateY(GLfloat a)
03259     {
03260         GgMatrix m;
03261         return m.loadRotateY(a);
03262     }
03263
03270     inline GgMatrix ggRotateZ(GLfloat a)
03271     {
03272         GgMatrix m;
03273         return m.loadRotateZ(a);
03274     }
03275
03285     inline GgMatrix ggRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03286     {
03287         GgMatrix m;
03288         return m.loadRotate(x, y, z, a);
03289     }
03290
03298     inline GgMatrix ggRotate(const GLfloat* r, GLfloat a)
03299     {

```

```

03300     GgMatrix m;
03301     return m.loadRotate(r[0], r[1], r[2], a);
03302 }
03303
03311 inline GgMatrix ggRotate(const GgVector& r, GLfloat a)
03312 {
03313     GgMatrix m;
03314     return m.loadRotate(r[0], r[1], r[2], a);
03315 }
03316
03323 inline GgMatrix ggRotate(const GLfloat* r)
03324 {
03325     GgMatrix m;
03326     return m.loadRotate(r[0], r[1], r[2], r[3]);
03327 }
03328
03335 inline GgMatrix ggRotate(const GgVector& r)
03336 {
03337     GgMatrix m;
03338     return m.loadRotate(r[0], r[1], r[2], r[3]);
03339 }
03340
03355 inline GgMatrix ggLookat(
03356     GLfloat ex, GLfloat ey, GLfloat ez,    // 視点の位置
03357     GLfloat tx, GLfloat ty, GLfloat tz,    // 目標点の位置
03358     GLfloat ux, GLfloat uy, GLfloat uz    // 上方向のベクトル
03359 )
03360 {
03361     GgMatrix m;
03362     return m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz);
03363 }
03364
03373 inline GgMatrix ggLookat(
03374     const GLfloat* e,                    // 視点の位置
03375     const GLfloat* t,                    // 目標点の位置
03376     const GLfloat* u                      // 上方向のベクトル
03377 )
03378 {
03379     GgMatrix m;
03380     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03381 }
03382
03391 inline GgMatrix ggLookat(
03392     const GgVector& e,                    // 視点の位置
03393     const GgVector& t,                    // 目標点の位置
03394     const GgVector& u                      // 上方向のベクトル
03395 )
03396 {
03397     GgMatrix m;
03398     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03399 }
03400
03412 inline GgMatrix ggOrthogonal(GLfloat left, GLfloat right,
03413     GLfloat bottom, GLfloat top,
03414     GLfloat zNear, GLfloat zFar)
03415 {
03416     GgMatrix m;
03417     return m.loadOrthogonal(left, right, bottom, top, zNear, zFar);
03418 }
03419
03431 inline GgMatrix ggFrustum(
03432     GLfloat left, GLfloat right,
03433     GLfloat bottom, GLfloat top,
03434     GLfloat zNear, GLfloat zFar
03435 )
03436 {
03437     GgMatrix m;
03438     return m.loadFrustum(left, right, bottom, top, zNear, zFar);
03439 }
03440
03450 inline GgMatrix ggPerspective(
03451     GLfloat fovy, GLfloat aspect,
03452     GLfloat zNear, GLfloat zFar
03453 )
03454 {
03455     GgMatrix m;
03456     return m.loadPerspective(fovy, aspect, zNear, zFar);
03457 }
03458
03465 inline GgMatrix ggTranspose(const GgMatrix& m)
03466 {
03467     return m.transpose();
03468 }
03469
03476 inline GgMatrix ggInvert(const GgMatrix& m)
03477 {
03478     return m.invert();

```



```

03479     }
03480
03487     inline GgMatrix ggNormal(const GgMatrix& m)
03488     {
03489         return m.normal();
03490     }
03491
03495     class GgQuaternion : public GgVector
03496     {
03497     // GgQuaternion 型の四元数 p と四元数 q の積を四元数 r に求める
03498         void multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const;
03499
03500     // GgQuaternion 型の四元数 q が表す回転の変換行列を m に求める
03501         void toMatrix(GLfloat* m, const GLfloat* q) const;
03502
03503     // 回転の変換行列 m が表す四元数を q に求める
03504         void toQuaternion(GLfloat* q, const GLfloat* m) const;
03505
03506     // 球面線形補間 q と r を t で補間した四元数を p に求める
03507         void slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const;
03508
03509     public:
03510
03514         GgQuaternion() = default;
03515
03524         constexpr GgQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat w) :
03525             GgVector{ x, y, z, w }
03526         {
03527         }
03528
03534         constexpr GgQuaternion(GLfloat c) :
03535             GgQuaternion{ c, c, c, c }
03536         {
03537         }
03538
03544         GgQuaternion(const GLfloat* a) :
03545             GgQuaternion{ a[0], a[1], a[2], a[3] }
03546         {
03547         }
03548
03554         GgQuaternion(const GgVector& v) :
03555             GgQuaternion{ v[0], v[1], v[2], v[3] }
03556         {
03557         }
03558
03564         GgQuaternion(const GgQuaternion& q) = default;
03565
03571         GgQuaternion(GgQuaternion&& q) = default;
03572
03576         ~GgQuaternion() = default;
03577
03584         GgQuaternion& operator=(const GgQuaternion& q) = default;
03585
03592         GgQuaternion& operator=(GgQuaternion&& q) = default;
03593
03599         GLfloat norm() const
03600         {
03601             return ggLength4(*this);
03602         }
03603
03613         GgQuaternion& loadAdd(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03614         {
03615             (*this)[0] += x;
03616             (*this)[1] += y;
03617             (*this)[2] += z;
03618             (*this)[3] += w;
03619
03620             return *this;
03621         }
03622
03629         GgQuaternion& loadAdd(const GLfloat* a)
03630         {
03631             return loadAdd(a[0], a[1], a[2], a[3]);
03632         }
03633
03640         GgQuaternion& loadAdd(const GgVector& v)
03641         {
03642             return loadAdd(v[0], v[1], v[2], v[3]);
03643         }
03644
03651         GgQuaternion& loadAdd(const GgQuaternion& q)
03652         {
03653             return loadAdd(q[0], q[1], q[2], q[3]);
03654         }
03655
03665         GgQuaternion& loadSubtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03666         {

```

```

03667     (*this)[0] -= x;
03668     (*this)[1] -= y;
03669     (*this)[2] -= z;
03670     (*this)[3] -= w;
03671
03672     return *this;
03673 }
03674
03681 GgQuaternion& loadSubtract(const GLfloat* a)
03682 {
03683     return loadSubtract(a[0], a[1], a[2], a[3]);
03684 }
03685
03692 GgQuaternion& loadSubtract(const GgVector& v)
03693 {
03694     return loadSubtract(v[0], v[1], v[2], v[3]);
03695 }
03696
03703 GgQuaternion& loadSubtract(const GgQuaternion& q)
03704 {
03705     return loadSubtract(q[0], q[1], q[2], q[3]);
03706 }
03707
03717 GgQuaternion& loadMultiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03718 {
03719     const GLfloat a[]={ x, y, z, w };
03720     return loadMultiply(a);
03721 }
03722
03729 GgQuaternion& loadMultiply(const GLfloat* a)
03730 {
03731     return *this = multiply(a);
03732 }
03733
03740 GgQuaternion& loadMultiply(const GgVector& v)
03741 {
03742     return loadMultiply(v.data());
03743 }
03744
03751 GgQuaternion& loadMultiply(const GgQuaternion& q)
03752 {
03753     return loadMultiply(q.data());
03754 }
03755
03765 GgQuaternion& loadDivide(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03766 {
03767     const GLfloat a[]={ x, y, z, w };
03768     return loadDivide(a);
03769 }
03770
03777 GgQuaternion& loadDivide(const GLfloat* a)
03778 {
03779     return *this = divide(a);
03780 }
03781
03788 GgQuaternion& loadDivide(const GgVector& v)
03789 {
03790     return loadDivide(v.data());
03791 }
03792
03799 GgQuaternion& loadDivide(const GgQuaternion& q)
03800 {
03801     return loadDivide(q.data());
03802 }
03803
03813 GgQuaternion add(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03814 {
03815     GgQuaternion s{ *this };
03816     return s.loadAdd(x, y, z, w);
03817 }
03818
03825 GgQuaternion add(const GLfloat* a) const
03826 {
03827     return add(a[0], a[1], a[2], a[3]);
03828 }
03829
03836 GgQuaternion add(const GgVector& v) const
03837 {
03838     return add(v[0], v[1], v[2], v[3]);
03839 }
03840
03847 GgQuaternion add(const GgQuaternion& q) const
03848 {
03849     return add(q[0], q[1], q[2], q[3]);
03850 }
03851
03861 GgQuaternion subtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const

```

```

03862     {
03863         GgQuaternion s{ *this };
03864         return s.loadSubtract(x, y, z, w);
03865     }
03866
03873 GgQuaternion subtract(const GLfloat* a) const
03874 {
03875     return subtract(a[0], a[1], a[2], a[3]);
03876 }
03877
03884 GgQuaternion subtract(const GgVector& v) const
03885 {
03886     return subtract(v[0], v[1], v[2], v[3]);
03887 }
03888
03895 GgQuaternion subtract(const GgQuaternion& q) const
03896 {
03897     return subtract(q[0], q[1], q[2], q[3]);
03898 }
03899
03909 GgQuaternion multiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03910 {
03911     const GLfloat a[]={ x, y, z, w };
03912     return multiply(a);
03913 }
03914
03921 GgQuaternion multiply(const GLfloat* a) const
03922 {
03923     GgQuaternion s;
03924     multiply(s.data(), data(), a);
03925     return s;
03926 }
03927
03934 GgQuaternion multiply(const GgVector& v) const
03935 {
03936     return multiply(v.data());
03937 }
03938
03945 GgQuaternion multiply(const GgQuaternion& q) const
03946 {
03947     return multiply(q.data());
03948 }
03949
03959 GgQuaternion divide(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03960 {
03961     const GLfloat a[]={ x, y, z, w };
03962     return divide(a);
03963 }
03964
03971 GgQuaternion divide(const GLfloat* a) const
03972 {
03973     GgQuaternion s, ia;
03974     ia.loadInvert(a);
03975     multiply(s.data(), data(), ia.data());
03976     return s;
03977 }
03978
03985 GgQuaternion divide(const GgVector& v) const
03986 {
03987     return divide(v.data());
03988 }
03989
03996 GgQuaternion divide(const GgQuaternion& q) const
03997 {
03998     return divide(q.data());
03999 }
04000
04001 // 演算子
04002 GgQuaternion& operator=(const GLfloat* a)
04003 {
04004     return *this = GgQuaternion(a);
04005 }
04006 GgQuaternion& operator=(const GgVector& v)
04007 {
04008     return *this = GgQuaternion(v);
04009 }
04010 GgQuaternion& operator+=(const GLfloat* a)
04011 {
04012     return loadAdd(a);
04013 }
04014 GgQuaternion& operator+=(const GgVector& v)
04015 {
04016     return loadAdd(v);
04017 }
04018 GgQuaternion& operator+=(const GgQuaternion& q)
04019 {
04020     return loadAdd(q);

```

```

04021     }
04022     GgQuaternion& operator-=(const GLfloat* a)
04023     {
04024         return loadSubtract(a);
04025     }
04026     GgQuaternion& operator-=(const GgVector& v)
04027     {
04028         return loadSubtract(v);
04029     }
04030     GgQuaternion& operator-=(const GgQuaternion& q)
04031     {
04032         return operator-=(q.data());
04033     }
04034     GgQuaternion& operator*=(const GLfloat* a)
04035     {
04036         return loadMultiply(a);
04037     }
04038     GgQuaternion& operator*=(const GgVector& v)
04039     {
04040         return loadMultiply(v);
04041     }
04042     GgQuaternion& operator*=(const GgQuaternion& q)
04043     {
04044         return operator*=(q.data());
04045     }
04046     GgQuaternion& operator/=(const GLfloat* a)
04047     {
04048         return loadDivide(a);
04049     }
04050     GgQuaternion& operator/=(const GgVector& v)
04051     {
04052         return loadDivide(v);
04053     }
04054     GgQuaternion& operator/=(const GgQuaternion& q)
04055     {
04056         return operator/=(q.data());
04057     }
04058     GgQuaternion operator+(const GLfloat* a) const
04059     {
04060         return add(a);
04061     }
04062     GgQuaternion operator+(const GgVector& v) const
04063     {
04064         return add(v);
04065     }
04066     GgQuaternion operator+(const GgQuaternion& q) const
04067     {
04068         return operator+(q.data());
04069     }
04070     GgQuaternion operator-(const GLfloat* a) const
04071     {
04072         return subtract(a);
04073     }
04074     GgQuaternion operator-(const GgVector& v) const
04075     {
04076         return subtract(v);
04077     }
04078     GgQuaternion operator-(const GgQuaternion& q) const
04079     {
04080         return operator-(q.data());
04081     }
04082     GgQuaternion operator*(const GLfloat* a) const
04083     {
04084         return multiply(a);
04085     }
04086     GgQuaternion operator*(const GgVector& v) const
04087     {
04088         return multiply(v);
04089     }
04090     GgQuaternion operator*(const GgQuaternion& q) const
04091     {
04092         return operator*(q.data());
04093     }
04094     GgQuaternion operator/(const GLfloat* a) const
04095     {
04096         return divide(a);
04097     }
04098     GgQuaternion operator/(const GgVector& v) const
04099     {
04100         return divide(v);
04101     }
04102     GgQuaternion operator/(const GgQuaternion& q) const
04103     {
04104         return operator/(q.data());
04105     }
04106
04113     GgQuaternion& loadMatrix(const GLfloat* a)

```

```

04114     {
04115         toQuaternion(data(), a);
04116         return *this;
04117     }
04118
04125 GgQuaternion& loadMatrix(const GgMatrix& m)
04126 {
04127     return loadMatrix(m.get());
04128 }
04129
04135 GgQuaternion& loadIdentity()
04136 {
04137     return *this = GgQuaternion(0.0f, 0.0f, 0.0f, 1.0f);
04138 }
04139
04149 GgQuaternion& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
04150
04158 GgQuaternion& loadRotate(const GLfloat* v, GLfloat a)
04159 {
04160     return loadRotate(v[0], v[1], v[2], a);
04161 }
04162
04169 GgQuaternion& loadRotate(const GLfloat* v)
04170 {
04171     return loadRotate(v[0], v[1], v[2], v[3]);
04172 }
04173
04180 GgQuaternion& loadRotateX(GLfloat a);
04181
04188 GgQuaternion& loadRotateY(GLfloat a);
04189
04196 GgQuaternion& loadRotateZ(GLfloat a);
04197
04207 GgQuaternion rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
04208 {
04209     GgQuaternion q;
04210     return multiply(q.loadRotate(x, y, z, a));
04211 }
04212
04220 GgQuaternion rotate(const GLfloat* v, GLfloat a) const
04221 {
04222     return rotate(v[0], v[1], v[2], a);
04223 }
04224
04231 GgQuaternion rotate(const GLfloat* v) const
04232 {
04233     return rotate(v[0], v[1], v[2], v[3]);
04234 }
04235
04242 GgQuaternion rotateX(GLfloat a) const
04243 {
04244     return rotate(1.0f, 0.0f, 0.0f, a);
04245 }
04246
04253 GgQuaternion rotateY(GLfloat a) const
04254 {
04255     return rotate(0.0f, 1.0f, 0.0f, a);
04256 }
04257
04264 GgQuaternion rotateZ(GLfloat a) const
04265 {
04266     return rotate(0.0f, 0.0f, 1.0f, a);
04267 }
04268
04277 GgQuaternion& loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll);
04278
04285 GgQuaternion& loadEuler(const GLfloat* e)
04286 {
04287     return loadEuler(e[0], e[1], e[2]);
04288 }
04289
04298 GgQuaternion euler(GLfloat heading, GLfloat pitch, GLfloat roll) const
04299 {
04300     GgQuaternion r;
04301     return multiply(r.loadEuler(heading, pitch, roll));
04302 }
04303
04310 GgQuaternion euler(const GLfloat* e) const
04311 {
04312     return euler(e[0], e[1], e[2]);
04313 }
04314
04324 GgQuaternion euler(const GgVector& e) const
04325 {
04326     return euler(e[0], e[1], e[2]);
04327 }
04328

```

```

04337 GgQuaternion& loadSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04338 {
04339     slerp(data(), a, b, t);
04340     return *this;
04341 }
04342
04351 GgQuaternion& loadSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04352 {
04353     return loadSlerp(q.data(), r.data(), t);
04354 }
04355
04364 GgQuaternion& loadSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04365 {
04366     return loadSlerp(q.data(), a, t);
04367 }
04368
04377 GgQuaternion& loadSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04378 {
04379     return loadSlerp(a, q.data(), t);
04380 }
04381
04388 GgQuaternion& loadNormalize(const GLfloat* a);
04389
04396 GgQuaternion& loadNormalize(const GgQuaternion& q)
04397 {
04398     return loadNormalize(q.data());
04399 }
04400
04407 GgQuaternion& loadConjugate(const GLfloat* a);
04408
04415 GgQuaternion& loadConjugate(const GgQuaternion& q)
04416 {
04417     return loadConjugate(q.data());
04418 }
04419
04426 GgQuaternion& loadInvert(const GLfloat* a);
04427
04434 GgQuaternion& loadInvert(const GgQuaternion& q)
04435 {
04436     return loadInvert(q.data());
04437 }
04438
04446 GgQuaternion slerp(GLfloat* a, GLfloat t) const
04447 {
04448     GgQuaternion p;
04449     slerp(p.data(), data(), a, t);
04450     return p;
04451 }
04452
04460 GgQuaternion slerp(const GgQuaternion& q, GLfloat t) const
04461 {
04462     GgQuaternion p;
04463     slerp(p.data(), data(), q.data(), t);
04464     return p;
04465 }
04466
04472 GgQuaternion normalize() const
04473 {
04474     GgQuaternion q;
04475     q.loadNormalize(data());
04476     return q;
04477 }
04478
04484 GgQuaternion conjugate() const
04485 {
04486     GgQuaternion q;
04487     q.loadConjugate(data());
04488     return q;
04489 }
04490
04496 GgQuaternion invert() const
04497 {
04498     GgQuaternion q;
04499     q.loadInvert(data());
04500     return q;
04501 }
04502
04508 void get(GLfloat* a) const
04509 {
04510     a[0] = data()[0];
04511     a[1] = data()[1];
04512     a[2] = data()[2];
04513     a[3] = data()[3];
04514 }
04515
04521 void getMatrix(GLfloat* a) const
04522 {

```

```

04523     toMatrix(a, data());
04524 }
04525
04531 void getMatrix(GgMatrix& m) const
04532 {
04533     getMatrix(m.data());
04534 }
04535
04541 GgMatrix getMatrix() const
04542 {
04543     GgMatrix m;
04544     getMatrix(m);
04545     return m;
04546 }
04547
04553 void getConjugateMatrix(GLfloat* a) const
04554 {
04555     GgQuaternion q;
04556     q.loadConjugate(data());
04557     toMatrix(a, q.data());
04558 }
04559
04565 void getConjugateMatrix(GgMatrix& m) const
04566 {
04567     getConjugateMatrix(m.data());
04568 }
04569
04575 GgMatrix getConjugateMatrix() const
04576 {
04577     GgMatrix m;
04578     getConjugateMatrix(m);
04579     return m;
04580 }
04581 };
04582
04588 inline GgQuaternion ggIdentityQuaternion()
04589 {
04590     GgQuaternion q;
04591     return q.loadIdentity();
04592 }
04593
04600 inline GgQuaternion ggMatrixQuaternion(const GLfloat* a)
04601 {
04602     GgQuaternion q;
04603     return q.loadMatrix(a);
04604 }
04605
04612 inline GgQuaternion ggMatrixQuaternion(const GgMatrix& m)
04613 {
04614     return ggMatrixQuaternion(m.get());
04615 }
04616
04623 inline GgMatrix ggQuaternionMatrix(const GgQuaternion& q)
04624 {
04625     GLfloat m[16];
04626     q.getMatrix(m);
04627     return GgMatrix{ m };
04628 }
04629
04636 inline GgMatrix ggQuaternionTransposeMatrix(const GgQuaternion& q)
04637 {
04638     GLfloat m[16];
04639     q.getMatrix(m);
04640     GgMatrix t;
04641     return t.loadTranspose(m);
04642 }
04643
04653 inline GgQuaternion ggRotateQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
04654 {
04655     GgQuaternion q;
04656     return q.loadRotate(x, y, z, a);
04657 }
04658
04666 inline GgQuaternion ggRotateQuaternion(const GLfloat* v, GLfloat a)
04667 {
04668     return ggRotateQuaternion(v[0], v[1], v[2], a);
04669 }
04670
04677 inline GgQuaternion ggRotateQuaternion(const GLfloat* v)
04678 {
04679     return ggRotateQuaternion(v[0], v[1], v[2], v[3]);
04680 }
04681
04690 inline GgQuaternion ggEulerQuaternion(GLfloat heading, GLfloat pitch, GLfloat roll)
04691 {
04692     GgQuaternion q;
04693     return q.loadEuler(heading, pitch, roll);

```

```

04694     }
04695
04702     inline GgQuaternion ggEulerQuaternion(const GLfloat* e)
04703     {
04704         return ggEulerQuaternion(e[0], e[1], e[2]);
04705     }
04706
04713     inline GgQuaternion ggEulerQuaternion(const GgVector& e)
04714     {
04715         return ggEulerQuaternion(e[0], e[1], e[2]);
04716     }
04717
04726     inline GgQuaternion ggSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04727     {
04728         GgQuaternion r;
04729         return r.loadSlerp(a, b, t);
04730     }
04731
04740     inline GgQuaternion ggSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04741     {
04742         return ggSlerp(q.data(), r.data(), t);
04743     }
04744
04753     inline GgQuaternion ggSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04754     {
04755         return ggSlerp(q.data(), a, t);
04756     }
04757
04766     inline GgQuaternion ggSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04767     {
04768         return ggSlerp(a, q.data(), t);
04769     }
04770
04777     inline GLfloat ggNorm(const GgQuaternion& q)
04778     {
04779         return q.norm();
04780     }
04781
04788     inline GgQuaternion ggNormalize(const GgQuaternion& q)
04789     {
04790         return q.normalize();
04791     }
04792
04799     inline GgQuaternion ggConjugate(const GgQuaternion& q)
04800     {
04801         return q.conjugate();
04802     }
04803
04810     inline GgQuaternion ggInvert(const GgQuaternion& q)
04811     {
04812         return q.invert();
04813     }
04814
04818     class GgTrackball : public GgQuaternion
04819     {
04820     public:
04821         bool drag;           // ドラッグ中か否か
04822         GLfloat start[2];    // ドラッグ開始位置
04823         GLfloat scale[2];    // マウスの絶対位置→ウィンドウ内での相対位置の換算係数
04824         GgQuaternion cq;     // 回転の初期値 (四元数)
04825         GgMatrix rt;         // 回転の変換行列
04826
04827     public:
04833         GgTrackball(const GgQuaternion& q = ggIdentityQuaternion())
04834         {
04835             reset(q);
04836         }
04837
04841         ~GgTrackball() = default;
04842
04848         GgTrackball& operator=(const GgQuaternion& q)
04849         {
04850             GgQuaternion::operator=(q);
04851             return *this;
04852         }
04853
04863         void region(GLfloat w, GLfloat h);
04864
04874         void region(int w, int h)
04875         {
04876             region(static_cast<GLfloat>(w), static_cast<GLfloat>(h));
04877         }
04878
04888         void begin(GLfloat x, GLfloat y);
04889
04899         void motion(GLfloat x, GLfloat y);
04900

```



```

04906     void rotate(const GgQuaternion& q);
04907
04917     void end(GLfloat x, GLfloat y);
04918
04924     void reset(const GgQuaternion& q = ggIdentityQuaternion());
04925
04931     const GLfloat* getStart() const
04932     {
04933         return start;
04934     }
04935
04939     const GLfloat& getStart(int direction) const
04940     {
04941         return start[direction];
04942     }
04943
04949     void getStart(GLfloat* position) const
04950     {
04951         position[0] = start[0];
04952         position[1] = start[1];
04953     }
04954
04960     const GLfloat* getScale() const
04961     {
04962         return scale;
04963     }
04964
04970     const GLfloat getScale(int direction) const
04971     {
04972         return scale[direction];
04973     }
04974
04980     void getScale(GLfloat* factor) const
04981     {
04982         factor[0] = scale[0];
04983         factor[1] = scale[1];
04984     }
04985
04991     const GgQuaternion& getQuaternion() const
04992     {
04993         return *this;
04994     }
04995
05001     const GgMatrix& getMatrix() const
05002     {
05003         return rt;
05004     }
05005
05011     const GLfloat* get() const
05012     {
05013         return rt.get();
05014     }
05015 };
05016
05027 extern bool ggSaveTga(
05028     const std::string& name,
05029     const void* buffer,
05030     unsigned int width,
05031     unsigned int height,
05032     unsigned int depth
05033 );
05034
05041 extern bool ggSaveColor(const std::string& name);
05042
05049 extern bool ggSaveDepth(const std::string& name);
05050
05061 extern bool ggReadImage(
05062     const std::string& name,
05063     std::vector<GLubyte>& image,
05064     GLsizei* pWidth,
05065     GLsizei* pHeight,
05066     GLenum* pFormat
05067 );
05068
05082 extern GLuint ggLoadTexture(
05083     const GLvoid* image,
05084     GLsizei width,
05085     GLsizei height,
05086     GLenum format = GL_RGB,
05087     GLenum type = GL_UNSIGNED_BYTE,
05088     GLenum internal = GL_RGB,
05089     GLenum wrap = GL_CLAMP_TO_EDGE,
05090     bool swizzle = false
05091 );
05092
05103 extern GLuint ggLoadImage(
05104     const std::string& name,

```

```

05105     GLsizei* pWidth = nullptr,
05106     GLsizei* pHeight = nullptr,
05107     GLenum internal = GL_RGBA,
05108     GLenum wrap = GL_CLAMP_TO_EDGE
05109 );
05110
05122 extern void ggCreateNormalMap(
05123     const GLubyte* hmap,
05124     GLsizei width,
05125     GLsizei height,
05126     GLenum format,
05127     GLfloat nz,
05128     GLenum internal,
05129     std::vector<GgVector>& nmap
05130 );
05131
05142 extern GLuint ggLoadHeight(
05143     const std::string& name,
05144     GLfloat nz,
05145     GLsizei* pWidth = nullptr,
05146     GLsizei* pHeight = nullptr,
05147     GLenum internal = GL_RGBA
05148 );
05149
05163 extern GLuint ggCreateShader(
05164     const std::string& vsrc,
05165     const std::string& fsrc = "",
05166     const std::string& gsrc = "",
05167     GLint nvarying = 0,
05168     const char* const* varyings = nullptr,
05169     const std::string& vtext = "vertex shader",
05170     const std::string& ftext = "fragment shader",
05171     const std::string& gtext = "geometry shader");
05172
05183 extern GLuint ggLoadShader(
05184     const std::string& vert,
05185     const std::string& frag = "",
05186     const std::string& geom = "",
05187     GLint nvarying = 0,
05188     const char* const* varyings = nullptr
05189 );
05190
05199 inline GLuint ggLoadShader(
05200     const std::array<std::string, 3>& files,
05201     GLint nvarying = 0,
05202     const char* const* varyings = nullptr
05203 )
05204 {
05205     return ggLoadShader(files[0], files[1], files[2], nvarying, varyings);
05206 }
05207
05208 #if !defined(__APPLE__)
05216 extern GLuint ggCreateComputeShader(
05217     const std::string& csrc,
05218     const std::string& ctext = "compute shader"
05219 );
05220
05227 extern GLuint ggLoadComputeShader(const std::string& comp);
05228 #endif
05229
05236 class GgTexture
05237 {
05238     // テクスチャ名
05239     GLuint texture;
05240
05241     // テクスチャの縦横の画素数
05242     GLsizei size[2];
05243
05244 public:
05245
05258     GgTexture(
05259         const GLvoid* image,
05260         GLsizei width,
05261         GLsizei height,
05262         GLenum format = GL_RGB,
05263         GLenum type = GL_UNSIGNED_BYTE,
05264         GLenum internal = GL_RGBA,
05265         GLenum wrap = GL_CLAMP_TO_EDGE,
05266         bool swizzle = false
05267     ) :
05268         texture{ ggLoadTexture(image, width, height, format, type, internal, wrap, swizzle) },
05269         size{ width, height }
05270     {
05271     }
05272
05278     GgTexture(const GgTexture& texture) = delete;
05279

```

```

05289     GgTexture(GgTexture&& o) noexcept :
05290         texture{ o.texture },
05291         size{ o.size[0], o.size[1] }
05292     {
05293         o.texture = 0;
05294     }
05295
05299     virtual ~GgTexture()
05300     {
05301         glBindTexture(GL_TEXTURE_2D, 0);
05302         glDeleteTextures(1, &texture);
05303     }
05304
05311     GgTexture& operator=(const GgTexture& texture) = delete;
05312
05322     GgTexture& operator=(GgTexture&& o) noexcept
05323     {
05324         if (&o != this)
05325         {
05326             // 保持しているテクスチャを削除する
05327             glBindTexture(GL_TEXTURE_2D, 0);
05328             glDeleteTextures(1, &texture);
05329
05330             // テクスチャ名の所有権を移動する
05331             texture = o.texture;
05332             size[0] = o.size[0];
05333             size[1] = o.size[1];
05334             o.texture = 0;
05335         }
05336
05337         return *this;
05338     }
05339
05343     void bind() const
05344     {
05345         glBindTexture(GL_TEXTURE_2D, texture);
05346     }
05347
05351     void unbind() const
05352     {
05353         glBindTexture(GL_TEXTURE_2D, 0);
05354     }
05355
05361     void swapRandB(bool swizzle) const;
05362
05368     const GLsizei& getWidth() const
05369     {
05370         return size[0];
05371     }
05372
05378     const GLsizei& getHeight() const
05379     {
05380         return size[1];
05381     }
05382
05388     void getSize(GLsizei* size) const
05389     {
05390         size[0] = getWidth();
05391         size[1] = getHeight();
05392     }
05393
05399     const GLsizei* getSize() const
05400     {
05401         return size;
05402     }
05403
05409     const GLuint& getTexture() const
05410     {
05411         return texture;
05412     }
05413 };
05414
05421 class GgColorTexture
05422 {
05423     // テクスチャ
05424     std::shared_ptr<GgTexture> texture;
05425
05426 public:
05427
05431     GgColorTexture()
05432     {
05433     }
05434
05447     GgColorTexture(
05448         const GLvoid* image,
05449         GLsizei width,
05450         GLsizei height,

```

```

05451     GLenum format = GL_RGB,
05452     GLenum type = GL_UNSIGNED_BYTE,
05453     GLenum internal = GL_RGB,
05454     GLenum wrap = GL_CLAMP_TO_EDGE,
05455     bool swizzle = true
05456 )
05457 {
05458     load(image, width, height, format, type, internal, wrap, swizzle);
05459 }
05460
05461 GgColorTexture(
05462     const std::string& name,
05463     GLenum internal = 0,
05464     GLenum wrap = GL_CLAMP_TO_EDGE
05465 )
05466 {
05467     load(name, internal, wrap);
05468 }
05469
05470 virtual ~GgColorTexture()
05471 {
05472 }
05473
05474 void load(
05475     const GLvoid* image,
05476     GLsizei width,
05477     GLsizei height,
05478     GLenum format = GL_RGB,
05479     GLenum type = GL_UNSIGNED_BYTE,
05480     GLenum internal = GL_RGB,
05481     GLenum wrap = GL_CLAMP_TO_EDGE,
05482     bool swizzle = true
05483 )
05484 {
05485     // テクスチャを作成する
05486     texture = std::make_shared<GgTexture>(image, width, height, format, type, internal, wrap,
05487     swizzle);
05488 }
05489
05490 void load(
05491     const std::string& name,
05492     GLenum internal = 0,
05493     GLenum wrap = GL_CLAMP_TO_EDGE
05494 );
05495
05496 explicit operator bool() const noexcept
05497 {
05498     return texture != nullptr;
05499 }
05500
05501 bool operator!() const noexcept
05502 {
05503     return !static_cast<bool>(*this);
05504 }
05505
05506 const GgTexture* get() const
05507 {
05508     return texture.get();
05509 }
05510
05511 GLuint getTexture() const
05512 {
05513     return texture ? texture->getTexture() : 0;
05514 }
05515
05516 GLsizei getWidth() const
05517 {
05518     return texture ? texture->getWidth() : 0;
05519 }
05520
05521 GLsizei getHeight() const
05522 {
05523     return texture ? texture->getHeight() : 0;
05524 }
05525
05526 void getSize(GLsizei* size) const
05527 {
05528     if (texture) texture->getSize(size);
05529 }
05530
05531 const GLsizei* getSize() const
05532 {
05533     return texture ? texture->getSize() : nullptr;
05534 }
05535
05536 void bind() const
05537 {

```

```

05609         if (texture) texture->bind();
05610     }
05611
05615     void unbind() const
05616     {
05617         if (texture) texture->unbind();
05618     }
05619 };
05620
05627 class GgNormalTexture
05628 {
05629     // テクスチャ
05630     std::shared_ptr<GgTexture> texture;
05631
05632 public:
05633
05637     GgNormalTexture()
05638     {
05639     }
05640
05651     GgNormalTexture(
05652         const GLubyte* image,
05653         GLsizei width,
05654         GLsizei height,
05655         GLenum format = GL_RED,
05656         GLfloat nz = 1.0f,
05657         GLenum internal = GL_RGBA
05658     )
05659     {
05660         // 法線マップのテクスチャを作成する
05661         load(image, width, height, format, nz, internal);
05662     }
05663
05671     GgNormalTexture(
05672         const std::string& name,
05673         GLfloat nz = 1.0f,
05674         GLenum internal = GL_RGBA
05675     )
05676     {
05677         // 法線マップのテクスチャを作成する
05678         load(name, nz, internal);
05679     }
05680
05684     virtual ~GgNormalTexture()
05685     {
05686     }
05687
05698     void load(
05699         const GLubyte* hmap,
05700         GLsizei width,
05701         GLsizei height,
05702         GLenum format = GL_RED,
05703         GLfloat nz = 1.0f,
05704         GLenum internal = GL_RGBA
05705     )
05706     {
05707         // 法線マップ
05708         std::vector<GgVector> nmap;
05709
05710         // 法線マップを作成する
05711         ggCreateNormalMap(hmap, width, height, format, nz, internal, nmap);
05712
05713         // テクスチャを作成する
05714         texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,
GL_REPEAT);
05715     }
05716
05724     void load(
05725         const std::string& name,
05726         GLfloat nz = 1.0f,
05727         GLenum internal = GL_RGBA
05728     );
05729
05735     explicit operator bool() const noexcept
05736     {
05737         return texture != nullptr;
05738     }
05739
05745     bool operator!() const noexcept
05746     {
05747         return !static_cast<bool>(*this);
05748     }
05749
05755     const GgTexture* get() const
05756     {
05757         return texture.get();
05758     }

```

```

05759
05765     GLuint getTexture() const
05766     {
05767         return texture ? texture->getTexture() : 0;
05768     }
05769
05775     GLsizei getWidth() const
05776     {
05777         return texture ? texture->getWidth() : 0;
05778     }
05779
05785     GLsizei getHeight() const
05786     {
05787         return texture ? texture->getHeight() : 0;
05788     }
05789
05795     void getSize(GLsizei* size) const
05796     {
05797         if (texture) texture->getSize(size);
05798     }
05799
05805     const GLsizei* getSize() const
05806     {
05807         return texture ? texture->getSize() : nullptr;
05808     }
05809
05813     void bind() const
05814     {
05815         if (texture) texture->bind();
05816     }
05817
05821     void unbind() const
05822     {
05823         if (texture) texture->unbind();
05824     }
05825 };
05826
05833     template <typename T>
05834     class GgBuffer
05835     {
05836     // ターゲット
05837         const GLenum target;
05838
05839     // バッファオブジェクトのアライメントを考慮したデータの間隔
05840         const GLsizei stride;
05841
05842     // データの数
05843         const GLsizei count;
05844
05845     // バッファオブジェクト (ムーブしたら 0 になる)
05846         GLuint buffer;
05847
05848     public:
05849
05859         GgBuffer(
05860             GLenum target,
05861             const T* data,
05862             GLsizei stride,
05863             GLsizei count,
05864             GLenum usage
05865         ) :
05866             target{ target },
05867             stride{ stride },
05868             count{ count },
05869             buffer{ [] { GLuint buffer; glGenBuffers(1, &buffer); return buffer; } () }
05870         {
05871             // バッファオブジェクトのメモリを確保してデータを転送する
05872             glBindBuffer(target, buffer);
05873             glBufferData(target, getStride() * count, data, usage);
05874         }
05875
05881         GgBuffer(const GgBuffer<T>& buffer) = delete;
05882
05893         GgBuffer(GgBuffer<T>&& o) noexcept :
05894             target{ o.target },
05895             stride{ o.stride },
05896             count{ o.count },
05897             buffer{ o.buffer }
05898         {
05899             o.buffer = 0;
05900         }
05901
05905         virtual ~GgBuffer()
05906         {
05907             // バッファオブジェクトを削除する (ムーブ済みなら buffer は 0 で無視される)
05908             glBindBuffer(target, 0);
05909             glDeleteBuffers(1, &buffer);

```

```

05910     }
05911
05918     GgBuffer<T>& operator=(const GgBuffer<T>& buffer) = delete;
05919
05925     const GLuint& getTarget() const
05926     {
05927         return target;
05928     }
05929
05935     GLsizeiptr getStride() const
05936     {
05937         return static_cast<GLsizeiptr>(stride);
05938     }
05939
05945     const GLsizei& getCount() const
05946     {
05947         return count;
05948     }
05949
05955     const GLuint& getBuffer() const
05956     {
05957         return buffer;
05958     }
05959
05963     void bind() const
05964     {
05965         glBindBuffer(target, buffer);
05966     }
05967
05971     void unbind() const
05972     {
05973         glBindBuffer(target, 0);
05974     }
05975
05981     void* map() const
05982     {
05983         glBindBuffer(target, buffer);
05984         #if defined(GL_GLES_PROTOTYPES)
05985             return glMapBufferRange(target, 0, getStride() * count, GL_MAP_WRITE_BIT);
05986         #else
05987             return glMapBuffer(target, GL_WRITE_ONLY);
05988         #endif
05989     }
05990
05998     void* map(GLint first, GLsizei count) const
05999     {
06000         // count が 0 なら全データをマップする
06001         if (count == 0) count = getCount();
06002         if (first + count > getCount()) count = getCount() - first;
06003
06004         glBindBuffer(target, buffer);
06005         return glMapBufferRange(target, getStride() * first, getStride() * count, GL_MAP_WRITE_BIT);
06006     }
06007
06011     void unmap() const
06012     {
06013         glUnmapBuffer(target);
06014     }
06015
06023     void send(const T* data, GLint first, GLsizei count) const
06024     {
06025         // count が 0 なら全データを転送する
06026         if (count == 0) count = getCount();
06027         if (first + count > getCount()) count = getCount() - first;
06028
06029         // データを既存のバッファオブジェクトに転送する
06030         glBindBuffer(target, buffer);
06031         glBufferSubData(target, getStride() * first, getStride() * count, data);
06032     }
06033
06041     void read(T* data, GLint first, GLsizei count) const
06042     {
06043         // count が 0 なら全データを抽出する
06044         if (count == 0) count = getCount();
06045         if (first + count > getCount()) count = getCount() - first;
06046
06047         // データをバッファオブジェクトから抽出する
06048         glBindBuffer(target, buffer);
06049         #if defined(GL_GLES_PROTOTYPES)
06050             const GLsizeiptr begin{ getStride() * first };
06051             const GLsizeiptr range{ getStride() * count };
06052             T* const source{ glMapBufferRange(target, begin, range, GL_MAP_READ_BIT) };
06053             std::copy(source, source + count, data);
06054             glUnmapBuffer(target);
06055         #else
06056             glGetBufferSubData(target, getStride() * first, getStride() * count, data);
06057         #endif

```

```

06058     }
06059
06068 void copy(GLuint src_buffer, GLint src_first = 0, GLint dst_first = 0, GLsizei count = 0) const
06069 {
06070     // count が 0 なら全データを複写する
06071     if (count == 0) count = getCount();
06072     if (src_first + count > getCount()) count = getCount() - src_first;
06073     if (dst_first + count > getCount()) count = getCount() - dst_first;
06074
06075     // データの間隔
06076     const GLsizeiptr stride{ getStride() };
06077
06078     glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06079     glBindBuffer(GL_COPY_WRITE_BUFFER, buffer);
06080     glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
06081         stride * src_first, stride * dst_first, stride * count);
06082     glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06083     glBindBuffer(GL_COPY_READ_BUFFER, 0);
06084 }
06085 };
06086
06093 template <typename T>
06094 class GgUniformBuffer
06095 {
06096     // ユニフォームバッファオブジェクト
06097     std::shared_ptr<GgBuffer<T>> uniform;
06098
06099 public:
06100
06104     GgUniformBuffer()
06105     {
06106     }
06107
06115     GgUniformBuffer(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06116     {
06117         load(data, count, usage);
06118     }
06119
06127     GgUniformBuffer(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06128     {
06129         load(data, count, usage);
06130     }
06131
06135     virtual ~GgUniformBuffer()
06136     {
06137     }
06138
06144     const GLuint& getTarget() const
06145     {
06146         return uniform->getTarget();
06147     }
06148
06154     GLsizeiptr getStride() const
06155     {
06156         return uniform->getStride();
06157     }
06158
06164     const GLsizei& getCount() const
06165     {
06166         return uniform->getCount();
06167     }
06168
06174     const GLuint& getBuffer() const
06175     {
06176         return uniform->getBuffer();
06177     }
06178
06182     void bind() const
06183     {
06184         uniform->bind();
06185     }
06186
06190     void unbind() const
06191     {
06192         uniform->unbind();
06193     }
06194
06200     void* map() const
06201     {
06202         return uniform->map();
06203     }
06204
06212     void* map(GLint first, GLsizei count) const
06213     {
06214         return uniform->map(first, count);
06215     }
06216

```



```

06220 void unmap() const
06221 {
06222     uniform->unmap();
06223 }
06224
06232 void load(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06233 {
06234     // バッファオブジェクト上のデータの間隔
06235     const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment )};
06236
06237     // ユニフォームバッファオブジェクトを確保する
06238     uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
06239
06240     // 確保したユニフォームバッファオブジェクトにデータを転送する
06241     if (data) send(data, 0, sizeof(T), 0, count);
06242 }
06243
06251 void load(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06252 {
06253     // バッファオブジェクト上のデータの間隔
06254     const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment )};
06255
06256     // ユニフォームバッファオブジェクトを確保する
06257     uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
06258
06259     // 確保したユニフォームバッファオブジェクトにデータを転送する
06260     fill(&data, 0, sizeof(T), 0, count);
06261 }
06262
06272 void send(
06273     const GLvoid* data,
06274     GLint offset = 0,
06275     GLsizei size = sizeof(T),
06276     GLint first = 0,
06277     GLsizei count = 0
06278 ) const
06279 {
06280     // count が 0 なら全データを転送する
06281     if (count == 0) count = getCount();
06282     if (first + count > getCount()) count = getCount() - first;
06283
06284     // 転送元のデータの先頭
06285     const char* source{ reinterpret_cast<const char*>(data) };
06286
06287     // ターゲット
06288     const GLuint target{ getTarget() };
06289
06290     // データの間隔
06291     const GLsizeiptr stride{ getStride() };
06292
06293     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06294     bind();
06295     for (GLsizei i = 0; i < count; ++i)
06296     {
06297         glBufferSubData(target, stride * (first + i) + offset, size, source + size * i);
06298     }
06299 }
06300
06310 void fill(
06311     const GLvoid* data,
06312     GLint offset = 0,
06313     GLsizei size = sizeof(T),
06314     GLint first = 0,
06315     GLsizei count = 0
06316 ) const
06317 {
06318     // count が 0 なら全データを転送する
06319     if (count == 0) count = getCount();
06320     if (first + count > getCount()) count = getCount() - first;
06321
06322     // ターゲット
06323     const GLuint target{ getTarget() };
06324
06325     // データの間隔
06326     const GLsizeiptr stride{ getStride() };
06327
06328     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06329     bind();
06330     for (GLsizei i = 0; i < count; ++i)
06331     {
06332         glBufferSubData(target, stride * (first + i) + offset, size, data);
06333     }
06334 }
06345 void read(

```

```

06346     GLvoid* data,
06347     GLint offset = 0,
06348     GLsizei size = sizeof(T),
06349     GLint first = 0,
06350     GLsizei count = 0
06351 ) const
06352 {
06353     // count が 0 なら全データを転送する
06354     if (count == 0) count = getCount();
06355     if (first + count > getCount()) count = getCount() - first;
06356
06357     // 抽出先のデータの先頭
06358     char* const destination{ reinterpret_cast<char*>(data) };
06359
06360     // ターゲット
06361     const GLuint target{ getTarget() };
06362
06363     // データの間隔
06364     const GLsizeiptr stride{ getStride() };
06365
06366     // データをユニフォームバッファオブジェクトから抽出する
06367     bind();
06368     #if defined(GL_GLES_PROTOTYPES)
06369     const char* const source{ static_cast<const char*>(glMapBufferRange(target, stride * first,
06370     stride * count, GL_MAP_READ_BIT)) };
06371     for (GLsizei i = 0; i < count; ++i)
06372     {
06373         const char* const begin{ source + stride * i + offset };
06374         const char* const end{ begin + size };
06375         std::copy(begin, end, destination + sizeof(T) * i);
06376     }
06377     glUnmapBuffer(target);
06378     #else
06379     for (GLsizei i = 0; i < count; ++i)
06380     {
06381         glGetBufferSubData(target, stride * (first + i) + offset, size, destination + sizeof(T) * i);
06382     }
06383     #endif
06384
06393     void copy(
06394         GLuint src_buffer,
06395         GLint src_first = 0,
06396         GLint dst_first = 0,
06397         GLsizei count = 0
06398     ) const
06399     {
06400         // count が 0 なら全データを複写する
06401         if (count == 0) count = getCount();
06402         if (src_first + count > getCount()) count = getCount() - src_first;
06403         if (dst_first + count > getCount()) count = getCount() - dst_first;
06404
06405         // データの間隔
06406         const GLsizeiptr stride{ getStride() };
06407
06408         // ユニフォームバッファオブジェクトではブロックごとに転送する
06409         glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06410         glBindBuffer(GL_COPY_WRITE_BUFFER, getBuffer());
06411         for (GLsizei i = 0; i < count; ++i)
06412         {
06413             glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
06414                 stride * (src_first + i), stride * (dst_first + i), sizeof(T));
06415         }
06416         glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06417         glBindBuffer(GL_COPY_READ_BUFFER, 0);
06418     }
06419 };
06420
06424 class GgVertexArray
06425 {
06426     // 頂点配列オブジェクト (ムーブしたら 0 になる)
06427     GLuint vao;
06428
06429 public:
06430
06436     GgVertexArray(GLenum mode = 0) :
06437     {
06438         vao{ [] { GLuint vao; glGenVertexArrays(1, &vao); return vao; } () }
06439     {
06440         glBindVertexArray(vao);
06441     }
06442
06447     GgVertexArray(const GgVertexArray& array) = delete;
06448
06459     GgVertexArray(GgVertexArray&& o) noexcept :
06460     {
06461         vao{ o.vao }
06462         {
06463             o.vao = 0;

```

```

06463     }
06464
06468     virtual ~GgVertexArray()
06469     {
06470         // 頂点配列オブジェクトを削除する (ムーブ済みなら vao は 0 で無視される)
06471         glBindVertexArray(0);
06472         glDeleteVertexArrays(1, &vao);
06473     }
06474
06481     GgVertexArray& operator=(const GgVertexArray& array) = delete;
06482
06493     GgVertexArray& operator=(GgVertexArray&& o) noexcept
06494     {
06495         if (&o != this)
06496         {
06497             // 保持している頂点配列オブジェクトを削除する
06498             glBindVertexArray(0);
06499             glDeleteVertexArrays(1, &vao);
06500
06501             // 頂点配列オブジェクト名の所有権を移動する
06502             vao = o.vao;
06503             o.vao = 0;
06504         }
06505
06506         return *this;
06507     }
06508
06514     const GLuint& get() const
06515     {
06516         return vao;
06517     }
06518
06522     void bind() const
06523     {
06524         glBindVertexArray(vao);
06525     }
06526 };
06527
06535     class GgShape
06536     {
06537         // 頂点配列オブジェクト
06538         std::shared_ptr<GgVertexArray> object;
06539
06540         // 基本図形の種類
06541         GLenum mode;
06542
06543     public:
06544
06550         GgShape(GLenum mode = 0) :
06551             object{ std::make_shared<GgVertexArray>() },
06552             mode{ mode }
06553         {
06554         }
06555
06559         virtual ~GgShape()
06560         {
06561         }
06562
06568         virtual explicit operator bool() const noexcept
06569         {
06570             return object.get() != nullptr;
06571         }
06572
06578         virtual bool operator!() const noexcept
06579         {
06580             return !static_cast<bool>(*this);
06581         }
06582
06588         const GLuint& get() const
06589         {
06590             return object->get();
06591         }
06592
06598         void setMode(GLenum mode)
06599         {
06600             this->mode = mode;
06601         }
06602
06608         const GLenum& getMode() const
06609         {
06610             return this->mode;
06611         }
06612
06619         virtual void draw(GLint first = 0, GLsizei count = 0) const
06620         {
06621             object->bind();
06622         }

```

```

06623 };
06624
06628 class GgPoints
06629 : public GgShape
06630 {
06631     // 頂点バッファオブジェクト
06632     std::shared_ptr<GgBuffer<GgVector>> position;
06633
06634 public:
06635
06639     GgPoints(GLenum mode = GL_POINTS) :
06640         GgShape(mode)
06641     {
06642     }
06643
06652     GgPoints(
06653         const GgVector* pos,
06654         GLsizei countv,
06655         GLenum mode = GL_POINTS,
06656         GLenum usage = GL_STATIC_DRAW
06657     ) :
06658         GgPoints(mode)
06659     {
06660         load(pos, countv, usage);
06661     }
06662
06666     virtual ~GgPoints()
06667     {
06668     }
06669
06675     explicit operator bool() const noexcept
06676     {
06677         return position.get() != nullptr;
06678     }
06679
06685     bool operator!() const noexcept
06686     {
06687         return !static_cast<bool>(*this);
06688     }
06689
06695     const GLsizei& getCount() const
06696     {
06697         return position->getCount();
06698     }
06699
06705     const GLuint& getBuffer() const
06706     {
06707         return position->getBuffer();
06708     }
06709
06717     void send(const GgVector* pos, GLint first = 0, GLsizei count = 0) const
06718     {
06719         position->send(pos, first, count);
06720     }
06721
06729     void load(const GgVector* pos, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06730
06737     virtual void draw(GLint first = 0, GLsizei count = 0) const;
06738 };
06739
06743 struct GgVertex
06744 {
06746     GgVector position;
06747
06749     GgVector normal;
06750
06754     GgVertex()
06755     {
06756     }
06757
06764     GgVertex(const GgVector& pos, const GgVector& norm) :
06765         position(pos),
06766         normal(norm)
06767     {
06768     }
06769
06780     GgVertex(
06781         GLfloat px, GLfloat py, GLfloat pz,
06782         GLfloat nx, GLfloat ny, GLfloat nz
06783     ) :
06784         position{ px, py, pz, 1.0f },
06785         normal{ nx, ny, nz, 0.0f }
06786     {
06787     }
06788
06795     GgVertex(const GLfloat* pos, const GLfloat* norm) :
06796         GgVertex(pos[0], pos[1], pos[2], norm[0], norm[1], norm[2])

```

```

06797     {
06798     };
06799 };
06800
06804 class GgTriangles
06805 : public GgShape
06806 {
06807     // 頂点属性
06808     std::shared_ptr<GgBuffer<GgVertex>> vertex;
06809
06810 public:
06811
06817     GgTriangles(GLenum mode = GL_TRIANGLES) :
06818         GgShape(mode)
06819     {
06820     }
06821
06830     GgTriangles(
06831         const GgVertex* vert,
06832         GLsizei count,
06833         GLenum mode = GL_TRIANGLES,
06834         GLenum usage = GL_STATIC_DRAW
06835     ) :
06836         GgTriangles(mode)
06837     {
06838         load(vert, count, usage);
06839     }
06840
06844     virtual ~GgTriangles()
06845     {
06846     }
06847
06853     const GLsizei& getCount() const
06854     {
06855         return vertex->getCount();
06856     }
06857
06863     const GLuint& getBuffer() const
06864     {
06865         return vertex->getBuffer();
06866     }
06867
06875     void send(const GgVertex* vert, GLint first = 0, GLsizei count = 0) const
06876     {
06877         vertex->send(vert, first, count);
06878     }
06879
06887     void load(const GgVertex* vert, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06888
06895     virtual void draw(GLint first = 0, GLsizei count = 0) const;
06896 };
06897
06901 class GgElements
06902 : public GgTriangles
06903 {
06904     // インデックスを格納する頂点バッファオブジェクト
06905     std::shared_ptr<GgBuffer<GLuint>> index;
06906
06907 public:
06908
06914     GgElements(GLenum mode = GL_TRIANGLES) :
06915         GgTriangles(mode)
06916     {
06917     }
06918
06929     GgElements(
06930         const GgVertex* vert,
06931         GLsizei countv,
06932         const GLuint* face,
06933         GLsizei countf,
06934         GLenum mode = GL_TRIANGLES,
06935         GLenum usage = GL_STATIC_DRAW
06936     ) :
06937         GgElements(mode)
06938     {
06939         load(vert, countv, face, countf, usage);
06940     }
06941
06945     virtual ~GgElements()
06946     {
06947     }
06948
06953     const GLsizei& getIndexCount() const
06954     {
06955         return index->getCount();
06956     }
06957

```

```

06963     const GLuint& getIndexBuffer() const
06964     {
06965         return index->getBuffer();
06966     }
06967
06978     void send(
06979         const GgVertex* vert,
06980         GLuint firstv,
06981         GLsizei countv,
06982         const GLuint* face = nullptr,
06983         GLuint firstf = 0,
06984         GLsizei countf = 0
06985     ) const
06986     {
06987         GgTriangles::send(vert, firstv, countv);
06988         if (face != nullptr && countf > 0) index->send(face, firstf, countf);
06989     }
06990
07000     void load(
07001         const GgVertex* vert,
07002         GLsizei countv,
07003         const GLuint* face,
07004         GLsizei countf,
07005         GLenum usage = GL_STATIC_DRAW
07006     )
07007     {
07008         // 頂点バッファオブジェクトを作成する
07009         GgTriangles::load(vert, countv, usage);
07010
07011         // インデックスの頂点バッファオブジェクトを作成する
07012         index = std::make_shared<GgBuffer<GLuint>>(GL_ELEMENT_ARRAY_BUFFER, face,
static_cast<GLsizei>(sizeof(GLuint)), countf, usage);
07013     }
07014
07021     virtual void draw(GLint first = 0, GLsizei count = 0) const;
07022 };
07023
07034 extern std::shared_ptr<GgPoints> ggPointsCube(
07035     GLsizei countv,
07036     GLfloat length = 1.0f,
07037     GLfloat cx = 0.0f,
07038     GLfloat cy = 0.0f,
07039     GLfloat cz = 0.0f
07040 );
07041
07052 extern std::shared_ptr<GgPoints> ggPointsSphere(
07053     GLsizei countv,
07054     GLfloat radius = 0.5f,
07055     GLfloat cx = 0.0f,
07056     GLfloat cy = 0.0f,
07057     GLfloat cz = 0.0f
07058 );
07059
07067 extern std::shared_ptr<GgTriangles> ggRectangle(
07068     GLfloat width = 1.0f,
07069     GLfloat height = 1.0f
07070 );
07071
07080 extern std::shared_ptr<GgTriangles> ggEllipse(
07081     GLfloat width = 1.0f,
07082     GLfloat height = 1.0f,
07083     GLuint slices = 16
07084 );
07085
07097 extern std::shared_ptr<GgTriangles> ggArraysObj(
07098     const std::string& name,
07099     bool normalize = false
07100 );
07101
07113 extern std::shared_ptr<GgElements> ggElementsObj(
07114     const std::string& name,
07115     bool normalize = false
07116 );
07117
07130 extern std::shared_ptr<GgElements> ggElementsMesh(
07131     GLuint slices,
07132     GLuint stacks,
07133     const GLfloat(*pos)[3],
07134     const GLfloat(*norm)[3] = nullptr
07135 );
07136
07147 extern std::shared_ptr<GgElements> ggElementsSphere(
07148     GLfloat radius = 1.0f,
07149     int slices = 16,
07150     int stacks = 8
07151 );
07152

```

```

07159 class GgShader
07160 {
07161     // プログラム名 (ムーブしたら 0 になる)
07162     GLuint program;
07163
07164 public:
07165
07166     GgShader(
07167         const std::string& vert,
07168         const std::string& frag = "",
07169         const std::string& geom = "",
07170         int nvarying = 0,
07171         const char* const* varyings = nullptr
07172     ) :
07173         program(ggLoadShader(vert, frag, geom, nvarying, varyings))
07174     {
07175     }
07176
07177     GgShader(
07178         const std::array<std::string, 3>& files,
07179         int nvarying = 0,
07180         const char* const* varyings = nullptr
07181     ) :
07182         GgShader(files[0], files[1], files[2], nvarying, varyings)
07183     {
07184     }
07185
07186     GgShader(const GgShader& shader) = delete;
07187
07188     GgShader(GgShader&& o) noexcept :
07189         program{ o.program }
07190     {
07191         o.program = 0;
07192     }
07193
07194     virtual ~GgShader()
07195     {
07196         // シェーダプログラムを削除する (ムーブ済みなら program は 0 で無視される)
07197         glUseProgram(0);
07198         glDeleteProgram(program);
07199     }
07200
07201     GgShader& operator=(const GgShader& shader) = delete;
07202
07203     GgShader& operator=(GgShader&& o) noexcept
07204     {
07205         if (&o != this)
07206         {
07207             // 保持しているシェーダプログラムを削除する
07208             glUseProgram(0);
07209             glDeleteProgram(program);
07210
07211             // プログラム名の所有権を移動する
07212             program = o.program;
07213             o.program = 0;
07214         }
07215
07216         return *this;
07217     }
07218
07219     void use() const
07220     {
07221         glUseProgram(program);
07222     }
07223
07224     void unuse() const
07225     {
07226         glUseProgram(0);
07227     }
07228
07229     GLuint get() const
07230     {
07231         return program;
07232     }
07233 };
07234
07235 class GgPointShader
07236 {
07237     // シェーダー
07238     std::shared_ptr<GgShader> shader;
07239
07240     // 投影変換行列の uniform 変数の場所
07241     GLint mpLoc;
07242
07243     // モデルビュー変換行列の uniform 変数の場所
07244     GLint mvLoc;
07245
07246
07247
07248
07249
07250
07251
07252
07253
07254
07255
07256
07257
07258
07259
07260
07261
07262
07263
07264
07265
07266
07267
07268
07269
07270
07271
07272
07273
07274
07275
07276
07277
07278
07279
07280
07281
07282
07283
07284
07285
07286
07287
07288
07289
07290
07291
07292
07293
07294
07295
07296
07297
07298
07299
07300
07301
07302
07303
07304
07305
07306
07307
07308

```

```

07309 public:
07310
07314 GgPointShader() :
07315     mpLoc{ -1 },
07316     mvLoc{ -1 }
07317 {
07318 }
07319
07329 GgPointShader(
07330     const std::string& vert,
07331     const std::string& frag = "",
07332     const std::string& geom = "",
07333     GLint nvarying = 0,
07334     const char* const* varyings = nullptr
07335 ) :
07336     GgPointShader()
07337 {
07338     load(vert, frag, geom, nvarying, varyings);
07339 }
07340
07348 GgPointShader(
07349     const std::array<std::string, 3>& files,
07350     int nvarying = 0,
07351     const char* const* varyings = nullptr
07352 ) :
07353     GgPointShader(files[0], files[1], files[2], nvarying, varyings)
07354 {
07355 }
07356
07360 virtual ~GgPointShader()
07361 {
07362 }
07363
07374 bool load(
07375     const std::string& vert,
07376     const std::string& frag = "",
07377     const std::string& geom = "",
07378     GLint nvarying = 0,
07379     const char* const* varyings = nullptr
07380 )
07381 {
07382     // シェーダを作成する
07383     shader = std::make_shared<GgShader>(vert, frag, geom, nvarying, varyings);
07384
07385     // プログラム名を取り出す
07386     const GLuint program(shader->get());
07387
07388     // プログラムオブジェクトが作成できていなければ戻る
07389     if (program == 0) return false;
07390
07391     // 変換行列の uniform 変数の場所
07392     mpLoc = glGetUniformLocation(program, "mp");
07393     mvLoc = glGetUniformLocation(program, "mv");
07394
07395     // プログラムオブジェクトの作成に成功した
07396     return true;
07397 }
07398
07407 bool load(
07408     const std::array<std::string, 3>& files,
07409     GLint nvarying = 0,
07410     const char* const* varyings = nullptr
07411 )
07412 {
07413     return load(files[0], files[1], files[2], nvarying, varyings);
07414 }
07415
07421 virtual void loadProjectionMatrix(const GLfloat* mp) const
07422 {
07423     glUniformMatrix4fv(mpLoc, 1, GL_FALSE, mp);
07424 }
07425
07431 virtual void loadProjectionMatrix(const GgMatrix& mp) const
07432 {
07433     loadProjectionMatrix(mp.get());
07434 }
07435
07441 virtual void loadModelviewMatrix(const GLfloat* mv) const
07442 {
07443     glUniformMatrix4fv(mvLoc, 1, GL_FALSE, mv);
07444 }
07445
07451 virtual void loadModelviewMatrix(const GgMatrix& mv) const
07452 {
07453     loadModelviewMatrix(mv.get());
07454 }
07455

```



```

07462     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07463     {
07464         loadProjectionMatrix(mp);
07465         loadModelviewMatrix(mv);
07466     }
07467
07474     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07475     {
07476         loadMatrix(mp.get(), mv.get());
07477     }
07478
07482     virtual void use() const
07483     {
07484         shader->use();
07485     }
07486
07492     void use(const GLfloat* mp) const
07493     {
07494         use();
07495         loadProjectionMatrix(mp);
07496     }
07497
07503     void use(const GgMatrix& mp) const
07504     {
07505         use(mp.get());
07506     }
07507
07514     void use(const GLfloat* mp, const GLfloat* mv) const
07515     {
07516         use(mp);
07517         loadModelviewMatrix(mv);
07518     }
07519
07526     void use(const GgMatrix& mp, const GgMatrix& mv) const
07527     {
07528         use(mp.get(), mv.get());
07529     }
07530
07534     void unuse() const
07535     {
07536         shader->unuse();
07537     }
07538
07544     GLuint get() const
07545     {
07546         return shader->get();
07547     };
07548
07549
07553     class GgSimpleShader
07554     : public GgPointShader
07555     {
07556         // モデルビュー変換の法線変換行列の uniform 変数の場所
07557         GLint mnLoc;
07558
07559     public:
07560
07564         GgSimpleShader() :
07565             GgPointShader(),
07566             mnLoc{ -1 }
07567         {
07568         }
07569
07579         GgSimpleShader(
07580             const std::string& vert,
07581             const std::string& frag = "",
07582             const std::string& geom = "",
07583             GLint nvarying = 0,
07584             const char* const* varyings = nullptr
07585         )
07586         {
07587             load(vert, frag, geom, nvarying, varyings);
07588         }
07589
07597         GgSimpleShader(
07598             const std::array<std::string, 3>& files,
07599             GLint nvarying = 0,
07600             const char* const* varyings = nullptr
07601         ) :
07602             GgSimpleShader(files[0], files[1], files[2], nvarying, varyings)
07603         {
07604         }
07605
07609         GgSimpleShader(const GgSimpleShader& o) :
07610             GgPointShader(o),
07611             mnLoc{ o.mnLoc }
07612         {

```

```

07613     }
07614
07618     virtual ~GgSimpleShader()
07619     {
07620     }
07621
07628     GgSimpleShader& operator=(const GgSimpleShader& o)
07629     {
07630         if (&o != this)
07631         {
07632             GgPointShader::operator=(o);
07633             mnLoc = o.mnLoc;
07634         }
07635
07636         return *this;
07637     }
07638
07649     bool load(
07650         const std::string& vert,
07651         const std::string& frag = "",
07652         const std::string& geom = "",
07653         GLint nvarying = 0,
07654         const char* const* varyings = nullptr
07655     );
07656
07665     bool load(
07666         const std::array<std::string, 3>& files,
07667         GLint nvarying = 0,
07668         const char* const* varyings = nullptr
07669     )
07670     {
07671         return load(files[0], files[1], files[2], nvarying, varyings);
07672     }
07673
07680     virtual void loadModelviewMatrix(const GLfloat* mv, const GLfloat* mn) const
07681     {
07682         GgPointShader::loadModelviewMatrix(mv);
07683         glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07684     }
07685
07692     virtual void loadModelviewMatrix(const GgMatrix& mv, const GgMatrix& mn) const
07693     {
07694         loadModelviewMatrix(mv.get(), mn.get());
07695     }
07696
07702     virtual void loadModelviewMatrix(const GLfloat* mv) const
07703     {
07704         loadModelviewMatrix(mv, GgMatrix(mv).normal().get());
07705     }
07706
07712     virtual void loadModelviewMatrix(const GgMatrix& mv) const
07713     {
07714         loadModelviewMatrix(mv.get());
07715     }
07716
07724     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
07725     {
07726         GgPointShader::loadMatrix(mp, mv);
07727         glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07728     }
07729
07737     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
07738     {
07739         loadMatrix(mp.get(), mv.get(), mn.get());
07740     }
07741
07748     virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07749     {
07750         loadMatrix(mp, mv, GgMatrix(mv).normal());
07751     }
07752
07759     virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07760     {
07761         loadMatrix(mp, mv, mv.normal());
07762     }
07763
07767     struct Light
07768     {
07769         GgVector ambient;
07770         GgVector diffuse;
07771         GgVector specular;
07772         GgVector position;
07773     };
07774
07778     class LightBuffer
07779     : public GgUniformBuffer<Light>
07780     {

```

```

07781     public:
07782
07790     LightBuffer(
07791         const Light* light = nullptr,
07792         GLsizei count = 1,
07793         GLenum usage = GL_STATIC_DRAW
07794     ) :
07795         GgUniformBuffer<Light>(light, count, usage)
07796     {
07797     }
07798
07806     LightBuffer(
07807         const Light& light,
07808         GLsizei count = 1,
07809         GLenum usage = GL_STATIC_DRAW
07810     ) :
07811         GgUniformBuffer<Light>(light, count, usage)
07812     {
07813     }
07814
07825     LightBuffer(
07826         GgVector ambient,
07827         GgVector diffuse,
07828         GgVector specular,
07829         GgVector position,
07830         GLsizei count = 1,
07831         GLenum usage = GL_STATIC_DRAW
07832     ) :
07833         GgUniformBuffer<Light>(Light{ ambient, diffuse, specular, position }, count, usage)
07834     {
07835     }
07836
07840     virtual ~LightBuffer()
07841     {
07842     }
07843
07854     void loadAmbient(
07855         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07856         GLint first = 0, GLsizei count = 1
07857     ) const;
07858
07866     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
07867
07875     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
07876     {
07877         // first 番目のブロックから count 個の ambient 要素に値を設定する
07878         send(ambient, offsetof(Light, ambient), sizeof(Light::ambient), first, count);
07879     }
07880
07891     void loadDiffuse(
07892         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07893         GLint first = 0, GLsizei count = 1
07894     ) const;
07895
07903     void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
07904
07912     void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
07913     {
07914         // first 番目のブロックから count 個の diffuse 要素に値を設定する
07915         send(diffuse, offsetof(Light, diffuse), sizeof(Light::diffuse), first, count);
07916     }
07917
07928     void loadSpecular(
07929         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07930         GLint first = 0, GLsizei count = 1
07931     ) const;
07932
07940     void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
07941
07949     void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
07950     {
07951         // first 番目のブロックから count 個の specular 要素に値を設定する
07952         send(specular, offsetof(Light, specular), sizeof(Light::specular), first, count);
07953     }
07954
07962     void loadColor(const Light& color, GLint first = 0, GLsizei count = 1) const;
07963
07974     void loadPosition(
07975         GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f,
07976         GLint first = 0, GLsizei count = 1
07977     ) const;
07978
07986     void loadPosition(const GgVector& position, GLint first = 0, GLsizei count = 1) const;
07987
07995     void loadPosition(const GLfloat* position, GLint first = 0, GLsizei count = 1) const
07996     {
07997         // first 番目のブロックから count 個の position 要素に値を設定する

```

```

07998     send(position, offsetof(Light, position), sizeof(Light::position), first, count);
07999 }
08000
08008 void loadPosition(const GgVector* position, GLint first = 0, GLsizei count = 1) const
08009 {
08010     loadPosition(position->data(), first, count);
08011 }
08012
08020 void load(const Light* light, GLint first = 0, GLsizei count = 1) const
08021 {
08022     send(light, 0, sizeof(Light), first, count);
08023 }
08024
08032 void load(const Light& light, GLint first = 0, GLsizei count = 1) const
08033 {
08034     load(&light, first, count);
08035 }
08036
08042 void select(GLint i = 0) const
08043 {
08044     // バッファオブジェクトの i 番目のブロックの位置
08045     const GLintptr offset(static_cast<GLintptr>(getStride()) * i);
08046     glBindBufferRange(getTarget(), LightBindingPoint, getBuffer(), offset, sizeof(Light));
08047 }
08048 };
08049
08053 struct Material
08054 {
08055     GgVector ambient;
08056     GgVector diffuse;
08057     GgVector specular;
08058     GLfloat shininess;
08059 };
08060
08064 class MaterialBuffer
08065     : public GgUniformBuffer<Material>
08066 {
08067 public:
08068
08076     MaterialBuffer(
08077         const Material* material = nullptr,
08078         GLsizei count = 1,
08079         GLenum usage = GL_STATIC_DRAW
08080     ) :
08081         GgUniformBuffer<Material>(material, count, usage)
08082     {
08083     }
08084
08092     MaterialBuffer(
08093         const Material& material,
08094         GLsizei count = 1,
08095         GLenum usage = GL_STATIC_DRAW
08096     ) :
08097         GgUniformBuffer<Material>(material, count, usage)
08098     {
08099     }
08100
08111     MaterialBuffer(
08112         GgVector ambient,
08113         GgVector diffuse,
08114         GgVector specular,
08115         GLfloat shininess,
08116         GLsizei count = 1,
08117         GLenum usage = GL_STATIC_DRAW
08118     ) :
08119         GgUniformBuffer<Material>(Material{ ambient, diffuse, specular, shininess }, count, usage)
08120     {
08121     }
08122
08126     virtual ~MaterialBuffer()
08127     {
08128     }
08129
08140     void loadAmbient(
08141         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08142         GLint first = 0, GLsizei count = 1
08143     ) const;
08144
08152     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
08153
08161     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
08162     {
08163         // first 番目のブロックから count 個のブロックの ambient 要素に値を設定する
08164         send(ambient, offsetof(Material, ambient), sizeof(Material::ambient), first, count);
08165     }
08166
08177     void loadDiffuse(

```

```

08178     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08179     GLint first = 0, GLsizei count = 1
08180 ) const;
08181
08189 void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
08190
08198 void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
08199 {
08200     // first 番目のブロックから count 個の diffuse 要素に値を設定する
08201     send(diffuse, offsetof(Material, diffuse), sizeof(Material::diffuse), first, count);
08202 }
08203
08214 void loadAmbientAndDiffuse(
08215     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08216     GLint first = 0, GLsizei count = 1
08217 ) const;
08218
08226 void loadAmbientAndDiffuse(const GgVector& color, GLint first = 0, GLsizei count = 1) const;
08227
08235 void loadAmbientAndDiffuse(const GLfloat* color, GLint first = 0, GLsizei count = 1) const;
08236
08247 void loadSpecular(
08248     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08249     GLint first = 0, GLsizei count = 1
08250 ) const;
08251
08259 void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
08260
08268 void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
08269 {
08270     // first 番目のブロックから count 個の specular 要素に値を設定する
08271     send(specular, offsetof(Material, specular), sizeof(Material::specular), first, count);
08272 }
08273
08281 void loadShininess(GLfloat shininess, GLint first = 0, GLsizei count = 1) const;
08282
08290 void loadShininess(const GLfloat* shininess, GLint first = 0, GLsizei count = 1) const;
08291
08299 void load(const Material* material, GLint first = 0, GLsizei count = 1) const
08300 {
08301     send(material, 0, sizeof(Material), first, count);
08302 }
08303
08311 void load(const Material& material, GLint first = 0, GLsizei count = 1) const
08312 {
08313     load(&material, first, count);
08314 }
08315
08321 void select(GLint i = 0) const
08322 {
08323     // バッファオブジェクトの i 番目のブロックの位置
08324     const GLintptr offset{ static_cast<GLintptr>(getStride()) * i };
08325     glBindBufferRange(getTarget(), MaterialBindingPoint, getBuffer(), offset, sizeof(Material));
08326 }
08327 };
08328
08332 void use() const
08333 {
08334     // プログラムオブジェクトは基底クラスで指定する
08335     GgPointShader::use();
08336 }
08337
08345 void use(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
08346 {
08347     // プログラムオブジェクトを指定する
08348     use();
08349
08350     // 変換行列を設定する
08351     loadMatrix(mp, mv, mn);
08352 }
08353
08361 void use(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
08362 {
08363     use(mp.get(), mv.get(), mn.get());
08364 }
08365
08372 void use(const GLfloat* mp, const GLfloat* mv) const
08373 {
08374     use(mp, mv, GgMatrix(mv).normal().get());
08375 }
08376
08383 void use(const GgMatrix& mp, const GgMatrix& mv) const
08384 {
08385     use(mp, mv, mv.normal());
08386 }
08387
08394 void use(const LightBuffer* light, GLint i = 0) const

```

```

08395     {
08396         // プログラムオブジェクトを指定する
08397         use();
08398
08399         // 光源を設定する
08400         light->select(i);
08401     }
08402
08409 void use(const LightBuffer& light, GLint i = 0) const
08410 {
08411     use(&light, i);
08412 }
08413
08423 void use(
08424     const GLfloat* mp,
08425     const GLfloat* mv,
08426     const GLfloat* mn,
08427     const LightBuffer* light,
08428     GLint i = 0
08429 ) const
08430 {
08431     // 光源を指定してプログラムオブジェクトを指定する
08432     use(light, i);
08433
08434     // 変換行列を設定する
08435     loadMatrix(mp, mv, mn);
08436 }
08437
08447 void use(
08448     const GgMatrix& mp,
08449     const GgMatrix& mv,
08450     const GgMatrix& mn,
08451     const LightBuffer& light,
08452     GLint i = 0
08453 ) const
08454 {
08455     use(mp.get(), mv.get(), mn.get(), &light, i);
08456 }
08457
08466 void use(
08467     const GLfloat* mp,
08468     const GLfloat* mv,
08469     const LightBuffer* light,
08470     GLint i = 0
08471 ) const
08472 {
08473     use(mp, mv, GgMatrix(mv).normal().get(), light, i);
08474 }
08475
08484 void use(
08485     const GgMatrix& mp,
08486     const GgMatrix& mv,
08487     const LightBuffer& light,
08488     GLint i = 0
08489 ) const
08490 {
08491     use(mp, mv, mv.normal(), light, i);
08492 }
08493
08501 void use(const GLfloat* mp, const LightBuffer* light, GLint i = 0) const
08502 {
08503     // 光源を指定してプログラムオブジェクトを指定する
08504     use(light, i);
08505
08506     // 投影変換行列を設定する
08507     loadProjectionMatrix(mp);
08508 }
08509
08517 void use(const GgMatrix& mp, const LightBuffer& light, GLint i = 0) const
08518 {
08519     // 光源を指定してプログラムオブジェクトを指定する
08520     use(mp.get(), &light, i);
08521 }
08522 };
08523
08534 extern bool ggLoadSimpleObj(
08535     const std::string& name,
08536     std::vector<std::array<GLuint, 3>>& group,
08537     std::vector<GgSimpleShader::Material>& material,
08538     std::vector<GgVertex>& vert,
08539     bool normalize = false
08540 );
08541
08553 extern bool ggLoadSimpleObj(
08554     const std::string& name,
08555     std::vector<std::array<GLuint, 3>>& group,
08556     std::vector<GgSimpleShader::Material>& material,

```

```

08557     std::vector<GgVertex>& vert,
08558     std::vector<GLuint>& face,
08559     bool normalize = false
08560 );
08561
08562 class GgSimpleObj
08563 {
08564     // 同じ材質を割り当てるポリゴングループごとの三角形数
08565     std::shared_ptr<std::vector<std::array<GLuint, 3>>> group;
08566
08567     // ポリゴングループごとの材質のユニフォームバッファ
08568     std::shared_ptr<GgSimpleShader::MaterialBuffer> material;
08569
08570     // この図形の形状データ
08571     std::shared_ptr<GgElements> data;
08572
08573 public:
08574     GgSimpleObj(const std::string& name, bool normalize = false);
08575
08576     virtual ~GgSimpleObj()
08577     {
08578     }
08579
08580     explicit operator bool() const noexcept
08581     {
08582         return data.get() != nullptr;
08583     }
08584
08585     bool operator!() const noexcept
08586     {
08587         return !static_cast<bool>(*this);
08588     }
08589
08590     const GgTriangles* get() const
08591     {
08592         return data.get();
08593     }
08594
08595     virtual void draw(GLint first = 0, GLsizei count = 0) const;
08596 };
08597
08598 }

```

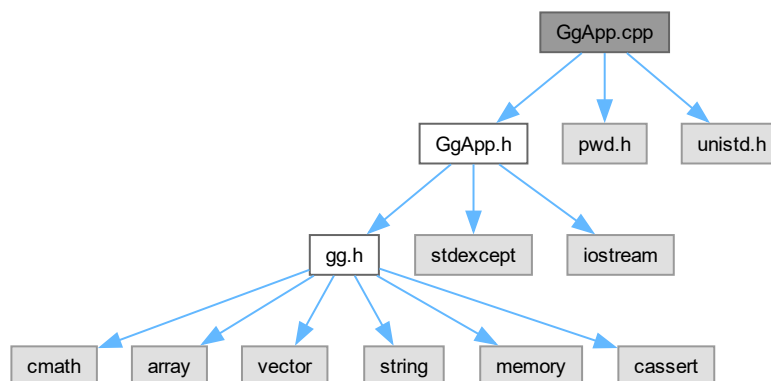
9.55 GgApp.cpp ファイル

```

#include "GgApp.h"
#include <pwd.h>
#include <unistd.h>

```

GgApp.cpp の依存先関係図:



9.55.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの実装.

著者

Kohe Tokoi

日付

July 27, 2025

[GgApp.cpp](#) に定義があります。

9.56 GgApp.cpp

[詳解]

```
00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00025 #include "GgApp.h"
00026 using namespace gg;
00027
00028 //
00029 // GLFW のエラー表示
00030 //
00031 static void glfwErrorCallback(int error, const char* description)
00032 {
00033     if defined(__aarch64__)
00034         if (error == 65544) return;
00035     #endif
00036     throw std::runtime_error(description);
00037 }
00038
00039 #if defined(__ANDROID__)
00040 struct android_app* GgApp::androidApp{ nullptr };
00041 #endif
00042
00043 //
00044 // GgApp クラスのコンストラクタ
00045 //
00046 GgApp::GgApp(int major, int minor)
00047 {
00048     #if defined(__ANDROID__)
00049         (void)major;
00050         (void)minor;
00051     #else
00052         // GLFW のエラー処理関数を登録する
00053         glfwSetErrorCallback(glfwErrorCallback);
00054     #endif
00055 }
```



```

00062 // GLFW を初期化する
00063 if (glfwInit() == GL_FALSE) throw std::runtime_error("Can't initialize GLFW");
00064
00065 // OpenGL の major 番号が指定されていれば
00066 if (major > 0)
00067 {
00068     // OpenGL のバージョンを指定する
00069     glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, major);
00070     glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, minor);
00071
00072 #if defined(GL_GLES_PROTOTYPES)
00073     // OpenGL ES 3 のコンテキストを指定する
00074     glfwWindowHint(GLFW_CLIENT_API, GLFW_OPENGL_ES_API);
00075     glfwWindowHint(GLFW_CONTEXT_CREATION_API, GLFW_EGL_CONTEXT_API);
00076 #else
00077     // OpenGL Version 3.2 以降なら
00078     if (major * 10 + minor >= 32)
00079     {
00080         // Core Profile を選択する (macOS の都合)
00081         glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
00082         glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
00083     }
00084 #endif
00085 }
00086 #endif
00087
00088 #if defined(IMGUI_VERSION)
00089 // ImGui のバージョンをチェックする
00090 ImGui::CheckVersion();
00091
00092 // ImGui のコンテキストを作成する
00093 ImGui::CreateContext();
00094 #endif
00095 }
00096
00097 //
00098 // デストラクタ
00099 //
00100 GgApp::~GgApp()
00101 {
00102     #if defined(IMGUI_VERSION)
00103     // Shutdown Platform/Renderer bindings
00104     ImGui_ImplOpenGL3_Shutdown();
00105     #if defined(_ANDROID_)
00106     ImGui_ImplAndroid_Shutdown();
00107     #else
00108     ImGui_ImplGlfw_Shutdown();
00109     #endif
00110     ImGui::DestroyContext();
00111 #endif
00112
00113 #if !defined(_ANDROID_)
00114     // プログラム終了時に GLFW を終了する
00115     glfwTerminate();
00116 #endif
00117 }
00118
00119 //
00120 // マウスや矢印キーによる平行移動量を初期化する
00121 //
00122 void GgApp::Window::HumanInterface::resetTranslation()
00123 {
00124     // 平行移動量を初期化する
00125     for (auto& t : translation)
00126     {
00127         std::fill(t.begin(), t.end(), GgVector{ 0.0f, 0.0f, 0.0f, 1.0f });
00128     }
00129
00130     // 矢印キーの設定値を初期化する
00131     std::fill(arrow.begin(), arrow.end(), std::array<int, 2>{ 0, 0 });
00132
00133     // マウスホイールの回転量を初期化する
00134     std::fill(wheel.begin(), wheel.end(), 0.0f);
00135 }
00136
00137 //
00138 // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00139 //
00140 void GgApp::Window::HumanInterface::calcTranslation(int button, const std::array<GLfloat, 3>&
velocity)
00141 {
00142     // マウスの相対変位
00143     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00144     const auto dx{ (mouse[0] - rotation[button].getStart(0)) * rotation[button].getScale(0) };
00145     const auto dy{ (rotation[button].getStart(1) - mouse[1]) * rotation[button].getScale(1) };
00146
00147     // 平行移動量

```

```

00148     auto& t{ translation[button] };
00149
00150     // 平行移動量の更新
00151     t[1][0] = dx * velocity[0] + t[0][0];
00152     t[1][1] = dy * velocity[1] + t[0][1];
00153
00154     // 回転量の更新
00155     rotation[button].motion(mouse[0], mouse[1]);
00156 }
00157
00158 #if !defined(__ANDROID__)
00159 //
00160 // ウィンドウのサイズ変更時の処理
00161 //
00162 void GgApp::Window::resize(GLFWwindow* window, int width, int height)
00163 {
00164     // このインスタンスの this ポインタを得る
00165     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00166
00167     if (instance)
00168     {
00169         // ウィンドウのサイズを保存する
00170         instance->size[0] = width;
00171         instance->size[1] = height;
00172
00173         // トラックボール処理の範囲を設定する
00174         for (auto& current_if : instance->interfaceData)
00175         {
00176             for (auto& t : current_if.rotation)
00177             {
00178                 t.region(width, height);
00179             }
00180         }
00181
00182         // ビューポートを更新する
00183         instance->updateViewport();
00184
00185         // ユーザー定義のコールバック関数の呼び出し
00186         if (instance->resizeFunc) (*instance->resizeFunc)(instance, width, height);
00187     }
00188 }
00189
00190 //
00191 // キーボードをタイプした時の処理
00192 //
00193 void GgApp::Window::keyboard(GLFWwindow* window, int key, int scancode, int action, int mods)
00194 {
00195     #if defined(IMGUI_VERSION)
00196         // ImGui がキーボードを使うときはキーボードの処理を行わない
00197         if (ImGui::GetIO().WantCaptureKeyboard) return;
00198     #endif
00199
00200     // このインスタンスの this ポインタを得る
00201     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00202
00203     if (instance && action)
00204     {
00205         // ユーザー定義のコールバック関数の呼び出し
00206         if (instance->keyboardFunc) (*instance->keyboardFunc)(instance, key, scancode, action, mods);
00207
00208         // 対象のユーザインタフェース
00209         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00210
00211         switch (key)
00212         {
00213             case GLFW_KEY_HOME:
00214
00215                 // トラックボールを初期化する
00216                 instance->resetRotation();
00217                 [[fallthrough]];
00218
00219             case GLFW_KEY_END:
00220
00221                 // 平行移動量を初期化する
00222                 instance->resetTranslation();
00223                 break;
00224
00225             case GLFW_KEY_UP:
00226
00227                 if (mods & GLFW_MOD_SHIFT)
00228                     current_if.arrow[1][1]++;
00229                 else if (mods & GLFW_MOD_CONTROL)
00230                     current_if.arrow[2][1]++;
00231                 else if (mods & GLFW_MOD_ALT)
00232                     current_if.arrow[3][1]++;
00233                 else
00234                     current_if.arrow[0][1]++;

```

```

00235         break;
00236
00237     case GLFW_KEY_DOWN:
00238
00239         if (mods & GLFW_MOD_SHIFT)
00240             current_if.arrow[1][1]--;
00241         else if (mods & GLFW_MOD_CONTROL)
00242             current_if.arrow[2][1]--;
00243         else if (mods & GLFW_MOD_ALT)
00244             current_if.arrow[3][1]--;
00245         else
00246             current_if.arrow[0][1]--;
00247         break;
00248
00249     case GLFW_KEY_RIGHT:
00250
00251         if (mods & GLFW_MOD_SHIFT)
00252             current_if.arrow[1][0]++;
00253         else if (mods & GLFW_MOD_CONTROL)
00254             current_if.arrow[2][0]++;
00255         else if (mods & GLFW_MOD_ALT)
00256             current_if.arrow[3][0]++;
00257         else
00258             current_if.arrow[0][0]++;
00259         break;
00260
00261     case GLFW_KEY_LEFT:
00262
00263         if (mods & GLFW_MOD_SHIFT)
00264             current_if.arrow[1][0]--;
00265         else if (mods & GLFW_MOD_CONTROL)
00266             current_if.arrow[2][0]--;
00267         else if (mods & GLFW_MOD_ALT)
00268             current_if.arrow[3][0]--;
00269         else
00270             current_if.arrow[0][0]--;
00271         break;
00272
00273     default:
00274         break;
00275     }
00276
00277     current_if.lastKey = key;
00278 }
00279 }
00280
00281 //
00282 // マウスボタンを操作したときの処理
00283 //
00284 void GgApp::Window::mouse(GLFWwindow* window, int button, int action, int mods)
00285 {
00286     #if defined(IMGUI_VERSION)
00287         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00288         if (ImGui::GetIO().WantCaptureMouse) return;
00289     #endif
00290
00291     // このインスタンスの this ポインタを得る
00292     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00293
00294     // マウスボタンの状態を記録する
00295     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00296     instance->status[button] = action != GLFW_RELEASE;
00297
00298     if (instance)
00299     {
00300         // ユーザー定義のコールバック関数の呼び出し
00301         if (instance->mouseFunc) (*instance->mouseFunc)(instance, button, action, mods);
00302
00303         // 対象のユーザインタフェース
00304         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00305
00306         // マウスの現在位置を得る
00307         const auto x{ current_if.mouse[0] };
00308         const auto y{ current_if.mouse[1] };
00309
00310         if (x < 0 || x >= instance->size[0] || y < 0 || y >= instance->size[1]) return;
00311
00312         if (action)
00313         {
00314             // ドラッグ開始
00315             current_if.rotation[button].begin(x, y);
00316         }
00317         else
00318         {
00319             // ドラッグ終了
00320             current_if.translation[button][0] = current_if.translation[button][1];
00321             current_if.rotation[button].end(x, y);

```

```

00322     }
00323   }
00324 }
00325
00326 //
00327 // マウスホイールを操作した時の処理
00328 //
00329 void GgApp::Window::wheel(GLFWwindow* window, double x, double y)
00330 {
00331     #if defined(IMGUI_VERSION)
00332         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00333         if (ImGui::GetIO().WantCaptureMouse) return;
00334     #endif
00335
00336     // このインスタンスの this ポインタを得る
00337     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00338
00339     if (instance)
00340     {
00341         // ユーザー定義のコールバック関数の呼び出し
00342         if (instance->wheelFunc) (*instance->wheelFunc)(instance, x, y);
00343
00344         // 対象のユーザインタフェース
00345         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00346
00347         // マウスホイールの回転量の保存
00348         current_if.wheel[0] += static_cast<GLfloat>(x);
00349         current_if.wheel[1] += static_cast<GLfloat>(y);
00350
00351         // マウスによる平行移動量の z 値の更新
00352         const auto z{ current_if.wheel[1] * instance->velocity[2] };
00353         for (auto& t : current_if.translation) t[1][2] = z;
00354     }
00355 }
00356 #else
00357 void GgApp::Window::resize(GLFWwindow*, int, int) {}
00358 void GgApp::Window::keyboard(GLFWwindow*, int, int, int, int) {}
00359 void GgApp::Window::mouse(GLFWwindow*, int, int, int) {}
00360 void GgApp::Window::wheel(GLFWwindow*, double, double) {}
00361 #endif
00362
00363 #if defined(__ANDROID__)
00364 //
00365 // EGL の初期化
00366 //
00367 bool GgApp::Window::initEgl(ANativeWindow* win)
00368 {
00369     display = eglGetDisplay(EGL_DEFAULT_DISPLAY);
00370     if (display == EGL_NO_DISPLAY) return false;
00371
00372     if (!eglInitialize(display, nullptr, nullptr)) return false;
00373
00374     const EGLint attribs[] = {
00375         EGL_RENDERABLE_TYPE, EGL_OPENGL_ES3_BIT,
00376         EGL_SURFACE_TYPE, EGL_WINDOW_BIT,
00377         EGL_BLUE_SIZE, 8,
00378         EGL_GREEN_SIZE, 8,
00379         EGL_RED_SIZE, 8,
00380         EGL_DEPTH_SIZE, 24,
00381         EGL_NONE
00382     };
00383
00384     EGLint numConfigs{ 0 };
00385     if (!eglChooseConfig(display, attribs, &config, 1, &numConfigs) || numConfigs <= 0)
00386     {
00387         const EGLint fallbackAttribs[] = {
00388             EGL_RENDERABLE_TYPE, EGL_OPENGL_ES3_BIT,
00389             EGL_SURFACE_TYPE, EGL_WINDOW_BIT,
00390             EGL_BLUE_SIZE, 8,
00391             EGL_GREEN_SIZE, 8,
00392             EGL_RED_SIZE, 8,
00393             EGL_DEPTH_SIZE, 16,
00394             EGL_NONE
00395         };
00396         if (!eglChooseConfig(display, fallbackAttribs, &config, 1, &numConfigs) || numConfigs <= 0)
00397         {
00398             return false;
00399         }
00400     }
00401
00402     EGLint format{ 0 };
00403     eglGetConfigAttrib(display, config, EGL_NATIVE_VISUAL_ID, &format);
00404     ANativeWindow_setBuffersGeometry(win, 0, 0, format);
00405
00406     surface = eglCreateWindowSurface(display, config, win, nullptr);
00407     if (surface == EGL_NO_SURFACE) return false;
00408 }

```

```

00409     const EGLint contextAttribs[] = {
00410         EGL_CONTEXT_CLIENT_VERSION, 3,
00411         EGL_NONE
00412     };
00413
00414     context = eglCreateContext(display, config, EGL_NO_CONTEXT, contextAttribs);
00415     if (context == EGL_NO_CONTEXT) return false;
00416
00417     if (!eglMakeCurrent(display, surface, surface, context)) return false;
00418     return true;
00419 }
00420
00421 //
00422 // EGL の破棄
00423 //
00424 void GgApp::Window::destroyEgl()
00425 {
00426     if (display != EGL_NO_DISPLAY)
00427     {
00428         eglMakeCurrent(display, EGL_NO_SURFACE, EGL_NO_SURFACE, EGL_NO_CONTEXT);
00429         if (context != EGL_NO_CONTEXT)
00430         {
00431             eglDestroyContext(display, context);
00432             context = EGL_NO_CONTEXT;
00433         }
00434         if (surface != EGL_NO_SURFACE)
00435         {
00436             eglDestroySurface(display, surface);
00437             surface = EGL_NO_SURFACE;
00438         }
00439         eglTerminate(display);
00440         display = EGL_NO_DISPLAY;
00441     }
00442 }
00443
00444 //
00445 // Window クラスのコンストラクタ (Android 版)
00446 //
00447 GgApp::Window::Window(const std::string& title, int width, int height, int fullscreen, void* share)
00448 {
00449     (void)title;
00450     (void)width;
00451     (void)height;
00452     (void)fullscreen;
00453     (void)share;
00454
00455     if (!androidApp) throw std::runtime_error("androidApp is null");
00456
00457     // Android のウィンドウが初期化されるまでイベントを処理
00458     while (androidApp->window == nullptr)
00459     {
00460         int ident;
00461         int events;
00462         struct android_poll_source* source;
00463         while ((ident = ALooper_pollOnce(0, nullptr, &events, (void**)&source)) >= 0)
00464         {
00465             if (source != nullptr) source->process(androidApp, source);
00466             if (androidApp->destroyRequested != 0) return;
00467         }
00468     }
00469
00470     window = androidApp->window;
00471
00472     if (!initEgl(window))
00473     {
00474         throw std::runtime_error("Failed to initialize EGL on Android.");
00475     }
00476
00477     int w{ ANativeWindow_getWidth(window) };
00478     int h{ ANativeWindow_getHeight(window) };
00479     size = { w, h };
00480     fboSize = { w, h };
00481     aspect = (h > 0) ? (static_cast<GLfloat>(w) / static_cast<GLfloat>(h)) : 1.0f;
00482
00483     ggInit();
00484
00485 #if defined(IMGUI_VERSION)
00486     static bool firstTime{ true };
00487     if (firstTime)
00488     {
00489         ImGui_ImplAndroid_Init(window);
00490         ImGui_ImplOpenGL3_Init("#version 310 es");
00491
00492         ImGuiIO& io{ ImGui::GetIO() };
00493         const float scale{ 2.0f };
00494         io.FontGlobalScale = scale;
00495     }

```

```

00496     ImGui::GetStyle().ScaleAllSizes(scale);
00497
00498     firstTime = false;
00499 }
00500 #endif
00501 }
00502 #else
00503 //
00504 // Window クラスのコンストラクタ
00505 //
00506 GgApp::Window::Window(const std::string& title, int width, int height, int fullscreen, GLFWwindow*
share) :
00507     size{ width, height },
00508     fboSize{ width, height }
00509 {
00510     // ディスプレイの情報
00511     GLFWmonitor* monitor{ nullptr };
00512
00513     // フルスクリーン表示
00514     if (fullscreen > 0)
00515     {
00516         // 接続されているモニタの数を数える
00517         int mcount;
00518         auto** const monitors{ glfwGetMonitors(&mcount) };
00519
00520         // セカンダリモニタがあればそれを使う
00521         if (fullscreen > mcount) fullscreen = mcount;
00522         monitor = monitors[fullscreen - 1];
00523
00524         // モニタのモードを調べる
00525         const auto* mode{ glfwGetVideoMode(monitor) };
00526
00527         // ウィンドウのサイズをディスプレイのサイズにする
00528         width = mode->width;
00529         height = mode->height;
00530     }
00531
00532     // GLFW のウィンドウを作成する
00533     window = glfwCreateWindow(width, height, title.c_str(), monitor, share);
00534
00535     // ウィンドウが作成できなければエラー
00536     if (!window) throw std::runtime_error("Unable to open the GLFW window.");
00537
00538     // 現在のウィンドウを処理対象にする
00539     glfwMakeContextCurrent(window);
00540
00541     // ゲームグラフィックス特論の都合による初期化を行う
00542     ggInit();
00543
00544     // このインスタンスの this ポインタを記録しておく
00545     glfwSetWindowUserPointer(window, this);
00546
00547     // キーボードを操作した時の処理を登録する
00548     glfwSetKeyCallback(window, keyboard);
00549
00550     // マウスボタンを操作したときの処理を登録する
00551     glfwSetMouseButtonCallback(window, mouse);
00552
00553     // マウスホイール操作時に呼び出す処理を登録する
00554     glfwSetScrollCallback(window, wheel);
00555
00556     // ウィンドウのサイズ変更時に呼び出す処理を登録する
00557     glfwSetFramebufferSizeCallback(window, resize);
00558
00559     // 垂直同期タイミングに合わせる
00560     glfwSwapInterval(1);
00561
00562     // 実際のフレームバッファのサイズを取得する
00563     glfwGetFramebufferSize(window, &width, &height);
00564
00565     // ビューポートと投影変換行列を初期化する
00566     resize(window, width, height);
00567
00568     #if defined(IMGUI_VERSION)
00569     // 最初のウィンドウを開いたとき
00570     static bool firstTime{ true };
00571     if (firstTime)
00572     {
00573         // Setup Platform/Renderer bindings
00574         ImGui_ImplGlfw_InitForOpenGL(window, true);
00575         ImGui_ImplOpenGL3_Init(nullptr);
00576
00577         // 実行済みであることを記録する
00578         firstTime = false;
00579     }
00580 #endif
00581 }

```

```

00582 #endif
00583
00584 //
00585 // Window クラスのムーブコンストラクタ
00586 //
00587 GgApp::Window::Window(Window&& w) noexcept :
00588     window{ w.window },
00589     size{ w.size },
00590     fboSize{ w.fboSize },
00591     interfaceData{ std::move(w.interfaceData) },
00592     interfaceNo{ w.interfaceNo },
00593     userPointer{ w.userPointer },
00594     resizeFunc{ w.resizeFunc },
00595     keyboardFunc{ w.keyboardFunc },
00596     mouseFunc{ w.mouseFunc },
00597     wheelFunc{ w.wheelFunc }
00598 {
00599     w.window = nullptr;
00600 #if !defined(__ANDROID__)
00601     if (window)
00602     {
00603         glfwSetWindowUserPointer(window, this);
00604     }
00605 #endif
00606 }
00607
00608 //
00609 // Window クラスのムーブ代入演算子
00610 //
00611 GgApp::Window& GgApp::Window::operator=(Window&& w) noexcept
00612 {
00613     if (&w != this)
00614     {
00615         if (window)
00616         {
00617             #if defined(__ANDROID__)
00618                 destroyEgl();
00619             #else
00620                 glfwDestroyWindow(window);
00621             #endif
00622         }
00623         window = w.window;
00624         size = w.size;
00625         fboSize = w.fboSize;
00626         interfaceData = std::move(w.interfaceData);
00627         interfaceNo = w.interfaceNo;
00628         userPointer = w.userPointer;
00629         resizeFunc = w.resizeFunc;
00630         keyboardFunc = w.keyboardFunc;
00631         mouseFunc = w.mouseFunc;
00632         wheelFunc = w.wheelFunc;
00633
00634         w.window = nullptr;
00635 #if !defined(__ANDROID__)
00636         if (window)
00637         {
00638             glfwSetWindowUserPointer(window, this);
00639         }
00640 #endif
00641     }
00642     return *this;
00643 }
00644
00645 // イベントを取得してループを継続すべきかどうか調べる
00646 //
00647 GgApp::Window::operator bool()
00648 {
00649     #if defined(__ANDROID__)
00650         if (!androidApp) return false;
00651         int ident;
00652         int events;
00653         struct android_poll_source* source;
00654         while ((ident = ALooper.pollOnce(0, nullptr, &events, (void**)&source)) >= 0)
00655         {
00656             if (source != nullptr) source->process(androidApp, source);
00657             if (androidApp->destroyRequested != 0) return false;
00658         }
00659         if (androidApp->window == nullptr) return false;
00660         int w{ ANativeWindow.getWidth(androidApp->window) };
00661         int h{ ANativeWindow.getHeight(androidApp->window) };
00662         if (w != size[0] || h != size[1])
00663         {
00664             size = { w, h };
00665             fboSize = { w, h };
00666         }
00667     }

```

```

00670     aspect = (h > 0) ? (static_cast<GLfloat>(w) / static_cast<GLfloat>(h)) : 1.0f;
00671     glViewport(0, 0, w, h);
00672 }
00673
00674 #if defined(IMGUI_VERSION)
00675     ImGui_ImplOpenGL3_NewFrame();
00676     ImGui_ImplAndroid_NewFrame();
00677     ImGui::NewFrame();
00678 #endif
00679
00680     return true;
00681 #else
00682     // イベントを取り出す
00683     glfwPollEvents();
00684
00685     // ウィンドウを閉じるべきなら false を返す
00686     if (shouldClose()) return false;
00687
00688     // 対象のユーザインタフェース
00689     auto& current_if{ interfaceData[interfaceNo] };
00690
00691 #if defined(IMGUI_VERSION)
00692     // ImGui の新規フレームを作成する
00693     ImGui_ImplOpenGL3_NewFrame();
00694     ImGui_ImplGlfw_NewFrame();
00695     ImGui::NewFrame();
00696
00697     // ImGui の状態を取り出す
00698     const ImGuiIO& io{ ImGui::GetIO() };
00699
00700     // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00701     if (io.WantCaptureMouse) return true;
00702
00703     // マウスの位置を更新する
00704     current_if.mouse = std::array<GLfloat, 2>{ io.MousePos.x, io.MousePos.y };
00705 #else
00706     // マウスの現在位置を調べる
00707     double x, y;
00708     glfwGetCursorPos(window, &x, &y);
00709
00710     // マウスの位置を更新する
00711     current_if.mouse = std::array<GLfloat, 2>{ static_cast<GLfloat>(x), static_cast<GLfloat>(y) };
00712 #endif
00713
00714     // マウスドラッグ
00715     for (int button = GLFW_MOUSE_BUTTON_1; button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT; ++button)
00716     {
00717         // マウスボタンを押していたら
00718         if (status[button])
00719         {
00720             // 現在位置と平行移動量を更新する
00721             current_if.calcTranslation(button, velocity);
00722         }
00723     }
00724
00725     return true;
00726 #endif
00727 }
00728
00729 //
00730 // カラーバッファを入れ替える
00731 //
00732 void GgApp::Window::swapBuffers() const
00733 {
00734     #if defined(IMGUI_VERSION)
00735         // ImGui の描画データがあればフレームをレンダリングする
00736         ImGui::Render();
00737         ImGuiDrawData* data{ ImGui::GetDrawData() };
00738         if (data) ImGui_ImplOpenGL3_RenderDrawData(data);
00739     #endif
00740
00741     // エラーチェック
00742     ggError();
00743
00744     #if defined(_ANDROID_)
00745         if (display != EGL_NO_DISPLAY && surface != EGL_NO_SURFACE)
00746         {
00747             eglSwapBuffers(display, surface);
00748         }
00749     #else
00750         // カラーバッファを入れ替える
00751         glfwSwapBuffers(window);
00752     #endif
00753 }
00754
00755 //
00756 // ビューポートのサイズを更新する

```



```

00757 //
00758 void GgApp::Window::updateViewport ()
00759 {
00760     #if defined(__ANDROID__)
00761         if (window)
00762         {
00763             fboSize[0] = ANativeWindow.getWidth(window);
00764             fboSize[1] = ANativeWindow.getHeight(window);
00765         }
00766     #else
00767         // フレームバッファの大きさを求める
00768         glfwGetFramebufferSize(window, &fboSize[0], &fboSize[1]);
00769     #endif
00770
00771     #if defined(IMGUI_VERSION)
00772         // フレームバッファの高さからメニューバーの高さを減じる
00773         fboSize[1] -= menubarHeight;
00774     #endif
00775
00776     // ウィンドウの縦横比を保存する
00777     aspect = static_cast<GLfloat>(fboSize[0]) / static_cast<GLfloat>(fboSize[1]);
00778
00779     // ビューポートを設定する
00780     restoreViewport();
00781 }
00782
00783 #if defined(GG_USE_OPENXR)
00784
00785 namespace
00786 {
00787     //
00788     // OpenXR の関数の戻り値を文字列にする
00789     //
00790     std::string xrMessage(XrInstance instance, XrResult result, const std::string& message)
00791     {
00792         char buffer[XR_MAX_RESULT_STRING_SIZE]{ '\0' };
00793         if (instance == XR_NULL_HANDLE
00794             || XR_FAILED(xrResultToString(instance, result, buffer))
00795             || buffer[0] == '\0')
00796         {
00797             std::snprintf(buffer, sizeof buffer, "XrResult(%d)", static_cast<int>(result));
00798         }
00799         return message + ": " + buffer;
00800     }
00801
00802     //
00803     // OpenXR の関数の戻り値を検査して、エラーなら例外を投げる
00804     //
00805     void xrCheck(XrInstance instance, XrResult result, const std::string& message)
00806     {
00807         if (XR_SUCCEEDED(result)) return;
00808         throw std::runtime_error(xrMessage(instance, result, message));
00809     }
00810
00811     //
00812     // OpenXR の関数の戻り値を検査して、エラーなら標準エラー出力に報告する
00813     //
00814     bool xrWarn(XrInstance instance, XrResult result, const std::string& message)
00815     {
00816         if (XR_SUCCEEDED(result)) return true;
00817         std::cerr << "OpenXR: " << xrMessage(instance, result, message) << '\n';
00818         return false;
00819     }
00820
00821     //
00822     // 固定長の文字列に安全にコピーする
00823     //
00824     void xrCopyString(char* destination, size_t size, const char* source)
00825     {
00826         if (size == 0) return;
00827         const auto length{ std::min(std::strlen(source), size - 1) };
00828         std::memcpy(destination, source, length);
00829         destination[length] = '\0';
00830     }
00831 }
00832
00833 //
00834 // コンストラクタ
00835 //
00836 GgApp::OpenXR::OpenXR()
00837 {
00838 }
00839
00840 //
00841 // デストラクタ
00842 //
00843 GgApp::OpenXR::~OpenXR()

```

```

00844 {
00845     // このオブジェクトは関数内 static なので、破棄されるのは main() が
00846     // 終了した後、すなわち OpenGL のコンテキストが失われた後である。
00847     // したがってここでは OpenGL の資源には触れず、OpenXR のハンドルだけを
00848     // 解放する (OpenGL の資源は terminate() で解放しておくこと)。
00849     destroyXr();
00850 }
00851
00852 //
00853 // アクションシステムを初期化する
00854 //
00855 void GgApp::OpenXR::initActions()
00856 {
00857     // アクションセットの作成
00858     XrActionSetCreateInfo actionSetInfo{ XR_TYPE_ACTION_SET_CREATE_INFO };
00859     xrCopyString(actionSetInfo.actionSetName, sizeof actionSetInfo.actionSetName, "gameplay");
00860     xrCopyString(actionSetInfo.localizedActionSetName, sizeof actionSetInfo.localizedActionSetName,
00861 "Gameplay");
00862     actionSetInfo.priority = 0;
00863     xrCheck(instance, xrCreateActionSet(instance, &actionSetInfo, &actionSet),
00864 "Can't create the OpenXR action set");
00865
00866     // サブアクションパスの取得
00867     xrCheck(instance, xrStringToPath(instance, "/user/hand/left", &handSubactionPath[Hand::Left]),
00868 "Can't convert the path of the left hand");
00869     xrCheck(instance, xrStringToPath(instance, "/user/hand/right", &handSubactionPath[Hand::Right]),
00870 "Can't convert the path of the right hand");
00871
00872     // アクション作成ヘルパー
00873     auto createAction = [this](const char* name, const char* localizedName, XrActionType type, XrAction&
00874 action)
00875     {
00876         XrActionCreateInfo createInfo{ XR_TYPE_ACTION_CREATE_INFO };
00877         xrCopyString(createInfo.actionName, sizeof createInfo.actionName, name);
00878         xrCopyString(createInfo.localizedActionName, sizeof createInfo.localizedActionName,
00879 localizedName);
00880         createInfo.actionType = type;
00881         createInfo.countSubactionPaths = Hand::Count;
00882         createInfo.subactionPaths = handSubactionPath;
00883         xrCheck(instance, xrCreateAction(actionSet, &createInfo, &action),
00884 "std::string(\"Can't create the OpenXR action \" + name + \"\")");
00885     };
00886
00887     createAction("aim_pose", "Aim Pose", XR_ACTION_TYPE_POSE_INPUT, aimPoseAction);
00888     createAction("grip_pose", "Grip Pose", XR_ACTION_TYPE_POSE_INPUT, gripPoseAction);
00889     createAction("trigger", "Trigger", XR_ACTION_TYPE_FLOAT_INPUT, triggerAction);
00890     createAction("grip", "Grip", XR_ACTION_TYPE_FLOAT_INPUT, gripAction);
00891     createAction("thumbstick", "Thumbstick", XR_ACTION_TYPE_VECTOR2F_INPUT, thumbstickAction);
00892     createAction("thumbstick_click", "Thumbstick Click", XR_ACTION_TYPE_BOOLEAN_INPUT,
00893 thumbstickClickAction);
00894     createAction("primary_button", "Primary Button", XR_ACTION_TYPE_BOOLEAN_INPUT, primaryButtonAction);
00895     createAction("secondary_button", "Secondary Button", XR_ACTION_TYPE_BOOLEAN_INPUT,
00896 secondaryButtonAction);
00897     createAction("menu_button", "Menu Button", XR_ACTION_TYPE_BOOLEAN_INPUT, menuButtonAction);
00898     createAction("haptic", "Haptic Vibration", XR_ACTION_TYPE_VIBRATION_OUTPUT, hapticAction);
00899
00900     // バインディング設定ヘルパー (対応していない対話プロファイルは読み飛ばす)
00901     auto suggestBindings = [this](const char* profileStr, const std::vector<std::pair<XrAction, const
00902 char*>>& bindings)
00903     {
00904         XrPath profilePath{ XR_NULL_PATH };
00905         if (XR_FAILED(xrStringToPath(instance, profileStr, &profilePath))) return;
00906
00907         std::vector<XrActionSuggestedBinding> suggestedBindings;
00908         suggestedBindings.reserve(bindings.size());
00909         for (const auto& [action, pathStr] : bindings)
00910         {
00911             XrPath path{ XR_NULL_PATH };
00912             if (XR_FAILED(xrStringToPath(instance, pathStr, &path))) continue;
00913             suggestedBindings.push_back(XrActionSuggestedBinding{ action, path });
00914         }
00915         if (suggestedBindings.empty()) return;
00916
00917         XrInteractionProfileSuggestedBinding profileSuggestedBindings{
00918 XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING };
00919         profileSuggestedBindings.interactionProfile = profilePath;
00920         profileSuggestedBindings.suggestedBindings = suggestedBindings.data();
00921         profileSuggestedBindings.countSuggestedBindings = static_cast<uint32_t>(suggestedBindings.size());
00922         xrWarn(instance, xrSuggestInteractionProfileBindings(instance, &profileSuggestedBindings),
00923 "std::string(\"Can't suggest the bindings for \" + profileStr);
00924     };
00925
00926     // Simple Controller のバインディング (すべてのランタイムが対応する最小限のもの)
00927     suggestBindings("/interaction_profiles/khr/simple_controller", {
00928         { aimPoseAction, "/user/hand/left/input/aim/pose" },
00929         { aimPoseAction, "/user/hand/right/input/aim/pose" },
00930         { gripPoseAction, "/user/hand/left/input/grip/pose" },
00931     },

```

```

00924     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00925     { triggerAction, "/user/hand/left/input/select/click" },
00926     { triggerAction, "/user/hand/right/input/select/click" },
00927     { menuButtonAction, "/user/hand/left/input/menu/click" },
00928     { menuButtonAction, "/user/hand/right/input/menu/click" },
00929     { hapticAction, "/user/hand/left/output/haptic" },
00930     { hapticAction, "/user/hand/right/output/haptic" }
00931   });
00932
00933   // Meta (Oculus) Touch コントローラーのバインディング
00934   suggestBindings("/interaction_profiles/oculus/touch_controller", {
00935     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00936     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00937     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00938     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00939     { triggerAction, "/user/hand/left/input/trigger/value" },
00940     { triggerAction, "/user/hand/right/input/trigger/value" },
00941     { gripAction, "/user/hand/left/input/squeeze/value" },
00942     { gripAction, "/user/hand/right/input/squeeze/value" },
00943     { thumbstickAction, "/user/hand/left/input/thumbstick" },
00944     { thumbstickAction, "/user/hand/right/input/thumbstick" },
00945     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },
00946     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
00947     { primaryButtonAction, "/user/hand/left/input/x/click" },
00948     { primaryButtonAction, "/user/hand/right/input/a/click" },
00949     { secondaryButtonAction, "/user/hand/left/input/y/click" },
00950     { secondaryButtonAction, "/user/hand/right/input/b/click" },
00951     { menuButtonAction, "/user/hand/left/input/menu/click" },
00952     { hapticAction, "/user/hand/left/output/haptic" },
00953     { hapticAction, "/user/hand/right/output/haptic" }
00954   });
00955
00956   // HTC Vive コントローラーのバインディング
00957   suggestBindings("/interaction_profiles/htc/vive_controller", {
00958     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00959     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00960     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00961     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00962     { triggerAction, "/user/hand/left/input/trigger/value" },
00963     { triggerAction, "/user/hand/right/input/trigger/value" },
00964     { gripAction, "/user/hand/left/input/squeeze/click" },
00965     { gripAction, "/user/hand/right/input/squeeze/click" },
00966     { thumbstickAction, "/user/hand/left/input/trackpad" },
00967     { thumbstickAction, "/user/hand/right/input/trackpad" },
00968     { thumbstickClickAction, "/user/hand/left/input/trackpad/click" },
00969     { thumbstickClickAction, "/user/hand/right/input/trackpad/click" },
00970     { menuButtonAction, "/user/hand/left/input/menu/click" },
00971     { menuButtonAction, "/user/hand/right/input/menu/click" },
00972     { hapticAction, "/user/hand/left/output/haptic" },
00973     { hapticAction, "/user/hand/right/output/haptic" }
00974   });
00975
00976   // Valve Index コントローラーのバインディング
00977   suggestBindings("/interaction_profiles/valve/index_controller", {
00978     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00979     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00980     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00981     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00982     { triggerAction, "/user/hand/left/input/trigger/value" },
00983     { triggerAction, "/user/hand/right/input/trigger/value" },
00984     { gripAction, "/user/hand/left/input/squeeze/value" },
00985     { gripAction, "/user/hand/right/input/squeeze/value" },
00986     { thumbstickAction, "/user/hand/left/input/thumbstick" },
00987     { thumbstickAction, "/user/hand/right/input/thumbstick" },
00988     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },
00989     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
00990     { primaryButtonAction, "/user/hand/left/input/a/click" },
00991     { primaryButtonAction, "/user/hand/right/input/a/click" },
00992     { secondaryButtonAction, "/user/hand/left/input/b/click" },
00993     { secondaryButtonAction, "/user/hand/right/input/b/click" },
00994     { hapticAction, "/user/hand/left/output/haptic" },
00995     { hapticAction, "/user/hand/right/output/haptic" }
00996   });
00997
00998   // Microsoft Mixed Reality モーションコントローラーのバインディング
00999   suggestBindings("/interaction_profiles/microsoft/motion_controller", {
01000     { aimPoseAction, "/user/hand/left/input/aim/pose" },
01001     { aimPoseAction, "/user/hand/right/input/aim/pose" },
01002     { gripPoseAction, "/user/hand/left/input/grip/pose" },
01003     { gripPoseAction, "/user/hand/right/input/grip/pose" },
01004     { triggerAction, "/user/hand/left/input/trigger/value" },
01005     { triggerAction, "/user/hand/right/input/trigger/value" },
01006     { gripAction, "/user/hand/left/input/squeeze/click" },
01007     { gripAction, "/user/hand/right/input/squeeze/click" },
01008     { thumbstickAction, "/user/hand/left/input/thumbstick" },
01009     { thumbstickAction, "/user/hand/right/input/thumbstick" },
01010     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },

```

```

01011     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
01012     { menuButtonAction, "/user/hand/left/input/menu/click" },
01013     { menuButtonAction, "/user/hand/right/input/menu/click" },
01014     { hapticAction, "/user/hand/left/output/haptic" },
01015     { hapticAction, "/user/hand/right/output/haptic" }
01016 });
01017
01018 // セッションにアクションセットをアタッチする (これ以降はバインディングを追加できない)
01019 XrSessionActionSetsAttachInfo attachInfo{ XR_TYPE_SESSION_ACTION_SETS_ATTACH_INFO };
01020 attachInfo.countActionSets = 1;
01021 attachInfo.actionSets = &actionSet;
01022 xrCheck(instance, xrAttachSessionActionSets(session, &attachInfo),
01023         "Can't attach the OpenXR action set to the session");
01024
01025 // アクションスペースの作成
01026 for (int i = 0; i < Hand::Count; ++i)
01027 {
01028     XrActionSpaceCreateInfo spaceInfo{ XR_TYPE_ACTION_SPACE_CREATE_INFO };
01029     spaceInfo.poseInActionSpace.orientation.w = 1.0f;
01030     spaceInfo.subactionPath = handSubactionPath[i];
01031
01032     spaceInfo.action = aimPoseAction;
01033     xrCheck(instance, xrCreateActionSpace(session, &spaceInfo, &aimSpace[i]),
01034             "Can't create the OpenXR action space for the aim pose");
01035
01036     spaceInfo.action = gripPoseAction;
01037     xrCheck(instance, xrCreateActionSpace(session, &spaceInfo, &gripSpace[i]),
01038             "Can't create the OpenXR action space for the grip pose");
01039 }
01040 }
01041
01042 //
01043 // アクション状態を更新する
01044 //
01045 void GgApp::OpenXR::pollActions()
01046 {
01047     // 入力を受け付けていなければコントローラーの状態を無効にする
01048     if (!isSessionRunning || actionSet == XR_NULL_HANDLE || sessionState != XR_SESSION_STATE_FOCUSED)
01049     {
01050         for (auto& state : controllerStates) state = ControllerState{};
01051         return;
01052     }
01053
01054     XrActiveActionSet activeActionSet{ actionSet, XR_NULL_PATH };
01055     XrActionsSyncInfo syncInfo{ XR_TYPE_ACTIONS_SYNC_INFO };
01056     syncInfo.countActiveActionSets = 1;
01057     syncInfo.activeActionSets = &activeActionSet;
01058     if (!xrWarn(instance, xrSyncActions(session, &syncInfo),
01059             "Can't synchronize the OpenXR actions")) return;
01060
01061     // 空間の姿勢を取り出すヘルパー (位置と向きが両方が有効なときだけ更新する)
01062     auto locate = [this](XrSpace space, XrPosef& pose)
01063     {
01064         if (space == XR_NULL_HANDLE) return false;
01065
01066         XrSpaceLocation location{ XR_TYPE_SPACE_LOCATION };
01067         if (XR_FAILED(xrLocateSpace(space, appSpace, frameState.predictedDisplayTime, &location)))
01068             return false;
01069
01070         constexpr XrSpaceLocationFlags valid
01071         {
01072             XR_SPACE_LOCATION_POSITION_VALID_BIT | XR_SPACE_LOCATION_ORIENTATION_VALID_BIT
01073         };
01074         if ((location.locationFlags & valid) != valid) return false;
01075
01076         pose = location.pose;
01077         return true;
01078     };
01079
01080     for (int i = 0; i < Hand::Count; ++i)
01081     {
01082         auto& state = controllerStates[i];
01083         const XrPath subaction = handSubactionPath[i];
01084
01085         XrActionStateGetInfo getInfo{ XR_TYPE_ACTION_STATE_GET_INFO };
01086         getInfo.subactionPath = subaction;
01087
01088         // グリップポーズの取得
01089         XrActionStatePose gripPoseState{ XR_TYPE_ACTION_STATE_POSE };
01090         getInfo.action = gripPoseAction;
01091         const bool gripActive
01092         {
01093             XR_SUCCEEDED(xrGetActionStatePose(session, &getInfo, &gripPoseState))
01094             && gripPoseState.isActive != XR_FALSE
01095         };
01096
01097         // エイムポーズの取得

```

```

01098     XrActionStatePose aimPoseState{ XR.TYPE_ACTION.STATE_POSE };
01099     getInfo.action = aimPoseAction;
01100     const bool aimActive
01101     {
01102         XR.SUCCEEDED(xrGetActionStatePose(session, &getInfo, &aimPoseState))
01103         && aimPoseState.isActive != XR.FALSE
01104     };
01105
01106     // どちらかの姿勢が有効ならコントローラーが接続されている
01107     state.isTracked = gripActive || aimActive;
01108     if (gripActive) locate(gripSpace[i], state.gripPose);
01109     if (aimActive) locate(aimSpace[i], state.aimPose);
01110
01111     // 連続値のアクションを取り出すヘルパー
01112     auto getFloat = [this, &getInfo](XrAction action, float& value)
01113     {
01114         getInfo.action = action;
01115         XrActionStateFloat floatState{ XR.TYPE_ACTION.STATE_FLOAT };
01116         value = XR.SUCCEEDED(xrGetActionStateFloat(session, &getInfo, &floatState))
01117             && floatState.isActive != XR.FALSE ? floatState.currentState : 0.0f;
01118     };
01119
01120     // 論理値のアクションを取り出すヘルパー
01121     auto getBoolean = [this, &getInfo](XrAction action, bool& value)
01122     {
01123         getInfo.action = action;
01124         XrActionStateBoolean booleanState{ XR.TYPE_ACTION.STATE_BOOLEAN };
01125         value = XR.SUCCEEDED(xrGetActionStateBoolean(session, &getInfo, &booleanState))
01126             && booleanState.isActive != XR.FALSE && booleanState.currentState != XR.FALSE;
01127     };
01128
01129     // トリガーとグリップ
01130     getFloat(triggerAction, state.trigger);
01131     getFloat(gripAction, state.grip);
01132
01133     // スティック
01134     getInfo.action = thumbstickAction;
01135     XrActionStateVector2f thumbstickState{ XR.TYPE_ACTION.STATE_VECTOR2F };
01136     if (XR.SUCCEEDED(xrGetActionStateVector2f(session, &getInfo, &thumbstickState))
01137         && thumbstickState.isActive != XR.FALSE)
01138         state.thumbstick = { thumbstickState.currentState.x, thumbstickState.currentState.y };
01139     else
01140         state.thumbstick = { 0.0f, 0.0f };
01141
01142     // ボタン
01143     getBoolean(thumbstickClickAction, state.thumbstickClick);
01144     getBoolean(primaryButtonAction, state.primaryButton);
01145     getBoolean(secondaryButtonAction, state.secondaryButton);
01146     getBoolean(menuButtonAction, state.menuButton);
01147 }
01148 }
01149
01150 //
01151 // OpenXR のイベントを処理する
01152 //
01153 void GgApp::OpenXR::pollEvents()
01154 {
01155     XrEventDataBuffer eventData{ XR.TYPE_EVENT.DATA_BUFFER };
01156
01157     while (xrPollEvent(instance, &eventData) == XR.SUCCESS)
01158     {
01159         switch (eventData.type)
01160         {
01161             case XR.TYPE_EVENT.DATA_INSTANCE_LOSS_PENDING:
01162
01163                 // OpenXR のインスタンスが失われるのでアプリケーションを終了する
01164                 isSessionRunning = false;
01165                 if (window) window->setClose(GLFW.TRUE);
01166                 break;
01167
01168             case XR.TYPE_EVENT.DATA_SESSION_STATE_CHANGED:
01169             {
01170                 const auto* stateChanged{ reinterpret_cast<const XrEventDataSessionStateChanged*>(&eventData) };
01171                 sessionState = stateChanged->state;
01172
01173                 switch (sessionState)
01174                 {
01175                     case XR.SESSION.STATE_READY:
01176                     {
01177                         // セッションを開始する
01178                         XrSessionBeginInfo beginInfo{ XR.TYPE_SESSION.BEGIN_INFO };
01179                         beginInfo.primaryViewConfigurationType = XR.VIEW_CONFIGURATION.TYPE_PRIMARY_STEREO;
01180                         if (xrWarn(instance, xrBeginSession(session, &beginInfo),
01181                             "Can't begin the OpenXR session")) isSessionRunning = true;
01182                         break;
01183                     }
01184

```

```

01185     case XR_SESSION_STATE_STOPPING:
01186
01187         // セッションを終了する
01188         isSessionRunning = false;
01189         frameBegun = false;
01190         xrWarn(instance, xrEndSession(session), "Can't end the OpenXR session");
01191         break;
01192
01193     case XR_SESSION_STATE_EXITING:
01194     case XR_SESSION_STATE_LOSS_PENDING:
01195
01196         // アプリケーションを終了する
01197         isSessionRunning = false;
01198         if (window) window->setClose(GLFW_TRUE);
01199         break;
01200
01201     default:
01202         break;
01203 }
01204 break;
01205 }
01206
01207 default:
01208     break;
01209 }
01210
01211 // 次のイベントを取り出す準備をする
01212 eventData = XrEventDataBuffer{ XR_TYPE_EVENT_DATA_BUFFER };
01213 }
01214 }
01215
01216 //
01217 // スワップチェーンを作成する
01218 //
01219 void GgApp::OpenXR::createSwapchains()
01220 {
01221     // ビュー構成を取得する
01222     uint32_t viewCount{ 0 };
01223     xrCheck(instance, xrEnumerateViewConfigurationViews(instance, systemId,
01224         XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, 0, &viewCount, nullptr),
01225         "Can't count the OpenXR view configuration views");
01226     views.assign(viewCount, XrViewConfigurationView{ XR_TYPE_VIEW_CONFIGURATION_VIEW });
01227     xrCheck(instance, xrEnumerateViewConfigurationViews(instance, systemId,
01228         XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, viewCount, &viewCount, views.data()),
01229         "Can't enumerate the OpenXR view configuration views");
01230     if (viewCount == 0) throw std::runtime_error("The OpenXR system has no view");
01231
01232     viewStates.assign(viewCount, XrView{ XR_TYPE_VIEW });
01233     currentIndex.assign(viewCount, 0);
01234     imageAcquired.assign(viewCount, false);
01235
01236     // 環境の合成方法を取得する (最初のが最も推奨される)
01237     uint32_t blendModeCount{ 0 };
01238     if (XR_SUCCEEDED(xrEnumerateEnvironmentBlendModes(instance, systemId,
01239         XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, 0, &blendModeCount, nullptr))
01240         && blendModeCount > 0)
01241     {
01242         std::vector<XrEnvironmentBlendMode> blendModes(blendModeCount);
01243         if (XR_SUCCEEDED(xrEnumerateEnvironmentBlendModes(instance, systemId,
01244             XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, blendModeCount,
01245             &blendModeCount, blendModes.data())))
01246         {
01247             blendMode = blendModes[0];
01248         }
01249     }
01250
01251     // 利用可能なスワップチェーンのカラーフォーマットを取得する
01252     uint32_t formatCount{ 0 };
01253     xrCheck(instance, xrEnumerateSwapchainFormats(session, 0, &formatCount, nullptr),
01254         "Can't count the OpenXR swapchain formats");
01255     std::vector<int64_t> formats(formatCount);
01256     xrCheck(instance, xrEnumerateSwapchainFormats(session, formatCount, &formatCount, formats.data()),
01257         "Can't enumerate the OpenXR swapchain formats");
01258     if (formats.empty()) throw std::runtime_error("The OpenXR runtime has no swapchain format");
01259
01260     // 使用したいカラーフォーマットの候補 (前にあるものを優先する)
01261     static const int64_t preferred[]{ GL_SRGB8_ALPHA8, GL_SRGB8, GL_RGBA8, GL_RGB10_A2 };
01262
01263     // 利用可能なカラーフォーマットの中から使用するものを選ぶ
01264     int64_t format{ formats[0] };
01265     for (const auto candidate : preferred)
01266     {
01267         if (std::find(formats.begin(), formats.end(), candidate) != formats.end())
01268         {
01269             format = candidate;
01270             break;
01271         }
01272     }

```

```

01272 }
01273
01274 // sRGB のフォーマットならリニア色空間で描画してガンマ補正をランタイムに任せる
01275 swapchainIsSrgb = format == GL_SRGB8_ALPHA8 || format == GL_SRGB8;
01276
01277 // ビューの数だけ FBO とデプスバッファを作成する
01278 openxrFbo.assign(viewCount, 0);
01279 openxrDepth.assign(viewCount, 0);
01280 glGenFramebuffers(static_cast<GLsizei>(viewCount), openxrFbo.data());
01281 glGenRenderbuffers(static_cast<GLsizei>(viewCount), openxrDepth.data());
01282
01283 for (uint32_t i = 0; i < viewCount; ++i)
01284 {
01285     // スワップチェーンを作成する
01286     XrSwapchainCreateInfo swapchainCreateInfo{ XR_TYPE_SWAPCHAIN_CREATE_INFO };
01287     swapchainCreateInfo.usageFlags = XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT
01288         | XR_SWAPCHAIN_USAGE_SAMPLED_BIT;
01289     swapchainCreateInfo.format = format;
01290     swapchainCreateInfo.sampleCount = 1;
01291     swapchainCreateInfo.width = views[i].recommendedImageRectWidth;
01292     swapchainCreateInfo.height = views[i].recommendedImageRectHeight;
01293     swapchainCreateInfo.faceCount = 1;
01294     swapchainCreateInfo.arraySize = 1;
01295     swapchainCreateInfo.mipCount = 1;
01296
01297     XrSwapchain swapchain{ XR_NULL_HANDLE };
01298     xrCheck(instance, xrCreateSwapchain(session, &swapchainCreateInfo, &swapchain),
01299         "Can't create the OpenXR swapchain");
01300     swapchains.push_back(swapchain);
01301
01302     // スワップチェーンのイメージを取得する
01303     uint32_t imageCount{ 0 };
01304     xrCheck(instance, xrEnumerateSwapchainImages(swapchain, 0, &imageCount, nullptr),
01305         "Can't count the OpenXR swapchain images");
01306     std::vector<XrSwapchainImageOpenGLKHR> images(imageCount,
01307         XrSwapchainImageOpenGLKHR{ XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_KHR });
01308     xrCheck(instance, xrEnumerateSwapchainImages(swapchain, imageCount, &imageCount,
01309         reinterpret_cast<XrSwapchainImageBaseHeader*>(images.data())),
01310         "Can't enumerate the OpenXR swapchain images");
01311     swapchainImages.push_back(std::move(images));
01312
01313     // 隠面消去処理に使うデプスバッファを作成する
01314     glBindRenderbuffer(GL_RENDERBUFFER, openxrDepth[i]);
01315     glRenderbufferStorage(GL_RENDERBUFFER, GL_DEPTH24_STENCIL8,
01316         static_cast<GLsizei>(swapchainCreateInfo.width),
01317         static_cast<GLsizei>(swapchainCreateInfo.height));
01318     glBindRenderbuffer(GL_RENDERBUFFER, 0);
01319
01320     // FBO にデプスバッファを取り付けておく (カラーバッファは select() で取り付ける)
01321     glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[i]);
01322     glFramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_STENCIL_ATTACHMENT,
01323         GL_RENDERBUFFER, openxrDepth[i]);
01324 }
01325
01326 glBindFramebuffer(GL_FRAMEBUFFER, 0);
01327 }
01328
01329 //
01330 // OpenXR のセッションを作成する
01331 //
01332 GgApp::OpenXR& GgApp::OpenXR::initialize(const Window& window,
01333     XrReferenceSpaceType spaceType, const char* appName)
01334 {
01335     static OpenXR openxr;
01336
01337     // 初期化済みならそのまま返す
01338     if (openxr.initialized) return openxr;
01339
01340     // 初期化に失敗していた場合に備えて後始末をしておく
01341     openxr.terminate();
01342
01343     openxr.window = &window;
01344     openxr.referenceSpaceType = spaceType;
01345
01346     try
01347     {
01348         // 利用可能な拡張機能を調べる
01349         uint32_t extensionCount{ 0 };
01350         xrCheck(XR_NULL_HANDLE,
01351             xrEnumerateInstanceExtensionProperties(nullptr, 0, &extensionCount, nullptr),
01352             "Can't count the OpenXR instance extensions");
01353         std::vector<XrExtensionProperties> extensionProperties(extensionCount,
01354             XrExtensionProperties{ XR_TYPE_EXTENSION_PROPERTIES });
01355         xrCheck(XR_NULL_HANDLE,
01356             xrEnumerateInstanceExtensionProperties(nullptr, extensionCount,
01357                 &extensionCount, extensionProperties.data()),
01358             "Can't enumerate the OpenXR instance extensions");
01359     }

```



```

01359
01360 // OpenGL との連携に必要な拡張機能が使えなければあきらめる
01361 const auto found{ std::any_of(extensionProperties.begin(), extensionProperties.end(),
01362     [](const XrExtensionProperties& p)
01363     {
01364         return std::strcmp(p.extensionName, XR_KHR_OPENGL_ENABLE_EXTENSION_NAME) == 0;
01365     }) };
01366 if (!found)
01367 {
01368     throw std::runtime_error(
01369         "The OpenXR runtime does not support " XR_KHR_OPENGL_ENABLE_EXTENSION_NAME);
01370 }
01371
01372 // XrInstance の作成
01373 XrInstanceCreateInfo createInfo{ XR_TYPE_INSTANCE_CREATE_INFO };
01374 xrCopyString(createInfo.applicationInfo.applicationName,
01375     sizeof createInfo.applicationInfo.applicationName, appName ? appName : "GgApp");
01376 createInfo.applicationInfo.applicationVersion = 1;
01377 xrCopyString(createInfo.applicationInfo.engineName,
01378     sizeof createInfo.applicationInfo.engineName, "GgApp");
01379 createInfo.applicationInfo.engineVersion = 1;
01380 createInfo.applicationInfo.apiVersion = XR_CURRENT_API_VERSION;
01381
01382 const char* const extensions[]{ XR_KHR_OPENGL_ENABLE_EXTENSION_NAME };
01383 createInfo.enabledExtensionCount = 1;
01384 createInfo.enabledExtensionNames = extensions;
01385
01386 xrCheck(XR_NULL_HANDLE, xrCreateInstance(&createInfo, &openxr.instance),
01387     "Can't create the OpenXR instance (is an OpenXR runtime installed and active?)");
01388
01389 // システムの取得
01390 XrSystemGetInfo systemInfo{ XR_TYPE_SYSTEM_GET_INFO };
01391 systemInfo.formFactor = XR_FORM_FACTOR_HEAD_MOUNTED_DISPLAY;
01392 xrCheck(openxr.instance, xrGetSystem(openxr.instance, &systemInfo, &openxr.systemId),
01393     "Can't get the OpenXR system (is the head mounted display connected?)");
01394
01395 // システムの名前の取得
01396 XrSystemProperties systemProperties{ XR_TYPE_SYSTEM_PROPERTIES };
01397 if (XR_SUCCEEDED(xrGetSystemProperties(openxr.instance, openxr.systemId, &systemProperties)))
01398 {
01399     openxr.systemName = systemProperties.systemName;
01400 }
01401
01402 // OpenGL との連携に必要な拡張機能の関数の取得
01403 PFN_xrGetOpenGLGraphicsRequirementsKHR pfnGetOpenGLGraphicsRequirementsKHR{ nullptr };
01404 xrCheck(openxr.instance, xrGetInstanceProcAddr(openxr.instance,
01405     "xrGetOpenGLGraphicsRequirementsKHR",
01406     reinterpret_cast<PFN_xrVoidFunction*>(&pfnGetOpenGLGraphicsRequirementsKHR)),
01407     "Can't get the address of xrGetOpenGLGraphicsRequirementsKHR");
01408 if (!pfnGetOpenGLGraphicsRequirementsKHR)
01409     throw std::runtime_error("Can't get the address of xrGetOpenGLGraphicsRequirementsKHR");
01410
01411 // OpenGL の要件の取得 (セッションの作成前に必ず呼ばなければならない)
01412 XrGraphicsRequirementsOpenGLKHR graphicsRequirements{ XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_KHR };
01413 xrCheck(openxr.instance, pfnGetOpenGLGraphicsRequirementsKHR(openxr.instance,
01414     openxr.systemId, &graphicsRequirements),
01415     "Can't get the OpenGL graphics requirements");
01416
01417 // OpenGL のバージョンが要件を満たしているかどうか調べる
01418 GLint major{ 0 }, minor{ 0 };
01419 glGetIntegerv(GL_MAJOR_VERSION, &major);
01420 glGetIntegerv(GL_MINOR_VERSION, &minor);
01421 if (XR_MAKE_VERSION(major, minor, 0) < graphicsRequirements.minApiVersionSupported)
01422 {
01423     char message[128];
01424     std::snprintf(message, sizeof message,
01425         "The OpenXR runtime requires OpenGL %d.%d or later, but %d.%d is current",
01426         static_cast<int>(XR_VERSION_MAJOR(graphicsRequirements.minApiVersionSupported)),
01427         static_cast<int>(XR_VERSION_MINOR(graphicsRequirements.minApiVersionSupported)),
01428         major, minor);
01429     throw std::runtime_error(message);
01430 }
01431
01432 // OpenGL のコンテキストをセッションに結びつける
01433 #if defined(XR_USE_PLATFORM_WIN32)
01434 XrGraphicsBindingOpenGLWin32KHR graphicsBinding{ XR_TYPE_GRAPHICS_BINDING_OPENGL_WIN32_KHR };
01435 graphicsBinding.hDC = wglGetCurrentDC();
01436 graphicsBinding.hGLRC = glfwGetWGLContext(window.get());
01437 #elif defined(XR_USE_PLATFORM_XLIB)
01438 XrGraphicsBindingOpenGLXlibKHR graphicsBinding{ XR_TYPE_GRAPHICS_BINDING_OPENGL_XLIB_KHR };
01439 graphicsBinding.xDisplay = glfwGetXlibDisplay();
01440 graphicsBinding.visualid = 0;
01441 graphicsBinding.glxFBConfig = nullptr;
01442 graphicsBinding.glxDrawable = glfwGetGLXWindow(window.get());
01443 graphicsBinding.glxContext = glfwGetGLXContext(window.get());
01444 #else
01445 # error "GG_USE_OPENXR is not supported on this platform"

```



```

01446 #endif
01447
01448 // セッションの作成
01449 XrSessionCreateInfo sessionCreateInfo{ XR_TYPE_SESSION_CREATE_INFO };
01450 sessionCreateInfo.next = &graphicsBinding;
01451 sessionCreateInfo.systemId = openxr.systemId;
01452 xrCheck(openxr.instance, xrCreateSession(openxr.instance, &sessionCreateInfo, &openxr.session),
01453         "Can't create the OpenXR session");
01454
01455 // 参照空間の作成 (要求されたものが使えなければ LOCAL にフォールバックする)
01456 XrReferenceSpaceCreateInfo spaceCreateInfo{ XR_TYPE_REFERENCE_SPACE_CREATE_INFO };
01457 spaceCreateInfo.referenceSpaceType = openxr.referenceSpaceType;
01458 spaceCreateInfo.poseInReferenceSpace.orientation.w = 1.0f;
01459 if (XR_FAILED(xrCreateReferenceSpace(openxr.session, &spaceCreateInfo, &openxr.appSpace)))
01460 {
01461     openxr.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
01462     spaceCreateInfo.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
01463     xrCheck(openxr.instance,
01464             xrCreateReferenceSpace(openxr.session, &spaceCreateInfo, &openxr.appSpace),
01465             "Can't create the OpenXR reference space");
01466 }
01467
01468 // アクションシステムの初期化
01469 openxr.initActions();
01470
01471 // スワップチェーンの作成
01472 openxr.createSwapchains();
01473 }
01474 catch (...)
01475 {
01476     // 途中まで確保した資源を解放してから例外を投げ直す
01477     openxr.terminate();
01478     throw;
01479 }
01480
01481 // OpenXR は xrWaitFrame() でフレームの表示速度を制御するので
01482 // ウィンドウ側の垂直同期の待ち合わせは行わない
01483 glfwSwapInterval(0);
01484
01485 openxr.initialized = true;
01486
01487 return openxr;
01488 }
01489 //
01490 //
01491 // OpenXR のハンドルを破棄する (OpenGL の資源には触れない)
01492 //
01493 void GgApp::OpenXR::destroyXr()
01494 {
01495     for (int i = 0; i < Hand::Count; ++i)
01496     {
01497         if (aimSpace[i] != XR_NULL_HANDLE) { xrDestroySpace(aimSpace[i]); aimSpace[i] = XR_NULL_HANDLE; }
01498         if (gripSpace[i] != XR_NULL_HANDLE) { xrDestroySpace(gripSpace[i]); gripSpace[i] = XR_NULL_HANDLE; }
01499     }
01500
01501     // アクションはアクションセットと一緒に破棄される
01502     if (actionSet != XR_NULL_HANDLE) xrDestroyActionSet(actionSet);
01503     actionSet = XR_NULL_HANDLE;
01504     aimPoseAction = gripPoseAction = XR_NULL_HANDLE;
01505     triggerAction = gripAction = XR_NULL_HANDLE;
01506     thumbstickAction = thumbstickClickAction = XR_NULL_HANDLE;
01507     primaryButtonAction = secondaryButtonAction = menuButtonAction = XR_NULL_HANDLE;
01508     hapticAction = XR_NULL_HANDLE;
01509
01510     for (auto swapchain : swapchains) xrDestroySwapchain(swapchain);
01511     swapchains.clear();
01512     swapchainImages.clear();
01513
01514     if (appSpace != XR_NULL_HANDLE) { xrDestroySpace(appSpace); appSpace = XR_NULL_HANDLE; }
01515     if (session != XR_NULL_HANDLE) { xrDestroySession(session); session = XR_NULL_HANDLE; }
01516     if (instance != XR_NULL_HANDLE) { xrDestroyInstance(instance); instance = XR_NULL_HANDLE; }
01517
01518     systemId = XR_NULL_SYSTEM_ID;
01519     sessionState = XR_SESSION_STATE_UNKNOWN;
01520     isSessionRunning = false;
01521     frameBegun = false;
01522     viewPoseValid = false;
01523     initialized = false;
01524 }
01525
01526 //
01527 // OpenXR のセッションを破棄する
01528 //
01529 void GgApp::OpenXR::terminate()
01530 {
01531     // 取得中のスワップチェーンイメージがあれば解放する (xrEndFrame() より前に行う)
01532     for (size_t i = 0; i < imageAcquired.size(); ++i)

```

```

01533 {
01534     if (!imageAcquired[i]) continue;
01535     XrSwapchainImageReleaseInfo releaseInfo{ XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO };
01536     xrReleaseSwapchainImage(swapchains[i], &releaseInfo);
01537     imageAcquired[i] = false;
01538 }
01539
01540 // 描画中のフレームがあれば完了しておく
01541 if (frameBegun) endFrame();
01542
01543 // OpenGL の資源を解放する
01544 if (!openxrFbo.empty())
01545 {
01546     glDeleteFramebuffers(static_cast<GLsizei>(openxrFbo.size()), openxrFbo.data());
01547     openxrFbo.clear();
01548 }
01549 if (!openxrDepth.empty())
01550 {
01551     glDeleteRenderbuffers(static_cast<GLsizei>(openxrDepth.size()), openxrDepth.data());
01552     openxrDepth.clear();
01553 }
01554
01555 // OpenXR のハンドルを破棄する
01556 destroyXr();
01557
01558 views.clear();
01559 viewStates.clear();
01560 currentIndex.clear();
01561 imageAcquired.clear();
01562 systemName.clear();
01563
01564 for (auto& state : controllerStates) state = ControllerState{};
01565
01566 // ウィンドウ側の設定を元に戻す
01567 if (window)
01568 {
01569     glDisable(GL_FRAMEBUFFER_SRGB);
01570     glfwSwapInterval(1);
01571     window = nullptr;
01572 }
01573 }
01574
01575 //
01576 // OpenXR による描画開始
01577 //
01578 bool GgApp::OpenXR::begin()
01579 {
01580     // 初期化されていなければ何もしない
01581     if (instance == XR_NULL_HANDLE) return false;
01582
01583     // OpenXR のイベントを処理する
01584     pollEvents();
01585
01586     // セッションが実行中でなければ描画しない
01587     if (!isSessionRunning) return false;
01588
01589     // 合成器がこのフレームの描画を始めるべき時刻まで待つ
01590     XrFrameWaitInfo waitInfo{ XR_TYPE_FRAME_WAIT_INFO };
01591     frameState = XrFrameState{ XR_TYPE_FRAME_STATE };
01592     if (!xrWarn(instance, xrWaitFrame(session, &waitInfo, &frameState),
01593         "Can't wait for the OpenXR frame")) return false;
01594
01595     // フレームの描画を開始する
01596     XrFrameBeginInfo beginInfo{ XR_TYPE_FRAME_BEGIN_INFO };
01597     if (!xrWarn(instance, xrBeginFrame(session, &beginInfo),
01598         "Can't begin the OpenXR frame")) return false;
01599
01600     // ここから先は必ず xrEndFrame() を呼ばなければならない
01601     frameBegun = true;
01602     viewPoseValid = false;
01603
01604     // コントローラーの状態を更新する
01605     pollActions();
01606
01607     // 描画すべきフレームなら視点の姿勢を取得する
01608     if (frameState.shouldRender != XR_FALSE)
01609     {
01610         XrViewLocateInfo viewLocateInfo{ XR_TYPE_VIEW_LOCATE_INFO };
01611         viewLocateInfo.viewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
01612         viewLocateInfo.displayTime = frameState.predictedDisplayTime;
01613         viewLocateInfo.space = appSpace;
01614
01615         XrViewState viewState{ XR_TYPE_VIEW_STATE };
01616         uint32_t viewCount{ 0 };
01617         if (XR_SUCCEEDED(xrLocateViews(session, &viewLocateInfo, &viewState,
01618             static_cast<uint32_t>(viewStates.size()), &viewCount, viewStates.data())))
01619         {

```

```

01620         // 位置と向きの両方が有効なときだけ描画する
01621         constexpr XrViewStateFlags valid
01622         {
01623             XR_VIEW_STATE_POSITION_VALID_BIT | XR_VIEW_STATE_ORIENTATION_VALID_BIT
01624         };
01625         viewPoseValid = (viewState.viewStateFlags & valid) == valid
01626             && viewCount == static_cast<uint32_t>(viewStates.size());
01627     }
01628 }
01629 // 描画するなら true を返す
01630 if (viewPoseValid) return true;
01631 // 描画しないフレームでもここで xrEndFrame() を呼んで辻褄を合わせる
01632 endFrame();
01633 return false;
01634 }
01635 //
01636 // 描画対象の目を指定してフレームバッファとビューポートを設定する
01637 //
01638 void GgApp::OpenXR::select(int eye)
01639 {
01640     // 描画すべきフレームでなければ何もしない
01641     if (!frameBegun || !viewPoseValid) return;
01642     // 視点の番号が範囲を外れていたら何もしない
01643     assert(eye >= 0 && eye < static_cast<int>(swapchains.size()));
01644     if (eye < 0 || eye >= static_cast<int>(swapchains.size())) return;
01645     // 取得済みなら描画先を結合し直すだけにする
01646     if (imageAcquired[eye])
01647     {
01648         glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[eye]);
01649         glViewport(0, 0,
01650             static_cast<GLsizei>(views[eye].recommendedImageRectWidth),
01651             static_cast<GLsizei>(views[eye].recommendedImageRectHeight));
01652         if (swapchainIsSrgb) glEnable(GL_FRAMEBUFFER_SRGB);
01653         return;
01654     }
01655     // 描画可能なスワップチェーンイメージを取得する
01656     XrSwapchainImageAcquireInfo acquireInfo{ XR_TYPE_SWAPCHAIN_IMAGE_ACQUIRE_INFO };
01657     if (!xrWarn(instance, xrAcquireSwapchainImage(swapchains[eye], &acquireInfo,
01658         &currentImageIndex[eye]), "Can't acquire the OpenXR swapchain image")) return;
01659     // そのスワップチェーンイメージが描画可能になるのを待つ
01660     XrSwapchainImageWaitInfo waitInfo{ XR_TYPE_SWAPCHAIN_IMAGE_WAIT_INFO };
01661     waitInfo.timeout = XR_INFINITE_DURATION;
01662     if (!xrWarn(instance, xrWaitSwapchainImage(swapchains[eye], &waitInfo),
01663         "Can't wait for the OpenXR swapchain image"))
01664     {
01665         XrSwapchainImageReleaseInfo releaseInfo{ XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO };
01666         xrReleaseSwapchainImage(swapchains[eye], &releaseInfo);
01667         return;
01668     }
01669     imageAcquired[eye] = true;
01670     // 描画先をこのスワップチェーンイメージに切り替える
01671     const GLuint texture{ swapchainImages[eye][currentImageIndex[eye]].image };
01672     glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[eye]);
01673     glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, texture, 0);
01674     // フレームバッファオブジェクトが完成しているか確かめる (最初の一度だけ報告する)
01675     static bool reported{ false };
01676     if (!reported && glCheckFramebufferStatus(GL_FRAMEBUFFER) != GL_FRAMEBUFFER_COMPLETE)
01677     {
01678         reported = true;
01679         std::cerr << "OpenXR: The framebuffer object for the swapchain image is not complete\n";
01680     }
01681     glViewport(0, 0,
01682         static_cast<GLsizei>(views[eye].recommendedImageRectWidth),
01683         static_cast<GLsizei>(views[eye].recommendedImageRectHeight));
01684     // sRGB のスワップチェーンならリニア色空間で描画する
01685     if (swapchainIsSrgb) glEnable(GL_FRAMEBUFFER_SRGB);
01686 }
01687 //
01688 // 描画対象の目を指定する (旧 LibOVR 仕様互換)
01689 //
01690 void GgApp::OpenXR::select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation)
01691 {
01692     select(eye);

```

```

01707
01708     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01709     const auto& pose = viewStates[eye].pose;
01710     const auto& fov = viewStates[eye].fov;
01711
01712     screen[0] = tanf(fov.angleLeft);
01713     screen[1] = tanf(fov.angleRight);
01714     screen[2] = tanf(fov.angleDown);
01715     screen[3] = tanf(fov.angleUp);
01716
01717     position[0] = pose.position.x;
01718     position[1] = pose.position.y;
01719     position[2] = pose.position.z;
01720
01721     orientation[0] = pose.orientation.x;
01722     orientation[1] = pose.orientation.y;
01723     orientation[2] = pose.orientation.z;
01724     orientation[3] = pose.orientation.w;
01725 }
01726
01727 //
01728 // 指定した目の描画を完了する
01729 //
01730 void GgApp::OpenXR::commit(int eye)
01731 {
01732     if (!frameBegun) return;
01733     assert(eye >= 0 && eye < static_cast<int>(swapchains.size()));
01734     if (eye < 0 || eye >= static_cast<int>(swapchains.size())) return;
01735     if (!imageAcquired[eye]) return;
01736
01737     // ガンマ補正を元に戻して描画先をウィンドウに戻す
01738     if (swapchainIsSrgb) glDisable(GL_FRAMEBUFFER_SRGB);
01739     glBindFramebuffer(GL_FRAMEBUFFER, 0);
01740
01741     // スワップチェーンイメージはミラー表示に使うので、ここでは解放しない
01742     // (解放は submit() の中でミラー表示を行った後に実施する)
01743 }
01744
01745 //
01746 // ミラー表示を行う
01747 //
01748 void GgApp::OpenXR::blitMirror() const
01749 {
01750     // ミラー表示を行わないなら何もしない
01751     if (mirrorView < 0 || !window) return;
01752     const auto eye{ static_cast<size_t>(mirrorView) };
01753     if (eye >= swapchains.size() || !imageAcquired[eye]) return;
01754
01755     // 転送元の大きさ
01756     const auto srcWidth{ static_cast<GLint>(views[eye].recommendedImageRectWidth) };
01757     const auto srcHeight{ static_cast<GLint>(views[eye].recommendedImageRectHeight) };
01758     if (srcWidth <= 0 || srcHeight <= 0) return;
01759
01760     // 転送先 (ウィンドウ) の大きさ
01761     const auto& fboSize{ window->getFboSize() };
01762     if (fboSize[0] <= 0 || fboSize[1] <= 0) return;
01763
01764     // 縦横比を保ったままウィンドウに収まる転送先の矩形を求める
01765     const auto scale{ std::min(
01766         static_cast<float>(fboSize[0]) / static_cast<float>(srcWidth),
01767         static_cast<float>(fboSize[1]) / static_cast<float>(srcHeight) ) };
01768     const auto dstWidth{ static_cast<GLint>(static_cast<float>(srcWidth) * scale) };
01769     const auto dstHeight{ static_cast<GLint>(static_cast<float>(srcHeight) * scale) };
01770     const auto dstLeft{ (static_cast<GLint>(fboSize[0]) - dstWidth) / 2 };
01771     const auto dstBottom{ (static_cast<GLint>(fboSize[1]) - dstHeight) / 2 };
01772
01773     // sRGB の再変換を避けるためにガンマ補正を無効にする
01774     if (swapchainIsSrgb) glDisable(GL_FRAMEBUFFER_SRGB);
01775
01776     // 上下左右の余白を黒で塗りつぶす (消去色は元に戻す)
01777     GLfloat clearColor[4];
01778     glGetFloatv(GL_COLOR_CLEAR_VALUE, clearColor);
01779     glBindFramebuffer(GL_DRAW_FRAMEBUFFER, 0);
01780     glDisable(GL_SCISSOR_TEST);
01781     glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
01782     glClear(GL_COLOR_BUFFER_BIT);
01783     glClearColor(clearColor[0], clearColor[1], clearColor[2], clearColor[3]);
01784
01785     // スワップチェーンイメージをウィンドウに転送する
01786     glBindFramebuffer(GL_READ_FRAMEBUFFER, openxrFbo[eye]);
01787     glBlitFramebuffer(0, 0, srcWidth, srcHeight,
01788         dstLeft, dstBottom, dstLeft + dstWidth, dstBottom + dstHeight,
01789         GL_COLOR_BUFFER_BIT, GL_LINEAR);
01790     glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
01791 }
01792
01793 //

```

```

01794 // 描画中のフレームを合成器に転送する
01795 //
01796 void GgApp::OpenXR::endFrame()
01797 {
01798     if (!frameBegun) return;
01799
01800     // 合成する層
01801     std::vector<XrCompositionLayerProjectionView> projectionViews;
01802     XrCompositionLayerProjection layer{ XR_TYPE_COMPOSITION_LAYER_PROJECTION };
01803     const XrCompositionLayerBaseHeader* layers[1]{ nullptr };
01804
01805     // 描画したのなら層を用意する
01806     if (viewPoseValid && frameState.shouldRender != XR_FALSE)
01807     {
01808         projectionViews.resize(swapchains.size());
01809         for (size_t i = 0; i < swapchains.size(); ++i)
01810         {
01811             auto& projectionView{ projectionViews[i] };
01812             projectionView = XrCompositionLayerProjectionView{ XR_TYPE_COMPOSITION_LAYER_PROJECTION_VIEW };
01813             projectionView.pose = viewStates[i].pose;
01814             projectionView.fov = viewStates[i].fov;
01815             projectionView.subImage.swapchain = swapchains[i];
01816             projectionView.subImage.imageRect.offset = { 0, 0 };
01817             projectionView.subImage.imageRect.extent = {
01818                 static_cast<int32_t>(views[i].recommendedImageRectWidth),
01819                 static_cast<int32_t>(views[i].recommendedImageRectHeight)
01820             };
01821             projectionView.subImage.imageArrayIndex = 0;
01822         }
01823
01824         // 環境の合成方法に応じて層の属性を設定する
01825         layer.layerFlags = blendMode == XR_ENVIRONMENT_BLEND_MODE_OPAQUE ? 0
01826             : XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT
01827             | XR_COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT;
01828         layer.space = appSpace;
01829         layer.viewCount = static_cast<uint32_t>(projectionViews.size());
01830         layer.views = projectionViews.data();
01831         layers[0] = reinterpret_cast<const XrCompositionLayerBaseHeader*>(&layer);
01832     }
01833
01834     // フレームを合成器に転送する
01835     XrFrameEndInfo endInfo{ XR_TYPE_FRAME_END_INFO };
01836     endInfo.displayTime = frameState.predictedDisplayTime;
01837     endInfo.environmentBlendMode = blendMode;
01838     endInfo.layerCount = layers[0] ? 1u : 0u;
01839     endInfo.layers = layers;
01840     xrWarn(instance, xrEndFrame(session, &endInfo), "Can't end the OpenXR frame");
01841
01842     frameBegun = false;
01843 }
01844
01845 //
01846 // フレームを転送して HMD に表示する
01847 //
01848 bool GgApp::OpenXR::submit(bool mirror)
01849 {
01850     // 描画中のフレームがなければ何もしない
01851     if (!frameBegun) return false;
01852
01853     // ミラー表示の有無を設定する
01854     if (!mirror) mirrorView = -1;
01855     else if (mirrorView < 0) mirrorView = 0;
01856
01857     // スワップチェーンイメージを解放する前にミラー表示を行う
01858     blitMirror();
01859
01860     // ウィンドウのビューポートを復帰して Dear ImGui などの描画に備える
01861     if (window) window->restoreViewport();
01862
01863     // 取得したスワップチェーンイメージを解放する
01864     for (size_t i = 0; i < imageAcquired.size(); ++i)
01865     {
01866         if (!imageAcquired[i]) continue;
01867         XrSwapchainImageReleaseInfo releaseInfo{ XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO };
01868         xrWarn(instance, xrReleaseSwapchainImage(swapchains[i], &releaseInfo),
01869             "Can't release the OpenXR swapchain image");
01870         imageAcquired[i] = false;
01871     }
01872
01873     // 合成器にフレームを転送する
01874     endFrame();
01875
01876     return true;
01877 }
01878
01879 //
01880 // ミラー表示を行うビューの番号を設定する

```

```

01881 //
01882 void GgApp::OpenXR::setMirror(int eye)
01883 {
01884     mirrorView = eye;
01885 }
01886
01887 //
01888 // ミラー表示を行うビューの番号を取得する
01889 //
01890 int GgApp::OpenXR::getMirror() const
01891 {
01892     return mirrorView;
01893 }
01894
01895 //
01896 // セッションが実行中かどうか調べる
01897 //
01898 bool GgApp::OpenXR::isRunning() const
01899 {
01900     return isSessionRunning;
01901 }
01902
01903 //
01904 // アプリケーションが入力を受け付けているかどうか調べる
01905 //
01906 bool GgApp::OpenXR::isFocused() const
01907 {
01908     return sessionState == XR_SESSION_STATE_FOCUSED;
01909 }
01910
01911 //
01912 // OpenXR のシステム (HMD) の名前を取得する
01913 //
01914 const std::string& GgApp::OpenXR::getSystemName() const
01915 {
01916     return systemName;
01917 }
01918
01919 //
01920 // 指定した目の透視投影変換行列を取得する
01921 //
01922 GgMatrix GgApp::OpenXR::getProjectionMatrix(int eye, GLfloat zNear, GLfloat zFar) const
01923 {
01924     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01925     const auto& fov{ viewStates[eye].fov };
01926     const GLfloat l{ tanf(fov.angleLeft) * zNear };
01927     const GLfloat r{ tanf(fov.angleRight) * zNear };
01928     const GLfloat b{ tanf(fov.angleDown) * zNear };
01929     const GLfloat t{ tanf(fov.angleUp) * zNear };
01930     return ggFrustum(l, r, b, t, zNear, zFar);
01931 }
01932
01933 //
01934 // 指定した目のビュー変換行列を取得する
01935 //
01936 GgMatrix GgApp::OpenXR::getViewMatrix(int eye) const
01937 {
01938     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01939     const auto& pose{ viewStates[eye].pose };
01940     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01941     return q.getConjugateMatrix() * ggTranslate(-pose.position.x, -pose.position.y, -pose.position.z);
01942 }
01943
01944 //
01945 // 指定した目の姿勢行列を取得する
01946 //
01947 GgMatrix GgApp::OpenXR::getPoseMatrix(int eye) const
01948 {
01949     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01950     const auto& pose{ viewStates[eye].pose };
01951     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01952     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
01953 }
01954
01955 //
01956 // 指定した目の視点位置を取得する
01957 //
01958 GgVector GgApp::OpenXR::getPosition(int eye) const
01959 {
01960     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01961     const auto& pos{ viewStates[eye].pose.position };
01962     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
01963 }
01964
01965 //

```

```

01966 // 指定した目の視線方向の回転四元数を取得する
01967 //
01968 GgQuaternion GgApp::OpenXR::getOrientation(int eye) const
01969 {
01970     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01971     const auto& ori{ viewStates[eye].pose.orientation };
01972     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
01973 }
01974
01975 //
01976 // 指定した目の視野角情報 (XrFovf) を取得する
01977 //
01978 const XrFovf& GgApp::OpenXR::getFov(int eye) const
01979 {
01980     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01981     return viewStates[eye].fov;
01982 }
01983
01984 //
01985 // 指定した目の姿勢情報 (XrPosef) を取得する
01986 //
01987 const XrPosef& GgApp::OpenXR::getPose(int eye) const
01988 {
01989     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01990     return viewStates[eye].pose;
01991 }
01992
01993 //
01994 // 視点の姿勢が有効かどうか調べる
01995 //
01996 bool GgApp::OpenXR::isPoseValid() const
01997 {
01998     return viewPoseValid;
01999 }
02000
02001 //
02002 // レンダリング推奨解像度の横幅を取得する
02003 //
02004 GLsizei GgApp::OpenXR::getWidth(int eye) const
02005 {
02006     assert(eye >= 0 && eye < static_cast<int>(views.size()));
02007     return static_cast<GLsizei>(views[eye].recommendedImageRectWidth);
02008 }
02009
02010 //
02011 // レンダリング推奨解像度の高さを取得する
02012 //
02013 GLsizei GgApp::OpenXR::getHeight(int eye) const
02014 {
02015     assert(eye >= 0 && eye < static_cast<int>(views.size()));
02016     return static_cast<GLsizei>(views[eye].recommendedImageRectHeight);
02017 }
02018
02019 //
02020 // アスペクト比を取得する
02021 //
02022 GLfloat GgApp::OpenXR::getAspect(int eye) const
02023 {
02024     assert(eye >= 0 && eye < static_cast<int>(views.size()));
02025     return static_cast<GLfloat>(views[eye].recommendedImageRectWidth)
02026         / static_cast<GLfloat>(views[eye].recommendedImageRectHeight);
02027 }
02028
02029 //
02030 // ビューの総数を取得する
02031 //
02032 uint32_t GgApp::OpenXR::getViewCount() const
02033 {
02034     return static_cast<uint32_t>(views.size());
02035 }
02036
02037 //
02038 // 現在の参照空間タイプを取得する
02039 //
02040 XrReferenceSpaceType GgApp::OpenXR::getReferenceSpaceType() const
02041 {
02042     return referenceSpaceType;
02043 }
02044
02045 //
02046 // コントローラーがトラッキングされているか取得する
02047 //
02048 bool GgApp::OpenXR::isTracked(int hand) const
02049 {
02050     assert(hand >= 0 && hand < Hand::Count);
02051     return controllerStates[hand].isTracked;
02052 }

```

```

02053
02054 //
02055 // コントローラーのグリップ変換行列を取得する
02056 //
02057 GgMatrix GgApp::OpenXR::getGripMatrix(int hand) const
02058 {
02059     assert(hand >= 0 && hand < Hand::Count);
02060     const auto& pose = controllerStates[hand].gripPose;
02061     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
02062     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
02063 }
02064 //
02065 // コントローラーのエイム変換行列を取得する
02066 //
02067 //
02068 GgMatrix GgApp::OpenXR::getAimMatrix(int hand) const
02069 {
02070     assert(hand >= 0 && hand < Hand::Count);
02071     const auto& pose = controllerStates[hand].aimPose;
02072     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
02073     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
02074 }
02075 //
02076 // コントローラーのグリップ位置を取得する
02077 //
02078 //
02079 GgVector GgApp::OpenXR::getGripPosition(int hand) const
02080 {
02081     assert(hand >= 0 && hand < Hand::Count);
02082     const auto& pos = controllerStates[hand].gripPose.position;
02083     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
02084 }
02085 //
02086 // コントローラーのグリップ回転四元数を取得する
02087 //
02088 //
02089 GgQuaternion GgApp::OpenXR::getGripOrientation(int hand) const
02090 {
02091     assert(hand >= 0 && hand < Hand::Count);
02092     const auto& ori = controllerStates[hand].gripPose.orientation;
02093     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
02094 }
02095 //
02096 // コントローラーのエイム位置を取得する
02097 //
02098 //
02099 GgVector GgApp::OpenXR::getAimPosition(int hand) const
02100 {
02101     assert(hand >= 0 && hand < Hand::Count);
02102     const auto& pos = controllerStates[hand].aimPose.position;
02103     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
02104 }
02105 //
02106 // コントローラーのエイム回転四元数を取得する
02107 //
02108 //
02109 GgQuaternion GgApp::OpenXR::getAimOrientation(int hand) const
02110 {
02111     assert(hand >= 0 && hand < Hand::Count);
02112     const auto& ori = controllerStates[hand].aimPose.orientation;
02113     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
02114 }
02115 //
02116 // トリガーの押し込み量を取得する
02117 //
02118 //
02119 float GgApp::OpenXR::getTrigger(int hand) const
02120 {
02121     assert(hand >= 0 && hand < Hand::Count);
02122     return controllerStates[hand].trigger;
02123 }
02124 //
02125 // グリップの押し込み量を取得する
02126 //
02127 //
02128 float GgApp::OpenXR::getGrip(int hand) const
02129 {
02130     assert(hand >= 0 && hand < Hand::Count);
02131     return controllerStates[hand].grip;
02132 }
02133 //
02134 // アナログスティックの入力値を取得する
02135 //
02136 //
02137 std::array<float, 2> GgApp::OpenXR::getThumbstick(int hand) const

```



```

02138 {
02139     assert(hand >= 0 && hand < Hand::Count);
02140     return controllerStates[hand].thumbstick;
02141 }
02142
02143 //
02144 // アナログスティックのクリック状態を取得する
02145 //
02146 bool GgApp::OpenXR::getThumbstickClick(int hand) const
02147 {
02148     assert(hand >= 0 && hand < Hand::Count);
02149     return controllerStates[hand].thumbstickClick;
02150 }
02151
02152 //
02153 // プライマリボタンの押下状態を取得する
02154 //
02155 bool GgApp::OpenXR::getPrimaryButton(int hand) const
02156 {
02157     assert(hand >= 0 && hand < Hand::Count);
02158     return controllerStates[hand].primaryButton;
02159 }
02160
02161 //
02162 // セカンダリボタンの押下状態を取得する
02163 //
02164 bool GgApp::OpenXR::getSecondaryButton(int hand) const
02165 {
02166     assert(hand >= 0 && hand < Hand::Count);
02167     return controllerStates[hand].secondaryButton;
02168 }
02169
02170 //
02171 // メニューボタンの押下状態を取得する
02172 //
02173 bool GgApp::OpenXR::getMenuButton(int hand) const
02174 {
02175     assert(hand >= 0 && hand < Hand::Count);
02176     return controllerStates[hand].menuButton;
02177 }
02178
02179 //
02180 // コントローラーに振動を出力する
02181 //
02182 void GgApp::OpenXR::applyHapticVibration(int hand, float durationSeconds, float frequency, float
    amplitude)
02183 {
02184     assert(hand >= 0 && hand < Hand::Count);
02185     if (session == XR_NULL_HANDLE || hapticAction == XR_NULL_HANDLE) return;
02186
02187     XrHapticVibration vibration{ XR_TYPE_HAPTIC_VIBRATION };
02188     vibration.duration = durationSeconds > 0.0f
02189         ? static_cast<XrDuration>(static_cast<double>(durationSeconds) * 1.0e9)
02190         : XR_MIN_HAPTIC_DURATION;
02191     vibration.frequency = frequency;
02192     vibration.amplitude = std::min(std::max(amplitude, 0.0f), 1.0f);
02193
02194     XrHapticActionInfo actionInfo{ XR_TYPE_HAPTIC_ACTION_INFO };
02195     actionInfo.action = hapticAction;
02196     actionInfo.subactionPath = handSubactionPath[hand];
02197
02198     xrWarn(instance, xrApplyHapticFeedback(session, &actionInfo,
02199         reinterpret_cast<const XrHapticBaseHeader*>(&vibration)),
02200         "Can't apply the haptic feedback");
02201 }
02202
02203 #endif
02204
02205 #if defined(_WIN32)
02206 # if !defined(_INC_WINDOWS) && !defined(_WINDOWS_)
02207 #     include <windows.h>
02208 # endif
02209 #else
02210 # include <pwd.h>
02211 # include <unistd.h>
02212 #endif
02213 //
02214 // ユーザ名を得る
02215 //
02216 std::string GgApp::getUsername()
02217 {
02218     // 環境変数からユーザ名を得る
02219     const char* user{
02220 #if defined(_WIN32)
02221         std::getenv("USERNAME")
02222 #else
02223         std::getenv("USER")

```

```

02224 #endif
02225 };
02226
02227 // 環境変数からユーザ名が得られたらそれを返す
02228 if (user) return user;
02229
02230 #if defined(_WIN32)
02231 // Win32 API を使ってユーザ名を得る
02232 char username[256];
02233 DWORD size{ sizeof(username) };
02234 if (GetUserNameA(username, &size)) return std::string(username);
02235 #else
02236 struct passwd* pw{ getpwuid(getuid()) };
02237 if (pw) return std::string(pw->pw_name);
02238 #endif
02239
02240 // ユーザ名が得られなかった
02241 return "unknown";
02242 }

```

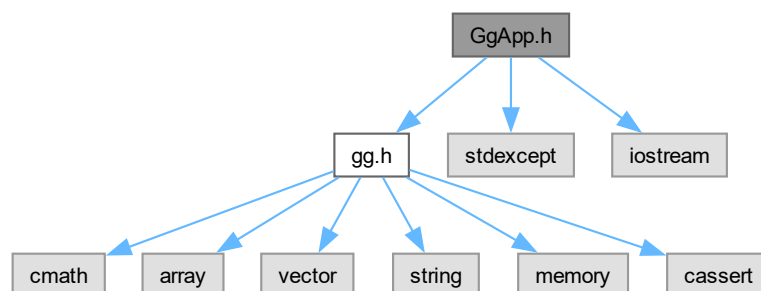
9.57 GgApp.h ファイル

```

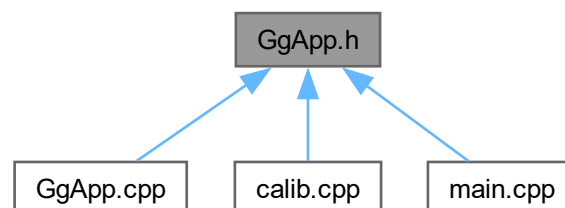
#include "gg.h"
#include <stdexcept>
#include <iostream>

```

GgApp.h の依存先関係図:



被依存関係図:



クラス

- class [GgApp](#)
- class [GgApp::Window](#)

マクロ定義

- #define [GG.BUTTON_COUNT](#) 3
- #define [GG.INTERFACE_COUNT](#) 5

9.57.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの定義。

著者

Kohe Tokoi

日付

July 17, 2025

[GgApp.h](#) に定義があります。

9.57.2 マクロ定義詳解

9.57.2.1 GG.BUTTON_COUNT

```
#define GG.BUTTON_COUNT 3
```

[GgApp.h](#) の 49 行目に定義があります。

9.57.2.2 GG.INTERFACE_COUNT

```
#define GG.INTERFACE_COUNT 5
```

[GgApp.h](#) の 54 行目に定義があります。

9.58 GgApp.h

[詳解]

```

00001 #pragma once
00002
00003 /*
00004 ゲームグラフィックス特論用補助プログラム GLFW3 版
00005
00006 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00007
00008 Permission is hereby granted, free of charge, to any person obtaining a copy
00009 of this software and associated documentation files (the "Software"), to deal
00010 in the Software without restriction, including without limitation the rights
00011 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00012 copies or substantial portions of the Software.
00013
00014 The above copyright notice and this permission notice shall be included in
00015 all copies or substantial portions of the Software.
00016
00017 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00018 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00019 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00020 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00021 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00022 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00023
00024 */
00025
00026 // Dear ImGui を使うなら
00027 #if !defined(GG_USE_IMGUI) && !defined(GG_NO_IMGUI)
00028 #   if defined(__has_include)
00029 #       if __has_include(<imgui.h>) || __has_include("imgui.h")
00030 #           define GG_USE_IMGUI
00031 #       endif
00032 #   endif
00033 #endif
00034
00035 // OpenXR を使うなら
00036 #define GG_USE_OPENXR
00037
00038 // 使用するマウスのボタン数
00039 #if !defined(GG_BUTTON_COUNT)
00040 #   define GG_BUTTON_COUNT 3
00041 #endif
00042
00043 // 使用するユーザインタフェースの数
00044 #if !defined(GG_INTERFACE_COUNT)
00045 #   define GG_INTERFACE_COUNT 5
00046 #endif
00047
00048 // 補助プログラム
00049 #include "gg.h"
00050 using namespace gg;
00051
00052 // 標準ライブラリ
00053 #include <stdexcept>
00054 #include <iostream>
00055
00056 // ImGui の組み込み
00057 #if defined(GG_USE_IMGUI)
00058 #   include "imgui.h"
00059 #   if defined(_ANDROID_)
00060 #       include "imgui_impl_android.h"
00061 #   else
00062 #       include "imgui_impl_glfw.h"
00063 #   endif
00064 #   include "imgui_impl_opengl3.h"
00065 #endif
00066
00067 // Android 固有のヘッダおよび定数定義
00068 #if defined(_ANDROID_)
00069 #   include <android.native_app_glue.h>
00070 #   include <android/native_window.h>
00071 #   include <EGL/egl.h>
00072 #endif
00073
00074 typedef struct GLFWwindow GLFWwindow;
00075 #define GLFW_MOUSE_BUTTON_1 0
00076 #define GLFW_MOUSE_BUTTON_2 1
00077 #define GLFW_MOUSE_BUTTON_3 2
00078 #define GLFW_RELEASE 0
00079 #define GLFW_PRESS 1
00080 #define GLFW_REPEAT 2
00081 #define GLFW_KEY_UNKNOWN -1
00082 #define GLFW_KEY_SPACE 32

```

```

00090 # define GLFW_ICONIFIED 0
00091 #endif
00092
00093 // OpenXR ライブラリの組み込み
00094 #if defined(GG_USE_OPENXR)
00095 # if defined(_WIN32)
00096 #     define XR_USE_PLATFORM_WIN32
00097 #     define XR_USE_GRAPHICS_API_OPENGL
00098 #     define GLFW_EXPOSE_NATIVE_WIN32
00099 #     define GLFW_EXPOSE_NATIVE_WGL
00100 #     include <GLFW/glfw3native.h>
00101 #     include <windows.h>
00102 #     include <unknwn.h>
00103 #     if defined(_MSC_VER)
00104 #         if defined(DEBUG)
00105 #             pragma comment(lib, "openxr_loaderd.lib")
00106 #         else
00107 #             pragma comment(lib, "openxr_loader.lib")
00108 #         endif
00109 #     endif
00110 # else
00111 #     if !defined(_gl.h)
00112 #         define _gl.h
00113 #     endif
00114 #     define XR_USE_PLATFORM_XLIB
00115 #     define XR_USE_GRAPHICS_API_OPENGL
00116 #     define GLFW_EXPOSE_NATIVE_X11
00117 #     define GLFW_EXPOSE_NATIVE_GLX
00118 #     include <GLFW/glfw3native.h>
00119 #     endif
00120 # include <openxr/openxr.h>
00121 # include <openxr/openxr.platform.h>
00122 # include <algorithm>
00123 # include <cstdio>
00124 # include <cstring>
00125 # include <string>
00126 # include <utility>
00127 # include <vector>
00128 #endif
00129
00133 class GgApp
00134 {
00135 public:
00136
00143     GgApp(int major = 0, int minor = 1);
00144
00150     GgApp(const GgApp& w) = delete;
00151
00157     GgApp(GgApp&& w) = default;
00158
00162     virtual ~GgApp();
00163
00170     GgApp& operator=(const GgApp& w) = delete;
00171
00178     GgApp& operator=(GgApp&& w) = default;
00179
00187     int main(int argc, const char* const* argv);
00188
00189 #if defined(__ANDROID__)
00191     static struct android_app* androidApp;
00192 #endif
00193
00200     class Window
00201     {
00202 #if defined(__ANDROID__)
00203         // Android の EGL ディスプレイ、サーフェス、コンテキスト
00204         EGLDisplay display{ EGL_NO_DISPLAY };
00205         EGLSurface surface{ EGL_NO_SURFACE };
00206         EGLContext context{ EGL_NO_CONTEXT };
00207         EGLConfig config{ nullptr };
00208         ANativeWindow* window{ nullptr };
00209
00210         // EGL の初期化
00211         bool initEgl(ANativeWindow* win);
00212
00213         // EGL の破棄
00214         void destroyEgl();
00215 #else
00216         // ウィンドウの識別子
00217         GLFWwindow* window{ nullptr };
00218 #endif
00219
00220         // ビューポートの横幅と高さ
00221         std::array<GLsizei, 2> size;
00222
00223         // フレームバッファの横幅と高さ
00224         std::array<GLsizei, 2> fboSize;

```

```

00225
00226 #if defined(IMGUI_VERSION)
00227     // メニューバーの高さ
00228     GLsizei menubarHeight{ 0 };
00229 #endif
00230
00231     // ビューポートの縦横比
00232     GLfloat aspect{ 1.0f };
00233
00234     // マウスの移動速度[X/Y/Z]
00235     std::array<GLfloat, 3> velocity{ 1.0f, 1.0f, 0.1f };
00236
00237     // マウスボタンの状態
00238     std::array<bool, GG.BUTTON_COUNT> status{};
00239
00240     // ユーザーインタフェースのデータ構造
00241     struct HumanInterface
00242     {
00243         // 最後にタイプしたキー
00244         int lastKey{ 0 };
00245
00246         // 矢印キー
00247         std::array<std::array<int, 2>, 4> arrow{};
00248
00249         // マウスの現在位置
00250         std::array<GLfloat, 2> mouse{};
00251
00252         // マウスホイールの回転量
00253         std::array<GLfloat, 2> wheel{};
00254
00255         // 平行移動量[ボタン][直前/更新][X/Y/Z]
00256         std::array<std::array<gg::GgVector, 2>, GG.BUTTON_COUNT> translation{};
00257
00258         // トラックボール
00259         std::array<gg::GgTrackball, GG.BUTTON_COUNT> rotation;
00260
00261         // コンストラクタ
00262         HumanInterface()
00263         {
00264             resetTranslation();
00265         }
00266
00267         //
00268         // マウスや矢印キーによる平行移動量を初期化する
00269         //
00270         void resetTranslation();
00271
00272         //
00273         // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00274         //
00275         void calcTranslation(int button, const std::array<GLfloat, 3>& velocity);
00276     };
00277
00278     // ヒューマンインタフェースデバイスのデータ
00279     std::array<HumanInterface, GG.INTERFACE_COUNT> interfaceData;
00280
00281     // ヒューマンインタフェースデバイスの番号
00282     int interfaceNo{ 0 };
00283
00284     //
00285     // ユーザー定義のコールバック関数へのポインタ
00286     //
00287     void* userPointer{ nullptr };
00288     void (*resizeFunc)(const Window* window, int width, int height){ nullptr };
00289     void (*keyboardFunc)(const Window* window, int key, int scancode, int action, int mods){ nullptr };
00290 };
00291
00292     void (*mouseFunc)(const Window* window, int button, int action, int mods){ nullptr };
00293     void (*wheelFunc)(const Window* window, double x, double y){ nullptr };
00294
00295     //
00296     // ウィンドウのサイズ変更時の処理
00297     //
00298     static void resize(GLFWwindow* window, int width, int height);
00299
00300     //
00301     // キーボードをタイプした時の処理
00302     //
00303     static void keyboard(GLFWwindow* window, int key, int scancode, int action, int mods);
00304
00305     //
00306     // マウスボタンを操作したときの処理
00307     //
00308     static void mouse(GLFWwindow* window, int button, int action, int mods);
00309
00310     //
00311     // マウスホイールを操作した時の処理
00312     //

```

```

00311     static void wheel(GLFWwindow* window, double x, double y);
00312
00313     public:
00314
00324     #if defined(__ANDROID__)
00325         Window(const std::string& title = "Android Window", int width = 0, int height = 0,
00326             int fullscreen = 0, void* share = nullptr);
00327     #else
00328         Window(const std::string& title = "GLFW Window", int width = 640, int height = 480,
00329             int fullscreen = 0, GLFWwindow* share = nullptr);
00330     #endif
00331
00337         Window(const Window& w) = delete;
00338
00344         Window(Window&& w) noexcept;
00345
00349         virtual ~Window()
00350         {
00351             // ウィンドウが作成されていなければ戻る
00352             if (!window) return;
00353
00354             #if defined(__ANDROID__)
00355                 destroyEgl();
00356             #else
00357                 // ウィンドウを破棄する
00358                 glfwDestroyWindow(window);
00359             #endif
00360         }
00361
00368         Window& operator=(const Window& w) = delete;
00369
00376         Window& operator=(Window&& w) noexcept;
00377
00383         auto* get() const
00384         {
00385             return window;
00386         }
00387
00393         void setClose(int flag = 1) const
00394         {
00395             #if !defined(__ANDROID__)
00396                 glfwSetWindowShouldClose(window, flag);
00397             #endif
00398         }
00399
00405         bool shouldClose() const
00406         {
00407             // ウィンドウを閉じるべきなら true を返す
00408             #if defined(__ANDROID__)
00409                 return !window;
00410             #else
00411                 return glfwWindowShouldClose(window) != GLFW_FALSE;
00412             #endif
00413         }
00414
00420         explicit operator bool();
00421
00425         void swapBuffers() const;
00426
00430         void restoreViewport() const
00431         {
00432             #if !defined(__ANDROID__)
00433                 if (!glfwGetWindowAttrib(window, GLFW_ICONIFIED)) glViewport(0, 0, fboSize[0], fboSize[1]);
00434             #else
00435                 glViewport(0, 0, fboSize[0], fboSize[1]);
00436             #endif
00437         }
00438
00442         void updateViewport();
00443
00444         #if defined(IMGUI_VERSION)
00450         void setMenubarHeight(GLsizei height)
00451         {
00452             // メニューバーの高さを保存する
00453             menubarHeight = height;
00454
00455             // ビューポートを復帰する
00456             updateViewport();
00457         }
00458     #endif
00459
00465         auto getWidth() const
00466         {
00467             return size[0];
00468         }
00469
00475         auto getHeight() const

```

```

00476     {
00477         return size[1];
00478     }
00479
00485     auto getFboWidth() const
00486     {
00487         return fboSize[0];
00488     }
00489
00495     auto getFboHeight() const
00496     {
00497         return fboSize[1];
00498     }
00499
00505     const auto& getSize() const
00506     {
00507         return size;
00508     }
00509
00515     void getSize(GLsizei* size) const
00516     {
00517         size[0] = getWidth();
00518         size[1] = getHeight();
00519     }
00520
00526     const auto& getFboSize() const
00527     {
00528         return fboSize;
00529     }
00530
00536     void getFboSize(GLsizei* fboSize) const
00537     {
00538         fboSize[0] = getFboWidth();
00539         fboSize[1] = getFboHeight();
00540     }
00541
00547     auto getAspect() const
00548     {
00549         return aspect;
00550     }
00551
00557     bool getKey(int key) const
00558     {
00559         #if !defined(_ANDROID_)
00560         #if defined(IMGUI_VERSION)
00561             // ImGui がキーボードを使うときはキーボードの処理を行わない
00562             if (ImGui::GetIO().WantCaptureKeyboard) return false;
00563         #endif
00564
00565         return glfwGetKey(window, key) != GLFW_RELEASE;
00566     #else
00567         (void)key;
00568         return false;
00569     #endif
00570     }
00571
00577     void selectInterface(int no)
00578     {
00579         assert(static_cast<size_t>(no) < interfaceData.size());
00580         interfaceNo = no;
00581     }
00582
00590     void setVelocity(GLfloat vx, GLfloat vy, GLfloat vz = 0.1f)
00591     {
00592         velocity = std::array<GLfloat, 3>{ vx, vy, vz };
00593     }
00594
00600     int getLastKey()
00601     {
00602         auto& current_if{ interfaceData[interfaceNo] };
00603         const int key{ current_if.lastKey };
00604         current_if.lastKey = 0;
00605         return key;
00606     }
00607
00615     auto getArrow(int direction = 0, int mods = 0) const
00616     {
00617         const auto& current_if{ interfaceData[interfaceNo] };
00618         return static_cast<GLfloat>(current_if.arrow[mods & 3][direction & 1]);
00619     }
00620
00627     auto getArrowX(int mods = 0) const
00628     {
00629         return getArrow(0, mods);
00630     }
00631
00638     auto getArrowY(int mods = 0) const

```



```

00639     {
00640         return getArrow(1, mods);
00641     }
00642
00649 void getArrow(GLfloat* arrow, int mods = 0) const
00650 {
00651     arrow[0] = getArrowX(mods);
00652     arrow[1] = getArrowY(mods);
00653 }
00654
00660 auto getShiftArrowX() const
00661 {
00662     return getArrow(0, 1);
00663 }
00664
00670 auto getShiftArrowY() const
00671 {
00672     return getArrow(1, 1);
00673 }
00674
00680 void getShiftArrow(GLfloat* shift_arrow) const
00681 {
00682     shift_arrow[0] = getShiftArrowX();
00683     shift_arrow[1] = getShiftArrowY();
00684 }
00685
00691 auto getControlArrowX() const
00692 {
00693     return getArrow(0, 2);
00694 }
00695
00701 auto getControlArrowY() const
00702 {
00703     return getArrow(1, 2);
00704 }
00705
00711 void getControlArrow(GLfloat* control_arrow) const
00712 {
00713     control_arrow[0] = getControlArrowX();
00714     control_arrow[1] = getControlArrowY();
00715 }
00716
00722 auto getAltArrowX() const
00723 {
00724     return getArrow(0, 3);
00725 }
00726
00732 auto getAltArrowY() const
00733 {
00734     return getArrow(1, 3);
00735 }
00736
00742 void getAltArrow(GLfloat* alt_arrow) const
00743 {
00744     alt_arrow[0] = getAltArrowX();
00745     alt_arrow[1] = getAltArrowY();
00746 }
00747
00753 const auto* getMouse() const
00754 {
00755     const auto& current_if{ interfaceData[interfaceNo] };
00756     return current_if.mouse.data();
00757 }
00758
00764 void getMouse(GLfloat* position) const
00765 {
00766     const auto& current_if{ interfaceData[interfaceNo] };
00767     position[0] = current_if.mouse[0];
00768     position[1] = current_if.mouse[1];
00769 }
00770
00777 auto getMouse(int direction) const
00778 {
00779     const auto& current_if{ interfaceData[interfaceNo] };
00780     return current_if.mouse[direction & 1];
00781 }
00782
00788 auto getMouseX() const
00789 {
00790     const auto& current_if{ interfaceData[interfaceNo] };
00791     return current_if.mouse[0];
00792 }
00793
00799 auto getMouseY() const
00800 {
00801     const auto& current_if{ interfaceData[interfaceNo] };
00802     return current_if.mouse[1];

```

```

00803     }
00804
00810     const auto* getWheel() const
00811     {
00812         const auto& current_if{ interfaceData[interfaceNo] };
00813         return current_if.wheel.data();
00814     }
00815
00821     void getWheel(GLfloat* rotation) const
00822     {
00823         const auto& current_if{ interfaceData[interfaceNo] };
00824         rotation[0] = current_if.wheel[0];
00825         rotation[1] = current_if.wheel[1];
00826     }
00827
00834     auto getWheel(int direction) const
00835     {
00836         const auto& current_if{ interfaceData[interfaceNo] };
00837         return current_if.wheel[direction & 1];
00838     }
00839
00845     auto getWheelX() const
00846     {
00847         const auto& current_if{ interfaceData[interfaceNo] };
00848         return current_if.wheel[0];
00849     }
00850
00856     auto getWheelY() const
00857     {
00858         const auto& current_if{ interfaceData[interfaceNo] };
00859         return current_if.wheel[1];
00860     }
00861
00868     const auto& getTranslation(int button = GLFW_MOUSE_BUTTON_1) const
00869     {
00870         const auto& current_if{ interfaceData[interfaceNo] };
00871         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00872         return current_if.translation[button][1];
00873     }
00874
00881     auto getTranslationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00882     {
00883         const auto& current_if{ interfaceData[interfaceNo] };
00884         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00885         const auto& t{ current_if.translation[button][1] };
00886
00887         gg::GgMatrix m
00888         {
00889             1.0f, 0.0f, 0.0f, 0.0f,
00890             0.0f, 1.0f, 0.0f, 0.0f,
00891             0.0f, 0.0f, 1.0f, 0.0f,
00892             t[0], t[1], t[2], 1.0f
00893         };
00894
00895         return m;
00896     }
00897
00904     auto getScrollMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00905     {
00906         const auto& current_if{ interfaceData[interfaceNo] };
00907         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00908         const auto& t{ current_if.translation[button][1] };
00909
00910         gg::GgMatrix m
00911         {
00912             t[2] + 1.0f, 0.0f, 0.0f, 0.0f,
00913             0.0f, t[2] + 1.0f, 0.0f, 0.0f,
00914             0.0f, 0.0f, 1.0f, 0.0f,
00915             t[0], t[1], 0.0f, 1.0f
00916         };
00917
00918         return m;
00919     }
00920
00927     auto getRotation(int button = GLFW_MOUSE_BUTTON_1) const
00928     {
00929         const auto& current_if{ interfaceData[interfaceNo] };
00930         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00931         return current_if.rotation[button].getQuaternion();
00932     }
00933
00940     auto getRotationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00941     {
00942         const auto& current_if{ interfaceData[interfaceNo] };
00943         assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00944         return current_if.rotation[button].getMatrix();
00945     }

```

```

00946
00950 void resetRotation()
00951 {
00952     // トラックボールを初期化する
00953     for (auto& tb : interfaceData[interfaceNo].rotation) tb.reset();
00954 }
00955
00959 void resetTranslation()
00960 {
00961     // 現在のインターフェースの平行移動量を初期化する
00962     interfaceData[interfaceNo].resetTranslation();
00963 }
00964
00968 void reset()
00969 {
00970     // トラックボール処理を初期化する
00971     resetRotation();
00972
00973     // 平行移動量を初期化する
00974     resetTranslation();
00975 }
00976
00982 void* getUserPointer() const
00983 {
00984     return userPointer;
00985 }
00986
00992 void setUserPointer(void* pointer)
00993 {
00994     userPointer = pointer;
00995 }
00996
01002 void setResizeFunc(void (*func)(const Window* window, int width, int height))
01003 {
01004     resizeFunc = func;
01005 }
01006
01012 void setKeyboardFunc(void (*func)(const Window* window, int key, int scancode, int action, int
01013 mods))
01014 {
01015     keyboardFunc = func;
01016 }
01022 void setMouseFunc(void (*func)(const Window* window, int button, int action, int mods))
01023 {
01024     mouseFunc = func;
01025 }
01026
01032 void setWheelFunc(void (*func)(const Window* window, double x, double y))
01033 {
01034     wheelFunc = func;
01035 }
01036 };
01037
01038 #if defined(GG_USE_OPENXR)
01047 class OpenXR
01048 {
01049 public:
01050
01054     enum Hand
01055     {
01056         Left = 0,
01057         Right = 1,
01058         Count = 2
01059     };
01060
01061 private:
01062
01063     // OpenXR のインスタンスとシステム
01064     XrInstance instance{ XR_NULL_HANDLE };
01065     XrSystemId systemId{ XR_NULL_SYSTEM_ID };
01066
01067     // OpenXR のシステムの名前
01068     std::string systemName;
01069
01070     // OpenXR のセッション
01071     XrSession session{ XR_NULL_HANDLE };
01072     XrSessionState sessionState{ XR_SESSION_STATE_UNKNOWN };
01073
01074     // OpenXR の参照空間
01075     XrSpace appSpace{ XR_NULL_HANDLE };
01076     XrReferenceSpaceType referenceSpaceType{ XR_REFERENCE_SPACE_TYPE_STAGE };
01077
01078     // ビュー設定とステート
01079     std::vector<XrViewConfigurationView> views;
01080     std::vector<XrView> viewStates;
01081

```

```

01082 // OpenXR のスワップチェーン
01083 std::vector<XrSwapchain> swapchains;
01084 std::vector<std::vector<XrSwapchainImageOpenGLKHR>> swapchainImages;
01085
01086 // OpenXR へのレンダリングに使う FBO とデスバッファ (ビューの数だけ確保する)
01087 std::vector<GLuint> openxrFbo;
01088 std::vector<GLuint> openxrDepth;
01089
01090 // スワップチェーンのカラーフォーマットが sRGB なら true
01091 bool swapchainIsSrgb{ true };
01092
01093 // 環境の合成方法
01094 XrEnvironmentBlendMode blendMode{ XR_ENVIRONMENT_BLEND_MODE_OPAQUE };
01095
01096 // フレームの同期状態
01097 XrFrameState frameState{ XR_TYPE_FRAME_STATE };
01098 bool isSessionRunning{ false };
01099
01100 // xrBeginFrame() を呼んで xrEndFrame() を呼んでいない状態なら true
01101 bool frameBegun{ false };
01102
01103 // 視点の姿勢が取得できていれば true
01104 bool viewPoseValid{ false };
01105
01106 // 初期化が完了していれば true
01107 bool initialized{ false };
01108
01109 // 各フレームで取得したスワップチェーンイメージのインデックス
01110 std::vector<uint32_t> currentImageIndex;
01111
01112 // スワップチェーンイメージを取得中のビューなら true
01113 std::vector<bool> imageAcquired;
01114
01115 // ミラー表示を行うビューの番号, ミラー表示を行わないなら -1
01116 int mirrorView{ 0 };
01117
01118 // OpenXR のミラー表示を行うウィンドウ
01119 const Window* window{ nullptr };
01120
01121 // アクションセット
01122 XrActionSet actionSet{ XR_NULL_HANDLE };
01123
01124 // アクション定義
01125 XrAction aimPoseAction{ XR_NULL_HANDLE };
01126 XrAction gripPoseAction{ XR_NULL_HANDLE };
01127 XrAction triggerAction{ XR_NULL_HANDLE };
01128 XrAction gripAction{ XR_NULL_HANDLE };
01129 XrAction thumbstickAction{ XR_NULL_HANDLE };
01130 XrAction thumbstickClickAction{ XR_NULL_HANDLE };
01131 XrAction primaryButtonAction{ XR_NULL_HANDLE };
01132 XrAction secondaryButtonAction{ XR_NULL_HANDLE };
01133 XrAction menuButtonAction{ XR_NULL_HANDLE };
01134 XrAction hapticAction{ XR_NULL_HANDLE };
01135
01136 // コントローラーのスペース
01137 XrSpace aimSpace[Hand::Count]{ XR_NULL_HANDLE, XR_NULL_HANDLE };
01138 XrSpace gripSpace[Hand::Count]{ XR_NULL_HANDLE, XR_NULL_HANDLE };
01139
01140 // コントローラーの状態キャッシュ
01141 struct ControllerState
01142 {
01143     bool isTracked{ false };
01144     XrPosef gripPose{ { 0.0f, 0.0f, 0.0f, 1.0f }, { 0.0f, 0.0f, 0.0f } };
01145     XrPosef aimPose{ { 0.0f, 0.0f, 0.0f, 1.0f }, { 0.0f, 0.0f, 0.0f } };
01146     float trigger{ 0.0f };
01147     float grip{ 0.0f };
01148     std::array<float, 2> thumbstick{ 0.0f, 0.0f };
01149     bool thumbstickClick{ false };
01150     bool primaryButton{ false };
01151     bool secondaryButton{ false };
01152     bool menuButton{ false };
01153 };
01154 ControllerState controllerStates[Hand::Count];
01155
01156 // サポートするパスの文字列
01157 XrPath handSubactionPath[Hand::Count]{ XR_NULL_PATH, XR_NULL_PATH };
01158
01159 //
01160 // アクションシステムを初期化する
01161 //
01162 void initActions();
01163
01164 //
01165 // アクション状態を更新する
01166 //
01167 void pollActions();
01168

```

```

01169 //
01170 // OpenXR のイベントを処理する
01171 //
01172 void pollEvents();
01173
01174 //
01175 // スワップチェーンを作成する
01176 //
01177 void createSwapchains();
01178
01179 //
01180 // 描画中のフレームを合成器に転送する
01181 //
01182 void endFrame();
01183
01184 //
01185 // OpenXR のハンドルを破棄する (OpenGL の資源には触れない)
01186 //
01187 void destroyXr();
01188
01189 //
01190 // ミラー表示を行う
01191 //
01192 void blitMirror() const;
01193
01194 //
01195 // コンストラクタ
01196 //
01197 OpenXR();
01198
01199 //
01200 // デストラクタ
01201 //
01202 virtual ~OpenXR();
01203
01204 public:
01205
01206 // シングルトンなのでコピー・ムーブ禁止
01207 OpenXR(const OpenXR&) = delete;
01208 OpenXR& operator=(const OpenXR&) = delete;
01209 OpenXR(OpenXR&&) = delete;
01210 OpenXR& operator=(OpenXR&&) = delete;
01211
01225 static OpenXR& initialize(const Window& window,
01226   XrReferenceSpaceType spaceType = XR_REFERENCE_SPACE_TYPE_STAGE,
01227   const char* appName = "GgApp");
01228
01236 void terminate();
01237
01250 bool begin();
01251
01257 void select(int eye);
01258
01267 void select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation);
01268
01278 void commit(int eye);
01279
01291 bool submit(bool mirror = true);
01292
01298 void setMirror(int eye);
01299
01305 int getMirror() const;
01306
01312 bool isRunning() const;
01313
01319 bool isFocused() const;
01320
01326 const std::string& getSystemName() const;
01327
01336 gg::GgMatrix getProjectionMatrix(int eye, GLfloat zNear = 0.1f, GLfloat zFar = 100.0f) const;
01337
01344 gg::GgMatrix getViewMatrix(int eye) const;
01345
01352 gg::GgMatrix getPoseMatrix(int eye) const;
01353
01360 gg::GgVector getPosition(int eye) const;
01361
01368 gg::GgQuaternion getOrientation(int eye) const;
01369
01376 const XrFovf& getFov(int eye) const;
01377
01384 const XrPosef& getPose(int eye) const;
01385
01391 bool isPoseValid() const;
01392
01399 GLsizei getWidth(int eye = 0) const;
01400

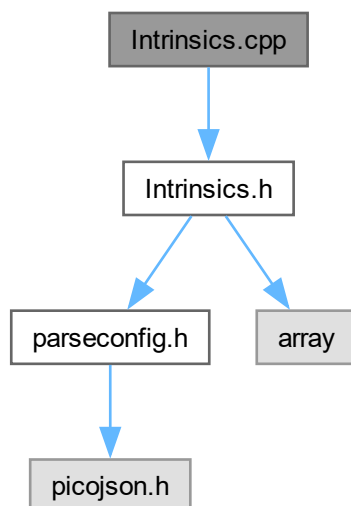
```

```
01407     GLsizei getHeight(int eye = 0) const;
01408
01415     GLfloat getAspect(int eye = 0) const;
01416
01422     uint32_t getViewCount() const;
01423
01429     XrReferenceSpaceType getReferenceSpaceType() const;
01430
01437     bool isTracked(int hand) const;
01438
01445     gg::GgMatrix getGripMatrix(int hand) const;
01446
01453     gg::GgMatrix getAimMatrix(int hand) const;
01454
01461     gg::GgVector getGripPosition(int hand) const;
01462
01469     gg::GgQuaternion getGripOrientation(int hand) const;
01470
01477     gg::GgVector getAimPosition(int hand) const;
01478
01485     gg::GgQuaternion getAimOrientation(int hand) const;
01486
01493     float getTrigger(int hand) const;
01494
01501     float getGrip(int hand) const;
01502
01509     std::array<float, 2> getThumbstick(int hand) const;
01510
01517     bool getThumbstickClick(int hand) const;
01518
01525     bool getPrimaryButton(int hand) const;
01526
01533     bool getSecondaryButton(int hand) const;
01534
01546     bool getMenuButton(int hand = Hand::Left) const;
01547
01556     void applyHapticVibration(int hand, float durationSeconds = 0.1f, float frequency =
XR_FREQUENCY_UNSPECIFIED, float amplitude = 0.5f);
01557 };
01558 #endif
01559
01565     static std::string getUsername();
01566 };
```

9.59 Intrinsic.cpp ファイル

```
#include "Intrinsic.h"
```

Intrinsic.cpp の依存先関係図:



9.59.1 詳解

キャプチャデバイス固有のパラメータクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Intrinsic.cpp](#) に定義があります。

9.60 Intrinsic.cpp

[詳解]

```
00001
00008 #include "Intrinsic.h"
00009
00010 //
00011 // 構成ファイルを読み込むときに使う
00012 // キャプチャデバイス固有のパラメータの構造体のコンストラクタ
```

```

00013 //
00014 Intrinsic::Intrinsic(const picojson::object& object)
00015 {
00016     // キャプチャデバイスの画角
00017     getValue(object, "fov", fov);
00018
00019     // キャプチャデバイスの中心位置
00020     getValue(object, "center", center);
00021
00022     // キャプチャデバイスの解像度
00023     getValue(object, "size", size);
00024
00025     // キャプチャデバイスの周波数
00026     getValue(object, "fps", fps);
00027 }
00028
00029 //
00030 // スクリーンの対角線長をもとにしてキャプチャデバイスのレンズの縦横の画角を変更する
00031 //
00032 void Intrinsic::setFov(float focal)
00033 {
00034     // 撮像面の画素数の対角線長×2
00035     const auto d{ hypotf(static_cast<float>(size[0]), static_cast<float>(size[1])) * 2.0f };
00036
00037     // 撮像面の画素数の対角線長に対するスクリーンの幅と高さの比の 1/2 (d は対角線長の 2 倍だから)
00038     const auto w{ size[0] / d };
00039     const auto h{ size[1] / d };
00040
00041     // 焦点距離が 1 のときの撮像面の対角線長
00042     const auto t{ sensorSize / focal };
00043
00044     // 焦点距離が 1 のときの撮像面の対角線長をこの比で横と縦に振り分ける
00045     const auto u{ t * w };
00046     const auto v{ t * h };
00047
00048     // 画角を求め単位を度に換算して設定する
00049     constexpr auto s{ 360.0f / 3.14159265359f };
00050     setFov(atan(u) * s, atan(v) * s);
00051 }

```

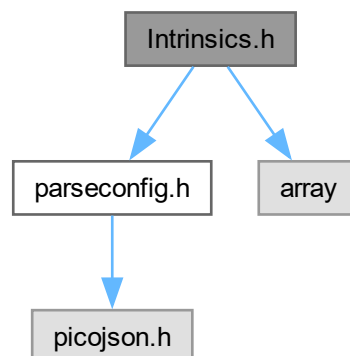
9.61 Intrinsic.h ファイル

```

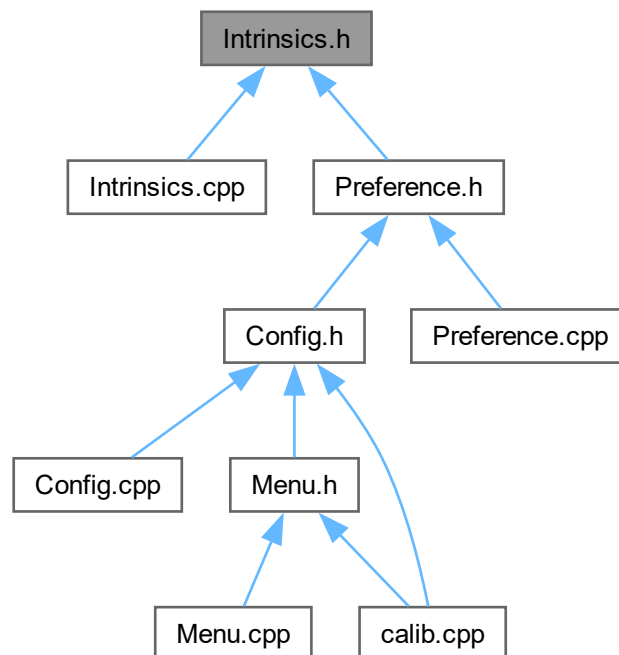
#include "parseconfig.h"
#include <array>

```

Intrinsic.h の依存先関係図:



被依存関係図:



クラス

- struct [Intrinsic](#)

9.61.1 詳解

キャプチャデバイス固有のパラメータクラスの定義

著者

Kohe Tokoi

日付

March 6, 2024

[Intrinsic.h](#) に定義があります。

9.62 Intrinsics.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 構成ファイルの読み取り補助
00012 #include "parseconfig.h"
00013
00014 // 標準ライブラリ
00015 #include <array>
00016
00020 struct Intrinsics
00021 {
00023     std::array<float, 2> fov{ 50.03f, 38.58f };
00024
00026     std::array<float, 2> center{ 0.0f, 0.0f };
00027
00029     std::array<int, 2> size{ 640, 480 };
00030
00032     double fps{ 30.0 };
00033
00039     Intrinsics() = default;
00040
00050     Intrinsics(const std::array<float, 2>& fov, const std::array<float, 2>& center,
00051               const std::array<int, 2> size, double fps)
00052         : fov{ fov }
00053         , center{ center }
00054         , size{ size }
00055         , fps{ fps }
00056     {
00057     }
00058
00063     Intrinsics(const picojson::object& object);
00064
00066     static constexpr auto sensorSize{ 35.0f };
00067
00073     void setFov(float focal);
00074
00081     void setFov(float fovx, float fovy)
00082     {
00083         // キャプチャデバイスのレンズの縦横の画角を設定する
00084         fov[0] = fovx;
00085         fov[1] = fovy;
00086     }
00087
00094     void setCenter(float x, float y)
00095     {
00096         // キャプチャデバイスのレンズの中心の位置を設定する
00097         center[0] = x;
00098         center[1] = y;
00099     }
00100
00101
00108     void setSize(int width, int height)
00109     {
00110         // 装置の解像度を設定する
00111         size[0] = width;
00112         size[1] = height;
00113     }
00114
00115
00121     void setFps(double frequency)
00122     {
00123         // 装置のフレームレートを設定する
00124         fps = frequency;
00125     }
00126 };

```

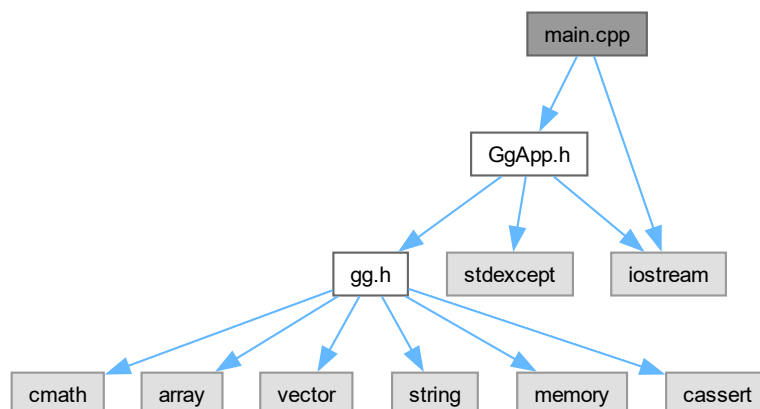
9.63 main.cpp ファイル

```

#include "GgApp.h"
#include <iostream>

```

main.cpp の依存先関係図:



マクロ定義

- `#define HEADER_STR "ゲームグラフィックス特論"`

関数

- `int main(int argc, const char *const *argv)`

9.63.1 詳解

ゲームグラフィックス特論宿題アプリケーション

著者

Kohe Tokoi

日付

February 20, 2024

main.cpp に定義があります。

9.63.2 マクロ定義詳解

9.63.2.1 HEADER_STR

```
#define HEADER_STR "ゲームグラフィックス特論"
```

main.cpp の 18 行目に定義があります。

9.63.3 関数詳解

9.63.3.1 main()

```
int main (
    int argc,
    const char *const * argv)
```

[main.cpp](#) の 122 行目に定義があります。

呼び出し関係図:



9.64 main.cpp

[詳解]

```

00001
00008 #include "GgApp.h"
00009
00010 // MessageBox の準備
00011 #if defined(WIN32)
00012 #   include <atlstr.h>
00013 #elif defined(__APPLE__)
00014 #   include <CoreFoundation/CoreFoundation.h>
00015 #else
00016 #   include <iostream>
00017 #endif
00018 #define HEADER_STR "ゲームグラフィックス特論"
00019
00020 #if defined(__ANDROID__)
00021 #include <android_native_app_glue.h>
00022 #include <android/asset_manager.h>
00023 #include <android/log.h>
00024 #include <unistd.h>
00025 #include <fstream>
00026
00027 extern void* ggAndroidAssetManager;
00028
00029 //
00030 // アセットをアプリ内内部ストレージへ展開し、作業ディレクトリを設定する
00031 //
00032 static void setupAndroidAssetsAndStorage(struct android_app* state)
00033 {
00034     if (!state || !state->activity) return;
00035
00036     const char* internalPath{ state->activity->internalDataPath };
00037     if (!internalPath) return;
00038
00039     // アプリの内部ストレージを作業ディレクトリに設定
00040     chdir(internalPath);
00041
00042     AAssetManager* assetMgr{ state->activity->assetManager };
00043     if (!assetMgr) return;
00044
00045     ggAndroidAssetManager = assetMgr;
00046
00047     // アセット一覧を走査して内部ストレージへ展開
00048     AAssetDir* assetDir{ AAssetManager_openDir(assetMgr, "") };
00049     if (assetDir)
00050     {
00051         const char* filename{ nullptr };
00052         while ((filename = AAssetDir_getNextFileName(assetDir)) != nullptr)
00053         {
00054             AAsset* asset{ AAssetManager_open(assetMgr, filename, AASSET_MODE_BUFFER) };
00055             if (asset)
00056             {
00057                 const off_t size{ AAsset_getLength(asset) };
00058                 const std::string destPath{ std::string(internalPath) + "/" + filename };
00059
00060                 bool needWrite{ true };
00061                 std::ifstream check(destPath, std::ios::binary | std::ios::ate);
00062                 if (check.is_open())
00063                 {
00064                     if (check.tellg() == size) needWrite = false;
00065                     check.close();
00066                 }
00067
00068                 if (needWrite)
00069                 {
00070                     const void* buffer{ AAsset_getBuffer(asset) };
00071                     std::ofstream out(destPath, std::ios::binary);
00072                     if (out.is_open())
00073                     {
00074                         out.write(static_cast<const char*>(buffer), size);
00075                         out.close();
00076                     }
00077                 }
00078                 AAsset_close(asset);
00079             }
00080         }
00081         AAssetDir_close(assetDir);
00082     }
00083 }
00084
00085 //
00086 // Android NativeActivity エントリーポイント
00087 //
00088 void android_main(struct android_app* state)

```

```

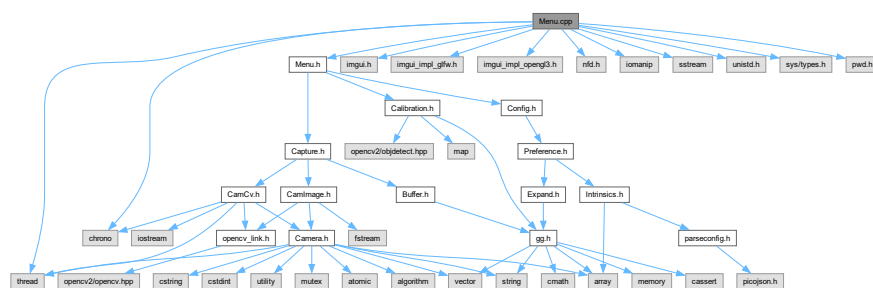
00089 {
00090     GgApp::androidApp = state;
00091
00092     // 入力イベントハンドラを登録
00093     state->onInputEvent = [](struct android_app*, AInputEvent* event) -> int32_t {
00094         #if defined(GG_USE_IMGUI)
00095             if (ImGui_ImplAndroid_HandleInputEvent(event))
00096             {
00097                 return 1;
00098             }
00099         #endif
00100         return 0;
00101     };
00102
00103     // アセットの展開と作業ディレクトリの設定
00104     setupAndroidAssetsAndStorage(state);
00105
00106     try
00107     {
00108         GgApp app(3, 1);
00109         app.main(0, nullptr);
00110     }
00111     catch (const std::exception& e)
00112     {
00113         __android_log_print(ANDROID_LOG_ERROR, "calib", "Application error: %s", e.what());
00114     }
00115 }
00116
00117 #else
00118
00119 //
00120 // メインプログラム
00121 //
00122 int main(int argc, const char* const* argv) try
00123 {
00124     // アプリケーションのオブジェクトを生成する
00125     #if defined(GL_GLES_PROTOTYPES)
00126         GgApp app(3, 1);
00127     #else
00128         GgApp app(4, 1);
00129     #endif
00130
00131     // アプリケーションを実行する
00132     return app.main(argc, argv);
00133 }
00134 catch (const std::runtime_error &e)
00135 {
00136     // エラーメッセージを表示する
00137     #if defined(WIN32)
00138         MessageBox(NULL, CString(e.what()), TEXT(HEADER_STR), MB_ICONERROR);
00139     #elif defined(__APPLE__)
00140         // the following code is copied from
00141         // http://blog.jorgearimany.com/2010/05/messagebox-from-windows-to-mac.html
00142         // convert the strings from char* to CFStringRef
00143         CFStringRef msg_ref = CFStringCreateWithCString(NULL, e.what(), kCFStringEncodingUTF8);
00144
00145         // result code from the message box
00146         CFOptionFlags result;
00147
00148         // launch the message box
00149         CFUserNotificationDisplayAlert(
00150             0, // no timeout
00151             kCFUserNotificationNoteAlertLevel, // change it depending message-type flags ( MB_ICONASTERISK....
00152             etc.),
00153             NULL, // icon url, use default, you can change it depending
00154             message_type flags
00155             NULL, // not used
00156             NULL, // localization of strings
00157             CFSTR(HEADER_STR), // header text
00158             msg_ref, // message text
00159             NULL, // default "ok" text in button
00160             NULL, // alternate button title
00161             NULL, // other button title, null--> no other button
00162             &result // response flags
00163         );
00164
00165         // Clean up the strings
00166         CFRelease(msg_ref);
00167     #else
00168         std::cerr << HEADER_STR << ": " << e.what() << '\n';
00169     #endif
00170 }
00171 #endif

```

9.65 Menu.cpp ファイル

```
#include "Menu.h"
#include "imgui.h"
#include "imgui_impl_glfw.h"
#include "imgui_impl_opengl3.h"
#include "nfd.h"
#include <iomanip>
#include <sstream>
#include <chrono>
#include <thread>
#include <unistd.h>
#include <sys/types.h>
#include <pwd.h>
```

Menu.cpp の依存先関係図:



関数

- void [getAnyList](#) (std::vector< std::string > &list)

9.65.1 詳解

メニューの描画クラスの実装

著者

Kohe Tokoi

日付

November 15, 2022

[Menu.cpp](#) に定義があります。

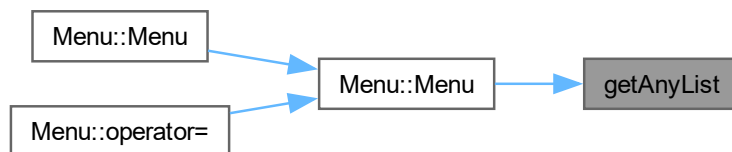
9.65.2 関数詳解

9.65.2.1 getAnyList()

```
void getAnyList (
    std::vector< std::string > & list)
```

Menu.cpp の 75 行目に定義があります。

被呼び出し関係図:



9.66 Menu.cpp

[詳解]

```

00001
00008 #include "Menu.h"
00009
00010 // ImGui
00011 #include "imgui.h"
00012 #include "imgui_impl_glfw.h"
00013 #include "imgui_impl_opengl3.h"
00014
00015 // ファイルダイアログ
00016 #if !defined(_ANDROID_)
00017 #include "nfd.h"
00018
00019 namespace
00020 {
00021     // JSON ファイル名のフィルタ
00022     constexpr nfdfilteritem_t jsonFilter[] { { "JSON", "json" } };
00023
00024     // 画像ファイル名のフィルタ
00025     constexpr nfdfilteritem_t imageFilter[] { "Images", "png,jpg,jpeg,jfif,bmp,dib" };
00026
00027     // 動画ファイル名のフィルタ
00028     constexpr nfdfilteritem_t movieFilter[] { "Movies", "mp4,m4v,mpg,mov,avi,ogg,mkv" };
00029 }
00030 #else
00031 #include <filesystem>
00032 #endif
00033
00034 // 初期表示の画像ファイル名
00035 std::string Config::initialImage{ "initial.jpg" };
00036
00037 // 標準ライブラリ
00038 #include <iomanip>
00039 #include <sstream>
00040 #include <chrono>
00041 #include <thread>
00042
00043 #if !defined(_WIN32) && !defined(_ANDROID_) && !defined(_APPLE_)
00044 // バックエンドのリスト
00045 const std::map<cv::VideoCaptureAPIs, const char*> Menu::backendList
00046 {
00047     # if defined(USE_LIBCAMERA)
```



```

00048 { CAP_LIBCAMERA, "libcamera" },
00049 # endif
00050 { cv::CAP_ANY, "(any)" },
00051 # if defined(__linux__)
00052 { cv::CAP_V4L2, "V4L2" },
00053 # endif
00054 { cv::CAP_FFMPEG, u8"動画ファイル履歴" }
00055 };
00056
00057 // コーデックのリスト
00058 const std::vector<const char*> Menu::codecList
00059 {
00060     "(any)",
00061     "MJPG",
00062     "H264",
00063     "BGR3",
00064     "YUY2",
00065     "I420",
00066     "NV12"
00067 };
00068
00069 // キャプチャデバイスのリスト
00070 std::map<cv::VideoCaptureAPIs, std::vector<std::string>> Menu::deviceList;
00071
00072 //
00073 // デフォルトのビデオデバイスの一覧を作る
00074 //
00075 void getAnyList(std::vector<std::string>& list)
00076 {
00077     list.emplace_back("(any)");
00078     list.emplace_back("Device 1");
00079     list.emplace_back("Device 2");
00080     list.emplace_back("Device 3");
00081     list.emplace_back("Device 4");
00082     list.emplace_back("Device 5");
00083     list.emplace_back("Device 6");
00084     list.emplace_back("Device 7");
00085 }
00086
00087 # if defined(__linux__)
00088 #     include <filesystem>
00089 #     include <fstream>
00090 #     include <fcntl.h>
00091 #     include <unistd.h>
00092 #     include <sys/ioctl.h>
00093 #     include <linux/videodev2.h>
00094
00095 //
00096 // Linux (V4L2) のビデオデバイスの一覧を作る
00097 //
00098 void getV4L2List(std::vector<std::string>& list)
00099 {
00100     namespace fs = std::filesystem;
00101     std::error_code ec;
00102
00103     // /sys/class/video4linux ディレクトリを走査する
00104     const fs::path v4l2Path{ "/sys/class/video4linux" };
00105     if (fs::exists(v4l2Path, ec))
00106     {
00107         std::map<int, std::string> cameraDevices;
00108         std::map<int, std::string> otherDevices;
00109
00110         for (const auto& entry : fs::directory_iterator(v4l2Path, ec))
00111         {
00112             const auto filename{ entry.path().filename().string() };
00113             // "video" で始まるノード (video0, video1, ...)
00114             if (filename.rfind("video", 0) == 0)
00115             {
00116                 try
00117                 {
00118                     const int index{ std::stoi(filename.substr(5)) };
00119                     const std::string devPath{ "/dev/" + filename };
00120
00121                     // デバイスファイルを開いてカーバリティを調べる
00122                     const int fd{ ::open(devPath.c_str(), O_RDONLY | O_NONBLOCK) };
00123                     if (fd >= 0)
00124                     {
00125                         v4l2_capability cap{};
00126                         if (::ioctl(fd, VIDIOC_QUERYCAP, &cap) == 0)
00127                         {
00128                             const uint32_t caps{ cap.device_caps ? cap.device_caps : cap.capabilities };
00129                             const std::string card{ reinterpret_cast<const char*>(cap.card) };
00130                             const std::string driver{ reinterpret_cast<const char*>(cap.driver) };
00131
00132                             // キャプチャ機能 (VIDEO_CAPTURE) を持ち、出力 (OUTPUT) や M2M ではないこと
00133                             const bool isCapture{ (caps & (V4L2_CAP_VIDEO_CAPTURE | V4L2_CAP_VIDEO_CAPTURE_MPLANE)) !=
00134
00135 
```

```

00134         const bool isM2M{ (caps & (V4L2_CAP_VIDEO_M2M | V4L2_CAP_VIDEO_M2M_MPLANE)) != 0 };
00135         const bool isOutput{ (caps & (V4L2_CAP_VIDEO_OUTPUT | V4L2_CAP_VIDEO_OUTPUT_MPLANE)) != 0
};
00136         const bool isMeta{ (caps & (V4L2_CAP_META_CAPTURE | V4L2_CAP_META_OUTPUT)) != 0 };
00137
00138         // Raspberry Pi の bcm2835-codec や bcm2835-isp, pisp 等の SoC 内部処理用ノードは除外
00139         const bool isSoCInternal{
00140             driver.find("bcm2835") != std::string::npos ||
00141             card.find("bcm2835") != std::string::npos ||
00142             driver.find("pisp") != std::string::npos ||
00143             card.find("pisp") != std::string::npos
00144         };
00145
00146         std::string displayName{ filename + ": " + (card.empty() ? driver : card) };
00147
00148         if (isCapture && !isM2M && !isOutput && !isMeta && !isSoCInternal)
00149         {
00150             cameraDevices[index] = displayName;
00151         }
00152         else
00153         {
00154             otherDevices[index] = displayName;
00155         }
00156     }
00157     ::close(fd);
00158 }
00159 }
00160 catch (...)
00161 {
00162 }
00163 }
00164 }
00165
00166 // カメラデバイスがあればそれを登録
00167 for (const auto& [idx, devName] : cameraDevices)
00168 {
00169     list.emplace_back(devName);
00170 }
00171
00172 // カメラデバイスが見つからなかった場合はフォールバックとしてその他を登録
00173 if (list.empty())
00174 {
00175     for (const auto& [idx, devName] : otherDevices)
00176     {
00177         list.emplace_back(devName);
00178     }
00179 }
00180 }
00181
00182 // デバイスが取得できなかった場合はフォールバック
00183 if (list.empty())
00184 {
00185     getAnyList(list);
00186 }
00187 }
00188 # endif
00189
00190 // パスワードのエントリからホームディレクトリの場所を得るときに使う
00191 # include <unistd.h>
00192 # include <sys/types.h>
00193 # include <pwd.h>
00194
00195 #endif // !defined(_WIN32) && !defined(__ANDROID__) && !defined(__APPLE__)
00196
00197 //
00198 // キャプチャデバイスを開く
00199 //
00200 bool Menu::openDevice()
00201 {
00202     #if defined(_WIN32) || defined(__ANDROID__) || defined(__APPLE__)
00203         // 何のデバイスも接続されていなければ戻る
00204         if (deviceNumber < 0) return false;
00205
00206         // キャプチャスレッドが動いていたら止める
00207         capture.stop();
00208
00209         // 前に開いていたキャプチャデバイスを閉じる
00210         capture.close();
00211
00212         // 選択したキャプチャデバイスを開く
00213         if (capture.openDevice(deviceNumber))
00214         {
00215             if (formatNumber < 0) formatNumber = 0;
00216
00217             updateFormatDropdowns();
00218
00219             // フォーマットを指定して開始できるように準備する

```

```

00220     if (capture.select(formatNumber))
00221     {
00222         // 実解像度と焦点距離から、この入力を見やすく表示する初期画角を設定する
00223         initializeInputIntrinsics(capture.getSize());
00224         return true;
00225     }
00226 }
00227
00228 // 開けなかった
00229 errorMessage = u8"デバイスが開けません";
00230 return false;
00231 #else
00232 // コーデック
00233 char codec[5]{};
00234 if (codecNumber > 0) strncpy(codec, codecList[codecNumber], 5);
00235
00236 // 実際のデバイス番号を決定する
00237 int actualDeviceNumber{ deviceNumber };
00238 # if defined(__linux__)
00239 #     if defined(USE_LIBCAMERA)
00240     if (backend == CAP_LIBCAMERA)
00241     {
00242         actualDeviceNumber = deviceNumber;
00243     }
00244     else
00245     # endif
00246     if (backend == cv::CAP_V4L2)
00247     {
00248         const auto& name{ getDeviceName(backend, deviceNumber) };
00249         if (name.rfind("video", 0) == 0)
00250         {
00251             try
00252             {
00253                 actualDeviceNumber = std::stoi(name.substr(5));
00254             }
00255             catch (...)
00256             {
00257             }
00258         }
00259     }
00260 # endif
00261
00262 // ダイアログで指定したキャプチャデバイスが開けなかったら
00263 if (!capture.openDevice(actualDeviceNumber,
00264     intrinsics.size, intrinsics.fps, backend, codec))
00265 {
00266     // 開けなかった
00267     errorMessage = u8"デバイスが開けません";
00268     return false;
00269 }
00270
00271 // 実解像度と焦点距離から、この入力を見やすく表示する初期画角を設定する
00272 initializeInputIntrinsics(capture.getSize());
00273
00274 // 使うことになったコーデックの番号を調べる
00275 for (size_t i = 0; i < codecList.size(); ++i)
00276 {
00277     if (strcmp(codec, codecList[i], 4) == 0)
00278     {
00279         // コーデックが分かった
00280         codecNumber = static_cast<int>(i);
00281         return true;
00282     }
00283 }
00284
00285 // コーデックが分からない
00286 codecNumber = 0;
00287 return true;
00288 #endif
00289 }
00290
00291 //
00292 // 画像ファイルを開く
00293 //
00294 void Menu::openImage()
00295 {
00296     #if !defined(__ANDROID__)
00297         // ファイルダイアログから得るパス
00298         nfdchar_t* filepath;
00299
00300         // ファイルダイアログを開く
00301         if (NFD_OpenDialog(&filepath, imageFilter, 1, NULL) == NFD_OKAY)
00302         {
00303             // スレッドが動作中なら停止する
00304             capture.stop();
00305
00306             // ダイアログで指定した画像ファイルが開いたら

```

```

00307     if (capture.openImage(filepath))
00308     {
00309         // 実解像度と焦点距離から、この画像を見やすく表示する初期画角を設定する
00310         initializeInputIntrinsics(capture.getSize());
00311     }
00312     else
00313     {
00314         // 開けなかった
00315         errorMessage = u8"画像ファイルが開けません";
00316     }
00317
00318     // ファイルパスの取り出しに使ったメモリを開放する
00319     NFD_FreePath(filepath);
00320 }
00321 #else
00322     fileModalType = FileModalType::Image;
00323     showFileModal = true;
00324 #endif
00325 }
00326
00327 //
00328 // 動画ファイルを開く
00329 //
00330 void Menu::openMovie()
00331 {
00332     #if !defined(__ANDROID__)
00333         // ファイルダイアログから得るパス
00334         nfdchar_t* filepath;
00335
00336         // ファイルダイアログを開く
00337         if (NFD_OpenDialog(&filepath, movieFilter, 1, NULL) == NFD_OKAY)
00338         {
00339             // スレッドが動作中なら停止する
00340             capture.stop();
00341
00342             // ダイアログで指定した動画ファイルが開いたら
00343             if (capture.openMovie(filepath))
00344             {
00345                 // 実解像度と焦点距離から、この動画を見やすく表示する初期画角を設定する
00346                 initializeInputIntrinsics(capture.getSize());
00347             }
00348             else
00349             {
00350                 // 開けなかった
00351                 errorMessage = u8"動画ファイルが開けません";
00352             }
00353
00354             // ファイルパスの取り出しに使ったメモリを開放する
00355             NFD_FreePath(filepath);
00356         }
00357     #else
00358         errorMessage = u8"動画ファイル再生に対応していません";
00359     #endif
00360 }
00361
00362 //
00363 // 構成ファイルを読み込む
00364 //
00365 void Menu::loadConfig()
00366 {
00367     #if !defined(__ANDROID__)
00368         // ファイルダイアログから得るパス
00369         nfdchar_t* filepath;
00370
00371         // ファイルダイアログを開く
00372         if (NFD_OpenDialog(&filepath, jsonFilter, 1, NULL) == NFD_OKAY)
00373         {
00374             // 現在の構成を構成ファイルの内容にする
00375             if (config.load(filepath))
00376             {
00377                 // 現在の設定に反映する
00378                 settings = config.getSettings();
00379
00380                 // 選択番号を有効範囲に収め、新しい投影方式の内部パラメータを反映する
00381                 selectPreference(preferenceNumber < static_cast<int>(config.getPreferences().size())
00382                     ? preferenceNumber : 0);
00383
00384                 // 校正設定も新しい構成に同期する
00385                 calibration.setDictionary(settings.dictionaryName, settings.checkerSize,
00386                     settings.checkerLength);
00387             }
00388             else
00389             {
00390                 // 読み込めなかった
00391                 errorMessage = u8"構成ファイルが読み込めません";
00392             }
00393         }
00394     }
00395 }

```

```

00393     // ファイルパスの取り出しに使ったメモリを開放する
00394     NFD_FreePath(filepath);
00395 }
00396 #else
00397 if (config.load("calib.config.json"))
00398 {
00399     settings = config.getSettings();
00400     selectPreference(preferenceNumber < static_cast<int>(config.getPreferences().size())
00401         ? preferenceNumber : 0);
00402     calibration.setDictionary(settings.dictionaryName, settings.checkerSize, settings.checkerLength);
00403 }
00404 else
00405 {
00406     errorMessage = u8"構成ファイルが読み込めません";
00407 }
00408 #endif
00409 }
00410
00411 //
00412 // 構成ファイルを保存する
00413 //
00414 void Menu::saveConfig() const
00415 {
00416     #if !defined(__ANDROID__)
00417         // ファイルダイアログから得るパス
00418         nfdchar_t* filepath;
00419
00420         // ファイルダイアログを開く
00421         if (NFD_SaveDialog(&filepath, jsonFilter, 1, NULL, "*.json") == NFD_OKAY)
00422         {
00423             // 現在の設定で構成を更新する
00424             config.setSettings(settings);
00425
00426             // 現在の構成を保存する
00427             if (!config.save(filepath))
00428             {
00429                 // 保存できなかった
00430                 errorMessage = u8"構成ファイルが保存できません";
00431             }
00432
00433             // ファイルパスの取り出しに使ったメモリを開放する
00434             NFD_FreePath(filepath);
00435         }
00436     #else
00437         config.setSettings(settings);
00438         if (!config.save("calib.config.json"))
00439         {
00440             errorMessage = u8"構成ファイルが保存できません";
00441         }
00442     #endif
00443 }
00444
00445 //
00446 // 校正ファイルを読み込む
00447 //
00448 void Menu::loadCalibration()
00449 {
00450     #if !defined(__ANDROID__)
00451         // ファイルダイアログから得るパス
00452         nfdchar_t* filepath;
00453
00454         // ファイルダイアログを開く
00455         if (NFD_OpenDialog(&filepath, jsonFilter, 1, NULL) == NFD_OKAY)
00456         {
00457             // 現在のキャリブレーションパラメータを校正ファイルの内容にする
00458             if (!calibration.loadParameters(filepath))
00459             {
00460                 // 読み込めなかった
00461                 errorMessage = u8"校正ファイルが読み込めません";
00462             }
00463
00464             // ファイルパスの取り出しに使ったメモリを開放する
00465             NFD_FreePath(filepath);
00466         }
00467     #else
00468         fileModalType = FileModalType::Calibration;
00469         showFileModal = true;
00470     #endif
00471 }
00472
00473 //
00474 // 校正ファイルを保存する
00475 //
00476 void Menu::saveParameters() const
00477 {
00478     // キャリブレーションが完了していなければ戻る
00479     if (!calibration.finished()) return;

```

```

00480
00481 // 現在時刻の取得
00482 const auto now{ std::chrono::system_clock::to_time_t(std::chrono::system_clock::now()) };
00483 const std::tm* const localtime{ std::localtime(&now) };
00484
00485 // 時刻を文字列に変換
00486 const auto timeString{ std::put_time(localtime, "cal_%Y%m%d%H%M%S.json") };
00487 const auto pathString{ static_cast<std::ostringstream&>(std::ostringstream() << timeString).str()
};
00488
00489 #if !defined(__ANDROID__)
00490 // ファイルダイアログから得るパス
00491 nfdchar_t* filepath;
00492
00493 // ファイルダイアログを開く
00494 if (NFD_SaveDialog(&filepath, jsonFilter, 1, NULL, pathString.c_str()) == NFD_OKAY)
00495 {
00496 // 現在のキャリブレーションパラメータを構成ファイルに保存する
00497 if (!calibration.saveParameters(filepath))
00498 {
00499 // 保存できなかった
00500 errorMessage = u8"校正ファイルが保存できません";
00501 }
00502
00503 // ファイルパスの取り出しに使ったメモリを開放する
00504 NFD_FreePath(filepath);
00505 }
00506 #else
00507 if (!calibration.saveParameters(pathString.c_str()))
00508 {
00509 errorMessage = u8"校正ファイルが保存できません";
00510 }
00511 #endif
00512 }
00513
00514 //
00515 // 校正用の画像ファイルを取得する (複数選択)
00516 //
00517 void Menu::recordFileCorners() const
00518 {
00519 #if !defined(__ANDROID__)
00520 // ファイルダイアログから得るパス
00521 const nfdpathset_t* outPaths;
00522
00523 // ファイルダイアログを開く
00524 if (NFD_OpenDialogMultiple(&outPaths, imageFilter, 1, NULL) == NFD_OKAY)
00525 {
00526 // ファイルパスの一覧を得る
00527 nfdpathsetenum_t enumerator;
00528 NFD_PathSet_GetEnum(outPaths, &enumerator);
00529
00530 // ファイルパスの一覧からファイルパスを一つずつ取り出して
00531 for (nfdchar_t* path = NULL; NFD_PathSet_EnumNext(&enumerator, &path) && path;)
00532 {
00533 // 画像の読み出し
00534 CamImage image;
00535
00536 // 画像ファイルが読み出せたら
00537 if (image.open(path))
00538 {
00539 // データのコピー先
00540 cv::Mat frame;
00541
00542 // データをコピーして
00543 image.lockFrame([&frame](const std::uint8_t* data, size_t length, int width, int height, int
channels) {
00544 frame.create(height, width, ((channels - 1) << 3));
00545 std::memcpy(frame.data, data, std::min(static_cast<size_t>(frame.total() * frame.elemSize()),
length));
00546 });
00547
00548 // ボードを検出して
00549 calibration.detectBoard(frame);
00550
00551 // コーナーを記録する
00552 calibration.recordCorners();
00553
00554 // 画像の読み出しを終わる
00555 image.close();
00556 }
00557
00558 // ファイルパスの取り出しに使ったメモリを開放する
00559 NFD_PathSet_FreePath(path);
00560 }
00561
00562 // ファイルパスの一覧に使ったメモリを開放する
00563 NFD_PathSet_FreeEnum(&enumerator);

```

```

00564
00565     // フォルダのパスに使ったメモリを開放する
00566     NFD.PathSet_Free(outPaths);
00567 }
00568 #else
00569 namespace fs = std::filesystem;
00570 std::error_code ec;
00571 for (const auto& entry : fs::directory_iterator(".", ec))
00572 {
00573     if (entry.is_regular_file())
00574     {
00575         const auto ext = entry.path().extension().string();
00576         if (ext == ".jpg" || ext == ".jpeg" || ext == ".png" || ext == ".bmp")
00577         {
00578             CamImage image;
00579             if (image.open(entry.path().string()))
00580             {
00581                 cv::Mat frame;
00582                 image.lockFrame([&frame](const std::uint8_t* data, size_t length, int width, int height, int
channels) {
00583                     frame.create(height, width, ((channels - 1) << 3));
00584                     std::memcpy(frame.data, data, std::min(static_cast<size_t>(frame.total() *
frame.elemSize()), length));
00585                 });
00586                 calibration.detectBoard(frame);
00587                 calibration.recordCorners();
00588                 image.close();
00589             }
00590         }
00591     }
00592 }
00593 #endif
00594 }
00595
00596 //
00597 // 校正用の ChArUco Board を作成する
00598 //
00599 void Menu::createCharuco() const
00600 {
00601     // ChArUco Board の画像を作成する
00602     cv::Mat boardImage;
00603     calibration.drawBoard(boardImage, 980, 692);
00604
00605     // ファイルに保存する
00606     saveImage(boardImage, "ChArUcoBoard.png");
00607 }
00608
00609 //
00610 // コンストラクタ
00611 //
00612 Menu::Menu(Config& config, Capture& capture, Calibration& calibration)
00613 : config{ config }
00614 , settings{ config.getSettings() }
00615 , intrinsics{ config.getPreferences().front().getIntrinsics() }
00616 , capture{ capture }
00617 , calibration{ calibration }
00618 {
00619     // ファイルダイアログ (Native File Dialog Extended) を初期化する
00620     #if !defined(__ANDROID__)
00621         NFD.Init();
00622     #endif
00623
00624     // Dear ImGui の入力デバイス
00625     //io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard; // キーボードコントロールを使う
00626     //io.ConfigFlags |= ImGuiConfigFlags_NavEnableGamepad; // ゲームパッドを使う
00627
00628     // Dear ImGui のスタイル
00629     //ImGui::StyleColorsDark(); // 暗めのスタイル
00630     //ImGui::StyleColorsClassic(); // 以前のスタイル
00631
00632     // 日本語を表示できるメニューフォントを読み込む
00633     // 基本の日本語グリフセット (常用・人名用漢字、ひらがな、カタカナ、英数字) に加え、
00634     // デバイス名等に含まれる一般句読点 (引用符、ダッシュ等) や文字様記号 (商標記号等) を追加する
00635     ImFontGlyphRangesBuilder builder;
00636     builder.AddRanges(ImGui::GetIO().Fonts->GetGlyphRangesJapanese());
00637
00638     // 追加の Unicode 範囲 (2要素で1範囲、0終端)
00639     static const ImWchar additionalRanges[] =
00640     {
00641         0x2000, 0x206F, // General Punctuation (引用符、ダッシュ、リーダー等)
00642         0x2100, 0x214F, // Letterlike Symbols (商標記号 TM 等)
00643         0x2190, 0x21FF, // Arrows (矢印記号等)
00644         0x2460, 0x24FF, // Enclosed Alphanumerics (丸数字・囲み英数字等)
00645         0x25A0, 0x25FF, // Geometric Shapes (幾何学模様・図形記号等)
00646         0,
00647     };
00648     builder.AddRanges(additionalRanges);

```

```

00649
00650 // フォントアトラス構築時までメモリを維持するため static で保持する
00651 static ImVector<ImWchar> glyphRanges;
00652 builder.BuildRanges(&glyphRanges);
00653
00654 if (!ImGui::GetIO().Fonts->AddFontFromFileTTF(config.getMenuFont().c_str(),
config.getMenuFontSize(),
00655     nullptr, glyphRanges.Data))
00656 {
00657     // メニューフォントが読み込めなかったらエラーにする
00658     throw std::runtime_error("Cannot find any menu fonts.");
00659 }
00660
00661 #if defined(_WIN32) || defined(__ANDROID__) || defined(__APPLE__)
00662 // 初期状態で最初のデバイスのフォーマットリストを取得しておく
00663 if (!config.getDeviceList().empty())
00664 {
00665     capture.updateFormatList(deviceNumber);
00666 }
00667 #else
00668 // バックエンドごとのキャプチャデバイスの一覧を初期化する
00669 for (auto& [api, name] : backendList)
00670 {
00671     // バックエンドごとに空のリストを追加する
00672     deviceList.emplace(api, std::vector<std::string>());
00673 }
00674
00675 // キャプチャデバイスの一覧を作る
00676 getAnyList(deviceList.at(cv::CAP_ANY));
00677 #if defined(_MSC_VER)
00678 getDirectShowList(deviceList.at(cv::CAP_DSHOW));
00679 getMediaFoundationList(deviceList.at(cv::CAP_MSMF));
00680 #elif defined(__linux__)
00681 # if defined(USE_LIBCAMERA)
00682 deviceList.at(CAP_LIBCAMERA) = CamLibcam::getDeviceList();
00683 if (!deviceList.at(CAP_LIBCAMERA).empty())
00684 {
00685     backend = CAP_LIBCAMERA;
00686 }
00687 # endif
00688 getV4L2List(deviceList.at(cv::CAP_V4L2));
00689 #endif
00690 #endif
00691 }
00692
00693 //
00694 // デストラクタ
00695 //
00696 Menu::~Menu()
00697 {
00698     // キャプチャスレッドが動いていたら止める
00699     capture.stop();
00700
00701     // 前に開いていたキャプチャデバイスを閉じる
00702     capture.close();
00703
00704     // ファイルダイアログ (Native File Dialog Extended) を終了する
00705 #if !defined(__ANDROID__)
00706     NFD.Quit();
00707 #endif
00708 }
00709
00710 //
00711 // 入力画像に合わせて内部パラメータを初期化する
00712 //
00713 void Menu::initializeInputIntrinsics(const std::array<int, 2>& size)
00714 {
00715     // 実際の入力解像度を処理系へ反映する
00716     intrinsics.size = size;
00717
00718     // 無効な入力によるゼロ除算を避け、有効な場合だけ焦点距離から初期画角を求める
00719     if (size[0] > 0 && size[1] > 0 && settings.focal > 0.0f)
00720     {
00721         intrinsics.setFov(settings.focal);
00722     }
00723 }
00724
00725 //
00726 // 選択中の入力設定を適用してキャプチャを開始する
00727 //
00728 bool Menu::startCapture()
00729 {
00730     // オープンとフォーマット適用を一つの入口に集約し、失敗時は開始処理を中断する
00731     if (!openDevice()) return false;
00732
00733     // デバイスが確定してから動作モードと入力に合う初期画角を反映し、取得スレッドを開始する
00734     capture.setPrioritizeLatency(prioritizeLatency);

```



```

00735     initializeInputIntrinsics (capture.getSize());
00736     capture.start();
00737     return true;
00738 }
00739 //
00740 // 選択中の投影方式とその内部パラメータを同期する
00741 //
00742 //
00743 void Menu::selectPreference(int index)
00744 {
00745     // 不正な選択番号では現在の投影状態を変更しない
00746     if (index < 0 || index >= static_cast<int>(config.getPreferences().size())) return;
00747     // 入力中の実解像度を、投影方式の構成値で上書きしないため回避する
00748     const auto size{intrinsics.size};
00749     preferenceNumber = index;
00750     intrinsics = getPreference().getIntrinsics();
00751     // 入力中は実解像度だけを戻し、画角と中心位置は選択した投影方式の設定値を使用する
00752     if (capture.isOpened()) intrinsics.size = size;
00753 }
00754 //
00755 // 解像度、フレームレート、コーデックの選択リストを更新する
00756 //
00757 void Menu::updateFormatDropdowns()
00758 {
00759     const auto& formatList{capture.getFormatList()};
00760     // デバイス列挙結果を UI 側にコピーし、各ドロップダウンの候補を作り直す
00761     availableFormats = formatList;
00762     uniqueResolutions.clear();
00763     uniqueFpsList.clear();
00764     uniqueCodecs.clear();
00765     // 構造化されたフォーマット情報から選択肢を作成する
00766     for (const auto& info : availableFormats)
00767     {
00768         if (std::find(uniqueResolutions.begin(), uniqueResolutions.end(), info.resolution) ==
00769             uniqueResolutions.end())
00770             uniqueResolutions.push_back(info.resolution);
00771         if (std::find(uniqueFpsList.begin(), uniqueFpsList.end(), info.fps) == uniqueFpsList.end())
00772             uniqueFpsList.push_back(info.fps);
00773         if (std::find(uniqueCodecs.begin(), uniqueCodecs.end(), info.codec) == uniqueCodecs.end())
00774             uniqueCodecs.push_back(info.codec);
00775     }
00776     // 現在の formatNumber のフォーマットに同期する
00777     const auto selected{std::find_if(availableFormats.begin(), availableFormats.end(),
00778         [this](const CaptureFormat& info) { return info.index == formatNumber; })};
00779     if (selected != availableFormats.end())
00780     {
00781         currentRes = selected->resolution;
00782         currentFps = selected->fps;
00783         currentCodec = selected->codec;
00784     }
00785     else
00786     {
00787         currentRes.clear();
00788         currentFps.clear();
00789         currentCodec.clear();
00790     }
00791     lastDeviceNumber = deviceNumber;
00792 }
00793 #endif
00794 //
00795 // シェーダを設定する
00796 //
00797 std::array<GLsizei, 2> Menu::setup(GLfloat aspect) const
00798 {
00799     // シェーダを設定する
00800     return config.getPreferences()[preferenceNumber].getShader().setup(settings.samples, aspect,
00801         pose, intrinsics.fov, intrinsics.center, settings.getFocal(), config.getBackground());
00802 }
00803 //
00804 // 指定した視点姿勢を加えてシェーダを設定する
00805 //
00806 std::array<GLsizei, 2> Menu::setup(GLfloat aspect, const gg::GgMatrix& viewPose) const
00807 {
00808     // メニューの補正を基準空間からカメラ空間への変換として先に適用する
00809     return config.getPreferences()[preferenceNumber].getShader().setup(settings.samples, aspect,
00810         pose * viewPose, intrinsics.fov, intrinsics.center, settings.getFocal(), config.getBackground());
00811 }
00812
00813

```

```

00821 //
00822 // メインメニューバーの描画
00823 //
00824 void Menu::drawMainMenuBar()
00825 {
00826     if (ImGui::BeginMainMenuBar())
00827     {
00828         // ファイルメニュー
00829         if (ImGui::BeginMenu(u8"ファイル"))
00830         {
00831             // 画像ファイルを開く
00832             if (ImGui::MenuItem(u8"画像ファイルを開く")) openImage();
00833
00834             // 動画ファイルを開く
00835             if (ImGui::MenuItem(u8"動画ファイルを開く")) openMovie();
00836
00837             // 構成ファイルを開く
00838             if (ImGui::MenuItem(u8"構成ファイルを開く")) loadConfig();
00839
00840             // 構成ファイルを保存する
00841             if (ImGui::MenuItem(u8"構成ファイルを保存")) saveConfig();
00842
00843             // キャリブレーションパラメータファイルを開く
00844             if (ImGui::MenuItem(u8"校正ファイルを開く")) loadCalibration();
00845
00846             // キャリブレーションパラメータファイルを保存する
00847             if (ImGui::MenuItem(u8"校正ファイルを保存")) saveParameters();
00848
00849             // フォルダ内の画像ファイルを使って校正する
00850             if (ImGui::MenuItem(u8"校正用画像から取得")) recordFileCorners();
00851
00852             // ChArUco Board の作成
00853             if (ImGui::MenuItem(u8"ChArUco 画像作成")) createCharuco();
00854
00855             // 終了
00856             quit = ImGui::MenuItem(u8"終了");
00857
00858             // File メニュー終了
00859             ImGui::EndMenu();
00860         }
00861
00862         // ウィンドウメニュー
00863         if (ImGui::BeginMenu(u8"ウィンドウ"))
00864         {
00865             // 入力パネルの表示
00866             ImGui::MenuItem(u8"入力", NULL, &showInputPanel);
00867
00868             // 校正パネルの表示
00869             ImGui::MenuItem(u8"校正", NULL, &showCalibrationPanel);
00870
00871             // File メニュー終了
00872             ImGui::EndMenu();
00873         }
00874
00875         // メニューバーの高さを保存しておく
00876         menubarHeight = static_cast<GLsizei>(ImGui::GetWindowHeight());
00877
00878         // メインメニューバー終了
00879         ImGui::EndMainMenuBar();
00880     }
00881 }
00882 //
00883 // 入力パネルの描画
00884 //
00885 //
00886 void Menu::drawInputPanel()
00887 {
00888     // 入力パネル
00889     if (showInputPanel)
00890     {
00891         // ウィンドウの位置とサイズ
00892         ImGui::SetNextWindowPos(ImVec2(2.0f, 2.0f + menubarHeight), ImGuiCond_Once);
00893         ImGui::SetNextWindowSize(ImVec2(231, 517), ImGuiCond_Once);
00894         ImGui::Begin(u8"入力", &showInputPanel);
00895
00896         // 投影方式の選択
00897         if (ImGui::BeginCombo(u8"投影方式", getPreference().getDescription().c_str()))
00898         {
00899             // すべての投影方式について
00900             for (int i = 0; i < static_cast<int>(config.getPreferences().size()); ++i)
00901             {
00902                 // その投影方式が選択されていれば真
00903                 const bool selected{ i == preferenceNumber };
00904
00905                 // 投影方式を（それが現在の投影方式ならハイライトして）コンボボックスに表示する
00906                 if (ImGui::Selectable(getPreference(i).getDescription().c_str(), selected))
00907             }

```

```

00908         // 表示した投影方式が選択されていたらそれを現在の選択とする
00909         selectPreference(i);
00910     }
00911
00912     // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00913     if (selected) ImGui::SetItemDefaultFocus();
00914 }
00915 ImGui::EndCombo();
00916 }
00917
00918 // レンズの画角を設定する
00919 ImGui::DragFloat2(u8"画角", intrinsics.fov.data(), 0.1f, -360.0f, 360.0f, "%.2f");
00920
00921 // レンズの中心位置
00922 ImGui::DragFloat2(u8"中心", intrinsics.center.data(), 0.001f, -1.0f, 1.0f, "%.4f");
00923
00924 // キャプチャデバイス固有のパラメータを元に戻す
00925 if (ImGui::Button(u8"回復"))
00926 {
00927     // 選択した投影方式のキャプチャデバイス固有のパラメータを回復する
00928     intrinsics = getPreference().getIntrinsics();
00929 }
00930
00931 // 姿勢
00932 ImGui::SliderAngle(u8"方位", &settings.euler[1], -180.0f, 180.0f, "%.2f");
00933 ImGui::SliderAngle(u8"仰角", &settings.euler[0], -180.0f, 180.0f, "%.2f");
00934 ImGui::SliderAngle(u8"傾斜", &settings.euler[2], -180.0f, 180.0f, "%.2f");
00935 pose = ggRotateY(settings.euler[1]).rotateX(settings.euler[0]).rotateZ(settings.euler[2]);
00936
00937 // 焦点距離
00938 ImGui::SliderFloat(u8"焦点距離", &settings.focal, settings.focalRange[0], settings.focalRange[1],
00939 "%.1f");
00940
00941 // 姿勢を元に戻す
00942 if (ImGui::Button(u8"復帰"))
00943 {
00944     settings.euler = config.getSettings().euler;
00945     settings.focal = config.getSettings().focal;
00946     settings.focalRange = config.getSettings().focalRange;
00947 }
00948
00949 ImGui::Separator();
00950
00951 #if defined(_WIN32) || defined(_ANDROID_) || defined(_APPLE_)
00952 // キャプチャデバイスが存在するとき
00953 if (!config.getDeviceList().empty())
00954 {
00955     // 装置関連項目
00956     ImGui::Text("%s", u8"以下の変更は [開始] で反映します");
00957
00958     // キャプチャデバイスの選択コンボボックス
00959     if (ImGui::BeginCombo(u8"装置", config.getDeviceName(deviceNumber).c_str()))
00960     {
00961         // すべてのキャプチャデバイスについて
00962         for (int i = 0; i < static_cast<int>(config.getDeviceList().size()); ++i)
00963         {
00964             // キャプチャデバイス名を (それを選択していればハイライトして) コンボボックスに表示する
00965             if (ImGui::Selectable(config.getDeviceName(i).c_str(), i == deviceNumber))
00966             {
00967                 // キャプチャデバイスが変わったら
00968                 if (deviceNumber != i)
00969                 {
00970                     // キャプチャスレッドが動いていたら止める
00971                     capture.stop();
00972
00973                     // 前に開いていたキャプチャデバイスを閉じる
00974                     capture.close();
00975
00976                     // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
00977                     deviceNumber = i;
00978
00979                     // キャプチャデバイスが変わったので最初のビデオフォーマットを選択する
00980                     formatNumber = 0;
00981
00982                     // 開始ボタンが押されるまでは、フォーマットリストだけを一時取得して更新する
00983                     capture.updateFormatList(deviceNumber);
00984
00985                     // 選択可能なドロップダウンのリストを更新する
00986                     updateFormatDropdowns();
00987                 }
00988             }
00989             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
00990             ImGui::SetItemDefaultFocus();
00991         }
00992     }
00993     ImGui::EndCombo();
00994 }

```

```

00994
00995 // 使用可能なビデオフォーマットの表示名のリスト
00996 const auto& formatList{ capture.getFormatList() };
00997
00998 // 使用可能なビデオフォーマットが存在するなら
00999 if (!formatList.empty())
01000 {
01001     // 必要な場合 (デバイス番号の不一致やリスト未作成時) にリストを更新する
01002     if (lastDeviceNumber != deviceNumber || availableFormats.empty())
01003     {
01004         updateFormatDropdowns();
01005     }
01006
01007     // 1. 解像度の選択コンボボックス
01008     if (ImGui::BeginCombo(u8"解像度", currentRes.c_str()))
01009     {
01010         for (const auto& res : uniqueResolutions)
01011         {
01012             if (ImGui::Selectable(res.c_str(), currentRes == res))
01013             {
01014                 currentRes = res;
01015                 // 選択した解像度において現在の fps / codec と一致するフォーマットを探し、
01016                 // なければその解像度で利用可能な最初のフォーマットに自動同期する
01017                 const bool matched{ std::any_of(availableFormats.begin(), availableFormats.end(),
01018                     [this](const CaptureFormat& f) {
01019                         return f.resolution == currentRes && f.fps == currentFps && f.codec == currentCodec;
01020                     }) };
01021                 if (!matched)
01022                 {
01023                     const auto it{ std::find_if(availableFormats.begin(), availableFormats.end(),
01024                         [this](const CaptureFormat& f) { return f.resolution == currentRes; }) };
01025                     if (it != availableFormats.end())
01026                     {
01027                         currentFps = it->fps;
01028                         currentCodec = it->codec;
01029                     }
01030                 }
01031             }
01032         }
01033         ImGui::EndCombo();
01034     }
01035
01036     // 2. フレームレートの選択コンボボックス
01037     std::string fpsLabel = currentFps + " fps";
01038     if (ImGui::BeginCombo(u8"コマ数", fpsLabel.c_str()))
01039     {
01040         for (const auto& fpsVal : uniqueFpsList)
01041         {
01042             std::string valLabel = fpsVal + " fps";
01043             if (ImGui::Selectable(valLabel.c_str(), currentFps == fpsVal))
01044             {
01045                 currentFps = fpsVal;
01046                 // 選択した fps において現在の resolution / codec と一致するフォーマットを探し、
01047                 // なければその fps で利用可能な最初のフォーマットに自動同期する
01048                 const bool matched{ std::any_of(availableFormats.begin(), availableFormats.end(),
01049                     [this](const CaptureFormat& f) {
01050                         return f.resolution == currentRes && f.fps == currentFps && f.codec == currentCodec;
01051                     }) };
01052                 if (!matched)
01053                 {
01054                     auto it{ std::find_if(availableFormats.begin(), availableFormats.end(),
01055                         [this](const CaptureFormat& f) { return f.fps == currentFps && f.resolution ==
01056 currentRes; }) };
01057                     if (it == availableFormats.end())
01058                     {
01059                         it = std::find_if(availableFormats.begin(), availableFormats.end(),
01060                             [this](const CaptureFormat& f) { return f.fps == currentFps; });
01061                     }
01062                     if (it != availableFormats.end())
01063                     {
01064                         currentRes = it->resolution;
01065                         currentCodec = it->codec;
01066                     }
01067                 }
01068             }
01069         }
01070         ImGui::EndCombo();
01071     }
01072
01073     // 3. コーデックの選択コンボボックス
01074     if (ImGui::BeginCombo(u8"符号化", currentCodec.c_str()))
01075     {
01076         for (const auto& cod : uniqueCodecs)
01077         {
01078             if (ImGui::Selectable(cod.c_str(), currentCodec == cod))
01079             {
01080                 currentCodec = cod;

```

```

01080         // 選択した codec において現在の resolution / fps と一致するフォーマットを探し、
01081         // なければその codec で利用可能な最初のフォーマットに自動同期する
01082         const bool matched{ std::any_of(availableFormats.begin(), availableFormats.end(),
01083             [this](const CaptureFormat& f) {
01084                 return f.resolution == currentRes && f.fps == currentFps && f.codec == currentCodec;
01085             }) };
01086         if (!matched)
01087         {
01088             auto it{ std::find_if(availableFormats.begin(), availableFormats.end(),
01089                 [this](const CaptureFormat& f) { return f.codec == currentCodec && f.resolution ==
01090                     currentRes; }) };
01091             if (it == availableFormats.end())
01092             {
01093                 it = std::find_if(availableFormats.begin(), availableFormats.end(),
01094                     [this](const CaptureFormat& f) { return f.codec == currentCodec; });
01095             }
01096             if (it != availableFormats.end())
01097             {
01098                 currentRes = it->resolution;
01099                 currentFps = it->fps;
01100             }
01101         }
01102     }
01103     ImGui::EndCombo();
01104 }
01105
01106 // 選択された組み合わせが availableFormats に存在するか探す
01107 int foundIndex = -1;
01108 for (const auto& info : availableFormats)
01109 {
01110     if (info.resolution == currentRes && info.fps == currentFps && info.codec == currentCodec)
01111     {
01112         foundIndex = info.index;
01113         break;
01114     }
01115 }
01116
01117 bool formatExists = (foundIndex != -1);
01118 if (formatExists)
01119 {
01120     // 存在する場合は、必要なら formatNumber を更新する
01121     if (formatNumber != foundIndex)
01122     {
01123         capture.stop();
01124         formatNumber = foundIndex;
01125     }
01126 }
01127
01128 // 4. レイテンシ優先のチェックボックス
01129 if (ImGui::Checkbox(u8"レイテンシ 優先", &prioritizeLatency))
01130 {
01131     if (capture) capture.setPrioritizeLatency(prioritizeLatency);
01132 }
01133
01134 // キャプチャの開始と停止
01135 if (capture)
01136 {
01137     // キャプチャスレッドが動いているので止める
01138     if (ImGui::Button(u8"停止")) capture.stop();
01139     ImGui::SameLine();
01140     ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"取得中");
01141 }
01142 else
01143 {
01144     if (formatExists)
01145     {
01146         // 「開始」 ボタンをクリックしたときデバイスが選択されていれば
01147         if (ImGui::Button(u8"開始") && deviceNumber >= 0)
01148         {
01149             startCapture();
01150         }
01151         ImGui::SameLine();
01152         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"停止中");
01153     }
01154     else
01155     {
01156         // 存在しない組み合わせの時はメッセージを表示する
01157         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"フォーマットが存在しません");
01158     }
01159 }
01160 }
01161 else
01162 {
01163     // キャプチャデバイスが開けなかった
01164     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが開けません");
01165 }

```

```

01166     }
01167     else
01168     {
01169         // キャプチャデバイスが存在しないとき
01170         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが見つかりません");
01171     }
01172 #else
01173     // 装置関連項目
01174     ImGui::Text("%s", u8"以下の変更は [開始] で反映します");
01175
01176     // デバイスプリファレンスを選択する
01177     if (ImGui::BeginCombo(u8"装置特性", backendList.at(backend)))
01178     {
01179         // すべての表示方式について
01180         for (auto& [apiId, apiName] : backendList)
01181         {
01182             // その表示方式が選択されていれば真
01183             const bool selected{ apiId == backend };
01184
01185             // 装置特性を（それが現在の装置特性ならハイライトして）コンボボックスに表示する
01186             if (ImGui::Selectable(apiName, selected))
01187             {
01188                 // 表示した装置特性が選択されていたらそれを現在の選択とする
01189                 backend = apiId;
01190
01191                 // 切り替え前の装置特性のデバイスが存在しなければ最初のデバイスの番号を選択する
01192                 if (deviceNumber < 0) deviceNumber = 0;
01193
01194                 // 選択されているデバイスの番号が接続されたキャプチャデバイスの数を超えないようにする
01195                 const int count{ getDeviceCount(backend) };
01196                 if (deviceNumber >= count) deviceNumber = count - 1;
01197             }
01198
01199             // この選択を次にコンボボックスを開いたときのデフォルトにしておく
01200             if (selected) ImGui::SetItemDefaultFocus();
01201         }
01202         ImGui::EndCombo();
01203     }
01204
01205     // キャプチャデバイスが存在すれば
01206     if (deviceNumber >= 0)
01207     {
01208         // キャプチャデバイスの選択コンボボックス
01209         if (ImGui::BeginCombo(u8"入力源", getDeviceName(backend, deviceNumber).c_str()))
01210         {
01211             // すべてのキャプチャデバイスについて
01212             for (int i = 0; i < static_cast<int>(getDeviceList(backend).size()); ++i)
01213             {
01214                 // キャプチャデバイス名を（それを選択していればハイライトして）コンボボックスに表示する
01215                 if (ImGui::Selectable(getDeviceName(backend, i).c_str(), i == deviceNumber))
01216                 {
01217                     // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
01218                     deviceNumber = i;
01219
01220                     // この選択を次にコンボボックスを開いたときのデフォルトにしておく
01221                     ImGui::SetItemDefaultFocus();
01222                 }
01223             }
01224             ImGui::EndCombo();
01225         }
01226     }
01227     else
01228     {
01229         // 使えるキャプチャデバイスがない
01230         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"デバイスが見つかりません");
01231     }
01232
01233     // キャプチャするサイズとフレームレート
01234     ImGui::InputInt2(u8"解像度", intrinsic.size.data());
01235     ImGui::InputDouble(u8"周波数", &intrinsic.fps, 1.0f, 1.0f, "%.1f");
01236
01237     // コーデックを選択する
01238     if (ImGui::BeginCombo(u8"符号化", codecList[codecNumber]))
01239     {
01240         // すべてのコーデックについて
01241         for (int i = 0; i < static_cast<int>(codecList.size()); ++i)
01242         {
01243             // コーデックを（それを選択していればハイライトして）コンボボックスに表示する
01244             if (ImGui::Selectable(codecList[i], i == codecNumber))
01245             {
01246                 // 表示したキャプチャデバイスが選択されていたらそのキャプチャデバイスを選択する
01247                 codecNumber = i;
01248
01249                 // この選択を次にコンボボックスを開いたときのデフォルトにしておく
01250                 ImGui::SetItemDefaultFocus();
01251             }
01252         }
01253     }

```

```

01253     ImGui::EndCombo();
01254 }
01255
01256 // キャプチャの開始と停止
01257 if (capture)
01258 {
01259     // キャプチャスレッドが動いているので止める
01260     if (ImGui::Button(u8"停止")) capture.stop();
01261     ImGui::SameLine();
01262     ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"取得中");
01263 }
01264 else
01265 {
01266     // キャプチャスレッドが止まっているので
01267     if (ImGui::Button(u8"開始") && deviceNumber >= 0)
01268     {
01269         startCapture();
01270     }
01271     ImGui::SameLine();
01272     ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", u8"停止中");
01273 }
01274 #endif
01275 ImGui::End();
01276 }
01277 }
01278
01279 //
01280 // 校正パネルの描画
01281 //
01282 void Menu::drawCalibrationPanel()
01283 {
01284     if (showCalibrationPanel)
01285     {
01286         // ウィンドウの位置とサイズ
01287         ImGui::SetNextWindowPos(ImVec2(235.0f, 2.0f + menubarHeight), ImGuiCond.Once);
01288         ImGui::SetNextWindowSize(ImVec2(218, 359), ImGuiCond.Once);
01289         ImGui::Begin(u8"校正", &showCalibrationPanel);
01290
01291         // 辞書の選択
01292         if (ImGui::BeginCombo(u8"辞書", settings.dictionaryName.c_str()))
01293         {
01294             // すべての辞書について
01295             for (auto d = calibration.dictionaryList.begin(); d != calibration.dictionaryList.end(); ++d)
01296             {
01297                 // その設定が現在選択されている設定なら真
01298                 const bool selected(d->first == settings.dictionaryName);
01299
01300                 // 設定を（それが現在の設定ならハイライトして）コンボボックスに表示する
01301                 if (ImGui::Selectable(d->first.c_str(), d->first == settings.dictionaryName))
01302                 {
01303                     // 表示した設定が選択されていたらそれを現在の選択とする
01304                     settings.dictionaryName = d->first;
01305
01306                     // 選択した ArUco Marker の辞書を設定する
01307                     calibration.setDictionary(settings.dictionaryName, settings.checkerSize,
01308 settings.checkerLength);
01309                 }
01310
01311                 // この選択を次にコンボボックスを開いたときのデフォルトにしておく
01312                 if (selected) ImGui::SetItemDefaultFocus();
01313             }
01314             ImGui::EndCombo();
01315         }
01316
01317         ImGui::Separator();
01318
01319         // ArUco Marker の検出
01320         if (ImGui::Checkbox(u8"ArUco Marker 検出", &detectMarker) && detectMarker) detectBoard = false;
01321
01322         // ArUco Marker の大きさ
01323         ImGui::InputFloat(u8"マーカー長", &settings.markerLength, 0.0f, 0.0f, "%.2f cm");
01324
01325         ImGui::Separator();
01326
01327         // 校正
01328         if (ImGui::Checkbox(u8"ChArUco Board 検出", &detectBoard) && detectBoard) detectMarker = false;
01329
01330         // ChArUco Board のマス目の数 (横, 縦)
01331         if (ImGui::InputInt2(u8"升目数", settings.checkerSize.data()))
01332         {
01333             // 升目の数は 2 以上とする
01334             settings.checkerSize[0] = std::max(2, settings.checkerSize[0]);
01335             settings.checkerSize[1] = std::max(2, settings.checkerSize[1]);
01336
01337             // ChArUco Board を作り直す
01338             calibration.createBoard(settings.checkerSize, settings.checkerLength);
01339         }
01340     }
01341 }

```

```

01339
01340 // ChArUco Board の大きさ
01341 if (ImGui::InputFloat2(u8"升目長", settings.checkerLength.data(), "%.2f cm"))
01342 {
01343     // ChArUco Board を作り直す
01344     calibration.createBoard(settings.checkerSize, settings.checkerLength);
01345 }
01346
01347 // 「取得」ボタンをクリックしたとき ChArUco Board の検出中なら
01348 if (ImGui::Button(u8"取得") && detectBoard)
01349 {
01350     // 検出したコーナーを記録する
01351     calibration.recordCorners();
01352 }
01353
01354 // 1つでも標本を取得していれば
01355 if (calibration.getSampleCount() > 0)
01356 {
01357     // 標本の「消去」ボタンを表示する
01358     ImGui::SameLine();
01359     if (ImGui::Button(u8"消去"))
01360     {
01361         calibration.discardCorners();
01362         autoCaptureStatusMessage.clear();
01363     }
01364
01365     // 標本を6つ以上取得していれば
01366     if (calibration.getSampleCount() >= 6)
01367     {
01368         // 「校正」ボタンを表示する
01369         ImGui::SameLine();
01370         if (ImGui::Button(u8"校正") && !calibration.calibrate())
01371         {
01372             // 校正失敗
01373             errorMessage = u8"校正に失敗しました";
01374         }
01375
01376         // 校正が完了していれば
01377         if (calibration.finished())
01378         {
01379             // 「完了」を表示する
01380             ImGui::SameLine();
01381             ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.0f, 1.0f), "%s", u8"完了");
01382         }
01383     }
01384 }
01385
01386 // 自動キャプチャ機能 (ChArUco Board 検出中のみ有効)
01387 if (detectBoard)
01388 {
01389     ImGui::Separator();
01390     ImGui::Checkbox(u8"自動キャプチャ", &autoCaptureEnabled);
01391     if (autoCaptureEnabled)
01392     {
01393         ImGui::SameLine();
01394         ImGui::Checkbox(u8"音", &autoCaptureBeep);
01395
01396         ImGui::SliderFloat(u8"静止時間", &autoCaptureMinStableTime, 0.3f, 1.5f, "%.2f s");
01397
01398         // クールダウンまたは静止プログレスバー表示
01399         if (autoCaptureCooldownTimer > 0.0f)
01400         {
01401             const float cdRatio{ autoCaptureCooldownTimer / autoCaptureCooldown };
01402             ImGui::ProgressBar(cdRatio, ImVec2(-1, 0), u8"姿勢変更待機中...");
01403         }
01404         else
01405         {
01406             const float prog{ calibration.getStableProgress(autoCaptureMinStableTime) };
01407             std::string progText{ u8"静止検知: " + std::to_string(static_cast<int>(prog * 100)) + "%" };
01408             if (prog >= 1.0f)
01409             {
01410                 progText = calibration.isDiverseEnough() ? u8"記録可能 (多様性OK)" : u8"多様性不足 (動かしてくだ
01411                 さい)";
01412             }
01413             ImGui::ProgressBar(prog, ImVec2(-1, 0), progText.c_str());
01414         }
01415
01416         // 変位量・静止状態テキスト
01417         const float motion{ calibration.getCurrentMotion() };
01418         if (calibration.isStable())
01419         {
01420             if (calibration.isDiverseEnough())
01421                 ImGui::TextColored(ImVec4(0.2f, 1.0f, 0.2f, 1.0f), "%s", u8"静止状態: 安定 (多様性あり)");
01422             else
01423                 ImGui::TextColored(ImVec4(0.8f, 0.8f, 0.3f, 1.0f), "%s", u8"静止状態: 安定 (前回姿勢と類似)");
01424         }
01425         else

```



```

01425     {
01426         if (motion < 100.0f)
01427             ImGui::Text(u8"変位量: %.1f px (進捗: %.0f%%)", motion,
calibration.getStableProgress(autoCaptureMinStableTime) * 100.0f);
01428         else
01429             ImGui::Text("%s", u8"変位量: -- px (コーナー不足)");
01430     }
01431
01432     // 自動記録ステータスメッセージがあれば表示
01433     if (!autoCaptureStatusMessage.empty())
01434     {
01435         ImGui::TextColored(ImVec4(0.3f, 0.8f, 1.0f, 1.0f), "%s", autoCaptureStatusMessage.c_str());
01436     }
01437 }
01438 }
01439
01440 ImGui::Separator();
01441
01442 // フレームレートの表示
01443 ImGui::Text(u8"フレームレート: %6.2f fps", ImGui::GetIO().Framerate);
01444
01445 // 検出数の表示
01446 ImGui::Text(u8"コーナー検出数: %d", calibration.getCornersCount());
01447
01448 // 標本数の表示
01449 ImGui::Text(u8"サンプル取得数: %d (%d)", calibration.getSampleCount(), calibration.getTotalCount());
01450
01451 // 再投影誤差の表示
01452 ImGui::Text(u8"再投影誤差: %.4f", calibration.getReprojectionError());
01453
01454 ImGui::End();
01455 }
01456 }
01457
01458 //
01459 // エラーダイアログの描画
01460 //
01461 void Menu::drawErrorDialog()
01462 {
01463     // エラーメッセージが設定されていたら
01464     if (errorMessage)
01465     {
01466         // ウィンドウの位置・サイズとタイトル
01467         ImGui::SetNextWindowPos(ImVec2(60, 60), ImGuiCond_Once);
01468         ImGui::SetNextWindowSize(ImVec2(240, 92), ImGuiCond_Always);
01469
01470         // ウィンドウを表示するとき true
01471         bool status{ true };
01472
01473         // エラーメッセージウィンドウを表示する
01474         ImGui::Begin(u8"エラー", &status);
01475
01476         // エラーメッセージの表示
01477         ImGui::TextColored(ImVec4(1.0f, 0.2f, 0.0f, 1.0f), "%s", errorMessage);
01478
01479         // クローズボックスか「閉じる」ボタンをクリックしたら
01480         if (!status || ImGui::Button(u8"閉じる"))
01481         {
01482             // エラーメッセージを消去する
01483             errorMessage = nullptr;
01484         }
01485         ImGui::End();
01486     }
01487 }
01488
01489 //
01490 // メニューの描画
01491 //
01492 void Menu::draw()
01493 {
01494     // 各ウィンドウの描画責務を分離し、この関数では一フレーム分の呼び出し順だけを管理する
01495     drawMainMenuBar();
01496     drawInputPanel();
01497     drawCalibrationPanel();
01498     drawErrorDialog();
01499
01500 #if defined(__ANDROID__)
01501     drawFileModal();
01502 #endif
01503
01504     // ChArUco Board の検出中にスペースバーをタイプしたなら
01505     if (detectBoard && ImGui::IsKeyPressed(ImGuiKey_Space))
01506     {
01507         // 検出したコーナーを記録する
01508         calibration.recordCorners();
01509     }
01510 }

```

```

01511
01512 //
01513 // 画像の保存
01514 //
01515 void Menu::saveImage(const cv::Mat& image, const std::string& filename) const
01516 {
01517 #if !defined(__ANDROID__)
01518 // 画像ファイル名のフィルタ
01519 constexpr nfdfilteritem_t imageFilter[] { "Images", "png,jpg,jpeg,jfif,bmp,dib" };
01520
01521 // ファイルダイアログから得るパス
01522 nfdchar_t* filepath;
01523
01524 // ファイルダイアログを開く
01525 if (NFD_SaveDialog(&filepath, imageFilter, 1, NULL, filename.c_str()) == NFD_OKAY)
01526 {
01527 // ファイルに保存する
01528 if (!CamImage::save(filepath, image)) errorMessage = u8"ファイルが保存できませんでした";
01529
01530 // ファイルパスの取り出しに使ったメモリを開放する
01531 NFD_FreePath(filepath);
01532 }
01533 #else
01534 if (!CamImage::save(filename, image)) errorMessage = u8"ファイルが保存できませんでした";
01535 #endif
01536 }
01537
01538 //
01539 // 自動キャプチャ処理を更新する
01540 //
01541 // 【目的】
01542 //   ChArUco ボードの静止状態および幾何多様性を判定し、
01543 //   条件を満たした場合に自動的に標本（コーナー）を記録する。
01544 //
01545 bool Menu::updateAutoCapture(float deltaTime)
01546 {
01547 // ChArUco Board 検出中でなければ何もしない
01548 if (!detectBoard) return false;
01549
01550 // 1. 静止判定（フレーム間変位追跡）を更新
01551 //   motionThresholdPx = 2.0px, minStableTime = autoCaptureMinStableTime, minCorners = 6
01552 calibration.updateMotion(deltaTime, 2.0f, autoCaptureMinStableTime, 6);
01553
01554 // 2. 姿勢変更クールダウンタイマーの更新
01555 if (autoCaptureCooldownTimer > 0.0f)
01556 {
01557   autoCaptureCooldownTimer -= deltaTime;
01558   if (autoCaptureCooldownTimer < 0.0f) autoCaptureCooldownTimer = 0.0f;
01559 }
01560
01561 // 3. 自動キャプチャが無効、またはクールダウン中なら記録判定をスキップ
01562 if (!autoCaptureEnabled || autoCaptureCooldownTimer > 0.0f) return false;
01563
01564 // 4. 静止判定かつ多様性チェック
01565 if (calibration.isStable() && calibration.isDiverseEnough())
01566 {
01567 // 標本を記録する
01568 calibration.recordCorners();
01569
01570 // 標本番号
01571 const int sampleIndex{ calibration.getSampleCount() };
01572 autoCaptureStatusMessage = u8"[自動] 標本 #" + std::to_string(sampleIndex) + u8" を記録しました";
01573
01574 // 姿勢変更のためのクールダウンを開始
01575 autoCaptureCooldownTimer = autoCaptureCooldown;
01576
01577 // 5. 音響フィードバック
01578 if (autoCaptureBeep)
01579 {
01580 #if defined(_WIN32)
01581   std::thread([] { Beep(1200, 100); }).detach();
01582 #elif !defined(__ANDROID__)
01583   std::cout << '\a' << std::flush;
01584 #endif
01585 }
01586
01587 return true;
01588 }
01589
01590 return false;
01591 }
01592
01593 #if defined(__ANDROID__)
01594 //
01595 // ファイル選択モーダルの描画
01596 //
01597 void Menu::drawFileModal()

```

```

01598 {
01599     if (!showFileModal) return;
01600
01601     ImGui::OpenPopup(u8"ファイル選択");
01602     if (ImGui::BeginPopupModal(u8"ファイル選択", &showFileModal, ImGuiWindowFlags_AlwaysAutoResize))
01603     {
01604         namespace fs = std::filesystem;
01605         std::error_code ec;
01606
01607         // 現在の作業ディレクトリ（内部ストレージ）内のファイルを列挙
01608         for (const auto& entry : fs::directory_iterator(".", ec))
01609         {
01610             if (entry.is_regular_file())
01611             {
01612                 const auto path = entry.path();
01613                 const auto ext = path.extension().string();
01614                 const auto name = path.filename().string();
01615
01616                 bool match = false;
01617                 if (fileModalType == FileModalType::Image)
01618                 {
01619                     match = (ext == ".jpg" || ext == ".jpeg" || ext == ".png" || ext == ".bmp");
01620                 }
01621                 else if (fileModalType == FileModalType::Calibration)
01622                 {
01623                     match = (ext == ".json");
01624                 }
01625
01626                 if (match)
01627                 {
01628                     if (ImGui::Selectable(name.c_str()))
01629                     {
01630                         if (fileModalType == FileModalType::Image)
01631                         {
01632                             capture.stop();
01633                             if (capture.openImage(name))
01634                             {
01635                                 initializeInputIntrinsics(capture.getSize());
01636                             }
01637                             else
01638                             {
01639                                 errorMessage = u8"画像ファイルが開けません";
01640                             }
01641                         }
01642                         else if (fileModalType == FileModalType::Calibration)
01643                         {
01644                             if (!calibration.loadParameters(name))
01645                             {
01646                                 errorMessage = u8"校正ファイルが読み込めません";
01647                             }
01648                         }
01649                         showFileModal = false;
01650                     }
01651                 }
01652             }
01653         }
01654
01655         ImGui::Separator();
01656         if (ImGui::Button(u8"キャンセル", ImVec2(120, 0)))
01657         {
01658             showFileModal = false;
01659         }
01660         ImGui::EndPopup();
01661     }
01662 }
01663 #endif

```

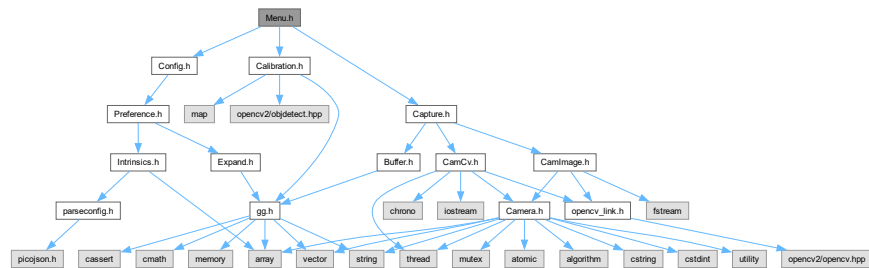
9.67 Menu.h ファイル

```

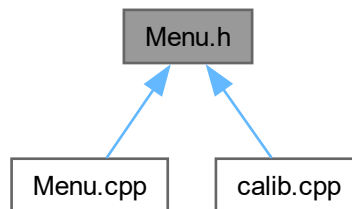
#include "Config.h"
#include "Capture.h"
#include "Calibration.h"

```

Menu.h の依存先関係図:



被依存関係図:



クラス

- class [Menu](#)

9.67.1 詳解

メニューの描画クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Menu.h](#) に定義があります。

9.68 Menu.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // 構成データ
00012 #include "Config.h"
00013
00014 // キャプチャデバイス
00015 #include "Capture.h"
00016
00017 // 校正オブジェクト
00018 #include "Calibration.h"
00019
00023 class Menu
00024 {
00026     Config& config;
00027
00029     Settings settings;
00030
00031 #if !defined(_WIN32) && !defined(__ANDROID__) && !defined(__APPLE__)
00033     static const std::map<cv::VideoCaptureAPIs, const char*> backendList;
00034
00036     static const std::vector<const char*> codecList;
00037
00039     static std::map<cv::VideoCaptureAPIs, std::vector<std::string>> deviceList;
00040
00042     std::vector<std::string> fileHistory;
00043
00050     const auto& getDeviceList (cv::VideoCaptureAPIs api) const
00051     {
00052         return deviceList.at(api);
00053     }
00054
00061     auto getDeviceCount (cv::VideoCaptureAPIs api) const
00062     {
00063         return static_cast<int>(deviceList.at(api).size());
00064     }
00065
00073     const auto& getDeviceName (cv::VideoCaptureAPIs api, int number) const
00074     {
00075         static const std::string empty{};
00076         const auto& list{ deviceList.at(api) };
00077         return list.empty() ? empty : list[number];
00078     }
00079 #endif
00080
00082     Intrinsics intrinsics;
00083
00085     Capture& capture;
00086
00088     Calibration& calibration;
00089
00091     int deviceNumber{ 0 };
00092
00093 #if defined(_WIN32) || defined(__ANDROID__) || defined(__APPLE__)
00095     int formatNumber{ 0 };
00096
00098     std::vector<CaptureFormat> availableFormats;
00099
00101     std::vector<std::string> uniqueResolutions;
00102
00104     std::vector<std::string> uniqueFpsList;
00105
00107     std::vector<std::string> uniqueCodecs;
00108
00110     std::string currentRes;
00111
00113     std::string currentFps;
00114
00116     std::string currentCodec;
00117
00119     int lastDeviceNumber{ -1 };
00120
00124     void updateFormatDropdowns();
00125 #else
00127     int codecNumber{ 0 };
00128
00130     cv::VideoCaptureAPIs backend{ cv::CAP_ANY };
00131 #endif
00132
00133 #if defined(__ANDROID__)
00135     enum class FileModalType { None, Image, Calibration };
00136

```

```

00138   FileModalType fileModalType{ FileModalType::None };
00139
00141   bool showFileModal{ false };
00142
00144   void drawFileModal();
00145 #endif
00146
00148   int preferenceNumber{ 0 };
00149
00151   GgMatrix pose{ ggIdentity() };
00152
00154   GLsizei menubarHeight{ 0 };
00155
00157   bool showInputPanel{ true };
00158
00160   bool showCalibrationPanel{ true };
00161
00163   bool quit{ false };
00164
00166   mutable const char* errorMessage{ nullptr };
00167
00172   bool openDevice();
00173
00179   bool startCapture();
00180
00184   void openImage();
00185
00189   void openMovie();
00190
00194   void loadConfig();
00195
00199   void saveConfig() const;
00200
00204   void loadCalibration();
00205
00209   void saveParameters() const;
00210
00214   void recordFileCorners() const;
00215
00219   void createCharuco() const;
00220
00229   void selectPreference(int index);
00230
00237   const auto& getPreference(int i) const
00238   {
00239       return config.getPreferences()[i];
00240   }
00241
00247   const auto& getPreference() const
00248   {
00249       return getPreference(preferenceNumber);
00250   }
00251
00255   void drawMainMenuBar();
00256
00260   void drawInputPanel();
00261
00265   void drawCalibrationPanel();
00266
00270   void drawErrorDialog();
00271
00272 public:
00273
00275   bool prioritizeLatency{ false };
00276
00278   bool detectMarker{ false };
00279
00281   bool detectBoard{ false };
00282
00284   bool autoCaptureEnabled{ false };
00285
00287   float autoCaptureMinStableTime{ 0.6f };
00288
00290   float autoCaptureCooldown{ 1.5f };
00291
00293   float autoCaptureCooldownTimer{ 0.0f };
00294
00296   bool autoCaptureBeep{ true };
00297
00299   std::string autoCaptureStatusMessage;
00300
00307   bool updateAutoCapture(float deltaTime);
00308
00316   Menu(Config& config, Capture& capture, Calibration& calibration);
00317
00323   Menu(const Menu& menu) = delete;
00324

```

```

00328     virtual ~Menu();
00329
00336     Menu& operator=(const Menu& menu) = delete;
00337
00343     explicit operator bool() const
00344     {
00345         return !quit;
00346     }
00347
00353     const auto& getPose() const
00354     {
00355         return pose;
00356     }
00357
00363     auto getMenuBarHeight() const
00364     {
00365         return menubarHeight;
00366     }
00367
00376     void initializeInputIntrinsics(const std::array<int, 2>& size);
00377
00383     const auto& getCheckerSize() const
00384     {
00385         return settings.checkerSize;
00386     }
00387
00393     const auto& getCheckerLength() const
00394     {
00395         return settings.checkerLength;
00396     }
00397
00403     auto getMarkerLength() const
00404     {
00405         return settings.markerLength;
00406     }
00407
00417     std::array<GLsizei, 2> setup(GLfloat aspect) const;
00418
00426     std::array<GLsizei, 2> setup(GLfloat aspect, const gg::GgMatrix& viewPose) const;
00427
00431     void draw();
00432
00439     void saveImage(const cv::Mat& image, const std::string& filename = "*.jpg") const;
00440 };

```

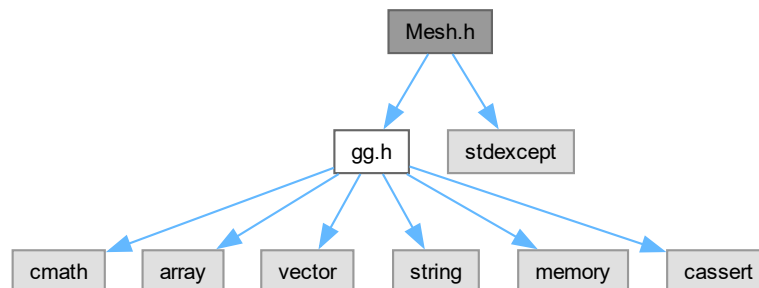
9.69 Mesh.h ファイル

```

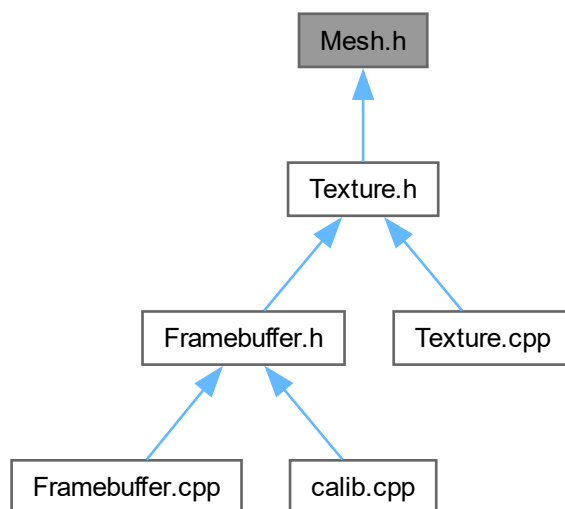
#include "gg.h"
#include <stdexcept>

```

Mesh.h の依存先関係図:



被依存関係図:



クラス

- class [Mesh](#)

9.69.1 詳解

メッシュの描画クラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Mesh.h](#) に定義があります。

9.70 Mesh.h

[詳解]

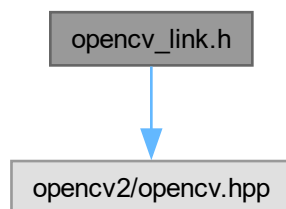
```

00001 #pragma once
00002
00010
00011 // 補助プログラム
00012 #include "gg.h"
00013
00014 // 標準ライブラリ
00015 #include <stdexcept>
00016
00020 class Mesh
00021 {
00023     const GLuint array;
00024
00026     const GLuint program;
00027
00029     const GLint imageLoc;
00030
00032     const GLint scaleLoc;
00033
00034 public:
00035
00039     Mesh()
00040     : array{ [] { GLuint array; glGenVertexArrays(1, &array); return array; }() }
00041     , program{ gg::ggLoadShader("draw.vert", "draw.frag") }
00042     , imageLoc{ glGetUniformLocation(program, "image") }
00043     , scaleLoc{ glGetUniformLocation(program, "scale") }
00044     {
00045         // 頂点配列オブジェクトが作れなかったら落とす
00046         assert(array);
00047
00048         // シェーダが作れなかったら落とす
00049         if (program == 0) throw std::runtime_error("Cannot create the shader for the mesh.");
00050     }
00051
00055     Mesh(const Mesh& mesh) = delete;
00056
00060     virtual ~Mesh()
00061     {
00062         glDeleteVertexArrays(1, &array);
00063         glDeleteProgram(program);
00064     }
00065
00072     Mesh& operator=(const Mesh& mesh) = delete;
00073
00084     void draw(const std::array<GLfloat, 2>& scale, GLint unit = 0) const
00085     {
00086         // 表示領域を背景色で塗りつぶす
00087         glClear(GL_COLOR_BUFFER_BIT);
00088
00089         // シェーダの使用開始
00090         glUseProgram(program);
00091
00092         // サンプラの指定
00093         glUniform1i(imageLoc, unit);
00094
00095         // スケールの設定
00096         glUniform2fv(scaleLoc, 1, scale.data());
00097
00098         // 描画
00099         glBindVertexArray(array);
00100         glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
00101         glBindVertexArray(0);
00102
00103         // シェーダの仕様終了
00104         glUseProgram(0);
00105     }
00106
00116     void drawMesh(const std::array<GLsizei, 2>& size) const
00117     {
00118         // 描画
00119         glBindVertexArray(array);
00120         glDrawArraysInstanced(GL_TRIANGLE_STRIP, 0, size[0] * 2, size[1] - 1);
00121         glBindVertexArray(0);
00122     }
00123 };

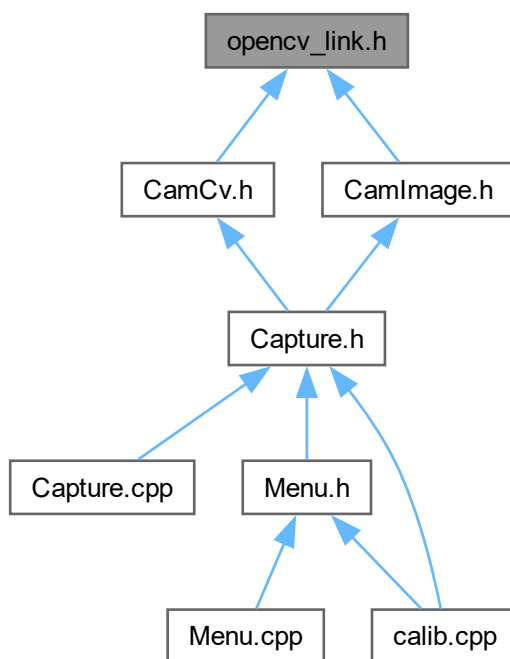
```

9.71 opencv_link.h ファイル

`#include <opencv2/opencv.hpp>`
opencv_link.h の依存先関係図:



被依存関係図:



9.71.1 詳解

MSVC環境用のOpenCV自動リンクおよびインクルード一括管理ヘッダー
[opencv_link.h](#) に定義があります。

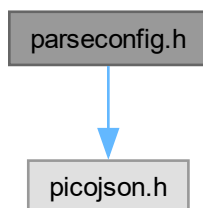
9.72 opencv_link.h

[詳解]

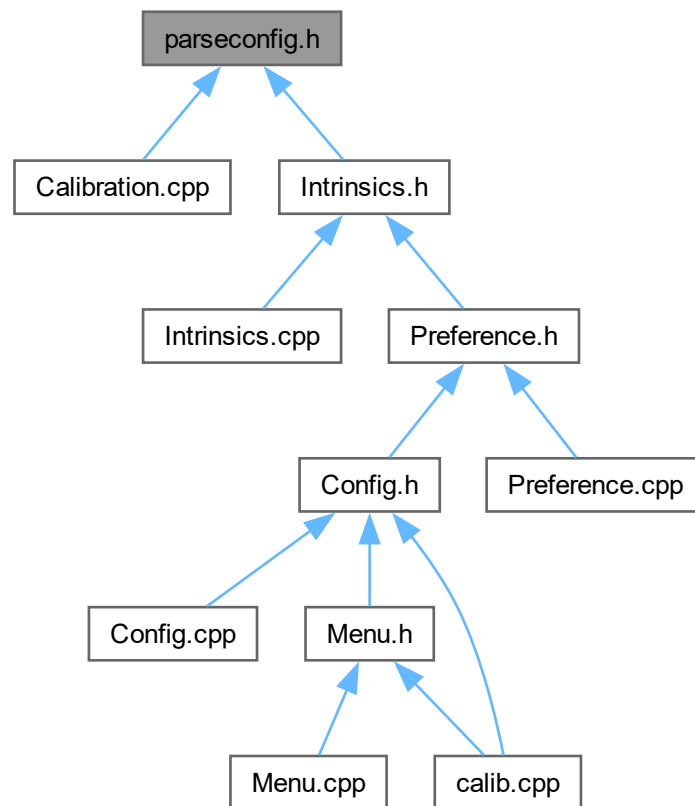
```
00001 #pragma once
00002
00008
00009 // OpenCV
00010 #pragma warning(disable:4819)
00011 #include <opencv2/opencv.hpp>
00012
00013 #if defined(_WIN32)
00014 #   define CV_VERSION_STR CVAUX_STR(CV_MAJOR_VERSION) CVAUX_STR(CV_MINOR_VERSION)
00015 #       CVAUX_STR(CV_SUBMINOR_VERSION)
00016 #   if defined(_DEBUG)
00017 #       define CV_EXT_STR "d.lib"
00018 #   else
00019 #       define CV_EXT_STR ".lib"
00020 #   endif
00021 #   pragma comment(lib, "opencv_world" CV_VERSION_STR CV_EXT_STR)
00022 #endif
```

9.73 parseconfig.h ファイル

```
#include "picojson.h"
parseconfig.h の依存先関係図:
```



被依存関係図:



関数

- `template<typename T>`
`bool getValue (const picojson::object &object, const std::string &key, T &scalar)`
- `template<typename T, std::size_t U>`
`bool getValue (const picojson::object &object, const std::string &key, std::array< T, U > &vector)`
- `bool getString (const picojson::object &object, const std::string &key, std::string &string)`
- `template<std::size_t U>`
`bool getString (const picojson::object &object, const std::string &key, std::array< std::string, U > &strings)`
- `bool getString (const picojson::object &object, const std::string &key, std::vector< std::string > &strings)`
- `template<typename T>`
`void setValue (picojson::object &object, const std::string &key, const T &scalar)`
- `template<typename T, std::size_t U>`
`void setValue (picojson::object &object, const std::string &key, const std::array< T, U > &vector)`
- `void setString (picojson::object &object, const std::string &key, const std::string &string)`
- `template<std::size_t U>`
`void setString (picojson::object &object, const std::string &key, const std::array< std::string, U > &strings)`
- `void setString (picojson::object &object, const std::string &key, const std::vector< std::string > &strings)`

9.73.1 詳解

構成ファイルの読み取り補助

著者

Kohe Tokoi

日付

November 15, 2022

[parseconfig.h](#) に定義があります。

9.73.2 関数詳解

9.73.2.1 getString() [1/3]

```
template<std::size_t U>
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::array< std::string, U > & strings)
```

構成ファイルの JSON オブジェクトから文字列の配列を取得する

テンプレート引数

<i>U</i>	構成ファイルから取得する配列の要素の文字列の数
----------	-------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

[parseconfig.h](#) の 105 行目に定義があります。

9.73.2.2 getString() [2/3]

```
bool getString (
    const picojson::object & object,
    const std::string & key,
    std::string & string) [inline]
```

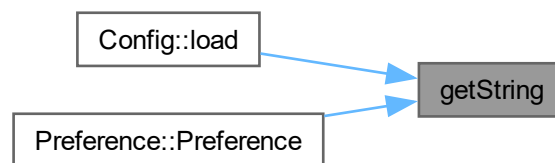
構成ファイルの JSON オブジェクトから文字列を取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>string</i>	取得した文字列を格納する変数

[parseconfig.h](#) の 81 行目に定義があります。

被呼び出し関係図:



9.73.2.3 getString() [3/3]

```

bool getString (
    const picojson::object & object,
    const std::string & key,
    std::vector< std::string > & strings) [inline]

```

構成ファイルの JSON オブジェクトから文字列のベクトルを取得する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>strings</i>	取得した文字列の配列を格納する変数

[parseconfig.h](#) の 138 行目に定義があります。

9.73.2.4 getValue() [1/2]

```

template<typename T, std::size_t U>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    std::array< T, U > & vector)

```

構成ファイルの JSON オブジェクトから数値の配列を取得する

テンプレート引数

<i>T</i>	構成ファイルから取得する配列の要素の数値のデータ型
<i>U</i>	構成ファイルから取得する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>vector</i>	取得した数値の配列を格納する変数

[parseconfig.h](#) の 48 行目に定義があります。

9.73.2.5 getValue() [2/2]

```
template<typename T>
bool getValue (
    const picojson::object & object,
    const std::string & key,
    T & scalar)
```

構成ファイルの JSON オブジェクトから数値を取得する

テンプレート引数

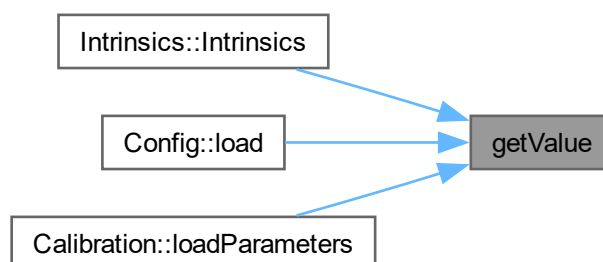
<i>T</i>	構成ファイルから取得する数値のデータ型
----------	---------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	取得する JSON オブジェクトのキー
<i>scalar</i>	取得した JSON オブジェクトの数値を格納する変数

[parseconfig.h](#) の 23 行目に定義があります。

被呼び出し関係図:



9.73.2.6 setString() [1/3]

```
template<std::size_t U>
void setString (
    picojson::object & object,
    const std::string & key,
    const std::array< std::string, U > & strings)
```

構成ファイルの JSON オブジェクトに文字列の配列を設定する

テンプレート引数

<i>U</i>	構成ファイルに設定する配列の要素の文字列の数
----------	------------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 225 行目に定義があります。

9.73.2.7 setString() [2/3]

```
void setString (
    picojson::object & object,
    const std::string & key,
    const std::string & string) [inline]
```

構成ファイルの JSON オブジェクトに文字列を設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>string</i>	設定する文字列

[parseconfig.h](#) の 210 行目に定義があります。

被呼び出し関係図:



9.73.2.8 setString() [3/3]

```
void setString (
    picojson::object & object,
    const std::string & key,
    const std::vector< std::string > & strings) [inline]
```

構成ファイルの JSON オブジェクトに文字列のベクトルを設定する

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>strings</i>	設定する文字列の配列

[parseconfig.h](#) の 250 行目に定義があります。

9.73.2.9 setValue() [1/2]

```
template<typename T, std::size_t U>
void setValue (
    picojson::object & object,
    const std::string & key,
    const std::array< T, U > & vector)
```

構成ファイルの JSON オブジェクトに数値の配列を設定する

テンプレート引数

<i>T</i>	構成ファイルに設定する配列の要素の数値のデータ型
<i>U</i>	構成ファイルに設定する配列の要素の数値の数

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>vector</i>	設定する数値の配列

[parseconfig.h](#) の 185 行目に定義があります。

9.73.2.10 setValue() [2/2]

```
template<typename T>
void setValue (
    picojson::object & object,
    const std::string & key,
    const T & scalar)
```

構成ファイルの JSON オブジェクトに数値を設定する

テンプレート引数

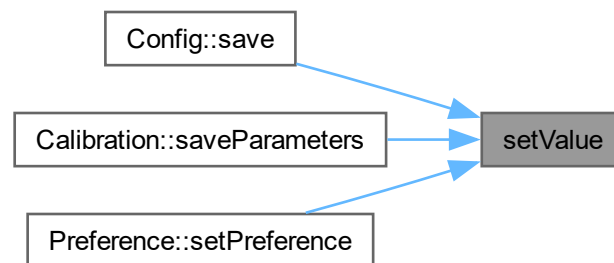
<i>T</i>	構成ファイルに設定する数値のデータ型
----------	--------------------

引数

<i>object</i>	構成ファイルの JSON オブジェクト
<i>key</i>	設定する JSON オブジェクトのキー
<i>scalar</i>	設定する数値

`parseconfig.h` の 169 行目に定義があります。

被呼び出し関係図:



9.74 parseconfig.h

[詳解]

```

00001 #pragma once
00002
00010
00011 // JSON
00012 #include "picojson.h"
00013
00022 template <typename T>
00023 bool getValue(const picojson::object& object,
00024             const std::string& key, T& scalar)
00025 {
00026     // key に一致するオブジェクトを探す
00027     const auto&& value{ object.find(key) };
00028
00029     // オブジェクトが無いか数値でなかったら戻る
00030     if (value == object.end() || !value->second.is<double>()) return false;
00031
00032     // 数値として格納する
00033     scalar = static_cast<T>(value->second.get<double>());
00034
00035     return true;
00036 }
00037
00047 template <typename T, std::size_t U>
00048 bool getValue(const picojson::object& object,
00049             const std::string& key, std::array<T, U>& vector)
00050 {
00051     // key に一致するオブジェクトを探す
00052     const auto&& value{ object.find(key) };
00053

```

```

00054 // オブジェクトが無いか配列でなかったら戻る
00055 if (value == object.end() || !value->second.is<picojson::array>()) return false;
00056
00057 // 配列を取り出す
00058 const auto& array{ value->second.get<picojson::array>() };
00059
00060 // 配列の要素数とデータの格納先の要素数の少ない方の数
00061 const auto n{ std::min(U, array.size()) };
00062
00063 // 配列の要素について
00064 for (std::size_t i = 0; i < n; ++i)
00065 {
00066     // 要素が数値なら格納する
00067     if (array[i].is<double>()) vector[i] = static_cast<T>(array[i].get<double>());
00068 }
00069
00070 return true;
00071 }
00072
00080 inline
00081 bool getString(const picojson::object& object,
00082               const std::string& key, std::string& string)
00083 {
00084     // key に一致するオブジェクトを探す
00085     const auto&& value{ object.find(key) };
00086
00087     // オブジェクトが無いか文字列でなかったら戻る
00088     if (value == object.end() || !value->second.is<std::string>()) return false;
00089
00090     // 文字列として格納する
00091     string = value->second.get<std::string>();
00092
00093     return true;
00094 }
00095
00104 template <std::size_t U>
00105 bool getString(const picojson::object& object,
00106               const std::string& key, std::array<std::string, U>& strings)
00107 {
00108     // key に一致するオブジェクトを探す
00109     const auto&& value{ object.find(key) };
00110
00111     // オブジェクトが無いか配列でなかったら戻る
00112     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00113
00114     // 配列を取り出す
00115     const auto& array{ value->second.get<picojson::array>() };
00116
00117     // 配列の要素数とデータの格納先の要素数の少ない方の数
00118     const auto n{ std::min(U, array.size()) };
00119
00120     // 配列の要素について
00121     for (std::size_t i = 0; i < n; ++i)
00122     {
00123         // 要素が文字列なら文字列として格納する
00124         strings[i] = array[i].is<std::string>() ? array[i].get<std::string>() : "";
00125     }
00126
00127     return true;
00128 }
00129
00137 inline
00138 bool getString(const picojson::object& object,
00139               const std::string& key, std::vector<std::string>& strings)
00140 {
00141     // key に一致するオブジェクトを探す
00142     const auto&& value{ object.find(key) };
00143
00144     // オブジェクトが無いか配列でなかったら戻る
00145     if (value == object.end() || !value->second.is<picojson::array>()) return false;
00146
00147     // 配列を取り出す
00148     const auto& array{ value->second.get<picojson::array>() };
00149
00150     // 配列のすべての要素について
00151     for (auto& element : array)
00152     {
00153         // 要素が文字列なら文字列として格納する
00154         strings.emplace_back(element.is<std::string>() ? element.get<std::string>() : "");
00155     }
00156
00157     return true;
00158 }
00159
00168 template <typename T>
00169 void setValue(picojson::object& object,
00170              const std::string& key, const T& scalar)

```

```

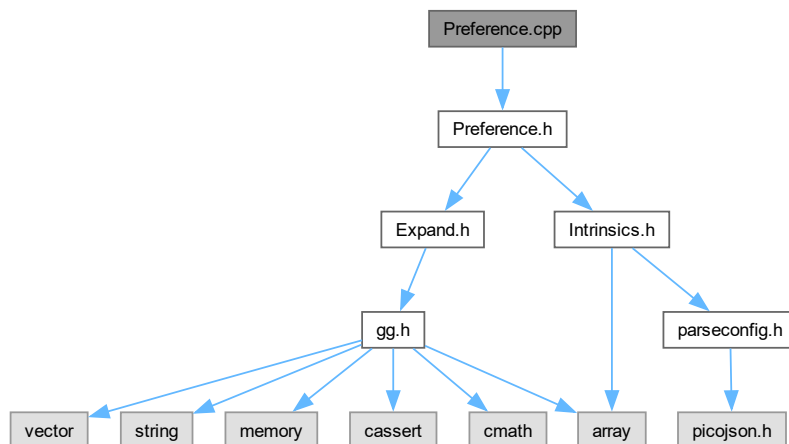
00171 {
00172     object.emplace(key, picojson::value(static_cast<double>(scalar)));
00173 }
00174
00184 template <typename T, std::size_t U>
00185 void setValue(picojson::object& object,
00186     const std::string& key, const std::array<T, U>& vector)
00187 {
00188     // picojson の配列
00189     picojson::array array;
00190
00191     // 配列のすべての要素について
00192     for (const auto& element : vector)
00193     {
00194         // 要素を picojson::array に追加する
00195         array.emplace_back(picojson::value(static_cast<double>(element)));
00196     }
00197
00198     // オブジェクトに追加する
00199     object.emplace(key, array);
00200 }
00201
00209 inline
00210 void setString(picojson::object& object,
00211     const std::string& key, const std::string& string)
00212 {
00213     object.emplace(key, picojson::value(string));
00214 }
00215
00224 template <std::size_t U>
00225 void setString(picojson::object& object,
00226     const std::string& key, const std::array<std::string, U>& strings)
00227 {
00228     // picojson の配列
00229     picojson::array array;
00230
00231     // 配列のすべての要素について
00232     for (const auto& string : strings)
00233     {
00234         // 要素を picojson::array に追加する
00235         array.emplace_back(picojson::value(string));
00236     }
00237
00238     // オブジェクトに追加する
00239     object.emplace(key, array);
00240 }
00241
00249 inline
00250 void setString(picojson::object& object,
00251     const std::string& key, const std::vector<std::string>& strings)
00252 {
00253     // picojson の配列
00254     picojson::array array;
00255
00256     // 配列のすべての要素について
00257     for (auto& string : strings)
00258     {
00259         // 要素を picojson::array に追加する
00260         array.emplace_back(picojson::value(string));
00261     }
00262
00263     // オブジェクトに追加する
00264     object.emplace(key, array);
00265 }

```

9.75 Preference.cpp ファイル

```
#include "Preference.h"
```

Preference.cpp の依存先関係図:



9.75.1 詳解

キャプチャデバイスの構成データクラスの実装

著者

Kohe Tokoi

日付

January 23, 2025

[Preference.cpp](#) に定義があります。

9.76 Preference.cpp

[詳解]

```
00001
00008 #include "Preference.h"
00009
00010 //
00011 // キャプチャデバイスの構成データのデフォルトコンストラクタ
00012 //
00013 Preference::Preference()
00014 : Preference{ "Default", "orthographic.vert", "normal.frag" }
00015 {
00016 }
00017
00018 //
```

```

00019 // シェーダのソースファイルを指定するときに使う
00020 // キャプチャデバイスの構成データのコンストラクタ
00021 //
00022 Preference::Preference(const std::string& description,
00023     const std::string& vert, const std::string& frag,
00024     const Intrinsic& intrinsics)
00025 : description{ description }
00026 , source{ vert, frag }
00027 , intrinsics{ intrinsics }
00028 {
00029 }
00030
00031 //
00032 // 構成ファイルを読み込むときに使う
00033 // キャプチャデバイスの構成データのコンストラクタ
00034 //
00035 Preference::Preference(const picojson::object& object)
00036 : intrinsics{ object }
00037 {
00038     // 説明の文字列
00039     getString(object, "description", description);
00040
00041     // 展開用シェーダのファイル名
00042     getString(object, "shader", source);
00043 }
00044
00045 //
00046 // 構成データのデストラクタ
00047 //
00048 Preference::~Preference()
00049 {
00050     // shaderList は全 Preference で共有するため、個々の破棄時には消去しない
00051     // 静的オブジェクトとしてプログラム終了時にまとめて破棄される
00052 }
00053
00054 //
00055 // シェーダをビルドする
00056 //
00057 void Preference::buildShader()
00058 {
00059     // ソースファイル名をつないでシェーダを検索するキーを作る
00060     const auto key{ source[0] + "\t" + source[1] };
00061
00062     // キーがシェーダリストに無ければシェーダを構築して追加する
00063     shader = &shaderList.try_emplace(key, source[0], source[1]).first->second;
00064 }
00065
00066 //
00067 // 構成を JSON オブジェクトに格納する
00068 //
00069 void Preference::setPreference(picojson::object& object) const
00070 {
00071     // 説明の文字列
00072     setString(object, "description", description);
00073
00074     // キャプチャデバイスのレンズの縦横の画角
00075     setValue(object, "fov", intrinsics.fov);
00076
00077     // キャプチャデバイスのレンズの中心（主点）位置
00078     setValue(object, "center", intrinsics.center);
00079
00080     // キャプチャデバイスの解像度
00081     setValue(object, "resolution", intrinsics.size);
00082
00083     // キャプチャデバイスのフレームレート
00084     setValue(object, "fps", intrinsics.fps);
00085
00086     // 展開用シェーダのファイル名
00087     setString(object, "shader", source);
00088 }
00089
00090 // すべての構成のシェーダーのリスト
00091 std::map<std::string, Expand> Preference::shaderList;

```

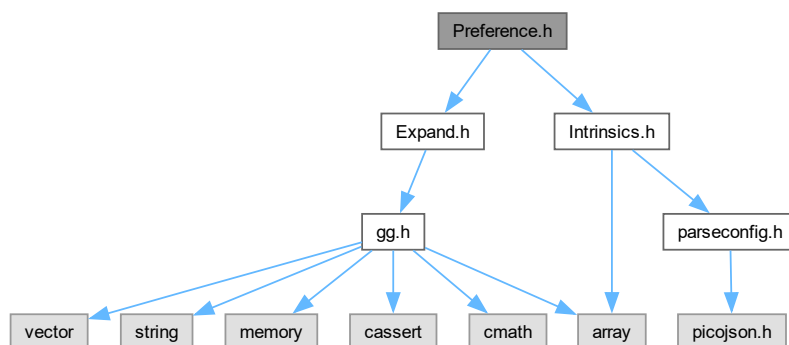
9.77 Preference.h ファイル

```

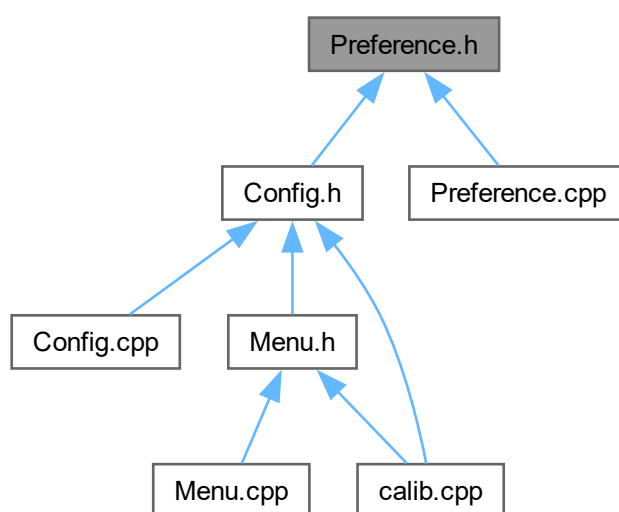
#include "Expand.h"
#include "Intrinsic.h"

```

Preference.h の依存先関係図:



被依存関係図:



クラス

- class [Preference](#)

9.77.1 詳解

キャプチャデバイスの構成データクラスの定義

著者

Kohe Tokoi

日付

January 23, 2025

[Preference.h](#) に定義があります。

9.78 Preference.h

[詳解]

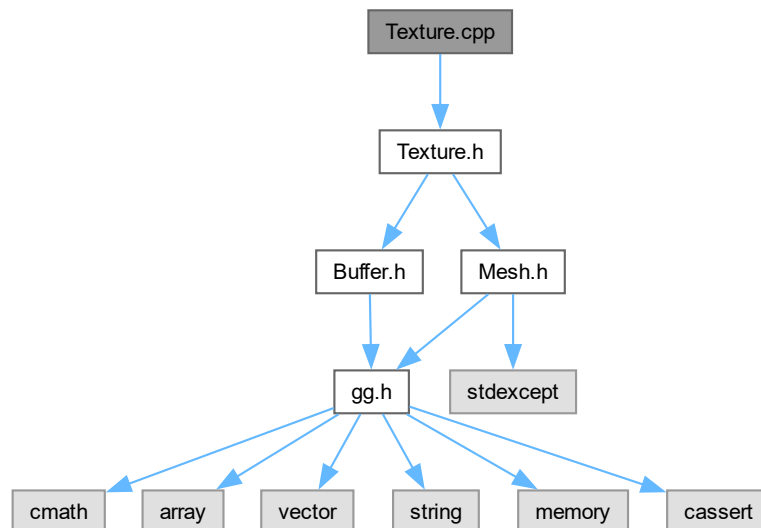
```
00001 #pragma once
00002
00010
00011 // 平面展開用のシェーダ
00012 #include "Expand.h"
00013
00014 // キャプチャデバイス固有のパラメータ
00015 #include "Intrinsics.h"
00016
00024 class Preference
00025 {
00027     std::string description;
00028
00030     std::array<std::string, 2> source;
00031
00033     const Intrinsics intrinsics;
00034
00036     const Expand* shader{ nullptr };
00037
00039     static std::map<std::string, Expand> shaderList;
00040
00041 public:
00042     Preference();
00048
00049     Preference(const std::string& description,
00059         const std::string& vert, const std::string& frag,
00060         const Intrinsics& intrinsics = Intrinsics{});
00061
00062     Preference(const picojson::object& object);
00067
00068     virtual ~Preference();
00072
00073     void buildShader();
00077
00078     const auto& getDescription() const
00084     {
00085         return description;
00086     }
00087
00088     const auto& getIntrinsics() const
00094     {
00095         return intrinsics;
00096     }
00097
00098     const auto& getShader() const
00104     {
00105         return *shader;
00106     }
00107
00108     void setPreference(picojson::object& object) const;
00114 };
00115
```


9.79 README.md ファイル

9.80 Texture.cpp ファイル

```
#include "Texture.h"
```

Texture.cpp の依存先関係図:



9.80.1 詳解

テクスチャクラスの実装

著者

Kohe Tokoi

日付

February 20, 2024

[Texture.cpp](#) に定義があります。

9.81 Texture.cpp

[詳解]

```

00001
00008 #include "Texture.h"
00009
00010 //
00011 // テクスチャを作成するコンストラクタ
00012 //
00013 Texture::Texture(GLsizei width, GLsizei height, int channels)
00014 {
00015     // テクスチャを作る
00016     Texture::create(width, height, channels);
00017 }
00018
00019 //
00020 // コピーコンストラクタ
00021 //
00022 Texture::Texture(const Texture& texture)
00023 {
00024     // テクスチャをコピーして作成する
00025     Texture::copy(texture);
00026 }
00027
00028 //
00029 // デストラクタ
00030 //
00031 Texture::~Texture()
00032 {
00033     // テクスチャを破棄する
00034     Texture::discard();
00035 }
00036
00037 //
00038 // 代入演算子
00039 //
00040 Texture& Texture::operator=(const Texture& texture)
00041 {
00042     // 代入元と代入先が同じなら何もしない
00043     if (&texture == this) return *this;
00044
00045     // 引数のテクスチャをこのテクスチャにコピーする
00046     Texture::copy(texture);
00047
00048     // このテクスチャを返す
00049     return *this;
00050 }
00051
00052 //
00053 // ムーブ代入演算子
00054 //
00055 Texture& Texture::operator=(Texture&& texture) noexcept
00056 {
00057     // ムーブ代入元とムーブ代入先が同じなら何もしない
00058     if (&texture == this) return *this;
00059
00060     // ムーブ代入元の基底クラスをムーブする
00061     static_cast<Buffer&>(*this) = std::move(texture);
00062
00063     // ムーブ代入元のテクスチャのメンバをムーブする
00064     textureSize = texture.textureSize;
00065     texture.textureSize = { 0, 0 };
00066     textureChannels = texture.textureChannels;
00067     texture.textureChannels = 0;
00068
00069     // ムーブ代入先のテクスチャを削除する
00070     glDeleteTextures(1, &textureName);
00071
00072     // ムーブ代入元のテクスチャ名をムーブ代入先に移す
00073     textureName = texture.textureName;
00074
00075     // ムーブ代入元のデストラクタでテクスチャが削除されないよう 0 にする
00076     texture.textureName = 0;
00077
00078     // このテクスチャを返す
00079     return *this;
00080 }
00081
00082 //
00083 // テクスチャを作成する
00084 //
00085 void Texture::create(GLsizei width, GLsizei height, int channels)
00086 {
00087     // 既存のバッファを破棄して新しいバッファを作成する
00088     Buffer::create(width, height, channels);

```

```

00089
00090 // まだメッシュの頂点配列オブジェクトが作られていなければ作る
00091 if (!mesh) mesh = std::make_shared<Mesh>();
00092
00093 // 指定したサイズが保持しているテクスチャのサイズと同じなら何もしない
00094 if (width == textureSize[0] && height == textureSize[1]
00095     && channels == textureChannels) return;
00096
00097 // テクスチャのサイズとチャンネル数を記録する
00098 textureSize = std::array<int, 2>{ width, height };
00099 textureChannels = channels;
00100
00101 // 以前のテクスチャを削除する
00102 glDeleteTextures(1, &textureName);
00103
00104 // テクスチャを作成する
00105 glGenTextures(1, &textureName);
00106
00107 // テクスチャのメモリを確保してデータを転送する
00108 glBindTexture(GL_TEXTURE_2D, textureName);
00109 glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, width, height, 0,
00110             channelsToFormat(channels), GL_UNSIGNED_BYTE, nullptr);
00111 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
00112 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
00113 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
00114 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
00115 glBindTexture(GL_TEXTURE_2D, 0);
00116 }
00117
00118 //
00119 // テクスチャをコピーする
00120 //
00121 void Texture::copy(const Buffer& texture) noexcept
00122 {
00123     // コピー元と同じサイズの空のテクスチャを作る
00124     Texture::create(texture.getWidth(), texture.getHeight(),
00125                     texture.getChannels());
00126
00127     // 作成したテクスチャのバッファにコピーする
00128     copyBuffer(texture);
00129 }
00130
00131 //
00132 // テクスチャを破棄する
00133 //
00134 void Texture::discard()
00135 {
00136     // デフォルトのテクスチャに戻す
00137     glBindTexture(GL_TEXTURE_2D, 0);
00138
00139     // テクスチャを削除する
00140     glDeleteTextures(1, &textureName);
00141     textureName = 0;
00142
00143     // テクスチャのサイズを 0 にする
00144     textureSize = std::array<int, 2>{ 0, 0 };
00145     textureChannels = 0;
00146 }
00147
00148 //
00149 // このテクスチャをマッピングしてメッシュを描画する
00150 //
00151 void Texture::draw(GLsizei width, GLsizei height, int unit) const
00152 {
00153     // テクスチャの縦横比
00154     const auto t{ static_cast<float>(textureSize[0] * height)};
00155
00156     // 表示領域の縦横比
00157     const auto d{ static_cast<float>(textureSize[1] * width)};
00158
00159     // テクスチャのスケール
00160     const std::array<GLfloat, 2> scale{ t > d // テクスチャの縦横比の方が
00161         ? std::array<GLfloat, 2>{ 1.0f, t / d } // 大きければ横方向いっぱい描く
00162         : std::array<GLfloat, 2>{ d / t, 1.0f } // それ以外は縦方向いっぱい描く
00163     };
00164
00165     // このオブジェクトのテクスチャを指定する
00166     glBindTexture(unit);
00167
00168     // メッシュを描画する
00169     mesh->draw(scale, unit);
00170
00171     // テクスチャの指定を解除する
00172     glBindTexture(0);
00173 }
00174
00175 //

```

```

00176 // テクスチャからピクセルバッファオブジェクトにデータをコピーする
00177 //
00178 void Texture::readPixels(
00179 #if defined(USE_PIXEL_BUFFER_OBJECT)
00180     GLuint buffer) const
00181 #else
00182     std::vector<GLubyte>& buffer)
00183 #endif
00184 {
00185     // 読み出し元のテクスチャを結合する
00186     glBindTexture(GL_TEXTURE_2D, textureName);
00187
00188 #if defined(USE_PIXEL_BUFFER_OBJECT)
00189     // 書き込み先のピクセルバッファオブジェクトを指定する
00190     glBindBuffer(GL_PIXEL_PACK_BUFFER, buffer);
00191
00192 #   if defined(GL_GLES_PROTOTYPES)
00193     // 現在使っているフレームバッファオブジェクトを調べる
00194     GLuint currentFbo;
00195     glGetIntegerv(GL_FRAMEBUFFER_BINDING, reinterpret_cast<GLint*>(&currentFbo));
00196
00197     // テクスチャをカラーバッファに持ってフレームバッファオブジェクトを作る
00198     GLuint fbo;
00199     glGenFramebuffers(1, &fbo);
00200     glBindFramebuffer(GL_FRAMEBUFFER, fbo);
00201     static const GLenum attachment{ GL_COLOR_ATTACHMENT0 };
00202     glFramebufferTexture2D(GL_FRAMEBUFFER, attachment, GL_TEXTURE_2D, textureName, 0);
00203     glDrawBuffers(1, &attachment);
00204     glReadBuffer(attachment);
00205
00206     // フレームバッファオブジェクトからピクセルバッファオブジェクトに読み出す
00207     glReadPixels(0, 0, textureSize[0], textureSize[1], channelsToFormat(textureChannels),
00208                 GL_UNSIGNED_BYTE, 0);
00209
00210     // 元のフレームバッファオブジェクトに戻す
00211     glBindFramebuffer(GL_FRAMEBUFFER, currentFbo);
00212
00213     // 作ったフレームバッファオブジェクトは削除する
00214     glDeleteFramebuffers(1, &fbo);
00215 #   else
00216     // テクスチャの内容をピクセルバッファオブジェクトに書き込む
00217     glGetTexImage(GL_TEXTURE_2D, 0, getFormat(), GL_UNSIGNED_BYTE, 0);
00218 #   endif
00219
00220     // 書き込み先のピクセルバッファオブジェクトの結合を解除する
00221     glBindBuffer(GL_PIXEL_PACK_BUFFER, 0);
00222 #else
00223     // テクスチャの内容をピクセルバッファオブジェクトに書き込む
00224     glGetTexImage(GL_TEXTURE_2D, 0, getFormat(), GL_UNSIGNED_BYTE, buffer.data());
00225 #endif
00226
00227 // 読み出し元のテクスチャの結合を解除する
00228 glBindTexture(GL_TEXTURE_2D, 0);
00229 }
00230
00231 //
00232 // ピクセルバッファオブジェクトからテクスチャにデータをコピーする
00233 //
00234 void Texture::drawPixels(
00235 #if defined(USE_PIXEL_BUFFER_OBJECT)
00236     GLuint
00237 #else
00238     const std::vector<GLubyte>&
00239 #endif
00240     buffer) const
00241 {
00242     // 書き込み先のテクスチャを結合する
00243     glBindTexture(GL_TEXTURE_2D, textureName);
00244
00245 #if defined(USE_PIXEL_BUFFER_OBJECT)
00246     // 読み出し元のピクセルバッファオブジェクトを指定する
00247     glBindBuffer(GL_PIXEL_UNPACK_BUFFER, buffer);
00248
00249     // ピクセルバッファオブジェクトの内容をテクスチャに書き込む
00250     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, textureSize[0], textureSize[1],
00251                     getFormat(), GL_UNSIGNED_BYTE, 0);
00252
00253     // 読み出し元のピクセルバッファオブジェクトの結合を解除する
00254     glBindBuffer(GL_PIXEL_UNPACK_BUFFER, 0);
00255 #else
00256     // ピクセルバッファオブジェクトの内容をテクスチャに書き込む
00257

```

```

00263     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, textureSize[0], textureSize[1],
00264         getFormat(), GL_UNSIGNED_BYTE, buffer.data());
00265
00266 #endif
00267
00268 // 書き込み先のテクスチャの結合を解除する
00269 glBindTexture(GL_TEXTURE_2D, 0);
00270 }
00271
00272 //
00273 // CPU メモリ上の画像をテクスチャへ転送する
00274 //
00275 void Texture::drawPixels(GLsizei width, GLsizei height, int channels, const void* pixels)
00276 {
00277     // 空画像や不正なサイズでは OpenGL を呼ばず、現在のテクスチャを維持する。
00278     if (!pixels || width <= 0 || height <= 0 || channels <= 0) return;
00279
00280     // OpenCV 補正後の画像サイズに合わせる。サイズが同じなら再確保は行われない。
00281     create(width, height, channels);
00282
00283     // cv::Mat の連続したCPU画素を、現在のテクスチャへ直接アップロードする。
00284     glBindTexture(GL_TEXTURE_2D, textureName);
00285
00286     // 3チャンネル画像など、1行のバイト数が4の倍数でない場合にも対応する。
00287     glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
00288     glTexSubImage2D(GL_TEXTURE_2D, 0, 0, 0, width, height,
00289         channelsToFormat(channels), GL_UNSIGNED_BYTE, pixels);
00290
00291     // 後続描画へ意図しないテクスチャ結合状態を残さない。
00292     glBindTexture(GL_TEXTURE_2D, 0);
00293 }
00294
00295 // 展開に用いるメッシュの頂点配列オブジェクト
00296 std::shared_ptr<Mesh> Texture::mesh;

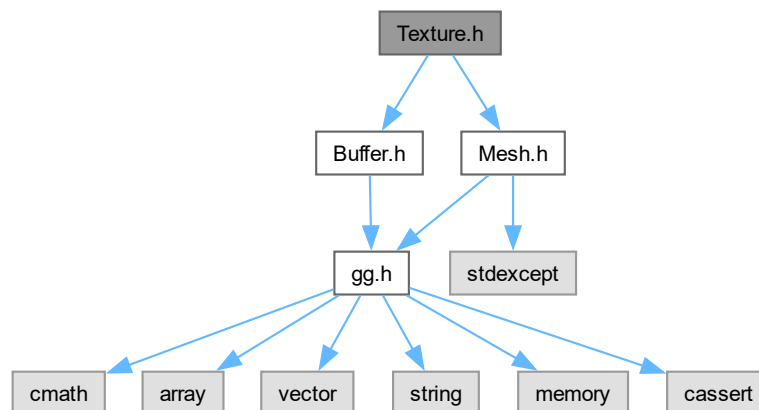
```

9.82 Texture.h ファイル

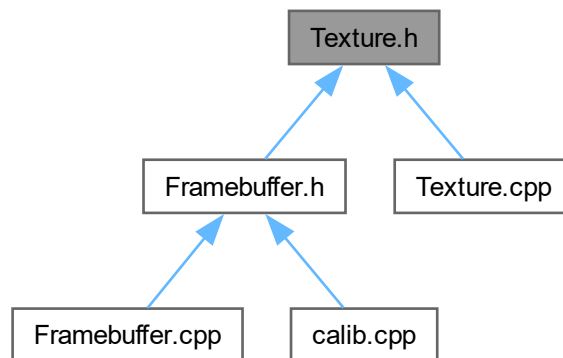
```
#include "Buffer.h"
```

```
#include "Mesh.h"
```

Texture.h の依存先関係図:



被依存関係図:



クラス

- class [Texture](#)

9.82.1 詳解

テクスチャクラスの定義

著者

Kohe Tokoi

日付

November 15, 2022

[Texture.h](#) に定義があります。

9.83 Texture.h

[詳解]

```
00001 #pragma once
00002
00010
00011 // バッファクラス
00012 #include "Buffer.h"
00013
00014 // テクスチャの展開に用いるメッシュ
00015 #include "Mesh.h"
00016
00020 class Texture : public Buffer
00021 {
00023     std::array<int, 2> textureSize{ 0, 0 };
```

```

00024
00026     int textureChannels{ 0 };
00027
00029     GLuint textureName{ 0 };
00030
00031 protected:
00032
00034     static std::shared_ptr<Mesh> mesh;
00035
00036 public:
00037
00041     Texture() = default;
00042
00050     Texture(GLsizei width, GLsizei height, int channels);
00051
00057     Texture(const Texture& texture);
00058
00064     Texture(Texture&& texture) noexcept
00065         : Buffer{ std::move(texture) }
00066         , textureSize{ texture.textureSize }
00067         , textureChannels{ texture.textureChannels }
00068         , textureName{ texture.textureName }
00069     {
00070         texture.textureSize = { 0, 0 };
00071         texture.textureChannels = 0;
00072
00073         // ムーブ元のデストラクタでテクスチャが削除されないよう 0 にする
00074         texture.textureName = 0;
00075     }
00076
00080     virtual ~Texture();
00081
00088     Texture& operator=(const Texture& texture);
00089
00096     Texture& operator=(Texture&& texture) noexcept;
00097
00109     virtual void create(GLsizei width, GLsizei height, int channels);
00110
00123     virtual void copy(const Buffer& texture) noexcept;
00124
00128     virtual void discard();
00129
00135     auto getTextureName() const
00136     {
00137         return textureName;
00138     }
00139
00143     virtual const std::array<GLsizei, 2>& getSize() const
00144     {
00145         return textureSize;
00146     }
00147
00151     virtual int getChannels() const
00152     {
00153         return textureChannels;
00154     }
00155
00165     void bindTexture(int unit = 0) const
00166     {
00167         // テクスチャユニットを指定する
00168         glActiveTexture(GL_TEXTURE0 + unit);
00169
00170         // テクスチャを指定する
00171         glBindTexture(GL_TEXTURE_2D, textureName);
00172     }
00173
00177     void unbindTexture() const
00178     {
00179         // デフォルトのテクスチャユニットに戻す
00180         glActiveTexture(GL_TEXTURE0);
00181
00182         // デフォルトのテクスチャに戻す
00183         glBindTexture(GL_TEXTURE_2D, 0);
00184     }
00185
00198     void draw(GLsizei width, GLsizei height, int unit = 0) const;
00199
00208     void readPixels(
00209 #if defined(USE_PIXEL_BUFFER_OBJECT)
00210         GLuint buffer) const
00211 #else
00212         std::vector<GLubyte>& buffer)
00213 #endif
00214     ;
00215
00224     void readPixels(Buffer& buffer)
00225 #if defined(USE_PIXEL_BUFFER_OBJECT)

```

```
00226     const
00227 #endif
00228 {
00229     // このテクスチャのバッファから引数のオブジェクトのバッファにコピーする
00230     readPixels();
00231
00232     // 引数のオブジェクトのサイズをこのテクスチャに合わせてコピーする
00233     buffer.Buffer::copy(*this);
00234 }
00235
00239 void readPixels()
00240 #if defined(USE_PIXEL_BUFFER_OBJECT)
00241     const
00242 #endif
00243 {
00244     // このテクスチャからこのバッファにコピーする
00245     readPixels(getBufferName());
00246 }
00247
00256 void drawPixels(
00257 #if defined(USE_PIXEL_BUFFER_OBJECT)
00258     GLuint
00259 #else
00260     const std::vector<GLubyte>&
00261 #endif
00262     buffer) const;
00263
00272 void drawPixels(const Texture& texture)
00273 {
00274     // このテクスチャのサイズを引数のオブジェクトのサイズに合わせてコピーする
00275     copy(texture);
00276
00277     // バッファからこのテクスチャにコピーする
00278     drawPixels();
00279 }
00280
00289 void drawPixels(const Buffer& buffer)
00290 {
00291     // このテクスチャのサイズを引数のオブジェクトのサイズに合わせてコピーする
00292     copy(buffer);
00293
00294     // バッファからこのテクスチャにコピーする
00295     drawPixels();
00296 }
00297
00301 void drawPixels()
00302 #if defined(USE_PIXEL_BUFFER_OBJECT)
00303     const
00304 #endif
00305 {
00306     // このバッファからこのテクスチャにコピーする
00307     drawPixels(getBufferName());
00308 }
00309
00320 void drawPixels(GLsizei width, GLsizei height, int channels, const void* pixels);
00321 };
```


Index

- `~ggError`
 - `gg`, [22](#)
- `~ggFBOError`
 - `gg`, [22](#)
- `~Buffer`
 - `Buffer`, [95](#)
- `~Calibration`
 - `Calibration`, [112](#)
- `~CamCv`
 - `CamCv`, [125](#)
- `~CamImage`
 - `CamImage`, [148](#)
- `~CamMf`
 - `CamMf`, [154](#)
- `~Camera`
 - `Camera`, [134](#)
- `~Capture`
 - `Capture`, [160](#)
- `~Compute`
 - `Compute`, [169](#)
- `~Config`
 - `Config`, [171](#)
- `~Expand`
 - `Expand`, [180](#)
- `~Framebuffer`
 - `Framebuffer`, [187](#)
- `~GgApp`
 - `GgApp`, [198](#)
- `~GgBuffer`
 - `gg::GgBuffer< T >`, [203](#)
- `~GgColorTexture`
 - `gg::GgColorTexture`, [212](#)
- `~GgElements`
 - `gg::GgElements`, [219](#)
- `~GgMatrix`
 - `gg::GgMatrix`, [229](#)
- `~GgNormalTexture`
 - `gg::GgNormalTexture`, [286](#)
- `~GgPointShader`
 - `gg::GgPointShader`, [299](#)
- `~GgPoints`
 - `gg::GgPoints`, [293](#)
- `~GgQuaternion`
 - `gg::GgQuaternion`, [318](#)
- `~GgShader`
 - `gg::GgShader`, [387](#)
- `~GgShape`
 - `gg::GgShape`, [390](#)
- `~GgSimpleObj`
 - `gg::GgSimpleObj`, [394](#)
- `~GgSimpleShader`
 - `gg::GgSimpleShader`, [400](#)
- `~GgTexture`
 - `gg::GgTexture`, [421](#)
- `~GgTrackball`
 - `gg::GgTrackball`, [430](#)
- `~GgTriangles`
 - `gg::GgTriangles`, [440](#)
- `~GgUniformBuffer`
 - `gg::GgUniformBuffer< T >`, [446](#)
- `~GgVector`
 - `gg::GgVector`, [461](#)
- `~GgVertexArray`
 - `gg::GgVertexArray`, [488](#)
- `~LightBuffer`
 - `gg::GgSimpleShader::LightBuffer`, [501](#)
- `~MaterialBuffer`
 - `gg::GgSimpleShader::MaterialBuffer`, [520](#)
- `~Menu`
 - `Menu`, [536](#)
- `~Mesh`
 - `Mesh`, [545](#)
- `~Preference`
 - `Preference`, [548](#)
- `~Texture`
 - `Texture`, [558](#)
- `~Window`
 - `GgApp::Window`, [577](#)
- `add`
 - `gg::GgQuaternion`, [318–320](#)
- `ambient`
 - `gg::GgSimpleShader::Light`, [497](#)
 - `gg::GgSimpleShader::Material`, [516](#)
- `autoCaptureBeep`
 - `Menu`, [542](#)
- `autoCaptureCooldown`
 - `Menu`, [542](#)
- `autoCaptureCooldownTimer`
 - `Menu`, [542](#)
- `autoCaptureEnabled`
 - `Menu`, [542](#)
- `autoCaptureMinStableTime`
 - `Menu`, [543](#)
- `autoCaptureStatusMessage`
 - `Menu`, [543](#)
- `begin`
 - `gg::GgTrackball`, [430](#)

- bind
 - gg::GgBuffer< T >, 204
 - gg::GgColorTexture, 213
 - gg::GgNormalTexture, 286
 - gg::GgTexture, 422
 - gg::GgUniformBuffer< T >, 446
 - gg::GgVertexArray, 488
- bindBuffer
 - Buffer, 95
- bindFramebuffer
 - Framebuffer, 187
- BindingPoints
 - gg, 22
- bindTexture
 - Texture, 559
- Buffer, 91
 - ~Buffer, 95
 - bindBuffer, 95
 - Buffer, 92–94
 - channelsToFormat, 96
 - copy, 96
 - copyBuffer, 97
 - create, 98
 - discard, 99
 - formatToChannels, 99
 - getAspect, 101
 - getBufferName, 101
 - getChannels, 102
 - getFormat, 102
 - getHeight, 103
 - getSize, 104
 - getWidth, 105
 - map, 105
 - operator=, 106
 - resize, 107, 108
 - unbindBuffer, 109
 - unmap, 109
- Buffer.cpp, 605
- Buffer.h, 608
 - USE_PIXEL_BUFFER_OBJECT, 609
- buildShader
 - Preference, 549
- calib, 3
- calib.cpp, 611
 - CONFIG_FILE, 612
- calibrate
 - Calibration, 112
- Calibration, 110
 - ~Calibration, 112
 - calibrate, 112
 - Calibration, 111
 - createBoard, 112
 - detectBoard, 113
 - detectMarkers, 113
 - dictionaryList, 123
 - discardCorners, 114
 - drawBoard, 114
 - finished, 115
 - getAllMarkerPoses, 115
 - getCameraMatrix, 116
 - getCornersCount, 116
 - getCurrentMotion, 116
 - getDistortionCoefficients, 117
 - getReprojectionError, 117
 - getSampleCount, 117
 - getStableProgress, 117
 - getTotalCount, 118
 - isDiverseEnough, 118
 - isStable, 118
 - loadParameters, 119
 - operator=, 119
 - recordCorners, 120
 - RvecTvecToPose, 120
 - saveParameters, 121
 - setDictionary, 121
 - updateMotion, 122
- Calibration.cpp, 615
 - USE_RODRIGUES, 616
- Calibration.h, 624
- CamAndroid.cpp, 628
- CamAndroid.h, 634
- CamAvf.h, 636
- CamAvf.mm, 637
- CamCv, 123
 - ~CamCv, 125
 - CamCv, 125
 - decreaseExposure, 126
 - decreaseGain, 126
 - getCodec, 127
 - getFps, 128
 - getPosition, 128
 - increaseExposure, 128
 - increaseGain, 129
 - onClose, 129
 - onStart, 129
 - onStop, 130
 - open, 130
 - setExposure, 131
 - setGain, 131
 - setPosition, 132
- CamCv.h, 647
- Camera, 133
 - ~Camera, 134
 - Camera, 134
 - captured, 143
 - channels, 143
 - close, 135
 - decreaseExposure, 135
 - decreaseGain, 135
 - getChannels, 136
 - getFormatList, 136
 - getFps, 136
 - getHeight, 136
 - getPrioritizeLatency, 137
 - getSize, 137
 - getWidth, 137

- height, 144
- image, 144
- increaseExposure, 137
- increaseGain, 138
- interval, 144
- isCaptured, 138
- isRunning, 138
- isStillImage, 138
- lockFrame, 139
- mtx, 144
- onClose, 140
- onStart, 140
- onStop, 141
- prioritizeLatency, 144
- running, 144
- selectFormat, 141
- setPrioritizeLatency, 142
- start, 142
- stop, 142
- thr, 145
- width, 145
- Camera.h, 652
- CamImage, 145
 - ~CamImage, 148
 - CamImage, 147
 - isOpened, 148
 - isStillImage, 148
 - load, 149
 - onClose, 149
 - onStart, 150
 - onStop, 150
 - open, 150
 - save, 151
- CamImage.h, 656
- CamLibcam.cpp, 660
- CamLibcam.h, 668
- CamMf, 152
 - ~CamMf, 154
 - CamMf, 154
 - capture, 155
 - getDeviceList, 155
 - getFormatList, 156
 - onClose, 156
 - onStart, 156
 - onStop, 157
 - open, 157
 - select, 158
 - selectFormat, 158
- CamMf.cpp, 670
 - SafeRelease, 671
 - SubTypeToName, 671
- CamMf.h, 687
- Capture, 159
 - ~Capture, 160
 - Capture, 160
 - close, 161
 - getFps, 161
 - getPrioritizeLatency, 161
 - getSize, 161
 - isImage, 162
 - isOpened, 162
 - openDevice, 162
 - openImage, 163
 - openMovie, 163
 - operator bool, 164
 - retrieve, 164, 165
 - setPrioritizeLatency, 165
 - start, 166
 - stop, 166
- capture
 - CamMf, 155
- Capture.cpp, 690
- Capture.h, 694
- captured
 - Camera, 143
- CaptureFormat, 166
 - CaptureFormat, 167
 - codec, 167
 - fps, 167
 - index, 168
 - resolution, 168
- center
 - Intrinsics, 494
- channels
 - Camera, 143
- channelsToFormat
 - Buffer, 96
- checkerLength
 - Settings, 551
- checkerSize
 - Settings, 551
- close
 - Camera, 135
 - Capture, 161
- codec
 - CaptureFormat, 167
- Compute, 168
 - ~Compute, 169
 - Compute, 168
 - execute, 169
 - get, 169
 - use, 170
- Compute.h, 697
- Config, 170
 - ~Config, 171
 - Config, 171
 - getBackground, 171
 - getCheckerLength, 171
 - getCheckerSize, 172
 - getDictionaryName, 172
 - getHeight, 173
 - getInitialImage, 173
 - getMarkerLength, 174
 - getMenuFont, 174
 - getMenuFontSize, 174
 - getPreferences, 175

- getSettings, 175
 - getTitle, 175
 - getWidth, 175
 - initialize, 176
 - load, 176
 - save, 177
 - setSettings, 178
- Config.cpp, 698
- Config.h, 703
- CONFIG_FILE
 - calib.cpp, 612
- conjugate
 - gg::GgQuaternion, 321
- copy
 - Buffer, 96
 - Framebuffer, 188
 - gg::GgBuffer< T >, 204
 - gg::GgUniformBuffer< T >, 446
 - Texture, 560
- copyBuffer
 - Buffer, 97
- create
 - Buffer, 98
 - Framebuffer, 189
 - Texture, 561
- createBoard
 - Calibration, 112
- decreaseExposure
 - CamCv, 126
 - Camera, 135
- decreaseGain
 - CamCv, 126
 - Camera, 135
- defaultCheckerLength
 - Settings, 551
- defaultCheckerSize
 - Settings, 552
- defaultEuler
 - Settings, 552
- defaultFocal
 - Settings, 552
- defaultFocalRange
 - Settings, 552
- defaultMarkerLength
 - Settings, 552
- detectBoard
 - Calibration, 113
 - Menu, 543
- detectMarker
 - Menu, 543
- detectMarkers
 - Calibration, 113
- dictionaryList
 - Calibration, 123
- dictionaryName
 - Settings, 552
- diffuse
 - gg::GgSimpleShader::Light, 497
 - gg::GgSimpleShader::Material, 516
- discard
 - Buffer, 99
 - Framebuffer, 190
 - Texture, 562
- discardCorners
 - Calibration, 114
- distance3
 - gg::GgVector, 461
- distance4
 - gg::GgVector, 461
- divide
 - gg::GgQuaternion, 321–323
- dot3
 - gg::GgVector, 462
- dot4
 - gg::GgVector, 463
- draw
 - gg::GgElements, 220
 - gg::GgPoints, 293
 - gg::GgShape, 390
 - gg::GgSimpleObj, 394
 - gg::GgTriangles, 440
 - Menu, 537
 - Mesh, 545
 - Texture, 563
- drawBoard
 - Calibration, 114
- drawMesh
 - Mesh, 545
- drawPixels
 - Texture, 564–566
- end
 - gg::GgTrackball, 430
- euler
 - gg::GgQuaternion, 324–326
 - Settings, 553
- execute
 - Compute, 169
- Expand, 178
 - ~Expand, 180
 - Expand, 179
 - getProgram, 180
 - operator=, 180
 - setup, 181
- Expand.cpp, 706
- Expand.h, 708
- fill
 - gg::GgUniformBuffer< T >, 447
- finished
 - Calibration, 115
- focal
 - Settings, 553
- focalRange
 - Settings, 553
- formatToChannels
 - Buffer, 99

- fov
 - Intrinsics, [494](#)
- fps
 - CaptureFormat, [167](#)
 - Intrinsics, [495](#)
- Framebuffer, [182](#)
 - ~Framebuffer, [187](#)
 - bindFramebuffer, [187](#)
 - copy, [188](#)
 - create, [189](#)
 - discard, [190](#)
 - Framebuffer, [185](#), [186](#)
 - getChannels, [190](#)
 - getFramebufferName, [190](#)
 - getSize, [191](#)
 - operator=, [191](#)
 - show, [193](#)
 - unbindFramebuffer, [193](#)
 - update, [194](#), [195](#)
- Framebuffer.cpp, [710](#)
- Framebuffer.h, [714](#)
- frustum
 - gg::GgMatrix, [229](#)
- get
 - Compute, [169](#)
 - gg::GgColorTexture, [213](#)
 - gg::GgMatrix, [230](#), [231](#)
 - gg::GgNormalTexture, [286](#)
 - gg::GgPointShader, [300](#)
 - gg::GgQuaternion, [327](#)
 - gg::GgShader, [387](#)
 - gg::GgShape, [391](#)
 - gg::GgSimpleObj, [394](#)
 - gg::GgTrackball, [431](#)
 - gg::GgVertexArray, [488](#)
 - GgApp::Window, [578](#)
- getAllMarkerPoses
 - Calibration, [115](#)
- getAltArrow
 - GgApp::Window, [578](#)
- getAltArrowX
 - GgApp::Window, [578](#)
- getAltArrowY
 - GgApp::Window, [579](#)
- getAnyList
 - Menu.cpp, [906](#)
- getArrow
 - GgApp::Window, [580](#)
- getArrowX
 - GgApp::Window, [581](#)
- getArrowY
 - GgApp::Window, [582](#)
- getAspect
 - Buffer, [101](#)
 - GgApp::Window, [583](#)
- getBackground
 - Config, [171](#)
- getBuffer
 - gg::GgBuffer< T >, [204](#)
 - gg::GgPoints, [294](#)
 - gg::GgTriangles, [440](#)
 - gg::GgUniformBuffer< T >, [448](#)
- getBufferName
 - Buffer, [101](#)
- getCameraMatrix
 - Calibration, [116](#)
- getChannels
 - Buffer, [102](#)
 - Camera, [136](#)
 - Framebuffer, [190](#)
 - Texture, [567](#)
- getCheckerLength
 - Config, [171](#)
 - Menu, [537](#)
- getCheckerSize
 - Config, [172](#)
 - Menu, [537](#)
- getCodec
 - CamCv, [127](#)
- getConjugateMatrix
 - gg::GgQuaternion, [327](#), [328](#)
- getControlArrow
 - GgApp::Window, [583](#)
- getControlArrowX
 - GgApp::Window, [583](#)
- getControlArrowY
 - GgApp::Window, [584](#)
- getCornersCount
 - Calibration, [116](#)
- getCount
 - gg::GgBuffer< T >, [205](#)
 - gg::GgPoints, [294](#)
 - gg::GgTriangles, [441](#)
 - gg::GgUniformBuffer< T >, [449](#)
- getCurrentMotion
 - Calibration, [116](#)
- getDescription
 - Preference, [549](#)
- getDeviceList
 - CamMf, [155](#)
- getDictionaryName
 - Config, [172](#)
- getDistortionCoefficients
 - Calibration, [117](#)
- getFboHeight
 - GgApp::Window, [585](#)
- getFboSize
 - GgApp::Window, [585](#), [586](#)
- getFboWidth
 - GgApp::Window, [586](#)
- getFocal
 - Settings, [551](#)
- getFormat
 - Buffer, [102](#)
- getFormatList
 - Camera, [136](#)

- CamMf, 156
- getFps
 - CamCv, 128
 - Camera, 136
 - Capture, 161
- getFramebufferName
 - Framebuffer, 190
- getHeight
 - Buffer, 103
 - Camera, 136
 - Config, 173
 - gg::GgColorTexture, 213
 - gg::GgNormalTexture, 286
 - gg::GgTexture, 422
 - GgApp::Window, 587
- getIndexBuffer
 - gg::GgElements, 220
- getIndexCount
 - gg::GgElements, 221
- getInitialImage
 - Config, 173
- getIntrinsics
 - Preference, 549
- getKey
 - GgApp::Window, 587
- getLastKey
 - GgApp::Window, 587
- getMarkerLength
 - Config, 174
 - Menu, 537
- getMatrix
 - gg::GgQuaternion, 329, 331
 - gg::GgTrackball, 431
- getMenubarHeight
 - Menu, 538
- getMenuFont
 - Config, 174
- getMenuFontSize
 - Config, 174
- getMode
 - gg::GgShape, 391
- getMouse
 - GgApp::Window, 588
- getMouseX
 - GgApp::Window, 589
- getMouseY
 - GgApp::Window, 589
- getPose
 - Menu, 538
- getPosition
 - CamCv, 128
- getPreferences
 - Config, 175
- getPrioritizeLatency
 - Camera, 137
 - Capture, 161
- getProgram
 - Expand, 180
- getQuaternion
 - gg::GgTrackball, 431
- getReprojectionError
 - Calibration, 117
- getRotation
 - GgApp::Window, 589
- getRotationMatrix
 - GgApp::Window, 589
- getSampleCount
 - Calibration, 117
- getScale
 - gg::GgTrackball, 432
- getScrollMatrix
 - GgApp::Window, 590
- getSettings
 - Config, 175
- getShader
 - Preference, 549
- getShiftArrow
 - GgApp::Window, 590
- getShiftArrowX
 - GgApp::Window, 590
- getShiftArrowY
 - GgApp::Window, 591
- getSize
 - Buffer, 104
 - Camera, 137
 - Capture, 161
 - Framebuffer, 191
 - gg::GgColorTexture, 213
 - gg::GgNormalTexture, 287
 - gg::GgTexture, 422
 - GgApp::Window, 592
 - Texture, 567
- getStableProgress
 - Calibration, 117
- getStart
 - gg::GgTrackball, 433
- getStride
 - gg::GgBuffer< T >, 205
 - gg::GgUniformBuffer< T >, 449
- getString
 - parseconfig.h, 935, 936
- getTarget
 - gg::GgBuffer< T >, 206
 - gg::GgUniformBuffer< T >, 450
- getTexture
 - gg::GgColorTexture, 214
 - gg::GgNormalTexture, 287
 - gg::GgTexture, 423
- getTextureName
 - Texture, 567
- getTitle
 - Config, 175
- getTotalCount
 - Calibration, 118
- getTranslation
 - GgApp::Window, 592

- getTranslationMatrix
 - GgApp::Window, 594
- getUsername
 - GgApp, 198
- getUserPointer
 - GgApp::Window, 594
- getValue
 - parseconfig.h, 936, 937
- getWheel
 - GgApp::Window, 594, 595
- getWheelX
 - GgApp::Window, 595
- getWheelY
 - GgApp::Window, 595
- getWidth
 - Buffer, 105
 - Camera, 137
 - Config, 175
 - gg::GgColorTexture, 214
 - gg::GgNormalTexture, 287
 - gg::GgTexture, 423
 - GgApp::Window, 595
- gg, 19
 - _ggError, 22
 - _ggFBOError, 22
 - BindingPoints, 22
 - ggArraysObj, 23
 - ggBufferAlignment, 89
 - ggConjugate, 23
 - ggCreateComputeShader, 24
 - ggCreateNormalMap, 24
 - ggCreateShader, 25
 - ggCross, 26, 28
 - ggDistance3, 28, 29
 - ggDistance4, 30, 31
 - ggDot3, 32
 - ggDot4, 33
 - ggElementsMesh, 34
 - ggElementsObj, 35
 - ggElementsSphere, 36
 - ggEllipse, 36
 - ggEulerQuaternion, 37, 39
 - ggFrustum, 40
 - ggIdentity, 40
 - ggIdentityQuaternion, 41
 - ggInit, 41
 - ggInvert, 42, 43
 - ggLength3, 43, 44
 - ggLength4, 45
 - ggLoadComputeShader, 46
 - ggLoadHeight, 47
 - ggLoadImage, 48
 - ggLoadShader, 48, 49
 - ggLoadSimpleObj, 50, 51
 - ggLoadTexture, 52
 - ggLookat, 53, 54
 - ggMatrixQuaternion, 55, 56
 - ggNorm, 56
 - ggNormal, 57
 - ggNormalize, 57
 - ggNormalize3, 58–60
 - ggNormalize4, 60–62
 - ggOrthogonal, 63
 - ggPerspective, 63
 - ggPointsCube, 64
 - ggPointsSphere, 65
 - ggQuaternionMatrix, 65
 - ggQuaternionTransposeMatrix, 66
 - ggReadImage, 67
 - ggRectangle, 68
 - ggRotate, 68–71
 - ggRotateQuaternion, 72, 73
 - ggRotateX, 74
 - ggRotateY, 74
 - ggRotateZ, 75
 - ggSaveColor, 75
 - ggSaveDepth, 77
 - ggSaveTga, 77
 - ggScale, 78–80
 - ggSlerp, 80–82
 - ggTranslate, 83, 84
 - ggTranspose, 86
 - LightBindingPoint, 22
 - MaterialBindingPoint, 22
 - operator+, 87
 - operator-, 88
 - operator/, 89
 - operator*, 87
- gg.cpp, 716
- gg.h, 794
 - ggError, 799
 - ggFBOError, 799
 - pathChar, 799
 - pathString, 799
 - TCharToUtf8, 799
 - Utf8ToTChar, 799
- gg::GgBuffer< T >, 201
 - ~GgBuffer, 203
 - bind, 204
 - copy, 204
 - getBuffer, 204
 - getCount, 205
 - getStride, 205
 - getTarget, 206
 - GgBuffer, 202, 203
 - map, 206, 207
 - operator=, 208
 - read, 208
 - send, 209
 - unbind, 209
 - unmap, 210
- gg::GgColorTexture, 210
 - ~GgColorTexture, 212
 - bind, 213
 - get, 213
 - getHeight, 213

- getSize, 213
- getTexture, 214
- getWidth, 214
- GgColorTexture, 211, 212
- load, 214, 215
- operator bool, 216
- operator!, 216
- unbind, 216
- gg::GgElements, 217
 - ~GgElements, 219
 - draw, 220
 - getIndexBuffer, 220
 - getIndexCount, 221
 - GgElements, 218, 219
 - load, 221
 - send, 222
- gg::GgMatrix, 223
 - ~GgMatrix, 229
 - frustum, 229
 - get, 230, 231
 - GgMatrix, 226–228
 - invert, 232
 - loadFrustum, 232
 - loadIdentity, 233
 - loadInvert, 234
 - loadLookat, 235, 236
 - loadNormal, 237, 238
 - loadOrthogonal, 238
 - loadPerspective, 239
 - loadRotate, 240, 241, 243, 244
 - loadRotateX, 245
 - loadRotateY, 245
 - loadRotateZ, 246
 - loadScale, 247, 248
 - loadTranslate, 249, 250
 - loadTranspose, 251
 - lookat, 252, 253
 - normal, 255
 - operator+, 260, 261
 - operator+=, 262
 - operator-, 263, 264
 - operator-=, 264, 265
 - operator/, 266
 - operator/=: 267, 268
 - operator=, 269, 270
 - operator*, 256, 258
 - operator*=, 259
 - orthogonal, 270
 - perspective, 271
 - projection, 271, 272
 - rotate, 273–275
 - rotateX, 276
 - rotateY, 277
 - rotateZ, 277
 - scale, 278, 280
 - translate, 281, 282
 - transpose, 283
- gg::GgNormalTexture, 284
 - ~GgNormalTexture, 286
 - bind, 286
 - get, 286
 - getHeight, 286
 - getSize, 287
 - getTexture, 287
 - getWidth, 287
 - GgNormalTexture, 285
 - load, 288, 289
 - operator bool, 289
 - operator!, 289
 - unbind, 290
- gg::GgPoints, 290
 - ~GgPoints, 293
 - draw, 293
 - getBuffer, 294
 - getCount, 294
 - GgPoints, 292
 - load, 294
 - operator bool, 295
 - operator!, 295
 - send, 295
- gg::GgPointShader, 296
 - ~GgPointShader, 299
 - get, 300
 - GgPointShader, 297–299
 - load, 300, 301
 - loadMatrix, 302
 - loadModelviewMatrix, 304, 305
 - loadProjectionMatrix, 305, 306
 - unuse, 307
 - use, 307–310
- gg::GgQuaternion, 311
 - ~GgQuaternion, 318
 - add, 318–320
 - conjugate, 321
 - divide, 321–323
 - euler, 324–326
 - get, 327
 - getConjugateMatrix, 327, 328
 - getMatrix, 329, 331
 - GgQuaternion, 314, 316–318
 - invert, 331
 - loadAdd, 332, 333
 - loadConjugate, 334, 335
 - loadDivide, 336, 337
 - loadEuler, 338, 340
 - loadIdentity, 341
 - loadInvert, 341, 342
 - loadMatrix, 343
 - loadMultiply, 344, 345
 - loadNormalize, 346, 348
 - loadRotate, 348, 349
 - loadRotateX, 350
 - loadRotateY, 351
 - loadRotateZ, 352
 - loadSlerp, 353–355
 - loadSubtract, 356, 357

- multiply, 359, 360, 362
- norm, 362
- normalize, 363
- operator+, 367
- operator+=, 368, 369
- operator-, 369, 370
- operator-=, 370, 371
- operator/, 372
- operator/=: 373, 374
- operator=, 374–376
- operator*, 364, 365
- operator*=, 365, 366
- rotate, 376, 377
- rotateX, 378
- rotateY, 379
- rotateZ, 379
- slerp, 380
- subtract, 381, 382
- gg::GgShader, 384
 - ~GgShader, 387
 - get, 387
 - GgShader, 384–386
 - operator=, 387, 388
 - unuse, 388
 - use, 388
- gg::GgShape, 389
 - ~GgShape, 390
 - draw, 390
 - get, 391
 - getMode, 391
 - GgShape, 390
 - operator bool, 392
 - operator!, 392
 - setMode, 392
- gg::GgSimpleObj, 393
 - ~GgSimpleObj, 394
 - draw, 394
 - get, 394
 - GgSimpleObj, 393
 - operator bool, 395
 - operator!, 395
- gg::GgSimpleShader, 396
 - ~GgSimpleShader, 400
 - GgSimpleShader, 398, 399
 - load, 400, 401
 - loadMatrix, 402–404
 - loadModelviewMatrix, 405, 406
 - operator=, 407
 - use, 409–413, 415–418
- gg::GgSimpleShader::Light, 496
 - ambient, 497
 - diffuse, 497
 - position, 497
 - specular, 497
- gg::GgSimpleShader::LightBuffer, 498
 - ~LightBuffer, 501
 - LightBuffer, 499, 500
 - load, 501, 502
 - loadAmbient, 503, 504
 - loadColor, 505
 - loadDiffuse, 506, 507
 - loadPosition, 508–510
 - loadSpecular, 512, 513
 - select, 514
- gg::GgSimpleShader::Material, 515
 - ambient, 516
 - diffuse, 516
 - shininess, 516
 - specular, 516
- gg::GgSimpleShader::MaterialBuffer, 517
 - ~MaterialBuffer, 520
 - load, 521
 - loadAmbient, 522, 523
 - loadAmbientAndDiffuse, 524–526
 - loadDiffuse, 527, 528
 - loadShininess, 529, 530
 - loadSpecular, 531, 532
 - MaterialBuffer, 519, 520
 - select, 533
- gg::GgTexture, 419
 - ~GgTexture, 421
 - bind, 422
 - getHeight, 422
 - getSize, 422
 - getTexture, 423
 - getWidth, 423
 - GgTexture, 419–421
 - operator=, 423, 424
 - swapRandB, 424
 - unbind, 425
- gg::GgTrackball, 425
 - ~GgTrackball, 430
 - begin, 430
 - end, 430
 - get, 431
 - getMatrix, 431
 - getQuaternion, 431
 - getScale, 432
 - getStart, 433
 - GgTrackball, 429
 - motion, 433
 - operator=, 434
 - region, 434, 435
 - reset, 436
 - rotate, 436
- gg::GgTriangles, 437
 - ~GgTriangles, 440
 - draw, 440
 - getBuffer, 440
 - getCount, 441
 - GgTriangles, 438, 439
 - load, 441
 - send, 442
- gg::GgUniformBuffer< T >, 443
 - ~GgUniformBuffer, 446
 - bind, 446

- copy, 446
- fill, 447
- getBuffer, 448
- getCount, 449
- getStride, 449
- getTarget, 450
- GgUniformBuffer, 443, 445
- load, 450, 451
- map, 452
- read, 453
- send, 454
- unbind, 455
- unmap, 455
- gg::GgVector, 456
 - ~GgVector, 461
 - distance3, 461
 - distance4, 461
 - dot3, 462
 - dot4, 463
 - GgVector, 457–460
 - length3, 463
 - length4, 464
 - normalize3, 464
 - normalize4, 465
 - operator+, 468, 470
 - operator+=", 470, 472
 - operator-, 472, 474
 - operator-=, 474, 476
 - operator/, 476, 478
 - operator/=", 478, 480
 - operator=, 480, 481
 - operator*, 465, 466
 - operator*=", 466, 468
- gg::GgVertex, 481
 - GgVertex, 483, 484
 - normal, 484
 - position, 484
- gg::GgVertexArray, 485
 - ~GgVertexArray, 488
 - bind, 488
 - get, 488
 - GgVertexArray, 485, 486
 - operator=, 488, 490
- GG_BUTTON_COUNT
 - GgApp.h, 885
- GG_INTERFACE_COUNT
 - GgApp.h, 885
- GgApp, 196
 - ~GgApp, 198
 - getUsername, 198
 - GgApp, 196, 197
 - main, 198
 - operator=, 200
- GgApp.cpp, 857
- GgApp.h, 884
 - GG_BUTTON_COUNT, 885
 - GG_INTERFACE_COUNT, 885
- GgApp::Window, 573
 - ~Window, 577
 - get, 578
 - getAltArrow, 578
 - getAltArrowX, 578
 - getAltArrowY, 579
 - getArrow, 580
 - getArrowX, 581
 - getArrowY, 582
 - getAspect, 583
 - getControlArrow, 583
 - getControlArrowX, 583
 - getControlArrowY, 584
 - getFboHeight, 585
 - getFboSize, 585, 586
 - getFboWidth, 586
 - getHeight, 587
 - getKey, 587
 - getLastKey, 587
 - getMouse, 588
 - getMouseX, 589
 - getMouseY, 589
 - getRotation, 589
 - getRotationMatrix, 589
 - getScrollMatrix, 590
 - getShiftArrow, 590
 - getShiftArrowX, 590
 - getShiftArrowY, 591
 - getSize, 592
 - getTranslation, 592
 - getTranslationMatrix, 594
 - getUserPointer, 594
 - getWheel, 594, 595
 - getWheelX, 595
 - getWheelY, 595
 - getWidth, 595
 - operator bool, 596
 - operator=, 596, 597
 - reset, 597
 - resetRotation, 598
 - resetTranslation, 598
 - restoreViewport, 598
 - selectInterface, 599
 - setClose, 599
 - setKeyboardFunc, 599
 - setMouseFunc, 600
 - setResizeFunc, 600
 - setUserPointer, 601
 - setVelocity, 601
 - setWheelFunc, 601
 - shouldClose, 602
 - swapBuffers, 602
 - updateViewport, 602
 - Window, 575–577
- ggArraysObj
 - gg, 23
- GgBuffer
 - gg::GgBuffer< T >, 202, 203
- ggBufferAlignment

- gg, [89](#)
- GgColorTexture
 - gg::GgColorTexture, [211](#), [212](#)
- ggConjugate
 - gg, [23](#)
- ggCreateComputeShader
 - gg, [24](#)
- ggCreateNormalMap
 - gg, [24](#)
- ggCreateShader
 - gg, [25](#)
- ggCross
 - gg, [26](#), [28](#)
- ggDistance3
 - gg, [28](#), [29](#)
- ggDistance4
 - gg, [30](#), [31](#)
- ggDot3
 - gg, [32](#)
- ggDot4
 - gg, [33](#)
- GgElements
 - gg::GgElements, [218](#), [219](#)
- ggElementsMesh
 - gg, [34](#)
- ggElementsObj
 - gg, [35](#)
- ggElementsSphere
 - gg, [36](#)
- ggEllipse
 - gg, [36](#)
- ggError
 - gg.h, [799](#)
- ggEulerQuaternion
 - gg, [37](#), [39](#)
- ggFBOError
 - gg.h, [799](#)
- ggFrustum
 - gg, [40](#)
- ggIdentity
 - gg, [40](#)
- ggIdentityQuaternion
 - gg, [41](#)
- ggInit
 - gg, [41](#)
- ggInvert
 - gg, [42](#), [43](#)
- ggLength3
 - gg, [43](#), [44](#)
- ggLength4
 - gg, [45](#)
- ggLoadComputeShader
 - gg, [46](#)
- ggLoadHeight
 - gg, [47](#)
- ggLoadImage
 - gg, [48](#)
- ggLoadShader
 - gg, [48](#), [49](#)
- ggLoadSimpleObj
 - gg, [50](#), [51](#)
- ggLoadTexture
 - gg, [52](#)
- ggLookat
 - gg, [53](#), [54](#)
- GgMatrix
 - gg::GgMatrix, [226–228](#)
- ggMatrixQuaternion
 - gg, [55](#), [56](#)
- ggNorm
 - gg, [56](#)
- ggNormal
 - gg, [57](#)
- ggNormalize
 - gg, [57](#)
- ggNormalize3
 - gg, [58–60](#)
- ggNormalize4
 - gg, [60–62](#)
- GgNormalTexture
 - gg::GgNormalTexture, [285](#)
- ggOrthogonal
 - gg, [63](#)
- ggPerspective
 - gg, [63](#)
- GgPoints
 - gg::GgPoints, [292](#)
- ggPointsCube
 - gg, [64](#)
- GgPointShader
 - gg::GgPointShader, [297–299](#)
- ggPointsSphere
 - gg, [65](#)
- GgQuaternion
 - gg::GgQuaternion, [314](#), [316–318](#)
- ggQuaternionMatrix
 - gg, [65](#)
- ggQuaternionTransposeMatrix
 - gg, [66](#)
- ggReadImage
 - gg, [67](#)
- ggRectangle
 - gg, [68](#)
- ggRotate
 - gg, [68–71](#)
- ggRotateQuaternion
 - gg, [72](#), [73](#)
- ggRotateX
 - gg, [74](#)
- ggRotateY
 - gg, [74](#)
- ggRotateZ
 - gg, [75](#)
- ggSaveColor
 - gg, [75](#)
- ggSaveDepth

- gg, 77
- ggSaveTga
 - gg, 77
- ggScale
 - gg, 78–80
- GgShader
 - gg::GgShader, 384–386
- GgShape
 - gg::GgShape, 390
- GgSimpleObj
 - gg::GgSimpleObj, 393
- GgSimpleShader
 - gg::GgSimpleShader, 398, 399
- ggSlerp
 - gg, 80–82
- GgTexture
 - gg::GgTexture, 419–421
- GgTrackball
 - gg::GgTrackball, 429
- ggTranslate
 - gg, 83, 84
- ggTranspose
 - gg, 86
- GgTriangles
 - gg::GgTriangles, 438, 439
- GgUniformBuffer
 - gg::GgUniformBuffer< T >, 443, 445
- GgVector
 - gg::GgVector, 457–460
- GgVertex
 - gg::GgVertex, 483, 484
- GgVertexArray
 - gg::GgVertexArray, 485, 486
- HEADER_STR
 - main.cpp, 901
- height
 - Camera, 144
- image
 - Camera, 144
- increaseExposure
 - CamCv, 128
 - Camera, 137
- increaseGain
 - CamCv, 129
 - Camera, 138
- index
 - CaptureFormat, 168
- initialize
 - Config, 176
- initializeInputIntrinsics
 - Menu, 539
- interval
 - Camera, 144
- Intrinsics, 491
 - center, 494
 - fov, 494
 - fps, 495
 - Intrinsics, 491, 492
 - sensorSize, 495
 - setCenter, 492
 - setFov, 493
 - setFps, 494
 - setSize, 494
 - size, 495
- Intrinsics.cpp, 897
- Intrinsics.h, 898
- invert
 - gg::GgMatrix, 232
 - gg::GgQuaternion, 331
- isCaptured
 - Camera, 138
- isDiverseEnough
 - Calibration, 118
- isImage
 - Capture, 162
- isOpened
 - CamImage, 148
 - Capture, 162
- isRunning
 - Camera, 138
- isStable
 - Calibration, 118
- isStillImage
 - Camera, 138
 - CamImage, 148
- length3
 - gg::GgVector, 463
- length4
 - gg::GgVector, 464
- LightBindingPoint
 - gg, 22
- LightBuffer
 - gg::GgSimpleShader::LightBuffer, 499, 500
- load
 - CamImage, 149
 - Config, 176
 - gg::GgColorTexture, 214, 215
 - gg::GgElements, 221
 - gg::GgNormalTexture, 288, 289
 - gg::GgPoints, 294
 - gg::GgPointShader, 300, 301
 - gg::GgSimpleShader, 400, 401
 - gg::GgSimpleShader::LightBuffer, 501, 502
 - gg::GgSimpleShader::MaterialBuffer, 521
 - gg::GgTriangles, 441
 - gg::GgUniformBuffer< T >, 450, 451
- loadAdd
 - gg::GgQuaternion, 332, 333
- loadAmbient
 - gg::GgSimpleShader::LightBuffer, 503, 504
 - gg::GgSimpleShader::MaterialBuffer, 522, 523
- loadAmbientAndDiffuse
 - gg::GgSimpleShader::MaterialBuffer, 524–526
- loadColor
 - gg::GgSimpleShader::LightBuffer, 505

- loadConjugate
 - gg::GgQuaternion, [334](#), [335](#)
- loadDiffuse
 - gg::GgSimpleShader::LightBuffer, [506](#), [507](#)
 - gg::GgSimpleShader::MaterialBuffer, [527](#), [528](#)
- loadDivide
 - gg::GgQuaternion, [336](#), [337](#)
- loadEuler
 - gg::GgQuaternion, [338](#), [340](#)
- loadFrustum
 - gg::GgMatrix, [232](#)
- loadIdentity
 - gg::GgMatrix, [233](#)
 - gg::GgQuaternion, [341](#)
- loadInvert
 - gg::GgMatrix, [234](#)
 - gg::GgQuaternion, [341](#), [342](#)
- loadLookat
 - gg::GgMatrix, [235](#), [236](#)
- loadMatrix
 - gg::GgPointShader, [302](#)
 - gg::GgQuaternion, [343](#)
 - gg::GgSimpleShader, [402–404](#)
- loadModelviewMatrix
 - gg::GgPointShader, [304](#), [305](#)
 - gg::GgSimpleShader, [405](#), [406](#)
- loadMultiply
 - gg::GgQuaternion, [344](#), [345](#)
- loadNormal
 - gg::GgMatrix, [237](#), [238](#)
- loadNormalize
 - gg::GgQuaternion, [346](#), [348](#)
- loadOrthogonal
 - gg::GgMatrix, [238](#)
- loadParameters
 - Calibration, [119](#)
- loadPerspective
 - gg::GgMatrix, [239](#)
- loadPosition
 - gg::GgSimpleShader::LightBuffer, [508–510](#)
- loadProjectionMatrix
 - gg::GgPointShader, [305](#), [306](#)
- loadRotate
 - gg::GgMatrix, [240](#), [241](#), [243](#), [244](#)
 - gg::GgQuaternion, [348](#), [349](#)
- loadRotateX
 - gg::GgMatrix, [245](#)
 - gg::GgQuaternion, [350](#)
- loadRotateY
 - gg::GgMatrix, [245](#)
 - gg::GgQuaternion, [351](#)
- loadRotateZ
 - gg::GgMatrix, [246](#)
 - gg::GgQuaternion, [352](#)
- loadScale
 - gg::GgMatrix, [247](#), [248](#)
- loadShininess
 - gg::GgSimpleShader::MaterialBuffer, [529](#), [530](#)
- loadSlerp
 - gg::GgQuaternion, [353–355](#)
- loadSpecular
 - gg::GgSimpleShader::LightBuffer, [512](#), [513](#)
 - gg::GgSimpleShader::MaterialBuffer, [531](#), [532](#)
- loadSubtract
 - gg::GgQuaternion, [356](#), [357](#)
- loadTranslate
 - gg::GgMatrix, [249](#), [250](#)
- loadTranspose
 - gg::GgMatrix, [251](#)
- lockFrame
 - Camera, [139](#)
- lookat
 - gg::GgMatrix, [252](#), [253](#)
- main
 - GgApp, [198](#)
 - main.cpp, [902](#)
- main.cpp, [900](#)
 - HEADER_STR, [901](#)
 - main, [902](#)
- map
 - Buffer, [105](#)
 - gg::GgBuffer< T >, [206](#), [207](#)
 - gg::GgUniformBuffer< T >, [452](#)
- markerLength
 - Settings, [553](#)
- MaterialBindingPoint
 - gg, [22](#)
- MaterialBuffer
 - gg::GgSimpleShader::MaterialBuffer, [519](#), [520](#)
- Menu, [534](#)
 - ~Menu, [536](#)
 - autoCaptureBeep, [542](#)
 - autoCaptureCooldown, [542](#)
 - autoCaptureCooldownTimer, [542](#)
 - autoCaptureEnabled, [542](#)
 - autoCaptureMinStableTime, [543](#)
 - autoCaptureStatusMessage, [543](#)
 - detectBoard, [543](#)
 - detectMarker, [543](#)
 - draw, [537](#)
 - getCheckerLength, [537](#)
 - getCheckerSize, [537](#)
 - getMarkerLength, [537](#)
 - getMenubarHeight, [538](#)
 - getPose, [538](#)
 - initializeInputIntrinsics, [539](#)
 - Menu, [535](#), [536](#)
 - operator bool, [539](#)
 - operator=, [539](#)
 - prioritizeLatency, [543](#)
 - saveImage, [540](#)
 - setup, [540](#), [541](#)
 - updateAutoCapture, [541](#)
- Menu.cpp, [905](#)
 - getAnyList, [906](#)
- Menu.h, [925](#)

- Mesh, [544](#)
 - ~Mesh, [545](#)
 - draw, [545](#)
 - drawMesh, [545](#)
 - Mesh, [544](#)
 - operator=, [546](#)
- mesh
 - Texture, [573](#)
- Mesh.h, [929](#)
- motion
 - gg::GgTrackball, [433](#)
- mtx
 - Camera, [144](#)
- multiply
 - gg::GgQuaternion, [359](#), [360](#), [362](#)
- norm
 - gg::GgQuaternion, [362](#)
- normal
 - gg::GgMatrix, [255](#)
 - gg::GgVertex, [484](#)
- normalize
 - gg::GgQuaternion, [363](#)
- normalize3
 - gg::GgVector, [464](#)
- normalize4
 - gg::GgVector, [465](#)
- onClose
 - CamCv, [129](#)
 - Camera, [140](#)
 - CamImage, [149](#)
 - CamMf, [156](#)
- onStart
 - CamCv, [129](#)
 - Camera, [140](#)
 - CamImage, [150](#)
 - CamMf, [156](#)
- onStop
 - CamCv, [130](#)
 - Camera, [141](#)
 - CamImage, [150](#)
 - CamMf, [157](#)
- open
 - CamCv, [130](#)
 - CamImage, [150](#)
 - CamMf, [157](#)
- opencv_link.h, [932](#)
- openDevice
 - Capture, [162](#)
- openImage
 - Capture, [163](#)
- openMovie
 - Capture, [163](#)
- operator bool
 - Capture, [164](#)
 - gg::GgColorTexture, [216](#)
 - gg::GgNormalTexture, [289](#)
 - gg::GgPoints, [295](#)
 - gg::GgShape, [392](#)
 - gg::GgSimpleObj, [395](#)
 - GgApp::Window, [596](#)
 - Menu, [539](#)
- operator!
 - gg::GgColorTexture, [216](#)
 - gg::GgNormalTexture, [289](#)
 - gg::GgPoints, [295](#)
 - gg::GgShape, [392](#)
 - gg::GgSimpleObj, [395](#)
- operator+
 - gg, [87](#)
 - gg::GgMatrix, [260](#), [261](#)
 - gg::GgQuaternion, [367](#)
 - gg::GgVector, [468](#), [470](#)
- operator+=
 - gg::GgMatrix, [262](#)
 - gg::GgQuaternion, [368](#), [369](#)
 - gg::GgVector, [470](#), [472](#)
- operator-
 - gg, [88](#)
 - gg::GgMatrix, [263](#), [264](#)
 - gg::GgQuaternion, [369](#), [370](#)
 - gg::GgVector, [472](#), [474](#)
- operator-=
 - gg::GgMatrix, [264](#), [265](#)
 - gg::GgQuaternion, [370](#), [371](#)
 - gg::GgVector, [474](#), [476](#)
- operator/
 - gg, [89](#)
 - gg::GgMatrix, [266](#)
 - gg::GgQuaternion, [372](#)
 - gg::GgVector, [476](#), [478](#)
- operator/=
 - gg::GgMatrix, [267](#), [268](#)
 - gg::GgQuaternion, [373](#), [374](#)
 - gg::GgVector, [478](#), [480](#)
- operator=
 - Buffer, [106](#)
 - Calibration, [119](#)
 - Expand, [180](#)
 - Framebuffer, [191](#)
 - gg::GgBuffer< T >, [208](#)
 - gg::GgMatrix, [269](#), [270](#)
 - gg::GgQuaternion, [374–376](#)
 - gg::GgShader, [387](#), [388](#)
 - gg::GgSimpleShader, [407](#)
 - gg::GgTexture, [423](#), [424](#)
 - gg::GgTrackball, [434](#)
 - gg::GgVector, [480](#), [481](#)
 - gg::GgVertexArray, [488](#), [490](#)
 - GgApp, [200](#)
 - GgApp::Window, [596](#), [597](#)
 - Menu, [539](#)
 - Mesh, [546](#)
 - Texture, [568](#)
- operator*
 - gg, [87](#)

- gg::GgMatrix, [256](#), [258](#)
- gg::GgQuaternion, [364](#), [365](#)
- gg::GgVector, [465](#), [466](#)
- operator*=
 - gg::GgMatrix, [259](#)
 - gg::GgQuaternion, [365](#), [366](#)
 - gg::GgVector, [466](#), [468](#)
- orthogonal
 - gg::GgMatrix, [270](#)
- parseconfig.h, [933](#)
 - getString, [935](#), [936](#)
 - getValue, [936](#), [937](#)
 - setString, [938](#), [939](#)
 - setValue, [939](#)
- pathChar
 - gg.h, [799](#)
- pathString
 - gg.h, [799](#)
- perspective
 - gg::GgMatrix, [271](#)
- position
 - gg::GgSimpleShader::Light, [497](#)
 - gg::GgVertex, [484](#)
- Preference, [546](#)
 - ~Preference, [548](#)
 - buildShader, [549](#)
 - getDescription, [549](#)
 - getIntrinsics, [549](#)
 - getShader, [549](#)
 - Preference, [547](#), [548](#)
 - setPreference, [549](#)
- Preference.cpp, [943](#)
- Preference.h, [944](#)
- prioritizeLatency
 - Camera, [144](#)
 - Menu, [543](#)
- projection
 - gg::GgMatrix, [271](#), [272](#)
- read
 - gg::GgBuffer< T >, [208](#)
 - gg::GgUniformBuffer< T >, [453](#)
- README.md, [947](#)
- readPixels
 - Texture, [570](#), [571](#)
- recordCorners
 - Calibration, [120](#)
- region
 - gg::GgTrackball, [434](#), [435](#)
- reset
 - gg::GgTrackball, [436](#)
 - GgApp::Window, [597](#)
- resetRotation
 - GgApp::Window, [598](#)
- resetTranslation
 - GgApp::Window, [598](#)
- resize
 - Buffer, [107](#), [108](#)
- resolution
 - CaptureFormat, [168](#)
- restoreViewport
 - GgApp::Window, [598](#)
- retrieve
 - Capture, [164](#), [165](#)
- rotate
 - gg::GgMatrix, [273–275](#)
 - gg::GgQuaternion, [376](#), [377](#)
 - gg::GgTrackball, [436](#)
- rotateX
 - gg::GgMatrix, [276](#)
 - gg::GgQuaternion, [378](#)
- rotateY
 - gg::GgMatrix, [277](#)
 - gg::GgQuaternion, [379](#)
- rotateZ
 - gg::GgMatrix, [277](#)
 - gg::GgQuaternion, [379](#)
- running
 - Camera, [144](#)
- RvecTvecToPose
 - Calibration, [120](#)
- SafeRelease
 - CamMf.cpp, [671](#)
- samples
 - Settings, [553](#)
- save
 - CamImage, [151](#)
 - Config, [177](#)
- savelImage
 - Menu, [540](#)
- saveParameters
 - Calibration, [121](#)
- scale
 - gg::GgMatrix, [278](#), [280](#)
- select
 - CamMf, [158](#)
 - gg::GgSimpleShader::LightBuffer, [514](#)
 - gg::GgSimpleShader::MaterialBuffer, [533](#)
- selectFormat
 - Camera, [141](#)
 - CamMf, [158](#)
- selectInterface
 - GgApp::Window, [599](#)
- send
 - gg::GgBuffer< T >, [209](#)
 - gg::GgElements, [222](#)
 - gg::GgPoints, [295](#)
 - gg::GgTriangles, [442](#)
 - gg::GgUniformBuffer< T >, [454](#)
- sensorSize
 - Intrinsics, [495](#)
- setCenter
 - Intrinsics, [492](#)
- setClose
 - GgApp::Window, [599](#)
- setDictionary

- Calibration, 121
- setExposure
 - CamCv, 131
- setFov
 - Intrinsics, 493
- setFps
 - Intrinsics, 494
- setGain
 - CamCv, 131
- setKeyboardFunc
 - GgApp::Window, 599
- setMode
 - gg::GgShape, 392
- setMouseFunc
 - GgApp::Window, 600
- setPosition
 - CamCv, 132
- setPreference
 - Preference, 549
- setPrioritizeLatency
 - Camera, 142
 - Capture, 165
- setResizeFunc
 - GgApp::Window, 600
- setSettings
 - Config, 178
- setSize
 - Intrinsics, 494
- setString
 - parseconfig.h, 938, 939
- Settings, 550
 - checkerLength, 551
 - checkerSize, 551
 - defaultCheckerLength, 551
 - defaultCheckerSize, 552
 - defaultEuler, 552
 - defaultFocal, 552
 - defaultFocalRange, 552
 - defaultMarkerLength, 552
 - dictionaryName, 552
 - euler, 553
 - focal, 553
 - focalRange, 553
 - getFocal, 551
 - markerLength, 553
 - samples, 553
- setup
 - Expand, 181
 - Menu, 540, 541
- setUserPointer
 - GgApp::Window, 601
- setValue
 - parseconfig.h, 939
- setVelocity
 - GgApp::Window, 601
- setWheelFunc
 - GgApp::Window, 601
- shininess
 - gg::GgSimpleShader::Material, 516
- shouldClose
 - GgApp::Window, 602
- show
 - Framebuffer, 193
- size
 - Intrinsics, 495
- slerp
 - gg::GgQuaternion, 380
- specular
 - gg::GgSimpleShader::Light, 497
 - gg::GgSimpleShader::Material, 516
- start
 - Camera, 142
 - Capture, 166
- stop
 - Camera, 142
 - Capture, 166
- subtract
 - gg::GgQuaternion, 381, 382
- SubTypeToName
 - CamMf.cpp, 671
- swapBuffers
 - GgApp::Window, 602
- swapRandB
 - gg::GgTexture, 424
- TCharToUtf8
 - gg.h, 799
- Texture, 554
 - ~Texture, 558
 - bindTexture, 559
 - copy, 560
 - create, 561
 - discard, 562
 - draw, 563
 - drawPixels, 564–566
 - getChannels, 567
 - getSize, 567
 - getTextureName, 567
 - mesh, 573
 - operator=, 568
 - readPixels, 570, 571
 - Texture, 556, 557
 - unbindTexture, 572
- Texture.cpp, 947
- Texture.h, 951
- thr
 - Camera, 145
- translate
 - gg::GgMatrix, 281, 282
- transpose
 - gg::GgMatrix, 283
- unbind
 - gg::GgBuffer< T >, 209
 - gg::GgColorTexture, 216
 - gg::GgNormalTexture, 290
 - gg::GgTexture, 425

- gg::GgUniformBuffer< T >, [455](#)
- unbindBuffer
 - Buffer, [109](#)
- unbindFramebuffer
 - Framebuffer, [193](#)
- unbindTexture
 - Texture, [572](#)
- unmap
 - Buffer, [109](#)
 - gg::GgBuffer< T >, [210](#)
 - gg::GgUniformBuffer< T >, [455](#)
- unuse
 - gg::GgPointShader, [307](#)
 - gg::GgShader, [388](#)
- update
 - Framebuffer, [194](#), [195](#)
- updateAutoCapture
 - Menu, [541](#)
- updateMotion
 - Calibration, [122](#)
- updateViewport
 - GgApp::Window, [602](#)
- use
 - Compute, [170](#)
 - gg::GgPointShader, [307–310](#)
 - gg::GgShader, [388](#)
 - gg::GgSimpleShader, [409–413](#), [415–418](#)
- USE_PIXEL_BUFFER_OBJECT
 - Buffer.h, [609](#)
- USE_RODRIGUES
 - Calibration.cpp, [616](#)
- Utf8ToTChar
 - gg.h, [799](#)
- width
 - Camera, [145](#)
- Window
 - GgApp::Window, [575–577](#)
- ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版., [1](#)